OKIEM EKSPERTA

Django 5

Praktyczne tworzenie aplikacji internetowych w Pythonie

Wydanie V

Antonio Melé

Helion

packt

Tytuł oryginału: Django 5 By Example: Build powerful and reliable Python web applications from scratch, 5th Edition

Tłumaczenie: Radosław Meryk

ISBN: 978-83-289-1883-2

Copyright © Packt Publishing 2024. First published in the English language under the title 'Django 5 By Example - Fifth Edition – (9781805125457)'

Polish edition copyright © 2025 by Helion S.A.

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording or by any information storage retrieval system, without permission from the Publisher.

Wszelkie prawa zastrzeżone. Nieautoryzowane rozpowszechnianie całości lub fragmentu niniejszej publikacji w jakiejkolwiek postaci jest zabronione. Wykonywanie kopii metodą kserograficzną, fotograficzną, a także kopiowanie książki na nośniku filmowym, magnetycznym lub innym powoduje naruszenie praw autorskich niniejszej publikacji.

Wszystkie znaki występujące w tekście są zastrzeżonymi znakami firmowymi bądź towarowymi ich właścicieli.

Autor oraz wydawca dołożyli wszelkich starań, by zawarte w tej książce informacje były kompletne i rzetelne. Nie biorą jednak żadnej odpowiedzialności ani za ich wykorzystanie, ani za związane z tym ewentualne naruszenie praw patentowych lub autorskich. Autor oraz wydawca nie ponoszą również żadnej odpowiedzialności za ewentualne szkody wynikłe z wykorzystania informacji zawartych w książce.

Drogi Czytelniku!
Jeżeli chcesz ocenić tę książkę, zajrzyj pod adres
https://helion.pl/user/opinie/dj5pr5_ebook
Możesz tam wpisać swoje uwagi, spostrzeżenia, recenzję.

Helion S.A.
ul. Kościuszki 1c, 44-100 Gliwice
tel. 32 230 98 63
e-mail: *helion@helion.pl*
WWW: *https://helion.pl* (księgarnia internetowa, katalog książek)

- Poleć książkę na Facebook.com
- Kup w wersji papierowej
- Oceń książkę

- Księgarnia internetowa
- Lubię to! » Nasza społeczność

Mojej mamie, Loli.

Spis treści

Słowo wstępne **19**

O autorze **21**

O recenzencie **21**

O czytelnikach wersji beta **22**

Przedmowa **23**

ROZDZIAŁ 1
Utworzenie aplikacji bloga **29**

Instalacja Pythona 30
Tworzenie środowiska wirtualnego w Pythonie 31
Instalacja Django 32
Instalowanie Django za pomocą pip 32
Ogólne informacje na temat frameworka Django 33
Główne komponenty frameworka 33
Architektura Django 34
Nowe funkcjonalności w Django 5 35
Tworzenie pierwszego projektu 36
Stosowanie początkowych migracji bazy danych 38
Uruchamianie serwera programistycznego 39
Ustawienia projektu 41
Projekty i aplikacje 42
Utworzenie aplikacji 42
Tworzenie modeli danych dla bloga 43
Utworzenie modelu Post 43
Dodawanie pól daty i godziny 45
Definiowanie domyślnej kolejności sortowania 46
Dodawanie indeksu bazy danych 47
Aktywacja aplikacji 48
Dodawanie pola stanu 48
Dodanie relacji wiele do jednego 51
Tworzenie i stosowanie migracji 52

Tworzenie witryny administracyjnej dla modeli ... 54
Tworzenie superużytkownika ... 55
Witryna administracyjna Django ... 55
Dodawanie modeli do witryny administracyjnej ... 56
Personalizacja sposobu wyświetlania modeli ... 58
Dodawanie liczby aspektów do filtrów ... 60
Praca z obiektami QuerySet i menedżerami ... 61
Tworzenie obiektów ... 62
Aktualizowanie obiektów ... 63
Pobieranie obiektów ... 63
Filtrowanie obiektów ... 64
Korzystanie z różnych metod dopasowywania pól ... 64
Łączenie filtrów w łańcuch ... 66
Wykluczanie obiektów ... 66
Sortowanie obiektów ... 66
Ograniczanie wynikowych obiektów QuerySet ... 67
Zliczanie obiektów ... 67
Sprawdzanie, czy obiekt istnieje ... 67
Usuwanie obiektów ... 68
Złożone dopasowania z wykorzystaniem obiektów Q ... 68
Kiedy następuje określenie zawartości kolekcji QuerySet? ... 68
Więcej informacji o obiektach QuerySet ... 69
Utworzenie menedżerów modelu ... 69
Przygotowanie widoków listy i szczegółów ... 70
Utworzenie widoków listy i szczegółów ... 71
Korzystanie ze skrótu get_object_or_404 ... 72
Dodanie wzorców adresów URL do widoków ... 72
Utworzenie szablonów dla widoków ... 74
Tworzenie szablonu bazowego ... 75
Utworzenie szablonu listy postów ... 75
Uruchomienie aplikacji ... 76
Tworzenie szablonu szczegółów posta ... 77
Cykl żądanie-odpowiedź ... 78
Polecenia zarządzające używane w tym rozdziale ... 79
Podsumowanie ... 80
Zasoby dodatkowe ... 80

ROZDZIAŁ 2
Usprawnianie bloga i dodanie funkcjonalności społecznościowych ... 82
Przegląd funkcjonalności ... 82
Kanoniczne adresy URL dla modeli ... 83

Tworzenie dla postów adresów URL przyjaznych dla SEO ... 85
Modyfikowanie wzorców adresów URL ... 86
Modyfikowanie widoków ... 87
Modyfikowanie kanonicznego adresu URL dla postów ... 88
Dodanie stronicowania ... 89
Dodanie stronicowania do widoku listy postów ... 89
Tworzenie szablonu stronicowania ... 90
Obsługa błędów stronicowania ... 92
Użycie widoków opartych na klasach ... 95
Po co korzystać z widoków opartych na klasach? ... 95
Użycie widoku opartego na klasie do wyświetlania listy postów ... 96
Polecanie postów przez e-mail ... 98
Tworzenie formularzy w Django ... 98
Obsługa formularzy w widokach ... 100
Wysyłanie wiadomości e-mail w Django ... 101
Korzystanie ze zmiennych środowiskowych ... 102
Wysyłanie wiadomości e-mail w widokach ... 106
Renderowanie formularzy w szablonach ... 108
Utworzenie systemu komentarzy ... 112
Tworzenie modelu komentarzy ... 112
Dodawanie modeli do witryny administracyjnej ... 114
Utworzenie formularzy na podstawie modeli ... 115
Obsługa klasy ModelForm w widokach ... 116
Tworzenie szablonów formularza komentarza ... 118
Dodawanie komentarzy do widoku szczegółów posta ... 120
Dodawanie komentarzy do szablonu szczegółów posta ... 121
Uproszczone szablony do renderowania pól formularzy ... 126
Podsumowanie ... 128
Dodatkowe zasoby ... 128

ROZDZIAŁ 3
Rozbudowa aplikacji bloga ... 130

Przegląd funkcjonalności ... 130
Implementacja systemu tagowania za pomocą modułu django-taggit ... 131
Pobieranie postów według podobieństwa ... 139
Utworzenie własnych filtrów i znaczników szablonu ... 144
Utworzenie własnych znaczników szablonu ... 145
Utworzenie tagu szablonu typu simple_tag ... 145
Tworzenie znacznika szablonu typu inclusion_tag ... 147
Tworzenie znacznika szablonu, który zwraca kolekcję QuerySet ... 149
Utworzenie własnych filtrów szablonu ... 151
Tworzenie filtra szablonu obsługującego składnię Markdown ... 151

Dodanie mapy witryny ... 156
Utworzenie kanału wiadomości dla postów bloga ... 160
Dodanie do bloga wyszukiwania pełnotekstowego ... 167
Instalacja Dockera ... 168
Instalacja PostgreSQL ... 168
Zrzucanie istniejących danych ... 169
Przełączanie bazy danych w projekcie ... 170
Ładowanie danych do nowej bazy danych ... 171
Proste wyszukiwania ... 172
Wyszukiwanie w wielu polach ... 173
Utworzenie widoku wyszukiwania ... 173
Stemming i ranking wyników ... 176
Stemming i usuwanie słów ze stoplisty dla różnych języków ... 178
Wagi zapytań ... 179
Wyszukiwanie z podobieństwem trygramu ... 180
Podsumowanie ... 182
Rozszerzanie projektu z wykorzystaniem sztucznej inteligencji ... 182
Dodatkowe zasoby ... 183

ROZDZIAŁ 4

Utworzenie witryny społecznościowej ... 185

Przegląd funkcjonalności ... 185
Utworzenie projektu witryny społecznościowej ... 186
Rozpoczęcie pracy nad aplikacją społecznościową ... 187
Użycie frameworka uwierzytelniania w Django ... 188
Utworzenie widoku logowania ... 189
Użycie wbudowanych widoków uwierzytelniania w Django ... 195
Widoki logowania i wylogowania ... 196
Widoki zmiany hasła ... 202
Widoki odzyskiwania hasła ... 204
Rejestracja użytkownika i profile użytkownika ... 211
Rejestracja użytkownika ... 212
Rozbudowa modelu User ... 217
Instalowanie modułu Pillow i udostępnianie plików multimedialnych ... 219
Tworzenie migracji dla modelu Profile ... 220
Podsumowanie ... 226
Dodatkowe zasoby ... 226

ROZDZIAŁ 5
Implementacja uwierzytelniania za pomocą witryn społecznościowych ... 227
Przegląd funkcjonalności ... 227
Wymagania techniczne ... 228
Użycie frameworka messages ... 228
Implementacja własnego backendu uwierzytelniania ... 232
Uniemożliwianie użytkownikom korzystania z istniejącego adresu e-mail ... 234
Dodanie do witryny uwierzytelnienia za pomocą innej witryny społecznościowej ... 235
Uruchomienie serwera programistycznego za pośrednictwem HTTPS ... 238
Uwierzytelnianie za pomocą serwisu Google ... 241
Tworzenie profili dla użytkowników rejestrujących się za pomocą uwierzytelniania społecznościowego ... 248
Podsumowanie ... 250
Dodatkowe zasoby ... 251

ROZDZIAŁ 6
Udostępnianie treści w witrynie internetowej ... 252
Przegląd funkcjonalności ... 252
Utworzenie witryny internetowej do kolekcjonowania obrazów ... 253
Utworzenie modelu Image ... 254
Zdefiniowanie relacji typu „wiele do wielu” ... 256
Rejestracja modelu Image w witrynie administracyjnej ... 257
Umieszczanie treści pochodzącej z innych witryn internetowych ... 257
Usunięcie zawartości pól formularza ... 258
Instalowanie biblioteki Requests ... 259
Nadpisanie metody save() egzemplarza ModelForm ... 259
Utworzenie bookmarkletu za pomocą JavaScript ... 264
Utworzenie szczegółowego widoku obrazu ... 276
Utworzenie miniatur za pomocą modułu easy-thumbnails ... 278
Dodawanie asynchronicznych operacji za pomocą JavaScript ... 280
Załadowanie JavaScript w modelu DOM ... 282
Ataki CSRF w żądaniach HTTP w JavaScript ... 283
Wykonywanie żądań HTTP za pomocą JavaScript ... 284
Dodanie do listy obrazów nieskończonego stronicowania ... 289
Podsumowanie ... 296
Dodatkowe zasoby ... 296

ROZDZIAŁ 7
Śledzenie działań użytkownika 298
Przegląd funkcjonalności 298
Utworzenie systemu obserwacji 299
Utworzenie relacji typu „wiele do wielu" za pomocą modelu pośredniego 300
Utworzenie widoków listy i szczegółów dla profilu użytkownika 303
Dodanie działań obserwowania i rezygnacji z obserwowania użytkownika za pomocą JavaScript 308
Opracowanie strumienia aktywności aplikacji 311
Użycie frameworka contenttypes 312
Dodanie do modelu relacji generycznych 313
Uniknięcie powielonych działań w strumieniu aktywności 316
Dodanie działania użytkownika do strumienia aktywności 318
Wyświetlanie strumienia aktywności 320
Optymalizacja kolekcji QuerySet dotyczącej powiązanych obiektów 321
Tworzenie szablonów dla działań użytkowników 323
Użycie sygnałów dla denormalizowanych zliczeń 325
Praca z sygnałami 325
Definiowanie klas konfiguracyjnych aplikacji 328
Korzystanie z paska narzędzi Django Debug Toolbar 330
Instalacja paska narzędzi Django Debug Toolbar 330
Panele paska narzędzi Django Debug Toolbar 333
Polecenia paska narzędzi Django Debug Toolbar 335
Zliczanie wyświetleń obrazu za pomocą bazy danych Redis 336
Instalacja bazy danych Redis 336
Użycie bazy danych Redis z Pythonem 338
Przechowywanie widoków obrazów w bazie danych Redis 339
Przechowywanie rankingu w bazie danych Redis 341
Kolejne kroki z bazą danych Redis 344
Podsumowanie 344
Rozszerzanie projektu przy użyciu sztucznej inteligencji 345
Dodatkowe zasoby 345

ROZDZIAŁ 8
Utworzenie sklepu internetowego 347
Przegląd funkcjonalności 347
Utworzenie projektu sklepu internetowego 348
Utworzenie modeli katalogu produktów 350
Rejestracja modeli katalogu w witrynie administracyjnej 353
Utworzenie widoków katalogu 355
Utworzenie szablonów katalogu 357

Utworzenie koszyka na zakupy ... 362
Użycie frameworka sessions Django ... 362
Ustawienia sesji ... 363
Wygaśnięcie sesji ... 364
Przechowywanie koszyka na zakupy w sesji ... 364
Utworzenie widoków koszyka na zakupy ... 368
Utworzenie procesora kontekstu dla bieżącego koszyka na zakupy ... 375
Rejestracja zamówień klienta ... 378
Utworzenie modeli zamówienia ... 379
Dołączenie modeli zamówienia do witryny administracyjnej ... 380
Utworzenie zamówień klienta ... 382
Zadania asynchroniczne ... 387
Wykorzystywanie zadań asynchronicznych ... 387
Wątki robocze, kolejki komunikatów i brokery komunikatów ... 387
Podsumowanie ... 398
Dodatkowe zasoby ... 398

ROZDZIAŁ 9
Zarządzanie płatnościami i zamówieniami ... 399

Przegląd funkcjonalności ... 399
Integracja bramki płatności ... 400
Utworzenie konta Stripe ... 401
Instalowanie biblioteki Pythona obsługi serwisu Stripe ... 403
Dodanie do projektu obsługi serwisu Stripe ... 404
Budowanie procesu płatności ... 405
Testowanie płatności ... 414
Korzystanie z webhooków do otrzymywania powiadomień o płatnościach ... 421
Odwoływanie się do płatności Stripe w zamówieniach ... 428
Wdrożenie do produkcji ... 432
Eksport zamówień do plików CSV ... 432
Dodanie własnych działań do witryny administracyjnej ... 432
Rozbudowa witryny administracyjnej za pomocą własnych widoków ... 435
Dynamiczne generowanie faktur w formacie PDF ... 440
Instalacja WeasyPrint ... 441
Utworzenie szablonu PDF ... 441
Renderowanie plików w formacie PDF ... 442
Wysyłanie dokumentów PDF za pomocą poczty elektronicznej ... 444
Podsumowanie ... 448
Dodatkowe zasoby ... 448

ROZDZIAŁ 10
Rozbudowa sklepu internetowego 450
Przegląd funkcjonalności 450
Utworzenie systemu kuponów 451
Utworzenie modeli kuponu 452
Zastosowanie kuponu w koszyku na zakupy 455
Zastosowanie kuponu w zamówieniu 462
Zastosowanie kuponów w sesji Stripe Checkout 466
Dodawanie kuponów do zamówień w serwisie administracyjnym oraz do faktur w formacie PDF 469
Utworzenie silnika rekomendacji produktów 471
Rekomendacja produktów na podstawie wcześniejszych transakcji 472
Podsumowanie 482
Dodatkowe zasoby 483

ROZDZIAŁ 11
Internacjonalizacja sklepu internetowego 484
Przegląd funkcjonalności 484
Internacjonalizacja za pomocą Django 485
Ustawienia internacjonalizacji i lokalizacji 486
Polecenia przeznaczone do zarządzania internacjonalizacją 487
Instalowanie zestawu narzędzi gettext 487
Jak dodać tłumaczenie do projektu Django? 487
W jaki sposób Django określa bieżący język? 488
Przygotowanie projektu do internacjonalizacji 489
Tłumaczenie kodu Pythona 490
Tłumaczenie standardowe 491
Tłumaczenie leniwe 491
Tłumaczenia zawierające zmienne 491
Liczba mnoga w tłumaczeniach 492
Tłumaczenie własnego kodu 492
Tłumaczenie szablonów 496
Znacznik szablonu {% translate %} 496
Znacznik szablonu {% blocktranslate %} 496
Tłumaczenie szablonów sklepu internetowego 497
Użycie interfejsu do tłumaczeń o nazwie Rosetta 500
Tłumaczenia niejednoznaczne 503
Wzorce adresów URL dla internacjonalizacji 503
Dodanie prefiksu języka do wzorców adresów URL 504
Tłumaczenie wzorców adresów URL 505

Umożliwienie użytkownikowi zmiany języka ... 508
Tłumaczenie modeli za pomocą django-parler ... 510
Instalacja django-parler ... 511
Tłumaczenie pól modelu ... 511
Integracja tłumaczeń w witrynie administracyjnej ... 513
Utworzenie migracji dla tłumaczeń modeli ... 514
Korzystanie z tłumaczeń w kolekcjach QuerySet ... 517
Adaptacja widoków dla tłumaczeń ... 518
Lokalizacja formatów ... 520
Użycie modułu django-localflavor do weryfikacji pól formularza ... 521
Rozszerzanie projektu przy użyciu sztucznej inteligencji ... 523
Podsumowanie ... 523
Dodatkowe zasoby ... 524

ROZDZIAŁ 12
Budowa platformy e-learningu ... 525
Przegląd funkcjonalności ... 525
Utworzenie platformy e-learningu ... 526
Obsługa plików multimedialnych ... 527
Utworzenie modeli kursu ... 528
Rejestracja modeli w witrynie administracyjnej ... 530
Użycie fikstur w celu dostarczenia początkowych danych dla modeli ... 530
Utworzenie modeli dla zróżnicowanej treści ... 534
Wykorzystanie dziedziczenia modeli ... 535
Utworzenie modeli treści ... 538
Utworzenie własnych pól modelu ... 541
Dodawanie porządkowania do modułów i obiektów treści ... 544
Dodanie widoków uwierzytelniania ... 547
Dodanie systemu uwierzytelniania ... 547
Utworzenie szablonów uwierzytelniania ... 548
Podsumowanie ... 551
Dodatkowe zasoby ... 552

ROZDZIAŁ 13
Tworzenie systemu zarządzania treścią ... 553
Przegląd funkcjonalności ... 553
Utworzenie systemu zarządzania treścią ... 555
Utworzenie widoków opartych na klasach ... 555
Użycie domieszek w widokach opartych na klasach ... 556
Praca z grupami i uprawnieniami ... 558

Zarządzanie modułami kursu i treścią ... 565
Użycie zbiorów formularzy dla modułów kursów ... 566
Dodanie treści do modułów kursów ... 570
Zarządzanie modułami i treścią ... 575
Zmiana kolejności modułów i treści ... 579
Używanie domieszek z pakietu django-braces ... 580
Podsumowanie ... 587
Dodatkowe zasoby ... 587

ROZDZIAŁ 14
Renderowanie i buforowanie treści ... 588
Przegląd funkcjonalności ... 589
Wyświetlanie katalogu kursów ... 590
Dodanie rejestracji uczestnika ... 595
Utworzenie widoku rejestracji uczestnika ... 595
Zapisanie się na kurs ... 597
Renderowanie treści kursu ... 601
Uzyskanie dostępu do treści kursu ... 601
Renderowanie różnych rodzajów treści ... 604
Użycie frameworka buforowania ... 606
Dostępne backendy buforowania ... 608
Instalacja Memcached ... 608
Instalowanie obrazu Docker mechanizmu Memcached ... 608
Instalacja powiązania Memcached dla języka Python ... 609
Ustawienia buforowania we frameworku Django ... 609
Dodanie Memcached do projektu ... 610
Poziomy buforowania ... 610
Użycie niskopoziomowego API buforowania ... 611
Sprawdzanie żądań pobierających dane z bufora za pomocą paska narzędzi Django Debug Toolbar ... 612
Niskopoziomowe buforowanie na podstawie danych dynamicznych ... 616
Buforowanie fragmentów szablonu ... 617
Buforowanie widoków ... 618
Użycie bufora dla poszczególnych witryn ... 618
Backend buforowania oparty na bazie danych Redis ... 620
Monitorowanie bazy danych Redis za pomocą Django Redisboard ... 621
Podsumowanie ... 622
Dodatkowe zasoby ... 622

ROZDZIAŁ 15
Utworzenie API ... 624
Przegląd funkcjonalności ... 624
Utworzenie API typu RESTful ... 626
Instalacja frameworka REST Django ... 627
Definiowanie serializatorów ... 627
Klasy parserów i renderowania formatów ... 628
Utworzenie widoków listy i szczegółowego ... 629
Wykorzystanie API ... 631
Rozszerzanie serializatorów ... 633
Dodawanie pól do serializatorów ... 633
Implementacja w serializatorach pól obliczanych za pomocą metod ... 634
Dodawanie stronicowania do widoków ... 636
Tworzenie serializatora kursów ... 637
Serializacja relacji ... 639
Opracowanie zagnieżdżonych serializatorów ... 640
Utworzenie kolekcji ViewSet i routerów ... 640
Tworzenie własnych widoków API ... 644
Obsługa uwierzytelniania ... 645
Implementacja podstawowego uwierzytelniania ... 646
Określenie uprawnień do widoków ... 646
Dołączanie dodatkowych operacji do kolekcji ViewSet ... 648
Tworzenie niestandardowych uprawnień ... 649
Serializacja treści kursu ... 649
Wykorzystanie API RESTful ... 651
Podsumowanie ... 655
Dodatkowe zasoby ... 655

ROZDZIAŁ 16
Budowanie serwera czatu ... 657
Przegląd funkcjonalności ... 657
Utworzenie aplikacji czatu ... 658
Implementacja widoku pokoju czatu ... 659
Komunikacja w czasie rzeczywistym w Django za pomocą frameworka Channels ... 662
Aplikacje asynchroniczne z wykorzystaniem ASGI ... 662
Cykl żądanie-odpowiedź z wykorzystaniem frameworka Channels ... 663
Instalacja frameworków Channels i Daphne ... 665
Pisanie konsumenta ... 667

Routing ... 668
Implementacja klienta WebSocket ... 669
Warstwa kanału komunikacyjnego ... 676
Kanały komunikacyjne i grupy ... 676
Konfiguracja warstwy kanału komunikacyjnego z wykorzystaniem Redis ... 676
Aktualizacja konsumenta w celu rozgłaszania wiadomości ... 677
Dodawanie kontekstu do wiadomości ... 681
Modyfikacja konsumenta w celu uzyskania pełnej asynchroniczności ... 684
Utrwalanie wiadomości w bazie danych ... 686
Utworzenie modelu dla wiadomości czatu ... 687
Dodanie modelu Message do witryny administracyjnej ... 688
Zapisywanie wiadomości w bazie danych ... 689
Wyświetlanie historii czatu ... 691
Integracja aplikacji czatu z istniejącymi widokami ... 692
Podsumowanie ... 694
Dodatkowe zasoby ... 695

ROZDZIAŁ 17
Wdrożenie ... 696

Tworzenie środowiska produkcyjnego ... 697
Zarządzanie ustawieniami dla wielu środowisk ... 697
Korzystanie z systemu Docker Compose ... 700
Instalacja Docker Compose za pośrednictwem Docker Desktop ... 701
Tworzenie pliku Dockerfile ... 701
Dodanie wymagań Pythona ... 702
Tworzenie pliku Docker Compose ... 703
Konfigurowanie usługi PostgreSQL ... 706
Stosowanie migracji bazy danych i tworzenie superużytkownika ... 709
Konfigurowanie usługi Redis ... 710
Serwowanie Django za pomocą WSGI i NGINX ... 711
Korzystanie z uWSGI ... 711
Konfiguracja uWSGI ... 712
Korzystanie z NGINX ... 713
Konfiguracja NGINX ... 714
Korzystanie z nazwy hosta ... 717
Udostępnianie zasobów statycznych i multimedialnych ... 717
Zabezpieczanie witryny za pomocą protokołu SSL/TLS ... 720
Sprawdzenie gotowości projektu do wdrożenia do produkcji ... 720
Konfiguracja projektu do obsługi SSL/TLS ... 721
Utworzenie certyfikatu SSL/TLS ... 722

Konfiguracja serwera NGINX do wykorzystania SSL/TLS 723
Przekierowywanie ruchu HTTP do HTTPS 725
Konfiguracja serwera Daphne do wykorzystania z frameworkiem Django Channels 726
Wykorzystanie bezpiecznych połączeń dla gniazd WebSocket 727
Uwzględnienie Daphne w konfiguracji NGINX 728
Utworzenie własnej warstwy middleware 731
Utworzenie oprogramowania middleware do obsługi subdomeny 733
Implementacja własnych poleceń administracyjnych 735
Podsumowanie 738
Rozszerzanie projektu przy użyciu sztucznej inteligencji 738
Dodatkowe zasoby 739

Słowo wstępne

Django to wysokopoziomowy framework do tworzenia aplikacji webowych w Pythonie, który pozwala na szybkie tworzenie aplikacji o czystym, praktycznym projekcie.

Oto opis Django, który można znaleźć na jego stronie internetowej i w repozytorium Git. Uważam, że definiuje go w zwięzły, ale kompletny sposób.

W ciągu prawie 20 lat od pierwszego wydania Django pojawiło się i szybko rozwinęło wiele nowych frameworków webowych, opartych na Pythonie i innych językach. Frameworki te zajmują swoje specyficzne nisze i wykorzystują nowe technologie.

Mimo wszystko Django dalej się rozwija. Wersja po wersji (aż do najnowszej) uzupełniano go o nowe funkcjonalności, poprawiano stabilność i wprowadzano innowacje w iteracyjny, ale konsekwentny sposób.

Zacząłem używać Django od wersji 1.3, po wcześniejszych doświadczeniach z Zope i Plone. Od początku byłem pod wrażeniem tego, jak bardzo framework pomógł mi w budowaniu rzeczywistych aplikacji w szybki, ale skuteczny sposób. Z biegiem lat charakter tworzonych przeze mnie aplikacji webowych się zmieniał, ale Django zawsze pozostawał ważnym narzędziem do reagowania na techniczne wyzwania.

Przez lata starałem się spisywać swoje doświadczenia z własnościami frameworka Django, które doceniam najbardziej, i stale próbowałem demonstrować je przykładami. Kiedy odkryłem tę książkę, od razu zobaczyłem w podejściu Antonio siebie — uzupełnianie poznawania Django konkretnymi aplikacjami, które tworzysz po drodze.

Oprócz opisu wielu powszechnie używanych własności Django doceniam sposób, w jaki ta książka przedstawia przydatne pakiety, takie jak Django REST framework i Django Debug Toolbar, a także przykłady korzystania z bardziej zaawansowanych funkcji, takich jak wyszukiwanie pełnotekstowe z PostgreSQL (którego jestem zagorzałym zwolennikiem) i integracja pamięci podręcznej z wykorzystaniem Redis.

Co więcej, w nowym wydaniu znajdziesz opis wielu nowych cech wprowadzonych w Django 5.0, takich jak wartości domyślne obliczane na podstawie bazy danych, nowe możliwości tworzenia stron administracyjnych oraz uproszczone szablony renderowania pól formularzy.

Projekty ze starego wydania przepisano, by wykorzystać Pythona 3.12 i zaktualizowane pakiety zewnętrzne. Zaktualizowano również instrukcje konfiguracji. Wprowadzono ponadto ciekawe dodatki, takie jak diagramy objaśniające i zaawansowane funkcjonalności frameworka DRF.

Kiedy za młodu zacząłem uczyć się HTML, próbowałem edytować odwiedzane strony internetowe za pomocą edytora tekstu. Nadal lubię praktyczne podejście do nauki technologii.

Lubię wykonywać praktyczne zadania przed dokładniejszym studiowaniem technologii, a najlepiej, gdy tworzę coś konkretnego. Gorąco polecam stosowanie takiego podejścia.

Ta książka zabierze Cię w podróż, w której będziesz na przemian studiować funkcjonalności Django i wykorzystywać je w konkretnych przykładach. Przygotuj się na to, że ubrudzisz sobie ręce!

— Paolo Melchiorre
Programista Pythona/ współtwórca frameworka Django
www.paulox.net

O autorze

Antonio Melé pełni funkcję dyrektora ds. informatyki w Backbase, firmie będącej liderem w dziedzinie transformacji cyfrowej dla instytucji finansowych. Dołączył do Backbase w 2023 roku, po przejęciu przez tę firmę Nucoro — cyfrowej platformy zarządzania majątkiem, której Antonio był współzałożycielem.

Antonio rozwija projekty Django dla klientów z różnych branż od 2006 roku. W 2009 roku Antonio założył Zenx IT — firmę programistyczną specjalizującą się w budowaniu produktów cyfrowych. Pracował jako CTO i konsultant techniczny dla wielu technicznych start-upów, a także zarządzał zespołami programistycznymi budującymi projekty dla dużych firm informatycznych. Antonio ma tytuł magistra w dziedzinie informatyki na Universidad Pontificia Comillas. Ukończył również Advanced Management Program na MIT Sloan. Zamiłowanie do komputerów i programowania wpoił mu ojciec.

Nie mógłbym napisać tej książki bez fantastycznej grupy entuzjastów i ekspertów Pythona, którzy poświęcili czas na zapoznanie się z wczesnymi wersjami tej książki. Ogromne podziękowania kieruję do wszystkich, którzy zrecenzowali książkę i przeczytali wersję beta. Wasze spostrzeżenia, poprawki i entuzjazm bardzo ożywiły ten projekt.

O recenzencie

Mark Walker wcześniej opracowywał oprogramowanie e-learningowe w sektorze rządowym i obronnym. Programowanie w Django rozpoczął od niszowej aplikacji offline, a następnie przeszedł do programowania projektów wykorzystywanych na największych imprezach biegowych na świecie. Następnie zajął się DevOps, testowaniem obciążenia i zabezpieczaniem aplikacji na potrzeby testów penetracyjnych. W ostatnich latach został liderem technicznym Django CMS Association i członkiem Django Software Foundation. Współuczestniczył w tworzeniu doskonałych gier sportowych. Ma doświadczenie w korzystaniu ze środowisk AWS i GCP. Mark stał się opiekunem kilku pakietów i nawigatorem programu kosmicznego Djangonaut, którego celem jest mentorowanie programistom pragnącym współtworzyć ekosystem Django.

O czytelnikach wersji beta

Poniższa lista zawiera czytelników z programu beta, którzy dzięki swoim komentarzom kierowali rozwojem tego wydania. Na serdeczne podziękowanie za cenną pomoc przy recenzowaniu tego wydania zasłużyły następujące osoby:

- Adewunmi Oladele
- Ashkan Ranjbar Beyranvand
- Ashraf Ibrahim Mohamed
- Attila Sebők
- Chedjou Soff o Rocelin
- Eduardo Andrés Muñoz Muñoz
- Emmanuel Aidoo
- Florian Bautry
- Gladys García Granadillo
- Henry Ukomah
- Ismir Kullolli
- Karen Stingel
- Melissa Strong
- Mohammad Ahmad Mohammad AbdelSalam
- Moustapha Diarra
- Oleg Belov
- Rasa Lazovic
- Safwan Samsudeen
- Sergio Delgado Quintero
- Takahiro Kato
- Yardley Luther Estiverne Estiverne

Przedmowa

Django to framework open source do tworzenia aplikacji webowych w Pythonie, pozwalający na szybkie tworzenie aplikacji o czystym, praktycznym projekcie. Zajmuje się większością problemów związanych z tworzeniem aplikacji webowych i charakteryzuje stosunkowo płytką krzywą uczenia się, dzięki czemu jest dość łatwy do przyswojenia przez początkujących programistów. Django podąża za filozofią Pythona „baterie są w zestawie". Dostarcza bogaty i wszechstronny zestaw modułów, które rozwiązują typowe problemy związane z tworzeniem aplikacji webowych. Prostota Django wraz z jego rozbudowanymi funkcjami sprawia, że jest atrakcyjny zarówno dla początkujących, jak i doświadczonych programistów. Framework Django zaprojektowano z myślą o prostocie, elastyczności, niezawodności i skalowalności.

Obecnie Django używają niezliczone start-upy i duże firmy, takie jak Instagram, Spotify, Pinterest, Udemy, Robinhood i Coursera. Nieprzypadkowo przez kilka ostatnich lat Django był konsekwentnie wybierany w corocznej ankiecie dla programistów serwisu Stack Overflow przez użytkowników z całego świata jako jeden z najbardziej lubianych frameworków webowych.

Ta książka przeprowadza czytelników przez cały proces tworzenia profesjonalnych aplikacji webowych z wykorzystaniem Django. Książka koncentruje się na wyjaśnieniu, jak działa framework Django, na przykładzie wielu projektów budowanych od podstaw. Autor nie tylko omawia najistotniejsze aspekty frameworka, ale także wyjaśnia, jak zastosować Django w bardzo różnych praktycznych sytuacjach.

Ta książka uczy nie tylko Django, ale także przedstawia inne popularne technologie, takie jak PostgreSQL, Redis, Celery, RabbitMQ i Memcached. Dowiesz się z niej, jak zintegrować te technologie ze swoimi projektami Django, aby tworzyć zaawansowane funkcjonalności i budować złożone aplikacje webowe.

W książce opisany został krok po kroku proces tworzenia praktycznych aplikacji, rozwiązywania typowych problemów i implementacji najlepszych praktyk z wykorzystaniem łatwych do powielenia procedur.

Po przeczytaniu tej książki będziesz dobrze rozumieć działanie Django i tworzyć praktyczne, zaawansowane aplikacje webowe w Pythonie.

Dla kogo jest ta książka?

Ta książka powinna służyć jako elementarz dla programistów, którzy niedawno rozpoczęli pracę z Django. Książka jest przeznaczona dla programistów posiadających umiejętności programowania w Pythonie, którzy chcą w praktyczny sposób nauczyć się Django.

Być może Django jest dla Ciebie zupełną nowością lub już masz pewną wiedzę na temat frameworka, ale chcesz wycisnąć z niego jak najwięcej. Ta książka pomoże Ci opanować najbardziej istotne obszary frameworka przez budowanie praktycznych projektów od podstaw. Aby przeczytać tę książkę, musisz posiadać pewną wiedzę na temat programowania. Oprócz podstawowej znajomości Pythona zakłada się podstawową znajomość HTML i JavaScript.

Co znajdziesz w książce?

Ta książka obejmuje szereg tematów związanych z tworzeniem aplikacji webowych z wykorzystaniem frameworka Django. Książka poprowadzi Czytelnika przez proces tworzenia czterech różnych, w pełni funkcjonalnych aplikacji webowych, które będziemy budowali podczas lektury 17 rozdziałów:

- Aplikacja bloga (rozdziały od 1. do 3.)
- Witryna do tworzenia zakładek do zdjęć (rozdziały od 4. do 7.)
- Sklep internetowy (rozdziały od 8. do 11.)
- Platforma e-learningowa (rozdziały od 12. do 17.)

W każdym rozdziale omówiono kilka funkcji Django:

Rozdział 1., „Utworzenie aplikacji bloga", zawiera wprowadzenie do frameworka na przykładzie opracowania aplikacji bloga. Utworzymy podstawowe modele bloga, widoki, szablony i adresy URL przeznaczone do wyświetlania postów. Dowiesz się, jak za pomocą warstwy **ORM** w Django tworzyć obiekty QuerySet, a także jak przeprowadzić konfigurację witryny administracyjnej.

Rozdział 2., „Usprawnianie bloga i dodanie funkcjonalności społecznościowych", nauczy Cię, jak dodać stronicowanie do swojego bloga oraz jak zaimplementować widoki Django oparte na klasach. Nauczysz się wysyłać e-maile za pomocą Django oraz obsługiwać formularze tworzone na podstawie modeli. Zaimplementujesz również system komentarzy do postów na blogu.

Rozdział 3., „Rozbudowa aplikacji bloga", omawia sposób integracji aplikacji zewnętrznych. Ten rozdział przeprowadzi Cię przez proces tworzenia systemu tagowania i nauczy, jak budować złożone zestawy zapytań umożliwiających polecanie podobnych postów. Ponadto dowiesz się, jak tworzyć niestandardowe tagi szablonów i filtry. Nauczysz się również, jak skorzystać z frameworka do tworzenia mapy witryny oraz jak utworzyć kanał wiadomości RSS dla publikowanych postów. Dowiesz się również, jak przygotować mapę witryny oraz kanał wiadomości RSS dla publikowanych postów. Budowę aplikacji bloga ukończymy, implementując silnik wyszukiwania z funkcjami wyszukiwania pełnotekstowego w PostgreSQL.

Rozdział 4., „Utworzenie witryny społecznościowej", dotyczy przygotowania witryny społecznościowej. Dowiesz się, jak implementować niestandardowe widoki uwierzytelniania oraz jak korzystać z frameworka uwierzytelniania Django. Zaimplementujesz mechanizm rejestracji użytkowników i rozszerzysz model użytkownika o niestandardowy model profilu.

Rozdział 5., „Implementacja uwierzytelniania za pomocą witryn społecznościowych", opisuje implementację uwierzytelniania społecznościowego i korzystanie z frameworka `messages`. Stworzysz niestandardowy backend uwierzytelniania i używając OAuth 2, zintegrujesz uwierzytelnianie społecznościowe z Google. Dowiesz się także, jak używać pakietu `django-extensions` do uruchamiania serwera programistycznego przez HTTPS i jak dostosować potok uwierzytelniania społecznościowego w celu zautomatyzowania procesu tworzenia profili użytkowników.

Rozdział 6., „Udostępnianie treści w witrynie internetowej", wyjaśnia, jak przekształcić aplikację społecznościową na witrynę przeznaczoną do kolekcjonowania zdjęć. W przygotowanych modelach wykorzystamy relacje typu „wiele do wielu", utworzymy w języku JavaScript bookmarklet, który następnie zintegrujemy z projektem. W tym rozdziale dowiesz się również, jak generować miniatury zdjęć. Ponadto nauczysz się implementacji asynchronicznych żądań HTTP z wykorzystaniem JavaScript i Django oraz mechanizmu stronicowania z nieskończonym przewijaniem.

Rozdział 7., „Śledzenie działań użytkownika", pokazuje, jak zbudować system obserwacji użytkowników. Pracę nad aplikacją przeznaczoną do kolekcjonowania obrazów ukończymy przez dodanie możliwości obserwacji działań użytkownika. Dowiesz się także, jak tworzyć generyczne relacje między modelami i optymalizować zestawy `QuerySet`.

Będziesz pracować z sygnałami oraz nauczysz się implementować denormalizację. Nauczysz się również korzystać z paska narzędzi Django Debug Toolbar w celu uzyskania odpowiednich informacji diagnostycznych. Na koniec zintegrujesz ze swoim projektem bazę danych Redis, co pozwoli zliczać wyświetlenia zdjęć, oraz stworzysz za pomocą Redis ranking najczęściej oglądanych ilustracji.

Rozdział 8., „Utworzenie sklepu internetowego", opisuje, jak utworzyć sklep internetowy. Zbudujesz modele katalogów oraz opracujesz koszyk na zakupy z wykorzystaniem sesji Django. Stworzysz procesor kontekstu dla koszyka na zakupy i nauczysz się zarządzać zamówieniami klientów. Z tego rozdziału dowiesz się także, jak za pomocą Celery i RabbitMQ wysyłać asynchroniczne powiadomienia. Nauczysz się ponadto monitorować Celery za pomocą systemu Flower.

Rozdział 9., „Zarządzanie płatnościami i zamówieniami", wyjaśnia, w jaki sposób zintegrować bramkę płatniczą z Twoim sklepem online. Zintegrujesz ze swoim sklepem bramkę płatniczą Stripe Checkout i będziesz otrzymywać w swojej aplikacji asynchroniczne powiadomienia o płatnościach. Zaimplementujesz w serwisie administracyjnym niestandardowe widoki oraz dostosujesz serwis administracyjny do eksportowania zamówień do plików CSV. Dowiesz się również, jak dynamicznie generować faktury w formacie PDF.

Rozdział 10., „Rozbudowa sklepu internetowego", uczy jak utworzyć system kuponów pozwalający na stosowanie rabatów do zamówień. Zaktualizujesz integrację bramki płatniczej Stripe Checkout w taki sposób, aby wprowadzać kupony rabatowe i stosować kupony do zamówień. Skorzystasz z bazy danych Redis do zapamiętywania produktów, które zwykle są kupowane razem, i wykorzystasz te informacje do zbudowania silnika rekomendacji produktów.

Rozdział 11., „Internacjonalizacja sklepu internetowego", pokazuje, jak dodać do projektu mechanizmy internacjonalizacji. Dowiesz się, jak generować pliki tłumaczeń i zarządzać nimi oraz jak tłumaczyć ciągi znaków w kodzie Pythona i szablonach Django. Do zarządzania tłumaczeniami i implementacji adresów URL dla poszczególnych języków skorzystasz z systemu Rosetta. Dowiesz się, jak tłumaczyć pola modelu za pomocą modułu `django-parler` oraz jak korzystać z tłumaczeń z wykorzystaniem mechanizmu ORM. Na koniec za pomocą pakietu `django-localflavor` utworzysz zlokalizowane pole formularza.

Rozdział 12., „Budowa platformy e-learningu", przeprowadzi Cię przez proces tworzenia platformy e-learningowej. Dodasz fikstury do projektu i utworzysz początkowe modele dla systemu zarządzania treścią. Skorzystasz także z dziedziczenia modelu do tworzenia modeli danych dla polimorficznej zawartości. Dowiesz się, jak tworzyć niestandardowe pola modelu przez budowanie pól do porządkowania obiektów. Zaimplementujesz również widoki uwierzytelniania dla systemu CMS.

Rozdział 13., „Tworzenie systemu zarządzania treścią", nauczy Cię, jak stworzyć system CMS przy użyciu widoków opartych na klasach i domieszek. Wykorzystasz system grup i uprawnień Django do ograniczania dostępu do widoków oraz zaimplementujesz zestawy formularzy do edycji treści kursów. Za pomocą JavaScript i Django stworzysz również funkcjonalność „przeciągnij i upuść", pozwalająca na zmianę kolejności modułów kursu i ich zawartości.

Rozdział 14., „Renderowanie i buforowanie treści" pokaże Ci, jak zaimplementować publiczne widoki katalogu kursów. Opracujesz system rejestracji uczestników kursu oraz funkcjonalności zapisywania się na kursy. Utworzysz także funkcjonalność odpowiedzialną za renderowanie różnych typów zawartości dla modułów kursów. Dowiesz się również, jak buforować treść za pomocą frameworka pamięci podręcznej Django i konfigurować dla swojego projektu backendy pamięci podręcznej `Memcached` i `Redis`. Na koniec dowiesz się, jak monitorować Redis za pomocą strony administracyjnej.

Rozdział 15., „Utworzenie API", dotyczy budowy API typu RESTful dla projektu. Do tego zadania wykorzystamy Django REST Framework (DRF). Dowiesz się, jak obsługiwać uwierzytelnianie API i implementować uprawnienia dla widoków API. Dowiesz się, jak obsługiwać uwierzytelnianie API i implementować uprawnienia dla widoków API.

Dowiesz się także, jak budować zestawy widoków API i routery. Nauczysz się ponadto, jak korzystać z interfejsu API przy użyciu biblioteki `Requests`.

Rozdział 16., „Budowanie serwera czatu" wyjaśnia, jak korzystać z technologii Django Channels w celu utworzenia działającego w czasie rzeczywistym serwera czatu dla uczestników kursów. Z rozdziału dowiesz się, jak zaimplementować funkcjonalności, które bazują na komunikacji asynchronicznej przez gniazda WebSocket. Stworzysz konsumenta WebSocket za pomocą Pythona oraz zaimplementujesz klienta WebSocket za pomocą JavaScript. Użyjesz bazy danych Redis do skonfigurowania warstwy kanału oraz nauczysz się konfigurowania konsumenta WebSocket w taki sposób, aby był w pełni asynchroniczny. Zaimplementujesz również historię czatu dzięki utrwalaniu w bazie danych wiadomości przesyłanych na czacie.

Rozdział 17., „Wdrożenie", pokaże Ci, jak tworzyć ustawienia dla wielu środowisk oraz jak za pomocą Docker Compose skonfigurować środowisko produkcyjne przy użyciu PostgreSQL,

Redis, uWSGI, NGINX i Daphne. Dowiesz się, jak bezpiecznie serwować swój projekt przez HTTPS i korzystać z wbudowanego w Django frameworka kontroli systemu. Nauczysz się także, jak można zaimplementować własną warstwę middleware oraz jak utworzyć własne polecenia zarządzania.

Jak wynieść z tej książki najwięcej?

- Czytelnik powinien mieć dobrą praktyczną znajomość języka Python.
- Czuć się komfortowo z projektami obejmującymi kod HTML i JavaScript.

  Przed rozpoczęciem pracy z tą książką przeczytać części od 1. do 3. oficjalnego samouczka Django, dostępnego pod adresem *https://docs.djangoproject.com/en/5.0/intro/tutorial01/*.

Pobieranie przykładów kodu

Spolonizowany przykładowy kod dla tej książki możesz pobrać pod adresem *https://ftp.helion.pl/przyklady/dj5pr5.zip*. Pakiet z przykładowym kodem do wersji oryginalnej można pobrać ze strony *https://github.com/PacktPublishing/Django-5-by-Example*. Pod adresem *https://github.com/PacktPublishing/* dostępne są również kody źródłowe z innych pozycji z naszego bogatego katalogu książek i filmów. Warto je przejrzeć!

Pobieranie kolorowych ilustracji

Dostarczamy także plik PDF, który zawiera kolorowe wersje zrzutów ekranu i schematów użytych w tej książce. Można je pobrać pod adresem *https://ftp.helion.pl/przyklady/dj5pr5.zip*.

Zastosowane konwencje

W książce zastosowano różne konwencje edytorskie.

`KodWTekście`. W taki sposób są wyświetlane słowa kluczowe w tekście, nazwy tabel baz danych, nazwy pakietów. Oto przykład: „Przeprowadź edycję pliku *models.py* aplikacji `shop`".

Blok kodu zapisywany jest w następujący sposób.

```
from django.contrib import admin
from .models import Post

admin.site.register(Post)
```

Gdy trzeba zwrócić uwagę na określony fragment bloku kodu, odpowiednie wiersze lub elementy kodu zaznaczane są **pogrubieniem**.

```
INSTALLED_APPS = [
'django.contrib.admin',
'django.contrib.auth',
```

```
'django.contrib.contenttypes',
'django.contrib.sessions',
'django.contrib.messages',
'django.contrib.staticfiles',
'blog.apps.BlogConfig',
]
```

Wejście lub wyjście komend w wierszu polecenia zapisujemy tak:

```
python manage.py runserver
```

Pogrubienie. Wskazuje nowy termin, ważne słowo lub słowa widoczne na ekranie. W ten sposób pojawiają się w tekście elementy menu lub okien dialogowych. Oto przykład: „Z panelu **Administracja** wybierz pozycję **Informacje o systemie**".

Uwaga

W taki sposób są oznaczane ostrzeżenia, ważne uwagi, porady i sztuczki.

Wskazówka

Wskazówki i porady są oznaczane w taki sposób.

Utworzenie aplikacji bloga

Rozdział 1

Z tej książki dowiesz się, jak budować profesjonalne projekty webowe Django. Ten początkowy rozdział przeprowadzi Cię przez podstawowe etapy budowania aplikacji Django — od instalacji po wdrożenie. Podrozdział „Instalacja Django" przeprowadzi Cię przez proces instalacji. Te wskazówki przydadzą się zwłaszcza tym czytelnikom, którzy jeszcze nie zainstalowali Django na swoich komputerach.

Zanim zaczniemy pierwszy projekt Django, poświęćmy chwilę, aby zobaczyć, czego się nauczysz. W tym rozdziale przedstawiono ogólny przegląd frameworka. Omówiono w nim główne komponenty frameworka pozwalające stworzyć w pełni funkcjonalną aplikację webową: modele, szablony, widoki i adresy URL. Po jego przeczytaniu będziesz mieć obszerną wiedzę na temat sposobu działania frameworka Django oraz interakcji pomiędzy jego komponentami.

W tym rozdziale poznasz także różnicę pomiędzy projektami a aplikacjami Django oraz najważniejsze ustawienia. Zbudujesz prostą aplikację bloga, która pozwoli użytkownikom przeglądać opublikowane posty i je czytać. Stworzysz również prosty interfejs administracyjny do zarządzania postami oraz ich publikowania. W dwóch następnych rozdziałach rozbudujesz aplikację bloga o bardziej zaawansowane funkcjonalności.

Ten rozdział powinien służyć jako przewodnik do zbudowania kompletnej aplikacji Django. Nie martw się, jeśli nie rozumiesz wszystkich aspektów jego działania. Poszczególne komponenty frameworka zostaną dość dokładnie omówione w dalszych rozdziałach książki.

Oto zagadnienia, na których skoncentruję się w tym rozdziale:

- Instalacja Pythona
- Tworzenie środowiska wirtualnego w Pythonie
- Instalacja Django
- Utworzenie i skonfigurowanie projektu Django
- Utworzenie aplikacji Django
- Projektowanie modeli danych
- Tworzenie migracji modeli i ich stosowanie
- Utworzenie witryny administracyjnej dla modeli
- Praca z obiektami QuerySet oraz menedżerami
- Tworzenie widoków, szablonów i adresów URL
- Cykl żądanie-odpowiedź frameworka Django

Na początek zainstalujesz na swoim komputerze Pythona.

Kod źródłowy przykładów z tego rozdziału można znaleźć pod adresem *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter01*.

Wszystkie pakiety Pythona użyte w tym rozdziale są zawarte w pliku *requirements.txt* dołączonym do plików z kodem źródłowym przykładów z tego rozdziału. Aby zainstalować każdy pakiet Pythona, możesz postępować zgodnie z instrukcjami zamieszczonymi w poniższych podrozdziałach. Możesz także zainstalować wszystkie wymagania jednocześnie za pomocą polecenia `pip install -r requirements.txt`.

Instalacja Pythona

Django 5.0 obsługuje Pythona w wersjach 3.10, 3.11 i 3.12. W przykładach w tej książce będziemy korzystać z wersji 3.12 Pythona.

Jeśli używasz Linuksa lub macOS X, prawdopodobnie masz zainstalowanego Pythona. Jeśli pracujesz w systemie Windows, możesz pobrać instalator Pythona z witryny python.org. Pythona dla swojego systemu operacyjnego możesz pobrać ze strony *https://www.python.org/downloads/*.

Otwórz wiersz polecenia powłoki komputera. Jeśli korzystasz z systemu macOS, naciśnij *Command+spacja*, aby otworzyć Spotlight, i wpisz *Terminal*, aby otworzyć aplikację *Terminal.app*. Jeśli używasz systemu Windows, otwórz menu *Start* i w polu wyszukiwania wpisz **`powers`**. Następnie kliknij aplikację *Windows PowerShell*, aby ją otworzyć. Możesz także użyć prostszego środowiska wiersza polecenia. Aby to zrobić, w polu wyszukiwania wpisz **`cmd`** i kliknij aplikację *Wiersz polecenia*.

Aby sprawdzić, czy język Python jest zainstalowany na Twoim komputerze, wpisz w wierszu powłoki polecenie:

```
python3 --version
```

Jeśli Python w wersji 3.*x* jest zainstalowany na Twoim komputerze, na ekranie wyświetli się wynik podobny do poniższego:

```
Python 3.12.3
```

Jeśli wyświetli się komunikat o błędzie, spróbuj użyć polecenia `python` zamiast `python3`. Użytkownikom systemu Windows zaleca się zastąpienie polecenia `python` skróconą wersją `py`.

Jeśli zainstalowana wersja Pythona jest niższa od 3.12 lub Python nie został zainstalowany, pobierz najnowszą wersję pod adresem *https://www.python.org/downloads/* i ją zainstaluj. Na stronie pobierania można znaleźć instalatory języka Python dla systemów Windows, macOS i Linux.

W tej książce środowisko języka Python będziemy wywoływali z wiersza polecenia za pomocą polecenia `python`. W niektórych systemach operacyjnych wymagane jest jednak użycie polecenia `python3`. Jeśli używasz Linuksa lub macOS, a domyślną wersją Pythona w Twoim systemie operacyjnym jest Python 2, skorzystanie z zainstalowanej wersji

Pythona 3 wymaga użycia polecenia `python3`. Należy pamiętać, że Python 2 został wycofany w styczniu 2020 roku i nie powinien być już używany.

W systemie Windows poleceniem uruchamiającym interpreter Pythona jest `python`, natomiast programem uruchamiającym środowisko Pythona jest `py`. Program uruchamiający Pythona dla Windows wprowadzono w wersji Pythona 3.3. Program ten wykrywa zainstalowane wersje Pythona na komputerze i automatycznie deleguje do najnowszej wersji.

Jeśli korzystasz z systemu Windows, powinieneś używać polecenia `py`. Więcej informacji na temat programu uruchamiającego Pythona w środowisku Windows możesz znaleźć pod adresem *https://docs.python.org/3/using/windows.html#launcher*.

W następnym podrozdziale utworzysz środowisko Pythona dla swojego projektu i zainstalujesz niezbędne biblioteki Pythona.

Tworzenie środowiska wirtualnego w Pythonie

Podczas pisania aplikacji w języku Python zwykle używa się pakietów i modułów, które nie należą do standardowej biblioteki języka Python. Niektóre aplikacje Pythona wymagają innej wersji tego samego modułu. Jednak w całym systemie może być zainstalowana tylko określona wersja modułu. Uaktualnienie wersji modułu dla aplikacji może spowodować problemy działania aplikacji wymagających starszej wersji tego modułu.

Aby rozwiązać ten problem, można użyć środowisk wirtualnych Pythona. Dzięki środowiskom wirtualnym, zamiast instalować moduły języka Python globalnie, można instalować je w odizolowanej lokalizacji. Każde środowisko wirtualne ma swoje własne binaria Pythona i może mieć swój własny niezależny zestaw zainstalowanych pakietów Pythona w oddzielnych katalogach.

Od wersji 3.3 wraz z dystrybucją Pythona jest dostarczana biblioteka `venv`, która zapewnia wsparcie dla tworzenia prostych środowisk wirtualnych. Wykorzystanie modułu Python `venv` do tworzenia izolowanych środowisk Pythona pozwala używać różnych wersji pakietów dla różnych projektów. Kolejną zaletą korzystania z mechanizmu `venv` jest brak konieczności posiadania specjalnych uprawnień administracyjnych w celu instalowania pakietów Pythona.

Jeśli używasz systemu Linux lub macOS, utwórz odizolowane środowisko za pomocą polecenia:

```
python -m venv my_env
```

Jeśli Twój system jest dostarczany z Pythonem w wersji 2, a zainstalowałeś Pythona w wersji 3, pamiętaj, aby zamiast polecenia `python` używać polecenia `python3`.

Jeśli używasz systemu Windows, skorzystaj z polecenia:

```
py -m venv my_env
```

Spowoduje to użycie windowsowego programu uruchamiającego środowisko Python.

Poprzednie polecenie utworzy środowisko Pythona w nowym katalogu o nazwie *my_env*. Wszystkie biblioteki Pythona, które są instalowane, kiedy Twoje środowisko wirtualne jest aktywne, zostaną umieszczone w katalogu *my_env/lib/python3.12/site-packages*.

Jeśli używasz systemu Linux lub macOS, aby aktywować środowisko wirtualne, uruchom polecenie:

```
source my_env/bin/activate
```

Jeśli używasz systemu Windows, skorzystaj z polecenia:

```
.\my_env\Scripts\activate.bat
```

Symbol zachęty powłoki zawiera ujętą w nawiasy nazwę aktywnego środowiska wirtualnego. Przykładowo:

```
(my_env) zenx@pc:~ zenx$
```

Za pomocą polecenia `deactivate` możesz w dowolnym momencie zdezaktywować środowisko. Więcej informacji na temat pakietu `venv` można znaleźć pod adresem *https://docs.python.org/3/library/venv.html*.

Instalacja Django

Jeżeli masz już zainstalowany framework Django 5.0, możesz pominąć ten podrozdział i od razu przejść do kolejnego, zatytułowanego „Tworzenie pierwszego projektu".

Framework Django jest dostarczany w postaci pakietu Pythona i dlatego może być zainstalowany w dowolnym środowisku Pythona. Jeżeli jeszcze nie zainstalowałeś Django, tutaj znajdziesz krótkie omówienie jego instalacji na potrzeby programowania w środowisku lokalnym.

Instalowanie Django za pomocą pip

Preferowaną metodą instalacji Django jest skorzystanie z systemu zarządzania pakietami `pip`. W Pythonie 3.12 pip jest zainstalowany domyślnie. Instrukcje instalacji pakietu pip można znaleźć na stronie *https://pip.pypa.io/en/stable/installation/*.

Aby zainstalować Django za pomocą `pip`, uruchom w wierszu polecenia powłoki polecenie:

```
python -m pip install Django~=5.0.4
```

Django w najnowszej wersji 5.0 zostanie zainstalowany w podkatalogu *site-packages* katalogu Twojego środowiska wirtualnego.

Teraz możesz sprawdzić, czy framework Django został pomyślnie zainstalowany. Uruchom w terminalu polecenie:

```
python -m Django --version
```

Jeśli otrzymasz w odpowiedzi wynik 5.0, to znaczy, że framework Django został pomyślnie zainstalowany na Twoim komputerze. Jeśli pojawi się komunikat *No module named*

Django, framework Django nie jest zainstalowany na Twoim komputerze. W razie problemów z instalacją Django zapoznaj się z różnymi opcjami instalacji dostępnymi pod adresem *https://docs.djangoproject.com/en/5.0/intro/install/*.

Wszystkie pakiety Pythona użyte w tym rozdziale są zawarte w pliku *requirements.txt* dołączonym do plików z kodem źródłowym przykładów z tego rozdziału. Aby zainstalować każdy pakiet Pythona, możesz postępować zgodnie z instrukcjami zamieszczonymi w poniższych punktach. Możesz także zainstalować wszystkie wymagania jednocześnie za pomocą polecenia `pip install -r requirements.txt`.

Ogólne informacje na temat frameworka Django

Django to framework składający się z zestawu komponentów, które rozwiązują typowe problemy związane z tworzeniem aplikacji webowych. Komponenty Django są luźno sprzężone, co oznacza, że mogą być zarządzane niezależnie. Pomaga to oddzielić obowiązki różnych warstw frameworka. Warstwa bazy danych „nic nie wie" o sposobie wyświetlania danych, system szablonów „nic nie wie" o obsłudze żądań i tak dalej.

Framework Django dzięki stosowaniu zasady DRY (ang. *do not repeat yourself* — dosłownie: nie powtarzaj się) w maksymalnym stopniu wspiera ideę wielokrotnego wykorzystywania kodu. Użycie frameworka Django sprzyja również szybkiemu tworzeniu aplikacji, a dzięki wykorzystaniu dynamicznych możliwości Pythona, takich jak introspekcja, pozwala pisać mniej kodu.

Więcej informacji na temat filozofii projektowania z wykorzystaniem frameworka Django możesz znaleźć pod adresem *https://docs.djangoproject.com/en/5.0/misc/design-philosophies/*.

Główne komponenty frameworka

Framework Django jest zgodny ze wzorcem MTV (*Model-Template-View* — dosłownie: model-szablon-widok). Jest to wzorzec nieco podobny do popularnego wzorca MVC (*Model-View-Controller*), w którym szablon pełni funkcję widoku, a funkcję kontrolera pełni sam framework.

Odpowiedzialność poszczególnych warstw we wzorcu MTV frameworka Django podzielono w następujący sposób.

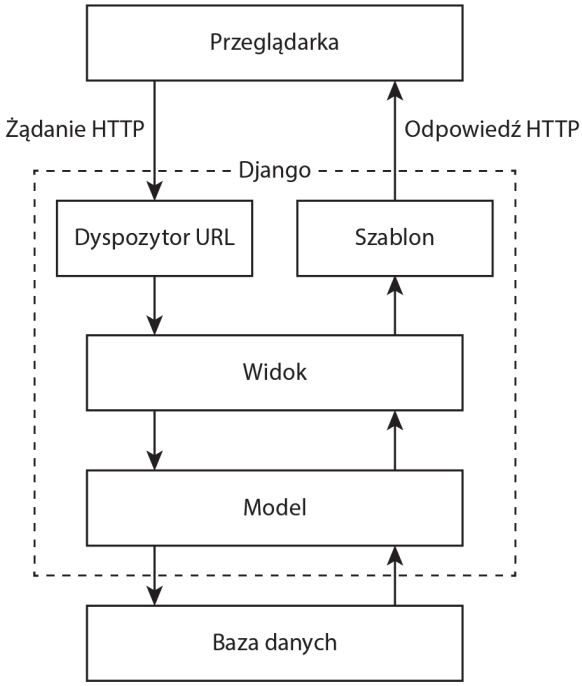
- **Model** — definiuje logiczną strukturę danych i jest handlerem danych między bazą danych a widokiem.
- **Szablon** — warstwa prezentacji. Django korzysta z systemu tekstowych szablonów, które zawierają wszystko, co renderuje przeglądarka.
- **Widok** — komunikuje się z bazą danych za pośrednictwem modelu i przesyła dane do szablonu w celu ich wyświetlenia.

Funkcję kontrolera pełni sam framework. Przesyła żądania do odpowiedniego widoku, zgodnie z konfiguracją URL frameworka Django.

Podczas tworzenia dowolnego projektu Django zawsze będziesz pracować z modelami, widokami, szablonami i adresami URL. W tym rozdziale dowiesz się, jakie zachodzą między nimi interakcje.

Architektura Django

Sposób przetwarzania żądań oraz zarządzania cyklem żądanie-odpowiedź przez różne komponenty frameworka Django: adresy URL, widoki, modele i szablony pokazano na rysunku 1.1.



Rysunek 1.1. Architektura Django

Oto w jaki sposób Django obsługuje żądania HTTP i generuje odpowiedzi:

1. Przeglądarka internetowa żąda strony o określonym adresie URL, a serwer WWW przekazuje żądanie HTTP do Django.
2. Django przegląda swoje skonfigurowane wzorce adresów URL i zatrzymuje się na pierwszym, który pasuje do żądanego adresu URL.
3. Django uruchamia widok, który odpowiada dopasowanemu wzorcowi adresu URL.
4. Do pobierania informacji z bazy danych widok potencjalnie używa modeli danych.

5. Modele danych określają definicję danych i ich zachowania. Są one używane do wysyłania zapytań do bazy danych.
6. W celu wyświetlenia danych widok renderuje szablon (zwykle HTML) i zwraca go wraz z odpowiedzią HTTP.

Do opisu cyklu żądanie-odpowiedź frameworka Django wrócimy na końcu tego rozdziału, w podrozdziale „Cykl żądanie-odpowiedź".

W procesie żądanie-odpowiedź framework Django wykorzystuje również haki, które nazywane są warstwą *middleware* (dosłownie: oprogramowaniem pośredniczącym). W celu zachowania prostoty opisu warstwa middleware celowo nie została uwzględniona na rysunku 1.1. Z warstwy middleware skorzystasz w różnych przykładach tej książki. W rozdziale 17., „Wdrożenie", dowiesz się również, jak tworzyć spersonalizowane warstwy middleware.

Omówiłem podstawowe komponenty Django i sposób, w jaki Django przetwarza żądania. Przyjrzyjmy się nowym funkcjonalnościom wprowadzonym w Django 5.

Nowe funkcjonalności w Django 5

W Django 5 wprowadzono kilka kluczowych funkcjonalności, z których będziemy korzystać w przykładach zamieszczonych w tej książce. W tej wersji Django niektóre funkcjonalności oznaczono jako przestarzałe oraz usunięto kilka, które były oznaczone jako przestarzałe we wcześniejszych wersjach. W Django 5.0 wprowadzono nowe funkcjonalności:

- **Filtry aspektów w witrynie administracyjnej**: w nowej wersji Django w witrynie administracyjnej można dodawać filtry aspektów (ang. *facet filters*). Po włączeniu tej opcji na liście obiektów administratora wyświetlane są liczby aspektów dla zastosowanych filtrów. Funkcjonalność tę opisałem w punkcie „Dodawanie liczby aspektów do filtrów", w dalszej części tego rozdziału.
- **Uproszczone szablony do renderowania pól formularzy**: dzięki możliwości definiowania grup pól z powiązanymi szablonami uproszczono renderowanie pól formularzy. Ma to na celu usprawnienie procesu renderowania powiązanych elementów formularza Django, takich jak etykiety, widżety, tekst pomocy i błędy. Przykład wykorzystania grup pól można znaleźć w podrozdziale „Tworzenie szablonów formularza komentarza" w rozdziale 2., „Usprawnianie bloga i dodanie funkcjonalności społecznościowych".
- **Wartości domyślne obliczone przez bazę danych**: w nowej wersji Django wprowadzono wartości domyślne obliczane przez bazę danych. Przykład tej funkcjonalności zaprezentowano w punkcie „Dodawanie pól daty i godziny" w dalszej części tego rozdziału.
- **Pola modelu generowane przez bazę danych**: Jest to nowy typ pola, umożliwiający tworzenie kolumn generowanych przez bazę danych. Do automatycznego ustawiania wartości pola przy każdej zmianie modelu definiuje się wyrażenie. Wartość pola ustawia się za pomocą instrukcji SQL `GENERATED ALWAYS`.

- **Więcej możliwości deklarowania opcji wyboru dla pól modelu**: pola obsługujące wybory w celu uzyskania dostępu do typów wyliczeniowych nie wymagają już dostępu do atrybutu `.choices`. Do rozszerzenia typów wyliczeniowych zamiast obiektów `iterable` można użyć bezpośrednio mapowania lub obiektów `callable`. Pola wyboru z typami wyliczeniowymi w przykładach w tej książce zaktualizowano w celu odzwierciedlenia tych zmian. Przykład można znaleźć w punkcie „Dodawanie pola stanu" w dalszej części tego rozdziału.

W Django 5 wprowadzono również pewne ulepszenia w obsłudze trybu asynchronicznego. **Obsługa ASGI** (ang. *Asynchronous Server Gateway Interface*) została wprowadzona w Django 3 i ulepszona w Django 4.1 z asynchronicznymi handlerami dla widoków opartych na klasach i asynchronicznym interfejsem ORM. W Django 5 dodano funkcje asynchroniczne do frameworka uwierzytelniania, zapewniono obsługę asynchronicznego wysyłania sygnałów i dodano obsługę trybu asynchronicznego dla wielu wbudowanych dekoratorów.

Z Django 5.0 usunięto obsługę Pythona 3.8 i 3.9.

Pełną listę zmian można przeczytać w dokumencie *release notes* dla Django 5.0, na stronie *https://docs.djangoproject.com/en/5.0/releases/5.0/*.

Ponieważ Django 5.0 to zaplanowane wydanie okresowe, nie wprowadzono w nim wielu podstawowych zmian. Dzięki temu aktualizacja aplikacji Django 4 do wydania 5.0 nie powinna stwarzać zbyt wielu trudności.

Aby szybko zaktualizować istniejący projekt Django do wydania 5.0, można użyć narzędzia `django-upgrade`. Ten pakiet przepisuje pliki projektu i wykorzystuje skrypty poprawiające kod do wersji docelowej (ang. *fixers*). Instrukcje korzystania z narzędzia `django-↪upgrade` można znaleźć pod adresem *https://github.com/adamchainz/django-upgrade*.

Narzędzie `django-upgrade` jest inspirowane pakietem `pyupgrade`. Z pakietu `pyupgrade` można skorzystać w celu automatycznego aktualizowania składni do nowszych wersji Pythona. Więcej informacji na temat pakietu `pyupgrade` można znaleźć pod adresem *https://docs.python.org/3/library/venv.html*.

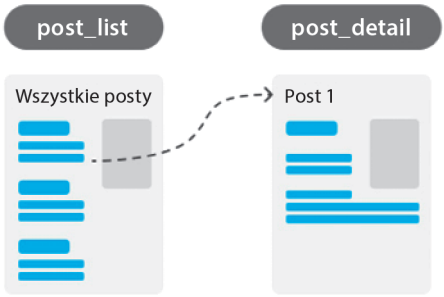
Tworzenie pierwszego projektu

Pierwszym projektem Django, który zbudujemy, będzie blog. Stworzenie tej aplikacji będzie solidnym wprowadzeniem pozwalającym zapoznać się z możliwościami i funkcjonalnościami Django.

Wziąwszy pod uwagę szeroki zakres wymaganych funkcjonalności bloga — od podstawowego zarządzania treścią po funkcjonalności zaawansowane, takie jak komentowanie, udostępnianie postów, wyszukiwanie i rekomendacje, idealnie nadaje się on do zbudowania kompletnego projektu Django. Projekt bloga omawiam w pierwszych trzech rozdziałach tej książki.

W tym rozdziale zaczniemy od utworzenia projektu Django i aplikacji Django `blog`. Następnie stworzymy modele danych i zsynchronizujemy je z bazą danych. Na koniec stworzymy stronę administracyjną dla bloga i zbudujemy widoki, szablony i adresy URL.

Reprezentację stron aplikacji bloga, które stworzymy, przedstawiono na rysunku 1.2.



Rysunek 1.2. Schemat funkcjonalności zbudowanych w rozdziale 1.

Aplikacja bloga składa się z listy postów zawierających tytuł posta, datę publikacji, autora, fragment posta i prowadzący do niego link. Stronę listy postów zaimplementujemy za pomocą widoku `post_list`. W tym rozdziale dowiesz się, jak tworzyć widoki.

Gdy czytelnik kliknie link do posta na stronie z listą postów, zostanie przekierowany do widoku pojedynczego posta (widoku szczegółów). Widok szczegółów wyświetla tytuł posta, datę publikacji, autora i całą treść posta.

Zacznijmy od utworzenia projektu Django dla bloga. Django dostarcza polecenie, które pozwala na opracowanie początkowej struktury plików projektu.

Uruchom ze swojej powłoki polecenie:

```
django-admin startproject mysite
```

Spowoduje to utworzenie projektu Django o nazwie `mysite`.

Wskazówka

Aby nie powodować konfliktów, unikaj nazywania projektów z użyciem takich samych nazw, jakie mają wbudowane moduły Pythona lub Django.

Przyjrzyjmy się wygenerowanej strukturze projektu.

```
mysite/
    manage.py
    mysite/
        __init__.py
       asgi.py
       settings.py
       urls.py
       wsgi.py
```

Zewnętrzny katalog *mysite/* jest kontenerem dla utworzonego projektu. Zawiera następujące pliki.

- *manage.py*: jest to narzędzie wiersza polecenia używane do interakcji z projektem. Zwykle nie będziesz edytować tego pliku.
- *mysite/*: pakiet języka Python dla projektu, który składa się z plików:
  - *__init__.py*: pusty plik, który informuje Pythona, aby traktował katalog *mysite* jako moduł Pythona;
  - *asgi.py*: konfiguracja do uruchomienia projektu jako aplikacji ASGI (ang. *Asynchronous Server Gateway Interface*) dla serwerów WWW kompatybilnych z ASGI. ASGI to nowy standard Pythona dla asynchronicznych serwerów WWW i aplikacji;
  - *settings.py*: zawiera parametry i konfigurację projektu oraz ustawienia domyślne;
  - *urls.py*: w tym pliku są zapisane wzorce adresów URL. Każdy zdefiniowany tutaj adres URL jest zmapowany na widok;
  - *wsgi.py*: konfiguracja do uruchomienia projektu jako aplikacji WSGI (ang. *Web Server Gateway Interface*) dla serwerów WWW zgodnych z WSGI.

Stosowanie początkowych migracji bazy danych

Aplikacje Django do przechowywania danych wymagają bazy danych. Plik *settings.py* w ustawieniu `DATABASES` zawiera konfigurację bazy danych dla projektu. W domyślnej konfiguracji używana jest baza danych SQLite3. Obsługa bazy danych SQLite jest dostarczana wraz z Pythonem 3 i może być używana w dowolnej aplikacji Pythona. SQLite to prosta baza danych, której można używać do programowania z Django. Jeśli planujesz wdrażać swoje aplikacje w środowisku produkcyjnym, powinieneś użyć bardziej rozbudowanej bazy danych, takiej jak PostgreSQL, MySQL lub Oracle. Więcej informacji na temat instalacji bazy danych i jej współpracy z Django znajdziesz na stronie *https://docs.djangoproject.com/en/5.0/topics/install/#database-installation.*

Wygenerowany plik *settings.py* zawiera również listę o nazwie `INSTALLED_APPS`, w której znajdują się popularne aplikacje Django, dodawane do projektu domyślnie. Omówię te aplikacje później, w punkcie „Ustawienia projektu".

Aplikacje Django zawierają modele danych, które są mapowane na tabele bazy danych. Własne modele utworzysz w podrozdziale „Tworzenie modeli danych dla bloga". Aby ukończyć konfigurację projektu, trzeba utworzyć w bazie danych tabele wymagane przez aplikacje wymienione na liście `INSTALLED_APPS`. Django ma system, który pomaga zarządzać migracjami baz danych.

Otwórz powłokę i uruchom polecenia:

```
cd mysite
python manage.py migrate
```

Wyświetli się wynik podobny do poniższego.

```
Applying contenttypes.0001_initial... OK
Applying auth.0001_initial... OK
Applying admin.0001_initial... OK
Applying admin.0002_logentry_remove_auto_add... OK
Applying admin.0003_logentry_add_action_flag_choices... OK
Applying contenttypes.0002_remove_content_type_name... OK
Applying auth.0002_alter_permission_name_max_length... OK
Applying auth.0003_alter_user_email_max_length... OK
Applying auth.0004_alter_user_username_opts... OK
Applying auth.0005_alter_user_last_login_null... OK
Applying auth.0006_require_contenttypes_0002... OK
Applying auth.0007_alter_validators_add_error_messages... OK
Applying auth.0008_alter_user_username_max_length... OK
Applying auth.0009_alter_user_last_name_max_length... OK
Applying auth.0010_alter_group_name_max_length... OK
Applying auth.0011_update_proxy_permissions... OK
Applying auth.0012_alter_user_first_name_max_length... OK
Applying sessions.0001_initial... OK
```

Powyższe wiersze opisują migracje bazy danych, które zostały zastosowane przez Django. W wyniku zastosowania migracji w bazie danych są tworzone tabele dla początkowych aplikacji wymienionych w ustawieniu `INSTALLED_APPS`.

Więcej informacji o poleceniu zarządzania migracjami `migrate` można znaleźć w punkcie „Tworzenie i stosowanie migracji", dalej w tym rozdziale.

Uruchamianie serwera programistycznego

Framework Django jest wyposażony w prosty serwer WWW, który umożliwia szybkie uruchamianie kodu bez konieczności poświęcania czasu na konfigurację serwera produkcyjnego. Serwer programistyczny Django po uruchomieniu sprawdza zmiany wprowadzone w kodzie. Automatycznie się przeładowuje, co zwalnia użytkownika z konieczności ponownego ładowania po wprowadzeniu zmian w kodzie. Może się jednak zdarzyć, że pewne działania, np. dodanie nowych plików do projektu, nie zostaną zauważone. Dlatego w takich przypadkach konieczne jest ręczne zrestartowanie serwera.

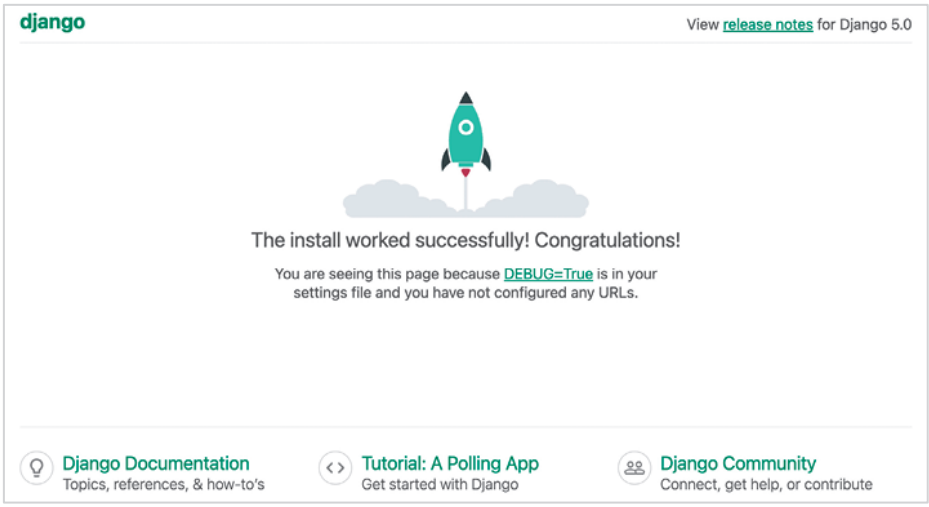
Aby uruchomić serwer programistyczny, wpisz w głównym folderze projektu polecenie:

```
python manage.py runserver
```

Powinieneś zobaczyć następujący wynik.

```
Watching for file changes with StatReloader
Performing system checks...
System check identified no issues (0 silenced).
January 01, 2024 - 10:00:00
Django version 5.0, using settings 'mysite.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CONTROL-C.
```

Teraz w przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/`. Powinieneś zobaczyć stronę z informacją o zakończonym powodzeniem uruchomieniu projektu, co pokazałem na rysunku 1.3.



Rysunek 1.3. Domyślna strona serwera programistycznego Django

Powyższy zrzut ekranu potwierdza, że Django działa. Jeśli spojrzysz na konsolę, zobaczysz wykonane przez Twoją przeglądarkę żądanie GET.

```
[01/Jan/2024 10:00:15] "GET / HTTP/1.1" 200 16351
```

Serwer programistyczny rejestruje w konsoli wszystkie żądania HTTP. W konsoli wyświetlają się również wszelkie błędy występujące podczas działania serwera.

Możesz zlecić Django, aby serwer programistyczny był uruchamiany na niestandardowych hoście i porcie.

```
python manage.py runserver 127.0.0.1:8001 --settings=mysite.settings
```

Wskazówka

Gdy w projekcie obsługujesz wiele środowisk, które wymagają różnych konfiguracji, możesz opracować dla każdego środowiska inny plik ustawień.

Należy zapamiętać, że serwer programistyczny jest przeznaczony tylko do przygotowania aplikacji. Aby wdrożyć Django w środowisku produkcyjnym, należy uruchomić go jako aplikację WSGI przy użyciu produkcyjnego serwera WWW, takiego jak Apache, Gunicorn lub uWSGI bądź też jako aplikację ASGI z wykorzystaniem serwera Uvicorn albo Daphne.

Więcej informacji na temat wdrażania Django na różnych serwerach internetowych można znaleźć na stronie *https://docs.djangoproject.com/en/5.0/howto/deployment/wsgi/*.

Sposób konfiguracji środowiska produkcyjnego dla projektów Django opisałem w rozdziale 17., „Wdrożenie”.

Ustawienia projektu

Otwórzmy plik *settings.py* i spójrzmy na konfigurację utworzonego projektu. Jest kilka ustawień, które Django czyta z tego pliku, ale to tylko część wszystkich dostępnych ustawień Django. Wszystkie ustawienia i ich domyślne wartości można znaleźć pod adresem *https://docs.djangoproject.com/en/5.0/ref/settings/*.

Warto przyjrzeć się bliżej następującym ustawieniom.

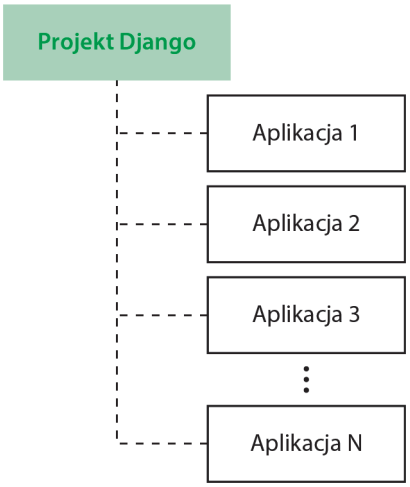
- `DEBUG` to wartość typu boolean; w zależności od ustawienia włącza bądź wyłącza tryb debugowania projektu. Jeśli jest ustawione na `True`, Django wyświetla szczegółowe strony błędów, gdy aplikacja zgłosi nieobsłużony wyjątek. Należy pamiętać, aby po przejściu do środowiska produkcyjnego ustawić je na `False`. Nigdy nie wdrażaj witryny do produkcji z włączonym ustawieniem `DEBUG`, ponieważ może to doprowadzić do ujawnienia poufnych danych związanych z projektem.
- Ustawienie `ALLOWED_HOSTS` nie jest stosowane przy włączonym trybie debugowania albo gdy są uruchamiane testy. Po przeniesieniu witryny do produkcji i ustawieniu `DEBUG` na `False`, aby umożliwić obsługę witryny Django, trzeba dodać do tego ustawienia domenę (hosta).
- `INSTALLED_APPS` to ustawienie, które trzeba wyedytować dla wszystkich projektów. Informuje ono Django o tym, które aplikacje są aktywne dla tej witryny. Domyślnie Django włącza następujące aplikacje:
  - `django.contrib.admin`: witryna administracyjna;
  - `django.contrib.auth`: framework uwierzytelniania;
  - `django.contrib.contenttypes`: framework obsługi typów zawartości;
  - `django.contrib.sessions`: framework obsługi sesji;
  - `django.contrib.messages`: framework obsługi komunikatów;
  - `django.contrib.staticfiles`: framework zarządzania statycznymi plikami, takimi jak arkusze CSS, pliki JavaScript i obrazy.
- `MIDDLEWARE` to lista zawierająca oprogramowanie middleware do uruchomienia.
- `ROOT_URLCONF` wskazuje moduł Pythona, w którym zdefiniowano wzorce URL aplikacji.
- `DATABASES` to słownik zawierający ustawienia wszystkich baz danych, które będą używane w projekcie. W projekcie zawsze musi być domyślna baza danych. W domyślnej konfiguracji używana jest baza danych SQLite3.
- `LANGUAGE_CODE` definiuje kod domyślnego języka dla witryny Django.
- Ustawienie `USE_TZ` informuje Django, aby aktywował (dezaktywował) obsługę stref czasowych. Django zapewnia obsługę typu `datetime` uwzględniającego strefy czasowe. Po utworzeniu nowego projektu za pomocą polecenia zarządzającego `startproject` opcja ta jest ustawiona na `True`.

Nie martw się, jeśli nie rozumiesz zbyt wiele z tego, co widzisz. Z różnymi ustawieniami Django zapoznasz się w kolejnych rozdziałach.

Projekty i aplikacje

Na stronach książki nieustannie pojawiać się będą słowa **projekt** i **aplikacja**. We frameworku Django za projekt jest uznawana instalacja Django wraz z pewnymi ustawieniami, natomiast aplikacja to grupa modeli, widoków, szablonów i adresów URL. Aplikacja współpracuje z frameworkiem w celu dostarczenia pewnych konkretnych funkcjonalności i może być wielokrotnie używana w różnych projektach. Projekt możesz potraktować jak witrynę internetową zawierającą kilka aplikacji, takich jak blog, wiki lub forum. Poszczególne aplikacje można później wykorzystać w innych projektach.

Strukturę projektu Django pokazałem na rysunku 1.4.



Rysunek 1.4. Struktura aplikacji (projektu) Django

Utworzenie aplikacji

Przystępujemy teraz do zbudowania pierwszej aplikacji Django. Zupełnie od początku opracujemy aplikację bloga.

Przejdź do katalogu głównego projektu, a następnie wydaj poniższe polecenie.

```
python manage.py startapp blog
```

Spowoduje to utworzenie podstawowej struktury aplikacji, która wygląda tak:

```
blog/
     __init__.py
     admin.py
     apps.py
     migrations/
         __init__.py
         models.py
         tests.py
         views.py
```

Utworzone pliki mają następującą zawartość.

- *__init__.py*: pusty plik, który informuje Python, aby traktował katalog blog jako moduł Pythona.
- *admin.py*: w tym pliku rejestrujemy modele, które mają być uwzględnione w witrynie administracyjnej Django — korzystanie z witryny administracyjnej Django jest opcjonalne.
- *apps.py*: zawiera główną konfigurację aplikacji bloga.
- *migrations*: ten katalog będzie zawierał migracje bazy danych Twojej aplikacji. Migracje pozwalają Django śledzić zmiany modelu i odpowiednio synchronizować bazę danych. Ten katalog zawiera pusty plik *__init__.py*.
- *models.py*: modele danych aplikacji — plik *models.py* musi znajdować się we wszystkich aplikacjach Django, choć może pozostać pusty.
- *tests.py*: w tym pliku można wprowadzić testy aplikacji.
- *views.py*: w tym pliku znajduje się logika aplikacji. Każdy widok otrzymuje żądanie HTTP, przetwarza je i zwraca odpowiedź.

Mając gotową strukturę aplikacji, możemy przystąpić do budowania modeli danych dla aplikacji bloga.

Tworzenie modeli danych dla bloga

Warto zapamiętać, że obiekt Pythona jest zbiorem danych i metod. Klasy są wzorcem łączenia ze sobą danych i funkcjonalności. Utworzenie nowej klasy powoduje utworzenie nowego typu obiektu, co umożliwia tworzenie egzemplarzy tego typu.

Model Django jest źródłem informacji i zachowań Twoich danych. Składa się z klasy Pythona, która jest podklasą klasy `django.db.models.Model`. Każdy model można zmapować na pojedynczą tabelę bazy danych, a każdy atrybut tej klasy reprezentuje pole bazy danych.

Kiedy definiujesz model, Django udostępnia praktyczny interfejs API do łatwego odpytywania obiektów w bazie danych.

Spróbujmy zdefiniować modele baz danych dla przykładowej aplikacji bloga. Następnie, aby utworzyć odpowiednie tabele bazy danych, wygenerujemy migracje bazy danych dla modeli. Gdy stosujemy migracje, Django tworzy tabelę dla każdego modelu zdefiniowanego w pliku *models.py* aplikacji.

Utworzenie modelu Post

Najpierw zdefiniujemy model `Post`.

Dodaj do pliku *models.py* aplikacji blog następujące wiersze. Nowe wiersze zostały wyróżnione pogrubioną czcionką.

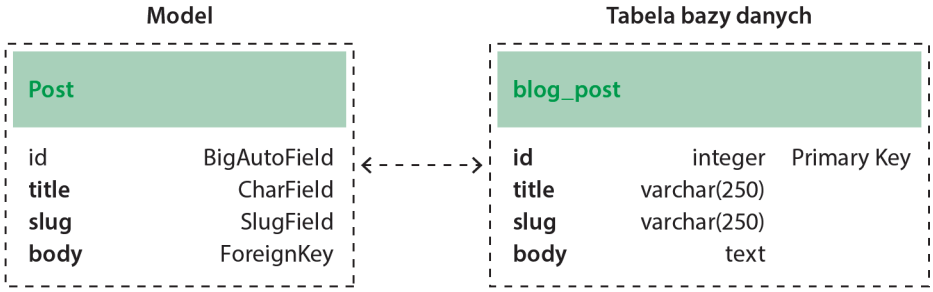
```
from django.db import models

class Post(models.Model):
    title = models.CharField(max_length=250)
    slug = models.SlugField(max_length=250)
    body = models.TextField()

    def __str__(self):
        return self.title
```

Jest to podstawowy model dla postów bloga. Posty mają tytuł (`title`), krótką etykietę określaną jako slug i treść (`body`). Przeanalizujemy teraz poszczególne kolumny zdefiniowane dla tego modelu.

- `title`: pole reprezentujące tytuł posta. Jest to pole typu `CharField`, co odpowiada kolumnie `VARCHAR` w bazie danych SQL.
- `slug`: jest to pole typu `SlugField`, co odpowiada kolumnie `VARCHAR` w bazie danych SQL. Jest to krótka etykieta zawierająca wyłącznie litery, cyfry, znaki podkreślenia lub łączniki. Post o tytule *Django Reinhardt: legenda jazzu* może mieć slug postaci *django-reinhardt-legenda-jazzu*. Pola `slug` będziemy używać w rozdziale 2., „Usprawnianie bloga i dodanie funkcjonalności społecznościowych", do tworzenia opisowych, przyjaznych dla SEO (ang. *Search Engine Optimization*) adresów URL postów na blogu.
- `body`: to pole zawiera treść posta. Jest to pole typu `TextField`, co odpowiada kolumnie `TEXT` w bazie danych SQL.

Do modelu klasy dodaliśmy również metodę `__str__()`. Jest to domyślna metoda Pythona zwracająca ciąg znaków z czytelną dla człowieka reprezentacją obiektu. Django używa jej w wielu miejscach, np. w witrynie administracyjnej.

Przyjrzyjmy się, w jaki sposób model i jego pola zostaną przetłumaczone na tabelę bazy danych i kolumny. Diagram na rysunku 1.5 pokazuje model `Post` i odpowiadającą mu tabelę bazy danych, którą Django utworzy, gdy zsynchronizujemy model z bazą danych.



Rysunek 1.5. Początkowy model Post i odpowiadająca mu tabela bazy danych

Django utworzy kolumnę bazy danych dla każdego z pól modelu: `title`, `slug` i `body`. Łatwo zauważyć, że każdy typ pola odpowiada typowi danych w bazie danych.

Do każdego modelu Django domyślnie dodaje automatycznie zwiększające się pole klucza głównego. Typ dla tego pola jest określony w konfiguracji każdej aplikacji lub globalnie,

za pomocą ustawienia DEFAULT_AUTO_FIELD. Podczas tworzenia aplikacji za pomocą polecenia `startapp` domyślną wartością opcji `DEFAULT_AUTO_FIELD` jest `BigAutoField`. Jest to 64-bitowa liczba całkowita, która zwiększa się automatycznie, zgodnie z dostępnymi identyfikatorami. Jeśli nie określisz klucza podstawowego dla swojego modelu, Django doda to pole automatycznie. Jako klucz główny możesz również zdefiniować jedno z pól modelu. Aby to zrobić, należy w tym polu użyć klauzuli `primary_key=True`.

Model `Post` rozszerzymy o dodatkowe pola i zachowania. Po zdefiniowaniu kompletnego modelu zsynchronizujemy go z bazą danych przez utworzenie migracji i zastosowanie jej.

Dodawanie pól daty i godziny

W następnej kolejności dodamy do modelu `Post` różne pola opisujące daty i godziny. Każdy post zostanie opublikowany w określonym dniu i o określonej godzinie. Dlatego potrzebujemy pola do przechowywania daty i godziny publikacji. Chcemy również przechowywać datę i godzinę utworzenia obiektu Post oraz datę jego ostatniej modyfikacji.

Wyedytuj plik *models.py* w aplikacji bloga i doprowadź go do postaci pokazanej na poniższym listingu.

```
from django.db import models
from django.utils import timezone

class Post(models.Model):
    title = models.CharField(max_length=250)
    slug = models.SlugField(max_length=250)
    body = models.TextField()
    publish = models.DateTimeField(default=timezone.now)

    def __str__(self):
        return self.title
```

Do modelu `Post` dodaliśmy pole `publish`. Jest to pole typu `DateTimeField`, co odpowiada kolumnie `DATETIME` w bazie danych SQL. Użyjemy go do przechowywania daty i godziny opublikowania posta. Wartość domyślna tego pola to wartość zwracana przez metodę `now` obiektu `timezone` frameworka Django. Zwróć uwagę, że aby skorzystać z tej metody, trzeba było zaimportować moduł `timezone`. Metoda `now` zwraca bieżącą wartość `DateTime` w formacie uwzględniającym strefę czasową. Wartość tę można uważać za wersję standardowej metody Pythona `datetime.now` z obsługą stref czasowych.

Inną metodą definiowania wartości domyślnych dla pól modelu jest użycie wartości domyślnych obliczonych przez bazę danych. Jest to nowa funkcjonalność wprowadzona w Django 5, pozwalająca na używanie funkcji bazy danych do generowania wartości domyślnych. Na przykład w poniższym kodzie do obliczenia wartości domyślnej pola `publish` wykorzystano bieżącą datę i godzinę serwera bazy danych:

```
from django.db import models
from django.db.models.functions import Now
class Post(models.Model):
    # ...
    publish = models.DateTimeField(db_default=Now())
```

Aby użyć wartości domyślnych wygenerowanych przez bazę danych, zamiast `default` używamy atrybutu `db_default`. W tym przykładzie skorzystaliśmy z funkcji bazy danych `Now`. Funkcja ta ma podobny cel co ustawienie `default=timezone.now`, ale zamiast daty i godziny wygenerowanej przez Pythona do wygenerowania wartości początkowej wykorzystuje funkcję bazy danych `NOW()`. Więcej informacji na temat atrybutu `db_default` można znaleźć pod adresem *https://docs.djangoproject.com/en/5.0/ref/models/fields/#django.db.models.Field.db_default*. Wszystkie dostępne funkcje bazy danych można znaleźć pod adresem *https://docs.djangoproject.com/en/5.0/ref/models/database-functions/*.

Kontynuujmy z poprzednią wersją pola:

```
class Post(models.Model):
    # ...
    publish = models.DateTimeField(default=timezone.now)
```

Wyedytuj plik *models.py* aplikacji bloga i dodaj wiersze, które na poniższym listingu zaznaczono pogrubieniem.

```
from django.db import models
from django.utils import timezone

class Post(models.Model):
    title = models.CharField(max_length=250)
    slug = models.SlugField(max_length=250)
    body = models.TextField()
    publish = models.DateTimeField(default=timezone.now)
    created = models.DateTimeField(auto_now_add=True)
    updated = models.DateTimeField(auto_now=True)

    def __str__(self):
        return self.title
```

Do modelu `Post` dodaliśmy pola:

- `created`: pole typu `DateTimeField`. Użyjemy go do przechowywania daty i godziny utworzenia posta. Ponieważ tutaj używamy opcji `auto_now_add`, data zostanie zapisana automatycznie podczas tworzenia obiektu.
- `updated`: pole typu `DateTimeField`. Użyjemy go do przechowywania daty i godziny aktualizacji posta. Ponieważ tutaj używamy opcji `auto_now`, data zostanie zaktualizowana automatycznie podczas zapisywania obiektu.

Wskazówka

Wykorzystanie pól daty `auto_now_add` i `auto_now` w modelach Django jest bardzo przydatne do śledzenia czasu utworzenia obiektów i ich ostatniej modyfikacji.

Definiowanie domyślnej kolejności sortowania

Posty na blogu są zwykle wyświetlane w odwrotnej kolejności chronologicznej, zatem najpierw są wyświetlane posty najnowsze. Dla tworzonego modelu zdefiniujemy domyślną kolejność. Ta kolejność będzie stosowana podczas pobierania obiektów z bazy danych, chyba że w zapytaniu zostanie wskazana konkretna kolejność.

Wyedytuj plik *models.py* w aplikacji bloga i doprowadź go do postaci pokazanej na poniższym listingu. Nowe wiersze zostały wyróżnione pogrubioną czcionką.

```
from django.db import models
from django.utils import timezone

class Post(models.Model):
    title = models.CharField(max_length=250)
    slug = models.SlugField(max_length=250)
    body = models.TextField()
    publish = models.DateTimeField(default=timezone.now)
    created = models.DateTimeField(auto_now_add=True)
    updated = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ['-publish']

    def __str__(self):
        return self.title
```

Dodaliśmy wewnątrz modelu klasę `Meta`. Ta klasa definiuje metadane dla modelu. Aby zlecić frameworkowi Django posortowanie wyników według pola `publish`, skorzystaliśmy z atrybutu `ordering`. Ustalona kolejność będzie stosowana domyślnie dla tych kwerend bazy danych, w których nie podano konkretnej kolejności. Określiliśmy kolejność malejącą przez podanie łącznika przed nazwą pola: `-publish`. Posty będą domyślnie zwracane w odwrotnej kolejności chronologicznej.

Dodawanie indeksu bazy danych

Zdefiniujmy indeks bazy danych dla pola `publish`. Poprawi to wydajność filtrowania zapytań lub sortowania wyników według tego pola. Spodziewamy się, że z tego indeksu skorzysta wiele zapytań, ponieważ pole `publish` jest wykorzystywane do sortowania wyników domyślnie.

Wyedytuj plik *models.py* w aplikacji bloga i doprowadź go do postaci pokazanej na poniższym listingu. Nowe wiersze zostały wyróżnione pogrubioną czcionką.

```
from django.db import models
from django.utils import timezone

class Post(models.Model):
    title = models.CharField(max_length=250)
    slug = models.SlugField(max_length=250)
    body = models.TextField()
    publish = models.DateTimeField(default=timezone.now)
    created = models.DateTimeField(auto_now_add=True)
    updated = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ['-publish']
        indexes = [
            models.Index(fields=['-publish']),
        ]
```

```
    def __str__(self):
        return self.title
```

Do klasy `Meta` modelu dodaliśmy opcję `indexes`. Ta opcja umożliwia zdefiniowanie dla modelu indeksów bazy danych, które mogą obejmować jedno lub wiele pól w porządku rosnącym lub malejącym, a także wywołania wyrażeń funkcyjnych i funkcji bazy danych. W tym przypadku dodaliśmy indeks dla pola `publish`. Aby zdefiniować indeks w porządku malejącym, przed nazwą pola należy wprowadzić łącznik. Utworzenie tego indeksu zostanie uwzględnione w migracjach bazy danych, które później wygenerujemy dla modeli aplikacji bloga.

Uwaga

Więcej informacji na temat definiowania indeksów dla modeli można znaleźć na stronie *https://docs.djangoproject.com/en/5.0/ref/models/indexes/*.

Więcej informacji na temat definiowania indeksów dla modeli można znaleźć na stronie *https://docs.djangoproject.com/en/5.0/ref/models/indexes/*.

Aktywacja aplikacji

Aby framework Django mógł śledzić aplikację i tworzyć tabele bazy danych dla jej modeli, trzeba uaktywnić aplikację blog w projekcie.

Aby to zrobić, wyedytuj plik *settings.py* i dodaj parametr `blog.apps.BlogConfig` do ustawienia `INSTALLED_APPS`. Powinno ono mieć postać pokazaną poniżej. Nowe wiersze zostały wyróżnione pogrubioną czcionką:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'blog.apps.BlogConfig',
]
```

Klasa `BlogConfig` zawiera konfigurację aplikacji. Teraz Django ma informację, że aplikacja jest dla tego projektu aktywna, i może wczytać jej modele.

Dodawanie pola stanu

Typową funkcją blogów jest zapisywanie postów jako wersji roboczej, dopóki nie będą gotowe do publikacji. Aby zarządzać stanem postów na blogu, dodamy do modelu aplikacji pole `status`. Do określania statusu postów wykorzystamy wartości `Draft` i `Published`.

Wyedytuj plik *models.py* w aplikacji bloga i doprowadź go do postaci pokazanej na poniższym listingu. Nowe wiersze zostały wyróżnione pogrubioną czcionką.

```
from django.db import models
from django.utils import timezone


class Post(models.Model):
     class Status(models.TextChoices):
        DRAFT = 'DF', 'Roboczy'
        PUBLISHED = 'PB', 'Opublikowany'
    title = models.CharField(max_length=250)
    slug = models.SlugField(max_length=250)
    body = models.TextField()
    publish = models.DateTimeField(default=timezone.now)
    created = models.DateTimeField(auto_now_add=True)
    updated = models.DateTimeField(auto_now=True)
    status = models.CharField(
         max_length=2,
         choices=Status,
         default=Status.DRAFT
    )

    class Meta:
        ordering = ['-publish']
        indexes = [
            models.Index(fields=['-publish']),
         ]

    def __str__(self):
         return self.title
```

Zdefiniowaliśmy klasę wyliczeniową `Status`, która jest podklasą klasy `models.TextChoices`. Dostępne opcje statusu wpisu to `DRAFT` i `PUBLISHED`. Odpowiadające im wartości to `DF` i `PB`, a etykiety lub czytelne nazwy to *Roboczy* i *Opublikowany*.

Django zapewnia typy wyliczeniowe, po których można dziedziczyć, co pozwala w prosty sposób zdefiniować dostępne opcje. Są one oparte na obiekcie enum ze standardowej biblioteki Pythona. Więcej informacji o typie enum można znaleźć pod adresem *https://docs.python.org/3/library/enum.html*.

W typach wyliczeniowych Django wprowadzono pewne modyfikacje w stosunku do typu `enum`. Więcej informacji na temat tych różnic można znaleźć pod adresem *https://docs.djangoproject.com/en/5.0/ref/models/fields/#enumeration-types*.

Aby wyświetlić dostępne opcje, możemy odczytać listę `Post.Status.choices`; nazwy czytelne dla człowieka są dostępne za pośrednictwem listy `Post.Status.labels`, natomiast lista `Post.Status.values` zawiera rzeczywiste wartości wyborów.

Dodaliśmy również do modelu nowe pole `status`, które jest egzemplarzem klasy `CharField`. Zawiera parametr `choices`, który ogranicza wartość pola do wyborów należących do kolekcji `Status.choices`. Za pomocą parametru `default` ustawiliśmy także wartość domyślną dla pola. Jako domyślnego wyboru dla tego pola używamy ustawienia `DRAFT`.

Wskazówka

Definiowanie wyborów wewnątrz klasy modelu i używanie typów wyliczeniowych jest dobrą praktyką. Umożliwia to łatwe odwoływanie się do etykiet wyboru, wartości lub nazw z dowolnego miejsca w kodzie. Aby skorzystać w dowolnym miejscu kodu ze statusu *Roboczy*, możesz zaimportować model `Post` i użyć odwołania `Post.Status.DRAFT`.

Przyjrzyjmy się, jak wchodzić w interakcje z opcjami statusu.

Otwórz terminal i uruchom następujące polecenie, aby otworzyć powłokę Pythona.

```
python manage.py shell
```

Następnie wpisz wiersze:

```
>>> from blog.models import Post
>>> Post.status.choices
```

Wyświetlą się opcje wyliczeniowe z parami etykieta-wartość, w następującej postaci.

```
[('DF', 'Roboczy'), ('PB', 'Opublikowany')]
```

Wpisz wiersz:

```
>>> Post.Status.labels
```

Wyświetlą się czytelne dla człowieka nazwy elementów typu enum w następującej postaci.

```
['Roboczy', 'Opublikowany']
```

Wpisz wiersz:

```
>>> Post.Status.values
```

Wyświetli się lista wartości typu `enum` w postaci pokazanej poniżej. Są to wartości pola statusu, które mogą być przechowywane w bazie danych.

```
['DF', 'PB']
```

Wpisz wiersz:

```
>>> Post.Status.names
```

Wyświetli się lista dostępnych wyborów w następującej postaci.

```
['Roboczy', 'Opublikowany']
```

Dostęp do określonego elementu wyliczenia możesz uzyskać za pomocą odwołania `Post.`
`↪Status.PUBLISHED`. Możesz również uzyskać dostęp do jego właściwości `.name` i `.value`.

Dodanie relacji wiele do jednego

Każdy post ma autora. Stworzymy relację między użytkownikami a postami, która będzie wskazywać, który użytkownik napisał które posty. Django zapewnia framework uwierzytelniania, który obsługuje konta użytkowników. Framework uwierzytelniania Django jest dostarczany za pośrednictwem pakietu `django.contrib.auth` i zawiera model `User`. Aby zdefiniować relację między użytkownikami a postami, użyjemy ustawienia `AUTH_USER_MODEL`, które domyślnie wskazuje na `auth.User`. To ustawienie umożliwia określenie innego modelu użytkownika dla projektu.

Wyedytuj plik *models.py* w aplikacji bloga i doprowadź go do postaci pokazanej na poniższym listingu. Nowe wiersze zostały wyróżnione pogrubioną czcionką.

```
from django.conf import settings
from django.db import models
from django.utils import timezone

class Post(models.Model):

    class Status(models.TextChoices):
        DRAFT = 'DF', 'Draft'
        PUBLISHED = 'PB', 'Published'

    title = models.CharField(max_length=250)
    slug = models.SlugField(max_length=250)
    author = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name='blog_posts'
    )
    body = models.TextField()
    publish = models.DateTimeField(default=timezone.now)
    created = models.DateTimeField(auto_now_add=True)
    updated = models.DateTimeField(auto_now=True)
    status = models.CharField(
        max_length=2,
        choices=Status,
        default=Status.DRAFT
    )

    class Meta:
        ordering = ['-publish']
        indexes = [
            models.Index(fields=['-publish']),
        ]

    def __str__(self):
        return self.title
```

Zaimportowaliśmy ustawienia projektu i dodaliśmy pole `author` do modelu `Post`. To pole definiuje relację wiele-do-jednego z domyślnym modelem użytkownika, co oznacza, że każdy post jest napisany przez użytkownika, a użytkownik może napisać dowolną liczbę postów. Za pomocą tego pola Django utworzy klucz obcy w bazie danych z wykorzystaniem klucza głównego powiązanego modelu.

Parametr `on_delete` określa zachowanie podczas usuwania obiektu. Nie jest ono specyficzne dla Django. Jest to zgodne ze standardem SQL. Jeśli wprowadzimy słowo kluczowe `CASCADE`, określamy, że po usunięciu użytkownika z bazy danych zostaną usunięte również powiązane z nim posty na blogu. Wszystkie możliwe opcje można znaleźć pod adresem *https://docs.djangoproject.com/en/5.0/ref/models/fields/#django.db.models.ForeignKey.on_delete*. Aby określić nazwę odwrotnej relacji — od użytkownika do posta — używamy odwołania `related_name`. Umożliwi to łatwy dostęp do powiązanych obiektów z poziomu obiektu użytkownika przy użyciu notacji `user.blog_posts`. Więcej na ten temat napiszę później.

W Django istnieją różne typy pól, których można użyć do zdefiniowania modeli. Wszystkie typy pól można znaleźć pod adresem *https://docs.djangoproject.com/en/5.0/ref/models/fields/*.

Model `Post` jest już gotowy i można go teraz zsynchronizować z bazą danych.

Tworzenie i stosowanie migracji

Teraz, gdy mamy model danych dla postów na blogu, musimy utworzyć odpowiednią tabelę bazy danych. Django posiada system migracji, który śledzi zmiany wprowadzone w modelach i umożliwia ich propagowanie w bazie danych.

Polecenie `migrate` stosuje migracje dla wszystkich aplikacji wymienionych w ustawieniu `INSTALLED_APPS`. Powoduje to synchronizację bazy danych z aktualnymi modelami i istniejącymi migracjami.

Najpierw dla naszego modelu `Post` należy utworzyć migrację początkową.

Uruchom następujące polecenie w wierszu polecenia powłoki, w głównym katalogu projektu.

```
python manage.py makemigrations blog
```

Powinieneś otrzymać wynik podobny do tego:

```
Migrations for 'blog':
    blog/migrations/0001_initial.py
        - Create model Post
        - Create index blog_post_publish_bb7600_idx on field(s)
          -publish of model post
```

Django właśnie utworzył w katalogu *migrations* aplikacji bloga *plik 0001_initial.py*. Ta migracja zawiera instrukcje SQL potrzebne do stworzenia tabeli bazy danych dla modelu `Post` oraz definicję indeksu bazy danych dla pola `publish`.

Aby zobaczyć, jak wygląda migracja, możesz otworzyć ten plik. Migracja określa zależności od innych migracji i operacji wykonywanych w bazie danych w celu zsynchronizowania ich ze zmianami w modelu.

Przyjrzyjmy się kodowi SQL, który Django wykonuje w bazie danych w celu utworzenia tabeli dla modelu `Post`. Polecenie `sqlmigrate` pobiera nazwy migracji i zwraca odpowiadający im kod SQL bez wykonywania go.

Aby sprawdzić instrukcje SQL pierwszej migracji, uruchom w wierszu polecenia powłoki polecenie:

```
python manage.py sqlmigrate blog 0001
```

Wynik powinien być podobny do następującego.

```
BEGIN;
--
-- Create model Post
--
CREATE TABLE "blog_post" (
  "id" integer NOT NULL PRIMARY KEY AUTOINCREMENT,
  "title" varchar(250) NOT NULL,
  "slug" varchar(250) NOT NULL,
  "body" text NOT NULL,
  "publish" datetime NOT NULL,
  "created" datetime NOT NULL,
  "updated" datetime NOT NULL,
  "status" varchar(10) NOT NULL,
  "author_id" integer NOT NULL REFERENCES "auth_user" ("id") DEFERRABLE
INITIALLY DEFERRED);
--
-- Create blog_post_publish_bb7600_idx on field(s) -publish of model post
--
CREATE INDEX "blog_post_publish_bb7600_idx" ON "blog_post" ("publish" DESC);
CREATE INDEX "blog_post_slug_b95473f2" ON "blog_post" ("slug");
CREATE INDEX "blog_post_author_id_dd7a8485" ON "blog_post" ("author_id");
COMMIT;
```

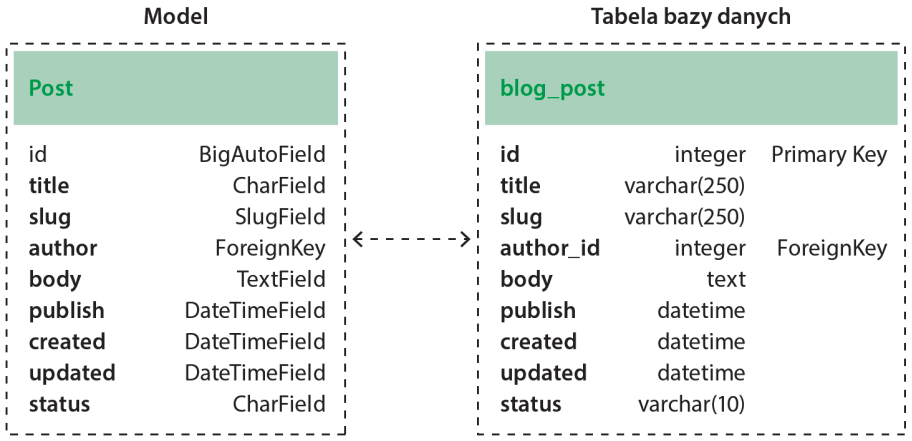
Dokładny wynik zależy od używanej bazy danych. Powyższy wynik został wygenerowany dla SQLite. Jak widać w powyższym wyniku, Django generuje nazwy tabel poprzez połączenie nazwy aplikacji i nazwy modelu zapisanych małymi literami (`blog_post`). Można jednak również, używając atrybutu `db_table`, podać niestandardową nazwę bazy danych dla modelu w klasie `Meta` modelu.

Django automatycznie tworzy klucz podstawowy dla każdego modelu. Można go jednak również nadpisać. W tym celu należy wpisać w jednym z pól modelu `primary_key = True`. Domyślnym kluczem podstawowym jest kolumna `id`, która zawiera automatycznie inkrementowaną liczbę całkowitą. Ta kolumna odpowiada polu `id`, które jest automatycznie dodawane do modeli.

Tworzone są trzy indeksy bazy danych:

- Indeks dla malejącego porządku w kolumnie `publish`. Jest to indeks, który jawnie zdefiniowaliśmy za pomocą opcji `indexes` w klasie `Meta` modelu.
- Indeks w kolumnie `slug`, ponieważ pola typu `SlugField` domyślnie implikują zdefiniowanie indeksu.
- Indeks w kolumnie `author_id`, ponieważ pola `ForeignKey` domyślnie implikują zdefiniowanie indeksu.

Sposób dopasowania pól modelu z kolumnami w bazie danych jest pokazany na rysunku 1.6. Porównajmy model `Post` z odpowiadającą mu tabelą `blog_post` w bazie danych:



Rysunek 1.6. Kompletny model Post i odpowiadająca mu tabela w bazie danych

Zsynchronizujmy bazę danych z nowym modelem.

Aby zastosować istniejące migracje, wykonaj w wierszu powłoki polecenie:

```
python manage.py migrate
```

Wyświetli się wynik, który kończy się wierszem:

```
Applying blog.0001_initial... OK
```

Właśnie zastosowaliśmy migracje do aplikacji wymienionych w ustawieniu `INSTALLED_APPS`, w tym do tworzonej aplikacji bloga. Po zastosowaniu migracji baza danych odzwierciedla bieżący stan modeli.

Jeśli zmodyfikujesz plik *models.py* w celu dodania, usunięcia lub zmiany pól istniejących modeli, albo jeśli dodasz nowe modele, powinieneś utworzyć nową migrację za pomocą polecenia `makemigrations`. Migracja pozwala frameworkowi Django śledzić zmiany w modelu. Następnie, aby zadbać o synchronizację bazy danych z modelami, trzeba zastosować tę migrację za pomocą polecenia `migrate`.

Tworzenie witryny administracyjnej dla modeli

Teraz, gdy model `Post` jest zsynchronizowany z bazą danych, możemy stworzyć prostą stronę administracyjną do zarządzania wpisami na blogu.

Django ma wbudowany interfejs administracyjny, który bardzo się przydaje do edycji treści. Strona administracyjna Django jest budowana dynamicznie poprzez czytanie metadanych modelu i udostępnianie gotowego do produkcji interfejsu edycji treści. Można go bardzo łatwo użyć. Wystarczy skonfigurować sposób wyświetlania modeli.

Aplikacja `django.contrib.admin` jest uwzględniona w ustawieniu `INSTALLED_APPS`, więc nie trzeba jej specjalnie dodawać.

Tworzenie superużytkownika

Najpierw trzeba utworzyć użytkownika do zarządzania stroną administracyjną. Uruchom polecenie:

```
python manage.py createsuperuser
```

Wyświetli się poniższy wynik. Wprowadź żądaną nazwę użytkownika, adres e-mail i hasło w następujący sposób.

```
Username (leave blank to use 'admin'): admin
Email address: admin@admin.com
Password: ********
Password (again): ********
```

Następnie wyświetli się komunikat o powodzeniu:

```
Superuser created successfully.
```

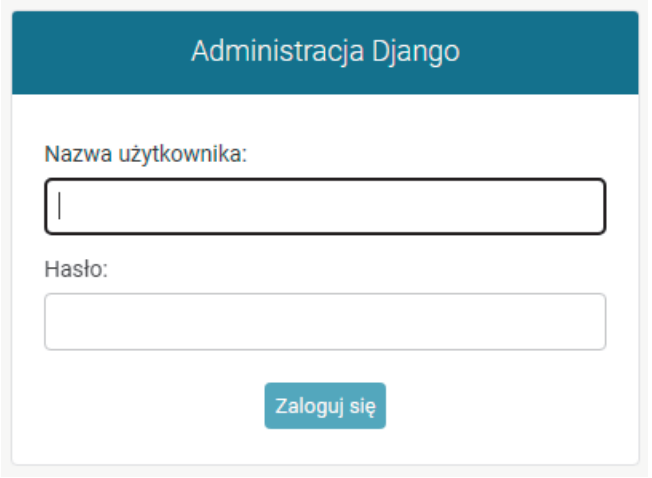
Właśnie utworzyłeś użytkownika admin z najwyższymi uprawnieniami.

Witryna administracyjna Django

Uruchom serwer programistyczny za pomocą polecenia:

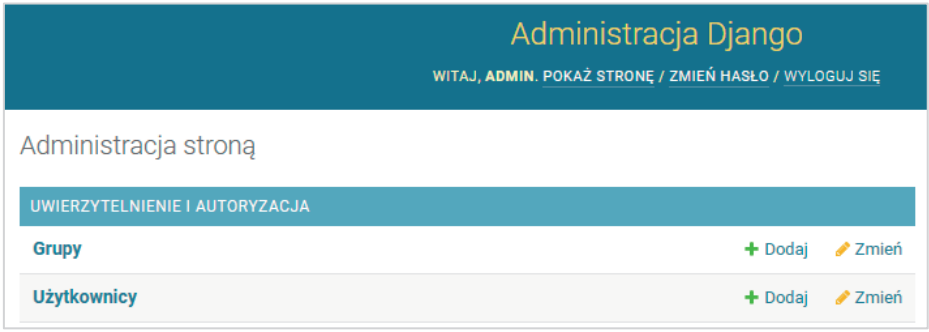
```
python manage.py runserver
```

Otwórz w przeglądarce stronę `http://127.0.0.1:8000/admin/`. Powinieneś zobaczyć stronę logowania administratora, taką jak pokazana na rysunku 1.7.



Rysunek 1.7. Ekran logowania do strony administracyjnej Django

Zaloguj się przy użyciu poświadczeń użytkownika utworzonego w poprzednim kroku. Powinieneś zobaczyć stronę główną witryny administracyjnej, taką jak pokazana na rysunku 1.8.



Rysunek 1.8. Strona główna administracyjnej witryny Django

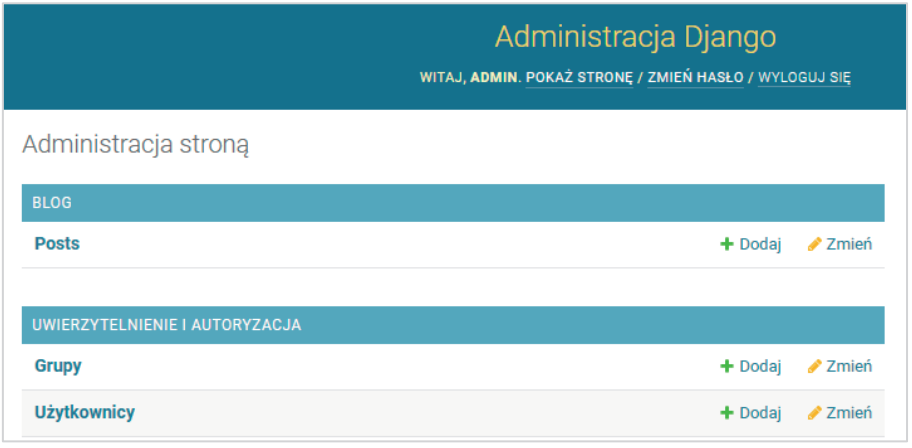
Modele `Group` i `User` widoczne na poprzednim zrzucie ekranu są częścią frameworka uwierzytelniania Django zaimplementowanego w aplikacji `django.contrib.auth`. Gdy klikniesz *Użytkownicy*, zobaczysz utworzonego wcześniej użytkownika.

Dodawanie modeli do witryny administracyjnej

Dodajmy do witryny administracyjnej modele bloga. Wyedytuj plik *admin.py* w aplikacji bloga i doprowadź go do postaci pokazanej poniżej. Nowe wiersze zostały wyróżnione pogrubioną czcionką.

```
from django.contrib import admin
from .models import Post


admin.site.register(Post)
```

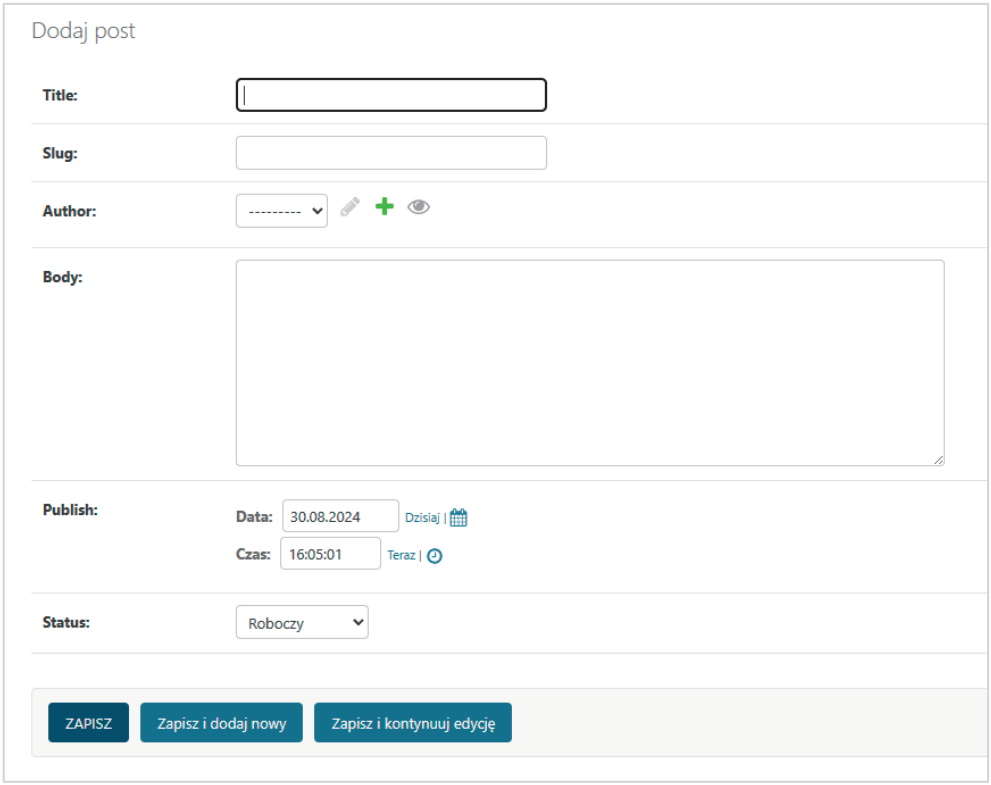
Teraz przeładuj witrynę administracyjną w przeglądarce. Powinieneś zobaczyć swój model `Post` w witrynie administracyjnej, tak jak pokazałem na rysunku 1.9.



Rysunek 1.9. Model Post aplikacji bloga dołączony do głównej strony administracyjnej witryny Django

To było łatwe, prawda? Kiedy rejestrujesz model na stronie administracyjnej Django, uzyskujesz wygenerowany przez introspekcję Twoich modeli przyjazny dla użytkownika interfejs, który pozwala w prosty sposób wyświetlać, edytować, tworzyć i usuwać obiekty.

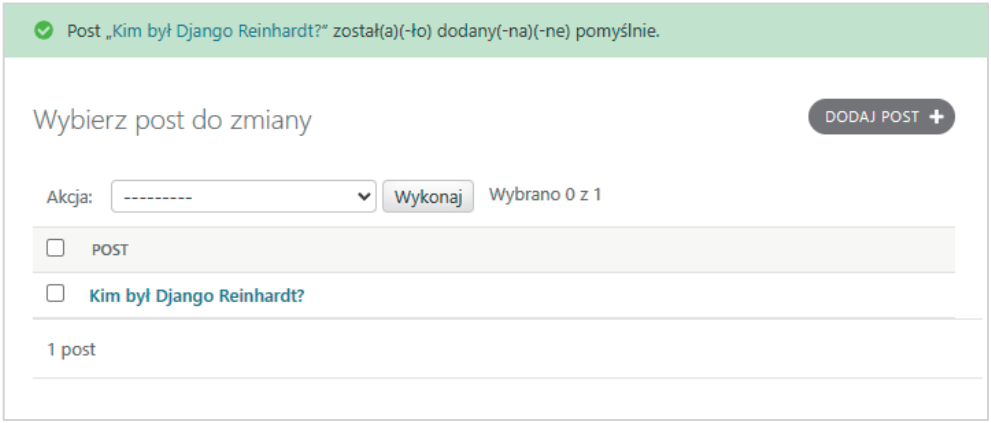
Aby dodać nowy post, kliknij łącze *Dodaj* obok *Posts*. Wyświetli się formularz, który Django wygenerował dynamicznie dla Twojego modelu, taki jak pokazany na rysunku 1.10.



Rysunek 1.10. Formularz edycji modelu Post na stronie administracyjnej Django

Django używa różnych widżetów dla poszczególnych typów pól. Nawet złożone pola, takie jak `DateTimeField`, są wyświetlane za pomocą wygodnego interfejsu, np. selektora daty JavaScript.

Wypełnij formularz i kliknij przycisk *ZAPISZ*. Powinieneś zostać przekierowany na stronę listy postów z komunikatem o udanej publikacji posta, który właśnie utworzyłeś, tak jak pokazałem na rysunku 1.11.



Rysunek 1.11. Na stronie administracyjnej Django wyświetla się widok modelu Post z komunikatem o pomyślnym dodaniu posta

Personalizacja sposobu wyświetlania modeli

Przyjrzymy się teraz, jak można spersonalizować witrynę administracyjną.

Wyedytuj plik *admin.py* swojej aplikacji bloga tak, by miał następującą zawartość. Nowe wiersze zostały wyróżnione pogrubioną czcionką.

```
from django.contrib import admin
from .models import Post

@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    list_display = ['title', 'slug', 'author', 'publish', 'status']
```

Poinformowaliśmy witrynę administracyjną Django, że model jest zarejestrowany przy użyciu niestandardowej klasy dziedziczącej po `ModelAdmin`. W tej klasie możemy podać informacje o tym, jak wyświetlić model w witrynie administracyjnej i jak się z nim komunikować.

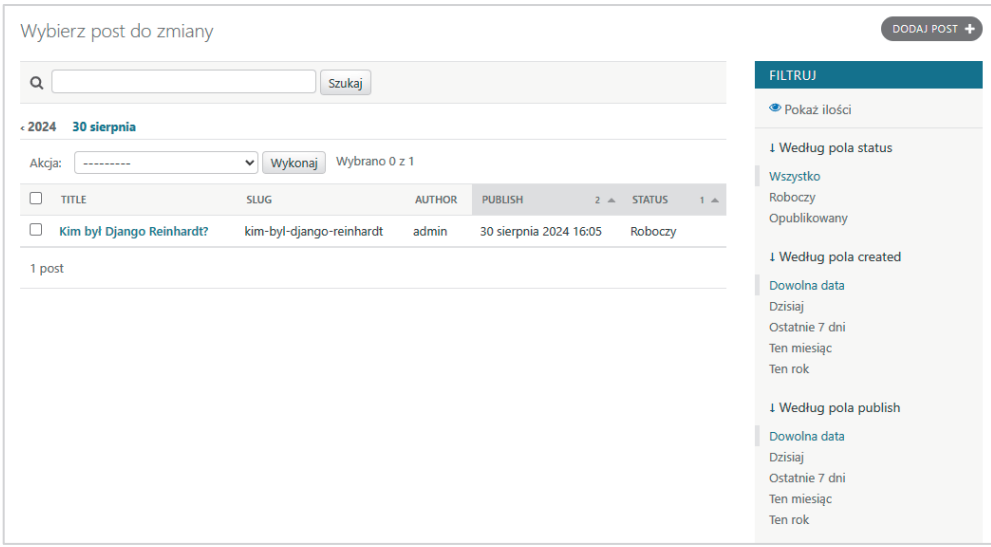
Za pomocą atrybutu `list_display` można ustawić pola modelu, które mają być wyświetlone na stronie listy obiektów administracyjnych. Dekorator `@admin.register()` wykonuje to samo działanie co funkcja `admin.site.register()`, którą zastąpiliśmy — rejestruje udekorowaną klasę `ModelAdmin`. Dostosujmy model administracyjny za pomocą kilku opcji.

Wyedytuj plik *admin.py* swojej aplikacji bloga tak, by miał następującą zawartość. Nowe wiersze zostały wyróżnione pogrubioną czcionką.

```
from django.contrib import admin
from .models import Post


@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    list_display = ['title', 'slug', 'author', 'publish', 'status']
```

```
list_filter = ['status', 'created', 'publish', 'author']
search_fields = ['title', 'body']
prepopulated_fields = {'slug': ('title',)}
raw_id_fields = ['author']
date_hierarchy = 'publish'
ordering = ['status', 'publish']
```

Wróć do przeglądarki i ponownie załaduj stronę listy postów. Teraz wygląda ona jak na rysunku 1.12.

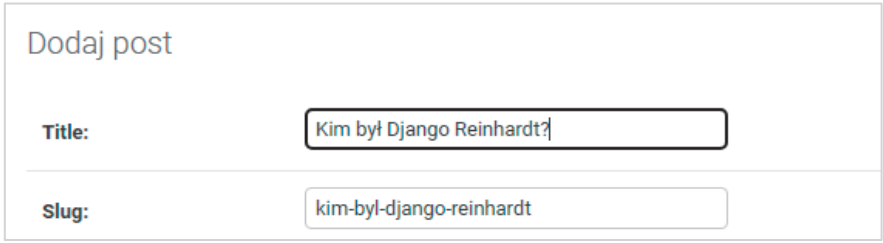


Rysunek 1.12. Formularz edycji modelu Post na stronie administracyjnej Django

Jak łatwo zauważyć, pola wyświetlane na stronie listy postów są tymi, które podaliśmy za pomocą atrybutu `list_display`. Strona listy zawiera teraz prawy pasek boczny, który pozwala filtrować wyniki według pól zawartych w atrybucie `list_filter`. Filtry dla pól klucza obcego, takich jak `author`, są wyświetlane na pasku bocznym tylko wtedy, gdy w bazie danych istnieje więcej niż jeden obiekt.

Na stronie pojawił się pasek Szukaj. Jest to efekt zdefiniowania za pomocą atrybutu `search_fields` listy pól do przeszukiwania. Tuż pod paskiem *Szukaj* znalazły się linki nawigacyjne umożliwiające poruszanie się po hierarchii dat: zdefiniowaliśmy ją za pomocą atrybutu `date_hierarchy`. Jak można zauważyć, posty są domyślnie posortowane według kolumn `STATUS` i `PUBLISH`. Określiliśmy domyślną kolejność za pomocą atrybutu `ordering`.

Teraz kliknij link *DODAJ POST*. Tutaj również można zauważyć kilka zmian. Podczas wpisywania tytułu nowego posta pole `slug` jest wypełniane automatycznie. Za pomocą atrybutu `prepopulated_fields` poinformowaliśmy framework Django, aby wraz z wprowadzaniem pola `title` wstępnie wypełnił pole `slug` (rysunek 1.13).



Rysunek 1.13. Pole slug wypełnia się teraz automatycznie podczas wpisywania tytułu

Ponadto teraz pole `author` jest wyświetlane z widżetem wyszukiwania, który — gdy mamy tysiące użytkowników — skaluje się znacznie lepiej niż kontrolka rozwijanego menu. Uzyskałem to dzięki zastosowaniu atrybutu `raw_id_fields`. Wygląd kontrolki do wprowadzania pola jest pokazany na rysunku 1.14.

Rysunek 1.14. Widżet do wyboru powiązanych obiektów dla pola author modelu Post

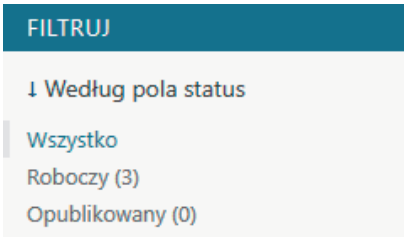
Dodawanie liczby aspektów do filtrów

W Django 5.0 wprowadzono do strony administracyjnej filtry aspektów, które umożliwiają wyświetlenie liczby aspektów. Wartości te wskazują liczbę obiektów odpowiadających każdemu filtrowi, co ułatwia identyfikację pasujących obiektów w widoku listy zmian administratora. W poniższym kodzie włączymy wyświetlanie filtrów aspektów dla modelu administratora `PostAdmin`.

Wyedytuj plik *admin.py* aplikacji bloga i dodaj wiersze, które na poniższym listingu zaznaczono pogrubieniem.

```
from django.contrib import admin
from .models import Post

@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    list_display = ['title', 'slug', 'author', 'publish', 'status']
    list_filter = ['status', 'created', 'publish', 'author']
    search_fields = ['title', 'body']
    prepopulated_fields = {'slug': ('title',)}
    raw_id_fields = ['author']
    date_hierarchy = 'publish'
    ordering = ['status', 'publish']
    show_facets = admin.ShowFacets.ALWAYS
```

Utwórz kilka postów za pomocą strony administracyjnej i otwórz przeglądarkę pod adresem `http://127.0.0.1:8000/admin/blog/post/`. Filtry powinny teraz zawierać całkowitą liczbę aspektów, jak pokazałem na rysunku 1.15.



Rysunek 1.15. Filtry pola statusu zawierające liczbę aspektów

Za pomocą kilku linijek kodu dostosowaliśmy sposób wyświetlania modelu na stronie administracyjnej. Istnieje wiele sposobów na spersonalizowanie i rozszerzenie witryny administracyjnej Django. Więcej informacji na ten temat znajdziesz dalej w tej książce.

Więcej informacji na temat strony administracyjnej Django można znaleźć pod adresem *https://docs.djangoproject.com/en/5.0/ref/contrib/admin/*.

Praca z obiektami QuerySet i menedżerami

Teraz, gdy mamy w pełni funkcjonalną witrynę administracyjną do zarządzania wpisami na blogu, nadszedł czas, aby nauczyć się programowo odczytywać i zapisywać zawartość bazy danych.

Django ma rozbudowany interfejs API do abstrakcji baz danych, który pozwala łatwo tworzyć, pobierać, aktualizować i usuwać obiekty. Mechanizm **mapowania obiektowo-relacyjnego frameworka** Django (ang. *object-relational mapper* — ORM) umożliwia generowanie zapytań SQL przy użyciu zorientowanego obiektowo paradygmatu języka Python. Jest to sposób interakcji z bazą danych w języku Python pozwalający korzystać z Pythona, zamiast pisać surowe zapytania SQL.

Mechanizm ORM odwzorowuje zdefiniowane modele na tabele bazy danych i zapewnia prosty interfejs Pythona do interakcji z bazą danych. ORM generuje zapytania SQL i odwzorowuje wyniki na obiekty modelu. Mechanizm ORM frameworka Django jest kompatybilny z bazami danych MySQL, PostgreSQL, SQLite i MariaDB.

Należy pamiętać, że bazę danych stosowaną w projekcie można zdefiniować za pomocą ustawienia `DATABASES` w pliku *settings.py* projektu. Django może pracować z wieloma bazami danych jednocześnie. W celu tworzenia niestandardowych schematów routingu można także zaprogramować routery baz danych.

Po utworzeniu modeli danych Django można skorzystać z bezpłatnego interfejsu API do interakcji z nimi. Opis interakcji z modelami danych można znaleźć w oficjalnej dokumentacji, pod adresem *https://docs.djangoproject.com/en/5.0/ref/models/*.

Mechanizm ORM frameworka Django bazuje na obiektach `QuerySet`. Obiekt `QuerySet` to zbiór zapytań do bazy danych służący do pobierania obiektów z bazy danych. Obiekty `QuerySet` pozwalają na stosowanie filtrów pozwalających zawęzić wyniki kwerendy na podstawie podanych parametrów. Obiekt `QuerySet` odpowiada instrukcji SQL `SELECT`, a filtry są ograniczającymi klauzulami SQL, takimi jak `WHERE` lub `LIMIT`.

W kolejnym punkcie pokażę, jak tworzyć i uruchamiać obiekty `QuerySet`.

Tworzenie obiektów

Otwórz terminal i uruchom następujące polecenie, aby otworzyć powłokę Pythona.

```
python manage.py shell
```

Następnie wpisz wiersze:

```
>>> from django.contrib.auth.models import User
>>> from blog.models import Post
>>> user = User.objects.get(username='admin')
>>> post = Post(title='Inny post',
...             slug='inny-post',
...             body='Treść posta.',
...             author=user)
>>> post.save()
```

Przeanalizujmy, co ten kod robi.

Najpierw pobieramy obiekt `user`, którego pole `username` to `admin`.

```
>>> user = User.objects.get(username='admin')
```

Wiemy już, że za pomocą metody `get()` możemy pobrać z bazy danych pojedynczy obiekt. Ta metoda wykonuje za kulisami instrukcję SQL `SELECT`. Należy pamiętać, że metoda oczekuje wyniku pasującego do zapytania. Jeśli baza danych nie zwróci żadnych wyników, metoda zgłosi wyjątek DoesNotExist, a jeśli baza danych zwróci więcej niż jeden wynik, zostanie zgłoszony wyjątek `MultipleObjectsReturned`. Oba wyjątki są atrybutami klasy modelu, na którym wykonywane jest zapytanie.

Następnie tworzymy egzemplarz klasy `Post` z niestandardowymi polami `title`, `slug` i `body` oraz ustawiamy jako autora posta użytkownika, którego wcześniej pobraliśmy.

```
>>> post = Post(title='Inny post', slug='inny-post', body='Treść posta.',
author=user)
```

Ten obiekt znajduje się w pamięci i nie jest utrwalony w bazie danych; stworzyliśmy obiekt Pythona, którego można używać w czasie wykonywania, ale który nie jest zapisywany w bazie danych.

Na koniec zapisujemy obiekt `Post` w bazie danych za pomocą metody `save()`.

```
>>> post.save()
```

W poprzedniej operacji „za kulisami" została wykonana instrukcja SQL `INSERT`.

Najpierw utworzyliśmy obiekt w pamięci, a następnie utrwaliliśmy go w bazie danych. Możesz jednak również utworzyć obiekt i zachować go w bazie danych w pojedynczej operacji przy użyciu metody `create()`.

```
>>> Post.objects.create(title='Jeszcze jeden post',
                        slug='jeszcze-jeden-post',
                        body='Treść posta.',
                        author=user)
```

W niektórych sytuacjach trzeba pobrać obiekt z bazy danych lub utworzyć go, jeśli go nie ma. Służy do tego metoda `get_or_create()`, która pobiera z bazy danych obiekt lub tworzy go, jeśli nie został znaleziony. Metoda ta zwraca krotkę z pobranym obiektem i wartością logiczną wskazującą, czy nowy obiekt został utworzony. Poniższy kod próbuje pobrać obiekt `User` z wartością pola `username` *user2*, a jeśli taki obiekt nie istnieje, tworzy go:

```
>>> user, created = User.objects.get_or_create(username='user2')
```

Aktualizowanie obiektów

Spróbujmy teraz zmodyfikować tytuł posta na jakiś inny i zapisać obiekt ponownie.

```
>>> post.title = 'Nowy tytuł'
>>> post.save()
```

Tym razem metoda `save()` wykonuje instrukcję SQL `UPDATE`.

Uwaga

Zmiany wprowadzone w obiekcie nie są zachowywane w bazie danych do momentu wywołania metody `save()`.

Pobieranie obiektów

Wiemy już, że za pomocą metody `get()` możemy pobrać z bazy danych pojedynczy obiekt. Dostęp do tej metody uzyskaliśmy za pomocą wywołania `Post.objects.get()`. Każdy model Django ma co najmniej jednego menedżera, a domyślny menedżer nazywa się `objects`. Za pomocą menedżera modelu uzyskujemy obiekt `QuerySet`.

Aby pobrać wszystkie obiekty z tabeli, wystarczy użyć metody `all()` domyślnego menedżera `objects`. Przykładowo:

```
>>> all_posts = Post.objects.all()
```

W ten sposób tworzymy obiekt `QuerySet`, który zwraca wszystkie obiekty w bazie danych. Zwróć uwagę, że ten obiekt `QuerySet` nie został jeszcze uruchomiony. Obiekty `QuerySet` frameworka Django są wartościowane leniwie — tylko wtedy, gdy jest to konieczne. To zachowanie sprawia, że obiekty `QuerySet` są bardzo wydajne. Jeśli nie przypiszemy obiektu `QuerySet` do zmiennej, a zamiast tego wywołamy go bezpośrednio w powłoce Pythona, instrukcja SQL z obiektu `QuerySet` zostanie wykonana, ponieważ zmuszamy obiekt do wyświetlenia wyników.

```
>>> Post.objects.all()
<QuerySet [<Post: Jeszcze jeden post>, <Post: Nowy tytuł>, <Post: Kim był Django
↪Reinhardt?>]>
```

Filtrowanie obiektów

Do filtrowania obiektu `QuerySet` można użyć metody `filter()` menedżera. Ta metoda za pomocą mechanizmu wyszukiwania pól pozwala określić zawartość klauzuli SQL `WHERE`.

Na przykład do filtrowania obiektów `Post` na podstawie ich tytułu można użyć poniższego wywołania:

```
>>> Post.objects.filter(title='Kim był Django Reinhardt?')
```

Ten obiekt `QuerySet` zwróci wszystkie posty, których tytuł to *Kim był Django Reinhardt?*. Przyjrzyjmy się instrukcji SQL wygenerowanej za pomocą tego obiektu `QuerySet`. Uruchom w powłoce kod:

```
>>> posts = Post.objects.filter(title='Kim był Django Reinhardt?')
>>> print(posts.query)
```

Wyświetlenie atrybutu `query` obiektu `QuerySet` pozwala uzyskać instrukcję SQL, którą ten obiekt wygenerował:

```
SELECT "blog_post"."id", "blog_post"."title", "blog_post"."slug", "blog_post".
↪"author_id", "blog_post"."body", "blog_post"."publish", "blog_post"."created",
"blog_post"."updated", "blog_post"."status" FROM "blog_post" WHERE "blog_post".
↪"title" = Kim był Django Reinhardt? ORDER BY "blog_post"."publish" DESC
```

Wygenerowana klauzula `WHERE` wykonuje dokładne dopasowanie w kolumnie `title`. Ponieważ w obiekcie `QuerySet` nie podaliśmy żadnej konkretnej kolejności, klauzula `ORDER BY` określa domyślną kolejność zdefiniowaną w atrybucie `ordering` opcji `Meta` modelu `Post`. O definiowaniu kolejności rekordów dowiesz się za chwilę. Należy zapamiętać, że atrybut `query` nie jest częścią publicznego interfejsu API obiektu `QuerySet`.

Korzystanie z różnych metod dopasowywania pól

Poprzedni przykład zastosowania obiektu `QuerySet` obejmuje filtr z dokładnym dopasowaniem. Interfejs obiektu `QuerySet` udostępnia wiele typów dopasowywania. Do zdefiniowania typu dopasowania używa się znaku podwójnego podkreślenia w formacie *`pole__dopasowanie`*. Na przykład poniższa instrukcja filtrowania wykorzystuje dokładne dopasowanie:

```
>>> Post.objects.filter(id__exact=1)
```

Jeśli nie podano konkretnego typu dopasowania, stosowany jest domyślny typ dopasowania: `exact`. Poniższe wyszukiwanie jest równoważne poprzedniemu:

```
>>> Post.objects.filter(id=1)
```

Przyjrzyjmy się innym popularnym typom dopasowywania. Dopasowanie `iexact` pozwala wyszukiwać rekordy bez uwzględniania wielkości liter:

```
>>> Post.objects.filter(title__iexact='kim był django reinhardt?')
```

Można również filtrować obiekty z wykorzystaniem testu zawartości. Dopasowanie `contains` odpowiada wyszukiwaniu SQL za pomocą operatora `LIKE`:

```
>>> Post.objects.filter(title__contains='Django')
```

Równoważna klauzula SQL to `WHERE title LIKE '%Django%'`. Dostępna jest również wersja niewrażliwa na wielkość liter o nazwie `icontains`:

```
>>> Post.objects.filter(title__icontains='django')
```

Za pomocą dopasowania in można sprawdzić obiekt iterowalny (często listę, krotkę lub inny obiekt `QuerySet`). Oto przykład instrukcji, która pobiera posty o wartości pola `id` 1 lub 3:

```
>>> Post.objects.filter(id__in=[1, 3])
```

Oto przykład zastosowania dopasowania `gt` (większy niż):

```
>>> Post.objects.filter(id__gt=3)
```

Równoważną klauzulą SQL jest `WHERE ID > 3`.

Oto przykład dopasowania `gte` (większy lub równy):

```
>>> Post.objects.filter(id__gte=3)
```

A tak wygląda wykorzystanie dopasowania `lt` (mniejszy niż):

```
>>> Post.objects.filter(id__lt=3)
```

To jest przykład dopasowania `lte` (mniejszy lub równy):

```
>>> Post.objects.filter(id__lte=3)
```

Dopasowania „zaczyna się" w wersjach z uwzględnieniem wielkości liter lub bez uwzględniania wielkości liter można wykonać za pomocą typów dopasowania, odpowiednio, `startswith` i `istartswith`:

```
>>> Post.objects.filter(title__istartswith='kim')
```

Dopasowania „kończy się na" z uwzględnieniem wielkości liter lub bez uwzględniania wielkości liter są reprezentowane przez typy dopasowania, odpowiednio, `endswith` i `iendswith`:

```
>>> Post.objects.filter(title__iendswith='reinhardt?')
```

Istnieją również różne typy dopasowania do wyszukiwania dat. Dokładne wyszukiwanie daty można wykonać w taki sposób:

```
>>> from datetime import date
>>> Post.objects.filter(publish__date=date(2024, 1, 31))
```

Oto jak należy filtrować pola `DateField` lub `DateTimeField` według roku:

```
>>> Post.objects.filter(publish__year=2024)
```

Można też filtrować według miesięcy:

```
>>> Post.objects.filter(publish__month=1)
```

A także według numeru dnia:

```
>>> Post.objects.filter(publish__day=1)
```

Można połączyć w łańcuch dodatkowe dopasowania dla daty, roku, miesiąca i dnia. Oto przykład wyszukiwania wartości większej od podanej daty:

```
>>> Post.objects.filter(publish__date__gt=date(2024, 1, 1))
```

Aby wyszukiwać powiązane pola obiektów, należy użyć notacji z dwoma podkreśleniami. Na przykład, aby pobrać posty napisane przez użytkownika o nazwie *admin*, użyj poniższej instrukcji:

```
>>> Post.objects.filter(author__username='admin')
```

Można też utworzyć łańcuch dodatkowych dopasowań dla powiązanych pól.

Na przykład, aby pobrać posty napisane przez użytkownika o nazwie zaczynającej się od *ad*, użyj poniższej instrukcji:

```
>>> Post.objects.filter(author__username__starstwith='ad')
```

Możemy także filtrować według wielu pól. Na przykład poniższe zapytanie pobiera wszystkie posty opublikowane w 2024 roku przez autora o nazwie *admin*.

```
>>> Post.objects.filter(publish__year=2024, author__username='admin')
```

Łączenie filtrów w łańcuch

Wynikiem zastosowania filtra do obiektu `QuerySet` jest inny obiekt `QuerySet`. Umożliwia to łączenie obiektów `QuerySet` w łańcuchy. Oto jak zbudować obiekt `QuerySet` równoważny z poprzednim z wykorzystaniem połączenia wielu filtrów w łańcuch:

```
>>> Post.objects.filter(publish__year=2024) \
>>>             .filter(author__username='admin')
```

Wykluczanie obiektów

Za pomocą metody `exclude()` menedżera można wykluczyć niektóre rezultaty z wynikowego obiektu `QuerySet`. Możemy np. pobrać wszystkie posty opublikowane w 2024 roku, których tytuły nie zaczynają się od „Dlaczego”.

```
>>> Post.objects.filter(publish__year=2024) \
>>> .exclude(title__startswith='Dlaczego')
```

Sortowanie obiektów

Domyślną kolejność można zdefiniować za pomocą opcji `ordering` w klasie `Meta` modelu. Domyślną kolejność można nadpisać za pomocą metody `order_by()` menedżera. Przykładowo za pomocą poniższego polecenia można pobrać wszystkie obiekty uporządkowane według tytułu.

```
>>> Post.objects.order_by('title')
```

Domyślny jest porządek rosnący. Aby uporządkować obiekty malejąco, można zastosować prefiks w postaci znaku minusa.

```
>>> Post.objects.order_by('-title')
```

Istnieje możliwość porządkowania według wielu pól. Poniższa instrukcja porządkuje obiekty według autora, a następnie tytułu:

```
>>> Post.objects.order_by('author', 'title')
```

Aby zastosować porządek losowy, można wykorzystać ciąg `'?'`, jak pokazano poniżej:

```
>>> Post.objects.order_by('?')
```

Ograniczanie wynikowych obiektów QuerySet

Aby wprowadzić ograniczenie dla obiektu `QuerySet` do określonej liczby wyników, można skorzystać z podzbioru składni Pythona do ustalania wycinków tablic. Przykładowo poniższa instrukcja ogranicza wyniki do pięciu obiektów:

```
>>> Post.objects.all()[:5]
```

Powyższa instrukcja odpowiada klauzuli SQL `LIMIT 5`. Warto zapamiętać, że indeksowanie ujemne nie jest obsługiwane.

```
>>> Post.objects.all()[3:6]
```

Powyższa instrukcja odpowiada klauzuli SQL `OFFSET 3 LIMIT 6`, która zwraca obiekty od czwartego do szóstego.

Aby pobrać pojedynczy obiekt, można użyć indeksu zamiast wycinka. Na przykład, aby pobrać pierwszy obiekt postów w losowej kolejności, można skorzystać z instrukcji:

```
>>> Post.objects.order_by('?')[0]
```

Zliczanie obiektów

Metoda `count()` zlicza całkowitą liczbę obiektów pasujących do wynikowego obiektu `QuerySet`. Metoda ta odpowiada instrukcji SQL `SELECT COUNT(*)`. Oto przykład instrukcji zwracającej całkowitą liczbę postów o identyfikatorze niższym od 3:

```
>>> Post.objects.filter(id_lt=3).count()
2
```

Sprawdzanie, czy obiekt istnieje

Metoda `exists()` pozwala sprawdzić, czy obiekt `QuerySet` zawiera jakiekolwiek wyniki. Ta metoda zwraca `True`, jeśli obiekt `QuerySet` zawiera jakieś elementy, i `False` w przeciwnym razie. Na przykład za pomocą poniższej instrukcji można sprawdzić, czy istnieją posty o tytule zaczynającym się od „Dlaczego”:

```
>>> Post.objects.filter(title__startswith='Dlaczego').exists() False
```

Usuwanie obiektów

Aby usunąć obiekt, można skorzystać z metody `delete()` egzemplarza obiektu.

```
>>> post = Post.objects.get(id=1)
>>> post.delete()
```

Należy zauważyć, że usunięcie obiektów spowoduje również usunięcie relacji zależnych dla obiektów `ForeignKey` zdefiniowanych przy użyciu klauzuli `on_delete` ustawionej na `CASCADE`.

Złożone dopasowania z wykorzystaniem obiektów Q

Dopasowania pól z wykorzystaniem funkcji `filter()` są łączone za pomocą operatora SQL `AND`. Na przykład instrukcja `filter(field1='foo ', field2='bar')` pobierze obiekty, dla których `field1` ma wartość *foo*, a `field2` ma wartość *bar*. Aby zbudować bardziej złożone zapytania, np. zapytania z instrukcjami `OR`, można użyć obiektów `Q`.

Obiekt `Q` pozwala na enkapsulację kolekcji dopasowań pól. Obiekty `Q` można łączyć za pomocą operatorów `&` (`and`), `|` (`or`) i `^` (`xor`).

Na przykład poniższy kod pobiera posty o tytule zaczynającym się od ciągu znaków *kim* **lub** *dlaczego* (wielkość liter nie ma znaczenia):

```
>>> from django.db.models import Q
>>> starts_who = Q(title__istartswith='kim')
>>> starts_why = Q(title__istartswith='dlaczego')
>>> Post.objects.filter(starts_who | starts_why)
```

W tym przypadku do utworzenia instrukcji `OR` użyliśmy operatora `|`.

Więcej informacji o obiektach `Q` można znaleźć na stronie *https://docs.djangoproject.com/en/5.0/topics/db/queries/#complex-lookups-with-q-objects*.

Kiedy następuje określenie zawartości kolekcji QuerySet?

Utworzenie obiektu `QuerySet` nie powoduje żadnego działania w bazie danych. Kolekcje `QuerySet` zwykle zwracają inne, nieokreślone kolekcje `QuerySet`. W kolekcji `QuerySet` można łączyć dowolną liczbę filtrów, ale operacje w bazie danych nie będą wykonywane aż do chwili wywołania kolekcji. Podczas określania zawartości obiektu `QuerySet` jest on tłumaczony na zapytanie SQL do bazy danych.

Określenie zawartości kolekcji `QuerySet` odbywa się jedynie w wymienionych poniżej sytuacjach.

- W trakcie pierwszej iteracji przez kolekcję.
- Podczas obliczania wycinka kolekcji, np. za pomocą polecenia `Post.objects.all()[:3]`.

- Podczas serializacji za pomocą modułu `pickle` lub podczas buforowania.
- Podczas wywoływania funkcji `repr()` lub `len()` względem kolekcji.
- Podczas jawnego wywoływania funkcji `list()` względem kolekcji.
- Podczas wykonywania w kolekcji polecenia testującego, takiego jak `bool()`, `or`, `and` lub `if`.

Więcej informacji o obiektach QuerySet

Obiekty `QuerySet` będą używane we wszystkich przykładach projektów przedstawionych w tej książce. W podrozdziale „Pobieranie postów według podobieństwa" w rozdziale 3., „Rozbudowa aplikacji bloga", dowiesz się, jak generować agregaty złożone z obiektów `QuerySet`.

W punkcie „Optymalizacja kolekcji QuerySet dotyczącej powiązanych obiektów", w rozdziale 7., „Śledzenie działań użytkownika", dowiesz się jak zoptymalizować obiekty `QuerySet`.

Dokumentacja API `QuerySet` jest dostępna pod adresem *https://docs.djangoproject.com/en/5.0/ref/models/querysets/*.

Więcej informacji o tworzeniu zapytań za pomocą mechanizmu ORM Django można znaleźć na stronie *https://docs.djangoproject.com/en/5.0/topics/db/queries/*.

Utworzenie menedżerów modelu

Domyślnym menedżerem dla każdego modelu jest menedżer `objects`. Ten menedżer pobiera wszystkie obiekty w bazie danych. Istnieje możliwość zdefiniowania własnych menedżerów dla modelu.

Stwórzmy niestandardowego menedżera, który pobiera wszystkie posty o statusie `PUBLISHED`.

Mamy dwa sposoby dodania menedżerów do modeli. Pierwszy polega na dodaniu dodatkowych metod menedżera, natomiast drugi na modyfikacji początkowego menedżera kolekcji `QuerySet`. Po zastosowaniu pierwszego podejścia mamy wywołanie w stylu `Post.objects.my_manager()`, w przypadku drugiego mamy `Post.my_manager.all()`.

Wybierzemy drugą metodę implementacji menedżera, która pozwoli nam pobierać posty za pomocą wywołania `Post.published.all()`.

Aby dodać własnego menedżera, wyedytuj plik *models.py* aplikacji bloga w sposób zaprezentowany na poniższym listingu. Nowe wiersze zostały wyróżnione pogrubioną czcionką.

```
class PublishedManager(models.Manager):
    def get_queryset(self):
        return (
            super().get_queryset().filter(status=Post.Status.PUBLISHED)
        )
```

```
class Post(models.Model):
    # pola modelu
    # …
    objects = models.Manager() # Menedżer domyślny
    published = PublishedManager() # Menedżer niestandardowy

    class Meta:
        ordering = ['-publish']
        indexes = [
            models.Index(fields=['-publish']),
        ]

    def __str__(self):
        return self.title
```

Pierwszy menedżer zadeklarowany w modelu staje się menedżerem domyślnym. W celu wskazania innego menedżera domyślnego można użyć atrybutu `Meta default_manager_name`. Jeśli w modelu nie został zdefiniowany menedżer, Django automatycznie tworzy `objects` jako domyślnego menedżera. Jeśli zadeklarujesz dowolnego menedżera dla Twojego modelu, ale chcesz zachować menedżera `objects`, musisz dodać go do modelu jawnie. W powyższym kodzie dodaliśmy do modelu `Post` domyślnego menedżera `objects` oraz niestandardowego menedżera `published`.

Wykonana tu zostanie metoda `get_queryset()`, której wartością zwrotną jest kolekcja `QuerySet`. Zastąpiliśmy tę metodę w celu stworzenia niestandardowego obiektu `QuerySet`, który filtruje posty według ich statusu i zwraca kolejny obiekt `QuerySet`, zawierający tylko posty o statusie `PUBLISHED`.

Właśnie zdefiniowaliśmy niestandardowego menedżera dla modelu `Post`. Przetestujmy go.

Uruchom ponownie serwer programistyczny za pomocą polecenia:

```
python manage.py shell
```

Teraz możesz pobrać wszystkie opublikowane posty, których tytuł zaczyna się od „Kim”, za pomocą kodu:

```
>>> from blog.models import Post
>>> Post.published.filter(title__startswith='Kim')
```

Aby uzyskać wyniki dla tego obiektu `QuerySet`, ustaw pole `status` na `PUBLISHED` w obiektach `Post`, których pole `title` zaczyna się od *Kim*.

Przygotowanie widoków listy i szczegółów

Na tym etapie wiesz już cokolwiek o sposobie użycia mapowania obiektowo-relacyjnego. Możemy więc przystąpić do przygotowania widoków aplikacji bloga. Widok frameworka Django to po prostu funkcja Pythona, która otrzymuje żądanie sieciowe i udziela na nie odpowiedzi. Wewnątrz widoku znajduje się cała logika odpowiedzialna za zwrócenie żądanej odpowiedzi.

Zaczniemy od przygotowania widoków aplikacji, następnie zajmiemy się zdefiniowaniem wzorca adresu URL dla poszczególnych widoków, a na końcu utworzymy szablony HTML przeznaczone do wyświetlenia danych wygenerowanych przez te widoki. Każdy widok przekazuje zmienne do szablonu i otrzymuje odpowiedź HTTP wraz z wygenerowanymi w szablonie danymi wyjściowymi.

Utworzenie widoków listy i szczegółów

Rozpoczynamy od opracowania widoku przeznaczonego do wyświetlenia listy postów.

Przeprowadź edycję pliku *views.py* aplikacji `blog` w taki sposób, aby otrzymać postać pokazaną na poniższym listingu. Nowe wiersze zostały wyróżnione pogrubioną czcionką.

```
from django.shortcuts import render
from .models import Post

def post_list(request):
    posts = Post.published.all()
    return render(
        request,
        'blog/post/list.html',
        {'posts': posts}
    )
```

To jest pierwszy widok Django, jaki utworzyliśmy w budowanej aplikacji. Widok `post_list` pobiera obiekt `request` jako jedyny parametr. Pamiętaj, że parametr ten jest wymagany przez wszystkie widoki.

W omawianym widoku za pomocą utworzonego wcześniej menedżera `published` pobieramy wszystkie posty, których status ma wartość `PUBLISHED`.

Na koniec używamy oferowanego przez Django skrótu w postaci wywołania `render()` w celu wygenerowania listy postów na podstawie wskazanego szablonu. Omawiana funkcja pobiera parametry w postaci obiektu `request`, ścieżki dostępu do szablonu oraz zmiennych kontekstu przeznaczonych do wyrenderowania w podanym szablonie. Funkcja zwraca obiekt `HttpResponse` wraz z wyrenderowanym tekstem (standardowo jest to kod HTML).

Wspomniany skrót w postaci wywołania `render()` uwzględnia kontekst żądania, więc w tym szablonie będą dostępne wszystkie zmienne ustawione przez procesory kontekstu szablonu. Procesory kontekstu szablonu to po prostu obiekty wywoływalne ustawiające zmienne w kontekście. Więcej informacji na temat korzystania z procesorów kontekstu znajdziesz w rozdziale 4., „Utworzenie witryny społecznościowej”.

Przechodzimy teraz do utworzenia drugiego widoku przeznaczonego do wyświetlenia pojedynczego posta. W pliku *views.py* umieść poniższą funkcję.

```
from django.http import Http404

def post_detail(request, id):
    try:
        post = Post.published.get(id=id)
    except Post.DoesNotExist:
        raise Http404("Nie znaleziono posta.")
```

```
    return render(
        request,
        'blog/post/detail.html',
        {'post': post}
    )
```

To jest widok `post_detail` wyświetlający szczegóły posta. Widok przyjmuje argument `id` posta. W widoku próbujemy pobrać obiekt `Post` o podanym `id`. W tym celu wywołujemy metodę `get()` na menedżerze `published`. Jeśli zostanie zgłoszony wyjątek modelu `DoesNot ↪Exist`, zgłaszamy wyjątek `Http404` w celu zwrócenia błędu HTTP 404, ponieważ nie znaleziono żadnego wyniku.

Na końcu używamy skrótu `render()` do wyrenderowania posta za pomocą wskazanego szablonu.

Korzystanie ze skrótu get_object_or_404

Django zapewnia skrót pozwalający na wywołanie metody `get()` na określonym menedżerze modelu, a gdy nie zostanie znaleziony żaden obiekt — zgłoszenie wyjątku `Http404` zamiast wyjątku `DoesNotExist`.

Wyedytuj plik *views.py*, aby zaimportować skrót `get_object_or_404`, i zmodyfikuj widok `post_detail` w sposób pokazany na poniższym listingu. Nowy kod został wyróżniony pogrubioną czcionką.

```
from django.shortcuts import get_object_or_404, render

# …

def post_detail(request, id):
    post = get_object_or_404(
        Post,
        id=id,
        status=Post.Status.PUBLISHED
    )
    return render(
        request,
        'blog/post/detail.html',
        {'post': post}
    )
```

W widoku szczegółowym do pobrania żądanego obiektu `Post` wykorzystujemy skrót w postaci wywołania `get_object_or_404()`. Wymieniona funkcja pobiera obiekt dopasowany do podanych parametrów lub zgłasza wyjątek `HTTP404` (nie znaleziono), gdy nie uda się dopasować żadnego obiektu.

Dodanie wzorców adresów URL do widoków

Wzorce adresów URL pozwalają mapować adresy URL na widoki. Wzorzec adresu URL składa się z wyrażenia regularnego Pythona, widoku i nazwy, co pozwala na otrzymanie nazwy unikatowej w całym projekcie. Django przegląda swoje skonfigurowane wzorce

adresów URL i zatrzymuje się na pierwszym, który pasuje do żądanego adresu URL. Następnie framework importuje widok dopasowanego wzorca adresu URL i wykonuje go, przekazując egzemplarz klasy `HttpRequest` oraz argumenty pozycyjne lub w postaci słów kluczowych.

Utwórz plik *urls.py* w katalogu aplikacji `blog`, a następnie umieść w nim poniższy fragment kodu.

```
from django.urls import path
from . import views

app_name = 'blog'

urlpatterns = [
    # widoki posta
    path('', views.post_list, name='post_list'),
    path('<int:id>/', views.post_detail, name='post_detail'),
]
```

W powyższym fragmencie kodu zdefiniowaliśmy przestrzeń nazw aplikacji za pomocą zmiennej `app_name`. To pozwala zorganizować adresy URL według aplikacji i używać nazwy aplikacji podczas odwoływania się do nich. Za pomocą funkcji `path()` zdefiniowaliśmy dwa różne wzorce. Pierwszy wzorzec adresu URL nie pobiera żadnych argumentów i jest mapowany na widok `post_list`. Drugi wzorzec jest odwzorowany na widok `post_detail` i przyjmuje tylko jeden argument, `id`, który odpowiada liczbie całkowitej ustawionej przez konwerter ścieżki `int`.

Do przechwytywania wartości z adresu URL używamy nawiasów trójkątnych. Każda wartość określona we wzorcu adresu URL jako `<parametr>` jest przechwytywana jako ciąg znaków. Używamy konwerterów ścieżek, np. `<int: year>`, aby dokładnie dopasować i zwrócić liczbę całkowitą. Na przykład `<slug:post>` dokładnie pasuje do slugu (ciąg składający się z liter lub cyfr ASCII oraz znaków łącznika i podkreślenia). Z wszystkimi konwerterami ścieżek dostarczonymi z frameworkiem Django można zapoznać się na stronie *https://docs.djangoproject.com/en/5.0/topics/http/urls/#path-converters*.

Jeśli użycie funkcji `path()` i konwerterów nie jest wystarczające, możesz skorzystać z metody `re_path()`, która pozwala na zdefiniowanie złożonych wzorców adresów URL za pomocą wyrażeń regularnych Pythona.

Wskazówka

Utworzenie pliku *urls.py* dla każdej aplikacji jest najlepszym sposobem zapewnienia możliwości wielokrotnego użycia tej aplikacji w innych projektach.

Teraz trzeba dołączyć wzorce adresów URL aplikacji bloga do głównych wzorców adresów URL projektu.

W tym celu przeprowadź edycję pliku *urls.py*, znajdującego się w katalogu *mysite* projektu, i doprowadź go do postaci pokazanej na poniższym listingu. Nowy kod został wyróżniony pogrubioną czcionką.

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('blog/', include('blog.urls', namespace='blog')),
]
```

Nowy wzorzec adresu URL zdefiniowany za pomocą `include` odnosi się do wzorców adresów URL zdefiniowanych w aplikacji blog, dlatego zostały dołączone w ścieżce *blog/*. Te wzorce dołączamy w ramach przestrzeni nazw `blog`. Przestrzenie nazw muszą być unikatowe w całym projekcie. Za pomocą przestrzeni nazw możemy później łatwo odwoływać się do adresów URL bloga. Adres URL może przyjąć np. format `blog: post_list` lub `blog:post_detail`. Więcej informacji na temat przestrzeni nazw URL można znaleźć pod adresem *https://docs.djangoproject.com/en/5.0/topics/http/urls/#url-namespaces*.

Utworzenie szablonów dla widoków

Utworzyliśmy widoki i wzorce adresów URL dla aplikacji `blog`. Wzorce adresów URL mapują adresy URL na widoki, a widoki decydują o tym, jakie dane będą zwrócone użytkownikowi. Szablony określają sposób wyświetlania danych; zwykle są one napisane w HTML w połączeniu z językiem opisu szablonów frameworka Django. Więcej informacji na temat języka szablonów frameworka Django można znaleźć pod adresem *https://docs.djangoproject.com/en/5.0/ref/templates/language/*.

Kolejnym krokiem jest dodanie szablonów, z pomocą których posty będą wyświetlane w sposób przyjazny dla użytkownika.

W katalogu aplikacji `blog` przygotuj taką strukturę katalogów i plików:

```
templates/
    blog/
        base.html
        post/
            list.html
            detail.html
```

Jest to struktura plików szablonów wykorzystywanych w aplikacji. Plik *base.html* będzie zawierał główną strukturę HTML strony internetowej z zachowaniem podziału zawartości na treść główną i pasek boczny. Z kolei pliki *list.html* i *detail.html* dziedziczą po pliku *base.html* i są odpowiedzialne za wygenerowanie widoków — odpowiednio — listy postów i szczegółów posta.

Django jest wyposażony w elastyczny język szablonów, pozwalający na określenie sposobu, w jaki mają być wyświetlane dane. Jego działanie opiera się na znacznikach szablonu, zmiennych szablonu oraz filtrach szablonów.

- Znaczniki szablonów decydują o sposobie renderowania szablonu i mają postać `{% znacznik %}`.
- Podczas renderowania zmienne szablonu są zastępowane wartościami. Wyglądają tak: `{{ zmienna }}`.

- Filtry szablonów pozwalają modyfikować zmienne do wyświetlania, np. `{{zmienna|filtr }}`.

Wszystkie wbudowane znaczniki szablonów i filtry możesz przejrzeć na stronie *https://docs.djangoproject.com/en/5.0/ref/templates/builtins/*.

Tworzenie szablonu bazowego

Przeprowadź edycję pliku *base.html* i umieść w nim poniższy fragment kodu:

```
{% load static %}
<!DOCTYPE html>
<html>
<head>
  <title>{% block title %}{% endblock %}</title>
  <link href="{% static "css/blog.css" %}" rel="stylesheet">
</head>
<body>
  <div id="content">
    {% block content %}
    {% endblock %}
  </div>
  <div id="sidebar">
    <h2>Mój blog</h2>
    <p>To jest mój blog.</p>
  </div>
</body>
</html>
```

Znacznik `{% load static %}` nakazuje Django wczytanie znaczników szablonu `static` dostarczanego przez aplikację `django.contrib.staticfiles` zawartą w ustawieniu `INSTALLED_APPS`. Po jego wczytaniu można używać znacznika `{% static %}` w całym szablonie. Za pomocą tego filtru szablonu można dołączać pliki statyczne, np. plik *blog.css*, który znajdziesz w materiałach przygotowanych dla książki, a dokładnie: w katalogu *static/* aplikacji `blog`. Aby zastosować arkusze stylów CSS, skopiuj wymieniony katalog do swojego projektu. Zawartość katalogów jest dostępna pod adresem *https://github.com/PacktPublishing/Django-5-by-example/tree/master/Chapter01/mysite/blog/static*.

W powyższym kodzie łatwo zauważysz dwa znaczniki `{% block %}`. Informują one framework, że chcemy zdefiniować blok we wskazanym miejscu. Szablony dziedziczące po tym szablonie będą mogły umieszczać treść we wspomnianych blokach. W omawianym przykładzie zdefiniowaliśmy dwa bloki o nazwach `title` i `content`.

Utworzenie szablonu listy postów

Przeprowadź edycję pliku *post/list.html* w taki sposób, aby jego zawartość przedstawiała się następująco.

```
{% extends "blog/base.html" %}

{% block title %}Mój blog{% endblock %}
{% block content %}
```

```
    <h1>Mój blog</h1>
    {% for post in posts %}
      <h2>
        <a href="{% url 'blog:post_detail' post.id %}">
          {{ post.title }}
        </a>
      </h2>
      <p class="date">
        Opublikowany {{ post.publish }} przez {{ post.author }}
      </p>
      {{ post.body|truncatewords:30|linebreaks }}
    {% endfor %}
  {% endblock %}
```

Za pomocą znacznika `{% extends %}` szablonu informujemy Django o konieczności dziedziczenia szablonu po szablonie *blog/base.html*. Następnie wypełniamy treścią bloki `title` i `content` szablonu bazowego. Później przeprowadzamy iterację przez posty i wyświetlamy ich tytuł, datę, autora oraz treść. Tytuł posta jest łączem w postaci kanonicznego adresu URL posta. Adres URL budujemy przy użyciu dostarczonego przez Django znacznika szablonu `{% url %}`.

Ten tag szablonu umożliwia dynamiczne tworzenie adresów URL według ich nazw. Aby odwołać się do adresu URL `post_detail` w przestrzeni nazw `blog`, używamy znacznika `blog:post_detail`. Aby zbudować adres URL dla każdego posta, przekazujemy wymagany parametr `post.id`.

Wskazówka

Do tworzenia adresów URL w swoich szablonach zamiast zapisywania adresów URL zakodowanych na sztywno zawsze używaj znaczników szablonu `{% url %}`. Dzięki temu adresy URL będą łatwiejsze w utrzymaniu.

W treści posta stosowane są dwa filtry szablonu. Pierwszy, `truncatewords`, powoduje ograniczenie treści do podanej liczby słów. Natomiast drugi, `linebreaks`, konwertuje dane wyjściowe na postać kodu HTML wraz z odpowiednimi znakami nowego wiersza. Istnieje możliwość wykorzystania dowolnej liczby filtrów szablonu, a każdy z nich będzie zastosowany względem danych wyjściowych wygenerowanych przez poprzedni.

Uruchomienie aplikacji

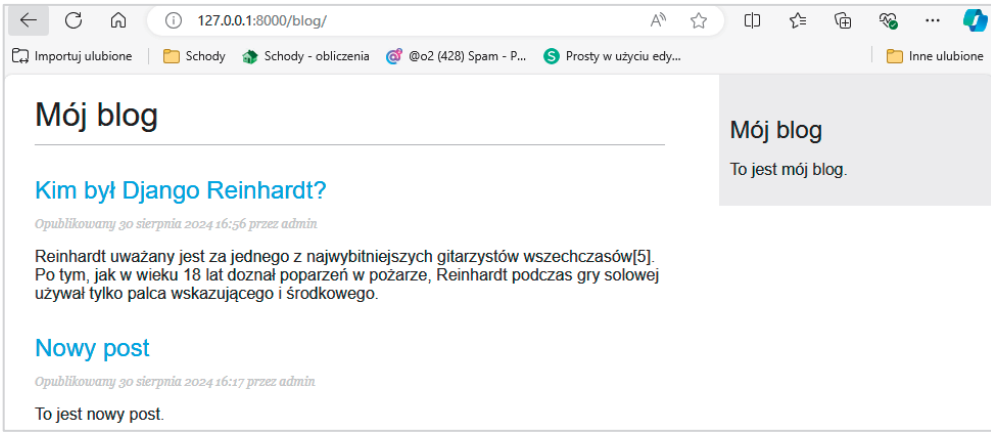
Zmień status początkowego postu na `Opublikowany`, jak pokazano na rysunku 1.16, i utwórz kilka nowych postów, również ze statusem `Opublikowany`.

Rysunek 1.16. Pole statusu dla opublikowanego posta

Przejdź do powłoki i w celu uruchomienia serwera programistycznego uruchom polecenie:

```
python manage.py runserver
```

Jeżeli w przeglądarce internetowej przejdziesz pod adres `http://127.0.0.1:8000/blog/`, przekonasz się, że wszystko działa. Powinieneś zobaczyć widok pokazany na rysunku 1.17.



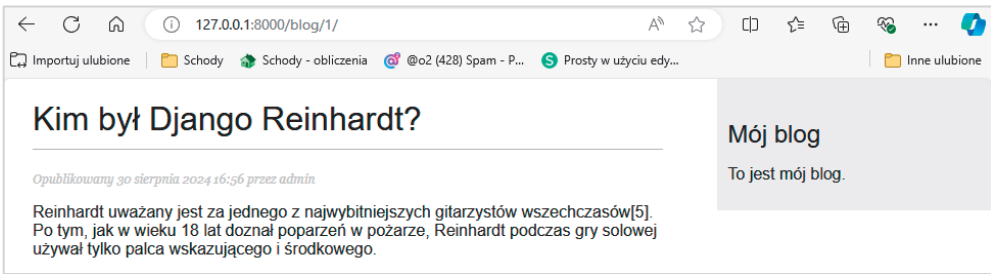
Rysunek 1.17. Strona widoku listy postów

Tworzenie szablonu szczegółów posta

Następnie przeprowadź edycję pliku *post/detail.html*.

```
{% extends "blog/base.html" %}

{% block title %}{{ post.title }}{% endblock %}

{% block content %}
  <h1>{{ post.title }}</h1>
  <p class="date">
    Opublikowany {{ post.publish }} przez {{ post.author }}
  </p>
  {{ post.body|linebreaks }}
{% endblock %}
```

Powróć do przeglądarki internetowej i kliknij jeden z tytułów posta, aby przejść do jego widoku szczegółowego. Powinieneś zobaczyć widok pokazany na rysunku 1.18.

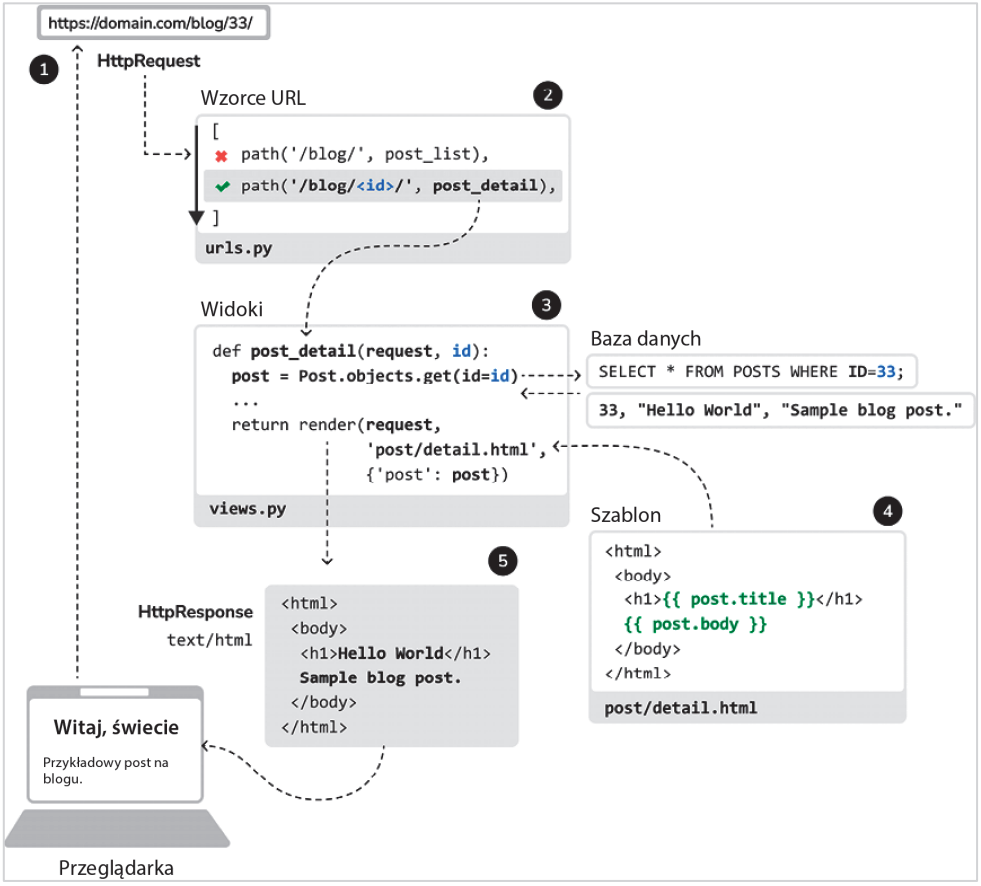


Rysunek 1.18. Strona widoku szczegółów posta

Powinien zawierać wygenerowany automatycznie identyfikator posta, np. `/blog/1/`.

Cykl żądanie-odpowiedź

Na przykładzie zbudowanej aplikacji przyjrzyjmy się cyklowi żądanie-odpowiedź frameworka Django. Uproszczony przykład tego, w jaki sposób Django przetwarza żądania HTTP i generuje odpowiedzi, pokazałem na rysunku 1.19.



Rysunek 1.19. Cykl żądanie-odpowiedź frameworka Django

Przyjrzyjmy się procesowi żądanie-odpowiedź frameworka Django:

1. Przeglądarka internetowa żąda strony za pomocą adresu URL, na przykład *https://domain.com/blog/33/*. Serwer WWW odbiera żądanie HTTP i przekazuje je do frameworka Django.
2. Django przegląda wszystkie wzorce adresów URL zdefiniowane w konfiguracji wzorców adresów URL. Framework sprawdza podaną ścieżkę URL z każdym ze wzorców, w kolejności ich zdefiniowania, i zatrzymuje się na pierwszym, który pasuje do żądanego adresu URL. W tym przypadku ścieżce `/blog/33/` odpowiada wzorzec `/blog/<id>/`.

3. Następnie framework importuje widok dopasowanego wzorca adresu URL i wykonuje go, przekazując egzemplarz klasy `HttpRequest` oraz argumenty pozycyjne lub w postaci słów kluczowych. Do pobierania informacji z bazy danych widok potencjalnie używa modeli danych. Mechanizm ORM frameworka Django tłumaczy obiekty `QuerySet` na zapytania SQL i wykonuje je w bazie danych.
4. Do renderowania szablonu HTML widok korzysta z funkcji `render()`, do której przekazuje obiekt `Post` jako zmienną `context`.
5. Wyrenderowana treść jest zwracana przez widok jako obiekt `HttpResponse`, domyślnie z typem zawartości `text/html`.

To jest podstawowy sposób, w jaki framework Django przetwarza żądania. W celu zachowania prostoty opisu w powyższym schemacie nie uwzględniono warstwy middleware frameworka Django. Z warstwy middleware skorzystasz w różnych przykładach tej książki. W rozdziale 17., „Wdrożenie", dowiesz się również, jak tworzyć spersonalizowane warstwy middleware.

Polecenia zarządzające używane w tym rozdziale

W tym rozdziale skorzystałem z szeregu poleceń zarządzających frameworka Django. Trzeba się z nimi zapoznać, ponieważ będą często używane w całej książce. Powróćmy do poleceń, które wykorzystałem w tym rozdziale.

Aby utworzyć strukturę plików dla nowego projektu Django o nazwie `mysite`, użyłem polecenia:

```
django-admin startproject mysite
```

Aby utworzyć strukturę plików dla nowej aplikacji Django o nazwie `blog`:

```
python manage.py startapp blog
```

Aby zastosować wszystkie migracje bazy danych:

```
python manage.py migrate
```

Aby utworzyć migracje dla modeli aplikacji `blog`:

```
python manage.py makemigrations blog
```

Aby wyświetlić instrukcje SQL, które zostaną wykonane podczas pierwszej migracji aplikacji `blog`:

```
python manage.py sqlmigrate blog 0001
```

Aby uruchomić serwer programistyczny frameworka Django:

```
python manage.py runserver
```

Aby uruchomić serwer programistyczny z podaniem hosta i portu oraz ze wskazaniem pliku ustawień:

```
python manage.py runserver 127.0.0.1:8001 --settings=mysite.settings
```

Aby uruchomić powłokę Django:

```
python manage.py shell
```

Aby utworzyć superużytkownika za pomocą frameworka uwierzytelniania Django:

```
python manage.py createsuperuser
```

Pełną listę dostępnych poleceń zarządzających można znaleźć na stronie *https://docs.djangoproject.com/en/5.0/ref/django-admin/*.

Podsumowanie

W tym rozdziale poznałeś podstawy frameworka aplikacji webowych Django na podstawie procesu utworzenia prostej aplikacji bloga. Zaprojektowałeś modele danych i przeprowadziłeś migracje w projekcie. Utworzyłeś widoki, szablony i adresy URL dla bloga.

W następnym rozdziale dowiesz się, jak usprawnić aplikację bloga przez utworzenie kanonicznych adresów URL dla modeli i jak budować przyjazne dla SEO adresy URL dla postów na blogu. Dowiesz się również, jak zaimplementować stronicowanie obiektów i jak budować widoki oparte na klasach. Zaimplementujesz także formularze Django, które umożliwiają użytkownikom polecanie postów przez e-mail i komentowanie postów.

Zasoby dodatkowe

Oto lista zasobów zawierających dodatkowe informacje związane z tematami omawianymi w tym rozdziale:

- Kod źródłowy przykładów zamieszczonych w tym rozdziale —*https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter01*.
- Strona pobierania Pythona — *https://www.python.org/downloads/*.
- Program uruchamiający Pythona dla Windows — *https://docs.python.org/3/using/windows.html#launcher*.
- Biblioteka Pythona `venv` dla środowisk wirtualnych — *https://docs.python.org/3/library/venv.html*.
- Instrukcje instalacji `pip` w Pythonie — *https://pip.pypa.io/en/stable/installation/*.
- Opcje instalacji Django — *https://docs.djangoproject.com/en/5.0/topics/install/*.
- Informacje o wydaniu Django 5.0 — *https://docs.djangoproject.com/en/5.0/releases/5.0/*.
- Narzędzie `django-upgrade` — *https://github.com/adamchainz/django-upgrade*.
- Narzędzie pyupgrade — *https://github.com/asottile/pyupgrade*.
- Filozofie projektowania dla frameworka Django — *https://docs.djangoproject.com/en/5.0/misc/design-philosophies/*.
- Dokumentacja pól modeli frameworka Django — *https://docs.djangoproject.com/en/5.0/ref/models/fields/*.

- Dokumentacja indeksów modelu — *https://docs.djangoproject.com/en/5.0/ref/models/indexes/*.
- Obsługa typów wyliczeniowych w Pythonie — *https://docs.python.org/3/library/enum.html*.
- Typy wyliczeniowe w modelach frameworka Django — *https://docs.djangoproject.com/en/5.0/ref/models/fields/#enumeration-types*.
- Dokumentacja ustawień frameworka Django — *https://docs.djangoproject.com/en/5.0/ref/settings/*.
- Wartości domyślne bazy danych dla pól modelu — *https://docs.djangoproject.com/en/5.0/ref/models/fields/#django.db.models.Field.db_default*.
- Funkcje bazy danych — *https://docs.djangoproject.com/en/5.0/ref/models/database-functions/*.
- Strona administracyjna Django — *https://docs.djangoproject.com/en/5.0/ref/contrib/admin/*.
- Dokumentacja API modeli — *https://docs.djangoproject.com/en/5.0/ref/models/*.
- Wykonywanie zapytań za pomocą ORM frameworka Django — *https://docs.djangoproject.com/en/5.0/topics/db/queries/*.
- Dokumentacja interfejsu API QuerySet — *https://docs.djangoproject.com/en/5.0/ref/models/querysets/*.
- Złożone dopasowania z wykorzystaniem obiektów `Q` — *https://docs.djangoproject.com/en/5.0/topics/db/queries/#complex-lookups-with-q-objects*.
- Dyspozytor URL frameworka Django — *https://docs.djangoproject.com/en/5.0/topics/http/urls/*.
- Język szablonów Django — *https://docs.djangoproject.com/en/5.0/ref/templates/language/*.
- Wbudowane tagi szablonów i filtry — *https://docs.djangoproject.com/en/5.0/ref/templates/builtins/*.
- Polecenia zarządzające frameworka Django — *https://docs.djangoproject.com/en/5.0/ref/django-admin/*.
- Pliki statyczne wykorzystywane w przykładach kodu w tym rozdziale — *https://github.com/PacktPublishing/Django-5-by-example/tree/master/Chapter01/mysite/blog/static*.

Rozdział 2

Usprawnianie bloga i dodanie funkcjonalności społecznościowych

W poprzednim rozdziale, przy okazji tworzenia prostej aplikacji blogowej, poznałeś główne komponenty frameworka Django. Podczas tworzenia aplikacji skorzystaliśmy z widoków, szablonów i adresów URL. W tym rozdziale rozszerzymy funkcjonalność aplikacji blogowej o funkcjonalności dostępne dziś na wielu platformach blogowych.

W tym rozdziale omówię następujące zagadnienia:

- Kanoniczne adresy URL dla modeli
- Tworzenie dla postów adresów URL przyjaznych dla SEO
- Dodanie stronicowania do widoku listy postów
- Użycie widoków opartych na klasach
- Wysyłanie wiadomości e-mail w Django
- Korzystanie z formularzy Django do udostępniania postów przez e-mail
- Dodawanie komentarzy do postów z wykorzystaniem formularzy generowanych z modeli

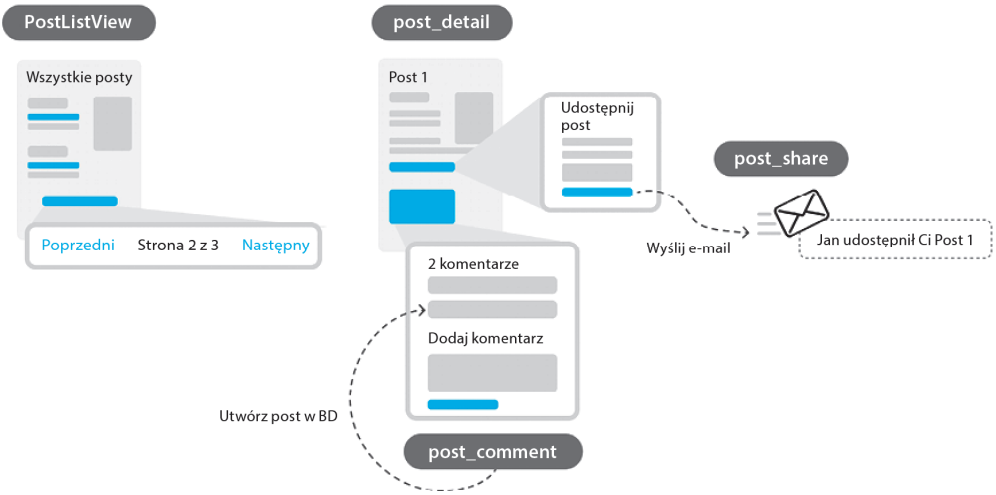
Przegląd funkcjonalności

Reprezentację widoków, szablonów i funkcjonalności, które zbudujemy w tym rozdziale, przedstawiono na rysunku 2.1.

W tym rozdziale, aby poruszać się po wszystkich postach, dodamy stronicowanie do strony z listą postów. Dowiesz się także, jak tworzyć w Django widoki oparte na klasach, i przekonwertujemy widok `post_list` na oparty na klasie widok o nazwie `PostListView`.

Utworzymy widok `post_share`, aby udostępniać posty za pośrednictwem poczty e-mail. Skorzystamy z formularzy Django w celu udostępniania postów i wysyłania rekomendacji e-mailem za pośrednictwem protokołu **SMTP** (ang. *Simple Mail Transfer Protocol*). Aby dodawać komentarze do postów, utworzymy model `Comment` do przechowywania komentarzy oraz, za pomocą formularzy dla modeli, zbudujemy widok `post_comment`.

Kod źródłowy przykładów z tego rozdziału można znaleźć pod adresem *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter02*.



Rysunek 2.1. Schemat funkcjonalności zbudowanych w rozdziale 2.

Wszystkie pakiety Pythona użyte w tym rozdziale są zawarte w pliku *requirements.txt* dołączonym do plików z kodem źródłowym przykładów z tego rozdziału. Aby zainstalować każdy pakiet Pythona, możesz postępować zgodnie z instrukcjami zamieszczonymi w poniższych podrozdziałach. Możesz także zainstalować wszystkie wymagania jednocześnie za pomocą polecenia `pip install -r requirements.txt`.

Kanoniczne adresy URL dla modeli

Witryna WWW może się składać z różnych stron, które wyświetlają tę samą treść. W aplikacji bloga początkowa część treści każdego posta jest wyświetlana zarówno na stronie z listą postów, jak i na stronie ze szczegółami posta. Kanoniczny adres URL to preferowany adres URL dla zasobu. Można go zinterpretować jako adres URL najbardziej reprezentatywnej strony dla określonej treści. W Twojej witrynie mogą się znajdować różne strony, na których są wyświetlane posty, ale jest jeden adres URL, którego używasz jako głównego adresu URL posta.

Kanoniczne adresy URL pozwalają określić adres URL głównej kopii strony. Django umożliwia zaimplementowanie w modelach metody `get_absolute_url()`, która zwraca kanoniczny adres URL obiektu.

Do zbudowania kanonicznego adresu URL dla obiektów `Post` użyjemy adresu URL `post_detail` zdefiniowanego we wzorcach adresów URL aplikacji. Django udostępnia różne funkcje rozpoznawania adresów URL, które umożliwiają dynamiczne tworzenie adresów URL przy użyciu nazw i wszelkich wymaganych parametrów. W tworzonej aplikacji użyjemy funkcji narzędziowej `reverse()` z modułu `django.urls`.

Zmodyfikuj plik *models.py* aplikacji `blog`, aby zaimportować funkcję `reverse()`, i dodaj do modelu `Post` metodę `get_absolute_url()` w następujący sposób. Nowy kod został wyróżniony pogrubioną czcionką.

```
from django.conf import settings
from django.db import models
from django.urls import reverse
from django.utils import timezone

class PublishedManager(models.Manager):
     def get_queryset(self):
        return (
             super().get_queryset().filter(status=Post.Status.PUBLISHED)
         )

class Post(models.Model):
    # …
     class Meta:
        ordering = ['-publish']
         indexes = [
             models.Index(fields=['-publish']),
         ]

    def __str__(self):
         return self.title

    def get_absolute_url(self):
         return reverse(
             'blog:post_detail',
             args=[self.id]
    )
```

Funkcja `reverse()` zbuduje adres URL dynamicznie, z wykorzystaniem nazwy adresu URL zdefiniowanej we wzorcach adresów URL. Użyliśmy przestrzeni nazw `blog`, po której następuje dwukropek i nazwa adresu URL `post_detail`. Pamiętaj, że przestrzeń nazw `blog` jest zdefiniowana w głównym pliku *urls.py* projektu. Aby z niej skorzystać, należy dołączyć wzorce adresów URL z modułu `blog.urls`. Adres URL `post_detail` jest zdefiniowany w pliku *urls.py* aplikacji `blog`.

Wynikowy ciąg, `blog:post_detail`, możesz wykorzystywać globalnie w swoim projekcie jako odniesienie do adresu URL szczegółów posta.

Do tego adresu URL trzeba przekazać obowiązkowy parametr — identyfikator posta do pobrania.

Aby przekazać identyfikator obiektu `Post` jako argument pozycyjny, skorzystaliśmy z odwołania `args=[self.id]`.

Więcej informacji na temat funkcji narzędziowych do obsługi adresów URL można znaleźć pod adresem *https://docs.djangoproject.com/en/5.0/ref/urlresolvers/*.

Zamieńmy szczegółowe adresy URL postów w szablonach nową metodą `get_absolute_url()`.

W pliku *blog/templates/blog/post/list.html* zastąp wiersz:

```
<a href="{% url 'blog:post_detail' post.id %}">
```

takim:

```
<a href="{{ post.get_absolute_url }}">
```

Plik *blog/post/list.html* powinien teraz mieć następującą zawartość.

```
{% extends "blog/base.html" %}

{% block title %}Mój Blog{% endblock %}

{% block content %}
  <h1>Mój Blog</h1>
  {% for post in posts %}
    <h2>
      <a href="{{ post.get_absolute_url }}">
        {{ post.title }}
      </a>
    </h2>
    <p class="date">
      Opublikowany {{ post.publish }} przez {{ post.author }}
    </p>
    {{ post.body|truncatewords:30|linebreaks }}
  {% endfor %}
{% endblock %}
```

Przejdź do powłoki i w celu uruchomienia serwera programistycznego, uruchom polecenie:

```
python manage.py runserver
```

Następnie w przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/blog/`. Linki do poszczególnych postów na blogu powinny nadal działać.

Django buduje je teraz przy użyciu metody `get_absolute_url()` modelu `Post`.

Tworzenie dla postów adresów URL przyjaznych dla SEO

Kanoniczny adres URL widoku szczegółów posta na blogu wygląda obecnie tak: */blog/1/*. W tym punkcie zmodyfikuję wzorzec adresu URL w taki sposób, aby utworzyć adresy URL dla postów przyjazne dla technik optymalizacji wyszukiwarek (ang. *search engine optimization* — SEO). Do zbudowania adresów URL dla pojedynczych postów skorzystamy z wartości atrybutów `publish` i `slug`. Dzięki włączeniu dat adres URL szczegółów posta będzie miał postać */blog/2022/1/1/kim-byl-django-reinhardt/*. W ten sposób dostarczymy wyszukiwarkom wygodne do indeksowania adresy URL zawierające zarówno tytuł, jak i datę posta.

Aby zapewnić możliwość pobierania za pomocą kombinacji daty publikacji i slugu pojedynczego posta, musimy zadbać o to, aby żaden post przechowywany w bazie danych nie miał takiego samego slugu i daty publikacji, co inny post. Aby zapobiec przechowywaniu duplikatów postów przez model `Post`, zdefiniujemy atrybut `slug` jako unikatowy dla daty publikacji posta.

Wyedytuj plik *models.py* i dodaj do pola `slug` modelu `Post` parametr `unique_for_date`.

```
class Post(models.Model):
    # …
    slug = models.SlugField(
         max_length=250,
         unique_for_date='publish'
     )
    # ...
```

Dzięki wykorzystaniu parametru `unique_for_date` pole `slug` musi być teraz unikatowe dla daty zapisanej w polu `publish`. Należy zauważyć, że pole `publish` jest egzemplarzem obiektu `DateTimeField`, ale sprawdzanie unikatowych wartości będzie wykonywane tylko względem daty (godziny nie będą uwzględniane). Django uniemożliwi zapisanie nowego postu z tym samym slugiem, jaki ma istniejący post dla danej daty publikacji. Dzięki zapewnieniu unikatowości slugów dla daty publikacji możemy pobierać pojedyncze posty według pól `publish` i `slug`.

Ponieważ zmodyfikowaliśmy modele, powinniśmy stworzyć migracje. Należy zapamiętać, że parametr `unique_for_date` nie jest wymuszany na poziomie bazy danych, więc migracja bazy danych nie jest konieczna. Framework Django wykorzystuje jednak migracje do śledzenia zmian w modelu. Migrację utworzymy tylko po to, aby zachować zgodność migracji z bieżącym stanem modelu.

W wierszu polecenia powłoki uruchom polecenie:

```
python manage.py makemigrations blog
```

Powinieneś otrzymać taki wynik:

```
Migrations for 'blog':
     blog/migrations/0002_alter_post_slug.py
     - Alter field slug on post
```

Django właśnie utworzył w katalogu *migrations* aplikacji `blog` plik *0002_alter_post_slug.py*.

Aby zastosować istniejące migracje, wykonaj następujące polecenie w wierszu polecenia powłoki.

```
python manage.py migrate
```

Wyświetli się wynik, który kończy się wierszem:

```
Applying blog.0002_alter_post_slug... OK
```

Django uzna, że wszystkie migracje zostały zastosowane, a modele są zsynchronizowane. W bazie danych nie zostaną wykonane żadne działania, ponieważ parametr `unique_for_date` nie jest egzekwowany na poziomie bazy danych.

Modyfikowanie wzorców adresów URL

Teraz zmodyfikujmy wzorce adresów URL w taki sposób, aby dla adresu URL szczegółów posta używały daty publikacji i slugu.

Wyedytuj plik *urls.py* aplikacji `blog` i zastąp wiersz:

```
path('<int:id>/', views.post_detail, name='post_detail'),
```

wierszami:

```
path(
    '<int:year>/<int:month>/<int:day>/<slug:post>/',
     views.post_detail,
     name='post_detail'
),
```

Plik *urls.py* powinien teraz mieć następującą zawartość.

```
from django.urls import path
from . import views

app_name = 'blog'

urlpatterns = [
     # Widoki postów
     path('', views.post_list, name='post_list'),
     path(
        '<int:year>/<int:month>/<int:day>/<slug:post>/',
         views.post_detail,
         name='post_detail'
     ),
]
```

Wzorzec adresu URL dla widoku `post_detail` przyjmuje argumenty:

- **`year`**. Wymaga podania liczby całkowitej.
- **`month`**. Wymaga podania liczby całkowitej.
- **`day`**. Wymaga podania liczby całkowitej.
- **`post`**. Wymaga slugu (jest to krótka etykieta zawierająca wyłącznie litery, cyfry, znaki podkreślenia lub łączniki).

Dla parametrów `year`, `month` i `day` używany jest konwerter ścieżki `int`, podczas gdy dla parametru `post` jest używany konwerter ścieżki `slug`. O konwerterach ścieżek dowiedziałeś się w poprzednim rozdziale. Opis wszystkich konwerterów ścieżek dostarczanych z frameworkiem Django można znaleźć na stronie *https://docs.djangoproject.com/en/5.0/topics/http/urls/#path-converters*.

Posty mają teraz adres URL przyjazny dla SEO, tworzony z datą i slugiem. Teraz zmodyfikujmy odpowiednio widok `post_detail`.

Modyfikowanie widoków

Teraz trzeba zmienić parametry widoku `post_detail` tak, aby pasowały do nowych parametrów adresu URL, i użyć ich do pobrania odpowiedniego obiektu `Post`.

Wyedytuj plik *views.py* i zmodyfikuj widok `post_detail` w następujący sposób.

```
def post_detail(request, year, month, day, post):
    post = get_object_or_404(
         Post,
         status=Post.Status.PUBLISHED,
         slug=post,
         publish__year=year,
```

```
            publish__month=month,
            publish__day=day)
        return render(
            request,
            'blog/post/detail.html',
            {'post': post}
        )
```

Zmodyfikowaliśmy widok `post_detail` w taki sposób, aby na podstawie parametrów `year`, `month`, `day` i `post` pobierał opublikowany post o podanym slugu i dacie. Dzięki temu, że wcześniej dodaliśmy do pola `slug` modelu `Post` atrybut `unique_for_date='publish'`, mamy pewność istnienia w bazie danych tylko jednego posta ze slugiem dla danej daty. W ten sposób możesz pobierać pojedyncze posty na podstawie daty i slugu.

Modyfikowanie kanonicznego adresu URL dla postów

Trzeba również zmodyfikować parametry kanonicznego adresu URL dla postów na blogu tak, aby pasowały do nowych parametrów adresu URL.

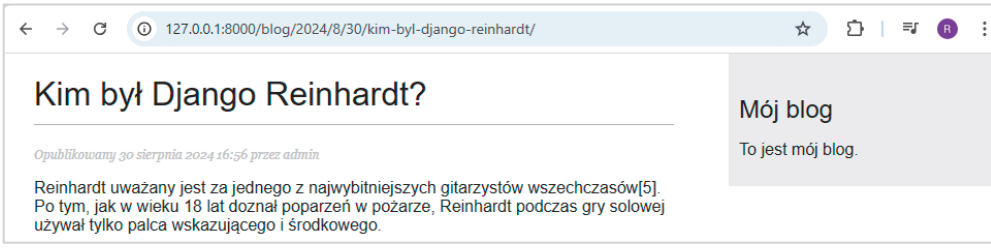
Zmodyfikuj plik *models.py* aplikacji `blog` i wprowadź zmiany w metodzie `get_absolute_url()` w taki sposób:

```
class Post(models.Model):
    # …
    def get_absolute_url(self):
        return reverse(
            'blog:post_detail',
            args=[
                self.publish.year,
                self.publish.month,
                self.publish.day,
                self.slug
            ]
        )
```

Otwórz powłokę i uruchom serwer programistyczny za pomocą polecenia:

```
python manage.py runserver
```

Powróć do przeglądarki internetowej i kliknij jeden z tytułów posta, aby przejść do jego widoku szczegółowego. Powinieneś zobaczyć widok podobny do pokazanego na rysunku 2.2.



Rysunek 2.2. Strona widoku szczegółów posta

Zaprojektowałeś przyjazne dla SEO adresy URL dla postów na blogu. Adres URL posta ma teraz postać */blog/2024/1/1/kim-byl-django-reinhardt/*.

Po zaimplementowaniu adresów URL przyjaznych dla SEO skoncentrujemy się na systemie poruszania się po postach z wykorzystaniem stronicowania.

Dodanie stronicowania

Po jakimś czasie dodawania treści do bloga liczba pozycji w bazie danych może osiągnąć kilkaset lub kilka tysięcy. Zamiast wyświetlać wszystkie posty na jednej stronie, możesz podzielić listę postów na kilka stron i dołączyć linki nawigacyjne do różnych stron. Taka funkcja nazywa się stronicowaniem i występuje w prawie każdej aplikacji webowej wyświetlającej długie listy elementów.

Na przykład wyszukiwarka Google używa stronicowania w celu podzielenia wyników wyszukiwania na wiele stron. Łącza stronicowania wyszukiwarki Google dla kolejnych stron z wynikami wyszukiwania są pokazane na rysunku 2.3.

Rysunek 2.3. Łącza stronicowania wyszukiwarki Google dla stron wyników wyszukiwania

Django ma wbudowane klasy obsługi stronicowania, które pozwalają na bardzo łatwe zarządzanie stronicowaniem danych. Możesz zdefiniować liczbę obiektów, które chcesz zwrócić na stronie, a następnie pobierać posty odpowiadające stronie zażądanej przez użytkownika.

Dodanie stronicowania do widoku listy postów

Aby można było łatwo poruszać się po postach opublikowanych na blogu, dodamy do widoku listy postów mechanizm stronicowania.

Przeprowadź edycję pliku *views.py* aplikacji `blog` w celu zaimportowania oferowanej przez Django klasy `Paginator` oraz zmodyfikuj widok `post_list` w następujący sposób.

```
from django.core.paginator import Paginator
from django.shortcuts import get_object_or_404, render
from .models import Post

def post_list(request):
    post_list = Post.published.all()
    # Stronicowanie z 3 postami na stronę
    paginator = Paginator(post_list, 3)
    page_number = request.GET.get('page', 1)
    posts = paginator.page(page_number)

    return render(
        request,
```

```
        'blog/post/list.html',
        {'posts': posts}
    )
```

Przeanalizujmy nowy kod, który dodaliśmy do widoku.

1. Tworzymy egzemplarz klasy `Paginator` z liczbą obiektów do zwrócenia na stronę. W tym przypadku na jednej stronie będziemy wyświetlać trzy posty.
2. Pobieramy parametr HTTP `GET` strony i przechowujemy go w zmiennej `page_number`. Ten parametr zawiera żądany numer strony. Jeśli w parametrach `GET` żądania nie ma pozycji `page`, używamy domyślnej wartości 1 w celu załadowania pierwszej strony wyników.
3. Pobieramy obiekty dla wybranej strony przez wywołanie metody `page()` klasy `Paginator`. Ta metoda zwraca obiekt `Page`, który zapisujemy w zmiennej `posts`.
4. Do szablonu przekazujemy obiekt `posts`.

Tworzenie szablonu stronicowania

Aby użytkownicy mogli przeglądać różne strony, trzeba stworzyć mechanizmy nawigacji po stronie. Do wyświetlania łączy stronicowania utworzymy oddzielny szablon. Zdefiniujemy go w sposób generyczny tak, aby można go było użyć do stronicowania dowolnej listy obiektów naszej aplikacji webowej.

W katalogu *templates/* utwórz nowy plik i nadaj mu nazwę *pagination.html*. Wprowadź w tym pliku następujący kod HTML.

```
<div class="pagination">
  <span class="step-links">
    {% if page.has_previous %}
      <a href="?page={{ page.previous_page_number }}">Poprzednia</a>
    {% endif %}
    <span class="current">
       Strona {{ page.number }} z {{ page.paginator.num_pages }}.
    </span>
    {% if page.has_next %}
      <a href="?page={{ page.next_page_number }}">Następna</a>
    {% endif %}
  </span>
</div>
```

Tak może wyglądać generyczny szablon stronicowania. Szablon stronicowania oczekuje obiektu `Page` w celu wygenerowania łączy pozwalających na przejście do poprzedniej i następnej strony wyników, także do wyświetlenia numeru strony bieżącej i całkowitej liczby stron wyników.

Powracamy do szablonu *blog/post/list.html* i dołączamy szablon *pagination.html* na końcu bloku `{% content %}`:

```
{% extends "blog/base.html" %}

{% block title %}Mój Blog{% endblock %}

{% block content %}
```

```
<h1>Mój Blog</h1>
{% for post in posts %}
  <h2>
    <a href="{{ post.get_absolute_url }}">
      {{ post.title }}
    </a>
  </h2>
  <p class="date">
    Opublikowany {{ post.publish }} przez {{ post.author }}
  </p>
  {{ post.body|truncatewords:30|linebreaks }}
{% endfor %}
{% include "pagination.html" with page=posts %}
{% endblock %}
```

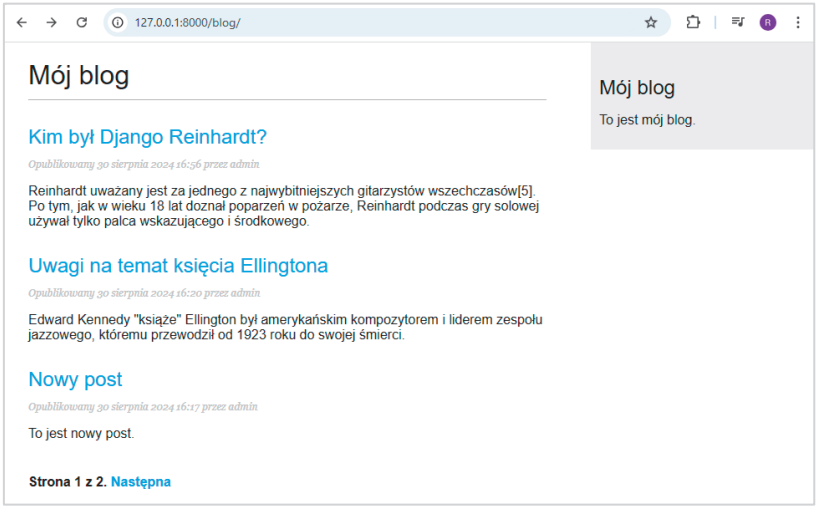
Tag szablonu `{% include %}` ładuje podany szablon i renderuje go przy użyciu bieżącego kontekstu szablonu. Aby przekazać do szablonu dodatkowe zmienne kontekstowe, skorzystaliśmy z konstrukcji `with`. Szablon stronicowania używa do renderowania zmiennej `page`, podczas gdy obiekt `Page`, który przekazujemy z naszego widoku do szablonu, ma nazwę `posts`. Do przekazania zmiennej oczekiwanej przez szablon stronicowania wykorzystaliśmy konstrukcję `with page=posts`. Metodę tę możesz zastosować, aby użyć szablonu stronicowania dla dowolnego typu obiektu.

Otwórz powłokę i uruchom serwer programistyczny za pomocą polecenia:

```
python manage.py runserver
```

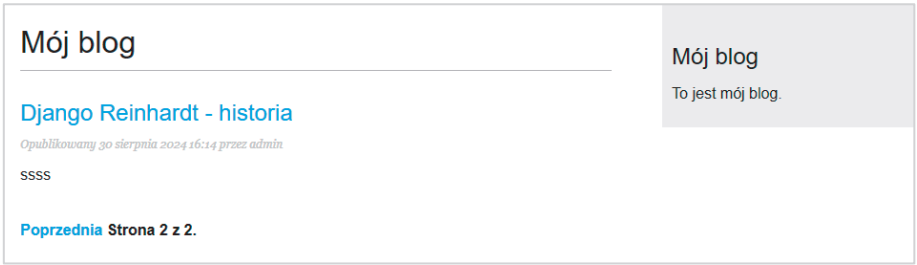
Otwórz w przeglądarce stronę `http://127.0.0.1:8000/admin/blog/post/` i za pomocą strony administracyjnej utwórz cztery różne posty. Pamiętaj, aby dla wszystkich z nich ustawić stan na *Opublikowany*.

Teraz w przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/blog/`. Powinieneś zobaczyć pierwsze trzy posty w odwrotnej kolejności chronologicznej, a następnie linki nawigacyjne na dole listy postów, tak jak pokazałem na rysunku 2.4.



Rysunek 2.4. Strona z listą postów z uwzględnieniem podziału na strony

Aby zobaczyć ostatni post, kliknij *Następna*. Adres URL drugiej strony zawiera parametr `GET ?page=2`. Widok wykorzystuje ten parametr do załadowania przez mechanizm stronicowania żądanej strony wyników (rysunek 2.5).



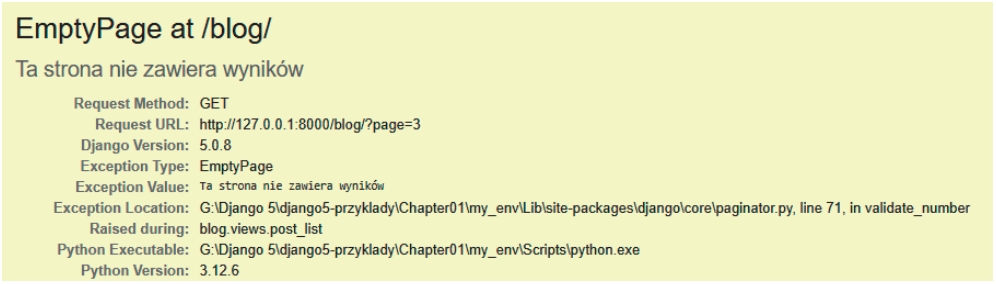
Rysunek 2.5. Druga strona wyników

Doskonale! Łącza stronicowania działają zgodnie z oczekiwaniami.

Obsługa błędów stronicowania

Teraz, gdy podział na strony działa, możemy dodać w widoku obsługę wyjątków dla błędów podziału na strony. Parametr `page` używany przez widok do pobierania danej strony może potencjalnie zostać użyty z nieprawidłowymi wartościami, na przykład z nieistniejącymi numerami stron lub wartością tekstową, której nie można użyć jako numeru strony. Zaimplementujemy odpowiednią obsługę błędów dla tych przypadków.

Otwórz w przeglądarce internetowej stronę spod adresu `http://127.0.0.1:8000/blog/?page=3`. Powinna zostać wyświetlona strona z komunikatem o błędzie pokazana na rysunku 2.6.



Rysunek 2.6. Strona błędu EmptyPage

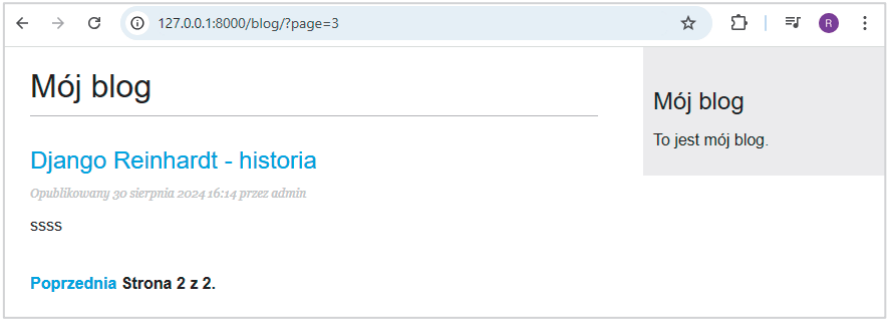
Podczas pobierania strony 3 obiekt `Paginator` zgłasza wyjątek `EmptyPage`, ponieważ strona jest poza zakresem. Nie ma wyników do wyświetlenia. Spróbujmy obsłużyć ten błąd w widoku.

Przeprowadź edycję pliku *views.py* aplikacji `blog` w celu dodania koniecznych instrukcji importowania oraz zmodyfikuj widok `post_list` w następujący sposób.

```
from django.core.paginator import EmptyPage, Paginator
from django.shortcuts import get_object_or_404, render
from .models import Post

def post_list(request):
    post_list = Post.published.all()
    # Stronicowanie z 3 postami na stronę
    paginator = Paginator(post_list, 3)
    page_number = request.GET.get('page', 1)
    try:
        posts = paginator.page(page_number)
    except EmptyPage:
        # Jeśli zmienna page_number jest poza zakresem, wyślij ostatnią stronę wyników
        posts = paginator.page(paginator.num_pages)
    return render(
        request,
        'blog/post/list.html',
        {'posts': posts}
    )
```

Do obsługi wyjątku `EmptyPage` podczas pobierania strony dodaliśmy bloki `try` i `except`. Jeśli żądana strona jest poza zakresem, zwracamy ostatnią stronę wyników. Za pomocą wywołania `paginator.num_pages` otrzymujemy całkowitą liczbę stron. Całkowita liczba stron jest taka sama jak numer ostatniej strony.

Ponownie otwórz w przeglądarce internetowej stronę spod adresu `http://127.0.0.1:8000/blog/?page=3`. Teraz widok obsługuje wyjątek, a ostatnia strona wyników jest zwracana w sposób pokazany na rysunku 2.7.

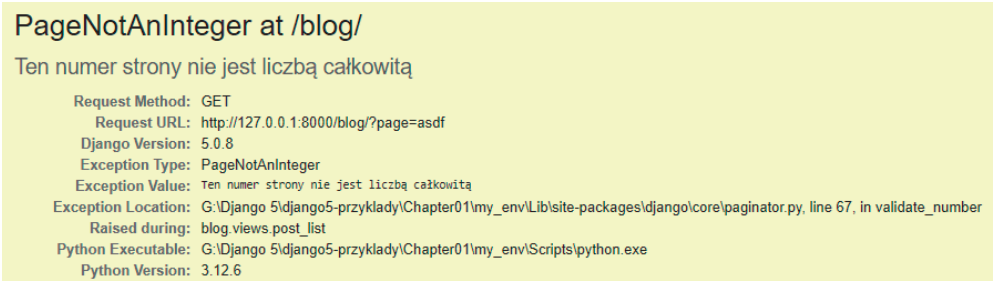


Rysunek 2.7. Ostatnia strona wyników

Widok powinien również obsłużyć przypadek, gdy w parametrze `page` zostanie przekazane coś innego niż liczba całkowita.

Otwórz w przeglądarce internetowej stronę spod adresu `http://127.0.0.1:8000/blog/?page=asdf`. Powinna zostać wyświetlona strona z komunikatem o błędzie pokazana na rysunku 2.8.

W tym przypadku podczas próby pobierania strony `asdf` obiekt `Paginator` zgłasza wyjątek `PageNotAnInteger`, ponieważ numery stron mogą być tylko liczbami całkowitymi. Spróbujmy obsłużyć ten błąd w widoku.



Rysunek 2.8. Strona błędu PageNotAnInteger

Przeprowadź edycję pliku *views.py* aplikacji `blog` w celu dodania koniecznych instrukcji importowania oraz zmodyfikuj widok `post_list` w taki sposób.

```
from django.shortcuts import get_object_or_404, render
from .models import Post
from django.core.paginator import EmptyPage, PageNotAnInteger, Paginator

def post_list(request):
    post_list = Post.published.all()
    # Stronicowanie z 3 postami na stronę
    paginator = Paginator(post_list, 3)
    page_number = request.GET.get('page')
    try:
        posts = paginator.page(page_number)
    except PageNotAnInteger:
        # Jeśli page_number nie jest liczbą całkowitą, zwróć pierwszą stronę
        posts = paginator.page(1)
    except EmptyPage:
        # Jeśli zmienna page_number jest poza zakresem, wyślij ostatnią stronę wyników
         posts = paginator.page(paginator.num_pages)
     return render(
         request,
         'blog/post/list.html',
        {'posts': posts}
    )
```

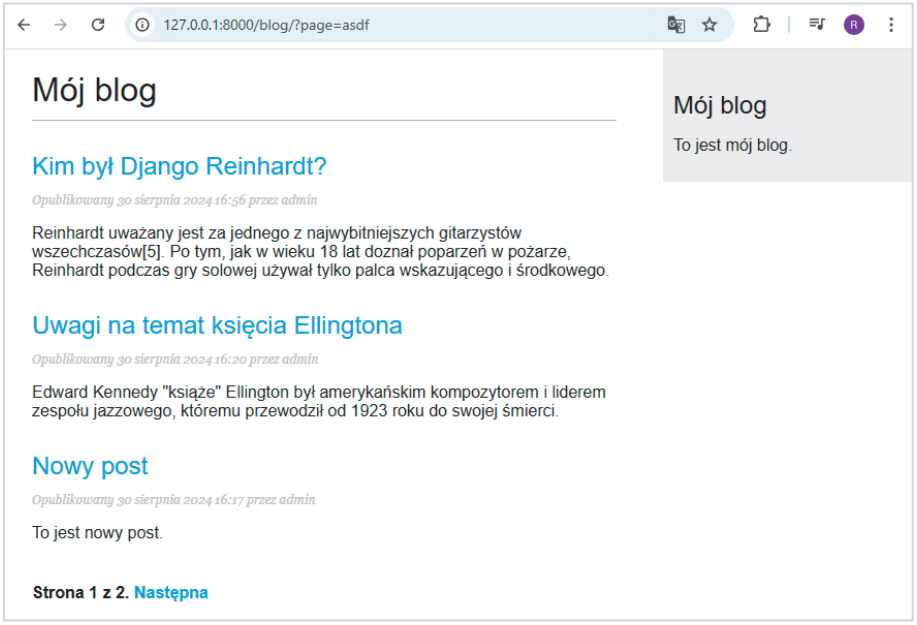
Aby podczas pobierania strony był obsługiwany wyjątek `PageNotAnInteger`, dodaliśmy nowy blok, `except`. Jeśli zmienna reprezentująca żądaną stronę nie jest liczbą całkowitą, zwracamy pierwszą stronę wyników.

Ponownie otwórz w przeglądarce internetowej stronę spod adresu `http://127.0.0.1:8000/blog/?page=asdf`. Teraz widok obsługuje wyjątek, a w wynikach zwracana jest pierwsza strona, tak jak pokazano na rysunku 2.9.

Stronicowanie dla postów na blogu jest teraz w pełni zaimplementowane.

O klasie `Paginator` możesz się dowiedzieć więcej pod adresem *https://docs.djangoproject.com/en/5.0/ref/paginator/*.

Po zaimplementowaniu stronicowania bloga przejdziemy do przekształcenia widoku `post_list` w równoważny widok zbudowany przy użyciu generycznych widoków Django i wbudowanego stronicowania.



Rysunek 2.9. Pierwsza strona wyników

Użycie widoków opartych na klasach

Aplikację blogową zbudowaliśmy przy użyciu widoków opartych na funkcjach. Widoki oparte na funkcjach są proste i wydajne, ale Django pozwala również budować widoki przy użyciu klas.

Widoki oparte na klasach są obok funkcji alternatywnym sposobem implementacji widoków jako obiektów Pythona. Ponieważ widok to wywoływana funkcja otrzymująca żądanie sieciowe i zwracająca odpowiedź na nie, więc istnieje możliwość zdefiniowania widoków jako metod klasy. Django udostępnia bazowe klasy widoków, których możesz użyć do zaimplementowania własnych widoków. Wszystkie tego rodzaju klasy dziedziczą po klasie `View`, która obsługuje metody HTTP oraz inne funkcjonalności.

Po co korzystać z widoków opartych na klasach?

Widoki oparte na klasach w niektórych przypadkach użycia mają przewagę nad widokami opartymi na funkcjach. Widoki oparte na klasach umożliwiają:

- Organizowanie kodu w powiązaniu z metodami HTTP, takimi jak `GET`, `POST` lub `PUT`, w oddzielnych metodach zamiast korzystania z instrukcji warunkowych.
- Używanie dziedziczenia wielokrotnego do tworzenia klas widoku wielokrotnego użytku (znanych również jako domieszki — ang. *mixin*).

Użycie widoku opartego na klasie do wyświetlania listy postów

Abyś zrozumiał, jak pisać widoki oparte na klasach, utworzymy nowy widok oparty na klasie, który będzie odpowiednikiem widoku `post_list`. Stworzymy klasę, która będzie dziedziczyć po oferowanym przez Django generycznym widoku `ListView`. Widok `ListView` pozwala wyświetlić listę obiektów dowolnego typu.

Przeprowadź edycję pliku *views.py* aplikacji `blog` i dodaj w nim poniższy fragment kodu.

```
from django.views.generic import ListView

class PostListView(ListView):
    """
    Alternatywny widok listy postów
    """
    queryset = Post.published.all()
    context_object_name = 'posts'
    paginate_by = 3
    template_name = 'blog/post/list.html'
```

Widok `PostListView` jest analogiczny do zbudowanego wcześniej widoku `post_list`. Zaimplementowaliśmy widok oparty na klasie, który dziedziczy po klasie `ListView`. Zdefiniowaliśmy widok z następującymi atrybutami.

- Używamy pola `queryset`, aby korzystać z niestandardowego obiektu `QuerySet` zamiast pobierania wszystkich obiektów. Zamiast zdefiniować atrybut `queryset`, można by podać `model = Post`, a Django za nas przygotuje ogólną kolekcję `QuerySet` `Post.objects.all()`.
- Dla wyników zapytania używamy zmiennej kontekstu `posts`. Jeśli nie zostanie podana żadna wartość `context_object_name`, wartością domyślną będzie `object_list`.
- Za pomocą klauzuli `paginate_by` definiujemy stronicowanie, tak by były wyświetlane po trzy obiekty na stronę.
- Za pomocą wywołania `template_name` wskazujemy własny szablon do wygenerowania strony. Jeżeli nie zostanie podany szablon domyślny, `ListView` będzie używać szablonu *blog/post/list.html*.

Teraz przejdź do pliku *urls.py* aplikacji `blog`, umieść w komentarzu poprzedni wzorzec adresu URL dla widoku `post_list`, a następnie dodaj nowy wzorzec URL dla widoku utworzonego na podstawie klasy `PostListView`, co pokazałem w poniższym fragmencie kodu.

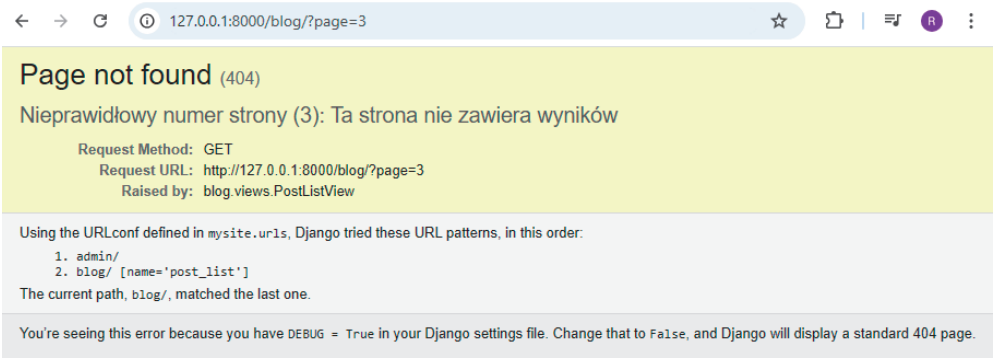
```
urlpatterns = [
    # Widoki postów
    # path('', views.post_list, name='post_list'),
    path('', views.PostListView.as_view(), name='post_list'),
    path(
        '<int:year>/<int:month>/<int:day>/<slug:post>/',
        views.post_detail,
        name='post_detail'
    ),
]
```

W celu umożliwienia działania stronicowania konieczne jest użycie odpowiedniego obiektu `page`, który należy przekazać do szablonu. Generyczny widok `ListView` frameworka Django przekazuje żądaną stronę w zmiennej o nazwie `page_obj`. Aby uwzględnić stronicowanie przy użyciu właściwej zmiennej, trzeba odpowiednio zmodyfikować szablon *post/list.html*.

```
{% extends "blog/base.html" %}

{% block title %}Mój Blog{% endblock %}

{% block content %}
  <h1>Mój Blog</h1>
  {% for post in posts %}
    <h2>
      <a href="{{ post.get_absolute_url }}">
        {{ post.title }}
      </a>
    </h2>
    <p class="date">
      Opublikowany {{ post.publish }} przez {{ post.author }}
    </p>
    {{ post.body|truncatewords:30|linebreaks }}
  {% endfor %}
  {% include "pagination.html" with page=page_obj %}
{% endblock %}
```

Otwórz w przeglądarce stronę `http://127.0.0.1:8000/blog/` i sprawdź, czy linki stronicowania działają zgodnie z oczekiwaniami. Zachowanie linków stronicowania powinno być takie samo jak w poprzednim widoku `post_list`.

Obsługa wyjątków w tym przypadku jest nieco inna. Jeśli spróbujesz załadować stronę spoza zakresu lub przekażesz w parametrze `page` wartość inną niż liczba całkowita, widok zwróci odpowiedź HTTP z kodem stanu `404` (Nie znaleziono strony), tak jak pokazano na rysunku 2.10.



Rysunek 2.10. Odpowiedź HTTP 404 (Nie znaleziono strony)

Obsługa wyjątków, która zwraca kod stanu HTTP 404, jest zapewniana przez widok `ListView`.

To jest prosty przykład pisania widoków opartych na klasach. Więcej informacji o widokach opartych na klasach znajdziesz w rozdziale 13., „Tworzenie systemu zarządzania treścią", oraz dalszych.

Z wprowadzeniem do widoków opartych na klasach możesz się zapoznać na stronie *https://docs.djangoproject.com/en/5.0/topics/class-based-views/intro/*.

Po zapoznaniu się ze sposobem korzystania z widoków opartych na klasach i używania obiektu stronicowania zaimplementujemy funkcjonalność, która pozwoli na zachęcanie do lektury postów — udostępnianie postów przez e-mail.

Polecanie postów przez e-mail

Teraz nauczysz się tworzyć formularze i wysyłać maile za pomocą Django. Stworzymy użytkownikom możliwość udostępniania postów na blogu innym osobom przez wysyłanie rekomendacji postów pocztą elektroniczną.

Zrób krótką przerwę i zastanów się, jak użyłbyś widoków, adresów URL i szablonów do przygotowania tego rodzaju funkcjonalności z wykorzystaniem wiedzy zdobytej w poprzednim rozdziale.

Aby umożliwić użytkownikom udostępnianie postów przez e-mail, należy:

1. Utworzyć formularz dla użytkownika pozwalający na podanie imienia, adresów e-mail nadawcy i odbiorcy, a także opcjonalnych komentarzy.
2. Utworzyć w pliku *views.py* widok odpowiedzialny za obsługę przekazywanych danych i ich wysyłania poprzez e-mail.
3. Dodać w pliku *urls.py* aplikacji `blog` wzorzec adresu URL dla nowego widoku.
4. Utworzyć szablon przeznaczony do wyświetlenia formularza.

Tworzenie formularzy w Django

Przystępujemy teraz do przygotowania formularza umożliwiającego informowanie o postach. Django ma wbudowany framework `forms` niezwykle ułatwiający ich tworzenie. Wspomniany framework `forms` pozwala na zdefiniowanie pól formularza, określenie sposobu ich wyświetlania, a także ustalenie wymagań w zakresie sprawdzania danych wejściowych. Framework jednocześnie oferuje elastyczność w zakresie generowania formularzy i obsługi danych.

Django dostarcza dwie klasy bazowe przeznaczone do tworzenia formularzy.

- `Form`: Umożliwia budowanie standardowych formularzy poprzez definiowanie pól i walidacji.
- `ModelForm`: Umożliwia budowanie formularzy powiązanych z egzemplarzami modelu. Zapewnia wszystkie funkcjonalności bazowej klasy `Form`, ale pola formularza mogą być jawnie deklarowane lub generowane automatycznie z pól modelu. Formularz może służyć do tworzenia lub edytowania egzemplarzy modelu.

Najpierw w katalogu aplikacji `blog` utwórz plik *forms.py* i dodaj do niego kod:

```
from django import forms

class EmailPostForm(forms.Form):
    name = forms.CharField(max_length=25)
    email = forms.EmailField()
    to = forms.EmailField()
    comments = forms.CharField(
        required=False,
        widget=forms.Textarea
    )
```

Właśnie zdefiniowaliśmy pierwszy formularz Django. Formularz `EmailPostForm` dziedziczy po bazowej klasie `Form`. Aby odpowiednio zweryfikować poprawność danych, używamy różnych typów pól.

Uwaga

Wprawdzie formularze mogą być umieszczane gdziekolwiek w projekcie Django, ale jeśli chcesz pozostać w zgodzie z obowiązującą konwencją, powinieneś je definiować w pliku *forms.py* dla każdej aplikacji.

Formularz zawiera pola:

- `name`: Egzemplarz klasy `CharField` o maksymalnej długości 25 znaków. Użyjemy go do przechowywania imienia i nazwiska osoby wysyłającej wiadomość e-mail.
- `email`: Egzemplarz klasy `EmailField`. Wykorzystamy go do przechowywania adresu e-mail osoby wysyłającej rekomendację posta.
- `to`: Egzemplarz klasy `EmailField`. Używamy go do przechowywania adresu e-mail odbiorcy, który otrzyma wiadomość e-mail z rekomendacją postu.
- `comments`: Egzemplarz klasy `CharField`. Będziemy go używać do umieszczania komentarzy w e-mailu z rekomendacjami postów. Ustawiliśmy to pole jako opcjonalne, przez ustawienie wartości opcji `required` na `False`, i określiliśmy niestandardowy widżet do renderowania tego pola.

Każdy typ pola ma domyślny widżet, który określa sposób jego wyświetlania na stronie internetowej. Pole `name` jest egzemplarzem klasy `CharField`. Ten typ pola jest renderowany jako element HTML `<input type="text">`. Widżet domyślny może być nadpisany przez atrybut `widget`. W polu `comments` użyliśmy widżetu `Textarea` do wyświetlania go jako elementu HTML `<textarea>` zamiast domyślnego elementu `<input>`.

Sposób walidacji danych wejściowych wprowadzanych w polu także zależy od typu pola. Przykładowo pola `email` i `to` są typu `EmailField`. Obydwa wymienione pola wymagają podania poprawnego adresu e-mail, ponieważ w przeciwnym razie nastąpi zgłoszenie wyjątku `forms.ValidationError` i operacja walidacji zakończy się niepowodzeniem. Podczas walidacji pól formularzy uwzględniane są również inne parametry, takie jak maksymalna długość pola `name`, wynosząca 25, i opcjonalność pola `comments`.

To tylko niektóre z typów pól, które framework Django udostępnia dla formularzy. Listę wszystkich dostępnych typów pól można znaleźć pod adresem *https://docs.djangoproject.com/en/5.0/ref/forms/fields/*.

Obsługa formularzy w widokach

Zdefiniowaliśmy formularz do polecania postów przez e-mail. Teraz potrzebujemy widoku, w którym stworzymy egzemplarz formularza i obsłużymy jego przesyłanie.

Przeprowadź edycję pliku *views.py* aplikacji `blog` i dodaj w nim poniższy fragment kodu.

```
from .forms import EmailPostForm

def post_share(request, post_id):
    # Pobierz post według identyfikatora
    post = get_object_or_404(
        Post,
        id=post_id,
        status=Post.Status.PUBLISHED
    )

    if request.method == 'POST':
        # Formularz został przesłany
        form = EmailPostForm(request.POST)
        if form.is_valid():
            # Pomyślnie zweryfikowano poprawność pól formularza
            cd = form.cleaned_data
            # ...wyślij email
    else:
        form = EmailPostForm()
    return render(
        request,
        'blog/post/share.html',
        {
            'post': post,
            'form': form
        }
    )
```

Zdefiniowaliśmy widok `post_share`, który jako parametry przyjmuje obiekt `request` i zmienną `post_id`. Do pobrania opublikowanego posta według jego identyfikatora używamy skrótu `get_object_or_404()`.

Tego samego widoku użyliśmy zarówno do wyświetlenia początkowego formularza, jak i przetworzenia wysłanych danych. Do ustalenia, czy formularz jest wysyłany, można skorzystać z atrybutu `method` obiektu `request` zwracającego metodę HTTP żądania. Wartość `GET` oznacza, że trzeba wyświetlić pusty formularz, natomiast wartość `POST` oznacza, że użytkownik przesyła wypełniony formularz. Aby rozróżnić te dwa scenariusze, używamy warunku `request.method == 'POST'`.

Oto proces wyświetlania formularza i obsługi jego przesyłania:

1. Gdy strona jest ładowana po raz pierwszy, widok otrzymuje żądanie `GET`. W takim przypadku tworzony jest nowy egzemplarz klasy `EmailPostForm`, który jest zapisywany w zmiennej `form`. Ten egzemplarz formularza zostanie użyty do wyświetlenia pustego formularza w szablonie:
   ```
   form = EmailPostForm()
   ```
2. Gdy użytkownik po wypełnieniu formularza przesyła go metodą `POST`, tworzony jest egzemplarz formularza na podstawie przesłanych danych zawartych w polu `request.POST`:
   ```
   if request.method == 'POST':
       # Formularz został przesłany
       form = EmailPostForm(request.POST)
   ```
3. Następnym krokiem jest przeprowadzenie walidacji wysłanych danych z wykorzystaniem metody `is_valid()` formularza. Metoda sprawdza prawidłowość danych wprowadzonych w formularzu i zwraca wartość `True`, gdy wszystkie pola zawierają prawidłowe dane. Jeżeli którekolwiek pole zawiera nieprawidłowe dane, funkcja `is_valid()` zwróci `False`. Listę błędów walidacji można uzyskać za pomocą wywołania `form.errors`.
4. Jeżeli formularz zawiera nieprawidłowe dane, ponownie wyświetlamy go w szablonie wraz z przesłanymi danymi. Ponadto szablon wyświetla błędy wykryte podczas walidacji formularza.
5. Jeśli formularz zawiera prawidłowe dane, sprawdzone dane są pobierane za pomocą atrybutu `form.cleaned_data`. Ten atrybut jest słownikiem zawierającym pola formularza i ich wartości. Formularze nie tylko walidują dane, ale także oczyszczają je przez znormalizowanie do spójnego formatu.

Uwaga

Jeżeli walidacja formularza zakończy się niepowodzeniem, słownik `cleaned_data` będzie zawierał jedynie prawidłowe pola formularza.

Zaimplementowaliśmy widok do wyświetlania formularza i obsługi wysyłania formularza. Dowiemy się teraz, jak wysyłać wiadomości e-mail za pomocą Django, a następnie dodamy tę funkcjonalność do widoku `post_share`.

Wysyłanie wiadomości e-mail w Django

Wysyłanie wiadomości e-mail w Django jest całkiem prostą operacją. Aby wysyłać e-maile za pomocą Django, musisz mieć lokalny serwer SMTP (ang. *Simple Mail Transfer Protocol*) lub potrzebujesz dostępu do zewnętrznego serwera SMTP, takiego jaki oferuje Twój dostawca usług e-mail.

Konfigurację SMTP do wysyłania wiadomości e-mail za pomocą Django pozwalają zdefiniować poniższe ustawienia:

- `EMAIL_HOST`: nazwa hosta serwera SMTP, wartością domyślną jest `localhost`.
- `EMAIL_PORT`: numer portu w serwerze SMTP, wartością domyślną jest 25.

- EMAIL_HOST_USER: nazwa użytkownika serwera SMTP.
- EMAIL_HOST_PASSWORD: hasło do serwera SMTP.
- EMAIL_USE_TLS: opcja wskazuje, czy będzie używane bezpieczne połączenie **TLS** (ang. *Transport Layer Security*).
- EMAIL_USE_SSL: opcja wskazuje, czy niejawnie będzie używane bezpieczne połączenie SSL.

Dodatkowo, można użyć ustawienia DEFAULT_FROM_EMAIL pozwalającego określić domyślnego nadawcę podczas wysyłania e-maili z Django. W tym przykładzie użyjemy serwera SMTP Google ze standardowym kontem Gmail.

Korzystanie ze zmiennych środowiskowych

W tym punkcie pokażę, jak dodać do projektu ustawienia konfiguracji SMTP i załadować poświadczenia SMTP ze zmiennych środowiskowych. Wykorzystanie zmiennych środowiskowych pozwala uniknąć osadzania danych uwierzytelniających w kodzie źródłowym. Jest wiele powodów, dla których warto oddzielić konfigurację od kodu:

- **Bezpieczeństwo:** Poświadczenia lub tajne klucze w kodzie, zwłaszcza jeśli kod jest udostępniany w publicznych repozytoriach, mogą prowadzić do niezamierzonego ujawnienia.
- **Elastyczność:** Utrzymywanie konfiguracji oddzielnie od kodu pozwala korzystać z tej samej bazy kodu, bez wprowadzania żadnych zmian, w różnych środowiskach. Sposób tworzenia wielu środowisk opisałem w rozdziale 17., „Wdrożenie".
- **Łatwość utrzymania:** Zmiana konfiguracji nie wymaga zmiany kodu, dzięki czemu projekt pozostaje spójny we wszystkich wersjach.

Aby ułatwić oddzielenie konfiguracji od kodu, skorzystamy z biblioteki python-decouple. Biblioteka ta upraszcza korzystanie ze zmiennych środowiskowych w projektach. Informacje na temat pakietu python-decouple można znaleźć na stronie *https://github.com/HBNetwork/python-decouple*.

Zacznij od zainstalowania pakietu python-decouple za pomocą menedżera pip. W tym celu uruchom polecenie:

```
python -m pip install python-decouple==3.8
```

Następnie w katalogu głównym projektu utwórz nowy plik i nadaj mu nazwę *.env*. Plik *.env* zawiera pary klucz-wartość zmiennych środowiskowych. Dodaj do nowego pliku wiersze:

```
EMAIL_HOST_USER=twoje_konto@gmail.com
EMAIL_HOST_PASSWORD=
DEFAULT_FROM_EMAIL=Mój Blog <twoje_konto@gmail.com>
```

Jeśli masz konto Gmail, zastąp *twoje_konto@gmail.com* swoim kontem Gmail. Zmienna EMAIL_HOST_PASSWORD nie ma jeszcze wartości, dodamy ją później. Zmienna DEFAULT_FROM_
↪EMAIL będzie używana do określenia domyślnego nadawcy wysyłanych wiadomości e-mail.

Jeśli nie masz konta Gmail, możesz skorzystać z konfiguracji serwera SMTP swojego dostawcy usług pocztowych.

Jeśli do przechowywania swojego kodu używasz repozytorium *git*, zadbaj o dołączenie pliku *.env* do pliku *.gitignore* w katalogu Twojego repozytorium. Dzięki temu unikniesz nieumyślnego przesłania poświadczeń do repozytorium.

Zmodyfikuj plik *settings.py* swojego projektu i dodaj do niego kod:

```
from decouple import config
# ...
# Konfiguracja serwera e-mail
EMAIL_HOST = 'smtp.gmail.com'
EMAIL_HOST_USER = config('EMAIL_HOST_USER')
EMAIL_HOST_PASSWORD = config('EMAIL_HOST_PASSWORD')
EMAIL_PORT = 587
EMAIL_USE_TLS = True
DEFAULT_FROM_EMAIL = config('DEFAULT_FROM_EMAIL')
```

Ustawienia `EMAIL_HOST_USER`, `EMAIL_HOST_PASSWORD` i `DEFAULT_FROM_EMAIL` są teraz ładowane ze zmiennych środowiskowych zdefiniowanych w pliku *.env*.

Podane ustawienia `EMAIL_HOST`, `EMAIL_PORT` i `EMAIL_USE_TLS` dotyczą serwera SMTP Gmail. Jeśli nie masz konta Gmail, możesz skorzystać z konfiguracji serwera SMTP swojego dostawcy usług pocztowych.

Zamiast Gmaila możesz też skorzystać z profesjonalnej, skalowalnej usługi pocztowej, która umożliwia wysyłanie e-maili przez SMTP z wykorzystaniem własnej domeny, takiej jak SendGrid (*https://sendgrid.com/*) lub Amazon **Simple Email Service** (*https://aws.amazon.com/ses/*). Obie usługi wymagają zweryfikowania domeny i konta e-mail nadawcy oraz zapewniają poświadczenia SMTP do wysyłania wiadomości e-mail. Zadanie dodawania do projektu dostawców usług e-mail, takich jak SendGrid lub Amazon SES, upraszcza aplikacja `django-anymail`. Instrukcje instalacji `django-anymail` można znaleźć na stronie *https://anymail.dev/en/stable/installation/*, a listę obsługiwanych dostawców usług e-mail na stronie *https://anymail.dev/en/stable/esps/*.

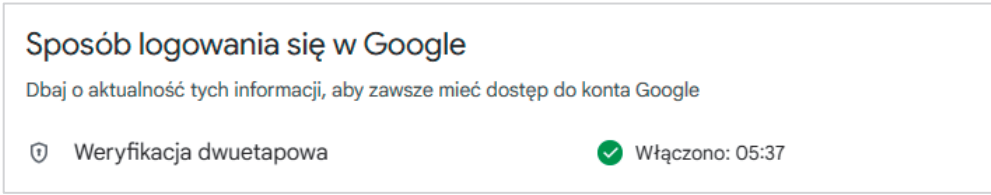
Jeśli nie możesz użyć serwera SMTP, możesz skonfigurować Django w taki sposób, aby pisał wiadomości e-mail na konsoli. W tym celu dodaj do pliku *settings.py* następujące ustawienie.

```
EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'
```

Po użyciu tego ustawienia Django zamiast wysyłania wiadomości e-mail, będzie wyświetlał je w powłoce. Jest to bardzo przydatne podczas testowania aplikacji bez serwera SMTP.

Aby wysyłać wiadomości e-mail za pomocą serwera SMTP Gmail, włącz na swoim koncie Gmail weryfikację dwuetapową.

Otwórz stronę w przeglądarce stronę *https://myaccount.google.com/security* i włącz weryfikację dwuetapową dla swojego konta, jak pokazałem na rysunku 2.11.

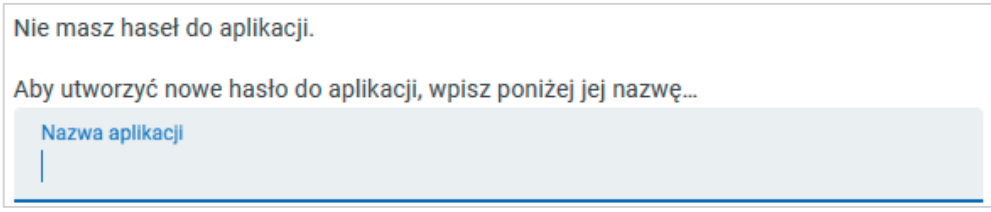


Rysunek 2.11. Strona logowania do Google dla kont Google

Następnie trzeba utworzyć hasło aplikacji i użyć go jako poświadczeń dla serwera SMTP. Hasło do aplikacji to 16-cyfrowy kod dostępu, który umożliwia mniej bezpiecznej aplikacji lub urządzeniu dostęp do konta Google.

Aby utworzyć hasło do aplikacji, otwórz w przeglądarce stronę *https://myaccount.google.com/apppasswords*.

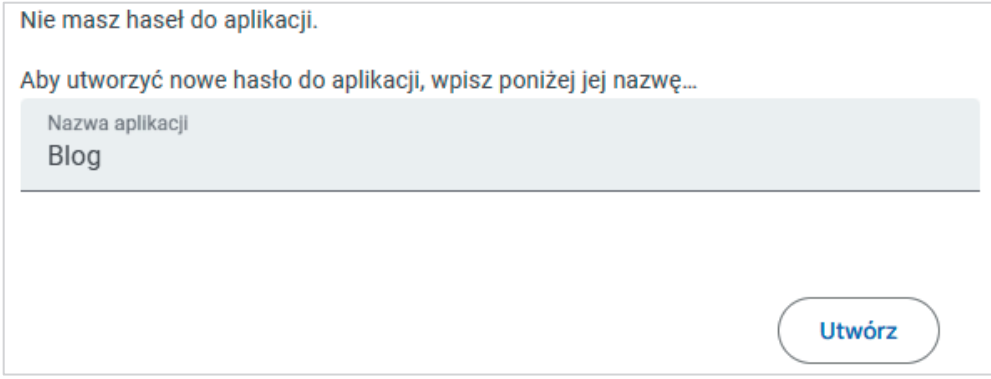
Wyświetli się ekran pokazany na rysunku 2.12.



Rysunek 2.12. Formularz do generowania nowego hasła do aplikacji Google

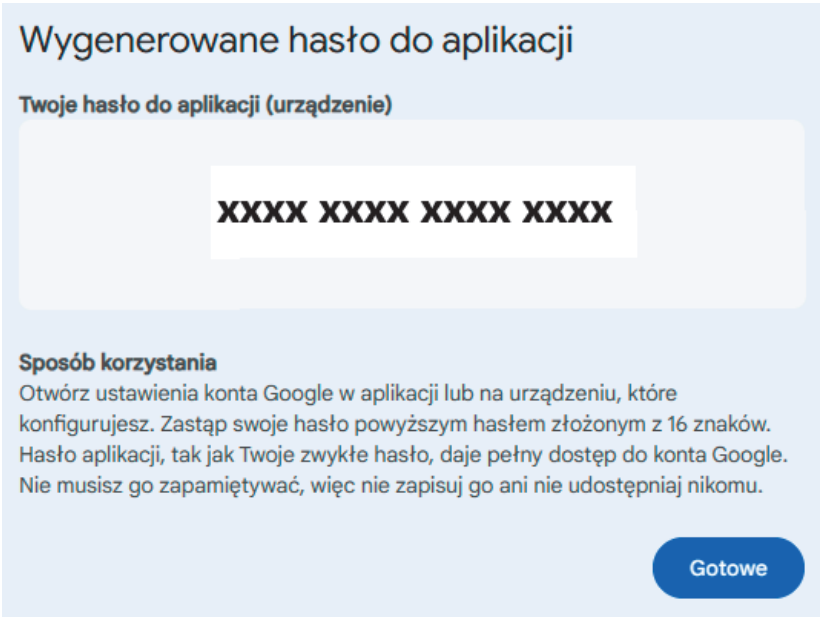
Jeśli nie widzisz opcji *Hasła do aplikacji*, być może na Twoim koncie nie została ustawiona weryfikacja dwuetapowa, Twoje konto jest kontem organizacji, a nie standardowym kontem Gmail, lub masz włączoną zaawansowaną ochronę Google. Powinieneś skorzystać ze standardowego konta Gmail i uaktywnić dla swojego konta Google weryfikację dwuetapową. Więcej informacji można znaleźć na stronie *https://support.google.com/accounts/answer/185833*.

Następnie wprowadź nazwę *Blog* i kliknij przycisk *Utwórz* (rysunek 2.13).



Rysunek 2.13. Formularz do generowania nowego hasła do aplikacji Google

Nowe hasło zostanie wygenerowane i wyświetlone, tak jak pokazałem na rysunku 2.14.



Rysunek 2.14. Wygenerowane hasło do aplikacji Google

Skopiuj wygenerowane hasło do aplikacji.

Następnie otwórz do edycji plik *.env* projektu i dodaj do zmiennej `EMAIL_HOST_PASSWORD` hasło aplikacji w taki sposób:

```
EMAIL_HOST_USER=twoje_konto@gmail.com
EMAIL_HOST_PASSWORD=xxxxxxxxxxxxxxxx
DEFAULT_FROM_EMAIL=Mój Blog <twoje_konto@gmail.com>
```

Aby otworzyć powłokę Pythona, otwórz terminal i uruchom polecenie:

```
python manage.py shell
```

W powłoce Pythona uruchom następujący kod.

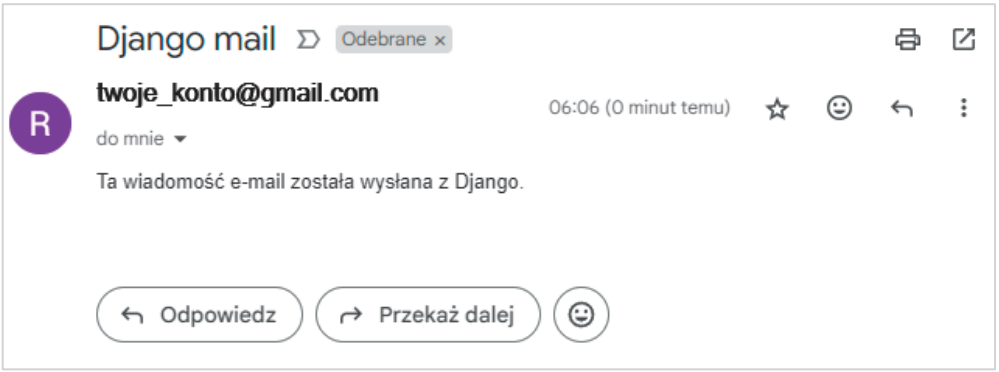
```
>>> from django.core.mail import send_mail
>>> send_mail('Django mail',
...           'Ta wiadomość e-mail została wysłana z Django.',
...           'twoje_konto@gmail.com',
...           ['twoje_konto@gmail.com'],
...           fail_silently=False)
```

Funkcja `send_mail()` pobiera wymagane argumenty, czyli temat, treść wiadomości, nadawcę oraz listę odbiorców. Przez ustawienie opcjonalnego argumentu `fail_silently=False` nakazujemy zgłoszenie wyjątku, gdy wysłanie wiadomości e-mail zakończy się niepowodzeniem. Jeżeli otrzymasz wynik `1`, oznacza to, że udało Ci się wysłać wiadomość.

Jeśli wyświetli się błąd `CERTIFICATE_VERIFY_FAILED`, zainstaluj moduł `certifi` za pomocą polecenia `pip install --upgrade certifi`. Jeśli używasz systemu macOS, aby zainstalować `certifi` i umożliwić Pythonowi dostęp do głównych certyfikatów macOS, uruchom w powłoce poniższe polecenie.

```
/Applications/Python\ 3.12/Install\ Certificates.command
```

Sprawdź swoją skrzynkę odbiorczą. Powinieneś otrzymać e-mail (rysunek 2.15).



Rysunek 2.15. Wysłana testowa wiadomość e-mail wyświetlona w Gmailu

Właśnie wysłałeś swój pierwszy e-mail za pomocą Django! Więcej informacji na temat wysyłania wiadomości e-mail za pomocą Django można znaleźć na stronie *https://docs.djangoproject.com/en/5.0/topics/email/*.

Dodajmy tę funkcjonalność do widoku `post_share`.

Wysyłanie wiadomości e-mail w widokach

Przeprowadź edycję widoku `post_share` w pliku *views.py* aplikacji `blog`, aby zawierał kod:

```
# ...

from django.core.mail import send_mail

# ...

def post_share(request, post_id):
    # Pobierz post według identyfikatora
    post = get_object_or_404(
        Post,
        id=post_id,
         status=Post.Status.PUBLISHED
    )
    sent = False

    if request.method == 'POST':
        # Formularz został przesłany
```

```
        form = EmailPostForm(request.POST)
        if form.is_valid():
            # Pomyślnie zweryfikowano poprawność pól formularza
            cd = form.cleaned_data
            post_url = request.build_absolute_uri(
                post.get_absolute_url()
            )
            subject = (
                f"{cd['name']} ({cd['email']}) "
                f"poleca Ci przeczytanie {post.title}"
            )
            message = (
                f"Przeczytaj {post.title} pod adresem {post_url}\n\n" \
                f"komentarze {cd['name']}: {cd['comments']}"
            )
            send_mail(
                 subject=subject,
                 message=message,
                 from_email=None,
                 recipient_list=[cd['to']]
            )
            sent = True
    else:
        form = EmailPostForm()
    return render(
        request,
        'blog/post/share.html',
        {
            'post': post,
            'form': form,
            'sent': sent
        }
    )
```

W powyższym kodzie zadeklarowaliśmy zmienną `sent` i zainicjowaliśmy ją wartością początkową `False`. Ustawiamy tę zmienną na `True` po wysłaniu wiadomości e-mail. Wymienioną zmienną wykorzystamy później w szablonie w celu wyświetlenia komunikatu informującego o zakończonym powodzeniem wysłaniu wiadomości e-mail, gdy uda się poprawnie wysłać formularz.

Ponieważ w wiadomości e-mail trzeba umieścić łącze do posta, bezwzględną ścieżkę dostępu posta pobieramy za pomocą metody `get_absolute_url()`. Następnie tej ścieżki dostępu używamy jako danych wejściowych dla metody `request.build_absolute_uri()` i budujemy kompletny adres URL zawierający schemat HTTP oraz nazwę hosta.

Temat i treść wiadomości tworzymy na podstawie oczyszczonych danych ze zweryfikowanego formularza. Na koniec wysyłamy wiadomość e-mail na adres e-mail podany w polu `to` formularza. W parametrze `from_email` przekazujemy wartość `None`, zatem dla nadawcy zostanie użyta wartość ustawienia `DEFAULT_FROM_EMAIL`.

Po przygotowaniu widoku nie zapomnij o dodaniu dla niego nowego wzorca adresu URL.

Otwórz plik *urls.py* aplikacji `blog` i dodaj wzorzec adresu URL dla widoku `post_share`.

```
from django.urls import path
from . import views


app_name = 'blog'

urlpatterns = [
    # Widoki postów
    # path('', views.post_list, name='post_list'),
     path('', views.PostListView.as_view(), name='post_list'),
     path(
         '<int:year>/<int:month>/<int:day>/<slug:post>/',
         views.post_detail,
         name='post_detail'),
     path('<int:post_id>/share/', views.post_share, name='post_share'),

]
```

Renderowanie formularzy w szablonach

Po utworzeniu formularza, przygotowaniu widoku i dodaniu wzorca adresu URL do zrobienia pozostało już tylko opracowanie szablonu dla tego widoku.

W katalogu *blog/templates/blog/post/* utwórz więc nowy plik o nazwie *share.html*.

Do nowego szablonu *share.html* dodaj kod:

```
{% extends "blog/base.html" %}

{% block title %}Udostępnij post {% endblock %}

{% block content %}
  {% if sent %}
    <h1>Wiadomość e-mail została pomyślnie wysłana </h1>
    <p>
       "Wysłanie posta {{ post.title }}" do {{ form.cleaned_data.to }} zakończyło
       ↪się sukcesem.
    </p>
  {% else %}
    <h1>Udostępnij "{{ post.title }}" przez e-mail</h1>
    <form method="post">
      {{ form.as_p }}
      {% csrf_token %}
      <input type="submit" value="Wyślij wiadomość e-mail">
    </form>
  {% endif %}
{% endblock %}
```

Jest to szablon używany zarówno do wyświetlania formularza udostępniania posta przez e-mail, jak i do wyświetlania komunikatu o powodzeniu po wysłaniu wiadomości e-mail. Oba przypadki rozróżniamy za pomocą znacznika `{% if sent %}`.

Aby wyświetlić formularz, zdefiniowaliśmy element HTML `form`, wskazujący na użycie metody `POST` podczas wysyłania formularza:

```
<form method="post">
```

Egzemplarz formularza dołączyliśmy za pomocą znacznika `{{ form.as_p }}`. Zleciliśmy frameworkowi Django wyrenderowanie pól formularza z wykorzystaniem elementów HTML `<p>` za pomocą metody `as_p`. Formularz może być także wygenerowany w postaci nieuporządkowanej listy (metoda `as_ul`) lub w tabeli HTML (metoda `as_table`).

Dodaliśmy znacznik szablonu `{% csrf_token %}`. Ten znacznik dodaje ukryte pole wraz z automatycznie wygenerowanym tokenem, co stanowi sposób ochrony przez atakami typu **CSRF** (ang. *cross-site request forgery*). Tego rodzaju ataki polegają na tym, że program lub inna witryna o złośliwym działaniu wykonują operacje niechciane przez użytkownika Twojej witryny. Więcej informacji na temat ataków CSRF można znaleźć pod adresem *https://owasp.org/www-community/attacks/csrf*.

Znacznik szablonu `{% csrf_token %}` generuje ukryte pole, które jest renderowane w taki sposób.

```
<input type='hidden' name='csrfmiddlewaretoken'
value='26JjKo2lcEtYkGoV9z4XmJIEHLXN5LDR' />
```

Uwaga

Sprawdzenie pod kątem tokenu CSRF Django domyślnie przeprowadza we wszystkich żądaniach POST. Pamiętaj więc o dołączaniu znacznika `csrf_token` do wszystkich formularzy wysyłanych za pomocą metody `POST`.

Wyedytuj szablon *blog/post/detail.html* i doprowadź go do następującej postaci.

```
{% extends "blog/base.html" %}

{% block title %}{{ post.title }}{% endblock %}

{% block content %}
  <h1>{{ post.title }}</h1>
  <p class="date">
    Opublikowany {{ post.publish }} przez {{ post.author }}
  </p>
  {{ post.body|linebreaks }}
  <p>
    <a href="{% url "blog:post_share" post.id %}">
      Udostępnij post
    </a>
  </p>
{% endblock %}
```

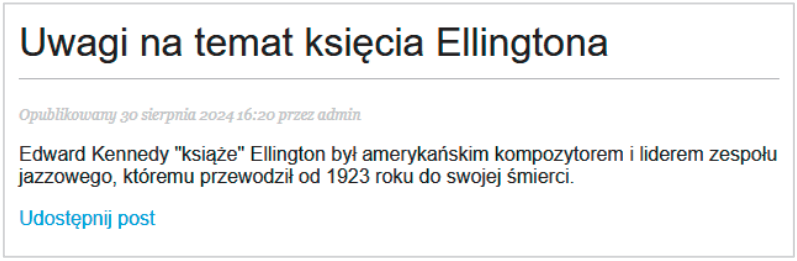
Dodaliśmy link do adresu URL `post_share`. Adres URL budujemy przy użyciu dostarczonego przez Django znacznika szablonu `{% url %}`. Używamy przestrzeni nazw o nazwie *blog* i adresu URL o nazwie `post_share`. W celu zbudowania adresu URL przekazujemy jako parametr identyfikator posta.

Przejdź do powłoki i w celu uruchomienia serwera programistycznego, uruchom polecenie:

```
python manage.py runserver
```

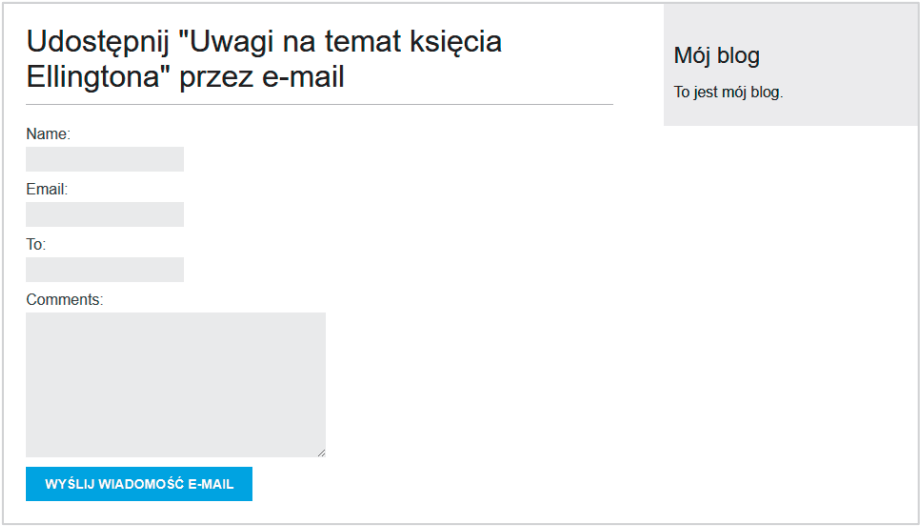
W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/blog/` i kliknij tytuł dowolnego posta, aby przejść na stronę jego szczegółów.

Pod treścią posta powinieneś zobaczyć dodany przed chwilą link, co pokazałem na rysunku 2.16.



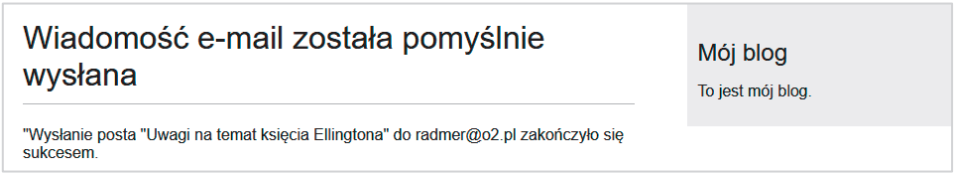
Rysunek 2.16. Strona szczegółów posta zawierająca łącze do udostępnienia posta

Po kliknięciu linka *Udostępnij post* zostanie wyświetlona strona wraz z formularzem pozwalającym na przekazanie posta za pomocą wiadomości e-mail (rysunek 2.17).



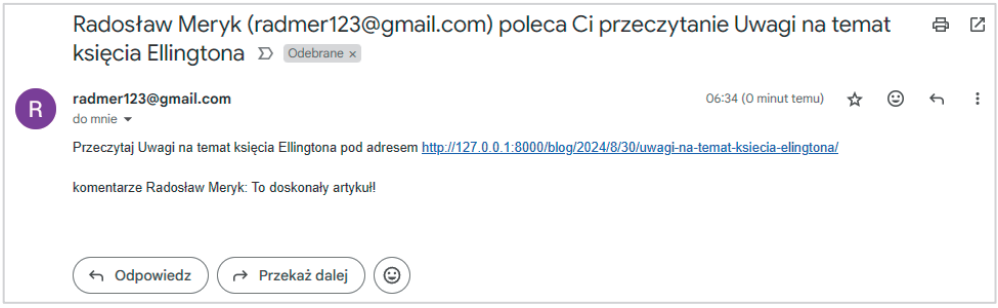
Rysunek 2.17. Strona do udostępniania posta za pomocą e-maila

Style CSS dla formularza znajdują się w materiałach dodatkowych przygotowanych dla książki, w katalogu *static/css/blog.css*. Kliknięcie przycisku *Wyślij wiadomość e-mail* powoduje wysłanie formularza i przeprowadzenie jego walidacji. Jeżeli wszystkie pola formularza zawierają prawidłowe dane, zostanie wyświetlony komunikat informujący o zakończonym powodzeniem wysłaniu wiadomości e-mail (rysunek 2.18).



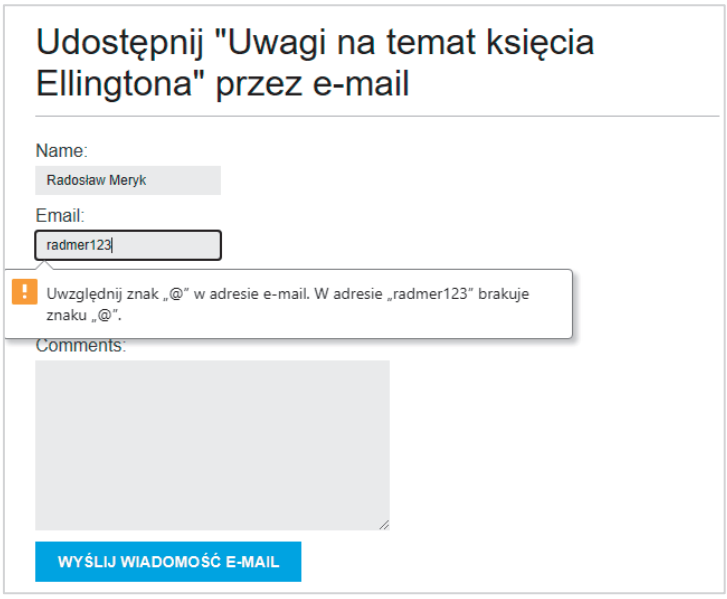
Rysunek 2.18. Komunikat o udanym udostępnieniu posta przez e-mail

Wyślij post na własny adres e-mail i sprawdź swoją skrzynkę odbiorczą. E-mail, który otrzymasz, powinien wyglądać jak na rysunku 2.19.



Rysunek 2.19. Wysłana testowa wiadomość e-mail wyświetlona w Gmailu

Jeśli prześlesz formularz z nieprawidłowymi danymi, formularz zostanie wyświetlony ponownie, łącznie ze wszystkimi błędami walidacji (rysunek 2.20).



Rysunek 2.20. Formularz udostępniania posta z komunikatem o błędzie dotyczącym niepoprawnych danych

Warto wiedzieć, że większość nowoczesnych przeglądarek uniemożliwia przesyłanie formularzy z pustymi lub błędnymi polami. Dzieje się tak dlatego, że przed wysłaniem formularza przeglądarka sprawdza poprawność pól na podstawie ich atrybutów. Kiedy stwierdzi błędy, formularz nie zostanie przesłany, a przeglądarka wyświetli komunikat o błędzie dla pól, które są źle wypełnione. Aby przetestować mechanizmy walidacji formularza frameworka Django za pomocą nowoczesnej przeglądarki, możesz pominąć walidację formularza przeglądarki. W tym celu dodaj atrybut `novalidate` do elementu HTML `<form>`, na przykład `<form method=post novalidate>`. Możesz dodać ten atrybut, aby uniemożliwić przeglądarce sprawdzanie poprawności pól i przetestować własne mechanizmy walidacji formularza. Po zakończeniu testowania usuń atrybut `novalidate`, aby przywrócić sprawdzanie poprawności formularza przez przeglądarkę.

Funkcjonalność udostępniania postów przez e-mail jest ukończona. Więcej informacji na temat korzystania z formularzy można znaleźć pod adresem *https://docs.djangoproject.com/en/5.0/topics/forms/*.

Utworzenie systemu komentarzy

Będziemy nadal rozszerzać naszą aplikację blogową o system komentarzy, który umożliwi użytkownikom komentowanie postów. Aby zbudować tego rodzaju system, będziemy musieli wykonać wymienione poniżej działania.

- Utworzenie modelu do przechowywania komentarzy użytkowników do postów.
- Zdefiniowanie formularza Django umożliwiającego przesyłanie komentarzy oraz obsługującego walidację danych.
- Dodanie widoku odpowiedzialnego za przetwarzanie tego formularza i zapisywanie nowego komentarza w bazie danych.
- Utworzenie listy komentarzy oraz formularza dodawania nowego komentarza, który można osadzić w szablonie szczegółów posta.

Tworzenie modelu komentarzy

Pracę zaczynamy od przygotowania modelu przeznaczonego do przechowywania komentarzy użytkowników.

Otwórz plik *models.py* aplikacji `blog` i dodaj w nim poniższy fragment kodu.

```
class Comment(models.Model):
    post = models.ForeignKey(
        Post,
        on_delete=models.CASCADE,
        related_name='comments'
    )
    name = models.CharField(max_length=80)
    email = models.EmailField()
    body = models.TextField()
    created = models.DateTimeField(auto_now_add=True)
    updated = models.DateTimeField(auto_now=True)
    active = models.BooleanField(default=True)
```

```
    class Meta:
        ordering = ['created']
        indexes = [
            models.Index(fields=['created']),
        ]

    def __str__(self):
        return f'Komentarz dodany przez {self.name} dla posta {self.post}'
```

To jest model `Comment`. Dodaliśmy pole `ForeignKey`, aby powiązać każdy komentarz z pojedynczym postem. W modelu została zdefiniowana relacja typu „wiele do jednego", ponieważ każdy komentarz jest przeznaczony dla jednego konkretnego posta, a sam post może mieć wiele komentarzy.

Atrybut `related_name` umożliwia nadanie nazwy atrybutowi, którego używamy do obsługi związku między dwoma obiektami. Za pomocą metody `comment.post()` możemy pobrać obiekt komentarza lub też użyć metody `post.comments.all()` do pobrania wszystkich komentarzy dla danego posta. Jeśli nie zdefiniujesz atrybutu `related_name`, Django użyje zapisanej małymi literami nazwy modelu wraz z przyrostkiem `_set` (w omawianym przykładzie to będzie `comment_set`) jako nazwy relacji powiązanego z tym obiektem modelu, w którym zdefiniowano tę relację.

Więcej informacji na temat relacji wiele-do-jednego możesz znaleźć na stronie *https://docs.djangoproject.com/en/5.0/topics/db/examples/many_to_one/*.

Do zarządzania stanem komentarzy zdefiniowaliśmy pole `active` typu `Boolean`. To pole pozwoli ręcznie dezaktywować nieodpowiednie komentarze za pomocą strony administracyjnej. Aby wskazać, że wszystkie komentarze są domyślnie aktywne, używamy ustawienia `default=True`.

Zdefiniowaliśmy także pole `created` do przechowywania daty i godziny utworzenia komentarza. Ponieważ tutaj używamy opcji `auto_now_add`, data zostanie zapisana automatycznie podczas tworzenia obiektu. W klasie `Meta` modelu dodaliśmy ustawienie `ordering ↪= ['created']`, aby domyślnie sortować komentarze w porządku chronologicznym, oraz dodaliśmy indeks dla pola `created` w celu porządkowania w kolejności rosnącej. Poprawi to wydajność przeszukiwania bazy danych lub porządkowania wyników przy użyciu pola `created`.

Utworzony przed chwilą nowy model `Comment` nie jest jeszcze zsynchronizowany z bazą danych. Aby utworzyć odpowiednią tabelę bazy danych, trzeba wygenerować nową migrację bazy danych.

W wierszu polecenia powłoki uruchom polecenie:

```
python manage.py makemigrations blog
```

Wyświetli się następujący wynik.

```
Migrations for 'blog':
  blog/migrations/0003_comment.py
    - Create model Comment
```

Framework Django wygenerował w katalogu *migrations/* aplikacji `blog` plik *0003_comment.py*. Teraz musisz utworzyć odpowiedni schemat bazy danych, a następnie wprowadzić w niej zmiany.

Wydanie poniższego polecenia spowoduje przeprowadzenie istniejących migracji.

```
python manage.py migrate
```

Wykonanie powyższego polecenia spowoduje wygenerowanie danych wyjściowych zawierających m.in. poniższy wiersz.

```
Applying blog.0003_comment... OK
```

Oznacza to, że utworzona przed chwilą migracja została zastosowana, więc baza danych zawiera tabelę o nazwie `blog_comment`.

Dodawanie modeli do witryny administracyjnej

Teraz utworzony model można dodać do witryny administracyjnej i tym samym mieć możliwość zarządzania komentarzami za pomocą prostego interfejsu.

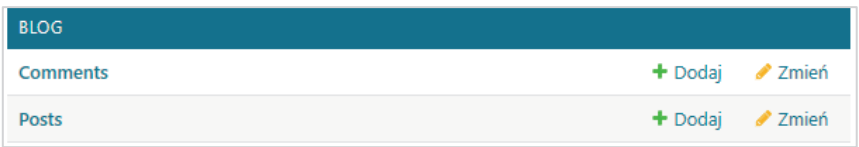
Otwórz plik *admin.py* aplikacji `blog` i dodaj polecenia importujące model `Comment` oraz przedstawioną poniżej klasę `CommentAdmin` dziedziczącą po klasie `ModelAdmin`.

```
from .models import Comment, Post


@admin.register(Comment)
class CommentAdmin(admin.ModelAdmin):
    list_display = ['name', 'email', 'post', 'created', 'active']
    list_filter = ['active', 'created', 'updated']
    search_fields = ['name', 'email', 'body']
```

Przejdź do powłoki i w celu uruchomienia serwera programistycznego, uruchom polecenie:

```
python manage.py runserver
```

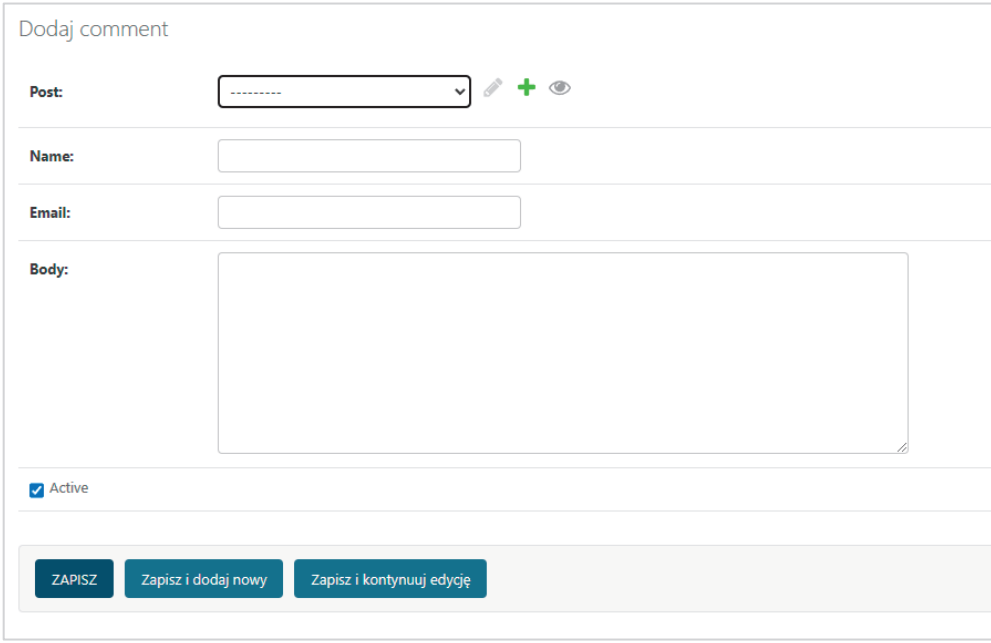
Otwórz w przeglądarce internetowej stronę spod adresu `http://127.0.0.1:8000/admin/`. Powinieneś zobaczyć, że w sekcji *Blog* został wymieniony nowy model, co pokazałem na rysunku 2.21.



Rysunek 2.21. Modele aplikacji bloga na głównej stronie administracyjnej Django

Model `Comment` jest teraz zarejestrowany na stronie administracyjnej.

W wierszu *Comments* kliknij *Dodaj*. Wyświetli się formularz dodania nowego komentarza (rysunek 2.22).



Rysunek 2.22. Formularz dodawania nowego komentarza na stronie administracyjnej Django

Teraz możemy zarządzać egzemplarzami modelu `Comment` za pomocą strony administracyjnej.

Utworzenie formularzy na podstawie modeli

Trzeba zbudować formularz pozwalający użytkownikom komentować wpisy na blogu. Pamiętaj, że Django ma dwie podstawowe klasy, których można użyć do tworzenia formularzy: `Form` i `ModelForm`. Aby umożliwić użytkownikom udostępnianie postów przez e-mail, użyliśmy klasy Form. Teraz, aby skorzystać z istniejącego modelu `Comment` i dynamicznie zbudować dla niego formularz, użyjemy klasy `ModelForm`.

W tym celu przeprowadź edycję pliku *forms.py* aplikacji `blog` i dodaj w nim poniższy fragment kodu.

```
from .models import Comment


class CommentForm(forms.ModelForm):
    class Meta:
        model = Comment
        fields = ['name', 'email', 'body']
```

Aby utworzyć formularz na podstawie modelu, trzeba jedynie wskazać model w klasie `Meta` formularza. Django przeanalizuje podany model i następnie dynamicznie utworzy formularz.

Każdy typ pola modelu ma odpowiedni domyślny typ pola formularza. Atrybuty pól modelu są brane pod uwagę przy walidacji formularza. Domyślnie Django tworzy pole formularza dla każdej kolumny podanej w modelu. Możemy jednak jawnie poinformować Django, które pola mają zostać uwzględnione w formularzu. Do tego celu możesz użyć atrybutu `fields` lub zdefiniować, które pola mają zostać wykluczone, za pomocą atrybutu `exclude`. W formularzu `CommentForm` jawnie uwzględniliśmy pola `name`, `email` i `body`. Są to jedyne pola, które będą uwzględnione w formularzu.

Więcej informacji na temat tworzenia formularzy na podstawie modeli znajdziesz na stronie *https://docs.djangoproject.com/en/5.0/topics/forms/modelforms/*.

Obsługa klasy ModelForm w widokach

Do udostępniania postów przez e-mail używaliśmy tego samego widoku do wyświetlania formularza i zarządzania jego przesyłaniem. Do rozróżnienia obu przypadków wykorzystaliśmy metodę HTTP `GET`, aby wyświetlić formularz, i `POST`, aby go przesłać. W tym przypadku dodamy formularz komentarza do strony szczegółów posta i zbudujemy osobny widok do obsługi przesyłania formularza. Nowy widok przetwarzający formularz pozwoli użytkownikowi powrócić do widoku szczegółów posta po zapisaniu komentarza w bazie danych.

Przeprowadź edycję pliku *views.py* aplikacji `blog` i dodaj w nim poniższy fragment kodu.

```
from django.core.mail import send_mail
from django.core.paginator import EmptyPage, PageNotAnInteger, Paginator
from django.shortcuts import get_object_or_404, render
from django.views.decorators.http import require_POST
from django.views.generic import ListView
from .forms import CommentForm, EmailPostForm
from .models import Post

# ...

@require_POST
def post_comment(request, post_id):
    post = get_object_or_404(
        Post,
        id=post_id,
        status=Post.Status.PUBLISHED
    )
    comment = None
    # Opublikowano komentarz
    form = CommentForm(data=request.POST)
    if form.is_valid():
        # Utwórz obiekt Comment bez zapisywania go w bazie danych
        comment = form.save(commit=False)
        # Przypisz post do komentarza
        comment.post = post
        # Zapisz komentarz do bazy danych
        comment.save()
    return render(
        request,
        'blog/post/comment.html',
```

```
        {
            'post': post,
            'form': form,
            'comment': comment
        },
    )
```

Zdefiniowaliśmy widok `post_comment`, który jako parametry przyjmuje obiekt `request` i zmienną `post_id`. Tego widoku będziemy używać do zarządzania przesyłaniem postów. Oczekujemy, że formularz zostanie wysłany metodą HTTP `POST`. Używamy dekoratora `require_POST` dostarczonego przez Django, aby dla tego widoku zezwalać wyłącznie na żądania `POST`. Django pozwala ograniczyć listę metod HTTP dozwolonych dla widoków. Jeśli spróbujesz uzyskać dostęp do widoku za pomocą dowolnej innej metody HTTP, Django zgłosi błąd HTTP 405 (ang. *method not allowed* — dosłownie: metoda niedozwolona).

W tym widoku zaimplementowaliśmy następujące działania.

1. Pobraliśmy opublikowany post na podstawie jego identyfikatora za pomocą skrótu `get_object_or_404()`.
2. Zdefiniowaliśmy zmienną `comment` o wartości początkowej `None`. Ta zmienna będzie używana do przechowywania obiektu `comment` po jego utworzeniu.
3. Tworzymy egzemplarz formularza na podstawie wysłanych danych `POST` oraz weryfikujemy jego poprawność z wykorzystaniem metody `is_valid()`. Jeżeli formularz zawiera nieprawidłowe dane, renderujemy szablon wraz z błędami wykrytymi podczas walidacji formularza.
4. Jeżeli formularz zawiera prawidłowe dane, tworzymy nowy obiekt `Comment` przez wywołanie metody `save()` formularza, co przedstawiłem poniżej:
   `comment = form.save(commit=False)`
5. Metoda `save()` powoduje utworzenie egzemplarza modelu powiązanego z formularzem oraz jego zapis w bazie danych. Wywołanie wraz z ustawieniem `commit=False` powoduje utworzenie egzemplarza modelu, który jednak nie będzie zapisany w bazie danych. To pozwala zmodyfikować obiekt przed jego ostatecznym zapisaniem.

Uwaga

Metoda `save()` jest dostępna dla klas formularzy dziedziczących po klasie `ModelForm`. Nie jest jednak dostępna dla formularzy dziedziczących po klasie `Form`, ponieważ nie są one powiązane z żadnym modelem.

6. Przypisujemy post do utworzonego przez nas komentarza:
   `comment.post = post`
7. Na końcu zapisujemy nowy komentarz w bazie danych, poprzez wywołanie jego metody `save()`:
   `comment.save()`
8. Renderujemy szablon *blog/post/comment.html* przez przekazanie w kontekście szablonu obiektów `post`, `form` i `comment`. Ten szablon jeszcze nie istnieje; utworzymy go później.

Utwórzmy wzorzec adresu URL dla tego widoku.

Przeprowadź edycję pliku *urls.py* aplikacji `blog` i dodaj w nim poniższy fragment kodu.

```
from django.urls import path
from . import views

app_name = 'blog'

urlpatterns = [
    # Widoki postów
    # path('', views.post_list, name='post_list'),
    path('', views.PostListView.as_view(), name='post_list'),
    path(
        '<int:year>/<int:month>/<int:day>/<slug:post>/',
        views.post_detail,
        name='post_detail',
    ),
    path('<int:post_id>/share/', views.post_share, name='post_share'),
    path(
        '<int:post_id>/comment/', views.post_comment, name='post_comment'
    ),
]
```

Zaimplementowaliśmy widok do zarządzania przesyłaniem komentarzy i odpowiadający mu adres URL. Stwórzmy teraz niezbędne szablony.

Tworzenie szablonów formularza komentarza

W tym punkcie stworzymy szablon formularza komentarza, którego będziemy używać w dwóch miejscach:

- W szablonie szczegółów postu powiązanym z widokiem `post_detail`, aby umożliwić użytkownikom publikowanie komentarzy.
- W szablonie komentarza do postu powiązanym z widokiem `post_comment`, aby ponownie wyświetlić formularz, jeśli występują w nim błędy.

Utworzymy szablon formularza i użyjemy tagu szablonu `{% include %}`, aby włączyć go do dwóch innych szablonów.

W katalogu *templates/blog/post/* utwórz nowy katalog *includes/*. Dodaj w tym nowym katalogu nowy plik i nadaj mu nazwę *comment_form.html*.

Struktura plików powinna wyglądać tak:

```
templates/
  blog/
     post/
       includes/
         comment_form.html
       detail.html
       list.html
       share.html
```

Wyedytuj nowy szablon *blog/post/includes/comment_form.html* i dodaj w nim kod:

```
<h2>Dodaj nowy komentarz</h2>
<form action="{% url "blog:post_comment" post.id %}" method="post">
  {{ form.as_p }}
  {% csrf_token %}
  <p><input type="submit" value="Dodaj komentarz"></p>
</form>
```

W tym szablonie dynamicznie budujemy adres URL action elementu HTML `<form>` za pomocą znacznika szablonu `{% url %}`. Zbudujemy adres URL widoku `post_comment`, który będzie przetwarzał formularz. Wyświetlamy formularz renderowany w akapitach, a ponieważ ten formularz zostanie przesłany metodą `POST`, w celu ochrony przed atakami CSRF dołączamy znacznik `{% csrf_token %}`.

W katalogu *templates/blog/post/* aplikacji blog utwórz nowy plik i nadaj mu nazwę *comment.html*.

Struktura plików powinna wyglądać następująco.

```
templates/
  blog/
    post/
     includes/
       comment_form.html
     comment.html
     detail.html
     list.html
     share.html
```

Wyedytuj nowy szablon *blog/post/comment.html* i wprowadź w nim kod:

```
{% extends "blog/base.html" %}

{% block title %}Dodaj komentarz{% endblock %}

{% block content %}
  {% if comment %}
    <h2>Twój komentarz został dodany.</h2>
    <p><a href="{{ post.get_absolute_url }}">Powrót do posta</a></p>
  {% else %}
    {% include "blog/post/includes/comment_form.html" %}
  {% endif %}
{% endblock %}
```

To jest szablon widoku komentarzy do postów. W przypadku tego widoku oczekujemy, że formularz zostanie wysłany metodą `POST`. Szablon obejmuje dwa różne scenariusze.

- Jeśli przesłane dane formularza będą prawidłowe, zmienna `comment` będzie zawierała utworzony obiekt komentarza i zostanie wyświetlony komunikat o powodzeniu.
- Jeśli dane formularza będą nieprawidłowe, zmienna `comment` będzie mieć wartość `None`. W takim przypadku wyświetlimy formularz komentarza. W celu włączenia wcześniej stworzonego szablonu `comment_form.html` korzystamy z tagu szablonu `{% include %}`.

Dodawanie komentarzy do widoku szczegółów posta

Aby uzupełnić funkcjonalność komentarzy, dodamy do widoku `post_detail` listę komentarzy i formularz komentarza.

Wyedytuj plik *views.py* aplikacji `blog` i zmodyfikuj widok `post_detail` w następujący sposób.

```
def post_detail(request, year, month, day, post):
    post = get_object_or_404(
        Post,
        status=Post.Status.PUBLISHED,
        slug=post,
        publish__year=year,
        publish__month=month,
        publish__day=day,
    )
    # Lista aktywnych komentarzy do tego posta
    comments = post.comments.filter(active=True)
    # Formularz do wprowadzania komentarzy użytkowników
    form = CommentForm()

    return render(
        request,
        'blog/post/detail.html',
        {
            'post': post,
            'comments': comments,
            'form': form
        },
    )
```

Przejrzyjmy kod, który dodaliśmy do widoku `post_detail`.

- Aby pobrać wszystkie aktywne komentarze dla posta, dodaliśmy instrukcję `QuerySet`:

  ```
  comments = post.comments.filter(active=True)
  ```

- Ten obiekt `QuerySet` został zbudowany z wykorzystaniem obiektu `post`. Zamiast bezpośrednio budować kolekcję `QuerySet` dla modelu `Comment`, do pobrania powiązanych obiektów `Comment` wykorzystujemy obiekt `post`. Dla powiązanych obiektów `Comment` używamy menedżera `comments`, który wcześniej zdefiniowaliśmy w modelu `Comment`. W tym celu posługujemy się atrybutem `related_name` dla pola `ForeignKey`, definiującego relację do modelu `Post`.
- Utworzyliśmy również egzemplarz formularza do wprowadzania komentarzy za pomocą instrukcji `form = CommentForm()`.

Dodawanie komentarzy do szablonu szczegółów posta

Teraz trzeba zmodyfikować szablon *blog/post/detail.html* do wykonywania następujących zadań.

- Wyświetlenie całkowitej liczby komentarzy dla posta.
- Wyświetlenie listy komentarzy.
- Wyświetlenie formularza pozwalającego użytkownikom na dodawanie nowych komentarzy.

Na początek zajmiemy się dodaniem informacji o całkowitej liczbie komentarzy dla danego posta.

Zmodyfikuj szablon *blog/post/detail.html* i zmień go tak:

```
{% extends "blog/base.html" %}

{% block title %}{{ post.title }}{% endblock %}

{% block content %}
  <h1>{{ post.title }}</h1>
  <p class="date">
    Opublikowany {{ post.publish }} przez {{ post.author }}
  </p>
  {{ post.body|linebreaks }}
  <p>
    <a href="{% url "blog:post_share" post.id %}">
      Udostępnij ten post
    </a>
  </p>
  {% with comments.count as total_comments %}
    <h2>
      {{total_comments} komentarzy
    </h2>
  {% endwith %}
{% endblock %}
```

W szablonie wykorzystaliśmy warstwę **ORM** Django do pobrania kolekcji `QuerySet` o nazwie `comments.count()`. Zwróć uwagę na fakt, że w języku szablonów nie używa się nawiasów podczas wywoływania metod. Znacznik `{% with %}` pozwala na przypisanie wartości nowej zmiennej, która będzie dostępna do użycia aż do wystąpienia znacznika `{% endwith %}`.

Uwaga

Znacznik `{% with %}` szablonu okazuje się użyteczny, gdy chcemy uniknąć wykonania zapytania do bazy danych lub wielokrotnego uzyskania dostępu do kosztownych metod.

Ponadto wykorzystaliśmy filtr szablonu o nazwie `pluralize` do wyświetlenia przyrostka dla słowa *komentarz* w zależności od wartości `total_comments`. Filtr szablonu jako dane wejściowe pobiera wartość zmiennej, do której został zastosowany, i zwraca odpowiednio obliczoną wartość. Więcej informacji na temat filtrów szablonów znajdziesz w rozdziale 3., „Rozbudowa aplikacji bloga”.

Działanie filtru szablonu o nazwie `pluralize` polega na wyświetleniu y, jeśli wartość jest inna niż 1. Dlatego też poprzedzający filtr tekst zostanie wyświetlony w postaci *0 komentarzy*, *1 komentarz*, *N komentarzy*.

Przechodzimy teraz do dodania listy aktywnych komentarzy do szablonu szczegółów posta.

Zmodyfikuj szablon *blog/post/detail.html* i zmień go w następujący sposób.

```
{% extends "blog/base.html" %}

{% block title %}{{ post.title }}{% endblock %}

{% block content %}
  <h1>{{ post.title }}</h1>
  <p class="date">
    Opublikowany {{ post.publish }} przez {{ post.author }}
  </p>
  {{ post.body|linebreaks }}
  <p>
    <a href="{% url "blog:post_share" post.id %}">
      Udostępnij ten post
    </a>
  </p>
  {% with comments.count as total_comments %}
    <h2>
      {{ total_comments }} komentarzy
    </h2>
  {% endwith %}
  {% for comment in comments %}
    <div class="comment">
      <p class="info">
        Komentarz {{ forloop.counter }} dodany przez {{ comment.name }}
        {{ comment.created }}
      </p>
      {{ comment.body|linebreaks }}
    </div>
  {% empty %}
    <p>Nie ma żadnych komentarzy.</p>
  {% endfor %}
{% endblock %}
```

Znacznik `{% for %}` wykorzystujemy do przeprowadzenia iteracji przez komentarze. Jeżeli lista `comments` jest pusta, wyświetlany jest komunikat domyślny informujący o braku komentarzy dla danego posta. Do umieszczenia komentarzy na liście używamy zmiennej `{{ forloop.counter }}` zawierającej licznik pętli w trakcie każdej iteracji. Przy każdym poście wyświetlamy nazwę użytkownika, który go opublikował, oraz datę i treść komentarza.

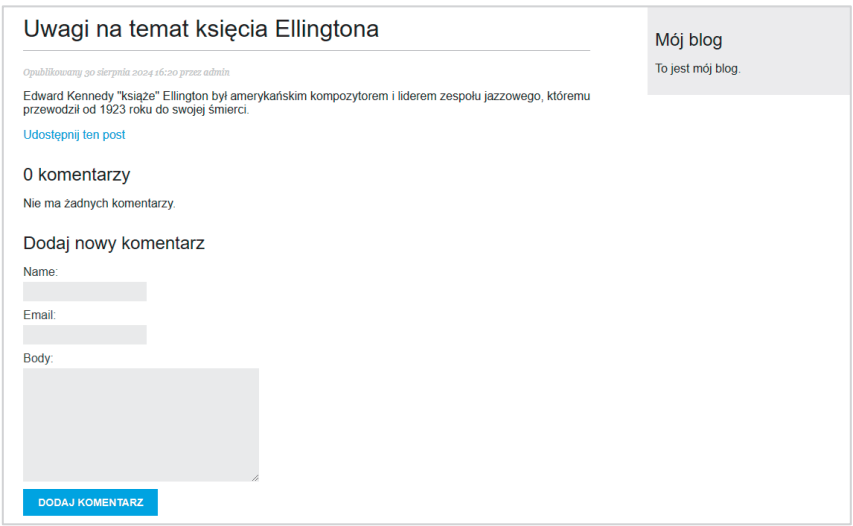
Na koniec dodajemy do szablonu formularz do wprowadzania komentarzy.

Wyedytuj szablon *blog/post/detail.html* i dołącz szablon formularza komentarza:

```
{% extends "blog/base.html" %}

{% block title %}{{ post.title }}{% endblock %}
{% block content %}
```

```
    <h1>{{ post.title }}</h1>
    <p class="date">
      Opublikowany {{ post.publish }} przez {{ post.author }}
    </p>
    {{ post.body|linebreaks }}
    <p>
      <a href="{% url "blog:post_share" post.id %}">
        Udostępnij ten post
      </a>
    </p>
    {% with comments.count as total_comments %}
      <h2>
        {{ total_comments }} komentarzy
      </h2>
    {% endwith %}
    {% for comment in comments %}
      <div class="comment">
        <p class="info">
          Komentarz {{ forloop.counter }} dodany przez {{ comment.name }}
          {{ comment.created }}
        </p>
        {{ comment.body|linebreaks }}
      </div>
    {% empty %}
      <p>Nie ma żadnych komentarzy.</p>
    {% endfor %}
    {% include "blog/post/includes/comment_form.html" %}
{% endblock %}
```

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/blog/` i kliknij tytuł posta, aby zobaczyć stronę szczegółów posta. Powinieneś zobaczyć stroną podobną do pokazanej na rysunku 2.23.

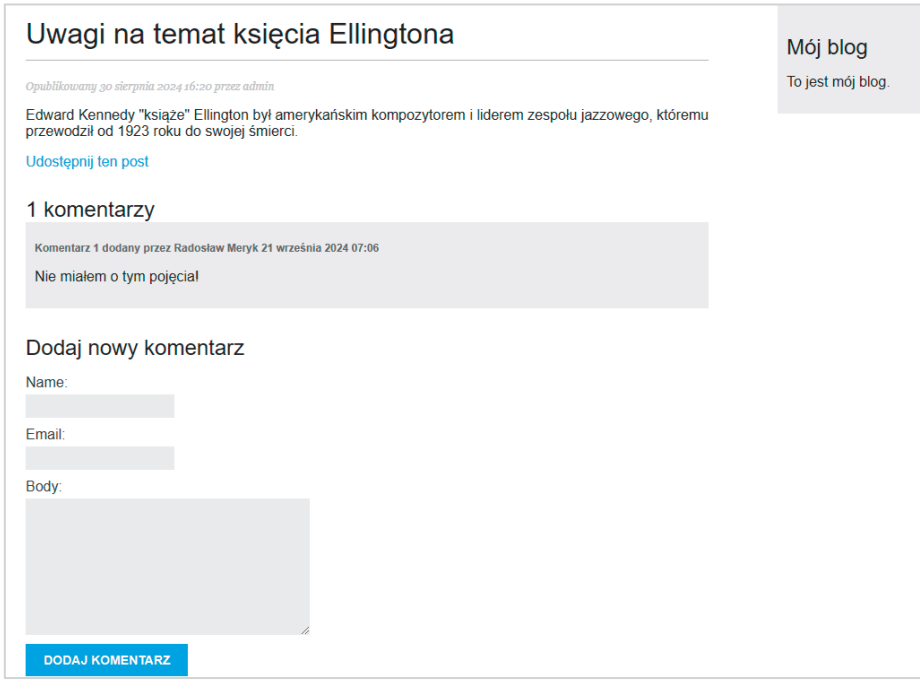


Rysunek 2.23. Strona szczegółów posta włącznie z formularzem do dodawania komentarzy

Wypełnij formularz dodawania komentarza prawidłowymi danymi i kliknij *Dodaj komentarz*. Powinieneś zobaczyć stronę pokazaną na rysunku 2.24.

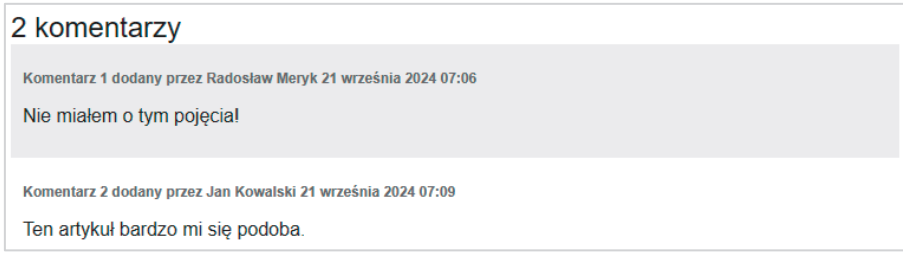
Rysunek 2.24. Strona z komunikatem o powodzeniu dodania komentarza

Kliknij łącze *Powróć do posta*. Powinieneś zostać przekierowany na stronę ze szczegółami posta i zobaczyć komentarz, który dodałeś przed chwilą (rysunek 2.25).



Rysunek 2.25. Strona szczegółów posta włącznie z dodanym komentarzem

Dodaj jeszcze jeden komentarz do posta. Komentarze powinny wyświetlić się pod treścią posta w porządku chronologicznym, w sposób pokazany na rysunku 2.26.

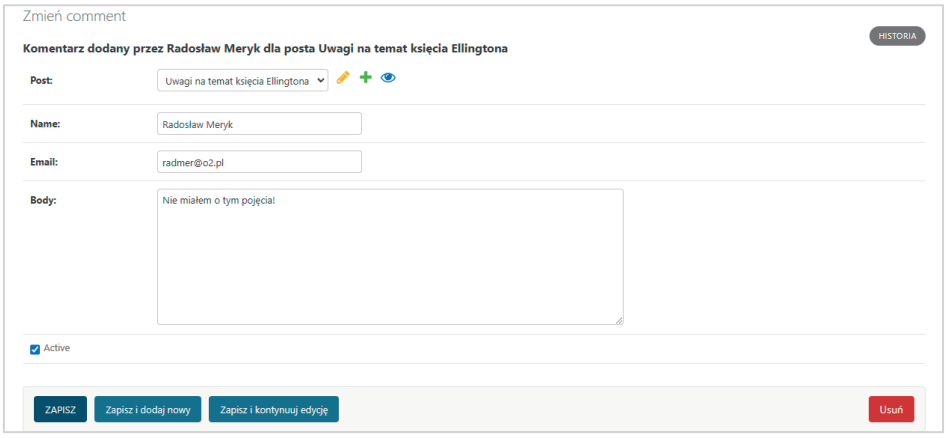


Rysunek 2.26. Lista komentarzy na stronie szczegółów posta

Otwórz w przeglądarce stronę `http://127.0.0.1:8000/admin/blog/comment/`. Zobaczysz stronę administracyjną z listą utworzonych komentarzy (rysunek 2.27).

Rysunek 2.27. Lista komentarzy na stronie administracyjnej

Kliknij nazwę jednego z postów, aby go edytować. Usuń zaznaczenie pola wyboru *Aktywny* w sposób pokazany na rysunku 2.28, po czym kliknij przycisk *Zapisz*.



Rysunek 2.28. Edycja komentarza na stronie administracyjnej

Zostaniesz przekierowany do listy komentarzy. W kolumnie *Aktywny* wyświetli się ikona nieaktywnego komentarza (rysunek 2.29).

Rysunek 2.29. Lista aktywnych i nieaktywnych komentarzy na stronie administracyjnej

Jeżeli powrócisz do widoku szczegółów posta, zobaczysz, że dezaktywowany przed chwilą komentarz nie jest już wyświetlany i nie jest także uwzględniany w całkowitej liczbie komentarzy (rysunek 2.30).

Rysunek 2.30. Pojedynczy aktywny komentarz wyświetlany na stronie szczegółów posta

W ten sposób za pomocą kolumny *active* można dezaktywować niewłaściwe komentarze i uniknąć ich wyświetlania na stronie posta.

Uproszczone szablony do renderowania pól formularzy

Do renderowania formularzy z zastosowaniem akapitów HTML użyliśmy znacznika `{{ form.as_p }}`. To bardzo prosta metoda renderowania formularzy, ale mogą wystąpić sytuacje, w których do renderowania formularzy trzeba wykorzystać niestandardowe znaczniki HTML.

Aby skorzystać z niestandardowego HTML do renderowania pól formularzy, można bezpośrednio sięgać do każdego pola formularza lub iterować po polach formularzy, jak w poniższym przykładzie:

```
{% for field in form %}
  <div class="my-div">
    {{ field.errors }}
    {{ field.label_tag }} {{ field }}
    <div class="help-text">{{ field.help_text|safe }}</div>
  </div>
{% endfor %}
```

W tym kodzie do renderowania błędów pola formularza używamy znacznika `{{ field.errors }}`. Do renderowania etykiet HTML formularza używamy znacznika `{{ field.label_tag }}`, a do renderowania rzeczywistych wartości pól — znacznika`{{ field }}`. Z kolei do renderowania tekstu pomocy HTML pola stosujemy znacznik `{{ field.help_text|safe }}`.

Ta metoda przydaje się do indywidualizowania sposobu renderowania formularzy, ale w przypadku niektórych pól może być konieczne dodanie określonych elementów HTML lub umieszczenie niektórych pól w kontenerach. W Django 5.0 wprowadzono grupy pól i szablony grup pól. Grupy pól upraszczają renderowanie etykiet, widżetów, tekstów pomocy i błędów pól. Skorzystajmy z tej nowej funkcjonalności w celu zindywidualizowania formularza do wprowadzania komentarzy.

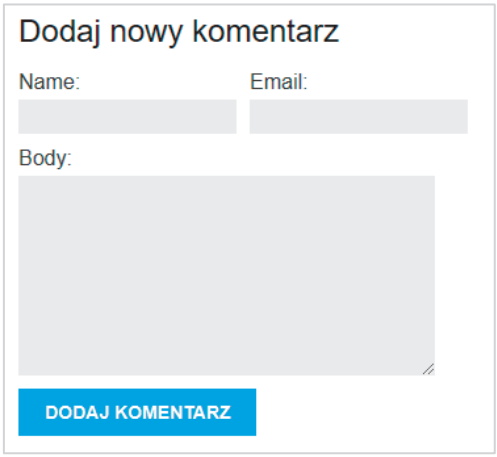
Zamierzamy użyć niestandardowych znaczników HTML, aby zmienić w formularzu położenie pól nazwy użytkownika i adresu e-mail.

Przeprowadź edycję pliku *blog/post/includes/comment_form.html* i zmodyfikuj go w następujący sposób. Nowy kod został wyróżniony pogrubioną czcionką.

```
<h2>Dodaj nowy komentarz</h2>
<form action="{% url "blog:post_comment" post.id %}" method="post">
  <div class="left">
    {{ form.name.as_field_group }}
  </div>
  <div class="left">
    {{ form.email.as_field_group }}
  </div>
  {{ form.body.as_field_group }}
  {% csrf_token %}
  <p><input type="submit" value="Dodaj komentarz"></p>
</form>
```

Dla pól nazwy i adresu e-mail dodaliśmy kontenery `<div>`, z niestandardową klasą CSS, aby przesunąć oba pola w lewo. Metoda `as_field_group` renderuje każde pole włącznie z tekstem pomocy i błędami. Ta metoda domyślnie używa szablonu *django/forms/field.html*. Zawartość tego szablonu można zobaczyć pod adresem *https://github.com/django/django/blob/stable/5.0.x/django/forms/templates/django/forms/field.html*. Możesz również tworzyć niestandardowe szablony pól i używać ich wielokrotnie, przez dodanie do dowolnego pola formularza atrybutu `template_name`. Więcej informacji o szablonach formularzy wielokrotnego użytku można znaleźć na stronie *https://docs.djangoproject.com/en/5.0/topics/forms/#reusable-field-group-templates*.

Otwórz wpis na blogu i spójrz na formularz do wprowadzania komentarza. Powinien teraz wyglądać jak na rysunku 2.31.



Rysunek 2.31. Formularz do wprowadzania komentarzy z nowymi znacznikami HTML

Pola `name` i `email` są teraz wyświetlane obok siebie. Grupy pól umożliwiają łatwą zmianę sposobu renderowania formularzy.

Podsumowanie

W tym rozdziale nauczyłeś się definiować kanoniczne adresy URL dla modeli. Utworzyłeś przyjazne dla SEO adresy URL dla postów na blogu i zaimplementowałeś stronicowanie obiektów dla listy postów. Nauczyłeś się również, jak pracować z formularzami Django i formularzami opartymi na modelach. Stworzyłeś system polecania postów przez e-mail oraz system dodawania komentarzy do bloga.

W następnym rozdziale utworzysz system tagowania dla bloga. Dowiesz się, jak budować złożone kolekcje `QuerySet`, aby wyszukiwać obiekty według podobieństwa. Dowiesz się także, jak tworzyć niestandardowe tagi szablonów i filtry. Zbudujemy również niestandardową mapę witryny i kanał wiadomości dla postów bloga. Zajmiemy się również tematem integracji w aplikacji pełnotekstowego silnika wyszukiwania.

Dodatkowe zasoby

Poniższe źródła zawierają dodatkowe informacje związane z tematami omawianymi w tym rozdziale:

- Kod źródłowy przykładów zamieszczonych w tym rozdziale — *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter02*.
- Funkcje narzędziowe do obsługi adresów URL — *https://docs.djangoproject.com/en/5.0/ref/urlresolvers/*.
- Konwertery ścieżek URL — *https://docs.djangoproject.com/en/5.0/topics/http/urls/#pathconverters*.
- Klasa `Paginator` frameworka Django — *https://docs.djangoproject.com/en/5.0/ref/paginator/*.
- Wprowadzenie do widoków opartych na klasach — *https://docs.djangoproject.com/en/5.0/topics/class-based-views/intro/*.
- Wysyłanie e-maili za pomocą frameworka Django — *https://docs.djangoproject.com/en/5.0/topics/email/*.
- Biblioteka `python-decouple` — *https://github.com/HBNetwork/python-decouple*.
- Biblioteka `django-anymail` — *https://anymail.dev/en/stable/installation/*.
- Dostawcy usług e-mail obsługiwani przez bibliotekę `django-anymail` — *https://anymail.dev/en/stable/esps/*.
- Typy pól formularzy Django — *https://docs.djangoproject.com/en/5.0/ref/forms/fields/*.
- Praca z formularzami — *https://docs.djangoproject.com/en/5.0/topics/forms/*.
- Tworzenie formularzy na podstawie modeli — *https://docs.djangoproject.com/en/5.0/topics/forms/modelforms/*.
- Relacje modelu wiele-do-jednego — *https://docs.djangoproject.com/en/5.0/topics/db/examples/many_to_one/*.

- Domyślny szablon pola formularza — *https://github.com/django/django/blob/stable/5.0.x/django/forms/templates/django/forms/field.html*.
- Szablony grup pól wielokrotnego użytku: *https://docs.djangoproject.com/en/5.0/topics/forms/#reusable-field-group-templates*.

Rozdział 3

Rozbudowa aplikacji bloga

W poprzednim rozdziale wykorzystałeś podstawowe formularze i poznałeś sposób tworzenia systemu komentarzy. Nauczyłeś się również, jak wysyłać e-maile za pomocą Django. W tym rozdziale rozbudujesz aplikację bloga o inne popularne funkcjonalności spotykane na platformach blogowych, takie jak tagowanie, polecanie podobnych postów, dostarczanie czytelnikom kanału RSS i umożliwianie im wyszukiwania postów. Podczas implementowania tych funkcjonalności poznasz nowe komponenty i funkcjonalności frameworka Django.

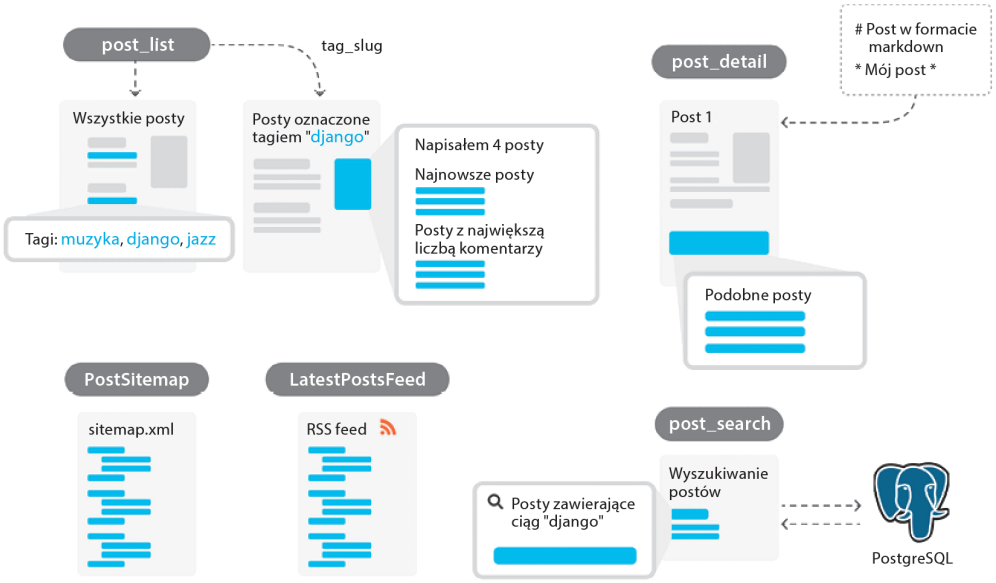
Oto zagadnienia, na których skoncentruję się w tym rozdziale:

- Korzystanie z modułu `django-taggit` do implementacji systemu tagowania
- Pobieranie postów według podobieństwa
- Tworzenie niestandardowych tagów szablonów i filtrów do wyświetlania na pasku bocznym listy najnowszych i najczęściej komentowanych postów
- Tworzenia mapy witryny
- Tworzenie kanałów dla wpisów na blogu
- Instalacja PostgreSQL
- Używanie fikstur do zrzucania danych do bazy danych i ładowania ich z bazy danych
- Implementacja wyszukiwarki pełnotekstowej z wykorzystaniem Django i PostgreSQL

Przegląd funkcjonalności

Reprezentację widoków, szablonów i funkcjonalności, które zbudujemy w tym rozdziale, przedstawiono na rysunku 3.1.

W tym rozdziale zbudujemy funkcjonalność dodawania tagów do postów. Aby filtrować posty według tagów, rozszerzymy widok `post_list`. Podczas ładowania w widoku `post_detail` pojedynczego posta będziemy pobierać podobne posty na podstawie wspólnych tagów. Aby wyświetlić pasek boczny z całkowitą liczbą postów, ostatnio opublikowanymi postami i najczęściej komentowanymi postami, utworzymy również niestandardowe tagi szablonu.



Rysunek 3.1. Schemat funkcjonalności zbudowanych w rozdziale 3.

Dodamy obsługę pisania postów ze składnią Markdown i konwertowania treści na HTML. Utworzymy mapę witryny dla bloga za pomocą klasy `PostSitemap` i zaimplementujemy kanał RSS z najnowszymi postami w klasie `LatestPostsFeed`. Na koniec zaimplementujemy wyszukiwarkę za pomocą widoku `post_search` i wykorzystamy możliwości wyszukiwania pełnotekstowego bazy danych PostgreSQL.

Kod źródłowy przykładów z tego rozdziału można znaleźć pod adresem *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter03*.

Wszystkie pakiety Pythona użyte w tym rozdziale są zawarte w pliku *requirements.txt* dołączonym do plików z kodem źródłowym przykładów z tego rozdziału. Aby zainstalować każdy pakiet Pythona, możesz postępować zgodnie z instrukcjami zamieszczonymi w poniższych podrozdziałach. Możesz także zainstalować wszystkie wymagania jednocześnie za pomocą polecenia `pip install -r requirements.txt`.

Implementacja systemu tagowania za pomocą modułu django-taggit

Powszechną funkcjonalnością blogów jest kategoryzowanie postów za pomocą tagów. Tagi pozwalają sklasyfikować zawartość w sposób niehierarchiczny, za pomocą prostych słów kluczowych. Tag to po prostu etykieta lub słowo kluczowe, które można przypisać do postów. W tym punkcie stworzymy system tagowania przez zintegrowanie z projektem zewnętrznej aplikacji Django do tagowania.

`django-taggit` to przeznaczona do wielokrotnego użycia aplikacja, która przede wszystkim oferuje model `Tag` i menedżera pozwalającego na łatwe dodawanie tagów do dowolnego

modelu. Kod źródłowy wymienionej aplikacji znajdziesz w serwisie GitHub, na stronie *https://github.com/alex/django-taggit*.

Do tworzonej aplikacji bloga dodajmy mechanizm tagowania. Pracę zaczniemy od zainstalowania pakietu `django-taggit` za pomocą menedżera `pip`. W tym celu wydaj poniższe polecenie:

```
python -m pip install django-taggit==5.0.1
```

Przejdź do pliku *settings.py* w projekcie *mysite* i dodaj `taggit` do listy `INSTALLED_APPS`, co pokazałem poniżej.

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'taggit',
    'blog.apps.BlogConfig',
]
```

Uwaga

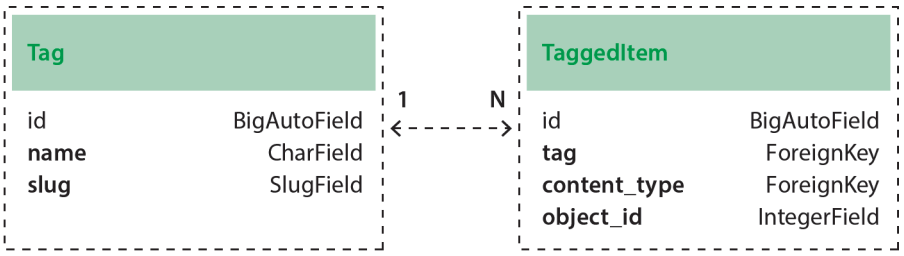
Dobrą praktyką jest wyszczególnianie pakietów Django na górze, pakietów firm zewnętrznych w środku, a lokalnych aplikacji na końcu ustawienia `INSTALLED_APPS`.

Otwórz plik *models.py* aplikacji `blog` i do modelu `Post` dodaj dostarczanego przez `django-taggit` menedżera `TaggableManager` za pomocą poniższego fragmentu kodu.

```
from taggit.managers import TaggableManager

class Post(models.Model):
    # ...
    tags = TaggableManager()
```

Menedżer `tags` umożliwia dodawanie, pobieranie i usuwanie tagów z obiektów `Post`.

Zdefiniowane przez `django-taggit` modele danych do tworzenia tagów i przechowywania powiązanych obiektów z tagami przedstawiłem na schemacie pokazanym na rysunku 3.2.



Rysunek 3.2. Modele Tag aplikacji django-taggit

Model `Tag` służy do przechowywania tagów. Zawiera pola `name` i `slug`.

Model `TaggedItem` służy do przechowywania powiązanych oznakowanych obiektów. Zawiera pole `ForeignKey` dla powiązanego obiektu `Tag`. Zawiera także klucz obcy (`ForeignKey`) do obiektu `ContentType` i pole `IntegerField` do przechowywania powiązanego identyfikatora oznakowanego obiektu. Połączone pola `content_type` i `object_id` tworzą ogólną relację z dowolnym modelem w projekcie. Pozwala to na tworzenie relacji między egzemplarzem modelu `Tag` a egzemplarzem dowolnego innego modelu aplikacji. Więcej informacji o relacjach generycznych znajdziesz w rozdziale 7., „Śledzenie działań użytkownika".

Wydaj poniższe polecenie w wierszu polecenia, aby przygotować migrację dla zmian wprowadzonych w modelu.

```
python manage.py makemigrations blog
```

Powinieneś otrzymać następujący wynik.

```
Migrations for 'blog':
  blog/migrations/0004_post_tags.py
    - Add field tags to post
```

Teraz wydaj poniższe polecenie, aby utworzyć niezbędne tabele bazy danych dla modelu `django-taggit` oraz zsynchronizować zmiany wprowadzone w modelu.

```
python manage.py migrate
```

Wyświetli się wynik, który wskazuje na zastosowanie migracji.

```
Applying taggit.0001_initial... OK
Applying taggit.0002_auto_20150616_2121... OK
Applying taggit.0003_taggeditem_add_unique_index... OK
Applying taggit.0004_alter_taggeditem_content_type_alter_taggeditem_tag... OK
Applying taggit.0005_auto_20220424_2025... OK
Applying taggit.0006_rename_taggeditem_content_type_object_id_taggit_tagg_
↪content_8fc721_idx... OK
Applying blog.0004_post_tags... OK
```

Baza danych jest teraz gotowa do użycia modeli `django-taggit`. Możemy zacząć korzystać z funkcjonalności aplikacji `django-taggit`.

Zobaczmy, jak można użyć menedżera tagów.

Przejdź do powłoki Django przez uruchomienie z terminala polecenia:

```
python manage.py shell
```

Aby pobrać jeden z postów (ten z identyfikatorem 1), uruchom następujący kod.

```
>>> from blog.models import Post
>>> post = Post.objects.get(id=1)
```

Następnie dodamy pewne tagi do pobranego posta i sprawdzimy, czy zostały pomyślnie dodane.

```
>>> post.tags.add('muzyka', 'jazz', 'django')
>>> post.tags.all()
<QuerySet [<Tag: jazz>, <Tag: django>, <Tag: muzyka>]>
```

Na koniec usuwamy tag `django` i ponownie sprawdzamy listę przypisanych tagów.

```
>>> post.tags.remove('django')
>>> post.tags.all()
<QuerySet [<Tag: jazz>, <Tag: muzyka>]>
```

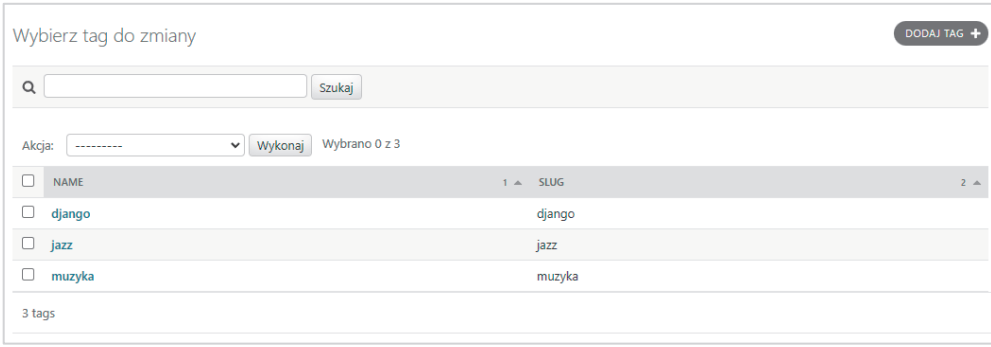
Pobieranie lub usuwanie tagów z modelu za pomocą zdefiniowanego przez siebie menedżera jest bardzo łatwe.

W terminalu uruchom serwer programistyczny za pomocą polecenia:

```
python manage.py runserver
```

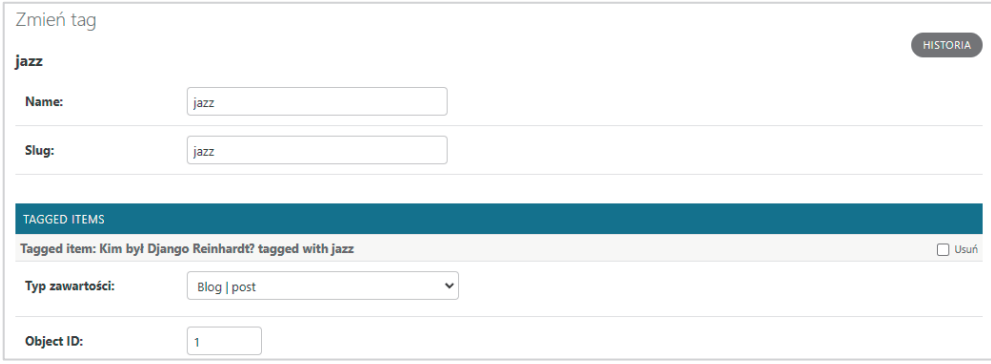
Otwórz w przeglądarce stronę `http://127.0.0.1:8000/admin/taggit/tag/`.

Powinieneś znaleźć się na stronie administracyjnej wyświetlającej listę obiektów `Tag` aplikacji `taggit`, co pokazałem na rysunku 3.3.



Rysunek 3.3. Widok zmiany listy tagów widoczny na stronie administracyjnej Django

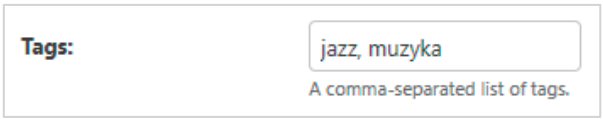
Kliknij tag `jazz`. Wyświetli się wynik pokazany na rysunku 3.4.



Rysunek 3.4: Widok edycji tagów na stronie administracyjnej Django

Przejdź pod adres `http://127.0.0.1:8000/admin/blog/post/1/change/`, aby edytować post o identyfikatorze 1.

Powinieneś zobaczyć, że post zawiera teraz nowe pole `Tags`, pozwalające łatwo dodać tagi (rysunek 3.5).



Rysunek 3.5. Powiązane pole Tags obiektu Post

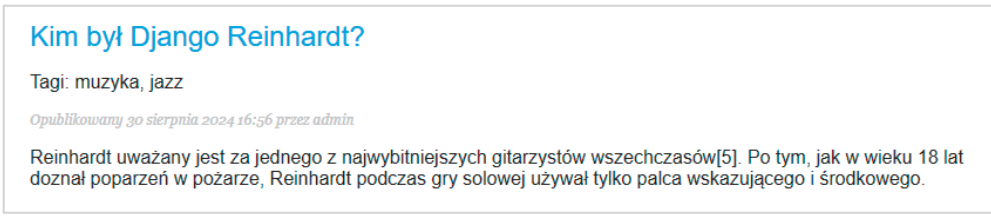
Kolejnym krokiem jest edycja postów bloga, aby przypisane tagi były wyświetlane.

Otwórz szablon *blog/post/list.html* i dodaj następujący kod HTML wyróżniony pogrubioną czcionką.

```
{% extends "blog/base.html" %}

{% block title %}Mój Blog{% endblock %}

{% block content %}
  <h1>Mój Blog</h1>
  {% for post in posts %}
    <h2>
      <a href="{{ post.get_absolute_url }}">
        {{ post.title }}
      </a>
    </h2>
    <p class="tags">Tagi: {{ post.tags.all|join:", " }}</p>
    <p class="date">
      Opublikowany {{ post.publish }} przez {{ post.author }}
    </p>
    {{ post.body|truncatewords:30|linebreaks }}
  {% endfor %}
  {% include "pagination.html" with page=page_obj %}
{% endblock %}
```

Filtr szablonu o nazwie `join` działa w sposób podobny do metody `join()` w Pythonie, czyli łączy podane elementy ze wskazanym ciągiem tekstowym. Na przykład połączenie tagów z listy `['muzyka', 'jazz', 'pianino']` z wykorzystaniem znaku `','` jako separatora metody `join()` spowoduje jej przekształcenie na pojedynczy ciąg `'muzyka,jazz,pianino'`.

Otwórz w przeglądarce stronę `http://127.0.0.1:8000/blog/`. Pod tytułem każdego posta powinieneś zobaczyć listę tagów (rysunek 3.6).



Rysunek 3.6. Element listy postów włącznie z powiązanymi tagami

Teraz przeprowadzimy edycję widoku `post_list`, a zmiany powinny umożliwić użytkownikom wyświetlenie wszystkich postów oznaczonych danym tagiem.

Otwórz plik *views.py* aplikacji blog, zaimportuj model `Tag` z `django-taggit`, a następnie zmień widok `post_list` w sposób pozwalający na opcjonalne filtrowanie postów na podstawie tagu. Nowy kod został wyróżniony pogrubioną czcionką.

```
from taggit.models import Tag

def post_list(request, tag_slug=None):
    post_list = Post.published.all()
    tag = None
    if tag_slug:
        tag = get_object_or_404(Tag, slug=tag_slug)
        post_list = post_list.filter(tags__in=[tag])
    # Stronicowanie z 3 postami na stronę
    paginator = Paginator(post_list, 3)
    page_number = request.GET.get('page', 1)
    try:
        posts = paginator.page(page_number)
    except PageNotAnInteger:
        #  Jeśli page_number nie jest liczbą całkowitą, zwróć pierwszą stronę
        posts = paginator.page(1)
    except EmptyPage:
        #  Jeśli zmienna page_number jest poza zakresem, wyślij ostatnią stronę wyników
        posts = paginator.page(paginator.num_pages)
    return render(
        request,
        'blog/post/list.html',
        {
            'posts': posts,
            'tag': tag
        }
    )
```

Po modyfikacjach widok `post_list` funkcjonuje w taki sposób.

1. Widok pobiera opcjonalny parametr `tag_slug`, którego wartością domyślną jest `None`. Parametr ten będzie użyty w adresie URL.
2. Wewnątrz widoku budujemy początkową kolekcję `QuerySet`, pobieramy wszystkie opublikowane posty i po znalezieniu wskazanego slugu tagu za pomocą skrótu w postaci wywołania `get_object_or_404()` otrzymujemy obiekt `Tag` wraz z danym slugiem.
3. Następnie filtrujemy listę postów, aby otrzymać te, które mają przypisany podany tag. Ponieważ mamy do czynienia z relacją typu „wiele do wielu”, konieczne jest filtrowanie względem tagów znajdujących się na liście. W omawianym przykładzie lista składa się z zaledwie jednego elementu. Skorzystaliśmy z wyszukiwania w polu za pomocą składni `__in`. Relacja „wiele do wielu” występuje, gdy wiele obiektów modelu jest powiązanych z wieloma obiektami innego modelu. W omawianej aplikacji post może mieć wiele tagów, a tag może być powiązany z wieloma postami. Sposób tworzenia relacji „wiele do wielu” omówię w rozdziale 6., „Udostępnianie treści w witrynie internetowej”.

Więcej informacji o relacjach wiele-do-wielu można znaleźć pod adresem *https://docs.djangoproject.com/en/5.0/topics/db/examples/many_to_many/*.

4. Na koniec funkcja `render()` przekazuje do szablonu nową zmienną `tag`.

Pamiętaj o leniwym wartościowaniu kolekcji `QuerySet`. Wymieniona kolekcja pobierze posty tylko podczas iteracji przez listę postów w trakcie renderowania szablonu.

Otwórz plik *urls.py* aplikacji `blog`, umieść znak komentarza na początku wiersza wzorca adresu URL na podstawie klasy `PostListView`, a następnie usuń znak komentarza z początku wiersza widoku `post_list`.

```
path('', views.post_list, name='post_list'),
#  path('', views.PostListView.as_view(), name='post_list'),
```

Dodaj poniższy dodatkowy wzorzec adresu URL, aby umożliwić wyświetlanie listy postów na podstawie tagu.

```
path(
     'tag/<slug:tag_slug>/', views.post_list, name='post_list_by_tag'
    ),
```

Jak możesz zobaczyć, oba wzorce prowadzą do dokładnie tego samego widoku, choć nadaliśmy im odmienne nazwy. Pierwszy wzorzec będzie wywoływał widok `post_list` bez żadnych opcjonalnych parametrów, podczas gdy drugi wywoła widok wraz z parametrem `tag_slug`. Wykorzystaliśmy konwerter ścieżki `slug` w celu dopasowania parametru jako ciągu zapisanego małymi literami i cyframi uzupełnionymi o znaki myślnika i podkreślenia.

Plik *urls.py* aplikacji `blog` powinien teraz mieć następującą zawartość.

```
from django.urls import path
from . import views

app_name = 'blog'

urlpatterns = [
    # Widoki postów
    path('', views.post_list, name='post_list'),
    #  path('', views.PostListView.as_view(), name='post_list'),
    path(
        'tag/<slug:tag_slug>/', views.post_list, name='post_list_by_tag'
    ),
    path(
        '<int:year>/<int:month>/<int:day>/<slug:post>/',
        views.post_detail,
        name='post_detail',
    ),
    path('<int:post_id>/share/', views.post_share, name='post_share'),
    path(
        '<int:post_id>/comment/', views.post_comment, name='post_comment'
    ),
]
```

Ponieważ używamy widoku `post_list`, przeprowadź edycję szablonu *blog/post/list.html* i zmodyfikuj mechanizm stronicowania, aby obiekt `posts` był używany tak:

```
{% include "pagination.html" with page=posts %}
```

Dodaj do szablonu *blog/post/list.html* następujące wiersze wyróżnione pogrubioną czcionką.

```
{% extends "blog/base.html" %}

{% block title %}Mój Blog{% endblock %}

{% block content %}
  <h1>Mój Blog</h1>
  {% if tag %}
    <h2>Posty oznaczone tagiem "{{ tag.name }}"</h2>
  {% endif %}
  {% for post in posts %}
    <h2>
      <a href="{{ post.get_absolute_url }}">
        {{ post.title }}
      </a>
    </h2>
    <p class="tags">Tagi: {{ post.tags.all|join:", " }}</p>
    <p class="date">
      Opublikowany {{ post.publish }} przez {{ post.author }}
    </p>
    {{ post.body|truncatewords:30|linebreaks }}
  {% endfor %}
  {% include "pagination.html" with page=posts %}
{% endblock %}
```

Jeżeli użytkownik przejdzie na blog, otrzyma listę wszystkich postów. Gdy przefiltruje posty na podstawie określonego tagu, otrzyma żądaną listę postów.

Teraz przeprowadź edycję szablonu *blog/post/list.html* i zmień sposób wyświetlania tagów, tak jak pokazałem na poniższym listingu. Nowe wiersze zostały wyróżnione pogrubioną czcionką.

```
{% extends "blog/base.html" %}

{% block title %}Mój Blog{% endblock %}

{% block content %}
  <h1>Mój Blog</h1>
  {% if tag %}
    <h2>Posty oznaczone tagiem "{{ tag.name }}"</h2>
  {% endif %}
  {% for post in posts %}
    <h2>
      <a href="{{ post.get_absolute_url }}">
        {{ post.title }}
      </a>
    </h2>
    <p class="tags">
      Tagi:
      {% for tag in post.tags.all %}
        <a href="{% url "blog:post_list_by_tag" tag.slug %}">
          {{ tag.name }}
        </a>{% if not forloop.last %}, {% endif %}
```

```
        {% endfor %}
      </p>
       <p class="date">
         Opublikowany {{ post.publish }} przez {{ post.author }}
       </p>
       {{ post.body|truncatewords:30|linebreaks }}
    {% endfor %}
    {% include "pagination.html" with page=posts %}
  {% endblock %}
```

W powyższym kodzie przeprowadzamy iterację przez wszystkie tagi posta, wyświetlając przy tym link do adresu URL pozwalającego na filtrowanie postów na podstawie danego tagu. Do zbudowania adresu URL używamy znacznika `{% url "blog:post_list_by_tag" ↪tag.slug %}`, zawierającego jako parametry nazwę adresu URL i slug tagu. Poszczególne tagi są rozdzielone przecinkami.

Otwórz w przeglądarce stronę `http://127.0.0.1:8000/blog/tag/jazz/`. Otrzymasz listę postów filtrowaną na podstawie wybranego tagu (rysunek 3.7).

Rysunek 3.7. Post odfiltrowany za pomocą tagu „jazz"

Pobieranie postów według podobieństwa

Skoro przygotowaliśmy mechanizm oznaczania postów tagami, możemy je teraz wykorzystać do wielu interesujących zastosowań. Przykładowo tagi doskonale się sprawdzają w tworzeniu niehierarchicznych klasyfikacji postów bloga. Ogólnie rzecz biorąc, posty o podobnej tematyce będą oznaczone wieloma tymi samymi tagami. Opracujemy teraz funkcjonalność pozwalającą na wyświetlenie podobnych postów na podstawie liczby współdzielonych przez nie tagów. W ten sposób, gdy użytkownik zakończy lekturę danego posta, będzie można zaproponować mu przejście do innych, podobnych postów.

W celu otrzymania listy postów podobnych do danego trzeba wykonać kilka działań. Oto one:

1. Pobranie wszystkich tagów dla bieżącego posta.
2. Pobranie wszystkich postów, które mają przypisany dowolny z tych tagów.
3. Wykluczenie bieżącego posta z listy, aby uniknąć rekomendacji tego samego posta.

4. Uporządkowanie wyników według liczby tagów współdzielonych z bieżącym postem.
5. Rekomendacja posta nowszego, w przypadku dwóch lub więcej postów o takiej samej liczbie współdzielonych tagów.
6. Ograniczenie liczby postów, które chcemy zarekomendować.

Powyższe kroki zostaną przekształcone na skomplikowaną kolekcję `QuerySet`. Aby uwzględnić sugestie postów oparte na podobieństwie, przeprowadź edycję widoku `post_detail`.

Przejdź do pliku *views.py* aplikacji `blog` i na jego początku umieść polecenie `import`:

```
from django.db.models import Count
```

Warstwa ORM w Django oferuje funkcję agregacji o nazwie `Count`. Funkcja jest przeznaczona do przeprowadzania zagregowanego zliczania. Aplikacja `django.db.models` dostarcza następujące funkcje agregacji.

- `Avg`: wartość średnia;
- `Max`: wartość maksymalna;
- `Min`: wartość minimalna;
- `Count`: całkowita liczba obiektów.

Więcej informacji na temat agregacji można uzyskać na stronie *https://docs.djangoproject.com/en/5.0/topics/db/aggregation/*.

Otwórz plik *views.py* aplikacji `blog` i dodaj następujące wiersze do widoku `post_detail`. Nowe wiersze zostały wyróżnione pogrubioną czcionką.

```
def post_detail(request, year, month, day, post):
    post = get_object_or_404(
        Post,
        status=Post.Status.PUBLISHED,
        slug=post,
        publish__year=year,
        publish__month=month,
        publish__day=day,
    )

    # Lista aktywnych komentarzy do tego posta
    comments = post.comments.filter(active=True)
    # Formularz do wprowadzania komentarzy użytkowników
    form = CommentForm()

    # Lista podobnych postów
    post_tags_ids = post.tags.values_list('id', flat=True)
    similar_posts = Post.published.filter(
        tags__in=post_tags_ids
    ).exclude(id=post.id)
    similar_posts = similar_posts.annotate(
        same_tags=Count('tags')
    ).order_by('-same_tags', '-publish')[:4]

    return render(
```

```
        request,
        'blog/post/detail.html',
        {
            'post': post,
            'comments': comments,
            'form': form,
            'similar_posts': similar_posts,
        },
    )
```

Oto w jaki sposób działa powyższy kod:

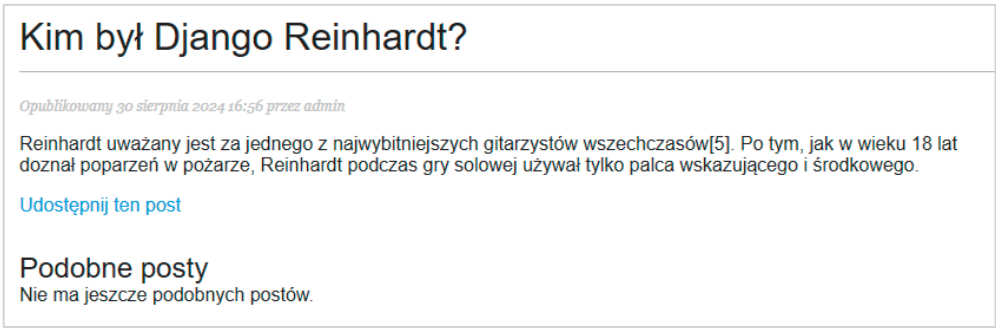
1. Pobieramy listę Pythona zawierającą identyfikatory dla tagów bieżącego posta. Kolekcja QuerySet `values_list()` zwraca krotki wraz z wartościami pochodzącymi z podanych pól. Ponieważ używamy parametru `flat=True`, w wyniku otrzymujemy płaską listę, taką jak `[1, 2, 3, ...]`, zamiast krotek jednoelementowych, takich jak `[(1,), (2,), (3,) ...]`.
2. Pobieramy wszystkie posty zawierające dowolny z otrzymanych wcześniej tagów, z wyłączeniem bieżącego posta.
3. Do wygenerowania wartości pola `same_tags` zawierającego listę tagów współdzielonych ze wszystkimi sprawdzanymi tagami wykorzystujemy funkcję agregacji `Count`.
4. Aby na liście najpierw znalazły się najnowsze posty o tej samej liczbie współdzielonych tagów, wynik porządkujemy według liczby współdzielonych tagów (malejąco) oraz względem wartości `publish`. Ostateczny wynik skracamy i pobieramy jedynie pierwsze cztery posty.
5. Obiekt `similar_posts` dodajemy do słownika kontekstu dla funkcji `render()`.

Teraz przeprowadź edycję szablonu *blog/post/detail.html* i dodaj poniższy fragment kodu HTML wyróżniony pogrubioną czcionką.

```
{% extends "blog/base.html" %}

{% block title %}{{ post.title }}{% endblock %}

{% block content %}
  <h1>{{ post.title }}</h1>
  <p class="date">
    Opublikowany {{ post.publish }} przez {{ post.author }}
  </p>
  {{ post.body|linebreaks }}
  <p>
    <a href="{% url "blog:post_share" post.id %}">
      Udostępnij ten post
    </a>
  </p>
  <h2>Podobne posty </h2>
  {% for post in similar_posts %}
    <p>
      <a href="{{ post.get_absolute_url }}">{{ post.title }}</a>
    </p>
```

```
      {% empty %}
        Nie ma jeszcze podobnych postów.
      {% endfor %}
      {% with comments.count as total_comments %}
        <h2>
          {{total_comments} komentarz{{total_comments|pluralize}}
        </h2>
      {% endwith %}
      {% for comment in comments %}
        <div class="comment">
          <p class="info">
            Komentarz {{ forloop.counter }} dodany przez {{ comment.name }}
            {{ comment.created }}
          </p>
          {{ comment.body|linebreaks }}
        </div>
      {% empty %}
        <p>Nie ma jeszcze żadnych komentarzy.</p>
      {% endfor %}
      {% include "blog/post/includes/comment_form.html" %}
    {% endblock %}
```

Po wprowadzeniu omówionych zmian strona szczegółów posta powinna wyglądać tak, jak pokazałem na rysunku 3.8.



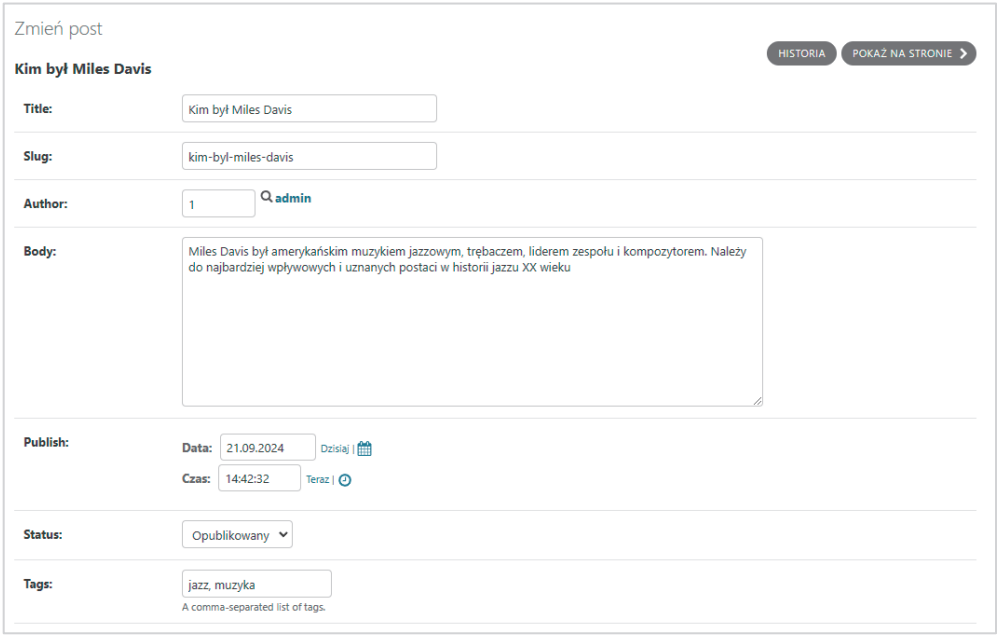
Rysunek 3.8. Strona szczegółów posta włącznie z listą podobnych postów

Przejdź w przeglądarce pod adres `http://127.0.0.1:8000/admin/blog/post/`, zmodyfikuj post, który nie ma tagów, i dodaj tagi `muzyka` i `jazz` w sposób pokazany na rysunku 3.9.

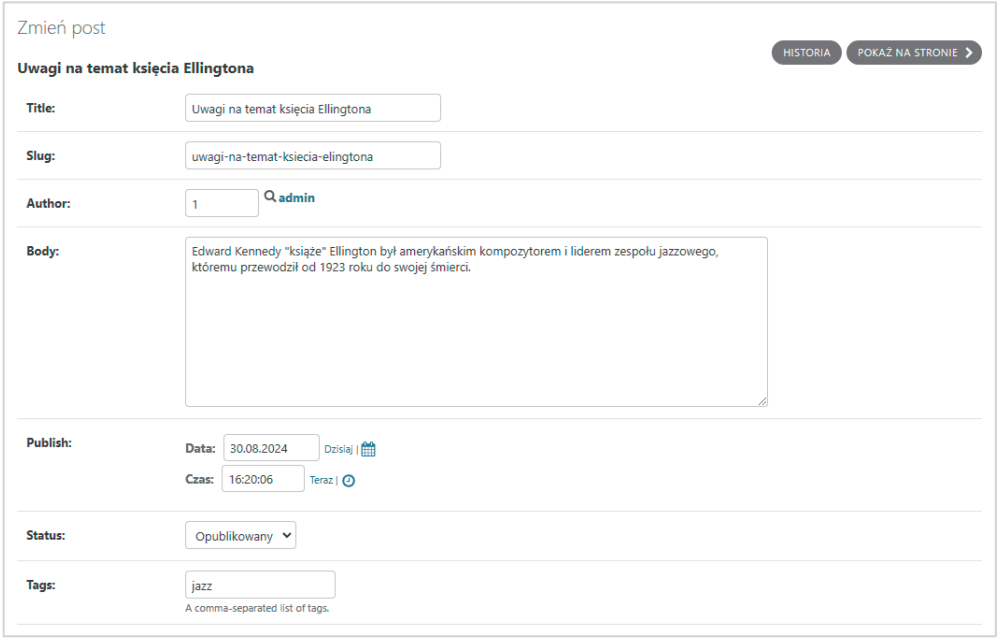
Zmodyfikuj inny post i dodaj tag `jazz` w sposób pokazany na rysunku 3.10.

Po wprowadzeniu omówionych zmian strona szczegółów posta powinna wyglądać tak, jak pokazałem na rysunku 3.11.

Posty polecane w sekcji strony *Podobne posty* są wyświetlane w kolejności malejącej liczby wspólnych tagów z postem oryginalnym.



Rysunek 3.9. Dodawanie do posta tagów jazz i muzyka



Rysunek 3.10. Dodawanie do posta tagu jazz

Rysunek 3.11. Strona szczegółów posta, włącznie z listą podobnych postów

Teraz możemy z powodzeniem polecać podobne posty czytelnikom. Aplikacja `django-taggit` zawiera również menedżera `related_objects()`, z którego można skorzystać do pobierania obiektów na podstawie wspólnych tagów. Informacje o wszystkich menedżerach django-taggit znajdziesz na stronie *https://django-taggit.readthedocs.org/en/latest/api.html*.

Możesz również dodać listę znaczników do szablonu szczegółów posta. W tym celu należy postępować tak samo, jak postąpiliśmy w przypadku szablonu *blog/post/list.html*.

Utworzenie własnych filtrów i znaczników szablonu

Django oferuje różne wbudowane znaczniki szablonów, takie jak `{% if %}` i `{% block %}`. Kilku standardowych znaczników szablonów używałeś już w rozdziale 1., „Utworzenie aplikacji bloga" oraz w rozdziale 2., „Usprawnianie bloga i dodanie funkcji społecznościowych". Pełną dokumentację wbudowanych tagów i filtrów szablonów można znaleźć na stronie *https://docs.djangoproject.com/en/5.0/ref/templates/builtins/*.

Django pozwala również utworzyć własne znaczniki szablonów w celu przeprowadzania niestandardowych działań. Tego rodzaju znaczniki okazują się niezwykle użyteczne, gdy trzeba dodać do szablonów funkcjonalności nieobsługiwane przez podstawowy zestaw znaczników szablonów Django. Może to być znacznik do uruchamiania zapytania `QuerySet` lub dowolnego przetwarzania po stronie serwera, które ma być wielokrotnie wykorzystane w szablonach. Można np. zbudować znacznik szablonu, aby wyświetlić listę najnowszych postów publikowanych na blogu. Można dołączyć go do listy na pasku bocznym bloga, tak aby był widoczny zawsze, niezależnie od widoku przetwarzającego żądanie.

Utworzenie własnych znaczników szablonu

Framework Django oferuje wymienione poniżej funkcje pomocnicze ułatwiające tworzenie własnych znaczników szablonu.

- `simple_tag`: przetwarza przekazane dane i zwraca ciąg tekstowy.
- `inclusion_tag`: przetwarza przekazane dane i zwraca wyrenderowany szablon.

Znaczniki szablonów muszą być zdefiniowane wewnątrz aplikacji Django.

W katalogu aplikacji `blog` utwórz nowy podkatalog o nazwie *templatetags*, a następnie dodaj do niego pusty plik *__init__.py*. Utwórz kolejny plik w tym samym podkatalogu i nadaj mu nazwę *blog_tags.py*. Struktura plików aplikacji `blog` powinna wyglądać następująco.

```
blog/
    __init__.py
    models.py
    ...
    templatetags/
        __init__.py
        blog_tags.py
```

Nazwa pliku jest ważna, ponieważ nazwa utworzonego powyżej modułu będzie wykorzystywana do wczytywania znaczników w szablonach.

Utworzenie tagu szablonu typu simple_tag

Pracę rozpoczynamy od utworzenia prostego znacznika odpowiedzialnego za podanie liczby wszystkich postów opublikowanych na blogu.

Przeprowadź edycję utworzonego przed chwilą pliku *templatetags/blog_tags.py* i dodaj do niego poniższy fragment kodu.

```
from django import template
from ..models import Post

register = template.Library()

@register.simple_tag
def total_posts():
    return Post.published.count()
```

W ten sposób utworzyliśmy prosty znacznik szablonu, który podaje liczbę postów opublikowanych dotąd na blogu.

Każdy moduł znaczników szablonu musi zawierać zmienną o nazwie `register`, aby mógł być uznawany za prawidłową bibliotekę znaczników. Wymieniona zmienna to egzemplarz klasy `template.Library`; jest ona używana do rejestrowania w aplikacji filtrów i znaczników szablonu.

W powyższym kodzie za pomocą prostej funkcji Pythona zdefiniowaliśmy tag o nazwie `total_posts`. Aby zarejestrować funkcję jako prosty tag, dodaliśmy do niej dekorator `@register.simple_tag`. Django w roli nazwy znacznika wykorzystuje nazwę funkcji.

Jeżeli chcesz go zarejestrować pod inną nazwą, możesz to zrobić przy użyciu atrybutu `name`, np. tak: `@register.simple_tag(name='prosty_znacznik')`.

Uwaga

Po dodaniu nowego modułu znaczników, aby można było rozpocząć korzystanie z nowo zdefiniowanych filtrów i znaczników szablonu, trzeba ponownie uruchomić serwer programistyczny Django.

Zanim zaczniesz używać niestandardowych znaczników szablonu, musisz je udostępnić szablonowi za pomocą znacznika `{% load %}`. Jak wcześniej wspomniałem, konieczne jest użycie nazwy modułu Pythona zawierającego filtry i znaczniki szablonu.

Otwórz szablon *blog/templates/base.html* i na początku pliku umieść polecenie `{% load ↪blog_tags %}`, odpowiedzialne za wczytanie modułu znaczników szablonu. Następnie utworzonego wcześniej znacznika możesz użyć do wyświetlenia liczby wszystkich opublikowanych postów. Nowe wiersze zostały wyróżnione pogrubioną czcionką.

```
{% load blog_tags %}
{% load static %}
<!DOCTYPE html>
<html>
<head>
  <title>{% block title %}{% endblock %}</title>
  <link href="{% static "css/blog.css" %}" rel="stylesheet">
</head>
<body>
  <div id="content">
    {% block content %}
    {% endblock %}
  </div>
  <div id="sidebar">
    <h2>Mój Blog</h2>
    <p>
      To jest mój blog.
      Dotychczas opublikowałem {% total_posts %} postów.
    </p>
  </div>
</body>
</html>
```

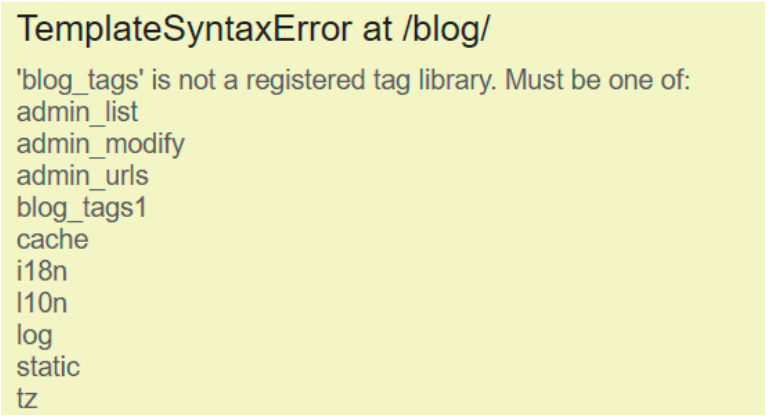
Aby śledzić nowe pliki dodane do projektu, trzeba ponownie uruchomić serwer. W tym celu należy zatrzymać działanie serwera programistycznego za pomocą klawiszy *Ctrl+C* i uruchomić go ponownie przez wydanie polecenia:

```
python manage.py runserver
```

Otwórz w przeglądarce stronę `http://127.0.0.1:8000/blog/`. Na pasku bocznym witryny powinieneś zobaczyć całkowitą liczbę postów (rysunek 3.12).

Jeśli widzisz komunikat o błędzie pokazany na rysunku 3.13, jest bardzo prawdopodobne, że nie zrestartowałeś serwera programistycznego.

Rysunek 3.12. Umieszczona na pasku bocznym informacja o łącznej liczbie opublikowanych postów



Rysunek 3.13. Komunikat o błędzie, gdy biblioteka znaczników szablonów nie została zarejestrowana

Potęga niestandardowych znaczników szablonu wiąże się z możliwością przetwarzania dowolnych danych i dodawania ich do wybranego szablonu niezależnie od wykonywanego widoku. W celu wyświetlenia wyników w szablonach możesz wykorzystać kolekcję `QuerySet` lub przetwarzać dowolne dane.

Tworzenie znacznika szablonu typu inclusion_tag

Teraz przechodzimy do utworzenia kolejnego znacznika. Za jego pomocą wyświetlimy w pasku bocznym tytuły postów ostatnio opublikowanych na blogu. Tym razem zaimplementujemy znacznik typu `inclusion_tag`. Za pomocą wymienionej funkcji można wygenerować szablon wraz ze zmiennymi kontekstu zwróconymi przez znacznik szablonu.

Przeprowadź edycję pliku *templatetags/blog_tags.py* i dodaj w nim poniższy fragment kodu.

```
@register.inclusion_tag('blog/post/latest_posts.html')
def show_latest_posts(count=5):
    latest_posts = Post.published.order_by('-publish')[:count]
    return {'latest_posts': latest_posts}
```

W powyższym kodzie za pomocą dekoratora `@register.inclusion_tag()` rejestrujemy znacznik szablonu. Określiliśmy, że wraz ze zwróconymi wartościami ma zostać wygenerowany szablon *blog/post/latest_posts.html*. Ten znacznik szablonu akceptuje opcjonalny

parametr `count` o wartości domyślnej 5. Wymieniony parametr pozwala na podanie liczby postów, które mają zostać wyświetlone. Wartość tę wykorzystamy do ograniczenia wyników zapytania `Post.published.order_by('-publish')[:count]`.

Zwróć uwagę na fakt, że funkcja zwraca słownik zmiennych zamiast prostej wartości. Znaczniki definiowane za pomocą funkcji `inclusion_tag()` zwracają słownik wartości używany jako kontekst do wygenerowania wskazanego szablonu. Utworzony tutaj znacznik szablonu może otrzymać opcjonalny parametr określający liczbę wyświetlanych postów, np. `{% show_latest_posts 3 %}`.

Teraz w katalogu *blog/post* utwórz nowy plik szablonu i nadaj mu nazwę *latest_posts.html*.

Wyedytuj nowy szablon *blog/post/latest_posts.html* i wprowadź w nim kod:

```
<ul>
  {% for post in latest_posts %}
    <li>
      <a href="{{ post.get_absolute_url }}">{{ post.title }}</a>
    </li>
  {% endfor %}
</ul>
```

Powyższy kod na podstawie wartości dostarczanej przez zmienną `latest_posts` zwróconą przez znacznik szablonu wyświetla nieuporządkowaną listę postów. Teraz przeprowadź edycję szablonu *blog/base.html* i dodaj nowy znacznik szablonu, aby wyświetlić tytuły trzech ostatnio opublikowanych postów. Nowe wiersze zostały wyróżnione pogrubioną czcionką.

```
{% load blog_tags %}
{% load static %}
<!DOCTYPE html>
<html>
<head>
  <title>{% block title %}{% endblock %}</title>
  <link href="{% static "css/blog.css" %}" rel="stylesheet">
</head>
<body>
  <div id="content">
    {% block content %}
    {% endblock %}
  </div>
  <div id="sidebar">
    <h2>Mój Blog</h2>
    <p>
      To jest mój blog.
      Dotychczas opublikowałem {% total_posts %} postów.
    </p>
    <h3>Najnowsze posty</h3>
    {% show_latest_posts 3 %}
  </div>
</body>
</html>
```

Wywołujemy znacznik szablonu i przekazujemy do niego liczbę postów, których tytuły mają zostać wyświetlone. Następnie jest renderowany szablon wraz z podanym kontekstem.

Wróć do przeglądarki internetowej i odśwież stronę. Po wprowadzonych zmianach pasek boczny powinien wyglądać tak, jak pokazałem na rysunku 3.14.

Rysunek 3.14. Pasek boczny zawierający ostatnio opublikowane posty

Tworzenie znacznika szablonu, który zwraca kolekcję QuerySet

Na koniec opracujemy prosty znacznik szablonu, który zwraca wartość. Wynik przechowamy w zmiennej, dzięki czemu będziemy mogli ponownie wykorzystać ją zamiast bezpośredniego wyświetlania. Utworzymy tag, aby wyświetlić najczęściej komentowane posty.

Przeprowadź edycję pliku *templatetags/blog_tags.py* i dodaj w nim poniższy fragment kodu.

```
from django.db.models import Count

@register.simple_tag
def get_most_commented_posts(count=5):
    return Post.published.annotate(
        total_comments=Count('comments')
    ).order_by('-total_comments')[:count]
```

Ta kolekcja `QuerySet` używa funkcji `annotate()` w celu agregacji łącznej liczby komentarzy dla każdego posta. Korzystamy z funkcji agregacji `Count` w celu zapisania w wyliczanym polu `total_comments` liczby komentarzy dla każdego obiektu `Post`. Następnie porządkujemy kolekcję `QuerySet` według obliczonej liczby komentarzy w porządku malejącym. Ponadto dostarczana jest opcjonalna zmienna `count` pozwalająca na ograniczenie liczby zwróconych obiektów do podanej wartości.

Oprócz funkcji `Count`, framework Django oferuje jeszcze inne funkcje agregacji, np. `Avg`, `Max`, `Min` i `Sum`. Więcej informacji na temat tych funkcji znajdziesz na stronie *https://docs.djangoproject.com/en/5.0/topics/db/aggregation/*.

Teraz przeprowadź edycję szablonu *blog/base.html* i dodaj poniższy fragment kodu HTML wyróżniony pogrubioną czcionką.

```
{% load blog_tags %}
{% load static %}
<!DOCTYPE html>
<html>
<head>
  <title>{% block title %}{% endblock %}</title>
  <link href="{% static "css/blog.css" %}" rel="stylesheet">
```

```
</head>
<body>
  <div id="content">
    {% block content %}
    {% endblock %}
  </div>
  <div id="sidebar">
    <h2>Mój Blog</h2>
    <p>
      To jest mój blog.
      Dotychczas opublikowałem {% total_posts %} postów.
    </p>
    <h3>Najnowsze posty</h3>
    {% show_latest_posts 3 %}
    <h3>Najczęściej komentowane posty</h3>
    {% get_most_commented_posts as most_commented_posts %}
    <ul>
      {% for post in most_commented_posts %}
        <li>
          <a href="{{ post.get_absolute_url }}">{{ post.title }}</a>
        </li>
      {% endfor %}
    </ul>
  </div>
</body>
</html>
```

W powyższym kodzie zapisaliśmy wynik w niestandardowej zmiennej z wykorzystaniem argumentu `as`, za którym występuje nazwa zmiennej. W przypadku omawianego znacznika użyliśmy `{% get_most_commented_posts as most_commented_posts %}`. W ten sposób wynik wykonania znacznika szablonu zostaje umieszczony w nowej zmiennej o nazwie `most_commented_posts`. Następnie zwrócone tytuły postów są wyświetlane za pomocą nieuporządkowanej listy HTML.

Powróć do przeglądarki internetowej i odśwież stronę, aby zobaczyć końcowy efekt, który powinien być podobny do pokazanego na rysunku 3.15.

Rysunek 3.15. Widok listy postów włącznie z kompletnym paskiem bocznym

Powinieneś teraz mieć czytelny pogląd na temat tworzenia niestandardowych tagów szablonów. Więcej informacji na temat niestandardowych znaczników szablonu znajdziesz na stronie *https://docs.djangoproject.com/en/5.0/howto/custom-template-tags/*.

Utworzenie własnych filtrów szablonu

Django oferuje wiele wbudowanych filtrów pozwalających modyfikować zmienne w szablonach. Przyjmują one postać funkcji Pythona pobierających jeden lub dwa parametry, takie jak stosowana wartość zmiennej oraz opcjonalny argument. Filtry zwracają dane, które mogą być przeznaczone do wyświetlenia lub odgrywać rolę innych filtrów.

Filtry zapisujemy w następujący sposób — `{{ zmienna|mój_filtr }}`. Filtry z argumentem są zapisywane tak: `{{ zmienna|mój_filtr:"foo"}}`. Można przykładowo skorzystać z filtru `capfirst`, aby wyświetlić pierwszą literę wartości jako wielką — `{{ value|capfirst }}`. Jeśli `value` to *django*, wyświetli się wartość *Django*. Istnieje możliwość zastosowania dowolnej liczby filtrów, np. `{{ zmienna|filtr1|filtr2 }}`, wówczas dane wyjściowe filtru są używane jako dane wejściowe kolejnego.

Listę wbudowanych filtrów szablonu frameworka Django można znaleźć pod adresem *https://docs.djangoproject.com/en/5.0/ref/templates/builtins/*.

Tworzenie filtra szablonu obsługującego składnię Markdown

Przystępujemy teraz do utworzenia własnego filtru pozwalającego na użycie składni Markdown w postach bloga, a następnie konwersji treści posta na kod HTML.

Wspomniany Markdown to składnia formatowania zwykłego tekstu, która jest bardzo prosta w użyciu i łatwa do konwersji na kod HTML. Możesz pisać posty z wykorzystaniem prostej składni Markdown, a treść zapisana za pomocą tej składni zostanie automatycznie przekształcona na HTML. Nauczenie się składni Markdown jest znacznie łatwiejsze niż nauczenie się składni HTML. Dzięki składni Markdown posty na blogu mogą łatwo pisać użytkownicy nieposiadający wiedzy technicznej na temat formatu HTML. Więcej informacji podstawowych na temat tej składni znajdziesz na stronie *https://daringfireball.net/projects/markdown/basics*.

Pracę zaczynamy od instalacji w Pythonie modułu o nazwie `markdown` za pomocą menedżera `pip`. W tym celu wydaj poniższe polecenie.

```
python -m pip install markdown==3.6
```

Następnie przeprowadź edycję pliku *templatetags/blog_tags.py* i umieść w nim poniższy fragment kodu.

```
import markdown
from django.utils.safestring import mark_safe

@register.filter(name='markdown')
def markdown_format(text):
    return mark_safe(markdown.markdown(text))
```

Rejestracja filtrów szablonu odbywa się dokładnie tak samo jak rejestracja znaczników szablonu. Aby uniknąć kolizji między nazwą funkcji i modułu `markdown`, w omawianym przykładzie nazwiemy funkcję `markdown_format()`. Natomiast nazwa filtru używana w szablonach to `markdown`, np. `{{ zmienna|markdown }}`.

Django oczyszcza kod HTML generowany przez filtry; znaki encji HTML są zastępowane kodami HTML. Przykładowo znacznik `<p>` zostanie przekonwertowany na `<p>` (symbol „mniejszy niż", znak p oraz symbol „większy niż").

Dostarczaną przez framework funkcję `mark_safe` wykorzystamy do oznaczenia wyniku jako bezpiecznego kodu HTML do wyrenderowania w szablonie. Domyślnie Django nie ufa żadnemu kodowi HTML i będzie przeprowadzać jego oczyszczenie przed umieszczeniem w danych wyjściowych. Jedynym wyjątkiem są zmienne oznaczone jako zawierające bezpieczny kod HTML. Dzięki takiemu podejściu Django chroni przed użyciem potencjalnie niebezpiecznego kodu HTML i pozwala na utworzenie wyjątku, gdy masz pewność, że dostarczasz bezpieczny kod HTML.

Uwaga

W Django zawartość HTML jest dla bezpieczeństwa domyślnie oczyszczana (ang. *escaped*). Używaj funkcji `mark_safe` ostrożnie, wyłącznie w odniesieniu do treści, nad którymi masz kontrolę. Aby zapobiec lukom w zabezpieczeniach, unikaj używania funkcji `mark_safe` w odniesieniu do treści przesłanych przez użytkowników nieuprzywilejowanych.

Teraz przeprowadź edycję szablonu *blog/post/detail.html* i dodaj poniższy fragment kodu HTML wyróżniony pogrubioną czcionką:

```
{% extends "blog/base.html" %}
{% load blog_tags %}

{% block title %}{{ post.title }}{% endblock %}

{% block content %}
  <h1>{{ post.title }}</h1>
  <p class="date">
    Opublikowany {{ post.publish }} przez {{ post.author }}
  </p>
  {{ post.body|markdown }}
  <p>
    <a href="{% url "blog:post_share" post.id %}">
      Udostępnij ten post
    </a>
  </p>
  <h2>Podobne posty </h2>
  {% for post in similar_posts %}
    <p>
      <a href="{{ post.get_absolute_url }}">{{ post.title }}</a>
    </p>
  {% empty %}
     Nie ma podobnych postów.
  {% endfor %}
```

```
  {% with comments.count as total_comments %}
    <h2>
      {{total_comments} komentarz{{total_comments|pluralize}}
    </h2>
  {% endwith %}
  {% for comment in comments %}
    <div class="comment">
      <p class="info">
        Comment {{ forloop.counter }} by {{ comment.name }}
        {{ comment.created }}
      </p>
      {{ comment.body|linebreaks }}
    </div>
  {% empty %}
    <p>Nie ma jeszcze żadnych komentarzy.</p>
  {% endfor %}
  {% include "blog/post/includes/comment_form.html" %}
{% endblock %}
```

Zastąpiliśmy filtr `linebreaks` w zmiennej szablonu `{{ post.body }}` filtrem `markdown`. Ten filtr nie tylko przekształci podziały linii w tagi `<p>`, ale przekształci również formatowanie Markdown w kod HTML.

Uwaga

Przechowywanie tekstu w bazie danych w formacie Markdown, a nie HTML, jest mądrą strategią bezpieczeństwa. Stosowanie formatu Markdown ogranicza możliwość wstrzykiwania złośliwej zawartości. Dzięki takiemu podejściu tekst jest bezpiecznie konwertowany na HTML dopiero w momencie renderowania szablonu.

Teraz przeprowadź edycję szablonu *blog/post/list.html* i dodaj poniższy fragment kodu HTML wyróżniony pogrubioną czcionką.

```
{% extends "blog/base.html" %}
{% load blog_tags %}

{% block title %}Mój Blog{% endblock %}

{% block content %}
  <h1>Mój Blog</h1>
  {% if tag %}
    <h2>Posty oznaczone tagiem "{{ tag.name }}"</h2>
  {% endif %}
  {% for post in posts %}
    <h2>
      <a href="{{ post.get_absolute_url }}">
        {{ post.title }}
      </a>
    </h2>
    <p class="tags">
      Tagi:
```

```
        {% for tag in post.tags.all %}
          <a href="{% url "blog:post_list_by_tag" tag.slug %}">
            {{ tag.name }}
          </a>
          {% if not forloop.last %}, {% endif %}
        {% endfor %}
      </p>
      <p class="date">
        Opublikowany {{ post.publish }} przez {{ post.author }}
      </p>
      {{ post.body|markdown|truncatewords_html:30 }}
    {% endfor %}
    {% include "pagination.html" with page=posts %}
  {% endblock %}
```

Do zmiennej szablonu `{{ post.body }}` dodaliśmy nowy filtr `markdown`. Ten filtr przekształci zawartość Markdown w HTML.

Dlatego zastąpiliśmy poprzedni filtr `truncatewords` filtrem `truncatewords_html`. Filtr `trun`
`↪catewords_html` powoduje skrócenie ciągu tekstowego po podanej liczbie słów, co w tym miejscu pomaga w uniknięciu niezamkniętych znaczników HTML.

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/admin/blog/`
`↪post/add/` i dodaj nowy post o treści:

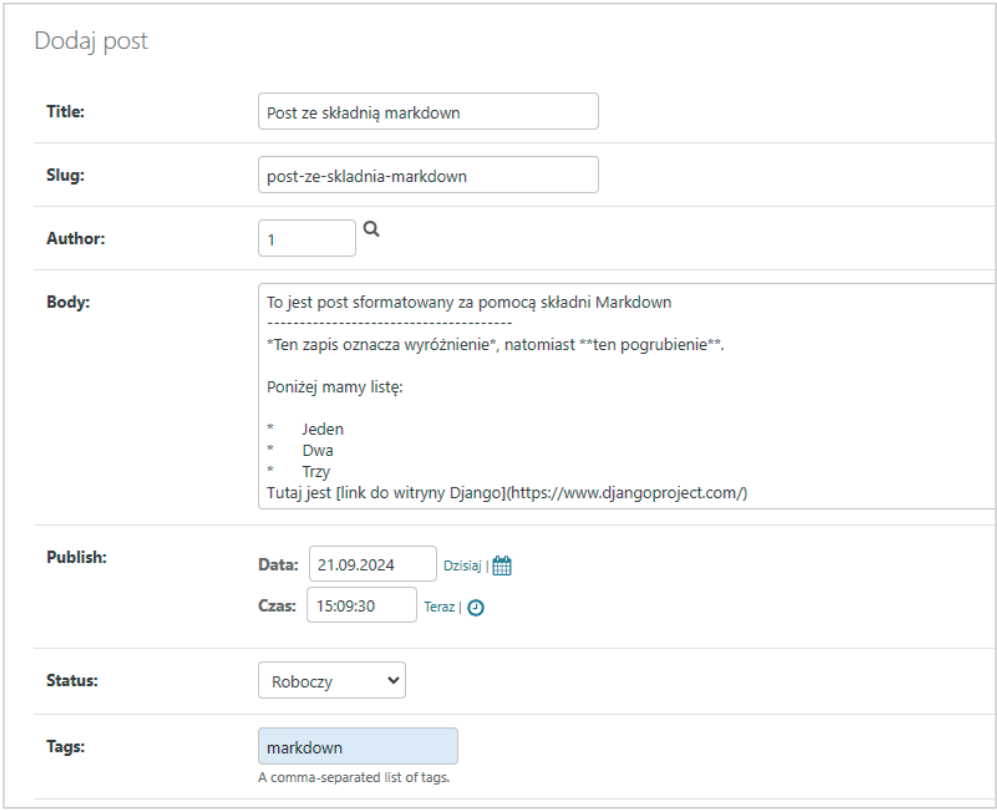
```
To jest post sformatowany za pomocą składni Markdown
-------------------------------------
*Ten zapis oznacza wyróżnienie*, natomiast **ten pogrubienie**.

Poniżej mamy listę:

*    Jeden
*    Dwa
*    Trzy

Tutaj jest [link do witryny Django](https://www.djangoproject.com/)
```

Powinieneś uzyskać formularz w postaci pokazanej na rysunku 3.16.

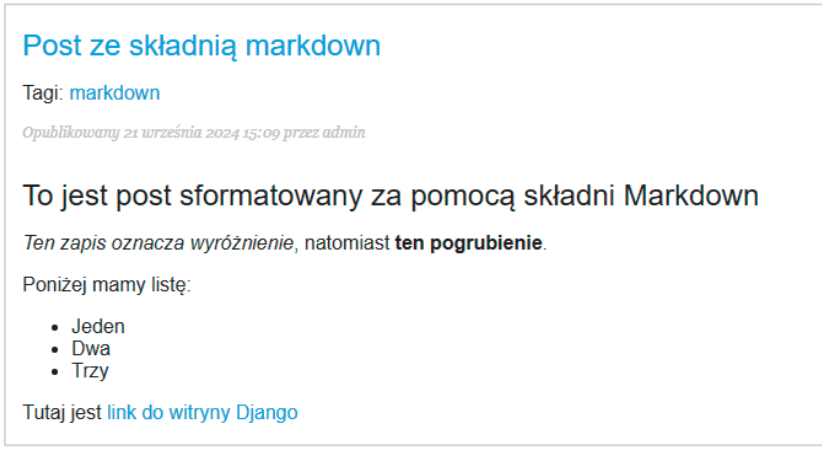
Otwórz w przeglądarce stronę `http://127.0.0.1:8000/blog/` i przyjrzyj się, w jaki sposób nowy post został wyrenderowany.

Powinieneś otrzymać wynik podobny do pokazanego na rysunku 3.17.

Jak możesz zobaczyć na rysunku 3.17, własne filtry szablonu okazują się bardzo użyteczne podczas modyfikowania formatowania. Więcej informacji na temat niestandardowych filtrów znajdziesz na stronie *https://docs.djangoproject.com/en/5.0/howto/custom-template-tags*.



Rysunek 3.16. Post napisany za pomocą składni Markdown wyrenderowany jako HTML



Rysunek 3.17. Post napisany za pomocą składni Markdown wyrenderowany jako HTML

Dodanie mapy witryny

Django oferuje wbudowany framework `sitemap` przeznaczony do dynamicznego generowania mapy witryny. Mapa witryny to po prostu plik XML przekazujący wyszukiwarkom internetowym informacje o stronach znajdujących się w witrynie, powiązaniach między nimi oraz częstotliwości uaktualniania tych stron. Używając mapy witryny, ułatwiasz robotom sieciowym indeksowanie treści zamieszczonej w witrynie internetowej.

Do prawidłowego funkcjonowania frameworka `sitemap` w Django konieczna jest aplikacja `django.contrib.sites`, która pozwala na powiązanie obiektów z określonymi witrynami internetowymi działającymi w ramach danego projektu. To okazuje się niezwykle użyteczne, gdy za pomocą pojedynczego projektu Django chcesz opracować wiele witryn internetowych. Aby zainstalować framework `sitemap`, konieczna jest aktywacja w projekcie aplikacji `sites` i `sitemap`. Zamierzamy zbudować dla aplikacji bloga mapę witryny, która będzie zawierać linki do wszystkich opublikowanych postów.

Przeprowadź edycję pliku *settings.py* projektu oraz umieść `django.contrib.sites` i `django.contrib.sitemaps` na liście `INSTALLED_APPS`. Ponadto w pokazany poniżej sposób zdefiniuj nowe ustawienie dla identyfikatora witryny. Nowy kod został wyróżniony pogrubioną czcionką.

```
# ...

SITE_ID = 1

# Definicje aplikacji
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.sites',
    'django.contrib.sitemaps',
    'django.contrib.staticfiles',
    'taggit',
    'blog.apps.BlogConfig',
]
```

Teraz po wydaniu poniższego polecenia nastąpi utworzenie w bazie danych tabel przeznaczonych do obsługi mapy witryny.

```
python manage.py migrate
```

Powinieneś otrzymać wynik zawierający wiersze:

```
Applying sites.0001_initial... OK
Applying sites.0002_alter_domain_unique... OK
```

W tym momencie aplikacja `sites` jest zsynchronizowana z bazą danych.

Następnie w katalogu aplikacji `blog` utwórz nowy plik i nazwij go *sitemaps.py*. Otwórz go i dodaj do niego kod:

```
from django.contrib.sitemaps import Sitemap
from .models import Post

class PostSitemap(Sitemap):
    changefreq = 'weekly'
    priority = 0.9

    def items(self):
        return Post.published.all()

    def lastmod(self, obj):
        return obj.updated
```

Aby utworzyć mapę witryny, korzystamy z dziedziczenia po klasie `Sitemap` z modułu `sitemaps`. Atrybuty `changefreq` i `priority` wskazują częstotliwość zmian stron posta, a także ich trafność w witrynie internetowej (wartość maksymalna wynosi 1).

Metoda `items()` zwraca kolekcję `QuerySet` obiektów, które mają znaleźć się w tworzonej mapie witryny. Aby pobrać adres URL każdego z obiektów, Django domyślnie wywołuje w każdym z nich metodę `get_absolute_url()`. Przypomnij sobie, że wymienioną metodę utworzyliśmy w rozdziale 2., „Usprawnianie bloga i dodanie funkcjonalności społecznościowych", w celu pobierania kanonicznego adresu URL dla postów. Jeżeli chcesz podać adres URL dla każdego obiektu, możesz dodać do klasy mapy witryny metodę `location`.

Z kolei metoda `lastmod` pobiera każdy obiekt zwrócony przez `items()`, a następnie podaje datę i godzinę ostatniej modyfikacji danego obiektu.

Wymienione wcześniej `changefreq` i `priority` mogą być zarówno metodami, jak i atrybutami. Pełny przewodnik po mapie witryny znajdziesz w oficjalnej dokumentacji Django na stronie *https://docs.djangoproject.com/en/5.0/ref/contrib/sitemaps/*.

Stworzyliśmy mapę witryny. Pozostało już tylko dodanie adresu URL przygotowanej mapy witryny.

W tym celu przeprowadź edycję pliku *urls.py* znajdującego się w katalogu głównym projektu (`mysite`) i dodaj mapę witryny w następujący sposób. Nowe wiersze zostały wyróżnione pogrubioną czcionką.

```
from django.contrib import admin
from django.contrib.sitemaps.views import sitemap
from django.urls import include, path
from blog.sitemaps import PostSitemap

sitemaps = {
    'posts': PostSitemap,
}

urlpatterns = [
     path('admin/', admin.site.urls),
     path('blog/', include('blog.urls', namespace='blog')),
     path(
         'sitemap.xml',
         sitemap,
```

```
            {'sitemaps': sitemaps},
            name='django.contrib.sitemaps.views.sitemap'
        )
]
```

W powyższym kodzie umieściliśmy niezbędne polecenia `import` oraz zdefiniowaliśmy słownik map witryn `sitemaps`. Dla witryny można zdefiniować wiele map witryn. Zdefiniowaliśmy wzorzec adresu URL dopasowujący plik *sitemap.xml* i używający dostarczanego przez Django widoku mapy witryny. Słownik `sitemaps` jest przekazywany do widoku `sitemap`.

W terminalu uruchom serwer programistyczny za pomocą polecenia:

```
python manage.py runserver
```

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/sitemap.xml`. Powinieneś otrzymać kod XML podobny do przedstawionego poniżej.

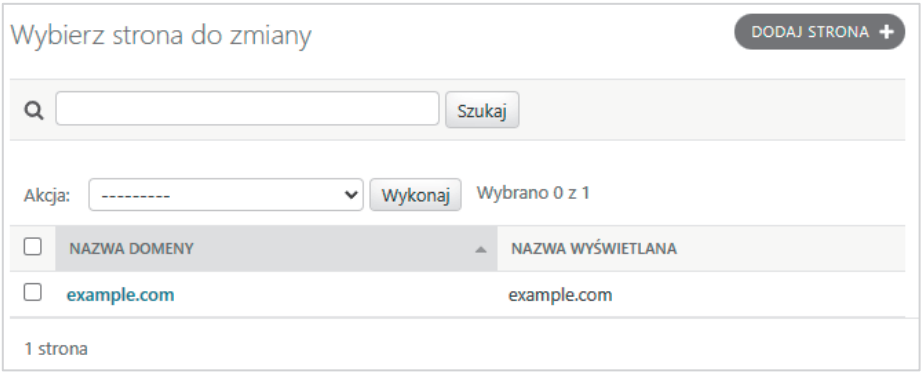
```
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9" xmlns:xhtml="http://
↪www.w3.org/1999/xhtml">
<url>
  <loc>http://example.com/blog/2024/9/21/post-ze-skladnia-markdown/</loc>
  <lastmod>2024-09-21</lastmod>
  <changefreq>weekly</changefreq>
  <priority>0.9</priority>
</url>
<url>
  <loc>http://example.com/blog/2024/9/21/kim-byl-miles-davis/</loc>
  <lastmod>2024-09-21</lastmod>
  <changefreq>weekly</changefreq>
  <priority>0.9</priority>
</url>
<url>
  <loc>http://example.com/blog/2024/8/30/kim-byl-django-reinhardt/</loc>
  <lastmod>2024-08-30</lastmod>
  <changefreq>weekly</changefreq>
  <priority>0.9</priority>
</url>
<url>
  <loc>http://example.com/blog/2024/8/30/uwagi-na-temat-ksiecia-elingtona/</loc>
  <lastmod>2024-09-21</lastmod>
  <changefreq>weekly</changefreq>
  <priority>0.9</priority>
</url>
<url>
  <loc>http://example.com/blog/2024/8/30/nowy-post/</loc>
  <lastmod>2024-08-30</lastmod>
  <changefreq>weekly</changefreq>
  <priority>0.9</priority>
</url>
<url>
  <loc>http://example.com/blog/2024/8/30/django-reinhardt-historia/</loc>
  <lastmod>2024-09-20</lastmod>
  <changefreq>weekly</changefreq>
  <priority>0.9</priority>
</url>
</urlset>
```

Adres URL każdego posta został utworzony za pomocą wywołania metody `get_absolute_url()` obiektu reprezentującego dany post.

Atrybut `lastmod` odpowiada kolumnie daty `updated`, jak wskazaliśmy w mapie witryny. Atrybuty `changefreq` i `priority` zostały pobrane z klasy `PostSitemap`.

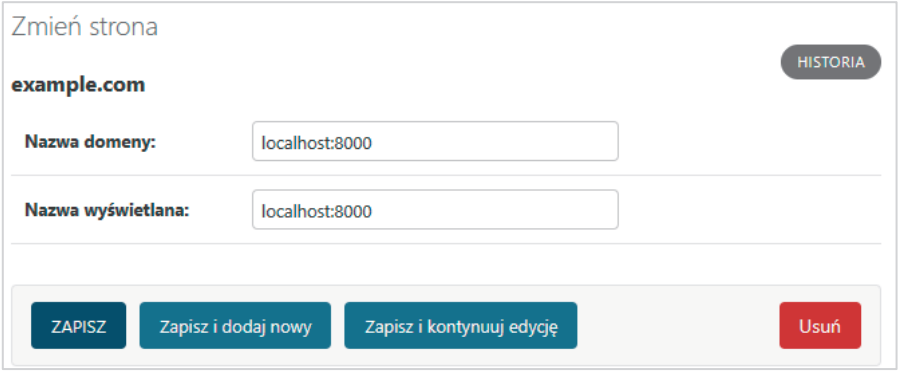
Do zbudowania adresu URL użyliśmy domeny *example.com*. Domena ta pochodzi z obiektu `Site` przechowywanego w bazie danych. Jest to obiekt domyślny utworzony podczas synchronizacji frameworka `sites` z bazą danych. Więcej informacji o frameworku `sites` znajdziesz na stronie *https://docs.djangoproject.com/en/5.0/ref/contrib/sites/*.

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/admin/sites/site/`. Powinieneś zobaczyć widok podobny do pokazanego na rysunku 3.18.



Rysunek 3.18. Widok edycji administracyjnej witryny Django dla modelu Site frameworka sites

Na rysunku 3.18 pokazałem listę widoku administracyjnego dostarczanego przez framework `sites`. W tym miejscu możesz skonfigurować domenę lub host używane przez framework `sites` oraz wykorzystujące go aplikacje. W celu wygenerowania adresu URL istniejącego w środowisku lokalnym nazwę domeny zmień na `localhost:8000`, co pokazałem na rysunku 3.19.



Rysunek 3.19. Widok edycji administracyjnej witryny Django dla modelu Site frameworka sites

Ponownie otwórz w przeglądarce internetowej stronę spod adresu `http://127.0.0.1:8000/sitemap.xml`. Adresy URL wyświetlane w Twoim kanale będą teraz używać nowej nazwy hosta i mieć postać podobną do następującej: `http://localhost:8000/blog/2024/1/22/markdown-post/`. Łącza są teraz dostępne w Twoim środowisku lokalnym. W środowisku produkcyjnym, aby wygenerować bezwzględne adresy URL, będziesz musiał użyć domeny swojej witryny.

Utworzenie kanału wiadomości dla postów bloga

Django zawiera wbudowany framework `feed` pozwalający na dynamiczne generowanie subskrybowanych kanałów wiadomości RSS lub Atom, podobnie jak podczas tworzenia mapy witryny za pomocą frameworka `sites`. Kanał internetowy to dane w określonym formacie (zwykle XML); dostarcza on użytkownikom często aktualizowaną zawartość. Użytkownicy mogą subskrybować Twój kanał za pomocą agregatora kanałów — oprogramowania służącego do czytania kanałów — i otrzymywać powiadomienia o nowych treściach.

W katalogu aplikacji `blog` utwórz nowy plik i nadaj mu nazwę *feeds.py*. Następnie w nowym pliku umieść poniższy fragment kodu.

```
import markdown
from django.contrib.syndication.views import Feed
from django.template.defaultfilters import truncatewords_html
from django.urls import reverse_lazy
from .models import Post


class LatestPostsFeed(Feed):

    title = 'Mój blog'
    link = reverse_lazy('blog:post_list')
    description = 'Nowe posty na moim blogu.'

    def items(self):
        return Post.published.all()[:5]

    def item_title(self, item):
        return item.title

    def item_description(self, item):
        return truncatewords_html(markdown.markdown(item.body), 30)

    def item_pubdate(self, item):
        return item.publish
```

W powyższym kodzie zdefiniowaliśmy kanał informacyjny przez utworzenie podklasy klasy `Feed` frameworka `syndication`. Atrybuty `title`, `link` i `description` odpowiadają elementom — odpowiednio — `<title>`, `<link>` i `<description>` kanału RSS.

Do wygenerowania adresu URL dla atrybutu link użyliśmy metody `reverse_lazy()`. Metoda `reverse()` pozwala na budowanie adresów URL na podstawie opcjonalnych parametrów `name` i `pass`. Z metody `reverse()` skorzystaliśmy w rozdziale 2., „Usprawnianie bloga i dodanie funkcjonalności społecznościowych".

Funkcja narzędziowa `reverse_lazy()` jest wartościowaną leniwie wersją metody `reverse()`. Pozwala na wygenerowanie adresu URL przed załadowaniem konfiguracji URL projektu.

Metoda `items()` pobiera obiekty, które mają być umieszczone w wiadomości kanału. Do uwzględnienia w kanale pobieramy pięć ostatnio opublikowanych postów.

Metody `item_title()` i `item_description()` pobierają wszystkie obiekty zwrócone przez `items()`, a następnie podają — odpowiednio — tytuł i opis pochodzące z danego obiektu.

W metodzie `item_description()` do konwersji treści Markdown na HTML wykorzystaliśmy metodę `markdown()`, natomiast do przygotowania opisu posta składającego się z pierwszych 30 słów wykorzystujemy wbudowany filtr szablonu o nazwie `truncatewords` (w ten sposób eliminujemy możliwość pozostawienia niezamkniętych tagów HTML).

Teraz przeprowadź edycję pliku *blog/urls.py*, zaimportuj utworzoną przed chwilą klasę `LatestPostsFeed`, a następnie utwórz egzemplarz kanału wiadomości w nowym wzorcu adresu URL. Nowe wiersze zostały wyróżnione pogrubioną czcionką:

```
from django.urls import path
from . import views
from .feeds import LatestPostsFeed

app_name = 'blog'

urlpatterns = [
    # Widoki postów
    path('', views.post_list, name='post_list'),
    #  path('', views.PostListView.as_view(), name='post_list'),
    path(
         'tag/<slug:tag_slug>/', views.post_list, name='post_list_by_tag'
    ),
    path(
        '<int:year>/<int:month>/<int:day>/<slug:post>/',
         views.post_detail,
         name='post_detail'
    ),
    path('<int:post_id>/share/', views.post_share, name='post_share'),
    path(
         '<int:post_id>/comment/', views.post_comment, name='post_comment'
    ),
    path('feed/', LatestPostsFeed(), name='post_feed'),
]
```

W przeglądarce internetowej przejdź pod adres *http://127.0.0.1:8000/blog/feed/*. Do uwzględnienia w kanale pobieramy pięć ostatnio opublikowanych postów.

```
<?xml version="1.0" encoding="utf-8"?>
<rss xmlns:atom="http://www.w3.org/2005/Atom" version="2.0">
  <channel>
    <title>Mój blog</title>
```

```
    <link>http://localhost:8000/blog/</link>
    <description>Nowe posty na moim blogu.</description>
    <atom:link href="http://localhost:8000/blog/feed/" rel="self"/>
    <language>pl-pl</language>
    <lastBuildDate>Tue, 02 Jan 2024 16:30:00 +0000</lastBuildDate>
    <item>
      <title>Post sformatowany markdown</title>
      <link>http://localhost:8000/blog/2024/1/2/markdown-post/</link>
      <description>To jest post sformatowany za pomocą...</description>
      <guid>http://localhost:8000/blog/2024/1/2/markdown-post/</guid>
    </item>
     ...
  </channel>
</rss>
```

Jeśli używasz przeglądarki Chrome, zobaczysz kod XML. Jeśli używasz Safari, zostaniesz poproszony o zainstalowanie czytnika kanałów RSS.

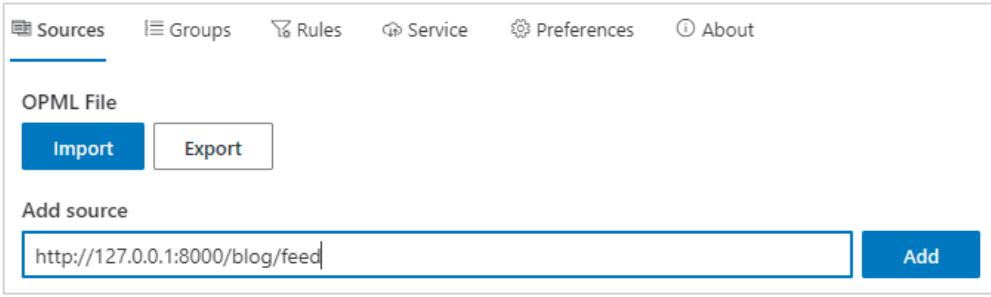
Aby przeglądać kanał RSS za pomocą przyjaznego dla użytkownika interfejsu, zainstalujemy stacjonarnego klienta RSS. Będziemy używać czytnika RSS Fluent Reader, który jest dostępny na wielu platformach.

Pobierz Fluent Reader dla systemu Linux, macOS lub Windows ze strony *https://github.com/yang991178/fluent-reader/releases*.

Zainstaluj czytnik Fluent Reader i go otwórz. Wyświetli się ekran pokazany na rysunku 3.20.

Rysunek 3.20. Fluent Reader bez źródeł kanałów RSS

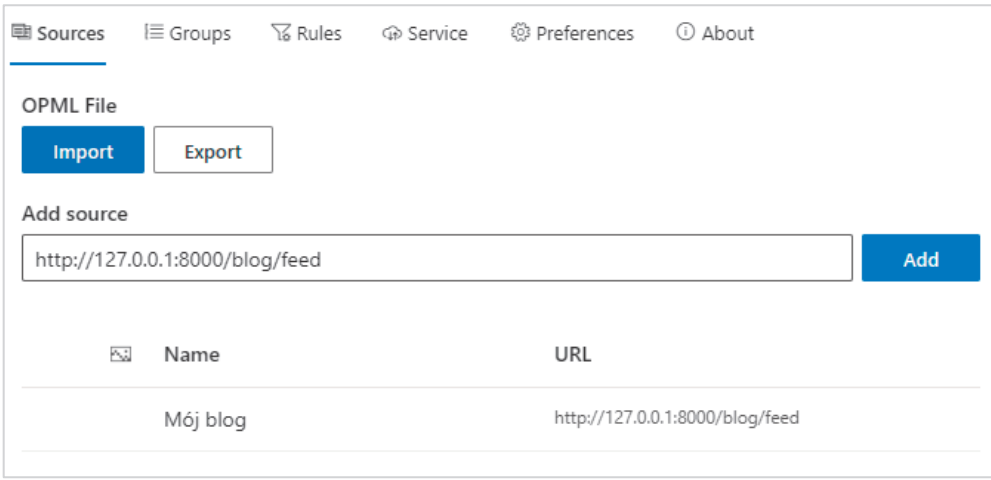
Kliknij ikonę ustawień w prawym górnym rogu okna. Zobaczysz ekran dodawania źródeł kanałów RSS, podobny do pokazanego na rysunku 3.21.



Rysunek 3.21. Dodawanie kanału RSS w programie Fluent Reader

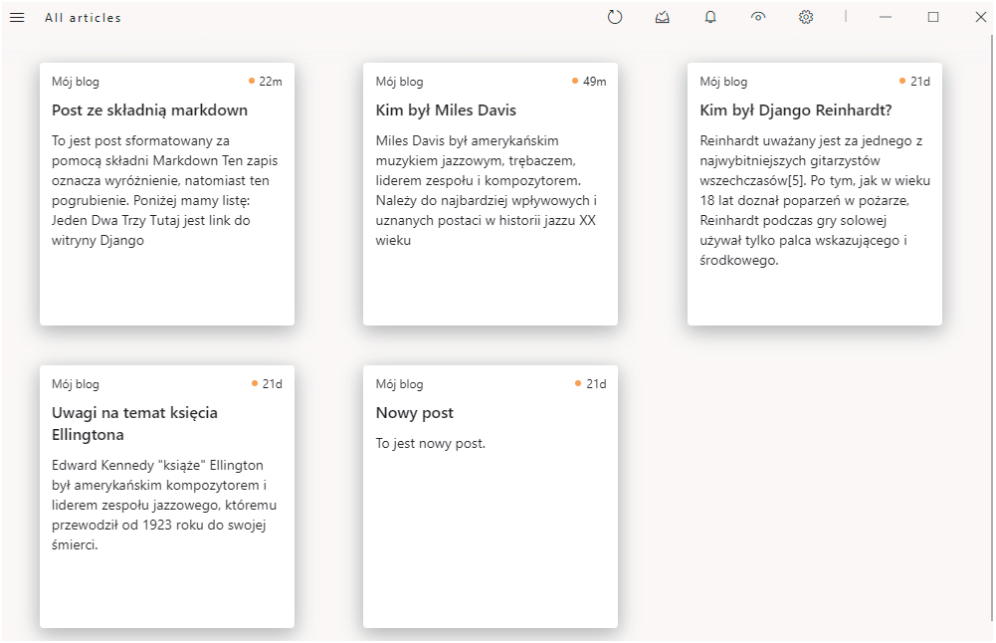
W polu *Add source* wprowadź `http://127.0.0.1:8000/blog/feed/`, a następnie kliknij przycisk *Add*.

W tabeli pod formularzem zobaczysz nowy wpis z kanałem RSS bloga, podobny do pokazanego na rysunku 3.22.



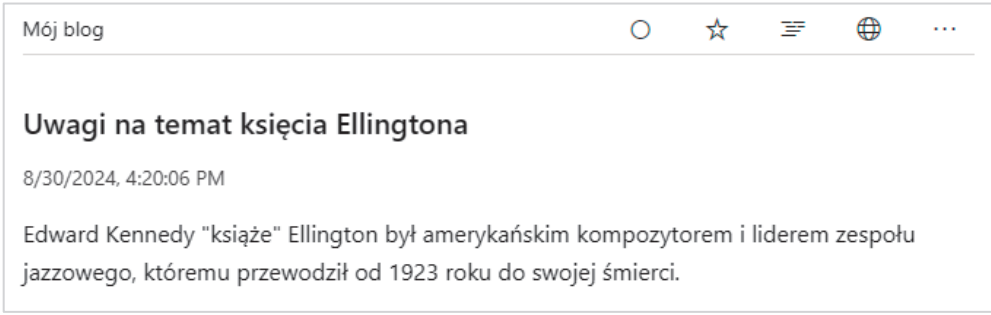
Rysunek 3.22. Dodawanie kanału RSS w programie Fluent Reader

Teraz wróć do głównego ekranu czytnika Fluent Reader. Powinieneś zobaczyć posty zawarte w kanale RSS bloga (rysunek 3.23).



Rysunek 3.23. Kanał RSS bloga wyświetlony w programie Fluent Reader

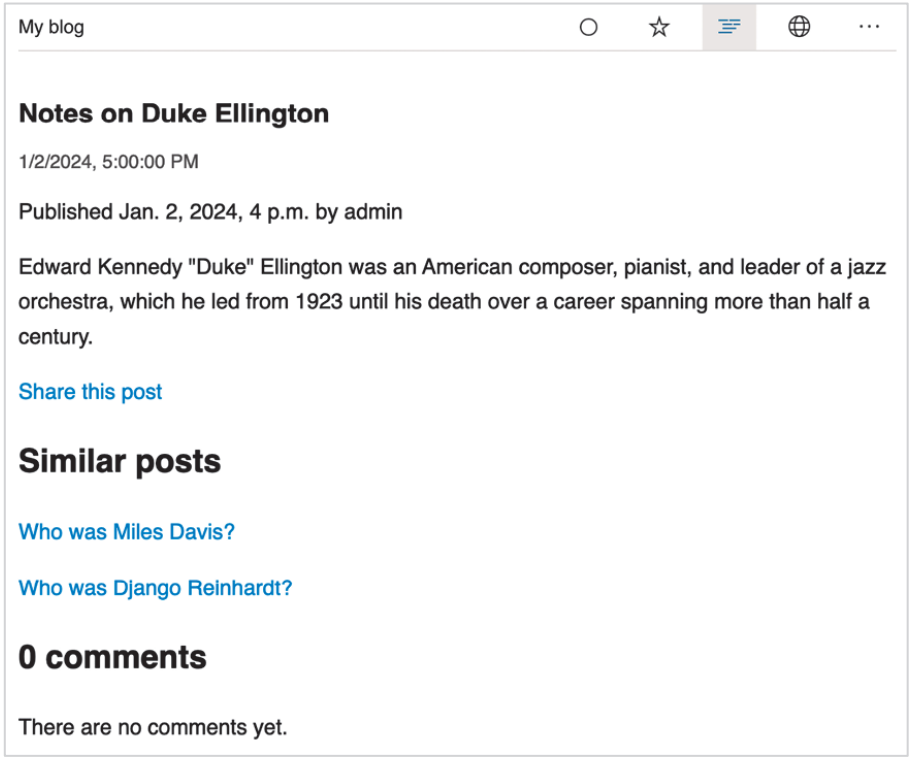
Aby zobaczyć opis, kliknij post (rysunek 3.24).



Rysunek 3.24. Opis posta w czytniku Fluent Reader

Aby załadować pełną zawartość strony posta, kliknij trzecią ikonę w prawym górnym rogu okna (rysunek 3.25).

Ostatnim krokiem jest umieszczenie w pasku bocznym bloga łącza pozwalającego na subskrypcję kanału wiadomości.



Rysunek 3.25. Pełna treść posta wyświetlona w programie Fluent Reader

Otwórz szablon *blog/base.html* i dodaj następujący kod wyróżniony pogrubioną czcionką.

```
{% load blog_tags %}
{% load static %}
<!DOCTYPE html>
<html>
<head>
  <title>{% block title %}{% endblock %}</title>
  <link href="{% static "css/blog.css" %}" rel="stylesheet">
</head>
<body>
  <div id="content">
    {% block content %}
    {% endblock %}
  </div>
  <div id="sidebar">
    <h2>Mój Blog</h2>
    <p>
      To jest mój blog.
      Dotychczas opublikowałem {% total_posts %} postów.
    </p>
    <p>
      <a href="{% url "blog:post_feed" %}">
        Subskrybuj mój kanał RSS
```

```
        </a>
      </p>
      <h3>Najnowsze posty</h3>
      {% show_latest_posts 3 %}
      <h3>Najczęściej komentowane posty</h3>
      {% get_most_commented_posts as most_commented_posts %}
      <ul>
        {% for post in most_commented_posts %}
          <li>
            <a href="{{ post.get_absolute_url }}">{{ post.title }}</a>
          </li>
        {% endfor %}
      </ul>
    </div>
  </body>
  </html>
```

Teraz w przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/blog/` i spójrz na pasek boczny. Kliknięcie nowego linka powinno przenieść Cię do kanału wiadomości postów bloga (rysunek 3.26).

Rysunek 3.26. Link subskrypcji kanału RSS dodany do paska bocznego

Więcej na temat frameworka kanałów RSS w Django możesz przeczytać na stronie *https://docs.djangoproject.com/en/5.0/ref/contrib/syndication/*.

Dodanie do bloga wyszukiwania pełnotekstowego

Przystępujemy teraz do zaimplementowania w aplikacji bloga funkcji wyszukiwania. Wyszukiwanie danych w bazie danych na podstawie danych wprowadzanych przez użytkownika jest częstą funkcjonalnością aplikacji webowych. Warstwa ORM w Django pozwala na przeprowadzanie prostej operacji dopasowywania za pomocą np. filtru `contains` (lub wersji nieuwzględniającej wielkość liter `icontains`). Do wyszukania postów zawierających w treści słowo *framework* można wykorzystać przykładowe poniższe zapytanie:

```
from blog.models import Post
Post.objects.filter(body__contains='framework')
```

Gdy jednak potrzebujesz nieco większych możliwości wyszukiwania — pobierania wyników według podobieństwa lub z wykorzystaniem wag terminów na podstawie częstości występowania w tekście lub istotności poszczególnych pól (na przykład istotności frazy występującej w tytule bądź treści) — będziesz musiał użyć pełnotekstowego silnika wyszukiwania. Kiedy chcesz przeszukiwać duże bloki tekstu, budowanie zapytań z operacjami na ciągach znaków nie wystarcza. Mechanizmy wyszukiwania pełnotekstowego podczas prób dopasowywania kryteriów wyszukiwania pozwalają analizować słowa i porównywać je z przechowywaną zawartością.

Django zapewnia użyteczne funkcje wyszukiwania bazujące na mechanizmach wyszukiwania pełnotekstowego bazy danych PostgreSQL. Funkcjonalności oferowane przez PostgreSQL, które nie są współużytkowane przez inne bazy danych obsługiwane przez Django, udostępnia moduł `django.contrib.postgres`. Więcej informacji na temat mechanizmów wyszukiwania pełnotekstowego PostgreSQL można znaleźć na stronie *https://www.postgresql.org/docs/16/textsearch.html*.

Uwaga

Mimo że Django jest niezależną od bazy danych platformą webową, udostępnia moduł dostarczający fragment bogatego zestawu funkcji oferowanych przez PostgreSQL. Moduł ten jest specyficzny dla PostgreSQL i nie jest dostępny dla innych baz danych obsługiwanych przez Django.

W tym momencie w projekcie `mysite` używamy bazy danych SQLite. W SQLite obsługa wyszukiwania pełnotekstowego jest ograniczona, a Django standardowo go nie zapewnia. Jednak baza danych PostgreSQL jest znacznie lepiej przystosowana do wyszukiwania pełnotekstowego. Aby skorzystać z możliwości wyszukiwania pełnotekstowego PostgreSQL, możemy użyć modułu `django.contrib.postgres`. Aby skorzystać z funkcji wyszukiwania pełnotekstowego, przeprowadzimy migrację danych z SQLite do PostgreSQL.

Uwaga

Baza danych SQLite wystarcza do celów programistycznych. Jednak w środowisku produkcyjnym potrzebna jest baza danych o większych możliwościach, na przykład PostgreSQL, MariaDB, MySQL lub Oracle.

PostgreSQL udostępnia obraz Docker, który bardzo ułatwia wdrożenie serwera PostgreSQL w standardowej konfiguracji.

Instalacja Dockera

Docker to popularna platforma open source do konteneryzacji. Umożliwia programistom pakowanie aplikacji w kontenery, co upraszcza proces tworzenia aplikacji, uruchamiania ich, zarządzania nimi oraz ich dystrybucji.

Najpierw pobierz i zainstaluj Dockera dla swojego systemu operacyjnego. Instrukcje dotyczące pobierania i instalowania Dockera w systemach Linux, macOS i Windows można znaleźć na stronie *https://docs.docker.com/get-docker/*. Instalacja obejmuje zarówno Docker Desktop, jak i narzędzia wiersza poleceń.

Instalacja PostgreSQL

Po zainstalowaniu Dockera na komputerze z systemem Linux, macOS lub Windows można łatwo pobrać obraz Docker bazy danych PostgreSQL. W wierszu polecenia powłoki uruchom polecenie:

```
docker pull postgres:16.2
```

Spowoduje to pobranie obrazu Docker bazy danych PostgreSQL na komputer lokalny. Informacje na temat oficjalnego obrazu Docker bazy danych PostgreSQL można znaleźć pod adresem *https://hub.docker.com/_/postgres*. Inne pakiety i instalatory PostgreSQL można znaleźć na stronie *https://www.postgresql.org/download/*.

Aby uruchomić kontener Docker PostgreSQL, wykonaj w powłoce polecenie:

```
docker run --name=blog_db -e POSTGRES_DB=blog -e POSTGRES_USER=blog -e POSTGRES_
↪PASSWORD=xxxxx -p 5432:5432 -d postgres:16.2
```

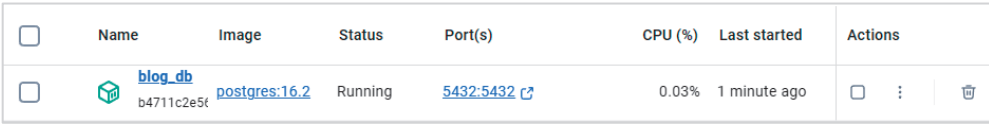
Zastąp `xxxxx` żądanym hasłem użytkownika bazy danych.

To polecenie uruchamia egzemplarz bazy danych PostgreSQL. Opcja `--name` służy do przypisania nazwy do kontenera, w tym przypadku `blog_db`. Opcja `-e` definiuje zmienne środowiskowe egzemplarza. Ustawiamy następujące zmienne środowiskowe:

- `POSTGRES_DB`: Nazwa bazy danych PostgreSQL. Jeśli jej nie zdefiniujesz, w roli nazwy bazy danych zostanie użyta wartość `POSTGRES_USER`.
- `POSTGRES_USER`: używana w połączeniu ze zmienną `POSTGRES_PASSWORD` do zdefiniowania nazwy użytkownika i hasła. Użytkownik jest tworzony z uprawnieniami superużytkownika.
- `POSTGRES_PASSWORD`: Ustawia dla PostgreSQL hasło superużytkownika.

Opcja `-p` służy do opublikowania portu `5432`, na którym działa PostgreSQL, na tym samym porcie interfejsu hosta. Pozwala to na dostęp do bazy danych zewnętrznym aplikacjom. Opcja `-d` oznacza *tryb odłączony*, który uruchamia kontener Docker w tle.

Otwórz aplikację Docker Desktop. Powinieneś zobaczyć uruchomiony nowy kontener, jak na rysunku 3.27.



Rysunek 3.27. Egzemplarz bazy danych PostgreSQL uruchomiony w Docker Desktop

Zobaczysz nowo utworzony kontener `blog_db` ze statusem *Running*. Kolumna *Actions* zawiera przyciski pozwalające zatrzymać lub ponownie uruchomić usługę. Możesz również usunąć kontener. Należy zwrócić uwagę, że usunięcie kontenera spowoduje także usunięcie bazy danych i wszystkich zawartych w niej danych. Sposób utrwalania danych PostgreSQL za pomocą Dockera w lokalnym systemie plików opisuję w rozdziale 17., „Wdrożenie".

Konieczne jest też zainstalowanie adaptera PostgreSQL dla Pythona `psycopg`. Aby go zainstalować, uruchom w powłoce polecenie:

```
python -m pip install psycopg==3.1.18
```

W kolejnym kroku przeprowadzimy migrację istniejących danych z bazy danych SQLite do nowego egzemplarza PostgreSQL.

Zrzucanie istniejących danych

Przed przełączeniem w projekcie Django bazy danych trzeba zrzucić dane dotychczas zapisane w bazie danych SQLite.

Najpierw wyeksportujemy dane, następnie przełączymy bazę danych projektu na PostgreSQL i na koniec zaimportujemy dane do nowej bazy danych.

Django oferuje prosty sposób ładowania i zrzucania danych z bazy danych do plików zwanych **fiksturami** (ang. *fixtures*). Django obsługuje fikstury w formatach JSON, XML lub YAML. Teraz stworzymy fiksturę ze wszystkimi danymi zawartymi w bazie danych.

Do zrzucenia danych z bazy danych na standardowe wyjście, domyślnie serializowane w formacie JSON, skorzystamy z polecenia `dumpdata`. Powstała struktura danych zawiera informacje o modelu i jego polach. Dzięki temu Django może załadować go do bazy danych.

Dane wyjściowe można ograniczyć do wskazanych modeli aplikacji. W tym celu należy przekazać do polecenia nazwy aplikacji lub wskazać pojedyncze modele do wyprowadzania danych przy użyciu formatu `app.Model`. Format możesz określić także za pomocą flagi `--format`. Polecenie `dumpdata` domyślnie wyprowadza zserializowane dane na standardowe wyjście. Można jednak wskazać plik wyjściowy za pomocą flagi `--output`. Flaga `--indent` pozwala określić wcięcia. Więcej informacji na temat parametrów polecenia `dumpdata` można uzyskać za pomocą polecenia `python manage. py dumpdata --help`.

W wierszu polecenia powłoki uruchom polecenie:

```
python manage.py dumpdata --indent=2 --output=mysite_data.json
```

Wszystkie istniejące dane zostały wyeksportowane w formacie JSON do nowego pliku o nazwie *mysite_data.json*. Wyświetl zawartość pliku, aby zobaczyć strukturę JSON

zawierającą wszystkie obiekty danych dla różnych modeli zainstalowanych aplikacji. Jeśli podczas uruchamiania polecenia wystąpi błąd kodowania, aktywuj tryb Python UTF-8 przez dołączenie flagi `-Xutf8` w następujący sposób.

```
python -Xutf8 manage.py dumpdata --indent=2 --output=mysite_data.json
```

Teraz przełączymy bazę danych w projekcie Django, a następnie zaimportujemy dane do nowej bazy danych.

Przełączanie bazy danych w projekcie

Teraz należy dodać konfigurację bazy danych PostgreSQL do ustawień projektu.

Przeprowadź edycję pliku *settings.py* projektu i zmodyfikuj ustawienie `DATABASES` tak, aby wyglądało jak poniżej. Nowy kod został wyróżniony pogrubioną czcionką.

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': config('DB_NAME'),
        'USER': config('DB_USER'),
        'PASSWORD': config('DB_PASSWORD'),
        'HOST': config('DB_HOST'),
    }
}
```

Silnikiem bazy danych jest obecnie `postgresql`. Poświadczenia bazy danych są ładowane ze zmiennych środowiskowych za pomocą polecenia `python-decouple`.

Dodajmy wartości do zmiennych środowiskowych. Przeprowadź edycję pliku *.env* projektu i dodaj następujące wiersze zaznaczone pogrubioną czcionką.

```
EMAIL_HOST_USER=your_account@gmail.com
EMAIL_HOST_PASSWORD=xxxxxxxxxxxx
DEFAULT_FROM_EMAIL=My Blog <your_account@gmail.com>
DB_NAME=blog
DB_USER=blog
DB_PASSWORD=xxxxx
DB_HOST=localhost
```

Zastąp `xxxxxx` hasłem użytym podczas uruchamiania kontenera PostgreSQL. Nowa baza danych jest pusta.

Aby zastosować wszystkie migracje do nowej bazy danych PostgreSQL, uruchom polecenie:

```
python manage.py migrate
```

Zobaczysz wynik zawierający wszystkie zastosowane migracje, podobny do zaprezentowanego poniżej.

```
Operations to perform:
  Apply all migrations: admin, auth, blog, contenttypes, sessions, sites, taggit
  ↪Running migrations:
  Applying contenttypes.0001_initial... OK
```

```
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying taggit.0001_initial... OK
  Applying taggit.0002_auto_20150616_2121... OK
  Applying taggit.0003_taggeditem_add_unique_index... OK
  Applying taggit.0004_alter_taggeditem_content_type_alter_taggeditem_tag... OK
  Applying taggit.0005_auto_20220424_2025... OK
  Applying taggit.0006_rename_taggeditem_content_type_object_id_taggit_tagg_
  ↪content_8fc721_idx... OK
  Applying blog.0001_initial... OK
  Applying blog.0002_alter_post_slug... OK
  Applying blog.0003_comment... OK
  Applying blog.0004_post_tags... OK
  Applying sessions.0001_initial... OK
  Applying sites.0001_initial... OK
  Applying sites.0002_alter_domain_unique... OK
```

Baza danych PostgreSQL jest teraz zsynchronizowana z Twoimi modelami danych. Możesz zatem uruchomić projekt Django wskazujący na nową bazę danych. Doprowadźmy bazę danych do tego samego stanu przez załadowanie danych, które wcześniej wyeksportowaliśmy z bazy danych SQLite.

Ładowanie danych do nowej bazy danych

Zamierzamy załadować wygenerowane wcześniej dane do nowej bazy danych PostgreSQL.

Aby załadować dane do bazy danych PostgreSQL, uruchom polecenie:

```
python manage.py loaddata mysite_data.json
```

Wyświetli się następujący wynik.

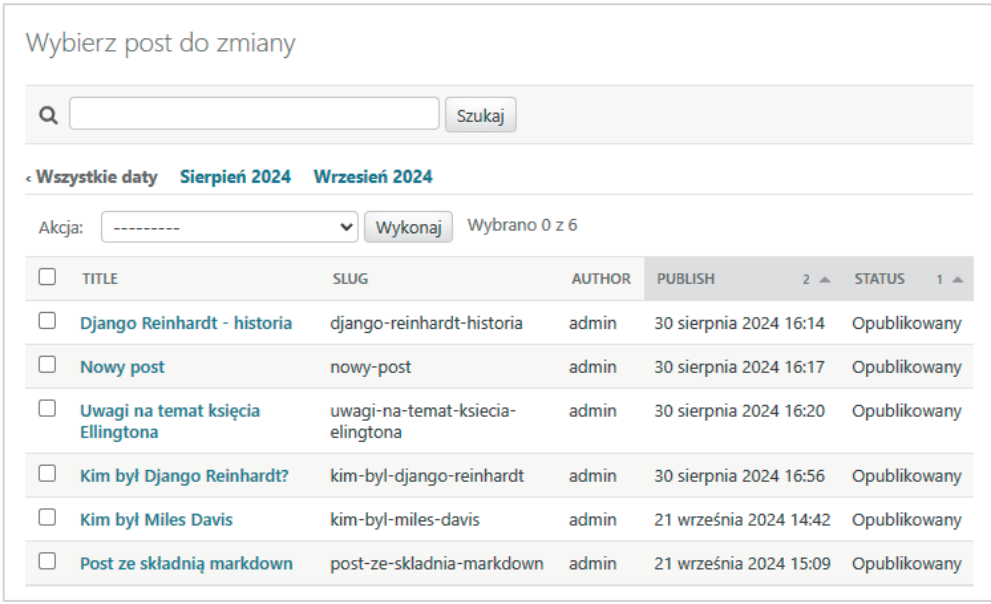
```
Installed 104 object(s) from 1 fixture(s)
```

Liczba obiektów może się różnić w zależności od liczby użytkowników, postów, komentarzy i innych obiektów utworzonych w bazie danych.

W terminalu uruchom serwer programistyczny za pomocą polecenia:

```
python manage.py runserver
```

Aby sprawdzić, czy wszystkie posty zostały załadowane do nowej bazy danych, otwórz w przeglądarce stronę `http://127.0.0.1:8000/admin/blog/post/`. Powinieneś zobaczyć wszystkie posty tak, jak pokazano na rysunku 3.28.



Rysunek 3.28. Lista postów na stronie administracyjnej

Proste wyszukiwania

Po włączeniu w projekcie bazy danych PostgreSQL możesz zbudować złożoną wyszukiwarkę wykorzystującą możliwości wyszukiwania pełnotekstowego PostgreSQL. Zaczniemy od podstawowych wyszukiwań i w celu zbudowania kompleksowej wyszukiwarki pełnotekstowej stopniowo włączymy bardziej zaawansowane funkcje, takie jak stemming, ranking i wagi zapytań.

Wyedytuj plik *settings.py* projektu i dodaj do ustawienia `INSTALLED_APPS` aplikację `django.contrib.postgres` w następujący sposób.

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.sites',
    'django.contrib.sitemaps',
    'django.contrib.staticfiles',
    'django.contrib.postgres',
    'taggit',
    'blog.apps.BlogConfig',
]
```

Przejdź do powłoki Django przez uruchomienie z terminala polecenia:

```
python manage.py shell
```

Teraz możesz wyszukiwać dane w jednym polu przy użyciu wyszukiwania `QuerySet`, tak jak pokazałem poniżej.

W powłoce Pythona uruchom następujący kod.

```
>>> from blog.models import Post
>>> Post.objects.filter(title__search='django')
<QuerySet [<Post: Kim był Django Reinhardt?>]>
```

W tym zapytaniu użyto bazy danych PostgreSQL do utworzenia wektora wyszukiwania dla pola `title` i zapytania wyszukiwania dla hasła *django*. Wyniki są zwracane poprzez dopasowanie zapytania do wektora.

Wyszukiwanie w wielu polach

Istnieje możliwość wyszukiwania z wykorzystaniem wielu pól. W takim przypadku trzeba zdefiniować obiekt `SearchVector`. Zbudujemy wektor, który pozwala wyszukiwać w polach `title` i `body` modelu `Post`.

W powłoce Pythona uruchom kod:

```
>>> from django.contrib.postgres.search import SearchVector
>>> from blog.models import Post
>>>
>>> Post.objects.annotate(
...     search=SearchVector('title', 'body'),
... ).filter(search='django')
<QuerySet [<Post: Markdown post>, <Post: Kim był Django Reinhardt?>]>
```

Wykorzystując metodę `annotate` i definiując obiekt `SearchVector` z dwoma polami, zapewniliśmy funkcjonalność dopasowania zapytania zarówno do tytułu, jak i treści postów.

Uwaga

Wyszukiwanie pełnotekstowe to intensywny proces. Jeśli szukasz więcej niż kilkuset wierszy, powinieneś zdefiniować indeks funkcjonalny odpowiadający używanemu wektorowi wyszukiwania. Django dostarcza dla modeli pola `SearchVectorField`. Więcej informacji na ten temat można znaleźć na stronie *https://docs.djangoproject.com/en/5.0/ref/contrib/postgres/search*.

Utworzenie widoku wyszukiwania

Przystępujemy teraz do utworzenia widoku wyszukiwania, aby umożliwić użytkownikom przeszukiwanie postów. Przede wszystkim potrzebujemy formularza wyszukiwania. Przeprowadź edycję pliku *forms.py* aplikacji blog i umieść w nim poniższy fragment kodu.

```
class SearchForm(forms.Form):
    query = forms.CharField()
```

Kolumnę `query` wykorzystamy w celu umożliwienia użytkownikom podania szukanego wyrażenia. Przeprowadź edycję pliku *views.py* aplikacji `blog` i dodaj w nim poniższy fragment kodu.

```
# ...
from django.contrib.postgres.search import SearchVector
from .forms import CommentForm, EmailPostForm, SearchForm

#...

def post_search(request):
    form = SearchForm()
    query = None
    results = []

    if 'query' in request.GET:
        form = SearchForm(request.GET)
        if form.is_valid():
            query = form.cleaned_data['query']
            results = (
                Post.published.annotate(
                    search=SearchVector('title', 'body'),
                )
                .filter(search=query)
            )

    return render(
        request,
        'blog/post/search.html',
        {
            'form': form,
            'query': query,
            'results': results
        },
    )
```

W tym widoku najpierw tworzymy egzemplarz klasy `SearchForm`. Aby ustalić, czy formularz został wysłany, szukamy parametru `query` w słowniku `request.GET`. Formularz wysyłamy za pomocą metody `GET` zamiast `POST`, dlatego wynikowy adres URL zawiera parametr `query` i można go łatwo współdzielić. Po wysłaniu formularza tworzymy jego egzemplarz wraz z danymi przekazanymi w żądaniu `GET` i sprawdzamy ich poprawność. Jeśli formularz jest prawidłowy, wyszukujemy posty z wykorzystaniem niestandardowego egzemplarza obiektu `SearchVector` zbudowanego na podstawie pól `title` i `body`.

Widok wyszukiwania jest gotowy. Musimy jeszcze przygotować szablon przeznaczony do wyświetlenia formularza oraz wyników po przeprowadzeniu operacji wyszukiwania.

W katalogu *templates/blog/post/* utwórz nowy plik o nazwie *search.html*, a następnie umieść w nim poniższy fragment kodu.

```
{% extends "blog/base.html" %}
{% load blog_tags %}

{% block title %}Szukaj{% endblock %}
```

```
{% block content %}
  {% if query %}
    <h1>Posty zawierające "{{ query }}"</h1>
    <h3>
      {% with results.count as total_results %}
        Znaleziono {{ total_results }} wyników
      {% endwith %}
    </h3>
    {% for post in results %}
      <h4>
        <a href="{{ post.get_absolute_url }}">
          {{ post.title }}
        </a>
      </h4>
      {{ post.body|markdown|truncatewords_html:12 }}
    {% empty %}
      <p>Brak wyników dla podanego zapytania.</p>
    {% endfor %}
    <p><a href="{% url "blog:post_search" %}">Szukaj ponownie</a></p>
  {% else %}
    <h1>Szukaj postów</h1>
    <form method="get">
      {{ form.as_p }}
      <input type="submit" value="Szukaj">
    </form>
  {% endif %}
{% endblock %}
```

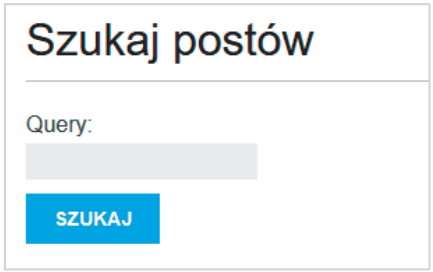
Podobnie jak w widoku wyszukiwania, także tutaj ustalenie, czy formularz został wysłany, opiera się na istnieniu parametru `query`. Gdy brakuje parametru `query`, wyświetlamy formularz oraz przycisk pozwalający na jego wysłanie. Jeśli natomiast parametr `query` istnieje, oznacza to, że formularz został wysłany. Po przesłaniu formularza wyszukiwania wyświetlamy wykonane zapytanie, całkowitą liczbę wyników oraz listę postów pasujących do zapytania.

Na końcu przeprowadź edycję pliku *urls.py* aplikacji `blog` i dodaj w nim poniższy wzorzec adresu URL oznaczony pogrubieniem.

```
urlpatterns = [
    # Widoki postów
    path('', views.post_list, name='post_list'),
    # path('', views.PostListView.as_view(), name='post_list'),
    path(
        'tag/<slug:tag_slug>/', views.post_list, name='post_list_by_tag'
    ),
    path(
        '<int:year>/<int:month>/<int:day>/<slug:post>/',
        views.post_detail,
        name='post_detail',
    ),
    path('<int:post_id>/share/', views.post_share, name='post_share'),
    path(
        '<int:post_id>/comment/', views.post_comment, name='post_comment'
    ),
```

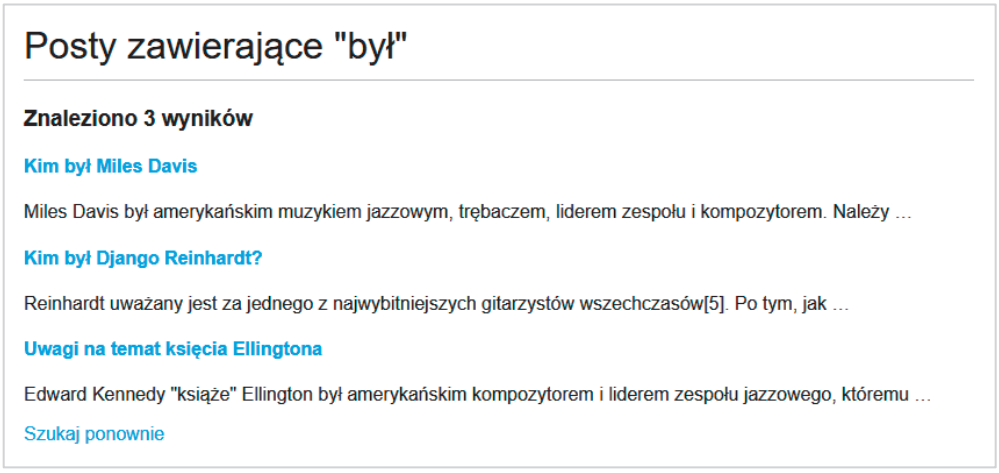
```
    path('feed/', LatestPostsFeed(), name='post_feed'),
    path('search/', views.post_search, name='post_search'),
]
```

Teraz w przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/blog/search/`. Powinieneś zobaczyć wyświetlony formularz wyszukiwania (rysunek 3.29).



Rysunek 3.29. Formularz z polem query do wyszukiwania postów

Wpisz szukane wyrażenie i kliknij przycisk *Szukaj*. Powinieneś otrzymać wyniki wyszukiwania, co pokazałem na rysunku 3.30.



Rysunek 3.30. Wyniki wyszukiwania hasła „jazz”

Gratulacje. Właśnie utworzyłeś prosty silnik wyszukiwania dla swojego bloga.

Stemming i ranking wyników

Stemming to proces redukowania słów do tematu lub formy podstawowej. Jest wykorzystywany przez wyszukiwarki w celu zmniejszenia liczby indeksowanych słów do form podstawowych i utworzenia możliwości dopasowywania słów pochodnych. Przykładowo słowa „muzyka” i „muzykant” mogą być uznane przez wyszukiwarki za podobne. Proces stemmingu normalizuje każdy token wyszukiwania w leksem, jednostkę znaczenia leksykalnego,

która leży u podstaw zestawu słów powiązanych przez fleksję. Podczas tworzenia zapytania wyszukiwania słowa „muzyka", „muzykant" i „muzykalność" zostaną zamienione na słowo „muzyka".

Django udostępnia klasę `SearchQuery`, która umożliwia tłumaczenie terminów na obiekt zapytania. Domyślnie terminy są przekazywane przez algorytmy stemmingu, które pomagają uzyskać lepsze dopasowania.

Wyszukiwarka PostgreSQL usuwa również słowa z tzw. **stoplisty** (ang. *stop words*). W języku angielskim są to, na przykład, takie słowa jak „a", „the", „on" i „of". Stoplista zawiera zbiór często używanych słów w języku. Są one usuwane podczas tworzenia zapytania wyszukiwania, ponieważ pojawiają się zbyt często, aby zawierające je wyniki wyszukiwania były trafne. Stoplistę wykorzystywaną przez PostgreSQL dla języka angielskiego można znaleźć pod adresem *https://github.com/postgres/postgres/blob/master/src/backend/snowball/stopwords/english.stop*.

Można również uporządkować wyniki według trafności. PostgreSQL zapewnia funkcję rankingu, która porządkuje wyniki na podstawie częstotliwości ich pojawiania się oraz podobieństwa.

W pliku *views.py* swojej aplikacji bloga dodaj instrukcje importu:

```
from django.contrib.postgres.search import (
     SearchVector,
     SearchQuery,
     SearchRank
)
```

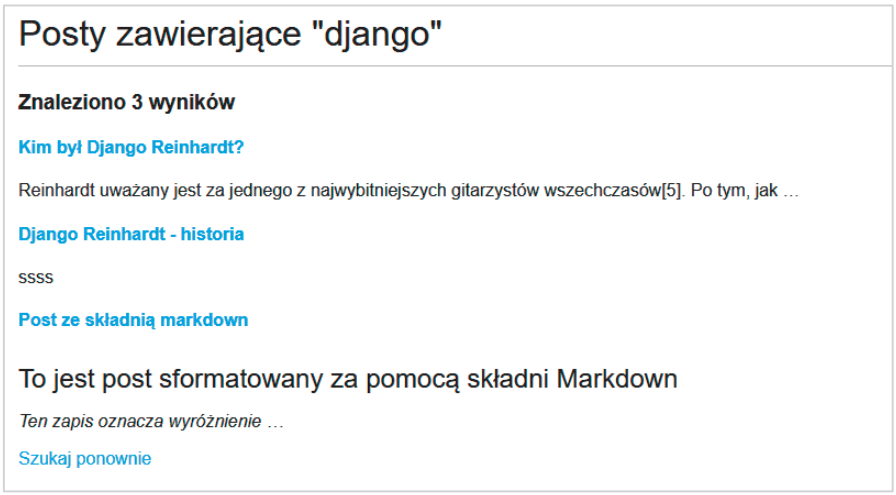
Następnie zmodyfikuj widok `post_search` w poniższy sposób. Nowy kod został wyróżniony pogrubioną czcionką.

```
def post_search(request):
    form = SearchForm()
    query = None
    results = []

    if 'query' in request.GET:
        form = SearchForm(request.GET)
        if form.is_valid():
            query = form.cleaned_data['query']
            search_vector = SearchVector('title', 'body')
            search_query = SearchQuery(query)
            results = (
                 Post.published.annotate(
                     search=search_vector,
                     rank=SearchRank(search_vector, search_query)
                 )
                 .filter(search=search_query)
                 .order_by('-rank')
            )
    return render(
        request,
        'blog/post/search.html',
        {
```

```
            'form': form,
            'query': query,
            'results': results
        }
    )
```

W powyższym kodzie utworzyliśmy obiekt `SearchQuery`, przefiltrowaliśmy wyniki z wykorzystaniem tego obiektu i użyliśmy obiektu `SearchRank`, aby uporządkować wyniki według trafności.

Żeby sprawdzić funkcje stemmingu i rankingu, możesz teraz otworzyć w przeglądarce stronę `http://127.0.0.1:8000/blog/search/` i przetestować różne wyszukiwania. Na rysunku 3.31 jest pokazany przykład rankingu według liczby wystąpień słowa *django* w tytule i treści postów.



Rysunek 3.31. Wyniki wyszukiwania hasła „django"

Stemming i usuwanie słów ze stoplisty dla różnych języków

Obiekty `SearchVector` i `SearchQuery` można skonfigurować w taki sposób, aby wykonywać stemming i usuwać słowa ze stoplisty w dowolnym języku. Aby użyć innej konfiguracji wyszukiwania, możemy przekazać do obiektów `SearchVector` i `SearchQuery` atrybut `config`. Dzięki temu możemy korzystać z różnych parserów i słowników językowych. Poniższy przykład wykonuje stemming i usuwa słowa ze stoplisty dla języka hiszpańskiego:

```
search_vector = SearchVector('title', 'body', config='spanish')
search_query = SearchQuery(query, config='spanish')
results = (
     Post.published.annotate(
         search=search_vector,
         rank=SearchRank(search_vector, search_query)
    )
```

```
        .filter(search=search_query)
        .order_by('-rank')
    )
```

Słownik słów ignorowanych używany przez PostgreSQL dla języka hiszpańskiego można znaleźć pod adresem *https://github.com/postgres/postgres/blob/master/src/backend/snowball/stopwords/spanish.stop*.

Wagi zapytań

Możesz poprawić dokładność niektórych wektorów, kiedy będziesz przypisywać im większą wagę przy porządkowaniu wyników według trafności. Można np. zastosować ten mechanizm, by nadać większą wagę postom, które są dopasowane według tytułu, a nie treści.

Wyedytuj plik *views.py* aplikacji `blog` i zmodyfikuj widok `post_search` w następujący sposób. Nowy kod został wyróżniony pogrubioną czcionką.

```
def post_search(request):
    form = SearchForm()
    query = None
    results = []

    if 'query' in request.GET:
        form = SearchForm(request.GET)
        if form.is_valid():
            query = form.cleaned_data['query']
            search_vector = SearchVector(
                'title', weight='A'
            ) + SearchVector('body', weight='B')
            search_query = SearchQuery(query)
            results = (
                 Post.published.annotate(
                     search=search_vector,
                     rank=SearchRank(search_vector, search_query)
                 )
                .filter(rank__gte=0.3)
                .order_by('-rank')
            )
    return render(
         request,
         'blog/post/search.html',
        {
            'form': form,
            'query': query,
            'results': results
        }
    )
```

W powyższym kodzie zastosowaliśmy różne wagi do wektorów wyszukiwania zbudowanych przy użyciu pól `title` i `body`. Domyślne wagi to `D`, `C`, `B` i `A`. Odpowiadają im liczby, kolejno `0.1`, `0.2`, `0.4` i `1.0`. Zastosowaliśmy wagę `1.0` dla wektora wyszukiwania pola `title` (`A`) oraz wagę `0.4` dla wektora wyszukiwania treści (`B`). Dopasowania tytułów będą dominować nad dopasowaniami treści. Wyniki filtrujemy tak, aby wyświetlać tylko te z rangą wyższą niż `0.3`.

Wyszukiwanie z podobieństwem trygramu

Inną techniką wyszukiwania jest stosowanie podobieństwa trygramów. Trygram to grupa trzech kolejnych znaków. Zliczając wspólne trygramy w dwóch łańcuchach znaków, można zmierzyć ich podobieństwo. Takie podejście okazuje się bardzo skuteczne w mierzeniu podobieństwa słów w wielu językach.

Aby używać techniki trygramów w PostgreSQL, trzeba najpierw zainstalować rozszerzenie `pg_trgm`. Framework Django oferuje operacje migracji bazy danych pozwalające na tworzenie rozszerzeń PostgreSQL. Dodajmy migrację, która utworzy rozszerzenie w bazie danych.

Aby utworzyć pustą migrację, w wierszu polecenia powłoki wykonaj polecenie:

```
python manage.py makemigrations --name=trigram_ext --empty blog
```

Spowoduje to utworzenie pustej migracji dla aplikacji `blog`. Wyświetli się taki wynik:

```
Migrations for 'blog':
   blog/migrations/0005_trigram_ext.py
```

Przeprowadź edycję pliku *blog/migrations/0005_trigram_ext.py* i dodaj następujące wiersze zaznaczone pogrubioną czcionką.

```
from django.contrib.postgres.operations import TrigramExtension
from django.db import migrations


class Migration(migrations.Migration):

    dependencies = [
         ('blog', '0004_post_tags'),
     ]

    operations = [
        TrigramExtension()
    ]
```

Dodaliśmy do migracji bazy danych operację `TrigramExtension`. Aby utworzyć rozszerzenie w PostgreSQL, ta operacja wykonuje polecenie SQL `CREATE EXTENSION pg_trgm`.

Więcej informacji na temat operacji migracji bazy danych można znaleźć na stronie *https://docs.djangoproject.com/en/5.0/ref/contrib/postgres/operations/*.

Teraz wykonaj migrację za pomocą polecenia:

```
python manage.py migrate blog
```

Wyświetli się taki oto wynik:

```
Running migrations:
  Applying blog.0005_trigram_ext... OK
```

W bazie danych zostało utworzone rozszerzenie `pg_trgm`. Spróbujmy zmodyfikować widok `post_search`, aby wyszukiwał trygramy.

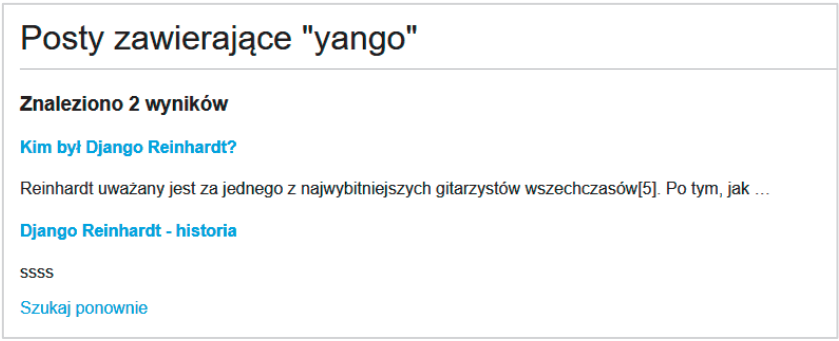
Przeprowadź edycję pliku *views.py* swojej aplikacji bloga i dodaj instrukcje importu:

```
from django.contrib.postgres.search import TrigramSimilarity
```

Następnie zmodyfikuj widok `post_search` w poniższy sposób. Nowy kod został wyróżniony pogrubioną czcionką.

```
def post_search(request):
    form = SearchForm()
    query = None
    results = []

    if 'query' in request.GET:
        form = SearchForm(request.GET)
        if form.is_valid():
            query = form.cleaned_data['query']
            results = (
                Post.published.annotate(
                    similarity=TrigramSimilarity('title', query),
                )
                .filter(similarity__gt=0.1)
                .order_by('-similarity')
            )

    return render(
        request,
        'blog/post/search.html',
        {
            'form': form,
            'query': query,
            'results': results
        },
    )
```

Otwórz w przeglądarce stronę `http://127.0.0.1:8000/blog/search/` i przetestuj różne wyszukiwania pod kątem działania trygramów. Na rysunku 3.32 pokazano wyszukiwanie możliwej literówki w haśle *django* — wyniki wyszukiwania terminu *yango*.



Rysunek 3.32. Wyniki wyszukiwania hasła „yango"

Właśnie utworzyłeś w swoim projekcie potężny silnik wyszukiwania.

Więcej informacji na temat wyszukiwania pełnotekstowego w Django i PostgreSQL można znaleźć na stronie *https://docs.djangoproject.com/en/5.0/ref/contrib/postgres/search/*.

Podsumowanie

W tym rozdziale zaimplementowałeś system tagowania przez zintegrowanie ze swoim projektem zewnętrznej aplikacji. Wygenerowałeś także rekomendacje postów za pomocą złożonych kolekcji QuerySet. Dowiedziałeś się również, jak w Django utworzyć własne znaczniki szablonu i filtry, aby tym samym dodać do szablonów niestandardowe możliwości. Przygotowaliśmy mapę witryny dla robotów silników wyszukiwania oraz kanał wiadomości RSS, który może być subskrybowany przez użytkowników. Na końcu zaimplementowaliśmy silnik wyszukiwania dla bloga z wykorzystaniem pełnotekstowego silnika wyszukiwania dla PostgreSQL.

W następnym rozdziale zobaczysz, jak utworzyć witrynę społecznościową za pomocą frameworka uwierzytelniania w Django oraz jak zaimplementować funkcjonalności obsługi kont użytkowników i niestandardowe profile użytkowników.

Rozszerzanie projektu z wykorzystaniem sztucznej inteligencji

Po ukończeniu aplikacji bloga prawdopodobnie masz wiele pomysłów na dodanie do swojego bloga nowych funkcjonalności. Ten podrozdział ma na celu dostarczenie pewnych informacji na temat odkrywania nowych funkcjonalności, które można włączyć do projektu za pomocą ChatGPT. ChatGPT to stworzony przez OpenAI **zaawansowany model językowy AI** (ang. *Large Language Model* — LLM), który na podstawie otrzymywanych zapytań generuje odpowiedzi podobne do udzielanych przez ludzi. W tym podrozdziale zaprezentuję zadanie rozszerzenia projektu, w którym wykorzystam przykładowe zapytanie do ChatGPT.

Aby zapoznać się z ChatGPT, odwiedź stronę *https://chat.openai.com/*. Podobne wskazówki znajdziesz po ukończeniu każdego projektu Django w tej książce, w rozdziale 7., „Śledzenie działań użytkownika", rozdziale 11., „Internacjonalizacja sklepu internetowego", i rozdziale 17., „Wdrożenie".

Spróbuj ulepszyć blog z wykorzystaniem ChatGPT. Obecnie Twój blog umożliwia filtrowanie postów według tagów. Dodanie tagów do mapy witryny pozwala znacznie poprawić optymalizację SEO bloga. Aby do mapy witryny dodać strony tagów, użyj zapytania dostępnego na stronie *https://github.com/PacktPublishing/Django-5-by-example/blob/main/Chapter03/prompts/task.md*. To zadanie jest doskonałą okazją, by odświeżyć swój projekt i pogłębić wiedzę o frameworku Django, a jednocześnie pozwala nauczyć się interakcji z ChatGPT.

Uwaga

ChatGPT jest przygotowany do udzielania pomocy w kwestiach związanych z kodem. Wystarczy udostępnić swój kod wraz z napotkanymi błędami, a ChatGPT pomoże zidentyfikować problemy i je rozwiązać.

Dodatkowe zasoby

Poniższe źródła zawierają dodatkowe informacje związane z tematami omawianymi w tym rozdziale.

- Kod źródłowy przykładów zamieszczonych w tym rozdziale — *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter03*.
- `django-taggit` — *https://github.com/jazzband/django-taggit*.
- Menedżery ORM `django-taggit` — *https://django-taggit.readthedocs.io/en/latest/api.html*.
- Relacje „wiele do wielu" — *https://docs.djangoproject.com/en/5.0/topics/db/examples/many_to_many/*.
- Funkcje agregacji frameworka Django — *https://docs.djangoproject.com/en/5.0/topics/db/aggregation/*.
- Wbudowane tagi szablonów i filtry — *https://docs.djangoproject.com/en/5.0/ref/templates/builtins/*.
- Pisanie niestandardowych tagów szablonów — *https://docs.djangoproject.com/en/5.0/howto/custom-template-tags/*.
- Dokumentacja formatu Markdown — *https://daringfireball.net/projects/markdown/basics*.
- Framework Django `sitemap` — *https://docs.djangoproject.com/en/5.0/ref/contrib/sitemaps/*.
- Framework Django `sites` — *https://docs.djangoproject.com/en/5.0/ref/contrib/sites/*.
- Framework kanałów informacyjnych syndication Django — *https://docs.djangoproject.com/en/5.0/ref/contrib/syndication/*.
- Instrukcje pobierania i instalacji Dockera — *https://docs.docker.com/get-docker/*.
- Obraz Docker bazy danych PostgreSQL — *https://hub.docker.com/_/postgres*.
- Strona pobierania bazy danych PostgreSQL — *https://www.postgresql.org/download/*.
- Możliwości wyszukiwania pełnotekstowego PostgreSQL — *https://www.postgresql.org/docs/16/textsearch.html*.
- Operacje migracji bazy danych — *https://docs.djangoproject.com/en/5.0/ref/contrib/postgres/operations/*.

- Obsługa wyszukiwania pełnotekstowego w PostgreSQL przez Django — *https://docs.djangoproject.com/en/5.0/ref/contrib/postgres/search/*.
- Interfejs ChatGPT — *https://chat.openai.com/*.
- Przykładowe zapytanie do ChatGPT o dodanie stron tagów do mapy witryny — *https://github.com/PacktPublishing/Django-5-by-example/blob/main/Chapter03/prompts/task.md*.

Utworzenie witryny społecznościowej

Rozdział 4

W poprzednim rozdziale nauczyłeś się, jak zaimplementować system tagowania i polecać podobne posty. Zaimplementowałeś niestandardowe tagi szablonów i filtry. Nauczyłeś się również, jak tworzyć mapy witryn i kanały wiadomości, a także zbudowałeś — za pomocą PostgreSQL — pełnotekstową wyszukiwarkę.

W tym rozdziale dowiesz się, jak opracować funkcjonalności związane z obsługą kont — w tym rejestrację użytkownika, zarządzanie hasłami, edycję profilu i uwierzytelnianie — w celu stworzenia serwisu społecznościowego. W następnych kilku rozdziałach zaimplementujemy w tej witrynie funkcje społecznościowe, pozwalające użytkownikom współdzielenie zdjęć i interakcje między sobą. Użytkownicy będą mogli oznaczyć w internecie dowolne zdjęcie i udostępnić je innym. Będą również mogli zobaczyć aktywność w internecie użytkowników, których obserwują, oraz polubić udostępniane przez nich zdjęcia (lub wyrazić dezaprobatę na ich temat).

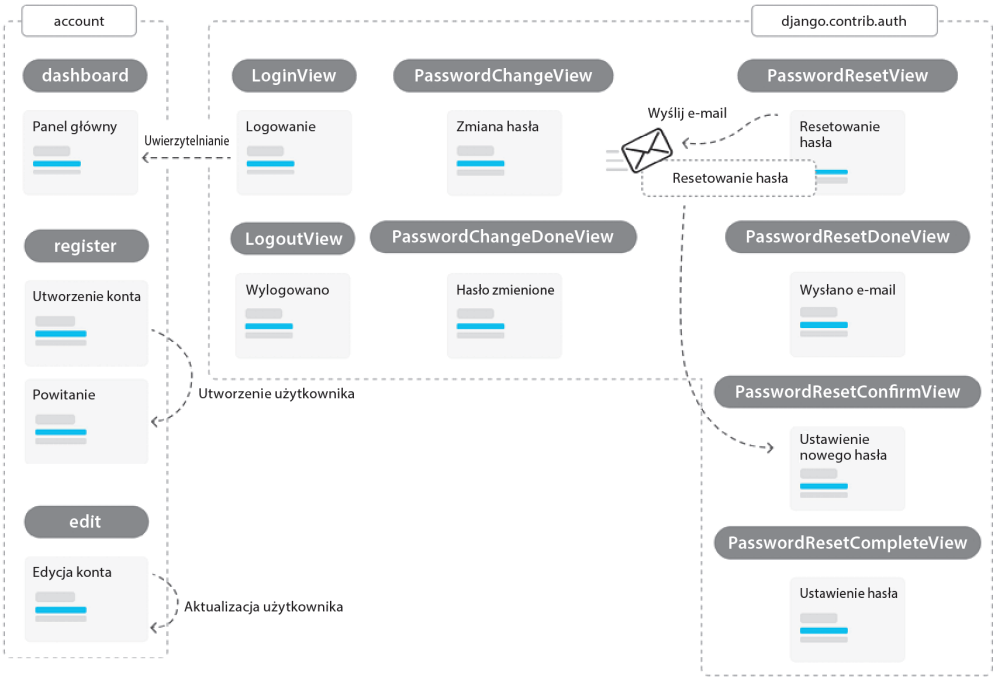
Oto zagadnienia, na których skoncentruję się w tym rozdziale:

- Utworzenie widoku logowania
- Użycie frameworka uwierzytelniania w Django
- Tworzenie w Django szablonów widoków logowania, wylogowania, zmiany hasła i resetowania hasła
- Utworzenie widoków pozwalających na rejestrację użytkowników
- Rozbudowa modelu `User` o obsługę niestandardowego profilu
- Konfigurowanie projektu do przesyłania plików multimedialnych

Przegląd funkcjonalności

Reprezentację widoków, szablonów i funkcjonalności, które zbudujemy w tym rozdziale, przedstawiono na rysunku 4.1.

W tym rozdziale stworzysz nowy projekt i użyjesz widoków logowania, wylogowania, zmiany hasła i odzyskiwania hasła dostarczanych przez Django w pakiecie `django.contrib.auth`. Utworzysz szablony widoków uwierzytelniania, a także widok pulpitu nawigacyjnego, do którego użytkownicy będą mieli dostęp po pomyślnym uwierzytelnieniu. Zaimplementujesz również mechanizm rejestracji użytkowników za pomocą widoku rejestracji (`register`). Na koniec rozszerzysz model użytkownika o niestandardowy model `Profile` i utworzysz widok edycji, by umożliwić użytkownikom edytowanie ich profili.



Rysunek 4.1. Schemat funkcjonalności zbudowanych w rozdziale 4.

Kod źródłowy przykładów z tego rozdziału można znaleźć pod adresem *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter04*.

Wszystkie pakiety Pythona użyte w tym rozdziale są zawarte w pliku *requirements.txt* dołączonym do plików z kodem źródłowym przykładów z tego rozdziału. Aby zainstalować każdy pakiet Pythona, możesz postępować zgodnie z instrukcjami zamieszczonymi w poniższych podrozdziałach. Możesz także zainstalować wszystkie wymagania jednocześnie za pomocą polecenia `pip install -r requirements.txt`.

Utworzenie projektu witryny społecznościowej

Przystępujemy teraz do budowy aplikacji społecznościowej umożliwiającej użytkownikom udostępnianie zdjęć znalezionych w internecie. Ten projekt jest istotny. Wykonanie go pomoże Ci zrozumieć, jak wbudować w witrynę funkcje społecznościowe, a także jak implementować zaawansowane funkcjonalności za pomocą frameworka Django i Java Script.

W ramach witryny do udostępniania zdjęć trzeba będzie zbudować komponenty:

- System uwierzytelniania pozwalający użytkownikowi na rejestrowanie, logowanie, edycję profilu oraz zmianę i zerowanie hasła.
- Uwierzytelnianie społecznościowe do logowania się za pomocą takich usług jak Google.

- Funkcjonalność pozwalającą na wyświetlanie udostępnianych zdjęć oraz udostępnianie obrazów z niemalże każdej witryny internetowej.
- Strumień aktywności dla każdego użytkownika pozwalający użytkownikom śledzić treść dodawaną przez obserwowanych użytkowników.
- System obserwacji pozwalający użytkownikom na śledzenie swoich poczynań.

W tym rozdziale zajmiemy się realizacją pierwszego z wymienionych punktów. Pozostałe punkty omówimy w rozdziałach od 5. do 7.

Rozpoczęcie pracy nad aplikacją społecznościową

Zaczniemy od skonfigurowania środowiska wirtualnego dla projektu i utworzenia początkowej struktury projektu.

Przejdź do powłoki i wydaj poniższe polecenia w celu utworzenia środowiska wirtualnego dla projektu, a następnie jego aktywacji.

```
mkdir env
python -m venv env/bookmarks
```

Jeśli używasz systemu Linux lub macOS, aby aktywować środowisko wirtualne, uruchom polecenie:

```
source env/bookmarks/bin/activate
```

Jeśli używasz systemu Windows, skorzystaj z polecenia:

```
.\env\bookmarks\Scripts\activate
```

Znak zachęty w powłoce wyświetla nazwę aktywnego środowiska wirtualnego, co pokazałem poniżej.

```
(bookmarks)laptop:~ zenx$
```

W przygotowanym środowisku wirtualnym zainstaluj framework Django, wydając poniższe polecenie.

```
python -m pip install Django~=5.0.4
```

Aby utworzyć nowy projekt, wydaj poniższe polecenie.

```
django-admin startproject bookmarks
```

W ten sposób zbudujemy nowy projekt Django o nazwie `bookmarks` wraz z początkową strukturą plików i katalogów. Teraz przejdź do nowego katalogu projektu i utwórz nową aplikację o nazwie `account`, wydając poniższe polecenia.

```
cd bookmarks/
django-admin startapp account
```

Pamiętaj, by dodać aplikację do projektu; zrobisz to, wpisując ją na listę `INSTALLED_APPS` w pliku *settings.py*.

Przeprowadź edycję pliku *settings.py* i dodaj do listy `INSTALLED_APPS` przed jakąkolwiek inną zainstalowaną aplikacją następujący wiersz (wyróżniony na poniższym listingu pogrubioną czcionką).

```
INSTALLED_APPS = [
    'account.apps.AccountConfig',
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
]
```

Django szuka szablonów według kolejności występowania aplikacji w ustawieniu `INSTALLED_APPS`. Aplikacja `django.contrib.admin` zawiera standardowe szablony uwierzytelniania, które zastąpimy w aplikacji `account`. Własne aplikacje zazwyczaj umieszczamy na końcu listy. W tym przypadku umieściliśmy aplikację na pierwszym miejscu w ustawieniu `INSTALLED_APPS`, aby zapewnić domyślne wykorzystywanie zdefiniowanych szablonów uwierzytelniania, a nie szablonów uwierzytelniania zawartych w aplikacji `django.contrib.admin`.

Uruchom poniższe polecenie, aby przeprowadzić synchronizację bazy danych z modelami aplikacji domyślnych wskazanymi na liście `INSTALLED_APPS`.

```
python manage.py migrate
```

Zobaczysz, że zostaną zastosowane wszystkie początkowe migracje bazy danych Django. W wyniku migracji zostały utworzone tabele bazy danych odpowiadające modelom Django zainstalowanych aplikacji. Teraz przystępujemy do budowy systemu uwierzytelniania w projekcie z wykorzystaniem frameworka uwierzytelniania Django.

Użycie frameworka uwierzytelniania w Django

Django jest dostarczany wraz z wbudowanym frameworkiem uwierzytelniania, który może obsługiwać uwierzytelnianie użytkowników, sesje, uprawnienia i grupy użytkowników. System uwierzytelniania oferuje widoki dla działań najczęściej podejmowanych przez użytkowników, takich jak logowanie, wylogowanie, zmiana hasła i zerowanie hasła.

Wspomniany framework uwierzytelniania znajduje się w aplikacji `django.contrib.auth` i jest używany także przez inne pakiety Django typu `contrib`. Framework uwierzytelniania wykorzystaliśmy już w rozdziale 1., „Utworzenie aplikacji bloga” do utworzenia superużytkownika dla aplikacji bloga, aby mieć dostęp do witryny administracyjnej.

Kiedy tworzysz nowy projekt Django za pomocą polecenia `startproject`, framework uwierzytelniania zostaje wymieniony w domyślnych ustawieniach projektu. Składa się z aplikacji `django.contrib.auth` oraz przedstawionych poniżej dwóch klas wymienionych w opcji MIDDLEWARE projektu.

- `AuthenticationMiddleware`: wiąże użytkowników z żądaniami za pomocą mechanizmu sesji.

- `SessionMiddleware`: zapewnia obsługę bieżącej sesji między poszczególnymi żądaniami.

Oprogramowanie middleware to klasy z metodami, które są uruchamiane globalnie podczas faz żądania lub odpowiedzi. W tej książce klasy oprogramowania pośredniczącego będziemy wykorzystywać w wielu sytuacjach. Temat tworzenia oprogramowania pośredniczącego zostanie dokładnie omówiony w rozdziale 17., „Wdrożenie".

Framework uwierzytelniania obejmuje również poniższe modele, które są zdefiniowane w modelach aplikacji `django.contrib.auth.models`.

- `User`: model użytkownika wraz z podstawowymi kolumnami, takimi jak `username`, `password`, `email`, `first_name`, `last_name` i `is_active`.
- `Group`: model grupy do nadawania kategorii użytkownikom.
- `Permission`: uprawnienia pozwalające na wykonywanie określonych operacji.

Opisywany framework zawiera także domyślne widoki uwierzytelniania i formularze, z których będziemy korzystać nieco później.

Utworzenie widoku logowania

Rozpoczynamy od użycia wbudowanego w Django frameworka uwierzytelniania w celu umożliwienia użytkownikom zalogowania się w witrynie. Stworzymy widok, który w celu zalogowania użytkownika wykona następujące czynności.

1. Wyświetlenie użytkownikowi formularza logowania.
2. Pobranie nazwy użytkownika i hasła z wysłanego formularza logowania.
3. Uwierzytelnienie użytkownika na podstawie danych przechowywanych w bazie danych.
4. Sprawdzenie, czy konto użytkownika jest aktywne.
5. Zalogowanie użytkownika w witrynie i rozpoczęcie uwierzytelnionej sesji.

Najpierw musimy przygotować formularz logowania.

Utwórz nowy plik *forms.py* w katalogu aplikacji `account` i umieść w nim poniższy fragment kodu.

```
from django import forms


class LoginForm(forms.Form):
    username = forms.CharField()
    password = forms.CharField(widget=forms.PasswordInput)
```

Formularz będzie używany do uwierzytelnienia użytkownika na podstawie informacji przechowywanych w bazie danych. Zwróć uwagę na wykorzystanie widżetu Password Input do wygenerowania elementu HTML `password`. Zawiera on atrybut `type="password"`, po to aby przeglądarka interpretowała wprowadzane dane jako hasło.

Przeprowadź edycję pliku *views.py* aplikacji `account` i umieść w nim poniższy fragment kodu.

```
from django.contrib.auth import authenticate, login
from django.contrib.auth.decorators import login_required
from django.http import HttpResponse
from django.shortcuts import render

from .forms import LoginForm

def user_login(request):
    if request.method == 'POST':
        form = LoginForm(request.POST)
        if form.is_valid():
            cd = form.cleaned_data
            user = authenticate(
                request,
                username=cd['username'],
                password=cd['password'],
            )
            if user is not None:
                if user.is_active:
                    login(request, user)
                    return HttpResponse('Uwierzytelnianie zakończyło się sukcesem')
                else:
                    return HttpResponse('Konto jest zablokowane')
            else:
                return HttpResponse('Niepoprawne dane uwierzytelniające')
    else:
        form = LoginForm()
    return render(request, 'account/login.html', {'form': form})
```

Oto jakie działania realizuje podstawowy widok logowania:

Po wywołaniu widoku `user_login` przez żądanie `GET` za pomocą wywołania `form = Login` ↪`Form()` tworzymy nowy egzemplarz formularza logowania. Następnie formularz jest przekazywany do szablonu.

Kiedy użytkownik wyśle formularz przy użyciu żądania `POST`, przeprowadzane są następujące działania.

Utworzenie egzemplarza formularza wraz z wysłanymi danymi. Do tego celu służy polecenie `form = LoginForm(request.POST)`.

Sprawdzenie za pomocą wywołania `form.is_valid()`, czy formularz jest prawidłowy. Jeżeli formularz jest nieprawidłowy, w szablonie wyświetlamy błędy wykryte podczas weryfikacji formularza (np. użytkownik nie wypełnił jednego z pól).

Jeżeli wysłane dane są prawidłowe, uwierzytelniamy użytkownika na podstawie informacji przechowywanych w bazie danych za pomocą metody `authenticate()`. Wymieniona metoda pobiera `username` i `password`, a zwraca obiekt `User`, gdy użytkownik zostanie uwierzytelniony, lub `None` w przeciwnym przypadku. Ponadto jeśli użytkownik nie będzie uwierzytelniony, zwracamy obiekt `HttpResponse` wraz z komunikatem „Nieprawidłowe dane uwierzytelniające".

W przypadku pomyślnego uwierzytelnienia użytkownika sprawdzamy za pomocą atrybutu `is_active`, czy jego konto jest aktywne. Wymieniony atrybut pochodzi z modelu `User`

dostarczanego przez Django. Gdy konto użytkownika jest nieaktywne, zwracamy obiekt `HttpResponse` wraz z komunikatem „Konto jest zablokowane".

Gdy konto użytkownika jest aktywne, logujemy go w witrynie internetowej. Rozpoczynamy także sesję dla użytkownika: wywoływana jest metoda `login()` i zwracany komunikat „Uwierzytelnienie zakończyło się sukcesem".

Uwaga

Zwróć uwagę na różnice między metodami `authenticate()` i `login()`. Metoda `authenticate()` sprawdza dane uwierzytelniające użytkownika i jeśli są prawidłowe, zwraca obiekt użytkownika. Natomiast metoda `login()` umieszcza użytkownika w bieżącej sesji.

Teraz musimy opracować wzorzec adresu URL dla nowo zdefiniowanego widoku.

Utwórz nowy plik *urls.py* w katalogu aplikacji `account` i umieść w nim poniższy fragment kodu.

```
from django.urls import path
from . import views

urlpatterns = [
    path('login/', views.user_login, name='login'),
]
```

Przeprowadź edycję głównego pliku *urls.py* znajdującego się katalogu projektu `bookmarks` i dodaj wzorzec adresu URL aplikacji `account`, co przedstawiłem poniżej. Nowy kod został wyróżniony pogrubioną czcionką.

```
from django.conf import settings
from django.urls import include, path

urlpatterns = [
    path('admin/', admin.site.urls),
    path('account/', include('account.urls')),
]
```

Widok logowania jest teraz dostępny za pomocą adresu URL.

Przechodzimy więc do przygotowania szablonu dla tego widoku. Ponieważ w projekcie nie mamy jeszcze żadnych szablonów, najpierw musimy utworzyć szablon bazowy, który następnie będzie mógł być rozszerzony przez szablon logowania.

W katalogu aplikacji `account` utwórz wymienioną poniżej strukturę plików i katalogów.

```
templates/
     account/
         login.html
     base.html
```

Przeprowadź edycję pliku *base.html* i umieść w nim poniższy fragment kodu.

```
{% load static %}
<!DOCTYPE html>
```

```
<html>
<head>
  <title>{% block title %}{% endblock %}</title>
  <link href="{% static "css/base.css" %}" rel="stylesheet">
</head>
<body>
  <div id="header">
    <span class="logo">Bookmarks</span>
  </div>
  <div id="content">
    {% block content %}
    {% endblock %}
  </div>
</body>
</html>
```

W ten sposób przygotowaliśmy szablon bazowy dla budowanej witryny internetowej. Podobnie jak w poprzednim projekcie, także w tym style CSS dołączamy w szablonie głównym. Niezbędne pliki statyczne znajdziesz w materiałach przygotowanych dla książki. Wystarczy skopiować podkatalog *static* aplikacji `account` ze wspomnianych materiałów i umieścić go w tej samej lokalizacji w budowanym projekcie. Zawartość katalogu można znaleźć pod adresem *https://github.com/PacktPublishing/Django-5-by-Example/tree/master/Chapter04/bookmarks/account/static*.

Szablon bazowy definiuje bloki `title` i `content`, które mogą być wypełniane przez treść szablonów rozszerzających szablon bazowy.

Przechodzimy do utworzenia szablonu dla formularza logowania.

W tym celu otwórz plik *account/login.html* i umieść w nim poniższy fragment kodu.

```
{% extends "base.html" %}

{% block title %}Logowanie{% endblock %}

{% block content %}
  <h1>Logowanie</h1>
  <p>Wypełnij poniższy formularz, aby się zalogować:</p>
  <form method="post">
    {{ form.as_p }}
    {% csrf_token %}
    <p><input type="submit" value="Zaloguj"></p>
  </form>
{% endblock %}
```

Ten szablon zawiera formularz, którego egzemplarz jest tworzony w widoku. Ponieważ formularz zostanie wysłany za pomocą metody `POST`, dołączamy znacznik szablonu `{% csrf_token %}` w celu zapewnienia ochrony przed atakami typu *CSRF*. Więcej informacji na temat ataków CSRF przedstawiłem w rozdziale 2., „Usprawnianie bloga i dodanie funkcjonalności społecznościowych”.

W bazie danych nie ma jeszcze żadnych kont użytkowników. Aby zapewnić sobie dostęp do witryny administracyjnej i zarządzać pozostałymi użytkownikami, trzeba najpierw utworzyć superużytkownika.

W wierszu polecenia powłoki uruchom polecenie:

```
python manage.py createsuperuser
```

Wyświetli się wynik pokazany poniżej. Wprowadź żądaną nazwę użytkownika, adres e-mail i hasło w następujący sposób.

```
Username (leave blank to use 'admin'): admin
Email address: admin@admin.com
Password: ********
Password (again): ********
```

Następnie wyświetli się komunikat o powodzeniu:

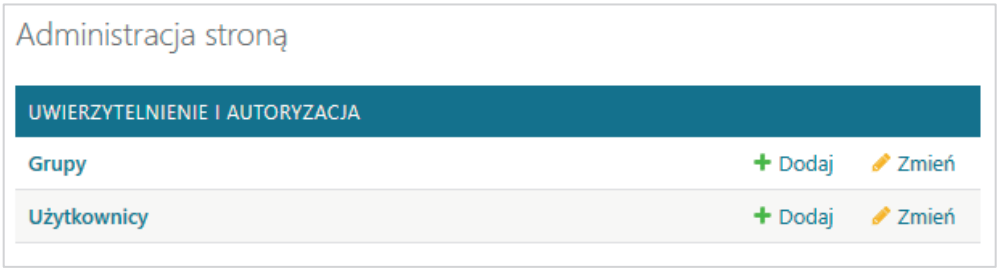
```
Superuser created successfully.
```

Uruchom serwer programistyczny za pomocą polecenia:

```
python manage.py runserver
```

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/admin/`. Dostęp do witryny administracyjnej uzyskasz po podaniu ustalonej przed chwilą nazwy użytkownika i hasła. Gdy znajdziesz się już w witrynie administracyjnej Django, zobaczysz modele `Users` (link *Użytkownicy*) i `Group` (link *Grupy*) dla wbudowanego w Django frameworka uwierzytelniania.

Strona wygląda jak na rysunku 4.2.



Rysunek 4.2. Główna strona witryny administracyjnej Django włącznie z modelami Users i Group

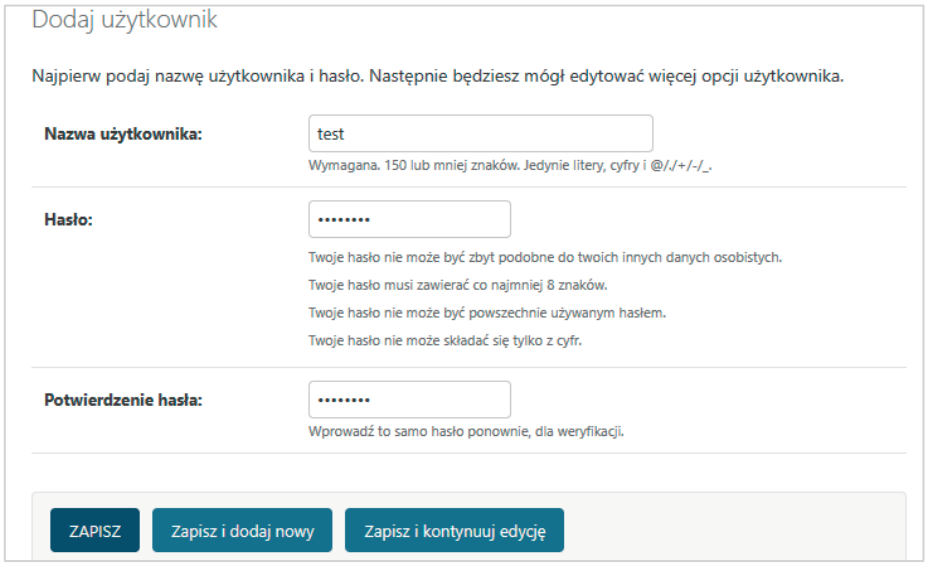
W wierszu *Użytkownicy* kliknij *Dodaj*.

W witrynie administracyjnej utwórz nowego użytkownika (rysunek 4.3).

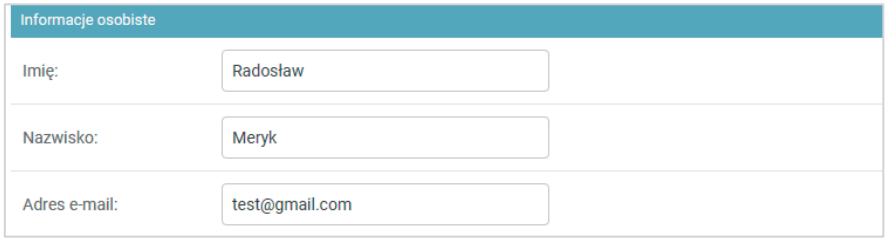
Wprowadź dane użytkownika i aby zapisać nowego użytkownika w bazie danych, kliknij przycisk *ZAPISZ*.

Następnie w obszarze *Informacje osobiste* wypełnij pola *Imię*, *Nazwisko* i *Adres e-mail* w sposób pokazany na rysunku 4.4, po czym kliknij przycisk *Zapisz*, aby zapisać zmiany.

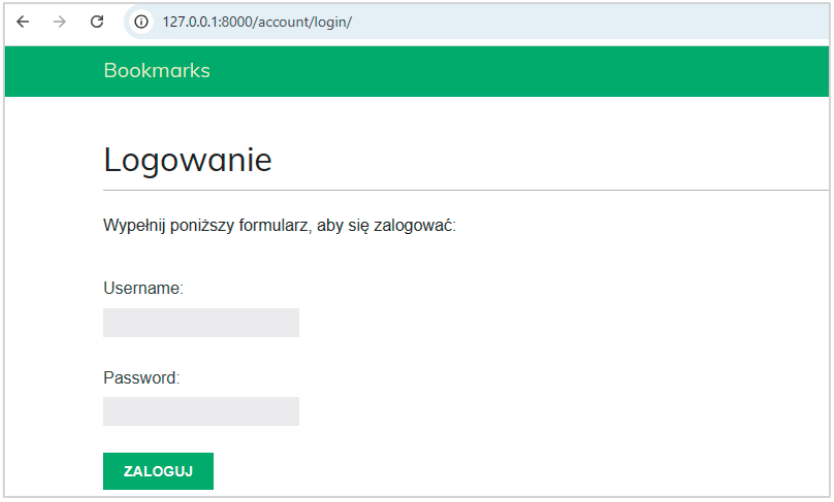
W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/account/login/`. Powinieneś zobaczyć wyrenderowany szablon wraz z formularzem logowania (rysunek 4.5).



Rysunek 4.3. Strona logowania użytkownika

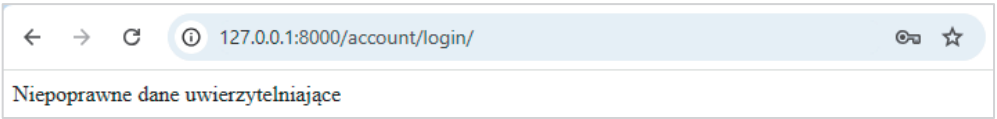


Rysunek 4.4. Formularz edycji danych użytkownika w witrynie administracyjnej Django



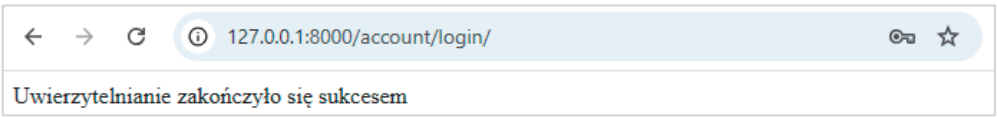
Rysunek 4.5. Strona logowania użytkownika

Wprowadź błędne dane uwierzytelniające i wyślij formularz. Powinieneś otrzymać komunikat *Niepoprawne dane uwierzytelniające* (rysunek 4.6).



Rysunek 4.6. Komunikat tekstowy informujący o niepoprawnym logowaniu

Wprowadź poprawne poświadczenia. Wyświetli się komunikat *Uwierzytelnianie zakończyło się sukcesem* (rysunek 4.7).



Rysunek 4.7. Komunikat tekstowy z informacją o pomyślnym uwierzytelnianiu

Nauczyłeś się uwierzytelniać użytkowników i tworzyć własny widok uwierzytelniania. Co prawda możesz tworzyć własne widoki uwierzytelniania, ale Django zapewnia gotowe do użycia widoki uwierzytelniania, z których możesz skorzystać.

Użycie wbudowanych widoków uwierzytelniania w Django

Framework uwierzytelniania w Django zawiera wiele formularzy i widoków gotowych do natychmiastowego użycia.

Utworzony przed chwilą widok logowania to dobre ćwiczenie pomagające w zrozumieniu procesu uwierzytelniania użytkowników w Django. Jednak w większości przypadków możesz wykorzystać wspomniane domyślne widoki uwierzytelniania.

Do obsługi uwierzytelniania Django oferuje wymienione poniżej widoki. Wszystkie są dostępne w module `django.contrib.auth.views`.

- `LoginView`: obsługa formularza logowania oraz proces zalogowania użytkownika.
- `LogoutView`: obsługa wylogowania użytkownika.

Do obsługi zmiany hasła Django oferuje wymienione poniżej widoki.

- `PasswordChangeView`: obsługa formularza pozwalającego użytkownikowi na zmianę hasła.
- `PasswordChangeDoneView`: strona informująca o sukcesie operacji; zostanie wyświetlona użytkownikowi, gdy zmiana hasła zakończy się powodzeniem.

Do obsługi operacji resetowania hasła Django oferuje zaś następujące widoki.

- `PasswordResetView`: Umożliwienie użytkownikowi ponownego ustawienia hasła. Django generuje link wraz z tokenem przeznaczonym do jednokrotnego użycia, który jest wysyłany na adres e-mail danego użytkownika.
- `PasswordResetDoneView`: Wyświetlenie użytkownikowi strony z informacją o wysłaniu wiadomości e-mail wraz z linkiem pozwalającym na ponowne ustawienie hasła.
- `PasswordResetConfirmView`: widok umożliwiający użytkownikowi zdefiniowanie nowego hasła.
- `PasswordResetCompleteView`: strona informująca o sukcesie operacji; zostanie wyświetlona użytkownikowi, gdy zmiana hasła zakończy się powodzeniem.

Zastosowanie wymienionych wyżej widoków może zaoszczędzić sporą ilość czasu podczas tworzenia witryny internetowej obsługującej konta użytkowników. W widokach tych używane są wartości domyślne, które oczywiście można nadpisać. Przykładem może być wskazanie położenia szablonu przeznaczonego do wygenerowania lub formularza wyświetlanego przez widok.

Więcej informacji na temat wbudowanych widoków uwierzytelniania można znaleźć pod adresem *https://docs.djangoproject.com/en/5.0/topics/auth/default/#all-authentication-views*.

Widoki logowania i wylogowania

Aby nauczyć się korzystać z widoków uwierzytelniania Django, zastąpimy niestandardowy widok logowania wbudowanym odpowiednikiem Django, a także zintegrujemy z aplikacją widok wylogowania.

Przeprowadź edycję pliku *urls.py* aplikacji `account` i dodaj kod wyróżniony pogrubioną czcionką.

```
from django.contrib.auth import views as auth_views
from django.urls import path
from . import views


urlpatterns = [
    # Poprzedni adres URL strony logowania
    # path('login/', views.user_login, name='login'),
    #  adresy url logowania / wylogowania
    path('login/', auth_views.LoginView.as_view(), name='login'),
    path('logout/', auth_views.LogoutView.as_view(), name='logout'),
]
```

W powyższym kodzie oznaczyliśmy jako komentarz wzorzec adresu URL dla utworzonego wcześniej widoku `user_login`. Teraz wykorzystamy widok `LoginView` oferowany przez wbudowany w Django framework uwierzytelniania. Dodaliśmy także wzorzec adresu URL dla widoku `LogoutView`.

Utwórz nowy podkatalog w katalogu *templates/* aplikacji `account` i nadaj mu nazwę *registration*. Ten podkatalog to domyślna lokalizacja, w której widoki uwierzytelniania Django spodziewają się znaleźć szablony.

Moduł `django.contrib.admin` zawiera kilka szablonów uwierzytelniania, które są używane w witrynie administracyjnej. Podczas konfigurowania projektu aplikację `account` umieściliśmy na początku ustawienia `INSTALLED_APPS`, aby Django domyślnie korzystał z niestandardowych szablonów zamiast szablonów uwierzytelniania zdefiniowanych w innych aplikacjach.

W katalogu *templates/registration* utwórz plik *login.html* i umieść w nim poniższy fragment kodu.

```
{% extends "base.html" %}

{% block title %}Logowanie{% endblock %}

{% block content %}
  <h1>Logowanie</h1>
  {% if form.errors %}
    <p>
      Nazwa użytkownika lub hasło są niepoprawne.
      Spróbuj ponownie.
    </p>
  {% else %}
    <p>
      Wypełnij poniższy formularz, aby się zalogować:
    </p>
  {% endif %}
  <div class="login-form">
    <form action="{% url 'login' %}" method="post">
      {{ form.as_p }}
      {% csrf_token %}
      <input type="hidden" name="next" value="{{ next }}" />
      <p><input type="submit" value="Zaloguj"></p>
    </form>
  </div>
{% endblock %}
```

Ten szablon logowania jest bardzo podobny do utworzonego wcześniej. Domyślnie Django używa formularza `AuthenticationForm` pochodzącego z modułu `django.contrib.auth.forms`. Formularz próbuje uwierzytelnić użytkownika i zgłasza błąd walidacji, gdy logowanie zakończy się niepowodzeniem. W takim przypadku, aby sprawdzić, czy podane zostały niepoprawne dane uwierzytelniające, można przeanalizować te błędy za pomocą znacznika szablonu `{% if form.errors %}`.

Zwróć uwagę na dodanie ukrytego elementu HTML `<input>`, przeznaczonego do wysłania wartości zmiennej o nazwie `next`. Ta zmienna będzie dostarczana do widoku logowania, jeśli przekażesz do żądania parametr o nazwie `next`, na przykład przez otwarcie w przeglądarce strony `http://127.0.0.1:8000/account/login/?next=/account/`.

Parametr `next` musi reprezentować adres URL. Jeżeli podamy ten parametr, to po pomyślnym zalogowaniu w widoku logowania Django przekieruje użytkownika pod wskazany adres URL.

Teraz utwórz w katalogu *registration* szablon *logged_out.html* i umieść w nim następujący fragment kodu.

```
{% extends "base.html" %}

{% block title %}Wylogowanie{% endblock %}

{% block content %}
  <h1>Wylogowanie</h1>
  <p>
    Zostałeś pomyślnie wylogowany.
    Możesz <a href="{% url "login" %}">zalogować się ponownie</a>.
  </p>
{% endblock %}
```

To jest szablon, który Django wyświetli po wylogowaniu się użytkownika.

Dodaliśmy wzorce i szablony adresów URL dla widoków logowania i wylogowania. Użytkownicy mogą teraz logować się i wylogowywać za pomocą oferowanych przez Django widoków uwierzytelniania.

Przystępujemy teraz do utworzenia nowego widoku. Będzie on przeznaczony do wyświetlenia użytkownikowi, po pomyślnym zalogowaniu się w aplikacji, panelu głównego (ang. *dashboard*).

Przeprowadź edycję pliku *views.py* aplikacji `account` i umieść w nim poniższy fragment kodu.

```
from django.contrib.auth.decorators import login_required

@login_required
def dashboard(request):
    return render(
        request,
        'account/dashboard.html',
        {'section': 'dashboard'}
    )
```

Utworzyliśmy widok `dashboard` i zastosowaliśmy do niego dekorator `login_required` frameworka uwierzytelniania. Zadanie dekoratora `login_required` polega na sprawdzeniu, czy bieżący użytkownik został uwierzytelniony.

Jeżeli użytkownik jest uwierzytelniony, następuje wykonanie udekorowanego widoku. Gdy natomiast użytkownik nie jest uwierzytelniony, zostaje przekierowany na stronę logowania, a adres URL, do którego próbował uzyskać dostęp, będzie podany jako wartość parametru next żądania `GET`.

Tym samym po udanym logowaniu użytkownik powróci na stronę, do której wcześniej próbował uzyskać dostęp. Pamiętaj, że do obsługi tego rodzaju sytuacji dodaliśmy w szablonie logowania ukryty element HTML `<input>`.

Zdefiniowaliśmy również zmienną section. Użyjemy tej zmiennej do zaznaczenia bieżącej sekcji w menu głównym witryny.

Teraz należy utworzyć szablon dla widoku panelu głównego.

Utwórz nowy plik w katalogu *templates/account/*, nadaj mu nazwę *dashboard.html* i umieść w nim przedstawiony poniżej kod.

```
{% extends "base.html" %}

{% block title %}Panel główny{% endblock %}

{% block content %}
  <h1>Panel główny</h1>
  <p>Witaj w panelu głównym.</p>
{% endblock %}
```

Przeprowadź edycję pliku *urls.py* aplikacji `account` i dodaj w nim poniższy wzorzec adresu URL oznaczony pogrubieniem.

```
urlpatterns = [
    # Poprzedni adres URL strony logowania
    #  path('login/', views.user_login, name='login'),

    # Adresy url logowania / wylogowania
    path('login/', auth_views.LoginView.as_view(), name='login'),
    path('logout/', auth_views.LogoutView.as_view(), name='logout'),

    path('', views.dashboard, name='dashboard'),
]
```

Teraz przeprowadź edycję pliku *settings.py* projektu i dodaj poniższy fragment kodu.

```
LOGIN_REDIRECT_URL = 'dashboard'
LOGIN_URL = 'login'
LOGOUT_URL = 'logout'
```

Zdefiniowaliśmy następujące ustawienia.

- `LOGIN_REDIRECT_URL`: wskazujemy Django adres URL, do którego ma nastąpić przekierowanie, gdy widok `contrib.auth.views.login` nie otrzymuje parametru `next`.
- `LOGIN_URL`: Adres URL, do którego ma nastąpić przekierowanie po zalogowaniu użytkownika (np. za pomocą dekoratora `login_required`).
- `LOGOUT_URL`: Adres URL, do którego ma nastąpić przekierowanie po wylogowaniu użytkownika.

Użyliśmy nazw adresów URL, które wcześniej zdefiniowaliśmy we wzorcach adresów URL za pomocą atrybutu `name` funkcji `path()`. Do tych ustawień zamiast nazw adresów URL można również użyć zakodowanych „na sztywno” adresów URL.

Oto krótkie podsumowanie przeprowadzonych dotąd działań:

- Do projektu dodaliśmy wbudowane we frameworku uwierzytelniania Django widoki logowania (`login`) i wylogowania (`logout`).
- Przygotowaliśmy własne szablony dla obu widoków i zdefiniowaliśmy prosty widok, do którego użytkownik zostanie przekierowany po zalogowaniu.
- Na koniec skonfigurowaliśmy ustawienia Django, aby wspomniane adresy URL były używane domyślnie.

Teraz do szablonu bazowego dodamy linki logowania i wylogowania. Aby to zrobić, trzeba ustalić, czy bieżący użytkownik jest zalogowany. Na tej podstawie zostanie wyświetlony właściwy link (logowania lub wylogowania). Warstwa middleware odpowiedzialna za uwierzytelnianie ustawia bieżącego użytkownika w obiekcie `HttpRequest`. Dostęp do niego

można uzyskać za pomocą `request.user`. Obiekt `request` zawiera obiekt `User` nawet wtedy, kiedy użytkownik nie został uwierzytelniony. W takim przypadku użytkownik będzie zdefiniowany w postaci egzemplarza obiektu `AnonymousUser`. Najlepszym sposobem zweryfikowania, czy użytkownik został uwierzytelniony, jest sprawdzenie wartości jego atrybutu „tylko do odczytu" `is_authenticated`.

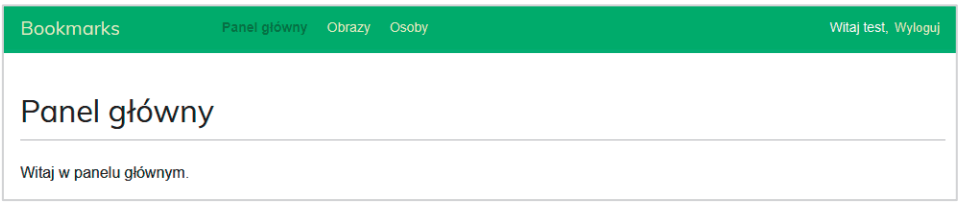
Przeprowadź edycję pliku *templates/base.html* i dodaj poniższe wiersze wyróżnione pogrubioną czcionką.

```
{% load static %}
<!DOCTYPE html>
<html>
<head>
  <title>{% block title %}{% endblock %}</title>
  <link href="{% static "css/base.css" %}" rel="stylesheet">
</head>
<body>
  <div id="header">
    <span class="logo">Bookmarks</span>
    {% if request.user.is_authenticated %}
      <ul class="menu">
        <li {% if section == "dashboard" %}class="selected"{% endif %}>
          <a href="{% url "dashboard" %}">Panel główny</a>
        </li>
        <li {% if section == "images" %}class="selected"{% endif %}>
          <a href="#">Obrazy</a>
        </li>
        <li {% if section == "people" %}class="selected"{% endif %}>
          <a href="#">Osoby</a>
        </li>
      </ul>
    {% endif %}
    <span class="user">
      {% if request.user.is_authenticated %}
        Witaj {{ request.user.first_name|default:request.user.username }},
        <form action="{% url "logout" %}" method="post">
          <button type="submit">Wyloguj</button>
          {% csrf_token %}
        </form>
      {% else %}
        <a href="{% url "login" %}">Zaloguj </a>
      {% endif %}
    </span>
  </div>
  <div id="content">
    {% block content %}
    {% endblock %}
  </div>
</body>
</html>
```

Menu witryny internetowej będzie wyświetlane jedynie uwierzytelnionym użytkownikom. Sprawdzana jest także zmienna `section`, reprezentująca sekcję witryny, w celu dodania klasy atrybutu `selected` do odpowiedniego elementu `<li>` i tym samym zaznaczenia za

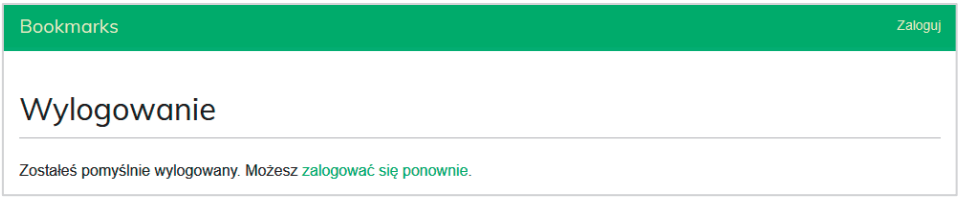
pomocą CSS nazwy bieżącej sekcji. Jeśli użytkownik jest zalogowany, wyświetlane jest imię uwierzytelnionego użytkownika i link pozwalający mu na wylogowanie. Jeżeli użytkownik nie jest uwierzytelniony, wyświetlony będzie link pozwalający mu na zalogowanie. Jeśli nazwa użytkownika jest pusta, wyświetlana jest wartość pola `username` za pomocą wywołania `request.user.first_name|default:request.user.username`. Zauważ, że dla akcji wylogowania używamy formularza z metodą `POST` i przycisku do przesłania formularza. To dlatego, że widok `LogoutView` wymaga żądań `POST`.

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/account/login/`. Powinieneś zobaczyć stronę logowania. Podaj poprawne dane uwierzytelniające i kliknij przycisk *Zaloguj*. Wyświetli się ekran pokazany na rysunku 4.8.



Rysunek 4.8. Strona panelu głównego

Dzięki zastosowaniu stylów CSS nazwa sekcji *Panel główny* została wyświetlona innym kolorem czcionki, ponieważ odpowiadającemu jej elementowi `<li>` przypisaliśmy klasę `selected`. Skoro użytkownik jest uwierzytelniony, jego imię wyświetlamy po prawej stronie nagłówka. Kliknij *Wyloguj*. Powinieneś zobaczyć stronę podobną do pokazanej na rysunku 4.9.



Rysunek 4.9. Strona wyświetlana po wylogowaniu użytkownika

Na tej stronie widać, że użytkownik jest wylogowany, w związku z czym menu witryny nie jest wyświetlane. Link znajdujący się po prawej stronie nagłówka zmienia się na *Zaloguj*.

Uwaga

Jeżeli zamiast przygotowanej wcześniej strony wylogowania zostanie wyświetlona strona wylogowania witryny administracyjnej Django, sprawdź listę `INSTALLED_APPS` projektu i zadbaj o to, aby wpis dotyczący aplikacji `django.contrib.admin` został wymieniony po `account`. Obie aplikacje zawierają szablony wylogowania znajdujące się w tej samej ścieżce względnej. Z tego powodu mechanizm ładujący szablony Django przejrzy różne aplikacje na liście `INSTALLED_APPS` i użyje pierwszego znalezionego szablonu.

Widoki zmiany hasła

Użytkownikom witryny musimy zapewnić możliwość zmiany hasła po zalogowaniu się. Zintegrujemy więc oferowane przez framework uwierzytelniania Django widoki przeznaczone do obsługi procedury zmiany hasła.

Otwórz plik *urls.py* aplikacji `account` i umieść w nim poniższe wzorce adresów URL.

```
urlpatterns = [
    # poprzedni widok logowania
    # path('login/', views.user_login, name='login'),

    # adresy url logowania / wylogowania
    path('login/', auth_views.LoginView.as_view(), name='login'),
    path('logout/', auth_views.LogoutView.as_view(), name='logout'),

    # Adresy URL przeznaczone do obsługi zmiany hasła
    path(
        'password-change/',
        auth_views.PasswordChangeView.as_view(),
        name='password_change'
    ),
    path(
        'password-change/done/',
        auth_views.PasswordChangeDoneView.as_view(),
        name='password_change_done'
    ),

    path('', views.dashboard, name='dashboard'),
]
```

Widok `PasswordChangeView` zapewnia obsługę formularza pozwalającego na zmianę hasła, natomiast `PasswordChangeDoneView` wyświetla komunikat informujący o sukcesie po udanej operacji zmiany hasła przez użytkownika. Przystępujemy więc do przygotowania szablonu dla wymienionych widoków.

Dodaj nowy plik w katalogu *templates/registration* aplikacji `account` i nadaj mu nazwę *password_change_form.html*. Następnie w nowym pliku umieść poniższy fragment kodu.

```
{% extends "base.html" %}

{% block title %}Zmiana hasła{% endblock %}

{% block content %}
  <h1>Zmiana hasła</h1>
  <p>Wypełnij poniższy formularz, aby zmienić hasło.</p>
  <form method="post">
    {{ form.as_p }}
    <p><input type="submit" value="Zmień"></p>
    {% csrf_token %}
  </form>
{% endblock %}
```

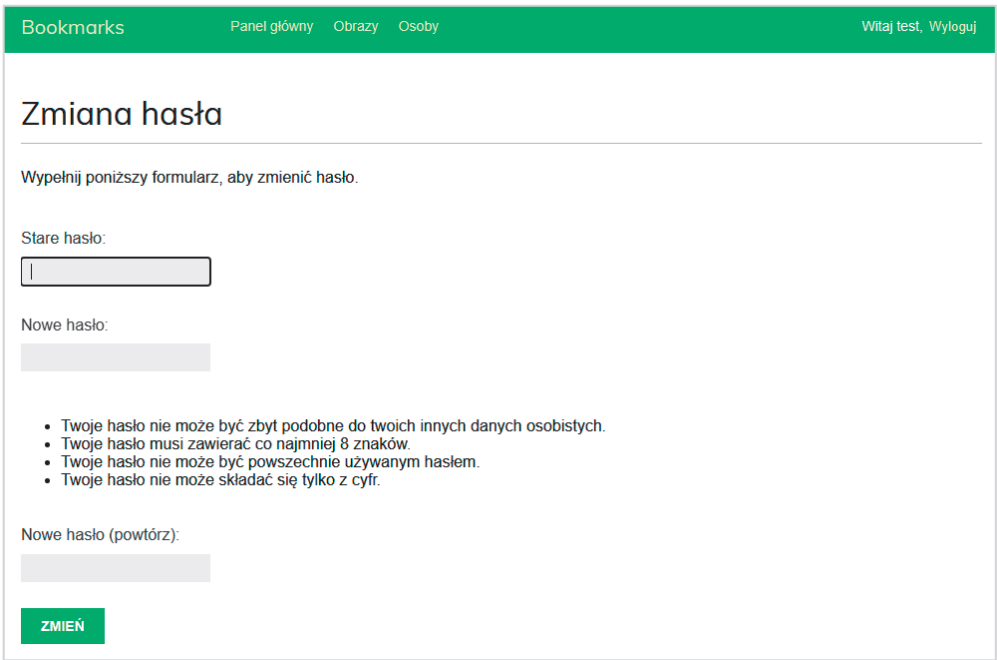
Przedstawiony szablon zawiera formularz przeznaczony do obsługi procedury zmiany hasła.

Teraz w tym samym katalogu utwórz kolejny plik i nadaj mu nazwę *password_change_done.html*. Następnie w nowym pliku umieść poniższy fragment kodu.

```
{% extends "base.html" %}

{% block title %}Hasło zostało zmienione{% endblock %}

{% block content %}
  <h1>Hasło zostało zmienione</h1>
  <p>Zmiana hasła zakończyła się powodzeniem.</p>
{% endblock %}
```

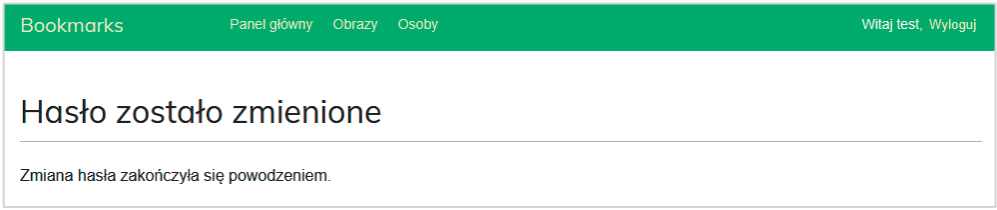
Ten szablon zawiera jedynie komunikat o sukcesie operacji wyświetlany, gdy przeprowadzona przez użytkownika operacja zmiany hasła zakończy się powodzeniem.

W przeglądarce internetowej przejdź pod adres *http://127.0.0.1:8000/account/password-change/*. Jeżeli użytkownik nie jest zalogowany, nastąpi przekierowanie na stronę logowania. Po udanym uwierzytelnieniu zobaczysz formularz pozwalający na zmianę hasła, pokazany na rysunku 4.10.



Rysunek 4.10. Formularz zmiany hasła

W wyświetlonym formularzu należy podać dotychczasowe hasło oraz nowe, a następnie kliknąć przycisk *ZMIEŃ*. Jeżeli operacja przebiegnie bez problemów, zostanie wyświetlona pokazana poniżej strona wraz z komunikatem informującym o sukcesie (rysunek 4.11).



Rysunek 4.11. Strona z komunikatem o pomyślnej zmianie hasła

Wyloguj się i zaloguj ponownie za pomocą nowego hasła, aby sprawdzić, że wszystko działa zgodnie z oczekiwaniami.

Widoki odzyskiwania hasła

Jeśli użytkownik zapomni hasła, powinien mieć możliwość odzyskania swojego konta. W tym punkcie zaimplementujemy funkcjonalność resetowania hasła. Umożliwi to użytkownikom odzyskanie dostępu do konta poprzez otrzymanie wiadomości e-mail zawierającej bezpieczny link, wygenerowany za pomocą unikatowego tokena, umożliwiający utworzenie nowego hasła.

Otwórz plik *urls*.py aplikacji `account` i umieść w nim poniższe wzorce adresów URL wyróżnione pogrubioną czcionką.

```
urlpatterns = [
    # Poprzedni adres URL strony logowania
    # path('login/', views.user_login, name='login'),

    # adresy url logowania / wylogowania
    path('login/', auth_views.LoginView.as_view(), name='login'),
    path('logout/', auth_views.LogoutView.as_view(), name='logout'),

    # Adresy URL przeznaczone do obsługi zmiany hasła
    path(
        'password-change/',
        auth_views.PasswordChangeView.as_view(),
        name='password_change'
    ),
    path(
        'password-change/done/',
        auth_views.PasswordChangeDoneView.as_view(),
        name='password_change_done'
    ),

    # Adresy URL przeznaczone do obsługi procedury odzyskiwania hasła
    path(
        'password-reset/',
        auth_views.PasswordResetView.as_view(),
        name='password_reset'
    ),
    path(
        'password-reset/done/',
```

```
            auth_views.PasswordResetDoneView.as_view(),
            name='password_reset_done'
        ),
        path(
            'password-reset/<uidb64>/<token>/',
            auth_views.PasswordResetConfirmView.as_view(),
            name='password_reset_confirm'
        ),
        path(
            'password-reset/complete/',
            auth_views.PasswordResetCompleteView.as_view(),
            name='password_reset_complete'
        ),
        path('', views.dashboard, name='dashboard'),
    ]
```

Dodaj nowy plik w katalogu *templates/registration/* aplikacji `account` i nadaj mu nazwę *password_reset_form.html*. Następnie w nowym pliku umieść poniższy fragment kodu.

```
{% extends "base.html" %}

{% block title %}Odzyskiwanie hasła{% endblock %}

{% block content %}
  <h1>Zapomniałeś hasła?</h1>
  <p>Podaj adres e-mail, aby zdefiniować nowe hasło.</p>
  <form method="post">
    {{ form.as_p }}
    <p><input type="submit" value="Wyślij wiadomość e-mail"></p>
    {% csrf_token %}
  </form>
{% endblock %}
```

Teraz utwórz w tym samym katalogu kolejny plik, tym razem o nazwie *password_reset_email.html*. Następnie w nowym pliku umieść poniższy fragment kodu.

```
Otrzymaliśmy żądanie odzyskania hasła dla użytkownika używającego adresu e-mail
➥{{email }}. Kliknij poniższy link:
{{ protocol }}://{{ domain }}{% url "password_reset_confirm" uidb64=uid token=token %}
Twoja nazwa użytkownika, na wypadek gdybyś zapomniał: {{ user.get_username }}
```

Szablon *password_reset_email.html* zostanie użyty do wygenerowania wiadomości e-mail wysyłanej użytkownikowi, który chce przeprowadzić operację odzyskania hasła. Wiadomość ta zawiera wygenerowany przez widok token, który jest potrzebny do ustawienia nowego hasła.

Utwórz w tym samym katalogu kolejny plik i nadaj mu nazwę *password_reset_done.html*. Następnie w nowym pliku umieść poniższy fragment kodu.

```
{% extends "base.html" %}

{% block title %}Odzyskiwanie hasła{% endblock %}

{% block content %}
  <h1>Odzyskiwania hasła</h1>
```

```
  <p> Wysłaliśmy Ci wiadomość e-mail wraz z instrukcjami pozwalającymi na
  ↪zdefiniowanie nowego hasła.</p>
  <p>Jeżeli nie otrzymałeś tej wiadomości, to upewnij się, że w formularzu wpisałeś
  ↪adres e-mail podany podczas zakładania konta użytkownika.</p>
{% endblock %}
```

Utwórz kolejny plik szablonu w tym samym katalogu i nadaj mu nazwę *password_reset_confirm.html*. Następnie w nowym pliku umieść poniższy fragment kodu.

```
{% extends "base.html" %}

{% block title %}Odzyskiwanie hasła{% endblock %}

{% block content %}
  <h1>Odzyskiwanie hasła</h1>
  {% if validlink %}
    <p>Dwukrotnie podaj nowe hasło:</p>
    <form method="post">
      {{ form.as_p }}
      {% csrf_token %}
      <p><input type="submit" value="Zmień hasło" /></p>
    </form>
  {% else %}
    <p>Link pozwalający na odzyskanie hasła jest niepoprawny, ponieważ
    ↪prawdopodobnie został już wcześniej użyty. Musisz ponownie rozpocząć procedurę
    ↪odzyskiwania hasła.</p>
   {% endif %}
{% endblock %}
```

W tym szablonie sprawdzamy, czy podany link jest poprawny. W tym celu weryfikowana jest zmienna `validlink`. Oferowany przez Django widok `PasswordResertConfirmView` ustawia zmienną i umieszcza ją w kontekście szablonu. Jeżeli link jest poprawny, wtedy wyświetlamy użytkownikowi formularz odzyskiwania hasła. Użytkownicy mogą ustawić nowe hasło tylko wtedy, gdy dysponują poprawnym linkiem odzyskiwania hasła.

Utwórz kolejny plik szablonu i nadaj mu nazwę *password_reset_complete.html*. Następnie umieść w nim poniższy fragment kodu.

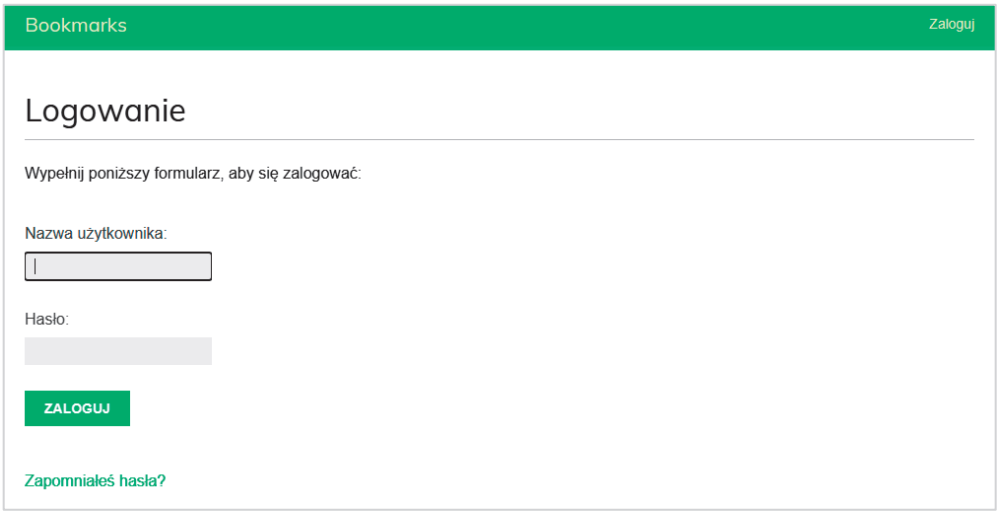
```
{% extends "base.html" %}

{% block title %}Odzyskiwanie hasła{% endblock %}

{% block content %}
  <h1>Ustawianie hasła</h1>
  <p>Hasło zostało zdefiniowane. Możesz się już <a href="{% url "login" %}">
  ↪zalogować</a>.</p>
{% endblock %}
```

Na koniec przeprowadź edycję szablonu *registration/login.html* aplikacji `account` i dodaj poniższe wiersze wyróżnione pogrubioną czcionką.

```
{% extends "base.html" %}
{% block title %}Logowanie{% endblock %}
{% block content %}
  <h1>Logowanie</h1>
  {% if form.errors %}
```

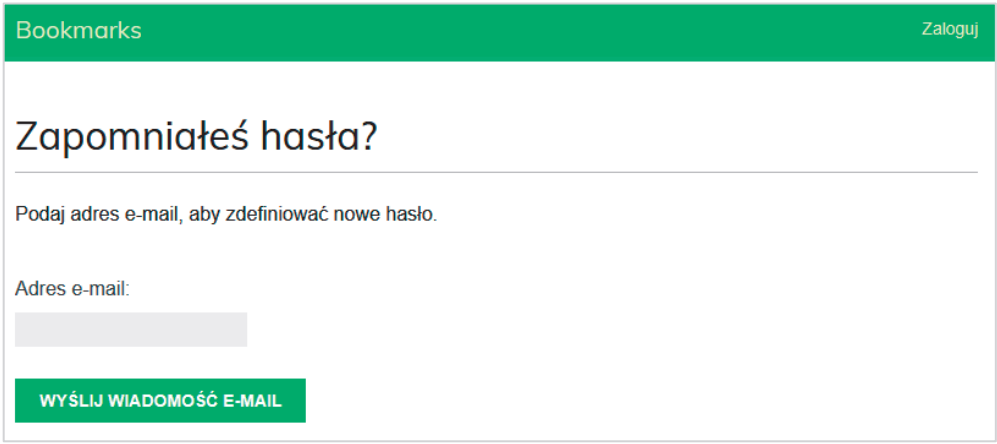
```
      <p>
        Nazwa użytkownika lub hasło są niepoprawne.
        Spróbuj ponownie.
      </p>
    {% else %}
      <p>Wypełnij poniższy formularz, aby się zalogować:</p>
    {% endif %}
    <div class="login-form">
      <form action="{% url 'login' %}" method="post">
        {{ form.as_p }}
        {% csrf_token %}
        <input type="hidden" name="next" value="{{ next }}" />
        <p><input type="submit" value="Zaloguj"></p>
      </form>
      <p>
        <a href="{% url "password_reset" %}">
          Zapomniałeś hasła?
        </a>
      </p>
    </div>
{% endblock %}
```

Teraz w przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/account/login/`. Strona logowania powinna teraz zawierać link do strony odzyskiwania hasła (rysunek 4.12).



Rysunek 4.12. Strona logowania zawierająca link do strony odzyskiwania hasła

Kliknij link *Zapomniałeś hasła?*. Powinieneś zobaczyć stronę podobną do pokazanej na rysunku 4.13.



Rysunek 4.13. Formularz przywracania hasła

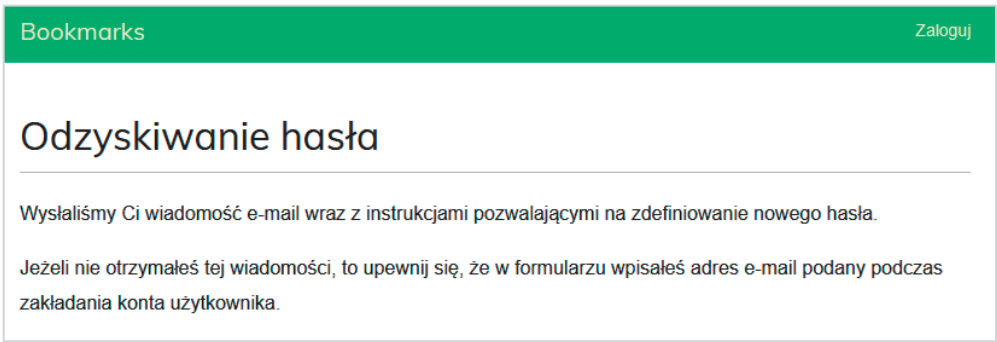
Na tym etapie, aby umożliwić Django wysyłanie wiadomości e-mail, w pliku *settings.py* projektu trzeba umieścić konfigurację serwera **SMTP**. Procedurę dodawania tego rodzaju konfiguracji do projektu omówiłem w rozdziale 2., „Usprawnianie bloga i dodanie funkcjonalności społecznościowych". Jednak podczas pracy nad aplikacją można skonfigurować Django do przekazywania wiadomości e-mail na standardowe wyjście zamiast ich faktycznego wysyłania za pomocą serwera SMTP. Framework Django oferuje mechanizm backend do wyświetlania wiadomości e-mail w konsoli.

W pliku *settings.py* swojego projektu dodaj poniższy fragment kodu.

```
EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'
```

Ustawienie `EMAIL_BACKEND` wskazuje klasę, która będzie używana do wysyłania wiadomości e-mail.

Wróć do przeglądarki internetowej, podaj adres e-mail istniejącego użytkownika i kliknij przycisk *WYŚLIJ WIADOMOŚĆ E-MAIL*. Powinieneś zobaczyć stronę podobną do pokazanej na rysunku 4.14.



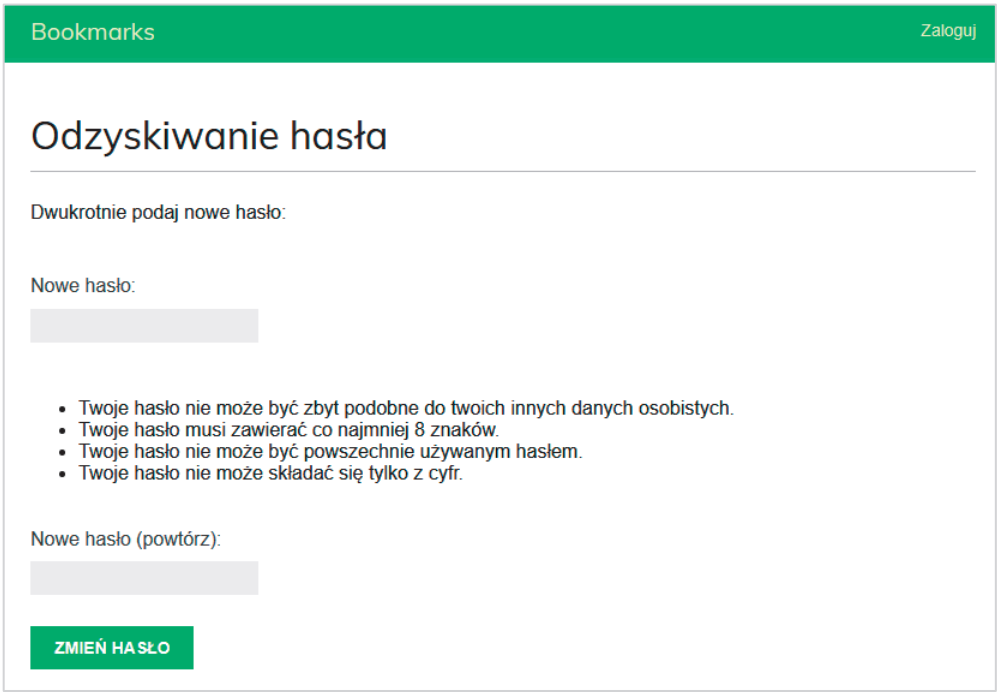
Rysunek 4.14. Strona z informacją o wysłaniu wiadomości e-mail dotyczącej odzyskiwania hasła

Spójrz na konsolę, na której został uruchomiony serwer programistyczny. Powinieneś zobaczyć wygenerowaną wiadomość e-mail.

```
Content-Type: text/plain; charset="utf-8"
MIME-Version: 1.0
Content-Transfer-Encoding: 8bit
Subject:
 =?utf-8?q?Reset_has=C5=82a_dla_konta_na_stronie_127=2E0=2E0=2E1=3A8000?=
From: webmaster@localhost
To: test@test.pl
Date: Sun, 22 Sep 2024 04:44:14 -0000
Message-ID: <172698025475.19188.14083776009677761263@DESKTOP-3ADGNTT>
Otrzymaliśmy żądanie odzyskania hasła dla użytkownika używającego adresu e-mail
↪test@test.pl. Kliknij poniższy link:
http://127.0.0.1:8000/account/password-reset/Mg/cdqtdq-76b2060878e0441fc8d4f60133257372/
Twoja nazwa użytkownika, na wypadek gdybyś zapomniał: test
```

Ta wiadomość e-mail została wygenerowana za pomocą utworzonego wcześniej szablonu *password_reset_email.html*. Adres URL pozwalający na przejście do strony odzyskiwania hasła zawiera token dynamicznie wygenerowany przez Django.

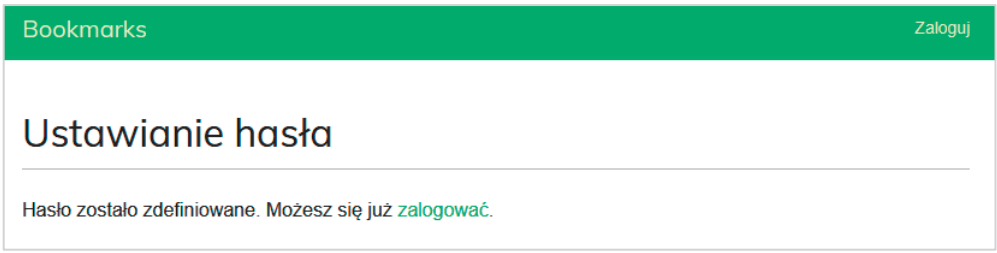
Skopiuj z wiadomości e-mail adres URL, który powinien wyglądać podobnie do następującego: `http://127.0.0.1:8000/account/password-reset/MQ/ardx0u-b4973cfa2c70d652a1↪90e79054bc479a/`, i otwórz go w przeglądarce. Powinieneś zobaczyć stronę podobną do pokazanej na rysunku 4.15.



Rysunek 4.15. Formularz odzyskiwania hasła

To jest strona umożliwiająca użytkownikowi podanie nowego hasła; odpowiada ona szablonowi *password_reset_confirm.html*. W obu polach formularza wpisz nowe hasło, a następnie kliknij przycisk *ZMIEŃ HASŁO*.

Django utworzy nowy skrót hasła i zapisze go w bazie danych. Jeżeli operacja przebiegnie bez problemów, zostanie wyświetlona strona pokazana na rysunku 4.16 wraz z komunikatem informującym o sukcesie.



Rysunek 4.16. Strona z komunikatem o pomyślnym odzyskaniu hasła

Teraz użytkownik może zalogować się na swoje konto, podając nowe hasło.

Każdy token przeznaczony do ustawienia nowego hasła może być użyty tylko jednokrotnie. Jeżeli ponownie otworzysz w przeglądarce internetowej otrzymany link, zostanie wyświetlony komunikat informujący o nieprawidłowym tokenie.

W ten sposób w projekcie zintegrowałeś widoki oferowane przez framework uwierzytelniania w Django. Wspomniane widoki są odpowiednie do użycia w większości sytuacji. Jednak zawsze możesz utworzyć własne widoki, jeśli potrzebna jest obsługa niestandardowego zachowania.

Wzorce adresów URL uwierzytelniania, które właśnie utworzyliśmy, udostępnia również framework Django. Teraz zastąpimy wzorce adresów URL uwierzytelniania wzorcami dostarczonymi przez Django.

Ujmij w komentarz wzorce adresów URL uwierzytelniania, które dodaliśmy do pliku *urls.py* aplikacji `account`, i zamiast tego dodaj aplikację `django.contrib.auth.urls` tak, jak pokazałem poniżej. Nowy kod został wyróżniony pogrubioną czcionką.

```
from django.urls import include, path
from django.contrib.auth import views as auth_views
from . import views

urlpatterns = [
    # poprzedni widok logowania
    # path('login/', views.user_login, name='login'),

    # path('login/', auth_views.LoginView.as_view(), name='login'),
    # path('logout/', auth_views.LogoutView.as_view(), name='logout'),

    # Adresy URL przeznaczone do obsługi zmiany hasła
    # path(
    #   'password-change/',
    #   auth_views.PasswordChangeView.as_view(),
```

```
        #    name='password_change'
        # ),
        # path(
        #    'password-change/done/',
        #    auth_views.PasswordChangeDoneView.as_view(),
        #    name='password_change_done'
        # ),
        # Adresy URL przeznaczone do obsługi procedury odzyskiwania hasła
        # path(
        #    'password-reset/',
        #    auth_views.PasswordResetView.as_view(),
        #    name='password_reset'
        # ),
        # path(
        #    'password-reset/done/',
        #    auth_views.PasswordResetDoneView.as_view(),
        #    name='password_reset_done'
        # ),
        # path(
        #    'password-reset/<uidb64>/<token>/',
        #    auth_views.PasswordResetConfirmView.as_view(),
        #     name='password_reset_confirm'
        # ),
        # path(
        #    'password-reset/complete/',
        #    auth_views.PasswordResetCompleteView.as_view(),
        #    name='password_reset_complete'
        # ),

        path('', include('django.contrib.auth.urls')),
        path('', views.dashboard, name='dashboard'),
    ]
```

Wzorce URL uwierzytelniania można znaleźć pod adresem *https://github.com/django/django/blob/stable/5.0.x/django/contrib/auth/urls.py*.

Dodaliśmy teraz do naszego projektu wszystkie niezbędne widoki uwierzytelniania. Następnie zaimplementujemy mechanizmy rejestracji użytkowników.

Rejestracja użytkownika i profile użytkownika

Istniejący użytkownicy mogą się zalogować, wylogować, zmienić hasło lub je odzyskać, jeśli go zapomną. Musimy teraz przygotować widok pozwalający nowym odwiedzającym witrynę na założenie w niej konta użytkownika. Powinni oni być w stanie zarejestrować się na stronie i utworzyć profil. Po zarejestrowaniu użytkownicy będą mogli zalogować się w aplikacji przy użyciu swoich poświadczeń.

Rejestracja użytkownika

Przystępujemy do utworzenia prostego widoku pozwalającego odwiedzającemu na zarejestrowanie się w witrynie internetowej. Zaczniemy od formularza, w którym nowy użytkownik wprowadzi nazwę użytkownika, swoje imię i nazwisko oraz hasło.

Przeprowadź edycję pliku *forms.py* w katalogu aplikacji `account` i umieść w nim poniższy fragment kodu.

```
from django import forms
from django.contrib.auth import get_user_model


class LoginForm(forms.Form):
    username = forms.CharField()
    password = forms.CharField(widget=forms.PasswordInput)


class UserRegistrationForm(forms.ModelForm):
    password = forms.CharField(
        label='Password',
        widget=forms.PasswordInput
    )
    password2 = forms.CharField(
        label='Repeat password',
        widget=forms.PasswordInput
    )

    class Meta:
        model = get_user_model()
        fields = ['username', 'first_name', 'email']
```

Utworzyliśmy formularz modelu (klasa `ModelForm`) dla modelu `User`. W przygotowanym formularzu będą uwzględnione jedynie pola `username`, `first_name` i `email`. Model użytkownika uzyskujemy dynamicznie, za pomocą funkcji `get_user_model()` dostarczanej przez aplikację `auth`. Framework Django pozwala na definiowanie niestandardowych modeli użytkowników, dlatego uruchomienie tej funkcji powoduje pobranie modelu użytkownika, który może być niestandardowym modelem, zamiast domyślnego modelu `User` aplikacji `auth`. Wartości wymienionych pól będą weryfikowane na podstawie odpowiadających im pól modelu. Jeśli na przykład użytkownik wybierze już istniejącą nazwę użytkownika, otrzyma komunikat o błędzie w trakcie walidacji formularza, ponieważ pole `username` jest zdefiniowane z ustawieniem `unique=True`.

Uwaga

Aby kod był generyczny, do pobrania modelu użytkownika używamy funkcji `get_user_model()`. Następnie, podczas definiowania relacji z tym modelem zamiast bezpośredniego odwoływania się do modelu `User` można korzystać z opcji `AUTH_USER_MODEL`. Więcej informacji na ten temat można przeczytać na stronie *https://docs.djangoproject.com/en/5.0/topics/auth/customizing/#django.contrib.auth.get_user_model*.

Dodaliśmy dwa nowe pola `password` i `password2` przeznaczone do zdefiniowania hasła i jego potwierdzenia. Dodajmy kod walidacji pola, aby sprawdzić, czy oba hasła są takie same.

Zmodyfikuj plik *forms.py* w aplikacji `account` i dodaj do klasy `UserRegistrationForm` zamieszczoną poniżej metodę `clean_password2()`. Nowy kod został wyróżniony pogrubioną czcionką.

```
class UserRegistrationForm(forms.ModelForm):
    password = forms.CharField(
        label='Hasło',
        widget=forms.PasswordInput
    )
    password2 = forms.CharField(
        label='Powtórz hasło',
        widget=forms.PasswordInput
    )

    class Meta:
        model = get_user_model()
        fields = ['username', 'first_name', 'email']

    def clean_password2(self):
        cd = self.cleaned_data
        if cd['password'] != cd['password2']:
            raise forms.ValidationError("Hasła nie są identyczne.")
        return cd['password2']
```

Zdefiniowaliśmy metodę `clean_password2()` w celu porównywania drugiego hasła z pierwszym i zgłoszenia błędu sprawdzania poprawności, jeśli hasła nie pasują. Metoda ta jest wywoływana podczas walidacji formularza za pomocą jego metody `is_valid()`. Istnieje możliwość dostarczenia metody `clean_<nazwa_pola>()` dla dowolnego pola formularza w celu wyczyszczenia jego wartości lub zgłoszenia błędu walidacji formularza dla określonego pola. Formularze zawierają także ogólną metodę `clean()` przeznaczoną do sprawdzenia całego formularza, co okazuje się użyteczne podczas walidacji pól zależnych wzajemnie od siebie. W tym przypadku zamiast zastępowania metody `clean()` formularza użyliśmy walidacji specyficznej dla pola `clean_password2()`. Pozwala to uniknąć zastępowania innych, specyficznych dla pól mechanizmów walidacji, które klasa `ModelForm` pobiera na podstawie mechanizmów kontroli (ang. *constraints*) ustawionych w modelu (na przykład sprawdzanie, czy nazwa użytkownika jest unikatowa).

Django oferuje również gotowy do natychmiastowego użycia formularz `UserCreationForm`. Znajdziesz go w aplikacji `django.contrib.auth.forms`, przy czym jest bardzo podobny do utworzonego przez nas wcześniej.

Przeprowadź edycję pliku *views.py* aplikacji `account` i umieść w nim poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
from django.contrib.auth import authenticate, login
from django.contrib.auth.decorators import login_required
from django.http import HttpResponse
from django.shortcuts import render
from .forms import LoginForm, UserRegistrationForm
```

```
# ...

def register(request):
    if request.method == 'POST':
        user_form = UserRegistrationForm(request.POST)
        if user_form.is_valid():
            # Utworzenie nowego obiektu użytkownika; jednak jeszcze nie zapisujemy go
            # w bazie danych
            new_user = user_form.save(commit=False)
            # Ustawienie wybranego hasła
            new_user.set_password(
                user_form.cleaned_data['password']
            )
            # Zapisanie obiektu User
            new_user.save()
            return render(
                request,
                'account/register_done.html',
                {'new_user': new_user},
            )
    else:
        user_form = UserRegistrationForm()
    return render(
        request,
        'account/register.html',
        {'user_form': user_form}
    )
```

Widok pozwalający na utworzenie nowego konta użytkownika jest całkiem prosty. Ze względów bezpieczeństwa zamiast zapisywać wprowadzone przez użytkownika hasło w postaci zwykłego tekstu, wykorzystujemy metodę `set_password()` modelu `User`. Metoda ta obsługuje haszowanie hasła przed zapisaniem go w bazie danych.

Django nie przechowuje haseł w postaci zwykłego tekstu; zamiast tego przechowuje skróty (ang. *hashes*) haseł. Haszowanie to proces przekształcania podanego klucza w inną wartość (tzw. skrót). Funkcja skrótu służy do generowania wartości o stałej długości zgodnie z algorytmem matematycznym. Dzięki haszowaniu haseł za pomocą bezpiecznych algorytmów Django znacząco utrudnia złamanie haseł przechowywanych w bazie danych.

Domyślnie do przechowywania wszystkich haseł użytkowników Django używa algorytmu haszowania `PBKDF2` ze skrótem SHA256. Framework obsługuje nie tylko sprawdzanie istniejących skrótów haseł obliczonych za pomocą algorytmu `PBKDF2`, ale także sprawdzanie przechowywanych skrótów haseł obliczonych za pomocą innych algorytmów, takich jak `PBKDF2SHA1`, `argon2`, `bcrypt` i `scrypt`.

Skróty haseł obsługiwane przez projekt Django są zdefiniowane za pomocą ustawienia `PASSWORD_HASHERS`. Oto domyślna lista `PASSWORD_HASHERS`:

```
PASSWORD_HASHERS = [
    'django.contrib.auth.hashers.PBKDF2PasswordHasher',
    'django.contrib.auth.hashers.PBKDF2SHA1PasswordHasher',
    'django.contrib.auth.hashers.Argon2PasswordHasher',
```

```
    'django.contrib.auth.hashers.BCryptSHA256PasswordHasher',
    'django.contrib.auth.hashers.ScryptPasswordHasher',
]
```

Do haszowania wszystkich haseł Django używa pierwszego wpisu na liście, w tym przypadku `PBKDF2PasswordHasher`. Reszty hasherów z listy Django może używać do sprawdzania istniejących haseł.

Uwaga

Hasher `scrypt` wprowadzono w Django 4.0. Jest bezpieczniejszy i bardziej zalecany niż `PBKDF2`. `PBKDF2` nadal jest jednak domyślnym hasherem, ponieważ `scrypt` wymaga zainstalowania OpenSSL 1.1+ i więcej pamięci.

Więcej informacji o sposobach przechowywania haseł w Django i o hasherach możesz znaleźć na stronie *https://docs.djangoproject.com/en/5.0/topics/auth/passwords/*.

Otwórz plik *urls.py* aplikacji `account` i umieść w nim poniższe wzorce adresów URL oznaczone pogrubioną czcionką.

```
urlpatterns = [

    # ...

    path('', include('django.contrib.auth.urls')),
    path('', views.dashboard, name='dashboard'),
    path('register/', views.register, name='register'),
]
```

Na koniec utwórz nowy szablon w katalogu *templates/account*, nadaj mu nazwę *register.html*, a następnie umieść w nim poniższy kod.

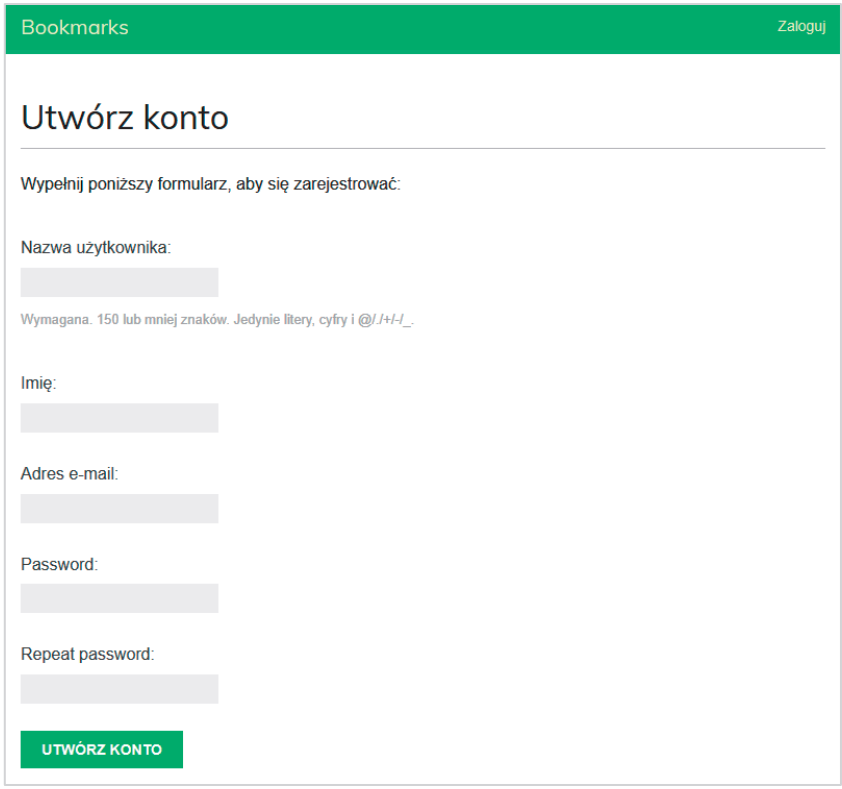
```
{% extends "base.html" %}

{% block title %}Utwórz konto{% endblock %}

{% block content %}
  <h1>Utwórz konto</h1>
  <p> Wypełnij poniższy formularz, aby się zarejestrować:</p>
  <form method="post">
    {{ user_form.as_p }}
    {% csrf_token %}
    <p><input type="submit" value="Utwórz konto"></p>
  </form>
{% endblock %}
```

Do tego samego katalogu dodaj nowy plik szablonu o nazwie *register_done.html*. Następnie w nowym pliku umieść poniższy fragment kodu.

```
{% extends "base.html" %}

{% block title %}Witaj{% endblock %}

{% block content %}
  <h1>Witaj {{ new_user.first_name }}!</h1>
  <p>
```

```
      Twoje konto zostało utworzone.
      Możesz się już <a href="{% url "login" %}">zalogować</a>.
    </p>
  {% endblock %}
```

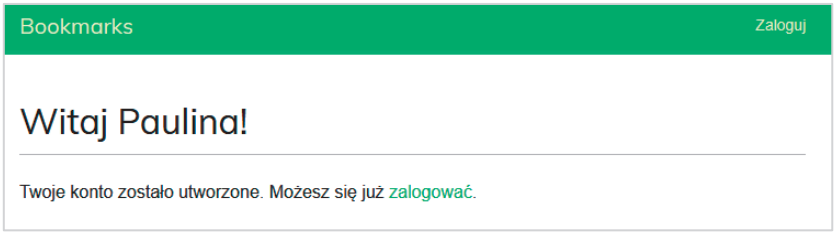
W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/account/register/`. Powinieneś zobaczyć wyświetloną stronę rejestracji nowego użytkownika (rysunek 4.17).



Rysunek 4.17. Formularz tworzenia konta

Podaj informacje potrzebne do utworzenia nowego konta użytkownika i kliknij przycisk *UTWÓRZ KONTO*.

Jeżeli wszystkie pola wypełnisz poprawnie, zostanie wyświetlona strona wraz z komunikatem informującym o pomyślnym utworzeniu nowego konta użytkownika (rysunek 4.18).



Rysunek 4.18. Strona z komunikatem o pomyślnym utworzeniu konta

Kliknij *Zaloguj*, a następnie podaj dane uwierzytelniające utworzonego przed chwilą użytkownika, aby potwierdzić, że możesz uzyskać dostęp do nowego konta.

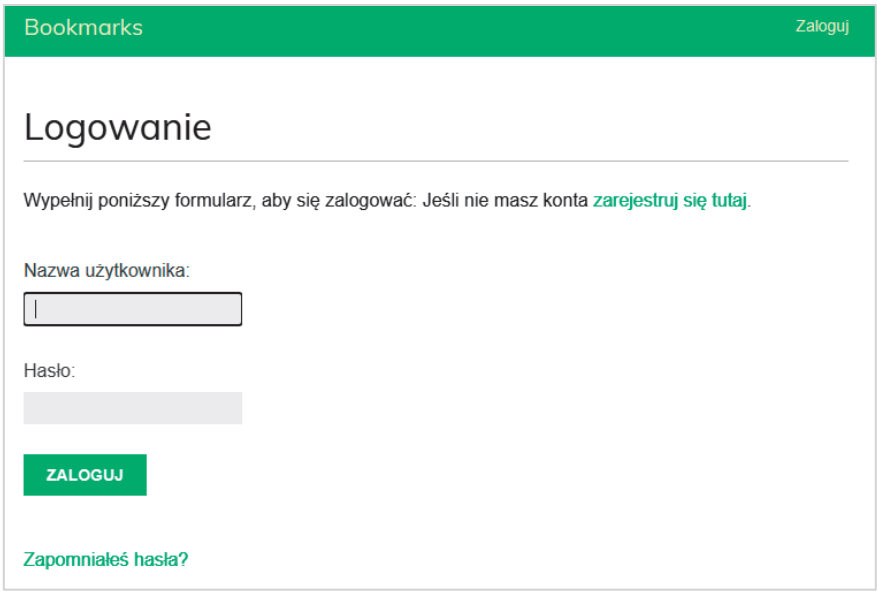
Teraz do szablonu logowania trzeba dodać link pozwalający na rejestrację użytkownika. Przeprowadź edycję szablonu *registration/login.html* i poniższy wiersz kodu:

```
<p>Wypełnij poniższy formularz, aby się zalogować:</p>
```

zastąp wierszami:

```
<p>
  Wypełnij poniższy formularz, aby się zalogować:
  Jeśli nie masz konta <a href="{% url "register" %}">zarejestruj się tutaj</a>.
</p>
```

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/account/login/`. Wynik powinien być podobny do pokazanego na rysunku 4.19.



Rysunek 4.19. Strona logowania zawierająca link do rejestracji

Stworzyliśmy stronę rejestracji dostępną ze strony logowania.

Rozbudowa modelu User

O ile model `User` dostarczany przez framework uwierzytelniania Django jest wystarczający do większości typowych scenariuszy, o tyle ma on ograniczony zestaw predefiniowanych pól. Jeśli chcesz uwzględnić dodatkowe szczegóły istotne dla Twojej aplikacji, możesz rozszerzyć domyślny model użytkownika. Na przykład domyślny model użytkownika zawiera pola `first_name` i `last_name`. Taka struktura może nie być zgodna z konwencjami pisowni nazwisk w niektórych krajach. Dodatkowo czasami trzeba przechowywać inne dane użytkownika lub skonstruować bardziej kompleksowy profil użytkownika.

Prostym sposobem na rozszerzenie modelu `User` jest utworzenie modelu profilu, który zawiera relację „jeden do jednego" z modelem `User` frameworka Django oraz wszelkie dodatkowe pola. Relacja „jeden do jednego" jest podobna do stosowania pola `ForeignKey` z parametrem `unique = True`. Druga strona relacji to niejawna relacja „jeden do jednego" z powiązanym modelem zamiast menedżera wielu elementów. Z każdej strony relacji pobieramy jeden powiązany obiekt.

Przeprowadź edycję pliku *models.py* aplikacji `account` i umieść w nim poniższy fragment kodu oznaczony pogrubioną czcionką.

```
from django.db import models
from django.conf import settings


class Profile(models.Model):
    user = models.OneToOneField(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE
    )
    date_of_birth = models.DateField(blank=True, null=True)
    photo = models.ImageField(
        upload_to='users/%Y/%m/%d/',
        blank=True
    )

    def __str__(self):
        return f'Profil użytkownika {self.user.username}'
```

Utworzony profil użytkownika będzie zawierał datę urodzenia użytkownika oraz jego zdjęcie.

Do powiązania profili z użytkownikami służy pole relacji „jeden do jednego" `user`. Aby odwoływać się do modelu użytkownika, zamiast wskazywać bezpośrednio na model `auth.User`, używamy ustawienia `AUTH_USER_MODEL`. Dzięki temu kod jest bardziej generyczny, ponieważ można go obsługiwać za pomocą niestandardowych modeli użytkowników. Użyliśmy klauzuli `CASCADE` dla parametru `on_delete`, aby po usunięciu użytkownika został również usunięty powiązany z nim profil.

Pole `date_of_birth` jest typu `DateField`. Ustawiliśmy to pole jako opcjonalne za pomocą opcji `blank=True`, a dzięki opcji `null=True` dopuściliśmy wprowadzanie wartości `null`.

Zdjęcie użytkownika jest przechowywane w polu `photo` typu `ImageField`. Ustawiliśmy to pole jako opcjonalne za pomocą opcji `blank=True`. Pole `ImageField` służy do przechowywania plików ze zdjęciami. Django sprawdza, czy podany plik jest prawidłowym zdjęciem, zapisuje plik ze zdjęciem w katalogu wskazanym przez parametr `upload_to` i zapisuje względną ścieżkę do pliku w odpowiednim polu bazy danych. Pole `ImageField` w bazie danych jest domyślnie tłumaczone na kolumnę `VARHAR(100)`. Jeśli wartość pola jest pusta, w bazie danych zostanie zapisany pusty ciąg.

Instalowanie modułu Pillow i udostępnianie plików multimedialnych

W celu zarządzania obrazami konieczne będzie zainstalowanie pakietu Pythona `Pillow`. Moduł `Pillow` jest standardem de facto przetwarzania zdjęć w Pythonie. Obsługuje wiele formatów obrazów i zapewnia zaawansowane funkcje przetwarzania obrazu. Moduł `Pillow` jest wymagany przez Django do obsługi zdjęć zapisanych w polach typu `ImageField`.

Zainstaluj pakiet `Pillow` przez wydanie w powłoce poniższego polecenia.

```
python -m pip install Pillow==10.3.0
```

Przeprowadź edycję pliku *settings.py* projektu i dodaj w nim poniższe wiersze kodu.

```
MEDIA_URL = 'media/'
MEDIA_ROOT = BASE_DIR / 'media'
```

Wprowadzenie tych ustawień umożliwi frameworkowi Django zarządzanie przesyłaniem plików i udostępnianiem plików multimedialnych. Opcja `MEDIA_URL` wskazuje bazowy adres URL określający lokalizację przeznaczoną do przechowywania plików multimedialnych przekazywanych przez użytkowników. Natomiast opcja `MEDIA_ROOT` określa lokalną ścieżkę dostępu dla tych plików. Ścieżki i adresy URL plików są budowane dynamicznie przez dodawanie do nich ścieżki projektu lub adresu URL multimediów, co ułatwia przenośność plików.

Teraz w głównym pliku *urls.py* projektu `bookmarks` zmodyfikuj znajdujący się w nim kod w pokazany niżej sposób. Nowe wiersze zostały wyróżnione pogrubioną czcionką.

```
from django.conf import settings
from django.conf.urls.static import static
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('account/', include('account.urls')),
]

if settings.DEBUG:
    urlpatterns += static(
        settings.MEDIA_URL,
         document_root=settings.MEDIA_ROOT
    )
```

Do serwowania plików multimedialnych przez serwer programistyczny Django podczas programowania (to znaczy gdy opcja `DEBUG` jest ustawiona na `True`) dodaliśmy funkcję pomocniczą `static()`.

Uwaga

Funkcja pomocnicza `static()` jest odpowiednia do stosowania w środowisku programistycznym, ale na pewno nie w produkcyjnym. Serwowanie statycznych plików przez Django jest bardzo niewydajne. Pamiętaj, aby w środowisku produkcyjnym nigdy nie udostępniać plików statycznych za pomocą Django. Sposób serwowania statycznych plików w środowisku produkcyjnym omówiłem w rozdziale 17., „Wdrożenie”.

Tworzenie migracji dla modelu Profile

Utwórzmy tabelę bazy danych dla nowego modelu `Profile`. Przejdź do powłoki i wydaj następujące polecenie, które spowoduje utworzenie migracji bazy danych dla nowego modelu.

```
python manage.py makemigrations
```

Powinieneś otrzymać taki wynik:

```
Migrations for 'account':
   account/migrations/0001_initial.py
     - Create model Profile
```

Kolejnym krokiem jest zsynchronizowanie bazy danych za pomocą poniższego polecenia:

```
python manage.py migrate
```

Wygenerowany wynik będzie zawierał m.in. następujący wiersz.

```
Applying account.0001_initial... OK
```

Przeprowadź edycję pliku *admin.py* aplikacji `account` i zarejestruj model `Profile` w witrynie administracyjnej, co pokazałem poniżej.

```
from django.contrib import admin
from .models import Profile

@admin.register(Profile)
class ProfileAdmin(admin.ModelAdmin):
    list_display = ['user', 'date_of_birth', 'photo']
    raw_id_fields = ['user']
```

Uruchom serwer programistyczny za pomocą polecenia:

```
python manage.py runserver
```

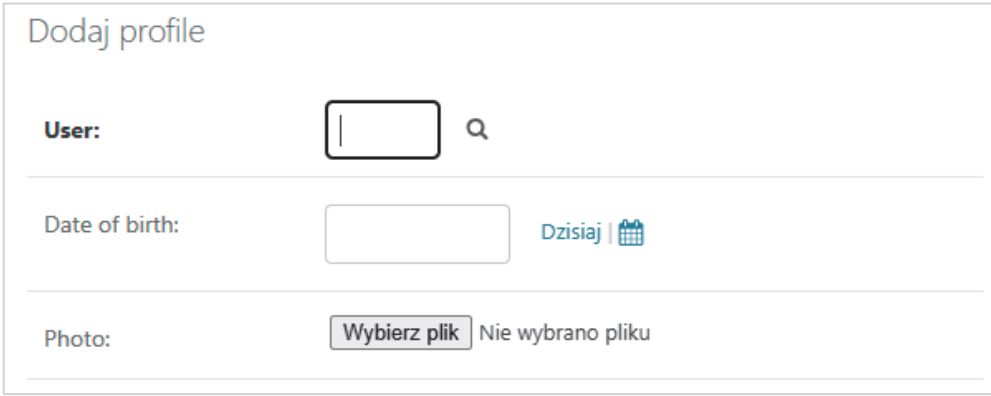
W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/admin/`. Teraz będziesz mógł zobaczyć model `Profile` w witrynie administracyjnej projektu, co pokazałem na rysunku 4.20.

Rysunek 4.20. Blok ACCOUNT głównej strony witryny administracyjnej

W wierszu *Profiles* kliknij *Dodaj*. Wyświetli się formularz służący do dodawania nowego profilu, pokazany na rysunku 4.21.

Utwórz ręcznie obiekt `Profile` dla każdego istniejącego użytkownika w bazie danych.

Teraz zajmiemy się umożliwieniem użytkownikom edycji profilu w witrynie internetowej.



Rysunek 4.21. Formularz dodawania profilu

Przeprowadź edycję pliku *forms.py* w aplikacji `account` i umieść w nim poniższe wiersze oznaczone pogrubioną czcionką.

```
# ...
from .models import Profile

# ...

class UserEditForm(forms.ModelForm):
    class Meta:
        model = get_user_model()
        fields = ['first_name', 'last_name', 'email']

class ProfileEditForm(forms.ModelForm):
     class Meta:
        model = Profile
        fields = ['date_of_birth', 'photo']
```

Oto krótkie omówienie dodanych formularzy:

- `UserEditForm`: formularz pozwala użytkownikowi na edycję imienia, nazwiska i adresu e-mail. Wymienione informacje są przechowywane we wbudowanym w Django modelu `User`.
- `ProfileEditForm`: formularz pozwala użytkownikowi na edycję danych dodatkowych, które zostaną zapisane w modelu `Profile`. Użytkownik będzie mógł podać datę urodzenia i wczytać obraz (tzw. awatar) dla swojego profilu.

Przeprowadź edycję pliku *views.py* w aplikacji `account` i umieść w nim poniższe wiersze oznaczone pogrubioną czcionką.

```
# ...
from .models import Profile

# ...

def register(request):
    if request.method == 'POST':
```

```
            user_form = UserRegistrationForm(request.POST)
            if user_form.is_valid():
                # Utworzenie nowego obiektu użytkownika; jednak jeszcze nie zapisujemy go
                # w bazie danych
                new_user = user_form.save(commit=False)
                # Ustawienie wybranego hasła
                new_user.set_password(
                    user_form.cleaned_data['password']
                )
                # Zapisanie obiektu User
                new_user.save()
                # Utworzenie profilu użytkownika
                Profile.objects.create(user=new_user)
                return render(
                    request,
                    'account/register_done.html',
                    {'new_user': new_user}
                )
        else:
            user_form = UserRegistrationForm()
        return render(
            request,
            'account/register.html',
            {'user_form': user_form}
        )
```

Gdy użytkownicy rejestrują się w witrynie, tworzony jest obiekt `Profile`, który jest powiązany z utworzonym obiektem `User`. Jednak dla użytkowników utworzonych za pośrednictwem witryny administracyjnej nie zostanie automatycznie utworzony powiązany obiekt `Profile`. W bazie danych mogą współistnieć użytkownicy z profilem, jak i bez niego (np. użytkownicy uprzywilejowani — ang. *staff users*).

Aby wymusić tworzenie profili dla wszystkich użytkowników, możesz użyć sygnałów Django. W ten sposób można zainicjować utworzenie obiektu `Profile` za każdym razem, gdy tworzony jest użytkownik. O sygnałach dowiesz się w rozdziale 7., „Śledzenie działań użytkownika", w którym w podrozdziale „Rozszerzanie projektu przy użyciu sztucznej inteligencji" wykonamy ćwiczenie implementujące tę funkcjonalność.

Teraz umożliwimy użytkownikowi edycję profilu.

Przeprowadź edycję pliku *views.py* aplikacji `account` i umieść w nim poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
from django.contrib.auth import authenticate, login
from django.contrib.auth.decorators import login_required
from django.http import HttpResponse
from django.shortcuts import render
from .forms import (
    LoginForm,
    UserRegistrationForm,
    UserEditForm,
    ProfileEditForm
)
from .models import Profile
```

```
# ...
@login_required
def edit(request):
    if request.method == 'POST':
        user_form = UserEditForm(
            instance=request.user,
            data=request.POST
        )
        profile_form = ProfileEditForm(
            instance=request.user.profile,
            data=request.POST,
            files=request.FILES
        )
        if user_form.is_valid() and profile_form.is_valid():
            user_form.save()
            profile_form.save()
    else:
        user_form = UserEditForm(instance=request.user)
        profile_form = ProfileEditForm(instance=request.user.profile)
    return render(
        request,
        'account/edit.html',
        {
            'user_form': user_form,
            'profile_form': profile_form
        }
    )
```

Aby umożliwić użytkownikom modyfikowanie swoich danych osobowych, dodaliśmy nowy widok `edit`. Dodaliśmy do widoku dekorator `login_required`, ponieważ tylko uwierzytelnieni użytkownicy będą mogli edytować swoje profile.

W tym widoku używamy dwóch formularzy opartych na modelu `UserEditForm`, przeznaczonym do przechowywania danych wbudowanego modelu User i `ProfileEditForm`, przeznaczonym do przechowywania dodatkowych danych profilu. Aby zweryfikować poprawność danych wysłanych w formularzu, sprawdzamy, czy wartością zwrotną metody `is_valid()` w obu wymienionych formularzach jest `True`. Jeżeli oba formularze zawierają prawidłowe dane, zapisujemy zawartość obu formularzy. W tym celu wywołujemy metodę `save()`, która uaktualnia odpowiedni obiekt w bazie danych.

Dodaj poniższy wzorzec adresu URL do pliku *urls.py* aplikacji `account`.

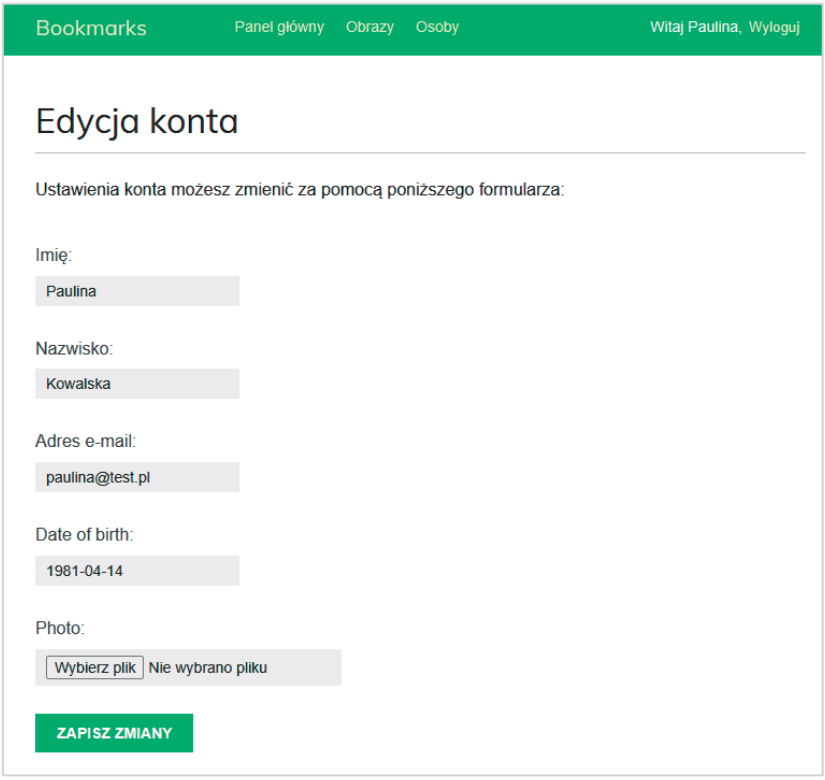
```
urlpatterns = [
    #...
    path('', include('django.contrib.auth.urls')),
    path('', views.dashboard, name='dashboard'),
    path('register/', views.register, name='register'),
    path('edit/', views.edit, name='edit'),
]
```

Na koniec w katalogu *templates/account/* utwórz nowy szablon dla widoku i nadaj mu nazwę *edit.html*. Następnie w tym pliku umieść poniższy fragment kodu.

```
{% extends "base.html" %}

{% block title %}Edycja konta{% endblock %}

{% block content %}
  <h1>Edycja konta</h1>
  <p>Ustawienia konta możesz zmienić za pomocą poniższego formularza:</p>
  <form method="post" enctype="multipart/form-data">
    {{ user_form.as_p }}
    {{ profile_form.as_p }}
    {% csrf_token %}
    <p><input type="submit" value="Zapisz zmiany"></p>
  </form>
{% endblock %}
```

Aby umożliwić przekazywanie plików, w powyższym kodzie dodaliśmy do elementu HTML `<form>` opcję `enctype="multipart/form-data"`. Wykorzystujemy tylko jeden formularz HTML do wysłania obu formularzy Django, czyli `user_form` i `profile_form`.

Otwórz w przeglądarce stronę `http://127.0.0.1:8000/account/register/` i zarejestruj nowego użytkownika. Następnie zaloguj się jako nowy użytkownik i otwórz stronę `http://127.0.0.1:8000/account/edit/`. Powinieneś zobaczyć stronę podobną do pokazanej na rysunku 4.22.



Rysunek 4.22. Formularz edycji profilu

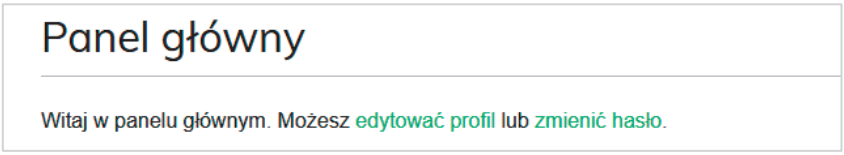
Możesz teraz dodać informacje o profilu i zapisać zmiany.

Teraz możemy zmodyfikować stronę panelu głównego i umieścić na niej linki prowadzące do stron pozwalających na edycję profilu i zmianę hasła.

Otwórz w przeglądarce szablon *templates/account/dashboard.html* i dodaj następujące wiersze wyróżnione pogrubioną czcionką.

```
{% extends "base.html" %}

{% block title %}Panel główny{% endblock %}

{% block content %}
  <h1>Panel główny</h1>
  <p>
    Witaj w panelu głównym. Możesz <a href="{% url "edit" %}">edytować profil</a>
    ↪lub <a href="{% url "password_change" %}">zmienić hasło</a>.
  </p>
{% endblock %}
```

Po wprowadzonych zmianach użytkownik będzie miał z poziomu panelu głównego dostęp do formularza umożliwiającego edycję profilu. Otwórz w przeglądarce stronę `http://127.0.0.1:8000/account/` i przetestuj nowy link edycji profilu użytkownika (rysunek 4.23). Panel główny powinien teraz wyglądać jak na rysunku 4.23.



Rysunek 4.23. Zawartość strony panelu głównego razem z linkami do edycji profilu i zmiany hasła

Użycie własnego modelu User

Django oferuje możliwość całkowitego zastąpienia modelu `User` własnym. Klasa modelu powinna dziedziczyć po klasie `AbstractUser` zapewniającej pełną implementację użytkownika domyślnego jako modelu abstrakcyjnego. Więcej o tej metodzie można przeczytać na stronie *https://docs.djangoproject.com/en/5.0/topics/auth/customizing/#substituting-a-custom-user-model*.

Zastosowanie własnego modelu użytkownika daje znacznie większą elastyczność, choć może oznaczać również nieco większą trudność podczas integracji z innymi aplikacjami, które współdziałają ze standardowym modelem `User`.

Podsumowanie

W tym rozdziale dowiedziałeś się, jak zaimplementować w budowanej witrynie internetowej system uwierzytelniania. Zaimplementowałeś wszystkie widoki potrzebne do rejestracji użytkownika, logowania i wylogowania oraz edycji i odzyskiwania hasła. Stworzyłeś również model do przechowywania niestandardowych profili użytkowników.

W następnym rozdziale poprawisz komfort użytkownika przez zaimplementowanie dostarczania informacji zwrotnych na temat jego działań za pomocą frameworka `messages` platformy Django. Rozszerzysz również zakres metod uwierzytelniania przez umożliwienie użytkownikom uwierzytelniania za pomocą adresu e-mail i zintegrowanie uwierzytelniania społecznościowego za pośrednictwem konta Google. Dowiesz się także, jak za pomocą rozszerzenia *Django Extensions* obsługiwać serwer programistyczny przez HTTPS oraz jak dostosować potok uwierzytelniania, aby automatycznie tworzyć profile użytkowników.

Dodatkowe zasoby

Oto lista zasobów zawierających dodatkowe informacje związane z tematami omawianymi w tym rozdziale:

- Kod źródłowy przykładów zamieszczonych w tym rozdziale — *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter04*.
- Wbudowane widoki uwierzytelniania — *https://docs.djangoproject.com/en/5.0/topics/auth/default/#all-authentication-views*.
- Wzorce adresów URL uwierzytelniania — *https://github.com/django/django/blob/stable/3.0.x/ django/contrib/auth/urls.py*.
- W jaki sposób Django zarządza hasłami i dostępnymi mechanizmami haszowania — *https://docs.djangoproject.com/en/5.0/topics/auth/passwords/*.
- Generyczny model użytkownika i metoda `get_user_model()` — *https://docs.djangoproject.com/en/5.0/topics/auth/customizing/#django.contrib.auth.get_user_model*.
- Korzystanie z niestandardowego modelu użytkownika — *https://docs.djangoproject.com/en/5.0/topics/auth/customizing/#substituting-a-custom-user-model*.

Implementacja uwierzytelniania za pomocą witryn społecznościowych

Rozdział 5

W poprzednim rozdziale utworzyłeś w projektowanej witrynie internetowej mechanizmy rejestracji i uwierzytelniania użytkowników. Zaimplementowałeś funkcjonalności zmiany i odzyskiwania hasła oraz nauczyłeś się tworzyć dla swoich użytkowników niestandardowy model profilu.

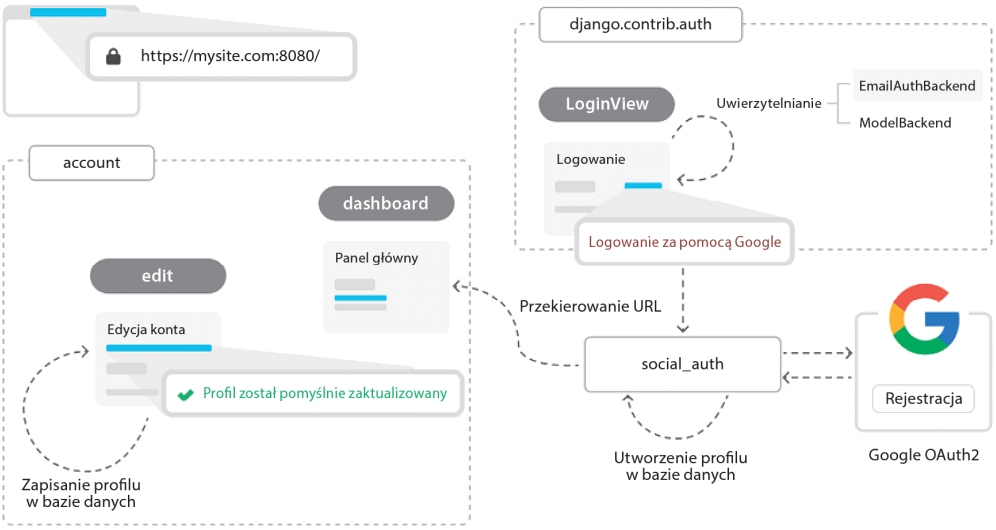
W tym rozdziale dodasz do swojej witryny mechanizmy uwierzytelniania za pomocą serwisu Google. Do zaimplementowania uwierzytelniania społecznościowego za pomocą OAuth 2.0 — standardowego w branży protokołu autoryzacji — użyjemy frameworka `Python Social Auth for Django`. Zmodyfikujesz również potok uwierzytelniania społecznościowego, aby automatycznie utworzyć profil użytkownika dla nowych użytkowników.

Oto zagadnienia, na których skoncentruję się w tym rozdziale:

- Użycie frameworka `messages`
- Implementacja własnego backendu uwierzytelniania
- Uniemożliwianie użytkownikom korzystania z istniejącego adresu e-mail
- Implementacja uwierzytelniania społecznościowego za pomocą modułu `Python Social Auth`
- Uruchomienie serwera programistycznego za pośrednictwem HTTPS z wykorzystaniem `Django Extensions`
- Uwierzytelnianie za pomocą serwisu Google
- Tworzenie profilu dla użytkowników rejestrujących się z wykorzystaniem uwierzytelniania społecznościowego

Przegląd funkcjonalności

Reprezentację widoków, szablonów i funkcjonalności, które zbudujemy w tym rozdziale, przedstawiono na rysunku 5.1.



Rysunek 5.1. Schemat funkcjonalności zbudowanych w rozdziale 5.

W tym rozdziale wygenerujesz w widoku `edit` komunikaty o sukcesie i błędzie z wykorzystaniem frameworka Django `messages`. Aby użytkownicy mogli uwierzytelniać się za pomocą swoich adresów e-mail, zbudujesz nowy backend uwierzytelniania o nazwie `EmailAuthBackend`. Nauczysz się serwowania witryny przez HTTPS w fazie programowania za pomocą pakietu `Django Extensions`, a ponadto za pomocą pakietu `Python Social Auth` zaimplementujesz na swojej stronie uwierzytelnianie społecznościowe przez Google. Po pomyślnym uwierzytelnieniu użytkownicy zostaną przekierowani do widoku panelu głównego. W celu automatycznego tworzenia profilu użytkownika, gdy nowy użytkownik zostanie utworzony za pośrednictwem uwierzytelniania społecznościowego, odpowiednio zmodyfikujesz potok uwierzytelniania.

Wymagania techniczne

Kod źródłowy przykładów z tego rozdziału można znaleźć pod adresem *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter05*.

Wszystkie pakiety Pythona użyte w tym rozdziale są zawarte w pliku *requirements.txt* dołączonym do plików z kodem źródłowym przykładów z tego rozdziału. Aby zainstalować każdy pakiet Pythona, możesz postępować zgodnie z instrukcjami zamieszczonymi w poniższych podrozdziałach. Możesz także zainstalować wszystkie wymagania jednocześnie za pomocą polecenia `pip install -r requirements.txt`.

Użycie frameworka messages

Podczas obsługi działań podejmowanych przez użytkowników może wystąpić konieczność informowania ich o skutkach podjętych przez nich czynności, takich jak pomyślne utworzenie obiektu w bazie danych lub przesłanie formularza.

Django oferuje wbudowany framework `messages`, pozwalający na wyświetlanie użytkownikom jednorazowych powiadomień. To bardzo wygodny mechanizm, który zapewnia użytkownikowi natychmiastową informację zwrotną na temat wykonywanych przez niego działań. Dzięki temu interfejs jest bardziej intuicyjny i przyjazny dla użytkownika.

Framework `messages` jest dostarczany przez aplikację `django.contrib.messages`, która została domyślnie umieszczona na liście `INSTALLED_APPS` w pliku *settings.py* podczas tworzenia nowego projektu za pomocą polecenia `python manage.py startproject`. W pliku *settings.py*, w sekcji `MIDDLEWARE`, jest również oprogramowanie pośredniczące `django.contrib.messages.middleware.MessageMiddleware`.

Framework `messages` zapewnia prosty sposób dodawania komunikatów przeznaczonych do wyświetlenia użytkownikom. Wspomniane komunikaty są przechowywane domyślnie w pliku cookie (jeśli go nie ma, to w pamięci trwałej sesji) i wyświetlane podczas następnego żądania wykonywanego przez danego użytkownika. Framework `messages` można wykorzystać w widokach — w tym celu należy zaimportować odpowiedni moduł — a następnie można już dodawać nowe komunikaty za pomocą prostych skrótów, co pokazałem poniżej.

```
from django.contrib import messages
messages.error(request, 'Coś poszło nie tak')
```

Nowe wiadomości możesz tworzyć z wykorzystaniem metody `add_message()` lub dowolnej z wymienionych poniżej metod skrótów.

- `success()`: komunikat o sukcesie wyświetlany po pomyślnym zakończeniu operacji.
- `info()`: komunikat informacyjny.
- `warning()`: wystąpił pewien problem, ale jeszcze nie mamy do czynienia z niepowodzeniem.
- `error()`: działanie zakończyło się niepowodzeniem lub doszło do awarii.
- `debug()`: komunikaty diagnostyczne, które w środowisku produkcyjnym będą usunięte lub zignorowane.

Dodajmy do projektu obsługę komunikatów. Framework `messages` jest stosowany globalnie w projekcie. Do wyświetlania klientowi wszelkich dostępnych komunikatów użyjemy szablonu bazowego. Dzięki temu będziemy mogli informować klienta o wynikach wszelkich działań na dowolnej stronie.

Otwórz szablon *templates/base.html* aplikacji `account` i dodaj następujący kod wyróżniony pogrubioną czcionką.

```
{% load static %}
<!DOCTYPE html>
<html>
<head>
  <title>{% block title %}{% endblock %}</title>
  <link href="{% static "css/base.css" %}" rel="stylesheet">
</head>
<body>
  <div id="header">
     ...
  </div>
```

```
    {% if messages %}
      <ul class="messages">
        {% for message in messages %}
          <li class="{{ message.tags }}">
            {{ message|safe }}
            <a href="#" class="close">x</a>
          </li>
        {% endfor %}
      </ul>
    {% endif %}
    <div id="content">
      {% block content %}
      {% endblock %}
    </div>
  </body>
  </html>
```

Framework `messages` zawiera procesor kontekstu `django.contrib.messages.context_processor.messages`, dodający do kontekstu żądania zmienną `messages`. Można go znaleźć na liście `context_processors` ustawienia `TEMPLATES` w Twoim projekcie. Zmiennej `messages` można używać w szablonach do wyświetlania użytkownikowi aktualnych komunikatów.

Uwaga

Procesor kontekstu to funkcja Pythona, która pobiera jako argument obiekt `request` i zwraca słownik, który jest dodawany do kontekstu żądania. Sposób tworzenia własnych procesorów kontekstu omówię w rozdziale 8., „Utworzenie sklepu internetowego”.

Teraz zmodyfikujmy widok edycji profilu, aby wykorzystać w nim framework `messages`.

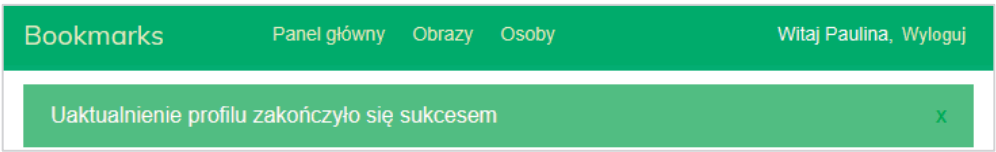
Przeprowadź edycję pliku *views.py* w aplikacji `account` i umieść w nim poniższe wiersze oznaczone pogrubioną czcionką.

```
from django.contrib import messages

# ...

@login_required
def edit(request):
     if request.method == 'POST':
         user_form = UserEditForm(
             instance=request.user,
             data=request.POST
         )
         profile_form = ProfileEditForm(
             instance=request.user.profile,
             data=request.POST,
             files=request.FILES
         )
         if user_form.is_valid() and profile_form.is_valid():
             user_form.save()
             profile_form.save()
             messages.success(
```

```
                request,
                'Uaktualnienie profilu zakończyło się sukcesem'
            )
        else:
            messages.error(request, 'Wystąpił błąd podczas uaktualniania profilu')
    else:
        user_form = UserEditForm(instance=request.user)
        profile_form = ProfileEditForm(
                        instance=request.user.profile)
    return render(
        request,
        'account/edit.html',
        {'user_form': user_form,'profile_form': profile_form}
)
```

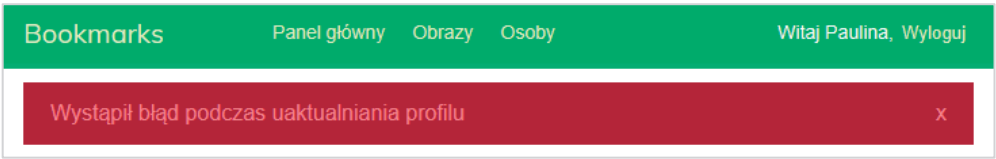
Dodaliśmy komunikat informujący o sukcesie wyświetlany, gdy operacja zmiany profilu zakończy się powodzeniem. Jeżeli którykolwiek formularz będzie wypełniony nieprawidłowo, nastąpi wyświetlenie komunikatu o błędzie.

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/account/edit/`, a następnie przeprowadź edycję profilu. Jeśli profil zostanie pomyślnie zaktualizowany, powinieneś zobaczyć komunikat informujący o sukcesie, co pokazałem na rysunku 5.2.



Rysunek 5.2. Komunikat o pomyślnej edycji profilu

Wprowadź błędną datę w polu *Date of birth* i ponownie wyślij formularz. Powinieneś zobaczyć komunikat pokazany na rysunku 5.3.



Rysunek 5.3. Komunikat o błędzie aktualizacji profilu

Generowanie komunikatów informujących użytkowników o wynikach ich działań jest bardzo proste. Komunikaty można dodawać również do innych widoków.

Więcej informacji na temat frameworka `messages` można znaleźć na stronie *https://docs.djangoproject.com/en/5.0/ref/contrib/messages/*.

Po utworzeniu wszystkich funkcji związanych z uwierzytelnianiem użytkownika i edycją profilu zajmiemy się personalizacją mechanizmów uwierzytelniania. Dowiesz się, jak zbudować niestandardowy backend uwierzytelniania, aby użytkownicy mogli się logować do serwisu za pomocą swojego adresu e-mail.

Implementacja własnego backendu uwierzytelniania

Django pozwala uwierzytelniać użytkowników na podstawie różnych źródeł, takich jak wbudowany system uwierzytelniania Django, zewnętrzne systemy uwierzytelniania, takie jak serwery **LDAP** (ang. *Lightweight Directory Access Protocol*), a nawet zewnętrzni dostawcy. Opcja `AUTHENTICATION_BACKENDS` to lista backendów uwierzytelniania, które mogą być stosowane w projekcie. Django umożliwia zdefiniowanie elastycznych schematów uwierzytelniania dzięki określeniu wielu backendów. Domyślna wartość ustawienia `AUTHENTICATION_BACKENDS` jest następująca:

```
['django.contrib.auth.backends.ModelBackend']
```

Domyślny model o nazwie `ModelBackend` uwierzytelnia użytkowników na podstawie bazy danych za pomocą modelu `User` dostarczanego przez django.`contrib.auth`. Takie rozwiązanie nadaje się do wykorzystania w większości projektów. Można jednak tworzyć niestandardowe backendy do uwierzytelniania użytkowników na podstawie innych źródeł.

Więcej informacji na temat dostosowania mechanizmu uwierzytelniania do własnych potrzeb można znaleźć na stronie *https://docs.djangoproject.com/en/5.0/topics/auth/customizing/#other-authentication-sources*.

Jeśli tylko użyjesz funkcji `authenticate()` zdefiniowanej w `django.contrib.auth`, Django spróbuje po kolei uwierzytelnić użytkownika względem każdego mechanizmu zdefiniowanego na liście `AUTHENTICATION_BACKENDS` aż do chwili, gdy którakolwiek próba zakończy się powodzeniem. Jeżeli nie uda się uwierzytelnić użytkownika za pomocą żadnego mechanizmu, wtedy pozostanie on niezalogowany w witrynie.

Django pozwala na bardzo łatwe zdefiniowanie własnego mechanizmu uwierzytelniania. Backend uwierzytelniania opiera się na klasie zapewniającej dwie wymienione poniżej metody.

- `authenticate()`: Metoda pobiera obiekt `request` i poświadczenia użytkownika jako parametry. Na tej podstawie zwraca obiekt `user` odpowiadający tym poświadczeniom, o ile są poprawne, lub `None` w przeciwnym razie. Parametr `request` to obiekt klasy `HttpRequest` lub `None` (jeśli go nie przekazano do metody `authenticate()`).
- `get_user()`: metoda pobiera parametr w postaci identyfikatora użytkownika i zwraca odpowiadający mu obiekt `User`.

Przygotowanie własnego mechanizmu uwierzytelnienia jest naprawdę łatwe i sprowadza się do utworzenia klasy Pythona implementującej obie wymienione metody. Zdefiniujemy teraz własny mechanizm uwierzytelniania, aby pozwolić użytkownikom na zalogowanie się do witryny za pomocą adresu e-mail zamiast nazwy użytkownika.

Utwórz nowy plik w katalogu aplikacji `account` i nadaj mu nazwę *authentication.py*. Następnie w tym pliku umieść poniższy fragment kodu.

```
from django.contrib.auth.models import User
class EmailAuthBackend:
```

```
    """
    Uwierzytelnia użytkownika na podstawie adresu e-mail.
    """
    def authenticate(self, request, username=None, password=None):
        try:
            user = User.objects.get(email=username)
            if user.check_password(password):
                return user
            return None
        except (User.DoesNotExist, User.MultipleObjectsReturned):
            return None

    def get_user(self, user_id):
        try:
            return User.objects.get(pk=user_id)
        except User.DoesNotExist:
            return None
```

To jest prosty mechanizm uwierzytelniania. Metoda `authenticate()` pobiera obiekt `request` i parametry opcjonalne `username` i `password`. Wprawdzie można by użyć innych parametrów, ale zdecydowałem się na wymienione, aby ułatwić współpracę tego mechanizmu z widokami frameworka uwierzytelniania. Oto dokładne omówienie sposobu działania powyższego kodu:

- `authenticate()`: próbujemy pobrać użytkownika na podstawie podanego adresu e-mail oraz sprawdzamy hasło za pomocą wbudowanej metody `check_password()` modelu `User`. Wymieniona metoda dla podanego przez użytkownika hasła generuje wartość skrótu, którą następnie porównuje z wartością przechowywaną w bazie danych. Przechwytywane są dwa różne wyjątki dotyczące kolekcji `QuerySet`: `DoesNotExist` i `MultipleObjectsReturned`. Wyjątek `DoesNotExist` jest zgłaszany w przypadku, gdy nie zostanie znaleziony żaden użytkownik o podanym adresie e-mail. Wyjątek `MultipleObjectsReturned` jest zgłaszany w przypadku, gdy zostanie znalezionych wielu użytkowników z tym samym adresem e-mail. Później, aby uniemożliwić użytkownikom korzystanie z istniejącego adresu e-mail, zmodyfikujemy widoki rejestracji i edycji.
- `get_user()`: metoda pobiera użytkownika na podstawie identyfikatora podanego w parametrze `user_id`. Do pobrania obiektu User na czas trwania sesji użytkownika Django wykorzystuje backend, który uwierzytelnił użytkownika. `pk` to skrót od *primary key* (klucz główny), który jest unikatowym identyfikatorem dla każdego rekordu w bazie danych. Pole, które służy jako klucz główny, ma każdy model Django. Domyślnie kluczem głównym jest automatycznie wygenerowane pole `id`. Więcej informacji na temat automatycznych pól klucza głównego można znaleźć na stronie *https://docs.djangoproject.com/en/5.0/topics/db/models/#automaticprimary-key-fields*.

Zmodyfikuj plik *settings.py* swojego projektu i dodaj do niego kod:

```
AUTHENTICATION_BACKENDS = [
    'django.contrib.auth.backends.ModelBackend',
    'account.authentication.EmailAuthBackend',
]
```

W powyższym ustawieniu zachowujemy domyślny mechanizm `ModelBackend`, który jest używany do uwierzytelniania za pomocą nazwy użytkownika i hasła, oraz dołączamy własny backend uwierzytelniania oparty na e-mailu `EmailAuthBackend`.

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/account/login/`. Ponieważ Django będzie próbował uwierzytelnić użytkownika za pomocą każdego mechanizmu, więc do witryny możesz się zalogować, podając nazwę użytkownika lub adres e-mail.

Poświadczenia użytkownika będą sprawdzane z wykorzystaniem mechanizmu uwierzytelniania `ModelBackend`, a jeśli nie zostanie zwrócony żaden użytkownik, poświadczenia będą sprawdzane przy użyciu mechanizmu uwierzytelniania `EmailAuthBackend`.

Uwaga

Kolejność umieszczenia mechanizmów uwierzytelniania na liście `AUTHENTICATION_BACKENDS` ma znaczenie. Jeżeli te same dane uwierzytelniające są poprawne dla wielu mechanizmów, Django zakończy próby na pierwszym mechanizmie, za pomocą którego uda się uwierzytelnić użytkownika. Oznacza to, że po znalezieniu dopasowania Django nie będzie kontynuować sprawdzania pozostałych backendów.

Uniemożliwianie użytkownikom korzystania z istniejącego adresu e-mail

Model `User` frameworka uwierzytelniania nie blokuje możliwości tworzenia użytkowników z tym samym adresem e-mail. Jeśli co najmniej dwa konta użytkowników mają ten sam adres e-mail, nie będziemy w stanie rozpoznać, który użytkownik się uwierzytelnia. Teraz, gdy użytkownicy mogą się logować przy użyciu swojego adresu e-mail, musimy zablokować im możliwość rejestracji przy użyciu istniejącego adresu e-mail.

Zmodyfikujemy teraz formularz rejestracji użytkownika, aby zapobiec rejestracji wielu użytkowników przy użyciu tego samego adresu e-mail.

Przeprowadź edycję pliku *forms.py* aplikacji `account` i umieść w nim poniższe wiersze oznaczone pogrubioną czcionką.

```
class UserRegistrationForm(forms.ModelForm):
    password = forms.CharField(
         label='Hasło',
         widget=forms.PasswordInput
     )
     password2 = forms.CharField(
         label='Powtórz hasło',
         widget=forms.PasswordInput
     )

    class Meta:
         model = User
         fields = ['username', 'first_name', 'email']
```

```
    def clean_password2(self):
         cd = self.cleaned_data
         if cd['password'] != cd['password2']:
            raise forms.ValidationError('Hasła nie są identyczne.')
         return cd['password2']

    def clean_email(self):
        data = self.cleaned_data['email']
        if User.objects.filter(email=data).exists():
            raise forms.ValidationError('Ten adres e-mail już został wykorzystany.')
        return data
```

Dodaliśmy walidację pola `email`, która uniemożliwia użytkownikom rejestrację przy użyciu istniejącego adresu e-mail. Aby znaleźć istniejących użytkowników z tym samym adresem e-mail, budujemy kolekcję `QuerySet`. Za pomocą metody `exists()` sprawdzamy, czy są jakieś wyniki. Metoda `exists()` zwraca `True`, jeśli kolekcja `QuerySet` zawiera jakieś wyniki, i `False` w przeciwnym razie.

Teraz dodaj do klasy `UserEditForm` następujące wiersze wyróżnione pogrubioną czcionką.

```
from django.contrib.auth.models import User

class UserEditForm(forms.ModelForm):
     class Meta:
        model = User
        fields = ['first_name', 'last_name', 'email']

    def clean_email(self):
        data = self.cleaned_data['email']
        qs = User.objects.exclude(
            id=self.instance.id
        ).filter(
            email=data
        )
        if qs.exists():
           raise forms.ValidationError('Ten adres e-mail już jest używany.')
        return data
```

W tym przypadku dodaliśmy walidację pola `email`, która uniemożliwia użytkownikom zmianę istniejącego adresu e-mail na istniejący adres e-mail innego użytkownika. Z kolekcji `QuerySet` wykluczamy bieżącego użytkownika. W przeciwnym razie aktualny adres e-mail użytkownika zostanie uznany za istniejący adres e-mail, a formularz nie zostanie zwalidowany.

Dodanie do witryny uwierzytelnienia za pomocą innej witryny społecznościowej

Uwierzytelnianie za pomocą serwisów społecznościowych to powszechnie stosowana funkcjonalność, umożliwiająca użytkownikom uwierzytelnianie z wykorzystaniem istniejącego konta dostawcy usług oraz mechanizmu **SSO** (ang. *single sign on* — dosłownie: pojedynczego logowania). Proces ten pozwala na uwierzytelnienie się w witrynie przy

użyciu istniejącego konta w usługach społecznościowych, takich jak Google, Facebook lub Twitter. W tym podrozdziale dodamy do witryny uwierzytelnianie społecznościowe za pomocą serwisu Google.

Do zaimplementowania uwierzytelniania społecznościowego skorzystamy z *OAuth 2.0* — standardowego w branży protokołu autoryzacji. *OAuth* to skrót od *Open Authorization*. OAuth 2.0 to standard zaprojektowany w celu umożliwienia witrynie internetowej lub aplikacji dostępu do zasobów hostowanych przez inne aplikacje internetowe w imieniu użytkownika. Google do uwierzytelniania i autoryzacji używa protokołu OAuth 2.0.

Dostępny jest moduł Pythona o nazwie `python-social-auth`, który ułatwia proces implementacji tego rodzaju uwierzytelnienia. Dzięki wykorzystaniu wymienionego modułu możesz zezwolić użytkownikowi na logowanie się do witryny internetowej za pomocą konta, które założył w innym serwisie. Kod tego modułu można znaleźć pod adresem *https://github.com/python-social-auth/social-app-django*.

Sam moduł jest dostarczany wraz z różnymi mechanizmami uwierzytelniania dla odmiennych frameworków Pythona, w tym także dla Django.

W wierszu polecenia powłoki uruchom polecenie:

```
python -m pip install social-auth-app-django==5.4.0
```

Spowoduje to zainstalowanie modułu `Python Social Auth`.

Po zainstalowaniu modułu umieść aplikację `social.apps.django_app.default` na liście `INSTALLED_APPS` w pliku *settings.py* projektu, tak jak pokazałem poniżej.

```
INSTALLED_APPS = [
    # ...
    'social_django',
]
```

W ten sposób dodajemy do projektu Django domyślną aplikację modułu `python-social-auth`. Teraz wydaj poniższe polecenie, aby przeprowadzić synchronizację modeli `python-social-auth` z bazą danych.

```
python manage.py migrate
```

Powinieneś zobaczyć zastosowanie migracji dla domyślnej aplikacji.

```
Applying social_django.0001_initial... OK
...
```

```
Applying social_django.0015_rename_extra_data_new_usersocialauth_extra_data...
OK
```

Moduł `Python Social Auth` oferuje obsługę mechanizmu uwierzytelniania wielu usług. Listę wszystkich dostępnych backendów można znaleźć pod adresem *https://python-social-auth.readthedocs.io/en/latest/backends/index. html#supported-backends*.

W tym rozdziale dodamy do projektu uwierzytelnianie społecznościowe, co umożliwi użytkownikom uwierzytelnianie za pomocą backendów dla serwisu Google.

Na początek konieczne jest dodanie do projektu wzorca adresu URL dla logowania za pomocą serwisu społecznościowego.

Otwórz główny plik *urls.py* w projekcie `bookmarks` i umieść w nim wzorce URL `social_django` tak, jak pokazałem poniżej. Nowe wiersze zostały wyróżnione pogrubioną czcionką.

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('account/', include('account.urls')),
    path(
       'social-auth/',
        include('social_django.urls', namespace='social')
    ),
]
```

Tworzona aplikacja webowa jest obecnie dostępna pod adresem IP lokalnego hosta, `127.0.0.1`, lub za pośrednictwem nazwy hosta, *localhost*. Serwis Google nie pozwala po pomyślnym uwierzytelnieniu na przekierowanie na adres *127.0.0.1* lub *localhost* — do przekierowania adresu URL oczekują one nazwy domeny. W tym projekcie będziemy symulować rzeczywiste środowisko przez serwowanie witryny pod nazwą domeny na lokalnej maszynie.

Znajdź na swoim komputerze plik *hosts*. Jeśli używasz systemu Linux lub macOS, plik *hosts* znajduje się w folderze */etc/hosts*. W systemie Windows plik *hosts* znajduje się w folderze *C:\Windows\System32\Drivers\etc\hosts*.

Przeprowadź edycję pliku *hosts* w swoim komputerze i dodaj do niego wiersz:

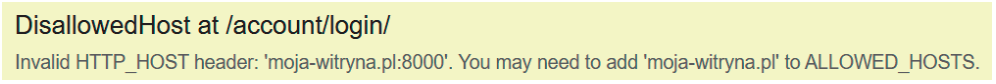
```
127.0.0.1 moja-witryna.pl
```

Powyższe polecenie oznacza, że po podaniu adresu *moja-witryna.pl* zostanie wskazany komputer lokalny.

Sprawdźmy, czy powiązanie nazwy hosta zadziałało. Uruchom serwer programistyczny za pomocą polecenia:

```
python manage.py runserver
```

W przeglądarce internetowej przejdź pod adres `http://moja-witryna.pl:8000/account/login/`. Wyświetli się komunikat o błędzie pokazany na rysunku 5.4.



Rysunek 5.4. Komunikat o nieprawidłowym nagłówku hosta

Django zarządza hostami, które mogą obsługiwać Twoją aplikację, używając ustawienia `ALLOWED_HOSTS`. Jest to środek bezpieczeństwa zapobiegający atakom z wykorzystaniem nagłówków hosta HTTP. Django zezwoli na serwowanie aplikacji tylko hostom z wymienionej listy.

Więcej informacji na temat ustawienia `ALLOWED_HOSTS` można znaleźć na stronie *https://docs.djangoproject.com/en/5.0/ref/settings/#allowed-hosts*.

Zmodyfikuj plik *settings.py* swojego projektu w taki sposób, aby ustawienie `ALLOWED_HOSTS` miało następującą postać. Nowy kod został wyróżniony pogrubioną czcionką.

```
ALLOWED_HOSTS = ['moja-witryna.pl', 'localhost', '127.0.0.1']
```

Oprócz hosta *moja-witryna.pl* jawnie dołączyliśmy hosty *localhost* i *127.0.0.1*. Zrobiliśmy to, aby można było uzyskać dostęp do witryny za pośrednictwem hosta *localhost*, co jest domyślnym zachowaniem Django, gdy opcja `DEBUG` ma wartość `True`, a lista `ALLOWED_HOSTS` jest pusta.

Otwórz ponownie w swojej przeglądarce stronę `http://moja-witryna.pl:8000/account/login/`. Teraz zamiast komunikatu o błędzie powinieneś zobaczyć stronę logowania do witryny.

Uruchomienie serwera programistycznego za pośrednictwem HTTPS

W tym rozdziale uruchomimy serwer programisty przez HTTPS, aby zasymulować prawdziwe środowisko, w którym zawartość wymieniana z przeglądarką jest zabezpieczona. Pomoże nam to później, w rozdziale 6., „Udostępnianie treści w witrynie internetowej", bezpiecznie obsługiwać witrynę i ładować narzędzie do tworzenia graficznych zakładek na dowolnej bezpiecznej stronie internetowej. Protokół **TLS** (ang. *Transport Layer Security*) jest standardem dla serwowania stron internetowych za pośrednictwem bezpiecznego połączenia. Poprzednikiem TLS jest **SSL** (ang. *Secure Sockets Layer*).

Chociaż protokół SSL jest obecnie uważany za przestarzały, w wielu bibliotekach i dokumentacji online można znaleźć odwołania zarówno do TLS, jak i SSL. Serwer programistyczny Django nie potrafi serwować witryn przy użyciu HTTPS, ponieważ to nie jest docelowe miejsce, w którym ma działać witryna. Aby przetestować funkcjonalność uwierzytelniania przez serwis społecznościowy, który serwuje witrynę za pomocą HTTPS, skorzystamy z rozszerzenia `RunServerPlus` pakietu `Django Extensions`. Ten pakiet zawiera zbiór przydatnych narzędzi Django. Należy pamiętać, że tej metody nie należy stosować do serwowania witryn w środowisku produkcyjnym. Służy ona wyłącznie do stosowania na serwerach programistycznych.

Aby zainstalować `Django Extensions`, skorzystaj z polecenia:

```
python -m pip install django-extensions==3.2.3
```

Teraz należy zainstalować `Werkzeug` — pakiet zawierający warstwę debuggera wymaganą przez rozszerzenie `RunServerPlus`. Aby go zainstalować, użyj polecenia:

```
python -m pip install werkzeug==3.0.2
```

Na koniec użyj poniższego polecenia, aby zainstalować `pyOpenSSL` — pakiet, który jest wymagany do korzystania z funkcjonalności SSL (TLS) rozszerzenia `RunServerPlus`.

```
python -m pip install pyOpenSSL==24.1.0
```

Przeprowadź edycję pliku *settings.py* w swoim projekcie i dodaj do ustawienia `INSTALLED_APPS` rozszerzenie `Django Extensions` tak, jak pokazałem poniżej.

```
INSTALLED_APPS = [
    # ...
    'django_extensions',
]
```

Aby uruchomić serwer programistyczny, skorzystaj z polecenia `runserver_plus` rozszerzeń `Django Extensions`.

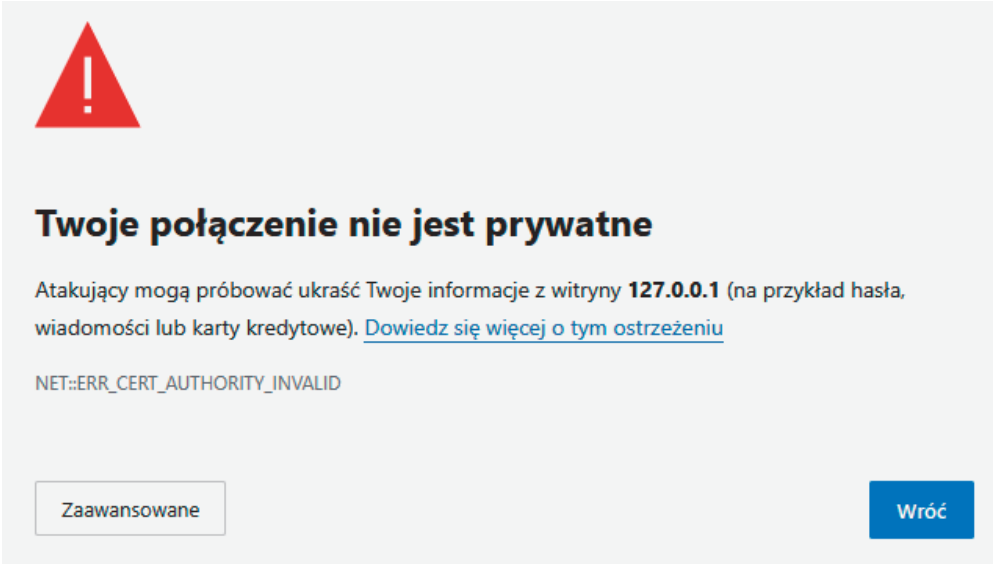
```
python manage.py runserver_plus --cert-file cert.crt
```

W poleceniu `runserver_plus` należy podać nazwę pliku zawierającego certyfikat SSL (TLS). Rozszerzenie `Django Extensions` automatycznie wygeneruje klucz wraz z certyfikatem.

W przeglądarce internetowej przejdź pod adres `https://127.0.0.1:8000/account/login/`. Teraz korzystasz z witryny za pośrednictwem HTTPS. Zauważ, że teraz adres zaczyna się od *https://* zamiast *http://*.

Ponieważ używasz certyfikatu z podpisem własnym zamiast zaufanego certyfikatu urzędu certyfikacji (CA), Twoja przeglądarka wyświetli ostrzeżenie o zabezpieczeniach.

Jeśli używasz przeglądarki Google Chrome, zobaczysz stronę podobną do pokazanej na rysunku 5.5.



Rysunek 5.5. Ostrzeżenie o zabezpieczeniach w przeglądarce Google Chrome

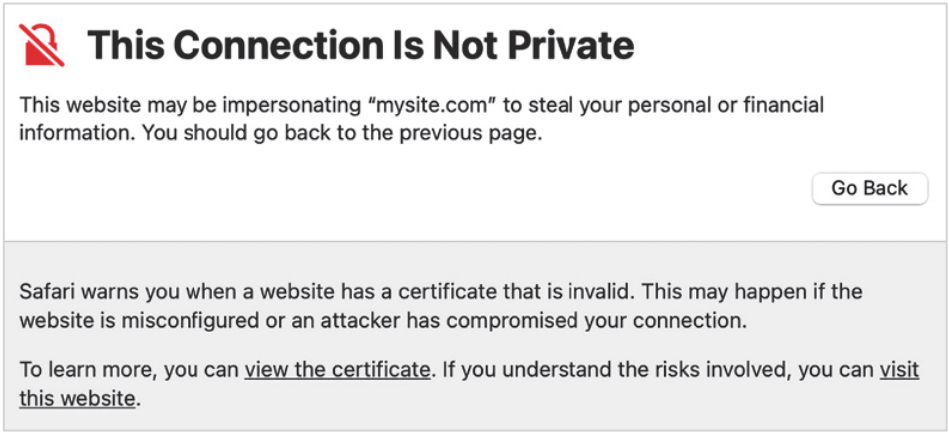
W takim przypadku kliknij łącze *Zaawansowane*, a następnie kliknij *Przejdź do 127.0.0.1 (niebezpieczne)*.

Jeśli używasz przeglądarki Safari, zobaczysz stronę podobną do pokazanej na rysunku 5.6.

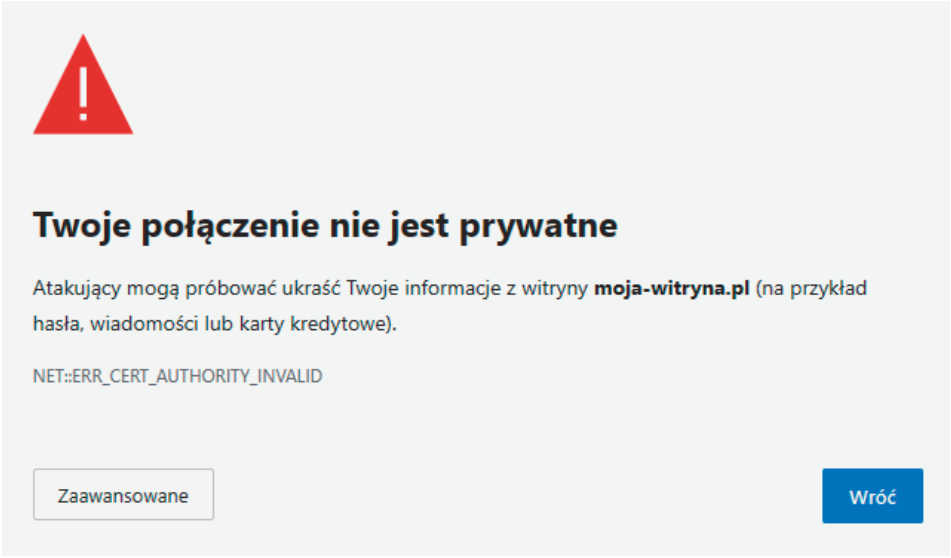
W takim przypadku kliknij *Pokaż szczegóły*, a następnie kliknij *Odwiedź tę witrynę*.

Jeśli używasz przeglądarki Microsoft Edge, zobaczysz stronę podobną do pokazanej na rysunku 5.7.

W takim przypadku kliknij *Zaawansowane*, a następnie *Przejdź do moja-witryna.pl (niebezpieczne)*.



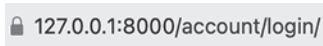
Rysunek 5.6. Błąd zabezpieczeń w przeglądarce Safari



Rysunek 5.7. Ostrzeżenie o zabezpieczeniach w przeglądarce Microsoft Edge

Jeśli używasz dowolnej innej przeglądarki, wyświetl zaawansowane informacje i zaakceptuj certyfikat z podpisem własnym, aby przeglądarka ufała certyfikatowi.

Zauważysz, że adres URL zaczyna się od *https://*, a ikona zabezpieczeń ma postać kłódki (rysunek 5.8), co wskazuje, że połączenie jest bezpieczne. Ponieważ używasz certyfikatu z podpisem własnym zamiast zaufanego, niektóre przeglądarki mogą wyświetlać ikonę zepsutej kłódki. Na potrzeby naszych testów nie będzie to stanowiło problemu.



Rysunek 5.8. Adres URL z ikoną bezpiecznego połączenia

Uwaga

Pakiet Django Extensions zapewnia wiele innych interesujących narzędzi i funkcji. Więcej informacji o tym pakiecie można znaleźć pod adresem *https://django-extensions.readthedocs.io/en/latest/*.

Możesz teraz podczas prac programistycznych serwować swoją witrynę przez HTTPS.

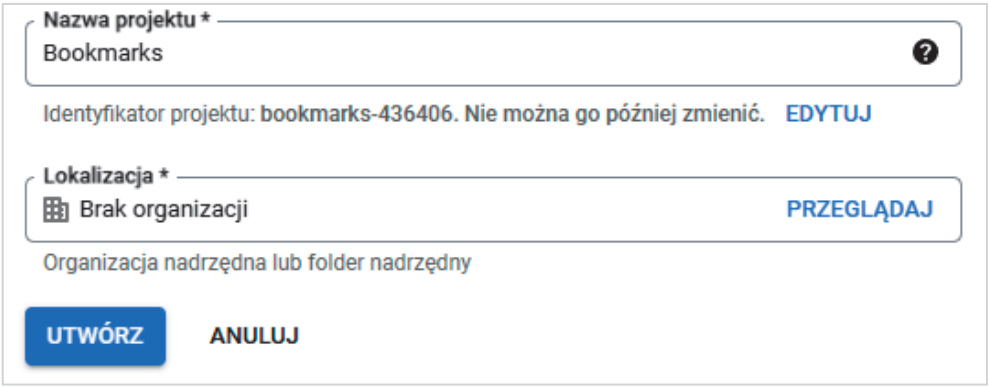
Uwierzytelnianie za pomocą serwisu Google

Serwis Google oferuje uwierzytelnianie społecznościowe za pomocą OAuth 2.0, co umożliwia użytkownikom logowanie się za pomocą kont Google. Więcej informacji na temat tej implementacji znajdziesz na stronie *https://developers.google.com/identity/protocols/OAuth2*.

Aby zaimplementować uwierzytelnianie za pomocą Google, dodaj do ustawienia `AUTHEN↪TICATION_BACKENDS` w pliku *settings.py* Twojego projektu następujący wiersz wyróżniony pogrubioną czcionką.

```
AUTHENTICATION_BACKENDS = [
    'django.contrib.auth.backends.ModelBackend',
    'account.authentication.EmailAuthBackend',
    'social_core.backends.google.GoogleOAuth2',
]
```

Pracę trzeba zacząć od utworzenia klucza API w Konsoli interfejsów API Google. W przeglądarce internetowej przejdź pod adres *https://console.cloud.google.com/projectcreate*. Wyświetli się ekran pokazany na rysunku 5.9.



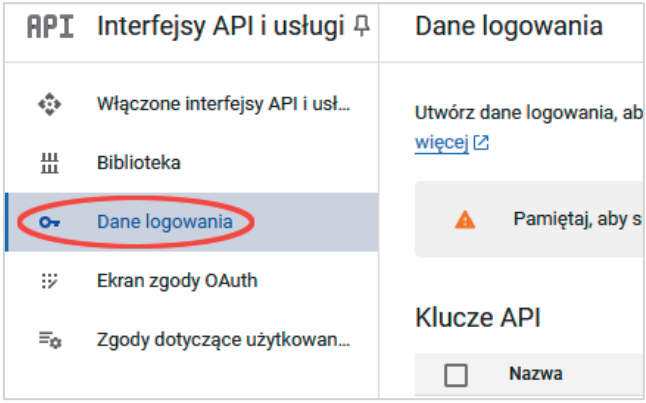
Rysunek 5.9. Formularz tworzenia projektu Google

W obszarze *Nazwa projektu* wpisz *Bookmarks* i kliknij przycisk *UTWÓRZ*.

Gdy nowy projekt będzie gotowy, upewnij się, że projekt jest wybrany na górnym pasku nawigacyjnym, tak jak pokazałem na rysunku 5.10.

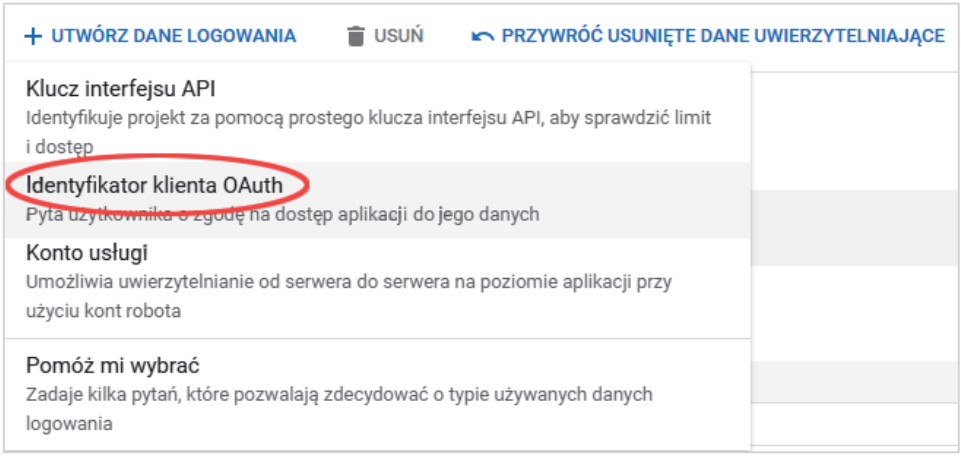
Rysunek 5.10. Górny pasek nawigacyjny Konsoli Programisty Google

Po utworzeniu projektu w obszarze *Interfejsy API i usługi* kliknij *Dane logowania* (rysunek 5.11).



Rysunek 5.11. Menu API i usługi w Google

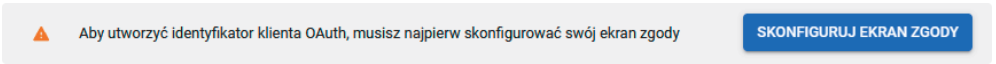
Wyświetli się ekran pokazany na rysunku 5.12.



Rysunek 5.12. Tworzenie poświadczeń API w Google API

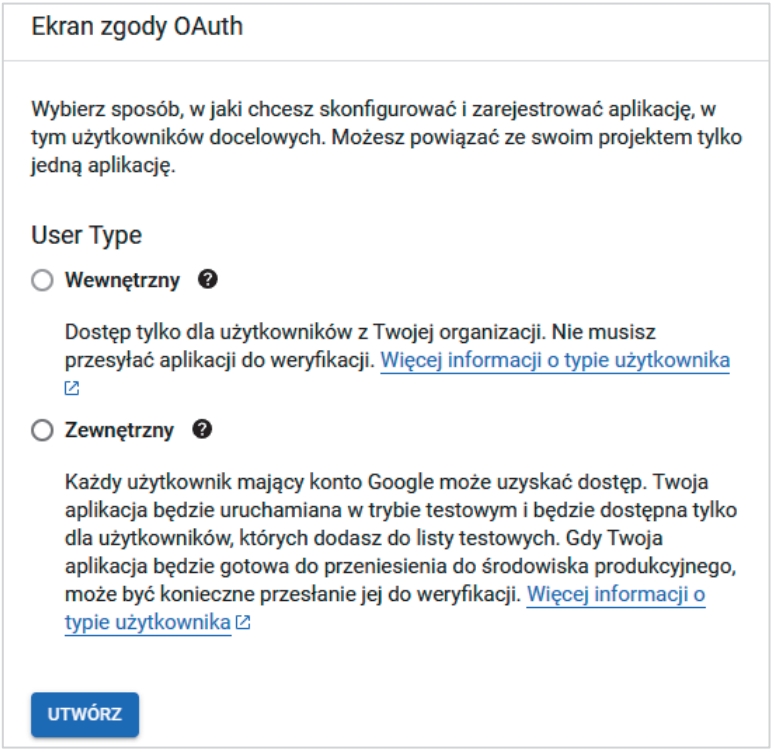
Następnie kliknij *UTWÓRZ DANE LOGOWANIA*, po czym kliknij *Identyfikator klienta OAuth*.

Google poprosi Cię najpierw o skonfigurowanie ekranu zgody (rysunek 5.13).



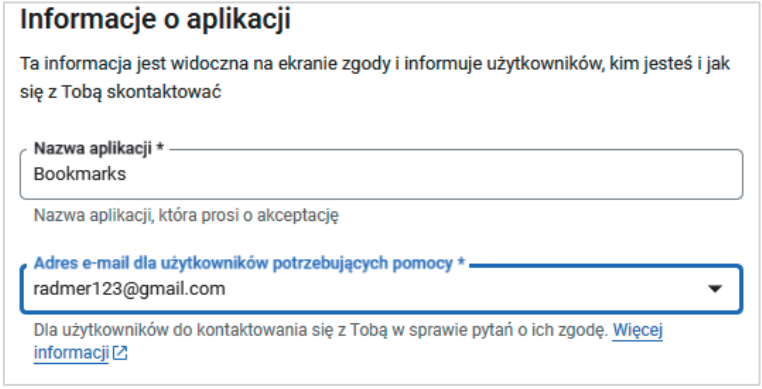
Rysunek 5.13. Alert dotyczący konfiguracji ekranu zgody OAuth

Skonfigurujemy stronę, która będzie wyświetlana użytkownikom po to, by wyrazili zgodę na dostęp do Twojej witryny za pomocą konta Google. Kliknij przycisk *SKONFIGURUJ EKRAN ZGODY*. Zostaniesz przekierowany na stronę pokazaną na rysunku 5.14.



Rysunek 5.14. Wybór typu użytkownika na ekranie konfigurowania ekranu akceptacji OAuth

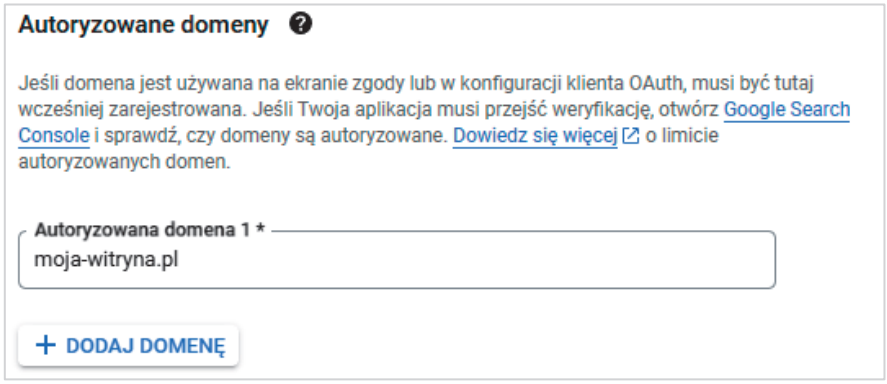
W obszarze *User Type* wybierz *Zewnętrzny*, po czym kliknij przycisk *UTWÓRZ*. Wyświetli się ekran pokazany na rysunku 5.15.



Rysunek 5.15. Konfiguracja ekranu zgody Google OAuth

W obszarze *Nazwa aplikacji* wpisz *Bookmarks* i w polu *Adres e-mail dla użytkowników potrzebujących pomocy* podaj swój e-mail.

W obszarze *Autoryzowane domeny* wprowadź *moja-witryna.pl* w sposób pokazany na rysunku 5.16.

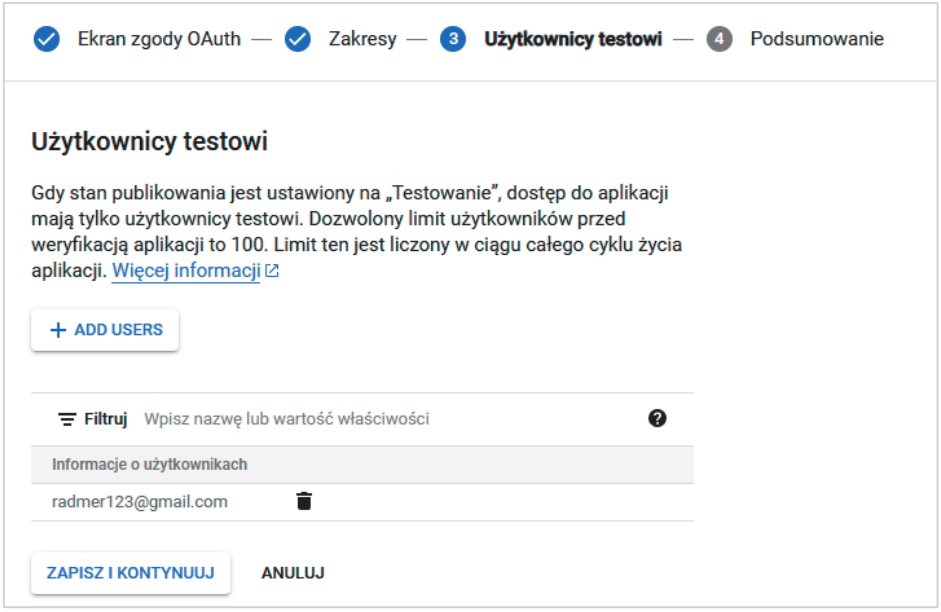


Rysunek 5.16. Domeny autoryzowane przez Google OAuth

W polu *Dane kontaktowe dewelopera* wpisz swój adres e-mail, a następnie kliknij *ZAPISZ I KONTYNUUJ*.

W kroku 2., *Zakresy*, nie zmieniaj niczego, a następnie kliknij *ZAPISZ I KONTYNUUJ*.

W kroku 3., *Użytkownicy testowi*, dodaj do listy użytkowników testowych swojego użytkownika Google, po czym kliknij *ZAPISZ I KONTYNUUJ* (rysunek 5.17).



Rysunek 5.17. Użytkownicy testowi Google OAuth

Wyświetli się podsumowanie konfiguracji ekranu akceptacji. Kliknij *POWRÓT DO PANELU*.

W menu na lewym pasku bocznym kliknij *Dane logowania* i ponownie kliknij *Utwórz dane logowania*, a następnie *Identyfikator klienta OAuth*.

W następnym kroku wprowadź następujące informacje.

- *Typ aplikacji:* wybierz *Aplikacja internetowa*.
- *Nazwa:* wpisz *Bookmarks*.
- *Autoryzowane źródła JavaScriptu*: dodaj *https://moja-witryna.pl:8000*.
- *Autoryzowane identyfikatory URI przekierowania*: dodaj `https://moja-witryna.pl:8000/social-auth/complete/googleoauth2/`.

Formularz powinien wyglądać tak, jak pokazałem na rysunku 5.18.

Kliknij przycisk *UTWÓRZ*. Otrzymasz identyfikator klienta i tajny klucz klienta (rysunek 5.19).

Następnie w katalogu głównym projektu utwórz nowy plik i nadaj mu nazwę *.env*. Plik *.env* zawiera pary klucz-wartość zmiennych środowiskowych. Dodaj do nowego pliku poświadczenia OAuth2 w taki sposób:

```
GOOGLE_OAUTH2_KEY=xxxx
GOOGLE_OAUTH2_SECRET=xxxx
```

Zastąp `xxxx` odpowiednio identyfikatorem i tajnym kluczem OAuth2.

Aby ułatwić oddzielenie konfiguracji od kodu, skorzystamy z biblioteki `python-decouple`. Z tej biblioteki korzystałeś już w rozdziale 2., „Usprawnianie bloga i dodanie funkcjonalności społecznościowych".

Zacznij od zainstalowania pakietu `python-decouple` za pomocą menedżera `pip`. W tym celu uruchom polecenie:

```
python -m pip install python-decouple==3.8
```

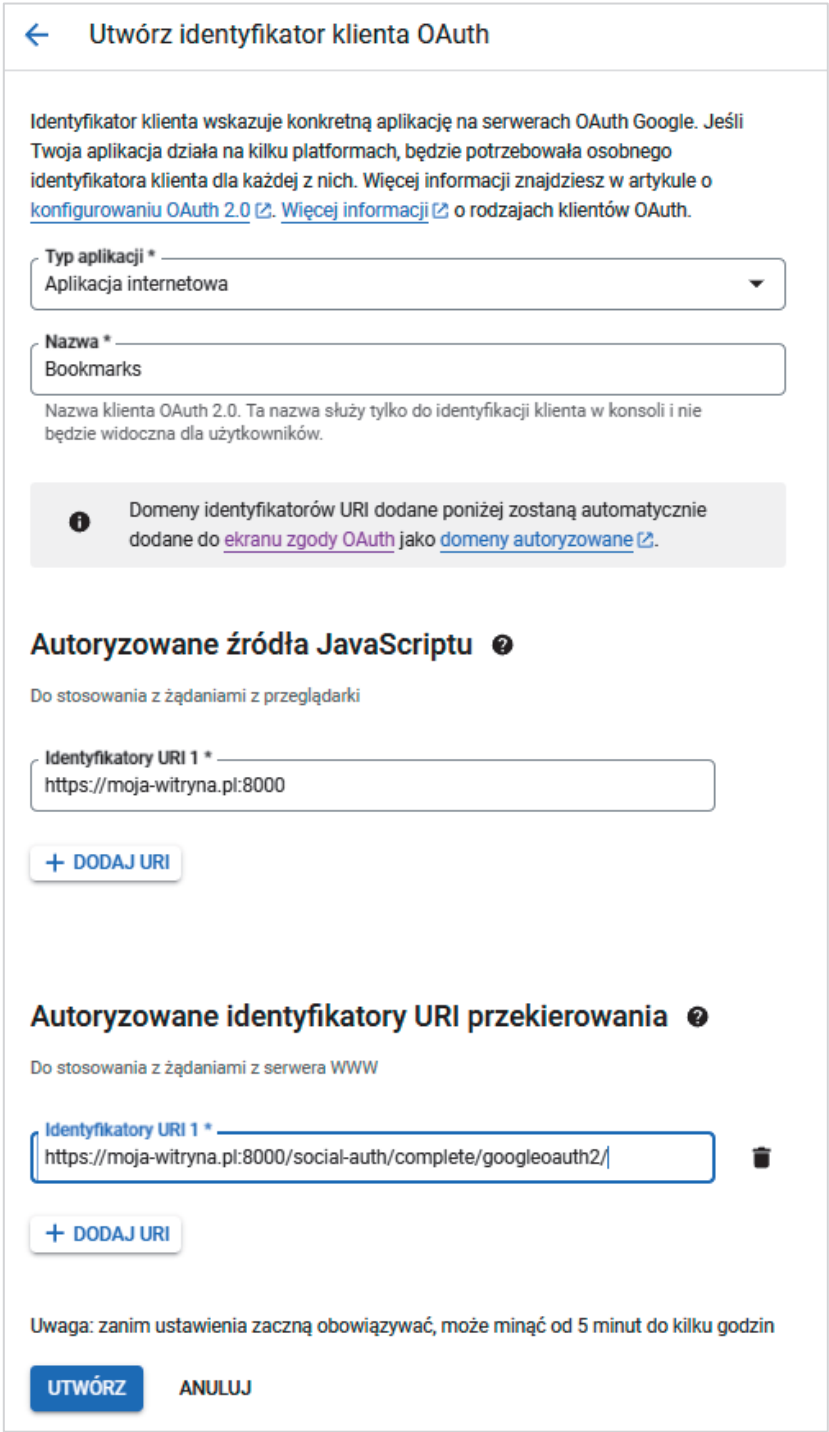
Przeprowadź edycję pliku *settings.py* swojego projektu i dodaj poniższy fragment kodu.

```
from decouple import config
# ...
SOCIAL_AUTH_GOOGLE_OAUTH2_KEY = config('GOOGLE_OAUTH2_KEY')
SOCIAL_AUTH_GOOGLE_OAUTH2_SECRET = config('GOOGLE_OAUTH2_SECRET')
```

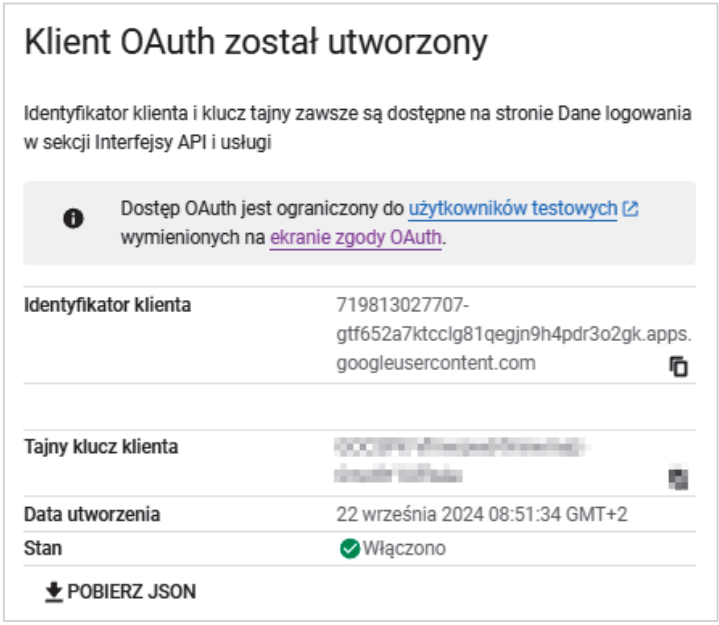
Ustawienia `SOCIAL_AUTH_GOOGLE_OAUTH2_KEY` i `SOCIAL_AUTH_GOOGLE_OAUTH2_SECRET` są ładowane ze zmiennych środowiskowych zdefiniowanych w pliku *.env*.

Teraz otwórz szablon *registration/login.html* aplikacji `account` i w bloku `content` dodaj poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
{% block content %}
  ...
  <div class="social">
    <ul>
      <li class="google">
        <a href="{% url "social:begin" "google-oauth2" %}">
          Zaloguj się za pomocą konta Google
        </a>
      </li>
    </ul>
  </div>
{% endblock %}
```



Rysunek 5.18. Formularz tworzenia identyfikatora klienta Google OAuth

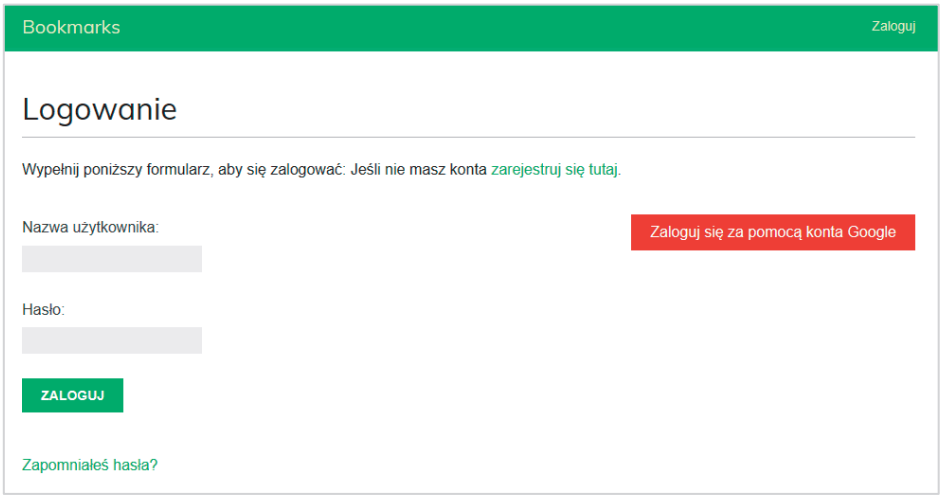


Rysek 5.19. Identyfikator klienta i tajny klucz klienta dla Google OAuth

Aby uruchomić serwer programistyczny, skorzystaj z polecenia `runserver_plus` rozszerzeń `Django Extensions`.

```
python manage.py runserver_plus --cert-file cert.crt
```

W przeglądarce internetowej przejdź pod adres `https://moja-witryna.pl:8000/account/login/`. Strona logowania powinna być podobna do pokazanej na rysunku 5.20.



Rysunek 5.20. Strona logowania zawierająca przyciski do uwierzytelniania za pośrednictwem serwisu Google

Kliknij przycisk *Zaloguj się za pomocą konta Google*. Wyświetli się ekran pokazany na rysunku 5.21.

Rysunek 5.21. Ekran autoryzacji aplikacji Google

Aby autoryzować aplikację, kliknij swoje konto Google. Jeżeli wszystko przebiegnie bez problemów, zostaniesz zalogowany i przeniesiony na stronę panelu głównego w budowanej witrynie internetowej. Pamiętaj, że ten adres URL został podany w opcji `LOGIN_REDIRECT_URL`. Jak możesz zobaczyć, implementacja w aplikacji uwierzytelnienia za pomocą innego serwisu społecznościowego jest naprawdę prosta.

Właśnie dodałeś mechanizm uwierzytelniania społecznościowego za pośrednictwem serwisu Google. Za pomocą modułu Pythona `social-auth` z łatwością możesz zaimplementować mechanizmy uwierzytelniania z wykorzystaniem innych usług online. W kolejnym podrozdziale zajmiemy się tworzeniem profili użytkowników rejestrujących się za pomocą uwierzytelniania społecznościowego.

Tworzenie profili dla użytkowników rejestrujących się za pomocą uwierzytelniania społecznościowego

Gdy użytkownik uwierzytelnia się za pomocą witryny społecznościowej, to jeśli nie ma istniejącego użytkownika powiązanego z tym profilem społecznościowym, w witrynie tworzony jest nowy obiekt `User`. Moduł `Python Social Auth` wykorzystuje potok składający się ze zbioru funkcji wykonywanych podczas przepływu uwierzytelniania w określonej kolejności. Funkcje te są odpowiedzialne za pobieranie dowolnych danych użytkownika, utworzenie profilu społecznościowego w bazie danych i powiązanie go z istniejącym użytkownikiem lub utworzenie nowego użytkownika.

W obecnym stanie witryny, którą budujemy, gdy są tworzeni nowi użytkownicy za pomocą uwierzytelniania społecznościowego, nie są dla nich tworzone obiekty `Profile`. Teraz dodamy do potoku nowy krok, dzięki któremu po utworzeniu nowego użytkownika zostanie automatycznie utworzony w bazie danych obiekt `Profile`.

Dodaj do pliku *settings.py* w tworzonym projekcie następujące ustawienie SOCIAL_AUTH_
↪PIPELINE.

```
SOCIAL_AUTH_PIPELINE = [
    'social_core.pipeline.social_auth.social_details',
    'social_core.pipeline.social_auth.social_uid',
    'social_core.pipeline.social_auth.auth_allowed',
    'social_core.pipeline.social_auth.social_user',
    'social_core.pipeline.user.get_username',
    'social_core.pipeline.user.create_user',
    'social_core.pipeline.social_auth.associate_user',
    'social_core.pipeline.social_auth.load_extra_data',
    'social_core.pipeline.user.user_details',
]
```

To jest domyślny potok uwierzytelniania używany przez Python Social Auth. Składa się z kilku funkcji, które podczas uwierzytelniania użytkownika wykonują różne zadania. Więcej informacji na temat domyślnego potoku uwierzytelniania można znaleźć pod adresem *https://python-social-auth.readthedocs.io/en/latest/pipeline.html*.

Zbudujmy funkcję, która za każdym razem, gdy zostanie utworzony nowy użytkownik, utworzy w bazie danych obiekt Profile. Następnie dodamy tę funkcję do potoku uwierzytelniania społecznościowego.

Przeprowadź edycję pliku *account/authentication.py* i dodaj do niego kod:

```
from account.models import Profile

def create_profile(backend, user, *args, **kwargs):
    """
    Utwórz profil użytkownika do uwierzytelniania społecznościowego
    """
    Profile.objects.get_or_create(user=user)
```

Funkcja create_profile przyjmuje dwa obowiązkowe argumenty.

- backend: Backend uwierzytelniania społecznościowego wykorzystywany do uwierzytelniania użytkownika. Pamiętaj, że backendy uwierzytelniania społecznościowego dodałeś do ustawienia AUTHENTICATION_BACKENDS w swoim projekcie.
- user: egzemplarz klasy User reprezentujący uwierzytelnionego użytkownika — nowego lub istniejącego.

Różne argumenty przekazywane do funkcji potoku możesz sprawdzić pod adresem *https://python-social-auth.readthedocs.io/en/latest/pipeline.html#extending-the-pipeline*.

W funkcji create_profile sprawdzamy, czy istnieje obiekt użytkownika. Następnie korzystamy z metody get_or_create() do wyszukania obiektu Profile dla danego użytkownika, a gdy taki obiekt nie istnieje — tworzymy go.

Teraz trzeba dodać nową funkcję do potoku uwierzytelniania. Dodaj do ustawienia SOCIAL_
↪AUTH_PIPELINE w pliku *settings.py* następujący wiersz wyróżniony pogrubioną czcionką.

```
SOCIAL_AUTH_PIPELINE = [
    'social_core.pipeline.social_auth.social_details',
    'social_core.pipeline.social_auth.social_uid',
    'social_core.pipeline.social_auth.auth_allowed',
    'social_core.pipeline.social_auth.social_user',
    'social_core.pipeline.user.get_username',
    'social_core.pipeline.user.create_user',
    'account.authentication.create_profile',
    'social_core.pipeline.social_auth.associate_user',
    'social_core.pipeline.social_auth.load_extra_data',
    'social_core.pipeline.user.user_details',
]
```

Dodaliśmy funkcję `create_profile` do funkcji `social_core.pipeline.create_user`. W tym momencie dostępny jest egzemplarz modelu `User`. Na tym etapie potoku może on reprezentować istniejącego użytkownika lub nowo utworzonego. Funkcja `create_profile` używa egzemplarza modelu `User` do wyszukania powiązanego z nim obiektu `Profile`, a jeśli to konieczne — utworzenia nowego.

Za pomocą strony dostępnej w witrynie administracyjnej pod adresem `https://moja-witryna.pl:8000/admin/auth/user/` odczytaj listę użytkowników. Usuń wszystkich użytkowników utworzonych za pomocą uwierzytelniania społecznościowego.

Następnie otwórz stronę `https://moja-witryna.pl:8000/account/login/` i przeprowadź uwierzytelnianie społecznościowe dla użytkownika, którego usunąłeś. Zostanie utworzony nowy użytkownik, a wraz z nim — również obiekt `Profile`. Aby sprawdzić, czy dla nowego użytkownika został utworzony profil, otwórz w przeglądarce stronę `https://moja-witryna.pl:8000/admin/account/profile/`.

Właśnie pomyślnie dodałeś do mechanizmu uwierzytelniania społecznościowego funkcję automatycznego tworzenia profilu użytkownika.

`Python Social Auth` oferuje również mechanizm potoku dla przepływu rozłączania użytkownika. Więcej informacji na ten temat znajdziesz na stronie *https://python-social-auth.readthedocs.io/en/latest/pipeline.html#disconnectionpipeline*.

Podsumowanie

W tym rozdziale znacznie poprawiłeś możliwości uwierzytelniania w witrynach społecznościowych. Utworzyłeś backend uwierzytelniania oparty na adresie e-mail i dodałeś uwierzytelnianie społecznościowe za pomocą konta Google. Poprawiłeś także wygodę pracy użytkownika przez zaimplementowanie dostarczania informacji zwrotnych na temat jego działań za pomocą frameworka `messages` platformy Django. Na koniec nauczyłeś się personalizacji potoku uwierzytelniania w celu automatycznego tworzenia profili użytkowników.

W następnym rozdziale stworzysz system katalogowania zdjęć. Stworzysz modele z relacjami wiele-do-wielu oraz dostosujesz zachowanie formularzy. Dowiesz się także, jak generować miniaturki obrazków oraz jak budować funkcjonalności AJAX z wykorzystaniem JavaScript i Django.

Dodatkowe zasoby

Oto lista zasobów zawierających dodatkowe informacje związane z tematami omawianymi w tym rozdziale:

- Kod źródłowy przykładów zamieszczonych w tym rozdziale — *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter05*.
- Framework `messages` w Django — *https://docs.djangoproject.com/en/5.0/ref/contrib/messages/*.
- Niestandardowe źródła uwierzytelniania — *https://docs.djangoproject.com/en/5.0/topics/auth/customizing/#other-authentication-sources*.
- Automatyczne pola klucza głównego — *https://docs.djangoproject.com/en/5.0/topics/db/models/#automatic-primary-key-fields*.
- `Python Social Auth` — *https://github.com/python-social-auth*.
- Backendy uwierzytelniania modułu Pythona Social Auth — *https://python-social-auth.readthedocs.io/en/latest/backends/index.html#supported-backends*.
- Ustawienia pcji `ALLOWED_HOSTS` w Django — *https://docs.djangoproject.com/en/5.0/ref/settings/#allowed-hosts*.
- Dokumentacja rozszerzeń `Django Extensions` — *https://django-extensions.readthedocs.io/en/latest/*.
- Implementacja Google OAuth2 — *https://developers.google.com/identity/protocols/OAuth2*.
- Poświadczenia Google API — *https://console.developers.google.com/apis/credentials*.
- Potok uwierzytelniania społecznościowego Pythona — *https://python-social-auth.readthedocs.io/en/latest/pipeline.html*.
- Rozszerzanie potoku `Python Social Auth` — *https://python-social-auth.readthedocs.io/en/latest/pipeline.html#extending-the-pipeline*.
- Potok `Python Social Auth` do obsługi rozłączania — *https://python-social-auth.readthedocs.io/en/latest/pipeline.html#disconnection-pipeline*.

Rozdział 6

Udostępnianie treści w witrynie internetowej

W poprzednim rozdziale dodałeś do swojej strony komunikaty o sukcesie i błędach z wykorzystaniem frameworka `messages` Django. Stworzyłeś też backend uwierzytelniania za pomocą adresu e-mail i dodałeś uwierzytelnianie społecznościowe do witryny za pomocą konta Google. Nauczyłeś się także, jak uruchomić na komputerze lokalnym serwer programistyczny przez HTTPS przy użyciu `Django Extensions`. Zmodyfikowałeś również potok uwierzytelniania społecznościowego, aby automatycznie tworzyć profile dla nowych użytkowników.

W tym rozdziale zobaczysz, jak utworzyć bookmarklet JavaScript pozwalający na udostępnianie w budowanej witrynie internetowej treści pochodzącej z innych witryn. Ponadto z zastosowaniem mechanizmów JavaScript i Django zaimplementujesz w projekcie obsługę w przeglądarce asynchronicznych żądań.

Oto zagadnienia, na których skoncentruję się w tym rozdziale:

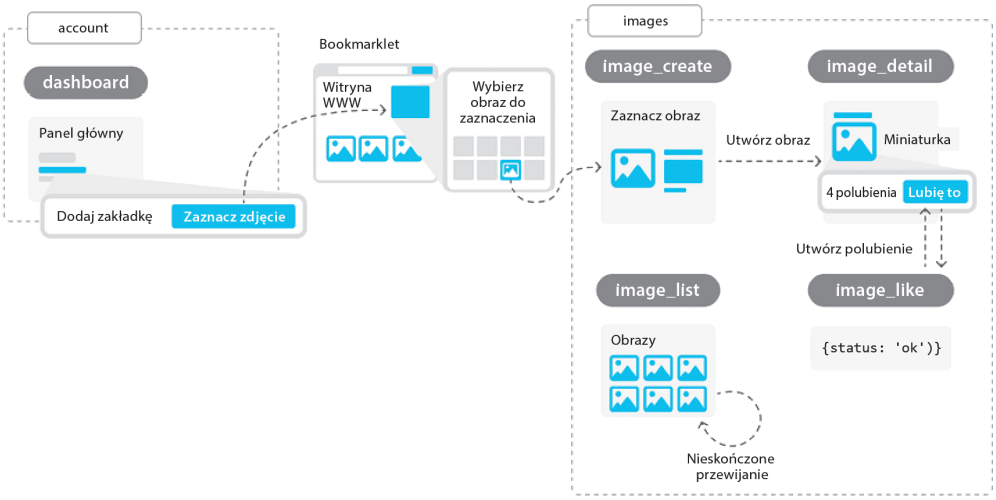
- Zdefiniowanie relacji typu „wiele do wielu”
- Dostosowanie do własnych potrzeb sposobu działania formularzy
- Wykorzystanie JavaScript z Django
- Budowanie bookmarkletu JavaScript
- Generowanie miniatur obrazów z wykorzystaniem modułu `easy-thumbnail`
- Implementacja asynchronicznych żądań HTTP za pomocą JavaScript i Django
- Budowa mechanizmu nieskończonego stronicowania podczas przewijania

W tym rozdziale stworzysz system katalogowania zdjęć. Utworzysz modele z relacjami „wiele do wielu” oraz dostosujesz zachowanie formularzy. Dowiesz się także, jak generować miniaturki obrazków oraz jak budować asynchroniczne funkcjonalności przeglądarki z wykorzystaniem JavaScript i Django.

Przegląd funkcjonalności

Reprezentację widoków, szablonów i funkcjonalności, które zbudujemy w tym rozdziale, przedstawiono na rysunku 6.1.

W tym rozdziale zaimplementujemy przycisk *Zaznacz zdjęcie*, który pozwoli użytkownikom dodawać zakładki do zdjęć z dowolnych stron internetowych. W celu wyświetlenia selektora zdjęć na górze dowolnej strony internetowej, tak aby użytkownicy mogli wybrać zdjęcie do utworzenia zakładki, skorzystamy z JavaScript. Zaimplementujemy także



Rysunek 6.1. Schemat funkcjonalności budowanych w rozdziale 6.

widok `image_create` i formularz pozwalający pobrać zdjęcie z jego oryginalnego źródła i zapisać na swojej stronie internetowej. Zbudujemy również widok `image_detail` do wyświetlania pojedynczych obrazów i automatycznie wygenerujemy miniatury obrazów za pomocą pakietu `easy-thumbnails`. Zaimplementujemy ponadto widok `image_like`, aby umożliwić użytkownikom polubienie (anulowanie polubienia) obrazów. Ten widok będzie obsługiwał asynchroniczne żądania HTTP wykonywane za pomocą JavaScript i zwracał odpowiedź w formacie JSON. Na koniec utworzymy widok `image_list` do wyświetlania wszystkich obrazów z zakładek oraz — za pomocą JavaScript i aplikacji Django `pagination` — zaimplementujesz mechanizm nieskończonego przewijania.

Kod źródłowy przykładów z tego rozdziału można znaleźć pod adresem *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter06*.

Wszystkie pakiety Pythona użyte w tym rozdziale są zawarte w pliku *requirements.txt* dołączonym do plików z kodem źródłowym przykładów z tego rozdziału. Aby zainstalować każdy pakiet Pythona, możesz postępować zgodnie z instrukcjami zamieszczonymi w poniższych podrozdziałach. Możesz także zainstalować wszystkie wymagania jednocześnie za pomocą polecenia `pip install -r requirements.txt`.

Utworzenie witryny internetowej do kolekcjonowania obrazów

Dowiesz się teraz, jak umożliwić użytkownikom dodawanie zakładek do zdjęć znalezionych w innych witrynach internetowych i udostępnianie ich na swojej stronie. Aby zaimplementować tę funkcjonalność, potrzebne będą takie elementy:

- Zdefiniowanie modelu przeznaczonego do przechowywania obrazów i informacji o nich.
- Utworzenie formularza i widoku, które pozwolą na przekazywanie obrazów do witryny internetowej.

- Kod bookmarkletu JavaScript, który można uruchomić na dowolnej stronie internetowej. Ten kod będzie odpowiedzialny za znajdowanie zdjęć na stronie i umożliwienie użytkownikom wyboru zdjęcia, które ma być dodane do zakładek.

Najpierw utwórz nową aplikację w katalogu projektu `bookmarks` za pomocą poniższego polecenia.

```
django-admin startapp images
```

Umieść nową aplikację na liście `INSTALLED_APPS` w pliku *settings.py*, tak jak pokazałem poniżej.

```
INSTALLED_APPS = [
    # ...
    'images.apps.ImagesConfig',
]
```

W tym momencie uaktywniłeś w projekcie aplikację `images`.

Utworzenie modelu Image

Przeprowadź edycję pliku *models.py* aplikacji `images` i dodaj poniższy fragment kodu.

```
from django.conf import settings
from django.db import models


class Image(models.Model):
    user = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        related_name='images_created',
        on_delete=models.CASCADE
    )
    title = models.CharField(max_length=200)
    slug = models.SlugField(max_length=200, blank=True)
    url = models.URLField(max_length=2000)
    image = models.ImageField(upload_to='images/%Y/%m/%d/')
    description = models.TextField(blank=True)
    created = models.DateTimeField(auto_now_add=True)

    class Meta:
       indexes = [
            models.Index(fields=['-created']),
        ]
        ordering = ['-created']
    def __str__(self):
        return self.title
```

To jest model, który będziemy wykorzystywać do przechowywania zdjęć na platformie. Spójrz na pola tego modelu.

- `user`: Obiekt `User` wskazujący użytkownika, który oznaczył dany obraz. Jest to pole klucza obcego (`ForeignKey`), ponieważ definiuje relację typu „jeden do wielu”: użytkownik może opublikować wiele obrazów, ale każdy obraz jest

publikowany przez jednego użytkownika. Dla parametru `on_delete` użyliśmy frazy `CASCADE`, aby po usunięciu użytkownika były usuwane powiązane z nim obrazy.

- `title`: Tytuł obrazu.
- `slug`: Krótka etykieta zawierająca jedynie litery, cyfry, znaki podkreślenia lub myślnik. Etykietę wykorzystujemy do przygotowania eleganckich adresów URL, które będą przyjazne mechanizmom SEO.
- `url`: Adres URL, pod którym został znaleziony obraz. Do określenia maksymalnej długości 2000 znaków używamy opcji `max_length`.
- `image`: Plik obrazu.
- `description`: Opcjonalny opis obrazu.
- `created`: Data i godzina utworzenia obiektu w bazie danych. Dodaliśmy opcję `auto_now_add`, aby podczas tworzenia obiektu automatycznie ustawić bieżącą datę i godzinę.

W klasie `Meta` modelu zdefiniowaliśmy dla tworzonego pola indeks bazy danych w celu porządkowania w kolejności malejącej. Aby zlecić frameworkowi Django posortowanie wyników domyślnie według pola `created`, skorzystaliśmy z atrybutu `ordering`. Kolejność malejącą wskazujemy za pomocą symbolu minus przed nazwą pola, na przykład `-created`.

Indeksy w bazie danych poprawiają wydajność wykonywania zapytań. Rozważ utworzenie indeksów dla pól, które są często wykorzystywane w zapytaniach z użyciem funkcji `filter()`, `exclude()` lub `order_by()`. Pola typu `ForeignKey` lub zdefiniowane z użyciem opcji `unique=True` będą zawierały indeks. Więcej informacji o indeksach baz danych znajdziesz pod adresem *https://docs.djangoproject.com/en/5.0/ref/models/options/#django.db.models.Options.indexes*.

Nadpiszemy metodę `save()` modelu `Image` w celu automatycznego wygenerowania wartości dla kolumny `slug` na podstawie wartości pola `title`. Zaimportuj funkcję `slugify()` i dodaj metodę `save()` do modelu `Image`, tak jak pokazałem poniżej. Nowe wiersze zostały wyróżnione pogrubioną czcionką.

```
from django.utils.text import slugify


class Image(models.Model):
    # ...


    def save(self, *args, **kwargs):
        if not self.slug:
            self.slug = slugify(self.title)
        super().save(*args, **kwargs)
```

Jeśli podczas zapisywania obiektu `Image` pole `slug` nie ma wartości, używana jest funkcja `slugify()`, która automatycznie generuje wartość `slug` na podstawie pola `title` obrazu. Następnie zapisujemy obiekt. Dzięki automatycznemu generowaniu pól `slug` na podstawie tytułu użytkownicy nie będą musieli podawać wartości slug podczas udostępniania obrazów w witrynie.

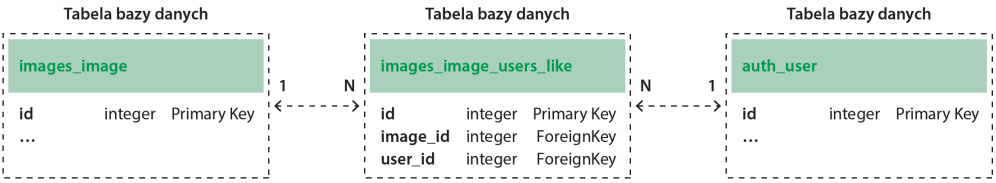
Zdefiniowanie relacji typu „wiele do wielu”

Do modelu `Image` dodamy kolejne pole przeznaczone do przechowywania informacji o polubieniu obrazu przez użytkownika. W takim przypadku konieczne jest zdefiniowanie relacji typu „wiele do wielu”, ponieważ użytkownik może polubić wiele obrazów, a ten sam obraz mógł zostać polubiony przez wielu użytkowników.

Dodaj poniższe pole do modelu `Image`.

```
users_like = models.ManyToManyField(
    settings.AUTH_USER_MODEL,
    related_name='images_liked',
    blank=True
)
```

Po zdefiniowaniu pola `ManyToManyField` Django utworzy pośrednią tabelę złączenia przy użyciu kluczy głównych obu modeli. Tabela bazy danych, która zostanie utworzona dla tej relacji, jest przedstawiona na rysunku 6.2.



Rysunek 6.2. Pośrednia tabela bazy danych dla relacji „wiele do wielu”

Django tworzy tabelę pośrednią `images_image_users_like`, która ma odniesienia do tabeli `images_image` (model `Image`) i tabeli `auth_user` (model `User`). Kolumna typu `ManyToManyField` może być zdefiniowana w dowolnym z dwóch powiązanych ze sobą modeli.

Podobnie jak w przypadku pól typu `ForeignKey`, atrybut `related_name` w `ManyToManyField` pozwala na nadanie nazwy relacji z powiązanego obiektu. Pola `ManyToManyField` zapewniają menedżera relacji typu „wiele do wielu” pozwalającego na pobieranie powiązanych obiektów, np. `image.users_like.all()`, lub z obiektu użytkownika, np. `user.images_liked.all()`.

Więcej informacji o relacjach „wiele do wielu” można znaleźć pod adresem *https://docs.djangoproject.com/en/5.0/topics/db/examples/many_to_many/*.

Przejdź do powłoki i wydaj poniższe polecenie w celu utworzenia początkowej migracji.

```
python manage.py makemigrations images
```

Powinieneś otrzymać wynik podobny do następującego.

```
Migrations for 'images':
  images/migrations/0001_initial.py
    - Create model Image
    - Create index images_imag_created_d57897_idx on field(s) -created of model image
```

Teraz wydaj poniższe polecenie, które spowoduje faktyczne przeprowadzenie migracji.

```
python manage.py migrate images
```

Otrzymasz wynik zawierający m. in. poniższy wiersz.

```
Applying images.0001_initial... OK
```

W tym momencie model `Image` został zsynchronizowany z bazą danych.

Rejestracja modelu Image w witrynie administracyjnej

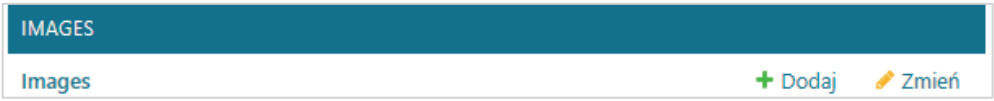
Przeprowadź edycję pliku *admin.py* aplikacji `images` i zarejestruj model `Image` w witrynie administracyjnej, co pokazałem poniżej.

```
from django.contrib import admin
from .models import Image

@admin.register(Image)
class ImageAdmin(admin.ModelAdmin):
    list_display = ['title', 'slug', 'image', 'created']
    list_filter = ['created']
```

Uruchom serwer programistyczny za pomocą poniższego polecenia.

```
python manage.py runserver_plus --cert-file cert.crt
```

W przeglądarce internetowej przejdź pod adres `https://127.0.0.1:8000/admin/`, a zobaczysz nowy model `Image` wyświetlony w witrynie administracyjnej Django (rysunek 6.3).



Rysunek 6.3. Blok Images na głównej stronie witryny administracyjnej Django

Właśnie stworzyłeś model do przechowywania zdjęć. W następnym podrozdziale dowiesz się, jak zaimplementować formularz do pobierania zdjęć według ich adresów URL i przechowywania ich za pomocą modelu `Image`.

Umieszczanie treści pochodzącej z innych witryn internetowych

Teraz umożliwimy użytkownikom dodawanie zakładek do zdjęć pochodzących z zewnętrznych witryn internetowych i udostępnianie ich na naszej stronie. Użytkownik wprowadzi adres URL zdjęcia, jego tytuł oraz opcjonalnie opis. Stworzymy formularz i widok do pobrania zdjęcia oraz utworzymy w bazie danych nowy obiekt `Image`.

Pracę rozpoczynamy od przygotowania formularza przeznaczonego do przekazywania obrazów.

Utwórz nowy plik *forms.py* w katalogu aplikacji `images` i umieść w nim poniższy fragment kodu.

```
from django import forms
from .models import Image


class ImageCreateForm(forms.ModelForm):
    class Meta:
        model = Image
        fields = ['title', 'url', 'description']
        widgets = {
             'url': forms.HiddenInput,
        }
```

Na podstawie modelu `Image` zdefiniowaliśmy formularz `ModelForm`, zawierający tylko pola `title`, `url` i `description`. Użytkownicy witryny nie będą wprowadzać adresu URL obrazu bezpośrednio w tym formularzu. Zamiast tego otrzymają do dyspozycji narzędzie Java Script pozwalające na wybranie obrazu z zewnętrznej witryny, a jego adres URL będzie w postaci parametru przekazany omawianemu formularzowi. Nadpiszemy domyślny widżet dla kolumny `url` i wykorzystamy widżet `HiddenInput`. Widżet ten jest generowany w postaci elementu HTML `<input>` wraz z atrybutem `type="hidden"`. Zdecydowałem się na takie rozwiązanie, ponieważ pole adresu URL nie powinno być widoczne dla użytkowników.

Usunięcie zawartości pól formularza

W celu potwierdzenia poprawności adresu URL obrazu sprawdzamy, czy rozszerzenie jego pliku to *.jpg*, *.jpeg* lub *.png*, ponieważ budowana witryna umożliwia przekazywanie obrazów jedynie w formatach JPEG i PNG. W poprzednim rozdziale do zaimplementowania mechanizmów walidacji pól użyliśmy konwencji `clean_<nazwa_pola>()`. Wymieniona metoda jest wywoływana dla każdego pola po wywołaniu `is_valid()` z poziomu egzemplarza formularza. W metodzie `clean` można zmienić wartość pola lub, jeśli trzeba, zgłosić błąd walidacji danego pola.

W pliku *forms.py* aplikacji `images` dodaj do klasy `ImageCreateForm` następującą metodę.

```
def clean_url(self):
    url = self.cleaned_data['url']
    valid_extensions = ['jpg', 'jpeg', 'png']
    extension = url.rsplit('.', 1)[1].lower()
    if extension not in valid_extensions:
        raise forms.ValidationError(
           'Podany adres URL nie zawiera obrazów w obsługiwanym formacie.'
        )
    return url
```

W tym kodzie definiujemy metodę `clean_url()` przeznaczoną do wyczyszczenia zawartości pola `url`. Oto działanie tego kodu:

1. Pobieramy wartość pola `url` przez uzyskanie dostępu do słownika `cleaned_data` egzemplarza formularza.

2. Przeprowadzamy podział adresu URL w celu otrzymania rozszerzenia pliku i sprawdzenia, czy odpowiada ono jednemu z poprawnych rozszerzeń. Jeżeli rozszerzenie nie zostanie dopasowane, następuje zgłoszenie wyjątku `ValidationError` i egzemplarz formularza nie będzie zweryfikowany poprawnie.

Oprócz sprawdzenia poprawności podanego adresu URL trzeba również pobrać plik obrazu i go zapisać. Do tego celu moglibyśmy na przykład użyć widoku obsługującego formularz pobierania pliku obrazu. Zamiast tego przyjmijmy bardziej ogólne podejście — przesłonimy metodę `save()` formularza modelu, aby wykonać to zadanie podczas zapisywania formularza.

Instalowanie biblioteki Requests

Gdy użytkownik doda zdjęcie do zakładek, będzie trzeba pobrać plik obrazu według jego adresu URL. Do tego celu wykorzystamy bibliotekę Pythona `Requests`. `Requests` to najpopularniejsza biblioteka obsługi HTTP dla Pythona. Biblioteka zawiera abstrakcje dla złożonych operacji obsługi żądań HTTP i zapewnia bardzo prosty interfejs do korzystania z usług HTTP. Dokumentację biblioteki `Requests` można znaleźć pod adresem *https://requests.readthedocs.io/en/master/*.

Otwórz powłokę i zainstaluj bibliotekę `Requests` za pomocą polecenia:

```
python -m pip install requests==2.31.0
```

Teraz nadpiszemy metodę `save()` klasy `ImageCreateForm` i skorzystamy z biblioteki `Requests` do pobrania obrazu na podstawie jego adresu URL.

Nadpisanie metody save() egzemplarza ModelForm

Jak wiesz, klasa `ModelForm` zawiera metodę `save()`, odpowiedzialną za zapisanie w bazie danych bieżącego egzemplarza modelu i zwrócenie obiektu. Metoda ta otrzymuje parametr typu `Boolean` o nazwie `commit` pozwalający na wskazanie, czy obiekt ma być na stałe zachowany w bazie danych. Jeżeli wartością parametru `commit` będzie `False`, metoda `save()` zwróci egzemplarz modelu, ale nie zapisze go w bazie danych. W budowanym formularzu nadpiszemy metodę `save()`, aby otrzymać możliwość pobierania pliku obrazu spod wskazanego adresu URL i zapisywania go w systemie plików.

Dodaj poniższe polecenia `import` na początku pliku *forms.py*.

```
import requests
from django.core.files.base import ContentFile
from django.utils.text import slugify
```

Następnie dodaj poniższą metodę `save()` do kodu formularza `ImageCreateForm`.

```
def save(self, force_insert=False, force_update=False, commit=True):
    image = super().save(commit=False)
    image_url = self.cleaned_data['url']
    name = slugify(image.title)
    extension = image_url.rsplit('.', 1)[1].lower()
    image_name = f'{name}.{extension}'
```

```
        # pobranie pliku obrazu z podanego adresu URL
        response = requests.get(image_url)
        image.image.save(
            image_name,
            ContentFile(response.content),
            save=False
        )
        if commit:
            image.save()
        return image
```

Nadpisaliśmy metodę `save()`, ale zachowaliśmy parametry wymagane przez klasę `Model` ↪`Form`. Działanie powyższego kodu przedstawia się następująco.

1. Utworzenie nowego egzemplarza `image` przez wywołanie metody `save()` formularza wraz z parametrem `commit=False`.
2. Pobranie adresu URL ze słownika `cleaned_data` formularza.
3. Wygenerowanie nazwy obrazu przez połączenie slugu tytułu z pierwotnym rozszerzeniem pliku.
4. Do pobrania obrazu przez wysłanie żądania HTTP `GET` pod wskazany adres URL skorzystaliśmy z biblioteki Pythona `Requests`. Odpowiedź została zwrócona za pomocą obiektu `response`.
5. Wywoływana jest metoda `save()`, do której przekazujemy obiekt `ContentFile` utworzony na podstawie zawartości pobranego pliku. W ten sposób plik zostanie zapisany w katalogu `media` projektu. Ponadto przekazywany jest parametr `save=False`, aby w tym momencie jeszcze nie zapisywać obiektu w bazie danych.
6. W celu zachowania pierwotnej funkcjonalności nadpisywanej metody `save()` formularz zapisywany jest w bazie danych tylko po podaniu parametru `commit` z wartością `True`.

Teraz potrzebujemy widoku, w którym utworzymy egzemplarz formularza i obsłużymy jego przesyłanie.

Przeprowadź edycję pliku *views.py* aplikacji `images` i umieść w nim poniższy fragment kodu. Nowy kod został wyróżniony pogrubioną czcionką.

```
from django.contrib import messages
from django.contrib.auth.decorators import login_required
from django.shortcuts import redirect, render
from .forms import ImageCreateForm


@login_required
def image_create(request):
    if request.method == 'POST':
        # Formularz został wysłany
        form = ImageCreateForm(data=request.POST)
        if form.is_valid():
            # Dane formularza są prawidłowe
            cd = form.cleaned_data
            new_image = form.save(commit=False)
```

```
            # Przypisanie bieżącego użytkownika do elementu
            new_image.user = request.user
            new_image.save()
            messages.success(request, 'Obraz został dodany pomyślnie')
            # Przekierowanie do widoku szczegółowego dla nowo utworzonego elementu
            return redirect(new_image.get_absolute_url())
    else:
        # Utworzenie formularza na podstawie danych dostarczonych przez bookmarklet
        # w żądaniu GET
        form = ImageCreateForm(data=request.GET)
    return render(
        request,
        'images/image/create.html',
        {'section': 'images', 'form': form}
    )
```

W powyższym kodzie utworzyliśmy widok do przechowywania zdjęć w witrynie. Aby uniemożliwić uzyskanie dostępu nieuwierzytelnionym użytkownikom, do widoku `image_create` dodaliśmy dekorator `login_required`. Oto sposób działania powyższego fragmentu kodu:

1. Oczekujemy otrzymania za pomocą żądania HTTP `GET` danych początkowych, na podstawie których utworzymy egzemplarz formularza. Dane te będą się składać z atrybutów `url` i `title` zdjęcia z zewnętrznej witryny internetowej. Oba parametry zostaną ustawione w żądaniu `GET` przez bookmarklet JavaScript, którego utworzeniem wkrótce się zajmiemy. Na razie możemy założyć, że te dane będą dostępne w żądaniu.
2. Gdy formularz zostaje przesłany z zastosowaniem żądania HTTP `POST`, za pomocą wywołania `form.is_valid()` jest weryfikowana jego poprawność. Jeśli dane formularza są prawidłowe, tworzony jest nowy egzemplarz modelu `Image` przez zapisanie formularza za pomocą wywołania `form.save(commit=False)`. Nowy egzemplarz nie jest zapisywany w bazie danych z powodu ustawienia `commit=False`.
3. Za pomocą wywołania `new_image.user = request.user` do nowego egzemplarza modelu `Image` dodawana jest relacja z bieżącym użytkownikiem wykonującym żądanie. W ten sposób będziemy później wiedzieć, kto przekazał dany obraz do aplikacji.
4. Zapisujemy obiekt `Image` w bazie danych.
5. Na koniec za pomocą frameworka `messages` w Django wyświetlamy komunikat informujący o sukcesie operacji i przekierowujemy użytkownika do kanonicznego adresu URL nowego obrazu. Jeszcze nie zaimplementowaliśmy metody `get_absolute_url()` dla modelu `Image`, zajmiemy się tym nieco później.

Utwórz nowy plik *urls.py* w aplikacji `images` i umieść w nim poniższy fragment kodu.

```
from django.urls import path
from . import views


app_name = 'images'
```

```
urlpatterns = [
     path('create/', views.image_create, name='create'),
]
```

Przeprowadź edycję głównego pliku *urls.py* projektu `bookmarks` w celu dołączenia utworzonych przed chwilą wzorców dla aplikacji `images`, tak jak pokazałem poniżej. Nowy kod został wyróżniony pogrubioną czcionką.

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('account/', include('account.urls')),
    path(
        'social-auth/',
        include('social_django.urls', namespace='social')
    ),
    path('images/', include('images.urls', namespace='images')),
]
```

Trzeba jeszcze przygotować szablon pozwalający na wyrenderowanie formularza. Dlatego też w katalogu aplikacji `images` utwórz poniższą strukturę katalogów i plików.

```
templates/
   images/
     image/
       create.html
```

Przeprowadź edycję nowego szablonu *create.html* i dodaj do niego poniższy fragment kodu.

```
{% extends "base.html" %}

{% block title %}Dodaj obraz{% endblock %}

{% block content %}
  <h1>Dodaj obraz</h1>
  <img src="{{ request.GET.url }}" class="image-preview">
  <form method="post">
    {{ form.as_p }}
    {% csrf_token %}
    <input type="submit" value="Zaznacz!">
  </form>
{% endblock %}
```

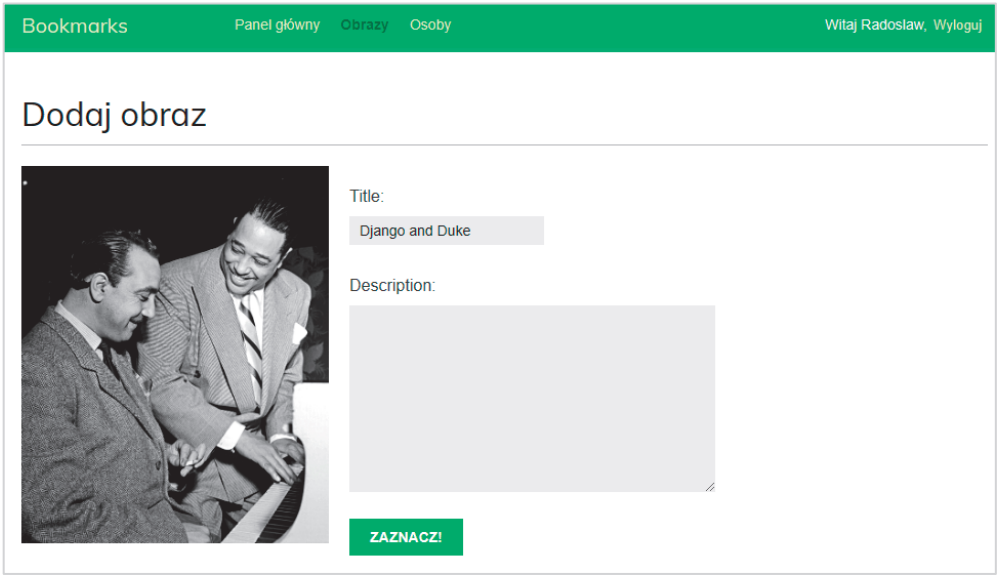
Otwórz powłokę i uruchom serwer programistyczny za pomocą polecenia:

```
python manage.py runserver_plus --cert-file cert.crt
```

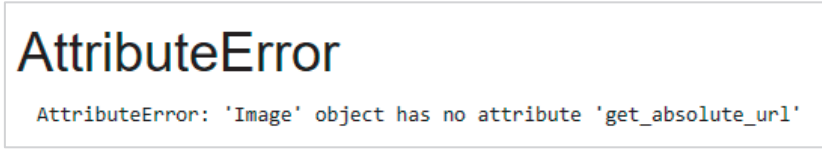
W przeglądarce internetowej przejdź pod adres `https://127.0.0.1:8000/images/create/?title=…&url=…`, podając w żądaniu `GET` odpowiednie wartości parametrów `title` i `url`. Możesz na przykład użyć następującego adresu URL: `https://127.0.0.1:8000/images/create/?title=%20Django%20and%20Duke&url=https://upload.wikimedia.org/wikipedia/commons/8/85/Django_Reinhardt_and_Duke_Ellington_%28Gottlieb%29.jpg`.

Zobaczysz formularz wraz z wyświetlonym podglądem obrazu, co pokazałem na rysunku 6.4.

Dodaj opis i kliknij przycisk *ZAZNACZ!*. Nowy obiekt `Image` zostanie zapisany w bazie danych. Otrzymasz jednak komunikat o błędzie informujący o braku metody `get_absolute_url()` w modelu `Image` (rysunek 6.5).



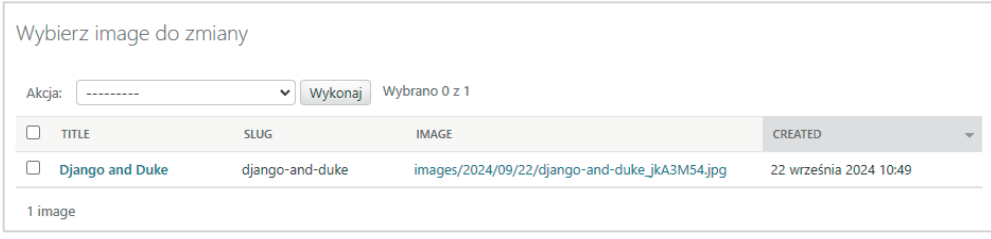
Rysunek 6.4. Strona tworzenia nowej zakładki dla obrazu



Rysunek 6.5. Komunikat o błędzie z informacją o braku atrybutu get_absolute_url w obiekcie Image

Na razie nie przejmuj się tym błędem; metodę `get_absolute_url` zaimplementujemy w modelu `Image` nieco później.

W przeglądarce internetowej przejdź pod adres `https://127.0.0.1:8000/admin/images/image/` i sprawdź, czy nowy obiekt obrazu faktycznie został umieszczony w bazie danych (rysunek 6.6).

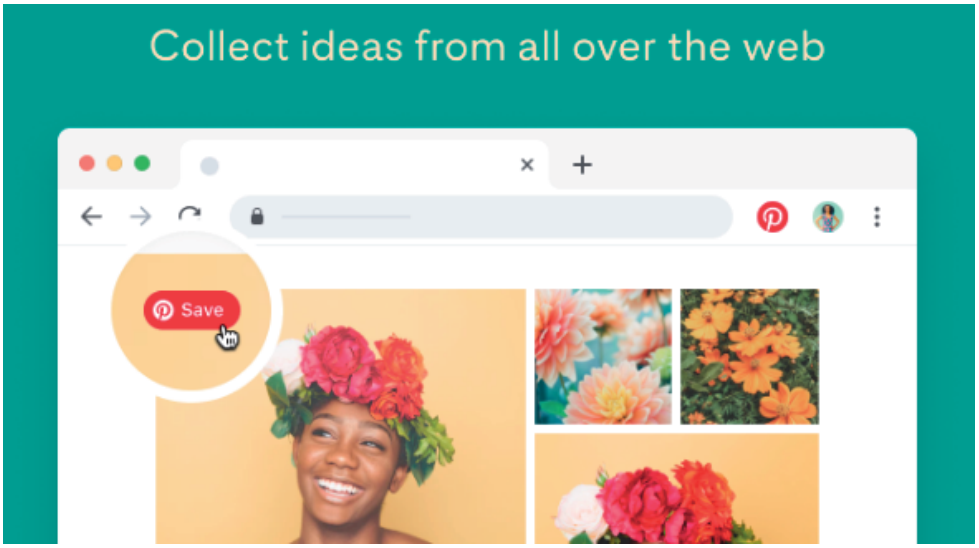


Rysunek 6.6. Strona witryny administracyjnej z listą obrazów zawierającą utworzony obiekt Image

Utworzenie bookmarkletu za pomocą JavaScript

Bookmarklet jest przechowywaną w przeglądarce internetowej zakładką, która zawiera kod JavaScript rozszerzający funkcjonalność przeglądarki. Po kliknięciu w przeglądarce bookmarkletu na pasku zakładek lub ulubionych stron następuje wykonanie zdefiniowanego w nim kodu JavaScript względem strony internetowej obecnie wyświetlonej w przeglądarce. Jest to bardzo użyteczna możliwość podczas tworzenia narzędzi współdziałających z innymi witrynami internetowymi.

Niektóre serwisy internetowe, np. Pinterest, implementują własne bookmarklety umożliwiające użytkownikom udostępnianie na danej platformie treści pochodzącej z innych witryn internetowych. Bookmarklet serwisu Pinterest jest zaimplementowany jako rozszerzenie przeglądarki i jest dostępny pod adresem *https://help.pinterest.com/en/article/save-pins-with-the-pinterest-browser-button*. Rozszerzenie *Pinterest Save* umożliwia użytkownikowi zapisywanie obrazów lub stron internetowych na jego koncie Pinterest za pomocą jednego kliknięcia w przeglądarce (rysunek 6.7).



Rysunek 6.7. Rozszerzenie Pinterest Save

Spróbujmy stworzyć podobny bookmarklet dla Twojej witryny internetowej. Do tego celu wykorzystamy JavaScript.

Poniżej przedstawiłem sposób, w jaki użytkownik dodaje bookmarklet do przeglądarki internetowej, a następnie korzysta z niego.

1. Użytkownik przeciąga link ze strony internetowej na pasek zakładek w przeglądarce. Wspomniany link zawiera w atrybucie `href` kod JavaScript. Ten kod będzie przechowywany w zakładce.
2. Użytkownik przechodzi na dowolną stronę internetową i klika bookmarklet w zakładkach lub na pasku ulubionych. W rezultacie uruchamiany jest kod JavaScript zakładki.

Ponieważ kod JavaScript będzie przechowywany jako zakładka, po dodaniu go przez użytkownika do paska zakładek nie będzie go można później uaktualnić. Jest to poważna wada, ale można sobie z nią poradzić przez zaimplementowanie skryptu uruchomieniowego. Użytkownicy zapiszą skrypt uruchomieniowy jako zakładkę, a następnie ten skrypt uruchomieniowy załaduje właściwy bookmarklet JavaScript z odpowiedniego adresu URL. W ten sposób będziesz mógł w każdej chwili zaktualizować kod bookmarkletu. Dokładnie takie podejście zastosujemy podczas opracowywania przykładowego bookmarkletu. Przystępujemy więc do pracy!

Utwórz nowy szablon w katalogu *images/templates* i nadaj mu nazwę *bookmarklet_launcher.js*. Będzie to wspomniany wcześniej skrypt uruchamiający rzeczywisty bookmarklet. W tym pliku umieść poniższy kod JavaScript.

```
(function(){
   if(!window.bookmarklet) {
     bookmarklet_js = document.body.appendChild(document.
createElement('script'));
     bookmarklet_js.src = '//127.0.0.1:8000/static/js/bookmarklet.js?r='+Math.
floor(Math.random()*9999999999999999);
     window.bookmarklet = true;
   }
   else {
     bookmarkletLaunch();
   }
})();
```

Powyższy skrypt sprawdza, czy bookmarklet został już załadowany. W tym celu za pomocą instrukcji `if(!window.bookmarklet)` sprawdzamy wartość zmiennej bookmarkletu `window`.

- Jeśli zmienna `window.bookmarklet` nie została zdefiniowana lub nie ma wartości `true`, następuje załadowanie pliku JavaScript przez dodanie do treści dokumentu HTML załadowanego w przeglądarce elementu `<script>`. Do załadowania adresu URL skryptu *bookmarklet.js* z losowym, 16-cyfrowym całkowitoliczbowym parametrem wygenerowanym za pomocą instrukcji `Math.random()*9999999999999999` wykorzystywany jest atrybut `src`. Jeśli bookmarklet JavaScript został wcześniej załadowany, inna wartość parametru zmusi przeglądarkę do ponownego załadowania skryptu ze źródłowego adresu URL. W ten sposób zyskujemy pewność, że bookmarklet zawsze uruchomi najbardziej aktualny kod JavaScript.
- Jeśli zmienna `window.bookmarklet` jest zdefiniowana i ma wartość `true`, wykonywana jest funkcja `bookmarkletLaunch()`. Zdefiniujemy ją jako funkcję globalną w skrypcie *bookmarklet.js*.

Dzięki sprawdzeniu zmiennej `window` bookmarkletu zapobiegamy ładowaniu kodu Java Script bookmarkletu więcej niż jeden raz w przypadku, gdy użytkownik kliknie bookmarklet wielokrotnie.

Właśnie utworzyłeś kod skryptu uruchamiającego bookmarklet. Właściwy kod bookmarkletu znajduje się w statycznym pliku *bookmarklet.js*. W ten sposób zachowujemy możliwość uaktualnienia bookmarkletu bez konieczności proszenia użytkowników o uaktualnienie bookmarkletu wcześniej dodanego przez nich do przeglądarki internetowej.

Przystępujemy teraz do utworzenia skryptu uruchamiającego bookmarklet i umieścimy go w panelu głównym, aby użytkownicy mogli go kopiować na listę zakładek w swoich przeglądarkach.

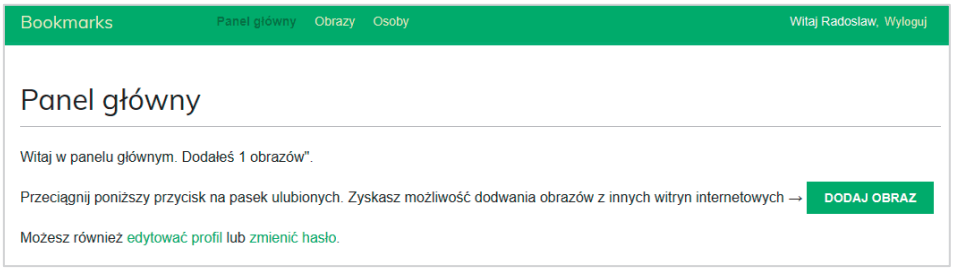
Przeprowadź edycję szablonu *account/dashboard.html*, znajdującego się w aplikacji `account`, i umieść w nim następujący kod. Nowe wiersze zostały wyróżnione pogrubioną czcionką.

```
{% extends "base.html" %}

{% block title %}Panel główny{% endblock %}

{% block content %}
  <h1>Panel główny</h1>
  {% with total_images_created=request.user.images_created.count %}
    <p>Witaj w panelu głównym. Dodałeś {{ total_images_created}} obrazów.</p>
  {% endwith %}
  <p>Przeciągnij poniższy przycisk na pasek ulubionych. Zyskasz możliwość dodawania
  ↪obrazów z innych witryn internetowych → <a href="javascript:{% include
  ↪"bookmarklet_launcher.js" %}" class="button">Dodaj obraz</a></p>
  <p>Możesz również <a href="{% url "edit" %}">edytować profil</a> lub <a href="
  ↪{% url "password_change" %}">zmienić hasło</a>.</p>
{% endblock %}
```

Zadbaj o to, aby tag szablonu nie był podzielony na wiele wierszy. Django nie obsługuje tagów złożonych z wielu wierszy.

Panel główny wyświetla teraz całkowitą liczbę obrazów przekazanych przez danego użytkownika. Do ustawienia zmiennej przechowującej wspomnianą liczbę wykorzystaliśmy znacznik szablonu `{% with %}`. Ponadto w kodzie znajduje się link wraz z atrybutem `href` zawierającym skrypt uruchomieniowy bookmarkletu. Dołączamy kod JavaScript pochodzący z szablonu *bookmarklet_launcher.js*.

W przeglądarce internetowej przejdź pod adres `https://127.0.0.1:8000/account/`. Powinieneś zobaczyć stronę podobną do pokazanej na rysunku 6.8.



Rysunek 6.8. Strona panelu głównego zawierająca liczbę oznaczonych zdjęć oraz przycisk dla bookmarkletu

Teraz w katalogu aplikacji `images` utwórz następujące pliki i katalogi.

```
static/
   js/
     bookmarklet.js
```

W kodzie przygotowanym dla książki katalog *static/css* znajdziesz w katalogu aplikacji `images`. Skopiuj katalog *css/* do katalogu *static/* budowanej aplikacji. Zawartość katalogu można pobrać spod adresu *https://github.com/PacktPublishing/Django-5-by-Example/tree/main/Chapter06/bookmarks/images/static*.

Arkusze stylu dla tworzonego bookmarkletu JavaScript są zapisane w pliku *css/bookmarklet.css*. Katalog *static/* powinien teraz zawierać taką strukturę plików:

```
css/
   bookmarklet.css
js/
   bookmarklet.js
```

Przeprowadź edycję statycznego pliku *bookmarklet.js* i dodaj do niego poniższy fragment kodu JavaScript.

```
const siteUrl = '//127.0.0.1:8000/';
const styleUrl = siteUrl + 'static/css/bookmarklet.css';
const minWidth = 200;
const minHeight = 200;
```

Zadeklarowałeś cztery stałe, które będą wykorzystywane przez bookmarklet. Te stałe to:

- `siteUrl` i `staticUrl`: Bazowy adres URL witryny i bazowy adres URL statycznych plików.
- `minWidth` i `minHeight`: Minimalna szerokość i wysokość obrazów w pikselach, które bookmarklet będzie pobierać z witryny. Bookmarklet zidentyfikuje obrazy, które mają co najmniej 250 pikseli szerokości i 250 pikseli wysokości.

Przeprowadź edycję statycznego pliku *bookmarklet.js* i dodaj do niego poniższy fragment kodu JavaScript wyróżniony pogrubioną czcionką.

```
const siteUrl = '//127.0.0.1:8000/';
const styleUrl = siteUrl + 'static/css/bookmarklet.css';
const minWidth = 200;
const minHeight = 200;

// Załaduj CSS
var head = document.getElementsByTagName('head')[0];
var link = document.createElement('link');
link.rel = 'stylesheet';
link.type = 'text/css';
link.href = styleUrl + '?r=' + Math.floor(Math.random()*9999999999999999);
head.appendChild(link);
```

Ten fragment kodu ładuje arkusz stylów CSS dla bookmarkletu. Wykorzystaliśmy JavaScript do wykonywania działań na **obiektowym modelu dokumentu** (ang. *Document Object Model* — DOM). Model DOM reprezentuje w pamięci dokument HTML i jest tworzony przez przeglądarkę podczas ładowania strony internetowej. Model DOM jest zbudowany jako drzewo obiektów, które składają się na strukturę i zawartość dokumentu HTML.

Powyższy kod generuje obiekt odpowiadający poniższemu kodowi JavaScript i dołącza go do elementu `<head>` strony HTML.

```
<link rel="stylesheet" type="text/css" href= "//127.0.0.1:8000/static/css/
↪bookmarklet.css?r=1234567890123456">
```

Przyjrzyjmy się, jak działa ten kod.

1. Za pomocą metody `document.getElementsByTagName()` pobieramy element `<head>` strony. Ta funkcja pobiera wszystkie elementy HTML strony z podanym tagiem. Dzięki wykorzystaniu indeksu `[0]` sięgamy do pierwszego znalezionego egzemplarza. Sięgamy do pierwszego elementu, ponieważ wszystkie dokumenty HTML powinny mieć taki sam element `<head>`.
2. Za pomocą metody `document.createElement('link')` tworzymy element `<link>`.
3. Ustawiamy atrybuty `rel` i `type` elementu `<link>`. Jest to odpowiednik kodu HTML `<link rel="stylesheet" type="text/css">`.
4. Atrybut `href` elementu `<link>` jest ustawiany na adres URL arkusza stylów *bookmarklet.css*. Aby uniemożliwić przeglądarce załadowanie pliku z pamięci podręcznej, jako parametr adresu URL używana jest 16-cyfrowa liczba losowa.
5. Za pomocą metody `head.appendChild(link)` dołączamy do strony HTML nowy element, `<link>`.

Teraz utworzymy element HTML, aby wyświetlić kontener na stronie internetowej, w którym uruchamiany jest bookmarklet. Kontener HTML służy do wyświetlania wszystkich zdjęć znalezionych w witrynie i umożliwi użytkownikom wybór zdjęcia, które chcą udostępnić. Użyje stylów CSS zdefiniowanych w arkuszu stylów *bookmarklet.css*.

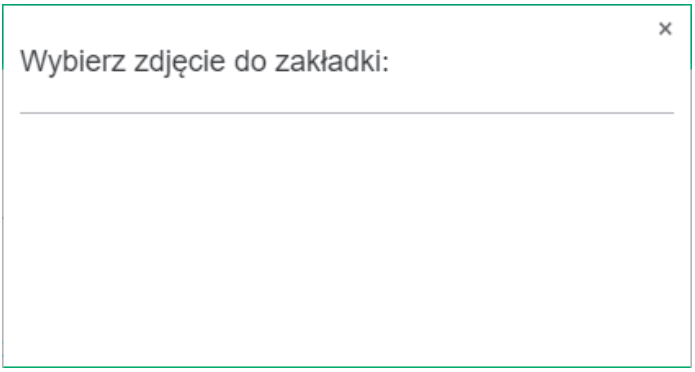
Przeprowadź edycję statycznego pliku *bookmarklet.js* i dodaj do niego poniższy fragment kodu JavaScript wyróżniony pogrubioną czcionką.

```
const siteUrl = '//127.0.0.1:8000/';
const styleUrl = siteUrl + 'static/css/bookmarklet.css';
const minWidth = 200;
const minHeight = 200;

// Załaduj CSS
var head = document.getElementsByTagName('head')[0];
var link = document.createElement('link');
link.rel = 'stylesheet';
link.type = 'text/css';
link.href = styleUrl + '?r=' + Math.floor(Math.random()*9999999999999999);
head.appendChild(link);

// Wczytanie kodu HTML
var body = document.getElementsByTagName('body')[0];
boxHtml = `
   <div id="bookmarklet">
    <a href="#" id="close">&times;</a>
    <h1>Wybierz zdjęcie do zakładki:</h1>
    <div class="images"></div>
   </div>`;
body.innerHTML += boxHtml;
```

Powyższy kod pobiera element <body> modelu DOM i dodaje do niego nowy kod HTML przez zmodyfikowanie powiązanej z nim właściwości innerHTML. Do treści strony zostanie dodany nowy element . Kontener składa się z następujących elementów.

- Link do zamknięcia kontenera zdefiniowany za pomocą kodu ×.
- Tytuł zdefiniowany za pomocą kodu <h1>Wybierz zdjęcie do zakładki:</h1>.
- Element do wyświetlenia listy zdjęć znalezionych w witrynie określonej za pomocą kodu . Ten kontener jest początkowo pusty i zostanie wypełniony zdjęciami znalezionymi na stronie.

Kontener HTML zawierający poprzednio załadowane style CSS powinien być podobny do pokazanego na rysunku 6.9.



Rysunek 6.9. Kontener wyboru zdjęcia

Teraz zaimplementujmy funkcję uruchamiającą bookmarklet. Przeprowadź edycję statycznego pliku *bookmarklet.js* i dodaj na jego końcu poniższy fragment kodu:

```
function bookmarkletLaunch()
{
   bookmarklet = document.getElementById('bookmarklet');
   var imagesFound = bookmarklet.querySelector('.images');

   // Wyczyszczenie znalezionych zdjęć
   imagesFound.innerHTML = '';
   // Wyświetlenie bookmarkletu
   bookmarklet.style.display = 'block';

   // Zdarzenie close
   bookmarklet.querySelector('#close')
              .addEventListener('click', function(){
                bookmarklet.style.display = 'none'
              });
}
// Uruchomienie bookmarkletu
bookmarkletLaunch();
```

To jest funkcja `bookmarkletLaunch()`. Przed zdefiniowaniem tej funkcji ładowany jest CSS dla bookmarkletu i do modelu DOM strony dodawany jest kontener HTML. Funkcja `bookmarkletLaunch()` działa w następujący sposób.

1. Za pomocą metody `document.getElementById()` przez pobranie elementu DOM z identyfikatorem `bookmarklet` pobierany jest główny kontener bookmarkletu.
2. Element `bookmarklet` służy do pobrania elementów potomnych klasy `images`. Pobieranie elementów DOM za pomocą selektorów CSS umożliwia metoda `querySelector()`. Selektory pozwalają znaleźć te elementy DOM, do których ma zastosowanie zestaw reguł CSS. Listę dostępnych selektorów CSS można znaleźć pod adresem *https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Selectors*. Więcej informacji na temat lokalizowania elementów DOM za pomocą selektorów można znaleźć pod adresem *https://developer.mozilla.org/en-US/docs/Web/API/Document_object_model/Locating_DOM_elements_using_selectors*.
3. Czyścimy kontener zdjęć przez ustawienie jego atrybutu `innerHTML` na pusty ciąg znaków, a następnie wyświetlamy bookmarklet przez ustawienie właściwości CSS `display` na `block`.
4. Do wyszukania elementu DOM o identyfikatorze `close` wykorzystujemy selektor `#close`. Za pomocą metody `addEventListener()` dołączamy do elementu zdarzenie `click`. Gdy użytkownik kliknie element, główny kontener bookmarkletu zostanie ukryty przez ustawienie jego właściwości `display` na `none`.
5. Po zdefiniowaniu funkcji `bookmarkletLaunch()` wywołujemy ją.

Po załadowaniu stylów CSS i kontenera HTML bookmarkletu trzeba znaleźć w modelu DOM bieżącej strony elementy `image`. Do kontenera HTML bookmarkletu dodajemy zdjęcia, które mają minimalny wymagany wymiar. Otwórz do edycji statyczny plik *js/bookmarklet.js* i umieść na końcu funkcji `bookmarklet()` poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
function bookmarkletLaunch()
{
   bookmarklet = document.getElementById('bookmarklet');
   var imagesFound = bookmarklet.querySelector('.images');

   // Wyczyszczenie znalezionych zdjęć
   imagesFound.innerHTML = '';
   // Wyświetlenie bookmarkletu
   bookmarklet.style.display = 'block';

   // Zdarzenie close
   bookmarklet.querySelector('#close')
              .addEventListener('click', function(){
               bookmarklet.style.display = 'none'
              });

   // Znajdź w modelu DOM obrazy o minimalnych wymiarach
   images = document.querySelectorAll('img[src$=".jpg"], img[src$=".jpeg"],
   ↪img[src$=".png"]');
   images.forEach(image => {
     if(image.naturalWidth >= minWidth
```

```
          && image.naturalHeight >= minHeight)
      {
        var imageFound = document.createElement('img');
        imageFound.src = image.src;
        imagesFound.append(imageFound);
      }
    })
}

// Uruchomienie bookmarkletu
bookmarkletLaunch();
```

Poprzedni kod używa selektorów `img[src$=".jpg"]`, `img[src$=".jpeg"]` i `img[src$=".png"]` w celu znalezienia wszystkich elementów `<img>`, których atrybut `src` kończy się, odpowiednio, na *.jpg*, *.jpeg* lub *.png*. Wykorzystanie tych selektorów w metodzie `document ↪.querySelectorAll()` pozwala znaleźć wszystkie wyświetlane na stronie internetowej zdjęcia w formacie *JPEG* i *PNG*. Za pomocą metody `forEach()` przeprowadzamy iterację po wynikach wyszukiwania. Zdjęcia o małych rozmiarach są odfiltrowywane, ponieważ nie jesteśmy nimi zainteresowani. Do wyników dołączane są tylko obrazy o rozmiarze większym niż określony za pomocą zmiennych `minWidth` i `minHeight`. Dla każdego znalezionego obrazu tworzony jest nowy element `<img>`, dla którego z oryginalnego obrazu jest kopiowany atrybut `src` źródłowego adresu URL, dodawany następnie do kontenera `imagesFound`.

Ze względów bezpieczeństwa przeglądarka nie dopuści do uruchomienia bookmarkletu przez HTTP w witrynach serwowanych przez HTTPS. To dlatego wciąż używamy do uruchamiania serwera programistycznego polecenia `RunServerPlus` i wykorzystujemy wygenerowany automatycznie certyfikat TLS/SSL. Sposób uruchamiania serwera programistycznego przez HTTPS został opisany w rozdziale 5., „Implementacja uwierzytelniania za pomocą witryn społecznościowych”.

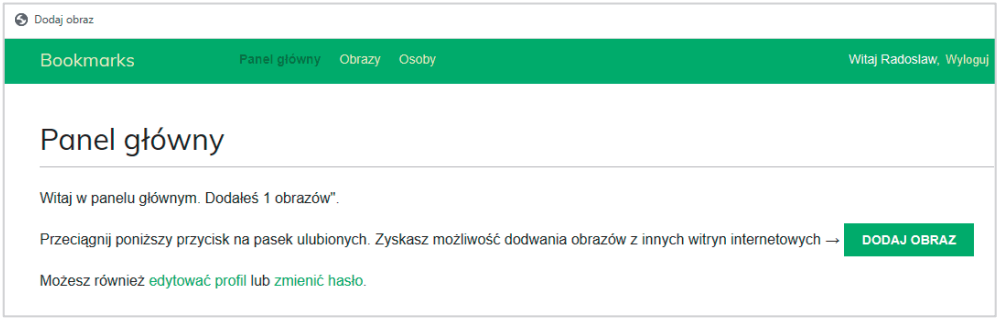
W środowisku produkcyjnym potrzebny będzie certyfikat TLS/SSL potwierdzony przez Urząd Certyfikacji (ang. *Certification Authority* — CA). Jeśli jesteś właścicielem nazwy domeny, możesz poprosić zaufany urząd certyfikacji (CA) o wydanie dla niej certyfikatu TLS/SSL, aby przeglądarki mogły zweryfikować jej tożsamość. Aby uzyskać zaufany certyfikat dla rzeczywistej domeny, możesz skorzystać z usługi Let’s Encrypt. Let’s Encrypt to organizacja non profit, która upraszcza bezpłatne uzyskiwanie i odnawianie zaufanych certyfikatów TLS/SSL. Więcej informacji na ten temat można znaleźć pod adresem *https://letsencrypt.org*.

W terminalu uruchom serwer programistyczny za pomocą polecenia:

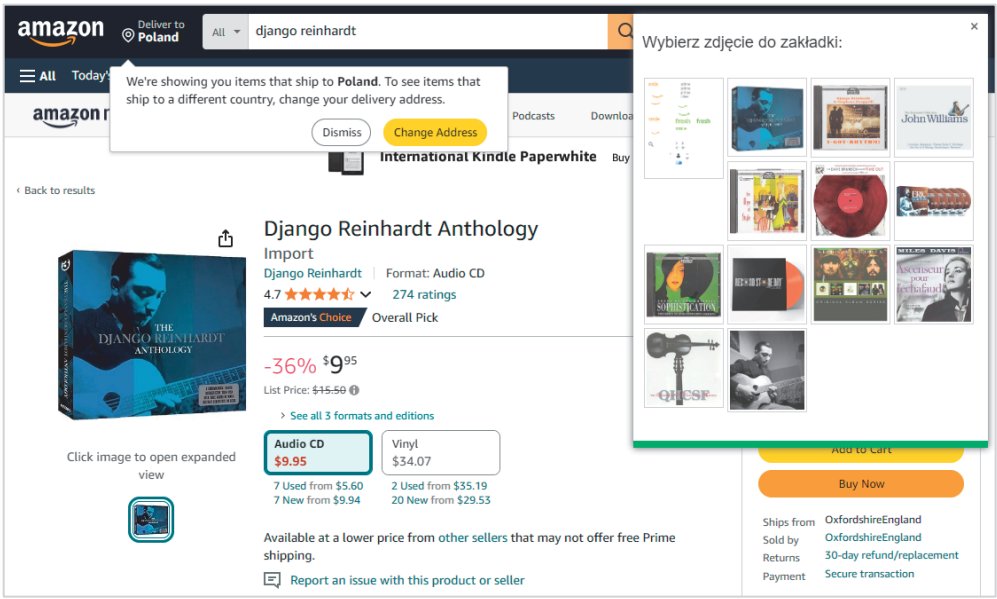
```
python manage.py runserver_plus --cert-file cert.crt
```

W przeglądarce internetowej przejdź pod adres `https://127.0.0.1:8000/account/`. Zaloguj się jako istniejący użytkownik, a następnie przeciągnij przycisk *DODAJ OBRAZ* na pasek zakładek przeglądarki w sposób pokazany na rysunku 6.10.

Otwórz w przeglądarce dowolną stronę internetową i kliknij bookmarklet na pasku zakładek. Zobaczysz, że w witrynie wyświetli się biała ramka zawierająca wszystkie znalezione obrazy JPEG o wymiarach większych niż 250×250 pikseli, którą pokazałem na rysunku 6.11.



Rysunek 6.10. Dodanie przycisku DODAJ OBRAZ do paska narzędzi aplikacji Bookmarks



Rysunek 6.11. Bookmarklet załadowany dla witryny amazon.com

Na rysunku 6.11 pokazałem bookmarklet uruchomiony na stronie *https://amazon.com/*.

Jeśli kontener HTML się nie wyświetli, sprawdź log konsoli w powłoce `RunServer`. Jeśli wyświetli się błąd typu MIME, najprawdopodobniej Twoje pliki *map* MIME są nieprawidłowe lub wymagają aktualizacji. Aby zastosować poprawne mapowanie dla plików JavaScript i CSS, należy dodać do pliku *settings.py* następujące wiersze.

```
if DEBUG:
    import mimetypes
    mimetypes.add_type('application/javascript', '.js', True)
    mimetypes.add_type('text/css', '.css', True)
```

Kontener HTML zawiera obrazy, które mogą być dodane do budowanej witryny internetowej. Teraz zaimplementujemy funkcjonalność umożliwiającą użytkownikom kliknięcie wybranego obrazu w celu dodania go do zakładek.

Otwórz do edycji statyczny plik *js/bookmarklet.js* i umieść poniższy fragment kodu na końcu funkcji `bookmarklet()`.

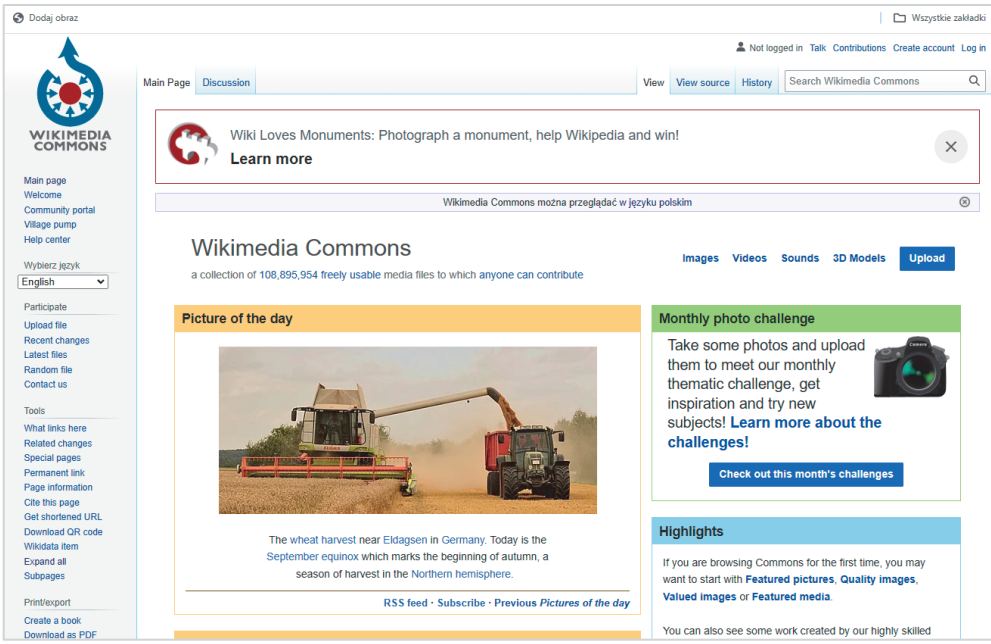
```
function bookmarkletLaunch() {
  bookmarklet = document.getElementById('bookmarklet');
  var imagesFound = bookmarklet.querySelector('.images');

  // Wyczyszczenie znalezionych zdjęć
  imagesFound.innerHTML = '';
  // Wyświetlenie bookmarkletu
  bookmarklet.style.display = 'block';

  // Zdarzenie close
  bookmarklet.querySelector('#close')
             .addEventListener('click', function(){
    bookmarklet.style.display = 'none'
  });

  //  Znajdź w modelu DOM obrazy o minimalnych wymiarach
  images = document.querySelectorAll('img[src$=".jpg"], img[src$=".jpeg"],
  ↪img[src$=".png"]');
  images.forEach(image => {
    if(image.naturalWidth >= minWidth
       && image.naturalHeight >= minHeight)
    {
      var imageFound = document.createElement('img');
      imageFound.src = image.src;
      imagesFound.append(imageFound);
    }
  })

  // Zdarzenie wyboru zdjęcia
  imagesFound.querySelectorAll('img').forEach(image => {
    image.addEventListener('click', function(event){
      imageSelected = event.target;
      bookmarklet.style.display = 'none';
      window.open(siteUrl + 'images/create/?url='
                  + encodeURIComponent(imageSelected.src)
                  + '&title='
                  + encodeURIComponent(document.title),
                  '_blank');
    })
  })
}

// Uruchomienie bookmarkletu
bookmarkletLaunch();
```

Powyższy kod działa w następujący sposób.

1. Do każdego elementu obrazu w kontenerze `imagesFound` dołączane jest zdarzenie `click()`.
2. Gdy użytkownik kliknie dowolny obraz, związany z nim element `image` zostanie zapisany w zmiennej `imageSelected`.

3. Następnie główny kontener bookmarkletu zostanie ukryty przez ustawienie jego właściwości `display` na `none`.
4. Otwiera się nowe okno przeglądarki z adresem URL umożliwiającym oznaczenie nowego zdjęcia w witrynie. Zawartość elementu `<title>` strony jest przekazywana do adresu URL w parametrze `GET` title, a adres URL wybranego zdjęcia jest przekazywany w parametrze `url`.

Teraz otwórz w przeglądarce nowy adres URL, na przykład *https://commons.wikimedia.org/*, jak pokazałem na rysunku 6.12.



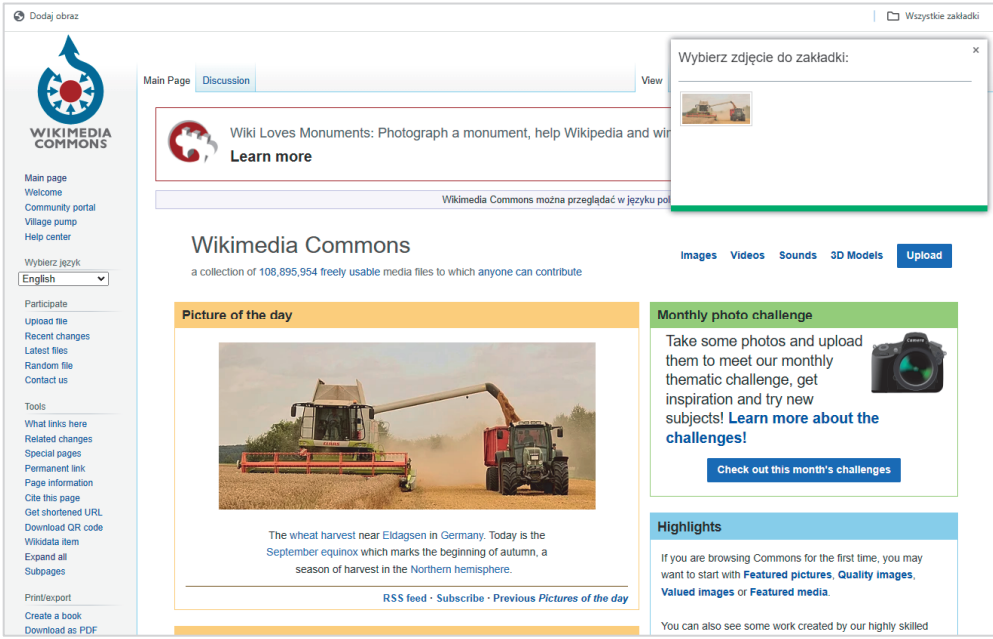
Rysunek 6.12. Witryna Wikimedia Commons

Uwaga

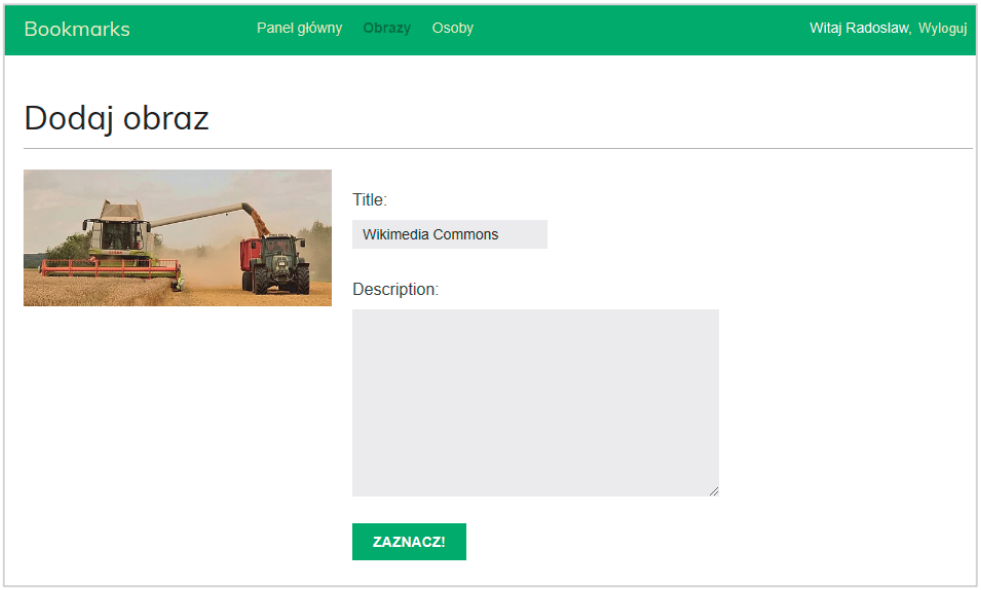
Rysunki od 6.12 do 6.15: *The wheat harvest near Eldagsen in Germany. The combine Claas Lexion 584 06833*, Michael Gäbler (licencja: Creative Commons Attribution-Share Alike 3.0 International: *https://creativecommons.org/licenses/by-sa/3.0/deed.en*)

Aby wyświetlić nakładkę wyboru zdjęcia, kliknij bookmarklet *Dodaj obraz*. Wyświetli się nakładka wyboru zdjęcia (rysunek 6.13).

Po kliknięciu obrazu nastąpi przekierowanie do strony tworzenia obrazu, a tytuł strony internetowej i adres URL wybranego obrazu zostaną przekazane jako parametry `GET`. Wyświetlana strona powinna wyglądać tak, jak pokazałem na rysunku 6.14.



Rysunek 6.13. Bookmarklet załadowany dla zewnętrznej witryny



Rysunek 6.14. Formularz dodawania zakładki do zdjęcia

Gratulacje! W ten sposób przygotowałeś pierwszy bookmarklet JavaScript, który został w pełni zintegrowany z projektem Django. Następnie utworzymy widok szczegółów dla zdjęć i zaimplementujemy kanoniczny adres URL dla obrazów.

Utworzenie szczegółowego widoku obrazu

Przystępujemy do utworzenia prostego widoku zawierającego szczegóły obrazów przekazywanych do budowanej witryny internetowej. Otwórz plik *views.py* aplikacji `images` i umieść w nim poniższy fragment kodu.

```
from django.shortcuts import get_object_or_404
from .models import Image


def image_detail(request, id, slug):
    image = get_object_or_404(Image, id=id, slug=slug)
    return render(
        request,
        'images/image/detail.html',
        {'section': 'images', 'image': image}
    )
```

To jest prosty widok przeznaczony do wyświetlenia obrazu. Przeprowadź edycję pliku *urls.py* aplikacji `images` i dodaj poniższy wzorzec adresu URL oznaczony pogrubioną czcionką.

```
urlpatterns = [
    path('create/', views.image_create, name='create'),
    path(
        'detail/<int:id>/<slug:slug>/',
        views.image_detail,
        name='detail'
    ),
]
```

Następnie przeprowadź edycję pliku *models.py* aplikacji `images` i umieść metodę `get_absolute_url()` w modelu `Image`, tak jak pokazałem poniżej.

```
from django.urls import reverse


class Image(models.Model):
    # ...
    def get_absolute_url(self):
        return reverse('images:detail', args=[self.id, self.slug])
```

Pamiętaj, że najczęściej stosowanym wzorcem dostarczenia kanonicznego adresu URL dla obiektu jest zdefiniowanie w modelu metody `get_absolute_url()`.

Na koniec utwórz szablon w katalogu */templates/images/image* aplikacji `images` i nadaj mu nazwę *detail.html*. Umieść w nim poniższy fragment kodu.

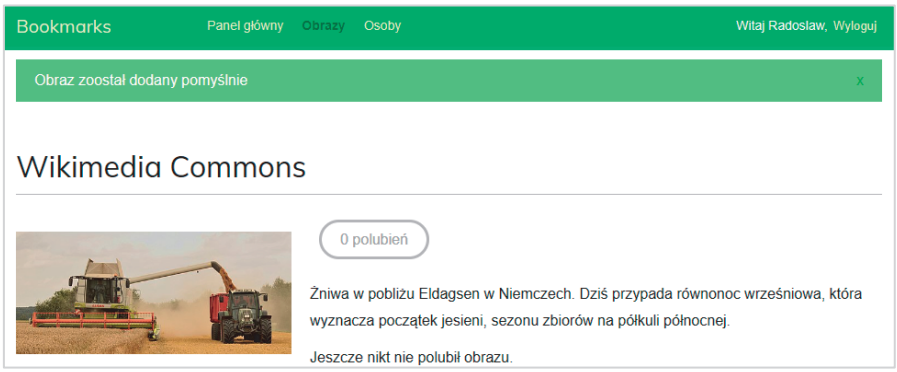
```
{% extends "base.html" %}

{% block title %}{{ image.title }}{% endblock %}

{% block content %}
  <h1>{{ image.title }}</h1>
  <img src="{{ image.image.url }}" class="image-detail">
  {% with total_likes=image.users_like.count %}
    <div class="image-info">
```

```
        <div>
          <span class="count">
            {{ total_likes }} polubień
          </span>
        </div>
        {{ image.description|linebreaks }}
      </div>
      <div class="image-likes">
        {% for user in image.users_like.all %}
          <div>
            {% if user.profile.photo %}
              <img src="{{ user.profile.photo.url }}">
            {% endif %}
            <p>{{ user.first_name }}</p>
          </div>
        {% empty %}
           Jeszcze nikt nie polubił obrazu.
        {% endfor %}
      </div>
    {% endwith %}
  {% endblock %}
```

Ten szablon jest przeznaczony do wyświetlania szczegółowych informacji o dodawanym obrazie. Do przechowywania w nowej zmiennej o nazwie `total_likes` wyniku działania kolekcji `QuerySet` zliczającej wszystkie polubienia danego obrazu przez użytkowników wykorzystaliśmy znacznik `{% with %}`. W ten sposób unikamy dwukrotnego wykonywania tej samej kolekcji `QuerySet`, gdyby liczba polubień była potrzebna w dalszej części kodu. Dołączyliśmy również opis zdjęcia i dodaliśmy pętlę `{% for %}` w celu iteracji po kolekcji `image.users_like.all`, aby wyświetlić wszystkich użytkowników, którym podoba się dany obraz.

Jeśli chcesz powtórzyć zapytanie w szablonie, użyj znacznika szablonu `{% with %}`. Dzięki temu unikniesz wykonywania dodatkowych zapytań do bazy danych.

Teraz otwórz zewnętrzny adres URL w przeglądarce i użyj bookmarkletu, aby dodać nowe zdjęcie do zakładek. Po przekazaniu obrazu zostaniesz przekierowany na stronę wyświetlającą szczegółowe informacje o danym obrazie. Na stronie będzie wyświetlony również komunikat informujący o sukcesie przeprowadzonej operacji (rysunek 6.15).



Rysunek 6.15. Strona szczegółów zaznaczonego obrazu

Świetnie! Zakończyłeś implementację funkcjonalności bookmarkletu. W kolejnym podrozdziale dowiesz się, jak tworzyć miniatury zdjęć.

Utworzenie miniatur za pomocą modułu easy-thumbnails

Na stronie szczegółów wyświetlamy zdjęcie w jego pierwotnej postaci, ale wymiary poszczególnych obrazów mogą się różnić. Ponadto niektóre obrazy mogą być naprawdę ogromne, więc ich wczytanie zajmie dłuższą chwilę. Najlepszym sposobem zoptymalizowania obrazów w spójny sposób jest wygenerowanie miniatur. Miniatura to obraz o małych rozmiarach reprezentujący większy obraz. Miniatury szybciej ładują się w przeglądarce i są świetnym sposobem na ujednolicenie wyświetlania zdjęć o zróżnicowanych rozmiarach. Do generowania miniatur zdjęć dodanych do zakładek przez użytkowników użyjemy aplikacji Django o nazwie `easy-thumbnails`.

Przejdź do powłoki, a następnie za pomocą poniższego polecenia zainstaluj moduł `easy-thumbnails`.

```
python -m pip install easy-thumbnails==2.8.5
```

Przeprowadź edycję pliku *settings.py* projektu `bookmarks` i umieść `easy_thumbnails` na liście `INSTALLED_APPS` tak, jak pokazałem poniżej.

```
INSTALLED_APPS = [
    # ...
    'easy_thumbnails',
]
```

Następnie w celu zsynchronizowania aplikacji z bazą danych wydaj poniższe polecenie.

```
python manage.py migrate
```

Powinieneś otrzymać wynik zawierający m.in. wiersze:

```
Applying easy_thumbnails.0001_initial... OK
Applying easy_thumbnails.0002_thumbnaildimensions... OK
```

Aplikacja `easy-thumbnails` oferuje różne sposoby definiowania miniatur obrazów. Dostarcza znacznik szablonu `{% thumbnail %}` przeznaczony do generowania miniatur w szablonach oraz niestandardową kolumnę `ImageField`, przydatną, jeśli chcesz zdefiniować miniatury w modelu. W omawianym przykładzie wykorzystamy podejście oparte na znaczniku szablonu.

Przeprowadź edycję szablonu *images/image/detail.html* i poniższy wiersz kodu:

```
<img src="{{ image.image.url }}" class="image-detail">
```

zastąp tymi:

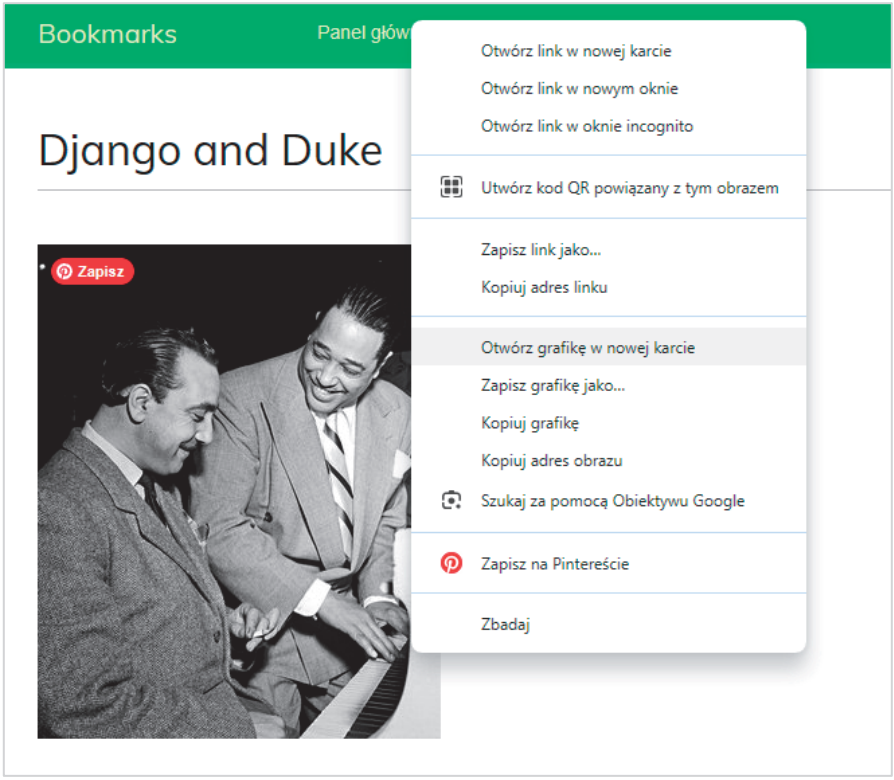
```
{% load thumbnail %}
<a href="{{ image.image.url }}">
  <img src="{% thumbnail image.image 300x0 %}" class="image-detail">
</a>
```

W ten sposób zdefiniowaliśmy miniaturę o stałej szerokości 300 pikseli i elastycznej wysokości (wartość 0) w celu utrzymania proporcji. Miniatura jest przechowywana w tym samym katalogu co plik wejściowy. Lokalizacja jest zdefiniowana przez ustawienie `MEDIA_ROOT` i atrybut `upload_to` pola `image` modelu `Image`. Wygenerowana miniatura będzie później używana w trakcie kolejnych żądań.

W terminalu uruchom serwer programistyczny za pomocą polecenia:

```
python manage.py runserver_plus --cert-file cert.crt
```

Wczytaj w przeglądarce stronę szczegółów obrazu dla istniejącego zdjęcia. Zostanie wygenerowana miniatura, która będzie wyświetlona na stronie. Kliknij zdjęcie prawym przyciskiem myszy i otwórz je w nowej karcie przeglądarki (rysunek 6.16).



Rysunek 6.16. Zdjęcie otwarte w nowej karcie przeglądarki

Sprawdź adres URL wygenerowanego obrazu w swojej przeglądarce. Powinien mieć postać jak na rysunku 6.17.

~~https~~://127.0.0.1:8000/media/images/2024/09/22/django-and-duke_EMgsl3R.jpg.300x0_q85.jpg

Rysunek 6.17. Adres URL wygenerowanego zdjęcia

Za wejściową nazwą pliku następują dodatkowe szczegóły ustawień użytych do utworzenia miniatury. W przypadku zdjęć w formacie JPEG nazwa pliku będzie miała format *nazwapliku.jpg.300x0_q85.jpg*, gdzie `300x0` to parametry rozmiaru używane do wygenerowania miniatury, a 85 to wartość domyślnej jakości *JPEG* używanej przez bibliotekę do generowania miniatury.

Za pomocą parametru `quality` można określić inną jakość. Aby ustawić największą jakość *JPEG*, możesz użyć wartości 100: `{% thumbnail image.image 300x0 quality=100 %}`. Im wyższa jakość, tym większy rozmiar pliku.

Aplikacja `easy-thumbnails` oferuje wiele opcji w zakresie dostosowania miniatur do własnych potrzeb, pozwala m.in. na wybór algorytmów przycinania oraz stosowanie na zdjęciach różnych efektów. Jeżeli napotkasz jakiekolwiek trudności podczas generowania miniatur, zawsze możesz w ustawieniach użyć opcji `THUMBNAIL_DEBUG=True`, aby otrzymać przydatne informacje procesu debugowania. Pełną dokumentację dla aplikacji `easy-thumbnails` znajdziesz na stronie *https://easy-thumbnails.readthedocs.io/*.

Dodawanie asynchronicznych operacji za pomocą JavaScript

Przechodzimy do dodania na stronie wyświetlającej szczegóły obrazu linka, którego kliknięcie pozwoli użytkownikowi na polubienie obrazu. Gdy użytkownik kliknie przycisk *Lubię to*, za pomocą JavaScript wyślemy żądanie HTTP do serwera WWW. Spowoduje to wykonanie działania polubienia bez przeładowywania całej strony. Na potrzeby tej funkcjonalności zaimplementujemy widok, który pozwala użytkownikom na polubienie (anulowanie polubienia) zdjęć.

Fetch API języka JavaScript to wbudowany framework wysyłania asynchronicznych żądań HTTP do serwerów internetowych z poziomu przeglądarek. Dzięki wykorzystaniu Fetch API możesz wysyłać i pobierać dane z serwera WWW bez konieczności odświeżania całej strony. Fetch API to nowoczesny następca wbudowanego w przeglądarkę obiektu `XMLHttpRequest` (XHR), służącego do wykonywania żądań HTTP bez przeładowywania strony. Zestaw technik tworzenia stron internetowych służący do asynchronicznego wysyłania i pobierania danych z serwerów WWW bez przeładowywania strony jest również znany jako *AJAX* (ang. *Asynchronous JavaScript and XML* — dosłownie: asynchroniczny JavaScript i XML). AJAX to myląca nazwa, ponieważ w żądaniach AJAX można wymieniać dane nie tylko w formacie XML, ale także w takich formatach jak JSON, HTML oraz format zwykłego tekstu. W dokumentach, które można znaleźć w internecie, nazwy Fetch API i AJAX są stosowane zamiennie.

Więcej informacji o Fetch API możesz znaleźć pod adresem *https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch*.

Zaczniemy od zaimplementowania widoku do wykonywania działań polubienia i anulowania polubienia, a następnie dodamy do powiązanego szablonu kod JavaScript w celu wykonywania asynchronicznych żądań HTTP.

Przeprowadź edycję pliku *views.py* aplikacji `images` i umieść w nim poniższy fragment kodu.

```
from django.http import JsonResponse
from django.views.decorators.http import require_POST


@login_required
@require_POST
def image_like(request):
    image_id = request.POST.get('id')
    action = request.POST.get('action')
    if image_id and action:
        try:
            image = Image.objects.get(id=image_id)
            if action == 'like':
                image.users_like.add(request.user)
            else:
                image.users_like.remove(request.user)
            return JsonResponse({'status': 'ok'})
        except Image.DoesNotExist:
            pass
    return JsonResponse({'status': 'error'})
```

Dla nowego widoku użyliśmy dwóch dekoratorów. Dekorator `login_required` uniemożliwia niezalogowanym użytkownikom uzyskanie dostępu do tego widoku. Z kolei dekorator `require_POST` zwraca obiekt `HttpResponseNotAllowed` (status `405`), jeśli żądanie HTTP nie zostanie wykonane za pomocą metody `POST`. W ten sposób widok będzie mógł być wyświetlony jedynie z wykorzystaniem żądań typu `POST`.

Django oferuje również dekoratory `require_GET`, pozwalające na wykonywanie jedynie żądań `GET`, i `require_http_methods`, któremu jako argument można przekazać listę dozwolonych metod HTTP.

Omawiany widok oczekuje następujących parametrów `POST`.

- `image_id`: identyfikator obiektu obrazu, na którym użytkownik wykonuje działania.
- `action`: operacja, którą użytkownik chce wykonać. Przyjmujemy założenie, że będzie to ciąg tekstowy o wartości `"like"` lub `"unlike"`.

Aby dodawać lub usuwać obiekty z tej relacji za pomocą metod, odpowiednio, `add()` i `remove()`, korzystamy z oferowanego przez Django menedżera dla kolumny `users_like` (relacja typu „wiele do wielu") w modelu `Image`. Wywołanie metody `add()` i przekazanie obiektu istniejącego już w kolekcji powiązanych obiektów nie spowoduje jego powielenia. Podobnie wywołanie `remove()` i wskazanie nieistniejącego w kolekcji obiektu do usunięcia nie spowoduje żadnego działania. Inną użyteczną metodą menedżera relacji typu „wiele do wielu" jest `clear()`, która powoduje usunięcie wszystkich obiektów z powiązanego zbioru obiektów.

Aby wygenerować odpowiedź widoku, użyjemy dostarczonej przez Django klasy `Json` `↪Response`, która zwraca odpowiedź HTTP zawierającą treść typu `application/json` i konwertuje dany obiekt na format JSON.

Przeprowadź edycję pliku *urls.py* aplikacji `images` i dodaj poniższy wzorzec adresu URL oznaczony pogrubioną czcionką.

```
urlpatterns = [
    path('create/', views.image_create, name='create'),
    path(
        'detail/<int:id>/<slug:slug>/',
        views.image_detail,
        name='detail'
    ),
    path('like/', views.image_like, name='like'),
]
```

Załadowanie JavaScript w modelu DOM

Do szablonu szczegółów obrazu trzeba dodać kod JavaScript. Aby używać w szablonach JavaScript, najpierw dodamy w szablonie *base.html* projektu bazowy wrapper.

Przeprowadź edycję pliku *base.html* aplikacji `account` i poniższy kod umieść przed zamykającym znacznikiem HTML `</body>`.

```
<!DOCTYPE html>
<html>
<head>
  ...
</head>
<body>
   ...
    <script>
     document.addEventListener('DOMContentLoaded', (event) => {
       // Model DOM załadowany
       {% block domready %}
       {% endblock %}
    })
  </script>
</body>
</html>
```

Aby włączyć kod JavaScript, dodaliśmy znacznik `<script>`. Do zdefiniowania funkcji, która zostanie wywołana w momencie zgłoszenia danego zdarzenia, służy metoda `document.addEventListener()`. Przekazujemy nazwę zdarzenia `DOMContentLoaded`, które zajdzie w chwili, gdy początkowy dokument HTML zostanie całkowicie załadowany, a hierarchia DOM zostanie w pełni zbudowana. Dzięki wykorzystaniu tego zdarzenia mamy pewność, że zanim wejdziemy w interakcję z elementami HTML i będziemy wykonywać działania na modelu DOM, będzie on w pełni skonstruowany. Kod w tej funkcji będzie wykonany dopiero po całkowitym przygotowaniu modelu DOM.

Wewnątrz handlera `document-ready` umieściliśmy blok szablonu Django o nazwie `domready`. Tego bloku może użyć dowolny szablon będący rozszerzeniem szablonu *base.html*, aby dołączyć określony kod JavaScript do wykonania w chwili, gdy model DOM będzie gotowy.

Nie pomyl kodu JavaScript ze znacznikami szablonów Django. Język szablonów Django jest przetwarzany po stronie serwera w trakcie generowania ostatecznego dokumentu HTML. Z kolei kod JavaScript jest wykonywany po stronie klienta. W pewnych przypadkach przydaje się możliwość dynamicznego generowania kodu JavaScript za pomocą

Django. Dzięki temu można wykorzystać wyniki zestawów `QuerySet` lub obliczeń po stronie serwera do definiowania zmiennych w JavaScript.

W przykładach przedstawionych w tym rozdziale kod JavaScript jest umieszczany w szablonach Django. Preferowaną metodą dodawania kodu JavaScript do szablonów, zwłaszcza jeśli używasz dużych skryptów, jest ładowanie plików *.js* dostarczanych jako pliki statyczne.

Ataki CSRF w żądaniach HTTP w JavaScript

Temat ataków *CSRF* poruszyłem w rozdziale 2., „Usprawnianie bloga i dodanie funkcjonalności społecznościowych". Gdy uaktywnimy zabezpieczenie przed tego rodzaju atakami, framework Django będzie sprawdzał wszystkie żądania `POST` pod kątem tokenu CSRF. W celu wysłania wraz z formularzem odpowiedniego tokenu można wykorzystać znacznik szablonu `{% csrf_token %}`. Żądania HTTP wykonane w JavaScript również muszą przekazywać token CSRF w każdym żądaniu `POST`.

Dlatego też Django pozwala na ustawienie w żądaniach HTTP własnego nagłówka `X-CSRF ↪Token` i przypisanie mu wartości tokenu CSRF.

Aby uwzględnić token w żądaniach HTTP pochodzących z JavaScript, trzeba pobrać token CSRF z pliku cookie `csrftoken`, który jest ustawiany przez Django, jeśli jest aktywna ochrona CSRF. Do obsługi plików cookie skorzystamy z biblioteki JavaScript `Cookie`. JavaScript `Cookie` to prosty pakiet API JavaScript do obsługi plików cookie. Więcej informacji na temat tego pakietu możesz znaleźć na stronie *https://github.com/js-cookie/js-cookie*.

Przeprowadź edycję szablonu *base.html* aplikacji `account` i dodaj na końcu elementu `<body>` poniższy kod wyróżniony pogrubioną czcionką.

```
<!DOCTYPE html>
<html>
<head>
  ...
</head>
<body>
   ...
  <script src="//cdn.jsdelivr.net/npm/js-
cookie@3.0.5/dist/js.cookie.min.js"></script>
  <script>
    const csrftoken = Cookies.get('csrftoken');
    document.addEventListener('DOMContentLoaded', (event) => {
      // Model DOM załadowany
      {% block domready %}
      {% endblock %}
    })
  </script>
</body>
</html>
```

Zaimplementowaliśmy następującą funkcjonalność.

- Z publicznej sieci **CDN** (ang. *content delivery network* — dosłownie: sieć dostarczania treści) wczytujemy wtyczkę JS Cookie.

- Za pomocą wywołania `Cookies.get()` odczytujemy wartość cookie o nazwie `csrftoken` i zapisujemy ją w stałej JavaScript `csrftoken`.

Token CSRF zostanie umieszczony we wszystkich żądaniach `fetch` JavaScript wykorzystujących niebezpieczne metody HTTP, takie jak `POST` i `PUT`. Później, podczas wysyłania żądań HTTP `POST`, dołączymy do niestandardowego nagłówka HTTP o nazwie `X-CSRFToken` stałą `csrftoken`.

Więcej informacji na temat ochrony przed atakami typu CSRF w żądaniach AJAX znajdziesz pod adresem *https://docs.djangoproject.com/en/5.0/ref/csrf/#ajax*.

W następnym podrozdziale zaimplementujemy kod HTML i JavaScript umożliwiający użytkownikom polubienie (anulowanie polubienia) zdjęć.

Wykonywanie żądań HTTP za pomocą JavaScript

Przeprowadź edycję szablonu *images/image/detail.html* i dodaj poniższy fragment kodu HTML wyróżniony pogrubioną czcionką.

```
{% extends "base.html" %}

{% block title %}{{ image.title }}{% endblock %}

{% block content %}
  <h1>{{ image.title }}</h1>
  {% load thumbnail %}
  <a href="{{ image.image.url }}">
    <img src="{% thumbnail image.image 300x0 %}" class="image-detail">
  </a>
  {% with total_likes=image.users_like.count users_like=image.users_like.all %}
    <div class="image-info">
      <div>
        <span class="count">
        <span class="total">{{ total_likes }}</span>
        polubień
        </span>
        <a href="#" data-id="{{ image.id }}" data-action="{% if request.user in
        ↪users_like %}nie{% endif %}lubię"
    class="like button">
          {% if request.user not in users_like %}
            lubię
          {% else %}
            nie lubię
          {% endif %}
        </a>
      </div>
      {{ image.description|linebreaks }}
    </div>
    <div class="image-likes">
      {% for user in users_like %}
        <div>
          {% if user.profile.photo %}
            <img src="{{ user.profile.photo.url }}">
          {% endif %}
```

```
            <p>{{ user.first_name }}</p>
          </div>
        {% empty %}
          Jeszcze nikt nie polubił obrazu.
        {% endfor %}
      </div>
    {% endwith %}
  {% endblock %}
```

W powyższym kodzie dodaliśmy do znacznika szablonu `{% with %}` kolejną zmienną, aby przechowywać wyniki zapytania `image.users_like.all` i uniknąć wielokrotnego wykonywania zapytania względem bazy danych. Ta zmienna służy do sprawdzenia, czy bieżący użytkownik znajduje się na liście. Służy do tego fragment `{% if request.user in users_like %}`, a następnie `{% if request.user not in users_like %}`. Potem ta sama zmienna jest używana do iterowania po liście użytkowników, którzy lubią ten obraz. Służy do tego instrukcja `{% for user in users_like %}`.

Do tej strony dodaliśmy łączną liczbę użytkowników, którym podoba się obraz, i umieściliśmy link umożliwiający użytkownikowi polubienie (anulowanie polubienia). Do wyświetlenia tekstu *Lubię* lub *Nie lubię* na podstawie bieżącej relacji między użytkownikiem a obrazem używany jest powiązany zestaw obiektów `users_like`. Za jego pomocą możemy sprawdzić, czy w powiązanym zestawie obiektów jest zawarty `request.user`. W elemencie HTML `<a>` umieszczamy poniższe atrybuty.

- `data-id`: atrybut zawierający identyfikator wyświetlonego obrazu.
- `data-action`: atrybut wskazuje działanie do wykonania po kliknięciu łącza przez użytkownika. Może to być `like` lub `unlike`.

Każdy atrybut, którego nazwa zaczyna się od `data-` w dowolnym elemencie HTML, to atrybut danych. Atrybuty danych są wykorzystywane do przechowywania niestandardowych danych aplikacji.

W żądaniu HTTP prześlemy do widoku `image_like` wartości atrybutów `data-id` i `data-action`. Gdy użytkownik kliknie link *like/unlike*, konieczne jest przeprowadzenie po stronie klienta poniższych działań.

1. Wysłanie do widoku `image_like` żądania HTTP `POST` oraz przekazanie do niego identyfikatora obrazu i parametru `action`.
2. Jeżeli wykonanie żądania HTTP zakończy się powodzeniem, uaktualniamy atrybut `data-action` elementu HTML `<a>` przez przypisanie przeciwnego działania (`like`/`unlike`) oraz odpowiednio modyfikujemy wyświetlany tekst.
3. Aktualizacja całkowitej liczby polubień wyświetlanych na stronie.

Na końcu szablonu *images/image/detail.html* umieść poniższy blok `domready`.

```
{% block domready %}
  const url = '{% url "images:like" %}';
  var options = {
    method: 'POST',
    headers: {'X-CSRFToken': csrftoken},
    mode: 'same-origin'
  }
```

```
    document.querySelector('a.like')
            .addEventListener('click', function(e){
     e.preventDefault();
     var likeButton = this;
    });
{% endblock %}
```

Oto sposób działania powyższego kodu:

1. Za pomocą znacznika szablonu `{% url %}` budujemy adres URL `images:like`. Wygenerowany adres URL jest przechowywany w stałej JavaScript `url`.
2. Tworzony jest obiekt `options`, zawierający opcje, które zostaną przekazane do żądania HTTP za pomocą *Fetch API*. Są to:
   - `method`. Wykorzystywana metoda HTTP. W tym przypadku jest to metoda `POST`.
   - `headers`. Dodatkowe nagłówki HTTP do uwzględnienia w żądaniu. Dołączamy nagłówek `X-CSRFToken` razem z wartością stałej `csrftoken`, którą zdefiniowaliśmy w szablonie *base.html*.
   - `mode`. Tryb żądania HTTP. Używamy ustawienia `same-origin`, aby wskazać, że żądanie jest wysyłane do tego samego źródła. Więcej informacji na temat dostępnych trybów żądań można znaleźć pod adresem *https://developer.mozilla.org/en-US/docs/Web/API/Request/mode*.
3. Do wyszukania wszystkich elementów `<a>` dokumentu HTML z klasą `like` wykorzystujemy selektor `a.like` oraz metodę `document.querySelector()`.
4. Dla elementów, na które wskazuje selektor, definiujemy detektor zdarzenia `click`. Funkcja ta będzie wykonywana za każdym razem, gdy użytkownik kliknie link polubienia lub rezygnacji z polubienia danego obrazu.
5. Aby zablokować domyślne zachowanie elementu `<a>` wewnątrz funkcji procedury obsługi, wykorzystujemy wywołanie metody `e.preventDefault()`. Zapobiegnie to domyślnemu zachowaniu elementu `link`: zatrzyma propagację zdarzenia i uniemożliwi przejście pod wskazywany przez niego adres URL.
6. Do przechowywania referencji do tego elementu, na którym zostało wywołane zdarzenie, służy zmienna `likeButton`.

Teraz trzeba wysłać żądanie HTTP za pomocą Fetch API. Przeprowadź edycję bloku `domready` w szablonie *images/image/detail.html* i dodaj poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
{% block domready %}
  const url = '{% url "images:like" %}';
  var options = {
    method: 'POST',
    headers: {'X-CSRFToken': csrftoken},
    mode: 'same-origin'
  }

  document.querySelector('a.like')
          .addEventListener('click', function(e){
    e.preventDefault();
```

```
    var likeButton = this;

    // dodanie treści żądania
    var formData = new FormData();
    formData.append('id', likeButton.dataset.id);
    formData.append('action', likeButton.dataset.action);
    options['body'] = formData;

    // wysłanie żądania HTTP
    fetch(url, options)
    .then(response => response.json())
    .then(data => {
      if (data['status'] === 'ok')
      {
      }
    })
  });
{% endblock %}
```

Oto sposób działania powyższego kodu:

1. Tworzymy obiekt `FormData` w celu skonstruowania zestawu par klucz-wartość reprezentujących pola formularza i ich wartości. Obiekt zapisujemy w zmiennej `formData`.
2. Do obiektu `formData` dodawane są parametry id i `action` oczekiwane przez widok `image_like` frameworka Django. Wartości tych parametrów są pobierane z elementu `likeButton`, który został kliknięty. Do atrybutów `data-id` i `data-action` sięgamy za pomocą wywołań `dataset.id` i dataset.`action`.
3. Do obiektu `options` dodajemy nowy klucz `body`, który będzie używany dla żądania HTTP. Wartością tego klucza jest obiekt `formData`.
4. Korzystamy z Fetch API przez wywołanie funkcji `fetch()`. Zdefiniowana wcześniej zmienna `url` jest przekazywana jako adres URL żądania, a obiekt `options` — jako opcje żądania.
5. Funkcja `fetch()` zwraca obietnicę (ang. *promise*), która jest rozwiązywana z obiektem `Response`, będącym reprezentacją odpowiedzi HTTP. Następnie do zdefiniowania procedury obsługi obietnicy wykorzystujemy metodę `.then()`. Aby wyodrębnić treść JSON, używamy metody `response.json()`. Więcej informacji o obiekcie `Response` możesz znaleźć pod adresem *https://developer.mozilla.org/en-US/docs/Web/API/Response*.
6. Następnie ponownie wykorzystujemy metodę `.then()` w celu zdefiniowania procedury obsługi danych wyodrębnionych do formatu JSON. Wewnątrz tej procedury posługujemy się atrybutem `status` odebranych danych w celu sprawdzenia, czy jego wartość jest poprawna.

Właśnie dodaliśmy funkcjonalność wysyłania żądania HTTP i obsługi odpowiedzi. Po udanym przesłaniu żądania, należy zmienić opis przycisku i powiązane z nim działanie na odwrotne: z *lubię* na *nie lubię* lub z *nie lubię* na *lubię*. W ten sposób użytkownicy mogą cofnąć swoje działanie.

Przeprowadź edycję bloku `domready` w szablonie *images/image/detail.html* i dodaj poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
{% block domready %}
  const url = '{% url "images:like" %}';
  var options = {
    method: 'POST',
    headers: {'X-CSRFToken': csrftoken},
    mode: 'same-origin'
  }

  document.querySelector('a.like')
          .addEventListener('click', function(e){
    e.preventDefault();
    var likeButton = this;

    // Dodanie treści żądania
    var formData = new FormData();
    formData.append('id', likeButton.dataset.id);
    formData.append('action', likeButton.dataset.action);
    options['body'] = formData;

    // Wysłanie żądania HTTP
    fetch(url, options)
    .then(response => response.json())
    .then(data => {
      if (data['status'] === 'ok')
      {
        var previousAction = likeButton.dataset.action;

        // Zmiana tekstu przycisku i operacji do wykonania
        var action = previousAction === 'lubię' ? 'nie lubię' : 'lubię';
        likeButton.dataset.action = action;
        likeButton.innerHTML = action;

        // Aktualizacja liczby polubień
        var likeCount = document.querySelector('span.count .total');
        var totalLikes = parseInt(likeCount.innerHTML);
        likeCount.innerHTML = previousAction === 'like' ? totalLikes + 1 :
        ↪totalLikes - 1;
      }
    })
  });
{% endblock %}
```

Oto sposób działania powyższego kodu:

1. Poprzednie działanie przycisku jest pobierane z atrybutu `data-action` linka i przechowywane w zmiennej `previousAction`.
2. Następuje przełączenie atrybutu `data-action` i tekstu linka. W ten sposób użytkownik może się wycofać z wykonanego działania.
3. Za pomocą selektora `span.count.total` pobieramy z modelu DOM całkowitą liczbę polubień i za pomocą metody `parseInt()` przetwarzamy wartość na liczbę całkowitą. Całkowitą liczbę polubień zwiększamy lub zmniejszamy o jeden w zależności od wykonanego działania (polubienie bądź anulowanie polubienia).

W przeglądarce internetowej otwórz stronę widoku szczegółowego dla przekazanego obrazu. Powinieneś zobaczyć początkową liczbę polubień obrazu oraz przycisk (rysunek 6.18).

Rysunek 6.18. Liczba polubień oraz przycisk LUBIĘ w szablonie szczegółów obrazu

Kliknij przycisk *LUBIĘ*. Zobaczysz, że całkowita liczba wskazująca polubienia obrazu zwiększyła się o jeden, a jednocześnie tekst wyświetlany przez przycisk zmienił się na *NIE LUBIĘ* (rysunek 6.19).

Rysunek 6.19. Liczba polubień oraz przycisk po kliknięciu przycisku LUBIĘ

Po kliknięciu przycisku *NIE LUBIĘ* zostanie wykonane działanie, a tekst wyświetlany przez przycisk zmieni się z powrotem na *LUBIĘ*. Ponadto odpowiednio zmodyfikowana będzie wartość wskazująca całkowitą liczbę polubień obrazu.

Podczas programowania w języku JavaScript, zwłaszcza w trakcie wykonywania żądań AJAX, zaleca się używanie narzędzia do debugowania kodu JavaScript i żądań HTTP. W narzędzia programistyczne do debugowania JavaScriptu wyposażono większość nowoczesnych przeglądarek. Zwykle, aby uzyskać dostęp do narzędzi programistycznych, wystarczy kliknąć prawym przyciskiem myszy gdziekolwiek na stronie, a następnie z menu kontekstowego wybrać opcję *Zbadaj*.

W następnym podrozdziale dowiesz się, jak za pomocą JavaScript i Django skorzystać z asynchronicznych żądań HTTP w celu implementacji stronicowania z nieskończonym przewijaniem.

Dodanie do listy obrazów nieskończonego stronicowania

Konieczne jest zapewnienie użytkownikowi możliwości wyświetlenia wszystkich dodanych przez niego obrazów. Do zbudowania funkcjonalności nieskończonego przewijania użyjemy żądań JavaScript. W tego rodzaju przewijaniu kolejne wyniki są wczytywane, gdy użytkownik będzie zbliżał się do końca strony.

Zaimplementujemy widok listy obrazów przeznaczony do obsługi stronicowania zarówno w standardowych żądaniach przeglądarki internetowej, jak i żądaniach generowanych za pomocą technologii JavaScript. Kiedy użytkownik przejdzie na stronę listy obrazów, na początek zostanie wyświetlona pierwsza strona obrazów. Gdy użytkownik przewinie stronę niemal do końca, wówczas za pomocą technologii JavaScript wczytamy kolejną stronę i dołączymy ją do obecnie wyświetlanej.

Standardowe stronicowanie, jak również stronicowanie w technologii AJAX będzie obsłużone przez ten sam widok. Przeprowadź edycję pliku *views.py* aplikacji `images` i umieść w nim poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
from django.core.paginator import EmptyPage, PageNotAnInteger, Paginator
from django.http import HttpResponse

# …

@login_required
def image_list(request):
    images = Image.objects.all()
    paginator = Paginator(images, 8)
    page = request.GET.get('page')
    images_only = request.GET.get('images_only')
    try:
        images = paginator.page(page)
    except PageNotAnInteger:
        # Jeżeli zmienna page nie jest liczbą całkowitą, wówczas pobierana jest pierwsza
        # strona wyników
        images = paginator.page(1)
    except EmptyPage:
        if images_only:
            # Jeśli żądanie AJAX i strona poza zakresem
            # zwróć pustą stronę
            return HttpResponse('')
        # Jeżeli zmienna page ma wartość większą niż numer ostatniej strony wyników,
        # wtedy pobierana jest ostatnia strona wyników
        images = paginator.page(paginator.num_pages)
    if images_only:
        return render(
            request,
            'images/image/list_images.html',
            {'section': 'images', 'images': images},
        )
    return render(
        request,
        'images/image/list.html',
        {'section': 'images', 'images': images},
    )
```

W przedstawionym widoku tworzymy kolekcję `QuerySet` przeznaczoną do pobrania wszystkich obrazów z bazy danych. Następnie tworzymy obiekt `Paginator` przeznaczony do obsługi stronicowania wyników przez pobieranie ośmiu obrazów na stronę. W celu uzyskania wybranego numeru strony pobierany jest parametr `page` żądania HTTP `GET`. Pobierany jest parametr HTTP `GET` `images_only`, który wskazuje na to, czy ma zostać wyrenderowana cała strona, czy tylko nowe obrazy.

Na żądanie przeglądarki wyrenderujemy całą stronę. Z kolei dla żądań Fetch API będziemy renderować kod HTML tylko z nowymi obrazami, ponieważ będziemy je dołączać do istniejącej strony HTML.

Jeżeli wartość żądanej strony będzie spoza zakresu, nastąpi zgłoszenie wyjątku `EmptyPage`. Jeśli tak się stanie i będzie trzeba wyrenderować tylko zdjęcia, zwrócony zostanie pusty obiekt `HttpResponse`. Umożliwi to zatrzymanie stronicowania AJAX po stronie klienta po dotarciu do ostatniej strony. Wyniki są renderowane przy użyciu dwóch różnych szablonów.

- W przypadku żądań HTTP JavaScript, które zawierają parametr `images_only`, zostanie wyrenderowany szablon *list_images.html*. Ten szablon będzie zawierał jedynie obrazy dla żądanej strony.
- W przypadku żądań przeglądarki zostanie wyrenderowany szablon *list.html*. Ten szablon rozszerza szablon bazowy `base.html`, aby wyświetlić całą stronę, i zawiera szablon *list_ajax.html* w celu dołączenia listy obrazów.

Przeprowadź edycję pliku *urls.py* aplikacji `images` i dodaj poniższy wzorzec adresu URL oznaczony pogrubioną czcionką.

```
urlpatterns = [
    path('create/', views.image_create, name='create'),
    path(
        'detail/<int:id>/<slug:slug>/',
        views.image_detail,
        name='detail',
    ),
    path('like/', views.image_like, name='like'),
    path('', views.image_list, name='list'),
]
```

Na koniec konieczne jest utworzenie wspomnianego wcześniej szablonu. W katalogu *images/image* utwórz nowy plik i nadaj mu nazwę *list_images.html*. Umieść w nim poniższy fragment kodu.

```
{% load thumbnail %}
{% for image in images %}
  <div class="image">
    <a href="{{ image.get_absolute_url }}">
      {% thumbnail image.image 300x300 crop="smart" as im %}
      <a href="{{ image.get_absolute_url }}">
        <img src="{{ im.url }}">
      </a>
    </a>
    <div class="info">
      <a href="{{ image.get_absolute_url }}" class="title">
        {{ image.title }}
      </a>
    </div>
  </div>
{% endfor %}
```

Szablon ten jest przeznaczony do wyświetlenia listy obrazów. Wykorzystamy go do zwrócenia wyników wykonania żądania w technologii AJAX. W tym kodzie iterujemy po obrazach i dla każdego z nich generujemy kwadratowe miniaturki. Rozmiar miniaturek normalizujemy do 300×300 pikseli. Skorzystamy także z opcji przycinania `smart`. Opcja ta powoduje przyrostowe przycinanie obrazu do żądanego rozmiaru poprzez usunięcie z krawędzi fragmentów o najmniejszej entropii.

Utwórz kolejny szablon w tym samym katalogu i nadaj mu nazwę *images/image/list.html*. Umieść w nim poniższy fragment kodu.

```
{% extends "base.html" %}

{% block title %}Dodane obrazy{% endblock %}

{% block content %}
  <h1>Dodane obrazy</h1>
  <div id="image-list">
    {% include "images/image/list_images.html" %}
  </div>
{% endblock %}
```

Szablon listy rozszerza szablon bazowy *base.html*. Aby uniknąć powtarzania kodu, dołączamy w nim szablon *images/image/list_images.html*, przeznaczony do wyświetlania obrazów. W szablonie *images/image/list.html* znajduje się kod JavaScript odpowiedzialny za wczytywanie kolejnych stron, gdy użytkownik przewinie bieżącą niemal do końca.

Przeprowadź edycję szablonu *images/image/list.html* i dodaj poniższy fragment kodu HTML wyróżniony pogrubioną czcionką.

```
{% extends "base.html" %}

{% block title %}Dodane obrazy{% endblock %}

{% block content %}
  <h1>Dodane obrazy</h1>
  <div id="image-list">
    {% include "images/image/list_images.html" %}
  </div>
{% endblock %}

{% block domready %}
  var page = 1;
  var emptyPage = false;
  var blockRequest = false;

  window.addEventListener('scroll', function(e) {
    var margin = document.body.clientHeight - window.innerHeight - 200;
    if(window.pageYOffset > margin && !emptyPage && !blockRequest) {
      blockRequest = true;
      page += 1;
      fetch('?images_only=1&page=' + page)
      .then(response => response.text())
      .then(html => {
        if (html === '') {
          emptyPage = true;
        }
        else {
          var imageList = document.getElementById('image-list');
          imageList.insertAdjacentHTML('beforeEnd', html);
          blockRequest = false;
        }
```

```
        })
      }
    });

    // Zainicjuj zdarzenie przewijania
    const scrollEvent = new Event('scroll');
    window.dispatchEvent(scrollEvent);
{% endblock %}
```

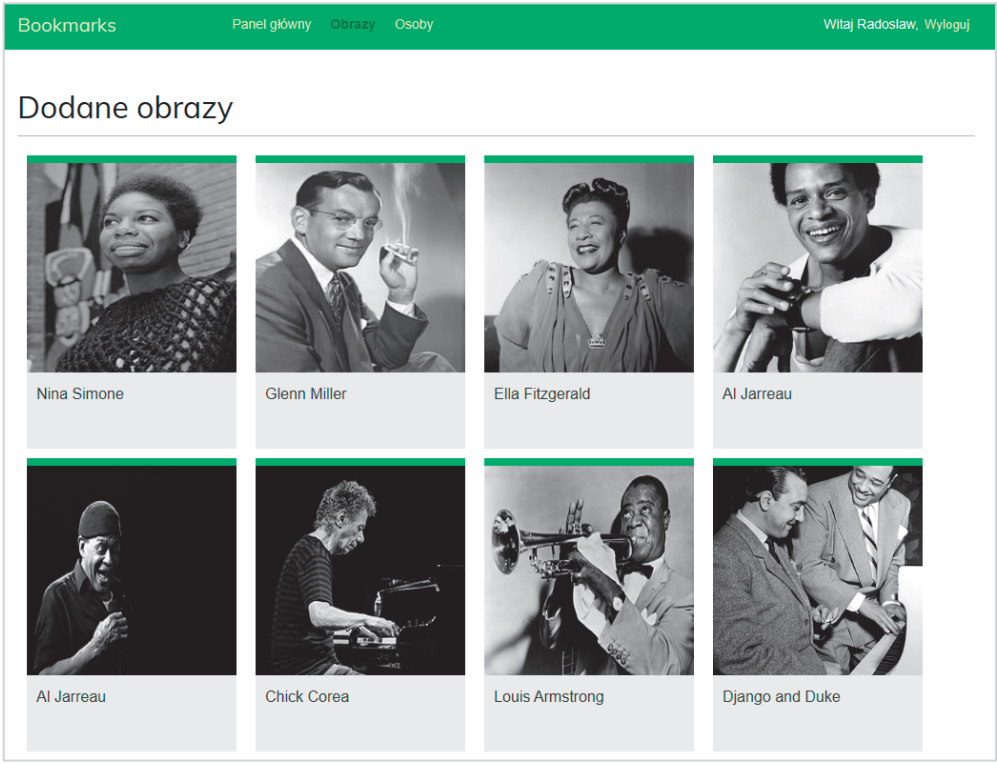
Powyższy kod definiuje funkcjonalność przewijania w nieskończoność. Dołączamy kod JavaScript w bloku `domready` zdefiniowanym w szablonie bazowym *base.html*. Oto sposób działania przedstawionego kodu:

1. Definiujemy następujące zmienne.
   - `page`: zmienna przechowuje numer bieżącej strony.
   - `empty_page`: zmienna pozwala na sprawdzenie, czy użytkownik znajduje się na ostatniej stronie i otrzyma pustą stronę. Gdy tylko otrzymamy pustą stroną, następuje zatrzymanie wykonywania kolejnych żądań HTTP, ponieważ przyjmujemy założenie o braku kolejnych wyników.
   - `block_request`: zmienna chroni przed wykonaniem dodatkowych żądań, gdy aktualnie przetwarzane jest żądanie HTTP.
2. Do przechwycenia zdarzenia `scroll` i zdefiniowania dla niego procedury obsługi wykorzystujemy metodę `window.addEventListener()`.
3. Obliczamy zmienną `margin` przez pobranie różnicy między całkowitą wysokością dokumentu i wysokością okna, ponieważ to będzie wysokość dla pozostałej treści do przewinięcia przez użytkownika. Od wyniku odejmujemy wartość `200`, aby można było wczytać kolejną stronę wyników, gdy użytkownik znajdzie się w odległości 200 pikseli od końca strony.
4. Przed wysłaniem żądania HTTP sprawdzamy, czy:
   - Przesunięcie `window.pageYOffset` jest większe niż obliczony margines.
   - Użytkownik nie dotarł do ostatniej strony wyników (wartość `emptyPage` musi mieć wartość `false`).
   - W danej chwili nie jest obsługiwane inne żądanie HTTP (zmienna `blockRequest` musi mieć wartość `false`).
5. Jeśli poprzednie warunki są spełnione, ustawiamy zmienną `blockRequest` na `true`, aby zdarzenie przewijania nie wyzwalało dodatkowych żądań HTTP, i zwiększamy licznik stron o 1, aby pobrać następną stronę.
6. Do wysłania żądania HTTP `GET` używamy metody `fetch()`. Przekazujemy do niej parametr adresu URL `image_only=1`, aby zamiast pobierania całej strony HTML pobrać tylko kod HTML dla obrazów, oraz `page`, oznaczający numer żądanej strony.
7. Za pomocą metody `response.text()` wyodrębniamy z odpowiedzi HTTP treść elementu `body` i odpowiednio przekształcamy zwrócony kod HTML:

- **Jeśli odpowiedź nie zawiera treści:** dotarliśmy do końca wyników i nie ma kolejnych stron do wczytania. W takim przypadku, aby uniknąć wykonywania kolejnych żądań HTTP, zmiennej `empty_page` przypisujemy wartość `true`.
- **Jeśli odpowiedź zawiera dane:** dołączamy dane do elementu HTML z identyfikatorem `image-list`. Zawartość strony zostaje rozbudowana w pionie, a gdy użytkownik dociera do końca strony, następuje dołączenie kolejnych wyników. Aby usunąć blokadę dla dodatkowych żądań HTTP, należy ustawić zmienną `blockRequest` na `false`.

8. Poniżej procedury nasłuchiwania zdarzeń symulujemy początkowe zdarzenie przewijania podczas ładowania strony. Zdarzenie definiujemy przez utworzenie nowego obiektu `Event`, a następnie, za pomocą metody `window.dispatchEvent()`, zgłaszamy je. W ten sposób zyskujemy pewność, że zdarzenie zostanie wywołane w przypadku, gdy początkowa zawartość mieści się w oknie i nie jest potrzebne przewijanie.

W przeglądarce internetowej przejdź pod adres `https://127.0.0.1:8000/images/`. Powinieneś zobaczyć listę obrazów, które dotąd dodałeś do witryny internetowej. Strona będzie podobna do pokazanej na rysunku 6.20.



Rysunek 6.20. Strona z listą obrazów z funkcjonalnością przewijania w nieskończoność

Uwaga

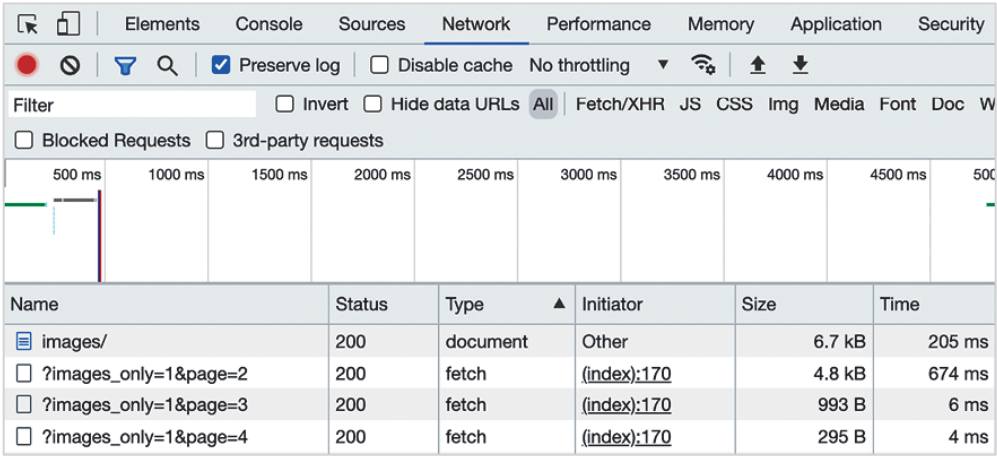
Atrybucje dla zdjęć przedstawionych na rysunku 6.20:

- ataelw *Chick Corea* (licencja: Creative Commons Attribution 2.0 Generic*: https://creativecommons.org/licenses/by/2.0/*).
- Eddi Laumanns, znany również jako RX-Guru *Al Jarreau — Düsseldorf 1981* (licencja: Creative Commons Attribution 3.0 Unported*: https://creativecommons.org/licenses/by/3.0/*).
- *Al Jarreau* Kingkongphoto *i www.celebrity-photos.com* (licencja: Creative Commons Attribution-ShareAlike 2.0 Generic*: https://creativecommons.org/licenses/by-sa/2.0/*).

Przewiń stronę do dołu, aby wczytać kolejne wyniki. Upewnij się, że za pomocą utworzonego wcześniej bookmarkletu dodałeś więcej niż osiem obrazów, ponieważ jest to liczba obrazów jednocześnie wyświetlanych na stronie.

Do śledzenia żądań AJAX możesz użyć narzędzi programistycznych swojej przeglądarki. Zwykle, aby uzyskać dostęp do narzędzi programistycznych, wystarczy kliknąć prawym przyciskiem myszy gdziekolwiek na stronie, a następnie z menu kontekstowego wybrać opcję *Zbadaj* lub *Zbadaj element*. W narzędziach poszukaj panelu żądań sieciowych.

Załaduj ponownie stronę i przewiń do dołu strony, aby załadować nowe obrazy. Wyświetli się żądanie pierwszej strony oraz żądania AJAX dotyczące dodatkowych stron (rysunek 6.21).



Rysunek 6.21. Żądania HTTP zarejestrowane w narzędziach programistycznych przeglądarki

Żądania wyświetlają się także w powłoce, w której korzystasz z Django:

```
[04/Jan/2024 08:14:20] "GET /images/ HTTP/1.1" 200
[04/Jan/2024 08:14:25] "GET /images/?images_only=1&page=2 HTTP/1.1" 200
[04/Jan/2024 08:14:26] "GET /images/?images_only=1&page=3 HTTP/1.1" 200
[04/Jan/2024 08:14:26] "GET /images/?images_only=1&page=4 HTTP/1.1" 200
```

Na koniec przeprowadź edycję szablonu *base.html* aplikacji `account` i zmodyfikuj poniższy adres URL dla elementu `images`. Odpowiedni wiersz został wyróżniony pogrubioną czcionką.

```
<ul class="menu">   ...
    ...
    <li {% if section == "images" %}class="selected"{% endif %}>
        <a href="{% url "images:list" %}">Obrazy</a>
      </li>
    ...
</ul>
```

Teraz do listy obrazów możesz przejść z poziomu menu głównego budowanej witryny internetowej.

Podsumowanie

W tym rozdziale stworzyłeś modele z relacjami „wiele do wielu" i nauczyłeś się dostosowywać zachowanie formularzy. Utworzyłeś bookmarklet JavaScript przeznaczony do pobierania obrazów z innej witryny internetowej i umieszczania ich w budowanej przez nas witrynie. W tym rozdziale omówiliśmy również tworzenie miniatur obrazów za pomocą aplikacji `easy-thumbnails`.

Na koniec zaimplementowałeś widoki AJAX przy użyciu JavaScript Fetch API i dodałeś nieskończone stronicowanie do widoku listy obrazów.

W następnym rozdziale dowiesz się, jak zbudować system obserwacji użytkownika i jego aktywności. Będziesz pracować z relacjami generycznymi, sygnałami i denormalizacją. Dowiesz się również, jak używać z frameworkiem Django bazy danych `Redis` do liczenia widoków obrazów i generowania rankingu.

Dodatkowe zasoby

Oto lista zasobów zawierających dodatkowe informacje związane z tematami omawianymi w tym rozdziale:

- Kod źródłowy przykładów zamieszczonych w tym rozdziale — *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter06*.
- Indeksy w bazach danych — *https://docs.djangoproject.com/en/5.0/ref/models/options/#django.db.models.Options.indexes*.
- Relacje „wiele do wielu" — *https://docs.djangoproject.com/en/5.0/topics/db/examples/many_to_many/*.
- Biblioteka obsługi żądań HTTP dla Pythona `Requests` — *https://docs.djangoproject.com/en/5.0/topics/db/examples/many_to_many/*.
- Rozszerzenie *Save* serwisu Pinterest — *https://help.pinterest.com/en/article/save-pins-with-the-pinterest-browser-button*.
- Statyczna zawartość dla aplikacji `account` — *https://github.com/PacktPublishing/Django-5-by-Example/tree/main/Chapter06/bookmarks/images/static*.

- Selektory CSS — *https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Selectors*.
- Wyszukiwanie elementów modelu DOM za pomocą selektorów CSS — *https://developer.mozilla.org/en-US/docs/Web/API/Document_object_model/Locating_DOM_elements_using_selectors*.
- Bezpłatny automatyczny urząd certyfikacji Let's Encrypt — *https://letsencrypt.org*.
- Aplikacja Django `easy-thumbnails` — *https://easy-thumbnails.readthedocs.io/*.
- Wykorzystanie Fetch API języka JavaScript — *https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch*.
- Biblioteka JavaScript `Cookie` — *https://github.com/js-cookie/js-cookie*.
- Zabezpieczenia CSRF w Django i AJAX — *https://docs.djangoproject.com/en/5.0/ref/csrf/#ajax*.
- Tryb żądania biblioteki Fetch API języka JavaScript — *https://developer.mozilla.org/en-US/docs/Web/API/Request/mode*.
- Tryb odpowiedzi biblioteki Fetch API języka JavaScript — *https://developer.mozilla.org/en-US/docs/Web/API/Response.*

Rozdział 7

Śledzenie działań użytkownika

W poprzednim rozdziale zaimplementowałeś bookmarklet JavaScript pozwalający na udostępnianie na budowanej platformie treści pochodzących z innych witryn internetowych. Za pomocą JavaScript zaimplementowałeś również operacje asynchroniczne oraz mechanizm nieskończonego stronicowania.

W tym rozdziale dowiesz się, jak zbudować systemy obserwacji użytkownika oraz strumienia jego aktywności. Poznasz sposób działania sygnałów w Django i zobaczysz, jak zintegrować z projektem oferującą szybkie operacje wejścia-wyjścia bazę danych Redis, którą będziemy wykorzystywać do przechowywania różnych elementów widoków.

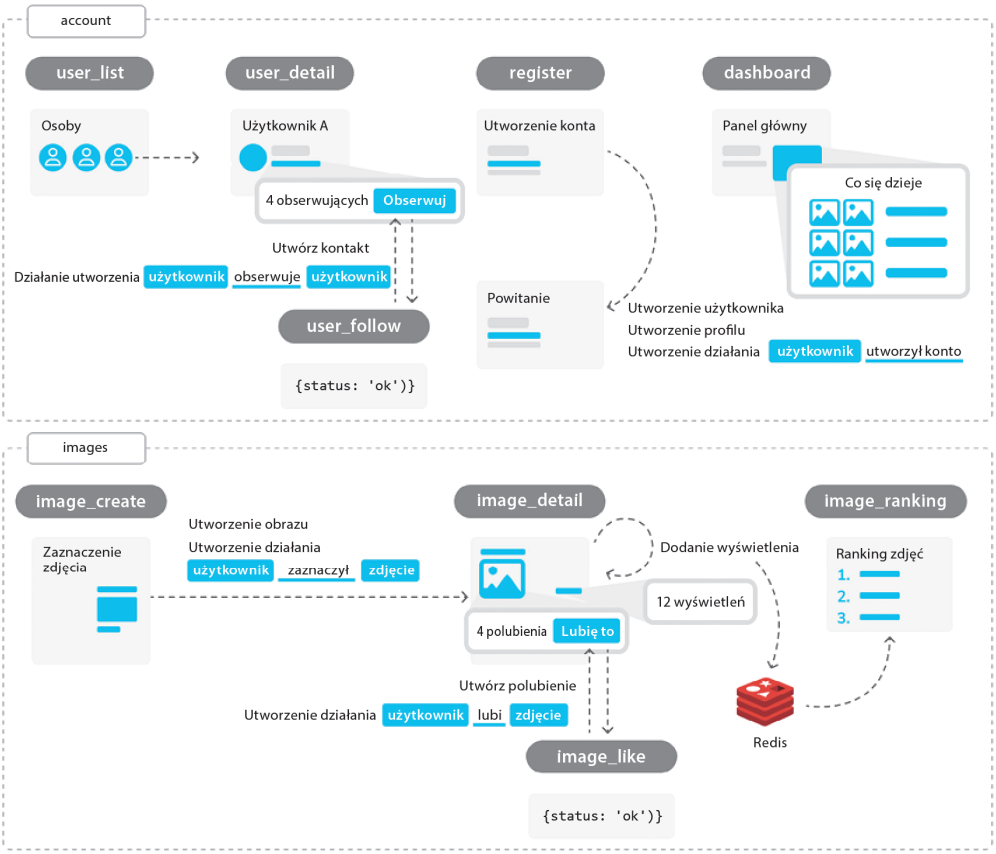
Oto zagadnienia, na których skoncentruję się w tym rozdziale:

- Utworzenie systemu obserwacji
- Utworzenie relacji typu „wiele do wielu" za pomocą modelu pośredniego
- Opracowanie strumienia aktywności aplikacji
- Dodanie do modeli relacji generycznych
- Optymalizacja kolekcji `QuerySet` pod kątem powiązanych obiektów
- Użycie sygnałów dla denormalizowanych zliczeń
- Korzystanie z paska narzędzi *Django Debug Toolbar* w celu uzyskania odpowiednich informacji diagnostycznych
- Zliczanie wyświetleń obrazu za pomocą bazy danych Redis
- Tworzenie rankingu najczęściej oglądanych zdjęć za pomocą bazy danych Redis

Przegląd funkcjonalności

Reprezentację widoków, szablonów i funkcjonalności, które zbudujemy w tym rozdziale, przedstawiono na rysunku 7.1.

W tym rozdziale utworzysz widok `user_list`, pozwalający na wyświetlenie listy wszystkich użytkowników, oraz widok `user_detail`, aby wyświetlić profil pojedynczego użytkownika. Za pomocą JavaScript zaimplementujesz system śledzenia użytkowników. Do przechowywania informacji o obserwowanych działaniach użyjesz widoku `user_follow`. Stworzysz system do przechowywania działań użytkownika oraz zaimplementujesz działania związane z tworzeniem konta, śledzeniem użytkownika, tworzeniem obrazu i polubieniem obrazu. Ten system wykorzystasz do wyświetlania w widoku panelu głównego strumienia aktywności zawierającego najnowsze działania użytkowników. Ponadto zastosujesz



Rysunek 7.1. Schemat funkcjonalności budowanych w rozdziale 7.

bazę danych Redis do przechowywania widoku za każdym razem, gdy ładowany jest widok `image_detail`, i utworzysz widok `image_ranking`, aby wyświetlić ranking najczęściej oglądanych obrazów.

Kod źródłowy przykładów z tego rozdziału można znaleźć pod adresem *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter07*.

Wszystkie pakiety Pythona użyte w tym rozdziale są zawarte w pliku *requirements.txt* dołączonym do plików z kodem źródłowym przykładów z tego rozdziału. Aby zainstalować każdy pakiet Pythona, możesz postępować zgodnie z instrukcjami zamieszczonymi w poniższych podrozdziałach. Możesz także zainstalować wszystkie wymagania jednocześnie za pomocą polecenia `pip install -r requirements.txt`.

Utworzenie systemu obserwacji

W tym podrozdziale zbudujemy w projekcie system obserwacji. Za jego pomocą użytkownicy będą mogli wzajemnie się obserwować i śledzić zdjęcia udostępniane przez innych w budowanej witrynie internetowej. Między użytkownikami zachodzi relacja typu

„wiele do wielu", ponieważ dany użytkownik może obserwować wielu innych, a przy tym sam może być obserwowany przez wielu użytkowników.

Utworzenie relacji typu „wiele do wielu" za pomocą modelu pośredniego

W poprzednich rozdziałach tworzyliśmy relację typu „wiele do wielu" przez dodanie kolumny `ManyToManyField` do jednego z powiązanych modeli, a framework Django budował tabelę bazy danych niezbędną do obsługi tej relacji. Wprawdzie w większości przypadków tego rodzaju rozwiązanie jest odpowiednie, ale czasami trzeba utworzyć dla relacji model pośredni. Tworzenie modelu pośredniego jest konieczne, gdy chcesz przechowywać dodatkowe informacje o relacji, np. datę jej powstania lub pole opisujące typ relacji.

Przygotujemy model pośredni pozwalający na budowanie relacji między użytkownikami. Są co najmniej dwa powody, dla których należy utworzyć model pośredni.

- Wykorzystujesz dostarczany przez Django model `User` i nie chcesz go modyfikować.
- Chcesz przechowywać informacje o dacie i godzinie utworzenia relacji.

Przeprowadź edycję pliku *models.py* aplikacji `account` i umieść w nim poniższy fragment kodu.

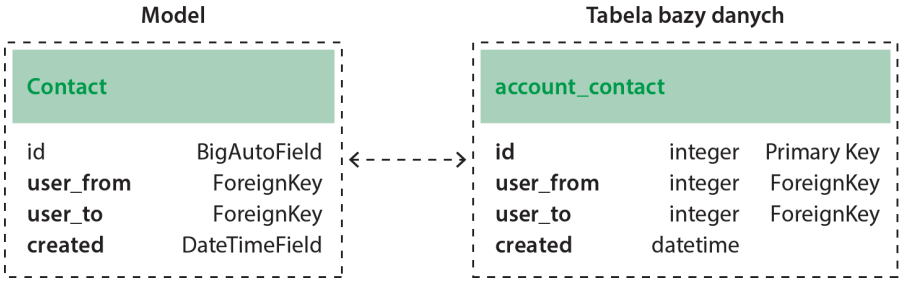
```
class Contact(models.Model):
    user_from = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        related_name='rel_from_set',
        on_delete=models.CASCADE,
    )
    user_to = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        related_name='rel_to_set',
        on_delete=models.CASCADE
    )
    created = models.DateTimeField(auto_now_add=True)

    class Meta:
        indexes = [
            models.Index(fields=['-created']),
        ]
        ordering = ['-created']

    def __str__(self):
        return f'{self.user_from} follows {self.user_to}'
```

Jest to model `Contact`, który wykorzystamy do utworzenia relacji pomiędzy użytkownikami. Wymieniony model zawiera pola:

- `user_from`: klucz obcy (`ForeignKey`) dla użytkownika tworzącego relację.
- `user_to`: klucz obcy (`ForeignKey`) dla obserwowanego użytkownika.
- `created`: pole typu `DateTimeField` z opcją `auto_now_add=True` przeznaczoną do przechowywania daty i godziny powstania relacji.

Na polach typu `ForeignKey` jest automatycznie tworzony indeks bazy danych. W klasie `Meta` modelu zdefiniowaliśmy dla tworzonego pola indeks bazy danych w celu porządkowania w kolejności malejącej. Aby zlecić frameworkowi Django posortowanie wyników domyślnie według pola `created`, skorzystaliśmy z atrybutu `ordering`. W tym konkretnym przypadku określiliśmy kolejność malejącą przez podanie znaku minus przed nazwą pola: `-created`.

Pośredni model `Contact` i odpowiadającą mu tabelę bazy danych przedstawiono na rysunku 7.2:



Rysunek 7.2. Pośredni model Contact i odpowiadająca mu tabela bazy danych

W celu utworzenia za pomocą warstwy ORM relacji dla użytkownika `user1` obserwującego użytkownika `user2` możemy wykorzystać poniższe polecenia.

```
user1 = User.objects.get(id=1)
user2 = User.objects.get(id=2)
Contact.objects.create(user_from=user1, user_to=user2)
```

Powiązane menedżery `rel_from_set` i `rel_to_set` zwrócą kolekcję `QuerySet` dla modelu `Contact`. W celu uzyskania dostępu do drugiej strony relacji z poziomu modelu `User` byłoby dobrze, aby model `User` zawierał pole `ManyToManyField`, np. takie jak pokazałem poniżej.

```
following = models.ManyToManyField(
    'self',
    through=Contact,
    related_name='followers',
    symmetrical=False
)
```

W omawianym przykładzie nakazujemy Django wykorzystanie do obsługi relacji niestandardowego modelu pośredniego. Odbywa się to przez dodanie do pola `ManyToManyField` opcji `through=Contact`. Jest to relacja typu „wiele do wielu" z modelu `User` do samego siebie. Przez wykorzystanie `self` w polu `ManyToManyField` wskazujemy na utworzenie relacji z tym samym modelem.

Uwaga

Gdy trzeba uwzględnić większą liczbę kolumn w relacji typu „wiele do wielu", utwórz niestandardowy model wraz z kolumną typu `ForeignKey` po obu stronach relacji. Możesz zarządzać relacją za pomocą modelu pośredniego albo w jednym z powiązanych modeli możesz dodać pole `ManyToManyField` i przez włączenie go do parametru `through` wskazać Django, że należy użyć modelu pośredniego.

Jeżeli w aplikacji był wykorzystywany model `User`, możesz dodać do modelu zawarte w nim pola. Nie możesz jednak bezpośrednio modyfikować klasy `User`, ponieważ należy ona do aplikacji `django.contrib.auth`. Wykorzystamy nieco inne podejście, polegające na dynamicznym dodaniu pola do modelu `User`.

W pliku *models.py* aplikacji `account` umieść poniższe wiersze kodu wyróżnione pogrubioną czcionką.

```
from django.contrib.auth import get_user_model

# ...

# Dynamiczne dodanie poniższego pola do modelu User
user_model = get_user_model()
user_model.add_to_class(
    'following',
    models.ManyToManyField(
        'self',
        through=Contact,
        related_name='followers',
        symmetrical=False
    )
)
```

W tym kodzie pobraliśmy model użytkownika za pomocą generycznej funkcji `get_user_model()` dostarczanej przez Django. Skorzystaliśmy z metody `add_to_class()` modeli Django w celu dynamicznego dodania pola do modelu `User`.

Pamiętaj, że użycie metody `add_to_class()` nie jest zalecanym sposobem dodawania pól do modeli. Jednak w tym przypadku odnosimy korzyści z wykorzystania tego sposobu, ponieważ dzięki temu unikamy tworzenia niestandardowego modelu użytkownika i zachowujemy wszystkie zalety wbudowanego modelu `User` frameworka Django.

Upraszczamy także sposób pobierania powiązanych ze sobą obiektów za pomocą metod warstwy ORM w Django `user.followers.all()` i `user.following.all()`. Wykorzystujemy model pośredni `Contact` i unikamy skomplikowanych zapytań do bazy danych, których wykonanie mogłoby angażować dodatkowe złączenia w bazie danych. Z taką sytuacją moglibyśmy mieć do czynienia po zdefiniowaniu relacji we własnym modelu `Profile`. Tabela dla tej relacji typu „wiele do wielu" będzie utworzona z wykorzystaniem modelu `Contact`. Dlatego też dodane dynamicznie pole `ManyToManyField` nie powoduje żadnych zmian w bazie danych dla modelu `User` frameworka Django.

Pamiętaj, że w większości przypadków zamiast majstrować przy wbudowanym modelu `User`, lepiej jest dodawać pola do utworzonego wcześniej modelu `Profile`. Byłoby idealnie, gdybyś nie zmieniał istniejącego modelu `User` frameworka Django. Django umożliwia korzystanie z niestandardowych modeli użytkowników. Jeżeli chcesz użyć własnego modelu użytkownika, zapoznaj się z dokumentacją przedstawioną na stronie *https://docs.djangoproject.com/en/5.0/topics/auth/customizing/#specifying-a-custom-user-model*.

Uwaga

W przypadku nowego projektu zdecydowanie zaleca się utworzenie niestandardowego modelu użytkownika, nawet jeśli domyślny model `User` jest wystarczający. Dzięki temu zyskujesz możliwość dostosowania modelu.

Jak możesz zobaczyć, relacja zawiera opcję `symmetrical=False`. Podczas definiowania pola `ManyToManyField` opisującego relację z tym samym modelem Django wymaga utworzenia relacji symetrycznej. W omawianym przykładzie używamy opcji `symmetrical=False` do zdefiniowania relacji niesymetrycznej. Dlatego też jeśli ja obserwuję Ciebie, to nie oznacza, że automatycznie Ty obserwujesz mnie.

Uwaga

Kiedy dla relacji typu „wiele do wielu" używasz modelu pośredniego, część metod powiązanego menedżera, np. `add()`, `create()` i `remove()`, będzie niedostępna. Zamiast tego należy utworzyć lub usunąć egzemplarze modelu pośredniego.

Aby wygenerować początkowe migracje dla aplikacji `account`, uruchom polecenie:

```
python manage.py makemigrations account
```

Powinieneś otrzymać następujące dane wyjściowe.

```
Migrations for 'account':
   account/migrations/0002_contact.py
     - Create model Contact
```

Następnie, w celu zsynchronizowania aplikacji z bazą danych, wydaj poniższe polecenie.

```
python manage.py migrate account
```

Powinieneś otrzymać dane wyjściowe zawierające m.in. poniższy wiersz.

```
Applying account.0002_contact... OK
```

W tym momencie model `Contact` jest zsynchronizowany z bazą danych i mamy możliwość tworzenia relacji między użytkownikami. Jednak budowana witryna internetowa nie oferuje jeszcze możliwości przeglądania użytkowników ani wyświetlania ich profili. Przystępujemy więc do utworzenia widoków listy i szczegółów dla modelu `User`.

Utworzenie widoków listy i szczegółów dla profilu użytkownika

Przeprowadź edycję pliku *views.py* aplikacji `account` i umieść w nim poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
from django.contrib.auth import authenticate, get_user_model, login
from django.shortcuts import get_object_or_404, render
# ...
```

```
User = get_user_model()

@login_required
def user_list(request):
    users = User.objects.filter(is_active=True)
    return render(
        request,
        'account/user/list.html',
        {'section': 'people', 'users': users},
    )

@login_required
def user_detail(request, username):
    user = get_object_or_404(User, username=username, is_active=True)
    return render(
        request,
        'account/user/detail.html',
        {'section': 'people', 'user': user},
    )
```

To są proste widoki listy i szczegółów dla obiektów `User`. Model użytkownika uzyskujemy dynamicznie, za pomocą funkcji `get_user_model()`. Widok `user_list` pobiera wszystkich aktywnych użytkowników. Oferowany przez Django model `User` zawiera opcję `is_active`, przeznaczoną do wskazania, czy dane konto użytkownika jest aktywne. Zapytanie filtrujemy za pomocą opcji `is_active=True`, aby zwrócić jedynie aktywnych użytkowników. Ten widok zwraca wszystkie wyniki, ale możesz go usprawnić przez dodanie stronicowania w dokładnie taki sam sposób jak w widoku `image_list`.

Widok `user_detail` do pobrania aktywnego użytkownika o podanej nazwie używa skrótu w postaci wywołania `get_object_or_404()`. Jeśli nie zostanie znaleziony aktywny użytkownik o podanej nazwie, widok zwraca kod statusu `HTTP 404`.

W pliku *urls.py* aplikacji `account` umieść wzorce adresów URL dla dodanych wcześniej widoków w przedstawiony poniżej sposób. Nowy kod został wyróżniony pogrubioną czcionką.

```
urlpatterns = [
    # ...
    path('', include('django.contrib.auth.urls')),
    path('', views.dashboard, name='dashboard'),
    path('register/', views.register, name='register'),
    path('edit/', views.edit, name='edit'),
    path('users/', views.user_list, name='user_list'),
    path('users/<username>/', views.user_detail, name='user_detail'),
]
```

Wzorzec adresu URL `user_detail` wykorzystamy w celu wygenerowania kanonicznego adresu URL dla użytkownika. W modelu zdefiniowałeś wcześniej metodę `get_absolute_`
`↪url()` odpowiedzialną za zwrócenie kanonicznego adresu URL dla poszczególnych obiektów. Innym sposobem określenia adresu URL dla modelu jest użycie w projekcie ustawienia `ABSOLUTE_URL_OVERRIDES`.

Uwaga

Wykorzystanie we wzorcu adresu URL `user_detail` nazwy użytkownika zamiast identyfikatora użytkownika poprawia zarówno wygodę korzystania z adresu, jak i bezpieczeństwo. Nazwy użytkowników, w przeciwieństwie do sekwencyjnych identyfikatorów, nie są wrażliwe na ataki wyliczeniowe, ponieważ nie ujawniają wykorzystywanej struktury danych. Utrudnia to napastnikom przewidywanie adresów URL i formułowanie wektorów ataku.

Przeprowadź edycję pliku *settings.py* projektu i umieść w nim poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
from django.urls import reverse_lazy

# ...

ABSOLUTE_URL_OVERRIDES = {
    'auth.user': lambda u: reverse_lazy('user_detail', args=[u.username])
}
```

Do każdego modelu wymienionego w opcji `ABSOLUTE_URL_OVERRIDES` Django dynamicznie dołącza metodę `get_absolute_url()`. Wymieniona metoda zwraca adres URL dla modelu wskazanego w opcji. W powyższym fragmencie kodu wygenerowano adres URL `user_detail` dla użytkownika reprezentowanego przez obiekt `auth.user`. Teraz, w celu pobrania odpowiedniego adresu URL, można wykorzystać metodę `get_absolute_url()` egzemplarza modelu `User`.

Przejdź do powłoki Pythona za pomocą polecenia.

```
python manage.py shell
```

Następnie, aby przetestować rozwiązanie, uruchom poniższy kod.

```
>>> from django.contrib.auth.models import User
>>> user = User.objects.latest('id')
>>> str(user.get_absolute_url())
'/account/users/paulina/'
```

Zwrócony adres URL ma oczekiwany format `/account/users/`*`<nazwa_użytkownika>`*`/`.

Dla utworzonych przed chwilą widoków trzeba przygotować szablony. Do katalogu *templates/account/* aplikacji `account` dodaj poniższy podkatalog i pliki.

```
/user/
    detail.html
    list.html
```

Przeprowadź edycję szablonu *account/user/list.html* i umieść w nim poniższy fragment kodu.

```
{% extends "base.html" %}
{% load thumbnail %} {% block title %}Ludzie{% endblock %}

{% block content %}
  <h1>Osoby</h1>
  <div id="people-list">
    {% for user in users %}
```

```
        <div class="user">
          <a href="{{ user.get_absolute_url }}">
            <img src="{% thumbnail user.profile.photo 180x180 %}">
          </a>
          <div class="info">
            <a href="{{ user.get_absolute_url }}" class="title">
              {{ user.get_full_name }}
            </a>
          </div>
        </div>
      {% endfor %}
    </div>
{% endblock %}
```

Szablon ten pozwala na wyświetlenie listy wszystkich aktywnych użytkowników witryny internetowej. Przeprowadzamy iterację przez danych użytkowników i wykorzystujemy znacznik szablonu `{% thumbnail %}` aplikacji `easy-thumbnail` do wygenerowania miniatur obrazu profilu użytkownika.

Zwróć uwagę, że użytkownicy muszą mieć zdjęcie profilowe. Aby użyć domyślnego obrazu dla użytkowników, którzy nie mają zdjęcia profilowego, możesz dodać instrukcję `if/else` sprawdzającą, czy użytkownik ma zdjęcie profilowe, na przykład `{% if user ↪.profile.photo %} {# miniaturka zdjęcia #} {% else %} {# obraz domyślny #} {% endif %}`.

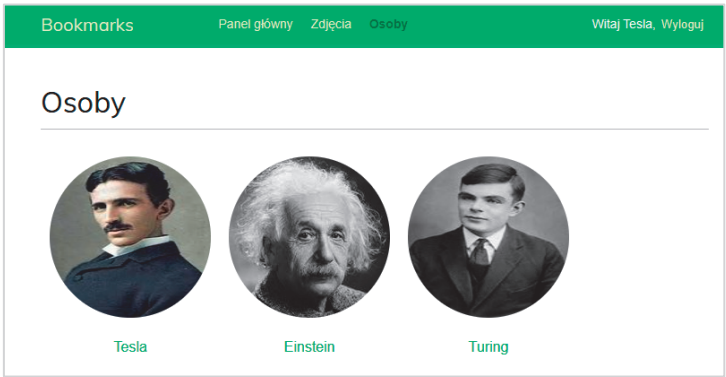
Otwórz szablon *base.html* projektu i umieść adres URL widoku `user_list` w atrybucie `href` przedstawionego poniżej elementu menu. Nowy kod został wyróżniony pogrubioną czcionką.

```
<ul class="menu">
   ...
  <li {% if section == "people" %}class="selected"{% endif %}>
    <a href="{% url "user_list" %}">Osoby</a>
  </li>
</ul>
```

Uruchom serwer programistyczny za pomocą poniższego polecenia.

```
python manage.py runserver
```

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/account/users/`. Powinieneś zobaczyć listę użytkowników podobną do pokazanej na rysunku 7.3.



Rysunek 7.3. Strona z listą użytkowników zawierająca miniaturki zdjęć powiązanych z profilami

W przypadku trudności z generowaniem miniatur możesz dodać do pliku *settings.py* ustawienie `THUMBNAIL_DEBUG = True`. Wtedy uzyskasz w powłoce informacje diagnostyczne.

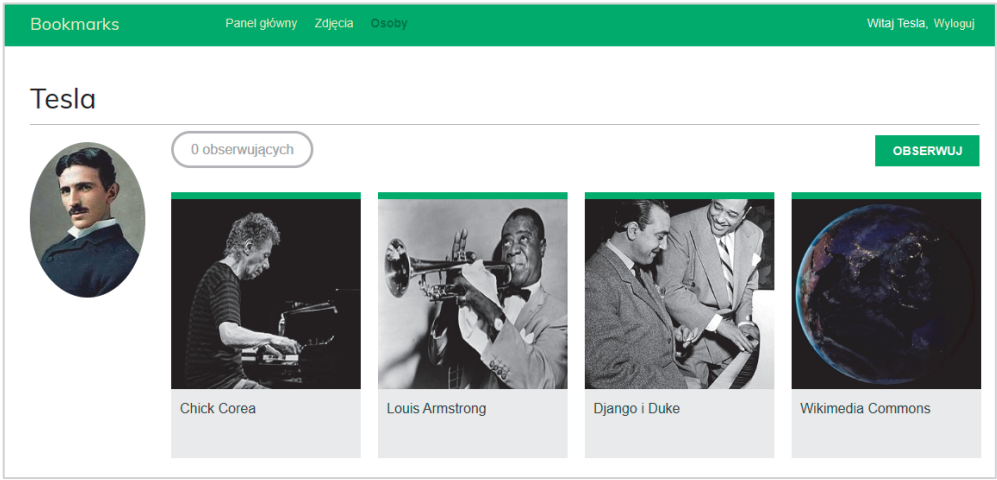
Przeprowadź edycję szablonu *account/user/detail.html* aplikacji `account` i umieść w nim poniższy fragment kodu.

```
{% extends "base.html" %}
{% load thumbnail %}

{% block title %}{{ user.get_full_name }}{% endblock %}
{% block content %}
  <h1>{{ user.get_full_name }}</h1>
  <div class="profile-info">
    <img src="{% thumbnail user.profile.photo 180x180 %}" class="user-detail">
  </div>
  {% with total_followers=user.followers.count %}
    <span class="count">
      <span class="total">{{ total_followers }}</span>
      obserwujących
    </span>
    <a href="#" data-id="{{ user.id }}" data-action="{% if request.user in
    ↪user.followers.all %}nie{% endif %}obserwuj" class="follow button">
      {% if request.user not in user.followers.all %}
        Obserwuj
      {% else %}
        Nie obserwuj
      {% endif %}
    </a>
    <div id="image-list" class="image-container">
      {% include "images/image/list_images.html" with images=user.images_created.all %}
     </div>
  {% endwith %}
{% endblock %}
```

Zadbaj o to, aby tag szablonu nie był podzielony na wiele wierszy. Django nie obsługuje tagów złożonych z wielu wierszy.

W szablonie widoku szczegółowego wyświetlamy profil użytkownika oraz wykorzystujemy znacznik szablonu `{% thumbnail %}` do wyświetlenia obrazu profilu. Ponadto na stronie znajdują się informacje o liczbie użytkowników obserwujących dany profil oraz link pozwalający obserwować tego użytkownika lub zrezygnować z obserwowania go. Kliknięcie tego linka spowoduje przełączenie pomiędzy obserwowaniem a zaprzestaniem obserwowania określonego użytkownika. Atrybuty `data-id` i `data-action` elementu HTML `<a>` zawierają identyfikator użytkownika i początkowe działanie do wykonania po kliknięciu elementu linka — *Obserwuj* lub *Nie obserwuj*. Początkowe działanie (obserwuj lub przestań obserwować) zależy od tego, czy użytkownik żądający wyświetlenia strony już jest obserwatorem użytkownika. Obrazy dodane przez użytkownika są wyświetlane po włączeniu szablonu *images/image/list_images.html*.

Ponownie przejdź do przeglądarki internetowej i wybierz użytkownika, który dodał kilka obrazów. Powinieneś zobaczyć stronę szczegółową jego profilu, podobną do pokazanej na rysunku 7.4.



Rysunek 7.4. Strona szczegółowych danych użytkownika

Uwaga

Zdjęcie Chicka Corei wykonane przez ataelw (licencja: Creative Commons Attribution 2.0 Generic: *https://creativecommons.org/licenses/by/2.0/*).

Dodanie działań obserwowania i rezygnacji z obserwowania użytkownika za pomocą JavaScript

Dodajmy funkcjonalność śledzenia użytkownika (zaprzestania jego obserwowania). Stworzymy nowy widok do obserwowania (zaprzestania obserwowania) użytkowników i zaimplementujemy za pomocą JavaScript asynchroniczne żądanie HTTP dla operacji `follow` — w celu rozpoczęcia obserwacji (`unfollow` — w celu jej zaprzestania).

Przeprowadź edycję pliku *views.py* aplikacji `account` i umieść w nim poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
from django.http import HttpResponse, JsonResponse
from django.views.decorators.http import require_POST
from .models import Contact, Profile

# ...

@require_POST
@login_required
def user_follow(request):
    user_id = request.POST.get('id')
    action = request.POST.get('action')
    if user_id and action:
        try:
            user = User.objects.get(id=user_id)
            if action == 'obserwuj':
```

```
                Contact.objects.get_or_create(
                    user_from=request.user,
                    user_to=user
                )
            else:
                Contact.objects.filter(
                    user_from=request.user,
                    user_to=user
                ).delete()
            return JsonResponse({'status':'ok'})
        except User.DoesNotExist:
            return JsonResponse({'status':'error'})
    return JsonResponse({'status':'error'})
```

Widok `user_follow` jest bardzo podobny do widoku `image_like` utworzonego w rozdziale 6., „Udostępnianie treści w witrynie internetowej". Ponieważ do obsługi relacji typu „wiele do wielu" między użytkownikami stosujemy własny model pośredni, domyślne metody `add()` i `remove()` automatycznego menedżera pola typu `ManyToManyField` są niedostępne. Do tworzenia lub usuwania relacji użytkownika wykorzystujemy model pośredni `Contact`.

Przeprowadź edycję pliku *urls.py* aplikacji `account` i dodaj poniższy wzorzec adresu URL oznaczony pogrubioną czcionką.

```
urlpatterns = [
    path('', include('django.contrib.auth.urls')),
    path('', views.dashboard, name='dashboard'),
    path('register/', views.register, name='register'),
    path('edit/', views.edit, name='edit'),
    path('users/', views.user_list, name='user_list'),
    path('users/follow/', views.user_follow, name='user_follow'),
    path('users/<username>/', views.user_detail, name='user_detail'), ]
```

Upewnij się, że umieściłeś powyższy wzorzec przed wzorcem adresu URL widoku `user_detail`. W przeciwnym razie wszystkie żądania do */users/follow/* zostaną dopasowane przez wyrażenie regularne wzorca adresu URL widoku `user_detail`, który w takim przypadku będzie użyty zamiast oczekiwanego `user_follow`. Pamiętaj, że w każdym żądaniu HTTP Django próbuje dopasować żądany adres URL do każdego wzorca w kolejności ich występowania w pliku *urls.py*.

Przeprowadź edycję szablonu *user/detail.html* aplikacji `account` i umieść w nim poniższy fragment kodu.

```
{% block domready %}
   const url = '{% url "user_follow" %}';
   var options = {
    method: 'POST',
     headers: {'X-CSRFToken': csrftoken},
     mode: 'same-origin'
   }
   document.querySelector('a.follow')
           .addEventListener('click', function(e){
    e.preventDefault();
     var followButton = this;
```

```
        // dodanie treści żądania
         var formData = new FormData();
         formData.append('id', followButton.dataset.id);
         formData.append('action', followButton.dataset.action);
         options['body'] = formData;

        // wysłanie żądania HTTP
         fetch(url, options)
         .then(response => response.json())
         .then(data => {
           if (data['status'] === 'ok')
           {
             var previousAction = followButton.dataset.action;

            // zmiana tekstu przycisku i działania data-action
             var action = previousAction === 'obserwuj' ? 'nie obserwuj' : 'obserwuj';
             followButton.dataset.action = action;
             followButton.innerHTML = action;

            // aktualizacja liczby obserwujących
             var followerCount = document.querySelector('span.count .total');
             var totalFollowers = parseInt(followerCount.innerHTML);
             followerCount.innerHTML = previousAction === 'obserwuj' ? totalFollowers +
             ↪1 : totalFollowers - 1;
           }
         })
       });
{% endblock %}
```

To jest kod JavaScript przeznaczony do wykonania asynchronicznego żądania HTTP pozwalającego na rozpoczęcie obserwacji lub rezygnację z obserwacji danego profilu użytkownika oraz na zmianę wyświetlanego w tym celu linka.

Do wykonania żądania AJAX wykorzystujemy Fetch API, a wartości atrybutu `data-action` i `text` elementu HTML są ustawiane na podstawie ich poprzedniej wartości. Po wykonaniu działania następuje aktualizacja wyświetlanej na stronie łącznej liczby obserwujących.

Aby wypróbować zaimplementowaną przed chwilą funkcjonalność, przejdź na stronę szczegółów istniejącego użytkownika, a następnie kliknij link *OBSERWUJ*. Zauważysz, że zwiększyła się liczba użytkowników obserwujących (rysunek 7.5).

Rysunek 7.5. Liczba obserwujących oraz przycisk OBSERWUJ (NIE OBSERWUJ)

System obserwacji jest teraz gotowy, a użytkownicy mogą się nawzajem obserwować. W następnym podrozdziale zbudujemy strumień aktywności przez utworzenie odpowiednich treści dla każdego użytkownika na podstawie obserwowanych przez niego osób.

Opracowanie strumienia aktywności aplikacji

Wiele społecznościowych witryn internetowych pokazuje strumień aktywności użytkowników, którzy w ten sposób mogą śledzić poczynania innych użytkowników na tej platformie. Strumień aktywności to po prostu lista ostatnich działań podjętych przez danego użytkownika lub grupę użytkowników. Przykładowo dziennik aktywności w serwisie Facebook to rodzaj strumienia aktywności. Przykładami działań mogą być sytuacje, w których *użytkownik X polubił zdjęcie Y* lub *użytkownik X teraz obserwuje użytkownika Y*.

Zaimplementujemy strumień aktywności, aby każdy użytkownik mógł śledzić ostatnie poczynania obserwowanych przez niego użytkowników. W tym celu potrzebny jest model pozwalający na zapisanie akcji przeprowadzanych przez użytkowników w witrynie internetowej oraz prosty sposób na dodanie tych akcji do kanału wiadomości.

Za pomocą poniższego polecenia utwórz w projekcie nową aplikację i nazwij ją `actions`.

```
python manage.py startapp actions
```

Umieść `'actions'` na liście `INSTALLED_APPS` w pliku *settings.py* projektu, aby poinformować Django, że nowa aplikacja jest aktywna. Nowy kod został wyróżniony pogrubioną czcionką.

```
INSTALLED_APPS = [
     # …
    'actions.apps.ActionsConfig',
]
```

Przeprowadź edycję pliku *models.py* aplikacji `actions` i umieść w nim poniższy fragment kodu.

```
from django.conf import settings
from django.db import models


class Action(models.Model):
    user = models.ForeignKey(
         settings.AUTH_USER_MODEL,
        related_name='actions',
         on_delete=models.CASCADE
    )
    verb = models.CharField(max_length=255)
    created = models.DateTimeField(auto_now_add=True)
    class Meta:
        indexes = [
            models.Index(fields=['-created']),
        ]
        ordering = ['-created']
```

Powyższy kod to implementacja modelu `Action`, który wykorzystamy do przechowywania działań podejmowanych przez użytkownika. Pola tego modelu przedstawiają się następująco.

- `user`: użytkownik, który przeprowadził dane działanie. Jest to kolumna typu `ForeignKey` dla modelu `User` frameworka Django.
- `verb`: czasownik określający działanie podjęte przez użytkownika.

- `created`: data i godzina podjęcia działania. W celu automatycznego przypisania bieżącej daty i godziny podczas pierwszego zapisywania obiektu w bazie danych użyliśmy opcji `auto_now_add=True`.

W klasie `Meta` modelu zdefiniowaliśmy dla tworzonego pola indeks bazy danych w celu porządkowania w kolejności malejącej. Aby zlecić frameworkowi Django posortowanie wyników domyślnie według pola `created`, skorzystaliśmy z atrybutu `ordering`.

Za pomocą tego podstawowego modelu możemy przechowywać jedynie działania typu „Użytkownik X zrobił to i to". Aby zapisywać działania obejmujące obiekt `target`, na przykład „Użytkownik X polubił obraz Y" lub „Użytkownik X teraz obserwuje użytkownika Y", potrzebujemy dodatkowego pola typu `ForeignKey`. Jak wiesz, zwykłe pole typu `ForeignKey` może wskazywać tylko jeden model. Trzeba więc znaleźć sposób, aby obiekt `target` działania był egzemplarzem dowolnego istniejącego modelu. Do tego celu można wykorzystać oferowany przez Django framework `contenttypes`.

Użycie frameworka contenttypes

Django oferuje framework `contentypes` umieszczony w aplikacji `django.contrib.contenttypes`. Aplikacja ta może śledzić wszystkie modele zainstalowane w projekcie i zapewnia generyczny interfejs przeznaczony do pracy z modelami.

Aplikacja `django.contrib.contenttypes` jest domyślnie umieszczana na liście `INSTALLED_APPS` podczas tworzenia nowego projektu za pomocą polecenia `startproject`. Wymieniona aplikacja jest również używana przez inne pakiety typu `contrib`, np. framework uwierzytelniania i aplikację administracyjną.

Aplikacja `contenttypes` zawiera model `ContentType`. Egzemplarze tego modelu reprezentują rzeczywiste modele aplikacji, a nowe egzemplarze `ContentType` są tworzone automatycznie podczas instalowania w aplikacji nowych modeli. Poniżej wymieniłem pola modelu `ContentType`.

- `app_label`: nazwa aplikacji, do której należy model. Nazwa jest pobierana automatycznie z atrybutu `app_label` opcji modelu `Meta`. Przykładowo model `Image` należy do aplikacji `images`.
- `model`: nazwa klasy modelu.
- `name`: czytelna dla człowieka nazwa modelu. Nazwa jest pobierana automatycznie z atrybutu `verbose_name` opcji modelu `Meta`.

Zobaczmy teraz, jak może wyglądać praca z obiektami `ContentType`. Przejdź do powłoki Pythona za pomocą polecenia:

```
python manage.py shell
```

Teraz możesz pobierać obiekty `ContentType` odpowiadające określonemu modelowi przez wykonywanie zapytań wraz z atrybutami `app_label` i `model`. Oto przykład:

```
>>> from django.contrib.contenttypes.models import ContentType
>>> image_type = ContentType.objects.get(app_label='images', model='image')
>>> image_type
<ContentType: images | image>
```

Możesz również pobrać klasę modelu z obiektu `ContentType` za pomocą wywołania metody `model_class()`.

```
>>> image_type.model_class()
<class 'images.models.Image'>
```

Dość powszechne jest również pobieranie obiektu `ContentType` dla określonej klasy modelu za pomocą poniższych poleceń.

```
>>> from images.models import Image
>>> ContentType.objects.get_for_model(Image)
<ContentType: images | image>
```

Powyżej przedstawiłem jedynie kilka przykładów użycia frameworka `contenttypes`. Django oferuje znacznie więcej możliwości w zakresie pracy z obiektami `ContentType`. Więcej informacji na temat frameworka `contenttypes` znajdziesz w oficjalnej dokumentacji dostępnej na stronie *https://docs.djangoproject.com/en/5.0/ref/contrib/contenttypes/*.

Dodanie do modelu relacji generycznych

W relacjach generycznych obiekty `ContentType` odgrywają rolę wskazującego model użyty dla danej relacji. Do zdefiniowania relacji generycznej w modelu konieczne jest wykorzystanie trzech kolumn. Oto one:

- Pole typu `ForeignKey` prowadzące do modelu `ContentType`. W ten sposób poznajemy model dla relacji.
- Pole przeznaczone do przechowywania klucza głównego powiązanego obiektu. Aby zachować dopasowanie do automatycznie definiowanych przez Django pól klucza głównego, to z reguły będzie pole `PositiveIntegerField`.
- Pole przeznaczone do definiowania relacji generycznej używającej dwóch poprzednich kolumn i zarządzania nią. Framework `contenttypes` oferuje do tego celu kolumnę `GenericForeignKey`.

Przeprowadź edycję pliku *models.py* aplikacji `actions` i umieść w nim poniższe wiersze kodu oznaczone pogrubioną czcionką.

```
from django.conf import settings
from django.contrib.contenttypes.fields import GenericForeignKey
from django.contrib.contenttypes.models import ContentType
from django.db import models


class Action(models.Model):
    user = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        related_name='actions',
        on_delete=models.CASCADE
    )
    verb = models.CharField(max_length=255)
    created = models.DateTimeField(auto_now_add=True)
    target_ct = models.ForeignKey(
        ContentType,
```

```
        blank=True,
        null=True,
        related_name='target_obj',
        on_delete=models.CASCADE
    )
    target_id = models.PositiveIntegerField(null=True, blank=True)
    target = GenericForeignKey('target_ct', 'target_id')

    class Meta:
        indexes = [
            models.Index(fields=['-created']),
            models.Index(fields=['target_ct', 'target_id']),
         ]
         ordering = ['-created']
```

Do modelu `Action` dodaliśmy pola:

- `target_ct`: pole typu `ForeignKey` prowadzące do modelu `ContentType`.
- `target_id`: pole typu `PositiveIntegerField` przeznaczone do przechowywania klucza głównego powiązanego obiektu.
- `target`: pole typu `GenericForeignKey` dla powiązanego obiektu na podstawie połączenia dwóch powyższych pól.

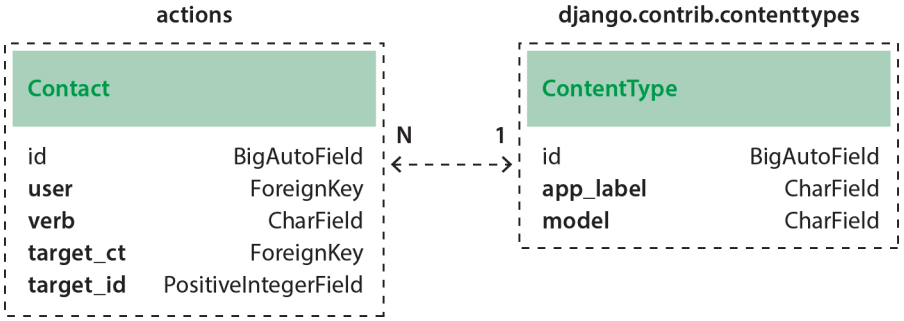
Dodaliśmy również indeks dla wielu pól, uwzględniający pola `target_ct` i `target_id`.

Django nie tworzy w bazie danych pól `GenericForeignKey`. Jedyne pola mapowane na pola w bazie danych to `target_ct` i `target_id`. Oba pola mają zdefiniowane atrybuty `blank=True` i `null=True`, więc podczas zapisywania obiektów `Action` nie jest wymagany obiekt `target`.

Uwaga

Dzięki zastosowaniu relacji generycznych zamiast kluczy obcych aplikacje stają się bardziej elastyczne. Relacje generyczne pozwalają kojarzyć modele w sposób niewyłączny, co umożliwia powiązanie pojedynczego modelu z wieloma innymi modelami.

Model `Action` w relacji z modelem `ContentType` pakietu `contenttypes` frameworka Django `contrib` pokazano na rysunku 7.6.



Rysunek 7.6. Model Action w relacji z modelem ContentType

W celu utworzenia początkowej migracji aplikacji wydaj poniższe polecenie.

```
python manage.py makemigrations actions
```

Powinieneś otrzymać następujące dane wyjściowe.

```
Migrations for 'actions':
   actions/migrations/0001_initial.py
     - Create model Action
     - Create index actions_act_created_64f10d_idx on field(s) -created of model
     ↪action
     - Create index actions_act_target__f20513_idx on field(s) target_ct, target_id
     ↪of model action
```

Następnie w celu przeprowadzenia synchronizacji aplikacji z bazą danych wydaj poniższe polecenie.

```
python manage.py migrate
```

Wygenerowane dane wyjściowe powinny wskazywać na zastosowanie nowej migracji.

```
Applying actions.0001_initial... OK
```

Kolejnym krokiem jest dodanie modelu `Action` do witryny administracyjnej. Przeprowadź edycję pliku *admin.py* aplikacji `actions` i umieść w nim poniższy fragment kodu.

```
from django.contrib import admin
from .models import Action

@admin.register(Action)
class ActionAdmin(admin.ModelAdmin):
    list_display = ['user', 'verb', 'target', 'created']
    list_filter = ['created']
    search_fields = ['verb']
```

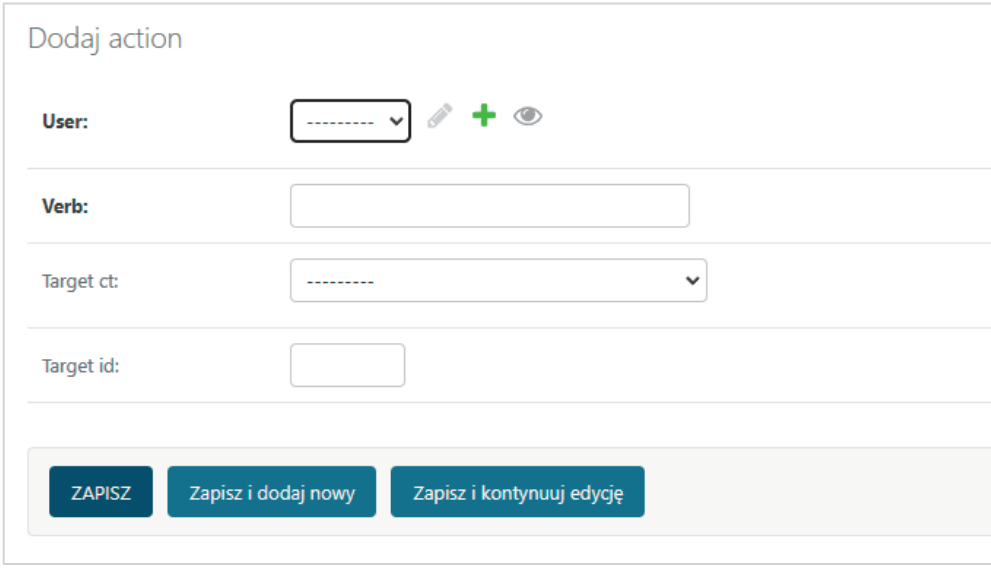
W ten sposób zarejestrowałeś model `Action` w witrynie administracyjnej Django.

Uruchom serwer programistyczny za pomocą poniższego polecenia.

```
python manage.py runserver
```

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/admin/actions/↪action/add/`. Powinieneś zobaczyć stronę przeznaczoną do tworzenia nowych obiektów Action (rysunek 7.7).

Jak widać na powyższym zrzucie ekranu, wyświetlane są jedynie kolumny `target_ct` i `target_id`, które są mapowane na rzeczywiste pola w bazie danych. Pole `GenericForeign↪Key` nie wyświetla się w formularzu. Pole `target_ct` pozwala na wybór dowolnego modelu zarejestrowanego w projekcie Django. Za pomocą atrybutu `limit_choices_to` pola `target_ct` można ograniczyć wybór typów zawartości do pewnego zbioru modeli. Atrybut `limit_choices_to` pozwala na ograniczenie zawartości pól typu `ForeignKey` do określonego zbioru wartości.



Rysunek 7.7. Strona dodawania działań w witrynie administracyjnej Django

Utwórz nowy plik w katalogu aplikacji `actions` i nadaj mu nazwę *utils.py*. Zdefiniujemy funkcję skrótu umożliwiającą utworzenie nowych obiektów `Action` w prosty sposób. Przeprowadź edycję nowego pliku *utils.py* i umieść w nim poniższy fragment kodu:

```
from django.contrib.contenttypes.models import ContentType
from .models import Action


def create_action(user, verb, target=None):
    action = Action(user=user, verb=verb, target=target)
    action.save()
```

Funkcja `create_action()` pozwala na utworzenie działań, które opcjonalnie będą zawierały obiekt `target`. Funkcję możemy wykorzystać w dowolnym miejscu kodu w celu dodawania nowych działań do strumienia aktywności.

Uniknięcie powielonych działań w strumieniu aktywności

Czasami użytkownik może wykonać jakieś działanie wielokrotnie. Być może kilka razy kliknął przycisk *Lubię* to lub *Nie lubię* bądź też w krótkim okresie wykonał tę samą operację wiele razy. W takim przypadku mogłoby dojść do przechowywania i wyświetlania powielających się działań. Aby tego uniknąć, nieco usprawnimy funkcję `create_action()`, a wprowadzone zmiany pomogą w eliminacji większości duplikatów.

Przeprowadź edycję pliku *utils.py* aplikacji `actions`, aby jego kod przedstawiał się następująco.

```
import datetime
from django.contrib.contenttypes.models import ContentType
from django.utils import timezone
from .models import Action


def create_action(user, verb, target=None):
    # Sprawdź, czy w ciągu ostatniej minuty nie wykonano podobnych działań
    now = timezone.now()
    last_minute = now - datetime.timedelta(seconds=60)
    similar_actions = Action.objects.filter(
        user_id=user.id,
        verb= verb,
        created__gte=last_minute
    )
    if target:
       target_ct = ContentType.objects.get_for_model(target)
       similar_actions = similar_actions.filter(
            target_ct=target_ct,
            target_id=target.id
       )
    if not similar_actions:
        # Nie znaleziono żadnych działań
        action = Action(user=user, verb=verb, target=target)
        action.save()
        return True
    return False
```

Aby uniknąć zapisywania duplikatów, zmodyfikowaliśmy funkcję `create_action()`. Funkcja zwraca wartość boolowską informującą, czy dane działanie zostało zapisane. Oto sposób, w jaki unikamy duplikatów:

1. Zaczynamy od pobrania bieżącej daty i godziny za pomocą dostarczanej przez Django metody `timezone.now()`. Działanie tej metody jest takie samo jak metody `datetime.datetime.now()`, ale obiekt zwracany przez metodę uwzględnia strefę czasową. Framework Django dostarcza opcję `USE_TZ` pozwalającą na włączenie lub wyłączenie obsługi stref czasowych. Domyślny plik *settings.py* tworzony przez polecenie `startproject` zawiera opcję `USE_TZ=True`.
2. Zmienną `last_minute` wykorzystujemy do przechowywania daty oraz godziny wskazującej ostatnią minutę i pobieramy wszystkie identyczne działania przeprowadzone przez użytkownika w tym okresie.
3. Jeżeli w ciągu ostatniej minuty nie było przeprowadzone identyczne działanie, tworzymy nowy obiekt `Action`. Jeżeli został utworzony obiekt `Action`, metoda zwraca `True`, w przeciwnym razie `False`.

Dodanie działania użytkownika do strumienia aktywności

Aby zbudować strumień aktywności użytkowników, możemy teraz przystąpić do dodawania pewnych działań do widoków. Będziemy przechowywać działania dla wymienionych poniżej rodzajów operacji:

- Użytkownik dodał zdjęcie.
- Użytkownik polubił zdjęcie.
- Użytkownik utworzył konto.
- Użytkownik zaczął obserwować profil innego użytkownika.

Przeprowadź edycję pliku *views.py* aplikacji `images` i dodaj następujące polecenie `import`.

```
from actions.utils import create_action
```

W widoku `image_create` dodaj po zapisaniu obrazu wywołanie funkcji `create_action()`, co pokazałem poniżej. Nowy kod został wyróżniony pogrubioną czcionką.

```
@login_required
def image_create(request):
    if request.method == 'POST':
        # Formularz został wysłany
        form = ImageCreateForm(data=request.POST)
        if form.is_valid():
            # Dane formularza są prawidłowe
            cd = form.cleaned_data
            new_image = form.save(commit=False)
            # Przypisanie bieżącego użytkownika do elementu
            new_image.user = request.user
            new_image.save()
            create_action(request.user, 'dodał obraz', new_image)
            messages.success(request, 'Pomyślnie dodano obraz')
            # Przekierowanie do widoku szczegółowego dla nowo utworzonego elementu
            return redirect(new_image.get_absolute_url())
    else:
        # Utworzenie formularza na podstawie danych dostarczonych przez bookmarklet
        # w żądaniu GET
        form = ImageCreateForm(data=request.GET)
    return render(
        request,
        'images/image/create.html',
        {'section': 'images', 'form': form}
    )
```

W widoku `image_like`, po dodaniu użytkownika do relacji `users_like`, dodaj wywołanie funkcji `create_action()`, tak jak pokazałem poniżej. Nowy kod został wyróżniony pogrubioną czcionką.

```
@login_required
@require_POST
def image_like(request):
    image_id = request.POST.get('id')
    action = request.POST.get('action')
```

```
    if image_id and action:
        try:
            image = Image.objects.get(id=image_id)
            if action == 'lubię':
                image.users_like.add(request.user)
                create_action(request.user, 'polubił', image)
            else:
                image.users_like.remove(request.user)
            return JsonResponse({'status':'ok'})
        except Image.DoesNotExist:
            pass
    return JsonResponse({'status':'error'})
```

Teraz przeprowadź edycję pliku *views.py* aplikacji `account` i umieść w nim poniższe polecenie `import`.

```
from actions.utils import create_action
```

W widoku `register`, po operacji utworzenia obiektu `Profile`, dodaj wywołanie funkcji `create_action()`, tak jak pokazałem poniżej. Nowy kod został wyróżniony pogrubioną czcionką.

```
def register(request):
    if request.method == 'POST':
        user_form = UserRegistrationForm(request.POST)
        if user_form.is_valid():
            # Utworzenie nowego obiektu użytkownika; jednak jeszcze nie zapisujemy go
            # w bazie danych
            new_user = user_form.save(commit=False)
            # Ustawienie wybranego hasła
            new_user.set_password(
                user_form.cleaned_data['password']
            )
            # Zapisanie obiektu User
            new_user.save()
            # Utworzenie profilu użytkownika
            Profile.objects.create(user=new_user)
            create_action(new_user, 'utworzył konto')
            return render(
                request,
                'account/register_done.html',
                {'new_user': new_user}
            )
    else:
        user_form = UserRegistrationForm()
    return render(
        request,
        'account/register.html',
        {'user_form': user_form}
    )
```

W widoku `user_follow` dodaj wywołanie funkcji `create_action()` w następujący sposób. Nowy kod został wyróżniony pogrubioną czcionką.

```
@require_POST
@login_required
```

```
def user_follow(request):
    user_id = request.POST.get('id')
    action = request.POST.get('action')
    if user_id and action:
        try:
            user = User.objects.get(id=user_id)
            if action == 'obserwuj':
                Contact.objects.get_or_create(
                    user_from=request.user,
                    user_to=user
                )
                create_action(request.user, 'obserwuje', user)
            else:
                Contact.objects.filter(
                    user_from=request.user,
                    user_to=user
                ).delete()
            return JsonResponse({'status':'ok'})
        except User.DoesNotExist:
            return JsonResponse({'status':'error'})
    return JsonResponse({'status':'error'})
```

Jak możesz zobaczyć, z wykorzystaniem modelu `Action` i funkcji pomocniczej można bardzo łatwo umieszczać w strumieniu aktywności użytkownika nowe działania.

Wyświetlanie strumienia aktywności

Trzeba jeszcze zadbać o możliwość wyświetlenia strumienia aktywności użytkownika. Strumień aktywności umieścimy na panelu głównym użytkownika. Przeprowadź edycję pliku *views.py* aplikacji `account`. Zaimportuj model `Action` oraz w następujący sposób zmodyfikuj panel główny. Nowy kod został wyróżniony pogrubioną czcionką.

```
from actions.models import Action

# ...

@login_required
def dashboard(request):
    # Domyślnie wyświetlane są wszystkie działania
    actions = Action.objects.exclude(user=request.user)
    following_ids = request.user.following.values_list(
        'id', flat=True
    )
    if following_ids:
    # Jeżeli użytkownik obserwuje innych, będzie otrzymywał jedynie informacje
    # o podejmowanych przez nich działaniach
        actions = actions.filter(user_id__in=following_ids)
    actions = actions[:10]
    return render(
        request,
        'account/dashboard.html',
        {'section': 'dashboard', 'actions': actions}
    )
```

W tym widoku pobieramy z bazy danych wszystkie działania z wyłączeniem podejmowanych przez bieżącego użytkownika. Domyślnie pobierane są najnowsze działania podejmowane przez wszystkich użytkowników platformy.

Jeśli użytkownik obserwuje innych, zapytanie pobiera jedynie działania podejmowane przez obserwowanych użytkowników. Na koniec ograniczamy wynik zapytania do 10 pierwszych zwróconych działań. Nie używamy tutaj funkcji `order_by()`, ponieważ opieramy się na domyślnej kolejności podanej w opcjach `Meta` modelu `Action`. Najnowsze działania są zwracane jako pierwsze, co wynika z ustawienia `ordering = ('-created')` w modelu `Action`.

Optymalizacja kolekcji QuerySet dotyczącej powiązanych obiektów

Za każdym razem, gdy pobierany jest obiekt `Action`, zwykle uzyskiwany jest dostęp do powiązanego z nim obiektu `User`, a także do powiązanego z użytkownikiem obiektu `Profile`. Warstwa ORM w Django oferuje prosty sposób na jednoczesne pobranie powiązanych ze sobą obiektów, co pozwala uniknąć dodatkowych zapytań do bazy danych.

Użycie metody select_related()

Django zawiera metodę kolekcji `QuerySet` o nazwie `select_related()` pozwalającą na pobieranie powiązanych obiektów dla relacji typu „jeden do wielu". Wprawdzie oznacza to jedną, bardziej skomplikowaną kolekcję `QuerySet`, ale jednocześnie pomaga w uniknięciu dodatkowych zapytań do bazy danych podczas uzyskiwania dostępu do powiązanych obiektów.

Metoda `select_related` jest przeznaczona dla pól typu `ForeignKey` i `OneToOne`. Jej działanie polega na wykonaniu polecenia SQL `JOIN` i uwzględnieniu w poleceniu `SELECT` pól obiektu powiązanego.

Aby wykorzystać zalety użycia metody `select_related()`, przeprowadź edycję poniższego wiersza kodu z poprzedniego fragmentu w pliku *views.py* aplikacji `account` i dodaj wywołanie metody `select_related`, przekazując pola, których będziesz używać, tak jak pokazałem na poniższym listingu.

Przeprowadź edycję pliku *views.py* aplikacji `account`. Nowy kod został wyróżniony pogrubioną czcionką.

```
@login_required
def dashboard(request):
    # Domyślnie wyświetlane są wszystkie działania
    actions = Action.objects.exclude(user=request.user)
    following_ids = request.user.following.values_list(
        'id', flat=True
    )
    if following_ids:
        # Jeżeli użytkownik obserwuje innych, będzie otrzymywał jedynie informacje
        # o podejmowanych przez nich działaniach
        actions = actions.filter(user_id__in=following_ids)
    actions = actions.select_related(
```

```
        'user', 'user__profile'
    )[:10]
    return render(
        request,
        'account/dashboard.html',
        {'section': 'dashboard', 'actions': actions}
    )
```

Wykorzystujemy `user__profile` w celu złączenia tabeli `Profile` w pojedynczym zapytaniu SQL. Jeżeli w wywołaniu `select_related()` nie przekażesz żadnych argumentów, pobrane zostaną obiekty ze wszystkich relacji `ForeignKey`. Zawsze ograniczaj wywołanie `select_related()` do relacji, które mają być zastosowane po wykonaniu danego wywołania.

Uwaga

Rozsądne użycie metody `select_related()` może znacznie skrócić czas wykonywania zapytania.

Użycie metody prefetch_related()

Jak widać, metoda `select_related()` może pomóc w zwiększeniu wydajności operacji pobierania powiązanych obiektów z relacji typu „jeden do wielu". Jednak metoda `select_related()` nie działa w przypadku relacji typu „wiele do wielu" lub „wiele do jednego" (pól `ManyToMany` lub odwróconych pól typu `ForeignKey`). Django oferuje kolejną metodę kolekcji `QuerySet` o nazwie `prefetch_related` działającą dla relacji typu „wiele do wielu" lub „wiele do jednego", poza oczywiście relacjami obsługiwanymi przez `select_related()`. Metoda `prefetch_related()` wykonuje dodatkowe wyszukiwanie poszczególnych relacji i otrzymane wyniki łączy za pomocą Pythona. Metoda ta obsługuje również pobieranie kolumn typu `GenericRelation` i `GenericForeignKey`.

Otwórz do edycji plik *views.py* aplikacji `account` i dokończ budowę zapytania przez dodanie wywołania `prefetch_related()` do kolumny `GenericForeignKey` obiektu docelowego, tak jak pokazałem poniżej. Nowy kod został wyróżniony pogrubioną czcionką.

```
@login_required
def dashboard(request):
    # Domyślnie wyświetlane są wszystkie działania
    actions = Action.objects.exclude(user=request.user)
    following_ids = request.user.following.values_list(
        'id', flat=True
    )
    if following_ids:
        # Jeżeli użytkownik obserwuje innych, będzie otrzymywał jedynie informacje
        # o podejmowanych przez nich działaniach
        actions = actions.filter(user_id__in=following_ids)
    actions = actions.select_related(
        'user', 'user__profile'
    ).prefetch_related('target')[:10]
    return render(
        request,
        'account/dashboard.html',
        {'section': 'dashboard', 'actions': actions}
    )
```

Przedstawione zapytanie zostało w ten sposób zoptymalizowane do pobierania działań użytkownika oraz powiązanych obiektów.

Tworzenie szablonów dla działań użytkowników

Przygotujemy teraz szablon przeznaczony do wyświetlania konkretnego obiektu `Action`. Utwórz nowy katalog w aplikacji `actions` i nadaj mu nazwę *templates*. Następnie umieść w nim poniższą strukturę plików i katalogów.

```
actions/
     action/
         detail.html
```

Przeprowadź edycję pliku szablonu *actions/action/detail.html* i umieść w nim poniższy fragment kodu.

```
{% load thumbnail %}

{% with user=action.user profile=action.user.profile %}
<div class="action">
  <div class="images">
    {% if profile.photo %}
      {% thumbnail user.profile.photo "80x80" crop="100%" as im %}
      <a href="{{ user.get_absolute_url }}">
         <img src="{{ im.url }}" alt="{{ user.get_full_name }}"
          class="item-img">
       </a>
    {% endif %}
    {% if action.target %}
      {% with target=action.target %}
        {% if target.image %}
          {% thumbnail target.image "80x80" crop="100%" as im %}
          <a href="{{ target.get_absolute_url }}">
            <img src="{{ im.url }}" class="item-img">
          </a>
        {% endif %}
      {% endwith %}
    {% endif %}
  </div>
  <div class="info">
    <p>
      <span class="date">{{ action.created|timesince }} temu</span>
      <br />
      <a href="{{ user.get_absolute_url }}">
        {{ user.first_name }}
      </a>
      {{ action.verb }}
      {% if action.target %}
        {% with target=action.target %}
          <a href="{{ target.get_absolute_url }}">{{ target }}</a>
        {% endwith %}
      {% endif %}
    </p>
```

```
  </div>
</div>
{% endwith %}
```

Jest to szablon przeznaczony do wyświetlania obiektu `Action`. Na początku używamy znacznika szablonu `{% with %}` w celu pobrania użytkownika wykonującego dane działanie oraz jego profilu. Następnie, jeśli obiekt `Action` ma powiązany z nim obiekt docelowy, wyświetlamy obraz obiektu docelowego. Na koniec wyświetlamy łącze do użytkownika przeprowadzającego dane działanie oraz obiekt docelowy, o ile taki istnieje.

Teraz przeprowadź edycję szablonu *account/dashboard.html* aplikacji `account` i umieść poniższy fragment kodu na końcu bloku `content`.

```
{% extends "base.html" %}

{% block title %}Panel główny{% endblock %}

{% block content %}

  ...

  <h2>Co nowego?</h2>
  <div id="action-list">
    {% for action in actions %}
      {% include "actions/action/detail.html" %}
    {% endfor %}
  </div>
{% endblock %}
```

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/account/`. Zaloguj się na konto istniejącego użytkownika i wykonaj kilka działań, aby mogły zostać zapisane w bazie danych. Następnie zaloguj się na konto innego użytkownika, obserwuj poprzedniego i spójrz na wygenerowany w panelu głównym strumień aktywności.

Powinieneś zobaczyć stronę podobną do pokazanej na rysunku 7.8.

Rysunek 7.8. Strumień aktywności dla bieżącego użytkownika

Uwaga

Atrybucje dla zdjęć przedstawionych na rysunku 7.8:

Ctac, *silnik indukcyjny Tesli* (licencja: Creative Commons Attribution Share-Alike 3.0 Unported: *https://creativecommons.org/licenses/by-sa/3.0/*).

Rocky Acosta, *model maszyny Turinga Davey, 2012* (licencja: Creative Commons Attribution 3.0 Unported: *https://creativecommons.org/licenses/by/3.0/*).

ataelw *Chick Corea* (licencja: Creative Commons Attribution 2.0 Generic: *https://creativecommons.org/licenses/by/2.0/*).

W ten sposób zakończyłeś implementację strumienia aktywności użytkowników, do którego można bardzo łatwo dodawać podejmowane przez nich działania. Do strumienia aktywności możesz dodać jeszcze funkcjonalność przewijania w nieskończoność przez implementację tego samego mechanizmu stronicowania AJAX, który został wykorzystany w widoku `image_list`. W następnym podrozdziale nauczysz się, jak używać sygnałów Django do denormalizacji liczników działań.

Użycie sygnałów dla denormalizowanych zliczeń

Zdarzają się sytuacje, gdy zachodzi potrzeba denormalizacji danych. Denormalizacja oznacza redundancję danych w sposób zapewniający optymalizację wydajności operacji odczytu. Na przykład, aby uniknąć kosztownych zapytań odczytu do bazy danych podczas pobierania powiązanych danych, możesz skopiować powiązane dane do obiektu. Przy denormalizacji należy zachować dużą ostrożność i stosować ją tylko i wyłącznie wtedy, gdy jest absolutnie konieczna. Największy problem, jaki pojawia się w przypadku denormalizacji, to trudność w zachowaniu aktualności denormalizowanych danych.

Przyjrzyjmy się, jak można poprawić wydajność wykonywania zapytań za pomocą denormalizacji zliczeń. Decydujemy się na denormalizację danych pochodzących z modelu `Image` i użycie sygnałów Django do zapewnienia aktualności danych.

Praca z sygnałami

Framework Django jest dostarczany wraz z dyspozytorem sygnałów, który pozwala funkcjom typu `receiver` na otrzymywanie powiadomień o wystąpieniu określonych działań. Sygnały okazują się niezwykle użyteczne, gdy trzeba wykonać zadanie po wystąpieniu innego zdarzenia. Sygnały pozwalają na rozdzielenie logiki: możesz przechwycić określone działanie niezależnie od aplikacji lub kodu, które je wyzwoliły, i zaimplementować logikę, która jest wykonywana za każdym razem, gdy to działanie ma miejsce. Na przykład możesz zbudować funkcję odbiornika sygnału, która będzie wykonywana za każdym razem, gdy jest zapisywany obiekt użytkownika. Oczywiście można także tworzyć własne sygnały, aby inne komponenty otrzymywały powiadomienia o wystąpieniu pewnych zdarzeń.

Django zapewnia kilka sygnałów dla modeli; zamieszczono je w aplikacji `django.db.models.signals`. Oto kilka wybranych sygnałów:

- `pre_save` i `post_save`: sygnały są wysyłane — odpowiednio — przed wywołaniem metody `save()` modelu i po jej wywołaniu.
- `pre_delete` i `post_delete`: sygnały są wysyłane — odpowiednio — przed wywołaniem metody `delete()` modelu lub kolekcji `QuerySet` i po jej wywołaniu.
- `m2m_changed`: sygnał jest wysyłany po zmianie w modelu wartości pola `ManyToManyField`.

Na liście wymieniłem zaledwie kilka sygnałów oferowanych przez Django. Pełną listę wbudowanych sygnałów Django znajdziesz na stronie *https://docs.djangoproject.com/en/5.0/ref/signals/*.

Przyjmujemy założenie, że chcemy pobierać obrazy na podstawie ich popularności. W tym celu można wykorzystać funkcje agregacji w Django i pobierać obrazy ułożone w kolejności liczby ich polubień przez użytkowników. Przypomnij sobie, jak używaliśmy funkcji agregacji w rozdziale 3., „Rozbudowa aplikacji bloga". Przedstawiony poniżej fragment kodu będzie pobierał obrazy na podstawie liczby ich polubień.

```
from django.db.models import Count
from images.models import Image

images_by_popularity = Image.objects.annotate(
     total_likes=Count('users_like')
).order_by('-total_likes')
```

Jednak uporządkowanie obrazów przez zliczanie ich polubień jest pod względem wydajności znacznie kosztowniejszą operacją niż ich układanie na podstawie kolumny zawierającej całkowitą liczbę polubień. Do modelu `Image` można więc dodać kolumnę przechowującą tę liczbę. Wprowadzamy tym samym denormalizację danych, choć dzięki temu zwiększamy wydajność wykonywania zapytań wykorzystujących tę nowo dodaną kolumnę. Powstaje w tym miejscu pytanie, jak uaktualniać wartość w nowej kolumnie?

Przeprowadź edycję pliku *models.py* aplikacji `images` i dodaj do modelu `Image` pole `total_likes`. Nowy kod został wyróżniony pogrubioną czcionką.

```
class Image(models.Model):
    # ...
    total_likes = models.PositiveIntegerField(default=0)

    class Meta:
        indexes = [
            models.Index(fields=['-created']),
            models.Index(fields=['-total_likes']),
        ]
        ordering = ['-created']
```

Pole `total_likes` pozwala na przechowywanie całkowitej liczby użytkowników, którzy polubili poszczególne obrazy. Denormalizowane zliczenie jest użyteczne, gdy trzeba filtrować lub układać kolekcję `QuerySet` względem tej liczby. Dodaliśmy dla pola `total_likes` indeks bazy danych dla porządku malejącego, ponieważ zamierzamy pobierać obrazy uporządkowane według ich łącznej liczby polubień w porządku malejącym.

Wskazówka

Istnieje wiele sposobów poprawy wydajności, które warto wziąć pod uwagę przed denormalizacją danych. Rozważ np. użycie indeksów bazy danych, optymalizację zapytań lub buforowanie.

W celu utworzenia migracji dla nowego pola dodanego do bazy danych wydaj w powłoce poniższe polecenie.

```
python manage.py makemigrations images
```

Powinieneś otrzymać takie dane wyjściowe:

```
Migrations for 'images':
   images/migrations/0002_image_total_likes_and_more.py
     - Add field total_likes to image
    - Create index images_imag_total_l_0bcd7e_idx on field(s) -total_likes of model
     ↪image
```

Teraz wydanie poniższego polecenia spowoduje faktyczne zastosowanie migracji.

```
python manage.py migrate images
```

Wygenerowane dane wyjściowe powinny zawierać m.in. następujący wiersz.

```
Applying images.0002_image_total_likes_and_more... OK
```

Dodamy teraz funkcję typu receiver do sygnału `m2m_changed`.

Utwórz nowy plik w katalogu aplikacji `images` i nadaj mu nazwę *signals.py*. Następnie w pliku umieść poniższy fragment kodu.

```
from django.db.models.signals import m2m_changed
from django.dispatch import receiver
from .models import Image

@receiver(m2m_changed, sender=Image.users_like.through)
def users_like_changed(sender, instance, **kwargs):
    instance.total_likes = instance.users_like.count()
    instance.save()
```

Przede wszystkim funkcję `users_like_changed` rejestrujemy jako funkcję typu `receiver` przy użyciu dekoratora `receiver()`. Dołączamy ją do sygnału `m2m_changed`. Aby wywołanie funkcji następowało jedynie po wysłaniu sygnału `m2m_changed` przez nadawcę, łączymy tę funkcję z `Image.users_like.through`. Istnieje również inna metoda rejestracji funkcji typu `receiver` polegająca na użyciu metody `connect()` obiektu `Signal`.

Uwaga

Sygnały w Django są synchroniczne i blokujące. Nie pomyl więc sygnałów z zadaniami asynchronicznymi. Można jednak połączyć sygnały i zadania asynchroniczne w celu ich wywoływania, gdy kod zostanie powiadomiony przez sygnał. Sposób tworzenia zadań asynchronicznych za pomocą Celery opisałem w rozdziale 8., „Utworzenie sklepu internetowego”.

Aby funkcja typu `receiver` była wywoływana za każdym razem, gdy jest wysyłany sygnał, trzeba połączyć ją z sygnałem. Zalecaną metodą rejestracji sygnałów jest ich zaimportowanie w metodzie `ready()` klasy konfiguracyjnej aplikacji. Django oferuje rejestr aplikacji pozwalający na konfigurację i introspekcję aplikacji.

Definiowanie klas konfiguracyjnych aplikacji

Django umożliwia wskazanie klas konfiguracyjnych aplikacji. Kiedy tworzysz aplikację za pomocą polecenia `startapp`, Django dodaje do katalogu aplikacji plik *apps.py*, włącznie z prostą konfiguracją aplikacji dziedziczącą po klasie `AppConfig`.

Klasa konfiguracyjna aplikacji pozwala na przechowywanie metadanych i konfiguracji dla aplikacji oraz umożliwia zastosowanie introspekcji. Więcej informacji na temat konfigurowania aplikacji znajdziesz pod adresem *https://docs.djangoproject.com/en/5.0/ref/applications/*.

W celu zarejestrowania funkcji typu `receiver`, kiedy używany jest dekorator `receiver()`, trzeba w metodzie `ready()` klasy `AppConfig` zaimportować moduł `signals`. Metoda ta jest wywoływana bezpośrednio po całkowitym wypełnieniu rejestru aplikacji. Wszelkie pozostałe funkcje inicjalizujące aplikacji również należy dołączać w wymienionej metodzie.

Otwórz do edycji plik *apps.py* aplikacji `images` i wprowadź w nim poniższy fragment kodu wyróżniony pogrubioną czcionką:

```
from django.apps import AppConfig

class ImagesConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'images'

    def ready(self):
        # Import procedur obsługi sygnałów
        import images.signals
```

W metodzie `ready()` przeprowadzamy import sygnałów do aplikacji. Zostaną one zaimportowane w momencie załadowania aplikacji `images`.

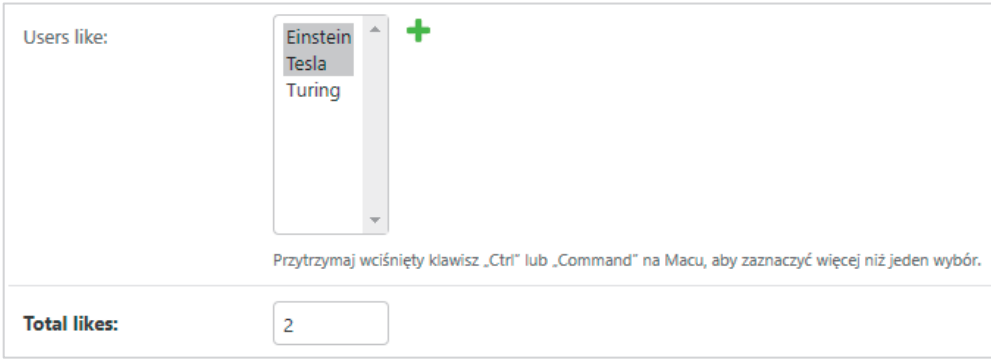
Uruchom serwer programistyczny za pomocą polecenia:

```
python manage.py runserver
```

Otwórz przeglądarkę internetową, przejdź na stronę widoku szczegółowego obrazu i kliknij przycisk polubienia obrazu.

Teraz w witrynie administracyjnej przejdź pod adres URL edycji obrazu, np. `http://127.0.0.1:8000/admin/images/image/1/change/`, i spójrz na atrybut `total_likes`. Powinieneś zobaczyć, że wymieniony atrybut został uaktualniony, co pokazałem na rysunku 7.9.

W tym momencie można używać atrybutu `total_likes` do porządkowania obrazów według ich popularności lub też do wyświetlania tej wartości w innych miejscach bez konieczności wykonywania skomplikowanego zapytania, aby ją obliczyć.



Rysunek 7.9. Strona edycji obrazu w witrynie administracyjnej włącznie z denormalizacją dla całkowitej liczby polubień

Poniższe zapytanie pozwala uzyskać obrazy uporządkowane według liczby polubień w kolejności malejącej.

```
from django.db.models import Count

images_by_popularity = Image.objects.annotate(
    likes=Count('users_like')
).order_by('-likes')
```

Powyższe zapytanie można teraz zapisać w następujący sposób.

```
images_by_popularity = Image.objects.order_by('-total_likes')
```

Dzięki denormalizacji całkowitej liczby polubień dla obrazów uzyskaliśmy mniej kosztowne zapytanie SQL. Przy okazji nauczyłeś się, jak możesz używać sygnałów w Django.

Uwaga

Zachowaj ostrożność podczas korzystania z sygnałów, ponieważ utrudniają one kontrolowanie przepływu sterowania. W wielu przypadkach można uniknąć użycia sygnałów, gdy wiadomo, które funkcje mają być poinformowane o wystąpieniu określonego zdarzenia.

Konieczne jest ustawienie wartości początkowej liczników dla pozostałych obiektów `Image` w taki sposób, aby odpowiadały bieżącemu stanowi zapisanemu w bazie danych.

Przejdź do powłoki Pythona za pomocą polecenia:

```
python manage.py shell
```

W powłoce Pythona uruchom kod:

```
>>> from images.models import Image
>>> for image in Image.objects.all():
...     image.total_likes = image.users_like.count()
...     image.save()
```

W ten sposób ręcznie zaktualizowałeś liczbę polubień dla istniejących obrazów w bazie danych. Od teraz za każdym razem, gdy zmienią się obiekty powiązane w relacji „wiele do wielu", aktualizację pola `total_likes` będzie obsługiwać funkcja odbiornika sygnału `users_like_changed`.

W następnym podrozdziale dowiesz się, jak korzystać z mechanizmu *Django Debug Toolbar* w celu pobrania odpowiednich informacji debugowania dla żądań, w tym czasu wykonania, wykonanych zapytań SQL, renderowanych szablonów, zarejestrowanych sygnałów i wielu innych.

Korzystanie z paska narzędzi Django Debug Toolbar

W tym momencie widok strony z informacjami diagnostycznymi Django powinien być już dla Ciebie znajomy. Podczas uruchamiania przykładów z poprzednich rozdziałów z pewnością co najmniej kilkakrotnie widziałeś charakterystyczną żółto-szarą stronę debugowania frameworka Django.

Na przykład w rozdziale 2., „Usprawnianie bloga i dodanie funkcjonalności społecznościowych", w punkcie „Obsługa błędów stronicowania", strona debugowania zawierała informacje dotyczące wyjątków nieobsługiwanych podczas implementowania stronicowania obiektów.

Strona debugowania Django zawiera przydatne informacje diagnostyczne. Framework Django zapewnia jednak aplikację, która zawiera bardziej szczegółowe informacje debugowania i może być bardzo pomocna podczas programowania.

Django Debug Toolbar to zewnętrzna aplikacja Django, która umożliwia wyświetlanie informacji diagnostycznych dotyczących bieżącego cyklu żądanie-odpowiedź. Informacje są podzielone na wiele paneli, które prezentują różne informacje, w tym dane żądanie-odpowiedź, wykorzystywane wersje pakietów Pythona, czas wykonania, ustawienia, nagłówki, zapytania SQL, używane szablony, pamięć podręczną, sygnały i logi.

Dokumentację paska narzędzi *Django Debug Toolbar* można znaleźć na stronie *https://django-debug-toolbar.readthedocs.io/*.

Instalacja paska narzędzi Django Debug Toolbar

Aby zainstalować aplikację `django-debug-toolbar` za pomocą menedżera `pip`, skorzystaj z polecenia:

```
python -m pip install django-debug-toolbar==4.3.0
```

Przeprowadź edycję pliku *settings.py* w swoim projekcie i dodaj do ustawienia `INSTALLED_APPS` aplikację `add_debug_toolbar` tak, jak pokazałem poniżej. Nowy kod został wyróżniony pogrubioną czcionką.

```
INSTALLED_APPS = [
     # ...
    'debug_toolbar',
     # ...
]
```

W tym samym pliku dodaj do ustawienia MIDDLEWARE wiersz wyróżniony na poniższym listingu pogrubioną czcionką.

```
MIDDLEWARE = [
    'debug_toolbar.middleware.DebugToolbarMiddleware',
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]
```

Pasek narzędzi *Django Debug Toolbar* jest najczęściej implementowany jako warstwa middleware. Kolejność aplikacji wewnątrz ustawienia MIDDLEWARE ma znaczenie. Aplikację DebugToolbarMiddleware należy umieścić przed innymi aplikacjami middleware, z wyjątkiem tych, które kodują treść odpowiedzi, na przykład GzipMiddleware (jeśli ta ostatnia aplikacja jest zainstalowana, powinna się znaleźć na pierwszym miejscu).

Dodaj na końcu pliku *settings.py* następujące wiersze:

```
INTERNAL_IPS = [
    '127.0.0.1',
]
```

Pasek narzędzi *Django Debug Toolbar* wyświetli się tylko wtedy, gdy Twój adres IP pasuje do wpisu w ustawieniu INTERNAL_IPS. Aby zapobiec wyświetlaniu informacji diagnostycznych w środowisku produkcyjnym, pasek narzędzi *Django Debug Toolbar* sprawdza, czy ustawienie DEBUG ma wartość True.

Przeprowadź edycję pliku *urls.py* swojego projektu i dodaj do listy urlpatterns poniższy wzorzec adresu URL oznaczony pogrubioną czcionką.

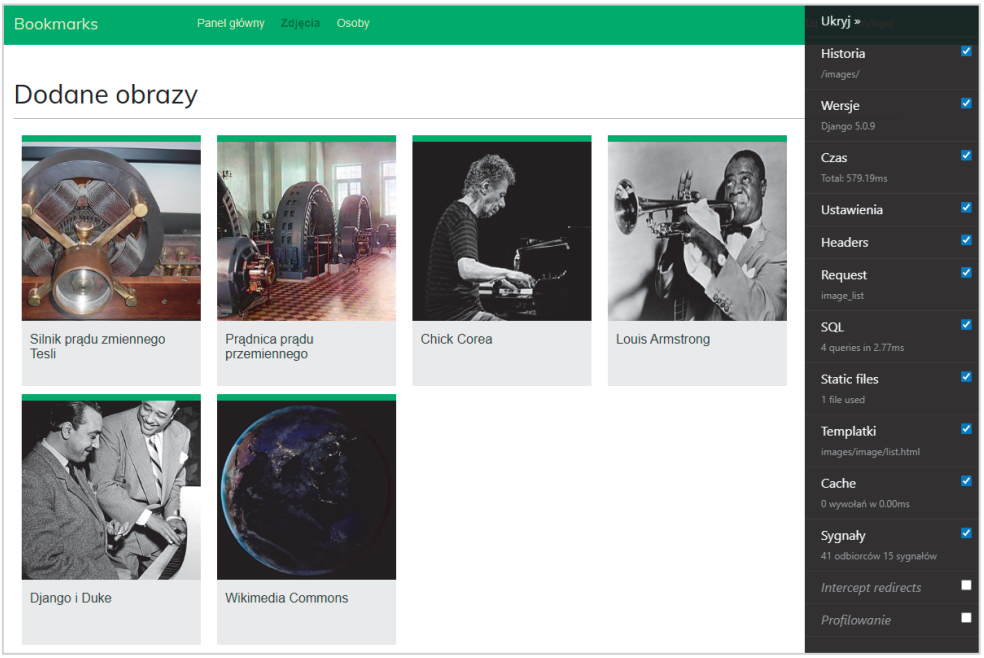
```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('account/', include('account.urls')),
    path(         'social-auth/',
        include('social_django.urls', namespace='social')
    ),
    path('images/', include('images.urls', namespace='images')),
    path('__debug__/', include('debug_toolbar.urls')),
]
```

Pasek narzędzi *Django Debug Toolbar* jest teraz zainstalowany w Twoim projekcie. Wypróbujmy go!

Uruchom serwer programistyczny za pomocą polecenia:

```
python manage.py runserver
```

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/images/`. Po prawej stronie powinieneś zobaczyć rozwijany pasek boczny. Powinien mieć postać pokazaną na rysunku 7.10.



Rysunek 7.10. Pasek boczny paska narzędzi debugowania Django Debug Toolbar

Uwaga

Atrybucje dla zdjęć przedstawionych na rysunku 7.10:

ataelw, *Chick Corea* (licencja*:* Creative Commons Attribution 2.0 Generic*:* *https://creativecommons.org/licenses/by/2.0/*).

Eddi Laumanns, znany również jako RX-Guru, *Al Jarreau — Düsseldorf 1981* (licencja: Creative Commons Attribution 3.0 Unported*: https://creativecommons .org/licenses/ by/3.0/*).

Kingkongphoto i *www.celebrity-photos.com*, *Al Jarreau* (licencja: Creative Commons Attribution-ShareAlike 2.0 Generic*: https://creativecommons.org/licenses/by-sa/2.0/*).

Jeśli pasek narzędzi się nie wyświetli, sprawdź log konsoli w powłoce RunServer. Jeśli wyświetli się błąd typu MIME, najprawdopodobniej Twoje pliki map MIME są nieprawidłowe lub wymagają aktualizacji.

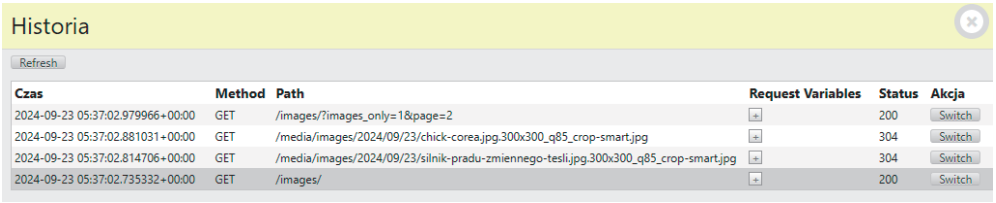
Aby zastosować poprawne mapowanie dla plików JavaScript i CSS, należy dodać do pliku *settings.py* następujące wiersze:

```
if DEBUG:
    import mimetypes
    mimetypes.add_type('application/javascript', '.js', True)
    mimetypes.add_type('text/css', '.css', True)
```

Panele paska narzędzi Django Debug Toolbar

Pasek narzędzi *Django Debug Toolbar* zawiera wiele paneli, które organizują informacje debugowania dla cyklu żądanie-odpowiedź. Aby aktywować lub dezaktywować dowolny panel, możesz użyć powiązanego z nim pola wyboru. Zmiana zostanie zastosowana przy następnym żądaniu. Jest to przydatne, gdy nie interesuje nas konkretny panel, a obliczenia dodają do żądania zbyt duży narzut.

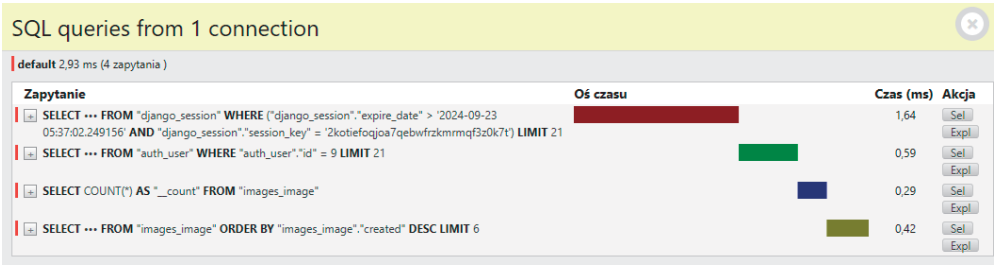
Kliknij w menu paska bocznego pozycję *Historia*. Wyświetli się panel pokazany na rysunku 7.11.



Rysunek 7.11. Pasek narzędzi Django Debug Toolbar — panel Historia

W systemach Linux i macOS dostępna jest zakładka *Czas*, zawierająca informacje o licznikach czasu dla różnych faz cyklu żądanie-odpowiedź. Wyświetla także czas procesora, czas obsługi żądania i liczbę przełączeń kontekstu. W systemie Windows panel *Czas* jest niedostępny. Na pasku narzędzi jest dostępny tylko całkowity czas obsługi żądania.

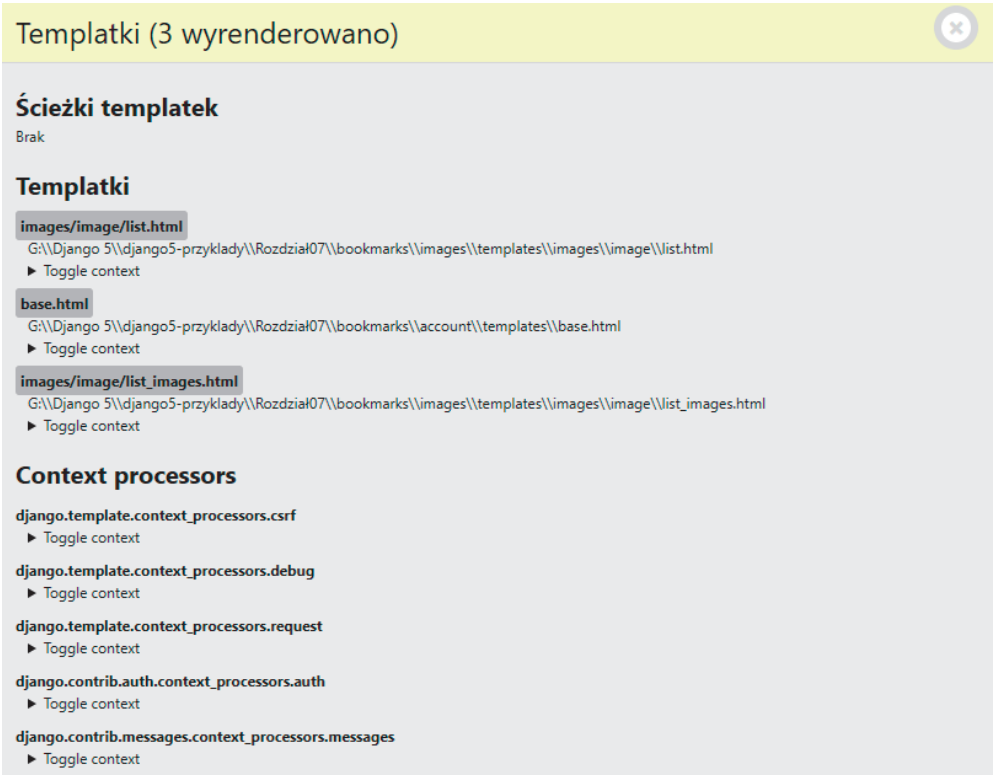
Kliknij w menu paska bocznego pozycję *SQL*. Wyświetli się panel pokazany na rysunku 7.12.



Rysunek 7.12. Pasek narzędzi Django Debug Toolbar — panel SQL

W tym panelu możesz zobaczyć różne wykonane zapytania SQL. Zamieszczone w nim informacje mogą Ci pomóc w zidentyfikowaniu niepotrzebnych zapytań, zapytań powielonych, które można ponownie wykorzystać, lub zapytań czasochłonnych, które można zoptymalizować. Na podstawie odczytanych informacji możesz udoskonalić kolekcje `QuerySet` w swoich widokach, w razie potrzeby utworzyć nowe indeksy w polach modelu lub zdecydować się na przeniesienie danych do pamięci podręcznej. W tym rozdziale nauczyłeś się, jak optymalizować zapytania obejmujące relacje za pomocą funkcji `select_related()` i `prefetch_related()`. Sposoby korzystania z pamięci podręcznej zostaną omówione w rozdziale 14., „Renderowanie i buforowanie treści”.

Kliknij w menu paska bocznego pozycję *Templatki*. Wyświetli się panel pokazany na rysunku 7.13.



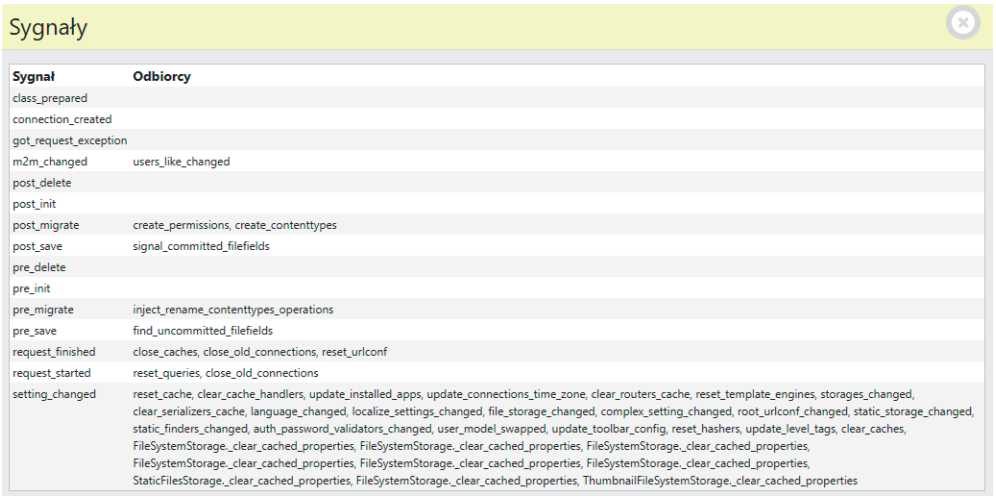
Rysunek 7.13. Pasek narzędzi Django Debug Toolbar — panel Templatki

Ten panel wyświetla różne szablony używane podczas renderowania zawartości, ścieżki szablonów oraz wykorzystywany kontekst. Możesz także wyświetlić używane procesory kontekstu. Więcej informacji na temat procesorów kontekstu znajdziesz w rozdziale 8., „Utworzenie sklepu internetowego".

Kliknij w menu paska bocznego pozycję *Sygnały*. Wyświetli się panel pokazany na rysunku 7.14.

W tym panelu możesz zobaczyć wszystkie sygnały zarejestrowane w Twoim projekcie oraz funkcje odbiornika dołączone do każdego sygnału. Na przykład możesz znaleźć utworzoną wcześniej funkcję odbiornika `users_like_changed` dołączoną do sygnału `m2m_changed`. Inne aplikacje Django korzystają z różnych sygnałów i powiązanych z nimi odbiorników.

Przejrzeliśmy niektóre panele dostarczane z paskiem narzędzi *Django Debug Toolbar*. Oprócz paneli wbudowanych możesz znaleźć dodatkowe panele podmiotów zewnętrznych, które są dostępne do pobrania na stronie *https://django-debug-toolbar.readthedocs.io/en/latest/panels.html#third-party-panels*.



Rysunek 7.14. Pasek narzędzi Django Debug Toolbar — panel Sygnały

Polecenia paska narzędzi Django Debug Toolbar

Oprócz paneli debugowania żądanie-odpowiedź pasek narzędzi *Django Debug Toolbar* udostępnia polecenia umożliwiające debugowanie zapytań SQL dla wywołań ORM. Polecenie narzędziowe `debugsqlshell` replikuje polecenie powłoki Django, ale generuje instrukcje SQL dla zapytań wykonywanych za pomocą Django ORM.

Przejdź do powłoki Pythona za pomocą polecenia:

```
python manage.py debugsqlshell
```

Uruchom polecenie:

```
>>> from images.models import Image
>>> Image.objects.get(id=17)
```

Wyświetli się następujący wynik.

```
SELECT "images_image"."id",
       "images_image"."user_id",
       "images_image"."title",
       "images_image"."slug",
       "images_image"."url",
       "images_image"."image",
       "images_image"."description",
       "images_image"."created",
       "images_image"."total_likes"
FROM "images_image"
WHERE "images_image"."id" = 17
LIMIT 21 [0.44ms]
<Image: Chick Corea>
```

Z powyższego polecenia możesz skorzystać w celu przetestowania zapytań ORM przed dodaniem ich do swoich widoków. Dla każdego wywołania ORM możesz sprawdzić wynikową instrukcję SQL i czas jej wykonania.

W następnym podrozdziale dowiesz się, jak zliczać wyświetlenia obrazu przy użyciu Redis — bazy danych w pamięci — która zapewnia niskie opóźnienia i wysoką przepustowość dostępu do danych.

Zliczanie wyświetleń obrazu za pomocą bazy danych Redis

Redis to zaawansowana baza danych typu klucz-wartość pozwalająca na przechowywanie danych różnego typu i charakteryzująca się niezwykle szybkimi operacjami wejścia-wyjścia. Redis przechowuje wszystko w pamięci, choć dane mogą być trwale zachowane przez ich zapis na dysku w określonych odstępach czasu lub przez dodanie każdego polecenia do dziennika zdarzeń. W porównaniu do innych magazynów typu klucz-wartość, Redis wyróżnia się wszechstronnością. Oferuje zestaw poleceń o użytecznych możliwościach, obsługuje różnorodne struktury danych, takie jak ciągi tekstowe, tablice asocjacyjne, listy, zbiory, uporządkowane zbiory, a nawet obiekty `bitmap` i `HyperLogLogs`.

Baza danych SQL jest odpowiednia w charakterze trwałego magazynu danych o zdefiniowanym schemacie, natomiast Redis oferuje wiele zalet podczas obsługi szybko zmieniających się danych, nietrwałych magazynów danych lub gdy trzeba użyć szybkiego bufora. W tym podrozdziale dowiesz się, jak można wykorzystać Redis do budowy nowej funkcjonalności w naszym projekcie.

Więcej informacji o Redis można znaleźć na jego stronie głównej pod adresem *https://redis.io/*.

Redis udostępnia obraz platformy Docker, który bardzo ułatwia wdrożenie serwera Redis o standardowej konfiguracji.

Instalacja bazy danych Redis

Aby zainstalować obraz Docker bazy danych Redis, najpierw upewnij się, że na Twoim komputerze jest zainstalowany Docker. Sposób instalacji Dockera opisałem w rozdziale 3., „Rozbudowa aplikacji bloga".

W wierszu polecenia powłoki uruchom polecenie:

```
docker pull redis:7.2.4
```

Spowoduje to pobranie obrazu Docker dla platformy Redis na komputer lokalny. Informacje dotyczące oficjalnego obrazu Docker bazy danych Redis można znaleźć na stronie *https://hub.docker.com/_/redis*. Opis innych metod instalacji Redis znajdziesz na stronie *https://redis.io/download/*.

Aby uruchomić kontener Docker bazy danych Redis, wykonaj w powłoce polecenie:

```
docker run -it --rm --name redis -p 6379:6379 redis:7.2.4
```

Za pomocą tego polecenia uruchamiamy bazę danych Redis w kontenerze Docker. Opcja `-it` zleca Dockerowi przejście do kontenera w celu interaktywnego wprowadzania danych. Opcja `--rm` poleca Dockerowi automatyczne wyczyszczenie kontenera i usunięcie go z systemu plików po zamknięciu. Opcja `--name` służy do przypisania do kontenera nazwy. Opcja `-p` służy do opublikowania portu `6379`, na którym działa Redis, do tego samego portu interfejsu hosta. `6379` to domyślny port dla bazy danych Redis.

Powinieneś zobaczyć dane wyjściowe zakończone wierszami:

```
# Server initialized
* Ready to accept connections
```

Pozostaw uruchomiony serwer Redis i otwórz inną powłokę. Uruchom kontener z bazą danych Redis za pomocą poniższego polecenia.

```
docker exec -it redis sh
```

Powinieneś zobaczyć wiersz z symbolem krzyżyka.

```
#
```

Uruchom klienta Redis, wydając w tym celu poniższe polecenie.

```
# redis-cli
```

Powinieneś zobaczyć powłokę klienta Redis przykładowo w następującej postaci.

```
127.0.0.1:6379>
```

Klient Redis pozwala na wykonywanie poleceń Redis bezpośrednio z powłoki. Wypróbujmy więc pewne polecenia. Wydaj polecenie `SET` w powłoce Redis, aby umieścić wartość w kluczu.

```
127.0.0.1:6379> SET name "Piotr"
OK
```

Powyższe polecenie powoduje utworzenie w bazie danych Redis klucza `name` wraz z wartością w postaci ciągu tekstowego `"Piotr"`. Wynik `OK` wskazuje na zakończoną powodzeniem operację zapisania klucza.

Jeżeli później zechcesz pobrać wartość tego klucza, musisz wydać polecenie `GET`, tak jak przedstawiłem poniżej.

```
127.0.0.1:6379> GET name
"Piotr"
```

Za pomocą `EXISTS` można sprawdzić, czy w bazie danych istnieje klucz o podanej nazwie. Polecenie zwraca 1, gdy klucz istnieje, i 0 w przeciwnym razie.

```
127.0.0.1:6379> EXISTS name
(integer) 1
```

Za pomocą polecenia `EXPIRE` można zdefiniować wyrażony w sekundach czas ważności danego klucza. Inną możliwością jest użycie polecenia `EXPIREAT` oczekującego podania znacznika czasu systemu UNIX. Wygaśnięcie klucza jest użyteczne podczas stosowania bazy danych Redis w charakterze bufora lub nietrwałego magazynu danych.

```
127.0.0.1:6379> GET name
"Piotr"
127.0.0.1:6379> EXPIRE name 2
(integer) 1
```

Odczekaj więcej niż dwie sekundy i ponownie spróbuj odczytać wartość tego samego klucza.

```
127.0.0.1:6379> GET name
(nil)
```

Odpowiedź (`nil`) to po prostu wartość `NULL` oznaczająca brak w bazie danych klucza o podanej nazwie. Do usunięcia klucza służy polecenie `DEL`, co pokazałem poniżej.

```
127.0.0.1:6379> SET total 1
OK
127.0.0.1:6379> DEL total
(integer) 1
127.0.0.1:6379> GET total
(nil)
```

Powyżej zaprezentowałem jedynie podstawowe polecenia służące do pracy z kluczami. Omówienie wszystkich poleceń bazy danych Redis znajdziesz na stronie *https://redis.io/commands/* and all Redis data types at *https://redis.io/docs/manual/data-types/*.

Użycie bazy danych Redis z Pythonem

Do pracy z bazą danych Redis w Pythonie potrzebujemy modułu definiującego tzw. powiązanie (ang. *binding*). Zainstaluj więc moduł `redis-py` za pomocą menedżera `pip`, wydając polecenie:

```
python -m pip install redis==5.0.4
```

Dokumentację modułu `redis-py` znajdziesz na stronie *https://redis-py.readthedocs.org/*.

Pakiet `redis-py` pozwala na interakcję z bazą Redis za pomocą interfejsu Pythona korzystającego ze składni poleceń Redis. Przejdź do powłoki Pythona za pomocą polecenia:

```
python manage.py shell
```

Następnie uruchom polecenie:

```
>>> import redis
>>> r = redis.Redis(host='localhost', port=6379, db=0)
```

Ten kod tworzy połączenie z bazą danych Redis, w której poszczególne bazy danych są identyfikowane za pomocą indeksu w postaci liczby całkowitej zamiast nazw baz danych. Liczba dostępnych dla klienta baz danych wynosi 16, ale tę wartość można zmienić w pliku konfiguracyjnym `redis.conf`.

Teraz zdefiniuj klucz za pomocą powłoki Pythona.

```
>>> r.set('foo', 'bar')
True
```

Polecenie zwróciło `True`, co wskazuje na zakończoną powodzeniem operację utworzenia klucza. Wartość klucza można pobrać za pomocą polecenia `get()`.

```
>>> r.get('foo')
b'bar'
```

Jak możesz zobaczyć, metody klasy `Redis` stosują składnię poleceń bazy danych Redis.

Przystępujemy teraz do integracji bazy danych Redis z naszym projektem. Przeprowadź edycję pliku *settings.py* projektu `bookmarks` i umieść w nim poniższe ustawienia.

```
REDIS_HOST = 'localhost'
REDIS_PORT = 6379
REDIS_DB = 0
```

Są to ustawienia przeznaczone dla serwera Redis i bazy danych, której będziemy używać w naszym projekcie.

Przechowywanie widoków obrazów w bazie danych Redis

Umieścimy w bazie danych całkowitą liczbę wyświetleń danego obrazu. Jeżeli do tego celu użyjemy warstwy ORM w Django, skutkiem będzie wykonanie polecenia SQL `UPDATE` w trakcie każdego wyświetlenia danego obrazu.

Natomiast w przypadku użycia bazy Redis po prostu inkrementujemy wartość licznika przechowywanego w pamięci, co oznacza znacznie lepszą wydajność całej operacji.

Przeprowadź edycję pliku *views.py* aplikacji `images` i umieść w nim, za instrukcjami `import`, poniższy fragment kodu.

```
import redis
from django.conf import settings

# Nawiązanie połączenia z bazą danych Redis
r = redis.Redis(
    host=settings.REDIS_HOST,
    port=settings.REDIS_PORT,
    db=settings.REDIS_DB
)
```

W powyższym fragmencie kodu nawiązaliśmy połączenie z bazą danych Redis, z której później będziemy korzystać w widokach. Przeprowadź edycję pliku *views.py* aplikacji `images` i zmodyfikuj widok `image_detail` w sposób pokazany na poniższym listingu. Nowy kod został wyróżniony pogrubioną czcionką.

```
def image_detail(request, id, slug):
    image = get_object_or_404(Image, id=id, slug=slug)
    # Inkrementacja o 1 całkowitej liczby wyświetleń danego obrazu
    total_views = r.incr(f'image:{image.id}:views')
    return render(
        request,
        'images/image/detail.html',
       {
```

```
            'section': 'images',
            'image': image,
            'total_views': total_views
        }
    )
```

W tym widoku używamy polecenia `incr`, które inkrementuje wartość klucza o 1. Jeśli klucz nie istnieje, polecenie `incr` go tworzy. Metoda `incr()` zwraca wartość klucza po przeprowadzeniu operacji. Następnie tę zwróconą wartość umieszczamy w zmiennej `total_views` i przekazujemy do kontekstu szablonu. Do utworzenia klucza Redis użyliśmy notacji *typ-obiektu*:*id*:*kolumna*, np. `image:33:id`.

Uwaga

Podczas nadawania nazw kluczom konwencją w bazie danych Redis jest użycie dwukropka jako separatora pomiędzy przestrzenią nazw a kluczem. Dzięki temu nazwy kluczy mogą być szczegółowo opisane, a powiązane klucze współdzielą w nazwie część tego samego schematu.

Przeprowadź edycję szablonu *images/image/detail.html* aplikacji `images` i dodaj poniższy fragment kodu HTML wyróżniony pogrubioną czcionką.

```
...
<div class="image-info">
  <div>
    <span class="count">
       <span class="total">{{ total_likes }}</span>
       like{{ total_likes|pluralize }}
    </span>
    <span class="count">
      {{ total_views }} wyświetleń
    </span>
    <a href="#" data-id="{{ image.id }}" data-action="{% if request.user in
    ↪users_like %}nie{% endif %} lubię"
     class="like button">
       {% if request.user not in users_like %}
        lubię
      {% else %}
        nie lubię
      {% endif %}
    </a>
  </div>
  {{ image.description|linebreaks }}
</div>
...
```

Uruchom serwer programistyczny za pomocą polecenia:

```
python manage.py runserver
```

Teraz w przeglądarce internetowej otwórz stronę widoku szczegółowego obrazu i wczytaj go kilkakrotnie. Powinieneś zobaczyć, że w trakcie każdego wykonania widoku następuje inkrementacja o 1 liczby wskazującej całkowitą liczbę wyświetleń danego obrazu (rysunek 7.15).

Rysunek 7.15. Strona szczegółów obrazu zawierająca liczbę polubień i wyświetleń

Doskonale! W ten sposób z powodzeniem zintegrowałeś z projektem bazę danych Redis, w której przechowujesz całkowitą liczbę wyświetleń danego obrazu. W następnym punkcie dowiesz się, jak za pomocą bazy danych Redis zbudować ranking najczęściej oglądanych obrazów.

Przechowywanie rankingu w bazie danych Redis

Zaimplementujemy jeszcze inną, nieco bardziej złożoną, funkcjonalność opartą na bazie danych Redis. Przygotujemy ranking obrazów najczęściej wyświetlanych na naszej platformie. W celu zbudowania tego rodzaju rankingu wykorzystamy posortowane zbiory Redis. Posortowany zbiór to niepowtarzająca się kolekcja ciągów tekstowych, w których każdemu elementowi jest przypisana liczba. Elementy są posortowane według wartości tej liczby.

Przeprowadź edycję pliku *views.py* aplikacji `images` i umieść w widoku `image_detail` poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
def image_detail(request, id, slug):
    image = get_object_or_404(Image, id=id, slug=slug)
    # Inkrementacja o 1 całkowitej liczby wyświetleń danego obrazu
    total_views = r.incr(f'image:{image.id}:views')
    # Inkrementacja o 1 rankingu danego obrazu
    r.zincrby('image_ranking', 1, image.id)
    return render(
        request,
        'images/image/detail.html',
        {
            'section': 'images',
            'image': image,
            'total_views': total_views
        }
    )
```

Do przechowywania w posortowanym zbiorze liczby wyświetleń obrazu zapisanej w kluczu o nazwie `image:ranking` wykorzystujemy wywołanie `zincrby()`. Przechowujemy identyfikator obrazu oraz wartość 1, która zostanie dodana do wartości całkowitej dla tego elementu w posortowanym zbiorze. W ten sposób możemy monitorować globalną liczbę wyświetleń obrazu i mieć posortowany zbiór, którego elementy są ułożone w kolejności liczby wyświetleń.

Przystępujemy teraz do utworzenia widoku przeznaczonego do wyświetlenia rankingu najczęściej oglądanych obrazów. W pliku *views.py* aplikacji `images` umieść poniższy fragment kodu.

```
@login_required
def image_ranking(request):
    # Pobranie słownika rankingu obrazów
    image_ranking = r.zrange(
        'image_ranking', 0, -1,
        desc=True
    )[:10]
    image_ranking_ids = [int(id) for id in image_ranking]
    # Pobranie najczęściej wyświetlanych obrazów
    most_viewed = list(
        Image.objects.filter(
            id__in=image_ranking_ids
        )
    )
    most_viewed.sort(key=lambda x: image_ranking_ids.index(x.id))
    return render(
        request,
        'images/image/ranking.html',
        {'section': 'images', 'most_viewed': most_viewed}
    )
```

To jest widok `image_ranking`. Jego działanie opisałem poniżej.

1. Za pomocą wywołania `zrange()` pobieramy elementy w posortowanym zbiorze. Wywołanie to oczekuje podania zakresu zdefiniowanego przez najmniejszą i największą wartość. Użycie wartości `0` dla najmniejszej wartości i `–1` dla największej powoduje, że Redis zwraca wszystkie elementy w posortowanym zbiorze. Podaliśmy również opcję `desc=True` w celu pobrania elementów ułożonych w kolejności malejącej. Na koniec notacja `[:10]` powoduje pobranie jedynie dziesięciu pierwszych elementów o największej wartości.
2. Budujemy listę zwróconych identyfikatorów obrazów i przechowujemy ją w zmiennej `image_ranking_ids` w postaci listy liczb całkowitych. Pobieramy obiekty `Image`, których identyfikatory są przechowywane w wymienionej zmiennej i wymuszamy wykonanie zapytania przez użycie funkcji `list()`. Bardzo ważne jest wymuszenie wykonania kolekcji `QuerySet`, ponieważ następnie wywołamy metodę `sort()` — w tym momencie konieczna jest lista obiektów zamiast kolekcji `QuerySet`.
3. Obiekty `Image` są sortowane według indeksu występowania w rankingu obrazów. Teraz możemy już wykorzystać listę `most_viewed` w szablonie do przedstawienia dziesięciu najczęściej wyświetlanych obrazów.

Utwórz nowy plik szablonu o nazwie *ranking.html* w katalogu *images/image* i umieść w nim poniższy fragment kodu.

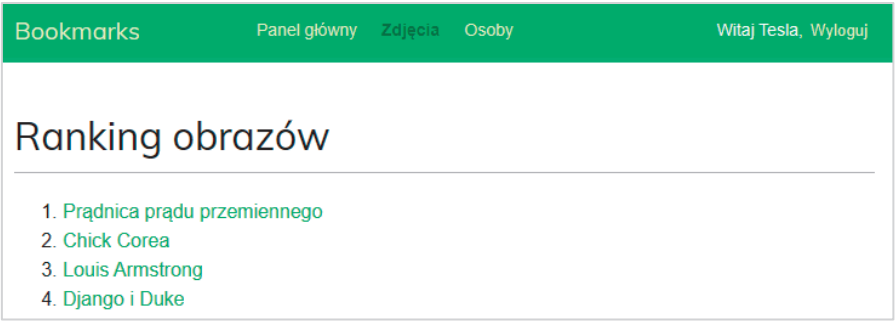
```
{% extends "base.html" %}

{% block title %}Ranking obrazów{% endblock %}

{% block content %}
  <h1>Ranking obrazów</h1>
  <ol>
    {% for image in most_viewed %}
      <li>
        <a href="{{ image.get_absolute_url }}">
          {{ image.title }}
        </a>
      </li>
    {% endfor %}
  </ol>
{% endblock %}
```

Szablon jest całkiem prosty, przeprowadzamy jedynie iterację przez obiekty `Image` znajdujące się na liście `most_viewed` i wyświetlamy ich nazwy razem z linkami do strony szczegółów.

Na koniec trzeba jeszcze utworzyć wzorzec adresu URL dla nowego widoku. Przeprowadź edycję pliku *urls.py* aplikacji `images` i dodaj poniższy wzorzec adresu URL oznaczony pogrubioną czcionką.

```
urlpatterns = [
     path('create/', views.image_create, name='create'),
     path('detail/<int:id>/<slug:slug>/',
          views.image_detail, name='detail'),
     path('like/', views.image_like, name='like'),
     path('', views.image_list, name='list'),
     path('ranking/', views.image_ranking, name='ranking'),
]
```

Uruchom serwer programistyczny. Przejdź w przeglądarce do Twojej witryny i wielokrotnie załaduj stronę szczegółów dla różnych obrazów. Następnie przejdź pod adres *http://127.0.0.1:8000/images/ranking/*. Powinieneś zobaczyć ranking obrazów, podobny do pokazanego na rysunku 7.16.



Rysunek 7.16. Strona rankingu zbudowana na podstawie danych pobranych z bazy danych Redis

Doskonale! Właśnie utworzyłeś ranking z wykorzystaniem bazy danych Redis.

Kolejne kroki z bazą danych Redis

Baza danych Redis nie jest zamiennikiem dla bazy danych SQL, ale charakteryzującym się dużą szybkością działania magazynem danych w pamięci, odpowiednim do wykonywania określonych zadań. Dodaj ją do swojego stosu i korzystaj z niej, gdy czujesz, że tak trzeba. Poniżej wymieniłem kilka scenariuszy, w których baza danych Redis sprawdza się całkiem dobrze.

- **Zliczanie.** Jak mogłeś zobaczyć, za pomocą bazy danych Redis bardzo łatwo można zarządzać licznikami. Do dyspozycji masz funkcje `incr()` i `incrby()`.
- **Przechowywanie ostatnich elementów.** Za pomocą funkcji `lpush()` i `rpush()` można dodawać elementy na początku i końcu listy. Do usunięcia i zwrócenia pierwszego lub ostatniego elementu służą funkcje — odpowiednio — `lpop()` i `rpop()`. Wielkość listy można zaś skrócić za pomocą funkcji `ltrim()`.
- **Kolejki.** Poza wywołaniami typu `push` i `pop`, Redis oferuje także blokujące polecenia kolejki.
- **Buforowanie.** Przy użyciu wywołań `expire()` i `expireat()` można wykorzystać Redis w charakterze bufora. Ponadto istnieją opracowane przez podmioty zewnętrzne rozwiązania pozwalające na użycie bufora Redis w Django.
- **Pub/sub**. Baza danych Redis oferuje polecenia przeznaczone do subskrypcji i rezygnacji z subskrypcji wysyłania komunikatów do kanałów wiadomości.
- **Rankingi i tablice wyników.** Posortowane zbiory Redis wraz z wartościami niezwykle ułatwiają tworzenie różnorodnych tablic wyników.
- **Monitorowanie w czasie rzeczywistym.** Bardzo szybkie operacje wejścia-wyjścia w bazie danych Redis czynią z niej doskonałe rozwiązanie w scenariuszach wymagających działania w czasie rzeczywistym.

Podsumowanie

W tym rozdziale przygotowałeś system obserwacji użytkownika z wykorzystaniem relacji „wiele do wielu" oraz modelu pośredniego. Utworzyłeś również strumień aktywności użytkownika z wykorzystaniem relacji generycznych oraz zoptymalizowałeś zestawy `QuerySet` do pobierania powiązanych obiektów. Dowiedziałeś się, jak Django korzysta z sygnałów. Napisałeś funkcję odbiorcy sygnału w celu denormalizacji powiązanych liczników obiektów. W rozdziale omówiłem także klasy konfiguracyjne aplikacji, które wykorzystałeś do załadowania procedur obsługi sygnałów. Ponadto dodałeś do swojego projektu pasek narzędzi *Django Debug Toolbar*. Dowiedziałeś się również, jak zainstalować i skonfigurować bazę danych Redis w projekcie Django. Na koniec skorzystałeś z bazy danych Redis w celu zapisania widoków elementów oraz opracowania rankingu obrazów.

W następnym rozdziale dowiesz się, jak zbudować sklep internetowy. Przygotujesz katalog produktów oraz zbudujesz koszyk na zakupy wykorzystujący mechanizm sesji.

Dowiesz się także, jak tworzyć niestandardowe procesory kontekstu. Nauczysz się ponadto zarządzać zamówieniami klientów i wysyłać asynchroniczne powiadomienia za pomocą Celery i RabbitMQ.

Rozszerzanie projektu przy użyciu sztucznej inteligencji

W tym podrozdziale zaprezentuję zadanie rozszerzenia projektu, do którego wykorzystam przykładowe zapytanie do ChatGPT. ChatGPT jest dostępny pod adresem *https://chat.openai.com/*. Jeśli jest to Twoja pierwsza interakcja z ChatGPT, możesz powrócić do podrozdziału „Rozszerzanie projektu z wykorzystaniem sztucznej inteligencji" w rozdziale 3., „Rozbudowa aplikacji bloga".

W tym przykładzie projektu nauczyłeś się, jak używać sygnałów Django, i z powodzeniem zaimplementowałeś odbiornik sygnału, by aktualizował całkowitą liczbę polubień obrazu za każdym razem, gdy nastąpi zmiana w liczbie polubień. Spróbujemy teraz zastosować ChatGPT do zbadania implementacji odbiornika sygnału, który automatycznie wygeneruje powiązany obiekt `Profile` zawsze wtedy, gdy jest tworzony obiekt `User`. W tym celu możesz wykorzystać zapytanie dostępne pod adresem *https://github.com/PacktPublishing/Django-5-by-example/blob/main/Chapter07/prompts/task.md*.

Po pomyślnym zaimplementowaniu odbiornika sygnału możesz usunąć kroki ręcznego tworzenia profilu, które wcześniej były zawarte w widoku `register` aplikacji `account` i potoku uwierzytelniania społecznościowego. Dzięki funkcji odbiornika dołączonej teraz do sygnału `post_save` modelu `User` dla nowych użytkowników będą automatycznie tworzone profile.

Uwaga

Jeśli masz problemy ze zrozumieniem konkretnego pojęcia lub tematu w książce, poproś ChatGPT o dostarczenie dodatkowych przykładów lub wyjaśnienie pojęcia. Dzięki takiemu zindywidualizowanemu podejściu można przyspieszyć naukę, a zrozumienie złożonych tematów staje się łatwiejsze.

Dodatkowe zasoby

Oto lista zasobów zawierających dodatkowe informacje związane z tematami omawianymi w tym rozdziale:

- Kod źródłowy przykładów zamieszczonych w tym rozdziale — *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter07.*
- Niestandardowe modele użytkownika — *https://docs.djangoproject.com/en/5.0/topics/auth/customizing/#specifying-a-custom-user-model*.
- Framework `contenttypes` — *https://docs.djangoproject.com/en/5.0/ref/contrib/contenttypes/*.

- Wbudowane sygnały Django — *https://docs.djangoproject.com/en/5.0/ref/signals/.*
- Klasy konfiguracyjne aplikacji — *https://docs.djangoproject.com/en/5.0/ref/applications/.*
- Dokumentacja paska narzędzi Django Debug Toolbar — *https://django-debug-toolbar.readthedocs.io/.*
- Tworzone przez firmy zewnętrzne panele paska narzędzi Django Debug Toolbar — *https://django-debug-toolbar.readthedocs.io/en/latest/panels.html#third-party-panels.*
- Magazyn danych w pamięci Redis — *https://redis.io/*.
- Oficjalny obraz Docker bazy danych Redis — *https://hub.docker.com/_/redis*.
- Opcje pobierania bazy danych Redis — *https://redis.io/download/*.
- Polecenia Redis — *https://redis.io/commands/*.
- Typy danych Redis — *https://redis.io/docs/manual/data-types/*.
- Dokumentacja modułu `redis-py` — *https://redis-py.readthedocs.io/*.

Utworzenie sklepu internetowego

Rozdział 8

W poprzednim rozdziale utworzyłeś system obserwacji użytkownika oraz strumień aktywności użytkownika. Dowiedziałeś się również, jak działają sygnały w Django oraz jak zintegrować z projektem bazę danych Redis i przechowywać w niej całkowitą liczbę wyświetleń danego obrazu.

W tym rozdziale zbudujesz nowy projekt Django zawierający w pełni funkcjonalny sklep internetowy. W tym i w kolejnych dwóch rozdziałach pokażę, jak zbudować podstawowe funkcjonalności platformy e-commerce. Twój sklep internetowy umożliwi klientom przeglądanie produktów, dodawanie ich do koszyka oraz stosowanie kodów rabatowych. Zaimplementujesz również proces płatności obejmujący korzystanie z kart kredytowych oraz mechanizm generowania faktur. Dołączysz też silnik rekomendacji pozwalający na polecanie produktów Twoim klientom i zastosujesz mechanizmy lokalizacji pozwalające na oferowanie Twojej witryny w wielu językach.

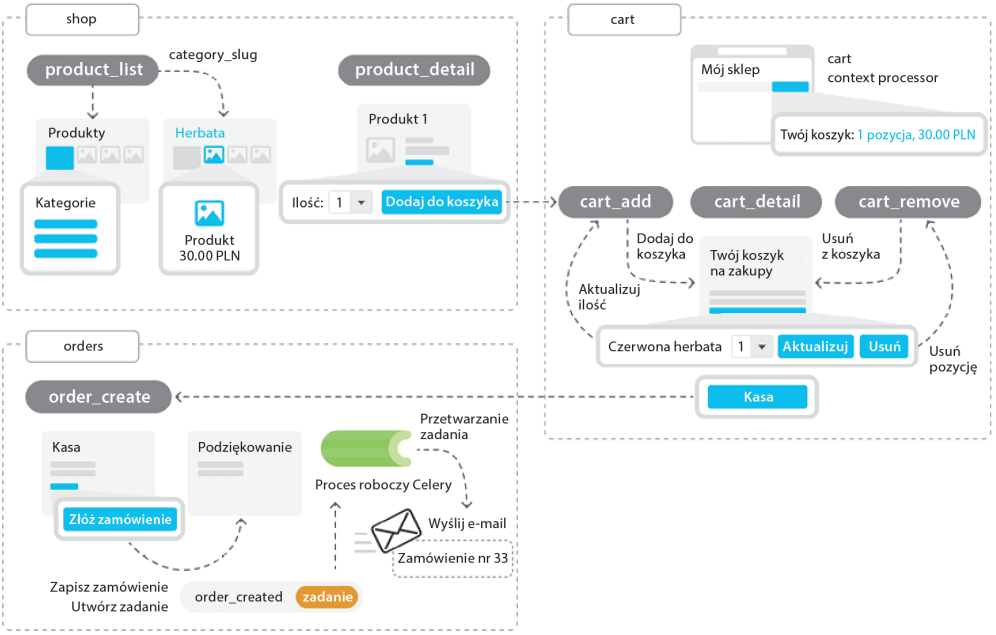
Oto zagadnienia, na których skoncentruję się w tym rozdziale:

- Tworzenie katalogu produktów
- Budowa koszyka na zakupy opartego na sesjach w Django
- Tworzenie niestandardowych procesorów kontekstu
- Zarządzanie zamówieniami klientów
- Konfigurowanie w projekcie Celery wraz z systemem RabbitMQ w roli brokera wiadomości
- Wysyłanie klientom asynchronicznych powiadomień za pomocą Celery
- Monitorowanie systemu Celery z wykorzystaniem systemu Flower

Przegląd funkcjonalności

Reprezentację widoków, szablonów i funkcjonalności, które zbudujemy w tym rozdziale, przedstawiono na rysunku 8.1.

W tym rozdziale zaimplementujesz widok `product_list`, przeznaczony do wyświetlania listy wszystkich produktów, oraz widok `product_detail`, do wyświetlania pojedynczego produktu. W widoku `product_list` zaimplementujesz również filtrowanie produktów według kategorii za pomocą parametru `category_slug`. Stworzysz koszyk na zakupy przy użyciu sesji i utworzysz widok `cart_detail`, aby wyświetlić elementy koszyka. Utworzysz też widok `cart_add`, przeznaczony do dodawania produktów do koszyka i aktualizowania



Rysunek 8.1. Schemat funkcjonalności zbudowanych w rozdziale 8.

ich ilości, oraz widok `cart_remove`, do usuwania produktów z koszyka. Aby wyświetlać liczbę pozycji w koszyku i ich łączny koszt w nagłówku witryny, zaimplementujesz procesor kontekstu szablonu koszyka. Oprócz tego utworzysz widok `order_create`, przeznaczony do składania zamówień, i użyjesz Celery do zaimplementowania asynchronicznego zadania `order_created`, wysyłającego do klientów, po złożeniu przez nich zamówienia, wiadomość e-mail z potwierdzeniem. Z tego rozdziału dowiesz się, jak zaimplementować w aplikacji sesje użytkownika oraz jak pracować z zadaniami asynchronicznymi. Jedno i drugie to bardzo często wykorzystywane umiejętności, które możesz zastosować w prawie każdym projekcie.

Kod źródłowy przykładów z tego rozdziału można znaleźć pod adresem *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter08*.

Wszystkie pakiety Pythona użyte w tym rozdziale są zawarte w pliku *requirements.txt* dołączonym do plików z kodem źródłowym przykładów z tego rozdziału. Aby zainstalować każdy pakiet Pythona, możesz postępować zgodnie z instrukcjami zamieszczonymi w poniższych podrozdziałach. Możesz także zainstalować wszystkie wymagania jednocześnie za pomocą polecenia `pip install -r requirements.txt`.

Utworzenie projektu sklepu internetowego

Przystępujemy do opracowania nowego projektu Django, w ramach którego zbudujemy sklep internetowy. Użytkownicy tego sklepu będą mogli przeglądać katalog produktów i dodawać produkty do koszyka na zakupy. Po wybraniu wszystkich interesujących produktów użytkownik będzie mógł złożyć zamówienie i sfinalizować zakup. Oto funkcjonalności sklepu internetowego, którymi zajmiemy się w tym rozdziale:

- Utworzenie modeli katalogu produktów, dodanie ich do witryny administracyjnej oraz przygotowanie podstawowych widoków przeznaczonych do wyświetlania katalogu.
- Opracowanie systemu koszyka na zakupy za pomocą sesji Django, co pozwoli użytkownikowi na zachowanie wybranych produktów podczas przeglądania zasobów witryny internetowej.
- Zbudowanie formularza i funkcjonalności, które pozwolą na składanie zamówień.
- Wysyłanie użytkownikowi asynchronicznego powiadomienia przez e-mail po złożeniu zamówienia.

Przejdź do powłoki i wydaj poniższe polecenia w celu utworzenia środowiska wirtualnego dla projektu wewnątrz katalogu */env*.

```
python -m venv env/myshop
```

Jeśli używasz systemu Linux lub macOS, aby aktywować środowisko wirtualne, uruchom polecenie:

```
source env/myshop/bin/activate
```

Jeśli używasz systemu Windows, użyj polecenia:

```
.\env\myshop\Scripts\activate
```

Znak zachęty w powłoce wyświetla nazwę aktywnego środowiska wirtualnego, co pokazałem poniżej.

```
(myshop)laptop:~ zenx$
```

W przygotowanym środowisku wirtualnym zainstaluj framework Django, wydając poniższe polecenie.

```
python -m pip install Django~=5.0.4
```

Utwórz nowy projekt o nazwie `myshop` wraz z aplikacją o nazwie `shop`. W tym celu przejdź do powłoki i wydaj poniższe polecenie.

```
django-admin startproject myshop
```

W ten sposób zbudowałeś początkową strukturę plików i katalogów. Teraz przejdź do nowego katalogu projektu i utwórz nową aplikację o nazwie `shop` poleceniami.

```
cd myshop/
django-admin startapp shop
```

Przeprowadź edycję pliku *settings.py* i dodaj do listy `INSTALLED_APPS` przed jakąkolwiek inną zainstalowaną aplikacją następujący wiersz wyróżniony pogrubioną czcionką.

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
```

```
    'django.contrib.staticfiles',
    'shop.apps.ShopConfig',
]
```

W tym momencie aplikacja jest aktywna w projekcie. Możemy przystąpić do zdefiniowania modeli potrzebnych do przygotowania katalogu produktów.

Utworzenie modeli katalogu produktów

Katalog budowanego sklepu internetowego będzie składał się z produktów pogrupowanych według różnych kategorii. Każdy produkt będzie miał nazwę, opcjonalny opis, opcjonalne zdjęcie, cenę oraz informacje o dostępności w magazynie.

Przeprowadź edycję pliku *models.py* utworzonej przed chwilą aplikacji `shop` i umieść w nim poniższy fragment kodu.

```
from django.db import models


class Category(models.Model):
    name = models.CharField(max_length=200)
    slug = models.SlugField(max_length=200, unique=True)

    class Meta:
        ordering = ['name']
        indexes = [
            models.Index(fields=['name']),
        ]
        verbose_name = 'category'
        verbose_name_plural = 'categories'

    def __str__(self):
        return self.name


class Product(models.Model):
    category = models.ForeignKey(
        Category,
        related_name='products',
        on_delete=models.CASCADE
    )
    name = models.CharField(max_length=200)
    slug = models.SlugField(max_length=200)
    image = models.ImageField(
        upload_to='products/%Y/%m/%d',
        blank=True
    )
    description = models.TextField(blank=True)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    available = models.BooleanField(default=True)
    created = models.DateTimeField(auto_now_add=True)
    updated = models.DateTimeField(auto_now=True)
```

```
    class Meta:
        ordering = ['name']
        indexes = [
            models.Index(fields=['id', 'slug']),
            models.Index(fields=['name']),
            models.Index(fields=['-created']),
        ]

    def __str__(self):
        return self.name
```

Są to modele `Category` i `Product`. Model `Category` składa się z kolumny `name` i unikatowej kolumny `slug`. W klasie `Meta` modelu `Category` zdefiniowaliśmy indeks dla pola `name`.

Z kolei w skład modelu `Product` wchodzą poniższe kolumny modelu.

- `category`. Klucz obcy (`ForeignKey`) dla modelu `Category`. Przedstawia relację typu „wiele do jednego”, ponieważ produkt zalicza się do jednej kategorii, w której może znajdować się wiele produktów.
- `name`. Nazwa produktu.
- `slug`. Slug dla produktu, służy do przygotowania wygodnego adresu URL.
- `image`. Opcjonalne zdjęcie produktu.
- `description`. Opcjonalny opis produktu.
- `price`. Do przechowywania liczby rzeczywistej o stałej precyzji. W tym polu używa się typu Pythona o nazwie `decimal.Decimal`. Maksymalna liczba cyfr (łącznie z tymi po przecinku dziesiętnym) jest określana za pomocą atrybutu `max_digits`, natomiast maksymalna liczba cyfr po przecinku dziesiętnym jest określana przez atrybut `decimal_places`.
- `available`. Wartość typu `Boolean` wskazująca, czy dany produkt jest dostępny. Dzięki tej wartości zyskujemy możliwość wyświetlania lub ukrywania danego produktu w katalogu.
- `created`. Pole zawiera datę i godzinę utworzenia danego obiektu.
- `updated`. Pole zawiera datę i godzinę ostatniej modyfikacji danego obiektu.

Aby uniknąć problemów związanych z zaokrąglaniem wartości, dla pola `price` używamy typu `DecimalField` zamiast `FloatField`.

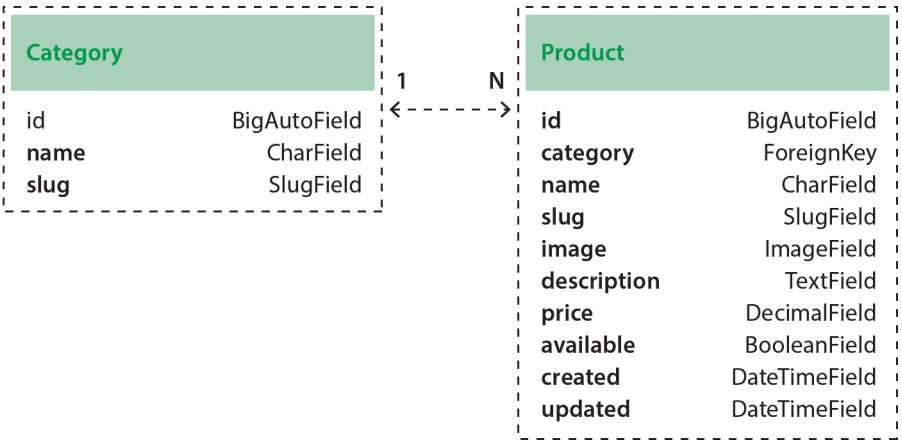
Uwaga

Do przechowywania wartości pieniężnych zawsze używaj kolumny typu `DecimalField`. Typ `FloatField` wewnętrznie opiera się na typie `float` Pythona, podczas gdy `Decimal` ↪`Field` bazuje na typie `Decimal` Pythona. Gdy korzystasz z typu `Decimal`, unikasz problemów z zaokrąglaniem wartości, jakie nierozerwalnie wiążą się z typem `float`.

W klasie `Meta` modelu `Product` zastosowaliśmy indeks złożony z wielu pól dla kolumn id i `slug`. Wspólne zindeksowanie obu kolumn powoduje, że następuje wyraźna poprawa wydajności zapytań wykorzystujących kolumny `id` i `slug`.

Planujemy wyszukiwać produkty zarówno według identyfikatora, jak i slugu. Dodaliśmy indeks dla pola `name` oraz indeks dla pola `created`. Aby zdefiniować indeks w porządku malejącym, przed nazwą pola należy wprowadzić znak minus.

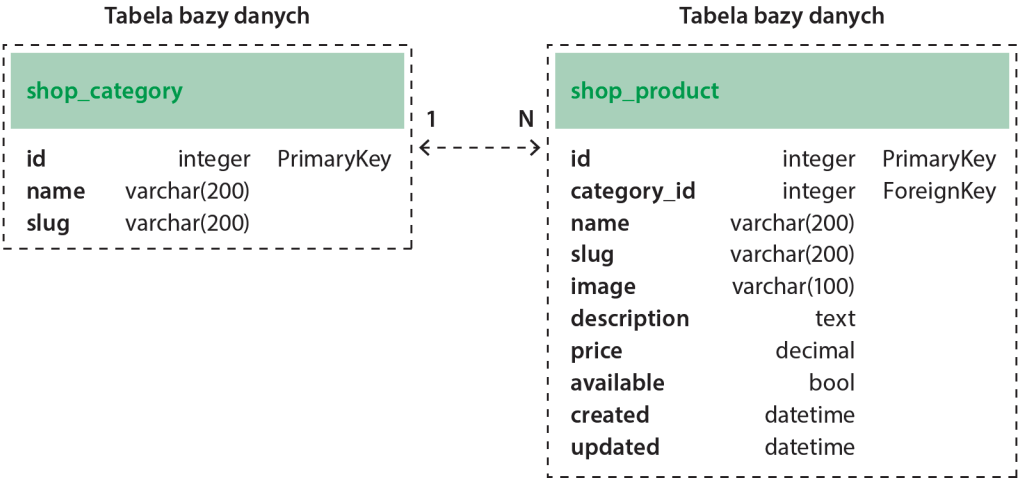
Dwa utworzone modele danych są pokazane na rysunku 8.2.



Rysunek 8.2. Modele katalogu produktów

Na rysunku 8.2 można zobaczyć różne pola modeli danych oraz relację jeden-do-wielu między modelami `Category` i `Product`.

Wynikiem tych modeli będą tabele bazy danych pokazane na rysunku 8.3.



Rysunek 8.3. Tabele bazy danych dla modeli katalogu produktów

Za pomocą pola `category_id` w tabeli `shop_product`, służącego do przechowywania identyfikatora powiązanej kategorii dla każdego obiektu `Product`, została zdefiniowana relacja jeden-do-wielu między obiema tabelami.

Przystępujemy do przygotowania początkowych migracji dla aplikacji `shop`. Ponieważ w modelach katalogu produktów będziemy obsługiwać zdjęcia, zainstalujemy bibliotekę `Pillow`. Sposób instalowania biblioteki `Pillow` do zarządzania obrazami omówiłem w rozdziale 4., „Utworzenie witryny społecznościowej". Otwórz powłokę i zainstaluj bibliotekę `Pillow` za pomocą polecenia:

```
python -m pip install Pillow==10.3.0
```

Następnie wydaj poniższe polecenie w celu przygotowania początkowych migracji projektu.

```
python manage.py makemigrations
```

Wyświetli się następujący wynik.

```
Migrations for 'shop':
  shop/migrations/0001_initial.py
    - Create model Category
    - Create model Product
```

W celu synchronizacji aplikacji z bazą danych wydaj polecenie:

```
python manage.py migrate
```

Powinieneś otrzymać dane wyjściowe zawierające m.in. poniższy wiersz.

```
Applying shop.0001_initial... OK
```

Teraz baza danych jest zsynchronizowana z modelami.

Rejestracja modeli katalogu w witrynie administracyjnej

Przygotowane modele dodamy do witryny administracyjnej, aby zyskać możliwość łatwego zarządzania kategoriami i produktami. Przeprowadź edycję pliku *admin.py* aplikacji `shop` i umieść w nim poniższy fragment kodu.

```
from django.contrib import admin
from .models import Category, Product


@admin.register(Category)
class CategoryAdmin(admin.ModelAdmin):
    list_display = ['name', 'slug']
    prepopulated_fields = {'slug': ('name',)}


@admin.register(Product)
class ProductAdmin(admin.ModelAdmin):
    list_display = [
        'name',
        'slug',
        'price',
        'available',
        'created',
        'updated',
    ]
```

```
    list_filter = ['available', 'created', 'updated']
    list_editable = ['price', 'available']
    prepopulated_fields = {'slug': ('name',)}
```

Pamiętaj, że użyliśmy atrybutu `prepopulated_fields` w celu wskazania pól, których wartość będzie wstawiana automatycznie na podstawie wartości innych kolumn. Jak przekonałeś się wcześniej, tego rodzaju rozwiązanie okazuje się wygodne podczas generowania slugów.

Atrybutu `list_editable` w klasie `ProductAdmin` używamy do wskazania kolumn, które mogą być edytowane z poziomu listy wyświetlanej na stronie witryny administracyjnej. Tym samym zyskujesz możliwość jednoczesnej edycji wielu rekordów. Każde pole na liście `list_editable` musi być również wymienione w atrybucie `list_display`, ponieważ edytowane mogą być jedynie wyświetlone pola.

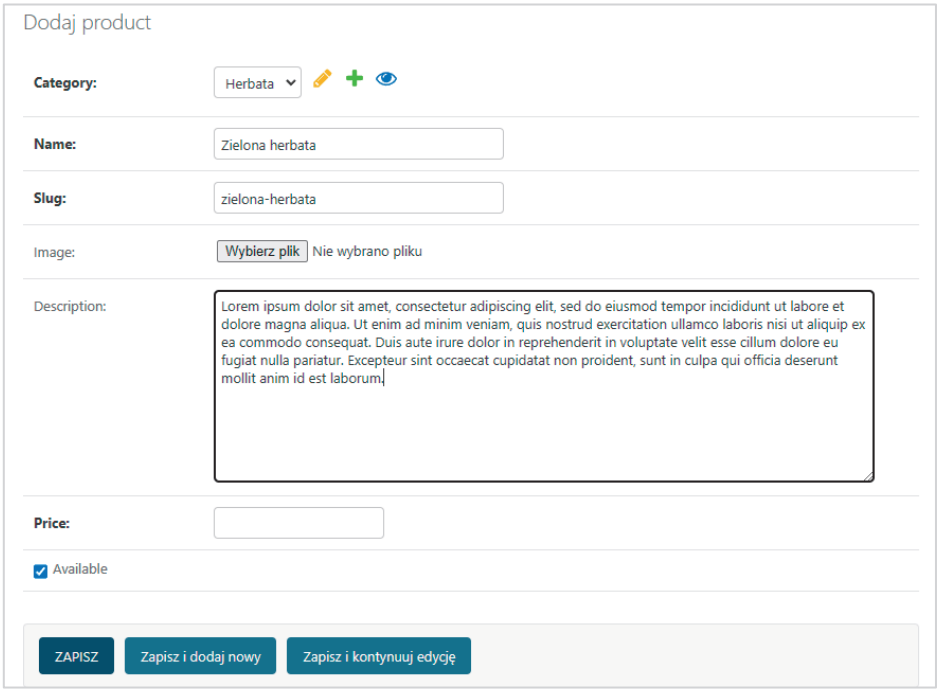
Teraz za pomocą poniższego polecenia utwórz superużytkownika.

```
python manage.py createsuperuser
```

Podaj wybraną nazwę użytkownika, adres e-mail i hasło. Uruchom serwer programistyczny za pomocą polecenia:

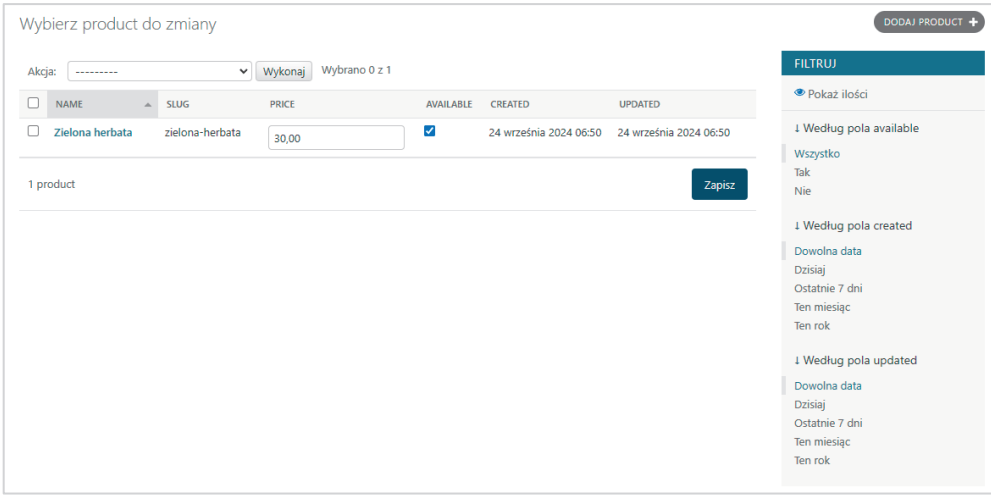
```
python manage.py runserver
```

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/admin/shop/product/`
`↪add/` i zaloguj się jako użytkownik, którego konto przed chwilą utworzyłeś. Następnie za pomocą interfejsu witryny administracyjnej dodaj nową kategorię i produkt. Formularz dodawania produktu powinien wyglądać jak na rysunku 8.4.



Rysunek 8.4. Formularz dodawania produktu

Kliknij przycisk *ZAPISZ*. Strona produktów w witrynie administracyjnej Django zmieni się i będzie podobna do pokazanej na rysunku 8.5.



Rysunek 8.5. Strona z listą zmian produktu

Utworzenie widoków katalogu

Aby można było wyświetlić katalog produktów, trzeba utworzyć widok przeznaczony do pokazania wszystkich produktów lub ich filtrowania na podstawie wybranej kategorii. Przeprowadź edycję pliku *views.py* aplikacji `shop` i umieść w nim poniższy fragment kodu wyróżniony pogrubioną czcionką:

```
from django.shortcuts import get_object_or_404, render
from .models import Category, Product


def product_list(request, category_slug=None):
    category = None
    categories = Category.objects.all()
    products = Product.objects.filter(available=True)
    if category_slug:
        category = get_object_or_404(Category, slug=category_slug)
        products = products.filter(category=category)
    return render(
        request,
        'shop/product/list.html',
        {
            'category': category,
            'categories': categories,
            'products': products,
        },
    )
```

Za pomocą opcji `available=True` kolekcji `QuerySet` filtrujemy produkty w celu pobrania jedynie dostępnych. Wykorzystujemy opcjonalny parametr `category_slug` do ewentualnego filtrowania produktów na podstawie danej kategorii.

Konieczne jest również przygotowanie widoku przeznaczonego do pobrania i wyświetlenia pojedynczego produktu. Dodaj kod poniższego widoku do pliku *views.py*.

```
def product_detail(request, id, slug):
    product = get_object_or_404(
        Product, id=id, slug=slug, available=True
    )
    return render(
        request,
        'shop/product/detail.html',
        {'product': product},
    )
```

Widok `product_detail` oczekuje podania parametrów `id` i `slug` w celu pobrania egzemplarza modelu `Product`. Egzemplarz pobieramy za pomocą jedynie identyfikatora, ponieważ to jest unikatowy atrybut. Jednak w adresie URL dołączamy także slug, aby dla produktu wygenerować adres URL, który będzie przyjazny mechanizmom SEO.

Po przygotowaniu widoków listy produktów i szczegółów produktu kolejnym krokiem jest zdefiniowanie dla nich wzorców adresów URL. Utwórz nowy plik w katalogu aplikacji `shop` i nadaj mu nazwę *urls.py*. Następnie w pliku umieść poniższy fragment kodu.

```
from django.urls import path
from . import views

app_name = 'shop'

urlpatterns = [
    path('', views.product_list, name='product_list'),
    path(
        '<slug:category_slug>/',
        views.product_list,
        name='product_list_by_category',
    ),
    path(
        '<int:id>/<slug:slug>/',
        views.product_detail,
        name='product_detail',
    ),
]
```

Są to wzorce adresów URL dla przygotowywanego tutaj katalogu produktów. Zdefiniowaliśmy dwa różne wzorce adresów URL dla widoku `product_list`. Pierwszy z nich nosi nazwę `product_list` i wywołuje widok `product_list` bez żadnych parametrów. Natomiast drugi nosi nazwę `product_list_by_category` i dostarcza widokowi parametr `category_slug` pozwalający na filtrowanie produktów według danej kategorii. Ponadto dodaliśmy wzorzec dla widoku `product_detail`, który przekazuje widokowi parametry `id` i `slug` pozwalające na pobranie konkretnego produktu.

Przeprowadź edycję pliku *urls.py* projektu `myshop` w następujący sposób.

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('shop.urls', namespace='shop')),
]
```

W głównych wzorcach adresów URL dla projektu umieścimy jeszcze adres URL dla aplikacji `shop`, przy czym wykorzystamy własną przestrzeń nazw `'shop'`.

Teraz w pliku *models.py* aplikacji `shop` zaimportuj funkcję `reverse()` i dodaj do modeli `Category` i `Product` metodę `get_absolute_url()`, co pokazałem poniżej. Nowy kod został wyróżniony pogrubioną czcionką.

```
from django.db import models
from django.urls import reverse


class Category(models.Model):
    # ...
    def get_absolute_url(self):
        return reverse(
            'shop:product_list_by_category', args=[self.slug]
        )


class Product(models.Model):
    # ...

    def get_absolute_url(self):
        return reverse('shop:product_detail', args=[self.id, self.slug])
```

Jak już doskonale wiesz, metoda `get_absolute_url()` to konwencja służąca do pobierania adresu URL konkretnego obiektu. W omawianym przykładzie będziemy używać zdefiniowanych w pliku *urls.py* wzorców adresów URL.

Utworzenie szablonów katalogu

Kolejnym krokiem jest przygotowanie szablonów dla widoków listy produktów i szczegółów produktu. Dlatego też w katalogu aplikacji `shop` utwórz pokazaną poniżej strukturę katalogów i plików.

```
templates/
    shop/
        base.html
        product/
            list.html
            detail.html
```

Musimy zdefiniować szablon bazowy, który następnie rozszerzymy w szablonach listy produktów i szczegółów produktu. Przeprowadź edycję szablonu *shop/base.html* i umieść w nim poniższy fragment kodu.

```
{% load static %}
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>{% block title %}Mój sklep{% endblock %}</title>
    <link href="{% static "css/base.css" %}" rel="stylesheet">
  </head>
  <body>
    <div id="header">
      <a href="/" class="logo">Mój sklep</a>
    </div>
    <div id="subheader">
      <div class="cart">
            Koszyk jest pusty.
      </div>
    </div>
    <div id="content">
      {% block content %}
      {% endblock %}
    </div>
  </body>
</html>
```

To jest szablon bazowy, z którego będziemy korzystać w budowanym sklepie internetowym. Aby dołączyć niezbędne style CSS i obrazy używane przez szablony, w materiałach przygotowanych dla książki odszukaj katalog plików statycznych (*static*) w katalogu aplikacji `shop`. Skopiuj go i umieść w katalogu głównym budowanego projektu. Zawartość katalogu znajdziesz pod adresem *https://github.com/PacktPublishing/Django-5-by-Example/tree/main/Chapter08/myshop/shop/static*.

Przeprowadź edycję szablonu *shop/product/list.html* i umieść w nim poniższy fragment kodu.

```
{% extends "shop/base.html" %}
{% load static %}

{% block title %}
  {% if category %}{{ category.name }}{% else %}Produkty{% endif %}
{% endblock %}

{% block content %}
  <div id="sidebar">
    <h3>Kategorie</h3>
    <ul>
      <li {% if not category %}class="selected"{% endif %}>
        <a href="{% url "shop:product_list" %}">Wszystkie</a>
      </li>
      {% for c in categories %}
        <li {% if category.slug == c.slug %}class="selected"
        {% endif %}>
          <a href="{{ c.get_absolute_url }}">{{ c.name }}</a>
        </li>
      {% endfor %}
```

```
    </ul>
  </div>
  <div id="main" class="product-list">
    <h1>{% if category %}{{ category.name }}{% else %}Produkty
    {% endif %}</h1>
    {% for product in products %}
      <div class="item">
        <a href="{{ product.get_absolute_url }}">
          <img src="{% if product.image %}{{ product.image.url }}{% else %}{% static
          ↪"img/no_image.png" %}{% endif %}">
        </a>
        <a href="{{ product.get_absolute_url }}">{{ product.name }}</a>
        <br>
        {{ product.price }} PLN
      </div>
    {% endfor %}
  </div>
{% endblock %}
```

Zadbaj o to, aby żaden znacznik szablonu nie obejmował więcej niż jednego wiersza.

To jest szablon listy produktów. Rozszerza szablon bazowy *shop/base.html* i wykorzystuje zmienną kontekstu `categories` w celu wyświetlenia w pasku bocznym wszystkich dostępnych kategorii. Ten sam szablon jest wykorzystywany w dwóch celach — do wyświetlenia listy wszystkich dostępnych produktów i do wyświetlenia listy produktów filtrowanych według danej kategorii. Ponieważ pole `image` modelu `Product` może być puste, konieczne jest dostarczenie obrazu domyślnego dla produktów, dla których nie zdefiniowano obrazu. Wspomniany plik obrazu znajduje się w katalogu plików statycznych, a jego względna ścieżka dostępu to *img/no_image.png*.

Ponieważ do przechowywania obrazów użyliśmy kolumny typu `ImageField`, serwer programistyczny musi dostarczać przekazane do niego pliki obrazów.

Przeprowadź więc edycję pliku *settings.py* aplikacji `myshop` i umieść w nim poniższe ustawienia.

```
MEDIA_URL = 'media/'
MEDIA_ROOT = BASE_DIR / 'media'
```

Zmienna `MEDIA_URL` określa bazowy adres URL dla plików multimedialnych przekazanych przez użytkowników. Natomiast zmienna `MEDIA_ROOT` podaje lokalną ścieżkę dostępu do wspomnianych plików.

Aby framework Django dostarczał za pomocą serwera programistycznego pliki multimedialne przekazane przez użytkowników, przeprowadź edycję głównego pliku *urls.py* aplikacji `myshop` i dodaj wiersze oznaczone na poniższym listingu pogrubioną czcionką.

```
from django.conf import settings
from django.conf.urls.static import static
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('shop.urls', namespace='shop')),
```

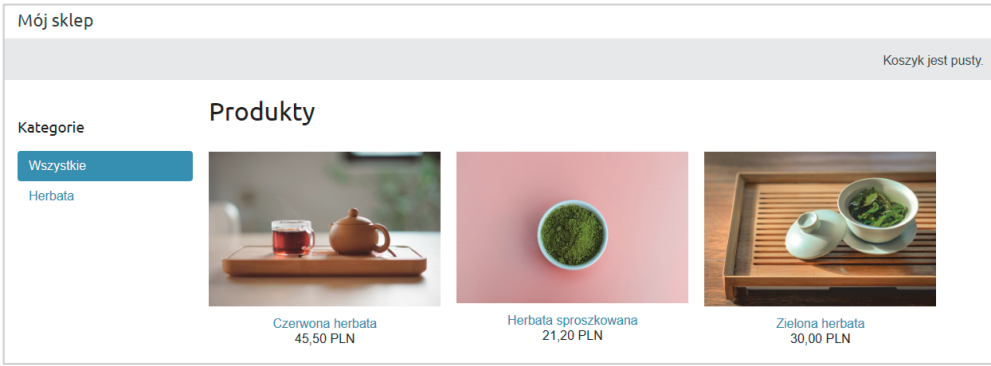
```
]

if settings.DEBUG:
    urlpatterns += static(
        settings.MEDIA_URL, document_root=settings.MEDIA_ROOT
    )
```

Pamiętaj, że pliki statyczne dostarczamy w taki sposób jedynie podczas pracy nad aplikacją. W środowisku produkcyjnym nigdy nie powinieneś dostarczać plików statycznych przy użyciu Django. Serwer programistyczny Django nie serwuje statycznych plików odpowiednio wydajnie. Sposób serwowania statycznych plików w środowisku produkcyjnym opisałem w rozdziale 17., „Wdrożenie”.

Uruchom serwer programistyczny za pomocą polecenia:

```
python manage.py runserver
```

Za pomocą witryny administracyjnej dodaj kilka produktów do sklepu, a następnie w przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/`. Powinieneś zobaczyć stronę listy produktów, podobną do pokazanej na rysunku 8.6.



Rysunek 8.6. Strona z listą produktów

Uwaga

Źródła zdjęć wykorzystanych w tym rozdziale:

- *Zielona herbata*: zdjęcie autorstwa Jia Ye pochodzące z serwisu Unsplash.
- *Czerwona herbata*: Zdjęcie autorstwa Manki Kim pochodzące z serwisu Unsplash.
- *Herbata sproszkowana*: Zdjęcie autorstwa Phuong Nguyen pochodzące z serwisu Unsplash.

Jeżeli z poziomu witryny administracyjnej utworzysz produkt, ale nie zdefiniujesz dla niego żadnego zdjęcia, wtedy domyślnie będzie wyświetlony obraz *no_image.png*, co pokazałem poniżej (rysunek 8.7).

Przeprowadź edycję pliku *shop/product/detail.html* i umieść w nim poniższy fragment kodu.

Rysunek 8.7. Lista produktów z domyślnym zdjęciem dla produktu, dla którego nie podano obrazu

```
{% extends "shop/base.html" %}
{% load static %}

{% block title %}
  {{ product.name }}
{% endblock %}

{% block content %}
  <div class="product-detail">
    <img src="{% if product.image %}{{ product.image.url }}{% else %}
    {% static "img/no_image.png" %}{% endif %}">
    <h1>{{ product.name }}</h1>
    <h2>
      <a href="{{ product.category.get_absolute_url }}">
        {{ product.category }}
      </a>
    </h2>
    <p class="price">{{ product.price }} PLN</p>
    {{ product.description|linebreaks }}
  </div>
{% endblock %}
```

Wywołujemy metodę `get_absolute_url()` w odpowiednim obiekcie kategorii, aby wyświetlić dostępne produkty zaliczające się do wybranej kategorii.

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/` i kliknij dowolny produkt, aby przejść na stronę wyświetlającą informacje szczegółowe o danym produkcie. Strona wygląda jak na rysunku 8.8.

Rysunek 8.8. Strona szczegółów dotyczących produktu

W ten sposób przygotowaliśmy prosty katalog produktów. W następnym podrozdziale zaimplementujesz koszyk, który pozwoli użytkownikom na dodanie do niego dowolnego produktu podczas przeglądania sklepu internetowego.

Utworzenie koszyka na zakupy

Po zbudowaniu katalogu produktów kolejnym krokiem jest opracowanie koszyka na zakupy pozwalającego użytkownikom na wybór produktów, które chcieliby kupić. Koszyk na zakupy umożliwia tymczasowe przechowywanie produktów wybranych przez użytkownika, który w tym czasie może kontynuować przeglądanie witryny aż do chwili, gdy zdecyduje się na złożenie zamówienia. Zawartość koszyka na zakupy musi być przechowywana w sesji, aby włożone do niego produkty pozostały w koszyku podczas przeglądania witryny przez użytkownika.

Do zachowania zawartości koszyka wykorzystamy oferowany przez Django framework `session`. Koszyk pozostanie zapisany w sesji aż do jej zakończenia lub złożenia zamówienia przez użytkownika. Trzeba również utworzyć kolejne modele Django przeznaczone do obsługi koszyka na zakupy i jego zawartości.

Użycie frameworka sessions Django

Django udostępnia framework `session`, obsługujący sesje użytkownika oraz anonimowe. Framework ten pozwala na przechowywanie dowolnych danych dla każdego odwiedzającego witrynę internetową. Jeśli nie użyjesz silnika sesji bazującej na plikach cookie, dane sesji są przechowywane po stronie serwera, a sam plik cookie zawiera identyfikator sesji. Wysyłaniem i odbieraniem plików cookie zarządza oprogramowanie middleware obsługi sesji. Wprawdzie domyślny silnik sesji przechowuje jej dane w bazie danych, ale jak wkrótce zobaczysz, istnieje możliwość wyboru różnych silników sesji.

Aby w ogóle użyć mechanizmu sesji, należy ustawić opcję `MIDDLEWARE` w projekcie na wartość `'django.contrib.sessions.middleware.SessionMiddleware'`. To warstwa middleware zarządza sesjami. Jest ona dodawana domyślnie do ustawienia `MIDDLEWARE` podczas tworzenia nowego projektu za pomocą polecenia `startproject`.

Oprogramowanie middleware obsługi sesji udostępnia bieżącą sesję w obiekcie `request`. Dostęp do bieżącej sesji możesz uzyskać za pomocą wywołania `request.session`, które zwraca słownik Pythona zawierający dane sesji. Ten słownik Pythona domyślnie akceptuje dowolny obiekt Pythona, który może być serializowany na postać danych w formacie JSON. Ustawienie zmiennej w sesji odbywa się tak:

```
request.session['foo'] = 'bar'
```

Pobranie klucza sesji odbywa się w poniższy sposób.

```
request.session.get('foo')
```

W celu usunięcia klucza przechowywanego w sesji należy użyć polecenia:

```
del request.session['foo']
```

Uwaga

Kiedy użytkownik loguje się w witrynie internetowej, jego sesja anonimowa jest usuwana i następuje utworzenie nowej sesji dla uwierzytelnionego użytkownika. Jeżeli w sesji anonimowej są przechowywane elementy, które mają w niej pozostać po zalogowaniu się użytkownika, konieczne jest skopiowanie danych ze starej sesji do nowej. Można to zrobić, pobierając dane sesji przed zalogowaniem użytkownika z wykorzystaniem funkcji `login()` systemu uwierzytelniania Django. Następnie dane te można zapisać w sesji uwierzytelnionej.

Ustawienia sesji

Istnieje kilka opcji, za pomocą których można skonfigurować sesje dla projektu. Najważniejsza z nich to `SESSION_ENGINE`. Opcja pozwala na zdefiniowanie miejsca przeznaczonego do przechowywania sesji. Domyślnie Django przechowuje sesje w bazie danych, wykorzystując do tego model `Session` aplikacji `django.contrib.sessions`.

Oto opcje związane z przechowywaniem danych sesji, które oferuje framework Django:

- **Sesje oparte na bazie danych**. Dane sesji są przechowywane w bazie danych. To jest domyślny silnik sesji.
- **Sesje oparte na plikach.** Dane sesji są przechowywane w systemie plików.
- **Sesje buforowane.** Dane sesji są przechowywane w warstwie backend pamięci podręcznej. Mechanizm backend buforowania można wskazać za pomocą opcji `CACHES`. Umieszczenie danych sesji w systemie buforowania zapewnia najlepszą wydajność.
- **Buforowane sesje oparte na bazie danych.** Dane sesji są przechowywane w buforze operacji zapisu i bazie danych. Odczyt danych następuje z bazy danych tylko wtedy, gdy dane nie znajdują się już w buforze.
- **Buforowane sesje oparte na plikach cookie.** Dane sesji są przechowywane w plikach cookie wysyłanych do przeglądarki internetowej.

Uwaga

W celu uzyskania lepszej wydajności możesz użyć silnika sesji działającego na bazie bufora. Django obsługuje Memcached oraz inne tego rodzaju mechanizmy dla bazy danych Redis oraz innych systemów buforowania.

Sesje można dostosować do własnych potrzeb za pomocą kolejnych ustawień. Poniżej wymieniłem najważniejsze opcje związane z sesją.

- `SESSION_COOKIE_AGE`. Wyrażony w sekundach czas trwania plików cookie sesji. Wartość domyślna wynosi `1209600`, czyli dwa tygodnie.
- `SESSION_COOKIE_DOMAIN`. Domena użyta dla plików cookie sesji. Przypisz wartość *.nazwa-domeny.pl*, jeśli chcesz zezwolić na użycie plików cookie między domenami, lub zastosuj wartość `None` dla standardowego pliku cookie domeny.

- SESSION_COOKIE_HTTPONLY. Ustawienie decyduje o używaniu flagi HttpOnly w plikach cookie sesji. Jeśli ta opcja będzie ustawiona na True, nie będzie można uzyskać dostępu do pliku cookie sesji z kodu JavaScript działającego po stronie klienta. W celu zwiększenia ochrony przed przejęciem sesji użytkownika, wartość domyślna to True.
- SESSION_COOKIE_SECURE. Wartość typu Boolean określająca, czy plik cookie powinien być przekazywany jedynie za pomocą bezpiecznego połączenia (HTTPS). Wartość domyślna to False.
- SESSION_EXPIRE_AT_BROWSER_CLOSE. Wartość typu Boolean określająca, czy sesja ma wygasnąć po zamknięciu przeglądarki internetowej. Wartość domyślna to False.
- SESSION_SAVE_EVERY_REQUEST. Wartość typu Boolean. W tym przypadku True oznacza zapisanie sesji w bazie danych podczas każdego żądania. Oczywiście za każdym razem będzie wówczas uaktualniana data i godzina wygaśnięcia sesji. Wartość domyślna to False.

Omówienie wszystkich ustawień dotyczących sesji znajdziesz na stronie *https://docs.djangoproject.com/en/5.0/ref/settings/#sessions*.

Wygaśnięcie sesji

Za pomocą opcji SESSION_EXPIRE_AT_BROWSER_CLOSE możesz wybrać między sesją trwałą lub kończoną w chwili zamknięcia przeglądarki internetowej. Wartością domyślną wymienionej opcji jest False, co oznacza wymuszenie trwania sesji przez okres czasu zdefiniowany w opcji SESSION_COOKIE_AGE. Jeżeli opcji SESSION_EXPIRE_AT_BROWSER_CLOSE będzie przypisana wartość True, sesja wygaśnie z chwilą zamknięcia przeglądarki internetowej przez użytkownika, a wartość opcji SESSION_COOKIE_AGE nie będzie miała żadnego znaczenia.

Istnieje możliwość użycia metody set_expiry() słownika request.session w celu zmiany czasu trwania bieżącej sesji.

Przechowywanie koszyka na zakupy w sesji

Konieczne jest przygotowanie prostej struktury, która będzie mogła być serializowana na postać danych w formacie JSON, aby umożliwić przechowywanie w sesji produktów umieszczonych w koszyku. Koszyk na zakupy przechowuje następujące dane dla każdego umieszczonego w nim produktu.

- Identyfikator (id) egzemplarza modelu Product.
- Liczba sztuk danego produktu.
- Cena jednostkowa danego produktu.

Ponieważ cena produktu może się zmieniać na przestrzeni czasu, przyjmujemy podejście polegające na przechowywaniu informacji o cenie produktu wraz z samym produktem po jego umieszczeniu w koszyku. W ten sposób zachowujemy tę samą cenę, jaką

użytkownik widział w chwili umieszczania danego produktu w koszyku, nawet jeśli od tego czasu cena uległa zmianie. Oznacza to, że cena towaru z chwili, gdy klient dodał go do koszyka, jest utrzymywana dla tego klienta w sesji do momentu złożenia zamówienia lub zakończenia sesji.

Musimy zająć się również zarządzaniem operacjami tworzenia koszyka na zakupy i powiązywania go z sesją. Sposób działania koszyka na zakupy przedstawia się tak:

- Kiedy potrzebny jest koszyk na zakupy, sprawdzamy, czy został zdefiniowany klucz sesji. Jeżeli w danej sesji nie znajduje się żaden koszyk na zakupy, tworzymy nowy i zapisujemy go w kluczu sesji.
- W trakcie kolejnych żądań przeprowadzamy tę samą operację sprawdzenia i pobieramy zawartość koszyka na zakupy z klucza sesji. Zawartość koszyka na zakupy pochodzi z sesji oraz powiązanych obiektów `Product` w bazie danych.

Przeprowadź edycję pliku *settings.py* projektu i dodaj do niego poniższy wiersz.

```
CART_SESSION_ID = 'cart'
```

To jest klucz, którego będziemy używać do przechowywania koszyka na zakupy w sesji użytkownika. Ponieważ sesje Django są tworzone dla poszczególnych użytkowników, dla wszystkich sesji można wykorzystać ten sam klucz.

Przystępujemy teraz do utworzenia aplikacji przeznaczonej do zarządzania koszykami na zakupy. Przejdź do powłoki i utwórz nową aplikację, wydając poniższe polecenie z poziomu katalogu projektu.

```
python manage.py startapp cart
```

Następnie przeprowadź edycję pliku *settings.py* projektu i umieść nową aplikację na liście `INSTALLED_APPS` za pomocą kodu oznaczonego poniżej pogrubioną czcionką.

```
INSTALLED_APPS = [
    # ...
    'cart.apps.CartConfig',
    'shop.apps.ShopConfig',
]
```

Utwórz nowy plik w katalogu aplikacji `cart` i nadaj mu nazwę *cart.py*. Następnie w pliku umieść poniższy fragment kodu.

```
from decimal import Decimal
from django.conf import settings
from shop.models import Product


class Cart:
    def __init__(self, request):
        """
        Zainicjuj koszyk.
        """
        self.session = request.session
        cart = self.session.get(settings.CART_SESSION_ID)
        if not cart:
            # Zapis pustego koszyka na zakupy w sesji.
            cart = self.session[settings.CART_SESSION_ID] = {}
        self.cart = cart
```

To jest kod klasy `Cart` pozwalającej na zarządzanie koszykiem na zakupy. Konieczne jest zainicjalizowanie koszyka na zakupy wraz z obiektem `request`. Bieżąca sesja jest przechowywana za pomocą polecenia `self.session = request.session`, aby była dostępna dla innych metod klasy `Cart`.

Najpierw próbujemy pobrać koszyk na zakupy z bieżącej sesji za pomocą polecenia `self.session.get(settings.CART_SESSION_ID)`. Jeżeli w sesji nie znajduje się koszyk na zakupy, tworzymy nowy przez po prostu zdefiniowanie w sesji pustego słownika.

Oczekujemy, że słownik koszyka na zakupy będzie używał identyfikatora produktu jako klucza, którego wartością będzie słownik wraz z liczbą sztuk danego produ jako klucza, którego wartością będzie słownik wraz z liczbą sztuk danego produktu i jego ceną jednostkową. W ten sposób mamy pewność, że dany produkt będzie mógł być dodany do koszyka tylko jednokrotnie. Ponadto ułatwiamy sobie sposób dostępu do wszystkich danych produktów umieszczonych w koszyku na zakupy.

Przystępujemy do utworzenia metody pozwalającej na dodawanie produktu do koszyka na zakupy lub zmianę ilości danego produktu w koszyku. Do klasy `Cart` dodaj wymienione poniżej metody `add()` i `save()`.

```
class Cart:
    # …
    def add(self, product, quantity=1, override_quantity=False):
        """
        Dodanie produktu do koszyka lub zmiana jego ilości.
        """
        product_id = str(product.id)
        if product_id not in self.cart:
            self.cart[product_id] = {
               'quantity': 0,
               'price': str(product.price)
            }
        if override_quantity:
           self.cart[product_id]['quantity'] = quantity
        else:
           self.cart[product_id]['quantity'] += quantity
        self.save()

    def save(self):
        # Oznaczenie sesji jako "zmodyfikowanej", aby upewnić się o jej zapisaniu.
        self.session.modified = True
```

Metoda `add()` przyjmuje poniższe parametry.

- `product`. Egzemplarz modelu `Product`, który będzie dodany lub uaktualniony w koszyku na zakupy.
- `quantity`. Opcjonalna liczba całkowita określająca liczbę sztuk produktu. Wartością domyślną jest `1`.
- `override_quantity`. Wartość typu `Boolean` wskazująca, czy liczbę sztuk produktu trzeba uaktualnić do podanej wartości (`True`), czy po prostu dodać ją do istniejącej (`False`).

W słowniku zawartości koszyka na zakupy używamy identyfikatora (`id`) produktu jako klucza. Identyfikator ten jest konwertowany na ciąg tekstowy, ponieważ Django korzysta z formatu JSON dla serializowanych danych sesji, a JSON pozwala na stosowanie kluczy jedynie w postaci ciągu tekstowego. W budowanym rozwiązaniu identyfikator produktu jest kluczem, a wartością słownik przechowujący informacje o liczbie sztuk danego produktu i jego cenie jednostkowej. Cena produktu zostaje skonwertowana z wartości typu `Decimal` na typ `string`, aby można było ją serializować. Na końcu wywołujemy metodę `save()` i zapisujemy koszyk w sesji.

Metoda `save()` zapisuje w sesji wszystkie zmiany wprowadzone w koszyku na zakupy i za pomocą `session.modified = True` oznacza sesję jako zmodyfikowaną. Dzięki temu framework Django jest informowany o zmianie sesji i konieczności jej zapisania.

Potrzebujemy także metody przeznaczonej do usuwania produktów z koszyka na zakupy. Dodaj poniższą metodę do klasy `Cart`.

```
class Cart:
    # …
    def remove(self, product):
        """
        Usunięcie produktu z koszyka na zakupy.
        """
        product_id = str(product.id)
        if product_id in self.cart:
            del self.cart[product_id]
            self.save()
```

Metoda `remove()` usuwa dany produkt ze słownika przechowującego zawartość koszyka na zakupy oraz wywołuje metodę `save()` w celu zaktualizowania koszyka w sesji.

Konieczne jest przeprowadzenie iteracji przez produkty umieszczone w koszyku na zakupy i uzyskanie dostępu do powiązanych z nimi egzemplarzy modelu `Product`. W tym celu można w klasie zdefiniować metodę `__iter__()`. Dodaj poniższą metodę do klasy `Cart`.

```
class Cart:
    # …

    def __iter__(self):
        """
        Iteracja przez elementy koszyka na zakupy i pobranie produktów z bazy danych.
        """
        product_ids = self.cart.keys()
        # Pobranie obiektów produktów i dodanie ich do koszyka na zakupy.
        products = Product.objects.filter(id__in=product_ids)
        cart = self.cart.copy()
        for product in products:
           cart[str(product.id)]['product'] = product
        for item in cart.values():
            item['price'] = Decimal(item['price'])
            item['total_price'] = item['price'] * item['quantity']
            yield item
```

W metodzie `__iter__()` pobieramy egzemplarze modelu `Product`, które reprezentują produkty znajdujące się w koszyku na zakupy. Kopiujemy bieżącą zawartość koszyka zapisaną w zmiennej cart i dodajemyz do niej egzemplarze modelu `Product`. Następnie przeprowadzamy iterację przez produkty w koszyku i konwertujemy ich cenę z powrotem na typ `Decimal` oraz dodajemy atrybut `total_price` dla każdego produktu. Dzięki metodzie `__iter__()` można bardzo łatwo przeprowadzić iterację przez produkty znajdujące się w koszyku na zakupy.

Potrzebny jest również sposób na podanie całkowitej liczby elementów znajdujących się w koszyku. Kiedy funkcja `len()` jest wykonywana w obiekcie, Python wywołuje jego metodę `__len__()` w celu ustalenia wielkości obiektu. Zdefiniujemy więc własną metodę `__len__()` przeznaczoną do podania całkowitej liczby elementów przechowywanych w koszyku na zakupy.

Dodaj do klasy `Cart` poniższą metodę `__len__()`.

```
class Cart:
    # …
    def __len__(self):
        """
        Obliczenie liczby wszystkich elementów w koszyku na zakupy.
        """
        return sum(item['quantity'] for item in self.cart.values())
```

Powyższa metoda zwraca liczbę wszystkich produktów znajdujących się w koszyku na zakupy.

Aby obliczyć wartość całkowitą kosztu wszystkich produktów znajdujących się w koszyku na zakupy, trzeba dodać jeszcze jedną metodę.

```
class Cart:
    # …
    def get_total_price(self):
        return sum(
            Decimal(item['price']) * item['quantity']
            for item in self.cart.values()
         )
```

Na końcu dodajemy metodę usuwającą zawartość koszyka na zakupy.

```
class Cart:
    # …
    def clear(self):
        # Usunięcie koszyka na zakupy z sesji.
        del self.session[settings.CART_SESSION_ID]
        self.save()
```

Klasa `Cart` jest teraz przygotowana do zarządzania koszykiem na zakupy.

Utworzenie widoków koszyka na zakupy

Skoro mamy już opracowaną klasę `Cart` do zarządzania koszykiem na zakupy, musimy przygotować widoki pozwalające na dodawanie, uaktualnianie i usuwanie produktów z koszyka. Konieczne jest więc utworzenie potrzebnych widoków. Oto one:

- Widok przeznaczony do dodawania lub uaktualniania produktów w koszyku na zakupy oraz obsługi bieżącej i nowej liczby sztuk danego produktu.
- Widok pozwalający na usunięcie produktu z koszyka na zakupy.
- Widok wyświetlający zawartość koszyka na zakupy i całkowitą wartość znajdujących się w nim produktów.

Dodanie elementów do koszyka

Aby użytkownik mógł dodać produkt do koszyka na zakupy, trzeba umożliwić mu podanie liczby sztuk dodawanego produktu. Utwórz plik *forms.py* w katalogu aplikacji `cart` i umieść w nim poniższy fragment kodu.

```
from django import forms


PRODUCT_QUANTITY_CHOICES = [(i, str(i)) for i in range(1, 21)]

class CartAddProductForm(forms.Form):
    quantity = forms.TypedChoiceField(
        choices=PRODUCT_QUANTITY_CHOICES,
        coerce=int
    )
    override = forms.BooleanField(
        required=False,
        initial=False,
        widget=forms.HiddenInput
    )
```

Powyższy formularz wykorzystamy podczas dodawania produktów do koszyka na zakupy. Klasa `CartAddProductForm` zawiera dwa pola:

- `quantity`. Pozwala użytkownikowi na wybór liczby sztuk dodawanego produktu (od 1 do 20). W celu konwersji danych wejściowych na postać liczby całkowitej, wykorzystujemy pole typu `TypedChoiceField` wraz z opcją `coerce=int`.
- `override`. Pozwala na wskazanie, czy podana ilość ma zostać dodana do obecnej ilości produktu w koszyku (`False`), czy też dotychczasowa ilość ma być zastąpiona tą nową (`True`). Dla tego pola użyjemy widżetu `HiddenInput` dla tego pola, ponieważ nie chcmy wyświetlać użytkownikowi omawianej tu wartości typu `Boolean`.

Przystępujemy do utworzenia widoku pozwalającego na dodanie produktów do koszyka na zakupy. Przeprowadź edycję pliku *views.py* aplikacji `cart` i umieść w nim poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
from django.shortcuts import get_object_or_404, redirect, render
from django.views.decorators.http import require_POST
from shop.models import Product
from .cart import Cart
from .forms import CartAddProductForm

@require_POST
def cart_add(request, product_id):
```

```
    cart = Cart(request)
    product = get_object_or_404(Product, id=product_id)
    form = CartAddProductForm(request.POST)
    if form.is_valid():
        cd = form.cleaned_data
        cart.add(
            product=product,
            quantity=cd['quantity'],
            override_quantity=cd['override']
        )
    return redirect('cart:cart_detail')
```

Jest to widok służący do dodawania produktów do koszyka na zakupy lub uaktualniania ilości produktów już się w nim znajdujących. Wykorzystujemy dekorator `require_POST`, aby były dozwolone wyłącznie żądania `POST`. Widok otrzymuje parametr w postaci identyfikatora produktu. Na podstawie identyfikatora pobieramy odpowiedni egzemplarz modelu `Product` i walidujemy formularz `CartAddProductForm`. Jeżeli formularz jest prawidłowy, następuje dodanie produktu do koszyka na zakupy lub też uaktualnienie już znajdującego się w nim. Widok powoduje przekierowanie do adresu URL widoku `cart_detail` i tym samym wyświetlenie bieżącej zawartości koszyka na zakupy. Utworzeniem widoku `cart_detail` zajmiemy się już wkrótce.

Potrzebny jest również widok pozwalający na usunięcie produktu z koszyka. Dodaj poniższy fragment kodu do pliku *views.py* aplikacji `cart`.

```
@require_POST
def cart_remove(request, product_id):
    cart = Cart(request)
    product = get_object_or_404(Product, id=product_id)
    cart.remove(product)
    return redirect('cart:cart_detail')
```

Widok `cart_remove` otrzymuje parametr w postaci identyfikatora produktu. Wykorzystujemy dekorator `require_POST`, aby były dozwolone wyłącznie żądania `POST`. Na podstawie identyfikatora następuje pobranie odpowiedniego egzemplarza modelu `Product` i usunięcie tego produktu z koszyka na zakupy. Po usunięciu produktu widok powoduje przekierowanie do adresu URL widoku `cart_detail`.

Oczywiście potrzebny jest jeszcze widok wyświetlający zawartość koszyka na zakupy. Dodaj poniższy widok do pliku *views.py* aplikacji `cart`.

```
def cart_detail(request):
    cart = Cart(request)
    return render(request, 'cart/detail.html', {'cart': cart})
```

Widok `cart_detail` pobiera bieżący koszyk na zakupy i wyświetla jego zawartość.

W ten sposób przygotowaliśmy widoki pozwalające na dodawanie produktów do koszyka, uaktualnianie ich ilości, usuwanie produktów z koszyka oraz wyświetlanie zawartości koszyka na zakupy. Kolejnym krokiem jest dodanie wzorców adresów URL dla tych widoków. Utwórz nowy plik w katalogu aplikacji `cart` i nadaj mu nazwę *urls.py*. Następnie w nowym pliku umieść poniższe wzorce adresów URL.

```
from django.urls import path
from . import views

app_name = 'cart'

urlpatterns = [
    path('', views.cart_detail, name='cart_detail'),
    path('add/<int:product_id>/', views.cart_add, name='cart_add'),
    path(
        'remove/<int:product_id>/',
        views.cart_remove,
        name='cart_remove'
    ),
]
```

Przeprowadź edycję głównego pliku *urls.py* aplikacji `myshop` i umieść w nim poniższy wzorzec adresu URL w celu dołączenia adresu URL dla koszyka na zakupy.

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('cart/', include('cart.urls', namespace='cart')),
    path('', include('shop.urls', namespace='shop')),
]
```

Upewnij się, że dodałeś nowy wzorzec adresu URL przed wzorcem `shop.urls`, ponieważ jest on znacznie dokładniejszy niż drugi z wymienionych.

Utworzenie szablonu do wyświetlenia koszyka na zakupy

Widoki `cart_add` i `cart_remove` nie renderują żadnych szablonów, ale konieczne jest utworzenie szablonu dla widoku `cart_detail`, który wyświetla zawartość koszyka na zakupy oraz wartość całkowitą znajdujących się w nim produktów.

W aplikacji `cart` utwórz poniższą strukturę plików i katalogów.

```
templates/
     cart/
         detail.html
```

Przeprowadź edycję szablonu *cart/detail.html* i umieść w nim poniższy fragment kodu.

```
{% extends "shop/base.html" %}
{% load static %}

{% block title %}
  Koszyk na zakupy
{% endblock %}

{% block content %}
  <h1>Twój koszyk</h1>
  <table class="cart">
    <thead>
      <tr>
        <th>Zdjęcie</th>
        <th>Produkt</th>
        <th>Ilość</th>
```

```
          <th>Usuń</th>
          <th>Cena jednostkowa</th>
          <th>Cena</th>
        </tr>
      </thead>
      <tbody>
        {% for item in cart %}
          {% with product=item.product %}
            <tr>
              <td>
                <a href="{{ product.get_absolute_url }}">
                  <img src="{% if product.image %}{{ product.image.url }}
                   {% else %}{% static "img/no_image.png" %}{% endif %}">
                </a>
              </td>
              <td>{{ product.name }}</td>
              <td>{{ item.quantity }}</td>
              <td>
                <form action="{% url "cart:cart_remove" product.id %}" method="post">
                   <input type="submit" value="Usuń">
                  {% csrf_token %}
                </form>
              </td>
              <td class="num">{{ item.price }} PLN</td>
              <td class="num">{{ item.total_price }} PLN</td>
            </tr>
          {% endwith %}
        {% endfor %}
        <tr class="total">
          <td>Kwota do zapłaty</td>
          <td colspan="4"></td>
          <td class="num">{{ cart.get_total_price }} PLN</td>
        </tr>
      </tbody>
    </table>
    <p class="text-right">
       <a href="{% url "shop:product_list" %}" class="button
       light">Kontynuuj zakupy</a>
       <a href="#" class="button">Do kasy</a>
    </p>
  {% endblock %}
```

Zadbaj o to, aby żaden znacznik szablonu nie obejmował więcej niż jednego wiersza.

Szablonu tego będziemy używać do wyświetlania zawartości koszyka na zakupy. Zawiera tabelę wraz z elementami przechowywanymi w bieżącym koszyku na zakupy. Użytkownik ma możliwość zmiany ilości poszczególnych produktów za pomocą formularza zdefiniowanego w widoku `cart_add`. Ponadto umożliwiamy użytkownikowi usunięcie produktu z koszyka za pomocą przycisku *Usuń* wyświetlanego obok każdego produktu. Wykorzystamy także formularz HTML z atrybutem `action` wskazującym adres URL `cart_remove` włącznie z identyfikatorem produktu.

Dodanie produktów do koszyka na zakupy

Kolejnym krokiem jest umieszczenie na stronie wyświetlającej szczegóły produktu przycisku *Dodaj do koszyka*. Przeprowadź edycję pliku *views.py* aplikacji `shop` i umieść kod formularza `CartAddProductForm` w widoku `product_detail` w następujący sposób.

```
from cart.forms import CartAddProductForm

# ...

def product_detail(request, id, slug):
    product = get_object_or_404(
        Product, id=id, slug=slug, available=True
    )
    cart_product_form = CartAddProductForm()
    return render(
        request,
        'shop/product/detail.html',
        {'product': product, 'cart_product_form': cart_product_form}
    )
```

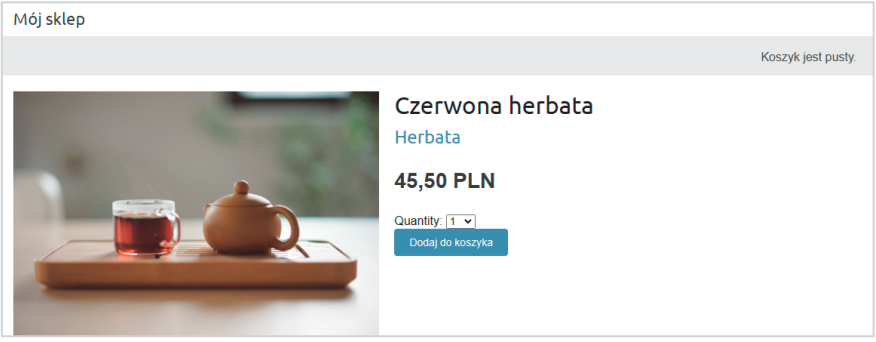
Następnie przeprowadź edycję szablonu *shop/product/detail.html* aplikacji `shop` i dodaj do szablonu szczegółów koszyka poniższy formularz. Nowe wiersze zostały wyróżnione pogrubioną czcionką.

```
...
<p class="price">{{ product.price }} PLN</p>
<form action="{% url "cart:cart_add" product.id %}" method="post">
  {{ cart_product_form }}
  {% csrf_token %}
  <input type="submit" value="Dodaj do koszyka">
</form>
{{ product.description|linebreaks }}
...
```

Uruchom serwer programistyczny za pomocą polecenia:

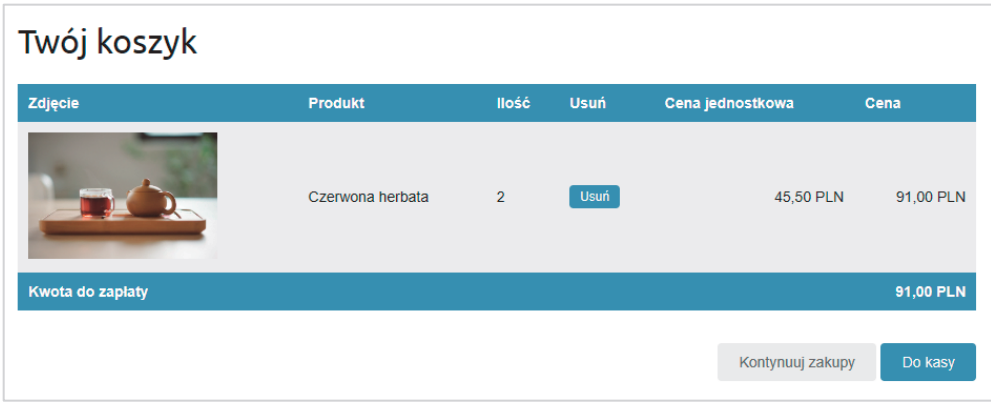
```
python manage.py runserver
```

Teraz w przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/` i wyświetl stronę szczegółów produktu. W tym momencie strona zawiera formularz pozwalający na wybór ilości danego produktu przed jego dodaniem do koszyka na zakupy. Strona będzie podobna do pokazanej na rysunku 8.9.



Rysunek 8.9. Strona szczegółów produktu włącznie z formularzem dodawania produktu do koszyka

Wybierz żądaną ilość i kliknij przycisk *Dodaj do koszyka*. Formularz zostanie wysłany do widoku `cart_add` z wykorzystaniem metody `POST`. Widok umieści w koszyku na zakupy dla bieżącej sesji wskazany produkt wraz z jego ceną i podaną ilością. Następnie użytkownik zostanie przekierowany na stronę wyświetlającą szczegóły koszyka na zakupy, co pokazałem na rysunku 8.10.



Rysunek 8.10. Strona szczegółów koszyka na zakupy

Uaktualnienie ilości produktu w koszyku na zakupy

Użytkownik, patrząc na koszyk zakupowy, może zdecydować się na zmianę ilości kupowanych produktów, zanim złoży zamówienie. Dlatego teraz umożliwimy użytkownikowi zmianę ilości zamawianego produktu bezpośrednio w koszyku na zakupy.

W pliku *views.py* w aplikacji `cart` umieść w widoku `cart_detail` poniższe wiersze oznaczone pogrubioną czcionką.

```
def cart_detail(request):
    cart = Cart(request)
    for item in cart:
        item['update_quantity_form'] = CartAddProductForm(
            initial={'quantity': item['quantity'], 'override': True}
        )
    return render(request, 'cart/detail.html', {'cart': cart})
```

Tworzymy w tym miejscu egzemplarz `CartAddProductForm` dla każdego produktu umieszczonego w koszyku na zakupy, aby tym samym pozwolić na zmianę jego ilości. Formularz zostaje zainicjalizowany wraz z ilością bieżącą, a pole `override` ma wartość `True`. Dlatego też po wysłaniu formularza do widoku `cart_add` bieżąca ilość zostaje zastąpiona nową.

Przeprowadź teraz edycję szablonu *cart/detail.html* aplikacji `cart` i znajdź poniższy wiersz kodu

```
<td>{{ item.quantity }}</td>
```

a następnie zastąp go takim fragmentem kodu:

```
<td>
  <form action="{% url "cart:cart_add" product.id %}" method="post">
```

```
        {{ item.update_quantity_form.quantity }}
        {{ item.update_quantity_form.update }}
        <input type="submit" value="Zmień">
        {% csrf_token %}
      </form>
    </td>
```

Uruchom serwer programistyczny za pomocą polecenia:

```
python manage.py runserver
```

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/cart/`.

Powinieneś zobaczyć formularz pozwalający na zmianę ilości każdego produktu znajdującego się w koszyku na zakupy, co pokazałem na rysunku 8.11.

Rysunek 8.11. Strona szczegółów koszyka na zakupy włącznie z formularzem aktualizacji ilości produktów

Zmień ilość produktu i kliknij przycisk *Zmień*, aby przetestować nową funkcjonalność. Możesz również usunąć pozycję z koszyka przy użyciu przycisku *Usuń*.

Utworzenie procesora kontekstu dla bieżącego koszyka na zakupy

Być może zauważyłeś komunikat *Koszyk jest pusty* wyświetlany w nagłówku witryny internetowej nawet wtedy, gdy są w nim towary. Zamiast tego komunikatu użytkownik powinien widzieć liczbę produktów w koszyku oraz ich całkowitą wartość. Ponieważ tego rodzaju informacje powinny być wyświetlane na wszystkich stronach, zaimplementujemy procesor kontekstu umieszczający bieżący koszyk na zakupy w kontekście żądania, niezależnie od aktualnie przetwarzanego widoku.

Procesory kontekstu

Procesor kontekstu to funkcja Pythona, która pobiera jako argument obiekt `request` i zwraca słownik, który jest dodawany do kontekstu żądania. Tego rodzaju funkcja okazuje się użyteczna, gdy trzeba wykonać zadanie, którego wynik ma być dostępny dla wszystkich szablonów.

Domyślnie po utworzeniu nowego projektu za pomocą polecenia `startproject` Django umieści w nim procesory kontekstu, które znajdziesz w opcji `context_processors` ustawienia `TEMPLATES`. Oto one:

- `django.template.context_processors.debug`. Ustawia w kontekście zmienną boolowską `debug` i `sql_queries` zawierającą listę zapytań SQL wykonanych w żądaniu.
- `django.template.context_processors.request`. Ustawia w kontekście zmienną `request`.
- `django.contrib.auth.context_processors.auth`. Ustawia w żądaniu zmienną `user`, reprezentującą użytkownika.
- `django.contrib.messages.context_processors.messages`. Ustawia w kontekście zmienną `messages`, zawierającą wszystkie komunikaty wysłane użytkownikowi za pomocą frameworka `messages`.

Django aktywuje również `django.template.context_processors.csrf`, aby pomóc w ochronie przed atakami typu **CSRF**. Ten procesor kontekstu nie znajduje się w ustawieniach, ale ze względów bezpieczeństwa zawsze pozostaje włączony i nie można go wyłączyć.

Listę wszystkich wbudowanych procesorów kontekstu znajdziesz na stronie *https://docs.djangoproject.com/en/5.0/ref/templates/api/#built-in-template-context-processors*.

Ustawienie koszyka na zakupy w kontekście żądania

Przystępujemy do utworzenia procesora kontekstu ustawiającego bieżący koszyk na zakupy w kontekście żądania dla szablonów. W ten sposób uzyskanie dostępu do koszyka na zakupy będzie możliwe z poziomu każdego szablonu.

W aplikacji `cart` utwórz nowy plik o nazwie *context_processors.py*. Wprawdzie procesory kontekstu mogą znajdować się w dowolnym miejscu kodu, ale zdefiniowanie ich w wymienionym pliku pomaga w zachowaniu dobrze zorganizowanego kodu. W nowym pliku umieść poniższy fragment kodu.

```
from .cart import Cart


def cart(request):
    return {'cart': Cart(request)}
```

W procesorze kontekstu tworzymy egzemplarz koszyka z wykorzystaniem obiektu `request` i udostępniamy go wszystkim szablonom jako zmienną o nazwie `cart`.

W pliku *settings.py* projektu dodaj `cart.context_processors.cart` do opcji `context_processors` w ustawieniach `TEMPLATES`, tak jak pokazałem poniżej. Nowy kod został wyróżniony pogrubioną czcionką.

```
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
```

```
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
                'cart.context_processors.cart',
            ],
        },
    },
]
```

W ten sposób utworzony procesor kontekstu będzie wykonywany w trakcie renderowania każdego szablonu za pomocą obiektu Django `RequestContext`. W kontekście szablonów zostanie ustawiona zmienna `cart`. Więcej informacji na temat obiektu `Request ↪Context` można znaleźć pod adresem *https://docs.djangoproject.com/en/5.0/ref/templates/api/#django.template.RequestContext*.

Uwaga

Procesory kontekstu są wykonywane we wszystkich żądaniach, w których użyto obiektu `RequestContext`. Jeżeli konieczne jest uzyskanie dostępu do bazy danych, zamiast procesora kontekstu powinieneś utworzyć własny znacznik szablonu.

Teraz przeprowadź edycję szablonu *shop/base.html* aplikacji `shop`. Odszukaj fragment:

```
<div class="cart">
   Koszyk jest pusty.
</div>
```

i zastąp go poniższym fragmentem kodu:

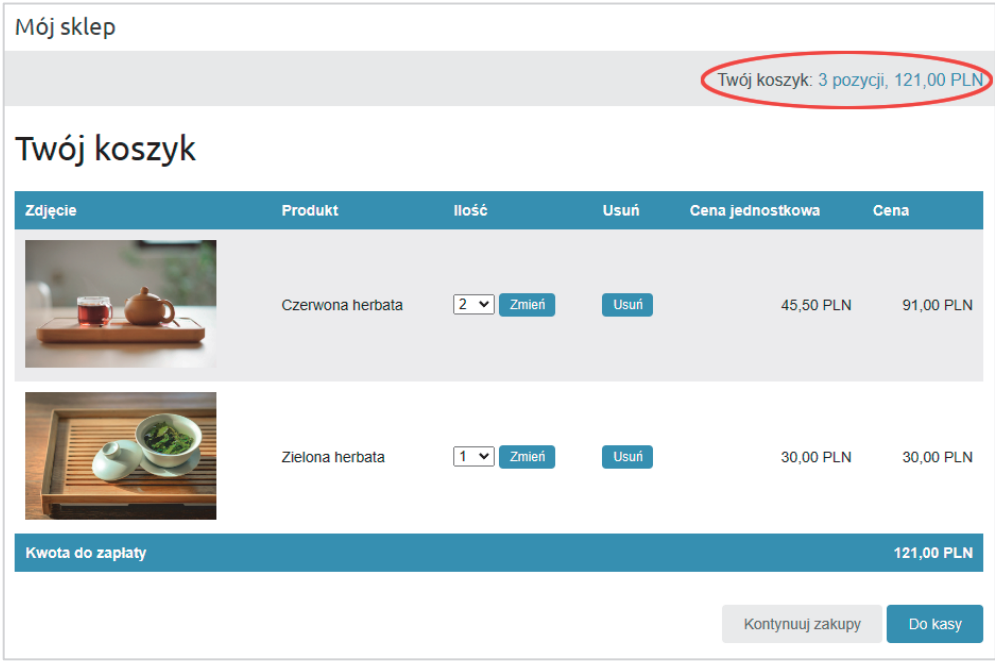
```
<div class="cart">
  {% with total_items=cart|length %}
    {% if cart|length > 0 %}
       Koszyk:
      <a href="{% url "cart:cart_detail" %}">
        {{ total_items }} item{{ total_items|pluralize }},
        {{ cart.get_total_price }} PLN
      </a>
    {% else %}
      Koszyk jest pusty.
    {% endif %}
  {% endwith %}
</div>
```

Uruchom serwer programistyczny za pomocą poniższego polecenia.

```
python manage.py runserver
```

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/`, a następnie dodaj wybrane produkty do koszyka na zakupy.

W nagłówku witryny internetowej powinieneś zobaczyć całkowitą liczbę produktów znajdujących się w koszyku na zakupy oraz ich wartość (rysunek 8.12).



Rysunek 8.12. Nagłówek witryny wyświetlający bieżące elementy w koszyku

Gratulacje! Funkcjonalność koszyka na zakupy jest gotowa. To ważny kamień milowy w projekcie sklepu internetowego. W następnym podrozdziale stworzysz funkcjonalność rejestrowania zamówień klientów, która jest kolejnym podstawowym elementem każdej platformy e-commerce.

Rejestracja zamówień klienta

Kiedy klient chce złożyć zamówienie, jego treść trzeba zapisać w bazie danych. Zamówienie będzie zawierało informacje o kliencie i kupowanych przez niego produktach.

Za pomocą poniższego polecenia utwórz nową aplikację przeznaczoną do zarządzania zamówieniami składanymi przez klienta.

```
python manage.py startapp orders
```

W pliku *settings.py* projektu dodaj nową aplikację na liście `INSTALLED_APPS`, tak jak pokazałem poniżej.

```
INSTALLED_APPS = [
    # ...
    'cart.apps.CartConfig',
    'orders.apps.OrdersConfig',
    'shop.apps.ShopConfig',
]
```

W ten sposób aktywowałeś aplikację `orders`.

Utworzenie modeli zamówienia

Potrzebujemy modelu przeznaczonego do przechowywania szczegółów zamówienia oraz drugiego do przechowywania kupowanych produktów, łącznie z informacjami o cenie i ilości. Przeprowadź edycję pliku *models.py* aplikacji `orders` i umieść w nim poniższy fragment kodu.

```
from django.db import models


class Order(models.Model):
    first_name = models.CharField(max_length=50)
    last_name = models.CharField(max_length=50)
    email = models.EmailField()
    address = models.CharField(max_length=250)
    postal_code = models.CharField(max_length=20)
    city = models.CharField(max_length=100)
    created = models.DateTimeField(auto_now_add=True)
    updated = models.DateTimeField(auto_now=True)
    paid = models.BooleanField(default=False)

    class Meta:
        ordering = ['-created']
        indexes = [
            models.Index(fields=['-created']),
        ]

    def __str__(self):
         return f'Zamówienie {self.id}'

    def get_total_cost(self):
        return sum(item.get_cost() for item in self.items.all())

class OrderItem(models.Model):
    order = models.ForeignKey(
        Order,
        related_name='items',
        on_delete=models.CASCADE
)
    product = models.ForeignKey(
        'shop.Product',
        related_name='order_items',
        on_delete=models.CASCADE
    )
    price = models.DecimalField(
        max_digits=10,
        decimal_places=2
    )
    quantity = models.PositiveIntegerField(default=1)

    def __str__(self):
        return str(self.id)

    def get_cost(self):
        return self.price * self.quantity
```

Model `Order` zawiera wiele pól przeznaczonych do przechowywania informacji o kliencie oraz kolumnę o nazwie `paid` typu `Boolean`, której wartością domyślną jest `False`. Później podczas prac nad aplikacją wykorzystamy tę kolumnę do ustalenia, czy zamówienie zostało opłacone. Ponadto zdefiniowaliśmy metodę `get_total_cost()` odpowiedzialną za pobranie całkowitej wartości produktów kupowanych w tym zamówieniu.

Z kolei model `OrderItem` pozwala na przechowywanie informacji o produkcie, zamawianej ilości i cenie jednostkowej danego produktu. Zdefiniowaliśmy metodę `get_cost()`, która zwraca koszt towaru poprzez pomnożenie jego ceny przez ilość. W polu `product` używamy ciągu znaków `'shop.Product'` z formatem `app.Model`, co jest kolejnym sposobem na wskazanie powiązanych modeli, a także dobrym sposobem na uniknięcie importów cyklicznych.

W celu utworzenia początkowych migracji w aplikacji `orders` wydaj polecenie:

```
python manage.py makemigrations
```

Zobaczysz wynik podobny do następującego.

```
Migrations for 'orders':
   orders/migrations/0001_initial.py
   - Create model Order
   - Create model OrderItem
```

Wydaj poniższe polecenie, aby zastosować nową migrację.

```
python manage.py migrate
```

Wyświetli się wynik:

```
Applying orders.0001_initial… OK
```

W tym momencie nowe modele są zsynchronizowane z bazą danych.

Dołączenie modeli zamówienia do witryny administracyjnej

Kolejnym krokiem jest dodanie nowych modeli do witryny administracyjnej. Przeprowadź edycję pliku *admin.py* aplikacji `orders` i umieść w nim poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
from django.contrib import admin
from .models import Order, OrderItem

class OrderItemInline(admin.TabularInline):
    model = OrderItem
    raw_id_fields = ['product']

@admin.register(Order)
class OrderAdmin(admin.ModelAdmin):
    list_display = [
        'id',
        'first_name',
        'last_name',
        'email',
```

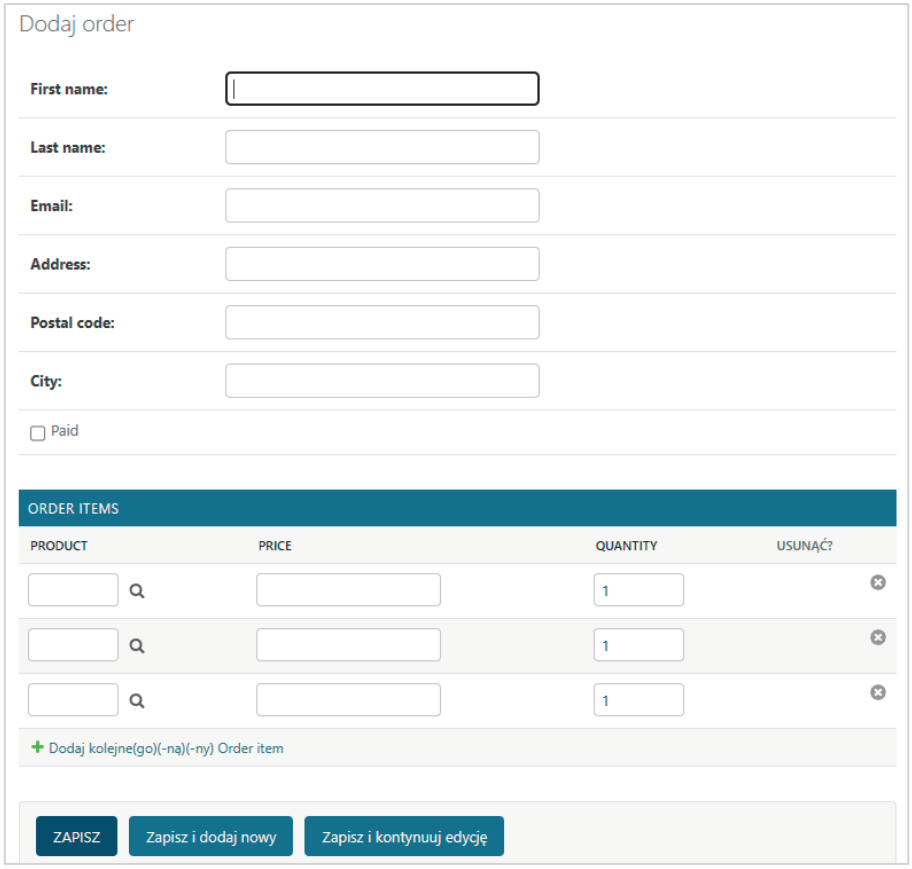
```
        'address',
        'postal_code',
        'city',
        'paid',
        'created',
        'updated'
    ]
    list_filter = ['paid', 'created', 'updated']
    inlines = [OrderItemInline]
```

Dla modelu `OrderItem` wykorzystujemy klasę `ModelInline`, aby dołączyć ją w trybie *inline*, w klasie `OrderAdmin`. Taki sposób dołączenia modelu pozwala na jego pojawienie się na tej samej stronie edycji, co strona modelu nadrzędnego.

Uruchom serwer programistyczny za pomocą polecenia:

```
python manage.py runserver
```

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/admin/orders/order/add/`. Powinieneś zobaczyć stronę pokazaną na rysunku 8.13.



Rysunek 8.13. Formularz dodawania zamówienia włącznie z obiektem OrderItemInline

Utworzenie zamówień klienta

Zbudowane przed chwilą modele wykorzystamy do trwałego zachowania produktów znajdujących się w koszyku na zakupy, gdy użytkownik zdecyduje się ostatecznie na złożenie zamówienia. Oto sposób działania funkcjonalności utworzenia nowego zamówienia:

1. Wyświetlenie użytkownikowi formularza zamówienia, w którym będzie mógł podać dane niezbędne do jego realizacji.
2. Utworzenie nowego egzemplarza modelu `Order` wraz z danymi wprowadzonymi przez użytkownika, a następnie opracowanie powiązanego z nim egzemplarza `OrderItem` dla każdego produktu w koszyku na zakupy.
3. Usunięcie zawartości koszyka i przekierowanie użytkownika na stronę informującą o sukcesie operacji.

Przede wszystkim potrzebujemy formularza umożliwiającego podanie informacji niezbędnych do realizacji zamówienia. W katalogu aplikacji `orders` utwórz nowy plik o nazwie *forms.py*. Dodaj do niego następujący fragment kodu:

```
from django import forms
from .models import Order

class OrderCreateForm(forms.ModelForm):
    class Meta:
        model = Order
        fields = [
            'first_name',
            'last_name',
            'email',
            'address',
            'postal_code',
            'city'
        ]
```

Jest to formularz, za pomocą którego będziemy tworzyć nowe obiekty `Order`. Teraz jest nam potrzebny widok przeznaczony do obsługi tego formularza i faktycznego utworzenia nowego zamówienia. Przeprowadź edycję pliku *views.py* aplikacji `orders` i umieść w nim poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
from cart.cart import Cart
from django.shortcuts import render
from .forms import OrderCreateForm
from .models import OrderItem

def order_create(request):
    cart = Cart(request)
    if request.method == 'POST':
        form = OrderCreateForm(request.POST)
        if form.is_valid():
            order = form.save()
            for item in cart:
                OrderItem.objects.create(
                    order=order,
                    product=item['product'],
```

```
                    price=item['price'],
                    quantity=item['quantity']
                )
            # Wyczyść zawartość koszyka
            cart.clear()
            return render(
                request, 'orders/order/created.html', {'order': order}
            )
    else:
        form = OrderCreateForm()
    return render(
        request,
        'orders/order/create.html',
        {'cart': cart, 'form': form}
    )
```

W widoku `order_create` pobieramy bieżący koszyk na zakupy z sesji, wykorzystując polecenie `cart = Cart(request)`. W zależności od metody żądania, wykonane będą wymienione poniżej zadania.

- **Żądanie GET.** Utworzenie egzemplarza formularza `OrderCreateForm` i wygenerowanie szablonu *orders/order/create.html*.
- **Żądanie POST.** Sprawdzenie poprawności przekazanych danych. Jeżeli dane są prawidłowe, za pomocą polecenia `order = form.save()` tworzymy nowy egzemplarz modelu `Order`. Następnie zapisujemy go w bazie danych i umieszczamy w zmiennej `order`. Po utworzeniu zmiennej order przeprowadzamy iterację przez produkty znajdujące się w koszyku na zakupy i dla każdego z nich tworzymy egzemplarz `OrderItem`. Na koniec usuwamy zawartość koszyka i renderujemy szablon *orders/order/created.html*.

Utwórz nowy plik w katalogu aplikacji `orders` i nadaj mu nazwę *urls.py*. Następnie w pliku umieść poniższy fragment kodu.

```
from django.urls import path
from . import views

app_name = 'orders'

urlpatterns = [
     path('create/', views.order_create, name='order_create'),
]
```

Jest to wzorzec adresu URL dla widoku `order_create`.

Przeprowadź edycję pliku *urls.py* aplikacji `myshop` i dołącz poniższy wzorzec. Pamiętaj o umieszczeniu go przed wzorcem *shop.urls*. Nowy kod został wyróżniony pogrubioną czcionką.

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('cart/', include('cart.urls', namespace='cart')),
    path('orders/', include('orders.urls', namespace='orders')),
    path('', include('shop.urls', namespace='shop')),
]
```

Przeprowadź edycję szablonu *cart/detail.html* aplikacji `cart` i odszukaj wiersz:

```
<a href="#" class="button">Do kasy</a>
```

Dodaj do atrybutu HTML `href` adres URL `order_create` w taki sposób:

```
<a href="{% url "orders:order_create" %}" class="button">
  Do kasy
</a>
```

Użytkownik może teraz przejść ze strony zawartości koszyka na zakupy do formularza zamówienia.

Nadal czeka nas zdefiniowanie szablonów pozwalających na składanie zamówień. W katalogu aplikacji `orders` utwórz następującą strukturę plików i katalogów.

```
templates/
    orders/
        order/
            create.html
            created.html
```

Przeprowadź edycję szablonu *orders/order/create.html* i umieść w nim poniższy fragment kodu.

```
{% extends "shop/base.html" %}

{% block title %}
  Do kasy
{% endblock %}

{% block content %}
  <h1>Kasa</h1>
  <div class="order-info">
    <h3>Zamówienie</h3>
    <ul>
      {% for item in cart %}
        <li>
          {{ item.quantity }}x {{ item.product.name }}
          <span>{{ item.total_price }} PLN</span>
        </li>
      {% endfor %}
    </ul>
    <p>Razem: {{ cart.get_total_price }} PLN</p>
  </div>
  <form method="post" class="order-form">
    {{ form.as_p }}
    <p><input type="submit" value="Złóż zamówienie"></p>
    {% csrf_token %}
  </form>
{% endblock %}
```

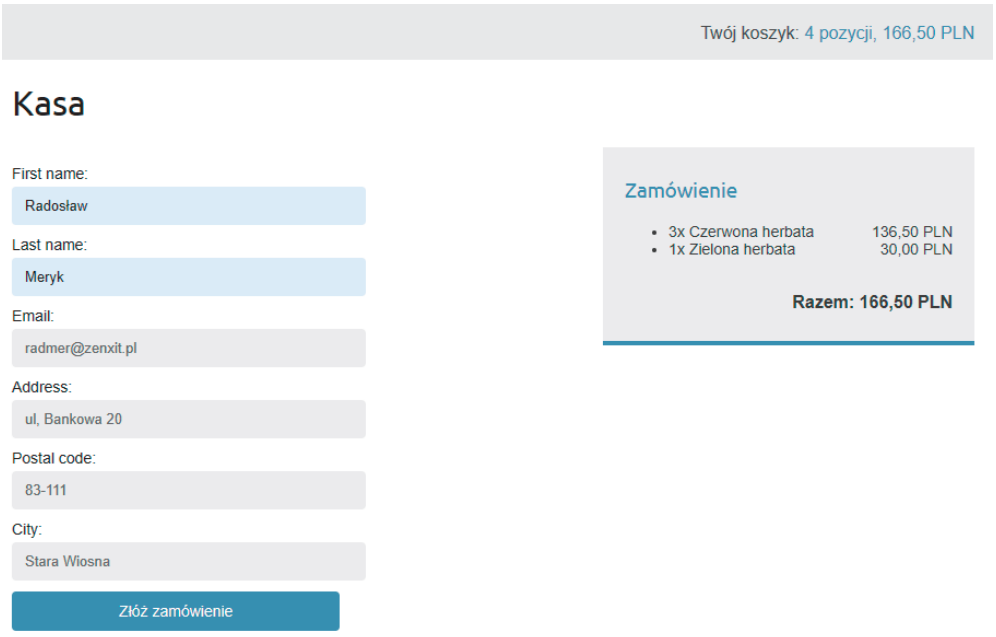
Szablon ten wyświetla produkty znajdujące się w koszyku na zakupy, ich wartość całkowitą oraz formularz pozwalający na złożenie zamówienia.

Teraz przeprowadź edycję szablonu *orders/order/created.html* i umieść w nim poniższy fragment kodu.

```
{% extends "shop/base.html" %}

{% block title %}
  Dziękujemy
{% endblock %}

{% block content %}
  <h1>Dziękujemy</h1>
  <p> Zamówienie zostało pomyślnie złożone. Jego numer to
  <strong>{{ order.id }}</strong>.</p>
{% endblock %}
```

Z kolei ten szablon będzie wyrenderowany, gdy operacja złożenia zamówienia zakończy się powodzeniem.

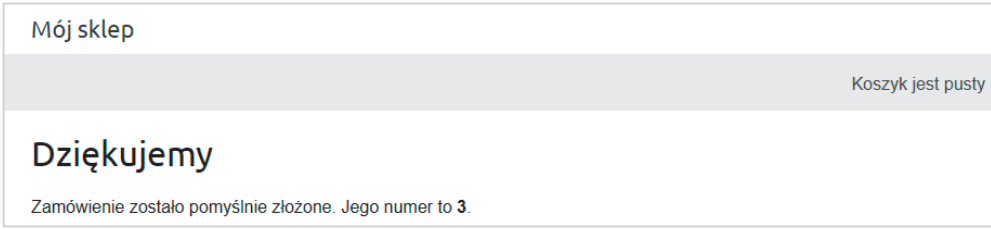
Uruchom serwer programistyczny, aby wypróbować nowe funkcjonalności. W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/`, dodaj kilka produktów do koszyka na zakupy, a następnie przejdź na stronę składania zamówienia. Powinieneś zobaczyć formularz pokazany na rysunku 8.14.



Rysunek 8.14. Strona tworzenia zamówienia włącznie z formularzem kasy i szczegółami zamówienia

Podaj w formularzu poprawne dane i kliknij przycisk *Złóż zamówienie*. Utworzone zostanie nowe zamówienie, a użytkownik zobaczy stronę wraz z komunikatem informującym o sukcesie operacji (rysunek 8.15).

Zamówienie zostało zarejestrowane, a koszyk wyczyszczony.



Rysunek 8.15. Szablon utworzonego zamówienia wyświetlający jego numer

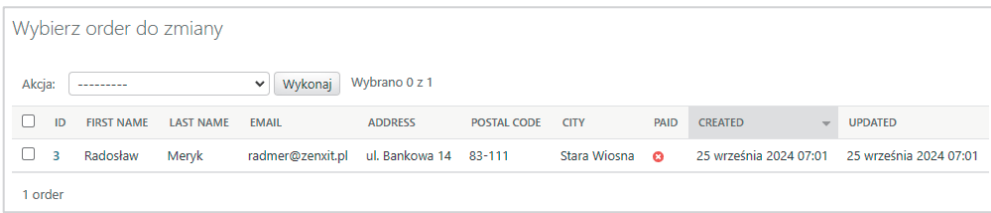
Łatwo zauważyć, że po zrealizowaniu zamówienia w nagłówku wyświetla się komunikat *Twój koszyk jest pusty*. To dlatego, że koszyk został wyczyszczony. Dla widoków, które mają obiekt `order` w kontekście szablonu, możemy łatwo uniknąć wyświetlania tego komunikatu.

Przeprowadź edycję szablonu *shop/base.html* aplikacji `shop` i wpisz poniższy wiersz oznaczony pogrubioną czcionką.

```
...
<div class="cart">
  {% with total_items=cart|length %}
    {% if total_items > 0 %}
       Koszyk:
      <a href="{% url "cart:cart_detail" %}">
        {{ total_items }} item{{ total_items|pluralize }},
        ${{ cart.get_total_price }}
      </a>
    {% elif not order %}
       Koszyk jest pusty.
    {% endif %}
  {% endwith %}
</div> ...
```

Komunikat *Koszyk jest pusty* nie będzie już wyświetlany podczas tworzenia zamówienia.

Teraz przejdź do strony administracyjnej pod adresem `http://127.0.0.1:8000/admin/orders/order/`. Przekonasz się, że zamówienie zostało pomyślnie utworzone (rysunek 8.16).



Rysunek 8.16. Lista wyboru zamówienia do zmiany w witrynie administracyjnej oraz utworzone zamówienie

System składania zamówień jest gotowy. W kolejnym podrozdziale dowiesz się, jak tworzyć zadania asynchroniczne. Dzięki nim po złożeniu zamówienia będziesz mógł wysłać do użytkownika wiadomość e-mail z potwierdzeniem.

Zadania asynchroniczne

Gdy aplikacja otrzymuje żądanie HTTP, powinna jak najszybciej zwrócić odpowiedź użytkownikowi. Jak pamiętamy z rozdziału 7., „Śledzenie działań użytkownika", do obserwowania czasów realizacji różnych faz cyklu żądanie-odpowiedź oraz czasu wykonywania zapytań SQL, skorzystaliśmy z paska narzędzi *Django Debug Toolbar*.

Na całkowity czas odpowiedzi składają się czasy wykonania wszystkich zadań wykonywanych w trakcie cyklu żądanie-odpowiedź. Długotrwałe zadania mogą poważnie spowolnić odpowiedź serwera. Jak zwrócić użytkownikowi szybką odpowiedź w przypadku wykonywania czasochłonnych zadań? Można to zrobić za pomocą zadań asynchronicznych.

Wykorzystywanie zadań asynchronicznych

Aby skrócić czas obsługi cyklu żądanie-odpowiedź, pewne zadania mogą być wykonywane w tle. Rozważmy np. platformę pozwalającą użytkownikom na przekazywanie klipów wideo, choć przetworzenie tych plików przez serwer może wymagać dużej ilości czasu. Witryna powinna więc udzielić użytkownikowi odpowiedzi wraz z informacją o rozpoczęciu przetwarzania pliku i jednocześnie asynchronicznie zacząć przetwarzać ten plik. Kolejnym przykładem może być wysyłanie wiadomości e-mail do użytkowników. Jeżeli witryna będzie wysyłać powiadomienia e-mail za pomocą widoku, połączenie z serwerem **SMTP** (ang. *Simple Mail Transfer Protocol*) może spowolnić lub wręcz uniemożliwić udzielenie odpowiedzi. Wysłanie e-maila asynchronicznie pozwala uniknąć blokowania wykonania kodu.

Asynchroniczne wykonywanie zadań jest szczególnie istotne w przypadku procesów intensywnie korzystających z danych, zasobów i czasu lub procesów, które uległy awarii i wymagają ponowienia próby.

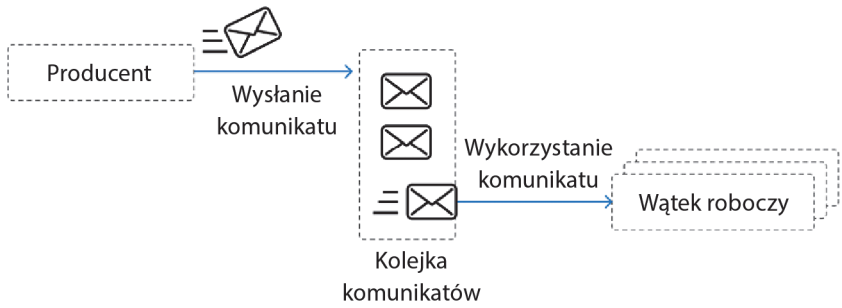
Wątki robocze, kolejki komunikatów i brokery komunikatów

W czasie gdy serwer WWW przetwarza żądania i zwraca odpowiedzi, do przetwarzania zadań asynchronicznych potrzebny jest drugi serwer zadaniowy, tzw. **wątek roboczy** (ang. *worker*). Zadania w tle może wykonywać jeden lub wiele wątków roboczych. Te wątki robocze mogą uzyskiwać dostęp do bazy danych, przetwarzać pliki, wysyłać e-maile itp. Wątki robocze mogą nawet tworzyć kolejki zadań, które mają być wykonane w przyszłości. Wszystko to nie obciąża swobody głównego serwera WWW w zakresie przetwarzania żądań HTTP.

Aby zlecać wątkom roboczym zadania do wykonania, potrzebny jest mechanizm wysyłania *komunikatów*. Komunikujemy się z brokerami poprzez dodawanie *komunikatów* do *kolejki*, która w gruncie rzeczy jest strukturą danych typu **FIFO** (ang. *first in first out* — dosłownie: pierwsze wchodzi, pierwsze wychodzi). Gdy broker staje się dostępny, pobiera z kolejki pierwszy komunikat i rozpoczyna wykonywanie odpowiedniego zadania. Po zakończeniu wykonywania tego zadania, broker pobiera następny komunikat z kolejki i wykonuje powiązane z nim zadanie. Brokery stają się bezczynne po opróżnieniu kolejki

komunikatów. W przypadku korzystania z wielu brokerów, każdy broker pobiera z kolejki pierwszy dostępny komunikat w miarę, gdy stają się one dostępne. Dzięki kolejce każdy broker otrzymuje tylko jedno zadanie na raz, a żadne zadanie nie jest przetwarzane przez więcej niż jeden wątek roboczy.

Sposób działania kolejki komunikatów pokazałem na rysunku 8.17.



Rysunek 8.17. Wykonywanie asynchroniczne przy użyciu kolejki komunikatów i wątków roboczych

Producent wysyła komunikat do kolejki, a wątki robocze przetwarzają komunikaty na zasadzie „pierwszy odebrany, pierwszy obsłużony". Oznacza to, że pierwszy komunikat dodany do kolejki komunikatów jest pierwszym komunikatem do przetworzenia przez wątek roboczy.

Do zarządzania kolejki komunikatów potrzebujemy *brokera komunikatów*. Broker komunikatów służy do tłumaczenia komunikatów na formalny protokół przesyłania komunikatów i zarządzania kolejkami komunikatów dla wielu odbiorców. Zapewnia niezawodne przechowywanie i gwarantowane dostarczanie komunikatów. Broker komunikatów pozwala tworzyć kolejki komunikatów, rozprowadzać komunikaty pomiędzy wątkami roboczymi, itp.

Aby w budowanym projekcie zaimplementować zadania asynchroniczne, użyjemy Celery do zarządzania kolejkami zadań i RabbitMQ jako brokera komunikatów, z którego korzysta Celery. Obie technologie omawiam w następnym podpunkcie.

Wykorzystanie Django z Celery i RabbitMQ

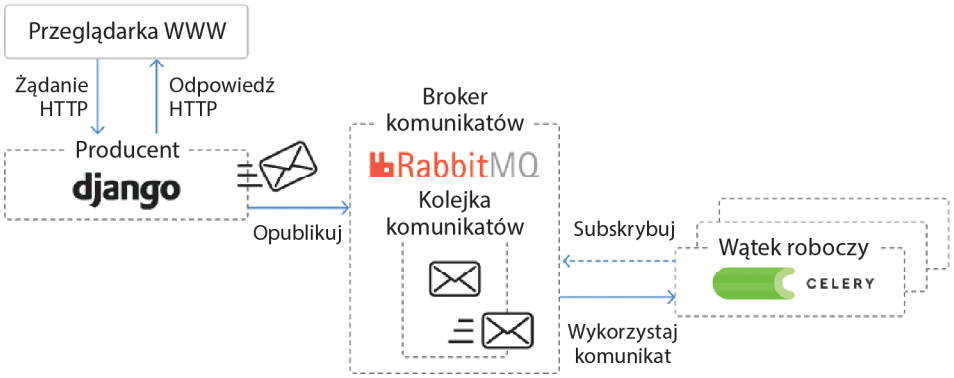
Celery to rozproszona kolejka zadań, w której można przetwarzać ogromne ilości komunikatów. Skorzystamy z Celery do definiowania zadań asynchronicznych jako funkcji Pythona w tworoznych aplikacjach Django. Uruchomimy wątki robocze Celery, którzy będą nasłuchiwać brokera komunikatów, aby uzyskać nowe komunikaty do przetwarzania asynchronicznych zadań.

Za pomocą Celery można nie tylko bardzo łatwo tworzyć zadania asynchroniczne, ale również wykonywać je z wykorzystaniem wątków roboczych, gdy tylko pojawi się możliwość. Ponadto można definiować wykonywanie zadań o określonej porze. Dokumentację kolejki Celery znajdziesz na stronie *https://docs.celeryq.dev/en/stable/index.html*.

Celery komunikuje się za pośrednictwem komunikatów, a do pośredniczenia między klientami, a wątkami roboczymi wymaga zastosowania brokera komunikatów. W zakresie brokerów komunikatów dla systemu Celery mamy do wyboru kilka opcji, m.in. magazyn wartości typu klucz-wartość, taki jak Redis, oraz rzeczywisty system komunikatów, taki jak RabbitMQ.

RabbitMQ to najczęściej stosowany broker komunikatów. Obsługuje wiele protokołów przesyłania komunikatów, takich jak **AMQP** (ang. *Advanced Message Queuing Protocol*) i jest zalecanym mechanizmem obsługi komunikatów dla kolejki Celery. RabbitMQ to system prosty, łatwy do wdrożenia oraz skonfigurowania pod kątem skalowalności i wysokiej dostępności.

Sposób wykorzystania Django, Celery i RabbitMQ do wykonywania zadań asynchronicznych pokazałem na rysunku 8.18.



Rysunek 8.18. Architektura zadań asynchronicznych z wykorzystaniem Django, RabbitMQ i Celery

Instalacja Celery

Przechodzimy do instalacji systemu Celery i jego integracji z budowanym projektem. Aby zainstalować Celery za pomocą menedżera `pip`, wydaj poniższe polecenie.

```
python -m pip install celery==5.4.0
```

Wprowadzenie do kolejki Celery można znaleźć pod adresem *https://docs.celeryq.dev/en/stable/getting-started/introduction.html*.

Instalacja RabbitMQ

Społeczność RabbitMQ udostępnia obraz platformy Docker, który bardzo ułatwia wdrożenie serwera RabbitMQ o standardowej konfiguracji. Sposób instalacji Dockera opisałem w rozdziale 3., „Rozbudowa aplikacji bloga”.

Po zainstalowaniu Dockera na swoim komputerze możesz łatwo pobrać obraz Docker RabbitMQ, przez uruchomienie z powłoki polecenia:

```
docker pull rabbitmq:3.13.1-management
```

Spowoduje to pobranie obrazu Docker systemu RabbitMQ na Twój lokalny komputer. Informacje dotyczące oficjalnego obrazu Docker brokera RabbitMQ można znaleźć na stronie *https://hub.docker.com/_/rabbitmq.*

Szczegółowe informacje potrzebne do zainstalowania RabbitMQ na swojej maszynie bez używania Dockera, dla różnych systemów operacyjnych znajdziesz na stronie *https://www.rabbitmq.com/download.html.*

Aby uruchomić serwer RabbitMQ w kontenerze Docker, wykonaj w powłoce polecenie:

```
docker run -it --rm --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:3.13.1
↪-management
```

Powyższe polecenie uruchamia serwer RabbitMQ w porcie `5672` oraz jego webowy interfejs użytkownika do zarządzania, w porcie `15672`.

Powinieneś otrzymać wynik zawierający m.in. wiersze:

```
Starting broker...
...
completed with 4 plugins.
Server startup complete; 4 plugins started.
```

System RabbitMQ działa w porcie `5672` i jest gotowy na otrzymywanie wiadomości.

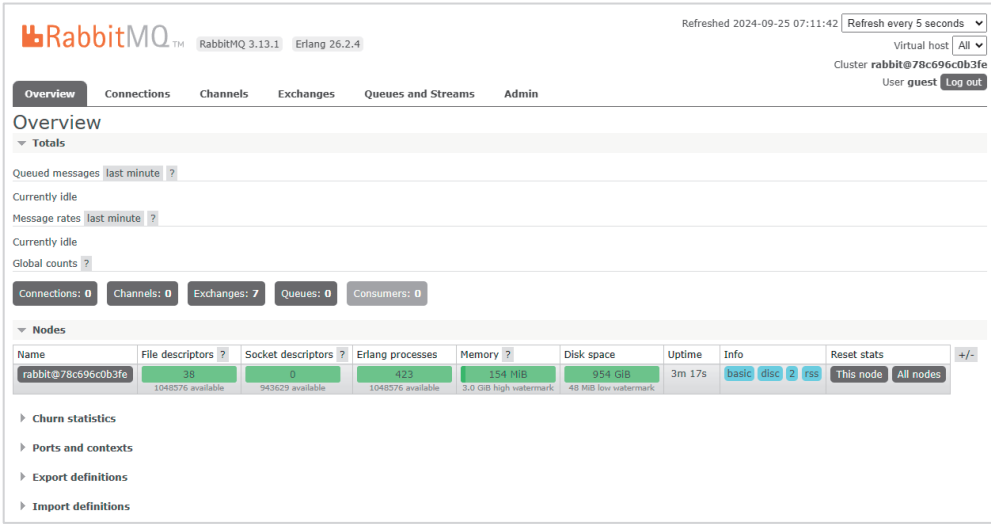
Korzystanie z interfejsu zarządzania RabbitMQ

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:15672/`. Wyświetli się ekran logowania do interfejsu zarządzania RabbitMQ. Strona będzie podobna do pokazanej na rysunku 8.19.

Rysunek 8.19. Ekran logowania interfejsu zarządzania systemem RabbitMQ

Wprowadź *guest* zarówno jako nazwę użytkownika, jak i hasło, a następnie kliknij *Login*. Wyświetli się ekran pokazany na rysunku 8.20.

Użytkownik *guest* to domyślny administrator systemu RabbitMQ. Na powyższym ekranie możesz monitorować bieżącą aktywność systemu RabbitMQ. Jak możesz zobaczyć, działa jeden węzeł bez zarejestrowanych połączeń lub kolejek.



Rysunek 8.20. Ekran logowania interfejsu zarządzania systemem RabbitMQ

Jeśli chcesz używać systemu RabbitMQ w środowisku produkcyjnym, utwórz nowego administratora i usuń domyślnego użytkownika *guest*. Możesz to zrobić w sekcji *Admin* aplikacji zarządzającej.

Teraz dodamy do projektu system Celery. Następnie uruchomimy Celery i przetestujemy połączenie z RabbitMQ.

Dodanie Celery do projektu

Konieczne jest przygotowanie konfiguracji dla egzemplarza Celery. W katalogu aplikacji `myshop` utwórz nowy plik o nazwie *celery.py* i umieść go w tym samym katalogu, w którym znajduje się plik *settings.py*. Plik *celery.py* będzie zawierał konfigurację dla naszego projektu. Następnie w pliku umieść poniższy fragment kodu.

```
import os
from celery import Celery

# Ustawienie domyślnego modułu ustawień Django dla programu 'celery'.
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'myshop.settings')

app = Celery('myshop')
app.config_from_object('django.conf:settings', namespace='CELERY')
app.autodiscover_tasks()
```

W powyższym fragmencie kodu wykonaliśmy następujące działania.

- Ustawiliśmy zmienną `DJANGO_SETTINGS_MODULE` dla działającego w powłoce programu Celery.
- Utworzyliśmy egzemplarz aplikacji za pomocą wywołania `app = Celery('myshop')`.
- Za pomocą metody `config_from_object()` wczytaliśmy konfigurację z ustawień projektu. Atrybut `namespace` określa prefiks wykorzystywany dla ustawień związanych z Celery w pliku *settings.py*. Jeśli ustawimy go na wartość `CELERY`,

to wszystkie ustawienia związane z Celery będą musiały zawierać w swojej nazwie prefiks `CELERY_` (np. `CELERY_BROKER_URL`).

- Na koniec nakazujemy Celery automatyczne wykrywanie zadań asynchronicznych zdefiniowanych dla aplikacji wymienionych na liście `INSTALLED_APPS`. Celery będzie szukać pliku *tasks.py* we wszystkich katalogach aplikacji i wczyta zdefiniowane tam zadania asynchroniczne.

Konieczne jest zaimportowanie modułu `celery` w pliku *__init__.py* projektu, aby mieć pewność, że zostanie wczytany podczas uruchamiania Django.

Przeprowadź edycję pliku *myshop/__init__.py* i umieść w nim poniższy fragment kodu.

```
# import celery.
from .celery import app as celery_app

__all__ = ['celery_app']
```

Właśnie dodałeś Celery do projektu Django. Teraz możesz zacząć z niego korzystać.

Uruchamianie wątku roboczego Celery

Wątek roboczy Celery to mechanizm, który obsługuje takie funkcje jak wysyłanie komunikatów do kolejki i odbieranie ich z niej, rejestrowanie zadań, zabijanie zawieszonych zadań, śledzenie statusu itp. Egzemplarz wątku roboczego może korzystać z dowolnej liczby kolejek komunikatów.

Przejdź do kolejnej powłoki i uruchom wątek roboczy Celery za pomocą poniższego polecenia[1].

```
celery -A myshop worker -l info
```

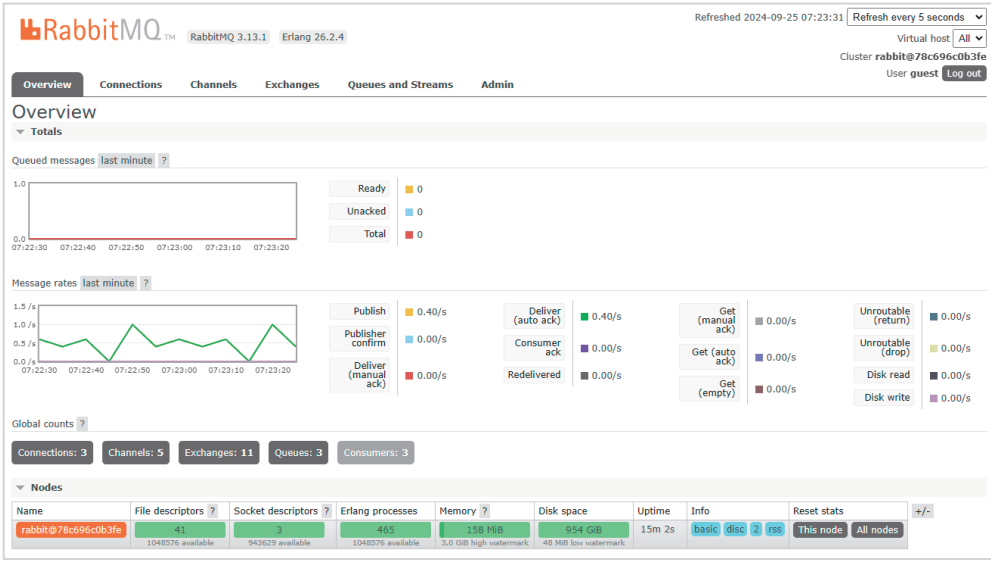
Wątek roboczy Celery jest teraz uruchomiony i gotowy na przetwarzanie zadań. Sprawdźmy, czy istnieje połączenie między Celery, a RabbitMQ.

Aby uruchomić aplikację zarządzającą RabbitMQ, otwórz w przeglądarce stronę `http://127.0.0.1:15672/`. W obszarze *Queued messages* zobaczysz wykres, a w obszarze *Message rates* — inny (rysunek 8.21).

Oczywiście w tej chwili nie ma żadnych komunikatów w kolejce, ponieważ jeszcze nie wysłaliśmy do niej żadnych komunikatów. Wykres w obszarze *Message rates* powinien aktualizować się co pięć sekund. Częstotliwość odświeżania wyświetla się w prawym górnym rogu ekranu. Tym razem zarówno liczba połączeń (*Connections*), jak i kolejek (*Queues*) powinny być większe od zera.

Teraz możemy przystąpić do programowania zadań asynchronicznych.

[1] W razie problemów z wykonywaniem w systemie Windows zadań asynchronicznych zainstaluj moduł `gevent` za pomocą polecenia `pip install gevent`, a następnie uruchom celery za pomocą polecenia `celery -A myshop worker -l info -P gevent` — *przyp. tłum.*



Rysunek 8.21. Pulpit zarządzania RabbitMQ z listą połączeń i kolejek

Uwaga

Opcja `CELERY_ALWAYS_EAGER` pozwala na wykonywanie zadań lokalnych w sposób synchroniczny zamiast ich przekazywania do kolejki. To jest niezwykle użyteczne podczas wykonywania testów jednostkowych lub pracy nad projektem w środowisku lokalnym bez działającego systemu Celery.

Dodawanie do aplikacji zadań asynchronicznych

Spróbujmy wysłać użytkownikowi powiadomienie e-mail po złożeniu przez niego zamówienia. Wysyłanie wiadomości e-mail zaimplementujemy za pomocą funkcji Pythona, którą zarejestrujemy jako zadanie w Celery. Następnie, aby wykonać zadanie asynchronicznie, dodamy ją do widoku `order_create`.

Gdy uruchomimy funkcję widoku `order_create`, Celery wyśle wiadomość do kolejki komunikatów zarządzanej przez RabbitMQ, a następnie broker Celery wykona zadanie zdefiniowane za pomocą funkcji Pythona w trybie asynchronicznym.

Zgodnie z konwencją, zadania asynchroniczne dla aplikacji powinny być umieszczane w module `tasks` w katalogu aplikacji.

W aplikacji `orders` utwórz nowy plik i nadaj mu nazwę *tasks.py*. W tym pliku Celery będzie szukać zadań asynchronicznych do wykonania. Następnie w pliku umieść poniższy fragment kodu.

```
from celery import shared_task
from django.core.mail import send_mail
from .models import Order

@shared_task
```

```
def order_created(order_id):
    """
    Zadanie wysyła e-mailem powiadomienie po pomyślnym utworzeniu zamówienia
    """
    order = Order.objects.get(id=order_id)
    subject = f'Zamówienie nr {order.id}'
    message = (
        f'Witaj {order.first_name},\n\n' \
        f'Złożyłeś zamówienie w naszym sklepie.'
        f'Identyfikator zamówienia to {order.id}.'
    )
    mail_sent = send_mail(
        subject, message, 'admin@myshop.com', [order.email]
    )
    return mail_sent
```

Zadanie `order_created` zdefiniowaliśmy za pomocą dekoratora `@shared_task`. Jak możesz zobaczyć, zadanie Celery to po prostu funkcja Pythona udekorowana przez `@shared_task`. Funkcja zadania `order_created` otrzymuje parametr `order_id`. Zawsze zaleca się przekazywanie do funkcji reprezentujących zadania samych identyfikatorów i pobieranie obiektów z bazy danych dopiero podczas wykonywania zadania. W ten sposób unikamy dostępu do nieaktualnych informacji, ponieważ podczas kolejkowania zadania dane w bazie danych mogły ulec zmianie. W prezentowanym przykładzie do wysyłania użytkownikowi powiadomienia e-mail po złożeniu przez niego zamówienia wykorzystujemy dostarczaną przez Django funkcję `send_mail()`.

Sposób konfigurowania Django w celu wykorzystania serwera SMTP omówiłem w rozdziale 2., „Usprawnianie bloga i dodanie funkcjonalności społecznościowych”. Jeśli nie chcesz konfigurować ustawień e-maili, możesz polecić Django, by zapisywał e-maile do konsoli. Aby to zrobić, wprowadź poniższe ustawienie do pliku *settings.py*.

```
EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'
```

Wskazówka

Zadań asynchronicznych możesz używać nie tylko w długo wykonywanych procesach, ale również w innych procesach, które mogą zakończyć się niepowodzeniem lub niewymagających dużej ilości czasu, choć jednocześnie podatnych na awarie połączenia bądź też wymagających powtórzenia.

Teraz przygotowane zadanie musimy dodać do widoku `order_create`. Przejdź do pliku *views.py* w aplikacji `orders`, zaimportuj moduł `tasks`, a następnie, po wyczyszczeniu koszyka, wywołaj asynchroniczne zadanie `order_created` tak, jak pokazałem poniżej.

```
# ...
from .tasks import order_created

def order_create(request):
    # ...
    if request.method == 'POST':
        # ...
        if form.is_valid():
```

```
            # ...
            cart.clear()
            # Uruchomienie zadania asynchronicznego.
            order_created.delay(order.id)
    # ...
```

W celu wywołania zadania w sposób asynchroniczny, używamy metody `delay()`. Zadanie będzie dodane do kolejki i wykonane przez wątek roboczy tak wcześnie, jak to możliwe.

Upewnij się, że serwer RabbitMQ działa. Następnie zatrzymaj wątek roboczy Celery i uruchom go ponownie za pomocą polecenia:

```
celery -A myshop worker -l info
```

Wątek roboczy Celery właśnie zarejestrował zadanie. W innej powłoce uruchom serwer programistyczny za pomocą polecenia:

```
python manage.py runserver
```

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/`, dodaj kilka produktów do koszyka na zakupy, a następnie sfinalizuj proces składania zamówienia. W powłoce, w której uruchomiłeś wątek roboczy Celery, powinieneś zobaczyć dane wyjściowe podobne do przedstawionych poniżej.

```
[2024-01-02 20:25:19,569: INFO/MainProcess] Task
orders.tasks.order_created[a94dc22e-372b-4339-bff7-52bc83161c5c] received
...
[2024-01-02 20:25:19,605: INFO/ForkPoolWorker-8] Task
orders.tasks.order_created[a94dc22e-372b-4339-bff7-52bc83161c5c] succeeded in
↪0.015824042027816176s: 1
```

Zadanie `order_created` zostało wykonane i wysłano powiadomienie e-mail dotyczące zamówienia. Jeśli w roli backendu obsługi poczty e-mail używasz warstwy `console.Email↪Backend`, wiadomość e-mail nie zostanie fizycznie wysłana, ale w wynikach wyświetlanych na konsoli powinien wyświetlić się wyrenderowany tekst wiadomości e-mail.

Monitorowanie Celery za pomocą systemu Flower

Poza aplikacją zarządzającą RabbitMQ, do monitorowania zadań asynchronicznych wykonywanych za pomocą Celery, możesz używać także innych narzędzi. Flower to aplikacja webowa przeznaczona do monitorowania Celery. Dokumentację programu Flower znajdziesz na stronie *https://flower.readthedocs.io/*.

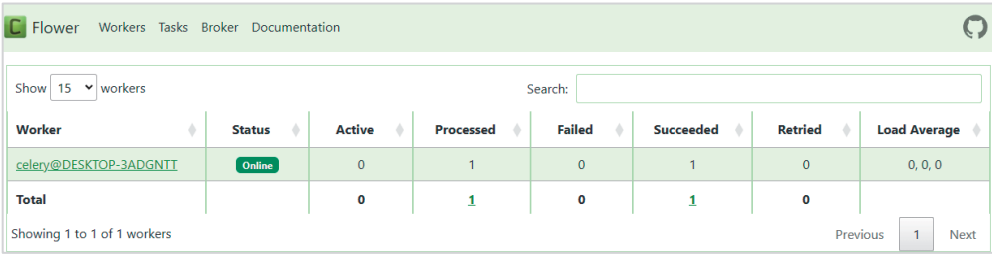
Aby zainstalować Flower, należy wydać poniższe polecenie.

```
python -m pip install flower==2.0.1
```

Po instalacji możesz uruchomić narzędzie Flower. W tym celu z poziomu katalogu projektu wydaj polecenie:

```
celery -A myshop flower
```

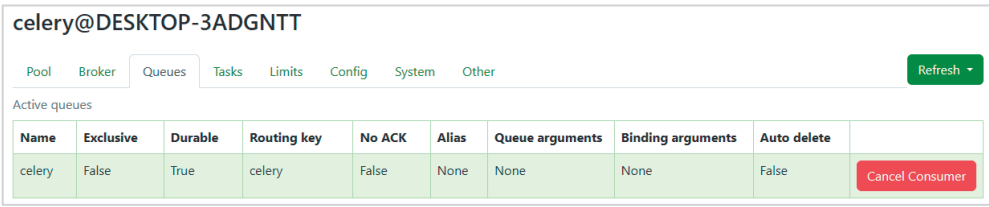
W przeglądarce internetowej przejdź pod adres `http://localhost:5555/`. Zobaczysz aktywne wątki robocze Celery oraz dane statystyczne dotyczące zadań asynchronicznych. Wynik powinien być podobny do pokazanego na rysunku 8.22.



Rysunek 8.22. Panel główny programu Flower

Zobaczysz aktywny wątek roboczy, którego nazwa zaczyna się od *celery@* i którego status to *Online*.

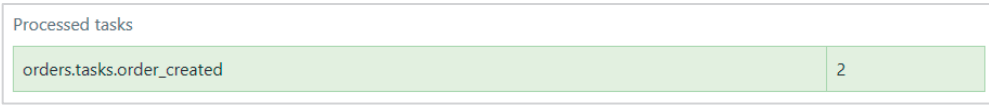
Kliknij nazwę wątku roboczego, a następnie kliknij zakładkę *Queues*. Wyświetli się ekran pokazany na rysunku 8.23.



Rysunek 8.23. System Flower — kolejki zadań wątku roboczego Celery

Na powyższym zrzucie ekranu widać aktywną kolejkę o nazwie *celery*. To jest konsument aktywnej kolejki połączony z brokerem komunikatów.

Kliknij zakładkę *Tasks*. Wyświetli się ekran pokazany na rysunku 8.24.



Rysunek 8.24. System Flower — zadania wątku roboczego Celery

Na tym ekranie widać zadania, które zostały przetworzone wraz z licznikami ich wykonania. Powinieneś zobaczyć zadanie `order_created` i całkowitą liczbę jego wykonania. Ta liczba może być różna, w zależności od liczby złożonych zamówień.

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/`. Dodaj kilka produktów do koszyka, a następnie dokończ proces realizacji zamówienia.

W przeglądarce internetowej przejdź pod adres `http://localhost:5555/`. System Flower zarejestrował zadanie jako przetworzone. Powinieneś teraz zobaczyć wartość `1` w polu *Processed* i wartość `1` w polu *Succeeded* (rysunek 8.25).

Worker	Status	Active	Processed	Failed	Succeeded	Retried	Load Average
celery@DESKTOP-3ADGNTT	Online	0	2	0	2	0	0, 0, 0
Total		0	2	0	2	0	

Rysunek 8.25. Flower — wątki robocze Celery

W polu *Tasks* możesz zobaczyć dodatkowe szczegóły dotyczące każdego zadania zarejestrowanego w Celery (rysunek 8.26).

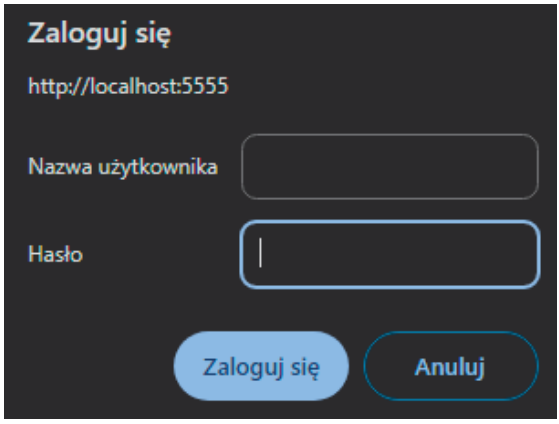
orders.tasks.order_created	547bfb16-6fd5-42af-a6cd-0c61036f1781	SUCCESS	(17,)	{}	1	2024-09-25 08:55:32.230	2024-09-25 08:55:32.232	0.03	celery@DESKTOP-3ADGNTT
orders.tasks.order_created	371c82f2-bdb3-4c82-a937-1caa70389023	SUCCESS	(16,)	{}	1	2024-09-25 08:49:44.688	2024-09-25 08:49:44.689	0.05	celery@DESKTOP-3ADGNTT

Rysunek 8.26. System Flower — zadania Celery

Nie należy otwarcie instalować narzędzia Flower w środowisku produkcyjnym bez zabezpieczeń. Dodajmy uwierzytelnianie do egzemplarza Flower. Zatrzymaj program Flower za pomocą kombinacji klawiszy *Ctrl+C* i uruchom go ponownie z opcją `--basic-auth` za pomocą polecenia:

```
celery -A myshop flower --basic-auth=user:pwd
```

Zastąp `user` i `pwd` własną nazwą użytkownika i hasłem. W przeglądarce internetowej przejdź pod adres `http://localhost:5555/`. Przeglądarka wyświetli teraz monit o podanie poświadczeń, jak pokazano na rysunku 8.27.



Rysunek 8.27. Podstawowe uwierzytelnianie wymagane do uzyskania dostępu do aplikacji Flower

Flower umożliwia zastosowanie także innych opcji uwierzytelniania, takich jak Google, GitHub lub Okta OAuth. Więcej o metodach uwierzytelniania Flower można przeczytać na stronie *https://flower.readthedocs.io/en/latest/auth.html*.

Podsumowanie

W tym rozdziale utworzyłeś prostą aplikację sklepu internetowego. Przygotowałeś katalog produktów i zbudowałeś koszyk na zakupy działający na bazie mechanizmu sesji. Zaimplementowałeś własny procesor kontekstu, aby koszyk na zakupy był dostępny dla wszystkich szablonów. Utworzyłeś również formularz pozwalający na składanie zamówień. Nauczyłeś się również implementować zadania asynchroniczne za pomocą Celery i RabbitMQ. Po lekturze tego rozdziału rozumiesz już podstawowe elementy budowania platformy e-commerce z Django, w tym zarządzanie produktami, przetwarzanie zamówień i obsługę zadań asynchronicznych. Teraz potrafisz budować projekty, które wydajnie przetwarzają transakcje użytkowników i skalują się, aby płynnie obsługiwać złożone operacje w tle.

W kolejnym rozdziale nauczysz się, jak zintegrować bramkę płatności ze swoim sklepem, dodać niestandardowe operacje do strony administracyjnej, eksportować dane w formacie CSV i dynamicznie generować pliki PDF.

Dodatkowe zasoby

Oto lista zasobów zawierających dodatkowe informacje związane z tematami omawianymi w tym rozdziale:

- Kod źródłowy przykładów do tego rozdziału — *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter08.*
- Pliki statyczne dla projektu — *https://github.com/PacktPublishing/Django-5-by-Example/tree/main/Chapter08/myshop/shop/static*.
- Ustawienia sesji Django — *https://docs.djangoproject.com/en/5.0/ref/settings/#sessions.*
- Wbudowane procesory kontekstu Django — *https://docs.djangoproject.com/en/5.0/ref/templates/api/#built-in-template-context-processors*.
- Dokumentacja klasy `RequestContext` — *https://docs.djangoproject.com/en/5.0/ref/templates/api/#django.template.RequestContext*.
- Dokumentacja Celery — *https://docs.celeryq.dev/en/stable/index.html.*
- Wprowadzenie do Celery — *https://docs.celeryq.dev/en/stable/getting-started/introduction.html*.
- Oficjalny obraz Docker RabbitMQ — *https://hub.docker.com/_/rabbitmq*.
- Instrukcje instalacji RabbitMQ — *https://www.rabbitmq.com/download.html.*
- Dokumentacja Flower — *https://flower.readthedocs.io/.*
- Metody uwierzytelniania w systemie Flower — *https://flower.readthedocs.io/en/latest/auth.html*.

Zarządzanie płatnościami i zamówieniami

Rozdział 9

W poprzednim rozdziale zbudowałeś prosty sklep internetowy wraz z katalogiem produktów i systemem składania zamówień. Nauczyłeś się korzystać z sesji Django i budować niestandardowe procesory kontekstu. Nauczyłeś się również implementować zadania asynchroniczne za pomocą Celery i RabbitMQ.

W tym rozdziale zajmiemy się zagadnieniem integracji bramki płatności z przygotowanym sklepem internetowym, aby umożliwić użytkownikom dokonywanie płatności za pomocą kart kredytowych i zarządzanie płatnościami za zamówienia. Ponadto rozbudujemy witrynę administracyjną o kolejne możliwości.

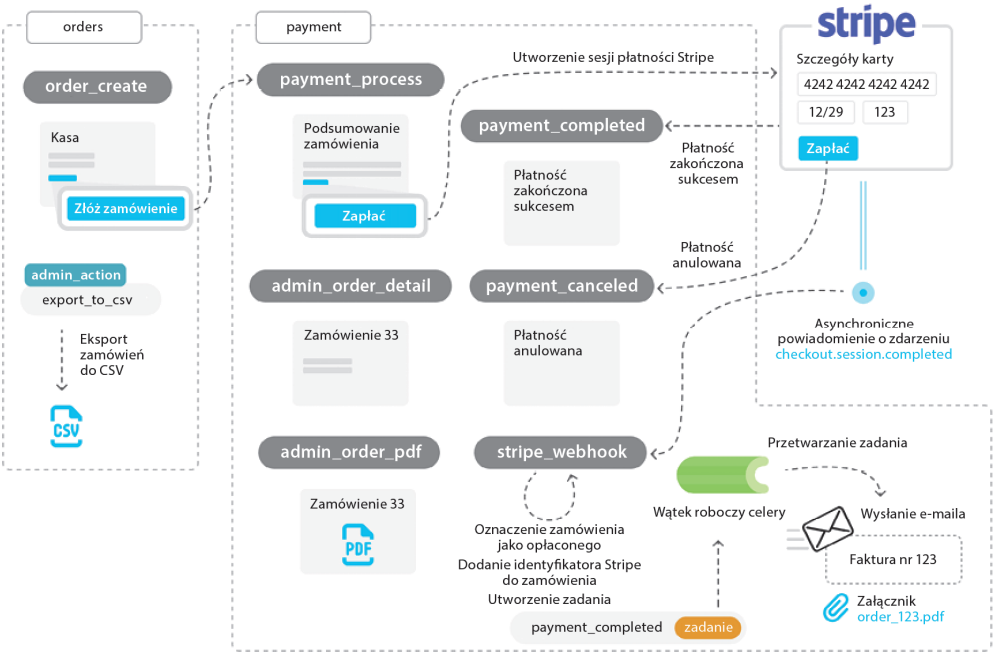
W tym rozdziale skoncentruję się na następujących zagadnieniach:

- Integracja z projektem bramki płatności Stripe
- Przetwarzanie płatności kartą kredytową za pomocą serwisu Stripe
- Obsługa powiadomień o płatnościach i oznaczanie zamówień jako opłacone
- Eksport zamówień do plików CSV
- Utworzenie własnych widoków dla witryny administracyjnej
- Dynamiczne generowanie rachunków w formacie PDF

Przegląd funkcjonalności

Reprezentację widoków, szablonów i funkcjonalności, które zbudujemy w tym rozdziale, przedstawiono na rysunku 9.1.

W tym rozdziale stworzysz nową aplikację płatności. Zaimplementujesz w niej widok `payment_process`, aby zainicjować sesję kasy pozwalającą opłacać zamówienia za pomocą bramki Stripe. Zbudujesz widok `payment_completed`, którego zadaniem będzie przekierowywanie użytkowników po udanych płatnościach, i widok `payment_canceled`, aby przekierowywać użytkowników, jeśli płatność zostanie anulowana. Zaimplementujesz także funkcję administratora `export_to_csv`, aby za pośrednictwem strony administracyjnej eksportować zamówienia w formacie CSV. Zbudujesz również widok administratora `admin_order_detail` do wyświetlania szczegółów zamówienia oraz widok `admin_order_pdf` do dynamicznego generowania faktur w formacie PDF. Zaimplementujesz webhook `stripe_webhook`, aby otrzymywać od Stripe asynchroniczne powiadomienia o płatnościach, jak też asynchroniczne zadanie `payment_completed`, aby wysyłać faktury do klientów po opłaceniu zamówień.



Rysunek 9.1. Schemat funkcjonalności zbudowanych w rozdziale 9.

Kod źródłowy przykładów z tego rozdziału można znaleźć pod adresem *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter09*.

Wszystkie pakiety Pythona użyte w tym rozdziale są zawarte w pliku *requirements.txt* dołączonym do plików z kodem źródłowym przykładów z tego rozdziału. Aby zainstalować każdy pakiet Pythona, możesz postępować zgodnie z instrukcjami zamieszczonymi w poniższych podrozdziałach. Możesz także zainstalować wszystkie wymagania jednocześnie za pomocą polecenia `pip install -r requirements.txt`.

Integracja bramki płatności

Bramka płatności to technologia używana przez sprzedawców do przetwarzania płatności online od klientów. Przy jej użyciu można zarządzać zamówieniami klientów i delegować przetwarzanie płatności do niezawodnego systemu firmy zewnętrznej. Dzięki wykorzystaniu zaufanej bramki płatności, nie będziesz musiał zajmować się złożonością techniczną, bezpieczeństwem i przepisami dotyczącymi przetwarzania kart kredytowych we własnym systemie.

Istnieje wielu dostawców bramek płatności, spośród których można wybierać. W naszym projekcie przedstawiam integrację z systemem Stripe, bardzo popularną bramką płatniczą wykorzystywaną między innymi przez takie serwisy internetowe, jak Shopify, Uber, Twitch czy GitHub.

Stripe dostarcza interfejs **API**, który umożliwia przetwarzanie płatności online za pomocą wielu metod płatności, takich jak karta kredytowa, Google Pay i Apple Pay. Więcej informacji o bramce płatniczej Stripe można znaleźć pod adresem *https://www.stripe.com/*.

Stripe dostarcza różnych produktów związanych z przetwarzaniem płatności. Między innymi pozwala zarządzać płatnościami jednorazowymi, płatnościami cyklicznymi za usługi subskrypcyjne oraz płatnościami wielostronnymi za korzystanie z platform handlowych.

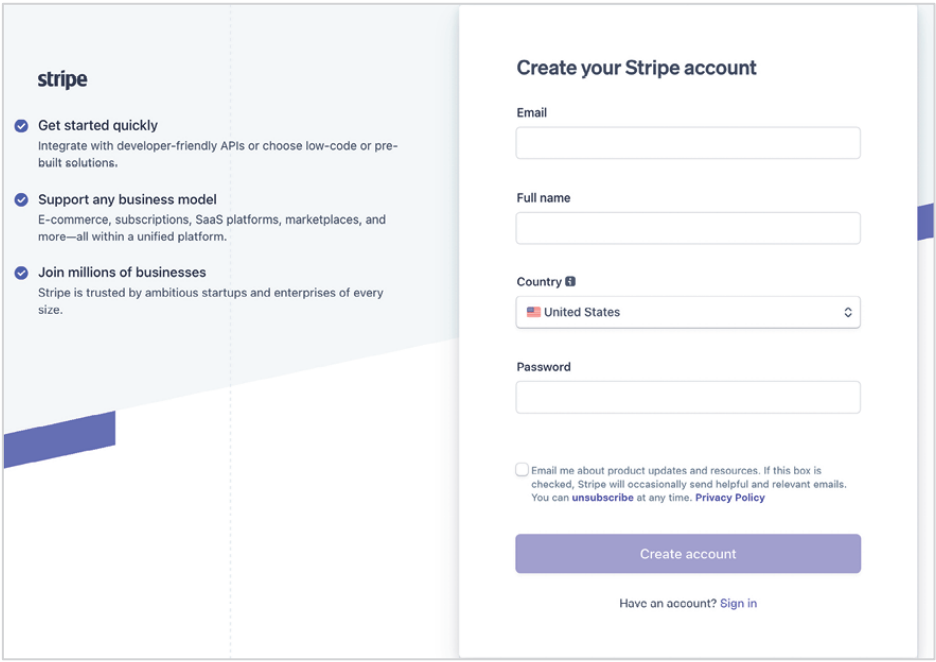
Stripe oferuje różne metody integracji, od formularzy płatności obsługiwanych przez Stripe po w pełni konfigurowalne przepływy płatności. W tym rozdziale zintegrujemy budowaną aplikację z produktem *Stripe Checkout*, na który składa się strona płatności zoptymalizowana pod kątem konwersji. Dzięki tej integracji użytkownicy będą mogli łatwo płacić za zamówione produkty kartą kredytową lub innymi metodami. Po dokonaniu płatności, serwis Stripe prześle do nas odpowiednie powiadomienia. Z dokumentacją produktu *Stripe Checkout* możesz zapoznać się pod adresem *https://stripe.com/docs/payments/checkout*.

Wykorzystanie produktu *Stripe Checkout* do przetwarzania płatności jest bezpieczne i zgodne z wymaganiami organizacji **Payment Card Industry** (**PCI**). Pozwala na odbieranie płatności, między innymi, za pośrednictwem serwisów Google Pay, Apple Pay, Afterpay, Alipay, poleceń zapłaty SEPA, Bacs i BECS, iDEAL, Sofort, GrabPay i FPX.

Utworzenie konta Stripe

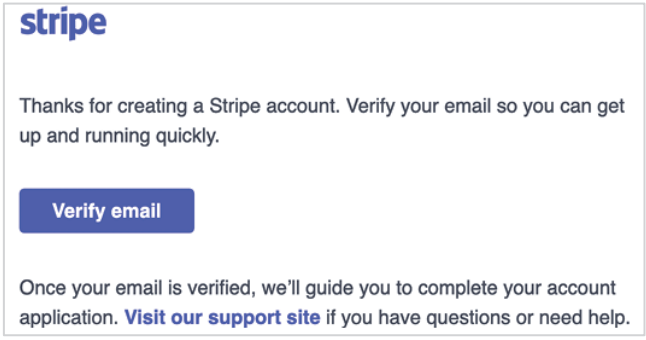
Aby zintegrować bramkę płatności z witryną, potrzebne jest konto w serwisie Stripe. Utwórzmy konto, aby przetestować API serwisu Stripe. W przeglądarce internetowej przejdź pod adres *https://dashboard.stripe.com/register*.

Wyświetli się formularz podobny do pokazanego na rysunku 9.2.



Rysunek 9.2. Formularz rejestracyjny Stripe

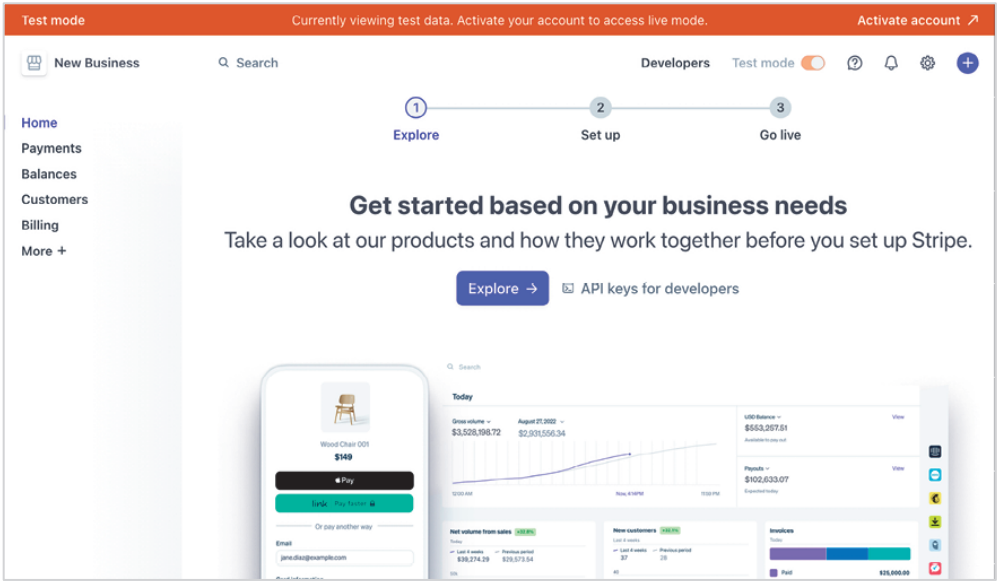
Wypełnij formularz własnymi danymi i kliknij przycisk *Create account*. Otrzymasz z serwisu Stripe wiadomość e-mail z linkiem do weryfikacji adresu e-mail. Strona będzie podobna do pokazanej na rysunku 9.3.



Rysunek 9.3. Wiadomość służąca do weryfikacji adresu e-mail

Otwórz wiadomość e-mail w swojej skrzynce odbiorczej i kliknij *Verify email*.

Zostaniesz przekierowany do ekranu głównego panelu Stripe, który będzie wyglądał jak na rysunku 9.4.

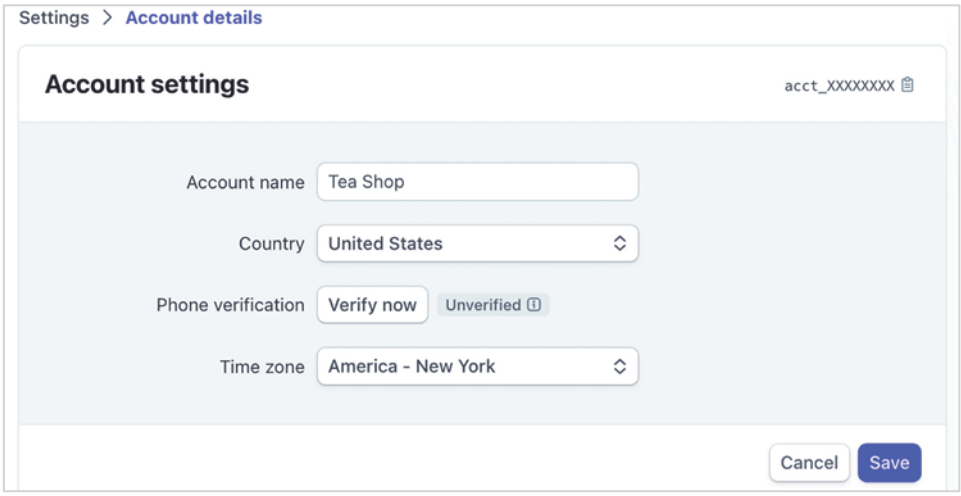


Rysunek 9.4. Panel główny Stripe po zweryfikowaniu adresu e-mail

W prawym górnym rogu ekranu widać, że jest włączony tryb testowy. Stripe zapewnia środowisko testowe i środowisko produkcyjne. Jeśli jesteś właścicielem firmy lub freelancerem, możesz dodać dane swojej firmy, aby aktywować konto i uzyskać dostęp do przetwarzania rzeczywistych płatności. Nie jest to jednak konieczne do zaimplementowania i przetestowania płatności za pośrednictwem Stripe. Do tego celu wystarczy środowisko testowe.

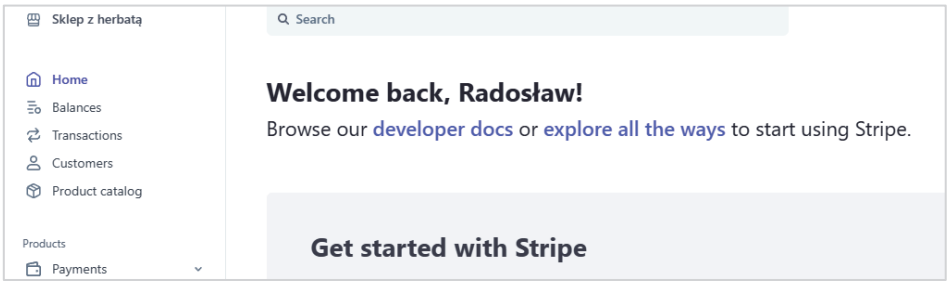
Aby przetwarzać płatności, trzeba dodać nazwę konta. W przeglądarce internetowej przejdź pod adres *https://dashboard.stripe.com/settings/account*.

Wyświetli się ekran pokazany na rysunku 9.5.



Rysunek 9.5. Ustawienia konta Stripe

W obszarze *Account name* wprowadź wybraną nazwę, a następnie kliknij *Save*. Wróć do panelu głównego serwisu Stripe. W nagłówku wyświetli się nazwa Twojego konta (rysunek 9.6).



Rysunek 9.6. Nagłówek panelu głównego serwisu Stripe zawierający nazwę konta

W następnym punkcie będziemy kontynuować instalację pakietu SDK serwisu Stripe i dodamy pakiet Stripe do projektu Django.

Instalowanie biblioteki Pythona obsługi serwisu Stripe

Serwis Stripe dostarcza moduł Pythona, który upraszcza obsługę API tego serwisu. W prezentowanym przykładzie zamierzamy zintegrować bramkę płatniczą z projektem za pomocą biblioteki `stripe`.

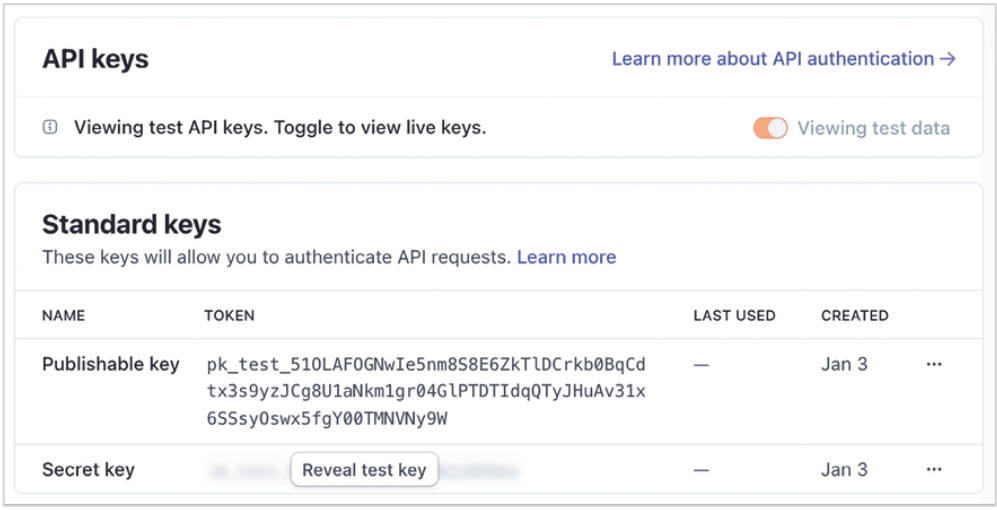
Kod źródłowy biblioteki Pythona Stripe można znaleźć pod adresem *https://github.com/stripe/stripe-python*.

Zainstaluj bibliotekę `stripe` z poziomu powłoki za pomocą polecenia:

```
python -m pip install stripe==9.3.0
```

Dodanie do projektu obsługi serwisu Stripe

W przeglądarce internetowej przejdź pod adres *https://dashboard.stripe.com/test/apikeys*. Dostęp do tej strony możesz uzyskać również ze strony panelu głównego Stripe. Aby to zrobić, kliknij pozycję *Developers*, a następnie kliknij *API keys*. Wyświetli się ekran pokazany na rysunku 9.7.



Rysunek 9.7. Ekran z testowymi kluczami API serwisu Stripe

Serwis Stripe dostarcza kluczy API dla dwóch różnych środowisk: testowego i produkcyjnego. Dla każdego środowiska dostępny jest *klucz do publikacji* i *klucz tajny*. Klucze do publikacji dla trybu testowego mają przedrostek `pk_test_`, natomiast klucze do publikacji dla trybu produkcyjnego mają przedrostek `pk_live_`. Klucze tajne dla trybu testowego mają przedrostek `sk_test_`, natomiast klucze tajne dla trybu produkcyjnego mają przedrostek `sk_live_`.

Będziesz potrzebował tych informacji do uwierzytelnienia żądań do interfejsu API serwisu Stripe. Klucz prywatny powinieneś zachować w tajemnicy i przechowywać w bezpiecznym miejscu. Klucz do publikacji może być wykorzystywany w kodzie po stronie klienta — na przykład w skryptach JavaScript. Więcej informacji na temat kluczy API w serwisie Stripe znajdziesz na stronie *https://stripe.com/docs/keys*.

Aby ułatwić oddzielenie konfiguracji od kodu, skorzystamy z biblioteki `python-decouple`. Z tej biblioteki korzystałeś już w rozdziale 2., „Usprawnianie bloga i dodanie funkcjonalności społecznościowych”.

W katalogu głównym projektu utwórz nowy plik i nadaj mu nazwę *.env*. Plik *.env* zawiera pary klucz-wartość zmiennych środowiskowych. Dodaj do nowego pliku poświadczenia Stripe w następujący sposób.

```
STRIPE_PUBLISHABLE_KEY=pk_test_XXXX
STRIPE_SECRET_KEY=sk_test_XXXX
```

Zastąp wartości `STRIPE_PUBLISHABLE_KEY` i `STRIPE_SECRET_KEY` testowym kluczem do publikacji i kluczem tajnym pobranymi z serwisu Stripe.

Jeśli do przechowywania swojego kodu korzystasz z repozytorium git, zadbaj o dołączenie pliku *.env* do pliku *.gitignore* w katalogu swojego repozytorium. Dzięki temu unikniesz nieumyślnego przesłania poświadczeń do repozytorium.

Zacznij od zainstalowania pakietu `python-decouple` za pomocą menedżera `pip`. W tym celu uruchom polecenie:

```
python -m pip install python-decouple==3.8
```

Przeprowadź edycję pliku *settings.py* swojego projektu i dodaj poniższy fragment kodu.

```
from decouple import config
# ...
STRIPE_PUBLISHABLE_KEY = config('STRIPE_PUBLISHABLE_KEY')
STRIPE_SECRET_KEY = config('STRIPE_SECRET_KEY')
STRIPE_API_VERSION = '2024-04-10'
```

W tym projekcie będziemy używać Stripe API w wersji `2024-04-10`. Informacje o tej wersji API można znaleźć pod adresem *https://stripe.com/docs/upgrades#2024-04-10*.

Uwaga

W tym projekcie będziemy używać kluczy dla środowiska testowego. Po wdrożeniu systemu do produkcji i zweryfikowaniu konta Stripe otrzymasz klucze środowiska produkcyjnego. Sposób konfigurowania ustawień dla wielu środowisk opisałem w rozdziale 17., „Wdrożenie".

Zintegrujmy bramkę płatności z procesem realizacji transakcji. Dokumentację biblioteki Pythona zapewniającej obsługę serwisu Stripe znajdziesz pod adresem *https://stripe.com/docs/api?lang=python*.

Budowanie procesu płatności

Proces złożenia zamówienia powinien przedstawiać się następująco.

1. Użytkownik dodaje produkty do koszyka na zakupy.
2. Użytkownik decyduje się na złożenie zamówienia.
3. Użytkownik wprowadza dane karty kredytowej i dokonuje zapłaty.

Do zarządzania płatnościami zamierzamy opracować nową aplikację. Za pomocą poniższego polecenia utwórz w projekcie nową aplikację.

```
python manage.py startapp payment
```

Przeprowadź edycję pliku *settings.py* projektu i dodaj nową aplikację na liście INSTALLED_APPS, tak jak pokazałem poniżej. The new line is highlighted in bold:

```
INSTALLED_APPS = [
    # ...
    'cart.apps.CartConfig',
    'orders.apps.OrdersConfig',
    'payment.apps.PaymentConfig',
    'shop.apps.ShopConfig',
]
```

Aplikacja payment jest teraz aktywna w projekcie.

W tej chwili użytkownicy mogą składać zamówienia, ale nie mogą za nie płacić. Gdy klient złoży zamówienie, trzeba przekierować go do procesu płatności.

Przeprowadź edycję pliku *views.py* aplikacji orders i dodaj instrukcje importu:

```
from django.shortcuts import render, get_object_or_404, redirect
```

W tym samym pliku znajdź w widoku order_create wiersze:

```
# Uruchomienie zadania asynchronicznego.
order_created.delay(order.id)
return render(
    request, 'orders/order/created.html', {'order': order}
)
```

Zastąp je poniższym fragmentem kodu.

```
# Uruchomienie zadania asynchronicznego.
order_created.delay(order.id)
# Umieszczenie zamówienia w sesji.
request.session['order_id'] = order.id
# Przekierowanie do płatności.
return redirect('payment:process')
```

Zmodyfikowany widok powinien mieć następującą postać.

```
from django.shortcuts import redirect, render
# ...

def order_create(request):
    cart = Cart(request)
    if request.method == 'POST':
        form = OrderCreateForm(request.POST)
        if form.is_valid():
            order = form.save()
            for item in cart:
                OrderItem.objects.create(
                    order=order,
                    product=item['product'],
                    price=item['price'],
                    quantity=item['quantity'],
                )
            # Wyczyść zawartość koszyka
            cart.clear()
            # Uruchomienie zadania asynchronicznego.
```

```
            order_created.delay(order.id)
            # Umieszczenie zamówienia w sesji.
            request.session['order_id'] = order.id
            # Przekierowanie do płatności.
            return redirect('payment:process')
    else:
        form = OrderCreateForm()
    return render(
        request,
        'orders/order/create.html',
        {'cart': cart, 'form': form},
    )
```

Zamiast renderowania szablonu *orders/order/created.html* podczas składania nowego zamówienia, identyfikator zamówienia jest przechowywany w sesji użytkownika, a użytkownik jest przekierowywany do adresu URL `paymet:process`. Ten adres URL zaimplementujemy wkrótce. Pamiętaj, że aby umieścić w kolejce zadanie `order_created` i je uruchomić, musisz uruchomić Celery.

Teraz przystąpimy do integracji bramki płatności.

Integracja bramki płatności Stripe Checkout

Integracja mechanizmu Stripe Checkout składa się z wyświetlenia hostowanej przez Stripe ze strony kasy umożliwiającej użytkownikowi wprowadzanie danych płatności (zwykle karty kredytowej) i pobrania płatności. Jeśli płatność się powiedzie, Stripe przekierowuje klienta na stronę z komunikatem o powodzeniu operacji. Jeśli płatność zostanie przez klienta anulowana, Stripe przekierowuje użytkownika na stronę z komunikatem o anulowaniu operacji.

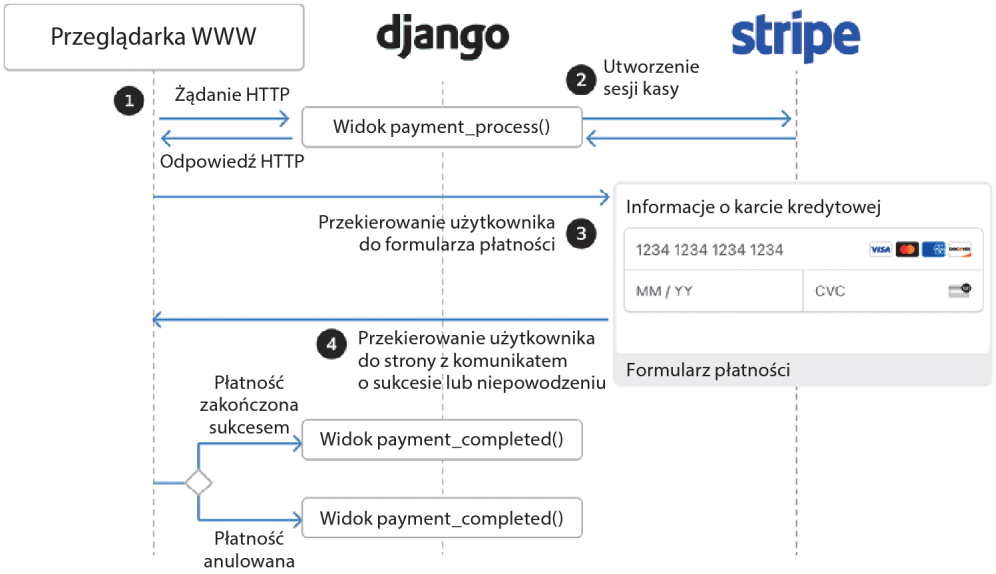
Zaimplementujemy trzy widoki.

- `payment_process`. Tworzy sesję *Stripe Checkout* i przekierowuje klienta do formularza płatności hostowanego przez serwis Stripe. Sesja *Stripe Checkout* to programowa reprezentacja tego, co widzi klient, gdy zostaje przekierowany do formularza płatności, w tym listy produktów, ilości, waluty i kwoty do zapłaty.
- `payment_completed`. Wyświetla komunikat o pomyślnych płatnościach. Widok do przekierowania użytkownika po udanym dokonaniu płatności.
- `payment_canceled`. Wyświetla komunikat o anulowanych płatnościach. Widok do przekierowania użytkownika po anulowaniu płatności.

Przepływ płatności został pokazany na rysunku 9.8.

Proces realizacji transakcji będzie przebiegać następująco.

1. Po utworzeniu zamówienia użytkownik zostaje przekierowany do widoku `payment_process`. Wyświetla się podsumowanie zamówienia oraz przycisk umożliwiający dokonanie płatności.
2. Gdy użytkownik przechodzi do płatności, tworzona jest sesja kasy serwisu Stripe. Sesja kasy obejmuje listę produktów, które użytkownik zamierza kupić, adres URL, do którego nastąpi przekierowanie użytkownika po udanej płatności, oraz adres URL, do którego nastąpi przekierowanie użytkownika w przypadku anulowania płatności.



Rysunek 9.8. Przepływ płatności na stronie kasy

3. Widok przekierowuje użytkownika do strony kasy hostowanej przez serwis Stripe. Ta strona zawiera formularz płatności. Użytkownik wprowadza dane swojej karty kredytowej i przesyła formularz.
4. Stripe przetwarza płatność i przekierowuje klienta do widoku `payment_completed`. Jeśli klient nie dokończy płatności, Stripe przekierowuje go do widoku `payment_canceled`.

Przystępujemy do budowania widoków płatności. Przeprowadź edycję pliku *views.py* aplikacji `payment` i umieść w nim poniższy fragment kodu.

```
from decimal import Decimal
import stripe
from django.conf import settings
from django.shortcuts import get_object_or_404, redirect, render
from django.urls import reverse
from orders.models import Order

# Utworzenie egzemplarza bramki płatności Stripe
stripe.api_key = settings.STRIPE_SECRET_KEY
stripe.api_version = settings.STRIPE_API_VERSION


def payment_process(request):
    order_id = request.session.get('order_id')
    order = get_object_or_404(Order, id=order_id)

    if request.method == 'POST':
        success_url = request.build_absolute_uri(
            reverse('payment:completed')
        )
```

```
            cancel_url = request.build_absolute_uri(
                reverse('payment:canceled')
            )
            # Dane sesji transakcji Stripe
            session_data = {
                'mode': 'payment',
                'client_reference_id': order.id,
                'success_url': success_url,
                'cancel_url': cancel_url,
                'line_items': [],
            }
            # utworzenie sesji Stripe Checkout
            session = stripe.checkout.Session.create(**session_data)
            # przekierowanie do formularza płatności Stripe
            return redirect(session.url, code=303)

        else:
            return render(request, 'payment/process.html', locals())
```

W powyższym kodzie importujemy moduł `stripe` i ustawiamy klucz API serwisu Stripe za pomocą ustawienia `STRIPE_SECRET_KEY`. Ustawiamy również wykorzystywaną wersję interfejsu API za pomocą wartości opcji `STRIPE_API_VERSION`.

Widok `payment_process` wykonuje następujące zadania.

1. Na podstawie klucza sesji `order_id`, który został wcześniej ustawiony w widoku `order_create`, pobieramy bieżące zamówienie.
2. Dla podanego identyfikatora pobieramy obiekt `Order`. Jeśli zamówienie o podanym identyfikatorze nie zostanie znalezione, za pomocą funkcji skrótu `get_object_or_404()` zgłaszamy wyjątek `Http404` (nie znaleziono strony).
3. Jeśli widok jest ładowany z wykorzystaniem żądania `GET`, renderowany jest szablon *payment/process.html*, który jest następnie wyświetlany. Ten szablon będzie zawierał podsumowanie zamówienia oraz przycisk umożliwiający dokonanie płatności. Kliknięcie tego przycisku spowoduje wygenerowanie żądania `POST` do widoku.
4. Jeśli widok jest ładowany za pomocą żądania `POST`, za pomocą wywołania `stripe.checkout.Session.create()` tworzona jest sesja kasy serwisu Stripe z parametrami:
   - `mode`. tryb sesji kasy. Dla płatności jednorazowej używamy trybu `payment`. Różne wartości akceptowane w przypadku tego parametru są dostępne na stronie *https://stripe.com/docs/api/checkout/sessions/object#checkout_session_object-mode*.
   - `client_reference_id`. unikatowy numer referencyjny dla tej płatności. Wartość tę wykorzystamy do uzgodnienia sesji płatności Stripe z wybranym zamówieniem. Dzięki przekazaniu identyfikatora zamówienia, łączymy w systemie płatności Stripe z zamówieniami. Dzięki temu będziemy mogli otrzymywać z serwisu Stripe powiadomienia o płatności, co pozwala oznaczyć zamówienia jako opłacone.

- `success_url`. widok do przekierowania użytkownika po pomyślnym dokonaniu płatności. Do wygenerowania bezwzględnego adresu URI na podstawie ścieżki URL, używamy wywołania `request.build_absolute_uri()`. Dokumentacja tej metody jest dostępna na stronie *https://docs.djangoproject.com/en/5.0/ref/request-response/#django.http.HttpRequest.build_absolute_uri*.
- `cancel_url`. widok do przekierowania użytkownika w przypadku anulowania płatności.
- `line_items`. Pusta lista. W dalszej części kodu wypełnimy ją pozycjami zamówienia odpowiadającymi produktom, które mają być zakupione.

5. Po utworzeniu sesji kasy zwracamy przekierowanie HTTP z kodem stanu `303`, co powoduje przekierowanie użytkownika do serwisu Stripe. Kod statusu `303` jest zalecany do przekierowywania aplikacji internetowych do nowego identyfikatora URI po wykonaniu żądania HTTP `POST`.

Wszystkie parametry potrzebne do utworzenia obiektu sesji Stripe można znaleźć na stronie *https://stripe.com/docs/api/checkout/sessions/create*.

Aby utworzyć sesję kasy, wypełnijmy listę `line_items` pozycjami zamówienia. Każdy produkt będzie opisany nazwą, kwotą do pobrania, walutą, której należy użyć, oraz zakupioną ilością.

Dodaj do widoku `payment_process` następujący kod wyróżniony pogrubioną czcionką.

```
def payment_process(request):
    order_id = request.session.get('order_id')
    order = get_object_or_404(Order, id=order_id)
    if request.method == 'POST':
        success_url = request.build_absolute_uri(
            reverse('payment:completed')
        )
        cancel_url = request.build_absolute_uri(
            reverse('payment:canceled')
        )
        # Dane sesji transakcji Stripe
        session_data = {
            'mode': 'payment',
            'client_reference_id': order.id,
            'success_url': success_url,
            'cancel_url': cancel_url,
            'line_items': []
        }
        # dodanie do sesji Stripe Checkout pozycji zamówienia
        for item in order.items.all():
            session_data['line_items'].append(
                {
                    'price_data': {
                        'unit_amount': int(item.price * Decimal('100')),
                        'currency': 'usd',
                        'product_data': {
                            'name': item.product.name,
                        },
```

```
                    },
                    'quantity': item.quantity,
                }
            )
        # utworzenie sesji Stripe Checkout
        session = stripe.checkout.Session.create(**session_data)
        # przekierowanie do formularza płatności Stripe
        return redirect(session.url, code=303)

    else:
        return render(request, 'payment/process.html', locals())
```

Dla każdej pozycji użyliśmy następujących informacji.

- `price_data`. Informacje dotyczące ceny.
  - `unit_amount`. Kwota płatności do pobrania wyrażona w groszach. Jest to dodatnia liczba całkowita reprezentująca wysokość opłaty wyrażona w najmniejszej jednostce waluty, bez miejsc po przecinku. Na przykład, aby obciążyć klienta kwotą 10,00 zł, wartość ta powinna wynosić `1000` (czyli 1000 groszy). W celu uzyskania wartości w groszach cena towaru, `item.price`, jest mnożona przez `Decimal('100')`, a następnie konwertowana na liczbę całkowitą.
  - `currency`. Używana waluta reprezentowana przez trzyliterowy format ISO. Dla kwoty w złotych używamy symbolu PLN. Listę obsługiwanych walut można zobaczyć na stronie *https://stripe.com/docs/currencies*.
- `product_data`. Informacje dotyczące produktu.
  - `name`. Nazwa produktu.
- `quantity`. Zakupowana liczba jednostek.

Widok `payment_process` jest teraz gotowy. Stworzymy teraz proste widoki stron do wyświetlania komunikatów o pomyślnej płatności oraz o jej anulowaniu.

Umieść poniższy fragment kodu w pliku *views.py* aplikacji `payment`.

```
def payment_completed(request):
    return render(request, 'payment/completed.html')

def payment_canceled(request):
    return render(request, 'payment/canceled.html')
```

Utwórz w katalogu aplikacji `payment` nowy plik i nadaj mu nazwę *urls.py*. Następnie w pliku umieść poniższy fragment kodu.

```
from django.urls import path
from . import views

app_name = 'payment'

urlpatterns = [
    path('process/', views.payment_process, name='process'),
    path('completed/', views.payment_completed, name='completed'),
    path('canceled/', views.payment_canceled, name='canceled'),
]
```

To są adresy URL wykorzystywane podczas przetwarzania płatności. Obejmują następujące wzorce adresów URL.

- `process`. Widok, który wyświetla użytkownikowi podsumowanie zamówienia, tworzy sesję płatności Stripe i przekierowuje użytkownika do formularza płatności hostowanego przez serwis Stripe.
- `completed`. Widok do przekierowania użytkownika po udanym dokonaniu płatności.
- `canceled`. Widok do przekierowania użytkownika w przypadku, gdy płatność się nie powiedzie.

Przeprowadź edycję głównego pliku *urls.py* projektu `myshop` i dodaj do niego wzorce adresu URL dla aplikacji `payment`.

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('cart/', include('cart.urls', namespace='cart')),
    path('orders/', include('orders.urls', namespace='orders')),
    path('payment/', include('payment.urls', namespace='payment')),
    path('', include('shop.urls', namespace='shop')),
]
```

Pamiętaj o umieszczeniu go przed wzorcem *shop.urls*, aby uniknąć dopasowania nieprawidłowego wzorca. Należy wziąć pod uwagę, że Django przetwarza poszczególne wzorce URL po kolei i zatrzymuje się na pierwszym, który pasuje do żądanego adresu URL.

Przystępujemy więc do przygotowania szablonu dla wymienionych widoków. W aplikacji `payment` utwórz poniższą strukturę plików i katalogów.

```
templates/
    payment/
        process.html
        completed.html
        canceled.html
```

Przeprowadź edycję szablonu *payment/process.html* i umieść w nim poniższy fragment kodu.

```
{% extends "shop/base.html" %}
{% load static %}

{% block title %}Zapłać za zamówienie{% endblock %}

{% block content %}
  <h1>Podsumowanie zamówienia</h1>
  <table class="cart">
    <thead>
      <tr>
        <th>Zdjęcie</th>
        <th>Produkt</th>
        <th>Cena</th>
        <th>Ilość</th>
        <th>Razem</th>
      </tr>
    </thead>
    <tbody>
```

```
        {% for item in order.items.all %}
          <tr class="row{% cycle "1" "2" %}">
            <td>
              <img src="{% if item.product.image %}{{ item.product.image.url }}
              {% else %}{% static "img/no_image.png" %}{% endif %}">
            </td>
            <td>{{ item.product.name }}</td>
            <td class="num">{{ item.price }} PLN</td>
            <td class="num">{{ item.quantity }}</td>
            <td class="num">{{ item.get_cost }} PLN</td>
          </tr>
        {% endfor %}
        <tr class="total">
          <td colspan="4">Razem</td>
          <td class="num">{{ order.get_total_cost }} PLN</td>
        </tr>
      </tbody>
    </table>
    <form action="{% url "payment:process" %}" method="post">
      <input type="submit" value="Zapłać teraz">
      {% csrf_token %}
    </form>
  {% endblock %}
```

To jest szablon, który służy do wyświetlania użytkownikowi podsumowania zamówienia i umożliwia dokonanie przez niego płatności. Zawiera formularz i przycisk *Zapłać teraz*, który powoduje przesłanie formularza metodą `POST`. Po przesłaniu formularza widok `payment_process` tworzy sesję kasy serwisu Stripe i przekierowuje użytkownika do hostowanego przez Stripe formularza płatności.

Przeprowadź edycję szablonu *payment/completed.html* i umieść w nim poniższy fragment kodu.

```
{% extends "shop/base.html" %}

{% block title %}Płatność zakończyła się sukcesem.{% endblock %}

{% block content %}
  <h1>Twoja płatność powiodła się</h1>
  <p>Twoja płatność została pomyślnie przetworzona.</p>
{% endblock %}
```

Szablon jest przeznaczony dla strony, na którą użytkownik zostanie przekierowany po udanym wykonaniu płatności.

Teraz przeprowadź edycję szablonu *payment/canceled.html* i umieść w nim poniższy fragment kodu.

```
{% extends "shop/base.html" %}

{% block title %}Płatność anulowana{% endblock %}

{% block content %}
  <h1>Twoja płatność nie została przetworzona</h1>
  <p>Wystąpił problem podczas przetwarzania płatności.</p>
{% endblock %}
```

Ten szablon jest przeznaczony dla strony, na którą użytkownik zostanie przekierowany, gdy transakcja się nie powiedzie.

Zaimplementowaliśmy niezbędne widoki do przetwarzania płatności, w tym wzorce i szablony adresów URL. Spróbujmy teraz przetestować proces płatności.

Testowanie płatności

Aby uruchomić serwer RabbitMQ w kontenerze Docker, wykonaj w powłoce polecenie:

```
docker run -it --rm --name rabbitmq -p 5672:5672 -p 15672:15672
rabbitmq:3.13.1-management
```

Spowoduje to uruchomienie serwera RabbitMQ w porcie `5672` i webowego interfejsu zarządzania w porcie `15672`.

Otwórz inną powłokę i z katalogu projektu uruchom wątek roboczy Celery przy użyciu polecenia:

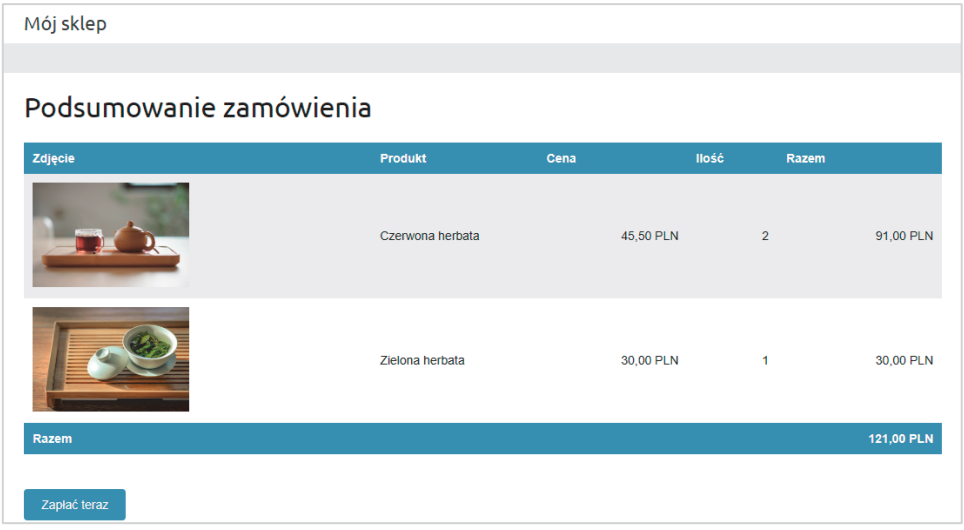
```
celery -A myshop worker -l info
```

Otwórz jeszcze jedną powłokę i uruchom serwer programistyczny, korzystając z polecenia.

```
python manage.py runserver
```

Otwórz w przeglądarce stronę `http://127.0.0.1:8000/`, dodaj kilka produktów do koszyka na zakupy i wypełnij formularz finalizacji transakcji. Kliknij przycisk *Złóż zamówienie*. Zamówienie zostanie zapisane w bazie danych, w bieżącej sesji zostanie zapisany identyfikator zamówienia oraz nastąpi przekierowanie do strony procesu płatności.

Strona procesu płatności będzie wyglądać jak na rysunku 9.9.



Rysunek 9.9. Strona procesu płatności zawierająca podsumowanie zamówienia

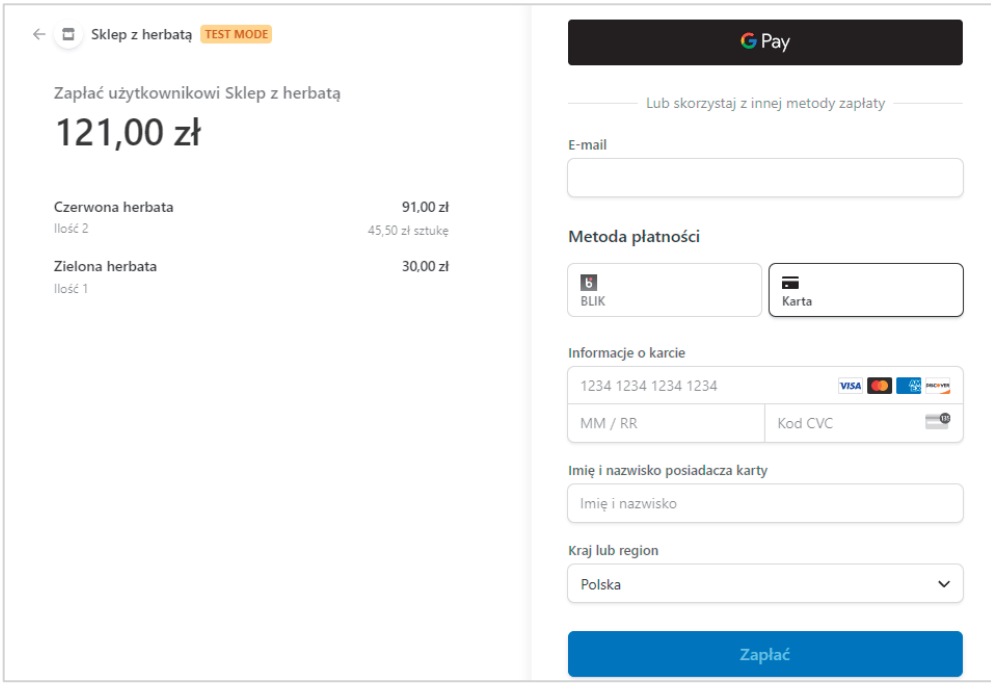
Uwaga

Zdjęcia wykorzystane w tym rozdziale:

- *Zielona herbata:* zdjęcie autorstwa Jia Ye pochodzące z serwisu Unsplash.
- *Czerwona herbata:* Zdjęcie autorstwa Manki Kim pochodzące z serwisu Unsplash.

Na tej stronie możesz zobaczyć podsumowanie zamówienia i przycisk *Zapłać teraz*. Kliknij *Zapłać teraz*. Widok `payment_process` utworzy sesję kasową Stripe i zostaniesz przekierowany do formularza płatności hostowanego przez serwis Stripe.

Powinieneś zobaczyć stronę pokazaną na rysunku 9.10.



Rysunek 9.10. Formularz kasy w serwisie Stripe

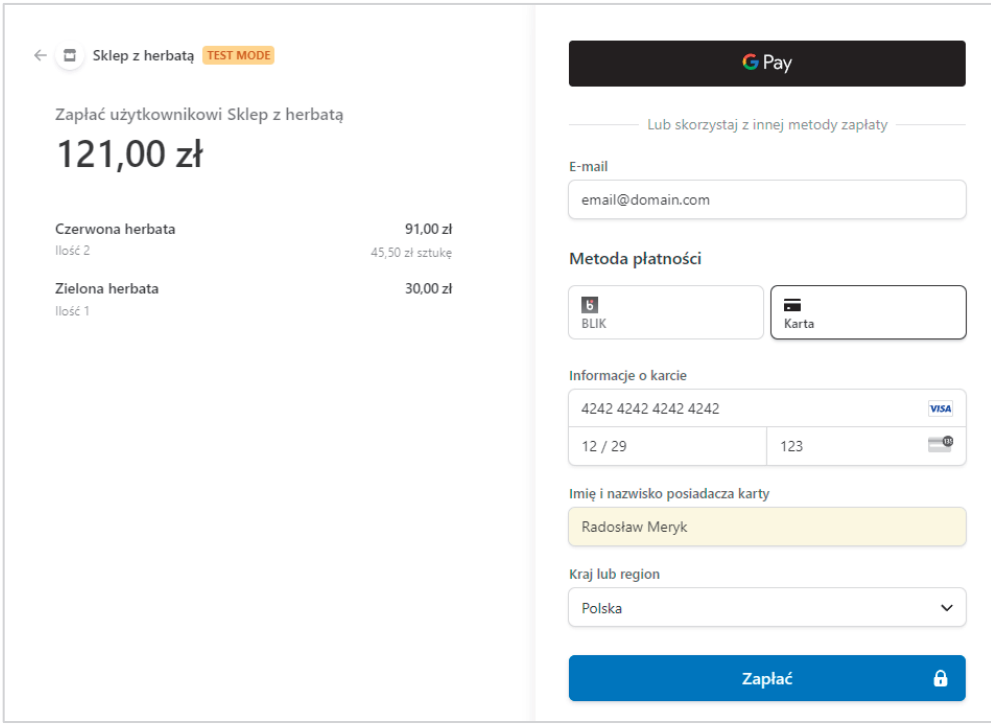
Korzystanie z testowych kart kredytowych

Stripe udostępnia testowe karty kredytowe dla różnych wydawców i krajów. Pozwala to symulować płatności w celu przetestowania wszystkich możliwych scenariuszy (płatność pomyślna, płatność odrzucona, itp.). Niektóre karty, które można przetestować pod kątem różnych scenariuszy, zostały zestawione w poniższej tabeli.

Wynik	Testowa karta kredytowa	CVC	Data wygaśnięcia
Pomyślna płatność	4242 4242 4242 4242	Dowolne 3 cyfry	Dowolna data w przyszłości
Nieudana płatność	4000 0000 0000 0002	Dowolne 3 cyfry	Dowolna data w przyszłości
Wymaganie bezpiecznego uwierzytelniania 3D	4000 0025 0000 3155	Dowolne 3 cyfry	Dowolna data w przyszłości

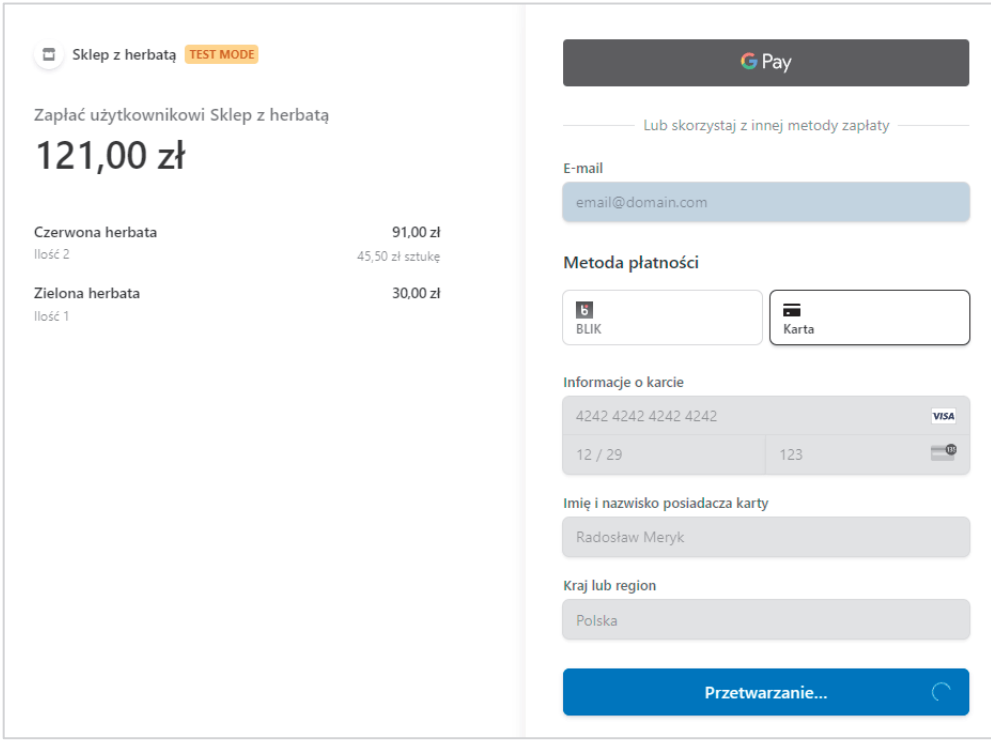
Pełną listę kart kredytowych do testowania można znaleźć na stronie *https://stripe.com/docs/testing*.

Zamierzamy użyć karty testowej VISA o numerze `4242 4242 4242 4242`, która zwraca sukces. Użyjemy kodu CVC `123` i dowolnej daty wygaśnięcia z przyszłości, na przykład `12/29`. Wprowadź w formularzu płatności dane karty kredytowej w sposób pokazany na rysunku 9.11.



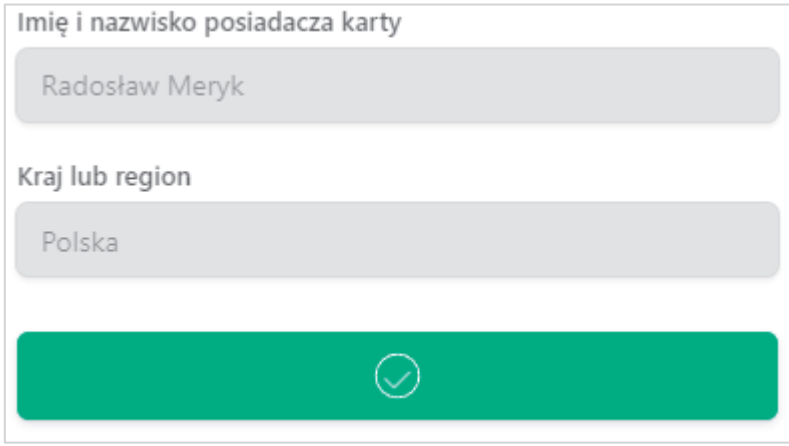
Rysunek 9.11. Formularz płatności z prawidłowymi danymi testowej karty kredytowej

Kliknij przycisk *Zapłać*. Tekst przycisku zmieni się na **Przetwarzanie...**, jak na rysunku 9.12.



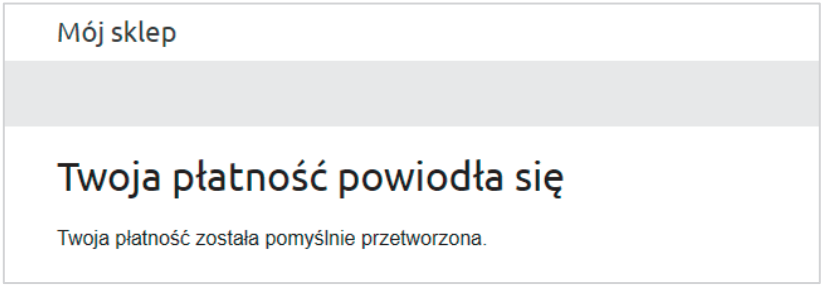
Rysunek 9.12. Formularz płatności podczas jej przetwarzania

Po kilku sekundach przycisk zmieni kolor na zielony, jak na rysunku 9.13.



Rysunek 9.13. Formularz płatności po pomyślnym wykonaniu płatności

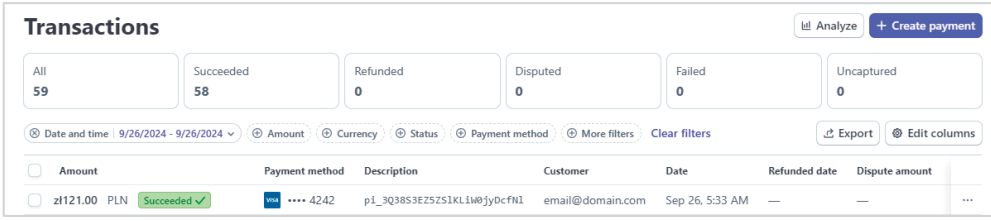
Następnie Stripe przekieruje Twoją przeglądarkę do adresu URL płatności zakończonej sukcesem, podanego podczas tworzenia sesji kasy. Powinieneś zobaczyć stronę pokazaną na rysunku 9.14.



Rysunek 9.14. Strona pomyślnie zakończonej udanej płatności

Sprawdzanie informacji o płatności w panelu głównym serwisu Stripe

Otwórz w przeglądarce główny panel serwisu Stripe pod adresem *https://dashboard.stripe.com/test/payments*. W obszarze *Payments* wyświetli się wykonana płatność (rysunek 9.15).



Rysunek 9.15. Obiekt płatności ze statusem Succeeded w panelu głównym serwisu Stripe

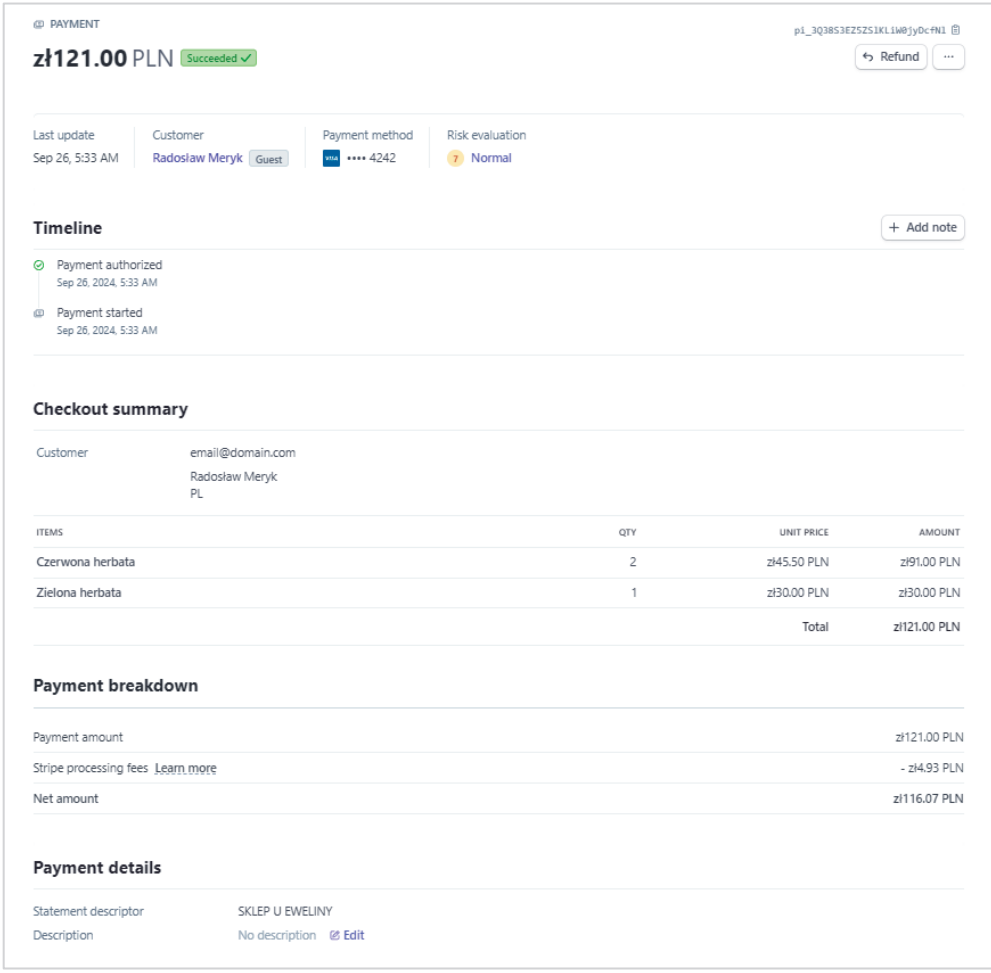
Transakcja ma status **Succeeded**. Opis płatności zawiera identyfikator zlecenia płatności rozpoczynający się od `pi_`. Po potwierdzeniu sesji kasowej Stripe tworzy powiązane z sesją zlecenie płatności. Zlecenie płatności służy do pobrania płatności od użytkownika. Serwis Stripe rejestruje jako zlecenia płatności wszystkie próby płatności. Każde zlecenie płatności ma unikatowy identyfikator i zawiera szczegóły transakcji, takie jak obsługiwane metody płatności, kwotę do pobrania i wybraną walutę. Aby uzyskać dostęp do szczegółów płatności, kliknij interesującą Cię transakcję.

Wyświetli się ekran pokazany na rysunku 9.16.

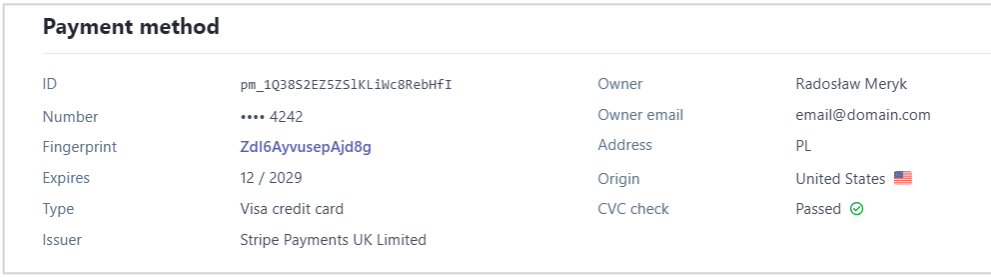
Na tym ekranie możesz zobaczyć informacje o płatności i harmonogram płatności, w tym zmiany jej statusu. W obszarze *Checkout summary* wyświetlają się zakupione pozycje, w tym nazwa, ilość, cena jednostkowa i kwota.

W obszarze *Payment details* wyświetla się zestawienie wpłaconej kwoty oraz opłata naliczona przez serwis Stripe za przetworzenie płatności.

W tym obszarze znajduje się panel *Payment details*, zawierający szczegółowe informacje na temat metody płatności i weryfikacji karty kredytowej przeprowadzanych przez serwis Stripe (rysunek 9.17).

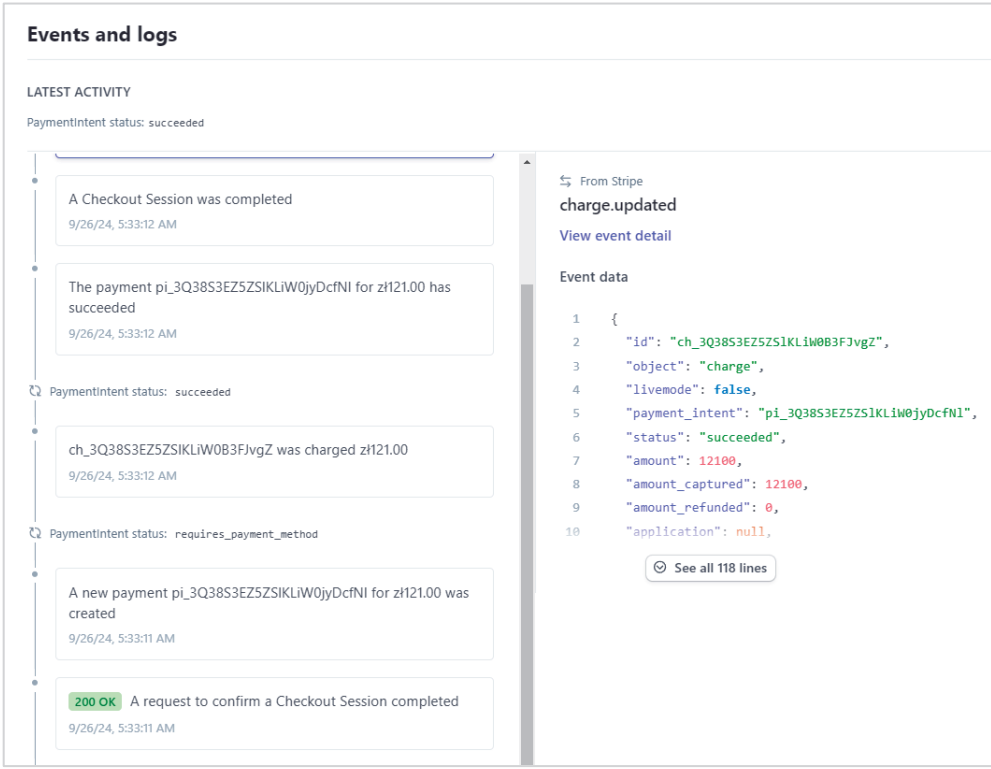


Rysunek 9.16. Szczegóły płatności dla transakcji Stripe



Rysunek 9.17. Metoda płatności wykorzystana w transakcji Stripe

W tym obszarze znajduje się także kolejny obszar o nazwie *Events and logs* (rysunek 9.18).



Rysunek 9.18. Zdarzenia i logi dla transakcji Stripe

W tym obszarze znajdziesz informacje dotyczące wszystkich czynności związanych z transakcją, w tym zapytania do interfejsu API serwisu Stripe. Aby wyświetlić żądanie HTTP do interfejsu Stripe API i odpowiedź w formacie JSON, możesz kliknąć dowolne żądanie na liście. Przeanalizujmy zdarzenia związane z płatnością w porządku chronologicznym — od dołu do góry.

1. Najpierw, przez wysłanie żądania `POST` do punktu końcowego Stripe API */v1/checkout/sessions*, tworzona jest nowa sesja kasowa serwisu Stripe. Metoda Stripe SDK `stripe.checkout.Session.create()` wywołana w widoku `payment_process` tworzy i wysyła żądanie do interfejsu API Stripe oraz obsługuje odpowiedź zwracającą obiekt sesji.
2. Użytkownik zostaje przekierowany na stronę kasy, gdzie przesyła formularz płatności. Strona kasy w serwisie Stripe wysyła żądanie potwierdzenia sesji kasowej.
3. Tworzone jest nowe zlecenie płatności.
4. Naliczana jest opłata związana ze zleceniem płatności.
5. Następuje realizacja zlecenia płatności, która kończy się pomyślnie.
6. Sesja kasy została zakończona.

Gratulacje! Pomyślnie zintegrowałeś Stripe Checkout ze swoim projektem. W następnym punkcie opowiem, jak otrzymywać powiadomienia o płatnościach z serwisu Stripe i jak odwoływać się do płatności Stripe w zamówieniach składanych w sklepie.

Korzystanie z webhooków do otrzymywania powiadomień o płatnościach

Serwis Stripe pozwala na przekazywanie zdarzeń do aplikacji w czasie rzeczywistym za pomocą webhooków. *Webhook*, zwany także wywołaniem zwrotnym (ang. *callback*), można traktować jako interfejs API sterowany zdarzeniami w odróżnieniu od interfejsu API sterowanego żądaniami. Zamiast częstego odpytywania API serwisu Stripe, aby się dowiedzieć, kiedy nowa płatność została zakończona, Stripe może w czasie rzeczywistym wysyłać żądania HTTP do adresu URL aplikacji, aby powiadomić o pomyślnie zakończonych płatnościach. Powiadomienia o tych zdarzeniach są wysyłane asynchronicznie, gdy wystąpi zdarzenie, niezależnie od synchronicznych wywołań API Stripe.

Stwórzmy punkt końcowy webhooka przeznaczony do odbierania zdarzeń Stripe. Webhook będzie się składał z widoku, który odbierze ładunek JSON z informacjami o zdarzeniu w celu jego przetworzenia. Informacje o zdarzeniu wykorzystamy do oznaczenia zamówień jako opłaconych, po pomyślnym zakończeniu sesji kasy.

Tworzenie punktu końcowego webhooka

Aby otrzymywać zdarzenia, możesz dodać do swojego konta Stripe adresy URL punktów końcowych webhooka. Ponieważ używamy webhooków i nie mamy hostowanej witryny internetowej dostępnej za pośrednictwem publicznego adresu URL, użyjemy do nasłuchiwania zdarzeń i przekazywania ich do naszego lokalnego środowiska interfejsu wiersza poleceń (ang. *command line interface* — **CLI**) serwisu Stripe.

W przeglądarce internetowej przejdź pod adres *https://dashboard.stripe.com/test/webhooks*. Wyświetli się ekran pokazany na rysunku 9.19.

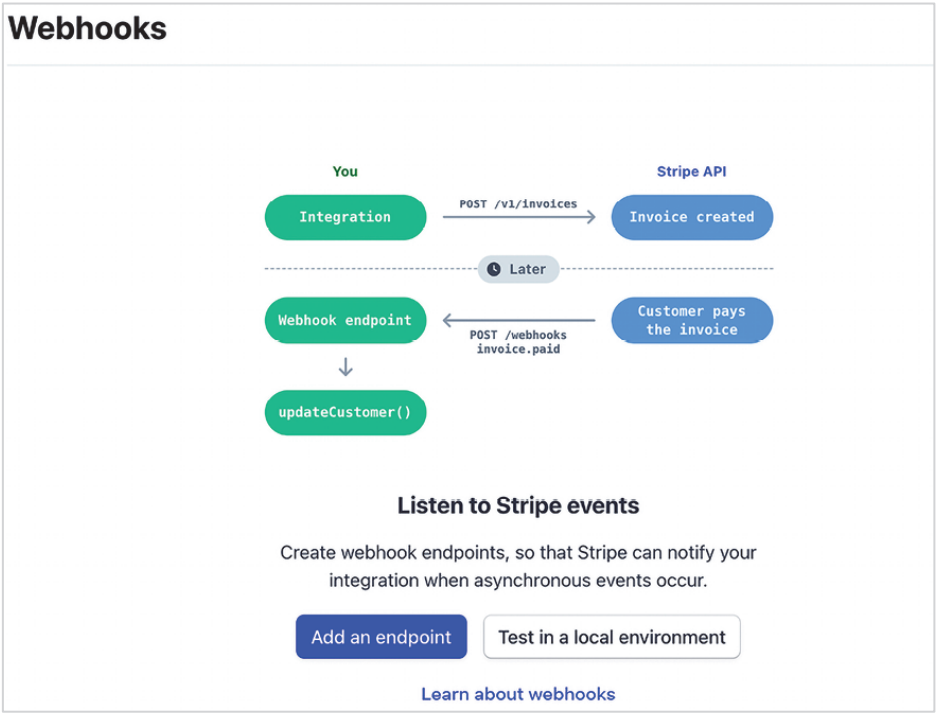
Na tym ekranie możesz zobaczyć schemat sposobu, w jaki Stripe asynchronicznie powiadamia o postępach płatności. Gdy tylko wydarzy się jakieś zdarzenie, będziesz w czasie rzeczywistym otrzymywać powiadomienia Stripe. Stripe wysyła zdarzenia różnego rodzaju, na przykład utworzenie sesji kasowej, utworzenie zlecenia płatności, aktualizacji zlecenia płatności lub zakończenie sesji kasowej. Listę wszystkich typów zdarzeń wysyłanych przez Stripe można znaleźć na stronie *https://stripe.com/docs/api/events/types*.

Kliknij przycisk *Test in local environment*. Wyświetli się ekran pokazany na rysunku 9.20.

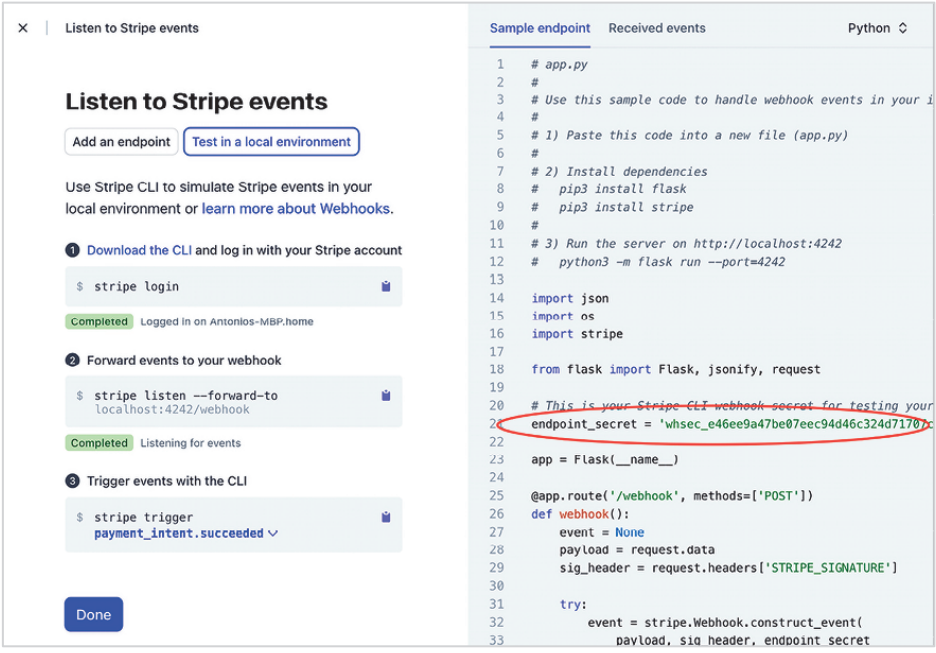
Na tym ekranie wyświetlają się działania, które należy wykonać, aby nasłuchiwać zdarzeń Stripe ze środowiska lokalnego. Zawiera również przykładowy punkt końcowy webhooka dla języka Python. Skopiuj samą wartość `endpoint_secret`.

Przeprowadź edycję pliku *.env* projektu i dodaj następujące wiersze zaznaczone pogrubioną czcionką.

```
STRIPE_PUBLISHABLE_KEY=pk_test_XXXX
STRIPE_SECRET_KEY=sk_test_XXXX
STRIPE_WEBHOOK_SECRET=whsec_XXXX
```



Rysunek 9.19. Domyślny ekran webhooków w serwisie Stripe



Rysunek 9.20. Ekran konfiguracji webhooków w serwisie Stripe

Zastąp wartość `STRIPE_WEBHOOK_SECRET` wartością `endpoint_secret`, którą skopiowałeś z serwisu Stripe.

Przeprowadź edycję pliku *settings.py* projektu `myshop` i umieść w nim poniższe ustawienie.

```
# ...
STRIPE_PUBLISHABLE_KEY = config('STRIPE_PUBLISHABLE_KEY')
STRIPE_SECERT_KEY = config('STRIPE_SECRET_KEY')
STRIPE_API_VERSION = '2024-04-10'
STRIPE_WEBHOOK_SECRET = config('STRIPE_WEBHOOK_SECRET')
```

Aby zbudować punkt końcowy webhooka, utworzymy widok, który odbierze ładunek JSON zawierający szczegółowe dane zdarzenia. Sprawdzimy szczegóły zdarzenia, aby określić, kiedy sesja realizacji transakcji została zakończona, i oznaczymy powiązane zamówienie jako opłacone.

Stripe podpisuje wysyłane do punktów końcowych zdarzenia webhooka, przez dołączenie do każdego zdarzenia nagłówka `Stripe-Signature` z podpisem. Dzięki sprawdzeniu podpisu Stripe, możesz zweryfikować, czy zdarzenia zostały wysłane przez Stripe, a nie przez inny podmiot. Jeśli nie sprawdzisz podpisu, narażasz się na ryzyko otrzymywania przez webhook fałszywych zdarzeń wysyłanych przez napastników. Metoda weryfikacji podpisów jest dostarczana w pakiecie SDK serwisu Stripe. Wykorzystamy ją do stworzenia webhooka, który weryfikuje podpis.

Dodaj do katalogu *payment* aplikacji nowy plik i nadaj mu nazwę *webhooks.py*. Dodaj do nowego pliku `webhooks.py` następujący kod.

```
import stripe
from django.conf import settings
from django.http import HttpResponse
from django.views.decorators.csrf import csrf_exempt
from orders.models import Order


@csrf_exempt
def stripe_webhook(request):
    payload = request.body
    sig_header = request.META['HTTP_STRIPE_SIGNATURE']
    event = None

    try:
        event = stripe.Webhook.construct_event(
            payload, sig_header, settings.STRIPE_WEBHOOK_SECRET
        )
    except ValueError as e:
        # Nieprawidłowy ładunek danych
        return HttpResponse(status=400)
    except stripe.error.SignatureVerificationError as e:
        # Niepoprawna sygnatura
        return HttpResponse(status=400)

    return HttpResponse(status=200)
```

Dekorator `@csrf_exempt` jest używany w celu zablokowania przeprowadzanej przez Django walidacji **CSRF**, wykonywanej domyślnie dla wszystkich żądań `POST`. Do weryfikacji nagłówka sygnatury zdarzenia używamy metody `stripe.Webhook.construct_event()` z biblioteki `stripe`. Jeśli ładunek danych zdarzenia jest nieprawidłowy lub sygnatura jest niepoprawna, zwracana jest odpowiedź HTTP `400 Bad Request`. W przeciwnym razie otrzymujemy odpowiedź HTTP `200 OK`.

Jest to podstawowa funkcjonalność wymagana do weryfikacji sygnatury i skonstruowania zdarzenia na podstawie ładunku JSON. Teraz możemy zaimplementować działania punktu końcowego webhooka.

Dodaj do widoku `stripe_webhook` następujący kod wyróżniony pogrubioną czcionką.

```
@csrf_exempt
def stripe_webhook(request):
    payload = request.body
    sig_header = request.META['HTTP_STRIPE_SIGNATURE']
    event = None
    try:
        event = stripe.Webhook.construct_event(
            payload, sig_header, settings.STRIPE_WEBHOOK_SECRET
        )
    except ValueError as e:
        # Nieprawidłowy ładunek danych
        return HttpResponse(status=400)
     except stripe.error.SignatureVerificationError as e:
        # Niepoprawna sygnatura
         return HttpResponse(status=400)

    if event.type == 'checkout.session.completed':
        session = event.data.object
        if (
             session.mode == 'payment'
             and session.payment_status == 'paid'
        ):
            try:
                order = Order.objects.get(
                     id=session.client_reference_id
                 )
            except Order.DoesNotExist:
                return HttpResponse(status=404)
            # Oznaczenie zamówienia jako zapłaconego
            order.paid = True
            order.save()

    return HttpResponse(status=200)
```

W nowym kodzie sprawdzamy, czy otrzymane zdarzenie to `checkout.session.completed`. To zdarzenie wskazuje, że sesja realizacji transakcji została pomyślnie zakończona. Jeśli otrzymamy to zdarzenie, pobieramy obiekt sesji i sprawdzamy, czy trybem sesji jest `payment`, ponieważ taki jest oczekiwany tryb dla płatności jednorazowych.

Następnie odczytujemy atrybut `client_reference_id`, którego użyliśmy podczas tworzenia sesji kasowej, i korzystamy z mechanizmu ORM frameworka Django do pobrania obiektu `Order` o podanym identyfikatorze. Jeśli zamówienie nie istnieje, zgłaszamy wyjątek HTTP `404`. W innym przypadku za pomocą instrukcji `order.paid = True` oznaczamy zamówienie jako opłacone i zapisujemy zamówienie do bazy.

Przeprowadź edycję pliku *urls.py* aplikacji `payment` i umieść w nim poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
from django.urls import path
from . import views, webhooks

app_name = 'payment'

urlpatterns = [
    path('process/', views.payment_process, name='process'),
    path('completed/', views.payment_completed, name='completed'),
    path('canceled/', views.payment_canceled, name='canceled'),
    path('webhook/', webhooks.stripe_webhook, name='stripe-webhook'),
]
```

Zaimportowaliśmy moduł `webhooks` i dodaliśmy dla webhooka Stripe wzorzec adresu URL.

Testowanie powiadomień webhooka

Aby przetestować webhooki, trzeba zainstalować interfejs CLI serwisu Stripe. Stripe CLI to narzędzie programistyczne, które umożliwia testowanie serwisu Stripe i zarządzanie integracją z tym serwisem bezpośrednio z powłoki. Instrukcje instalacji znajdziesz na stronie *https://stripe.com/docs/stripe-cli#install*.

Jeśli używasz systemu macOS lub Linux, możesz zainstalować Stripe CLI za pomocą menedżera Homebrew następującym poleceniem.

```
brew install stripe/stripe-cli/stripe
```

Jeśli używasz systemu Windows lub systemu macOS bądź Linux bez menedżera Homebrew, pobierz najnowszą wersję Stripe CLI dla systemu macOS, Linux lub Windows, ze strony *https://github.com/stripe/stripe-cli/releases/latest*, a następnie rozpakuj plik. Jeśli używasz systemu Windows, uruchom rozpakowany plik `.exe`.

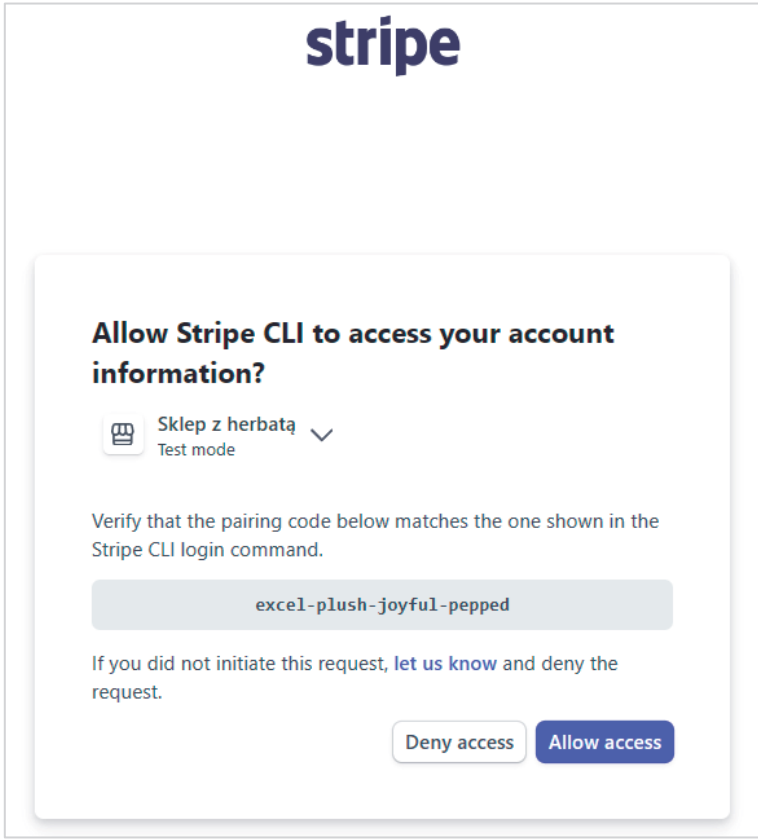
Po zainstalowaniu Stripe CLI uruchom z powłoki polecenie:

```
stripe login
```

Wyświetli się taki wynik:

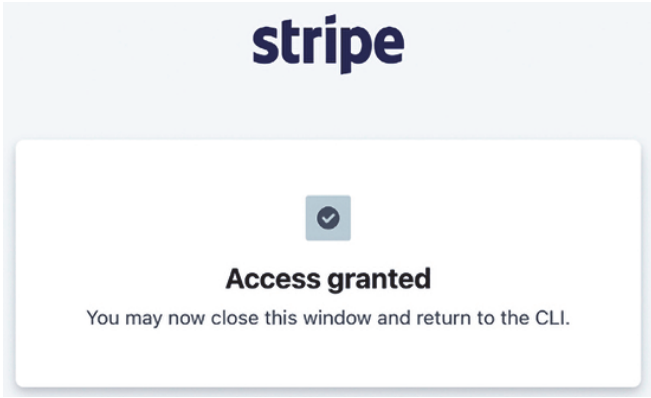
```
Your pairing code is: xxxx-yyyy-zzzz-oooo
This pairing code verifies your authentication with Stripe.
Press Enter to open the browser or visit
https://dashboard.stripe.com/stripecli/confirm_auth?t=....
```

Naciśnij *Enter* lub otwórz wyświetlony adres URL w przeglądarce. Wyświetli się ekran pokazany na rysunku 9.21.



Rysunek 9.21. Ekran parowania Stripe CLI

Sprawdź, czy kod parowania w Stripe CLI jest zgodny z kodem pokazanym na stronie internetowej, po czym kliknij *Allow access*. Wyświetli się komunikat jak na rysunku 9.22.



Rysunek 9.22. Potwierdzenie parowania Stripe CLI

W wierszu polecenia powłoki uruchom polecenie:

```
stripe listen --forward-to localhost:8000/payment/webhook/
```

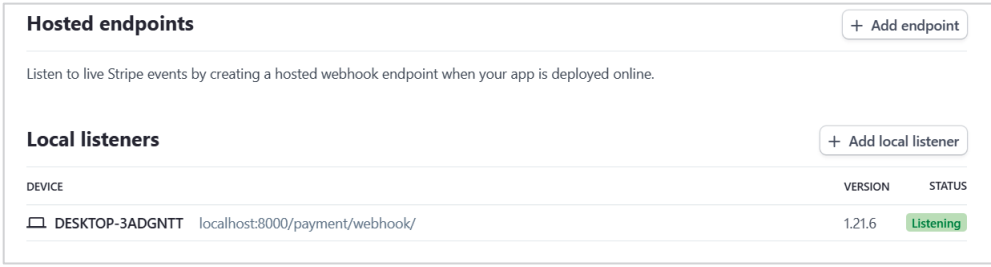
Za pomocą tego polecenia zlecamy serwisowi Stripe nasłuchiwanie zdarzeń i przekazywanie ich do lokalnego hosta. Używamy portu 8000, na którym działa serwer programistyczny Django, oraz ścieżki */payment/webhook/*, która odpowiada wzorcowi adresu URL stworzonego webhooka.

Wyświetli się następujący wynik.

```
Getting ready... > Ready! You are using Stripe API Version [2024-04-10].
Your webhook signing secret is xxxxxxxxxxxxxxxxxx (^C to quit)
```

Tutaj wyświetla się tajny klucz webhooka. Sprawdź, czy hasło wykorzystywane do podpisania webhooka, jest zgodne z ustawieniem `STRIPE_WEBHOOK_SECRET` w pliku *settings.py* Twojego projektu.

W przeglądarce internetowej przejdź pod adres *https://dashboard.stripe.com/test/webhooks*. Wyświetli się ekran pokazany na rysunku 9.23.



Rysunek 9.23. Strona Webhooks w serwisie Stripe

W obszarze *Local listeners* zobaczysz lokalnego słuchacza, którego utworzyliśmy przed chwilą.

> **Uwaga**
>
> W środowisku produkcyjnym narzędzie Stripe CLI nie jest potrzebne. Zamiast tego, przy użyciu adresu URL hostowanej aplikacji, należy dodać punkt końcowy webhooka.

Otwórz w przeglądarce stronę `http://127.0.0.1:8000/`, dodaj do koszyka na zakupy kilka produktów i wypełnij formularz finalizacji transakcji.

Sprawdź powłokę, w której uruchomiłeś narzędzie Stripe CLI.

```
2024-01-03 18:06:13   --> payment_intent.created [evt_...]
2024-01-03 18:06:13  <--  [200] POST http://127.0.0.1:8000/payment/webhook/
[evt_...]
2024-01-03 18:06:13   --> payment_intent.succeeded [evt_...]
2024-01-03 18:06:13  <--  [200] POST http://127.0.0.1:8000/payment/webhook/
[evt_...]
2024-01-03 18:06:13   --> charge.succeeded [evt_...]
```

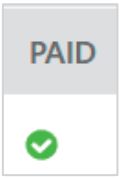
```
2024-01-03 18:06:13  <--  [200] POST http://127.0.0.1:8000/payment/webhook/
[evt_...]
2024-01-03 18:06:14   --> checkout.session.completed [evt_...]
2024-01-03 18:06:14  <--  [200] POST http://127.0.0.1:8000/payment/webhook/
[evt_...]
```

Wyświetlają się tu różne zdarzenia, które zostały wysłane przez Stripe do lokalnego punktu końcowego webhooka. Zdarzenia mogą być w innej kolejności niż powyżej. Stripe nie gwarantuje dostarczenia zdarzeń w kolejności, w jakiej zostały wygenerowane. Oto te zdarzenia:

- `payment_intent.created`. Utworzono zlecenie płatności.
- `payment_intent.succeeded`. Zlecenie płatności powiodło się.
- `charge.succeeded`. Obciążenie powiązane ze zleceniem płatności powiodło się.
- `checkout.session.completed`. Sesja kasowa została zakończona. Zdarzenie, które wykorzystujemy do oznaczenia zamówienia jako opłacone.

Na wszystkie żądania wysłane przez webhook serwisu Stripe wywołanie `stripe_webhook` zwraca odpowiedź HTTP `200 OK`. Jednak aby oznaczyć zamówienie związane z płatnością jako opłacone, przetwarzamy tylko zdarzenie `checkout.session.completed`.

Teraz w przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/admin/orders/order/`. Zamówienie powinno być teraz oznaczone jako opłacone (rysunek 9.24).



Rysunek 9.24. Zamówienie na liście zamówień w witrynie administracyjnej oznaczone jako opłacone

Teraz, dzięki powiadomieniom o płatnościach Stripe, zamówienia są automatycznie oznaczane jako opłacone. W dalszej części rozdziału dowiesz się, jak odwoływać się do płatności Stripe w zamówieniach składanych w sklepie.

Odwoływanie się do płatności Stripe w zamówieniach

Z każdą płatnością Stripe jest powiązany unikatowy identyfikator. Możemy skorzystać z identyfikatora płatności, aby powiązać każde zamówienie z odpowiadającą mu płatnością Stripe. Abyśmy mogli odwoływać się do powiązanej płatności za pomocą jej identyfikatora, dodamy do modelu `Order` w aplikacji `orders` nowe pole. Dzięki temu będziemy mogli powiązać każde zamówienie z odpowiednią transakcją Stripe.

Przeprowadź edycję pliku *models.py* aplikacji `orders` i dodaj do modelu `Order` nowe pole. Nowe pole zostało wyróżnione pogrubioną czcionką.

```
class Order(models.Model):
    #…
    stripe_id = models.CharField(max_length=250, blank=True)
```

Teraz należy zsynchronizować to pole z bazą danych. Aby wygenerować migracje bazy danych dla projektu, użyj polecenia:

```
python manage.py makemigrations
```

Wyświetli się taki wynik:

```
Migrations for 'orders':
   orders/migrations/0002_order_stripe_id.py
    - Add field stripe_id to order
```

Zastosuj migrację do bazy danych za pomocą polecenia:

```
python manage.py migrate
```

Wyświetli się wynik zakończony wierszem:

```
Applying orders.0002_order_stripe_id... OK
```

Zmiany modelu są teraz zsynchronizowane z bazą danych. Teraz będziesz mógł przechowywać identyfikator płatności Stripe dla każdego zamówienia.

Przeprowadź edycję pliku *webhooks.py* w aplikacji `payment` i umieść w nim poniższe wiersze oznaczone pogrubioną czcionką.

```
# ...
@csrf_exempt
def stripe_webhook(request):
    # ...

        if event.type == 'checkout.session.completed':
        session = event.data.object
        if (
            session.mode == 'payment'
            and session.payment_status == 'paid'
        ):
            try:
                order = Order.objects.get(
                    id=session.client_reference_id
                )
            except Order.DoesNotExist:
                return HttpResponse(status=404)
            # Oznaczenie zamówienia jako opłaconego
            order.paid = True
            # Zapisanie identyfikatora płatności Stripe
            order.stripe_id = session.payment_intent
            order.save()
    return HttpResponse(status=200)
```

Dzięki tej zmianie po otrzymaniu powiadomienia webhooka o zakończonej sesji realizacji zakupu w polu `stripe_id` obiektu `order` zostanie zapisany identyfikator zlecenia płatności.

Otwórz w przeglądarce stronę `http://127.0.0.1:8000/`, dodaj kilka `produktów` do koszyka na zakupy i wypełnij formularz finalizacji transakcji. Następnie otwórz w przeglądarce stronę `http://127.0.0.1:8000/admin/orders/order/`, po czym aby zmodyfikować najnowsze zamówienie, kliknij jego identyfikator. Pole `stripe_id` powinno zawierać identyfikator zlecenia płatności, tak jak pokazałem na rysunku 9.25.

Rysunek 9.25. Pole Stripe ID z identyfikatorem zlecenia płatności

Doskonale! W zamówieniach możemy bezproblemowo odwoływać się do płatności Stripe. Teraz możemy dodać identyfikatory płatności Stripe do listy zamówień na stronie administracyjnej. Możemy również dołączyć link do każdego identyfikatora płatności, aby wyświetlić szczegóły płatności w panelu głównym serwisu Stripe.

Przeprowadź edycję pliku *models.py* aplikacji `orders` i umieść w nim poniższe wiersze kodu oznaczone pogrubioną czcionką.

```
from django.conf import settings
from django.db import models


class Order(models.Model):
    # …
    class Meta:
        # ...

    def __str__(self):
        return f'Zamówienie {self.id}'

    def get_total_cost(self):
        return sum(item.get_cost() for item in self.items.all())

    def get_stripe_url(self):
        if not self.stripe_id:
            # Z zamówieniem nie jest powiązana żadna płatność
            return ''
        if '_test_' in settings.STRIPE_SECRET_KEY:
            # Ścieżka dla płatności testowych Stripe
            path = '/test/'
        else:
            # Ścieżka dla rzeczywistych płatności Stripe
            path = '/'
        return f'https://dashboard.stripe.com{path}payments/{self.stripe_id}'
```

Dodaliśmy do modelu `Order` nową metodę `get_stripe_url()`. Ta metoda zwraca adres URL głównego panelu serwisu Stripe dla płatności powiązanej z zamówieniem. Jeśli w polu `stripe_id` obiektu `Order` nie jest przechowywany żaden identyfikator płatności, metoda zwraca pusty ciąg znaków. W przeciwnym razie zwracany jest adres URL płatności z panelu głównego serwisu Stripe. Aby odróżnić środowisko produkcyjne od środowiska

testowego sprawdzamy, czy w ustawieniu STRIPE_SECRET_KEY występuje ciąg _test_. Płatności w środowisku produkcyjnym są zgodne ze wzorcem *https://dashboard.stripe.com/payments/{id}*, natomiast płatności testowe z wzorcem *https://dashboard.stripe.com/payments/test/{id}*.

Na koniec w witrynie administracyjnej na stronie wyświetlającej listę zamówień dodajemy do każdego obiektu Order link do płatności.

Przeprowadź edycję pliku *admin.py* aplikacji orders i umieść w nim poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
# ...
from django.utils.safestring import mark_safe

def order_payment(obj):
    url = obj.get_stripe_url()
    if obj.stripe_id:
        html = f'<a href="{url}" target="_blank">{obj.stripe_id}</a>'
        return mark_safe(html)
    return ''
order_payment.short_description = 'Płatność Stripe'

@admin.register(Order)
class OrderAdmin(admin.ModelAdmin):
    list_display = [
        'id',
        'first_name',
        'last_name',
        'email',
        'address',
        'postal_code',
        'city',
        'paid',
        order_payment,
        'created',
        'updated'
    ]
    # ...
```

Funkcja order_stripe_payment() pobiera jako argument obiekt Order i zwraca link HTML z adresem URL płatności w serwisie Stripe. Domyślnie Django neutralizuje dane wyjściowe HTML. Aby uniknąć wspomnianej neutralizacji, konieczne jest użycie funkcji mark_safe.

Uwaga

Aby uniknąć ataków typu **XSS** (ang. *cross-site scripting*), nie korzystaj z funkcji mark_safe w odniesieniu do danych wprowadzanych przez użytkownika. Ataki XSS umożliwiają napastnikom wstrzykiwanie do treści webowej przeglądanej przez użytkowników skryptów działających po stronie klienta.

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/admin/orders/order/`. Zobaczysz nową kolumnę o nazwie *PŁATNOŚĆ STRIPE*. Wyświetli się powiązany identyfikator płatności Stripe dla ostatniego zamówienia. Jeśli klikniesz na identyfikator płatności, zostaniesz przeniesiony do adresu URL płatności w serwisie Stripe, gdzie znajdziesz dodatkowe szczegóły płatności (rysunek 9.26).

Rysunek 9.26. Identyfikator płatności Stripe dla obiektu zamówienia w witrynie administracyjnej

Teraz po otrzymaniu powiadomień o płatnościach automatycznie zapisujemy identyfikatory płatności Stripe w rekordach zamówień. Na tym zakończylismy integrację bramki płatniczej Stripe Checkout z naszym projektem.

Wdrożenie do produkcji

Po przetestowaniu integracji możesz utworzyć produkcyjne konto Stripe. Gdy będziesz gotowy do wdrożenia witryny do produkcji, pamiętaj o zastąpieniu w pliku *settings.py* testowych poświadczeń Stripe poświadczeniami produkcyjnymi. Zamiast korzystania ze Stripe CLI będziesz także musiał dodać dla swojej hostowanej witryny pod adresem *https://dashboard.stripe.com/webhooks* punkt końcowy webhooka. Sposób konfigurowania ustawień dla wielu środowisk opisałem w rozdziale 17., „Wdrożenie".

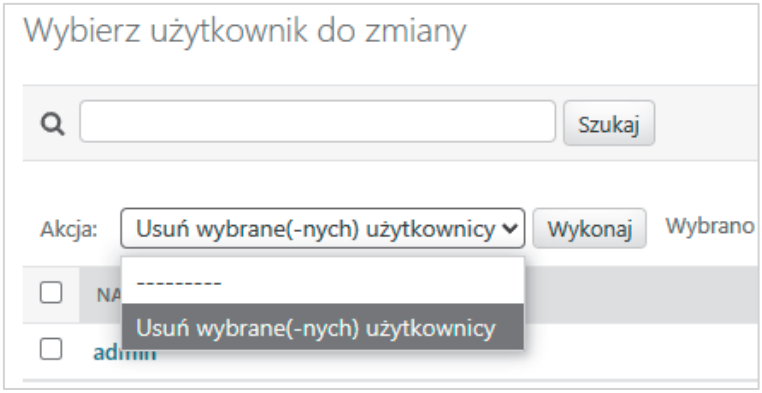
Eksport zamówień do plików CSV

Czasami może zaistnieć potrzeba, aby informacje znajdujące się w modelu zostały wyeksportowane do pliku, który następnie będzie można zaimportować w innym systemie. Jednym z formatów najczęściej używanych do eksportu i importu danych jest **CSV** (ang. *comma-separated values*), czyli wartości rozdzielone przecinkami. Plik CSV to zwykły plik tekstowy zawierający dowolną liczbę rekordów. Zwykle wiersz zawiera informacje o jednym rekordzie, a poszczególne wartości są rozdzielone ustalonym znakiem, najczęściej przecinkiem. Przeprowadzimy teraz modyfikację witryny administracyjnej, aby zyskać możliwość eksportu zamówień do plików CSV.

Dodanie własnych działań do witryny administracyjnej

Django oferuje wiele opcji pozwalających na dostosowanie witryny administracyjnej do własnych potrzeb. W tym miejscu przeprowadzimy modyfikację widoku listy obiektów i zamieścimy na niej własne działanie administracyjne. Niestandardowe działania administracyjne możesz zaimplementować w celu umożliwienia użytkownikom personelu stosowanie operacji w widoku listy zmian do wielu elementów jednocześnie.

Działanie administracyjne przebiega w następujący sposób: za pomocą pól wyboru użytkownik wybiera obiekty na stronie zawierającej listę obiektów, wskazuje operację, która ma zostać wykonana dla zaznaczonych elementów, i uruchamia tę operację. Przykład operacji umieszczonej w witrynie administracyjnej Django pokazałem na zrzucie ekranu na rysunku 9.27.



Rysunek 9.27. Rozwijane menu operacji administracyjnych frameworka Django

Przygotowanie własnej operacji odbywa się przez utworzenie zwykłej funkcji, która otrzymuje następujące parametry.

- Aktualnie wyświetlany model `ModelAdmin`.
- Bieżący obiekt żądania jako egzemplarz `HttpRequest`.
- Kolekcja `QuerySet` dla obiektów wybranych przez użytkownika.

Funkcja ta zostanie wykonana po wywołaniu operacji z poziomu witryny administracyjnej.

Utworzymy teraz własną akcję administracyjną polegającą na pobraniu listy zamówień w postaci pliku CSV.

Przeprowadź edycję pliku *admin.py* aplikacji `orders` i umieść w nim poniższy fragment kodu przed klasą `OrderAdmin`.

```
import csv
import datetime
from django.http import HttpResponse


def export_to_csv(modeladmin, request, queryset):
    opts = modeladmin.model._meta
    content_disposition = (
        f'attachment; filename={opts.verbose_name}.csv'
    )
    response = HttpResponse(content_type='text/csv')
    response['Content-Disposition'] = content_disposition
    writer = csv.writer(response)
    fields = [
        field
```

```
            for field in opts.get_fields()
            if not field.many_to_many and not field.one_to_many
        ]
        # Zapis pierwszego rekordu wraz z informacjami nagłówka.
        writer.writerow([field.verbose_name for field in fields])
        # Zapis rekordów danych.
        for obj in queryset:
            data_row = []
            for field in fields:
                value = getattr(obj, field.name)
                if isinstance(value, datetime.datetime):
                    value = value.strftime('%d/%m/%Y')
                data_row.append(value)
            writer.writerow(data_row)
        return response
    export_to_csv.short_description = 'Eksport do CSV'
```

W przedstawionym kodzie wykonujemy następujące zadania.

1. Utworzenie egzemplarza `HttpResponse` zawierającego treść oznaczoną niestandardowym typem `text/csv`, aby nakazać przeglądarce internetowej potraktowanie zawartości jako pliku CSV. Ponadto dodany został nagłówek `Content-Disposition` wskazujący, że odpowiedź HTTP zawiera dołączony plik.
2. Utworzenie obiektu CSV `writer` odpowiedzialnego za zapis danych w obiekcie `response`.
3. Pobranie dynamicznie kolumny modelu za pomocą metody `get_fields()` opcji modelu `_meta`. Wykluczone zostały relacje typu „wiele do wielu" i „jeden do wielu".
4. Zapisanie rekordu nagłówka zawierającego nazwy kolumn.
5. Przeprowadzenie iteracji przez daną kolekcję `QuerySet` i zapisanie rekordu dla każdego obiektu zwróconego przez `QuerySet`. Zajęto się formatowaniem obiektów `datetime`, ponieważ dane wyjściowe w formacie CSV muszą mieć postać ciągu tekstowego.
6. Dostosowanie do własnych potrzeb wyświetlanej w szablonie nazwy operacji za pomocą atrybutu funkcji `short_description`.

Utworzyliśmy w ten sposób generyczną operację administracyjną, którą można dodać do klasy `ModelAdmin`.

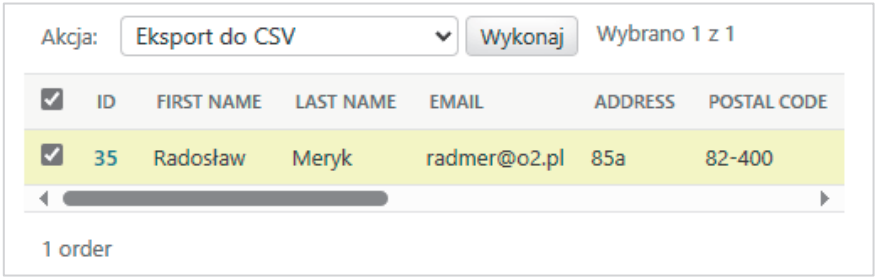
Na koniec dodaj do klasy `OrderAdmin` nową operację administracyjną `export_to_csv`, tak jak pokazałem poniżej. Nowy kod został wyróżniony pogrubioną czcionką.

```
@admin.register(Order)
class OrderAdmin(admin.ModelAdmin):
    # …
    actions = [export_to_csv]
```

Uruchom serwer programistyczny za pomocą poniższego polecenia.

```
python manage.py runserver
```

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/admin/orders/order/`. Powinieneś zobaczyć stronę pokazaną na rysunku 9.28.



Rysunek 9.28. Korzystanie z niestandardowej operacji administracyjnej Eksport do CSV

Zaznacz kilka zamówień, z rozwijanego menu *Akcja* wybierz *Eksport do CSV*, a następnie kliknij przycisk *Wykonaj*. Przeglądarka internetowa powinna pobrać wygenerowany plik CSV o nazwie *order.csv*. Otwórz ten plik w edytorze tekstu. Powinieneś zobaczyć zawartość w pokazanym poniżej formacie wraz z wierszem nagłówka oraz kolejnymi wierszami, po jednym dla każdego zaznaczonego obiektu `Order`.

```
ID,first name,last name,email,address,postal
code,city,created,updated,paid,stripe id
4,Antonio,Melé,email@domain.com,20 W 34th St,10001,New
York,03/01/2024,03/01/2024,True,pi_3ORvzkGNwIe5nm8S1wVd7l7i
...
```

Jak widać, utworzenie operacji administracyjnej jest całkiem łatwym zadaniem. Więcej informacji o generowaniu plików CSV z Django możesz znaleźć na stronie *https://docs.djangoproject.com/en/5.0/howto/outputting-csv/*.

Jeśli chcesz dodać do swojej strony administracyjnej bardziej zaawansowane funkcje importu (eksportu), możesz użyć zewnętrznej aplikacji `django-import-export`. Dokumentację tej aplikacji znajdziesz na stronie *https://django-import-export.readthedocs.io/en/latest/*.

Zaimplementowany powyżej przykład działa dobrze w przypadku małych i średnich zbiorów danych. Ponieważ eksport odbywa się w ramach żądania HTTP, bardzo duże zestawy danych mogą powodować przekroczenie limitu czasu serwera, serwer bowiem może zamknąć połączenie przed zakończeniem procesu eksportowania. Aby poradzić sobie z tym problemem, można generować asynchroniczne operacje eksportu za pomocą Celery. Do tego celu służy aplikacja `django-import-export-celery`. Ten projekt jest dostępny pod adresem *https://github.com/auto-mat/django-import-export-celery*.

W kolejnym punkcie będziemy kontynuować personalizację witryny administracyjnej. Tym razem stworzymy niestandardowy widok administracyjny.

Rozbudowa witryny administracyjnej za pomocą własnych widoków

Czasami będzie trzeba dostosować witrynę administracyjną do własnych potrzeb w zakresie znacznie wykraczającym poza możliwości oferowane przez klasę konfiguracyjną `ModelAction`, własne akcje administracyjne i nadpisywanie szablonów administracyjnych.

Możesz zaimplementować dodatkowe funkcjonalności, które nie są dostępne w istniejących widokach lub szablonach administracyjnych. W takim przypadku należy utworzyć niestandardowy widok administracyjny. Za pomocą niestandardowych widoków możesz zbudować dowolną funkcjonalność; musisz jedynie upewnić się, że tylko pracownicy personelu będą mogli uzyskać dostęp do tego rodzaju widoku i że podczas rozszerzania szablonu administracyjnego zachowujesz wygląd i działanie administracyjne.

Przystępujemy zatem do utworzenia niestandardowego widoku przeznaczonego do wyświetlania informacji o zamówieniu. Przeprowadź edycję pliku *views.py* aplikacji `orders` i umieść w nim poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
from django.contrib.admin.views.decorators import staff_member_required
from django.shortcuts import get_object_or_404, redirect, render
from cart.cart import Cart
from .forms import OrderCreateForm
from .models import Order, OrderItem
from .tasks import order_created


def order_create(request):
    # …

@staff_member_required
def admin_order_detail(request, order_id):
    order = get_object_or_404(Order, id=order_id)
    return render(
        request, 'admin/orders/order/detail.html', {'order': order}
    )
```

Dekorator `staff_member_required` sprawdza, czy kolumny `is_active` i `is_staff` użytkownika wykonującego żądanie do danej strony mają wartość `True`. W tym widoku pobieramy obiekt `Order` o podanym identyfikatorze i renderujemy szablon przeznaczony do wyświetlenia zamówienia.

Teraz przeprowadź edycję pliku *urls.py* aplikacji `orders` i umieść w nim poniższy wzorzec adresu URL oznaczony pogrubioną czcionką.

```
urlpatterns = [
    path('create/', views.order_create, name='order_create'),
    path(
        'admin/order/<int:order_id>/',
        views.admin_order_detail,
        name='admin_order_detail'
    ),
]
```

Utwórz poniższą strukturę plików i katalogów w katalogu *templates* aplikacji `orders`.

```
admin/
    orders/
        order/
            detail.html
```

Przeprowadź edycję szablonu *detail.html* i umieść w nim fragment kodu:

```
{% extends "admin/base_site.html" %}

{% block title %}
  Zamówienie {{ order.id }} {{ block.super }}
{% endblock %}

{% block breadcrumbs %}
  <div class="breadcrumbs">
    <a href="{% url "admin:index" %}">Strona główna</a> &rsaquo;
    <a href="{% url "admin:orders_order_changelist" %}">Zamówienia</a>
    &rsaquo;
    <a href="{% url "admin:orders_order_change" order.id %}">Zamówienie {{ order.id
}}</a>
    &rsaquo; Szczegóły
  </div>
{% endblock %}

{% block content %}
<div class="module">
  <h1>Zamówienie {{ order.id }}</h1>
  <ul class="object-tools">
    <li>
      <a href="#" onclick="window.print();">
        Drukuj zamówienie
      </a>
    </li>
  </ul>
  <table>
    <tr>
      <th>Utworzone</th>
      <td>{{ order.created }}</td>
    </tr>
    <tr>
      <th>Klient</th>
      <td>{{ order.first_name }} {{ order.last_name }}</td>
    </tr>
    <tr>
      <th>E-mail</th>
      <td><a href="mailto:{{ order.email }}">{{ order.email }}</a></td>
    </tr>
    <tr>
      <th>Adres</th>
    <td>
      {{ order.address }},
      {{ order.postal_code }} {{ order.city }}
    </td>
    </tr>
    <tr>
      <th>Razem </th>
      <td>{{ order.get_total_cost }} PLN</td>
    </tr>
    <tr>
      <th>Stan</th>
```

```
        <td>{% if order.paid %}Opłacone{% else %}Oczekiwanie na płatność
        ↪{% endif %}</td>
      </tr>
      <tr>
        <th>Płatność Stripe</th>
        <td>
          {% if order.stripe_id %}
            <a href="{{ order.get_stripe_url }}" target="_blank">
              {{ order.stripe_id }}
            </a>
          {% endif %}
        </td>
      </tr>
    </table>
  </div>
  <div class="module">
    <h2>Kupione produkty</h2>
    <table style="width:100%">
      <thead>
        <tr>
          <th>Produkt</th>
          <th>Cena</th>
          <th>Ilość</th>
          <th>Razem</th>
        </tr>
      </thead>
      <tbody>
        {% for item in order.items.all %}
          <tr class="row{% cycle "1" "2" %}">
            <td>{{ item.product.name }}</td>
            <td class="num">{{ item.price }} PLN</td>
            <td class="num">{{ item.quantity }}</td>
            <td class="num">{{ item.get_cost }} PLN</td>
          </tr>
        {% endfor %}
        <tr class="total">
          <td colspan="3">Razem</td>
          <td class="num">{{ order.get_total_cost }} PLN</td>
        </tr>
      </tbody>
    </table>
  </div>
  {% endblock %}
```

Zadbaj o to, aby żaden znacznik szablonu nie obejmował więcej niż jednego wiersza.

Szablon ten jest przeznaczony do wyświetlenia szczegółów zamówienia w witrynie administracyjnej. Omawiany szablon rozszerza szablon *admin/base_site.html* dostarczany w witrynie administracyjnej Django, zawiera podstawową strukturę kodu HTML i stylów CSS dla użytkowników o uprawnieniach administratora. Wykorzystaliśmy bloki zdefiniowane w szablonie nadrzędnym i wprowadziliśmy do nich własną treść. Wyświetlamy informacje na temat zamówienia oraz zakupionych towarów.

Jeżeli chcesz rozszerzyć szablon administracyjny, musisz poznać jego strukturę oraz zidentyfikować istniejące bloki. Wszystkie szablony administracyjne znajdziesz na stronie *https://github.com/django/django/tree/5.0/django/contrib/admin/templates/admin*.

Szablon administracyjny możesz również nadpisać, jeśli trzeba. W tym celu skopiuj go do katalogu *templates*, zachowując taką samą względną ścieżkę dostępu i nazwę pliku. Dzięki temu witryna administracyjna Django użyje przygotowanego przez Ciebie szablonu zamiast domyślnego.

Na koniec dodamy link do każdego obiektu `Order` na stronie wyświetlającej listę zamówień w witrynie administracyjnej. Przeprowadź edycję pliku *admin.py* aplikacji `orders` i umieść w nim poniższy fragment kodu przed kodem klasy `OrderAdmin`.

```
from django.urls import reverse


def order_detail(obj):
    url = reverse('orders:admin_order_detail', args=[obj.id])
    return mark_safe(f'<a href="{url}">Wyświetl</a>')
```

Funkcja ta jako argument pobiera obiekt `Order` i zwraca link HTML dla adresu URL widoku `admin_order_detail`. Domyślnie Django neutralizuje dane wyjściowe HTML. Aby uniknąć wspomnianej neutralizacji, konieczne jest użycie funkcji `mark_safe`.

Teraz przeprowadź edycję klasy `OrderAdmin`, aby wyświetlić wspomniany wcześniej link. Nowy kod został wyróżniony pogrubioną czcionką.

```
class OrderAdmin(admin.ModelAdmin):
    list_display = [
        'id',
        'first_name',
        'last_name',
        'email',
        'address',
        'postal_code',
        'city',
        'paid',
        order_payment,
        'created',
        'updated',
        order_detail,
    ]
    # …
```

Uruchom serwer programistyczny za pomocą poniższego polecenia.

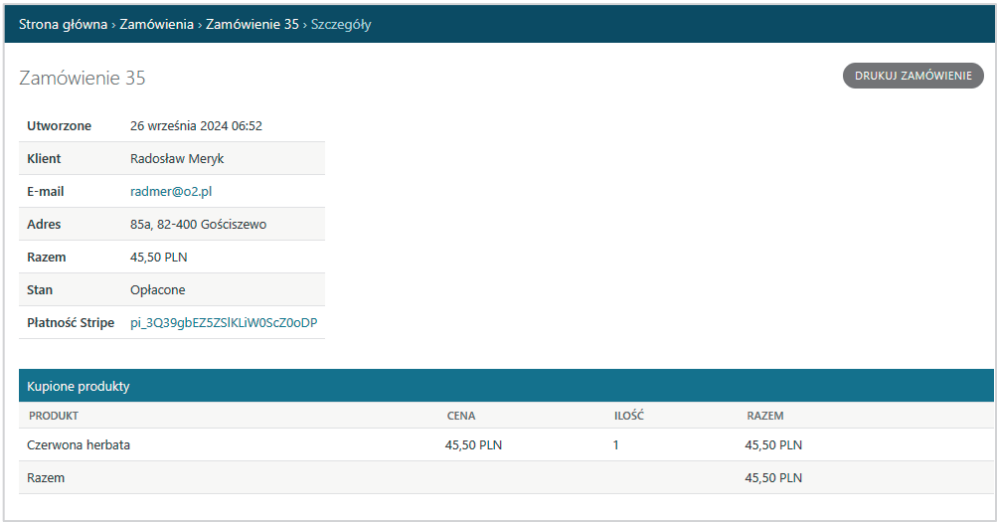
```
python manage.py runserver
```

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/admin/orders/order/`. Każdy wiersz zawiera teraz link *Wyświetl*, co pokazałem na rysunku 9.29.

PAID	PŁATNOŚĆ STRIPE	CREATED	UPDATED	ORDER DETAIL
✓	pi_3Q39gbEZ5ZSlKLiW0ScZ0oDP	26 września 2024 06:52	26 września 2024 06:52	Wyświetl

Rysunek 9.29. Link Wyświetl dołączony do każdego wiersza zamówienia

Kliknij link *Wyświetl* obok dowolnego zamówienia, a zobaczysz stronę zawierającą szczegóły danego zamówienia. Powinieneś zobaczyć stronę pokazaną na rysunku 9.30.



Rysunek 9.30. Niestandardowa strona szczegółów zamówienia w witrynie administracyjnej

Po utworzeniu strony szczegółów produktu dowiesz się, jak dynamicznie generować faktury zamówień w formacie PDF.

Dynamiczne generowanie faktur w formacie PDF

Skoro mamy przygotowany pełny system składania zamówień i dokonywania płatności, możemy pokusić się o wygenerowanie faktury w formacie PDF dla każdego zamówienia. Dla Pythona opracowano wiele bibliotek pozwalających na generowanie dokumentów PDF. Jedną z bardziej popularnych jest `Reportlab`. Więcej informacji na temat generowania plików PDF za pomocą biblioteki `Reportlab` znajdziesz na stronie *https://docs.djangoproject.com/en/5.0/howto/outputting-pdf/*.

W większości przypadków dla treści umieszczanej w plikach PDF trzeba dodać własne style i formatowanie. Znacznie wygodniejszym rozwiązaniem może być wygenerowanie szablonu HTML i jego konwersja na dokument PDF, co pozwala na trzymanie Pythona z daleka od warstwy prezentacyjnej. Wykorzystamy więc takie podejście i użyjemy modułu do generowania za pomocą Django plików w formacie PDF. W omawianym projekcie zdecydowałem się na `WeasyPrint`, czyli bibliotekę Pythona pozwalającą na generowanie dokumentów PDF na podstawie szablonów HTML.

Instalacja WeasyPrint

Przede wszystkim zainstaluj wszystkie zależności WeasyPrint dla używanego systemu operacyjnego. Można je pobrać na stronie *https://doc.courtbouillon.org/weasyprint/stable/first_steps.html*. Następnie zainstaluj WeasyPrint za pomocą menedżera `pip`, wydając poniższe polecenie.

```
python -m pip install WeasyPrint==61.2
```

Utworzenie szablonu PDF

Potrzebujemy dokumentu HTML używanego w charakterze danych wejściowych dla WeasyPrint. Przystępujemy więc do utworzenia szablonu HTML, następnie wygenerujemy go w Django i przekażemy do WeasyPrint w celu wygenerowania dokumentu PDF.

Utwórz nowy plik szablonu w katalogu *templates/orders/order* aplikacji `orders` i nadaj mu nazwę *pdf.html*. Następnie w pliku umieść poniższy fragment kodu.

```
<html>
<body>
  <h1>Mój sklep</h1>
  <p>
    Faktura nr. {{ order.id }}<br>
    <span class="secondary">
      {{ order.created|date:"M d, Y" }}
    </span>
  </p>
  <h3>Nabywca</h3>
  <p>
    {{ order.first_name }} {{ order.last_name }}<br>
    {{ order.email }}<br>
    {{ order.address }}<br>
    {{ order.postal_code }}, {{ order.city }}
  </p>
  <h3>Kupione produkty</h3>
  <table>
    <thead>
      <tr>
        <th>Produkt</th>
        <th>Cena</th>
        <th>Ilość</th>
        <th>Wartość</th>
      </tr>
    </thead>
    <tbody>
      {% for item in order.items.all %}
        <tr class="row{% cycle "1" "2" %}">
          <td>{{ item.product.name }}</td>
          <td class="num">{{ item.price }} PLN</td>
          <td class="num">{{ item.quantity }}</td>
          <td class="num">{{ item.get_cost }} PLN</td>
        </tr>
```

```
        {% endfor %}
        <tr class="total">
          <td colspan="3">Razem</td>
          <td class="num">{{ order.get_total_cost }} PLN</td>
        </tr>
      </tbody>
    </table>

    <span class="{% if order.paid %}paid{% else %}pending{% endif %}">
       {% if order.paid %}Zapłacono{% else %}Oczekiwanie na płatność{% endif %}
    </span>
  </body>
  </html>
```

Jest to szablon dla faktury w formacie PDF. W tym szablonie wszystkie szczegóły zamówienia, łącznie z kupionymi przez klienta produktami, wyświetlamy w tabeli HTML (element `<table>`). Ponadto dołączamy komunikat informujący o opłaceniu zamówienia lub oczekiwaniu na wpłatę.

Renderowanie plików w formacie PDF

Kolejnym krokiem jest utworzenie widoku przeznaczonego do generowania rachunku PDF dla istniejącego zamówienia przy użyciu witryny administracyjnej. Przeprowadź edycję pliku *views.py* w katalogu aplikacji `orders` i umieść w nim poniższy fragment kodu.

```
import weasyprint
from django.contrib.staticfiles import finders
from django.http import HttpResponse
from django.template.loader import render_to_string


@staff_member_required
def admin_order_pdf(request, order_id):
    order = get_object_or_404(Order, id=order_id)
    html = render_to_string('orders/order/pdf.html', {'order': order})
    response = HttpResponse(content_type='application/pdf')
    response['Content-Disposition'] = f'filename=order_{order.id}.pdf'
    weasyprint.HTML(string=html).write_pdf(
        response,
        stylesheets=[weasyprint.CSS(finders.find('css/pdf.css'))]
    )
    return response
```

Jest to widok przeznaczony do wygenerowania rachunku PDF dla zamówienia. Użyliśmy dekoratora `staff_member_required`, aby mieć pewność, że tylko pracownik będzie mógł wyświetlić ten widok.

Kod pobiera obiekt `Order` o podanym identyfikatorze, a następnie za pomocą dostarczanej przez Django funkcji `render_to_string()` generuje szablon *orders/order/pdf.html*. Wyrenderowany kod HTML jest zapisywany w zmiennej `html`.

Kolejnym krokiem jest wygenerowanie obiektu HttpResponse z podaniem typu treści *application/pdf* i nagłówkiem `Content-Disposition` określającym nazwę pliku. WeasyPrint

wykorzystujemy do wygenerowania na podstawie danego kodu HTML pliku w formacie PDF i zapisania go w obiekcie `HttpResponse`.

Style CSS dla wygenerowanego dokumentu PDF są pobierane ze statycznego pliku *css/pdf.css*. Aby zlokalizować plik, należy użyć funkcji `finders()` z modułu `staticfiles`. Na koniec zwracamy tak przygotowaną odpowiedź.

Jeśli brakuje stylów CSS, pamiętaj, aby skopiować pliki statyczne znajdujące się w katalogu *static/* aplikacji `shop` do tej samej lokalizacji projektu.

Zawartość katalogu znajdziesz pod adresem *https://github.com/PacktPublishing/Django-5-by-Example/tree/main/Chapter09/myshop/shop/static*.

Ponieważ użyliśmy opcji `STATIC_ROOT`, konieczne jest dodanie jej do projektu. Będzie ona wskazywała ścieżkę dostępu do plików statycznych w projekcie. Przeprowadź edycję pliku *settings.py* projektu `myshop` i umieść w nim poniższy wiersz kodu.

```
STATIC_ROOT = BASE_DIR / 'static'
```

Teraz wydaj poniższe polecenie.

```
python manage.py collectstatic
```

Powinieneś otrzymać dane wyjściowe zawierające m.in. wiersz:

```
131 static files copied to 'code/myshop/static'.
```

Polecenie `collectstatic` kopiuje wszystkie pliki statyczne z aplikacji do katalogu zdefiniowanego w opcji `STATIC_ROOT`. Dzięki temu każda aplikacja może mieć własne pliki statyczne w katalogu *static*. Za pomocą opcji `STATICFILES_DIRS` można zdefiniować dodatkowe źródła plików statycznych. Wszystkie katalogi wymienione na liście `STATICFILES_DIRS` również zostaną skopiowane do katalogu `STATIC_ROOT` podczas wykonywania polecenia `collectstatic`. Za każdym razem, gdy wykonujesz polecenie `collectstatic` kolejny raz, wyświetli się pytanie, czy chcesz zastąpić istniejące pliki statyczne.

Przeprowadź edycję pliku *urls.py* aplikacji `orders` i dodaj poniższy wzorzec adresu URL oznaczony pogrubioną czcionką.

```
urlpatterns = [
    # ...
    path('admin/order/<int:order_id>/pdf/',
        views.admin_order_pdf,
        name='admin_order_pdf'
    ),
]
```

Teraz możesz zmodyfikować stronę administracyjną wyświetlającą listę modelu `Order` i dodać link pozwalający na wygenerowanie pliku PDF. Przeprowadź edycję pliku *admin.py* w aplikacji `orders` i umieść poniższy fragment kodu przed kodem klasy `OrderAdmin`.

```
def order_pdf(obj):
    url = reverse('orders:admin_order_pdf', args=[obj.id])
    return mark_safe(f'<a href="{url}">PDF</a>')
order_pdf.short_description = 'Faktura'
```

Jeśli określisz atrybut `short_description` dla obiektu wywoływalnego (ang. *callable*), Django użyje go jako nazwy kolumny.

Następnie dodaj `order_pdf` do atrybutu `list_display` klasy `OrderAdmin`, tak jak pokazałem poniżej.

```
class OrderAdmin(admin.ModelAdmin):
    list_display = [
        'id',
        'first_name',
        'last_name',
        'email',
        'address',
        'postal_code',
        'city',
        'paid',
        order_payment,
        'created',
        'updated',
        order_detail,
        order_pdf,
    ]
```

Upewnij się, że serwer programistyczny jest uruchomiony. W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/admin/orders/order/`. Teraz w każdym wierszu zamówienia powinien znajdować się link pozwalający na wygenerowanie dokumentu PDF (rysunek 9.31).

CREATED	UPDATED	ORDER DETAIL	FAKTURA
26 września 2024 06:52	26 września 2024 06:52	Wyświetl	PDF

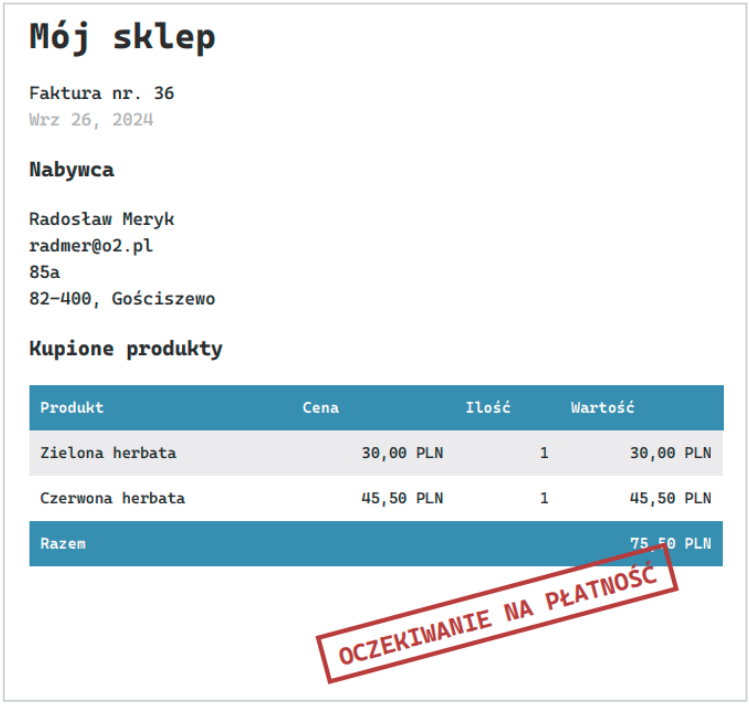
Rysunek 9.31. Link PDF dołączony do każdego wiersza zamówienia

Kliknij link *PDF* dla dowolnego zamówienia. Powinieneś zobaczyć wygenerowany plik PDF. W przypadku zamówień, które jeszcze nie zostały opłacone, powinien być podobny do pokazanego na rysunku 9.32.

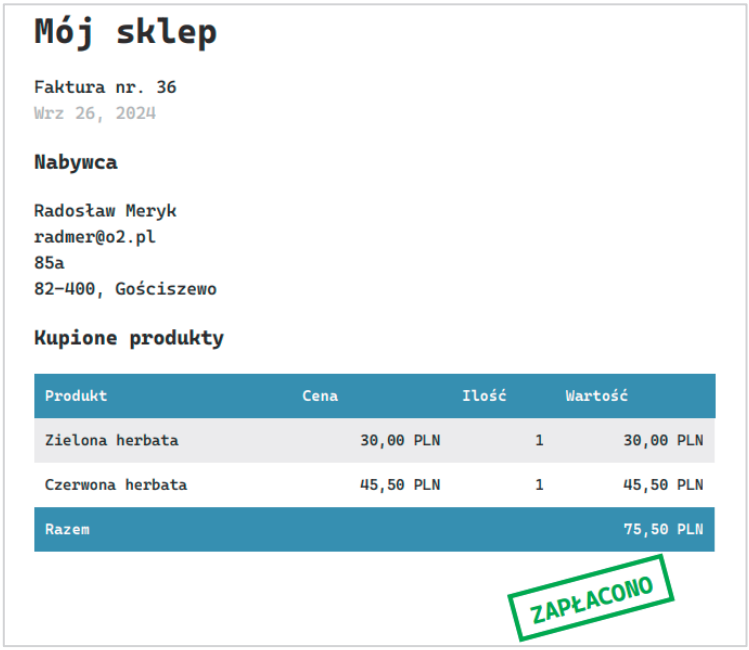
Jeżeli zamówienie zostało opłacone, wygenerowany rachunek PDF ma postać pokazaną na rysunku 9.33.

Wysyłanie dokumentów PDF za pomocą poczty elektronicznej

Przyjmujemy założenie, że po otrzymaniu płatności należy wysłać klientowi wiadomość e-mail wraz z fakturą w formacie PDF. Do wykonania tej operacji utworzymy asynchroniczne zadanie.



Rysunek 9.32. Faktura PDF dla nieopłaconego zamówienia



Rysunek 9.33. Faktura PDF dla opłaconego zamówienia

Utwórz nowy plik w katalogu aplikacji `payment` i nadaj mu nazwę *tasks.py*. Następnie w pliku umieść poniższy fragment kodu.

```
from io import BytesIO
import weasyprint
from celery import shared_task
from django.contrib.staticfiles import finders
from django.core.mail import EmailMessage
from django.template.loader import render_to_string
from orders.models import Order


@shared_task
def payment_completed(order_id):
    """
    Zadanie wysyła e-mailem powiadomienie po pomyślnym opłaceniu zamówienia
    """
    order = Order.objects.get(id=order_id)
    # Utworzenie wiadomości e-mail z fakturą.
    subject = f'Mój sklep - faktura nr {order.id}'
    message = (
        'W załączniku przesyłamy fakturę dla ostatniego zakupu.'
    )
    email = EmailMessage(
        subject, message, 'admin@myshop.com', [order.email]
    )
    #  Wygenerowanie dokumentu PDF.
    html = render_to_string('orders/order/pdf.html', {'order': order})
    out = BytesIO()
    stylesheets=[weasyprint.CSS(finders.find('css/pdf.css'))]
    weasyprint.HTML(string=html).write_pdf(out, stylesheets=stylesheets)
    #  Dołączenie pliku w formacie PDF.
    email.attach(
        f'order_{order.id}.pdf', out.getvalue(), 'application/pdf'
    )
    # Wysłanie wiadomości e-mail.
    email.send()
```

Za pomocą dekoratora `@shared_task` zdefiniowaliśmy zadanie `payment_completed`. W tym zadaniu wykorzystaliśmy dostarczaną przez Django klasę `EmailMessage` do utworzenia obiektu `email`. Następnie do zmiennej `html` wyrenderowaliśmy zawartość szablonu HTML. Kolejnym krokiem jest wygenerowanie dokumentu PDF na podstawie wspomnianego szablonu i przekazanie tego dokumentu do egzemplarza `BytesIO`, który jest umieszczonym w pamięci buforem bajtów. Tak przygotowany dokument PDF dołączamy do obiektu `EmailMessage` za pomocą metody `attach()`, nie zapominając przy tym o zawartości bufora `out`. Na koniec wysyłamy wiadomość e-mail.

Koniecznie musisz pamiętać o odpowiednich ustawieniach **SMTP** w pliku *settings.py* projektu, ponieważ są one niezbędne do wysyłania wiadomości e-mail. Jeżeli chcesz zobaczyć przykład działającej konfiguracji SMTP, zajrzyj do rozdziału 2., „Usprawnianie bloga i dodanie funkcjonalności społecznościowych”. Jeśli nie chcesz konfigurować ustawień e-maili, możesz polecić Django, by zapisywał e-maile do konsoli. Aby to zrobić, wprowadź poniższe ustawienie do pliku *settings.py*.

```
EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'
```

Teraz do punktu końcowego webhooka obsługującego zdarzenia zakończenia płatności dodamy zadanie `payment_completed`.

Przeprowadź edycję pliku *webhooks.py* aplikacji `payment` i zmodyfikuj go tak, by miał następującą zawartość.

```
import stripe
from django.conf import settings
from django.http import HttpResponse
from django.views.decorators.csrf import csrf_exempt
from orders.models import Order
from .tasks import payment_completed


@csrf_exempt
def stripe_webhook(request):
    payload = request.body
    sig_header = request.META['HTTP_STRIPE_SIGNATURE']
    event = None
    try:
        event = stripe.Webhook.construct_event(
            payload, sig_header, settings.STRIPE_WEBHOOK_SECRET
        )
    except ValueError as e:
        # Nieprawidłowy ładunek danych
        return HttpResponse(status=400)
    except stripe.error.SignatureVerificationError as e:
        # Niepoprawna sygnatura
        return HttpResponse(status=400)
    if event.type == 'checkout.session.completed':
        session = event.data.object
        if (
            session.mode == 'payment'
            and session.payment_status == 'paid'
        ):
            try:
                order = Order.objects.get(
                    id=session.client_reference_id
                )
            except Order.DoesNotExist:
                return HttpResponse(status=404)
            # Oznaczenie zamówienia jako opłaconego
            order.paid = True
            # Zapisanie identyfikatora płatności Stripe
            order.stripe_id = session.payment_intent
            order.save()
            # Uruchomienie zadania asynchronicznego
            payment_completed.delay(order.id)
    return HttpResponse(status=200)
```

Zadanie `payment_completed` zostało umieszczane w kolejce przez wywołanie jego metody `delay()`. Zadanie będzie dodane do kolejki i uruchomione przez wątek roboczy Celery tak szybko, jak to będzie możliwe.

Wkrótce potem powinieneś otrzymać wiadomość e-mail wraz z fakturą w formacie PDF. Jeśli używasz aplikacji `console.EmailBackend` w roli warstwy backend poczty e-mail, w powłoce, w której używasz Celery, wyświetlą się takie dane wyjściowe:

```
MIME-Version: 1.0
Subject: My Shop - Invoice no. 7
From: admin@myshop.com
To: email@domain.com
Date: Wed, 3 Jan 2024 20:15:24 -0000
Message-ID: <164841212458.94972.10344068999595916799@amele-mbp.home>
--===============8908668108717577350==
Content-Type: text/plain; charset="utf-8"
MIME-Version: 1.0
Content-Transfer-Encoding: 7bit
W załączniku przesyłamy fakturę dla ostatniego zakupu.
--===============8908668108717577350==
Content-Type: application/pdf
MIME-Version: 1.0
Content-Transfer-Encoding: base64
Content-Disposition: attachment; filename="order_7.pdf"
JVBERiOxLjcKJfCflqQKMSAwIG9iago8PAovVHlwZSA...
```

Ten wynik pokazuje, że wiadomość e-mail zawiera załącznik. We wcześniejszej części książki nauczyłeś się dołączać pliki do wiadomości e-mail i wysyłać je programowo.

Gratulacje! Zakończyłeś integrację z serwisem Stripe i dodałeś wartościową funkcjonalność do swojego sklepu.

Podsumowanie

W tym rozdziale zintegrowałeś bramkę płatności Stripe ze swoim projektem i utworzyłeś punkt końcowy webhooka do otrzymywania powiadomień o płatnościach. Zbudowałeś niestandardową operację administracyjną w celu eksportu zamówień do formatu CSV. Dostosowałeś także do własnych potrzeb witrynę administracyjną Django, korzystając z niestandardowych widoków i szablonów. Na koniec dowiedziałeś się, jak generować pliki w formacie PDF za pomocą WeasyPrint i wysyłać je e-mailem.

W następnym rozdziale opracujemy za pomocą Django system kuponów i zbudujemy silnik rekomendacji produktu z wykorzystaniem Redis.

Dodatkowe zasoby

Oto lista zasobów zawierających dodatkowe informacje związane z tematami omawianymi w tym rozdziale:

- Kod źródłowy przykładów zamieszczonych w tym rozdziale — *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter09*.
- Witryna Stripe — *https://www.stripe.com/*.
- Dokumentacja Stripe Checkout — *https://stripe.com/docs/payments/checkout*.

- Tworzenie konta Stripe — *https://dashboard.stripe.com/register*.
- Ustawienia konta Stripe — *https://dashboard.stripe.com/settings/account*.
- Biblioteka Stripe Pythona — *https://github.com/stripe/stripe-python*.
- Testowe klucze API serwisu Stripe — *https://dashboard.stripe.com/test/apikeys*.
- Dokumentacja kluczy API serwisu Stripe — *https://stripe.com/docs/keys*.
- Informacje o wydaniu Stripe API z 10.04.2024 r. — *https://stripe.com/docs/upgrades#2024-04-10*.
- Tryby sesji Stripe Checkout — *https://stripe.com/docs/api/checkout/sessions/object#checkout_session_object-mode*.
- Budowanie bezwzględnych adresów URI za pomocą Django — *https://docs.djangoproject.com/en/5.0/ref/request-response/#django.http.HttpRequest.build_absolute_uri*.
- Tworzenie sesji Stripe — *https://stripe.com/docs/api/checkout/sessions/create*.
- Waluty obsługiwane przez Stripe — *https://stripe.com/docs/currencies*.
- Pulpit Stripe Payments — *https://dashboard.stripe.com/test/payments*.
- Karty kredytowe do testowania płatności Stripe — *https://stripe.com/docs/testing*.
- Webhooki w serwisie Stripe — *https://dashboard.stripe.com/test/webhooks*.
- Rodzaje zdarzeń wysyłanych przez Stripe — *https://stripe.com/docs/api/events/types*.
- Instalacja Stripe CLI — *https://stripe.com/docs/stripe-cli#install*.
- Najnowsza wersja Stripe CLI — *https://github.com/stripe/stripe-cli/releases/latest*.
- Generowanie plików CSV za pomocą Django — *https://docs.djangoproject.com/en/5.0/howto/outputting-csv/*.
- Aplikacja `django-import-export` — *https://django-import-export.readthedocs.io/en/latest/*.
- Aplikacja `django-import-export-celery` — *https://github.com/auto-mat/django-import-export-celery*.
- Szablony administracyjne Django — *https://github.com/django/django/tree/5.0/django/contrib/admin/templates/admin*.
- Generowanie plików PDF za pomocą biblioteki ReportLab — *https://docs.djangoproject.com/en/5.0/howto/outputting-pdf/*.
- Instalacja WeasyPrint — *https://doc.courtbouillon.org/weasyprint/stable/first_steps.html*.
- Pliki statyczne do tego rozdziału — *https://github.com/PacktPublishing/Django-5-by-Example/tree/main/Chapter09/myshop/shop/static*.

Rozdział 10

Rozbudowa sklepu internetowego

W poprzednim rozdziale dowiedziałeś się, jak zintegrować bramkę płatności ze sklepem internetowym. Nauczyłeś się także generować pliki w formatach CSV i PDF.

W tym rozdziale dodamy do budowanego sklepu system kuponów i stworzymy silnik rekomendacji produktów.

Oto zagadnienia, na których skoncentruję się w tym rozdziale:

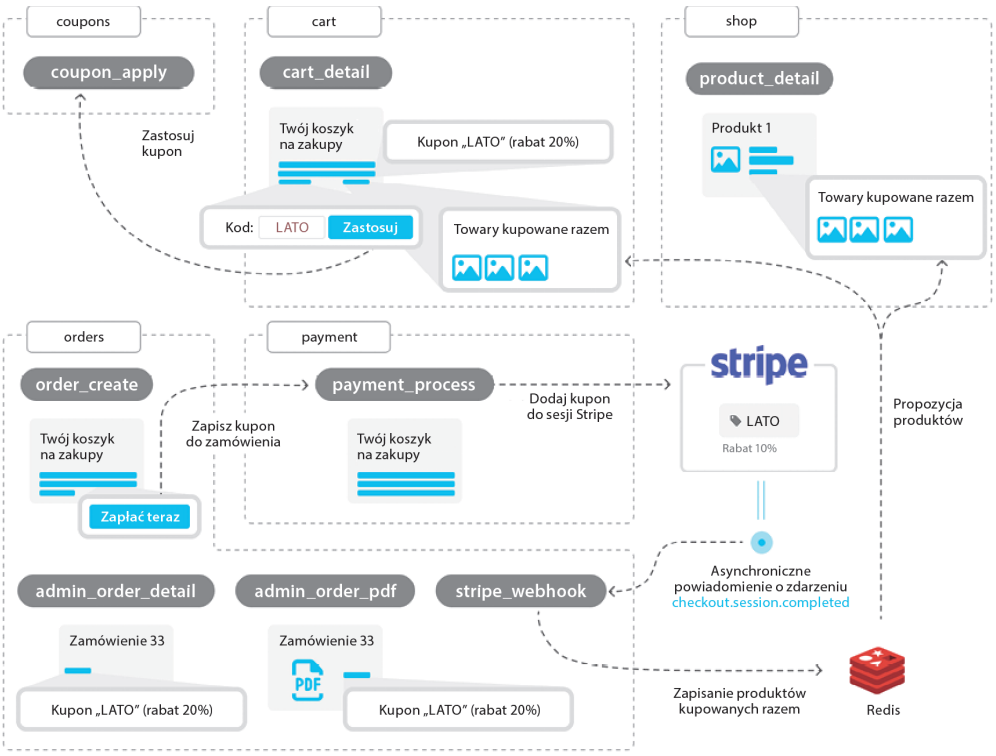
- Utworzenie systemu kuponów
- Zastosowanie kuponów do produktów w koszyku na zakupy
- Zastosowanie kuponów do zamówień
- Zastosowanie kuponów w sesji Stripe Checkout
- Zapisywanie produktów, które są zwykle kupowane razem
- Opracowanie silnika rekomendacji produktu za pomocą Redis

Przegląd funkcjonalności

Reprezentację widoków, szablonów i funkcjonalności, które zbudujemy w tym rozdziale, przedstawiono na rysunku 10.1.

W tym rozdziale, aby zastosować kupony rabatowe do sesji koszyka na zakupy, zbudujesz nową aplikację `coupons` i utworzysz widok `coupon_apply`. W widoku `cart_detail` aplikacji `cart` zaimplementujesz możliwość udzielania rabatu. Po utworzeniu zamówienia w widoku `order_create` aplikacji `orders` zostanie w nim zapisany kupon. Następnie, gdy utworzysz sesję Stripe w widoku `payment_process` aplikacji `payment`, przed przekierowaniem użytkownika do Stripe w celu dokończenia płatności dodasz ten kupon do sesji płatności Stripe. Do szablonów widoków administracyjnych `admin_order_detail` i `admin_order_pdf` aplikacji `order` dodamy możliwość udzielania rabatu. Oprócz systemu kuponów zaimplementujesz system rekomendacji. Po odebraniu w widoku `stripe_webhook` zdarzenia Stripe `checkout.session.completed` produkty, które zostały kupione razem, zostaną zapisane w bazie danych Redis. Do widoków `product_detail` i `cart_detail` zostaną dodane rekomendacje produktów przez pobranie z bazy danych Redis towarów, które są często kupowane razem.

Kod źródłowy przykładów z tego rozdziału można znaleźć pod adresem *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter10*.



Rysunek 10.1. Schemat funkcjonalności budowanych w rozdziale 10.

Wszystkie pakiety Pythona użyte w tym rozdziale są zawarte w pliku *requirements.txt* dołączonym do plików z kodem źródłowym przykładów z tego rozdziału. Aby zainstalować każdy pakiet Pythona, możesz postępować zgodnie z instrukcjami zamieszczonymi w poniższych podrozdziałach. Możesz także zainstalować wszystkie wymagania jednocześnie za pomocą polecenia `pip install -r requirements.txt`.

Utworzenie systemu kuponów

Wiele sklepów internetowych rozdaje klientom kupony, które pozwalają na otrzymanie rabatów podczas kolejnych zakupów. Tego rodzaju kupon najczęściej ma postać kodu przekazywanego klientowi i ważnego przez ustalony czas.

Zaimplementujemy system kuponów dla budowanej aplikacji sklepu internetowego. Kupon będzie ważny jedynie przez podany czas. Ponadto kupon będzie mógł być użyty wielokrotnie, a rabat zostanie naliczony od całkowitej wartości koszyka na zakupy.

Do przygotowania tego rodzaju funkcjonalności musimy utworzyć model przeznaczony do przechowywania kodu kuponu, okresu jego ważności oraz wysokości udzielonego rabatu.

Za pomocą poniższego polecenia utwórz nową aplikację w projekcie `myshop`.

```
python manage.py startapp coupons
```

Przeprowadź edycję pliku *settings.py* projektu `myshop` i umieść nową aplikację na liście `INSTALLED_APPS`, tak jak pokazałem poniżej.

```
INSTALLED_APPS = [
    # ...
    'coupons.apps.CouponsConfig',
]
```

W tym momencie nowa aplikacja w projekcie Django jest aktywna.

Utworzenie modeli kuponu

Pracę rozpoczynamy od opracowania modelu `Coupon`. Przeprowadź edycję pliku *models.py* aplikacji `coupons` i umieść w nim poniższy fragment kodu.

```
from django.core.validators import MaxValueValidator, MinValueValidator
from django.db import models


class Coupon(models.Model):
    code = models.CharField(max_length=50, unique=True)
    valid_from = models.DateTimeField()
    valid_to = models.DateTimeField()
    discount = models.IntegerField(
        validators=[MinValueValidator(0), MaxValueValidator(100)],
        help_text='Percentage value (0 to 100)'
    )
    active = models.BooleanField()

    def __str__(self):
        return self.code
```

Ten model wykorzystamy do przechowywania kuponów. Model `Coupon` zawiera następujące pola.

- `code`. Kod, który użytkownik musi wprowadzić, aby uzyskać rabat podczas zakupów.
- `valid_from`. Wartość daty i godziny wskazująca, od kiedy dany kupon jest aktywny.
- `valid_to`. Wartość daty i godziny wskazująca, do kiedy dany kupon jest aktywny.
- `discount`. Wysokość udzielonego rabatu (jest to wartość procentowa, więc dozwolone wartości mieszczą się w zakresie od `0` do `100`). Dla tej kolumny zastosujemy walidatory, aby ograniczyć minimalną i maksymalną akceptowaną wartość.
- `active`. Wartość typu `Boolean` określająca, czy dany kupon jest aktywny.

Wydaj poniższe polecenie w celu wygenerowania początkowej migracji aplikacji `coupons`.

```
python manage.py makemigrations
```

Powinieneś otrzymać dane wyjściowe zawierające m.in. poniższe wiersze.

```
Migrations for 'coupons':
   coupons/migrations/0001_initial.py
     - Create model Coupon
```

Następnie wydaj polecenie faktycznie przeprowadzające migrację.

```
python manage.py migrate
```

Powinieneś otrzymać dane wyjściowe zawierające m.in. poniższy wiersz.

```
Applying coupons.0001_initial... OK
```

W tym momencie migracje zostały zastosowane w bazie danych. Przystępujemy do dodania modelu `Coupon` do witryny administracyjnej. Przeprowadź edycję pliku *admin.py* aplikacji `coupons` i umieść w nim poniższy fragment kodu.

```
from django.contrib import admin
from .models import Coupon

@admin.register(Coupon)
class CouponAdmin(admin.ModelAdmin):
    list_display = [
        'code',
        'valid_from',
        'valid_to',
        'discount',
        'active'
    ]
    list_filter = ['active', 'valid_from', 'valid_to']
    search_fields = ['code']
```

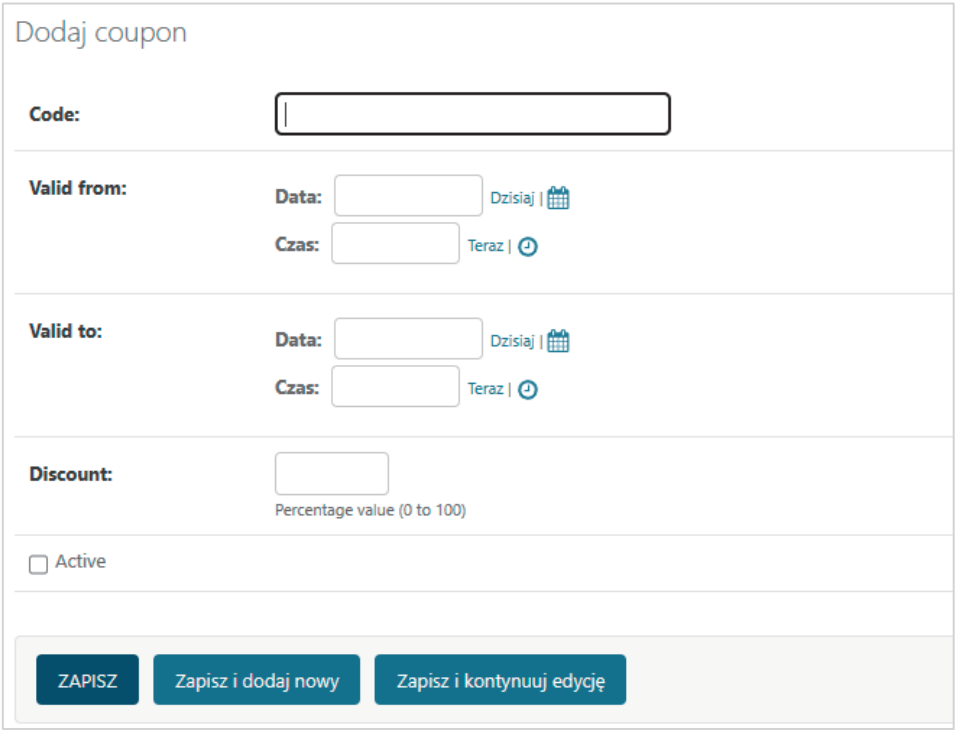
Model `Coupon` jest teraz zarejestrowany w witrynie administracyjnej Django. Za pomocą poniższego polecenia uruchom lokalny serwer programistyczny.

```
python manage.py runserver
```

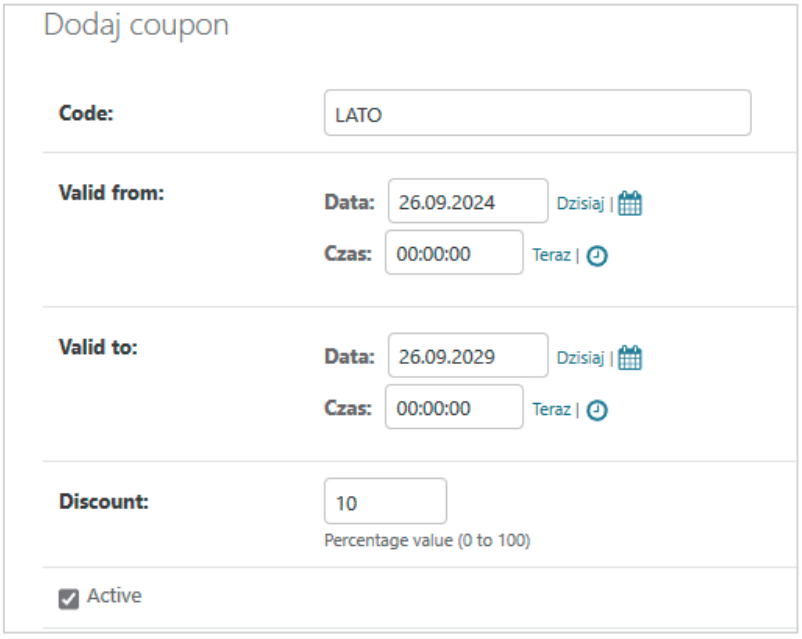
W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/admin/coupons/coupon/add/`.

Powinieneś zobaczyć formularz jak rysunku 10.2.

Wypełnij formularz w celu utworzenia nowego kuponu ważnego od zaraz. Upewnij się, że zaznaczone jest pole wyboru *Active*, i kliknij przycisk *ZAPISZ*. Przykład utworzenia kuponu został pokazany na rysunku 10.3.

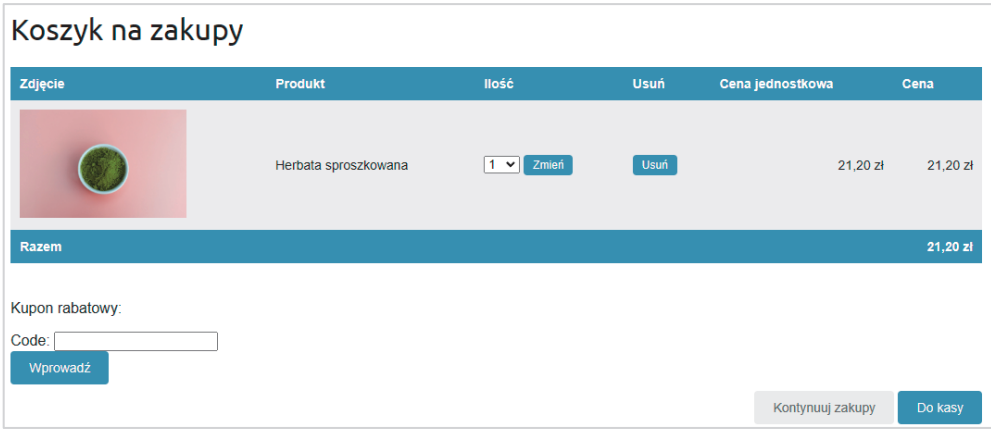


Rysunek 10.2. Formularz dodawania kuponu na stronie administracyjnej Django



Rysunek 10.3. Formularz dodawania kuponu z przykładowymi danymi

Po utworzeniu kuponu strona z listą zmian kuponów w witrynie administracyjnej będzie wyglądać podobnie jak na rysunku 10.4.



Rysunek 10.4. Widok zmiany listy kuponów widoczny na stronie administracyjnej Django

Następnie zaimplementujemy funkcjonalność dodawania kuponów do koszyka na zakupy.

Zastosowanie kuponu w koszyku na zakupy

Teraz możemy już przechowywać kupony w bazie danych i wykonywać zapytania w celu ich pobierania. Potrzebny jest więc sposób umożliwiający klientom zastosowanie kuponu w trakcie zakupów. Mechanizm zastosowania kuponu powinien być następujący.

1. Użytkownik dodaje produkty do koszyka na zakupy.
2. W formularzu wyświetlonym na stronie szczegółów koszyka na zakupy użytkownik wprowadza kupon.
3. Po wprowadzeniu kuponu i wysłaniu formularza szukamy istniejącego i aktywnego kuponu o podanym kodzie. Konieczne jest sprawdzenie, czy kod kuponu dokładnie odpowiada wprowadzonemu przez użytkownika, czy wartością atrybutu `active` jest `True` oraz czy bieżąca data i godzina mieszczą się w zakresie od `valid_from` do `valid_to`.
4. Jeżeli odpowiedni kupon zostanie znaleziony, zapisujemy go w sesji użytkownika, wyświetlamy koszyk na zakupy wraz z zastosowanym kuponem oraz uaktualniamy całkowitą kwotę do zapłaty.
5. Kiedy użytkownik składa zamówienie, zapisujemy kupon wraz z danym zamówieniem.

W katalogu aplikacji `coupons` utwórz nowy plik o nazwie *forms.py*. Następnie w pliku umieść poniższy fragment kodu.

```
from django import forms


class CouponApplyForm(forms.Form):
    code = forms.CharField()
```

Jest to formularz, który pozwoli użytkownikowi na wprowadzenie kodu kuponu. Przeprowadź edycję pliku *views.py* w katalogu aplikacji `coupons` i umieść w nim poniższy fragment kodu.

```
from django.shortcuts import redirect
from django.utils import timezone
from django.views.decorators.http import require_POST
from .forms import CouponApplyForm
from .models import Coupon


@require_POST
def coupon_apply(request):
    now = timezone.now()
    form = CouponApplyForm(request.POST)
    if form.is_valid():
        code = form.cleaned_data['code']
        try:
            coupon = Coupon.objects.get(
                code__iexact=code,
                valid_from__lte=now,
                valid_to__gte=now,
                active=True
            )
            request.session['coupon_id'] = coupon.id
        except Coupon.DoesNotExist:
            request.session['coupon_id'] = None
    return redirect('cart:cart_detail')
```

Widok `coupon_apply` weryfikuje kupon i przechowuje go w sesji użytkownika. W tym widoku stosujemy dekorator `require_ POST`, aby ograniczyć jego wyświetlanie jedynie do żądań `POST`. W widoku przeprowadzane są następujące działania.

1. Utworzenie egzemplarza formularza `CouponApplyForm` na podstawie przekazanych danych oraz sprawdzenie, czy formularz jest prawidłowy.
2. Jeżeli formularz jest prawidłowy, ze słownika `cleaned_data` pobieramy kod (`code`) wprowadzony przez użytkownika. Próbujemy pobrać obiekt `Coupon` wraz z danym kodem. Wykorzystujemy `iexact` do przeprowadzenia operacji wyszukiwania bez uwzględnienia wielkości liter. Kupon musi być aktywny (`active=True`) i ważny w bieżącym czasie. Do pobrania bieżącej daty i godziny z uwzględnieniem strefy czasowej używamy dostarczanej przez Django funkcji `timezone.now()`. Następnie porównujemy ją z wartościami w kolumnach `valid_from` i `valid_to`, przeprowadzając operacje wyszukiwania, odpowiednio, `lte` (mniejszy niż lub równy) i `gte` (większy niż lub równy).
3. Identyfikator kuponu zapisujemy w sesji użytkownika.
4. Przekierowujemy użytkownika do adresu URL widoku `cart_detail`, aby wyświetlić zawartość koszyka na zakupy wraz z zastosowanym kuponem.

Dla widoku `coupon_apply` potrzebny jest wzorzec adresu URL. Utwórz nowy plik w katalogu aplikacji `coupons` i nadaj mu nazwę *urls.py*. Następnie w pliku umieść poniższy fragment kodu.

```
from django.urls import path
from . import views


app_name = 'coupons'

urlpatterns = [
    path('apply/', views.coupon_apply, name='apply'),
]
```

Kolejnym krokiem jest przeprowadzenie edycji głównego pliku *urls.py* projektu `myshop` i dodanie wzorca adresu URL `coupons` za pomocą następującego wiersza wyróżnionego pogrubioną czcionką:

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('cart/', include('cart.urls', namespace='cart')),
    path('orders/', include('orders.urls', namespace='orders')),
    path('payment/', include('payment.urls', namespace='payment')),
    path('coupons/', include('coupons.urls', namespace='coupons')),
    path('', include('shop.urls', namespace='shop')),
]
```

Pamiętaj o umieszczeniu tego wzorca przed wzorcem *shop.urls*.

Teraz przeprowadź edycję pliku *cart.py* aplikacji `cart` i dodaj pokazane niżej polecenie `import`.

```
from coupons.models import Coupon
```

W celu zainicjowania kuponu z bieżącej sesji, dodaj na końcu metody `__init__()` klasy `Cart` poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
class Cart:
    def __init__(self, request):
        """
        Zainicjuj koszyk.
        """
        self.session = request.session
        cart = self.session.get(settings.CART_SESSION_ID)
        if not cart:
            # Zapis pustego koszyka na zakupy w sesji.
            cart = self.session[settings.CART_SESSION_ID] = {}
        self.cart = cart
        # Zapisanie wykorzystywanego kuponu.
        self.coupon_id = self.session.get('coupon_id')
```

W tym kodzie próbujemy pobrać klucz sesji `coupon_id` z bieżącej sesji i przechowywać jego wartość w obiekcie `Cart`. Dołącz do obiektu `Cart` poniższe metody wyróżnione pogrubioną czcionką.

```
class Cart:
    # ...

    @property
    def coupon(self):
        if self.coupon_id:
```

```
            try:
                return Coupon.objects.get(id=self.coupon_id)
            except Coupon.DoesNotExist:
                pass
        return None

    def get_discount(self):
        if self.coupon:
            return (
                self.coupon.discount / Decimal(100)
            ) * self.get_total_price()
        return Decimal(0)

    def get_total_price_after_discount(self):
        return self.get_total_price() - self.get_discount()
```

Oto sposób działania dodanych metod:

- `coupon()`. Definiujemy tę metodę jako właściwość (z dekoratorem `@property`). Jeżeli koszyk na zakupy zawiera funkcję `coupon_id`, metoda zwróci obiekt `Coupon` o podanym identyfikatorze.
- `get_discount()`. Jeżeli koszyk na zakupy zawiera obiekt `coupon`, pobieramy wskazaną w nim wysokość rabatu i zwracamy kwotę, która powinna zostać odjęta od wartości całkowitej koszyka na zakupy.
- `get_total_price_after_discount()`. Zwracamy wartość całkowitą koszyka na zakupy po odjęciu kwoty obliczonej w metodzie `get_discount()`.

Klasa `Cart` jest w tym momencie przygotowana na obsługę kuponu użytego w bieżącej sesji i zastosowanie odpowiedniego rabatu.

Przechodzimy do uwzględnienia systemu kuponów w widoku szczegółowym koszyka na zakupy. Przeprowadź edycję pliku *views.py* aplikacji `cart` i umieść poniższe polecenie `import` na początku pliku.

```
from coupons.forms import CouponApplyForm
```

Następnie przeprowadź edycję widoku `cart_detail` i umieść w nim nowy formularz, tak jak pokazałem poniżej.

```
def cart_detail(request):
    cart = Cart(request)
    for item in cart:
        item['update_quantity_form'] = CartAddProductForm(
            initial={'quantity': item['quantity'], 'override': True}
        )
    coupon_apply_form = CouponApplyForm()
    return render(
        request,
        'cart/detail.html',
        {
            'cart': cart,
            'coupon_apply_form': coupon_apply_form
        }
    )
```

Przeprowadź edycję szablonu *cart/detail.html* aplikacji `cart` i znajdź poniższe wiersze kodu.

```
<tr class="total">
  <td>Kwota do zapłaty</td>
  <td colspan="4"></td>
  <td class="num">{{ cart.get_total_price }} PLN</td>
</tr>
```

Zastąp je poniższym fragmentem kodu.

```
{% if cart.coupon %}
  <tr class="subtotal">
    <td>Suma częściowa</td>
    <td colspan="4"></td>
    <td class="num">{{ cart.get_total_price|floatformat:2 }} zł</td>
  </tr>
  <tr>
    <td>
      kupon "{{ cart.coupon.code }}"
      (rabat {{ cart.coupon.discount }}%)
    </td>
    <td colspan="4"></td>
    <td class="num neg">
      - {{ cart.get_discount|floatformat:"2" }} PLN
    </td>
  </tr>
{% endif %}
<tr class="total">
  <td>Razem</td>
  <td colspan="4"></td>
  <td class="num">
    {{ cart.get_total_price_after_discount|floatformat:2 }} PLN
  </td>
</tr>
```

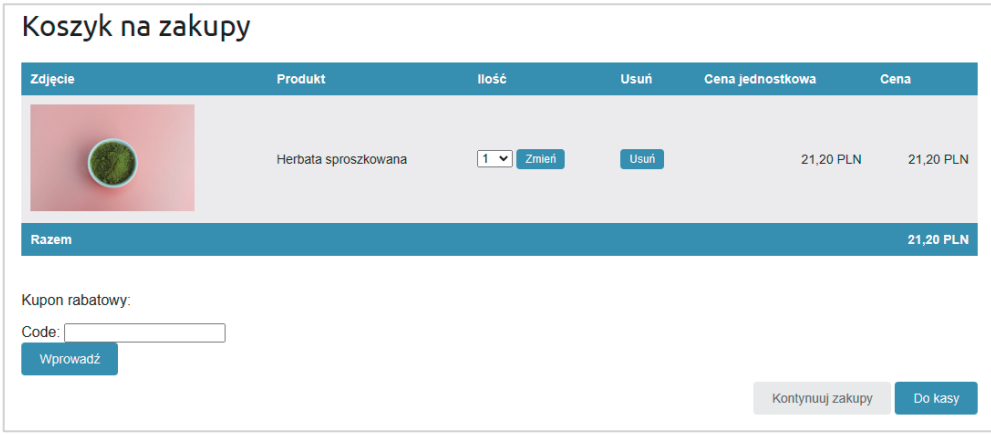
Kod ten jest przeznaczony do wyświetlenia opcjonalnego kuponu oraz wysokości udzielonego rabatu. Jeżeli koszyk na zakupy zawiera kupon, wyświetlamy pierwszy wiersz wraz z wartością całkowitą koszyka na zakupy. Następnie w wierszu drugim pokazujemy bieżący kupon zastosowany w tym koszyku. Na koniec wyświetlamy całkowitą kwotę do zapłaty po uwzględnieniu rabatu, co odbywa się za pomocą wywołania metody `get_total_price_after_discount()` obiektu `cart`.

W tym samym pliku umieść poniższy fragment kodu po znaczniku HTML `</table>`.

```
<p>Kupon rabatowy:</p>
<form action="{% url "coupons:apply" %}" method="post">
  {{ coupon_apply_form }}
  <input type="submit" value="Wprowadź">
  {% csrf_token %}
</form>
```

Kod jest odpowiedzialny za wyświetlenie formularza pozwalającego na wprowadzenie kuponu i jego zastosowanie w bieżącym koszyku na zakupy.

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/`, a następnie dodaj wybrane produkty do koszyka na zakupy. Powinieneś zobaczyć, że koszyk na zakupy uwzględnia rabat udzielony przez dany kupon (rysunek 10.5).

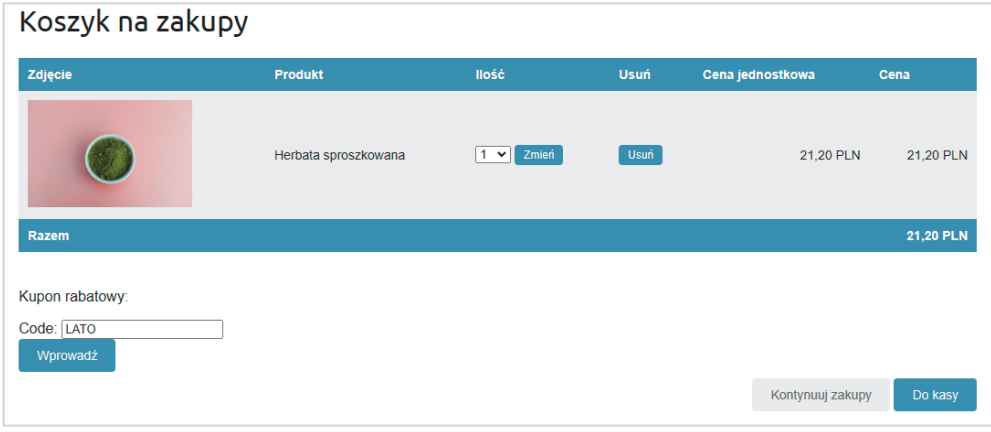


Rysunek 10.5. Strona szczegółów koszyka włącznie z danymi kuponu i formularzem do jego zastosowania

Uwaga

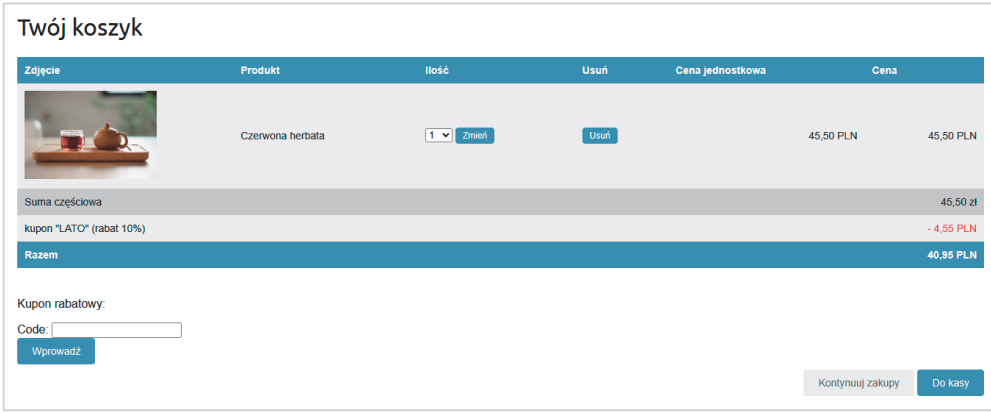
Zdjęcie *herbaty sproszkowanej*: Phuong Nguyen. Zdjęcie pochodzi z serwisu Unsplash.

W polu *Code* wprowadź kod kuponu utworzony za pomocą strony administracyjnej (rysunek 10.6).



Rysunek 10.6. Strona szczegółów koszyka włącznie z danymi kuponu i formularzem do jego zastosowania

Kliknij przycisk *Wprowadź*. Kupon zostanie zastosowany, a koszyk wyświetli rabat związany z kuponem, tak jak pokazałem na rysunku 10.7.



Rysunek 10.7. Strona szczegółów koszyka włącznie z danymi kuponu i formularzem do jego zastosowania

Przechodzimy teraz do dodania kuponu do kolejnego kroku procesu składania zamówienia. Przeprowadź edycję szablonu *orders/order/create.html* aplikacji `orders` i znajdź poniższe wiersze kodu.

```
<ul>
  {% for item in cart %}
    <li>
      {{ item.quantity }}x {{ item.product.name }}
      <span>{{ item.total_price }} PLN</span>
    </li>
  {% endfor %}
</ul>
```

Zastąp je kodem:

```
<ul>
  {% for item in cart %}
    <li>
      {{ item.quantity }}x {{ item.product.name }}
      <span>{{ item.total_price|floatformat:2 }} PLN</span>
    </li>
  {% endfor %}
  {% if cart.coupon %}
    <li>
      "rabat {{ cart.coupon.code }}" ({{ cart.coupon.discount }}%)
      <span class="neg">-{{ cart.get_discount|floatformat:2 }} PLN</span>
    </li>
  {% endif %}
</ul>
```

Podsumowanie zamówienia powinno teraz uwzględniać zastosowany (ewentualnie) kupon. Odszukaj poniższy wiersz kodu:

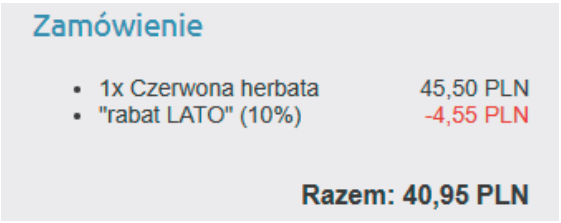
```
<p>Razem: {{ cart.get_total_price }} PLN</p>
```

i zastąp go tym:

```
<p>Razem: {{ cart.get_total_price_after_discount|floatformat:2 }} PLN</p>
```

W ten sposób całkowita kwota do zapłaty będzie obliczona wraz z uwzględnieniem rabatu wynikającego z użytego kuponu.

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/orders/create/`. Powinieneś zobaczyć, że podsumowanie zamówienia uwzględnia teraz zastosowany kupon, co pokazałem na rysunku 10.8.



Rysunek 10.8. Podsumowanie zamówienia włącznie z zastosowanym kuponem

Użytkownik może teraz stosować kupon w koszyku na zakupy. Jednak nadal musimy przechowywać informacje o kuponie w zamówieniu, które jest tworzone, gdy użytkownik zdecyduje się na przejście do kasy.

Zastosowanie kuponu w zamówieniu

Będziemy przechowywać kupony, które zostały zastosowane w poszczególnych zamówieniach. Przede wszystkim trzeba zmodyfikować model `Order`, aby przechowywać powiązany z nim obiekt `Coupon`, o ile istnieje.

Przeprowadź edycję pliku *models.py* aplikacji `orders` i umieść w nim poniższe polecenia `import`.

```
from decimal import Decimal
from django.core.validators import MaxValueValidator, MinValueValidator
from coupons.models import Coupon
```

Następnie dodaj do modelu `Order` poniższe pola.

```
class Order(models.Model):
    # …
    coupon = models.ForeignKey(
        Coupon,
        related_name='orders',
        null=True,
        blank=True,
        on_delete=models.SET_NULL
    )
    discount = models.IntegerField(
        default=0,
        validators=[MinValueValidator(0), MaxValueValidator(100)]
    )
```

Te pola pozwalają na przechowywanie opcjonalnego kuponu zastosowanego w zamówieniu oraz wysokości rabatu udzielonego w tym kuponie. Wprawdzie wysokość rabatu

jest przechowywana w powiązanym obiekcie `Coupon`, ale umieścimy ją również w modelu `Order`, aby ją zachować na wypadek modyfikacji lub usunięcia kuponu. Ustawiliśmy zdarzenie `on_delete` na wartość `models.SET_NULL`. Dzięki temu w przypadku usunięcia kuponu pole `coupon` zostanie ustawione na `Null`, ale rabat będzie zachowany.

Ponieważ model `Order` został zmieniony, konieczne jest przygotowanie migracji. Z poziomu powłoki wydaj poniższe polecenie.

```
python manage.py makemigrations
```

Powinieneś otrzymać poniższe dane wyjściowe.

```
Migrations for 'orders':
    orders/migrations/0003_order_coupon_order_discount.py
     - Add field coupon to order
     - Add field discount to order
```

Teraz przeprowadź faktyczną migrację za pomocą poniższego polecenia.

```
python manage.py migrate orders
```

Powinieneś otrzymać potwierdzenie przeprowadzenia migracji.

```
Applying orders.0003_order_coupon_order_discount… OK
```

Zmiany wprowadzone w polach modelu `Order` zostały zsynchronizowane z bazą danych.

Przeprowadź edycję pliku *models.py* i dodaj do modelu `Order` dwie nowe metody: `get_total_cost_before_discount()` i `get_discount()`, tak jak pokazałem poniżej. Nowy kod został wyróżniony pogrubioną czcionką.

```
class Order(models.Model):
    # …
    def get_total_cost_before_discount(self):
        return sum(item.get_cost() for item in self.items.all())

    def get_discount(self):
        total_cost = self.get_total_cost_before_discount()
        if self.discount:
            return total_cost * (self.discount / Decimal(100))
        return Decimal(0)
```

Następnie zmodyfikuj metodę `get_total_cost()` modelu `Order` w następujący sposób. Nowy kod został wyróżniony pogrubioną czcionką.

```
    def get_total_cost(self):
        total_cost = self.get_total_cost_before_discount()
        return total_cost - self.get_discount()
```

Metoda `get_total_cost()` modelu `Order` będzie teraz uwzględniała ewentualny rabat.

Przeprowadź edycję pliku *views.py* aplikacji `orders` i zmodyfikuj widok `order_create`, aby podczas tworzenia nowego zamówienia zapisać powiązany z nim kupon i wysokość udzielonego rabatu. Dodaj do widoku `order_create` następujący kod wyróżniony pogrubioną czcionką.

```
def order_create(request):
    cart = Cart(request)
    if request.method == 'POST':
        form = OrderCreateForm(request.POST)
        if form.is_valid():
            order = form.save(commit=False)
            if cart.coupon:
                order.coupon = cart.coupon
                order.discount = cart.coupon.discount
            order.save()
            for item in cart:
                OrderItem.objects.create(
                    order=order,
                    product=item['product'],
                    price=item['price'],
                    quantity=item['quantity']
                )
            # Wyczyść zawartość koszyka
            cart.clear()
            # Uruchomienie zadania asynchronicznego.
            order_created.delay(order.id)
            # Umieszczenie zamówienia w sesji.
            request.session['order_id'] = order.id
            # Przekierowanie do płatności.
            return redirect('payment:process')
    else:
        form = OrderCreateForm()
    return render(
        request,
        'orders/order/create.html',
        {'cart': cart, 'form': form}
    )
```

W nowym kodzie tworzymy obiekt `Order` za pomocą metody `save()` formularza `Order ↪CreateForm`. Kiedy używamy `commit=False`, nie zapisujemy go jeszcze w bazie danych. Jeżeli koszyk na zakupy zawiera kupon, przechowujemy powiązany z nim kupon oraz wysokość udzielonego rabatu. Następnie obiekt `order` zapisujemy w bazie danych.

Przeprowadź edycję szablonu *payment/process.html* aplikacji `payment` i znajdź wiersze:

```
<tr class="total">
  <td colspan="4">Razem</td>
  <td class="num">{{ order.get_total_cost }} PLN</td>
</tr>
```

Zastąp je poniższym fragmentem kodu. Nowe wiersze zostały wyróżnione pogrubioną czcionką.

```
{% if order.coupon %}
  <tr class="subtotal">
    <td>Suma częściowa</td>
    <td colspan="3"></td>
    <td class="num">
      {{ order.get_total_cost_before_discount|floatformat:2 }} zł
    </td>
```

```
    </tr>
    <tr>
      <td>
        Kupon "{{ order.coupon.code }}"
        (rabat {{ order.discount }}%)
      </td>
      <td colspan="3"></td>
      <td class="num neg">
        - {{ order.get_discount|floatformat:2 }} PLN
      </td>
    </tr>
  {% endif %}
  <tr class="total">
    <td>Razem</td>
    <td colspan="3"></td>
    <td class="num">
      {{ order.get_total_cost|floatformat:2 }} PLN
    </td>
  </tr>
```

Zaktualizowaliśmy podsumowanie zamówienia przed dokonaniem płatności.

Uruchom serwer programistyczny za pomocą poniższego polecenia.

```
python manage.py runserver
```

Upewnij się, że Docker jest uruchomiony, a następnie, aby uruchomić serwer RabbitMQ w kontenerze Dockera, wykonaj w innej powłoce polecenie:

```
docker run -it --rm --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:3.13.1-
↪management
```

Otwórz inną powłokę i z katalogu projektu uruchom wątek roboczy Celery przy użyciu polecenia:

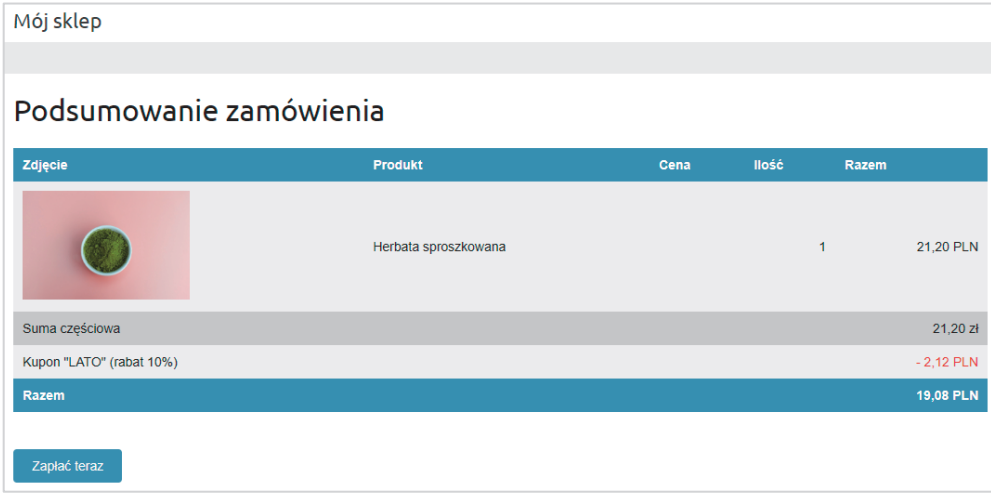
```
celery -A myshop worker -l info
```

Otwórz jeszcze jedną dodatkową powłokę i aby zainicjować przekazywanie zdarzeń Stripe do lokalnego adresu URL webhooka wykonaj polecenie:

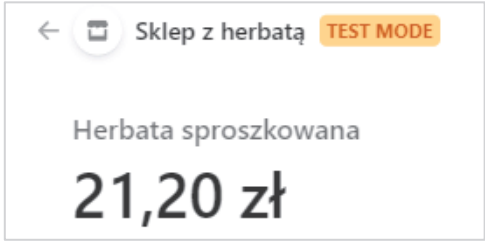
```
stripe listen --forward-to localhost:8000/payment/webhook/
```

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/` i stwórz zamówienie, używając utworzonego wcześniej kuponu. Po zweryfikowaniu poprawności pozycji w koszyku na stronie podsumowania zamówienia wyświetli się kupon zastosowany do zamówienia (rysunek 10.9).

Jeśli klikniesz *Zapłać teraz*, zauważysz, że serwis Stripe „nie wie” o zastosowanym rabacie (rysunek 10.10).



Rysunek 10.9. Podsumowanie zamówienia włącznie z zastosowanym kuponem



Rysunek 10.10. Szczegóły pozycji zamówienia na stronie Stripe Checkout nie uwzględniają kuponu rabatowego

W serwisie Stripe wyświetla się pełna kwota do zapłaty, bez żadnych potrąceń. To dlatego, że nie przekazaliśmy do serwisu Stripe informacji o rabacie. Pamiętaj, że w widoku `payment_process` przekazujemy do serwisu Stripe pozycje zamówienia jako listę `line_items` obejmującą dla każdej pozycji zamówienia koszt i ilość.

Zastosowanie kuponów w sesji Stripe Checkout

Stripe umożliwia definiowanie kuponów rabatowych i powiązanie ich z jednorazowymi płatnościami. Więcej informacji na temat tworzenia rabatów dla sesji Stripe Checkout znajdziesz pod adresem *https://stripe.com/docs/payments/checkout/discounts*.

Aby utworzyć kupon dla sesji Stripe Checkout, zmodyfikujemy widok `payment_process`. Przeprowadź edycję pliku *views.py* aplikacji `payment` i umieść w nim poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
def payment_process(request):
    order_id = request.session.get('order_id')
    order = get_object_or_404(Order, id=order_id)
```

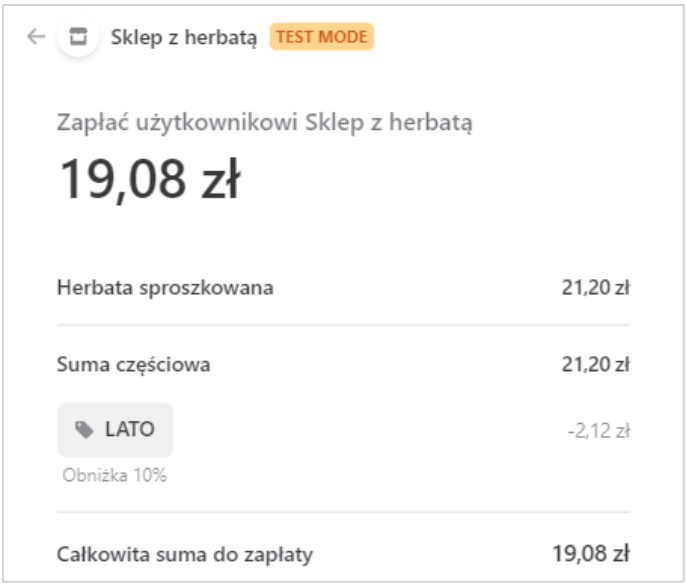
```
    if request.method == 'POST':
        success_url = request.build_absolute_uri(
            reverse('payment:completed')
        )
        cancel_url = request.build_absolute_uri(
            reverse('payment:canceled')
        )
        # Dane sesji transakcji Stripe
        session_data = {
            'mode': 'payment',
            'client_reference_id': order.id,
            'success_url': success_url,
            'cancel_url': cancel_url,
            'line_items': []
        }
        # Dodanie do sesji Stripe Checkout pozycji zamówienia
        for item in order.items.all():
            session_data['line_items'].append(
                {
                    'price_data': {
                        'unit_amount': int(item.price * Decimal('100')),
                        'currency': 'pln',
                        'product_data': {
                            'name': item.product.name,
                        },
                    },
                    'quantity': item.quantity,
                }
            )
        # Kupon serwisu Stripe
        if order.coupon:
            stripe_coupon = stripe.Coupon.create(
                name=order.coupon.code,
                percent_off=order.discount,
                duration='once'
            )
            session_data['discounts'] = [{'coupon': stripe_coupon.id}]
        # Utwórz sesję Stripe Checkout
        session = stripe.checkout.Session.create(**session_data)
        # Przekierowanie do formularza płatności Stripe
        return redirect(session.url, code=303)
    else:
        return render(request, 'payment/process.html', locals())
```

W nowym kodzie sprawdzamy, czy z zamówieniem jest powiązany kupon. W takim przypadku używamy Stripe SDK. W celu utworzenia kuponów Stripe korzystamy z wywołania `stripe.Coupon.create()`. Dla kuponu używamy następujących atrybutów.

- `name`. Kod kuponu związanego z przedmiotem zamówienia.
- `percent_off`. Rabat dla obiektu zamówienia.
- `duration`. Używamy wartości `once`. Dla serwisu Stripe oznacza ona, że jest to kupon na jednorazową płatność.

Po utworzeniu kuponu jego identyfikator `id` jest dodawany do słownika `session_data` służącego do tworzenia sesji Stripe Checkout. Identyfikator umożliwia powiązanie kuponu z sesją kasy.

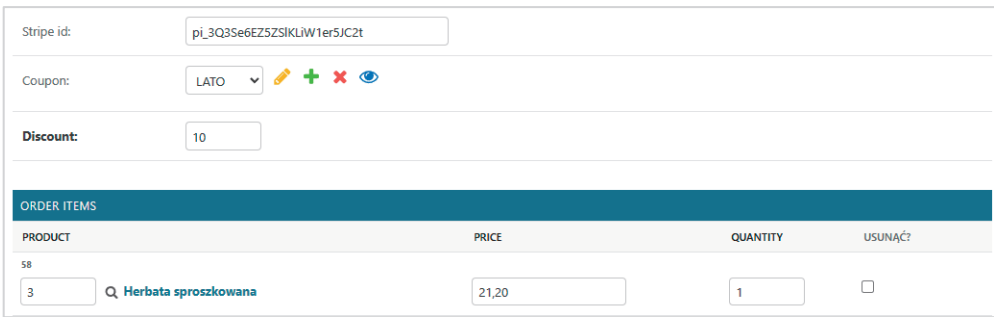
W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/` i dokończ zamówienie, używając utworzonego wcześniej kuponu. Po przekierowaniu na stronę Stripe Checkout, zauważysz zastosowany kupon (rysunek 10.11).



Rysunek 10.11. Szczegóły pozycji zamówienia na stronie Stripe Checkout z uwzględnieniem kuponu rabatowego o nazwie LATO

Strona Stripe Checkout zawiera teraz kupon zamówienia, a łączna kwota do zapłaty obejmuje teraz kwotę odliczoną za pomocą kuponu.

Dokończ zakup, a następnie otwórz w przeglądarce stronę `http://127.0.0.1:8000/admin/orders/order/`. Kliknij obiekt zamówienia, dla którego zastosowano kupon. W formularzu edycji wyświetli się zastosowany rabat, jak pokazano na rysunku 10.12.



Rysunek 10.12. Formularz edycji zamówienia włącznie z kuponem i zastosowanym rabatem

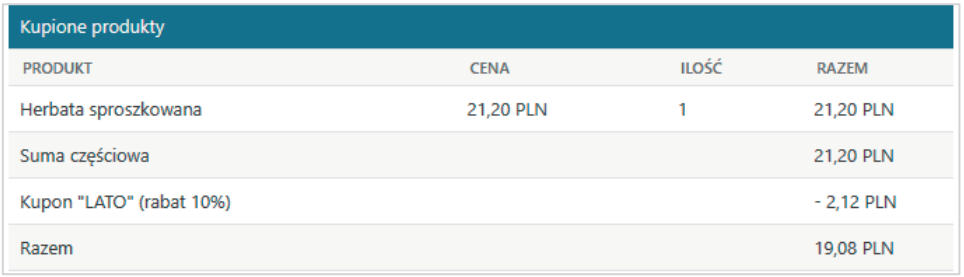
Z powodzeniem przechowujesz kupony do zamówień i przetwarzasz płatności z rabatami. W kolejnym kroku dodamy kupony do widoku szczegółów zamówienia na stronie administracyjnej oraz do faktur PDF za zamówienia.

Dodawanie kuponów do zamówień w serwisie administracyjnym oraz do faktur w formacie PDF

Dodajmy kupon do strony szczegółów zamówienia na stronie administracyjnej. Przeprowadź edycję szablonu *admin/orders/order/detail.html* aplikacji `orders` i dodaj poniższy fragment kodu HTML wyróżniony pogrubioną czcionką.

```
...
<table style="width:100%">
   ...
  <tbody>
    {% for item in order.items.all %}
      <tr class="row{% cycle "1" "2" %}">
        <td>{{ item.product.name }}</td>
        <td class="num">{{ item.price }} PLN</td>
        <td class="num">{{ item.quantity }}</td>
        <td class="num">{{ item.get_cost }} PLN</td>
      </tr>
    {% endfor %}
    {% if order.coupon %}
      <tr class="subtotal">
        <td colspan="3">Suma częściowa</td>
        <td class="num">
          {{ order.get_total_cost_before_discount|floatformat:2 }} PLN
        </td>
      </tr>
      <tr>
        <td colspan="3">
          Kupon "{{ order.coupon.code }}"
          (rabat {{ order.discount }}%)
        </td>
        <td class="num neg">
          - {{ order.get_discount|floatformat:2 }} PLN
        </td>
      </tr>
    {% endif %}

    <tr class="total">
      <td colspan="3">Razem</td>
      <td class="num">
        {{ order.get_total_cost|floatformat:2 }} PLN
      </td>
    </tr>
  </tbody>
</table>
...
```

Następnie otwórz w przeglądarce stronę `http://127.0.0.1:8000/admin/orders/order/`, po czym kliknij link *Wyświetl* odpowiadający najnowszemu zamówieniu. Tabela *Kupione przedmioty* będzie teraz zawierała wykorzystany kupon, jak pokazano na rysunku 10.13:



Kupione produkty

PRODUKT	CENA	ILOŚĆ	RAZEM
Herbata sproszkowana	21,20 PLN	1	21,20 PLN
Suma częściowa			21,20 PLN
Kupon "LATO" (rabat 10%)			- 2,12 PLN
Razem			19,08 PLN

Rysunek 10.13. Strona szczegółów produktu w witrynie administracyjnej z uwzględnieniem wykorzystanego kuponu

Teraz zmodyfikujmy szablon faktury zamówienia, aby zawierał kupon wykorzystany do zamówienia. Przeprowadź edycję szablonu *orders/order/pdf.html* aplikacji `orders` i dodaj poniższy fragment kodu HTML wyróżniony pogrubioną czcionką.

```
...
<table>
  <thead>
    <tr>
      <th>Produkt</th>
      <th>Cena</th>
      <th>Ilość</th>
      <th>Wartość</th>
    </tr>
  </thead>
  <tbody>
    {% for item in order.items.all %}
      <tr class="row{% cycle "1" "2" %}">
        <td>{{ item.product.name }}</td>
        <td class="num">{{ item.price }} PLN</td>
        <td class="num">{{ item.quantity }}</td>
        <td class="num">{{ item.get_cost }} PLN</td>
      </tr>
    {% endfor %}

    {% if order.coupon %}
      <tr class="subtotal">
        <td colspan="3">Suma częściowa</td>
        <td class="num">
          {{ order.get_total_cost_before_discount|floatformat:2 }} PLN
        </td>
      </tr>
      <tr>
        <td colspan="3">
          Kupon "{{ order.coupon.code }}"
          (rabat {{ order.discount }}%)
        </td>
```

```
            <td class="num neg">
              - {{ order.get_discount|floatformat:2 }} PLN
            </td>
          </tr>
        {% endif %}

        <tr class="total">
          <td colspan="3">Razem</td>
          <td class="num">{{ order.get_total_cost|floatformat:2 }} PLN</td>
        </tr>
      </tbody>
    </table>
    ...
```

Następnie otwórz w przeglądarce stronę `http://127.0.0.1:8000/admin/orders/order/`, po czym kliknij link *PDF* odpowiadający najnowszemu zamówieniu. Tabela *Kupione przedmioty* będzie teraz zawierała wykorzystany kupon, jak pokazano na rysunku 10.14.

Rysunek 10.14. Faktura zamówienia w formacie PDF wraz z wykorzystanym kuponem

Pomyślnie dodałeś do swojego sklepu system kuponów. W następnym podrozdziale zajmiemy się implementacją w budowanym sklepie internetowym silnika rekomendacji produktów.

Utworzenie silnika rekomendacji produktów

Silnik rekomendacji to system przewidujący preferencje lub ocenę, jaką użytkownik dałby danemu produktowi. System wybiera określone produkty dla użytkownika na podstawie jego zachowania i zebranych o nim informacji. Obecnie systemy rekomendacji są używane w wielu serwisach i usługach internetowych. Pomagają użytkownikom w wyborze produktów, które mogą ich zainteresować, spośród ogromnej ilości nieinteresujących ich produktów. Zaoferowanie dobrych rekomendacji zwiększa szansę zakupu przez użytkownika. Ponadto witryny typu e-commerce również korzystają z rekomendacji produktów, ponieważ każdy zakup wykonany przez użytkownika zwiększa ich zyski.

W tym podrozdziale zbudujemy dość prosty, choć jednocześnie skuteczny silnik rekomendacji sugerujący produkty, które były najczęściej kupowane razem. Silnik zasugeruje produkty na podstawie wcześniejszych transakcji, a więc będzie identyfikował produkty, które były już kupowane razem. Rekomendacja produktów będzie się odbywać w dwóch różnych scenariuszach.

- **Strona szczegółów dotyczących produktu.** Wyświetlimy listę produktów, które zwykle były kupowane z danym produktem. Będzie to miało postać komunikatu, takiego jak „Użytkownicy, którzy kupili ten produkt, kupowali również X, Y, Z". Potrzebna będzie struktura danych pozwalająca na przechowywanie informacji o tym, ile razy dany produkt był kupowany wraz z wyświetlanym produktem.
- **Strona szczegółów koszyka na zakupy.** Na podstawie produktów umieszczonych przez użytkownika w koszyku na zakupy zasugerujemy mu produkty, które najczęściej były kupowane wraz z wybranymi przez kupującego. W takim przypadku konieczne jest zagregowanie punktów w celu ustalenia powiązanych produktów.

Do przechowywania produktów kupowanych razem wykorzystamy bazę danych Redis. Jak pewnie pamiętasz, z tej bazy danych korzystaliśmy już w rozdziale 7., „Śledzenie działań użytkownika". We wspomnianym rozdziale znajdziesz informacje dotyczące procedury instalacji bazy Redis, jeżeli jeszcze jej nie zainstalowałeś.

Rekomendacja produktów na podstawie wcześniejszych transakcji

Będziemy polecać użytkownikom produkty na podstawie towarów, które często są kupowane razem. Do tego celu wykorzystamy posortowane zbiory danych Redis. Jak pamiętasz, posortowanych zbiorów danych użyłeś w rozdziale 7., „Śledzenie działań użytkownika", aby utworzyć ranking najczęściej oglądanych obrazów w witrynie.

Reprezentację posortowanego zbioru danych, w którym elementy zbioru są ciągami powiązanymi z wynikiem, przedstawiłem na rysunku 10.15.

Rysunek 10.15. Reprezentacja posortowanego zbioru danych Redis

W bazie danych Redis będziemy przechowywać klucz dla każdego produktu kupionego w witrynie internetowej. Klucz produktu będzie zawierał kolekcję Redis posortowaną według punktów. Liczba punktów dla danego produktu będzie inkrementowana o 1 po każdym zakupie. Posortowany zbiór pozwoli Ci wystawić punkty produktom, które są kupowane razem. W roli punktacji określonego przedmiotu wykorzystamy liczbę zakupów produktu razem z innym produktem.

Najpierw zainstaluj moduł `redis-py` w swoim środowisku za pomocą poleceni:.

```
python -m pip install redis==5.0.4
```

Przeprowadź edycję pliku *settings.py* projektu i dodaj do niego poniższe wiersze.

```
# Ustawienia Redis
REDIS_HOST = 'localhost'
REDIS_PORT = 6379
REDIS_DB = 1
```

Są to ustawienia wymagane do nawiązania połączenia z serwerem Redis. W katalogu aplikacji `shop` utwórz nowy plik o nazwie *recommender.py*. Następnie w pliku umieść poniższy fragment kodu.

```
import redis
from django.conf import settings
from .models import Product


# Nawiązanie połączenia z bazą danych Redis
r = redis.Redis(
    host=settings.REDIS_HOST,
    port=settings.REDIS_PORT,
    db=settings.REDIS_DB
)

class Recommender:
    def get_product_key(self, id):
        return f'product:{id}:purchased_with'

    def products_bought(self, products):
        product_ids = [p.id for p in products]
        for product_id in product_ids:
            for with_id in product_ids:
                # Pobranie innych produktów kupionych razem z analizowanymi.
                if product_id != with_id:
                    # Inkrementacja punktacji dla produktu kupionego razem z analizowanym.
                    r.zincrby(
                        self.get_product_key(product_id), 1, with_id
                    )
```

To jest klasa `Recommender` pozwalająca na przechowywanie kupionych produktów oraz pobieranie sugestii dla danego produktu lub produktów.

Metoda `get_product_key()` otrzymuje identyfikator obiektu `Product` i tworzy klucz Redis dla posortowanej kolekcji zawierającej powiązane produkty. Struktura ta ma postać taką, jak `product:[id]:purchased_with`.

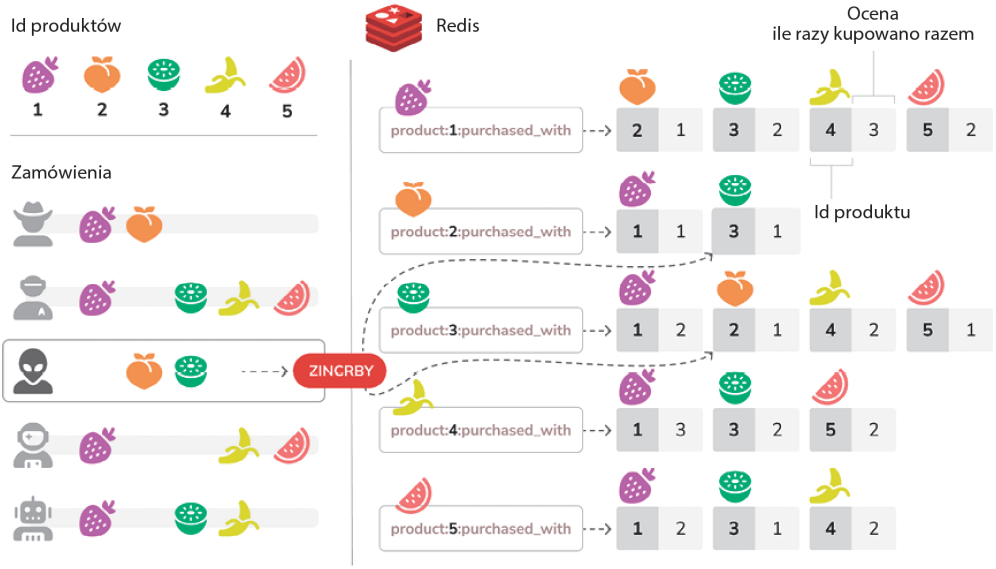
Metoda `products_bought()` otrzymuje listę obiektów `Product`, które zostały kupione razem (tzn. należą do tego samego zamówienia).

W tej metodzie są wykonywane następujące zadania.

1. Pobranie identyfikatorów produktów dla danych obiektów `Product`.

2. Iteracja przez identyfikatory produktów. Dla każdego identyfikatora przeprowadzamy iterację przez identyfikatory produktów i pomijamy ten sam produkt, aby w ten sposób otrzymać listę produktów kupionych razem z analizowanym.
3. Za pomocą metody `get_product_id()` pobieramy klucz produktu w bazie danych Redis. W przypadku produktu o identyfikatorze `33` ta metoda zwraca klucz `product:33:purchased_with`. Jest to klucz dla posortowanej kolekcji zawierającej identyfikatory produktów kupionych razem z analizowanym.
4. Za pomocą operacji Redis `ZINCRBY` inkrementujemy o 1 punktację każdego produktu o identyfikatorze znajdującym się w posortowanej kolekcji. Punktacja pokazuje, ile razy inny produkt został kupiony razem z analizowanym.

Przykład pięciu różnych produktów o identyfikatorach od 1 do 5 i pięciu zamówień zakupu różnych kombinacji produktów przedstawiłem na rysunku 10.16.



Rysunek 10.16. Pięć produktów z ich identyfikatorami i kombinacjami zakupów

Na rysunku można zobaczyć posortowany zbiór danych utworzony w bazie danych Redis dla każdego produktu, z kluczem `product:<id>:purchased_with`, gdzie `<id>` jest unikatowym identyfikatorem produktu. Posortowane elementy zbioru danych to identyfikatory produktów, które zostały kupione wraz z produktem głównym. Ocena punktowa dla każdego elementu zbioru odzwierciedla łączną liczbę wspólnych zakupów. Rysunek przedstawia operację `ZINCRBY` bazy danych Redis zwiększającą o 1 punktację produktów kupionych razem w jednym zamówieniu.

W ten sposób przygotowaliśmy metodę przeznaczoną do przechowywania i oceny produktów kupowanych razem. Teraz potrzebujemy metody pobierającej produkty kupowane razem dla listy danych produktów. Dodaj do klasy `Recommender` poniższą metodę `suggest_products_for()`.

```
def suggest_products_for(self, products, max_results=6):
    product_ids = [p.id for p in products]
    if len(products) == 1:
        # Tylko jeden produkt.
        suggestions = r.zrange(
            self.get_product_key(product_ids[0]), 0, -1, desc=True
        )[:max_results]
    else:
        # Wygenerowanie tymczasowego klucza.
        flat_ids = ''.join([str(id) for id in product_ids])
        tmp_key = f'tmp_{flat_ids}'
        # Wiele produktów, sumowanie punktów wszystkich produktów,
        # umieszczenie w kluczu tymczasowym posortowanej kolekcji wynikowej.
        keys = [self.get_product_key(id) for id in product_ids]
        r.zunionstore(tmp_key, keys)
        # Usunięcie identyfikatorów produktów, dla których przygotowujemy rekomendacje.
        r.zrem(tmp_key, *product_ids)
        # Pobranie identyfikatorów produktów według ich punktacji, w kolejności malejącej.
        suggestions = r.zrange(
            tmp_key, 0, -1, desc=True
        )[:max_results]
        # Usunięcie tymczasowego klucza
        r.delete(tmp_key)
    suggested_products_ids = [int(id) for id in suggestions]
    # Pobranie sugerowanych produktów i ułożenie ich w kolejności pojawienia się.
    suggested_products = list(
        Product.objects.filter(id__in=suggested_products_ids)
    )
    suggested_products.sort(
        key=lambda x: suggested_products_ids.index(x.id)
    )
    return suggested_products
```

Metoda `suggest_products_for()` otrzymuje poniższe parametry.

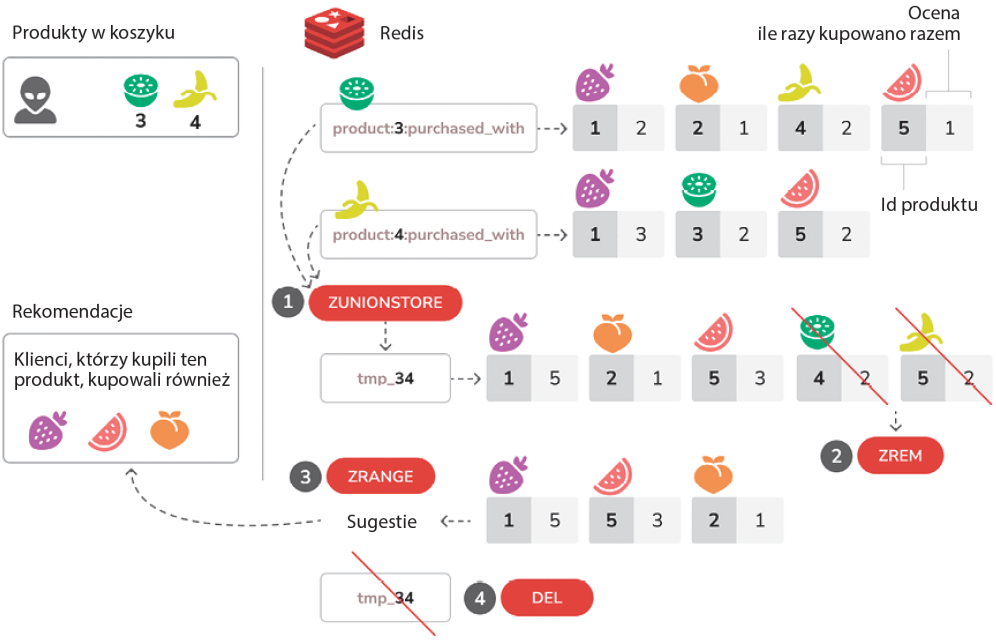
- `products`. Lista obiektów `Product`, dla których potrzebne są rekomendacje. Na liście może znajdować się więcej niż tylko jeden produkt.
- `max_results`. Liczba całkowita przedstawiająca maksymalną liczbę rekomendacji, które mają zostać zwrócone.

W wymienionej metodzie są wykonywane następujące zadania.

1. Pobranie identyfikatorów produktów dla danych obiektów `Product`.
2. Jeżeli na liście znajduje się tylko jeden produkt, pobieramy identyfikatory produktów kupowanych razem z nim, ułożone w kolejności liczby przypadków, kiedy oba produkty były zakupione razem. W tym celu wykorzystujemy polecenie `ZRANGE` bazy danych Redis. Ograniczamy liczbę wyników do wartości wskazanej przez atrybut `max_results` (domyślnie to `6`). Więcej informacji na temat polecenia `ZRANGE` można znaleźć na stronie *https://redis.io/commands/zrange/*.
3. Jeżeli lista zawiera więcej niż tylko jeden produkt, generujemy tymczasowy klucz Redis utworzony z identyfikatorów tych produktów.

4. Dla każdego z analizowanych produktów łączymy i sumujemy wszystkie punkty dla produktów znajdujących się w posortowanej kolekcji. W tym celu wykorzystujemy polecenie `ZUNIONSTORE` bazy danych Redis. Polecenie tworzy unię posortowanych kolekcji o danym kluczu i przechowuje w nowym kluczu Redis zagregowane sumy punktów elementów. Więcej informacji na temat polecenia `ZUNIONSTORE` można znaleźć na stronie *https://redis.io/commands/zunionstore/*. Zagregowane wyniki zapisujemy w kluczu tymczasowym.
5. Ponieważ agregujemy punktację, możemy otrzymać te same produkty, dla których pobieramy rekomendacje. Za pomocą polecenia `ZREM` usuwamy więc je z wygenerowanej posortowanej kolekcji. Więcej informacji na temat polecenia `ZREM` można znaleźć na stronie *https://redis.io/commands/zrem/*.
6. Pobieramy identyfikatory produktów z klucza tymczasowego, posortowane według punktów za pomocą polecenia `ZRANGE`. Ograniczamy liczbę wyników do wartości wskazanej przez atrybut `max_results`.
7. Następnie usuwamy klucz tymczasowy za pomocą metody `delete()` modułu `redis-py`, która wykonuje polecenie Redis `DEL`. Więcej informacji na temat polecenia `DEL` można znaleźć na stronie *https://redis.io/commands/del/*.
8. Na koniec pobieramy obiekty `Product` o wskazanych identyfikatorach i układamy produkty w takiej samej kolejności jak identyfikatory.

Przykład sesji, w której zostały dodane do koszyka dwa produkty i zostały wykonane operacje Redis w celu uzyskania rekomendacji powiązanych produktów, przedstawiłem na rysunku 10.17.



Rysunek 10.17. System rekomendacji produktów

Na rysunku można zobaczyć cztery kroki generowania rekomendacji produktów dla produktów znajdujących się w koszyku:

1. Do agregowania punktacji często kupowanych produktów z produktami w koszyku wykorzystano polecenie Redis `ZUNIONSTORE`. Wynikowy posortowany zbiór danych tej operacji jest przechowywany w nowym kluczu Redis o nazwie odpowiadającej identyfikatorom produktów w koszyku: w przypadku identyfikatorów `3` i `4` nazwa klucza to `tmp_34`.
2. Do usuwania kupowanych produktów z posortowanego zestawu, aby uniknąć polecania produktów, które już znajdują się w koszyku, służy polecenie `ZREM`.
3. W celu zwrócenia posortowanych elementów zbioru `tmp_34` uporządkowanych według punktacji wykorzystujemy polecenie `ZRANGE`.
4. Na koniec w celu usunięcia klucza Redis `tmp_34` używamy polecenia `DEL`.

Z powodów praktycznych zdefiniujemy jeszcze metodę usuwającą rekomendacje. Dodaj do klasy `Recommender` poniższą metodę.

```
def clear_purchases(self):
    for id in Product.objects.values_list('id', flat=True):
        r.delete(self.get_product_key(id))
```

Wypróbujmy teraz przygotowany silnik rekomendacji. Upewnij się, że w bazie danych umieściłeś wiele obiektów `Product`, i zainicjuj serwer Redis za pomocą poniższego polecenia wydanego z poziomu powłoki.

```
docker run -it --rm --name redis -p 6379:6379 redis:7.2.4
```

Przejdź do kolejnego okna powłoki i wydaj poniższe polecenie, aby otworzyć powłokę Pythona.

```
python manage.py shell
```

Zadbaj o to, aby w bazie danych znalazły się co najmniej cztery produkty. Spróbuj pobrać cztery różne produkty na podstawie nazwy.

```
>>> from shop.models import Product
>>> czarna_herbata = Product.objects.get(name='Czarna herbata')
>>> czerwona_herbata = Product.objects.get(name='Czerwona herbata')
>>> zielona_herbata = Product.objects.get(name='Zielona herbata')
>>> sproszkowana_herbata = Product.objects.get(name='Sproszkowana herbata')
```

Teraz dodajemy kilka przykładowych zakupów dla silnika rekomendacji.

```
>>> from shop.recommender import Recommender
>>> r = Recommender()
>>> r.products_bought([czarna_herbata, czerwona_herbata])
>>> r.products_bought([czarna_herbata, zielona_herbata])
>>> r.products_bought([czerwona_herbata, czarna_herbata, sproszkowana_herbata])
>>> r.products_bought([zielona_herbata, sproszkowana_herbata])
>>> r.products_bought([czarna_herbata, sproszkowana_herbata])
>>> r.products_bought([czerwona_herbata, zielona_herbata])
```

W bazie danych przechowujemy następującą punktację.

```
czarna_herbata:  czerwona_herbata (2), sproszkowana_herbata (2), zielona_herbata (1)
czerwona_herbata:    czarna_herbata (2), sproszkowana_herbata (1), zielona_herbata (1)
zielona_herbata:  czarna_herbata (1), sproszkowana_herbata (1), czerwona_herbata (1)
sproszkowana_herbata: czarna_herbata (2), czerwona_herbata (1), zielona_herbata (1)
```

Jest to reprezentacja produktów, które zostały kupione razem z każdym z produktów, włącznie z liczbą razy, kiedy zostały one zakupione razem.

Pobierzmy rekomendacje produktowe dla pojedynczego produktu:

```
>>> r.suggest_products_for([czarna_herbata])
[<Product: Sproszkowana herbata>, <Product: Czerwona herbata>, <Product: Zielona
↪herbata>]
>>> r.suggest_products_for([czerwona_herbata])
[<Product: Czarna herbata>, <Product: Sproszkowana herbata>, <Product: Zielona
↪herbata>]
>>> r.suggest_products_for([zielona_herbata])
[<Product: Czarna herbata>, <Product: Sproszkowana herbata>, <Product: Czerwona
↪herbata>]
>>> r.suggest_products_for([sproszkowana_herbata])
[<Product: Czarna herbata>, <Product: Czerwona herbata>, <Product: Zielona herbata>]
```

Jak możesz zobaczyć, polecanie produktów odbywa się na podstawie ich oceny punktowej. Przechodzimy teraz do rekomendacji dla wielu produktów za pomocą zagregowanych wyników.

```
>>> r.suggest_products_for([czarna_herbata, czerwona_herbata])
[<Product: Sproszkowana herbata>, <Product: Zielona herbata>]
>>> r.suggest_products_for([zielona_herbata, czerwona_herbata])
[<Product: Czarna herbata>, <Product: Sproszkowana herbata>]
>>> r.suggest_products_for([sproszkowana_herbata, czarna_herbata])
[<Product: Czerwona herbata>, <Product: Zielona herbata>]
```

Jak możesz zobaczyć, kolejność sugerowanych produktów odpowiada zagregowanym wynikom, np. produktami sugerowanymi dla `czarna_herbata` i `czerwona_herbata` są `sproszkowana_herbata` (2+1) i `zielona_herbata` (1+1).

Potwierdziliśmy tym samym, że algorytm rekomendacji działa zgodnie z oczekiwaniami.

Przystępujemy więc do implementacji mechanizmu zapisywania za każdym razem, gdy dokonywana jest płatność, produktów, które są kupowane łącznie. W pliku *webhooks.py* aplikacji `payment` umieść poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
# ...
from shop.models import Product
from shop.recommender import Recommender


@csrf_exempt
def stripe_webhook(request):
    # ...
    if event.type == 'checkout.session.completed':
        session = event.data.object
        if(
            session.mode == 'payment'
            and session.payment_status == 'paid'
```

```
        ):
            try:
                order = Order.objects.get(
                    id=session.client_reference_id
                )
            except Order.DoesNotExist:
                return HttpResponse(status=404)

            # Oznaczenie zamówienia jako zapłaconego
            order.paid = True

            # Zapisanie identyfikatora płatności Stripe
            order.stripe_id = session.payment_intent
            order.save()

            # Zapisanie kupionych towarów na potrzeby rekomendacji
            product_ids = order.items.values_list('product_id')
            products = Product.objects.filter(id__in=product_ids)
            r = Recommender()
            r.products_bought(products)

            # Uruchomienie zadania asynchronicznego
            payment_completed.delay(order.id)

    return HttpResponse(status=200)
```

W nowym kodzie po potwierdzeniu płatności za nowe zamówienie pobierane są obiekty `Product` powiązane z pozycjami zamówienia. Następnie tworzymy egzemplarz klasy `Recommender` i wywołujemy metodę `products_bought()`, która przechowuje w bazie danych Redis produkty kupione razem.

Teraz zapisujemy powiązane produkty, które są kupowane razem, przy okazji opłacania zamówień. Przystępujemy zatem do zaimplementowania wyświetlania rekomendacji dla produktów kupowanych w budowanym sklepie internetowym.

Przeprowadź edycję pliku *views.py* aplikacji `shop`. Dodaj funkcjonalność pobierania maksymalnie czterech polecanych produktów w widoku `product_detail`, tak jak pokazałem poniżej.

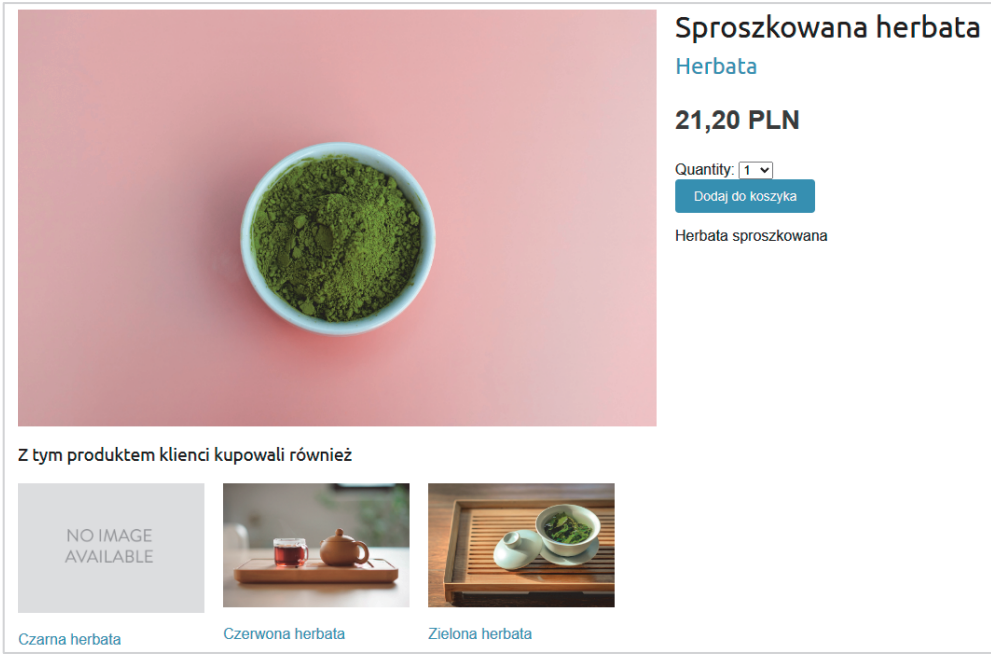
```
from .recommender import Recommender


def product_detail(request, id, slug):
    product = get_object_or_404(
        Product, id=id, slug=slug, available=True
    )
    cart_product_form = CartAddProductForm()
    r = Recommender()
    recommended_products = r.suggest_products_for([product], 4)
    return render(
        request,
        'shop/product/detail.html',
        {
            'product': product,
```

```
            'cart_product_form': cart_product_form,
            'recommended_products': recommended_products
        }
    )
```

Kolejnym krokiem jest przeprowadzenie edycji szablonu *shop/product/detail.html* aplikacji `shop` i umieszczenie poniższego fragmentu kodu po znaczniku `{{product.description ↪|linebreaks}}`.

```
{% if recommended_products %}
  <div class="recommendations">
    <h3>Z tym produktem klienci kupowali również</h3>
    {% for p in recommended_products %}
      <div class="item">
        <a href="{{ p.get_absolute_url }}">
          <img src="{% if p.image %}{{ p.image.url }}{% else %}
          {% static  "img/no_image.png" %}{% endif %}">
        </a>
        <p><a href="{{ p.get_absolute_url }}">{{ p.name }}</a></p>
      </div>
    {% endfor %}
  </div>
{% endif %}
```

Uruchom serwer programistyczny, a później w przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/`. Kliknij dowolny produkt, aby przejść na stronę wyświetlającą szczegółowe informacje na jego temat. Powinieneś zobaczyć, że rekomendowane produkty są wyświetlane na dole strony, co pokazałem na rysunku 10.18.



Rysunek 10.18. Strona szczegółów produktu włącznie z produktami rekomendowanymi

Uwaga

Zdjęcia wykorzystane w tym rozdziale:

- *Zielona herbata:* zdjęcie autorstwa Jia Ye pochodzące z serwisu Unsplash.
- *Czerwona herbata:* Zdjęcie autorstwa Manki Kim pochodzące z serwisu Unsplash.
- *Herbata sproszkowana:* zdjęcie autorstwa Phuong Nguyen pochodzące z serwisu Unsplash.

Produkty rekomendowane będziemy wyświetlać także w koszyku na zakupy. W tym przypadku wybór produktów odbywa się na podstawie zakupów umieszczonych w koszyku.

Przeprowadź edycję pliku *views.py* w katalogu aplikacji `cart`, zaimportuj klasę `Recommender` i zmodyfikuj widok `cart_detail` tak, aby przyjął poniższą postać.

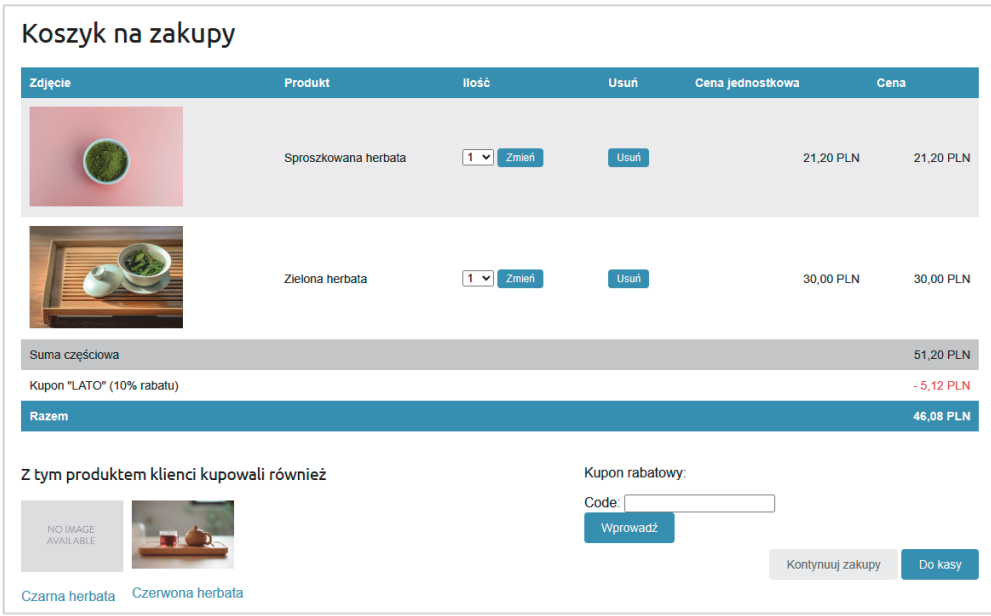
```
from shop.recommender import Recommender

def cart_detail(request):
    cart = Cart(request)
    for item in cart:
        item['update_quantity_form'] = CartAddProductForm(
            initial={'quantity': item['quantity'], 'override': True}
        )
    coupon_apply_form = CouponApplyForm()

    r = Recommender()
    cart_products = [item['product'] for item in cart]
    if(cart_products):
        recommended_products = r.suggest_products_for(
            cart_products, max_results=4
        )
    else:
        recommended_products = []
    return render(
         request,
        'cart/detail.html',
        {
            'cart': cart,
            'coupon_apply_form': coupon_apply_form,
            'recommended_products': recommended_products
        }
    )
```

Teraz przeprowadź edycję szablonu *cart/detail.html* aplikacji `cart` i umieść w nim poniższy fragment kodu, tuż po znaczniku HTML `</table>`.

```
{% if recommended_products %}
  <div class="recommendations cart">
    <h3>Z tym produktem klienci kupowali również</h3>
    {% for p in recommended_products %}
      <div class="item">
        <a href="{{ p.get_absolute_url }}">
          <img src="{% if p.image %}{{ p.image.url }}{% else %}
```

```
            {% static "img/no_image.png" %}{% endif %}">
          </a>
          <p><a href="{{ p.get_absolute_url }}">{{ p.name }}</a></p>
        </div>
      {% endfor %}
    </div>
  {% endif %}
```

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000` i dodaj kilka produktów do koszyka na zakupy. Po przejściu na stronę `http://127.0.0.1:8000/cart/` powinieneś zobaczyć rekomendowane produkty na podstawie zagregowanych wyników obliczonych dla produktów znajdujących się w koszyku na zakupy (rysunek 10.19).



Rysunek 10.19. Strona szczegółów koszyka na zakupy włącznie z produktami rekomendowanymi

Gratulacje! Za pomocą frameworka Django i bazy danych Redis zaimplementowałeś pełny silnik rekomendacji produktu.

Podsumowanie

W tym rozdziale utworzyłeś system kuponów działający w oparciu o mechanizm sesji frameworka Django i zintegrowałeś go z serwisem Stripe. Na koniec zbudowałeś system rekomendacji produktu działający w oparciu o bazę danych, żeby polecać produkty, które zwykle są kupowane razem.

W kolejnym rozdziale zajmiemy się tematem internacjonalizacji i lokalizacji projektów Django. Nauczysz się tłumaczyć kod i zarządzać tłumaczeniami za pomocą systemu Rosetta.

Zaimplementujesz adresy URL do tłumaczeń i zbudujesz selektor języka. Zaimplementujesz również tłumaczenia modeli za pomocą modułu `django-parler` oraz nauczysz się walidować zlokalizowane pola formularzy za pomocą modułu `django-localflavor`.

Dodatkowe zasoby

Oto lista zasobów zawierających dodatkowe informacje związane z tematami omawianymi w tym rozdziale:

- Kod źródłowy przykładów zamieszczonych w tym rozdziale — *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter10*.
- Obsługa rabatów w Stripe Checkout — *https://stripe.com/docs/payments/checkout/discounts.*
- Polecenie bazy danych Redis `ZRANGE` — *https://redis.io/commands/zrange/*.
- Polecenie bazy danych Redis `ZUNIONSTORE` — *https://redis.io/commands/zunionstore/*.
- Polecenie bazy danych Redis `ZREM` — *https://redis.io/commands/zrem/*.
- Polecenie bazy danych Redis `DEL` — *https://redis.io/commands/del/*.

Rozdział 11

Internacjonalizacja sklepu internetowego

W poprzednim rozdziale dodałeś do sklepu internetowego system kuponów i stworzyłeś silnik rekomendacji produktów.

W tym rozdziale dowiesz się, jak działa internacjonalizacja i lokalizacja. Udostępnienie aplikacji w wielu językach pozwala obsługiwać szersze grono użytkowników. Dodatkowo dostosowanie aplikacji do lokalnych konwencji formatowania, np. dat lub liczb, poprawia wygodę korzystania z aplikacji. Dzięki przetłumaczeniu i zlokalizowaniu Twojej aplikacji stanie się ona bardziej intuicyjna dla użytkowników z różnych środowisk kulturowych.

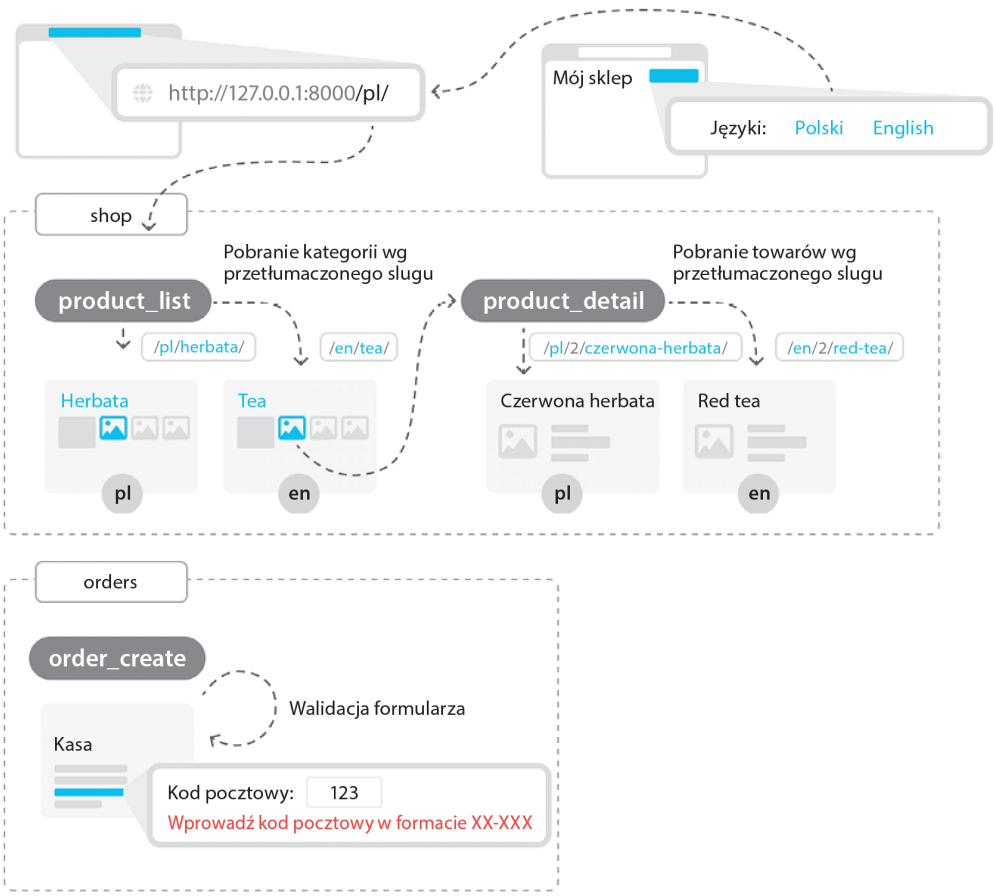
Oto zagadnienia, na których skoncentruję się w tym rozdziale:

- Przygotowanie projektu do internacjonalizacji
- Zarządzanie plikami tłumaczeń
- Tłumaczenie kodu Pythona
- Tłumaczenie szablonów
- Użycie systemu Rosetta do zarządzania tłumaczeniami
- Tłumaczenie wzorców adresów URL i używanie przedrostków językowych w adresach URL
- Umożliwienie użytkownikowi zmiany języka
- Tłumaczenie modeli za pomocą modułu `django-parler`
- Używanie tłumaczeń z mechanizmem ORM
- Dostosowanie widoków do korzystania z tłumaczeń
- Używanie zlokalizowanych pól formularzy dla modułu `django-localflavor`

Przegląd funkcjonalności

Reprezentację widoków, szablonów i funkcjonalności, które zbudujemy w tym rozdziale, przedstawiłem na rysunku 11.1.

W tym rozdziale zaimplementujesz internacjonalizację projektu sklepu internetowego. Przetłumaczysz szablony, adresy URL i modele. Dodasz do nagłówka witryny linki wyboru języka i utworzysz specyficzne dla języka adresy URL. Zmodyfikujesz widoki `product_list` i `product_detail` aplikacji `shop`, co pozwoli pobierać obiekty `Category` i `Product` według przetłumaczonych wartości pola `slug`. Ponadto do formularza używanego w widoku `order_create` zostanie dodany zlokalizowany kod pocztowy.



Rysunek 11.1. Schemat funkcjonalności zbudowanych w rozdziale 11.

Kod źródłowy przykładów z tego rozdziału można znaleźć pod adresem *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter11*.

Wszystkie pakiety Pythona użyte w tym rozdziale są zawarte w pliku *requirements.txt* dołączonym do plików z kodem źródłowym przykładów z tego rozdziału. Aby zainstalować każdy pakiet Pythona, możesz postępować zgodnie z instrukcjami zamieszczonymi w poniższych podrozdziałach. Możesz także zainstalować wszystkie wymagania jednocześnie za pomocą polecenia `pip install -r requirements.txt`.

Internacjonalizacja za pomocą Django

Django oferuje kompletną obsługę internacjonalizacji i lokalizacji projektu. Pozwala przetłumaczyć aplikację na wiele języków i zapewnia obsługę charakterystycznego dla danego języka sposobu formatowania daty, godziny, liczb i strefy czasowej. Zacznę od wyjaśnienia różnicy między internacjonalizacją a lokalizacją.

- **Internacjonalizacja** (często określana skrótem **i18n**) to proces dostosowania oprogramowania do potencjalnego użycia różnych języków i lokalizacji, więc nie jest na stałe powiązana z określonym językiem bądź lokalizacją.
- **Lokalizacja** (często określana skrótem **l10n**) to proces rzeczywistego przetłumaczenia oprogramowania i dostosowania go do danej lokalizacji. Framework Django został przetłumaczony na ponad pięćdziesiąt języków przy użyciu oferowanego frameworka internacjonalizacji.

Framework internacjonalizacji pozwala na łatwe oznaczenie ciągów tekstowych do przetłumaczenia zarówno w kodzie Pythona, jak i w szablonach. Działanie frameworka opiera się na narzędziach GNU `gettext` do generowania plików typu *message file* i zarządzania nimi. Plik typu *message file* to zwykły plik tekstowy reprezentujący język. Zawiera część lub wszystkie możliwe do przełożenia ciągi tekstowe w aplikacji oraz ich odpowiednie tłumaczenie dla pojedynczego języka. Pliki typu *message file* mają rozszerzenie *.po*. Po zakończeniu tłumaczenia pliki typu *message file* są kompilowane w celu zapewnienia szybkiego dostępu do przetłumaczonych ciągów tekstowych. Skompilowane pliki z tłumaczeniami mają rozszerzenie *.mo*.

Przejrzyjmy ustawienia zapewniające możliwość internacjonalizacji i lokalizacji dostępne we frameworku Django.

Ustawienia internacjonalizacji i lokalizacji

Django oferuje kilka ustawień dla internacjonalizacji. Poniżej wymieniłem jedynie najważniejsze.

- `USE_I18N`. Wartość typu Boolean określająca, czy system tłumaczeń w Django został włączony. Domyślnie `True`.
- `USE_TZ`. Wartość typu Boolean określająca, czy data i godzina uwzględniają strefę czasową. Po utworzeniu projektu za pomocą polecenia `startproject`, opcja ma przypisaną wartość `True`.
- `LANGUAGE_CODE`. Domyślny kod języka dla projektu. Tutaj jest stosowany standardowy format identyfikatora języka, np. `'en-us'` dla języka angielskiego używanego w USA, `'en-gb'` dla języka angielskiego używanego w Wielkiej Brytanii lub `'pl-pl'` dla języka polskiego. Ta opcja wymaga przypisania wartości `True` opcji `USE_I18N`. Listę poprawnych identyfikatorów języków znajdziesz na stronie *http://www.i18nguy.com/unicode/language-identifiers.html*.
- `LANGUAGES`. Krotka, w której wymieniono języki dostępne dla projektu. Krotka zawiera składające się z dwóch elementów krotki kodu języka i nazwy języka. Lista dostępnych języków znajduje się w ustawieniu `django.conf.global_settings`. Kiedy wybierzesz języki dla witryny internetowej, opcja `LANGUAGES` będzie zawierała podzbiór tej listy.
- `LOCALE_PATHS`. Lista katalogów, w których framework Django będzie szukał plików typu *message file* zawierających tłumaczenia dla danego projektu.
- `TIME_ZONE`. Ciąg tekstowy przedstawiający strefę czasową dla projektu. Po utworzeniu nowego projektu za pomocą polecenia `startproject` opcja ta jest ustawiona na `'UTC'`. Możesz przypisać inną strefę czasową, np. `'Europe/Warsaw'`.

Dostępne są jeszcze inne ustawienia internacjonalizacji i lokalizacji. Ich pełną listę znajdziesz na stronie *https://docs.djangoproject.com/en/5.0/ref/settings/#globalization-i18n-l10n*.

Po zapoznaniu się z najważniejszymi ustawieniami internacjonalizacji i lokalizacji przystępujemy do stworzenia tłumaczeń dla budowanej aplikacji.

Polecenia przeznaczone do zarządzania internacjonalizacją

Django do zarządzania tłumaczeniami oferuje wymienione poniżej polecenia.

- `makemessages`. Polecenie powoduje przejście przez drzewo kodu źródłowego projektu i wyszukanie wszystkich ciągów tekstowych przeznaczonych do tłumaczenia oraz utworzenie lub uaktualnienie plików typu *.po* w katalogu *locale*. Dla każdego języka tworzony jest jeden plik *.po*.
- `compilemessages`. Polecenie powoduje kompilację istniejących plików *.po* na pliki *.mo*, które będą używane do pobierania przetłumaczonych ciągów tekstowych.

Do generowania i kompilowania plików tłumaczeń framework Django wykorzystuje zestaw narzędzi `gettext`. Przyjrzyjmy się, jak go zainstalować.

Instalowanie zestawu narzędzi gettext

W celu tworzenia, uaktualnienia i kompilacji plików typu *message file* konieczne jest zainstalowanie w systemie zestawu narzędzi `gettext`. Większość dystrybucji systemu Linux ma go zainstalowanego domyślnie. Jeżeli używasz systemu macOS, wtedy prawdopodobnie najłatwiejszym sposobem będzie instalacja wymaganych narzędzi za pomocą Homebrew (*http://brew.sh/*) po wydaniu polecenia:

```
brew install gettext
```

Może być również konieczne wymuszenie połączenia za pomocą polecenia:

```
brew link --force gettext
```

Jeśli korzystasz z systemu Windows, wykonaj czynności opisane na stronie *https://docs.djangoproject.com/en/5.0/topics/i18n/translation/#gettext-on-windows*. Wstępnie skompilowany binarny instalator `gettext` dla systemu Windows możesz pobrać pod adresem *https://mlocati.github.io/articles/gettext-iconv-windows.html*.

Po zainstalowaniu zestawu narzędzi `gettext` możesz rozpocząć tłumaczenie projektu. Przede wszystkim trzeba zapoznać się z sekwencją działań potrzebnych do przetłumaczenia projektu oraz dowiedzieć, jak Django określa język użytkownika.

Jak dodać tłumaczenie do projektu Django?

Zobaczmy teraz, jak wygląda proces internacjonalizacji projektu. Oto czynności, które trzeba wykonać, aby przetłumaczyć projekt Django:

1. W kodzie Pythona oraz w szablonach oznaczamy ciągi tekstowe jako przeznaczone do tłumaczenia.
2. Wydajemy polecenie `makemessages` w celu utworzenia lub uaktualnienia plików typu *message file*, które będą zawierać wszystkie możliwe do przełożenia ciągi tekstowe z kodu.
3. Tłumaczymy ciągi tekstowe zawarte w plikach *message file*.
4. Kompilujemy pliki *message file* za pomocą polecenia `compilemessages`.

W całym rozdziale będziemy postępować zgodnie z tym procesem, aby dodać tłumaczenia do budowanego projektu.

W następnym punkcie wyjaśnię, jak Django określa język bieżącego użytkownika.

W jaki sposób Django określa bieżący język?

Framework Django jest dostarczany wraz z oprogramowaniem middleware przeznaczonym do ustalenia bieżącego języka na podstawie danych żądania. Jest to oprogramowanie `LocaleMiddleware` znajdujące się w module `django.middleware.locale.Locale` ↪`Middleware`, które służy do wykonywania następujących zadań.

1. Jeżeli używasz `i18_patterns`, czyli przetłumaczonych wzorców adresów URL, warstwa middleware szuka prefiksu języka w żądanym adresie URL i na jego podstawie ustala bieżący język. Wzorce adresów URL nauczysz się tłumaczyć w punkcie „Tłumaczenie wzorców adresów URL”.
2. Jeżeli wspomniany prefiks nie zostanie znaleziony, Django szuka klucza `LANGUAGE_SESSION_KEY` w sesji bieżącego użytkownika.
3. Jeżeli w sesji nie znajdują się informacje o języku, Django szuka istniejącego cookie wraz z informacjami o bieżącym języku. Własną nazwę dla tego rodzaju cookie można podać za pomocą opcji `LANGUAGE_COOKIE_NAME`. Domyślną nazwą dla tego cookie jest `django_language`.
4. Jeżeli cookie nie zostanie znalezione, Django sprawdza nagłówek `Accept-Language` żądania HTTP.
5. Jeżeli nagłówek `Accept-Language` nie wskazuje języka, Django używa języka zdefiniowanego w opcji `LANGUAGE_CODE`.

Domyślnie Django zastosuje język zdefiniowany w opcji `LANGUAGE_CODE`, chyba że podasz `LocaleMiddleware`. Przedstawiony powyżej proces ma zastosowanie tylko podczas użycia wymienionego oprogramowania middleware.

Można również pozwolić użytkownikom na zmianę języka. Sposób implementacji selektora języka omówiłem w podrozdziale „Umożliwienie użytkownikowi zmiany języka”.

Zacznijmy od przygotowania projektu do internacjonalizacji.

Przygotowanie projektu do internacjonalizacji

Teraz przygotujemy projekt do użycia różnych języków. Przygotujemy polską i angielską wersję budowanego sklepu internetowego.

Przeprowadź edycję pliku *settings.py* projektu (`myshop`) i dodaj do opcji `LANGUAGES` poniższe ustawienia. Umieść je obok ustawień `LANGUAGE_CODE`.

```
LANGUAGES = [
    ('pl', 'polski'),
    ('en', 'angielski'),
]
```

Opcja `LANGUAGES` zawiera dwie krotki składające się z kodu języka i jego nazwy. Kod języka może być specyficzny dla lokalizacji, np. `en-us` lub `en-gb`, bądź też ogólny, np. `en`. Korzystając z przedstawionej powyżej konfiguracji, wskazujemy, że dana aplikacja będzie dostępna jedynie w językach polskim i angielskim. Jeżeli więc nie zdefiniujemy własnego ustawienia `LANGUAGES`, witryna internetowa będzie dostępna we wszystkich językach, na które został przetłumaczony framework Django.

Zmień wartość opcji `LANGUAGE_CODE` na taką:

```
LANGUAGE_CODE = 'pl'
```

Dodaj do opcji `MIDDLEWARE_CLASSES` ustawienie `'django.middleware.locale.LocaleMiddleware'`. Upewnij się, że to oprogramowanie zostało dodane po `SessionMiddleware`, ponieważ do działania `LocaleMiddleware` wymaga danych sesji. Ponadto oprogramowanie middleware powinno być umieszczone przed `CommonMiddleware`, a to dlatego, że drugie z wymienionych do rozwiązania żądanego adresu URL wymaga aktywnego języka. Opcja `MIDDLEWARE` powinna mieć postać pokazaną poniżej.

```
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.locale.LocaleMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]
```

Wskazówka

Kolejność oprogramowania middleware jest bardzo ważna, ponieważ działanie jednej warstwy middleware może zależeć od zbioru danych dostarczonego przez uruchomioną wcześniej inną warstwę middleware. Podczas przetwarzania żądania zastosowanie warstwy middleware odbywa się w kolejności jego występowania w ustawieniu `MIDDLEWARE`, natomiast podczas udzielania odpowiedzi w odwrotnej kolejności.

Utwórz poniższą strukturę katalogów w głównym katalogu projektu. Struktura ta powinna znaleźć się na tym samym poziomie, na którym znajduje się plik *manage.py*.

```
locale/
     pl/
     en/
```

Katalog *locale* jest miejscem przeznaczonym na wszystkie pliki typu *message file* aplikacji.

Ponownie przeprowadź edycję pliku *settings.py* i umieść w nim poniższe ustawienia.

```
LOCALE_PATHS = [
    BASE_DIR / 'locale',
]
```

Opcja `LOCALE_PATHS` określa katalogi sprawdzane przez Django w poszukiwaniu plików z tłumaczeniami. Lokalne ścieżki dostępu wymienione na początku mają pierwszeństwo.

Kiedy używasz polecenia `makemessages` z poziomu katalogu projektu, pliki typu *message file* zostaną wygenerowane w katalogu *locale* wskazanym wcześniej przez wymienioną ścieżkę dostępu. Jednak w aplikacjach zawierających katalog *locale* pliki typu *message file* będą wygenerowane właśnie w tym katalogu.

Projekt jest teraz przygotowany do internacjonalizacji. W kolejnym podrozdziale dowiesz się, jak tłumaczyć ciągi znaków w kodzie Pythona.

Tłumaczenie kodu Pythona

W tym podrozdziale poznasz różne metody obsługi tłumaczeń w kodzie Pythona. Omówię metody:

- **Tłumaczenie standardowe**.
- **Tłumaczenie leniwe.** Wykonywane w chwili dostępu do wartości, zamiast w momencie wywoływania funkcji.
- **Tłumaczenia zawierające zmienne.** Służy do interpolacji zmiennych w ciągach znaków, które mają zostać przetłumaczone.
- **Liczba mnoga w tłumaczeniu.** Techniki zarządzania tłumaczeniami zależnymi od wielkości liczbowych, które mogą mieć wpływ na tłumaczony ciąg znaków.

Aby przetłumaczyć literały w kodzie Pythona, ciągi tekstowe przeznaczone do przełożenia możesz oznaczyć za pomocą funkcji `gettext()` oferowanej przez moduł `django ↪.utils.translation`. Funkcja tłumaczy komunikat i zwraca ciąg tekstowy. Powszechnie stosowana konwencja polega na zaimportowaniu tej funkcji jako skrótu w postaci _ (znak podkreślenia).

Dokumentację dotyczącą tłumaczeń znajdziesz na stronie *https://docs.djangoproject.com/en/5.0/topics/i18n/translation/*.

Przyjrzyjmy się różnym metodom tłumaczenia ciągów znaków Pythona.

Tłumaczenie standardowe

Poniższy fragment kodu pokazuje, jak oznaczyć ciąg tekstowy do tłumaczenia.

```
from django.utils.translation import gettext as _
output = _('Tekst przeznaczony do tłumaczenia.')
```

Ta metoda umożliwia stosowanie tłumaczeń do większości ciągów znaków w kodzie Pythona. Do tego celu wykorzystywana jest funkcja `gettext()`, która dla wygody jest reprezentowana przez alias _.

Tłumaczenie leniwe

Django udostępnia *leniwe* wersje wszystkich funkcji dotyczących tłumaczenia, mają one wówczas przyrostek `_lazy()`. Podczas użycia leniwych wersji funkcji ciągi tekstowe są tłumaczone w chwili uzyskiwania dostępu do ich wartości, a nie w momencie wywoływania funkcji (stąd bierze się nazwa leniwe). Leniwe funkcje tłumaczenia okazują się użyteczne, gdy ciągi tekstowe przeznaczone do tłumaczenia znajdują się w ścieżkach dostępu wykonywanych podczas wczytywania modułów.

Typowym przykładem, w którym przydają się leniwe tłumaczenia, jest plik *settings.py* projektu, w którym natychmiastowe tłumaczenie nie ma sensu, ponieważ ustawienia muszą zostać zdefiniowane, zanim system tłumaczenia będzie w pełni gotowy.

Uwaga

W przypadku użycia `gettext_lazy()` zamiast `gettext()` ciągi tekstowe są tłumaczone w chwili uzyskania dostępu do wartości, a nie podczas wywoływania funkcji. Django oferuje leniwe wersje dla wszystkich funkcji dotyczących tłumaczenia.

Tłumaczenia zawierające zmienne

Ciągi tekstowe oznaczone do tłumaczenia mogą zawierać miejsca zarezerwowane przeznaczone do wstawienia zmiennych w przetłumaczonym tekście. Poniższy fragment kodu jest przykładem tłumaczenia ciągu tekstowego wraz ze wspomnianym miejscem zarezerwowanym.

```
from django.utils.translation import gettext as _
month = _('kwiecień')
day = '14'
output = _('Dzisiaj mamy %(day) %(month)a ') % {'month': month, 'day': day}
```

Użycie miejsc zarezerwowanych sprawia, że można zmienić kolejność zmiennych. Przykładowo w języku angielskim tłumaczenie powyższego przykładu ma postać „Today is April 14”, natomiast w języku hiszpańskim to będzie „Hoy es 14 de Abril”. Gdy w tłumaczonym ciągu tekstowym znajduje się więcej niż tylko jeden parametr, zawsze używaj interpolacji ciągu tekstowego zamiast interpolacji pozycyjnej. W ten sposób będziesz mógł zmieniać kolejność tekstu w miejscach zarezerwowanych.

Liczba mnoga w tłumaczeniach

W przypadku liczby mnogiej masz do dyspozycji funkcje `ngettext()` i `ngettext_lazy()`. Funkcje te pozwalają na tłumaczenie form w postaci liczby pojedynczej i mnogiej, w zależności od argumentu wskazującego liczbę obiektów. Poniżej pokazałem przykład użycia tych funkcji.

```
output = ngettext(
    'mamy %(count)d produkt',    # Liczba pojedyncza
    'mamy %(count)d produktów',  # Liczba mnoga
    count                        # Wartość liczbowa określająca formę
) % {'count': count}
```

W tym przykładzie, jeśli `count` wynosi 1, `ngettext()` użyje pierwszego ciągu i wyświetli „mamy 1 produkt". W przypadku dowolnej innej liczby użyje drugiego ciągu znaków, co spowoduje wyświetlenie na przykład „mamy 5 produktów". Zastosowanie opisanych mechanizmów pozwala na bardziej dokładne i gramatycznie poprawne tłumaczenie w językach, w których stosowanie zasad pluralizacji jest niezbędne.

Skoro poznałeś podstawy tłumaczenia literałów w kodzie Pythona, możemy przystąpić do tłumaczenia projektu.

Tłumaczenie własnego kodu

Na początek przetłumaczymy nazwy języków. W tym celu postępuj zgodnie z poniższymi instrukcjami.

W pliku *settings.py* projektu zaimportuj funkcję `gettext_lazy()` i zmień ustawienia opcji `LANGUAGES` w pokazany niżej sposób.

```
from django.utils.translation import gettext_lazy as _
# ...

LANGUAGES = [
    ('pl', _('polski')),
    ('en', _('angielski')),
]
```

Wykorzystamy tutaj funkcję `gettext_lazy()` zamiast funkcji `gettext()` w celu uniknięcia zapętlonego importu, a nazwy języków zostaną przetłumaczone w chwili uzyskania dostępu do tych danych.

Przejdź do powłoki i z poziomu katalogu projektu wydaj poniższe polecenie.

```
django-admin makemessages --all
```

Powinieneś otrzymać następujące dane wyjściowe.

```
processing locale en
processing locale pl
```

Spójrz na katalog *locale*. Powinieneś zobaczyć taką strukturę plików:

```
pl/
     LC_MESSAGES/
          django.po
```

```
en/
     LC_MESSAGES/
         django.po
```

Dla każdego języka został utworzony plik z rozszerzeniem *.po*.

W edytorze tekstu otwórz plik *en/LC_MESSAGES/django.po*. Na końcu pliku powinieneś zobaczyć następującą zawartość.

```
#: myshop/settings.py:118
msgid "polski"
msgstr ""
#: myshop/settings.py:119
msgid "angielski"
msgstr ""
```

Każdy tłumaczony ciąg tekstowy jest poprzedzony komentarzem przedstawiającym informacje szczegółowe o pliku oraz wierszu, w którym dany ciąg został znaleziony. Dla każdego tłumaczenia mamy po dwa ciągi tekstowe.

- `msgid`. Ciąg tekstowy do przetłumaczenia w postaci, w jakiej znajduje się w kodzie źródłowym.
- `msgstr`. Ciąg tekstowy przetłumaczony na dany język, domyślnie to jest pusty ciąg tekstowy. W tym miejscu wprowadzasz rzeczywiste tłumaczenie danego ciągu tekstowego.

Wprowadź w `msgstr` tłumaczenie ciągów tekstowych podanych w `msgid`.

```
#: myshop/settings.py:118
msgid "polski"
msgstr "Polish"
#: myshop/settings.py:119
msgid "angielski"
msgstr "English"
```

Zapisz zmodyfikowany plik typu *message file*, przejdź do powłoki i wydaj poniższe polecenie.

```
django-admin compilemessages
```

Jeśli wszystko przebiegło pomyślnie, powinieneś otrzymać następujący wynik.

```
processing file django.po in myshop/locale/en/LC_MESSAGES
processing file django.po in myshop/locale/pl/LC_MESSAGES
```

Wygenerowane dane wyjściowe informują o skompilowanych plikach typu *message file*. Ponownie zajrzyj do katalogu *locale* w projekcie `myshop`, powinieneś zobaczyć przedstawioną poniżej strukturę plików i katalogów.

```
pl/
    LC_MESSAGES/
        django.mo
        django.po
en/
    LC_MESSAGES/
        django.mo
        django.po
```

Jak możesz zobaczyć, dla każdego języka został wygenerowany skompilowany plik *.mo*.

Teraz przystępujemy do tłumaczenia nazw pól modelu wyświetlanych w witrynie internetowej.

Przeprowadź edycję pliku *models.py* aplikacji `orders` i dodaj do modelu `Order` nowe pole.

```
from django.utils.translation import gettext_lazy as _


class Order(models.Model):
    first_name = models.CharField(_('imię'), max_length=50)
    last_name = models.CharField(_('nazwisko'), max_length=50)
    email = models.EmailField(_('e-mail'))
    address = models.CharField(_('adres'), max_length=250)
    postal_code = models.CharField(_('kod pocztowy'), max_length=20)
    city = models.CharField(_('miejscowość'), max_length=100)
    # ...
```

W ten sposób przygotowaliśmy nazwy dla kolumn wyświetlanych po złożeniu zamówienia przez użytkownika, czyli `first_name`, `last_name`, `email`, `address`, `postal_code` i `city`. Są to `imię`, `nazwisko`, `e-mail`, `adres`, `kod pocztowy` i `miejscowość`. Pamiętaj o możliwości użycia atrybutu `verbose_name` dla nazw wymienionych kolumn.

W katalogu aplikacji `orders` utwórz poniższą strukturę katalogów.

```
locale/
    pl/
    en/
```

Gdy utworzysz katalog *locale*, przetłumaczone ciągi tekstowe aplikacji będą przechowywane w plikach umieszczonych w tym katalogu, zamiast po prostu w głównych plikach typu *message file*. W ten sposób możliwe jest wygenerowanie oddzielnych plików z tłumaczeniem dla każdej aplikacji. W ten sposób możliwe jest wygenerowanie oddzielnych plików z tłumaczeniem dla każdej aplikacji.

Przejdź do powłoki z poziomu katalogu projektu i wydaj poniższe polecenie.

```
django-admin makemessages --all
```

Powinieneś otrzymać takie dane wyjściowe.

```
processing locale en
processing locale pl
```

W edytorze tekstu otwórz plik *en/LC_MESSAGES/django.po* aplikacji `order`. Zobaczysz ciągi tekstowe przeznaczone dla tłumaczenia modelu `Order`. W wierszu zawierającym `msgstr` wpisz tłumaczenia na podstawie ciągów tekstowych znajdujących się w wierszu `msgid`, co pokazałem poniżej.

```
#: orders/models.py:12
msgid "imię"
msgstr "first name"
#: orders/models.py:14
msgid "nazwisko"
msgstr "last name"
```

```
#: orders/models.py:16
msgid "e-mail"
msgstr "e-mail"
#: orders/models.py:17
msgid "adres"
msgstr "address"
#: orders/models.py:19
msgid "kod pocztowy"
msgstr "postal code"
#: orders/models.py:21
msgid "miejscowość"
msgstr "city"
```

Po zakończeniu dodawania tłumaczeń zapisz plik.

Do tłumaczenia ciągów tekstowych, poza edytorem tekstu, możesz jeszcze użyć narzędzia *Poedit*. Poedit to oprogramowanie do edycji tłumaczeń, które wykorzystują funkcję `gettext`. Jest dostępne dla systemów Linux, Windows i macOS. Narzędzie Poedit możesz pobrać pod adresem *https://poedit.net/*.

Przystępujemy do tłumaczenia formularzy projektu. Formularz `OrderCreateForm` aplikacji `orders` nie wymaga tłumaczenia, ponieważ to egzemplarz klasy `ModelForm` i podczas wyświetlania etykiet pól formularza używa atrybutu `verbose_name` pól modelu `Order`. Musimy natomiast przetłumaczyć formularze aplikacji `cart` i `coupons`.

Przeprowadź edycję pliku `forms.py` w katalogu aplikacji `cart` i dodaj atrybut `label` do pola `quantity` formularza `CartAddProductForm`, a później oznacz to pole jako przeznaczone do przetłumaczenia w następujący sposób.

```
from django import forms
from django.utils.translation import gettext_lazy as _

PRODUCT_QUANTITY_CHOICES = [(i, str(i)) for i in range(1, 21)]

class CartAddProductForm(forms.Form):
    quantity = forms.TypedChoiceField(
        choices=PRODUCT_QUANTITY_CHOICES,
        coerce=int,
        label=_('Ilość')
    )
    override = forms.BooleanField(
        required=False,
        initial=False,
        widget=forms.HiddenInput
    )
```

Przeprowadź edycję pliku *forms.py* aplikacji `coupons` i przetłumacz formularz `Coupon ↪ApplyForm`, tak jak pokazałem poniżej.

```
from django import forms
from django.utils.translation import gettext_lazy as _

class CouponApplyForm(forms.Form):
    code = forms.CharField(label=_('Kod'))
```

Do pola `code` została dodana etykieta, oznaczona jako przeznaczona do tłumaczenia.

Na tym zakończyłeś oznaczanie ciągów w kodzie Pythona przeznaczonych do tłumaczenia. W następnym podrozdziale dowiesz się, jak tłumaczyć ciągi znaków w szablonach.

Tłumaczenie szablonów

Django oferuje znaczniki szablonu {% translate %} i {% blocktranslate %} przeznaczone do obsługi tłumaczenia ciągów tekstowych. W celu wykorzystania wymienionych znaczników na początku szablonu należy umieścić znacznik {% load i18n %} odpowiedzialny za wczytanie wcześniej wymienionych.

Znacznik szablonu {% translate %}

Znacznik szablonu {% translate %} pozwala na oznaczenie zawartości ciągu tekstowego, stałej lub zmiennej jako przeznaczonych do tłumaczenia. Wewnętrznie Django wykonuje funkcję gettext() dla danego tekstu. Oto sposób oznaczenia ciągu tekstowego w szablonie jako przeznaczonego do tłumaczenia:

```
{% translate "Tekst przeznaczony do tłumaczenia." %}
```

Istnieje możliwość użycia słowa kluczowego as do umieszczenia przetłumaczonej treści w zmiennej, którą będzie można później wykorzystywać w szablonie. W poniższym fragmencie kodu przetłumaczony tekst został umieszczony w zmiennej o nazwie greeting.

```
{% translate "Witaj!" as greeting %}
<h1>{{ greeting }}</h1>
```

Znacznik {% translate %} doskonale sprawdza się przy tłumaczeniu prostych ciągów tekstowych, ale nie może obsługiwać przeznaczonej do tłumaczenia treści, która zawiera zmienne.

Znacznik szablonu {% blocktranslate %}

Znacznik szablonu {% blocktranslate %} pozwala na oznaczenie do tłumaczenia treści zawierającej literały i zmienne wykorzystujące miejsca zarezerwowane. Poniżej pokazałem przykład użycia znacznika {% blocktranslate %} do przetłumaczenia treści zawierającej zmienną name.

```
{% blocktranslate %}Witaj, {{ name }}!{% endblocktranslate %}
```

Można użyć słowa kluczowego with w celu dołączenia rozszerzeń szablonu, aby np. uzyskać dostęp do atrybutów obiektu lub zastosować filtry w zmiennych. Do wymienionych zadań zawsze można użyć miejsc zarezerwowanych. Wewnątrz bloku blocktranslate nie można uzyskać dostępu do wyrażeń ani atrybutów. W poniższym fragmencie kodu pokazałem przykład zastosowania słowa kluczowego with w celu dołączenia atrybutu obiektu, dla którego będzie zastosowany filtr capfirst.

```
{% blocktranslate with name=user.name|capfirst %}
  Witaj, {{ name }}!
{% endblocktranslate %}
```

Wskazówka

Gdy w ciągu tekstowym przeznaczonym do tłumaczenia trzeba wstawić zawartość przechowywaną w zmiennej, używaj znacznika `{% blocktranslate %}` zamiast `{% translate %}`.

Po zapoznaniu się ze znacznikami przeznaczonymi do tłumaczenia szablonów spróbujmy je wykorzystać.

Tłumaczenie szablonów sklepu internetowego

Przeprowadź edycję szablonu *shop/base.html* aplikacji `shop`. Upewnij się, że wczytany został znacznik `i18n` na początku szablonu i ciągi tekstowe do przetłumaczenia oznaczone zostały w pokazany niżej sposób. Nowy kod został wyróżniony pogrubioną czcionką.

```
{% load i18n static %}
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>
    {% block title %}{% translate "Mój sklep" %}{% endblock %}
  </title>
  <link href="{% static "css/base.css" %}" rel="stylesheet">
</head>
<body>
  <div id="header">
    <a href="/" class="logo">{% translate "Mój sklep " %}</a>
  </div>
  <div id="subheader">
    <div class="cart">
      {% with total_items=cart|length %}
        {% if total_items > 0 %}
          {% translate "Twój Koszyk" %}:
          <a href="{% url "cart:cart_detail" %}">
            {% blocktranslate with total=cart.get_total_price count items=total_
            ↪items %}
               {{ items }} pozycja, {{ total }} PLN
            {% plural %}
              {{ items }} pozycji, {{ total }} PLN
            {% endblocktranslate %}
          </a>
        {% elif not order %}
          {% translate "Koszyk jest pusty" %}
        {% endif %}
      {% endwith %}
    </div>
  </div>
  <div id="content">
    {% block content %}
    {% endblock %}
```

```
  </div>
</body>
</html>
```

Zadbaj o to, aby żaden znacznik szablonu nie obejmował więcej niż jednego wiersza.

Zwróć uwagę na to, jak został użyty znacznik {% blocktranslate %} do wyświetlenia podsumowania koszyka na zakupy. Wcześniej za wyświetlenie tego rodzaju podsumowania odpowiadał następujący fragment kodu.

```
{{ total_items }} pozycji,
{{ cart.get_total_price }} PLN
```

Zmieniliśmy to i teraz, aby ustawić zastępczą sumę z wartością `cart.get_total_price` (tutaj wywołano metodę obiektu), używasz wywołania {% blocktranslate with ... %}. Wykorzystujesz także wywołanie `count`, które pozwala ustawić zmienną do zliczania obiektów, aby framework Django wybrał odpowiednią formę liczby mnogiej. W celu zliczania obiektów ustawiamy zmienną `items` na wartość `total_items`.

Dzięki temu można ustawić tłumaczenie dla liczby pojedynczej i mnogiej. Obie formy oddzielamy w bloku {% blocktranslate %} znacznikiem {% plural %}. Wynikowy kod ma taką postać:

```
{% blocktranslate with total=cart.get_total_price count items=total_items %}
  {{ items }} pozycja, {{ total }} PLN
{% plural %}
  {{ items }} pozycji, {{ total }} PLN
{% endblocktranslate %}
```

Kolejnym krokiem jest edycja szablonu *shop/product/detail.html* w aplikacji `shop` i wczytanie znaczników `i18n` na początku pliku.

```
{% extends "shop/base.html" %}
{% load i18n static %}
 …
```

Zwróć uwagę, że znacznik {% load %} pozwala na załadowanie wszystkich tagów szablonu jednocześnie. W tym celu należy oddzielić włączane moduły spacjami. W tym przypadku ładujemy moduły `i18n` i `static`, zawierające znaczniki szablonów.

Teraz znajdź poniższy wiersz kodu:

```
<input type="submit" value="Dodaj do koszyka">
```

i zastąp go tym:

```
<input type="submit" value="{% translate "Dodaj do koszyka" %}">
```

Następnie znajdź poniższy wiersz kodu:

```
<h3>Z tym produktem klienci kupowali również</h3>
```

i zastąp go tym:

```
<h3>{% translate "Z tym produktem klienci kupowali również" %}</h3>
```

Następnie przetłumacz szablony aplikacji `orders`. Przeprowadź edycję szablonu *orders/order/create.html* aplikacji `orders` i oznacz tekst przeznaczony do tłumaczenia, tak jak pokazałem poniżej.

```
{% extends "shop/base.html" %}
{% load i18n %}

{% block title %}
  {% translate "Do kasy" %}
{% endblock %}

{% block content %}
  <h1>{% translate "Kasa" %}</h1>
  <div class="order-info">
    <h3>{% translate "Zamówienie" %}</h3>
    <ul>
      {% for item in cart %}
        <li>
          {{ item.quantity }}x {{ item.product.name }}
          <span>{{ item.total_price }} PLN</span>
        </li>
      {% endfor %}
      {% if cart.coupon %}
        <li>
          {% blocktranslate with code=cart.coupon.code discount=cart.coupon.discount %}
             "{{ code }}" ({{ discount }}% rabatu)
          {% endblocktranslate %}
          <span class="neg">-{{ cart.get_discount|floatformat:2 }} zł</span>
        </li>
      {% endif %}
    </ul>
    <p>{% translate "Razem" %}: {{cart.get_total_price_after_discount|
    ↪floatformat:2 }} PLN</p>
   </div>
  <form method="post" class="order-form">
    {{ form.as_p }}
    <p><input type="submit" value="{% translate "Złóż zamówienie " %}"></p>
    {% csrf_token %}
  </form>
{% endblock %}
```

Zadbaj o to, aby żaden znacznik szablonu nie obejmował więcej niż jednego wiersza. W kodzie projektu spójrz na poniższe pliki modyfikowane w tym rozdziale, a zobaczysz, w jaki sposób są oznaczane do przetłumaczenia ciągi tekstowe.

- Aplikacja `shop`: szablon *shop/product/list.html*.
- Aplikacja `orders`: szablon *orders/order/pdf.html*.
- Aplikacja `cart`: szablon *cart/detail.html*.
- Aplikacja `payments` : szablony *payment/process.html*, *payment/completed.html* i *payment/canceled.html*.

Uwaga

Przypominam, że kod źródłowy przykładów z tego rozdziału można znaleźć pod adresem *https://github.com/PacktPublishing/Django-5-by-Example/tree/master/Chapter11*.

Przystępujemy teraz do uaktualnienia plików typu *message file* w celu dołączenia przetłumaczonych ciągów tekstowych.

Przejdź do powłoki, a następnie wydaj poniższe polecenie.

```
django-admin makemessages --all
```

Pliki *.po* znajdują się w katalogu *locale* projektu `myshop`; jeśli tam zajrzysz, zobaczysz, że w aplikacji `orders` do przetłumaczenia zostały oznaczone wszystkie ciągi tekstowe.

Przeprowadź edycję plików tłumaczeń z rozszerzeniem *.po* projektu i aplikacji `orders`, a następnie dołącz tłumaczenie w języku angielskim, co należy ustawić za pomocą opcji `msgstr`. W trakcie tego zadania możesz skorzystać z przetłumaczonych plików *.po*, które znajdziesz w materiałach przygotowanych dla książki.

Aby skompilować pliki tłumaczeń, wydaj poniższe polecenie.

```
django-admin compilemessages
```

Wyświetli się następujący wynik.

```
processing file django.po in myshop/locale/en/LC_MESSAGES
processing file django.po in myshop/locale/pl/LC_MESSAGES
processing file django.po in myshop/orders/locale/en/LC_MESSAGES
processing file django.po in myshop/orders/locale/pl/LC_MESSAGES
```

Dla każdego pliku tłumaczeń *.po* został wygenerowany plik *.mo* zawierający skompilowane tłumaczenia.

W tym punkcie edytowałeś pliki *.po* za pomocą edytora tekstu lub narzędzia Poedit. W następnym podrozdziale użyjemy aplikacji Django do edycji tłumaczeń bezpośrednio w przeglądarce.

Użycie interfejsu do tłumaczeń o nazwie Rosetta

Rosetta to aplikacja zewnętrzna pozwalająca na tłumaczenie ciągów tekstowych za pomocą takiego samego interfejsu, jaki znasz z witryny administracyjnej Django. Rosetta ułatwia edycję plików *.po* i uaktualnia skompilowane pliki tłumaczeń. Eliminuje to konieczność pobierania i przesyłania plików tłumaczeń oraz umożliwia wspólną edycję plików *.po* przez wielu użytkowników.

Przystępujemy teraz do zintegrowania narzędzia Rosetta z budowanym projektem.

Zainstaluj aplikację Rosetta za pomocą menedżera pakietów `pip`, używając poniższego polecenia.

```
python -m pip install django-rosetta==0.10.0
```

Następnie umieść `'rosetta'` na liście `INSTALLED_APPS` w pliku *settings.py* projektu.

```
INSTALLED_APPS = [
    # ...
    'rosetta',
]
```

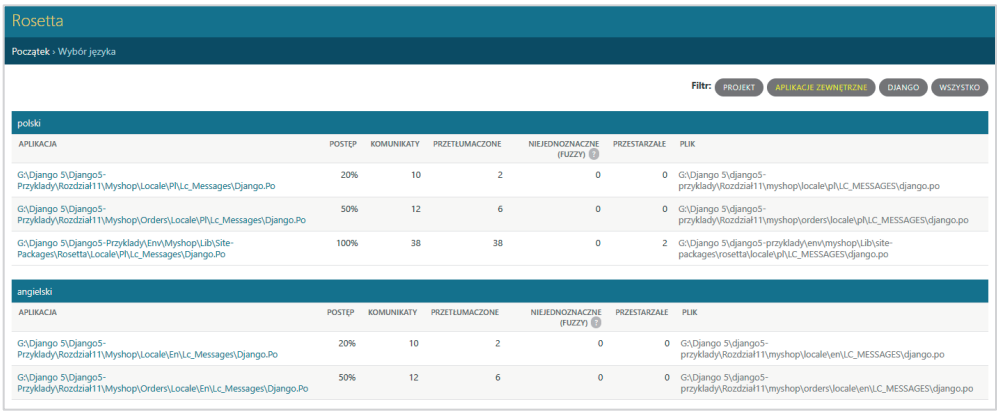
Konieczne jest również dodanie do głównej konfiguracji adresów URL nowego adresu URL dla aplikacji Rosetta. Przeprowadź edycję pliku *urls.py* swojego projektu i dodaj do listy `urlpatterns` poniższy wzorzec adresu URL oznaczony pogrubioną czcionką.

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('cart/', include('cart.urls', namespace='cart')),
    path('orders/', include('orders.urls', namespace='orders')),
    path('payment/', include('payment.urls', namespace='payment')),
    path('coupons/', include('coupons.urls', namespace='coupons')),
    path('rosetta/', include('rosetta.urls')),
    path('', include('shop.urls', namespace='shop')),
]
```

Pamiętaj o umieszczeniu go przed wzorcem *shop.urls*, aby uniknąć dopasowania nieprawidłowego wzorca.

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/admin/` i zaloguj się jako superużytkownik. Następnie przejdź pod adres `http://127.0.0.1:8000/rosetta/`. W menu *Filtr* kliknij link *APLIKACJE ZEWNĘTRZNE*. Wyświetli się lista dostępnych plików komunikatów włącznie z tymi, które należą do aplikacji `orders`.

Powinieneś zobaczyć listę dostępnych języków (rysunek 11.2).

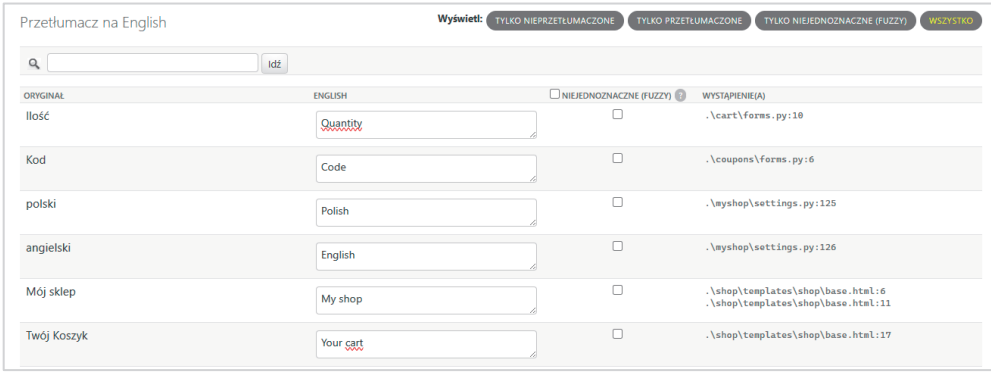


Rysunek 11.2. Interfejs administracyjny Rosetta

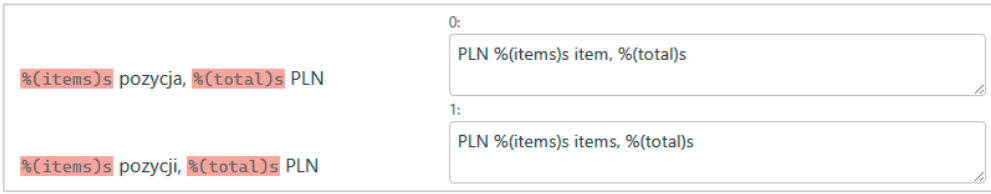
Kliknij pozycję odpowiadającą tłumaczeniom aplikacji `myshop` w sekcji *angielski*, co pozwoli na przeprowadzenie edycji angielskiego tłumaczenia aplikacji. Powinieneś otrzymać listę ciągów tekstowych przeznaczonych do tłumaczenia (rysunek 11.3).

Tłumaczenie w języku angielskim wpisz w kolumnie *ENGLISH*. Kolumna *WYSTĄPIENIE(A)* wyświetla nazwy plików i numery wierszy kodu, w których został znaleziony dany ciąg tekstowy do przetłumaczenia.

Tłumaczenia wraz z miejscami zarezerwowanymi przedstawiają się tak, jak pokazałem na rysunku 11.4.



Rysunek 11.3. Edycja tłumaczenia na angielski z wykorzystaniem programu Rosetta



Rysunek 11.4. Tłumaczenia z miejscami zarezerwowanymi

Dla miejsc zarezerwowanych Rosetta używa innego koloru tła. Kiedy będziesz tłumaczyć treść, upewnij się, że miejsca zarezerwowane są nieprzetłumaczone. Przykładowo spójrz na poniższy ciąg tekstowy.

```
%(items)s produktów %(total)s zł
```

Po jego przetłumaczeniu na język angielski otrzymujemy następującą postać ciągu tekstowego.

```
%(items)s items $%(total)s
```

Do przygotowania tłumaczenia projektu na język angielski możesz wykorzystać kod źródłowy dla tego rozdziału.

Po zakończeniu edycji tłumaczenia, aby zapisać przetłumaczony tekst w pliku *.po*, kliknij przycisk *Zapisz i tłumacz następny blok*. Podczas zapisywania pliku z tłumaczeniem Rosetta przeprowadza jego kompilację, więc nie trzeba ręcznie wydawać polecenia `compilemessages`. Rosetta wymaga jednak w celu zapisywania plików uprawnienia zapisu do katalogów *locale*. Dlatego też upewnij się, że nadałeś odpowiednie uprawnienia do tych katalogów.

Jeżeli chcesz umożliwić edycję tłumaczeń innym użytkownikom, w przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/admin/auth/group/add/` i utwórz nową grupę o nazwie *translators*. Następnie przejdź na stronę `http://127.0.0.1:8000/admin/auth/user/` w celu wyboru użytkowników, którzy otrzymają uprawnienia do edycji tłumaczeń. Podczas edycji ustawień użytkownika w sekcji *Permissions* dodaj grupę *translators* do listy *Chosen Groups* użytkownika. Aplikacja Rosetta jest dostępna jedynie dla superużytkowników oraz użytkowników należących do grupy *translators*.

Dokumentację aplikacji Rosetta znajdziesz na stronie *https://django-rosetta.readthedocs.io/*.

Uwaga

Kiedy dodajesz nowe tłumaczenie w środowisku produkcyjnym i korzystasz z Django w rzeczywistym serwerze WWW, wówczas zastosowanie wprowadzonych zmian — po wydaniu polecenia `compilemessages` lub po zapisaniu tłumaczenia za pomocą aplikacji Rosetta — wymaga ponownego uruchomienia serwera.

Podczas edycji tłumaczenie może zostać oznaczone jako rozmyte (ang. *fuzzy*). Przyjrzyjmy się, czym są tłumaczenia rozmyte.

Tłumaczenia niejednoznaczne

Być może zwróciłeś w aplikacji Rosetta uwagę na kolumnę *Niejednoznaczne (fuzzy)*. Nie jest to funkcja oferowana przez Rosettę, ale dostarczana przez `gettext`. Jeżeli opcja *FUZZY* jest aktywna dla danego tłumaczenia, nie zostanie ono umieszczone w skompilowanych plikach typu *message file*. Opcja po prostu oznacza, że dany ciąg tekstowy powinien być jeszcze przejrzany przez tłumacza. Po uaktualnieniu plików *.po* za pomocą nowych ciągów tekstowych do przetłumaczenia istnieje możliwość, że część z nich będzie oznaczona jako *fuzzy*. Dzieje się tak, bo `gettext`, gdy wykrywa wprowadzenie niewielkich modyfikacji w elementach `msgid`, paruje je z tłumaczeniem uznawanym za starsze i dlatego oznacza je opcją *fuzzy*, aby tłumaczenie zostało jeszcze przejrzane przez tłumacza. Następnie tłumacz przegląda ciągi tekstowe oznaczone jako fuzzy, usuwa opcję *FUZZY* i ponownie kompiluje plik typu *message file*.

Wprawdzie przetłumaczyłeś interfejs swojego projektu, ale internacjonalizacja na tym się nie kończy. Możesz również przetłumaczyć wzorce adresów URL, co pozwala zaoferować spersonalizowane adresy URL dostosowane do każdego obsługiwanego języka.

Wzorce adresów URL dla internacjonalizacji

Django oferuje możliwości internacjonalizacji adresów URL. Dostępne są dwie podstawowe funkcje przeznaczone do internacjonalizacji adresów URL.

- **Prefiks języka we wzorcach adresów URL.** Dodanie prefiksu języka do adresu URL pozwala na dostarczanie poszczególnych wersji językowych witryny internetowej za pomocą innego adresu bazowego URL.
- **Przetłumaczone wzorce adresów URL.** Oznaczenie wzorców adresów URL do tłumaczenia, więc ten sam adres URL będzie różny w poszczególnych językach.

Powodem tłumaczenia adresów URL może być optymalizacja witryny internetowej pod kątem silników wyszukiwarek internetowych. Dzięki dodaniu prefiksu do wzorców można zindeksować adresy URL dla poszczególnych języków, zamiast używać jednego dla wszystkich. Co więcej, po przetłumaczeniu adresu URL na każdy język wyszukiwarka internetowa otrzymuje adresy URL, które będą lepiej notowane dla poszczególnych języków.

Dodanie prefiksu języka do wzorców adresów URL

Django pozwala na dodanie prefiksu języka do wzorców adresów URL. Przykładowo angielska wersja witryny może być udostępniana przez ścieżkę dostępu rozpoczynającą się od */en/*, podczas gdy wersja polska — z prefiksem */pl/*. Aby użyć języków we wzorcach adresów URL, konieczne jest zastosowanie ustawienia `LocaleMiddleware`. Django wykorzysta to oprogramowanie middleware do ustalenia bieżącego języka na podstawie żądanego adresu URL. Opcję `LocaleMiddleware` dodaliśmy wcześniej do ustawienia `MIDDLEWARE` projektu, a zatem nie musimy robić tego teraz.

Dodamy teraz prefiks języka do naszych wzorców adresów URL.

Przeprowadź edycję głównego pliku *urls.py* aplikacji `myshop` i umieść w nim wywołanie `i18n_patterns()`, co pokazałem poniżej.

```
from django.conf.urls.i18n import i18n_patterns

urlpatterns = i18n_patterns(
    path('admin/', admin.site.urls),
    path('cart/', include('cart.urls', namespace='cart')),
    path('orders/', include('orders.urls', namespace='orders')),
    path('payment/', include('payment.urls', namespace='payment')),
    path('coupons/', include('coupons.urls', namespace='coupons')),
    path('rosetta/', include('rosetta.urls')),
    path('', include('shop.urls', namespace='shop')),
)
```

Istnieje możliwość łączenia wzorców adresów URL w wywołaniach `patterns()` i `i18n_patterns()`, więc pewne wzorce zawierają prefiks języka, a inne nie. Jednak najlepiej używać przetłumaczonych adresów URL jedynie w celu uniknięcia sytuacji, w której niestarannie przetłumaczony adres URL zostanie dopasowany do nieprzetłumaczonego wzorca adresu URL.

Uruchom serwer programistyczny i przejdź w przeglądarce internetowej pod adres `http://127.0.0.1:8000/`. W celu ustalenia bieżącego języka Django wykona omówione wcześniej w rozdziale kroki (patrz punkt „W jaki sposób Django określa bieżący język?") i przekieruje użytkownika do tego samego adresu URL, ale wraz z prefiksem języka. Spójrz na adres URL wyświetlony w przeglądarce internetowej, powinien mieć teraz postać `http://127.0.0.1:8000/pl/`. Bieżący język zostanie ustawiony za pomocą nagłówka `Accept-Language` przeglądarki internetowej (jeżeli to polski lub angielski) bądź też będzie domyślnie wskazany przez `LANGUAGE_CODE` (polski) w ustawieniach projektu.

Właśnie dodałeś prefiks języka do swoich adresów URL, dzięki czemu dla każdego dostępnego języka został wygenerowany inny adres URL. Takie działanie pomaga w indeksowaniu różnych wersji językowych w wyszukiwarkach.

W kolejnym punkcie zajmiemy się tłumaczeniem wzorców adresów URL, co pozwoli na dodanie do budowanej witryny w pełni przetłumaczonych adresów URL.

Tłumaczenie wzorców adresów URL

Django obsługuje przetłumaczone ciągi tekstowe we wzorcach adresów URL. Istnieje możliwość użycia różnych tłumaczeń dla poszczególnych języków w pojedynczym wzorcu adresu URL. Oznaczenie wzorca do przetłumaczenia odbywa się dokładnie tak samo jak w przypadku literałów, czyli za pomocą funkcji `gettext_lazy()`. Aby to zrobić, wykonaj działania przedstawione poniżej.

Przeprowadź edycję głównego pliku *urls.py* projektu `myshop` i dodaj tłumaczenie ciągów dla wyrażeń regularnych wzorców adresów URL aplikacji `cart`, `orders`, `payment` i `coupons`, tak jak pokazałem poniżej.

```
from django.utils.translation import gettext_lazy as _

urlpatterns = i18n_patterns(
    path('admin/', admin.site.urls),
    path(_('cart/'), include('cart.urls', namespace='cart')),
    path(_('orders/'), include('orders.urls', namespace='orders')),
    path(_('payment/'), include('payment.urls', namespace='payment')),
    path(_('coupons/'), include('coupons.urls', namespace='coupons')),
    path('rosetta/', include('rosetta.urls')),
    path('', include('shop.urls', namespace='shop')),
)
```

Teraz przeprowadź edycję pliku *urls.py* aplikacji `orders` i w poniższy sposób oznacz wzorce adresów URL przeznaczone do tłumaczenia.

```
from django.utils.translation import gettext_lazy as _
urlpatterns = [
    path(_('create/'), views.order_create, name='order_create'),
    # …
]
```

Kolejny krok to edycja pliku *urls.py* aplikacji `payment` i zmiana znajdującego się w nim kodu na następujący.

```
from django.utils.translation import gettext_lazy as _

urlpatterns = [
    path(_('process/'), views.payment_process, name='process'),
    path(_('completed/'), views.payment_completed, name='completed'),
    path(_('canceled/'), views.payment_canceled, name='canceled'),
    path('webhook/', webhooks.stripe_webhook, name='stripe-webhook'),
]
```

Zwróć uwagę, że te wzorce adresów URL będą zawierać prefiks języka, ponieważ są zawarte w metodzie `i18n_ pattern()` w głównym pliku *urls.py* projektu. Spowoduje to, że każdy wzorzec adresu URL będzie miał inny identyfikator URI dla każdego dostępnego języka, jeden zaczynający się od */en/*, drugi od */pl/* i tak dalej. Potrzebny jest jednak pojedynczy adres URL, aby Stripe mógł powiadamiać o zdarzeniach, i należy unikać prefiksów językowych w adresie URL elementu *webhook*.

Usuń z pliku *urls.py* aplikacji `payment` wzorzec adresu URL *webhook*. Kod powinien mieć taką postać:

```
from django.utils.translation import gettext_lazy as _

urlpatterns = [
    path(_('process/'), views.payment_process, name='process'),
    path(_('completed/'), views.payment_completed, name='completed'),
    path(_('canceled/'), views.payment_canceled, name='canceled'),
]
```

Następnie dodaj wzorzec adresu URL *webhook* do głównego pliku *urls.py* projektu `myshop`. Nowy kod został wyróżniony pogrubioną czcionką.

```
from django.utils.translation import gettext_lazy as _
from payment import webhooks

urlpatterns = i18n_patterns(
    path('admin/', admin.site.urls),
    path(_('cart/'), include('cart.urls', namespace='cart')),
    path(_('orders/'), include('orders.urls', namespace='orders')),
    path(_('payment/'), include('payment.urls', namespace='payment')),
    path(_('coupons/'), include('coupons.urls', namespace='coupons')),
    path('rosetta/', include('rosetta.urls')),
    path('', include('shop.urls', namespace='shop')),
)

urlpatterns += [
    path(
        'payment/webhook/',
        webhooks.stripe_webhook,
        name='stripe-webhook'
    ),
]
if settings.DEBUG:
    urlpatterns += static(
        settings.MEDIA_URL, document_root=settings.MEDIA_ROOT
    )
```

Dodaliśmy wzorzec adresu URL elementu *webhook* do ustawienia `urlpatterns` poza metodą `i18n_patterns()`, aby mieć pewność, że utrzymujemy jeden adres URL dla powiadomień o zdarzeniach Stripe.

Nie ma konieczności tłumaczenia wzorców adresów URL aplikacji `shop`, ponieważ są one budowane za pomocą zmiennych i nie zawierają żadnych innych literałów.

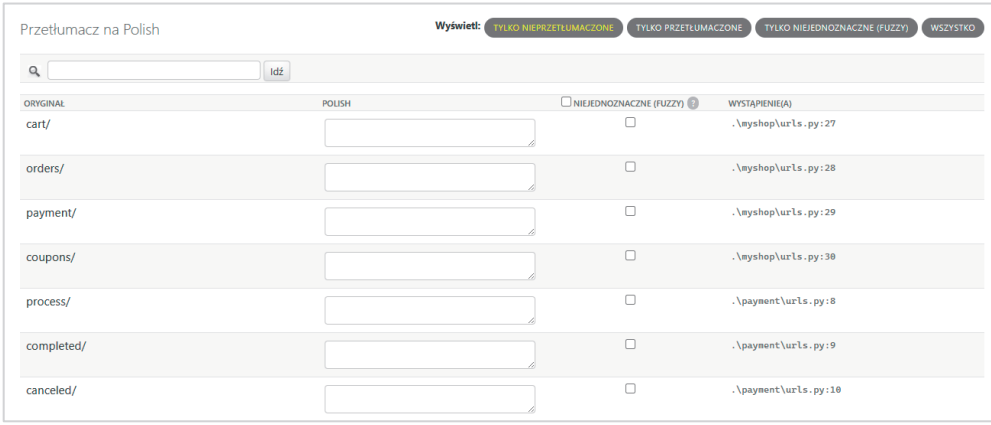
Przejdź do powłoki i wydaj poniższe polecenie, które spowoduje uaktualnienie plików typu *message file* nowymi tłumaczeniami.

```
django-admin makemessages --all
```

Uruchom serwer programistyczny za pomocą poniższego polecenia.

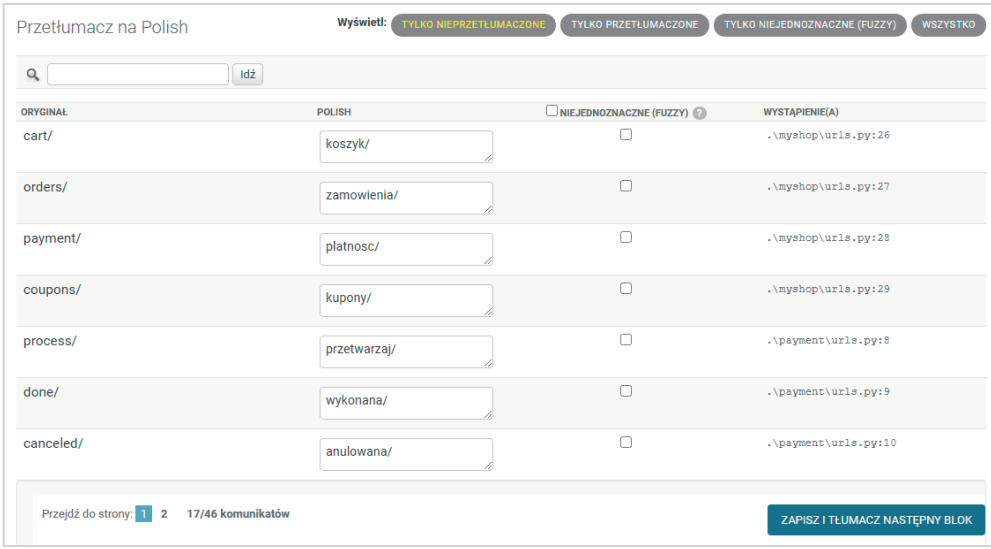
```
python manage.py runserver
```

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/pl/rosetta/` i w sekcji *Polish* kliknij link *myshop*. Aby wyświetlić wyłącznie te ciągi znaków, które nie zostały jeszcze przetłumaczone, możesz kliknąć link *Tylko nieprzetłumaczone*. Wyświetlą się wzorce adresów URL do tłumaczenia (rysunek 11.5).



Rysunek 11.5. Angielskie tłumaczenia wzorców adresów URL w interfejsie Rosetta

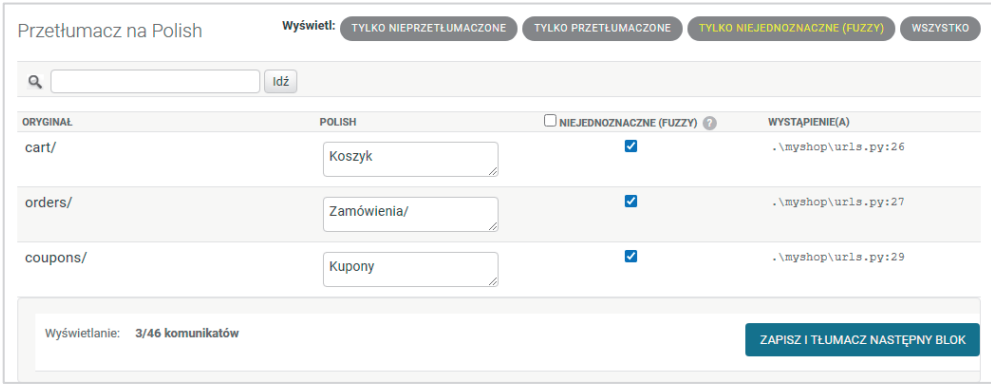
Dodaj dla każdego adresu URL inny ciąg tłumaczenia. Nie zapomnij umieścić znaku ukośnika / na końcu każdego adresu URL, jak pokazałem na rysunku 11.6.



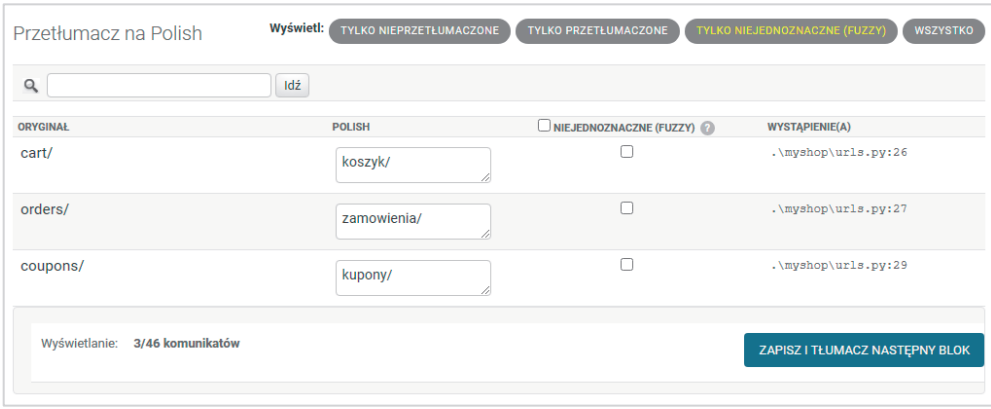
Rysunek 11.6. Polskie tłumaczenia wzorców adresów URL w interfejsie Rosetta

Po zakończeniu kliknij *ZAPISZ I TŁUMACZ NASTĘPNY BLOK*. Następnie kliknij *TYLKO NIEJEDNOZNACZNE (FUZZY)*. Wyświetlą się tłumaczenia, które zostały oznaczone jako rozmyte, ponieważ zostały sparowane ze starym tłumaczeniem podobnego oryginalnego ciągu znaków. W przypadku pokazanym na rysunku 11.7 tłumaczenia są błędne i należy je poprawić.

Wprowadź poprawny tekst tłumaczeń rozmytych. Podczas wprowadzania nowego tekstu do tłumaczenia Rosetta automatycznie anuluje zaznaczenie pola wyboru *FUZZY*. Po zakończeniu kliknij *ZAPISZ I TŁUMACZ NASTĘPNY BLOK* (rysunek 11.8).



Rysunek 11.7. Rozmyte tłumaczenia w interfejsie Rosetta



Rysunek 11.8. Poprawianie rozmytych tłumaczeń w interfejsie Rosetta

Możesz teraz wrócić do strony `http://127.0.0.1:8000/en/rosetta/files/third-party/` i edytować również angielskie tłumaczenie aplikacji `orders`.

Po przetłumaczeniu ciągów znaków na język angielski budowana aplikacja będzie dostępna w dwóch językach. Wcześniej dowiedziałeś się, w jaki sposób Django określa bieżący język. Użytkownicy mogą jednak chcieć zmienić język.

Stwórzmy funkcjonalność, która pozwoli użytkownikom na zmianę ich preferencji językowych.

Umożliwienie użytkownikowi zmiany języka

Ponieważ oferujemy treść dostępną w wielu językach, konieczne jest umożliwienie użytkownikowi zmiany języka używanego w witrynie internetowej. Dodamy teraz tę możliwość do budowanej witryny. Selektor wyboru języka będzie składał się z listy dostępnych języków wyświetlonych w postaci linków.

Przeprowadź edycję szablonu *shop/base.html* aplikacji `shop` i odszukaj poniższe wiersze kodu:

```
<div id="header">
  <a href="/" class="logo">{% translate "Mój sklep " %}</a>
</div>
```

Zastąp je poniższym fragmentem kodu.

```
<div id="header">
  <a href="/" class="logo">{% translate "Mój sklep " %}</a>
  {% get_current_language as LANGUAGE_CODE %}
  {% get_available_languages as LANGUAGES %}
  {% get_language_info_list for LANGUAGES as languages %}
  <div class="languages">
    <p>{% translate "Język" %}:</p>
    <ul class="languages">
      {% for language in languages %}
        <li>
          <a href="/{{ language.code }}/"
          {% if language.code == LANGUAGE_CODE %} class="selected"{% endif %}>
            {{ language.name_local }}
          </a>
        </li>
      {% endfor %}
    </ul>
  </div>
</div>
```

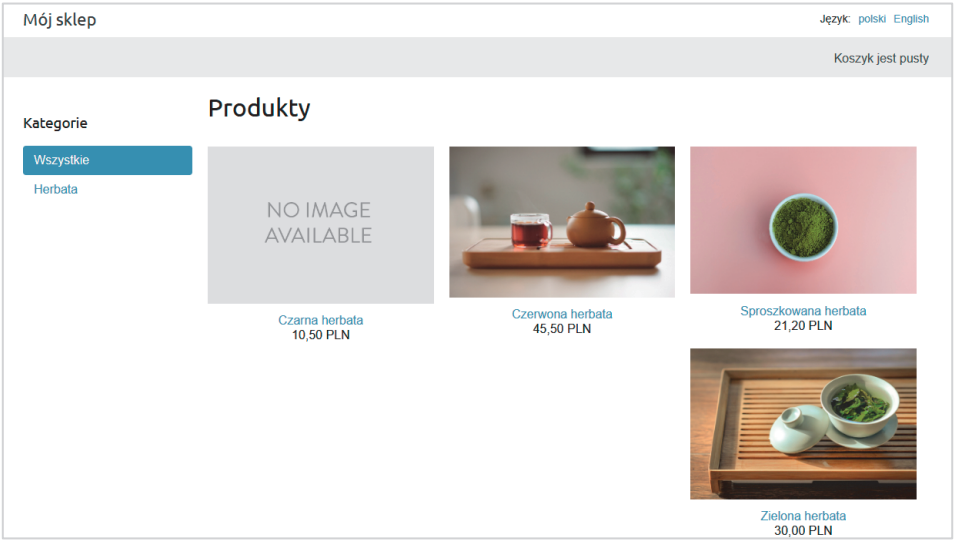
Pamiętaj, aby żaden znacznik szablonu nie był rozdzielony na wiele wierszy.

Oto sposób tworzenia sekcji wyboru języka:

1. Za pomocą tagu `{% load i18n %}` wczytujemy znaczniki przeznaczone do obsługi internacjonalizacji.
2. Do pobrania bieżącego języka używamy znacznika `{% get_current_language %}`.
3. Języki zdefiniowane na liście `LANGUAGES` pobieramy za pomocą znacznika szablonu `{% get_available_languages %}`.
4. Z kolei znacznik `{% get_language_info_list %}` umożliwia bardzo łatwe zapewnienie dostępu do atrybutów języka.
5. Tworzymy listę HTML wyświetlającą wszystkie dostępne języki, a następnie dodajemy klasę `selected` do elementu wskazującego aktualnie aktywny język.

W kodzie sekcji wyboru języka wykorzystujemy dostarczane przez `i18n` znaczniki szablonu na podstawie języków udostępnionych w ustawieniach projektu. Teraz w przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/` i spójrz na wyświetloną stronę. W prawym górnym rogu powinieneś zobaczyć sekcję wyboru języka (rysunek 11.9).

Od tego momentu użytkownicy mogą płynnie przełączać się na preferowany język przez wybranie jednej z opcji dostępnych w selektorze języków.



Rysunek 11.9. Strona listy produktów z selektorem języka w nagłówku

Uwaga

Ilustracje wykorzystane w tym rozdziale:
Zielona herbata: Jia Ye, zdjęcie z serwisu Unsplash.
Czerwona herbata: Manki Kim, zdjęcie z serwisu Unsplash.
Herbata sproszkowana: Phuong Nguyen, zdjęcie z serwisu Unsplash.

Tłumaczenie modeli za pomocą django-parler

Django nie oferuje standardowo rozwiązania pozwalającego na tłumaczenie modeli. Konieczne jest więc zaimplementowanie własnego rozwiązania do zarządzania przechowywaną treścią w różnych językach lub też użycie do tłumaczenia modeli zewnętrznego modułu. Istnieje kilka aplikacji zewnętrznych pozwalających na tłumaczenie kolumn modeli. Jedną z tego rodzaju aplikacji jest `django-parler`. Moduł oferuje niezwykle efektywny sposób tłumaczenia modeli i elegancko integruje się z witryną administracyjną Django.

Moduł `django-parler` generuje oddzielną tabelę bazy danych dla każdego modelu zawierającego tłumaczenia. Tabela ta zawiera wszystkie przetłumaczone pola oraz klucz zewnętrzny do pierwotnego obiektu, do którego należy tłumaczenie. Ponadto w tabeli znajduje się pole określające język, ponieważ każdy rekord przechowuje treść dla tylko jednego języka.

Uwaga

Pakiet `django-parler` od kilku lat nie był aktualizowany. Mimo to wielu programistów nadal z niego korzysta ze względu na jego sprawdzoną skuteczność w ułatwianiu tłumaczenia modeli.

Instalacja django-parler

Aby zainstalować moduł `django-parler` za pomocą menedżera `pip`, wydaj poniższe polecenie.

```
python -m pip install django-parler==2.3
```

Następnie przeprowadź edycję pliku *settings.py* projektu i umieść `'parler'` na liście `INSTALLED_APPS` tak, jak pokazałem poniżej.

```
INSTALLED_APPS = [
    # ...
    'parler',
]
```

Ponadto dodaj poniższy fragment kodu do pliku *settings.py*.

```
# ustawienia django-parler
PARLER_LANGUAGES = {
    None: (
        {'code': 'en'},
        {'code': 'pl'},
    ),
    'default': {
        'fallback': 'pl',
        'hide_untranslated': False,
    }
}
```

Powyższe ustawienia definiują języki `en` i `pl` jako dostępne dla `django-parler`. Określamy `pl` jako język domyślny oraz nakazujemy modułowi wyświetlanie nieprzetłumaczonej treści.

Parler jest teraz aktywny w budowanym projekcie. Dodajmy możliwość tłumaczenia do pól modelu.

Tłumaczenie pól modelu

Przystępujemy teraz do przetłumaczenia katalogu produktów. Moduł `django-parler` dostarcza klasę modelu `TranslatableModel` i opakowanie `TranslatedField` przeznaczone do tłumaczenia kolumn modelu. Aby to zrobić, wykonaj działania opisane poniżej:

Przeprowadź edycję pliku *models.py* w katalogu aplikacji `shop` i dodaj poniższe polecenie `import`.

```
from parler.models import TranslatableModel, TranslatedFields
```

Teraz zmień model `Category`, aby kolumny `name` i `slug` zostały oznaczone jako kolumny do przetłumaczenia.

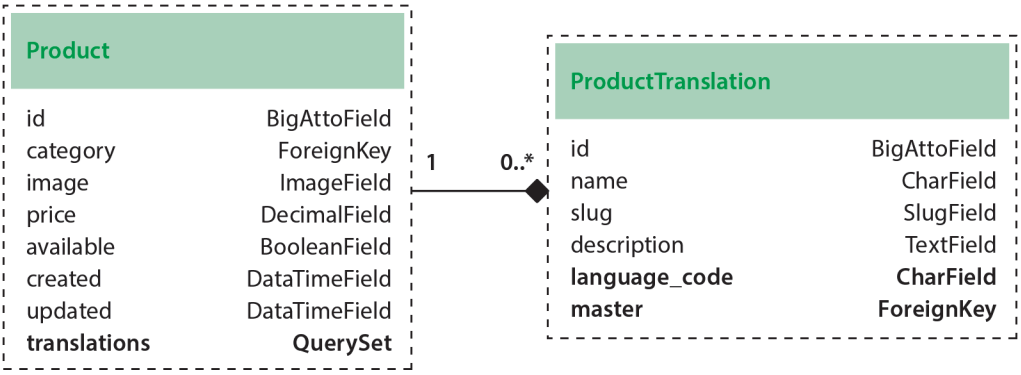
```
class Category(TranslatableModel):
    translations = TranslatedFields(
        name = models.CharField(max_length=200),
        slug = models.SlugField(max_length=200, unique=True),
    )
```

Model `Category` dziedziczy teraz po `TranslatableModel` zamiast po `models.Model`, a kolumny `name` i `slug` zostały umieszczone w opakowaniu `TranslatedFields`.

Przeprowadź edycję modelu `Product`, dodając tłumaczenia dla kolumn `name`, `slug` i `description`, tak jak pokazałem poniżej.

```
class Product(TranslatableModel):
    translations = TranslatedFields(
        name = models.CharField(max_length=200),
        slug = models.SlugField(max_length=200),
        description = models.TextField(blank=True)
    )
    category = models.ForeignKey(
        Category,
        related_name='products',
        on_delete=models.CASCADE
    )
    image = models.ImageField(
        upload_to='products/%Y/%m/%d',
        blank=True
    )
    price = models.DecimalField(max_digits=10, decimal_places=2)
    available = models.BooleanField(default=True)
    created = models.DateTimeField(auto_now_add=True)
    updated = models.DateTimeField(auto_now=True)
```

Moduł `django-parler` zarządza tłumaczeniami dzięki wygenerowaniu oddzielnego modelu dla każdego modelu oznaczonego do przetłumaczenia. Na rysunku poniżej możesz zobaczyć pola modelu `Product` i wygenerowany dla nich model `ProductTranslation` (rysunek 11.10).



Rysunek 11.10. Model Product i powiązany z nim model ProductTranslation wygenerowany przez moduł django-parler

Model `ProductTranslation` wygenerowany przez moduł `django-parler` zawiera oznaczone do przetłumaczenia pola `name`, `slug` i `description` oraz `language_code` i `ForeignKey` dla pierwotnego obiektu `Product`. Mamy tutaj relację typu „jeden do wielu" od modelu `Product` do `ProductTranslation`. Obiekt `ProductTranslation` będzie istniał dla każdego dostępnego języka poszczególnych obiektów `Product`.

Ponieważ w Django używa się oddzielnych tabel dla tłumaczeń, z pewnych funkcji frameworka nie można skorzystać. Przykładowo nie ma możliwości użycia domyślnej kolejności według przetłumaczonych kolumn. Zapytania można filtrować według przetłumaczonych kolumn, ale nie można ich umieszczać w opcjach `ordering` klasy `Meta`. Dla tłumaczonych pól nie można również używać indeksów. Pola te nie istnieją w oryginalnym modelu, ponieważ są umieszczone w modelu tłumaczenia.

Przeprowadź edycję pliku *models.py* aplikacji `shop` i umieść znak komentarza na początku atrybutów `ordering` i `indexes` w klasie `Meta` modelu `Category`.

```
class Category(TranslatableModel):
    # ...
    class Meta:
        # ordering = ['name']
        # indexes = [
        #    models.Index(fields=['name']),
        # ]
        verbose_name = 'category'
        verbose_name_plural = 'categories'
```

Konieczne jest również umieszczenie znaku komentarza na początku wierszy `ordering` i `index` klasy `Meta` modelu `Product` odwołujących się do tłumaczonych pól. Oznacz jako komentarz następujące wiersze klasy `Meta` modelu `Product`.

```
class Product(TranslatableModel):
    # ...
    class Meta:
        # ordering = ['name']
        indexes = [
            # models.Index(fields=['id', 'slug']),
            # models.Index(fields=['name']),
            models.Index(fields=['-created']),
        ]
```

Więcej informacji na temat zgodności modułu `django-parler` z frameworkiem Django znajdziesz na stronie *http://django-parler.readthedocs.org/en/latest/compatibility.html*.

W następnym punkcie pokażę, jak zintegrować przetłumaczalne modele z witryną administracyjną.

Integracja tłumaczeń w witrynie administracyjnej

Moduł `django-parler` integruje się płynnie z witryną administracyjną Django. Pozwala to łatwo edytować tłumaczenia różnych obiektów za pomocą wygodnego interfejsu administratora. Zawiera klasę `TranslatableAdmin`, która przesłania udostępnioną przez Django klasę `ModelAdmin` i służy do zarządzania tłumaczeniami modelu.

Zmodyfikuj plik *admin.py* aplikacji `shop` i dodaj do niego instrukcję importu:

```
from parler.admin import TranslatableAdmin
```

Zmodyfikuj klasy `CategoryAdmin` i `ProductAdmin`, aby dziedziczyły po klasie `TranslatableAdmin` zamiast po klasie `ModelAdmin`. Moduł `django-parler` nie obsługuje atrybutu `prepopulated_fields`, ale obsługuje metodę `get_prepopulated_fields()`, która zapewnia taką samą funkcjonalność. Spróbujmy to odpowiednio zmodyfikować. Przeprowadź edycję pliku *admin.py* tak, aby miał następującą zawartość.

```
from django.contrib import admin
from parler.admin import TranslatableAdmin
from .models import Category, Product

@admin.register(Category)
class CategoryAdmin(TranslatableAdmin):
    list_display = ['name', 'slug']

    def get_prepopulated_fields(self, request, obj=None):
        return {'slug': ('name',)}

@admin.register(Product)
class ProductAdmin(TranslatableAdmin):
    list_display = [
        'name',
        'slug',
        'price',
        'available',
        'created',
        'updated'
    ]
    list_filter = ['available', 'created', 'updated']
    list_editable = ['price', 'available']

    def get_prepopulated_fields(self, request, obj=None):
        return {'slug': ('name',)}
```

Właśnie zaadaptowaliśmy witrynę administracyjną do pracy z nowymi przetłumaczonymi modelami. Możemy teraz zsynchronizować bazę danych ze zmianami modelu, które wprowadziliśmy.

Utworzenie migracji dla tłumaczeń modeli

Aby utworzyć migracje, postępuj zgodnie z poniższymi instrukcjami.

Otwórz powłokę i uruchom poniższe polecenie tworzące nową migrację dla tłumaczeń modelu.

```
python manage.py makemigrations shop --name "translations"
```

Wyświetli się następujący wynik.

```
Migrations for 'shop':
shop/migrations/0002_translations.py
- Create model CategoryTranslation
- Create model ProductTranslation
- Change Meta options on category
```

```
- Change Meta options on product
- Remove index shop_catego_name_289c7e_idx from category
- Remove index shop_produc_id_f21274_idx from product
- Remove index shop_produc_name_a2070e_idx from product
- Remove field name from category
- Remove field slug from category
- Remove field description from product
- Remove field name from product
- Remove field slug from product
- Add field master to producttranslation
- Add field master to categorytranslation
- Alter unique_together for producttranslation (1 constraint(s))
- Alter unique_together for categorytranslation (1 constraint(s))
```

Migracja automatycznie obejmuje modele `CategoryTranslation` i `ProductTranslation` utworzone dynamicznie przez moduł `django-parler`. Należy zapamiętać, że ta migracja usuwa z modeli wcześniejsze pola.

Oznacza to, że po jej uruchomieniu utracimy zapisane w nich dane i będziemy musieli ponownie ustawić kategorie i produkty w witrynie administracyjnej.

Przeprowadź edycję pliku *migrations/0002_translations.py* aplikacji `shop` i zastąp dwa wystąpienia wiersza:

```
bases=(parler.models.TranslatedFieldsModelMixin, models.Model),
```

takim:

```
bases=(parler.models.TranslatableModel, models.Model),
```

Jest to poprawka niewielkiego błędu w używanej w tym projekcie wersji modułu `django-parler`. Zmiana ta jest konieczna, aby nie dopuścić do niepowodzenia migracji podczas jej stosowania. Problem jest związany z tworzeniem tłumaczeń dla istniejących pól w modelu i prawdopodobnie zostanie poprawiony w nowszych wersjach modułu `django-parler`.

Aby zastosować migrację, uruchom polecenie:

```
python manage.py migrate shop
```

Wyświetli się wynik zakończony wierszem:

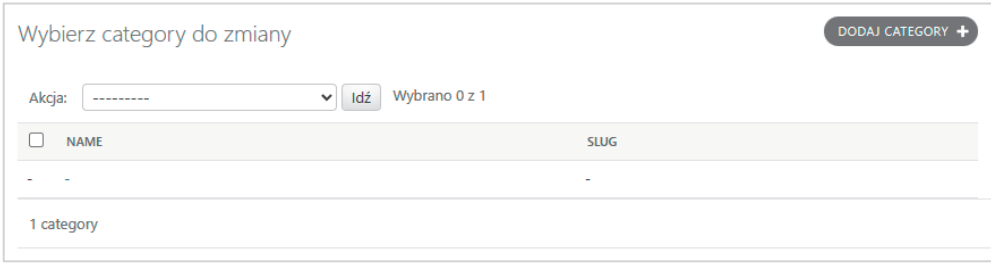
```
Applying shop.0002_ categorytranslation_producttranslation_and_more... OK
```

Modele w budowanym projekcie są teraz zsynchronizowane z bazą danych.

Uruchom serwer programistyczny za pomocą polecenia:

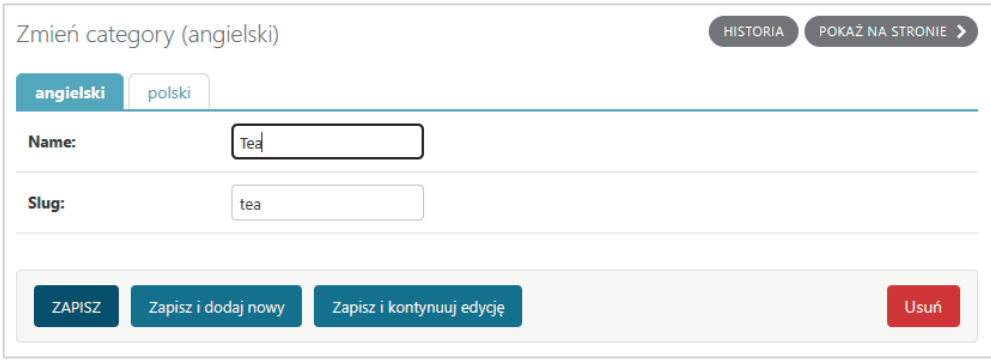
```
python manage.py runserver
```

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/pl/admin/shop/category/`. Zobaczysz, że ze względu na usunięcie pól i użycie zamiast nich modeli do tłumaczenia wygenerowanych przez moduł `django-parler` istniejące kategorie utraciły nazwę i slug. W każdym z tych pól wyświetli się łącznik (rysunek 11.11).



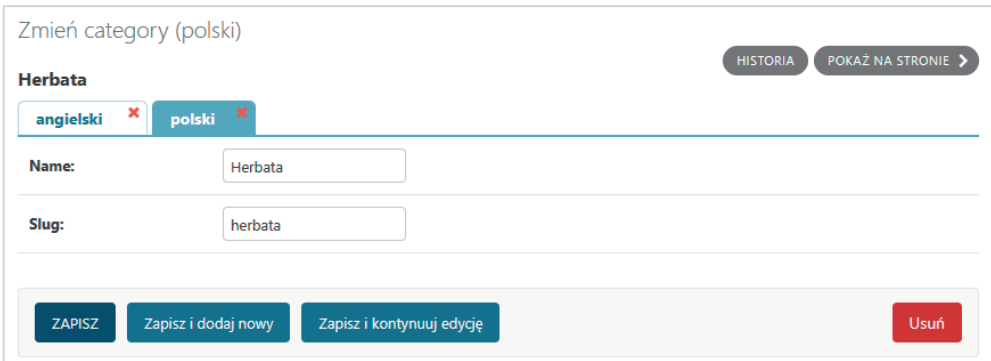
Rysunek 11.11. Lista kategorii na stronie administracyjnej Django po utworzeniu modeli tłumaczeń

Kliknij łącznik pod nazwą kategorii, aby ją edytować. Zobaczysz, że strona *Zmień category (angielski)* zawiera dwie różne karty, jedną dla języka angielskiego i jedną dla polskiego (rysunek 11.12).



Rysunek 11.12. Formularz edycji kategorii włącznie z kartami języków dodanymi przez moduł django-parler

Wprowadź nazwę i slug dla wszystkich istniejących kategorii. Podczas modyfikowania kategorii, wprowadź szczegóły w języku angielskim, po czym kliknij *Zapisz i kontynuuj edycję*. Następnie kliknij *Polski*, dodaj polskie tłumaczenie pól i kliknij *ZAPISZ* (rysunek 11.13).



Rysunek 11.13. Polskie tłumaczenie formularza edycji kategorii

Pamiętaj, aby przed przełączeniem między zakładkami języka zapisać zmiany. Po uzupełnieniu danych dla istniejących kategorii otwórz w przeglądarce stronę `http://127.0.0.1:8000/en/admin/shop/product/` i zmodyfikuj wszystkie produkty przez wprowadzenie angielskich i polskich wersji nazwy, slugu i opisu.

Po przygotowaniu tłumaczeń pokażę, jak wchodzić w interakcje z przetłumaczonymi polami za pośrednictwem Django ORM.

Korzystanie z tłumaczeń w kolekcjach QuerySet

Przyjrzyjmy się, jak korzystać z tłumaczeń w kolekcjach `QuerySet`. Aby otworzyć powłokę Pythona, z poziomu powłoki wydaj polecenie.

```
python manage.py shell
```

Zwróć uwagę, w jaki sposób możesz wykonywać zapytania i pobierać dane z przetłumaczonych pól. Aby pobrać zawartość kolumny dla aktualnie aktywnego języka, można skorzystać z funkcji Django `activate()`, tak jak pokazałem poniżej.

```
>>> from shop.models import Product
>>> from django.utils.translation import activate
>>> activate('en')
>>> product=Product.objects.first()
>>> product.name
'Green tea'
```

Innym sposobem jest skorzystanie z menedżera `language()` dostarczonego przez moduł `django-parler`. Można to zrobić tak, jak pokazałem niżej.

```
>>> product=Product.objects.language('pl').first()
>>> product.name
'Zielona herbata'
```

Podczas uzyskiwania dostępu do przetłumaczonych pól używany jest aktualnie wybrany język. Oczywiście istnieje możliwość ustawienia innego języka dla obiektu, aby tym samym uzyskać dostęp do określonego tłumaczenia.

```
>>> product.set_current_language('en')
>>> product.name
'Black tea'
>>> product.get_current_language()
'en'
```

Podczas wykonywania zapytania `QuerySet` za pomocą metody `filter()` można przeprowadzić filtrowanie powiązanych obiektów tłumaczenia z użyciem składni `translations__`, tak jak pokazałem poniżej.

```
>>> Product.objects.filter(translations__name='Zielona herbata')
<TranslatableQuerySet [<Product: Green tea>]>
```

Zastosujmy zdobytą wiedzę w widokach aplikacji.

Adaptacja widoków dla tłumaczeń

Przystępujemy teraz do adaptacji widoków katalogu produktów.

Przeprowadź edycję pliku *views.py* aplikacji `shop` i umieść w widoku `product_list` poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
def product_list(request, category_slug=None):
    category = None
    categories = Category.objects.all()
    products = Product.objects.filter(available=True)
    if category_slug:
        language = request.LANGUAGE_CODE
        category = get_object_or_404(
            Category,
            translations__language_code=language,
            translations__slug=category_slug
        )
        products = products.filter(category=category)
    return render(
        request,
        'shop/product/list.html',
        {
            'category': category,
            'categories': categories,
            'products': products
        }
    )
```

Następnie przeprowadź edycję widoku `product_detail` i dodaj następujący kod wyróżniony pogrubioną czcionką.

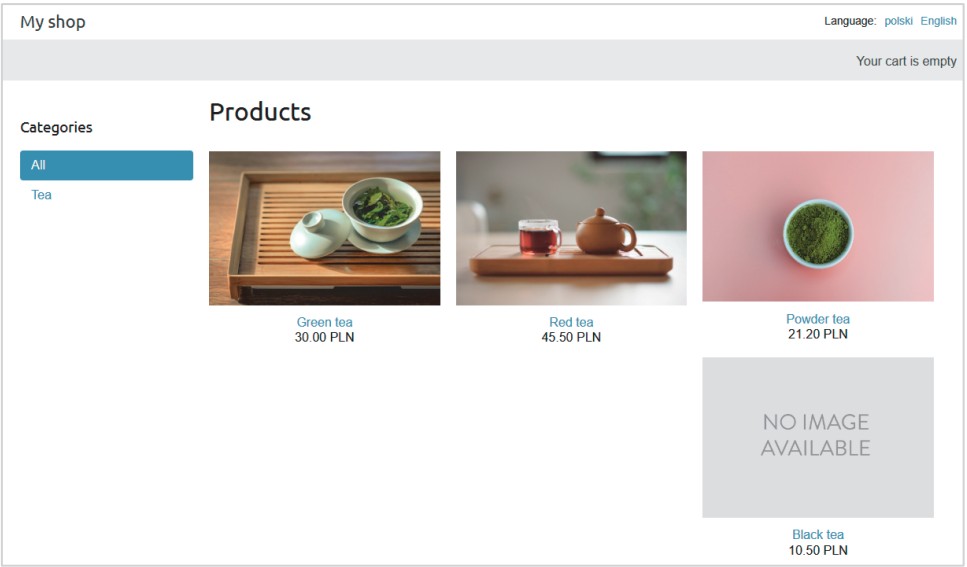
```
def product_detail(request, id, slug):
    language = request.LANGUAGE_CODE
    product = get_object_or_404(
        Product,
        id=id,
        translations__language_code=language,
        translations__slug=slug,
        available=True
    )
    cart_product_form = CartAddProductForm()
    r = Recommender()
    recommended_products = r.suggest_products_for([product], 4)
    return render(
        request,
        'shop/product/detail.html',
        {
            'product': product,
            'cart_product_form': cart_product_form,
            'recommended_products': recommended_products
        }
    )
```

Widoki `product_list` i `product_detail` są teraz zaadaptowane do pobierania obiektów za pomocą przetłumaczonych pól.

Uruchom serwer programistyczny za pomocą polecenia:

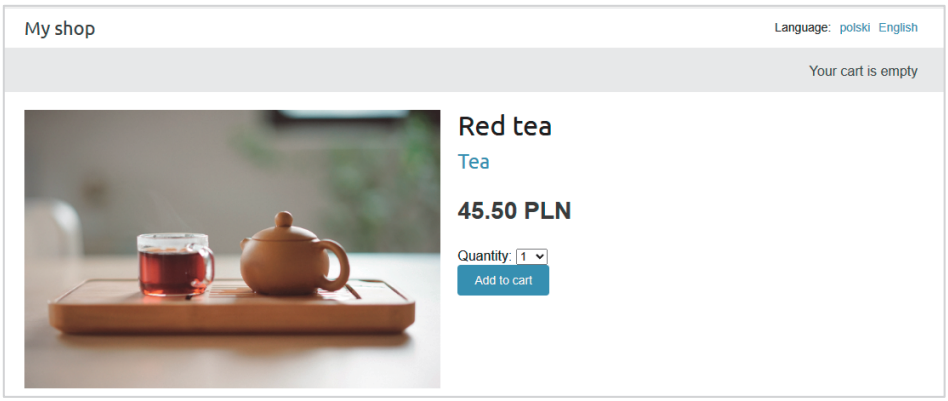
```
python manage.py runserver
```

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/en/`. Powinieneś zobaczyć, że strona zawierająca listę produktów oraz nazwy samych produktów zostały przetłumaczone na język angielski, co pokazałem na poniższym rysunku (rysunek 11.14).



Rysunek 11.14. Angielska wersja strony z listą produktów

Teraz adres URL każdego produktu jest tworzony za pomocą kolumny `slug` przetłumaczonej na bieżący język. Przykładowo adres URL dla produktu wyświetlanego na stronie w języku polskim ma postać `http://127.0.0.1:8000/pl/1/zielona-herbata/`, podczas gdy adres URL tego samego produktu na stronie w języku angielskim ma postać `http://127.0.0.1:8000/en/1/green-tea/`. Gdy przejdziesz na stronę szczegółów danego produktu, zobaczysz przetłumaczony nie tylko adres URL, ale również treść strony (rysunek 11.15).



Rysunek 11.15. Angielska wersja strony szczegółów produktu

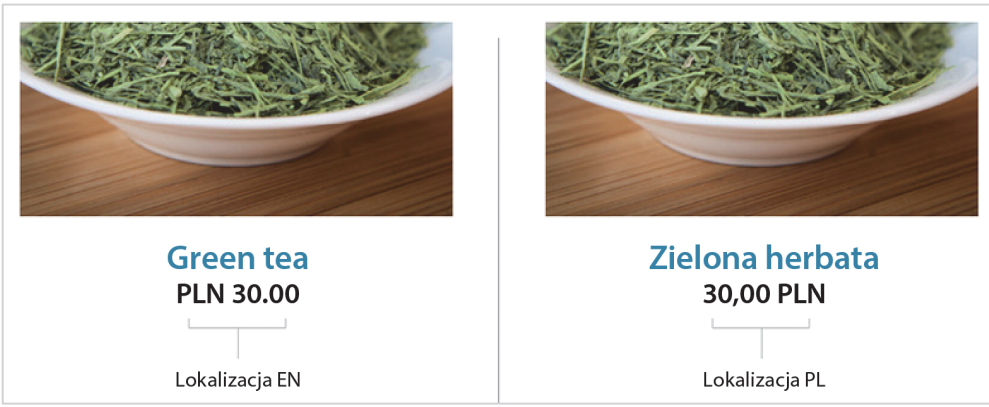
Pełną dokumentację modułu `django-parler` znajdziesz na stronie *http://django-parler.readthedocs.org/en/latest/*.

Dowiedziałeś się, jak przetłumaczyć kod Pythona, szablony, wzorce adresów URL i kolumny modeli. Aby dokończyć proces internacjonalizacji i lokalizacji, konieczne jest jeszcze zapewnienie wyświetlania odpowiednio sformatowanych i zlokalizowanych dat, godzin oraz liczb.

Lokalizacja formatów

Aby zwiększyć wygodę pracy użytkownika, należy prezentować daty, godziny i liczby w formatach zgodnych z lokalizacją użytkownika. Dostosowanie witryny do formatów danych znanych użytkownikom w różnych regionach znacznie poprawia dostępność aplikacji. Począwszy od wersji Django 5.0 zlokalizowane formatowanie danych jest zawsze włączone. Django wyświetla liczby i daty z wykorzystaniem formatu bieżącego ustawienia `locale`.

Django próbuje zlokalizować odpowiedni format podczas umieszczania wartości w szablonie. Na rysunku 11.16 pokazałem lokalizację formatu liczb dziesiętnych w językach angielskim i polskim.



Rysunek 11.16. Lokalizacja formatu w językach angielskim i polskim

Jak możesz zobaczyć, w angielskiej wersji witryny separatorem dziesiętnym jest kropka, w polskiej zaś przecinek. Zmiany te wynikają z formatu lokalizacji wskazanego przez framework dla lokalizacji `pl` i `en`. Konfigurację formatowania w języku angielskim można znaleźć pod adresem *https://github.com/django/django/blob/stable/5.0.x/django/conf/locale/en/formats.py*, a informacje dla języka polskiego są dostępne pod adresem *https://github.com/django/django/blob/stable/5.0.x/django/conf/locale/pl/formats.py*.

Domyślnie Django stosuje lokalizację formatu dla każdego ustawienia lokalizacji. Jednak mogą się zdarzyć sytuacje, w których nie chcesz korzystać ze zlokalizowanych wartości. Przykładem może być wyświetlanie danych JavaScript lub JSON, które mają być w formacie czytelnym dla komputera.

Django oferuje znacznik szablonu {% localize %} pozwalający na włączenie lub wyłączenie lokalizacji dla wskazanych fragmentów szablonu. Korzystając z tego znacznika, zyskujesz kontrolę nad zlokalizowanym formatowaniem. Przed skorzystaniem ze znacznika {% localize %} trzeba wczytać do szablonu znaczniki l10n. Poniżej przedstawiłem przykład włączenia i wyłączenia lokalizacji w szablonie.

```
{% load l10n %}

{% localize on %}
  {{ value }}
{% endlocalize %}

{% localize off %}
  {{ value }}
{% endlocalize %}
```

Django oferuje również filtry szablonów localize i unlocalize wymuszające użycie bądź uniknięcie lokalizacji wartości. Filtry można stosować następująco.

```
{{ value|localize }}
{{ value|unlocalize }}
```

Istnieje możliwość przygotowania własnych plików formatu w celu wskazania sposobu formatowania lokalizacji. Więcej informacji na temat formatu lokalizacji znajdziesz na stronie *https://docs.djangoproject.com/en/5.0/topics/i18n/formatting/*.

W następnym podrozdziale dowiesz się, jak tworzyć zlokalizowane pola formularza.

Użycie modułu django-localflavor do weryfikacji pól formularza

Moduł django-localflavor to opracowany przez firmę zewnętrzną moduł zawierający kolekcję określonych narzędzi, takich jak pola formularza lub modelu specyficzne dla poszczególnych krajów. Moduł jest niezwykle użyteczny podczas weryfikacji ustawień regionalnych, lokalnych formatów numerów telefonów, numerów dokumentów identyfikacyjnych, numerów ubezpieczeń społecznych itd. Pakiet jest zorganizowany w postaci serii modułów o nazwach zgodnych z kodami krajów według normy ISO 3166. Aby skonfigurować moduł, postępuj zgodnie z poniższymi instrukcjami:

Żeby zainstalować moduł django-localflavor, wydaj poniższe polecenie.

```
python -m pip install django-localflavor==4.0
```

Przeprowadź edycję pliku *settings.py* projektu i umieść 'localflavor' na liście INSTALLED_
↪APPS tak, jak pokazałem poniżej.

```
INSTALLED_APPS = [
    # …
    'localflavor',
]
```

Przykładowo dodamy polecenie sprawdzające format kodu pocztowego. Podanie prawidłowego kodu pocztowego jest wymagane do utworzenia nowego zamówienia. Przeprowadź edycję pliku *forms.py* aplikacji `orders`, aby przedstawiał się w następujący sposób.

```
from django import forms
from localflavor.pl.forms import PLPostalCodeField
from .models import Order

class OrderCreateForm(forms.ModelForm):
    postal_code = PLPostalCodeField()
    class Meta:
        model = Order
        fields = [
            'first_name',
            'last_name',
            'email',
            'address',
            'postal_code',
            'city'
        ]
```

Z pakietu `pl` modułu `django-localflavor` importujemy pole `PLPostalCodeField` i używamy go dla pola `postal_code` w formularzu `OrderCreateForm`.

Uruchom serwer programistyczny za pomocą polecenia:

```
python manage.py runserver
```

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/pl/zamowienia/create/`. Wypełnij wszystkie pola, wprowadź trzyliterowy kod pocztowy, a następnie wyślij formularz. Otrzymasz komunikat o błędzie wygenerowany podczas weryfikacji pola typu `PLPostalCodeField`.

```
Wpisz kod pocztowy w formacie XX-XXX.
```

Błąd sprawdzania poprawności formularza jest pokazany na rysunku 11.17.

Rysunek 11.17. Błąd sprawdzania poprawności formatu kodu pocztowego w Polsce

Jest to jedynie krótki przykład pokazujący, jak można w celu weryfikacji danych używać we własnych projektach niestandardowych pól modułu `django-localflavor`. Komponenty lokalne dostarczane przez wymieniony moduł są niezwykle użyteczne podczas adaptacji aplikacji dla określonych krajów. Dokumentację modułu `django-localflavor` i omówienie wszystkich dostępnych komponentów lokalnych dla poszczególnych krajów znajdziesz na stronie *https://django-localflavor.readthedocs.io/en/latest/*.

Rozszerzanie projektu przy użyciu sztucznej inteligencji

W tym podrozdziale zaprezentuję zadanie rozszerzenia projektu, do którego wykorzystam przykładowe zapytanie do ChatGPT. ChatGPT jest dostępny pod adresem *https://chat.openai.com/*. Jeśli jest to pierwsza interakcja z ChatGPT, możesz powrócić do punktu „Rozszerzanie projektu z wykorzystaniem sztucznej inteligencji" w rozdziale 3., „Rozbudowa aplikacji bloga".

W tym przykładzie projektu zaimplementowaliśmy sklep internetowy. Dodaliśmy zamówienia, płatności i system kuponów. Kolejną typową funkcjonalnością platform e-commerce jest zarządzanie kosztami wysyłki. Rozważmy dodanie do produktów atrybutu wagi i zaimplementowanie kosztów wysyłki na podstawie całkowitej wagi wysyłanych produktów. Wykorzystaj ChatGPT, aby zaimplementować koszty wysyłki produktów w zależności od wagi produktów. Sprawdź, czy serwis Stripe pobiera poprawną kwotę, włącznie z obliczonymi kosztami wysyłki. W tym celu możesz wykorzystać zapytanie dostępne pod adresem *https://github.com/PacktPublishing/Django-5-by-example/blob/main/Chapter11/prompts/task.md*.

Wskazówka

Korzystaj z ChatGPT jako pomoc podczas debugowania. Jeśli utkniesz na szczególnie upartym błędzie, opisz problem i kontekst. Dzięki ChatGPT możesz uzyskać świeżą perspektywę, często skłaniającą do rozważenia aspektów, które mogłeś przeoczyć. Dzięki temu szybciej rozwiążesz występujące problemy.

Podsumowanie

W tym rozdziale poznałeś podstawy internacjonalizacji i lokalizacji projektów Django. Oznaczyłeś kody i ciągi szablonów do tłumaczenia i dowiedziałeś się, jak wygenerować i skompilować pliki z tłumaczeniami. Zainstalowałeś również w projekcie system Rosetta w celu zarządzania tłumaczeniami za pośrednictwem interfejsu w przeglądarce. Przetłumaczyłeś wzorce adresów URL i utworzyłeś selektor języka, z pomocą którego użytkownik może przełączać język witryny. Następnie skorzystałeś z modułu `django-parler` w celu przetłumaczenia modeli oraz zastosowałeś moduł `django-localflavor`, aby zweryfikować zlokalizowane pola formularza.

W następnym rozdziale rozpoczniesz nowy projekt Django, który będzie składał się z platformy e-learningowej. Dowiesz się, jak korzystać z dziedziczenia modeli w celu zaimplementowania polimorfizmu i stworzysz podstawy elastycznego systemu zarządzania treścią. Utworzysz modele aplikacji i nauczysz się, jak tworzyć i stosować fikstury, aby dostarczyć dla modeli początkowe dane. Nauczysz się budować niestandardowe pola i używać ich w swoich modelach. Zbudujesz również widoki uwierzytelniania dla swojej nowej aplikacji.

Dodatkowe zasoby

Oto lista zasobów zawierających dodatkowe informacje związane z tematami omawianymi w tym rozdziale:

- Kod źródłowy przykładów zamieszczonych w tym rozdziale — *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter11*.
- Lista prawidłowych identyfikatorów języków — *http://www.i18nguy.com/unicode/language-identifiers.html*.
- Lista ustawień internacjonalizacji i lokalizacji — *https://docs.djangoproject.com/en/5.0/ref/settings/#globalization-i18n-l10n*.
- Menedżer pakietów Homebrew — *https://brew.sh/*.
- Instalowanie `gettext` w systemie Windows — *https://docs.djangoproject.com/en/5.0/topics/i18n/translation/#gettext-on-windows*.
- Skompilowany binarny instalator `gettext` dla systemu Windows — *https://mlocati.github.io/articles/gettext-iconv-windows.html*.
- Dokumentacja dotycząca tłumaczeń — *https://docs.djangoproject.com/en/5.0/topics/i18n/translation/*.
- Edytor plików tłumaczeń Poedit — *https://poedit.net/*.
- Dokumentacja systemu Rosetta dla frameworka Django — *https://django-rosetta.readthedocs.io/*.
- Informacje na temat zgodności modułu `django-parler` z frameworkiem Django — *https://django-parler.readthedocs.io/en/latest/compatibility.html*.
- Dokumentacja modułu `django-parler` — *https://django-parler.readthedocs.io/en/latest/*.
- Konfiguracja formatowania Django dla polskich ustawień regionalnych — *https://github.com/django/django/blob/stable/5.0.x/django/conf/locale/pl/formats.py*.
- Konfiguracja formatowania Django dla angielskich ustawień regionalnych — *https://github.com/django/django/blob/stable/5.0.x/django/conf/locale/en/formats.py*.
- Lokalizacja formatów w Django — *https://docs.djangoproject.com/en/5.0/topics/i18n/formatting/*.
- Dokumentacja modułu `django-localflower` — *https://django-localflavor.readthedocs.io/en/latest/*.

Budowa platformy e-learningu

Rozdział 12

W poprzednim rozdziale poznałeś podstawy internacjonalizacji i lokalizacji projektów Django. Dostosowałeś swój projekt do lokalnych formatów i języków użytkowników.

W tym rozdziale przystąpimy do utworzenia nowego projektu Django — opracujemy platformę e-learningu z własnym **systemem zarządzania treścią** (ang. *content management system* — CMS). Platformy e-learningowe to doskonały przykład aplikacji, w których trzeba zaoferować elastyczne narzędzia do generowania treści. Z tego rozdziału dowiesz się, jak tworzyć elastyczne modele danych, obsługujące różne typy danych, i odkryjesz, jak zaimplementować niestandardowe funkcje modelu, które można zastosować w przyszłych projektach Django.

Oto zagadnienia, na których skoncentruję się w tym rozdziale:

- Utworzenie modeli dla systemu CMS
- Utworzenie fikstur dla modeli i ich zastosowanie
- Wykorzystanie dziedziczenia modelu do tworzenia modeli danych dla polimorficznej zawartości
- Utworzenie własnych pól modelu
- Zawartość i moduły związane z zamawianiem kursów
- Budowa widoków uwierzytelniania dla systemu CMS

Przegląd funkcjonalności

Diagramy na początku poprzednich rozdziałów przedstawiały widoki, szablony i kompleksowe funkcjonalności. W tym rozdziale skupimy się jednak na implementacji dziedziczenia modeli i tworzeniu niestandardowych pól modeli, co nie jest łatwe do zaprezentowania na tego rodzaju diagramach. Zamiast tego konkretne diagramy ilustrujące te pojęcia pojawią się w treści rozdziału.

Kod źródłowy przykładów z tego rozdziału można znaleźć pod adresem *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter12*.

Wszystkie pakiety Pythona użyte w tym rozdziale są zawarte w pliku *requirements.txt* dołączonym do plików z kodem źródłowym przykładów z tego rozdziału. Aby zainstalować każdy pakiet Pythona, możesz postępować zgodnie z instrukcjami zamieszczonymi w poniższych podrozdziałach. Możesz także zainstalować wszystkie wymagania jednocześnie za pomocą polecenia `pip install -r requirements.txt`.

Utworzenie platformy e-learningu

Ostatnim praktycznym projektem przedstawionym w tej książce będzie platforma e-learningu. Na początek, za pomocą poniższego polecenia, utwórz w katalogu *env/* środowisko wirtualne dla nowego projektu.

```
python -m venv env/educa
```

Jeśli używasz systemu Linux lub macOS, aby aktywować środowisko wirtualne, uruchom polecenie:

```
source env/educa/bin/activate
```

Jeśli używasz systemu Windows, użyj polecenia:

```
.\env\educa\Scripts\activate
```

W przygotowanym środowisku wirtualnym zainstaluj framework Django, wydając poniższe polecenie.

```
python -m pip install Django~=5.0.4
```

Ponieważ w projekcie będziemy zarządzać przekazywaniem obrazów, więc konieczne jest zainstalowanie modułu `Pillow` przy użyciu polecenia:

```
python -m pip install Pillow==10.3.0
```

Utwórz nowy projekt za pomocą poniższego polecenia.

```
django-admin startproject educa
```

Wejdź do nowego katalogu `educa` i z zastosowaniem poniższych poleceń utwórz nową aplikację.

```
cd educa
django-admin startapp courses
```

Przeprowadź edycję pliku *settings.py* projektu `educa` i umieść `courses` na liście `INSTALLED_APPS`, tak jak pokazałem poniżej. Nowy kod został wyróżniony pogrubioną czcionką.

```
INSTALLED_APPS = [
    'courses.apps.CoursesConfig',
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
]
```

W tym momencie aplikacja `courses` jest już aktywna w projekcie. W kolejnym podrozdziale przygotujemy projekt do obsługi plików multimedialnych oraz zdefiniujemy modele i treści kursów.

Obsługa plików multimedialnych

Przed stworzeniem modeli i treści kursów przygotujemy projekt do obsługi plików multimedialnych. Za pomocą systemu CMS, który zbudujemy, instruktorzy kursu będą mogli przesyłać do treści kursu pliki multimedialne. Dlatego skonfigurujemy projekt do obsługi plików multimedialnych.

Przeprowadź edycję pliku *settings.py* projektu i dodaj w nim poniższe wiersze kodu.

```
MEDIA_URL = 'media/'
MEDIA_ROOT = BASE_DIR / 'media'
```

Wprowadzenie tych ustawień umożliwi frameworkowi Django zarządzanie przesyłaniem plików i udostępnianiem plików multimedialnych. Opcja `MEDIA_URL` wskazuje bazowy adres URL określający lokalizację przeznaczoną do przechowywania plików multimedialnych przekazywanych przez użytkowników. Natomiast opcja `MEDIA_ROOT` określa lokalną ścieżkę dostępu dla tych plików. Ścieżki i adresy URL plików są budowane dynamicznie przez dodawanie do nich ścieżki projektu lub adresu URL multimediów, co ułatwia przenośność plików.

Teraz przeprowadź edycję pliku głównego *urls.py* projektu `educa` i w następujący sposób zmodyfikuj znajdujący się w nim kod. Nowe wiersze zostały wyróżnione pogrubioną czcionką.

```
from django.conf import settings
from django.conf.urls.static import static
from django.contrib import admin
from django.urls import path

urlpatterns = [
    path('admin/', admin.site.urls),
]

if settings.DEBUG:
    urlpatterns += static(
        settings.MEDIA_URL, document_root=settings.MEDIA_ROOT
    )
```

Do serwowania plików multimedialnych przez serwer programistyczny Django podczas programowania (to znaczy gdy opcja `DEBUG` jest ustawiona na `True`) dodaliśmy funkcję pomocniczą `static()`.

Uwaga

Funkcja pomocnicza `static()` jest odpowiednia do stosowania w środowisku programistycznym, ale na pewno nie w produkcyjnym. Serwowanie statycznych plików przez Django jest bardzo niewydajne. Pamiętaj, aby w środowisku produkcyjnym nigdy nie udostępniać plików statycznych za pomocą Django. Sposób serwowania statycznych plików w środowisku produkcyjnym omówiłem w rozdziale 17., „Wdrożenie".

Projekt jest teraz gotowy do udostępniania plików multimedialnych. Przystępujemy do zdefiniowania modeli oraz treści dla kursów.

Utworzenie modeli kursu

Budowana platforma e-learningu będzie oferować kursy z różnych dziedzin. Każdy kurs zostanie podzielony na konfigurowaną liczbę modułów, natomiast poszczególne moduły będą zawierać konfigurowaną ilość treści różnego rodzaju, czyli tekst, pliki, obrazy i klipy wideo. Poniższy przykład pokazuje, jak powinna wyglądać struktura danych katalogu *course*.

```
Dziedzina 1
  Kurs 1
    Moduł 1
      Treść 1 (obraz)
      Treść 2 (tekst)
    Moduł 2
      Treść 3 (tekst)
      Treść 4 (plik)
      Treść 5 (wideo)
      ...
```

Przystępujemy teraz do budowy modeli kursu. Przeprowadź edycję pliku *models.py* aplikacji `courses` i umieść w nim poniższy fragment kodu.

```
from django.contrib.auth.models import User
from django.db import models

class Subject(models.Model):
    title = models.CharField(max_length=200)
    slug = models.SlugField(max_length=200, unique=True)

    class Meta:
        ordering = ['title']

    def __str__(self):
         return self.title

class Course(models.Model):
    owner = models.ForeignKey(
        User,
        related_name='courses_created',
        on_delete=models.CASCADE
    )
    subject = models.ForeignKey(
        Subject,
        related_name='courses',
        on_delete=models.CASCADE
    )
    title = models.CharField(max_length=200)
    slug = models.SlugField(max_length=200, unique=True)
    overview = models.TextField()
    created = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ['-created']
```

```
        def __str__(self):
            return self.title

    class Module(models.Model):
        course = models.ForeignKey(
            Course, related_name='modules', on_delete=models.CASCADE
        )
        title = models.CharField(max_length=200)
        description = models.TextField(blank=True)

        def __str__(self):
            return self.title
```

W ten sposób przygotowaliśmy początkowe modele `Subject`, `Course` i `Module`. Model `Course` zawiera następujące pola.

- `owner`. Instruktor, który utworzył dany kurs.
- `subject`. Dziedzina, do której zalicza się dany kurs. Pole typu `ForeignKey` prowadzące do modelu `Subject`.
- `title`. Tytuł danego kursu.
- `slug`. Slug dla danego kursu, później będzie wykorzystywany w adresach URL.
- `overview`. Pole typu `TextField` zawierające ogólny opis kursu.
- `created`. Data i godzina utworzenia danego kursu. Wartość będzie automatycznie wstawiona przez Django podczas budowania nowych obiektów, ponieważ użyty został atrybut `auto_now_add=True`.

Każdy kurs jest podzielony na wiele modułów. Dlatego też model `Module` zawiera pole klucza obcego (`ForeignKey`) wskazujące model `Course`.

Przejdź do powłoki i wydaj poniższe polecenie w celu utworzenia początkowej migracji dla budowanej aplikacji.

```
python manage.py makemigrations
```

Wyświetli się taki wynik:

```
Migrations for 'courses':
   courses/migrations/0001_initial.py:
     - Create model Course
     - Create model Module
     - Create model Subject
     - Add field subject to course
```

Teraz wydaj poniższe polecenie, aby zastosować wszystkie migracje w bazie danych.

```
python manage.py migrate
```

Powinieneś otrzymać dane wyjściowe wskazujące na zastosowanie wszystkich migracji, łącznie z przygotowanymi przez framework Django. Dane wyjściowe będą zawierały m.in. wiersz:

```
Applying courses.0001_initial... OK
```

Powyższy wiersz informuje, że modele aplikacji `courses` zostały zsynchronizowane z bazą danych. Przystępujemy teraz do dodania modeli kursów do witryny administracyjnej.

Rejestracja modeli w witrynie administracyjnej

Zarejestrujmy modele kursów na stronie administracyjnej, aby można było łatwo zarządzać danymi. Przeprowadź edycję pliku *admin.py* w katalogu aplikacji `courses` i umieść w nim poniższy fragment kodu.

```
from django.contrib import admin
from .models import Subject, Course, Module

@admin.register(Subject)
class SubjectAdmin(admin.ModelAdmin):
    list_display = ['title', 'slug']
    prepopulated_fields = {'slug': ('title',)}

class ModuleInline(admin.StackedInline):
    model = Module

@admin.register(Course)
class CourseAdmin(admin.ModelAdmin):
    list_display = ['title', 'subject', 'created']
    list_filter = ['created', 'subject']
    search_fields = ['title', 'overview']
    prepopulated_fields = {'slug': ('title',)}
    inlines = [ModuleInline]
```

W ten sposób modele dla aplikacji `courses` zostały zarejestrowane w witrynie administracyjnej. Zapamiętaj, że do rejestracji modeli w witrynie administracyjnej służy dekorator `@admin.register()`.

W następnym punkcie dowiesz się, jak utworzyć początkowe dane w celu wypełnienia modeli.

Użycie fikstur w celu dostarczenia początkowych danych dla modeli

Czasami trzeba wstępnie umieścić w bazie danych na stałe zdefiniowane dane. Użyteczne jest umieszczenie danych początkowych w konfiguracji projektu, nie ma wtedy konieczności ich ręcznego dodawania. Django oferuje prosty sposób ładowania i zrzucania danych z bazy danych do plików zwanych *fiksturami* (ang. *fixtures*). Django obsługuje fikstury w formatach JSON, XML lub YAML. Struktura fikstury bardzo przypomina reprezentację API modelu, co ułatwia tłumaczenie danych między wewnętrznymi formatami baz danych a aplikacjami zewnętrznymi. Przechodzimy do utworzenia fikstur zawierających pewne początkowe obiekty `Subject` dla tworzonego projektu.

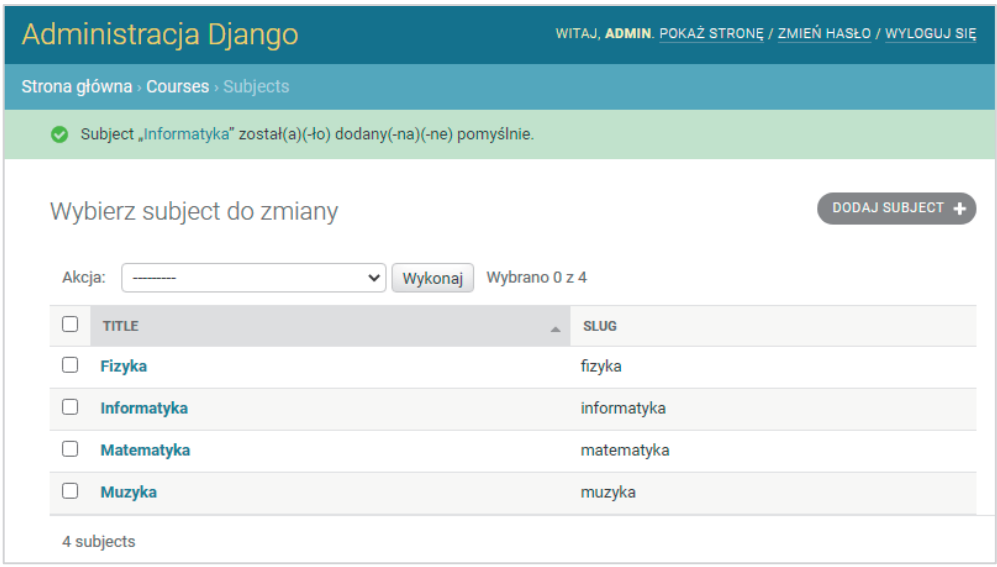
Pracę zaczniesz od utworzenia superużytkownika przez wydanie polecenia:

```
python manage.py createsuperuser
```

Teraz uruchom serwer programistyczny za pomocą polecenia:

```
python manage.py runserver
```

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/admin/courses/subject/`. Za pomocą witryny administracyjnej utwórz kilka dziedzin. Strona wyświetlająca ich listę powinna być podobna do pokazanej na rysunku 12.1.



Rysunek 12.1. Widok listy zmiany tematów na stronie administracyjnej

W wierszu polecenia powłoki uruchom polecenie:

```
python manage.py dumpdata courses --indent=2
```

Otrzymasz dane wyjściowe podobne do przedstawionych poniżej.

```
[
{
  "model": "courses.subject",
  "pk": 1,
  "fields": {
    "title": "Matematyka",
    "slug": "matematyka"
  }
},
{
  "model": "courses.subject",
  "pk": 2,
  "fields": {
    "title": "Muzyka",
    "slug": "muzyka"
  }
},
{
  "model": "courses.subject",
  "pk": 3,
  "fields": {
```

```
    "title": "Fizyka",
    "slug": "fizyka"
  }
},
{
  "model": "courses.subject",
  "pk": 4,
  "fields": {
    "title": "Programowanie",
    "slug": "programowanie",
  }
}
]
```

Polecenie `dumpdata` przekazuje do standardowego wyjścia dane pochodzące z bazy danych, domyślnie serializowane w formacie JSON. Powstała struktura danych zawiera informacje o modelu i jego polach. Dzięki temu Django może załadować go do bazy danych.

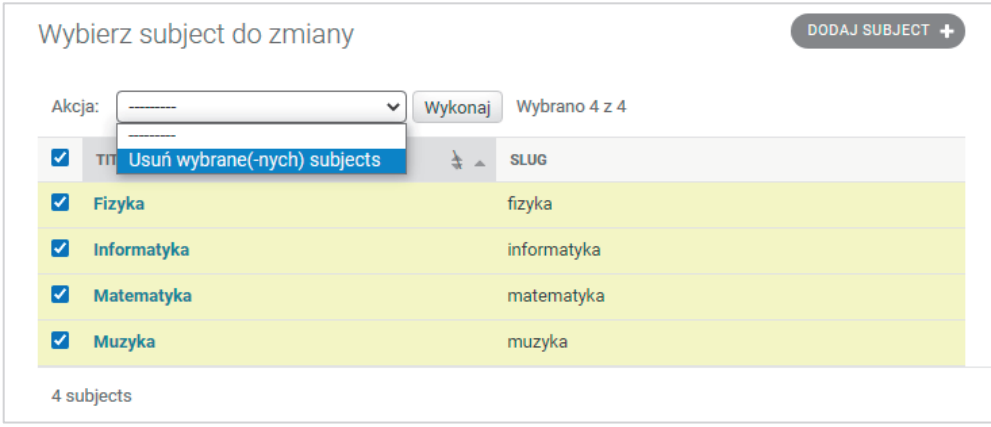
Dane wyjściowe można ograniczyć do wskazanych modeli aplikacji. W tym celu należy przekazać do polecenia nazwy aplikacji lub wskazać pojedyncze modele do wyprowadzania danych przy użyciu formatu `app.Model`.

Do określenia formatu służy opcja `--format`. Domyślnie narzędzie `dumpdata` przekazuje zserializowane dane do standardowego wyjścia. Można jednak wskazać plik wyjściowy za pomocą flagi `--output`, co pozwala na zapisanie wyniku. Z kolei opcja `--indent` pozwala na określenie poziomu wcięć. Więcej informacji na temat parametrów polecenia `dumpdata` można uzyskać za pomocą polecenia `python manage. py dumpdata --help`.

Informacje zapisz do pliku fikstur umieszczonego w katalogu *fixtures* aplikacji `courses` za pomocą poniższych poleceń.

```
mkdir courses/fixtures
python manage.py dumpdata courses --indent=2 --output=courses/fixtures/subjects.json
```

Uruchom serwer programistyczny i wykorzystaj witrynę administracyjną do usunięcia utworzonych wcześniej dziedzin kursów (rysunek 12.2).

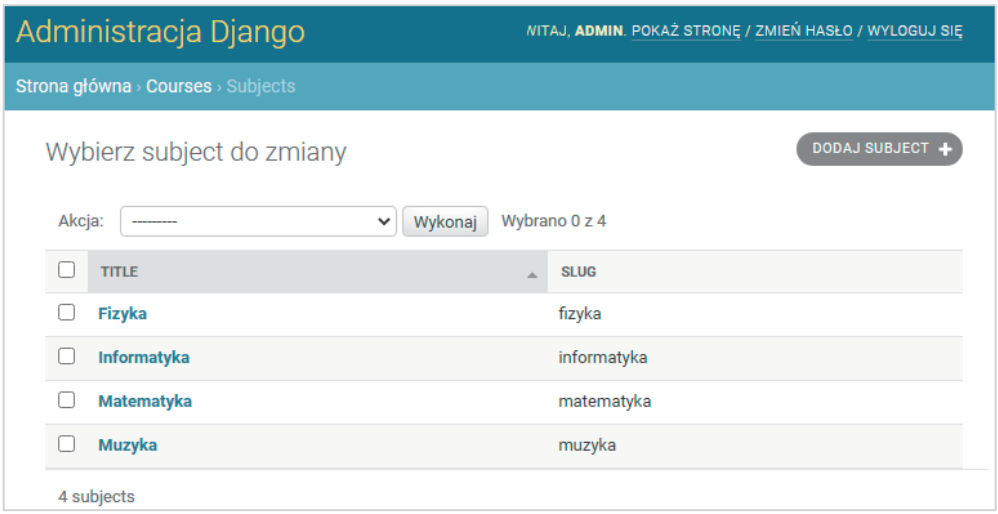


Rysunek 12.2. Usuwanie wszystkich istniejących dziedzin kursów

Po usunięciu wszystkich dziedzin, załaduj fikstur do bazy danych za pomocą polecenia:

```
python manage.py loaddata subjects.json
```

Wszystkie obiekty `Subject` znajdujące się w fikstur zostały ponownie wczytane do bazy danych (rysunek 12.3).



Rysunek 12.3. Dziedziny zapisane w fikstur zostały ponownie załadowane do bazy danych

Domyślnie Django szuka plików w katalogu *fixtures* aplikacji, ale istnieje możliwość podania w poleceniu `loaddata` pełnej ścieżki dostępu do katalogu fikstur. Jeszcze inne rozwiązanie to użycie opcji konfiguracyjnej `FIXTURE_DIRS` i za jej pomocą wskazanie frameworkowi dodatkowych katalogów, w których ma szukać fikstur.

Uwaga

Fikstury okazują się użyteczne nie tylko do ładowania danych początkowych, ale również jako mechanizm ładowania przykładowych danych aplikacji lub danych wymaganych do przeprowadzania testów. Można ich użyć również do wypełniania niezbędnych danych dla środowisk produkcyjnych.

Więcej informacji na temat używania fikstur do testowania znajdziesz na stronie *https://docs.djangoproject.com/en/5.0/topics/testing/tools/#fixture-loading*.

Jeżeli chcesz wczytywać fikstury w migracjach modeli, zajrzyj do dokumentacji Django poświęconej migracji danych. Dokumentację dotyczącą migracji danych znajdziesz na stronie *https://docs.djangoproject.com/en/5.0/topics/migrations/#data-migrations*.

Stworzyłeś modele do zarządzania przedmiotami kursów, kursami i ich modułami. W następnym podrozdziale utworzysz modele do zarządzania różnymi typami zawartości modułów.

Utworzenie modeli dla zróżnicowanej treści

Planujemy dodawanie dla modułów kursów różnego rodzaju treści, np. tekstu, obrazów, plików i klipów wideo. *Polimorfizm* to udostępnianie jednego interfejsu podmiotom różnych typów. Potrzebujemy wszechstronnego modelu danych pozwalającego na przechowywanie zróżnicowanej treści dostępnej za pomocą takiego samego interfejsu. W rozdziale 7., „Śledzenie działań użytkownika", przekonałeś się, jak wygodne jest użycie generycznych relacji do tworzenia zewnętrznych kluczy, które mogą wskazywać obiekty w dowolnym modelu. Przystępujemy teraz do utworzenia modelu `Content` reprezentującego treść modułów oraz zdefiniowania generycznej relacji w celu powiązania dowolnego rodzaju treści.

Przeprowadź edycję pliku *models.py* aplikacji `courses` i dodaj do niego poniższe polecenia import.

```
from django.contrib.contenttypes.fields import GenericForeignKey
from django.contrib.contenttypes.models import ContentType
```

Następnie umieść poniższy fragment kodu na końcu wymienionego pliku.

```
class Content(models.Model):
    module = models.ForeignKey(
        Module,
        related_name='contents',
        on_delete=models.CASCADE
    )
    content_type = models.ForeignKey(
        ContentType,
        on_delete=models.CASCADE
    )
    object_id = models.PositiveIntegerField()
    item = GenericForeignKey('content_type', 'object_id')
```

To jest model `Content`. Moduł może zawierać różnorodną treść, więc definiujemy kolumnę typu `ForeignKey` prowadzącą do modelu `Module`. Konfigurujemy również generyczną relację w celu powiązania obiektów pochodzących z różnych modeli i przedstawiających odmienne rodzaje treści. Pamiętaj o konieczności dodania trzech oddzielnych kolumn niezbędnych do zdefiniowania relacji generycznej. W omawianym tutaj modelu `Content` będą to następujące pola.

- `content_type`. Pole typu `ForeignKey` prowadzące do modelu `ContentType`.
- `object_id`. Pole typu `PositiveIntegerField` przeznaczone do przechowywania klucza głównego powiązanego obiektu.
- `item`. Pole typu `GenericForeignKey` prowadzące do powiązanego obiektu przez połączenie dwóch poprzednich pól.

Jedynie pola `content_type` i `object_id` mają odpowiadające im kolumny w tabeli bazy danych tego modelu. Natomiast pole `item` pozwala na bezpośrednie pobranie zbioru powiązanych obiektów, a jego funkcjonalność została zbudowana na podstawie dwóch wcześniej wymienionych pól.

Zamierzamy zastosować odmienne modele dla każdego rodzaju treści: tekstu, zdjęć, wideo i dokumentów. Modele `Content` będą miały pewne wspólne pola, ale będą różnić się

konkretnymi danymi, które przechowują. Na przykład w przypadku treści tekstowych przechowywany będzie rzeczywisty tekst, natomiast w przypadku treści wideo będzie przechowywany adres URL nagrania. Aby uzyskać tę funkcjonalność, trzeba zastosować dziedziczenie modeli. Przed zbudowaniem modeli `Content` przyjrzymy się szczegółowo opcjom dziedziczenia modeli oferowanym przez Django.

Wykorzystanie dziedziczenia modeli

Django obsługuje dziedziczenie modeli. Sposób działania jest bardzo podobny do standardowego mechanizmu dziedziczenia klas w Pythonie. Dla przypomnienia, dziedziczenie klas polega na zdefiniowaniu nowej klasy, która uzyskuje metody i właściwości z istniejącej klasy. Ułatwia to wielokrotne wykorzystanie kodu i upraszcza tworzenie powiązanych klas. Więcej o dziedziczeniu klas można przeczytać na stronie *https://docs.python.org/3/tutorial/classes.html#inheritance*.

Framework Django oferuje trzy opcje dziedziczenia modeli.

- **Modele abstrakcyjne.** Opcja okazuje się użyteczna, kiedy te same informacje mają zostać umieszczone w kilku modelach.
- **Model dziedziczenia oparty na wielu tabelach.** Opcja jest stosowana, gdy każdy model w hierarchii jest uznawany za kompletny.
- **Modele proxy.** Opcja okazuje się użyteczna, gdy trzeba zmodyfikować zachowanie modelu, np. dodać kolejne metody, zmienić domyślnego menedżera lub użyć zupełnie innych opcji `meta`.

Teraz opiszę dokładniej poszczególne modele dziedziczenia.

Modele abstrakcyjne

Model abstrakcyjny to klasa bazowa, w której definiujemy pola mające się znaleźć w modelach potomnych. Django nie buduje w bazie danych tabeli dla modelu abstrakcyjnego. Natomiast tabela bazy danych jest tworzona dla każdego modelu potomnego, m.in. dla kolumn dziedziczonych po klasie abstrakcyjnej i zdefiniowanych w modelu potomnym.

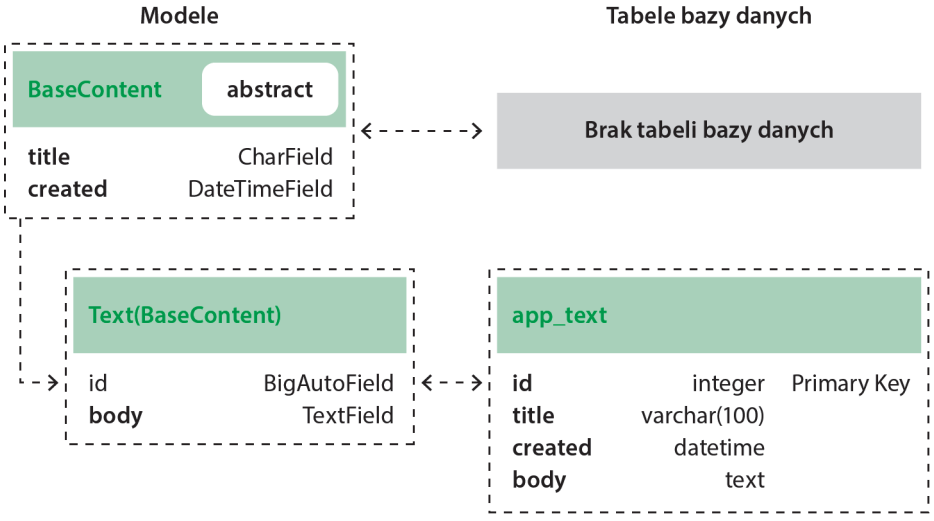
Aby oznaczyć model jako abstrakcyjny, konieczne jest użycie atrybutu `abstract=True` w klasie `Meta`. Django rozpozna, że ma do czynienia z modelem abstrakcyjnym i nie utworzy dla niego tabeli w bazie danych. W celu opracowania modeli potomnych konieczne jest po prostu utworzenie podklas modelu abstrakcyjnego.

Poniżej przedstawiłem przykład modelu abstrakcyjnego `BaseContent` i modelu potomnego `Text`.

```
from django.db import models

class BaseContent(models.Model):
    title = models.CharField(max_length=100)
    created = models.DateTimeField(auto_now_add=True)
    class Meta:
        abstract = True

class Text(BaseContent):
    body = models.TextField()
```

W powyższym przykładzie Django utworzy tabelę jedynie dla modelu `Text` i umieści w niej kolumny `title`, `created` i `body`.

Modele i powiązane tabele bazy danych dla podanego przykładu kodu przedstawiłem na rysunku 12.4.



Rysunek 12.4. Przykładowe modele i tabele bazy danych dla dziedziczenia z wykorzystaniem modeli abstrakcyjnych

W następnym podpunkcie zapoznasz się z innym podejściem do dziedziczenia modelu — takim, w którym tworzonych jest wiele tabel bazy danych.

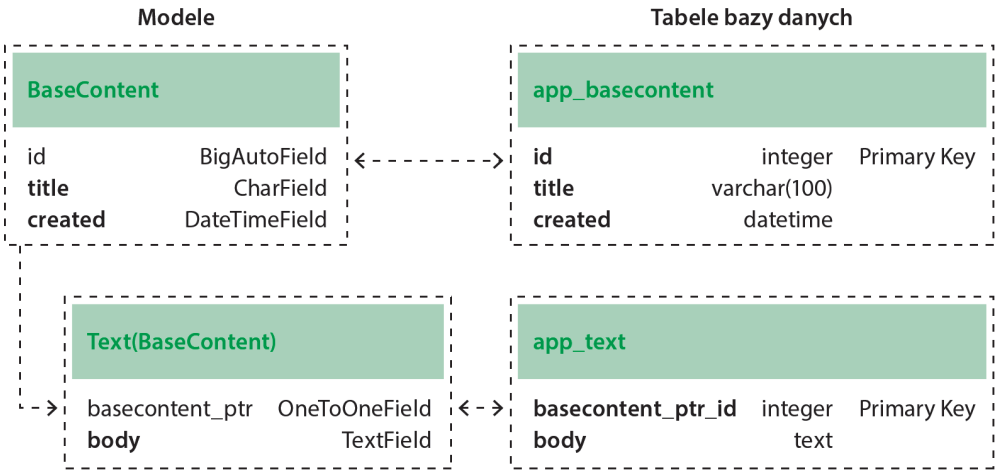
Model dziedziczenia oparty na wielu tabelach

W dziedziczeniu opartym na wielu tabelach każdemu modelowi odpowiada tabela w bazie danych. Django utworzy w modelu potomnym pole typu `OneToOneField` dla relacji z jego modelem nadrzędnym. W celu zastosowania dziedziczenia opartego na wielu tabelach należy utworzyć podklasę istniejącego modelu. Framework Django zbuduje tabelę bazy danych dla modelu zarówno pierwotnego, jak i potomnego. Oto przykład dziedziczenia opartego na wielu tabelach:

```
from django.db import models

class BaseContent(models.Model):
    title = models.CharField(max_length=100)
    created = models.DateTimeField(auto_now_add=True)

class Text(BaseContent):
    body = models.TextField()
```

Django dołączy w modelu `Text` automatycznie wygenerowane pole `OneToOneField`, wskazujące na model `BaseContent`. Nazwa tego pola to `basecontent_ptr`, gdzie `ptr` oznacza *pointer* (czyli wskaźnik). Dla każdego modelu tworzona jest tabela bazy danych.

Modele i powiązane tabele bazy danych dla podanego przykładu kodu dziedziczenia modelu z wieloma tabelami przedstawiłem na rysunku 12.5.



Rysunek 12.5. Przykładowe modele i tabele bazy danych dla dziedziczenia z wykorzystaniem modeli abstrakcyjnych

W następnym podpunkcie zapoznasz się z jeszcze innym podejściem do dziedziczenia modelu — takim, w którym wiele modeli pełni funkcję pośrednika do jednej tabeli bazy danych.

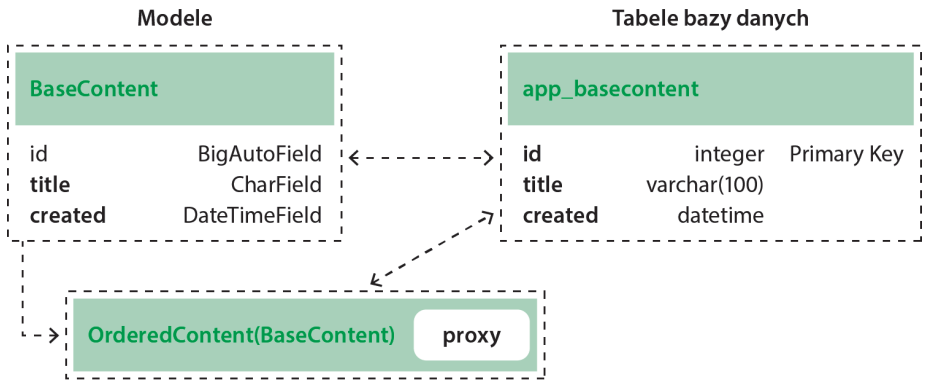
Modele proxy

Modele proxy są używane do zmiany zachowania modelu. Oba modele operują na tabeli bazy danych dla modelu pierwotnego. Pozwala to na dostosowanie zachowania dla różnych modeli bez tworzenia nowej tabeli bazy danych. W ten sposób tworzymy różne wersje tego samego modelu, dostosowane do różnych celów. W celu utworzenia modelu proxy należy dodać do klasy `Meta` modelu atrybut `proxy=True`. Sposób utworzenia modelu proxy pokazałem w poniższym przykładzie.

```
from django.db import models
from django.utils import timezone

class BaseContent(models.Model):
    title = models.CharField(max_length=100)
    created = models.DateTimeField(auto_now_add=True)

class OrderedContent(BaseContent):
    class Meta:
        proxy = True
        ordering = ['created']

    def created_delta(self):
        return timezone.now() - self.created
```

W kodzie zdefiniowaliśmy model `OrderedContent` będący modelem proxy dla modelu `Content`. Model zapewnia domyślne sortowanie kolekcji `QuerySet` oraz metodę dodatkową o nazwie `created_delta()`. Oba modele, `Content` i `OrderedContent`, działają na tej samej tabeli bazy danych, a obiekty są dostępne za pomocą ORM dla dowolnego modelu.

Modele i powiązane tabele bazy danych dla podanego przykładu kodu dziedziczenia z wykorzystaniem modelu proxy przedstawiłem na rysunku 12.6.



Rysunek 12.6. Przykładowe modele i tabele bazy danych dla dziedziczenia z wykorzystaniem modelu proxy

Zapoznałeś się już z trzema typami dziedziczenia modeli. Dodatkowe informacje na temat dziedziczenia modeli można znaleźć na stronie *https://docs.djangoproject.com/en/5.0/topics/db/models/#model-inheritance*. Teraz zastosujemy dziedziczenie modeli w praktyce. Wykorzystamy bazowy model abstrakcyjny do stworzenia modeli dla różnych typów treści.

Utworzenie modeli treści

Spróbujmy wykorzystać dziedziczenie modeli do zaimplementowania modeli polimorficznych. W tym punkcie stworzysz uniwersalny model danych umożliwiający przechowywanie różnorodnych treści dostępnych za pośrednictwem jednolitego interfejsu. Idealnym podejściem dla tego przypadku użycia jest stworzenie abstrakcyjnego modelu bazowego, który następnie będzie rozszerzony o inne modele, każdy przeznaczony do przechowywania określonego typu danych: tekstu, obrazu, wideo i plików. To elastyczne podejście da Ci narzędzia przydatne w scenariuszach, w których wymagany jest polimorfizm.

Model `Content` dla budowanej aplikacji `courses` zawiera relację generyczną przeznaczoną do powiązania z nim różnych rodzajów treści. Utworzymy oddzielny model dla każdego rodzaju treści. Wszystkie modele treści będą zawierały te same niektóre pola oraz dodatkowe przeznaczone do przechowywania własnych danych. Zaczynamy od przygotowania modelu abstrakcyjnego dostarczającego pola, które będą stosowane we wszystkich modelach treści.

Przeprowadź edycję pliku *models.py* aplikacji `courses` i umieść w nim poniższy fragment kodu.

```
class ItemBase(models.Model):
    owner = models.ForeignKey(User,
        related_name='%(class)s_related',
        on_delete=models.CASCADE
    )
    title = models.CharField(max_length=250)
    created = models.DateTimeField(auto_now_add=True)
    updated = models.DateTimeField(auto_now=True)

    class Meta:
        abstract = True

    def __str__(self):
        return self.title

class Text(ItemBase):
    content = models.TextField()

class File(ItemBase):
    file = models.FileField(upload_to='files')

class Image(ItemBase):
    file = models.FileField(upload_to='images')

class Video(ItemBase):
    url = models.URLField()
```

W powyższym kodzie zdefiniowaliśmy model abstrakcyjny o nazwie `ItemBase`. Dlatego też konieczne było użycie atrybutu `abstract=True` w jego klasie `Meta`.

W tym modelu mamy kolumny `owner`, `title`, `created` i `updated`. To wspólne pola używane we wszystkich modelach treści.

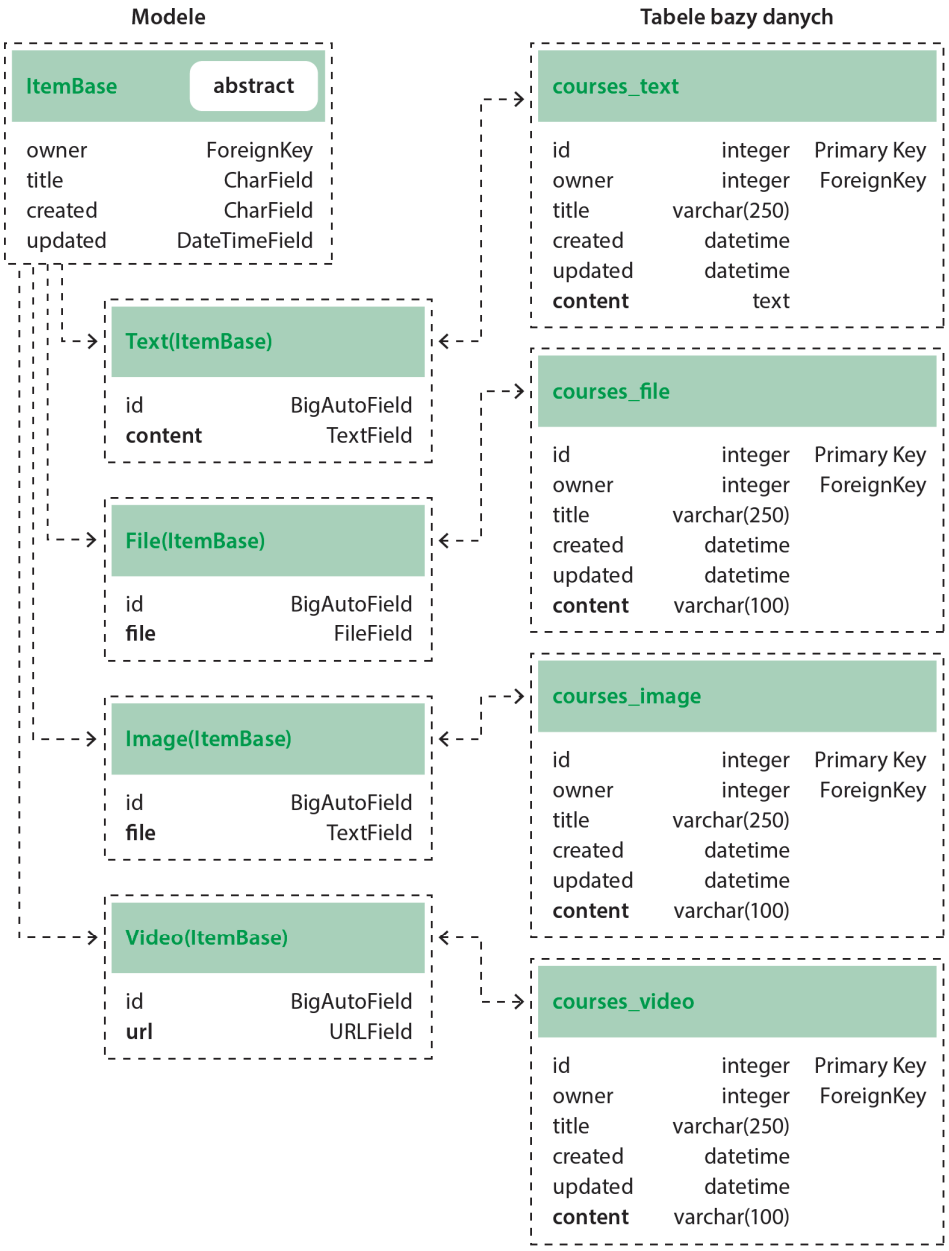
W polu `owner` będziemy przechowywać nazwę użytkownika, który utworzył daną treść. Ponieważ wymienione pole zostało zdefiniowane w klasie abstrakcyjnej, potrzebujemy różnych kolumn `related_name` dla każdego modelu potomnego. Django pozwala na określenie miejsca zarezerwowanego dla nazwy klasy modelu, co odbywa się za pomocą atrybutu `related_name` jako `'%(class)s_related'`. W ten sposób wartość `related_name` dla każdego modelu potomnego będzie generowana automatycznie. Ponieważ używamy `'%(class)s_related'` jako `related_name`, więc odwrotną relacją dla modeli potomnych będzie — odpowiednio — `text_related`, `file_related`, `image_related` i `video_related`.

Zdefiniowaliśmy cztery różne modele treści, które dziedziczą po modelu abstrakcyjnym `ItemBase`. Oto one:

- `Text`. Przeznaczony do przechowywania treści w postaci tekstu.
- `File`. Przeznaczony do przechowywania treści w postaci plików, np. dokumentów PDF.
- `Image`. Przeznaczony do przechowywania treści w postaci plików obrazów.
- `Video`. Przeznaczony do przechowywania treści w postaci plików wideo. Kolumnę typu `URLField` możemy wykorzystać w celu podania adresu URL osadzonego w wideo.

Każdy model potomny zawiera pola zdefiniowane w klasie `ItemBase` oraz własne. Tabela bazy danych zostanie utworzona dla modeli — odpowiednio — `Text`, `File`, `Image` i `Video`. Z modelem `ItemBase` nie jest skojarzona żadna tabela bazy danych, ponieważ jest to model abstrakcyjny.

Modele treści i powiązane z nimi tabele bazy danych przedstawiłem na rysunku 12.7.



Rysunek 12.7. Modele treści i powiązane z nimi tabele bazy danych

Przeprowadź edycję utworzonego wcześniej modelu `Content` i zmodyfikuj jego kolumnę `content_type`, tak jak pokazałem poniżej.

```
content_type = models.ForeignKey(
        ContentType,
        on_delete=models.CASCADE,
        limit_choices_to={
            'model__in':('text', 'video', 'image', 'file')
        }
    )
```

W celu ograniczenia obiektów `ContentType`, które mogą być używane w relacji generycznej, dodaliśmy argument `limit_choices_to`. Ponadto, do filtrowania zapytania, aby otrzymać jedynie obiekty `ContentType`, których atrybut `model` ma wartość `'text'`, `'video'`, `'image'` lub `'file'`, wykorzystujemy kolumnę wyszukiwania `model__in`.

Przechodzimy do przygotowania migracji obejmującej nowo dodane modele. Z poziomu powłoki wydaj poniższe polecenie.

```
python manage.py makemigrations
```

Wyświetli się taki wynik:

```
Migrations for 'courses':
   courses/migrations/0002_video_text_image_file_content.py
     - Create model Video
     - Create model Text
     - Create model Image
     - Create model File
     - Create model Content
```

Teraz wydaj poniższe polecenie, aby zastosować przygotowane wcześniej migracje.

```
python manage.py migrate
```

Powinieneś otrzymać dane wyjściowe kończące się poniższym wierszem.

```
Applying courses.0002_video_text_image_file_content… OK
```

W ten sposób utworzyliśmy modele odpowiednie do przechowywania zróżnicowanej treści dla modułów kursów. Jednak nadal mamy pewne braki w modelach. Moduły kursów i treści powinny stosować określoną kolejność. Potrzebujemy więc pola umożliwiającego łatwe wymuszanie zastosowania pewnej kolejności.

Utworzenie własnych pól modelu

Framework Django jest dostarczany wraz z pełną kolekcją pól modeli, które można wykorzystać podczas tworzenia własnych modeli. Oczywiście istnieje również możliwość opracowania własnych kolumn modeli przeznaczonych do przechowywania niestandardowych danych lub zmodyfikowania sposobu zachowania istniejących kolumn. Niestandardowe pola umożliwiają przechowywanie unikatowych typów danych, implementowanie niestandardowych reguł walidacji, hermetyzację złożonej logiki danych związanej z polem lub definiowanie określonych formularzy renderowanych przy użyciu niestandardowych widżetów.

W budowanym projekcie jest potrzebne pole pozwalające na określenie kolejności obiektów. Jeżeli zastanawiasz się nad łatwym rozwiązaniem w tym zakresie standardowo oferowanym przez Django, prawdopodobnie rozważasz dodanie pola typu `PositiveIntegerField`. Kolejność obiektów można łatwo określić za pomocą liczb całkowitych. Możemy utworzyć niestandardowe pole dziedziczące po klasie `PositiveIntegerField` i zapewniające obsługę dodatkowej funkcjonalności.

W polu określającym kolejność obiektów są wbudowane dwie funkcjonalności.

- **Automatyczne przypisanie wartości określającej kolejność, gdy nie została podana żadna konkretna kolejność.** Jeżeli podczas zapisywania obiektu nie podamy informacji o jego kolejności, pole powinno automatycznie przypisać mu następną wartość na podstawie ostatniego obiektu ułożonego w określonej kolejności. Jeśli np. mamy dwa obiekty o kolejności — odpowiednio — `1` i `2`, wtedy podczas zapisywania trzeciego należy mu automatycznie przypisać kolejność `3`, gdy nie będzie wyraźnie podana żadna inna.
- **Kolejność obiektów z uwzględnieniem innych pól.** Moduły kursu będą ułożone w kolejności z uwzględnieniem kursu, do którego należą, oraz treści modułu z uwzględnieniem modułu, do którego należą.

Utwórz nowy plik *fields.py* w katalogu aplikacji `courses` i umieść w nim poniższy fragment kodu.

```
from django.core.exceptions import ObjectDoesNotExist
from django.db import models

class OrderField(models.PositiveIntegerField):
    def __init__(self, for_fields=None, *args, **kwargs):
        self.for_fields = for_fields
        super().__init__(*args, **kwargs)

    def pre_save(self, model_instance, add):
        if getattr(model_instance, self.attname) is None:
            # Brak bieżącej wartości.
            try:
                qs = self.model.objects.all()
                if self.for_fields:
                    # Filtrowanie względem obiektów o tych samych wartościach pola
                    # dla pól w "for_fields".
                    query = {
                        field: getattr(model_instance, field)
                        for field in self.for_fields
                    }
                    qs = qs.filter(**query)
                # Pobranie kolejności ostatniego elementu.
                last_item = qs.latest(self.attname)
                value = getattr(last_item, self.attname) + 1
            except ObjectDoesNotExist:
                value = 0
            setattr(model_instance, self.attname, value)
            return value
        else:
            return super().pre_save(model_instance, add)
```

Tak przedstawia się utworzone niestandardowe pole `OrderField`. Dziedziczy po polu typu `PositiveIntegerField` udostępnianym przez Django. Pole `OrderField` pobiera opcjonalny parametr `for_fields`, pozwalający na wskazanie pól, które należy uwzględnić podczas ustalania kolejności dla danych.

Zdefiniowane pole nadpisuje metodę `pre_save()` pola typu `PositiveIntegerField`, która jest wywoływana przed umieszczeniem wartości pola w bazie danych. W wymienionej metodzie są wykonywane następujące zadania.

1. Sprawdzenie, czy dla danego pola w egzemplarzu modelu istnieje już wartość. Wykorzystujemy `self.attname`, czyli nazwę atrybutu nadaną polu w tym modelu. Jeżeli wartość atrybutu będzie inna niż `None`, to obliczamy kolejność w taki sposób:
   A. Tworzymy kolekcję `QuerySet` w celu pobrania wszystkich obiektów dla modelu pola. Klasę modelu, w którym znajduje się pole, określamy za pomocą `self.model`.
   B. Filtrujemy kolekcję `QuerySet` względem wartości bieżącego pola dla pól modelu zdefiniowanych w parametrze `for_fields`, o ile taki został podany. W ten sposób ustalamy kolejność z uwzględnieniem podanych pól.
   C. Za pomocą `last_item =qs.latest(self.attname)` pobieramy z bazy danych obiekt o największej wartości określającej kolejność. Jeżeli nie zostanie znaleziony żaden obiekt, przyjmujemy założenie, że będzie to pierwszy obiekt i przypisujemy mu kolejność `0`.
   D. W przypadku znalezienia obiektu dodajemy `1` do największej wartości określającej kolejność.
   E. Obliczoną kolejność za pomocą wywołania `setattr()` przypisujemy do wartości pola w egzemplarzu modelu i zwracamy ją.
2. Jeżeli egzemplarz modelu ma wartość dla bieżącej kolumny, nie podejmujemy żadnego działania.

Wskazówka

Podczas tworzenia niestandardowych kolumn modelu pamiętaj, aby pozostały jak najbardziej ogólne. Unikaj definiowania na stałe danych, które zależą od konkretnego modelu lub kolumny. Przygotowane przez Ciebie pole powinno działać w dowolnym modelu.

Więcej informacji na temat tworzenia niestandardowych pól modeli można znaleźć na stronie *https://docs.djangoproject.com/en/5.0/howto/custom-model-fields/*.

W następnym punkcie pokażę, w jaki sposób wykorzystać utworzone niestandardowe pole.

Dodawanie porządkowania do modułów i obiektów treści

Przystępujemy do dodania nowego pola do tworzonych modeli. Przeprowadź edycję pliku *models.py* aplikacji `courses`, zaimportuj klasę `OrderField` i zdefiniuj pole modelu `Module` za pomocą poniższego fragmentu kodu.

```
from .fields import OrderField
class Module(models.Model):
    # …
    order = OrderField(blank=True, for_fields=['course'])
```

Nowemu polu nadajemy nazwę `order` i za pomocą `for_fields=['course']` określamy, że kolejność ma zostać ustalona z uwzględnieniem kursu. Oznacza to, że podczas obliczania kolejności dla nowego modułu będzie dodane `1` do wartości oznaczającej kolejność ostatniego modułu w tym samym obiekcie `Course`.

Teraz można przeprowadzić edycję metody `__str__()` modelu `Module` w celu uwzględnienia jego kolejności, tak jak pokazałem poniżej.

```
class Module(models.Model):
    # …
    def __str__(self):
        return f'{self.order}.{self.title}'
```

Do treści modułu również należy stosować określoną kolejność. Dodaj kolumnę `OrderField` do modelu `Content`, tak jak pokazałem poniżej.

```
class Content(models.Model):
    # …
    order = OrderField(blank=True, for_fields=['module'])
```

Tym razem wskazujemy, że kolejność ma zostać ustalona z uwzględnieniem pola `module`.

Na końcu dodajemy do obu modeli kolejność domyślną. W modelach `Module` i `Content` umieść poniższą definicję klasy `Meta`.

```
class Module(models.Model):
    # …
    class Meta:
        ordering = ['order']

class Content(models.Model):
    # …
    class Meta:
        ordering = ['order']
```

Na tym etapie modele `Module` i `Content` powinny przedstawiać się tak:

```
class Module(models.Model):
    course = models.ForeignKey(
        Course, related_name='modules', on_delete=models.CASCADE
    )
    title = models.CharField(max_length=200)
    description = models.TextField(blank=True)
```

```
        order = OrderField(blank=True, for_fields=['course'])

        class Meta:
            ordering = ['order']

        def __str__(self):
            return f'{self.order}. {self.title}'

    class Content(models.Model):
        module = models.ForeignKey(
            Module,
            related_name='contents',
            on_delete=models.CASCADE
        )
        content_type = models.ForeignKey(
            ContentType,
            on_delete=models.CASCADE,
            limit_choices_to={
                'model__in':('text', 'video', 'image', 'file')
            }
        )
        object_id = models.PositiveIntegerField()
        item = GenericForeignKey('content_type', 'object_id')
        order = OrderField(blank=True, for_fields=['module'])

        class Meta:
                ordering = ['order']
```

Przystępujemy do przygotowania kolejnej migracji modelu odzwierciedlającej nowe pola określające kolejność. Przejdź do powłoki, a następnie wydaj poniższe polecenie.

```
python manage.py makemigrations courses
```

Wyświetli się następujący wynik.

```
It is impossible to add a non-nullable field 'order' to content without specifying
↪a default. This is because the database needs something to populate existing rows.
Please select a fix:
1) Provide a one-off default now (will be set on all existing rows with a null
↪value for this column)
2) Quit and manually define a default value in models.py.
Select an option:
```

Django informuje, że konieczne jest podanie wartości domyślnej dla rekordów istniejących w bazie danych, ponieważ do już utworzonego modelu dodaliśmy nowe pole `order`. Jeżeli pole ma atrybut `null=True`, zaakceptuje wartości `Null` i wtedy Django przeprowadzi migrację bez pytania o wartość domyślną. Możemy podać wartość domyślną lub przerwać migrację i przed przygotowaniem migracji dodać w pliku `models.py` do pola `order` atrybut `default`.

Aby podać wartość domyślną dla istniejących rekordów, wpisz **1** i naciśnij klawisz *Enter*. Wyświetli się następujący wynik.

```
Please enter the default value as valid Python.
```

```
The datetime and django.utils.timezone modules are available, so it is possible
↪to provide e.g. timezone.now as a value.
Type 'exit' to exit this prompt
>>>
```

Podaj `0` jako wartość domyślną dla istniejących rekordów i naciśnij klawisz *Enter*. Django zapyta również o wartość domyślną dla modelu `Module`. Wybierz pierwszą opcję i ponownie podaj `0` jako wartość domyślną.

Na koniec otrzymasz dane wyjściowe podobne do przedstawionych poniżej.

```
Migrations for 'courses':
courses/migrations/0003_alter_content_options_alter_module_options_and_more.py
     - Change Meta options on content
     - Change Meta options on module
     - Add field order to content
     - Add field order to module
```

Teraz możesz już zastosować nową migrację za pomocą polecenia:

```
python manage.py migrate
```

Wygenerowane dane wyjściowe będą wskazywały na zakończone powodzeniem przeprowadzenie migracji.

```
Applying courses.0003_alter_content_options_alter_module_options_and_more… OK
```

Przetestujmy teraz nowe pole. Przejdź do powłoki Pythona za pomocą polecenia:

```
python manage.py shell
```

Utwórz nowy kurs, tak jak pokazałem poniżej.

```
>>> from django.contrib.auth.models import User
>>> from courses.models import Subject, Course, Module
>>> user = User.objects.last()
>>> subject = Subject.objects.last()
>>> c1 = Course.objects.create(subject=subject, owner=user, title='Kurs 1',
↪slug='kurs1')
```

W ten sposób utworzyliśmy w bazie danych nowy kurs. Kolejnym krokiem jest dodanie modułów do kursu, aby zobaczyć, jak będzie przebiegało automatyczne określanie kolejności modułów. Tworzymy więc początkowy moduł i sprawdzamy jego kolejność.

```
>>> m1 = Module.objects.create(course=c1, title='Moduł 1')
>>> m1.order
0
```

Pole `OrderField` określiło jego kolejność jako `0`, ponieważ to jest pierwszy obiekt `Module` utworzony dla danego kursu. Dla tego samego kursu możemy utworzyć drugi moduł.

```
>>> m2 = Module.objects.create(course=c1, title='Moduł 2')
>>> m2.order
1
```

Dla drugiego modułu pole `OrderField` określiło kolejność przez dodanie 1 do najwyższej wartości istniejącego obiektu. Przystępujemy do utworzenia trzeciego modułu i wymuszenia dla niego określonej kolejności.

```
>>> m3 = Module.objects.create(course=c1, title='Moduł 3', order=5)
>>> m3.order
5
```

W przypadku podania konkretnej kolejności podczas tworzenia lub zapisywania obiektu pole `OrderField` nie ingeruje w jej wartość i użyta zostaje wartość podana dla opcji `order`.

Dodajmy czwarty moduł.

```
>>> m4 = Module.objects.create(course=c1, title='Moduł 4')
>>> m4.order
6
```

Kolejność rekordów dla tego modułu została ustalona automatycznie. Pole `OrderField` nie gwarantuje, że pola określające porządkowanie występują w modelu po kolei. Jednak respektuje istniejące wartości tych pól i zawsze przypisuje następną wartość na podstawie najwyższej istniejącej.

Utwórzmy drugi kurs i dodajmy do niego moduł.

```
>>> c2 = Course.objects.create(subject=subject, title='Kurs 2', slug='kurs2',
↪owner=user)
>>> m5 = Module.objects.create(course=c2, title='Moduł 1')
>>> m5.order
0
```

Aby obliczyć kolejność nowego modułu, pod uwagę są brane jedynie pola istniejących modułów, które należą do tego samego kursu. Ponieważ mamy do czynienia z pierwszym modułem drugiego kursu, więc wartość kolejności wynosi `0`. Wynika to z faktu użycia `for_fields=['course']` w polu `order` modelu `Module`.

Gratulacje! W ten sposób utworzyłeś pierwsze niestandardowe pole modelu. W następnym podrozdziale stworzysz mechanizm uwierzytelniania dla systemu CMS.

Dodanie widoków uwierzytelniania

Po utworzeniu polimorficznego modelu danych, zbudujemy system CMS do zarządzania kursami i ich zawartością. Pierwszym krokiem jest dodanie systemu uwierzytelniania dla CMS.

Dodanie systemu uwierzytelniania

Na budowanej platformie e-learningu wykorzystamy framework uwierzytelniania Django. Sposób korzystania z widoków uwierzytelniania frameworka Django omówiłem w rozdziale 4., „Utworzenie witryny społecznościowej”.

Konta zarówno instruktorów, jak i uczestników kursów będą egzemplarzami modelu `User` w Django. Wymienione osoby będą miały możliwość zalogowania się do witryny internetowej za pomocą widoków uwierzytelnienia dostarczanych przez moduł `django.contrib.auth`.

Przeprowadź edycję głównego pliku *urls.py* projektu `educa` i dodaj do niego widoki `login` i `logout` dostarczane przez framework uwierzytelniania w Django.

```
from django.conf import settings
from django.conf.urls.static import static
from django.contrib import admin
from django.contrib.auth import views as auth_views
from django.urls import path

urlpatterns = [
    path(
        'accounts/login/', auth_views.LoginView.as_view(), name='login'
    ),
    path(
        'accounts/logout/',
        auth_views.LogoutView.as_view(),
        name='logout'
    ),
    path('admin/', admin.site.urls), ]

if settings.DEBUG:
    urlpatterns += static(
        settings.MEDIA_URL, document_root=settings.MEDIA_ROOT
    )
```

W następnym punkcie utworzymy szablony uwierzytelniania dla widoków uwierzytelniania Django.

Utworzenie szablonów uwierzytelniania

W katalogu aplikacji `courses` utwórz następującą strukturę plików i katalogów.

```
templates/
     base.html
     registration/
         login.html
         logged_out.html
```

Zanim przystąpimy do budowy szablonów uwierzytelniania, konieczne jest opracowanie szablonu bazowego dla projektu. Przeprowadź edycję pliku szablonu *base.html* i umieść w nim poniższy fragment kodu.

```
{% load static %}
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>{% block title %}Educa{% endblock %}</title>
    <link href="{% static "css/base.css" %}" rel="stylesheet">
```

```
    </head>
    <body>
      <div id="header">
        <a href="/" class="logo">Educa</a>
        <ul class="menu">
          {% if request.user.is_authenticated %}
            <li>
              <form action="{% url "logout" %}" method="post">
                <button type="submit">Wyloguj</button>
                {% csrf_token %}
              </form>
            </li>
          {% else %}
            <li><a href="{% url "login" %}">Zaloguj</a></li>
          {% endif %}
        </ul>
      </div>
      <div id="content">
        {% block content %}
        {% endblock %}
      </div>
      <script>
        document.addEventListener('DOMContentLoaded', (event) => {
          // Model DOM załadowany
          {% block domready %}
          {% endblock %}
        })
      </script>
    </body>
  </html>
```

Jest to szablon bazowy, który będzie rozszerzany przez pozostałe szablony. W przedstawionym szablonie zdefiniowaliśmy następujące bloki.

- `title`. Blok dla innych szablonów pozwalający na dodanie własnego tytułu dla danej strony.
- `content`. Główny blok dla treści. Wszystkie szablony rozszerzające ten szablon bazowy powinny umieszczać treść w bloku `content`.
- `domready`. Blok umieszczony wewnątrz funkcji JavaScript nasłuchującej zdarzenia `DOMContentLoaded`. Pozwala na wykonanie kodu po zakończeniu wczytywania modelu **DOM**.

Style CSS użyte w omawianym szablonie znajdują się w katalogu *static* aplikacji `courses`, której kod umieściłem w materiałach przygotowanych dla książki. Skopiuj stamtąd katalog *static* i umieść go w tym samym miejscu budowanego projektu. Zawartość katalogu znajdziesz pod adresem *https://github.com/PacktPublishing/Django-5-by-Example/tree/main/Chapter12/educa/courses/static*.

Przeprowadź edycję szablonu *registration/login.html* i umieść w nim poniższy fragment kodu.

```
{% extends "base.html" %}

{% block title %}Logowanie{% endblock %}
```

```
{% block content %}
  <h1>Logowanie</h1>
  <div class="module">
    {% if form.errors %}
      <p>Nazwa użytkownika lub hasło są niepoprawne. Spróbuj ponownie.</p>
    {% else %}
      <p>Wypełnij poniższy formularz, aby się zalogować:</p>
    {% endif %}
    <div class="login-form">
      <form action="{% url 'login' %}" method="post">
        {{ form.as_p }}
        {% csrf_token %}
        <input type="hidden" name="next" value="{{ next }}" />
        <p><input type="submit" value="Zaloguj"></p>
      </form>
    </div>
  </div>
{% endblock %}
```

To standardowy szablon logowania dostarczanego przez Django widoku `login`.

Przeprowadź edycję szablonu *registration/logged_out.html* i umieść w nim poniższy fragment kodu.

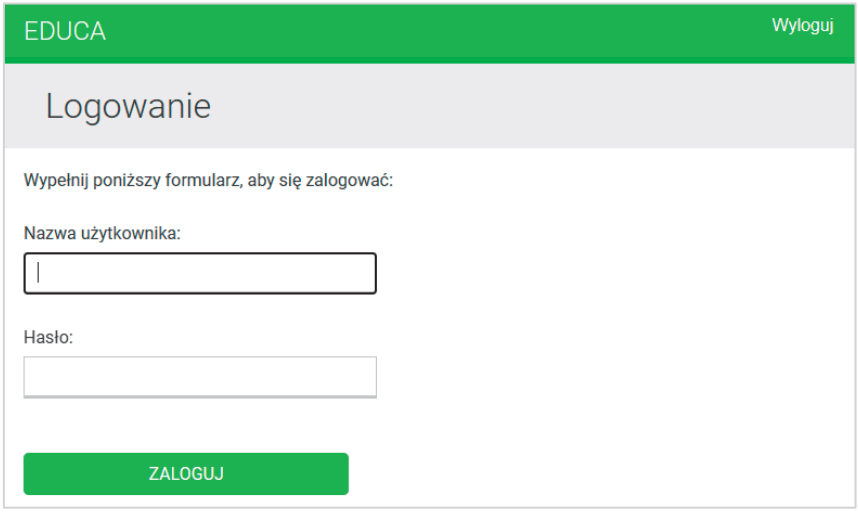
```
{% extends "base.html" %}

{% block title %}Wylogowanie{% endblock %}

{% block content %}
  <h1>Wylogowanie</h1>
  <div class="module">
    <p>
      Zostałeś pomyślnie wylogowany.
       Możesz <a href="{% url "login" %}">zalogować się ponownie</a>.
     </p>
  </div>
{% endblock %}
```

Jest to szablon wyświetlany użytkownikowi po wylogowaniu. Uruchom serwer programistyczny za pomocą polecenia:

```
python manage.py runserver
```

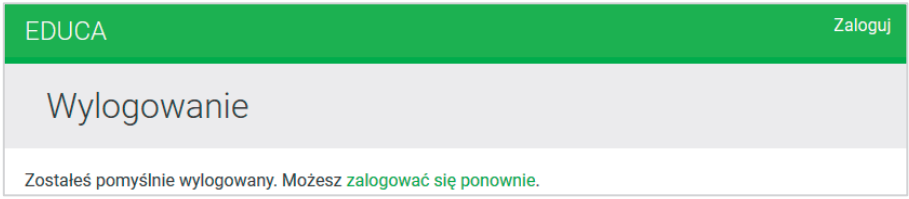
W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/accounts/login/`. Powinieneś zobaczyć stronę logowania (rysunek 12.8).

Zaloguj się przy użyciu poświadczeń superużytkownika. Nastąpi przekierowanie na adres URL `http://127.0.0.1:8000/accounts/profile/`, który jest domyślnym adresem URL przekierowania dla modułu uwierzytelniania. Otrzymasz odpowiedź `HTTP 404`, ponieważ podany adres URL jeszcze nie istnieje. Adres URL do przekierowania użytkowników po pomyślnym zalogowaniu można zdefiniować za pomocą ustawienia `LOGIN_REDIRECT_URL`. Sposób definiowania niestandardowego adresu URL przekierowania zaprezentuję w rozdziale 14., „Renderowanie i buforowanie treści”.



Rysunek 12.8. Strona logowania użytkownika

Otwórz ponownie w swojej przeglądarce stronę `http://127.0.0.1:8000/accounts/login/`. Teraz powinieneś zobaczyć w nagłówku strony przycisk *Wyloguj*. Kliknij przycisk *Wyloguj*. Powinieneś teraz zobaczyć stronę *Wylogowano* (rysunek 12.9).



Rysunek 12.9. Strona wyświetlana po wylogowaniu użytkownika

Pomyślnie utworzyłeś system uwierzytelniania dla CMS.

Podsumowanie

W tym rozdziale dowiedziałeś się, jak używać fikstur do dostarczania początkowych danych dla modeli. Korzystając z dziedziczenia modelu, stworzyłeś elastyczny system do zarządzania różnymi typami treści modułów kursu. Zaimplementowałeś również niestandardowe pole modelu dla obiektów zamówień i stworzyłeś system uwierzytelniania dla platformy e-learningowej.

W następnym rozdziale zaimplementujesz funkcjonalność CMS do zarządzania treścią kursu za pomocą widoków opartych na klasach. Wykorzystasz system grup i uprawnień Django do ograniczania dostępu do widoków oraz zaimplementujesz zestawy formularzy do edycji treści kursów. Za pomocą JavaScript i Django stworzysz również funkcjonalność „przeciągnij i upuść", pozwalająca na zmianę kolejności modułów kursu i ich zawartości.

Dodatkowe zasoby

Oto lista zasobów zawierających dodatkowe informacje związane z tematami omawianymi w tym rozdziale:

- Kod źródłowy przykładów zamieszczonych w tym rozdziale — *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter12*.
- Używanie fikstur Django do testowania — *https://docs.djangoproject.com/en/5.0/topics/testing/tools/#fixture-loading*.
- Migracje danych — *https://docs.djangoproject.com/en/5.0/topics/migrations/#datamigrations*.
- Dziedziczenie klas w Pythonie — *https://docs.python.org/3/tutorial/classes.html#inheritance*.
- Dziedziczenie modeli w Django — *https://docs.djangoproject.com/en/5.0/topics/db/models/#model-inheritance*.
- Tworzenie niestandardowych pól modelu — *https://docs.djangoproject.com/en/5.0/howto/custom-model-fields/*.
- Katalog *static* dla projektu e-learningowego — *https://github.com/PacktPublishing/Django-5-by-Example/tree/main/Chapter12/educa/courses/static*.

Tworzenie systemu zarządzania treścią

Rozdział 13

W poprzednim rozdziale stworzyłeś modele aplikacji dla platformy e-learningowej i nauczyłeś się tworzyć i stosować dla modeli fikstury danych. Utworzyłeś niestandardowe pole modelu do określania kolejności obiektów i zaimplementowałeś uwierzytelnianie użytkowników.

W tym rozdziale dowiesz się, w jaki sposób zbudować dla instruktorów funkcjonalność do opracowywania kursów oraz zarządzania ich zawartością w uniwersalny i wydajny sposób.

Poznasz widoki oparte na klasach, które w porównaniu z widokami opartymi na funkcjach, wykorzystywanymi w poprzednich przykładach, oferują nową perspektywę tworzenia aplikacji. Zapoznasz się również z możliwościami wielokrotnego wykorzystywania kodu i modułowością z zastosowaniem mixinów. Techniki te będziesz mógł zastosować w przyszłych projektach.

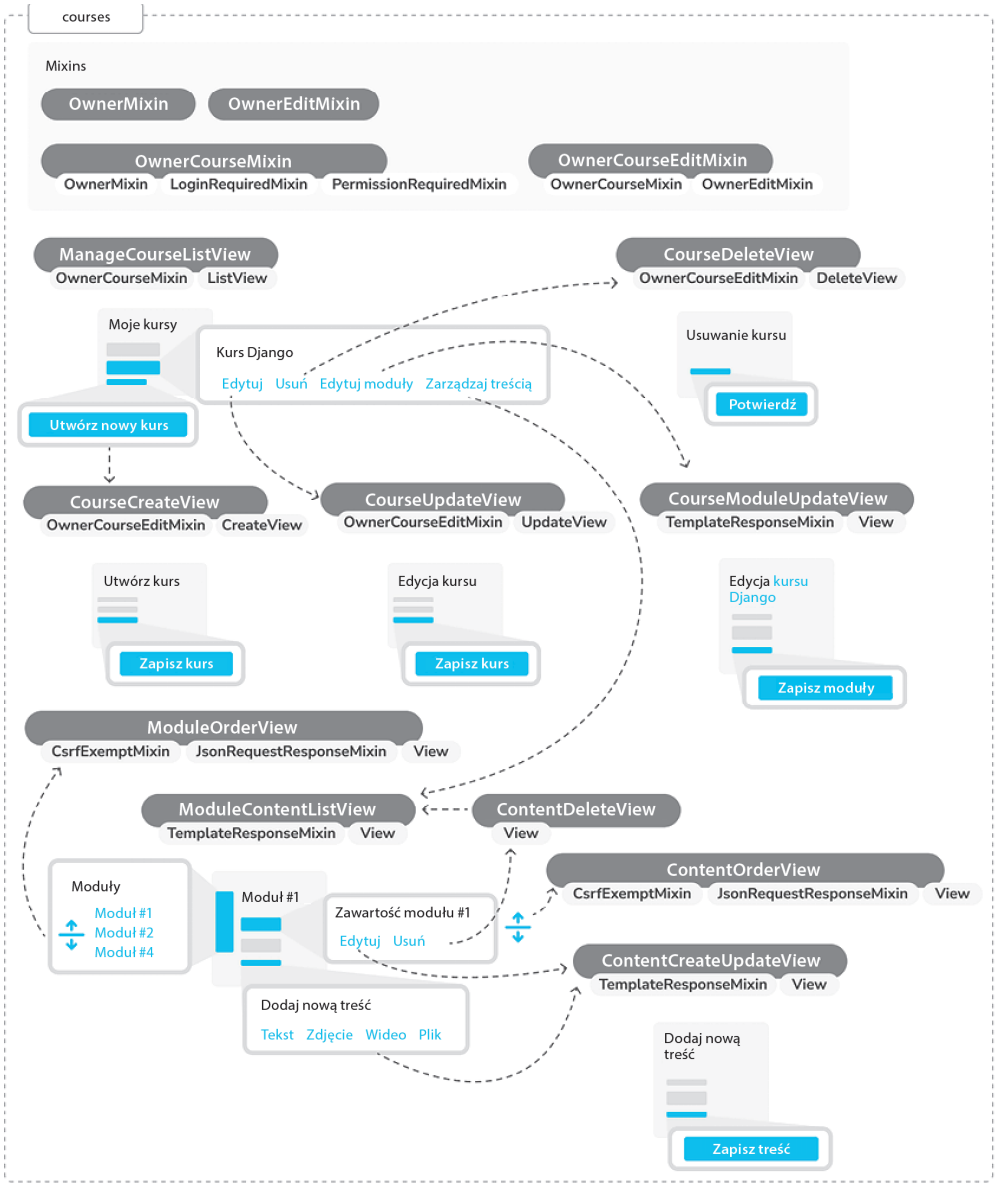
Oto zagadnienia, na których skoncentruję się w tym rozdziale:

- Utworzenie **systemu zarządzania treścią** z wykorzystaniem widoków opartych na klasach i mixinów
- Tworzenie zestawów formularzy i ich modeli w celu edycji modułów kursów i ich zawartości
- Zarządzanie grupami i uprawnieniami
- Implementacja funkcjonalności przeciągnij i upuść do zmiany kolejności modułów i treści

Przegląd funkcjonalności

Reprezentację widoków, szablonów i funkcjonalności, które zbudujemy w tym rozdziale, przedstawiłem na rysunku 13.1.

W tym rozdziale zaimplementujesz różne widoki oparte na klasach. Utworzysz także klasy mixin `OwnerMixin`, `OwnerEditMixin` i `OwnerCourseMixin`, zawierające wspólne funkcjonalności, które będzie można wykorzystać wielokrotnie w innych klasach. Utworzysz też widoki *CRUD* (*Create*, *Read*, *Update*, *Delete* — dosłownie: utwórz, czytaj, aktualizuj, usuwaj) dla modelu `Course`. W tym celu zaimplementujesz widok `ManageCourseListView` do wyświetlania listy kursów, `CourseCreateView` do tworzenia kursów, `CourseUpdateView` do ich aktualizowania i `CourseDeleteView` do ich usuwania. Zbudujesz widok `CourseModuleUpdateView`



Rysunek 13.1. Schemat funkcjonalności zbudowanych w rozdziale 13.

do dodawania, edytowania i usuwania modułów kursu oraz `ModuleContentListView` do wyświetlania zawartości modułu. Zaimplementujesz ponadto widok `ContentCreateUpdateView` do tworzenia i aktualizowania zawartości kursu oraz `ContentDeleteView` do usuwania zawartości. Na koniec zaimplementujesz funkcjonalność przeciągnij i upuść pozwalającą na zmianę kolejności modułów kursu i treści za pomocą widoków, odpowiednio, `Module` ↪`OrderView` i `ContentOrderView`.

Zwróć uwagę, że wszystkie widoki dziedziczące mixin `OwnerCourseMixin` po pomyślnym wykonaniu działania przekierowują użytkownika do widoku `ManageCourseListView`. Te przekierowania, dla uproszczenia, nie zostały dodane do diagramu.

Kod źródłowy przykładów z tego rozdziału można znaleźć pod adresem *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter13*.

Wszystkie pakiety Pythona użyte w tym rozdziale są zawarte w pliku *requirements.txt* dołączonym do plików z kodem źródłowym przykładów z tego rozdziału. Aby zainstalować każdy pakiet Pythona, możesz postępować zgodnie z instrukcjami zamieszczonymi w poniższych podrozdziałach. Możesz także zainstalować wszystkie wymagania jednocześnie za pomocą polecenia `pip install -r requirements.txt`.

Utworzenie systemu zarządzania treścią

Po przygotowaniu wszechstronnego modelu danych możemy przejść do zbudowania systemu zarządzania treścią (ang. *content management system* — CMS). Za pomocą tego systemu instruktorzy będą mogli opracowywać kursy i zarządzać ich treścią. Naszym celem jest dostarczenie następujących funkcjonalności.

- Wyświetlanie listy kursów utworzonych przez danego instruktora.
- Tworzenie, edycja i usuwanie kursów.
- Dodawanie modułów do kursów i zmiana ich kolejności.
- Dodawanie różnych rodzajów treści do poszczególnych modułów oraz zmiana kolejności tej treści.
- Zmiana kolejności modułów i treści kursu

Zacznijmy od opracowania podstawowych widoków CRUD.

Utworzenie widoków opartych na klasach

Przechodzimy do przygotowania widoków przeznaczonych do tworzenia, edycji i usuwania kursów. Do tego celu wykorzystamy widoki oparte na klasach.

Przeprowadź edycję pliku *views.py* aplikacji `courses` i umieść w nim poniższy fragment kodu.

```
from django.views.generic.list import ListView
from .models import Course


class ManageCourseListView(ListView):
    model = Course
    template_name = 'courses/manage/course/list.html'
    def get_queryset(self):
        qs = super().get_queryset()
        return qs.filter(owner=self.request.user)
```

To jest widok `ManageCourseListView`, który dziedziczy po generycznym widoku Django o nazwie `ListView`. Aby pobierać jedynie kursy utworzone przez bieżącego użytkownika,

nadpisujemy metodę `get_queryset()` tego widoku. Aby uniemożliwić użytkownikom edycję, uaktualnianie lub usuwanie nienależących do nich kursów, konieczne jest również nadpisanie metody `get_queryset()` w widokach tworzenia, uaktualniania i usuwania kursów. Kiedy trzeba dostarczyć określone zachowanie do wielu widoków opartych na klasach, wówczas zaleca się zastosowanie tzw. domieszek (ang. *mixins*).

Użycie domieszek w widokach opartych na klasach

Domieszka to specjalnego rodzaju klasa dziedziczenia wielokrotnego. Osobom, które dotąd nie spotkały się z domieszkami w Pythonie, wyjaśniam, że są one rodzajem klas przeznaczonych do dostarczania metod do innych klas, ale nie są przeznaczone do samodzielnego użytku. Pozwala to na tworzenie współdzielonych funkcjonalności, które można modułowo włączać do różnych klas. Aby to zrobić, wystarczy zastosować dziedziczenie tych klas z domieszek. Koncepcja jest podobna do klasy bazowej, ale do rozszerzenia funkcjonalności danej klasy można użyć wielu domieszek.

Domieszek używamy w dwóch podstawowych sytuacjach.

- Chcemy zapewnić klasie obsługę wielu opcjonalnych funkcji.
- Chcemy użyć w kilku klasach konkretnej funkcji.

Framework Django jest dostarczany z kilkoma domieszkami zapewniającymi dodatkową funkcjonalność klasom opartym na widokach. Więcej informacji o domieszkach znajdziesz na stronie *https://docs.djangoproject.com/en/5.0/topics/class-based-views/mixins/*.

Za pomocą klas-domieszek zaimplementujemy teraz wspólne zachowanie dla wielu widoków, a następnie wykorzystamy te klasy w widokach kursu. Przeprowadź edycję pliku *views.py* aplikacji `courses` i zmodyfikuj go w następujący sposób.

```
from django.urls import reverse_lazy
from django.views.generic.edit import CreateView, UpdateView, DeleteView
from django.views.generic.list import ListView
from .models import Course

class OwnerMixin:
    def get_queryset(self):
        qs = super().get_queryset()
        return qs.filter(owner=self.request.user)

class OwnerEditMixin:
    def form_valid(self, form):
        form.instance.owner = self.request.user
        return super().form_valid(form)

class OwnerCourseMixin(OwnerMixin):
    model = Course
    fields = ['subject', 'title', 'slug', 'overview']
    success_url = reverse_lazy('manage_course_list')

class OwnerCourseEditMixin(OwnerCourseMixin, OwnerEditMixin):
    template_name = 'courses/manage/course/form.html'
```

```
class ManageCourseListView(OwnerCourseMixin, ListView):
    template_name = 'courses/manage/course/list.html'

class CourseCreateView(OwnerCourseEditMixin, CreateView):
    pass

class CourseUpdateView(OwnerCourseEditMixin, UpdateView):
    pass

class CourseDeleteView(OwnerCourseMixin, DeleteView):
    template_name = 'courses/manage/course/delete.html'
```

W powyższym kodzie utworzyliśmy domieszki `OwnerMixin` i `OwnerEditMixin`, które wykorzystamy razem w widokach `ListView`, `CreateView`, `UpdateView` i `DeleteView` dostarczanych przez Django. Domieszka `OwnerMixin` implementuje metodę `get_queryset()`, która jest używana przez widoki do pobrania bazowej kolekcji `QuerySet`. Przygotowana domieszka nadpisuje tę metodę, aby filtrować obiekty względem atrybutu `owner` i pobrać jedynie te obiekty, które należą do bieżącego użytkownika (`request.user`).

Z kolei domieszka `OwnerEditMixin` implementuje metodę `form_valid()`. Metoda jest używana przez widoki wykorzystujące domieszkę Django o nazwie `ModelFormMixin`, czyli np. widoki formularzy lub widoki oparte na modelach, takie jak `CreateView` i `UpdateView`. Metoda `form_valid()` jest wywoływana, gdy wysłany formularz będzie prawidłowy.

Domyślne działanie metody polega na zapisaniu egzemplarza (dla formularzy modeli) i przekierowaniu użytkownika do adresu wskazanego przez `success_url`. Nadpisujemy tę metodę, aby automatycznie ustawiała bieżącego użytkownika w atrybucie `owner` zapisywanego obiektu. W ten sposób właściciela obiektu definiujemy automatycznie podczas zapisywania danego obiektu.

Klasa `OwnerMixin` może być używana w widokach prowadzących interakcję z dowolnym modelem zawierającym atrybut `owner`.

Definiujemy również klasę `OwnerCourseMixin` dziedziczącą po `OwnerMixin` i dostarczającą poniższy atrybut widokom potomnym.

- `model`. Model używany jest przez kolekcję `QuerySet`. Atrybut jest stosowany we wszystkich widokach.
- `fields`. Pola modelu wykorzystane do utworzenia formularza modelu dla widoków `CreateView` i `UpdateView`.
- `success_url`. Atrybut używany przez widoki `CreateView` i `UpdateView` do przekierowania użytkownika po zakończonym powodzeniem wysłaniu formularza. Wykorzystujemy adres URL o nazwie zdefiniowanej przez listę `manage_course_list`, którą wkrótce utworzymy.

Domieszkę `OwnerCourseEditMixin` definiujemy z poniższym atrybutem.

- `template_name`. Szablon używany przez widoki `CreateView` i `UpdateView`.

Na koniec przygotowujemy wymienione poniżej widoki przez utworzenie podklas `Owner` ↪`CourseMixin`.

- `ManageCourseListView`. Lista kursów utworzonych przez użytkownika. Dziedziczy po klasach `OwnerCourseMixin` i `ListView`. Definiuje określony atrybut `template_name` dla szablonu w celu wyświetlania listy kursów.
- `CourseCreateView`. Używa formularza modelu do utworzenia nowego obiektu `Course`. Do zbudowania formularza modelu oraz podklas `CreateView` wykorzystuje pola zdefiniowane w domieszce `OwnerCourseEditMixin`. Stosuje szablon zdefiniowany w domieszce `OwnerCourseEditMixin`.
- `CourseUpdateView`. Pozwala na edycję istniejącego obiektu `Course`. Do zbudowania formularza modelu oraz podklas `UpdateView` wykorzystuje pola zdefiniowane w domieszce `OwnerCourseEditMixin`. Stosuje szablon zdefiniowany w domieszce `OwnerCourseEditMixin`.
- `CourseDeleteView`. Dziedziczy po klasie `OwnerCourseMixin` i generycznym widoku `DeleteView`. Definiuje atrybut `template_name` określający szablon do potwierdzenia usunięcia kursu.

Przygotowaliśmy podstawowe widoki przeznaczone do zarządzania kursami. Chociaż w tym projekcie samodzielnie zaimplementowaliśmy widoki CRUD, aplikacja zewnętrzna Neapolitan umożliwia implementację w pojedynczym widoku standardowych widoków listy, szczegółów, tworzenia i usuwania. Więcej informacji o aplikacji Neapolitan znajdziesz na stronie *https://github.com/carltongibson/neapolitan*.

W kolejnym podrozdziale w celu ograniczenia dostępu do tych widoków użyjesz grup uwierzytelniania Django i mechanizmu uprawnień.

Praca z grupami i uprawnieniami

Obecnie dostęp do widoków w celu zarządzania kursami może uzyskać każdy użytkownik. Trzeba więc wprowadzić ograniczenia pozwalające na tworzenie kursów i zarządzanie nimi jedynie instruktorom.

Framework uwierzytelniania w Django zawiera system uprawnień pozwalający na przypisywanie uprawnień użytkownikom i grupom. Domyślnie framework Django generuje dla każdego modelu w zainstalowanych aplikacjach cztery uprawnienia: add (dodawanie), `view` (wyświetlanie), `change` (zmiana) i `delete` (usuwanie). Uprawnienia te odpowiadają działaniom tworzenia nowych egzemplarzy modelu, przeglądania egzemplarzy istniejących oraz ich edytowania i usuwania.

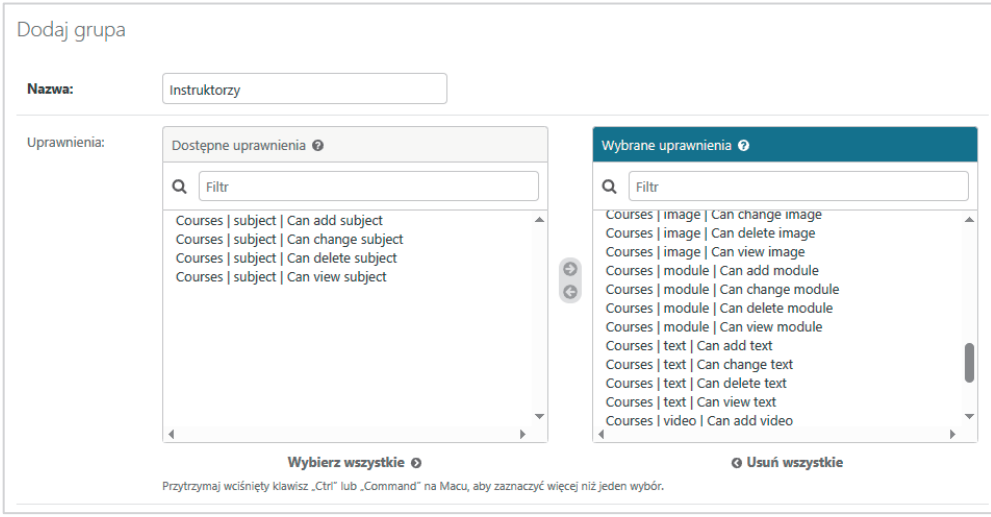
Uprawnienia można przypisywać bezpośrednio poszczególnym użytkownikom lub grupom użytkowników. Takie podejście upraszcza zarządzanie użytkownikami, ponieważ umożliwia grupowanie uprawnień, i zwiększa bezpieczeństwo aplikacji.

Przystępujemy teraz do utworzenia grupy dla instruktorów i przypisania im uprawnień do tworzenia, uaktualniania i usuwania kursów.

Uruchom serwer programistyczny za pomocą polecenia:

```
python manage.py runserver
```

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/admin/auth/group/➥add/` i utwórz nowy obiekt `Group`. Nadaj mu nazwę `Instruktorzy` i wybierz wszystkie uprawnienia aplikacji `courses` poza modelem `Subject`, tak jak pokazałem na rysunku 13.2.

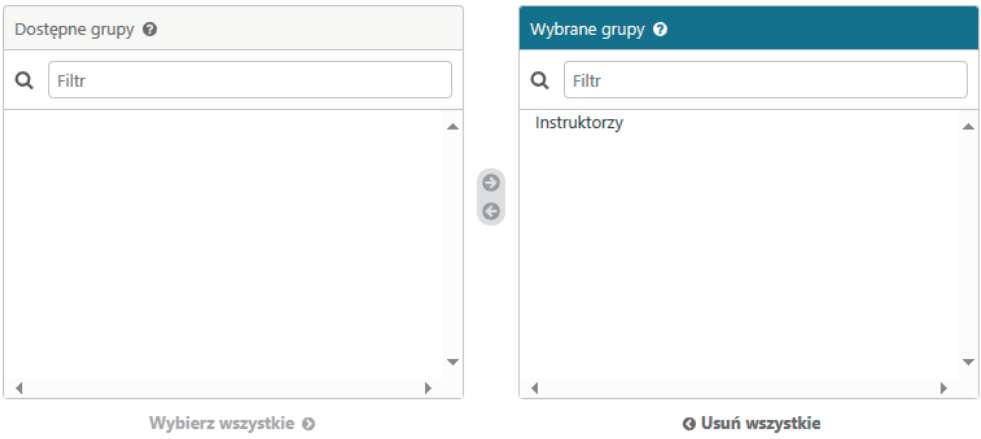


Rysunek 13.2. Uprawnienia dla grupy Instructors

Jak możesz zobaczyć, mamy cztery różne uprawnienia dla poszczególnych modeli, czyli *can view*, *can add*, *can change* i *can delete*. Po wybraniu uprawnień dla tej grupy kliknij przycisk *ZAPISZ*.

Wprawdzie Django automatycznie utworzy uprawnienia dla modeli, ale można również zdefiniować własne. Tworzenia własnych uprawnień w Django nauczysz się w rozdziale 15. „Utworzenie API". Więcej informacji na temat definiowania niestandardowych uprawnień w Django znajdziesz na stronie *https://docs.djangoproject.com/en/5.0/topics/auth/customizing/#custom-permissions*.

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/admin/auth/user/➥add/` i utwórz nowego użytkownika. Przeprowadź edycję nowego użytkownika i dodaj go do grupy *Instruktorzy* (rysunek 13.3).



Rysunek 13.3. Wybór grup dla użytkownika

Użytkownicy dziedziczą uprawnienia grupy, do której zostali dodani. Istnieje również możliwość dodawania pojedynczych uprawnień użytkownikowi za pomocą witryny administracyjnej Django. Użytkownik wraz z atrybutem `is_superuser` o wartości `True` automatycznie otrzymuje wszystkie uprawnienia.

W następnym podpunkcie zastosujemy uprawnienia w praktyce do widoków aplikacji.

Ograniczenie dostępu do widoków opartych na klasach

Ograniczymy teraz dostęp do widoków opartych na klasach, aby tylko użytkownicy z odpowiednimi uprawnieniami mogli dodawać, zmieniać i usuwać obiekty `Course`. W celu ograniczenia dostępu do widoków zamierzamy użyć dwóch domieszek dostarczonych przez aplikację `django.contrib.auth`. Oto one:

- `LoginRequiredMixin`. Powiela funkcjonalność dekoratora `login_required`.
- `PermissionRequiredMixin`. Udziela dostępu do widoku jedynie użytkownikom, którzy mają określone uprawnienia. Pamiętaj, że superużytkownik automatycznie otrzymuje wszystkie uprawnienia.

Przeprowadź edycję pliku *views.py* aplikacji `courses` i umieść w nim poniższe polecenie `import`.

```
from django.contrib.auth.mixins import (
    LoginRequiredMixin,
    PermissionRequiredMixin
)
```

Zdefiniuj klasę `OwnerCourseMixin` dziedziczącą po `LoginRequiredMixin`, w następujący sposób.

```
class OwnerCourseMixin(
    OwnerMixin, LoginRequiredMixin, PermissionRequiredMixin):
    model = Course
    fields = ['subject', 'title', 'slug', 'overview']
    success_url = reverse_lazy('manage_course_list')
```

Następnie dodaj do widoków tworzenia, uaktualniania i usuwania kursów atrybut `permission_required`, tak jak pokazałem w poniższym fragmencie kodu.

```
class ManageCourseListView(OwnerCourseMixin, ListView):
    template_name = 'courses/manage/course/list.html'
    permission_required = 'courses.view_course'

class CourseCreateView(OwnerCourseEditMixin, CreateView):
    permission_required = 'courses.add_course'

class CourseUpdateView(OwnerCourseEditMixin, UpdateView):
    permission_required = 'courses.change_course'

class CourseDeleteView(OwnerCourseMixin, DeleteView):
    template_name = 'courses/manage/course/delete.html'
    permission_required = 'courses.delete_course'
```

Domieszka `PermissionRequiredMixin` sprawdza, czy użytkownik uzyskujący dostęp do widoku ma uprawnienia wymienione w atrybucie `permission_required`. Po wprowadzeniu powyższych zmian widoki aplikacji są dostępne jedynie dla użytkowników, którzy mają odpowiednie uprawnienia.

Przystępujemy teraz do przygotowania adresów URL dla dodanych widoków. Utwórz nowy plik w katalogu aplikacji `courses` i nadaj mu nazwę *urls.py*. Następnie w pliku umieść poniższy fragment kodu.

```
from django.urls import path
from . import views

urlpatterns = [
    path(
        'mine/',
        views.ManageCourseListView.as_view(),
        name='manage_course_list'
    ),
    path(
        'create/',
        views.CourseCreateView.as_view(),
        name='course_create'
    ),
    path(
        '<pk>/edit/',
        views.CourseUpdateView.as_view(),
        name='course_edit'
    ),
    path(
        '<pk>/delete/',
        views.CourseDeleteView.as_view(),
        name='course_delete'
    ),
]
```

Są to wzorce adresów URL dla widoków tworzenia, edycji i usuwania kursu. Parametr `pk` odnosi się do pola klucza głównego. Nazwa `pk` to skrót od *primary key* (klucz główny). Pole, które służy jako klucz główny, ma każdy model Django. Domyślnie kluczem głównym jest automatycznie wygenerowane pole `id`. Generyczne widoki Django dla pojedynczych obiektów pobierają obiekt na podstawie jego pola `pk`. Przeprowadź edycję głównego pliku *urls.py* projektu `educa` i dołącz do niego wzorce adresów URL aplikacji `courses`, tak jak pokazałem poniżej.

Nowy kod został wyróżniony pogrubioną czcionką.

```
from django.conf import settings
from django.conf.urls.static import static
from django.contrib import admin
from django.contrib.auth import views as auth_views
from django.urls import include, path

urlpatterns = [
    path(
        'accounts/login/', auth_views.LoginView.as_view(), name='login'
    ),
    path(
        'accounts/logout/',
        auth_views.LogoutView.as_view(),
        name='logout'
```

```
    ),
    path('admin/', admin.site.urls),
    path('course/', include('courses.urls')),
]

if settings.DEBUG:
    urlpatterns += static(
        settings.MEDIA_URL, document_root=settings.MEDIA_ROOT
    )
```

Kolejnym krokiem jest zdefiniowanie szablonów dla przygotowanych wcześniej widoków. W katalogu *templates* aplikacji `courses` utwórz poniższą strukturę katalogów i plików.

```
courses/
     manage/
         course/
             list.html
             form.html
             delete.html
```

Przeprowadź edycję szablonu *courses/manage/course/list.html* i umieść w nim poniższy fragment kodu.

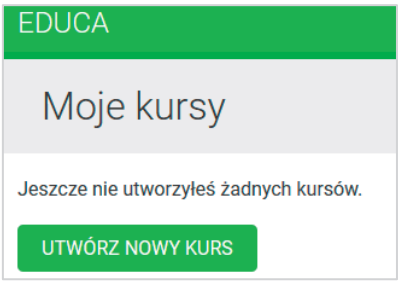
```
{% extends "base.html" %}

{% block title %}Moje kursy{% endblock %}

{% block content %}
  <h1>Moje kursy</h1>
  <div class="module">
    {% for course in object_list %}
      <div class="course-info">
        <h3>{{ course.title }}</h3>
        <p>
          <a href="{% url "course_edit" course.id %}">Edytuj</a>
          <a href="{% url "course_delete" course.id %}">Usuń</a>
        </p>
      </div>
    {% empty %}
      <p>Jeszcze nie utworzyłeś żadnych kursów.</p>
    {% endfor %}
    <p>
      <a href="{% url "course_create" %}" class="button">Utwórz nowy kurs</a>
    </p>
  </div>
{% endblock %}
```

Jest to szablon widoku `ManageCourseListView`. W tym szablonie wyświetlamy listę kursów utworzonych przez bieżącego użytkownika. Ponadto mamy linki pozwalające na edytowanie lub usunięcie kursu, a także link do budowania nowych kursów.

Uruchom serwer programistyczny za pomocą polecenia:

```
python manage.py runserver
```

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/accounts/login/?next=/course/mine/` i zaloguj się za pomocą konta użytkownika należącego do grupy *Instruktorzy*. Po zalogowaniu zostaniesz przekierowany na stronę o adresie URL *`http://127.0.0.1:8000/course/mine/`*. Wyświetli się strona podobna do pokazanej na rysunku 13.4.



Rysunek 13.4. Strona instruktora kursów, na której nie ma kursów

Na wyświetlonej stronie będzie znajdowała się lista wszystkich kursów utworzonych przez bieżącego użytkownika.

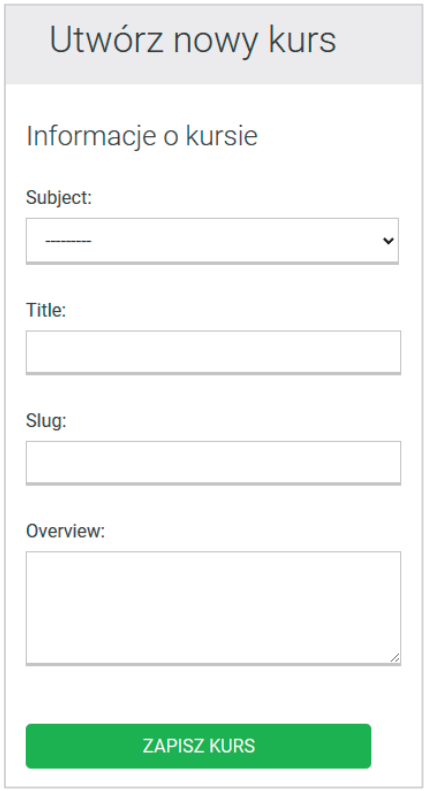
Przechodzimy teraz do przygotowania szablonu wyświetlającego formularz dla widoków tworzenia i uaktualniania kursu. Przeprowadź edycję szablonu *courses/manage/course/form.html* i umieść w nim poniższy fragment kodu.

```
{% extends "base.html" %}

{% block title %}
  {% if object %}
    Edytuj kurs "{{ object.title }}"
  {% else %}
    Utwórz nowy kurs
  {% endif %}
{% endblock %}

{% block content %}
  <h1>
    {% if object %}
      Edytuj kurs "{{ object.title }}"
    {% else %}
      Utwórz nowy kurs
    {% endif %}
  </h1>
  <div class="module">
    <h2>Informacje o kursie</h2>
    <form method="post">
      {{ form.as_p }}
      {% csrf_token %}
      <p><input type="submit" value="Zapisz kurs"></p>
    </form>
  </div>
{% endblock %}
```

Szablon *form.html* jest używany zarówno przez widok `CourseCreateView`, jak i widok `CourseUpdateView`. W tym szablonie sprawdzamy, czy w kontekście została ustawiona

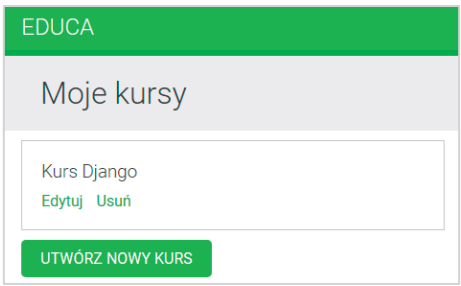
zmienna `object`. Jeżeli zmienna `object` istnieje w kontekście, wówczas wiadomo, że uaktualniamy istniejący kurs i podajemy jego nazwę w tytule strony. Natomiast brak wymienionej zmiennej oznacza tworzenie nowego obiektu `Course`.

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/course/mine/` i kliknij przycisk *UTWÓRZ NOWY KURS*. Powinieneś zobaczyć stronę pokazaną na rysunku 13.5.



Rysunek 13.5. Formularz do utworzenia nowego kursu

Wypełnij formularz i kliknij przycisk *ZAPISZ KURS*. Kurs będzie zapisany, a Ty zostaniesz przekierowany na stronę wyświetlającą listę kursów (rysunek 13.6).



Rysunek 13.6. Strona kursów instruktora z jednym kursem

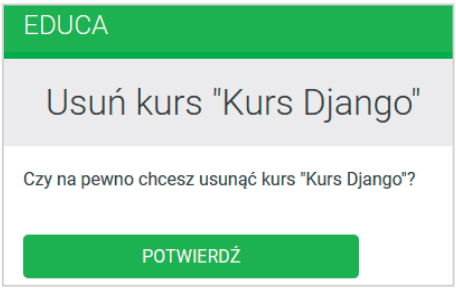
Kliknij link *Edytuj* dla utworzonego przed chwilą kursu. Ponownie zobaczysz ten sam formularz, ale tym razem będziemy edytować istniejący obiekt `Course`, zamiast tworzyć nowy.

Na koniec przeprowadź edycję szablonu *courses/manage/course/delete.html* i umieść w nim poniższy fragment kodu.

```
{% extends "base.html" %}

{% block title %}Usuń kurs{% endblock %}

{% block content %}
  <h1>Usuń kurs "{{ object.title }}"</h1>
  <div class="module">
    <form action="" method="post">
      {% csrf_token %}
      <p>Czy na pewno chcesz usunąć kurs "{{ object }}"?</p>
      <input type="submit" value="Potwierdź">
    </form>
  </div>
{% endblock %}
```

Jest to szablon dla widoku `CourseDeleteView`. Dziedziczy on po widoku `DeleteView` oferowanym przez Django i oczekuje od użytkownika potwierdzenia operacji usunięcia obiektu.

Powróć do przeglądarki internetowej i kliknij link *Usuń* wygenerowany dla utworzonego przed chwilą kursu. Powinieneś zobaczyć stronę pokazaną na rysunku 13.7.



Rysunek 13.7. Strona do potwierdzania usunięcia kursu

Kliknij przycisk *POTWIERDŹ*. Kurs będzie usunięty, a Ty ponownie zostaniesz przekierowany na stronę wyświetlającą listę kursów.

Po wprowadzonych zmianach instruktorzy mogą tworzyć, edytować i usuwać kursy. Kolejną funkcjonalnością, jaką trzeba im zapewnić w systemie zarządzania treścią, jest możliwość dodawania modułów i treści kursów. Pracę rozpoczynamy od zarządzania modułami kursu.

Zarządzanie modułami kursu i treścią

Przystępujemy do zbudowania systemu zarządzania modułami kursu i ich zawartością. W tym celu utworzymy formularze, które można wykorzystać do zarządzania wieloma modułami dla jednego kursu i różnymi rodzajami treści dla każdego modułu. Zarówno moduły, jak i treści będą musiały być zgodne z konkretnym zamówieniem i powinniśmy być w stanie zmienić ich kolejność za pomocą CMS.

Użycie zbiorów formularzy dla modułów kursów

Framework Django jest dostarczany wraz z warstwą abstrakcji przeznaczoną do pracy z wieloma formularzami na tej samej stronie. Tego rodzaju grupy formularzy są nazywane *zbiorami formularzy* (ang. *formset*). Taki zbiór formularzy pozwala na zarządzanie wieloma egzemplarzami określonych klas `Form` lub `ModelForm`. Wszystkie formularze są wysyłane jednocześnie, a zbiór formularzy zajmuje się obsługą zadań, takich jak ustalenie początkowej liczby wyświetlanych formularzy, ograniczenie maksymalnej liczby formularzy, jakie mogą być wysłane, i weryfikacja wszystkich formularzy.

Zbiór formularzy zawiera metodę `is_valid()` przeznaczoną do jednoczesnej weryfikacji wszystkich formularzy. Istnieje również możliwość dostarczenia danych początkowych dla formularzy i określenia liczby dodatkowych pustych formularzy do wyświetlenia. Więcej informacji na temat zbioru formularzy znajdziesz na stronie *https://docs.djangoproject.com/en/5.0/topics/forms/formsets/*, a o modelach zbioru formularzy — na stronie *https://docs.djangoproject.com/en/5.0/topics/forms/modelforms/#model-formsets*.

Ponieważ kursy są podzielone na różną liczbę modułów, więc sensowne wydaje się wykorzystanie w tym miejscu zbioru formularzy. Utwórz plik *forms.py* w katalogu aplikacji `courses` i umieść w nim poniższy fragment kodu.

```
from django.forms.models import inlineformset_factory
from .models import Course, Module

ModuleFormSet = inlineformset_factory(
    Course,
    Module,
    fields=['title', 'description'],
    extra=2,
    can_delete=True
)
```

Jest to zbiór formularzy `ModuleFormSet`. Zbudowaliśmy go za pomocą funkcji `inlineformset_factory()` dostarczonej przez Django. Osadzone elementy zbioru formularzy to niewielkie abstrakcje na bazie klasy `Formset`, które ułatwiają pracę z powiązanymi obiektami. Wymieniona funkcja pozwala na dynamiczne utworzenie modelu zbioru formularzy dla obiektów `Module` powiązanych z obiektami `Course`.

Do zbudowania zbioru formularzy wykorzystujemy następujące parametry.

- `fields`. Pola, które będą umieszczone w każdym formularzu znajdującym się w zbiorze formularzy.
- `extra`. Parametr pozwala na określenie liczby dodatkowych pustych formularzy do wyświetlenia w zbiorze formularzy.
- `can_delete`. Wartość `True` tego parametru oznacza, że Django umieści w każdym formularzu pole typu `Boolean`, które będzie renderowane w postaci pola wyboru. W ten sposób otrzymujemy możliwość oznaczenia obiektów przeznaczonych do usunięcia.

Przeprowadź edycję pliku *views.py* aplikacji `courses` i umieść w nim poniższy fragment kodu.

```
from django.shortcuts import get_object_or_404, redirect
from django.views.generic.base import TemplateResponseMixin, View
from .forms import ModuleFormSet

class CourseModuleUpdateView(TemplateResponseMixin, View):
    template_name = 'courses/manage/module/formset.html'
    course = None

    def get_formset(self, data=None):
        return ModuleFormSet(instance=self.course, data=data)

    def dispatch(self, request, pk):
        self.course = get_object_or_404(
            Course, id=pk, owner=request.user
        )
        return super().dispatch(request, pk)

    def get(self, request, *args, **kwargs):
        formset = self.get_formset()
        return self.render_to_response(
            {'course': self.course, 'formset': formset}
        )

    def post(self, request, *args, **kwargs):
        formset = self.get_formset(data=request.POST)
        if formset.is_valid():
            formset.save()
            return redirect('manage_course_list')
        return self.render_to_response(
            {'course': self.course, 'formset': formset}
        )
```

Obsługą zbioru formularzy przeznaczonego do dodawania, uaktualniania i usuwania modułów dla określonego kursu zajmuje się widok `CourseModuleUpdateView`. Widok ten dziedziczy po następujących domieszkach i widokach.

- `TemplateResponseMixin`. Domieszka jest odpowiedzialna za wyrenderowanie szablonów i zwrócenie odpowiedzi HTTP. Wymaga atrybutu `template_name` wskazującego szablon do wygenerowania i oferuje metodę `render_to_response()` w celu przekazania szablonu do kontekstu i wygenerowania go.
- `View`. Dostarczany przez Django podstawowy widok oparty na klasie.

W przedstawionym widoku implementujemy metody:

- `get_formset()`. Definiujemy tę metodę, aby uniknąć powielania kodu odpowiedzialnego za utworzenie zbioru formularzy. Metoda tworzy obiekt `ModuleFormSet` dla danego obiektu `Course` wraz z opcjonalnymi danymi.
- `dispatch()`. Metoda jest dostarczana przez klasę `View`. Pobiera żądanie HTTP i jego parametry, a następnie próbuje delegować do zapisanej małymi literami metody dopasowanej do metody HTTP następujące żądania: żądanie `GET` jest delegowane do metody `get()`, natomiast żądanie `POST` do metody `post()`. W tej metodzie w celu pobrania obiektu `Course` dla danego parametru `id` należącego do bieżącego użytkownika używamy skrótu w postaci funkcji `get_object_or_404()`.

Ten kod został umieszczony w metodzie `dispatch()`, ponieważ kurs musimy pobrać zarówno w żądaniu `GET`, jak i `POST`. Następnie, aby kurs był dostępny dla pozostałych metod, zapisujemy go w atrybucie `course` widoku.

- `get()`. Metoda wykonywana dla żądań `GET`. Budujemy pusty element `ModuleFormSet` i generujemy go w szablonie wraz z bieżącym obiektem `Course` za pomocą metody `render_to_response()` dostarczanej przez `TemplateResponseMixin`.
- `post()`. Metoda wykonywana dla żądań `POST`. W wymienionej metodzie są wykonywane następujące zadania.
  1) Utworzenie egzemplarza `ModuleFormSet` na podstawie wysłanych danych.
  2) Wykonanie metody `is_valid()` dla zbioru formularzy i weryfikacja wszystkich znajdujących się w nim formularzy.
  3) Zapisanie zbioru formularzy za pomocą wywołania metody `save()`, pod warunkiem że zbiór formularzy jest prawidłowy. Na tym etapie wszelkie wprowadzone zmiany, takie jak dodanie, uaktualnienie lub zaznaczenie modułu do usunięcia, będą zastosowane w bazie danych. Następnie użytkownik zostaje przekierowany do adresu URL wskazanego przez `manage_course_list`. Jeżeli zbiór formularzy jest nieprawidłowy, następuje wygenerowanie szablonu wraz z odpowiednimi komunikatami o błędach.

Przeprowadź edycję pliku *urls.py* aplikacji `courses` i umieść w nim poniższy wzorzec adresu URL.

```
path(
    '<pk>/module/',
    views.CourseModuleUpdateView.as_view(),
    name='course_module_update'
),
```

Utwórz nowy podkatalog w katalogu szablonów *courses/manage/* i nadaj mu nazwę *module*. Następnie stwórz szablon *courses/manage/module/formset.html* i umieść w nim poniższy fragment kodu.

```
{% extends "base.html" %}

{% block title %}
  Edycja "{{ course.title }}"
{% endblock %}

{% block content %}
  <h1>Edycja "{{ course.title }}"</h1>
  <div class="module">
    <h2>Moduły kursu</h2>
    <form method="post">
      {{ formset }}
      {{ formset.management_form }}
      {% csrf_token %}
      <input type="submit" value="Zapisz moduły">
    </form>
  </div>
{% endblock %}
```

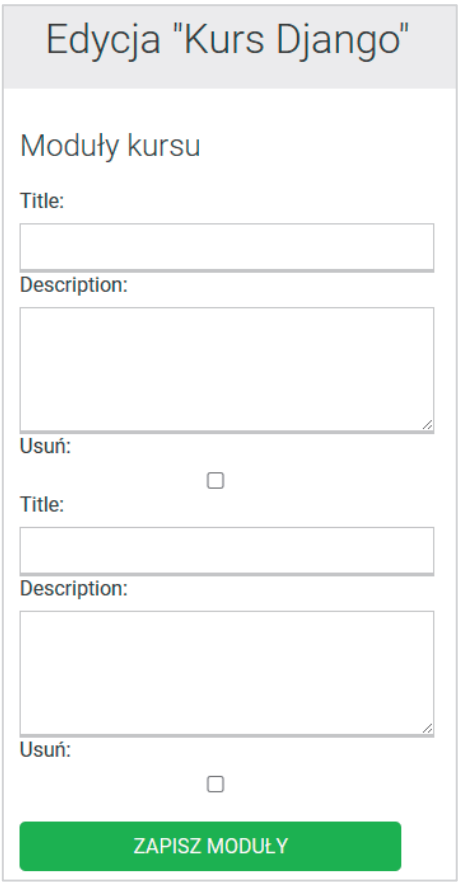
W szablonie tym tworzymy element HTML `<form>`, w którym dołączamy zbiór formularzy. Ponadto za pomocą zmiennej `{{ formset.management_form }}` dołączany jest także formularz przeznaczony do zarządzania zbiorem formularzy. Ten formularz zarządzający zawiera ukryte pola przeznaczone do kontrolowania początkowej, całkowitej, minimalnej i maksymalnej liczby formularzy.

W szablonie *courses/manage/course/list.html* umieść poniższy link do adresu URL wskazywanego przez `course_module_update`. Nowy link umieść poniżej linków przeznaczonych do edytowania i usunięcia kursu.

```
<a href="{% url "course_edit" course.id %}">Edytuj</a>
<a href="{% url "course_delete" course.id %}">Usuń</a>
<a href="{% url "course_module_update" course.id %}">Edytuj moduły</a>
```

W ten sposób w module kursu zamieściliśmy link umożliwiający jego edycję.

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/course/mine/`. Utwórz kurs i kliknij wygenerowany dla kursu link *Edytuj moduły*. Powinieneś zobaczyć zbiór formularzy, który pokazałem na rysunku 13.8.



Rysunek 13.8. Strona edycji kursu włącznie ze zbiorem formularzy dla modułów kursu

Ten zbiór formularzy zawiera formularz dla każdego obiektu `Module` znajdującego się w kursie. Po nich znajdują się dwa dodatkowe, puste formularze, ponieważ w `Module ↪FormSet` użyliśmy opcji `extra=2`. Podczas zapisywania zbioru formularzy Django wykorzysta dwa dodatkowe pola na dodanie nowych modułów.

Jak mogłeś zauważyć, zbiory formularzy są bardzo przydatne do zarządzania wieloma egzemplarzami formularzy na jednej stronie. Zbiory formularzy upraszczają proces efektywnego gromadzenia i walidacji danych pochodzących z grupy podobnych formularzy.

Po zapoznaniu Cię z działaniem zbiorów formularzy zaprezentuję ich zaawansowane możliwości. Pokażę, jak można dynamicznie tworzyć formularze, które dostosowują się do różnych typów treści dodawanych do modułów kursu.

Dodanie treści do modułów kursów

Potrzebne jest teraz rozwiązanie pozwalające na dodanie treści do modułów kursu. Mamy cztery różne rodzaje treści: tekst, obrazy, klipy wideo i pliki. Możemy rozważyć utworzenie czterech różnych widoków przeznaczonych do tworzenia treści, po jednym dla każdego modelu. Zastosujemy jednak nieco ogólniejsze podejście i przygotujemy widok obsługujący tworzenie lub uaktualnianie obiektów dowolnego modelu treści. Formularz dla tego widoku będzie tworzony dynamicznie, zgodnie z typem zawartości, którą instruktor chce dodać do kursu: `Text`, `Video`, `Image` lub `File`.

Przeprowadź edycję pliku *views.py* aplikacji `courses` i umieść w nim poniższy fragment kodu.

```
from django.apps import apps
from django.forms.models import modelform_factory
from .models import Module, Content

class ContentCreateUpdateView(TemplateResponseMixin, View):
    module = None
    model = None
    obj = None
    template_name = 'courses/manage/content/form.html'

    def get_model(self, model_name):
        if model_name in ['text', 'video', 'image', 'file']:
            return apps.get_model(
                app_label='courses', model_name=model_name
            )
        return None

    def get_form(self, model, *args, **kwargs):
        Form = modelform_factory(
            model, exclude=['owner', 'order', 'created', 'updated']
        )
        return Form(*args, **kwargs)

    def dispatch(self, request, module_id, model_name, id=None):
        self.module = get_object_or_404(
            Module, id=module_id, course__owner=request.user
```

```
        )
        self.model = self.get_model(model_name)
        if id:
            self.obj = get_object_or_404(
                self.model, id=id, owner=request.user
            )
        return super().dispatch(request, module_id, model_name, id)
```

Jest to część pierwsza widoku `ContentCreateUpdateView`. Pozwala ona na tworzenie i uaktualnianie treści różnych modeli. W widoku zdefiniowano następujące metody.

- `get_model()`. W tej funkcji sprawdzamy, czy podana nazwa modelu jest jednym z czterech modeli treści, takich jak tekst, wideo, z czterech modeli treści, takich jak tekst, wideo, obraz lub plik. Następnie za pomocą modułu Django `apps` pobieramy właściwą klasę dla danej nazwy modelu. Jeżeli sprawdzana nazwa modelu nie jest jedną z prawidłowych, funkcja zwróci `None`.
- `get_form()`. W tej funkcji dynamicznie budujemy formularz za pomocą dostarczanej przez framework funkcji `modelform_factory()`. Ponieważ ma być zbudowany formularz dla modeli `Text`, `Video`, `Image` lub `File`, używamy parametru `exclude` w celu wskazania pól, które mają być wykluczone z formularza, a wszystkie pozostałe atrybuty będą dołączone automatycznie. W ten sposób nie musimy wiedzieć, które pola mają być dołączane w zależności od modelu.
- `dispatch()`. Funkcja pobiera wymienione poniżej parametry adresu URL i przechowuje jako atrybuty klasy — odpowiednio — moduł, model i obiekt treści.
  - `module_id`. Identyfikator modułu, którego treść jest lub będzie powiązana z obiektem.
  - `model_name`. Nazwa modelu treści, która będzie utworzona lub uaktualniona.
  - `id`. Identyfikator uaktualnianego obiektu. Wartość `None` oznacza utworzenie nowego obiektu.

Dodaj do widoku `ContentCreateUpdateView` przedstawione poniżej metody `get()` i `post()`.

```
def get(self, request, module_id, model_name, id=None):
    form = self.get_form(self.model, instance=self.obj)
    return self.render_to_response(
        {'form': form, 'object': self.obj}
    )

def post(self, request, module_id, model_name, id=None):
    form = self.get_form(
        self.model,
        instance=self.obj,
        data=request.POST,
        files=request.FILES
    )
    if form.is_valid():
        obj = form.save(commit=False)
        obj.owner = request.user
        obj.save()
```

```
            if not id:
                # Nowa treść.
                Content.objects.create(module=self.module, item=obj)
            return redirect('module_content_list', self.module.id)
        return self.render_to_response(
            {'form': form, 'object': self.obj}
        )
```

Oto sposób działania dodanych metod:

- `get()`. Metoda jest wykonywana po otrzymaniu żądania `GET`. Budujemy tutaj model formularza dla uaktualnianego egzemplarza `Text`, `Video`, `Image` lub `File`. W przeciwnym razie nie jest przekazywany żaden egzemplarz do utworzenia nowego obiektu, ponieważ w przypadku niedostarczenia identyfikatora wartość `self.obj` wynosi `None`.
- `post()`. Metoda jest wykonywana po otrzymaniu żądania `POST`. Budujemy tutaj egzemplarz `ModelForm` na podstawie przekazanych danych i plików. Następnie przeprowadzamy jego walidację. Jeżeli formularz jest prawidłowy, tworzymy nowy obiekt i przed jego umieszczeniem w bazie danych przypisujemy `request.user` jako właściciela. Przeprowadzamy sprawdzenie pod kątem parametru `id`. Jeżeli nie został podany, wówczas wiadomo, że użytkownik tworzy nowy obiekt, a nie uaktualnia istniejącego. W przypadku nowego obiektu tworzymy obiekt `Content` na podstawie danego modułu i wiążemy z nim nową treść.

Przeprowadź edycję pliku *urls.py* aplikacji `courses` i umieść w nim poniższe wzorce adresów URL.

```
path(
    'module/<int:module_id>/content/<model_name>/create/',
    views.ContentCreateUpdateView.as_view(),
    name='module_content_create'
),
path(
    'module/<int:module_id>/content/<model_name>/<id>/',
    views.ContentCreateUpdateView.as_view(),
    name='module_content_update'
),
```

Oto nowe wzorce adresów URL:

- `module_content_create`. Wzorzec ten odpowiada za utworzenie nowego obiektu tekstu, klipu wideo, obrazu lub pliku i jego dodania do modułu. Ten wzorzec zawiera parametry `module_id` i `model_name`. Pierwszy z nich pozwala na połączenie nowego obiektu treści z danym modułem. Natomiast drugi określa model treści, dla którego zostanie zbudowany formularz.
- `module_content_update`. Wzorzec ten odpowiada za uaktualnienie istniejącego obiektu tekstu, klipu wideo, obrazu lub pliku. Ten wzorzec zawiera parametry `module_id`, `model_name` i `id` identyfikujące uaktualnianą treść.

Utwórz nowy podkatalog w katalogu szablonów *courses/manage* i nadaj mu nazwę `content`. Następnie utwórz szablon *courses/manage/content/form.html* i umieść w nim poniższy fragment kodu.

```
{% extends "base.html" %}

{% block title %}
  {% if object %}
    Edycja treści "{{ object.title }}"
  {% else %}
    Dodanie nowej treści
  {% endif %}
{% endblock %}

{% block content %}
  <h1>
    {% if object %}
      Edycja treści "{{ object.title }}"
    {% else %}
      Dodanie nowej treści
    {% endif %}
  </h1>
  <div class="module">
    <h2>Informacje o kursie</h2>
    <form action="" method="post" enctype="multipart/form-data">
      {{ form.as_p }}
      {% csrf_token %}
      <p><input type="submit" value="Zapisz treść"></p>
    </form>
  </div>
{% endblock %}
```

Jest to szablon dla widoku `ContentCreateUpdateView`. W tym szablonie sprawdzamy, czy w kontekście została ustawiona zmienna `object`. Jeżeli zmienna istnieje, wiadomo, że uaktualniamy istniejący obiekt. W przeciwnym razie wiemy, że jest tworzony nowy obiekt.

Do elementu HTML `<form>` dodaliśmy atrybut `enctype="multipart/form-data"`, ponieważ formularz umożliwia przekazywanie plików dla modeli treści `File` i `Image`.

Uruchom serwer programistyczny. Otwórz w przeglądarce stronę `http://127.0.0.1:8000/course/mine/`, dla istniejącego kursu, kliknij *Edytuj moduły* i utwórz moduł.

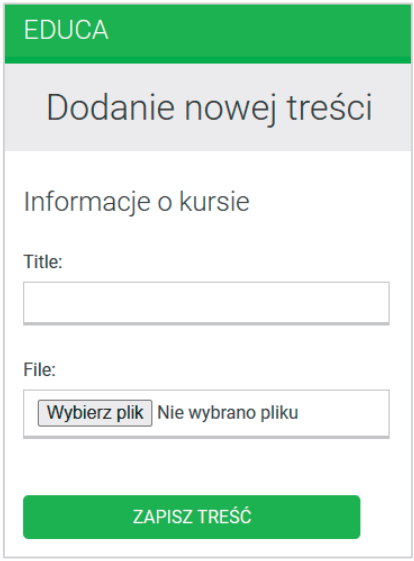
Przejdź do powłoki Pythona za pomocą polecenia:

```
python manage.py shell
```

Uzyskaj identyfikator ostatnio utworzonego modułu, tak jak pokazałem poniżej.

```
>>> from courses.models import Module
>>> Module.objects.latest('id').id
6
```

Uruchom serwer programistyczny. W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/course/module/6/content/image/create/`. Jeżeli trzeba, zmień identyfikator modułu w adresie URL na ten, który wcześniej odczytałeś. Powinieneś zobaczyć formularz przeznaczony do utworzenia obiektu `Image` pokazany na rysunku 13.9.



Rysunek 13.9. Formularz dodawania nowej treści kursu

Nie wysyłaj jeszcze formularza. Gdy spróbujesz to zrobić, operacja zakończy się niepowodzeniem, ponieważ nie zdefiniowaliśmy wzorca adresu URL `module_content_list`. Tym zajmiemy się nieco później.

Potrzebny jest widok pozwalający na usuwanie treści. Przeprowadź edycję pliku *views.py* aplikacji `courses` i umieść w nim poniższy fragment kodu.

```
class ContentDeleteView(View):
    def post(self, request, id):
        content = get_object_or_404(
            Content, id=id, module__course__owner=request.user
        )
        module = content.module
        content.item.delete()
        content.delete()
        return redirect('module_content_list', module.id)
```

Widok `ContentDeleteView` pobiera obiekt `content` o podanym identyfikatorze, a następnie usuwa powiązane z nim obiekty `Text`, `Video`, `Image` lub `File`. Na koniec usunięty będzie obiekt `content`, a użytkownik zostanie przekierowany na stronę wskazywaną przez wzorzec adresu URL zdefiniowany w `module_content_list`, co pozwoli na wyświetlenie pozostałej treści modułu.

Przeprowadź edycję pliku *urls.py* aplikacji `courses` i umieść w nim poniższy wzorzec adresu URL.

```
path(
    'content/<int:id>/delete/',
    views.ContentDeleteView.as_view(),
    name='module_content_delete'
),
```

W tym momencie instruktorzy mogą już z łatwością tworzyć, edytować i usuwać treść. Podejście, którego nauczyłeś się w tym podrozdziale, jest bardzo przydatne do zarządzania formularzami z różnorodnymi danymi w generyczny sposób. Metodę tę można wykorzystać w innych sytuacjach, np. kiedy trzeba zastosować elastyczne rozwiązanie do obsługi danych wejściowych.

W następnym punkcie zajmiemy się tworzeniem widoków i szablonów do wyświetlania modułów i treści kursu.

Zarządzanie modułami i treścią

Przygotowaliśmy widoki przeznaczone do tworzenia, edycji i usuwania modułów kursu oraz jego treści. Teraz potrzebny jest sposób na wyświetlenie wszystkich modułów w danym kursie oraz treści dla konkretnego modułu.

Przeprowadź edycję pliku *views.py* aplikacji `courses` i umieść w nim poniższy fragment kodu.

```
class ModuleContentListView(TemplateResponseMixin, View):
    template_name = 'courses/manage/module/content_list.html'

    def get(self, request, module_id):
        module = get_object_or_404(
            Module, id=module_id, course__owner=request.user
        )
        return self.render_to_response({'module': module})
```

Jest to widok `ModuleContentListView`. Pobiera on obiekt `Module` o podanym identyfikatorze należącym do bieżącego użytkownika i generuje szablon wraz z danym modułem.

Przeprowadź edycję pliku *urls.py* aplikacji `courses` i umieść w nim poniższy wzorzec adresu URL.

```
path(
    'module/<int:module_id>/',
    views.ModuleContentListView.as_view(),
    name='module_content_list'
),
```

Utwórz nowy szablon w katalogu *templates/courses/manage/module* i nadaj mu nazwę *content_list.html*. Następnie w pliku umieść poniższy fragment kodu.

```
{% extends "base.html" %}

{% block title %}
  Moduł {{ module.order|add:1 }}: {{ module.title }}
{% endblock %}

{% block content %}
{% with course=module.course %}
  <h1>Kurs "{{ course.title }}"</h1>
  <div class="contents">
    <h3>Moduły</h3>
    <ul id="modules">
```

```
    {% for m in course.modules.all %}
       <li data-id="{{ m.id }}" {% if m == module %}
        class="selected"{% endif %}>
         <a href="{% url "module_content_list" m.id %}">
          <span>
            Moduł <span class="order">{{ m.order|add:1 }}</span>
          </span>
          <br>
          {{ m.title }}
         </a>
       </li>
    {% empty %}
      <li>Brak modułów.</li>
    {% endfor %}
  </ul>
  <p><a href="{% url "course_module_update" course.id %}">
  Edytuj moduły</a></p>
</div>
<div class="module">
  <h2>Moduł {{ module.order|add:1 }}: {{ module.title }}</h2>
  <h3>Zawartość modułu:</h3>
  <div id="module-contents">
    {% for content in module.contents.all %}
      <div data-id="{{ content.id }}">
        {% with item=content.item %}
          <p>{{ item }}</p>
          <a href="#">Edytuj</a>
          <form action="{% url "module_content_delete" content.id %}"
            method="post">
             <input type="submit" value="Usuń">
            {% csrf_token %}
          </form>
        {% endwith %}
      </div>
    {% empty %}
      <p>Ten moduł nie zawiera jeszcze treści.</p>
    {% endfor %}
  </div>
  <h3> Dodaj nową treść:</h3>
  <ul class="content-types">
    <li>
      <a href="{% url "module_content_create" module.id "text" %}">
        Tekst
      </a>
    </li>
    <li>
      <a href="{% url "module_content_create" module.id "image" %}">
        Obraz
      </a>
    </li>
    <li>
      <a href="{% url "module_content_create" module.id "video" %}">
        Wideo
      </a>
    </li>
```

```
        <li>
          <a href="{% url "module_content_create" module.id "file" %}">
            Plik
          </a>
        </li>
      </ul>
    </div>
  {% endwith %}
  {% endblock %}
```

Zadbaj o to, aby żaden znacznik szablonu nie obejmował więcej niż jednego wiersza; silnik szablonów Django oczekuje, że tagi będą czytelnie zdefiniowane i nie będą przerwane.

Jest to szablon wyświetlający wszystkie moduły kursu oraz treść wybranego modułu. Przeprowadzamy iterację przez moduły kursu w celu wyświetlenia ich w pasku bocznym. Ponadto przeprowadzamy iterację przez treść modułu i uzyskujemy dostęp do `content.item`, aby pobrać powiązane z nim obiekty `Text`, `Video`, `Image` lub `File`. Dołączamy również linki do tworzenia nowych treści tekstowych, wideo, zdjęć lub plików.

Chcemy znać typ obiektu powiązanego z danym obiektem `item`, takim jak `Text`, `Video`, `Image` lub `File`. Do wygenerowania adresu URL pozwalającego na edycję obiektu potrzebna jest nazwa modelu. Ponadto poszczególne obiekty `item` można wyświetlić w szablonie inaczej, na podstawie typu obiektu. Model dla obiektu można pobrać z klasy `Meta` modelu przez uzyskanie dostępu do atrybutu `_meta` obiektu. To i tak bez znaczenia. Django nie pozwoli na uzyskanie w szablonach dostępu do zmiennych lub atrybutów o nazwach rozpoczynających się od znaku podkreślenia, aby uniemożliwić pobranie atrybutów prywatnych lub wywołanie metod prywatnych. Rozwiązaniem może być opracowanie własnego filtru szablonu.

W katalogu aplikacji `courses` utwórz taką strukturę plików i katalogów.

```
templatetags/
    __init__.py
    course.py
```

Przeprowadź edycję pliku *course.py* i umieść w nim poniższy fragment kodu.

```
from django import template

register = template.Library()

@register.filter
def model_name(obj):
    try:
        return obj._meta.model_name
    except AttributeError:
        return None
```

Jest to filtr szablonu o nazwie `model_name`. W szablonach możemy go zastosować jako `object|model_name`, aby pobrać nazwę modelu dla danego obiektu.

Przeprowadź edycję szablonu *templates/courses/manage/module/content_list.html* i umieść w nim poniższy wiersz kodu, po znaczniku szablonu `{% extends "base.html" %}`.

```
{% load course %}
```

Ten wiersz spowoduje wczytanie znaczników szablonu `course`. Teraz znajdź wiersze:

```
<p>{{ item }}</p>
<a href="#">Edytuj</a>
```

Zastąp je poniższym fragmentem kodu.

```
<p>{{ item }} ({{ item|model_name }})</p>
<a href="{% url "module_content_update" module.id item|model_name item.id %}">
  Edytuj
</a>
```

W powyższym kodzie wyświetlamy model elementu w szablonie, a nazwę modelu wykorzystujemy do utworzenia linka pozwalającego na edycję obiektu.

W szablonie *courses/manage/course/list.html* umieść link do wzorca adresu URL `module_content_list`.

```
<a href="{% url "course_module_update" course.id %}">Edytuj moduły</a>
{% if course.modules.count > 0 %}
  <a href="{% url "module_content_list" course.modules.first.id %}">
    Zarządzaj treścią
  </a>
{% endif %}
```

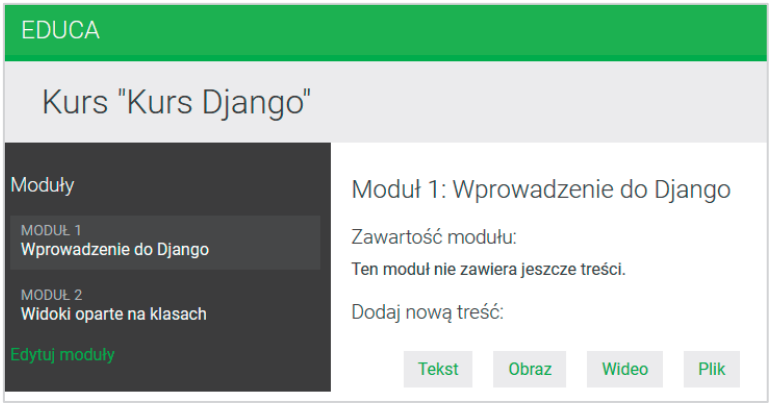
Nowy link pozwala użytkownikom na uzyskanie dostępu do treści pierwszego modułu kursu, o ile taki istnieje.

W tym celu należy zatrzymać działanie serwera programistycznego za pomocą klawiszy *Ctrl+C* i uruchomić go ponownie przez wydanie polecenia:

```
python manage.py runserver
```

Dzięki zatrzymaniu serwera programistycznego i ponownemu jego uruchomieniu zyskujemy pewność, że zostanie załadowany plik ze znacznikami szablonu `course`.

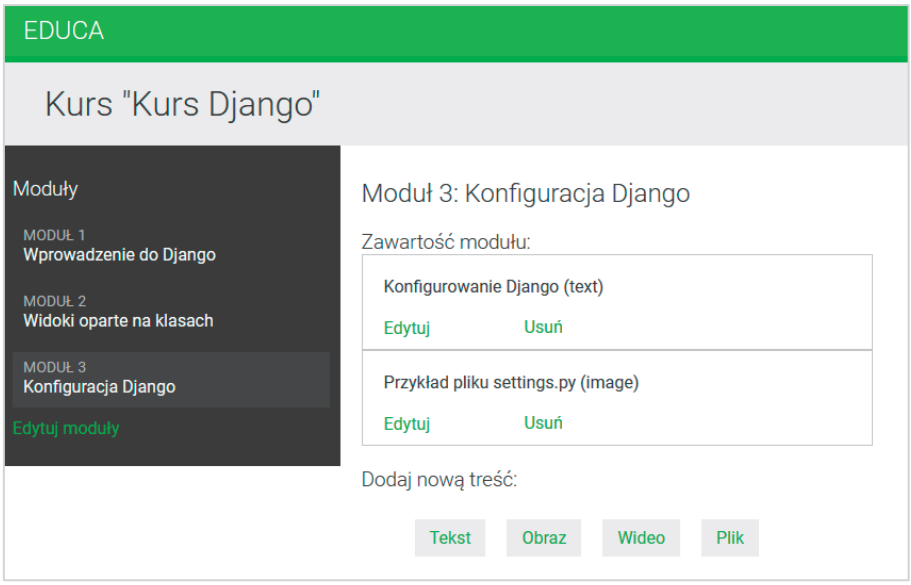
W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/course/mine/` i kliknij link *Zarządzaj treścią* dla kursu zawierającego przynajmniej jeden moduł. Powinieneś zobaczyć stronę podobną do pokazanej na rysunku 13.10.



Rysunek 13.10. Strona do zarządzania treścią modułu kursu

Kiedy klikniesz moduł w pasku bocznym po lewej, wybrana treść zostanie wyświetlona na stronie. Szablon generuje również linki pozwalające na dodanie nowego tekstu, klipu wideo, obrazu lub pliku dla właśnie wyświetlanego modułu.

Dodaj do modułu kilka różnego rodzaju treści, a następnie spójrz na otrzymany wynik. Treść będzie wyświetlona poniżej nagłówka *Treść modułu*, tak jak pokazałem na rysunku 13.11.



Rysunek 13.11. Zarządzanie różnymi modułami treści

W następnym podrozdziale umożliwimy instruktorom kursów zmianę kolejności modułów i ich zawartości za pomocą prostej funkcji „przeciągnij i upuść".

Zmiana kolejności modułów i treści

W celu umożliwienia instruktorom kursu zmiany kolejności modułów kursu przez ich przeciąganie zaimplementujemy funkcjonalność JavaScript typu „przeciągnij i upuść". Operacje typu „przeciągnij i upuść" uatrakcyjniają interfejs użytkownika. Oferują naturalny sposób zmiany kolejności elementów, który jest bardziej intuicyjny niż używanie liczb lub klikanie przycisków. Jest to również oszczędność czasu dla instruktorów kursów, którzy będą mogli łatwo reorganizować moduły kursów wraz z ich zawartością.

Do zaimplementowania tej funkcjonalności wykorzystamy bibliotekę HTML5 Sortable, która upraszcza proces tworzenia sortowalnych list przy użyciu natywnego API HTML5 Drag and Drop.

Kiedy użytkownik zakończy przeciąganie modułu, skorzystamy z interfejsu Fetch API języka JavaScript do wysłania do serwera asynchronicznego żądania HTTP zawierającego nową kolejność modułów.

Więcej informacji na temat interfejsu API HTML5 Drag and Drop można znaleźć na stronie *https://www.w3schools.com/html/html5_draganddrop.asp*. Przykłady zbudowane przy użyciu biblioteki HTML5 Sortable można znaleźć na stronie*https://lukasoppermann.github.io/html5sortable/*. Dokumentacja biblioteki HTML5 Sortable jest dostępna na stronie *https://github.com/lukasoppermann/html5sortable*.

W następnym punkcie zaimplementujemy widoki pozwalające zaktualizować kolejność modułów kursu i ich zawartość.

Używanie domieszek z pakietu django-braces

Pakiet `django-braces` jest dostarczanym przez zewnętrzną firmę modułem, który zawiera kolekcję generycznych domieszek dla Django. Domieszki zapewniają dodatkową funkcjonalność dla widoków opartych na klasach. Mogą one okazać się przydatne w różnych popularnych scenariuszach. Listę wszystkich domieszek dostarczonych przez moduł `django-braces` można znaleźć pod adresem *https://django-braces.readthedocs.io/.readthedocs.io/*.

Wykorzystamy następujące domieszki modułu `django-braces`.

- `CsrfExemptMixin`. Tę domieszkę zastosowaliśmy, aby uniknąć sprawdzenia pod kątem tokenu CSRF w żądaniach `POST`. Jest to niezbędne w celu wykonywania żądań AJAX `POST` bez konieczności generowania `csrf_token`.
- `JsonRequestResponseMixin`. Domieszka przetwarza dane żądania jako JSON, serializuje odpowiedź na postać danych JSON oraz zwraca odpowiedź HTTP jako typ `application/json`.

Zainstaluj moduł `django-braces` za pomocą menedżera `pip`, używając poniższego polecenia.

```
python -m pip install django-braces==1.15.0
```

Potrzebny jest widok pobierający nową kolejność modułu podaną w formacie danych JSON i odpowiednio ją aktualizujący. Przeprowadź edycję pliku *views.py* aplikacji `courses` i umieść w nim poniższy fragment kodu.

```
from braces.views import CsrfExemptMixin, JsonRequestResponseMixin

class ModuleOrderView(CsrfExemptMixin, JsonRequestResponseMixin, View):
    def post(self, request):
        for id, order in self.request_json.items():
            Module.objects.filter(
                id=id, course__owner=request.user
            ).update(order=order)
        return self.render_json_response({'saved': 'OK'})
```

Jest to widok `ModuleOrderView`, który umożliwia aktualizację kolejności modułów kursu.

Podobny widok możemy utworzyć do zmiany kolejności treści modułu. Umieść poniższy fragment kodu w pliku *views.py*.

```
class ContentOrderView(CsrfExemptMixin, JsonRequestResponseMixin, View):
    def post(self, request):
        for id, order in self.request_json.items():
```

```
            Content.objects.filter(
                id=id, module__course__owner=request.user
            ).update(order=order)
        return self.render_json_response({'saved': 'OK'})
```

Teraz przeprowadź edycję pliku *urls.py* aplikacji `courses` i umieść w nim poniższe wzorce adresów URL.

```
path(
    'module/order/',
    views.ModuleOrderView.as_view(),
    name='module_order'
),
path(
    'content/order/',
    views.ContentOrderView.as_view(),
    name='content_order'
),
```

Na końcu implementujemy w szablonie funkcjonalność typu przeciągnij i upuść. Do tego celu wykorzystamy bibliotekę HTML5 Sortable, która upraszcza tworzenie sortowalnych elementów przy użyciu standardowego API HTML Drag and Drop. Dostępne są także inne biblioteki JavaScript pozwalające osiągnąć to samo. Wybrałem bibliotekę HTML5 Sortable, ponieważ jest prosta oraz wykorzystuje natywny interfejs API HTML5 Drag and Drop.

Przeprowadź edycję szablonu *base.html* znajdującego się w katalogu *templates/* aplikacji `courses` i dodaj następujący blok zaznaczony pogrubioną czcionką:

```
{% load static %}
<!DOCTYPE html>
<html>
  <head>
     # ...
  </head>
  <body>
    <div id="header">
       # ...
    </div>
    <div id="content">
      {% block content %}
      {% endblock %}
    </div>
    {% block include_js %}
    {% endblock %}
     <script>
       document.addEventListener('DOMContentLoaded', (event) => {
        // Model DOM załadowany
        {% block domready %}
        {% endblock %}
      })
    </script>
  </body>
</html>
```

Ten nowy blok o nazwie `include_js` pozwala wstawiać pliki JavaScript do dowolnego szablonu, który rozszerza szablon *base.html*.

Teraz przeprowadź edycję szablonu *courses/manage/module/content_list.html* i na końcu pliku umieść poniższy fragment kodu oznaczony pogrubioną czcionką.

```
# ...
{% block content %}
   # …
{% endblock %}

{% block include_js %}
  <script src="https://cdnjs.cloudflare.com/ajax/libs/html5sortable/0.13.3/
  ↪html5sortable.min.js"></script>
{% endblock %}
```

W tym kodzie ładujemy z publicznej sieci **CDN** bibliotekę HTML5 Sortable. Jak pamiętasz, wcześniej ładowałeś bibliotekę JavaScript z sieci CDN w rozdziale 6., „Udostępnianie treści w witrynie internetowej”.

Teraz dodaj do szablonu *courses/manage/module/content_list.html* następujący blok `domready` zaznaczony pogrubioną czcionką:

```
# …
{% block content %}
   # …
{% endblock %}
{% block include_js %}
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/html5sortable/0.13.3/html5sortable.min.
↪js"></script>
{% endblock %}

{% block domready %}
  var options = {
      method: 'POST',
      mode: 'same-origin'
  }

  const moduleOrderUrl = '{% url "module_order" %}';
{% endblock %}
```

W tych nowych wierszach kodu dodajemy kod JavaScript do bloku `{% block domready %}`, który został zdefiniowany w procedurze nasłuchu zdarzenia `DOMContentLoaded` w szablonie *base.html*. Mamy zatem gwarancję wykonania kodu JavaScript po wczytaniu strony. Za pomocą tego kodu zdefiniowaliśmy opcje żądania HTTP w celu zmiany kolejności modułów. Zmianę tę zaimplementujemy w następnej kolejności. Aby zaktualizować kolejność modułów, wyślemy za pomocą Fetch API żądanie `POST`. Ścieżka adresu URL `module_order` została stworzona i zapisana w stałej JavaScript `moduleOrderUrl`.

Dodaj do bloku `domready` następujący kod zaznaczony pogrubioną czcionką:

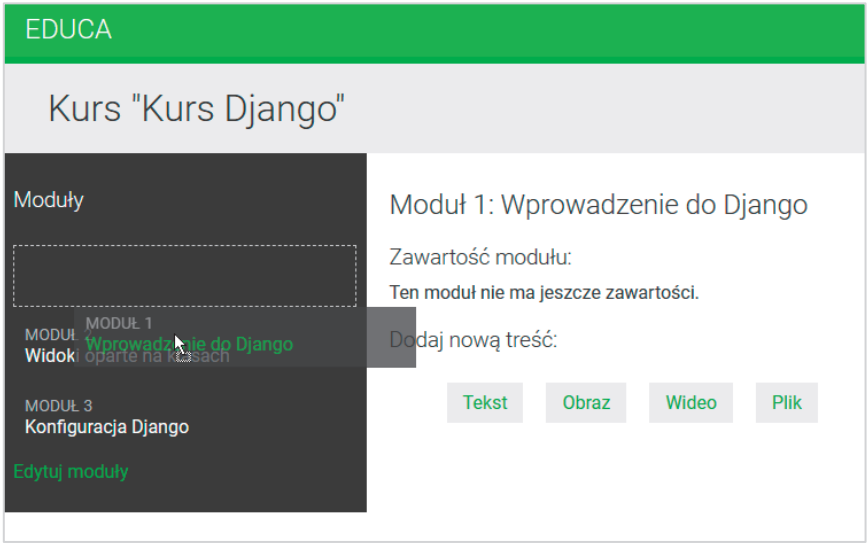
```
{% block domready %}
  var options = {
      method: 'POST',
```

```
            mode: 'same-origin'
        }

        const moduleOrderUrl = '{% url "module_order" %}';

        sortable('#modules', {
          forcePlaceholderSize: true,
          placeholderClass: 'placeholder'
        });
    {% endblock %}
```

W nowym kodzie definiujemy element `sortable` dla elementu HTML z atrybutem `id="modules"`, reprezentującym listę modułów wyświetlaną na pasku bocznym. Pamiętaj, że do wybrania elementu o podanym id używamy selektora CSS `#`. Kiedy zaczynasz przeciągać element, biblioteka HTML5 Sortable tworzy element zastępczy, dzięki czemu możesz łatwo zobaczyć, gdzie zostanie umieszczony element.

Aby wymusić określoną wysokość elementu zastępczego, ustawiliśmy opcję `forcePlace` ↪`hoderSize` na `true`. Ponadto do zdefiniowania klasy CSS dla elementu zastępczego zastosowaliśmy zmienną `placeholderClass`. Użyliśmy klasy o nazwie `placeholder`, która jest zdefiniowana w statycznym pliku *css/base.css* załadowanym w szablonie *base.html*.

Otwórz w przeglądarce stronę `http://127.0.0.1:8000/course/mine/` i kliknij link *Zarządzaj treścią* dla dowolnego kursu. Możesz teraz przeciągać i upuszczać moduły kursu na lewym pasku bocznym, jak pokazałem na rysunku 13.12.



Rysunek 13.12. Zmiana kolejności modułów z wykorzystaniem funkcjonalności przeciągnij i upuść

Podczas przeciągania elementu zobaczysz oznaczony linią przerywaną element zastępczy utworzony przez bibliotekę Sortable. Element zastępczy umożliwia określenie pozycji, w której ma być upuszczony przeciągany element.

Gdy przeciągasz moduł w inne miejsce, musisz wysłać do serwera żądanie HTTP, aby zapisać nową kolejność. Aby to zrobić, można dołączyć do elementu `sortable` procedurę obsługi zdarzenia i wysłać żądanie do serwera za pomocą JavaScript Fetch API.

Przeprowadź edycję szablonu *courses/manage/module/content_list.html* i na końcu pliku umieść poniższy fragment kodu oznaczony pogrubioną czcionką.

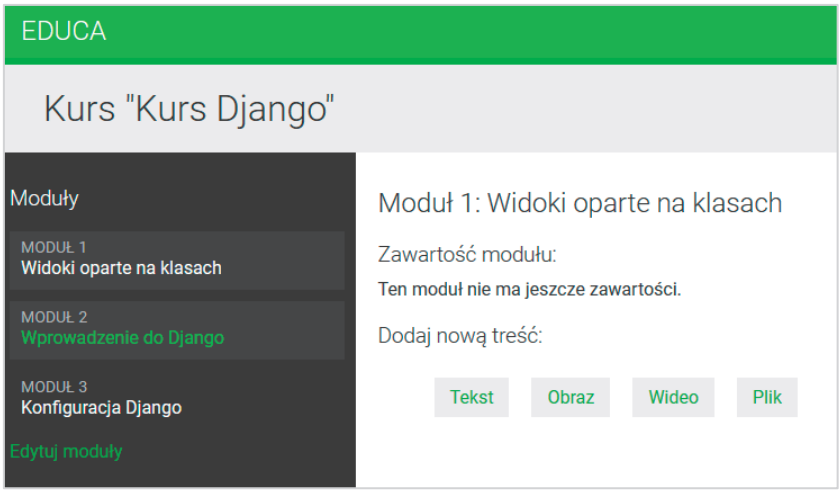
```
{% block domready %}
   var options = {
       method. 'POST',
       mode: 'same-origin'
   }
   const moduleOrderUrl = '{% url "module_order" %}';

   sortable('#modules', {
     forcePlaceholderSize: true,
     placeholderClass: 'placeholder'
  })[0].addEventListener('sortupdate', function(e) {

    modulesOrder = {};
     var modules = document.querySelectorAll('#modules li');
     modules.forEach(function (module, index) {
       // zaktualizuj indeks modułu
       modulesOrder[module.dataset.id] = index;
       // zaktualizuj indeks w elemencie HTML
       module.querySelector('.order').innerHTML = index + 1;
     });
     // dodaj nową kolejność do opcji żądania HTTP
     options['body'] = JSON.stringify(modulesOrder);
     // wysłanie żądania HTTP
     fetch(moduleOrderUrl, options)
   });
{% endblock %}
```

W nowym kodzie dla zdarzenia `sortupdate` elementu `sortable` tworzona jest procedura nasłuchu zdarzeń. Zdarzenie `sortupdate` jest wyzwalane w chwili upuszczenia elementu w inne miejsce. W funkcji obsługi zdarzenia wykonywane są następujące zadania.

1. Tworzymy pusty słownik `modulesOrder`. Kluczami dla tego słownika będą identyfikatory modułów, natomiast wartościami — kolejność przypisana poszczególnym modułom.
2. Za pomocą instrukcji `document.querySelectorAll()` przy użyciu selektora CSS `#modules li`. wybierane są elementy HTML `list` elementu `#modules`.
3. Wykorzystujemy funkcję `forEach()` do iterowania po elementach listy.
4. W słowniku `modulesOrder` zapisujemy nowy indeks dla każdego modułu. Za pomocą instrukcji `module.dataset.id` pobieramy z atrybutu HTML `data-id` identyfikator każdego modułu. Używamy tego identyfikatora jako klucza słownika `modulesOrder`, a jego wartością jest nowy indeks danego modułu.
5. Aktualizujemy wyświetlaną kolejność dla każdego modułu poprzez wybranie elementu z wykorzystaniem klasy CSS `order`. Ponieważ indeks jest liczony od zera, a chcemy wyświetlić indeks od jedynki, dodajemy do indeksu 1.

6. Do słownika `options` dodajemy klucz o nazwie `body` oraz nową kolejność zapisaną w słowniku `modulesOrder`. Za pomocą metody `JSON.stringify()` dokonujemy konwersji obiektu JavaScript na ciąg znaków JSON. Jest to treść żądania HTTP służąca do zaktualizowania kolejności modułów.
7. W celu zaktualizowania kolejności modułów korzystamy z Fetch API poprzez utworzenia żądania HTTP `fetch()`. Odpowiednim uaktualnieniem kolejności modułów zajmuje się widok `ModuleOrderView` odpowiadający adresowi URL danego modułu.

Możesz teraz przeciągać i upuszczać moduły. Po zakończeniu przeciągania modułu, w celu zaktualizowania kolejności modułów, do adresu URL `module_order` wysyłane jest żądanie HTTP. Jeśli odświeżysz stronę, najnowsza kolejność modułów zostanie zachowana, ponieważ została ona zaktualizowana w bazie danych. Zmienioną kolejność modułów na pasku bocznym po posortowaniu ich metodą „przeciągnij i upuść" pokazałem na rysunku 13.13.



Rysunek 13.13. Nowa kolejność modułów po zmianie ich kolejności metodą „przeciągnij i upuść"

Jeśli napotkasz jakiekolwiek problemy, pamiętaj, aby do debugowania żądań JavaScript i HTTP użyć narzędzi programistycznych przeglądarki. Zwykle, aby uzyskać dostęp do narzędzi programistycznych, wystarczy kliknąć prawym przyciskiem myszy gdziekolwiek na stronie, a następnie z menu kontekstowego wybrać opcję *Zbadaj* lub *Zbadaj element*.

Dodajmy teraz tę samą funkcjonalność „przeciągnij i upuść", aby umożliwić instruktorom kursów również sortowanie zawartości modułów.

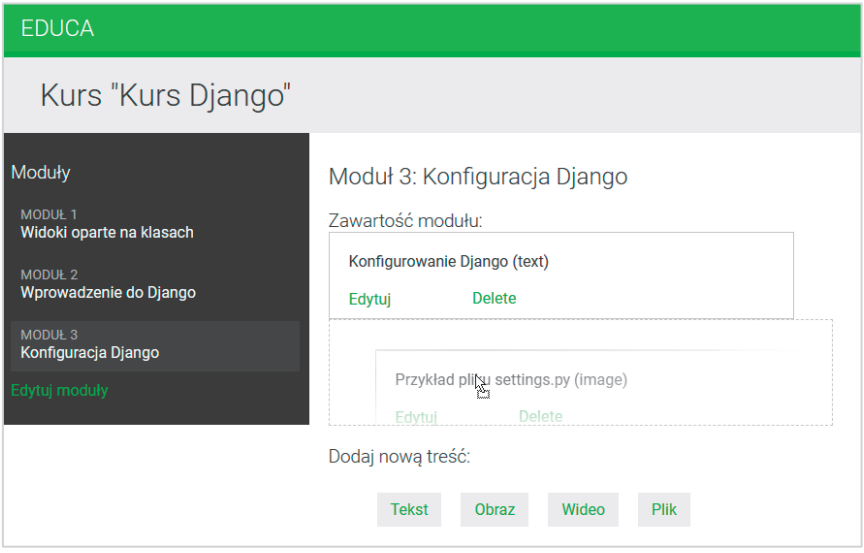
Przeprowadź edycję szablonu *courses/manage/module/content_list.html* i na końcu pliku umieść poniższy fragment kodu oznaczony pogrubioną czcionką.

```
{% block domready %}

  // ...
  const contentOrderUrl = '{% url "content_order" %}';
```

```
    sortable('#module-contents', {
      forcePlaceholderSize: true,
      placeholderClass: 'placeholder'
    })[0].addEventListener('sortupdate', function(e) {

      contentOrder = {};
      var contents = document.querySelectorAll('#module-contents div');
      contents.forEach(function (content, index) {
        // Aktualizacja indeksu treści
        contentOrder[content.dataset.id] = index;
    });

      // Dodaj nową kolejność do opcji żądania HTTP
      options['body'] = JSON.stringify(contentOrder);

      // Wysłanie żądania HTTP
      fetch(contentOrderUrl, options)
    });
  {% endblock %}
```

W tym przypadku zamiast `module_order` użyliśmy adresu URL `content_order` i zbudowaliśmy funkcjonalność `sortable` dla elementu HTML z identyfikatorem `module-contents`. Ogólnie rzecz biorąc funkcjonalność jest taka sama jak w przypadku kolejności modułów kursu. W tym przypadku nie ma potrzeby aktualizacji numeracji treści, ponieważ nie zawierają one widocznego indeksu.

Możesz teraz przeciągać i upuszczać zarówno moduły, jak i ich zawartość, tak jak pokazałem na rysunku 13.14.



Rysunek 13.14. Zmiana kolejności treści modułów z wykorzystaniem funkcjonalności przeciągnij i upuść

Doskonale! Zbudowałeś uniwersalny serwis CMS dla instruktorów kursów.

Podsumowanie

W tym rozdziale na przykładzie tworzenia systemu zarządzania treścią nauczyłeś się korzystania z widoków opartych na klasach i domieszek. Zdobyłeś wiedzę na temat kodu wielokrotnego użytku i modułowości, którą możesz zastosować do tworzenia przyszłych aplikacji. Aby ograniczyć dostęp do swoich widoków, korzystałeś także z grup i uprawnień. Dzięki temu poznałeś zasady zabezpieczeń i sposób kontrolowania działań na danych. Dowiedziałeś się jak używać zestawów formularzy i modeli formularzy do zarządzania modułami kursu i ich zawartością. Za pomocą JavaScript stworzyłeś również funkcjonalność „przeciągnij i upuść", pozwalająca na zmianę kolejności modułów kursu i ich zawartości.

W następnym rozdziale opracujesz system rejestracji uczestników kursu. Dowiesz się także, jak renderować różne rodzaje treści oraz w jaki sposób poprawić wydajność swojej aplikacji przez buforowanie zawartości za pomocą frameworka buforowania Django.

Dodatkowe zasoby

Oto lista zasobów zawierających dodatkowe informacje związane z tematami omawianymi w tym rozdziale:

- Kod źródłowy przykładów zamieszczonych w tym rozdziale — *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter13*.
- Dokumentacja domieszek frameworka Django — *https://docs.djangoproject.com/en/5.0/topics/class-based-views/mixins/*.
- Pakiet `Neapolitan` do tworzenia widoków CRUD — *https://github.com/carltongibson/neapolitan*.
- Tworzenie niestandardowych uprawnień — *https://docs.djangoproject.com/en/5.0/topics/auth/customizing/#custom-permissions*.
- Zestawy formularzy we frameworku Django — *https://docs.djangoproject.com/en/5.0/topics/forms/formsets/*.
- Zestawy formularzy dla modeli we frameworku Django — *https://docs.djangoproject.com/en/5.0/topics/forms/modelforms/#model-formsets*.
- Interfejs API „przeciągnij i upuść" HTML5 — *https://www.w3schools.com/html/html5_draganddrop.asp*.
- Dokumentacja biblioteki HTML5 Sortable — *https://github.com/lukasoppermann/html5sortable*.
- Przykłady korzystania z biblioteki HTML5 Sortable — *https://lukasoppermann.github.io/html5sortable/*.
- Dokumentacja modułu `django-braces` — *https://django-braces.readthedocs.io/*.

Rozdział 14

Renderowanie i buforowanie treści

W poprzednim rozdziale wykorzystaliśmy model dziedziczenia i relacje generyczne do utworzenia elastycznych modeli dla treści kursu. Zaimplementowałeś własne pole modelu i zbudowałeś system zarządzania kursami, używając widoków opartych na klasach. Na koniec, używając asynchronicznych żądań HTTP stworzyłeś w języku JavaScript funkcjonalność „przeciągnij i upuść" mającą na celu uporządkowania modułów kursu i ich zawartości.

W tym rozdziale zbudujesz funkcjonalność umożliwiającą uzyskanie dostępu do treści kursów. Utworzysz system rejestracji uczestników kursów i zarządzania zapisami uczestników na kursy. Zaimplementujesz renderowanie różnych typów treści wykorzystywanych w kursach i dowiesz się, jak buforować dane za pomocą frameworka `cache` Django.

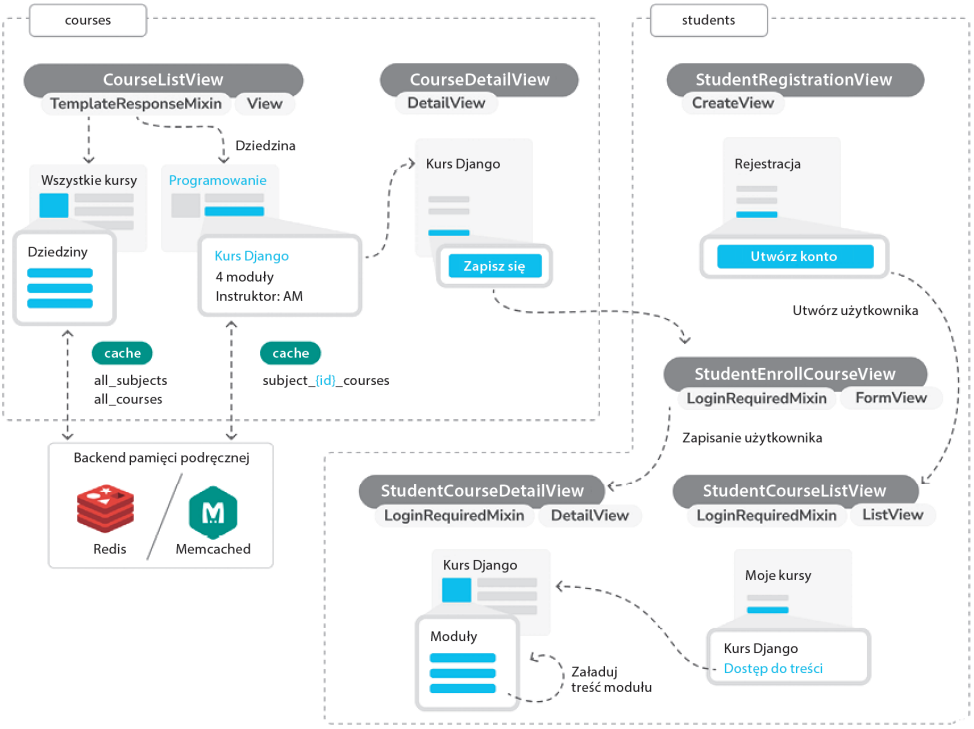
Renderowanie różnorodnych typów treści na platformach e-learningowych, na których kursy zazwyczaj są zbudowane z elastycznych modułów, zawierających połączenie tekstu, obrazów, filmów i dokumentów, jest w istocie niezbędne. W tym kontekście kluczowe staje się również buforowanie. Ponieważ treść kursu zwykle pozostaje niezmieniona przez dłuższy czas — przez wiele dni, tygodni, a nawet miesięcy — buforowanie pomaga oszczędzać moc obliczeniową i zmniejsza potrzebę przeszukiwania bazy danych za każdym razem, gdy uczniowie uzyskują dostęp do tych samych materiałów. Buforowanie danych nie tylko oszczędza zasoby systemowe, ale także poprawia wydajność podczas dostarczania treści do dużej liczby uczestników kursu.

W tym rozdziale skoncentruję się na następujących zagadnieniach:

- Utworzenie widoków publicznych przeznaczonych do wyświetlania informacji o kursie
- Utworzenie systemu rejestracji uczestników kursu
- Zarządzanie zapisywaniem się uczestników na kursy
- Generowanie różnorodnej treści modułów kursu
- Instalacja i konfiguracja frameworka `Memcached`
- Buforowanie treści za pomocą frameworka buforowania
- Korzystanie z mechanizmów buforowania `Memcached` i Redis
- Monitorowanie serwera Redis w witrynie administracyjnej Django

Przegląd funkcjonalności

Reprezentację widoków, szablonów i funkcjonalności, które zbudujemy w tym rozdziale, przedstawiłem na rysunku 14.1.



Rysunek 14.1. Schemat funkcjonalności zbudowanych w rozdziale 14.

W tym rozdziale zaimplementujesz publiczny widok `CourseListView` przeznaczony do wyświetlania listy kursów oraz `CourseDetailView` do wyświetlania szczegółów kursu. Zaimplementujesz również widok `StudentRegistrationView`, pozwalający uczestnikom kursu na tworzenie kont użytkowników, oraz `StudentCourseListView`, dzięki któremu uczestnicy mogą zapisywać się na kursy. Utworzysz widok `StudentCourseListView` przeznaczony dla uczestników, aby mogli zobaczyć listę kursów, na które są zapisani, oraz widok `StudentCourseDetailView`, pozwalający im uzyskać dostęp do całej zawartości kursu, zorganizowanej w różnych modułach. Dodasz także bufor do swoich widoków z użyciem frameworka buforowania Django, najpierw z backendem `Memcached`, a następnie z backendem buforowania Redis.

Kod źródłowy przykładów z tego rozdziału można znaleźć pod adresem *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter14*.

Wszystkie pakiety Pythona użyte w tym rozdziale są zawarte w pliku *requirements.txt* dołączonym do plików z kodem źródłowym przykładów z tego rozdziału. Aby zainstalować każdy pakiet Pythona, możesz postępować zgodnie z instrukcjami zamieszczonymi w poniższych podrozdziałach. Możesz także zainstalować wszystkie wymagania jednocześnie za pomocą polecenia `pip install -r requirements.txt`.

Wyświetlanie katalogu kursów

Być może chciałbyś jak najszybciej przejść do zagadnień renderowania i buforowania. Jest jednak kilka elementów, które trzeba skonfigurować, zanim będzie można to zrobić. Zacznijmy od katalogu kursów. Dla katalogu kursów konieczne jest przygotowanie następujących funkcjonalności.

- Wyświetlenie listy dostępnych kursów, opcjonalnie filtrowanych według tematu.
- Wyświetlenie ogólnego opisu kursu.

Pozwoli to uczestnikom kursów wyświetlić wszystkie kursy dostępne na platformie i zapisać się na te, które ich interesują. Przeprowadź edycję pliku *views.py* aplikacji `courses` i umieść w nim poniższy fragment kodu.

```
from django.db.models import Count
from .models import Subject


class CourseListView(TemplateResponseMixin, View):
    model = Course
    template_name = 'courses/course/list.html'
    def get(self, request, subject=None):
        subjects = Subject.objects.annotate(
            total_courses=Count('courses')
        )
        courses = Course.objects.annotate(
            total_modules=Count('modules')
        )
        if subject:
            subject = get_object_or_404(Subject, slug=subject)
            courses = courses.filter(subject=subject)
        return self.render_to_response(
            {
                'subjects': subjects,
                'subject': subject,
                'courses': courses
            }
        )
```

Jest to widok `CourseListView`, który dziedziczy po `TemplateResponseMixin` i `View`. W widoku są wykonywane następujące zadania.

1. Pobieramy wszystkie tematy oraz całkowitą liczbę kursów dostępnych dla poszczególnych tematów. W tym celu wykorzystujemy oferowaną przez warstwę ORM w Django metodę `annotate()` wraz z funkcją agregującą `Count()`.
2. Pobieramy wszystkie dostępne kursy oraz całkowitą liczbę modułów znajdujących się w poszczególnych kursach.
3. Jeżeli podany jest parametr w postaci slugu adresu URL dla danego tematu, pobieramy odpowiadający mu obiekt `subject` i ograniczamy zapytanie do kursów zaliczających się do danego tematu.
4. Wykorzystujemy metodę `render_to_response()` dostarczaną przez domieszkę `TemplateResponseMixin` w celu wygenerowania obiektów w szablonie i udzielenia odpowiedzi HTTP.

Przystępujemy do utworzenia widoku przeznaczonego do wyświetlenia ogólnego opisu kursu. Dodaj poniższy fragment kodu do pliku *views.py*.

```
from django.views.generic.detail import DetailView

class CourseDetailView(DetailView):
    model = Course
    template_name = 'courses/course/detail.html'
```

Ten widok dziedziczy po generycznym widoku `DetailView` dostarczanym przez Django. Określamy atrybuty `model` i `template_name`. W celu pobrania pojedynczego obiektu dla danego modelu widok Django `DetailView` oczekuje podania klucza podstawowego lub slugu adresu URL. Następnie renderuje szablon podany w `template_name`, z uwzględnieniem obiektu `Course` w zmiennej reprezentującej obiekt kontekstu szablonu.

Przeprowadź edycję głównego pliku *urls.py* projektu `educa` i umieść w nim poniższy wzorzec adresu URL.

```
from courses.views import CourseListView

urlpatterns = [
    # …
    path('', CourseListView.as_view(), name='course_list'),
]
```

Dodajemy wzorzec adresów URL `course_list` do głównego pliku *urls.py* projektu, ponieważ chcemy wyświetlać listę wszystkich kursów na stronie pod adresem URL `http://127.0.0.1:8000/`, natomiast wszystkie pozostałe adresy URL aplikacji courses będą miały prefiks */course/*.

Przeprowadź edycję pliku *urls.py* aplikacji `courses` i umieść w nim poniższe wzorce adresów URL.

```
path(
    'subject/<slug:subject>/',
    views.CourseListView.as_view(),
    name='course_list_subject' ),
path(
    '<slug:slug>/',
    views.CourseDetailView.as_view(),
    name='course_detail'
),
```

Zdefiniowaliśmy takie wzorce adresów URL:

- `course_list_subject`. Wzorzec przeznaczony do wyświetlania wszystkich kursów na dany temat.
- `course_detail`. Wzorzec przeznaczony do wyświetlania ogólnego opisu pojedynczego kursu.

Przystępujemy do przygotowania szablonów dla widoków `CourseListView` i `CourseDetailView`.

Utwórz poniższą strukturę plików i katalogów w katalogu *templates/courses/* aplikacji `courses`.

```
course/
    list.html
    detail.html
```

Przeprowadź edycję szablonu *courses/course/list.html* aplikacji `courses` i umieść w nim poniższy fragment kodu.

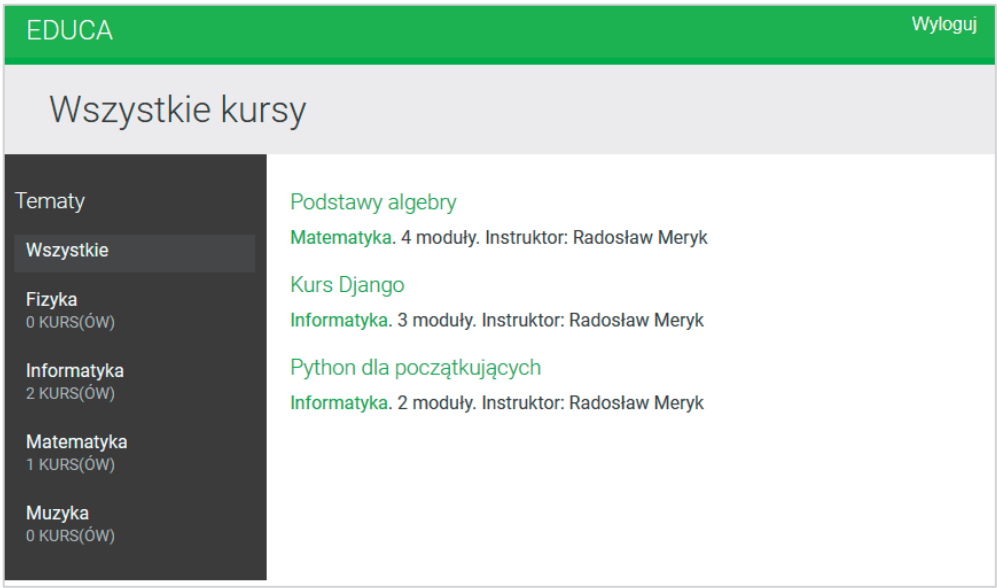
```
{% extends "base.html" %}

{% block title %}
  {% if subject %}
    Kursy z dziedziny: {{ subject.title }}
  {% else %}
    Wszystkie kursy
  {% endif %}
{% endblock %}

{% block content %}
  <h1>
    {% if subject %}
      Kursy z dziedziny: {{ subject.title }}
    {% else %}
      Wszystkie kursy
    {% endif %}
  </h1>
  <div class="contents">
    <h3>Tematy</h3>
    <ul id="modules">
      <li {% if not subject %}class="selected"{% endif %}>
        <a href="{% url "course_list" %}">Wszystkie</a>
      </li>
      {% for s in subjects %}
        <li {% if subject == s %}class="selected"{% endif %}>
          <a href="{% url "course_list_subject" s.slug %}">
            {{ s.title }}
            <br>
            <span>
              {{ s.total_courses }} kurs(ów)
            </span>
          </a>
        </li>
      {% endfor %}
    </ul>
  </div>
  <div class="module">
    {% for course in courses %}
      {% with subject=course.subject %}
        <h3>
          <a href="{% url "course_detail" course.slug %}">
            {{ course.title }}
          </a>
        </h3>
        <p>
          <a href="{% url "course_list_subject" subject.slug %}">{{ subject }}</a>.
            {{ course.total_modules }} moduły.
```

```
            Instruktor: {{ course.owner.get_full_name }}
          </p>
        {% endwith %}
      {% endfor %}
    </div>
  {% endblock %}
```

Zadbaj o to, aby żaden znacznik szablonu nie obejmował więcej niż jednego wiersza.

Jest to szablon przeznaczony do wyświetlenia dostępnych kursów. Tworzymy listę HTML wyświetlającą wszystkie obiekty `Subject` i dla każdego z nich budujemy link do wzorca adresu URL `course_list_subject`. Należy również uwzględnić całkowitą liczbę kursów dla każdego tematu i użyć filtru szablonu `pluralize`, aby dodać sufiks liczby mnogiej do słowa *kurs*, gdy liczba kursów jest różna od 1. Dodajemy klasę HTML `selected` w celu podświetlenia bieżącego tematu, o ile taki został wybrany. Przeprowadzamy iterację przez wszystkie obiekty `Course`, wyświetlamy całkowitą liczbę modułów oraz imię i nazwisko instruktora.

Uruchom serwer programistyczny i przejdź w przeglądarce internetowej pod adres `http://127.0.0.1:8000/`. Powinieneś zobaczyć stronę podobną do pokazanej na rysunku 14.2.



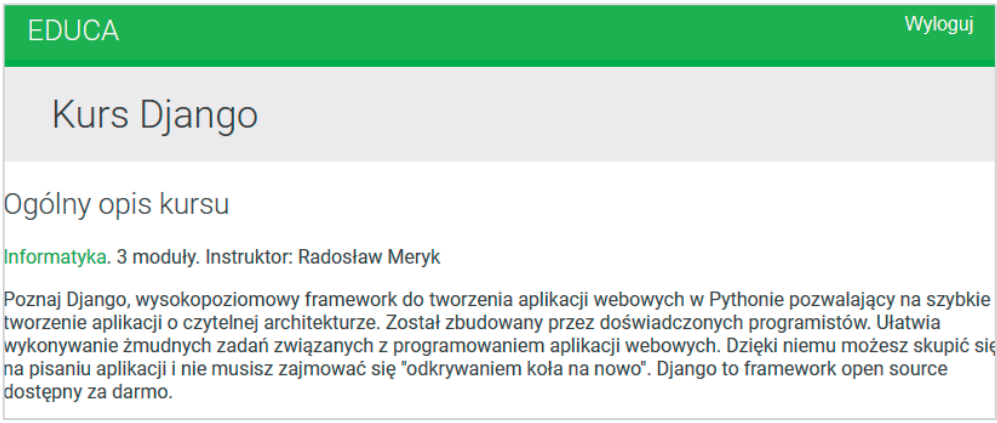
Rysunek 14.2. Strona z listą kursów

Znajdujący się po lewej stronie pasek boczny wyświetla wszystkie tematy, a także całkowitą liczbę kursów dla każdego z nich. Aby filtrować wyświetlane kursy, możesz kliknąć dowolny temat.

Przeprowadź edycję szablonu *courses/course/detail.html* i umieść w nim poniższy fragment kodu.

```
{% extends "base.html" %}

{% block title %}
  {{ object.title }}
{% endblock %}

{% block content %}
  {% with subject=object.subject %}
    <h1>
      {{ object.title }}
    </h1>
    <div class="module">
      <h2>Ogólny opis kursu</h2>
      <p>
        <a href="{% url "course_list_subject" subject.slug %}">
          {{ subject.title }}</a>.
          {{ object.modules.count }} moduły.
          Instruktor: {{ object.owner.get_full_name }}
      </p>
      {{ object.overview|linebreaks }}
    </div>
  {% endwith %}
{% endblock %}
```

W tym szablonie wyświetlamy ogólny opis oraz szczegóły dotyczące pojedynczego kursu. W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/` *i kliknij dowolny kurs.* Powinieneś zobaczyć stronę o strukturze pokazanej na rysunku 14.3.



Rysunek 14.3. Strona z ogólnym opisem kursu

Utworzyliśmy publiczny obszar przeznaczony do wyświetlania kursów. Przechodzimy teraz do umożliwienia użytkownikom rejestracji w charakterze uczestników i zapisywania się na wybrane kursy.

Dodanie rejestracji uczestnika

Trzeba zaimplementować mechanizm rejestracji uczestników kursu, umożliwiający zapisywanie się na kurs i dostęp do jego treści. Utwórz nową aplikację za pomocą poniższego polecenia.

```
python manage.py startapp students
```

Przeprowadź edycję pliku *settings.py* projektu `educa` i umieść nową aplikację na liście `INSTALLED_APPS`, tak jak pokazałem poniżej.

```
INSTALLED_APPS = [
    # …
    'students.apps.StudentsConfig',
]
```

Utworzenie widoku rejestracji uczestnika

Przeprowadź edycję pliku *views.py* aplikacji `students` i umieść w nim poniższy fragment kodu.

```
from django.contrib.auth import authenticate, login
from django.contrib.auth.forms import UserCreationForm
from django.urls import reverse_lazy
from django.views.generic.edit import CreateView

class StudentRegistrationView(CreateView):
    template_name = 'students/student/registration.html'
    form_class = UserCreationForm
    success_url = reverse_lazy('student_course_list')

    def form_valid(self, form):
        result = super().form_valid(form)
        cd = form.cleaned_data
        user = authenticate(
            username=cd['username'], password=cd['password1']
        )
        login(self.request, user)
        return result
```

Jest to widok pozwalający na rejestrację uczestników. Wykorzystaliśmy generyczny widok `CreateView` oferujący funkcjonalność przeznaczoną do tworzenia obiektów modeli. Widok wymaga podania atrybutów:

- `template_name`. Ścieżka dostępu prowadząca do widoku przeznaczonego do wyrenderowania.
- `form_class`. Formularz (typu `ModelForm`) przeznaczony do tworzenia obiektów. Oferowany przez Django `UserCreationForm` wykorzystujemy jako formularz do budowania obiektów `User`.
- `success_url`. Adres URL, na który użytkownik zostanie przekierowany po pomyślnym wysłaniu formularza. Do wyświetlenia kursów, na które jest zapisany dany uczestnik, wykorzystujemy wzorzec adresu URL `student_course_list`. Jego utworzeniem zajmiemy się nieco później, w punkcie „Uzyskanie dostępu do treści kursu”.

Metoda `form_valid()` jest wykonywana po przekazaniu prawidłowych danych formularza. Metoda zwraca odpowiedź HTTP. Metodę tę nadpisujemy w celu zalogowania użytkownika po udanej rejestracji.

Utwórz nowy plik w katalogu aplikacji `students` i nadaj mu nazwę *urls.py*. Następnie w pliku umieść poniższy fragment kodu.

```
from django.urls import path
from . import views


urlpatterns = [
    path(
        'register/',
        views.StudentRegistrationView.as_view(),
        name='student_registration'
    ),
]
```

Przeprowadź edycję głównego pliku *urls.py* projektu `educa` i dodaj adres URL do aplikacji `students` poprzez dodanie poniższego wzorca do istniejącej konfiguracji URL.

```
urlpatterns = [
    # ...
    path('students/', include('students.urls')),
]
```

Utwórz poniższą strukturę plików w aplikacji `students`.

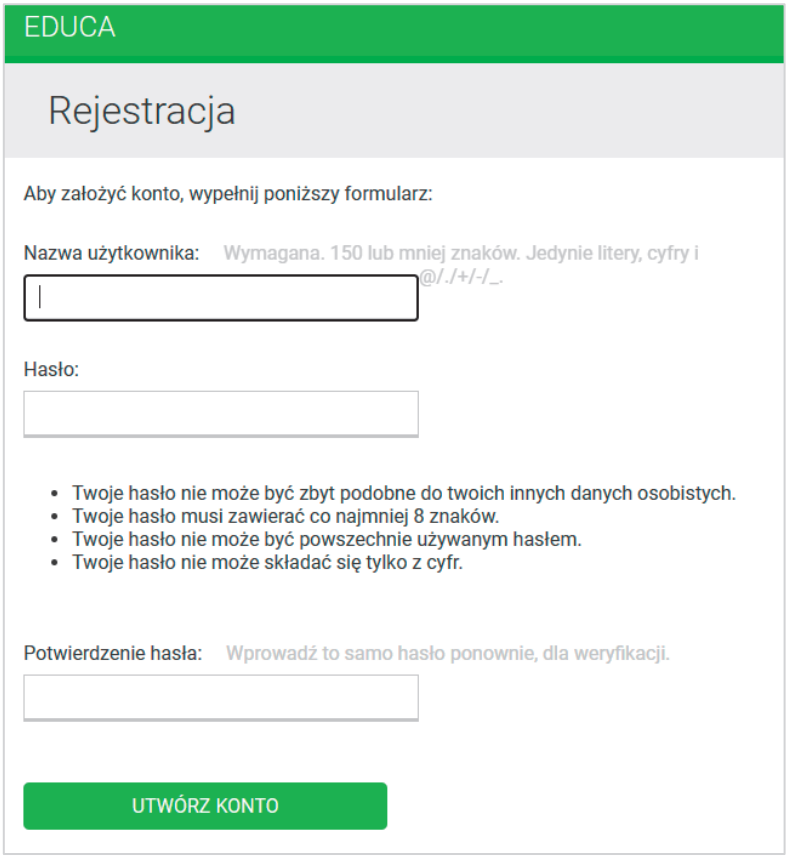
```
templates/
    students/
        student/
            registration.html
```

Przeprowadź edycję szablonu *students/student/registration.html* i umieść w nim poniższy fragment kodu.

```
{% extends "base.html" %}

{% block title %}
  Rejestracja
{% endblock %}

{% block content %}
  <h1>
    Rejestracja
  </h1>
  <div class="module">
    <p>Aby założyć konto, wypełnij poniższy formularz:</p>
    <form method="post">
      {{ form.as_p }}
      {% csrf_token %}
      <p><input type="submit" value="Utwórz konto"></p>
    </form>
  </div>
{% endblock %}
```

Uruchom serwer programistyczny i przejdź w przeglądarce internetowej pod adres `http://127.0.0.1:8000/students/register/`.

Powinieneś zobaczyć formularz pokazany na rysunku 14.4.



Rysunek 14.4. Formularz rejestracji uczestnika kursu

Zauważ, że adres URL `student_course_list` określony za pomocą atrybutu `success_url` w widoku `StudentRegistrationView` jeszcze nie istnieje. Gdy prześlesz formularz, Django nie znajdzie adresu URL do przekierowania po udanej rejestracji. Jak wspominałem wcześniej, ten adres URL utworzymy w punkcie „Uzyskanie dostępu do treści kursu".

Zapisanie się na kurs

Po utworzeniu konta użytkownik będzie mógł zapisywać się na kursy. W celu przechowywania informacji o tym, na jakie kursy jest zapisany dany użytkownik, musimy zdefiniować relację typu „wiele do wielu" pomiędzy modelami `Course` i `User`.

Przeprowadź edycję pliku *models.py* aplikacji `courses` i umieść poniższą kolumnę w modelu `Course`.

```
    students = models.ManyToManyField(
        User,
        related_name='courses_joined',
        blank=True
    )
```

Przejdź do powłoki, a następnie wydaj poniższe polecenie, aby przygotować migracje dla wprowadzonej zmiany.

```
python manage.py makemigrations
```

Powinieneś otrzymać takie dane wyjściowe.

```
Migrations for 'courses':
  courses/migrations/0004_course_students.py
    - Add field students to course
```

Kolejnym krokiem jest wydanie poniższego polecenia, aby faktycznie zastosować oczekujące migracje.

```
python manage.py migrate
```

Powinieneś otrzymać wynik kończący się wierszem:

```
Applying courses.0004_course_students... OK
```

Teraz można już powiązać uczestników z kursami, na które się zapisali. Przechodzimy do przygotowania funkcjonalności pozwalającej uczestnikom zapisywać się na kursy.

Utwórz nowy plik w katalogu aplikacji `students` i nadaj mu nazwę *forms.py*. Następnie w pliku umieść poniższy fragment kodu.

```
from django import forms
from courses.models import Course

class CourseEnrollForm(forms.Form):
    course = forms.ModelChoiceField(
        queryset=Course.objects.none(),
        widget=forms.HiddenInput
    )

    def __init__(self, *args, **kwargs):
        super(CourseEnrollForm, self).__init__(*args, **kwargs)
        self.fields['course'].queryset = Course.objects.all()
```

Ten formularz będzie używany do obsługi zapisywania się uczestników na kursy. Pole `course` wskazuje kurs, na który jest zapisany dany użytkownik. Dlatego też jest to kolumna typu `ModelChoiceField`. Użyliśmy widżetu `HiddenInput`, ponieważ wymienionego pola nie będziemy wyświetlać użytkownikowi. Początkowo definiujemy obiekt `QuerySet` jako `Course.objects.none()`. Użycie metody `none()` tworzy pusty obiekt `QuerySet`, który nie zwraca żadnych obiektów i, co ważne, nie wysyła zapytania do bazy danych. Pozwala to uniknąć niepotrzebnego obciążenia bazy danych podczas inicjalizacji formularza. Obiekt `QuerySet` jest wypełniany w metodzie `__init__()`. To dynamiczne ustawienie pozwala na dostosowanie formularza do różnych sytuacji, takich jak filtrowanie dostępnych kursów na podstawie określonych kryteriów. Takie podejście, ogólnie rzecz biorąc,

zapewnia większą elastyczność w zarządzaniu danymi formularzy, ponieważ dzięki niemu dane są pobierane na podstawie kontekstu, w którym jest używany formularz. Ten sposób jest również zgodny z najlepszymi praktykami wydajnej obsługi formularzy `QuerySet` w Django.

Omawiany formularz wykorzystamy w widoku `CourseDetailView` do wyświetlenia przycisku pozwalającego zapisać się na kurs. Przeprowadź edycję pliku *views.py* aplikacji `students` i umieść w nim poniższy fragment kodu.

```
from django.contrib.auth.mixins import LoginRequiredMixin
from django.views.generic.edit import FormView
from .forms import CourseEnrollForm


class StudentEnrollCourseView(LoginRequiredMixin, FormView):
    course = None
    form_class = CourseEnrollForm

    def form_valid(self, form):
        self.course = form.cleaned_data['course']
        self.course.students.add(self.request.user)
        return super().form_valid(form)

    def get_success_url(self):
        return reverse_lazy(
            'student_course_detail', args=[self.course.id]
        )
```

Jest to widok `StudentEnrollCourseView` odpowiedzialny za obsługę zapisu uczestników na kursy. Widok dziedziczy po domieszce `LoginRequiredMixin`, aby tylko zalogowany użytkownik mógł uzyskać do niego dostęp. Dziedziczy również po oferowanym przez Django widoku `FormView`, ponieważ obsługujemy operację wysłania formularza. Dla atrybutu `form_class` używamy formularza `CourseEnrollForm`, a ponadto definiujemy atrybut `course` przeznaczony do przechowywania danego obiektu `Course`.

Jeżeli formularz jest prawidłowy, dodajemy bieżącego użytkownika do uczestników zapisanych na dany kurs.

Metoda `get_success_url()` zwraca adres URL, do którego użytkownik zostanie przekierowany po udanym wysłaniu formularza. Metoda jest odpowiednikiem atrybutu `success_url`. Do wyświetlenia treści kursu wykorzystujemy wzorzec adresu URL `student_course_detail`, którego utworzeniem zajmiemy się nieco później.

Przeprowadź edycję pliku *urls.py* aplikacji `students` i umieść w nim poniższy wzorzec adresu URL.

```
path(
    'enroll-course/',
    views.StudentEnrollCourseView.as_view(),
    name='student_enroll_course'
),
```

Na stronie zawierającej ogólny opis kursu umieścimy teraz przycisk pozwalający użytkownikowi zapisać się na ten kurs. Przeprowadź edycję pliku *views.py* aplikacji `courses` i zmodyfikuj widok `CourseDetailView` w następujący sposób.

```
from students.forms import CourseEnrollForm


class CourseDetailView(DetailView):
    model = Course
    template_name = 'courses/course/detail.html'

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context['enroll_form'] = CourseEnrollForm(
            initial={'course':self.object}
        )
        return context
```

W celu umieszczenia formularza zapisu na kurs w kontekście po to, by wyrenderować szablon, używamy metody `get_context_data()`. W formularzu inicjalizujemy ukryte pole przez umieszczenie w nim bieżącego obiektu `Course`, aby mógł zostać wysłany bezpośrednio.

Przeprowadź edycję szablonu *courses/course/detail.html* i znajdź poniższy wiersz.

```
{{ object.overview|linebreaks }}
```

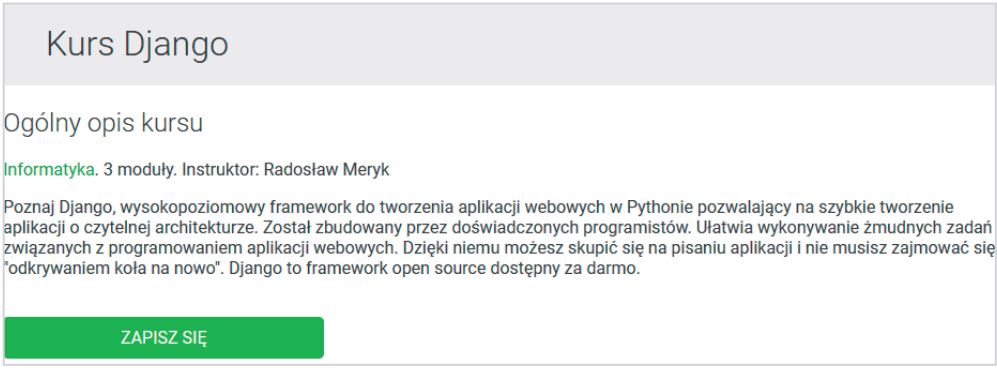
Zastąp go fragmentem kodu:

```
{{ object.overview|linebreaks }}
{% if request.user.is_authenticated %}
  <form action="{% url "student_enroll_course" %}" method="post">
    {{ enroll_form }}
    {% csrf_token %}
    <input type="submit" value="Zapisz się">
  </form>
{% else %}
  <a href="{% url "student_registration" %}" class="button">
    Zarejestruj się, aby zostać uczestnikiem kursu
  </a>
{% endif %}
```

Jest to przycisk umożliwiający zapisanie się na kurs. Jeżeli użytkownik jest uwierzytelniony, wyświetlamy ten przycisk zawierający ukryty formularz prowadzący do wzorca adresu URL `student_enroll_course`. Jeżeli użytkownik nie jest uwierzytelniony, wyświetlamy link pozwalający zarejestrować się na platformie.

Upewnij się, że serwer programistyczny działa. Następnie w przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/` i kliknij dowolny kurs. Jeżeli jesteś zalogowany, powinieneś zobaczyć przycisk *ZAPISZ SIĘ* wyświetlony pod ogólnym opisem kursu, co pokazałem na rysunku 14.5.

Jeżeli jeszcze nie jesteś zalogowany, zamiast przycisku zobaczysz link *Zarejestruj się, aby zostać uczestnikiem kursu*.



Rysunek 14.5. Strona z ogólnym opisem kursu zawierająca przycisk ZAPISZ SIĘ

Renderowanie treści kursu

Gdy uczestnicy są zapisani na kursy, trzeba im zapewnić centralną lokalizację umożliwiającą dostęp do wszystkich kursów, na które są zapisani. Trzeba skompilować listę kursów, na które jest zapisany uczestnik, i zapewnić mu dostęp do treści każdego z nich. Następnie trzeba zaimplementować system do renderowania różnych rodzajów treści, takich jak tekst, obrazy, filmy i dokumenty, które składają się na moduły kursu. Zbudujmy niezbędne widoki i szablony, aby użytkownicy mogli uzyskać dostęp do treści kursów.

Uzyskanie dostępu do treści kursu

Potrzebne są nam widok do wyświetlania kursów, na które zapisali się uczestnicy, oraz widok pozwalający na uzyskanie dostępu do rzeczywistej treści kursu. Przeprowadź edycję pliku *views.py* aplikacji `students` i umieść w nim poniższy fragment kodu.

```
from django.views.generic.list import ListView
from courses.models import Course


class StudentCourseListView(LoginRequiredMixin, ListView):
    model = Course
    template_name = 'students/course/list.html'

    def get_queryset(self):
        qs = super().get_queryset()
        return qs.filter(students__in=[self.request.user])
```

Widok ten jest przeznaczony dla uczestników i wyświetla kursy, na które się zapisali. Dziedziczy po domieszce `LoginRequiredMixin`, aby zagwarantować, że jedynie zalogowani użytkownicy będą mogli uzyskać dostęp do tego widoku. Dziedziczy również po generycznym widoku `ListView` w celu wyświetlenia listy obiektów `Course`. Nadpisujemy metodę `get_queryset()` i za jej pomocą pobieramy jedynie kursy, na które zapisał się uczestnik — filtrujemy kolekcję `QuerySet` względem uczestników podanych w kolumnie `ManyToManyField`.

Kolejnym krokiem jest umieszczenie poniższego fragmentu kodu w pliku *views.py* aplikacji `students`.

```
from django.views.generic.detail import DetailView


class StudentCourseDetailView(LoginRequiredMixin, DetailView):
    model = Course
    template_name = 'students/course/detail.html'

    def get_queryset(self):
        qs = super().get_queryset()
        return qs.filter(students__in=[self.request.user])

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        # Pobranie obiektu kursu.
        course = self.get_object()
        if 'module_id' in self.kwargs:
            # Pobranie bieżącego modułu.
            context['module'] = course.modules.get(
                id=self.kwargs['module_id']
            )
        else:
            # Pobranie pierwszego modułu.
            context['module'] = course.modules.all()[0]
        return context
```

Jest to widok `StudentCourseDetailView`. Nadpisujemy metodę `get_queryset()` w celu ograniczenia bazowej kolekcji `QuerySet` do jedynie tych kursów, na które zapisał się uczestnik. Ponadto nadpisujemy metodę `get_context_data()`, aby wybrać moduł kursu i podać go w kontekście, jeśli wprowadzony został parametr adresu URL o nazwie `module_id`. W przeciwnym razie wybieramy pierwszy moduł kursu. W ten sposób uczestnik ma możliwość poruszania się po modułach wewnątrz kursu.

Przeprowadź edycję pliku *urls.py* aplikacji `students` i umieść w nim poniższe wzorce adresów URL.

```
path(
    'courses/',
    views.StudentCourseListView.as_view(),
    name='student_course_list' ),
path(
    'course/<pk>/',
    views.StudentCourseDetailView.as_view(),
    name='student_course_detail' ),
path(
    'course/<pk>/<module_id>/',
    views.StudentCourseDetailView.as_view(),
    name='student_course_detail_module'
),
```

W katalogu *templates/students/* aplikacji `students` utwórz poniższą strukturę plików i katalogów.

```
course/
    detail.html
    list.html
```

Przeprowadź edycję szablonu *students/course/list.html* i umieść w nim poniższy fragment kodu.

```
{% extends "base.html" %}

{% block title %}Moje kursy{% endblock %}

{% block content %}
  <h1>Moje kursy</h1>
  <div class="module">
    {% for course in object_list %}
      <div class="course-info">
        <h3>{{ course.title }}</h3>
        <p><a href="{% url "student_course_detail" course.id %}">
        Dostęp do treści</a></p>
      </div>
    {% empty %}
      <p>
        Jeszcze nie zapisałeś się na żaden kurs.
        <a href="{% url "course_list" %}">Przejrzyj kursy</a>
        i zapisz się na wybrany.
       </p>
    {% endfor %}
  </div>
{% endblock %}
```

Szablon wyświetla kursy, na które zapisał się uczestnik. Warto zapamiętać, że kiedy nowy uczeń pomyślnie zarejestruje się na platformie, zostanie przekierowany na adres URL `student_course_list`. Na ten adres URL przekierujemy również użytkowników, którzy zalogują się na platformie.

Zmodyfikuj plik *settings.py* w projekcie `educa` poprzez dodanie do niego poniższego fragmentu kodu.

```
from django.urls import reverse_lazy
LOGIN_REDIRECT_URL = reverse_lazy('student_course_list')
```

Jest to ustawienie używane przez moduł `auth` do przekierowania użytkownika po pomyślnym zalogowaniu, jeśli w żądaniu nie ma parametru `next`. Po pomyślnym zalogowaniu się użytkownicy zostaną przekierowani na adres URL `student_course_list`, gdzie wyświetlą się kursy, na które są zapisani.

Przeprowadź edycję szablonu *students/course/detail.html* i umieść w nim poniższy fragment kodu.

```
{% extends "base.html" %}

{% block title %}
  {{ object.title }}
{% endblock %}

{% block content %}
  <h1>
```

```
      {{ module.title }}
    </h1>
    <div class="contents">
      <h3>Moduły</h3>
      <ul id="modules">
        {% for m in object.modules.all %}
          <li data-id="{{ m.id }}" {% if m == module %}class="selected"{% endif %}>
            <a href="{% url "student_course_detail_module" object.id m.id %}">
              <span>
                Moduł <span class="order">{{ m.order|add:1 }}</span>
              </span>
              <br>
              {{ m.title }}
            </a>
          </li>
        {% empty %}
          <li>Brak modułów.</li>
        {% endfor %}
      </ul>
    </div>
    <div class="module">
      {% for content in module.contents.all %}
        {% with item=content.item %}
          <h2>{{ item.title }}</h2>
          {{ item.render }}
        {% endwith %}
      {% endfor %}
    </div>
  {% endblock %}
```

Zadbaj o to, aby żaden znacznik szablonu nie obejmował więcej niż jednego wiersza. Jest to szablon dla zapisanych uczestników; pozwala im na uzyskanie dostępu do treści kursu. Zaczynamy od zbudowania listy HTML zawierającej wszystkie moduły kursu i podświetlamy bieżący. Następnie przeprowadzamy iterację przez treść bieżącego modułu i uzyskujemy do niej dostęp, aby wyświetlić tę treść za pomocą `{{ item.render }}`. W kolejnym kroku do modeli treści dodamy metodę `render()`. Metoda będzie odpowiadała za poprawne wyrenderowanie treści.

Możesz teraz przejść w przeglądarce pod adres `http://127.0.0.1:8000/students/register/`, zarejestrować nowe konto studenta i zapisać się na dowolny kurs.

Renderowanie różnych rodzajów treści

Aby wyświetlić zawartość kursu, potrzebna jest możliwość renderowania różnych typów zawartości, które tworzysz: tekstu, zdjęć, klipów wideo i plików.

Przeprowadź edycję pliku *models.py* w katalogu aplikacji `courses` i umieść poniższą metodę `render()` w modelu `ItemBase`, tak jak pokazałem poniżej.

```
from django.template.loader import render_to_string


class ItemBase(models.Model):
```

```
        # …
        def render(self):
            return render_to_string(
                f'courses/content/{self._meta.model_name}.html',
                {'item': self}
            )
```

Metoda ta wykorzystuje funkcję `render_to_string()` w celu wyrenderowania szablonu, a następnie zwraca przygotowaną treść w postaci ciągu tekstowego. Każdy rodzaj treści jest renderowany za pomocą szablonu o nazwie odpowiadającej modelowi treści. Do dynamicznego przygotowania odpowiedniej nazwy szablonu dla każdego modelu treści używamy wartości `self._meta.model_name`. Metoda `render()` dostarcza wspólny interfejs przeznaczony do renderowania różnorodnej treści.

Utwórz poniższą strukturę plików i katalogów w katalogu *templates/courses/* aplikacji `courses`.

```
content/
    text.html
    file.html
    image.html
    video.html
```

Przeprowadź edycję szablonu *courses/content/text.html* i umieść w nim poniższy wiersz kodu.

```
{{ item.content|linebreaks }}
```

Jest to szablon przeznaczony do renderowania zawartości tekstowej. Szablon filtru `linebreaks` zastępuje znaki podziału wiersza w zwykłym tekście dokumentu znacznikami HTML podziału wiersza.

Przeprowadź edycję szablonu *courses/content/file.html* i umieść w nim poniższy fragment kodu.

```
<p>
  <a href="{{ item.file.url }}" class="button">Pobierz plik</a>
</p>
```

To jest szablon do renderowania plików. Wygenerowaliśmy link do pobierania pliku.

Teraz przeprowadź edycję szablonu *courses/content/image.html* i umieść w nim fragment kodu:

```
<p>
  <img src="{{ item.file.url }}" alt="{{ item.title }}">
</p>
```

To jest szablon do renderowania obrazów.

Trzeba także przygotować widok przeznaczony do renderowania obiektów `Video`. Do obsługi osadzonej treści wideo wykorzystamy moduł Django o nazwie `django-embed-video`. Jest to opracowana przez firmę zewnętrzną aplikacja Django pozwalająca na osadzanie w szablonach klipów wideo pochodzących z serwisów, takich jak YouTube i Vimeo. Do działania wystarczy jedynie publiczny adres URL danego klipu wideo.

Zainstaluj pakiet za pomocą poniższego polecenia.

```
python -m pip install django-embed-video==1.4.9
```

Przeprowadź edycję pliku *settings.py* projektu i dodaj nową aplikację na liście `INSTALLED_APPS`, tak jak pokazałem poniżej.

```
INSTALLED_APPS = [
    # …
    'embed_video',
]
```

Dokumentację modułu `django-embed-video` znajdziesz na stronie *https://django-embed-video.readthedocs.io/en/latest/*.

Przeprowadź edycję szablonu *courses/content/video.html* i umieść w nim poniższy fragment kodu.

```
{% load embed_video_tags %}
{% video item.url "small" %}
```

To jest szablon do renderowania treści wideo.

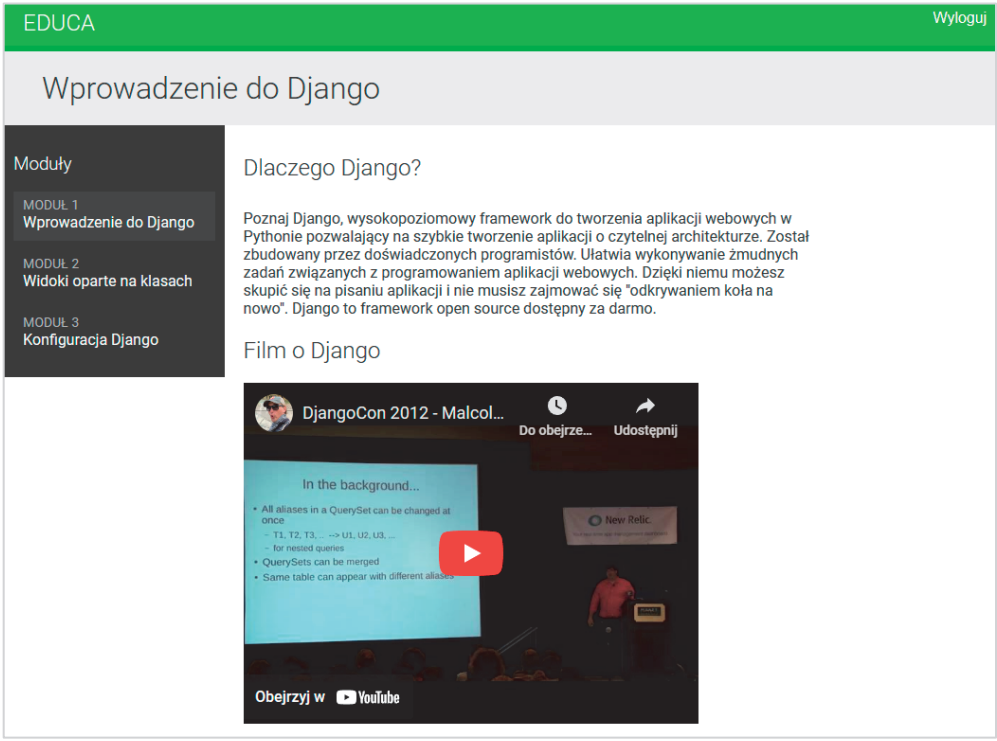
Teraz uruchom serwer programistyczny i przejdź w przeglądarce internetowej pod adres `http://127.0.0.1:8000/course/mine`. Zaloguj się do witryny internetowej za pomocą użytkownika należącego do grupy *Instruktorzy* lub superużytkownika oraz dodaj różne rodzaje treści do kursu. W przypadku treści wideo wystarczy skopiować adres URL klipu w serwisie YouTube, np. *https://www.youtube.com/watch?v=bgV39DlmZ2U*, i umieścić w polu `url` formularza.

Po dodaniu treści do kursu przejdź pod adres `http://127.0.0.1:8000/`, kliknij kurs, a następnie przycisk *ZAPISZ SIĘ*. Powinieneś zostać zapisany na kurs i przekierowany do adresu URL wskazywanego przez `student_course_detail`. Przykładową treść dodaną do kursu pokazałem na zrzucie ekranu widocznym na rysunku 14.6.

Użycie frameworka buforowania

Żądania HTTP do aplikacji sieciowej zwykle obejmują dostęp do bazy danych, przetwarzanie danych oraz renderowanie szablonu. Pod względem przetwarzania jest to operacja znacznie kosztowniejsza niż udostępnianie statycznej witryny internetowej. Gdy wyraźnie zwiększy się ruch w witrynie internetowej, obciążenie powodowane przez niektóre żądania może być znaczne. Dzięki zastosowaniu buforowania zapytań, wyników obliczeń oraz treści wygenerowanej w żądaniu HTTP można uniknąć ponownego wykonywania kosztownych operacji w kolejnych żądaniach wymagających tych samych danych. A to oznacza krótszy czas udzielenia odpowiedzi i mniejsze obciążenie serwera.

Framework Django oferuje solidny system buforowania pozwalający na buforowanie danych na różnych poziomach szczegółowości. Można buforować pojedyncze zapytanie, dane wyjściowe konkretnego widoku, fragmenty wygenerowanej treści szablonu lub nawet całą witrynę internetową. Elementy są przechowywane w buforze systemu przez domyślną ilość czasu. Oczywiście istnieje możliwość zdefiniowania czasu ważności buforowanych elementów.



Rysunek 14.6. Strona z zawartością kursu

Doskonale! Przygotowałeś interfejs przeznaczony do renderowania kursów z różnymi rodzajami treści.

Oto sposób, w jaki najczęściej używany jest framework buforowania, gdy aplikacja otrzymuje żądanie HTTP::

1. Próba odszukania w buforze żądanych danych.
2. Jeżeli dane zostaną znalezione, framework zwraca je żądającemu.
3. Jeżeli dane nie zostaną znalezione, wykonywane są poniższe kroki.
   A. Wykonanie zapytania do bazy danych lub operacji przetwarzania wymaganej do otrzymania żądanych danych.
   B. Zapis wygenerowanych danych w buforze.
   C. Zwrócenie danych.

Dokładne informacje o systemie buforowania w Django znajdziesz w dokumentacji dostępnej na stronie *https://docs.djangoproject.com/en/5.0/topics/cache/*.

Dostępne backendy buforowania

Framework Django oferuje kilka backendów buforowania. Oto one:

- `backends.memcached. MemcachedCache` lub `backends.memcached.PyLibMCCache`. `Memcached backends. Memcached` jest szybkim i wydajnym serwerem buforowania przechowującym dane w pamięci. Konkretny backend zależy od wybranego modułu powiązania `Memcached` dla Pythona.
- `backends.redis.RedisCache`. Backend buforowania oparty na bazie danych Redis. Ten backend buforowania dodano w Django 4.0.
- `backends.db.DatabaseCache`. Użycie bazy danych jako systemu buforowania.
- `backends.filebased.FileBasedCache`. Użycie systemu plików jako backendu buforowania. Każda wartość bufora jest serializowana i przechowywana w oddzielnym pliku.
- `backends.locmem.LocMemCache`. Backend buforowania oparty na pamięci lokalnej. Stosowany domyślnie.
- `backends.dummy.DummyCache`. Testowy mechanizm buforowania przeznaczony do użycia jedynie w środowisku programistycznym. Implementuje interfejs buforowania bez rzeczywistego buforowania jakichkolwiek danych. Mechanizm jest przeznaczony dla poszczególnych procesów i zapewnia bezpieczeństwo wątków.

Uwaga

W celu uzyskania optymalnej wydajności należy użyć mechanizmu buforowania przechowującego dane w pamięci, takiego jak `Memcached` lub Redis.

Instalacja Memcached

`Memcached` to popularny wydajny serwer buforowania działający w pamięci. W omawianym przykładzie będziemy używać backendu `Memcached` oraz `PyMemcacheCache`.

Instalowanie obrazu Docker mechanizmu Memcached

Aby pobrać obraz Docker `Memcached`, uruchom z poziomu powłoki polecenie:

```
docker pull memcached:1.6.26
```

Spowoduje to pobranie na komputer lokalny obrazu Docker dla systemu buforowania `Memchached`. Informacje dotyczące oficjalnego obrazu Docker systemu `Memcached` można znaleźć na stronie *https://hub.docker.com/_/memcached*. Jeśli nie chcesz korzystać z Dockera, możesz pobrać `Memcached` na stronie *https://memcached.org/downloads.*

Uruchom kontener Docker `Memcached` za pomocą polecenia:

```
docker run -it --rm --name memcached -p 11211:11211 memcached:1.6.26 -m 64
```

Domyślnie Memcached działa na porcie numer 11211. Opcja -p służy do opublikowania portu 11211 do tego samego portu interfejsu hosta. Opcja -m służy do ograniczenia pamięci dla kontenera do 64 MB. Memcached działa w pamięci i na swoje potrzeby wykorzystuje pewną ilość pamięci RAM. Po zapełnieniu przeznaczonej dla niego pamięci RAM Memcached zaczyna usuwanie najstarszych danych z bufora w celu zrobienia miejsca dla nowych danych. Jeśli chcesz uruchomić bufor w trybie odłączonym (ang. *detached mode* — tzn. w tle terminala), możesz użyć opcji -d.

Więcej informacji o Memcached znajdziesz w witrynie *https://memcached.org/*.

Instalacja powiązania Memcached dla języka Python

Po zainstalowaniu bufora Memcached, trzeba zainstalować powiązanie Memcached dla Pythona. W omawianym przykładzie zainstalujemy pymemcache — szybkiego klienta Memcached napisanego w czystym Pythonie. W wierszu polecenia powłoki uruchom polecenie:

```
python -m pip install pymemcache==4.0.0
```

Więcej informacji o bibliotece pymemcache można znaleźć na stronie *https://github.com/pinterest/pymemcache.*

Ustawienia buforowania we frameworku Django

Oto ustawienia buforowania, które są obsługiwane przez Django:

- CACHES. Słownik zawierający wszystkie bufory dostępne dla projektu.
- CACHE_MIDDLEWARE_ALIAS. Alias bufora do użycia jako magazynu danych.
- CACHE_MIDDLEWARE_KEY_PREFIX. Prefiks do użycia dla kluczy bufora. Tego rodzaju prefiks należy zdefiniować, aby uniknąć kolizji kluczy, gdy ten sam bufor jest współdzielony przez wiele witryn internetowych.
- CACHE_MIDDLEWARE_SECONDS. Domyślna liczba sekund, przez które mają być buforowane strony.

System buforowania dla projektu może być skonfigurowany za pomocą opcji CACHES. Jest to słownik pozwalający na podanie konfiguracji dla wielu backendów buforowania. Dla każdego bufora uwzględnionego w słowniku CACHES można określić następujące dane.

- BACKEND. Wskazuje używany backend buforowania.
- KEY_FUNCTION. Ciąg tekstowy zawierający ścieżkę dostępu do obiektu wywoływalnego, który przyjmuje argumenty w postaci prefiksu, numeru wersji i klucza oraz zwraca ostateczny klucz bufora.
- KEY_PREFIX. Prefiks w postaci ciągu tekstowego dla wszystkich kluczy bufora, którego użycie pozwala na uniknięcie kolizji.

- `LOCATION`. Położenie bufora. W zależności od użytego backendu buforowania, mogą to być katalog, nazwa hosta i numer portu bądź nazwa backendu przechowującego dane w pamięci.
- `OPTIONS`. Wszelkie parametry dodatkowe przekazywane mechanizmowi buforowania.
- `TIMEOUT`. Wyrażony w sekundach domyślny czas utraty ważności kluczy bufora. Wartością domyślną jest 300 sekund, czyli 5 minut. Przypisanie wartości `None` oznacza, że klucze bufora nie będą miały ustalonego czasu utraty ważności.
- `VERSION`. Domyślny numer wersji kluczy buforowania. Przydatny do wersjonowania bufora.

Więcej informacji na temat ustawień `CACHES` można znaleźć na stronie *https://docs.djangoproject.com/en/5.0/ref/settings/#caches*.

Dodanie Memcached do projektu

Przystępujemy do skonfigurowania bufora dla budowanego projektu. Zmodyfikuj plik *settings.py* w projekcie `educa` poprzez dodanie do niego poniższego fragmentu kodu.

```
CACHES = {
    'default': {
        'BACKEND': 'django.core.cache.backends.memcached.PyMemcacheCache',
        'LOCATION': '127.0.0.1:11211',
    }
}
```

Używamy backendu `PyMemcacheCache`. Jego położenie wskazujemy za pomocą notacji `adres:port`. Jeżeli masz wiele egzemplarzy backendu `Memcached`, możesz użyć listy dla opcji `LOCATION`.

Właśnie skonfigurowaliśmy mechanizm `Memcached` dla projektu. Możemy zacząć buforować dane!

Poziomy buforowania

Django oferuje opisane poniżej poziomy buforowania; wymieniłem je w malejącej kolejności szczegółowości.

- **Niskopoziomowe API buforowania.** Zapewnia największy poziom szczegółowości, ponieważ pozwala na buforowanie określonych zapytań lub obliczeń.
- **Bufor szablonu**. Pozwala na buforowanie fragmentów szablonu.
- **Bufor dla poszczególnych widoków**. Zapewnia buforowanie poszczególnych widoków.
- **Bufor dla poszczególnych witryn**. Bufor najwyższego poziomu, zapewnia buforowanie całej witryny.

Wskazówka

Dokładnie przemyśl strategię buforowania, zanim zaczniesz ją implementować. Najpierw skoncentruj się na najkosztowniejszych zapytaniach lub obliczeniach, które nie są przeprowadzane dla poszczególnych użytkowników.

W kolejnych punktach pokażę, jak wykorzystać każdy z wymienionych poziomów buforowania w budowanym projekcie.

Zacznijmy od nauczenia się sposobu korzystania z niskopoziomowego API buforowania w kodzie Pythona.

Użycie niskopoziomowego API buforowania

Niskopoziomowe API buforowania pozwala na przechowywanie obiektów w buforze z zachowaniem dowolnego poziomu szczegółowości. Tego rodzaju API zostało zdefiniowane w aplikacji `django.core.cache` i można je zastosować w projekcie po zaimportowaniu w poniższy sposób:

```
from django.core.cache import cache
```

W takim przypadku używany jest bufor domyślny. Jest to odpowiednik polecenia `caches ↪['default']`. Uzyskanie dostępu do określonego bufora jest również możliwe za pomocą jego aliasu.

```
from django.core.cache import caches
my_cache = caches['alias']
```

Spójrzmy na sposób działania API buforowania. Przejdź do powłoki za pomocą polecenia:

```
python manage.py shell
```

Następnie uruchom polecenie:

```
>>> from django.core.cache import cache
>>> cache.set('muzyk', 'Django Reinhardt', 20)
```

Uzyskujemy dostęp do domyślnego bufora i używamy polecenia `set(klucz, wartość, ↪czas-utraty-ważności)` do przechowywania klucza o nazwie `'muzyk'` i wartości w postaci ciągu tekstowego `'Django Reinhardt'` przez 20 sekund. Jeżeli nie zostanie podany czas utraty ważności, Django użyje domyślnego zdefiniowanego w ustawieniu `CACHES`. Teraz wykonaj poniższy fragment kodu.

```
>>> cache.get('muzyk')
'Django Reinhardt'
```

Klucz został pobrany z bufora. Odczekaj 20 sekund i ponownie wykonaj ten sam kod.

```
>>> cache.get('muzyk')
```

Nie została zwrócona żadna wartość. Klucz bufora `'muzyk'` wygasł, więc metoda `get()` zwraca wartość `None`, ponieważ klucz o wskazanej nazwie nie znajduje się już w buforze.

Wskazówka

Unikaj przechowywania wartości `None` w kluczu bufora, ponieważ wtedy nie będziesz mógł rozróżnić rzeczywistej wartości od nietrafienia bufora.

Spróbujmy buforować kolekcję QuerySet.

```
>>> from courses.models import Subject
>>> subjects = Subject.objects.all()
>>> cache.set('wszystkie_dziedziny', subjects)
```

Przeprowadzamy zapytanie do modelu `Subject` i zwrócony obiekt przechowujemy w kluczu o nazwie `'wszystkie_dziedziny'`. Spróbujmy teraz pobrać buforowane dane.

```
>>> cache.get('wszystkie_dziedziny')
<QuerySet [<Subject: Matematyka>, <Subject: Muzyka>, <Subject: Fizyka>,
<Subject: Informatyka>]>
```

Tego rodzaju zapytanie będziemy buforować w widokach. Przeprowadź edycję pliku *views.py* aplikacji `courses` i umieść w nim poniższe polecenie `import`.

```
from django.core.cache import cache
```

W metodzie `get()` widoku `CourseListView` poniższe wiersze kodu:

```
subjects = Subject.objects.annotate(
    total_courses=Count('courses')
)
```

zastąp fragmentem:

```
subjects = cache.get('wszystkie_dziedziny')
if not subjects:
    subjects = Subject.objects.annotate(
        total_courses=Count('courses')
    )
    cache.set('wszystkie_dziedziny', subjects)
```

W tym kodzie najpierw próbujemy pobrać klucz `wszystkie_dziedziny` z bufora za pomocą funkcji `cache.get()`. Jeżeli wskazany klucz nie zostanie znaleziony, funkcja zwróci `None`. Jeżeli żaden klucz nie będzie znaleziony (niebuforowany jeszcze lub buforowany, ale wygasły), wykonujemy zapytanie w celu pobrania wszystkich obiektów `Subject` oraz ich liczby kursów, a wynik buforujemy za pomocą `cache.set()`.

Sprawdzanie żądań pobierających dane z bufora za pomocą paska narzędzi Django Debug Toolbar

W celu przetestowania zapytań do pamięci podręcznej, dodajmy do projektu pasek narzędzi Django Debug Toolbar. Z paska narzędzi Django Debug Toolbar nauczyłeś się korzystać w rozdziale 7., „Śledzenie działań użytkownika”.

Najpierw zainstaluj pasek narzędzi Django Debug Toolbar za pomocą polecenia:

```
python -m pip install django-debug-toolbar==4.3.0
```

Przeprowadź edycję pliku *settings.py* w swoim projekcie i dodaj do ustawienia `INSTALLED_APPS` aplikację `debug_toolbar` tak, jak pokazałem poniżej. Nowy kod został wyróżniony pogrubioną czcionką.

```
INSTALLED_APPS = [
    # …
    'debug_toolbar',
]
```

W tym samym pliku dodaj do ustawienia `MIDDLEWARE` wiersz wyróżniony na poniższym listingu pogrubioną czcionką.

```
MIDDLEWARE = [
    'debug_toolbar.middleware.DebugToolbarMiddleware',
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]
```

Aplikację `DebugToolbarMiddleware` należy umieścić przed innymi aplikacjami middleware, z wyjątkiem tych, które kodują treść odpowiedzi, na przykład `GzipMiddleware` (jeśli ta ostatnia aplikacja jest zainstalowana, powinna znaleźć się na pierwszym miejscu).

Dodaj na końcu pliku *settings.py* wiersze:

```
INTERNAL_IPS = [
    '127.0.0.1',
]
```

Pasek narzędzi Django Debug Toolbar wyświetli się tylko wtedy, gdy Twój adres IP pasuje do wpisu w ustawieniu `INTERNAL_IPS`.

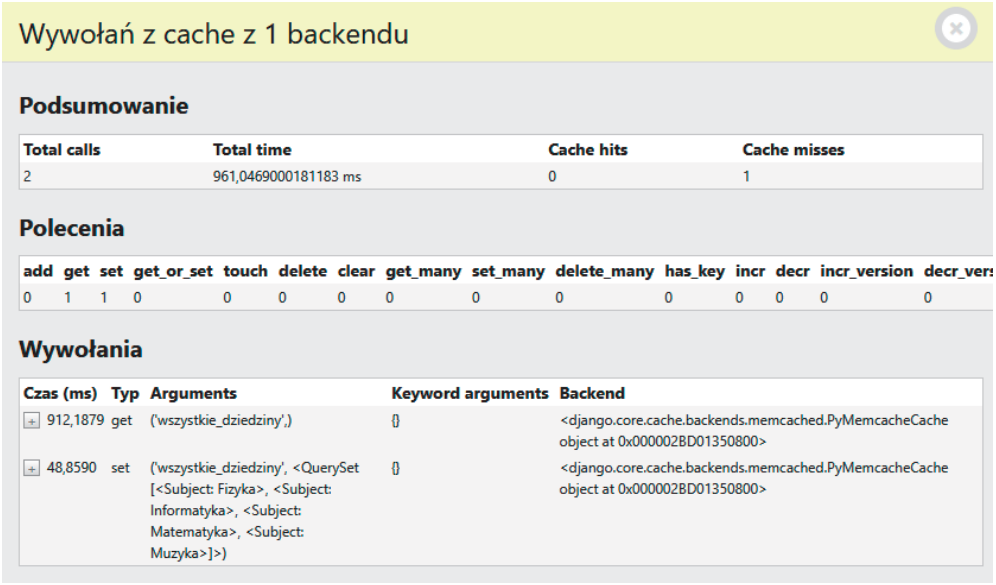
Przeprowadź edycję głównego pliku *urls.py* projektu i dodaj do listy `urlpatterns` poniższy wzorzec adresu URL.

```
path('__debug__/', include('debug_toolbar.urls')),
```

Uruchom serwer programistyczny i przejdź w przeglądarce internetowej pod adres `http://127.0.0.1:8000/`.

Po prawej stronie ekranu powinieneś zobaczyć pasek narzędzi Django Debug Toolbar. W menu paska bocznego kliknij pozycję *Cache*. Wyświetli się panel pokazany na rysunku 14.7.

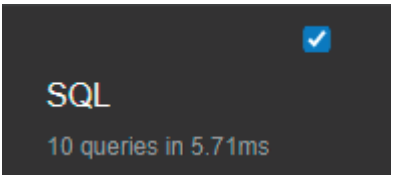
W obszarze *Total calls* powinna być widoczna liczba 2. Przy pierwszym uruchomieniu widoku `CourseListView` są dwa żądania do pamięci podręcznej. W obszarze *Commands* zobaczysz, że polecenie `get` zostało wykonane raz. Polecenie `set` również zostało wykonane raz. Polecenie `get` odpowiada wywołaniu, które pobiera klucz pamięci podręcznej



Rysunek 14.7. Panel Cache paska narzędzi Django Debug Toolbar, w tym żądania pamięci podręcznej dla widoku CourseListView w przypadku braku wpisu w pamięci podręcznej

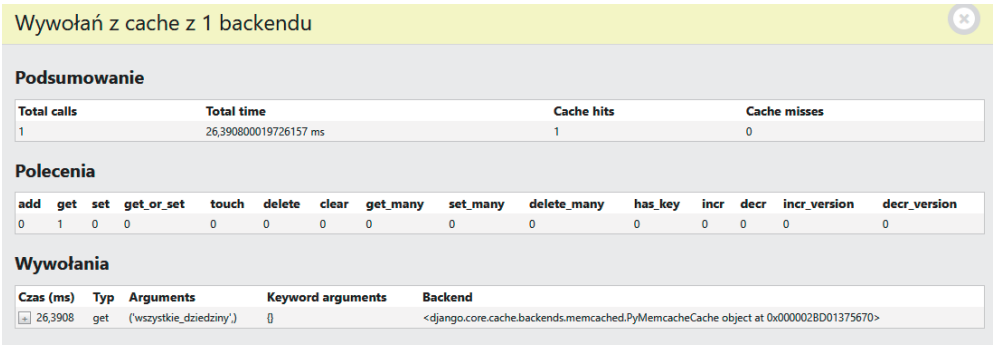
`wszystkie_dziedziny`. To jest pierwsze połączenie wyświetlane w obszarze *Calls*. Przy pierwszym uruchomieniu widoku występuje błąd nietrafienia wpisu w pamięci podręcznej, ponieważ w tym momencie w pamięci podręcznej nie są jeszcze przechowywane żadne dane. Dlatego w obszarze *Cache misses* wyświetla się wartość 1. Następnie, w celu przechowania wyników odczytu kolekcji `QuerySet` w pamięci podręcznej przy użyciu klucza `wszystkie_dziedziny`, używamy polecenia `set`. Jest to drugie wywołanie wyświetlane w obszarze *Calls*.

W pozycji *SQL* menu paska narzędzi Django Debug Toolbar zobaczysz całkowitą liczbę zapytań SQL wykonanych w tym żądaniu. Obejmuje to także zapytanie wykonywane w celu pobrania wszystkich tematów, które następnie są przechowywane w pamięci podręcznej (rysunek 14.8).



Rysunek 14.8. Zapytania SQL wykonane dla widoku CourseListView w przypadku nietrafienia wpisu w pamięci podręcznej

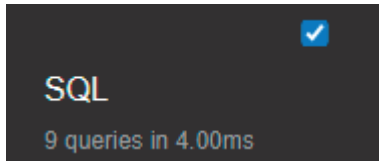
Ponownie załaduj stronę w przeglądarce i w menu paska bocznego kliknij pozycję *Cache* (rysunek 14.9).



Rysunek 14.9. Panel Cache paska narzędzi Django Debug Toolbar, w tym żądania pamięci podręcznej dla widoku CourseListView w przypadku trafienia odwołania do pamięci podręcznej

Teraz jest tylko jedno żądanie pamięci podręcznej. W obszarze *Total calls* powinna wyświetlić się wartość 1. W obszarze *Commands* możesz zaobserwować, że żądanie pamięci podręcznej odpowiada poleceniu `get`. W tym przypadku mamy trafienie w pamięci podręcznej (patrz *Cache hits*) zamiast chybienia, ponieważ w pamięci podręcznej zostały znalezione dane. W obszarze *Calls* możesz zobaczyć żądanie pobrania klucza pamięci podręcznej `wszystkie_dziedziny`.

Sprawdź na pasku narzędzi debugowania pozycję menu *SQL*. Powinieneś zobaczyć, że w tym żądaniu jest o jedno zapytanie SQL mniej. Zaoszczędziliśmy jedno zapytanie SQL, ponieważ widok znalazł dane w pamięci podręcznej i nie musi ich pobierać z bazy danych (rysunek 14.10).



Rysunek 14.10. Zapytania SQL wykonane dla widoku CourseListView w przypadku trafienia wpisu w pamięci podręcznej

W tym przykładzie w przypadku pojedynczego żądania pobranie elementu z pamięci podręcznej zajmuje więcej czasu niż czas zaoszczędzony na dodatkowym zapytaniu SQL. Jednak gdy do Twojej witryny uzyskuje dostęp wielu użytkowników, dzięki pobieraniu danych z pamięci podręcznej zamiast sięgania po nie do bazy danych znacznie skrócisz czas dostępu do strony i będziesz mógł obsługiwać w witrynie znacznie większą liczbę jednoczesnych użytkowników.

Kolejne żądania kierowane pod ten sam adres URL spowodują pobieranie danych z pamięci podręcznej. Ponieważ podczas buforowania danych w widoku `CourseListView` za pomocą funkcji `cache.set('wszystkie_dziedziny', Subjects)` nie określiliśmy limitu czasu dla wpisów w pamięci podręcznej, w tym przypadku zostanie użyty domyślny limit czasu (domyślnie 300 sekund, czyli 5 minut). Po osiągnięciu limitu czasu następne żądanie do adresu URL wygeneruje błąd chybienia odwołania do pamięci podręcznej. W efekcie

zostanie wykonana kolekcja `QuerySet`, a uzyskane dane będą przechowywane w pamięci podręcznej przez kolejne 5 minut. Aby zdefiniować inny domyślny limit czasu, możesz wprowadzić odpowiednie ustawienie w opcji `TIMEOUT` ustawienia `CACHES`.

Niskopoziomowe buforowanie na podstawie danych dynamicznych

Wielokrotnie spotkasz się z koniecznością buforowania czegoś bazującego na danych dynamicznych. W takich przypadkach trzeba utworzyć klucze dynamiczne zawierające wszystkie informacje niezbędne do unikatowej identyfikacji buforowanych danych.

Przeprowadź edycję pliku *views.py* aplikacji `courses` i zmodyfikuj widok `CourseListView` w następujący sposób.

```
class CourseListView(TemplateResponseMixin, View):
    model = Course
    template_name = 'courses/course/list.html'

    def get(self, request, subject=None):
        subjects = cache.get('wszystkie_dziedziny')
        if not subjects:
            subjects = Subject.objects.annotate(
                total_courses=Count('courses')
            )
            cache.set('all_subjects', subjects)
        all_courses = Course.objects.annotate(
            total_modules=Count('modules')
        )
        if subject:
            subject = get_object_or_404(Subject, slug=subject)
            key = f'subject_{subject.id}_courses'
            courses = cache.get(key)
            if not courses:
                courses = all_courses.filter(subject=subject)
                cache.set(key, courses)
        else:
            courses = cache.get('wszystkie_kursy')
            if not courses:
                courses = all_courses
                cache.set('wszystkie_kursy', courses)
        return self.render_to_response(
            {
                'subjects': subjects,
                'subject': subject,
                'courses': courses
            }
        )
```

W omawianym przykładzie buforujemy wszystkie kursy oraz kursy filtrowane według tematu. Do przechowywania wszystkich kursów, gdy nie został podany temat, wykorzystaliśmy klucz `wszystkie_kursy`. Natomiast po podaniu tematu klucz tworzymy dynamicznie za pomocą instrukcji `'subject_{}_courses'.format(subject.id)`.

Warto zwrócić uwagę na jeden fakt — nie można użyć buforowanej kolekcji `QuerySet` do przygotowania innej, ponieważ buforowane dane to w rzeczywistości wynik wykonania zapytania z wykorzystaniem kolekcji `QuerySet`. Dlatego też nie można zastosować poniższego rozwiązania.

```
courses = cache.get('wszystkie_kursy')
courses.filter(subject=subject)
```

W zamian trzeba za pomocą wywołania `Course.objects.annotate(total_modules=Count('modules'))` utworzyć bazową kolekcję `QuerySet`, która nie zostanie wykonana, aż do chwili gdy wystąpi taka konieczność. Dopiero wówczas można ją wykorzystać do dalszego ograniczenia `QuerySet` za pomocą wywołania `all_courses.filter(subject=subject)` na wypadek nieznalezienia danych w buforze.

Buforowanie fragmentów szablonu

Buforowanie fragmentów szablonu to podejście na wyższym poziomie. Konieczne jest wczytanie znaczników buforowania w szablonie za pomocą wywołania `{% load cache %}`. Następnie można użyć znacznika szablonu `{% cache %}` do buforowania określonych fragmentów szablonu. Zwykle ze znaczników szablonu korzystamy w następujący sposób.

```
{% cache 300 nazwa_fragmentu %}
    ...
{% endcache %}
```

Znacznik `{% cache %}` wymaga dwóch argumentów — wyrażonego w sekundach czasu wygaśnięcia ważności elementu oraz nazwy dla fragmentu. Jeżeli trzeba buforować treści na podstawie danych dynamicznych, można przekazać dodatkowe argumenty do znacznika szablonu `{% cache %}`, aby unikatowo zidentyfikować ten fragment.

Przeprowadź edycję szablonu */students/course/detail.html* aplikacji `students`. Na początku pliku, zaraz po znaczniku `{% extends %}` umieść poniższy wiersz kodu.

```
{% load cache %}
```

Następnie poniższe wiersze kodu:

```
{% for content in module.contents.all %}
  {% with item=content.item %}
    <h2>{{ item.title }}</h2>
    {{ item.render }}
  {% endwith %}
{% endfor %}
```

zastąp fragmentem:

```
{% cache 600 module_contents module %}
  {% for content in module.contents.all %}
    {% with item=content.item %}
      <h2>{{ item.title }}</h2>
      {{ item.render }}
    {% endwith %}
  {% endfor %}
{% endcache %}
```

Fragment szablonu buforujemy, używając nazwy `module_contents` i przekazując jej bieżący obiekt `Module`. W ten sposób unikatowo identyfikujemy dany fragment. Bardzo ważne jest, aby unikać buforowania treści modułu i dostarczania nieprawidłowej treści, gdy żądany jest inny moduł.

Jeżeli wartością opcji `USE_I18N` jest `True`, bufor dla poszczególnych witryn internetowych będzie respektował aktywny język. Jeżeli używasz znacznika szablonu `{% cache %}`, wówczas w celu osiągnięcia tego samego efektu musisz użyć jednej ze specjalnie przeznaczonych dla tłumaczeń zmiennej dostępnej w szablonach, np. `{% cache 600 name request ↪.LANGUAGE_CODE %}`.

Buforowanie widoków

Za pomocą dekoratora `cache_page` zdefiniowanego w `django.views.decorators.cache` można buforować dane wyjściowe poszczególnych widoków. Dekorator wymaga podania wyrażonego w sekundach argumentu określającego czas wygaśnięcia ważności buforowanych danych.

Spróbujmy teraz buforować widoki aplikacji. Przeprowadź edycję pliku *urls.py* aplikacji `students` i umieść w nim poniższe polecenie `import`.

```
from django.views.decorators.cache import cache_page
```

Następnie zastosuj dekorator `cache_page` względem wzorców adresów URL `student_ ↪course_detail` i `student_course_detail_module`, tak jak pokazałem poniżej.

```
path(
    'course/<pk>/',
    cache_page(60 * 15)(views.StudentCourseDetailView.as_view()),
    name='student_course_detail' ),
path(
    'course/<pk>/<module_id>/',
    cache_page(60 * 15)(views.StudentCourseDetailView.as_view()),
    name='student_course_detail_module' ),
```

Po wprowadzeniu tych ustawień cała zawartość widoku `StudentCourseDetailView` jest buforowana przez 15 minut.

Wskazówka

Bufor dla poszczególnych widoków korzysta z adresu URL do utworzenia klucza bufora. Dlatego też wiele adresów URL prowadzących do tego samego widoku będzie buforowanych oddzielnie.

Użycie bufora dla poszczególnych witryn

Jest to bufor najwyższego poziomu i pozwala na buforowanie całej witryny internetowej. Aby umożliwić buforowanie poszczególnych witryn internetowych, przeprowadź edycję pliku *settings.py* projektu polegającą na dodaniu `UpdateCacheMiddleware` i `Fetch ↪FromCacheMiddleware` do ustawienia `MIDDLEWARE`, tak jak pokazałem poniżej.

Opcja MIDDLEWARE może mieć następującą postać:

```
MIDDLEWARE = [
    'debug_toolbar.middleware.DebugToolbarMiddleware',
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.cache.UpdateCacheMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.cache.FetchFromCacheMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]
```

Pamiętaj, że warstwy middleware są uruchamiane w podanej kolejności podczas fazy żądania oraz w odwrotnej kolejności podczas fazy odpowiedzi. Oprogramowanie UpdateCacheMiddleware zostało umieszczone przed CommonMiddleware, ponieważ jest wykonywane w fazie odpowiedzi, gdy lista oprogramowania middleware jest wykonywana w odwrotnej kolejności. Z kolei FetchFromCacheMiddleware zostało celowo umieszczone po CommonMiddleware, ponieważ musi uzyskać dostęp do danych żądania zdefiniowanych przez drugie z wymienionych.

Następnie umieść poniższe ustawienia w pliku *settings.py*.

```
CACHE_MIDDLEWARE_ALIAS = 'default'
CACHE_MIDDLEWARE_SECONDS = 60 * 15  # 15 minut
CACHE_MIDDLEWARE_KEY_PREFIX = 'educa'
```

W powyższych ustawieniach określiliśmy domyślny mechanizm buforowania dla warstwy middleware buforowania, a także globalnie zdefiniowaliśmy czas wygaśnięcia ważności buforowanych danych po upływie 15 minut. Ponadto wskazaliśmy prefiks dla wszystkich kluczy buforowania, aby uniknąć kolizji na wypadek, gdyby ten sam mechanizm Memcached był stosowany w wielu projektach. Budowana witryna internetowa jest teraz buforowana i zwraca buforowaną treść dla wszystkich żądań GET.

Dostęp do różnych stron w celu sprawdzenia żądań pamięci podręcznej możesz uzyskać za pomocą paska narzędzi Django Debug Toolbar. Buforowanie dla poszczególnych witryn w przypadku, gdy jest ich wiele jest nieopłacalne. Wpływa ono bowiem na wszystkie widoki — nawet te, których nie chcesz buforować. Na przykład buforowane są widoki zarządzania, w przypadku których, w celu odzwierciedlenia najnowszych zmian, zawsze chcesz zwracać dane z bazy danych.

W tworzonym projekcie najlepszym podejściem jest zastosowanie buforowania do szablonów lub widoków używanych do wyświetlania treści kursu studentom i jednoczesne zachowanie bez buforowania widoków zarządzania treścią przeznaczonych dla instruktorów.

Z tego powodu zdezaktywujemy opcję buforowania dla poszczególnych witryn. Przeprowadź edycję pliku *settings.py* swojego projektu i oznacz jako komentarz klasy UpdateCacheMiddleware i FetchFromCacheMiddleware w ustawieniu MIDDLEWARE w następujący sposób.

```
MIDDLEWARE = [
    'debug_toolbar.middleware.DebugToolbarMiddleware',
    'django.middleware.security.SecurityMiddleware',
```

```
    'django.contrib.sessions.middleware.SessionMiddleware',
    # 'django.middleware.cache.UpdateCacheMiddleware',
    'django.middleware.common.CommonMiddleware',
    # 'django.middleware.cache.FetchFromCacheMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]
```

Przedstawiłem ogólne omówienie dostarczanych przez framework Django metod buforowania danych. Powinieneś zawsze rozważnie zdefiniować strategię buforowania i preferować buforowanie najkosztowniejszych kolekcji `QuerySet` lub obliczeń, danych które nie będą się często zmieniać oraz danych, do których będzie miało jednoczesny dostęp wielu użytkowników.

Backend buforowania oparty na bazie danych Redis

Framework Django zawiera backend buforowania oparty na bazie danych Redis. Spróbujmy zmienić ustawienia w taki sposób, aby w budowanym projekcie zamiast `Memcached` używać backendu buforowania opartego na bazie danych Redis. Jak pewnie pamiętasz, z tej bazy danych korzystaliśmy już w rozdziale 7., „Śledzenie działań użytkownika", i w rozdziale 10., „Rozbudowa sklepu internetowego".

Najpierw zainstaluj moduł `redis-py` w swoim środowisku za pomocą polecenia:

```
python -m pip install redis==5.0.4
```

Następnie przeprowadź edycję pliku *settings.py* projektu `educa` i zmodyfikuj ustawienie `CACHES` w następujący sposób.

```
CACHES = {
    'default': {
        'BACKEND': 'django.core.cache.backends.redis.RedisCache',
        'LOCATION': 'redis://127.0.0.1:6379',
    }
}
```

Projekt będzie teraz korzystał z backendu buforowania `RedisCache`. Lokalizacja została zdefiniowana w formacie *redis://[host]:[port]*. W tym projekcie użyliśmy ustawienia adresu hosta `127.0.0.1`, aby wskazać lokalnego hosta i portu `6379`, który jest domyślnym portem dla Redis.

Więcej informacji o backendzie buforowania opartym na Redis można znaleźć na stronie *https://docs.djangoproject.com/en/5.0/topics/cache/#redis*.

Zainicjuj kontener Docker bazy danych Redis za pomocą polecenia:

```
docker run -it --rm --name redis -p 6379:6379 redis:7.2.4
```

Aby uruchomić polecenie w tle (w trybie odłączonym), możesz użyć opcji `-d`.

Uruchom serwer programistyczny i przejdź w przeglądarce internetowej pod adres `http://127.0.0.1:8000/`. W panelu *Cache* na pasku narzędzi Django Debug Toolbar sprawdź żądania pamięci podręcznej. Teraz zamiast `Memcached` używasz jako backendu pamięci podręcznej swojego projektu bazy danych Redis.

Monitorowanie bazy danych Redis za pomocą Django Redisboard

Możesz monitorować swój serwer Redis za pomocą narzędzia Django Redisboard. Django Redisboard dodaje statystyki Redis do strony administracyjnej Django. Więcej informacji na temat Django Redisboard można znaleźć pod adresem *https://github.com/ionelmc/django-redisboard*.

Zainstaluj moduł `django-redisboard` w swoim środowisku za pomocą polecenia:

```
python -m pip install django-redisboard==8.4.0
```

Przeprowadź edycję pliku *settings.py* projektu i dodaj nową aplikację na liście `INSTALLED_APPS`, tak jak pokazałem poniżej.

```
INSTALLED_APPS = [
    # …
    'redisboard',
]
```

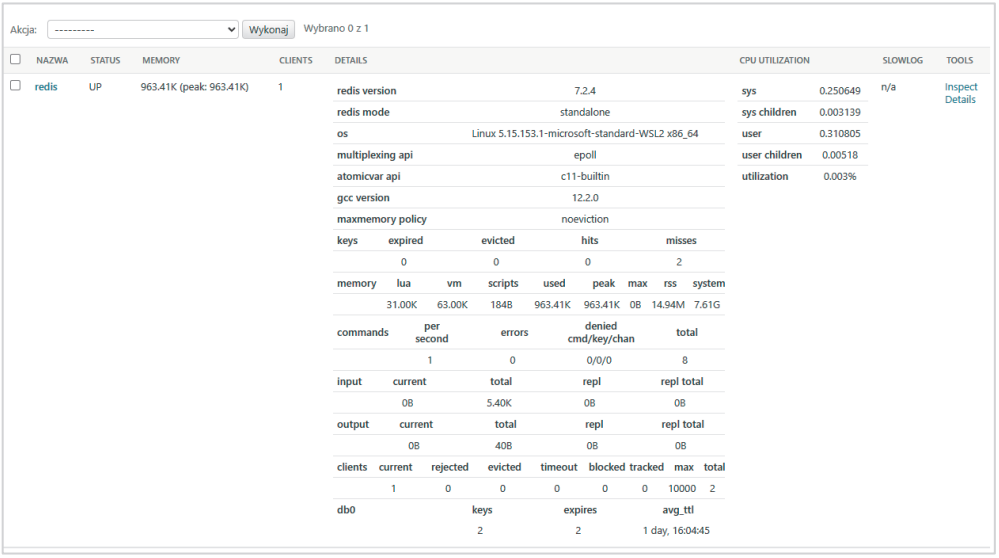
Aby uruchomić migracje Django Redisboard uruchom z katalogu swojego projektu polecenie:

```
python manage.py migrate redisboard
```

Aby dodać serwer Redis do monitorowania uruchom serwer programistyczny i przejdź w przeglądarce internetowej pod adres *http://127.0.0.1:8000/admin/redisboard/redisserver/add/*. W polu *Label* wpisz **`redis`**, a w polu *URL* wpisz **`redis://localhost:6379/0`**, jak pokazałem na rysunku 14.11.

Rysunek 14.11. Formularz dodawania serwera Redis dla Django Redisboard w witrynie administracyjnej

Będziemy monitorować egzemplarz bazy danych Redis działający na lokalnym hoście i korzystający z portu 6379 oraz bazy danych Redis o numerze 0. Kliknij *ZAPISZ*. Informacje zostaną zapisane w bazie danych, a konfigurację i metryki Redis będzie można zobaczyć na stronie administracyjnej Django (rysunek 14.12).



Rysunek 14.12. Monitorowanie Redis Django Redisboard na stronie administracyjnej

Gratulacje! Pomyślnie zaimplementowałeś buforowanie dla swojego projektu.

Podsumowanie

W tym rozdziale przygotowałeś publiczne widoki dla katalogu kursów, a także system pozwalający na rejestrację uczestników i zapisywanie się na kursy. Utworzyłeś także funkcjonalność odpowiedzialną za renderowanie różnych typów zawartości dla modułów kursów. Na koniec dowiedziałeś się, jak skorzystać z frameworka buforowania Django i zainstalowałeś backend buforowania `Memcached`, który następnie nauczyłeś się monitorować.

W następnym rozdziale zajmiemy się utworzeniem API typu RESTful dla budowanego projektu z wykorzystaniem frameworka API REST Django.

Dodatkowe zasoby

Oto lista zasobów zawierających dodatkowe informacje związane z tematami omawianymi w tym rozdziale:

- Kod źródłowy przykładów zamieszczonych w tym rozdziale — *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter14*.

- Dokumentacja modułu `django-embed-video` — *https://django-embed-video.readthedocs.io/en/latest/*
- Dokumentacja frameworka buforowania Django — *https://docs.djangoproject.com/en/5.0/topics/cache/*.
- Obraz Docker backendu `Memcached` — *https://hub.docker.com/_/memcached*.
- Strona pobierania `Memcached` — *https://memcached.org/downloads*.
- Oficjalna witryna internetowa `Memcached` — *https://memcached.org*.
- Dokumentacja ustawienia `CACHES` frameworka Django — *https://docs.djangoproject.com/en/5.0/ref/settings/#caches*.
- Kod źródłowy `pymemcache` — *https://github.com/pinterest/pymemcache*.
- Backend buforowania oparty na bazie danych Redis — *https://docs.djangoproject.com/en/5.0/topics/cache/#redis*.
- Oficjalny obraz Docker bazy danych Redis — *https://hub.docker.com/_/redis*.
- Opcje pobierania bazy danych Redis — *https://redis.io/download/*.
- Kod źródłowy Django Redisboard — *https://github.com/ionelmc/django-redisboard*.

Rozdział

15 Utworzenie API

W poprzednim rozdziale opracowałeś system rejestracji uczestników oraz zapisywania na kursy. Przygotowałeś widoki przeznaczone do wyświetlania treści kursu oraz wykorzystałeś oferowany przez Django framework buforowania.

W tym rozdziale utworzysz API RESTful dla platformy e-learningu. API to wspólny programowalny interfejs, którego można używać na wielu platformach, takich jak witryny internetowe, aplikacje mobilne, wtyczki itp. Możesz np. opracować API wykorzystywane przez aplikację mobilną dla platformy e-learningu. Jeśli dostarczysz API podmiotom zewnętrznym, będą mogły pobierać z Twojej aplikacji informacje i komunikować się z nią programowo. API pozwala programistom na zautomatyzowanie działań na platformie oraz na integrację Twojej usługi z innymi aplikacjami lub usługami online. W tym rozdziale zbudujesz w pełni funkcjonalne API dla swojej platformy e-learningowej.

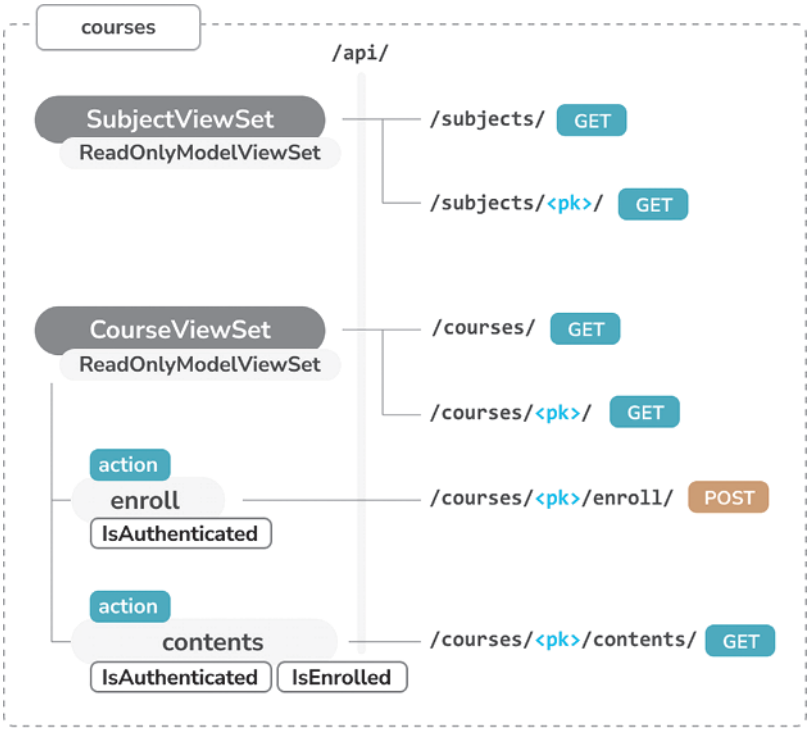
Oto zagadnienia, na których skoncentruję się w tym rozdziale:

- Instalacja frameworka REST Django
- Tworzenie serializatorów dla modeli
- Utworzenie API typu RESTful
- Implementacja w serializatorach pól metod
- Opracowanie zagnieżdżonych serializatorów
- Opracowanie kolekcji `ViewSet` i routerów API
- Zbudowanie niestandardowych widoków API
- Obsługa uwierzytelnienia API
- Obsługa uprawnień dla widoków API
- Utworzenie niestandardowych uprawnień
- Wykorzystanie biblioteki `Requests` do korzystania z API

Przegląd funkcjonalności

Reprezentację widoków, szablonów i funkcjonalności, które zbudujemy w tym rozdziale, przedstawiłem na rysunku 15.1.

W tym rozdziale utworzysz dwa różne zestawy widoków API: `SubjectViewSet` i `Course ViewSet`. Pierwszy z nich będzie zawierał widoki listy i szczegółów dla dziedzin kursów. Drugi będzie zawierał widoki listy i szczegółów dla kursów. W zestawie widoków `Course ↪ViewSet` zaimplementujesz działanie zapisu, pozwalające na zapisywanie uczestników na



Rysunek 15.1. Schemat widoków API i punktów końcowych, które utworzymy w rozdziale 15.

kursy. Dzięki wykorzystaniu uprawnienia `IsAuthenticated` to działanie będzie dostępne tylko dla uwierzytelnionych użytkowników. Aby uzyskać dostęp do zawartości kursu, utworzysz w zestawie widoków `CourseViewSet` działanie `content`. Dostęp do zawartości kursu mogą uzyskać tylko użytkownicy, którzy są uwierzytelnieni i zapisani na dany kurs. Aby zagwarantować dostęp do zawartości wyłącznie użytkownikom zapisanym na kurs, zaimplementujesz niestandardowe uprawnienie `IsEnrolled`.

Czytelnikom nieznającym tematyki punktów końcowych API wyjaśniam, że są to określone lokalizacje w API, pozwalające na akceptację żądań i udzielanie na nie odpowiedzi. Każdy punkt końcowy odpowiada adresowi URL, który może obsługiwać jedną lub więcej metod HTTP, takich jak `GET`, `POST`, `PUT` lub `DELETE`.

Kod źródłowy przykładów z tego rozdziału można znaleźć pod adresem *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter15*.

Wszystkie pakiety Pythona użyte w tym rozdziale są zawarte w pliku *requirements.txt* dołączonym do plików z kodem źródłowym przykładów z tego rozdziału. Aby zainstalować każdy pakiet Pythona, możesz postępować zgodnie z instrukcjami zamieszczonymi w poniższych podrozdziałach. Możesz także zainstalować wszystkie wymagania jednocześnie za pomocą polecenia `pip install -r requirements.txt`.

Utworzenie API typu RESTful

Wprawdzie istnieje kilka sposobów strukturyzacji API, ale zaleca się stosowanie podejścia opartego na architekturze REST.

Nazwa architektury pochodzi od *REpresentational STate transfer* — dosłownie: transfer reprezentatywnych stanów. API typu RESTful opiera się na zasobach. Modele przedstawiają zasoby i metody HTTP, takie jak `GET`, `POST`, `PUT` i `DELETE`, używane — odpowiednio — do pobierania, tworzenia, uaktualniania i usuwania obiektów. W tym kontekście są również stosowane kody odpowiedzi HTTP. Poszczególne kody odpowiedzi HTTP są zwracane w celu wskazania wyniku wykonania żądania HTTP, np. kod typu `2XX` oznacza sukces, `4XX` oznacza błąd itd.

Formaty najczęściej stosowane do wymiany danych w API typu RESTful to JSON i XML. W budowanym projekcie opracujemy API typu RESTful z serializacją JSON. Przygotowane API będzie zapewniać kilka funkcjonalności. Oto one:

- Pobieranie tematów.
- Pobieranie dostępnych kursów.
- Pobieranie treści kursu.
- Zapisywanie się na kurs.

Framework Django pozwala na zbudowanie API od podstaw, przez utworzenie własnych widoków. Istnieje jednak wiele modułów ułatwiających budowanie API dla projektu opracowanych przez podmioty zewnętrzne. Najpopularniejszy to *Django Rest Framework* (**DRF**).

DRF zapewnia kompleksowy zestaw narzędzi do tworzenia dla projektów interfejsów API RESTful. Oto niektóre z najważniejszych komponentów, których użyjemy do zbudowania API w tym rozdziale:

- **Serializatory.** Służą do przekształcania danych do standardowego formatu, zrozumiałego dla innych programów, czyli *deserializacji* danych. W tym celu przeprowadzają konwersję danych na format, który można wykorzystać w programie.
- **Parsery i renderery.** Zajmują się renderowaniem (czyli formatowaniem) zserializowanych danych przed ich zwróceniem w odpowiedzi HTTP. Są również odpowiedzialne za parsowanie przychodzących danych w celu przekształcenia ich do prawidłowego formatu.
- **Widoki API.** Służą do implementowania logiki aplikacji.
- **Adresy URL**. Wykorzystuje się je do zdefiniowania dostępnych punktów końcowych API.
- **Uwierzytelnianie i uprawnienia**. Umożliwiają definiowanie metod uwierzytelniania dla interfejsu API oraz uprawnień wymaganych dla każdego widoku.

Zaczniemy od zainstalowania DRF, a następnie opowiem więcej o wymienionych powyżej komponentach, abyśmy mogli zbudować pierwszy interfejs API.

Instalacja frameworka REST Django

Wyczerpujące informacje na temat frameworka **DRF** można znaleźć na stronie *https://www.django-rest-framework.org/*.

Przejdź do powłoki i zainstaluj framework za pomocą poniższego polecenia.

```
python -m pip install djangorestframework==3.15.1
```

Przeprowadź edycję pliku *settings.py* projektu `educa` i umieść `rest_framework` na liście `INSTALLED_APPS` w celu aktywacji aplikacji, tak jak pokazałem poniżej.

```
INSTALLED_APPS = [
    # …
    'rest_framework',
]
```

Następnie umieść w pliku *settings.py* poniższy fragment kodu.

```
REST_FRAMEWORK = {
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.DjangoModelPermissionsOrAnonReadOnly'
    ]
}
```

Za pomocą opcji `REST_FRAMEWORK` można podać określoną konfigurację dla API. Framework oferuje szeroką gamę ustawień przeznaczonych do konfigurowania domyślnego zachowania. Opcja `DEFAULT_PERMISSION_CLASSES` definiuje domyślne uprawnienia odczytu, tworzenia, uaktualniania i usuwania obiektów. Istnieje możliwość wskazania `DjangoModel` ↪`PermissionsOrAnonReadOnly` jako jedynej domyślnej klasy uprawnień. Działanie tej klasy opiera się na systemie uprawnień Django w zakresie zezwalania użytkownikom na tworzenie, uaktualnianie lub usuwanie obiektów i jednocześnie umożliwia użytkownikom anonimowym jedynie odczyt. Więcej informacji na temat uprawnień przedstawię dalej w tym rozdziale, w punkcie „Określenie uprawnień do widoków”.

Pełna lista ustawień dostępnych dla frameworka REST znajduje się na stronie *https://www.django-rest-framework.org/api-guide/settings/*.

Definiowanie serializatorów

Po skonfigurowaniu frameworka REST trzeba określić sposób serializacji danych. Dane wyjściowe mogą być serializowane na wskazany format, a dane wejściowe będą deserializowane w celu ich przetworzenia. Omawiany framework udostępnia wymienione poniżej klasy przeznaczone do zdefiniowania serializacji dla pojedynczych obiektów.

- `Serializer`. Zapewnia obsługę serializacji dla egzemplarzy zwykłych klas Pythona.
- `ModelSerializer`. Zapewnia obsługę serializacji dla egzemplarzy klas modelu.
- `HyperlinkedModelSerializer`. Tak samo jak w przypadku klasy `ModelSerializer`, ale reprezentuje relacje pomiędzy obiektami za pomocą linków zamiast kluczy głównych.

Przystępujemy zatem do stworzenia pierwszego serializatora. W katalogu aplikacji `courses` utwórz taką strukturę plików i katalogów:

```
api/
    __init__.py
    serializers.py
```

Cała funkcjonalność API zostanie umieszczona w katalogu *api*, co pozwoli zachować elegancką organizację projektu. Przeprowadź edycję pliku *api/serializers.py* i umieść w nim poniższy fragment kodu.

```
from rest_framework import serializers
from courses.models import Subject


class SubjectSerializer(serializers.ModelSerializer):
    class Meta:
        model = Subject
        fields = ['id', 'title', 'slug']
```

Jest to serializator dla modelu `Subject`. W podobny sposób definiujemy serializację dla klas Django `Form` i `ModelForm`. Klasa `Meta` pozwala na wskazanie modelu przeznaczonego do serializacji, a także pól, które mają być uwzględnione w trakcie tego procesu. Jeżeli nie podasz atrybutu `fields`, serializowane będą wszystkie pola.

Teraz wypróbujemy przygotowany serializator. Przejdź do powłoki za pomocą polecenia:

```
python manage.py shell
```

Uruchom następujący fragment kodu.

```
>>> from courses.models import Subject
>>> from courses.api.serializers import SubjectSerializer
>>> subject = Subject.objects.latest('id')
>>> serializer = SubjectSerializer(subject)
>>> serializer.data
{'id': 4, 'title': 'Informatyka', 'slug': 'informatyka'}
```

W tym przykładzie pobieramy obiekt `Subject`, tworzymy egzemplarz klasy `Subject` ↪`Serializer`, a następnie uzyskujemy dostęp do serializowanych danych. Jak można zauważyć, dane modelu zostały przekształcone w natywny typ danych Pythona.

Więcej informacji na temat serializatorów można znaleźć na stronie *https://www.django-rest-framework.org/api-guide/serializers/*.

Klasy parserów i renderowania formatów

Serializowane dane, zanim zostaną zwrócone w odpowiedzi HTTP, muszą być wyrenderowane do określonego formatu. Podobnie, po otrzymaniu żądania HTTP konieczne jest sparsowanie danych przychodzących i ich deserializacja, zanim będą mogły być użyte do właściwej operacji. Framework DRF oferuje parsery i mechanizmy renderowania formatów odpowiedzialne za wymienione zadania.

Zobaczmy, jak można parsować przychodzące dane. W powłoce Pythona uruchom następujący kod.

```
>>> from io import BytesIO
>>> from rest_framework.parsers import JSONParser
>>> data = b'{"id":4,"title":"Informatyka","slug":"informatyka"}'
>>> JSONParser().parse(BytesIO(data))
{'id': 4, 'title': 'Informatyka', 'slug': 'informatyka'}
```

Dla danych wejściowych w postaci ciągu tekstowego JSON można użyć klasy `JSONParser` dostarczanej przez framework DRF i skonwertować te dane na postać obiektu Pythona.

Framework DRF zawiera również klasę `Renderer`, pozwalającą na formatowanie odpowiedzi API. Framework ustala, która klasa potomna klasy `Renderer` powinna być użyta podczas negocjacji treści. Odbywa się to przez analizę wartości nagłówka `Accept` w celu określenia oczekiwanego typu treści w odpowiedzi. Opcjonalnie klasa do użycia może być ustalona na podstawie formatu przyrostka adresu URL. Na przykład adres URL `http://127.0.0.1:8000/api/data.json` może być punktem końcowym, który uruchamia obiekt `JSONRenderer` w celu zwrócenia odpowiedzi JSON.

Powróć do powłoki i aby wygenerować obiekt serializatora na podstawie wcześniejszego przykładu, wykonaj poniższy fragment kodu.

```
>>> from rest_framework.renderers import JSONRenderer
>>> JSONRenderer().render(serializer.data)
```

Wyświetli się taki wynik:

```
b'{"id":4,"title":"Informatyka","slug":"informatyka"}'
```

Wykorzystaliśmy klasę `JSONRenderer` do wyrenderowania serializowanych danych na format JSON. Domyślnie framework DRF używa do renderowania dwóch różnych klas: `JSONRenderer` i `BrowsableAPIRenderer`. Druga z wymienionych oferuje interfejs webowy pozwalający na łatwe przeglądanie API. Domyślne klasy renderowania można zmienić w ustawieniach `REST_FRAMEWORK` za pomocą opcji `DEFAULT_RENDERER_CLASSES`.

Więcej informacji na temat parserów i klas renderowania określonych formatów znajdziesz na stronach — odpowiednio — *https://www.django-rest-framework.org/api-guide/renderers/* i *https://www.django-rest-framework.org/api-guide/parsers/*.

W kolejnym kroku nauczysz się budowania widoków API i używania serializatorów w widokach.

Utworzenie widoków listy i szczegółowego

Framework DRF jest dostarczany wraz z zestawem generycznych widoków i domieszek gotowych do użycia w budowanych widokach API. Z widoków generycznych korzystałeś w całej książce, począwszy od rozdziału 2., „Usprawnianie bloga i dodanie funkcjonalności społecznościowych”, a o domieszkach dowiedziałeś się w rozdziale 13., „Tworzenie systemu zarządzania treścią”.

Zapewniają one funkcjonalność pobierania, tworzenia, uaktualniania i usuwania obiektów modelu. Informacje o wszystkich generycznych widokach i domieszkach oferowanych przez framework DRF znajdziesz na stronie *http://www.django-rest-framework.org/api-guide/generic-views/*.

Przystępujemy do utworzenia widoków listy i szczegółowego pozwalających na pobieranie obiektów `Subject`. Utwórz nowy plik w katalogu *courses/api* i nadaj mu nazwę *views.py*. Następnie umieść w nim poniższy fragment kodu.

```
from rest_framework import generics
from courses.api.serializers import SubjectSerializer
from courses.models import Subject

class SubjectListView(generics.ListAPIView):
    queryset = Subject.objects.all()
    serializer_class = SubjectSerializer

class SubjectDetailView(generics.RetrieveAPIView):
    queryset = Subject.objects.all()
    serializer_class = SubjectSerializer
```

W tym kodzie wykorzystaliśmy dostarczane przez framework DRF generyczne widoki `ListAPIView` i `RetrieveAPIView`. Oba widoki mają wymienione poniżej atrybuty.

- `queryset`. Bazowe zapytanie `QuerySet` mające na celu pobranie obiektów.
- `serializer_class`. Klasa przeznaczona do serializacji obiektów.

Przechodzimy do dodania wzorców adresów URL dla nowych widoków. W katalogu *courses/api* utwórz nowy plik o nazwie *urls.py* i umieść w nim następujący fragment kodu.

```
from django.urls import path
from . import views


app_name = 'courses'

urlpatterns = [
    path(
        'subjects/',
        views.SubjectListView.as_view(),
        name='subject_list'
    ),
    path(
        'subjects/<pk>/',
        views.SubjectDetailView.as_view(),
        name='subject_detail'
    ),
]
```

Aby pobrać obiekt z podanym kluczem głównym modelu `Subject`, którym jest pole `id`, należy dołączyć do wzorca adresu URL dla widoku `SubjectDetailView` parametr URL `pk`. Przeprowadź edycję głównego pliku *urls.py* projektu `educa` i umieść w nim poniższy wzorzec adresu URL dla budowanego API.

```
urlpatterns = [
    # …
    path('api/', include('courses.api.urls', namespace='api')),
]
```

Dla adresów URL budowanego API użyliśmy przestrzeni nazw `api`. Początkowe punkty końcowe interfejsu API są teraz gotowe do użycia.

Wykorzystanie API

Dzięki temu, że za pośrednictwem adresów URL udostępniliśmy widoki, stworzyliśmy pierwsze punkty końcowe API. Spróbujmy teraz wykorzystać utworzone przed chwilą API. Uruchom serwer programistyczny za pomocą poniższego polecenia.

```
python manage.py runserver
```

Do wykorzystania interfejsu API użyjemy programu `curl`. `curl` to narzędzie wiersza poleceń, które umożliwia przesyłanie danych do serwera i odbieranie ich z niego. Jeśli używasz Linuksa, macOS lub Windows 10/11, najprawdopodobniej `curl` jest zainstalowany w Twoim systemie. Możesz jednak pobrać go pod adresem *https://curl.se/download.html*.

Przejdź do powłoki i pobierz za pomocą `curl` treść znajdującą się pod adresem `http://127.0.0.1:8000/api/subjects`.

```
curl http://127.0.0.1:8000/api/subjects/
```

Powinieneś otrzymać odpowiedź zawierającą dane wyjściowe podobne do następujących.

```
[
    {
        "id":1,
        "title":"Matematyka",
        "slug":"matematyka"
    },
    {
        "id":2,
        "title":"Muzyka",
        "slug":"muzyka"
    },
    {
        "id":3,
        "title":"Fizyka",
        "slug":"fizyka"
    },
    {
        "id":4,
        "title":"Informatyka",
        "slug":"informatyka"
    }
]
```

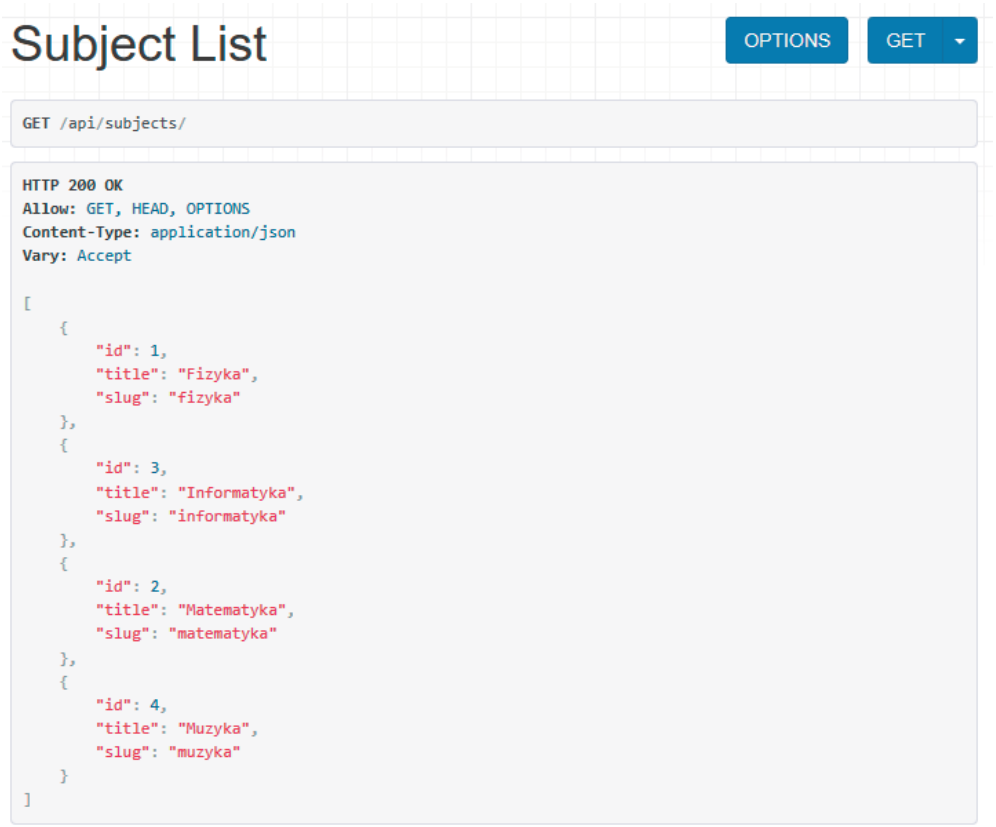
Aby uzyskać bardziej czytelną odpowiedź JSON, z estetycznymi wcięciami, możesz skorzystać z polecenia `curl` razem z narzędziem `json_pp`, tak jak poniżej.

```
curl http://127.0.0.1:8000/api/subjects/ | json_pp
```

Ta odpowiedź HTTP zawiera listę obiektów `Subject` w formacie JSON.

Do wykonywania własnych żądań HTTP można zamiast `curl` użyć innego dowolnego narzędzia, np. rozszerzenia przeglądarki internetowej o nazwie Postman, które znajdziesz w witrynie *https://www.getpostman.com/*.

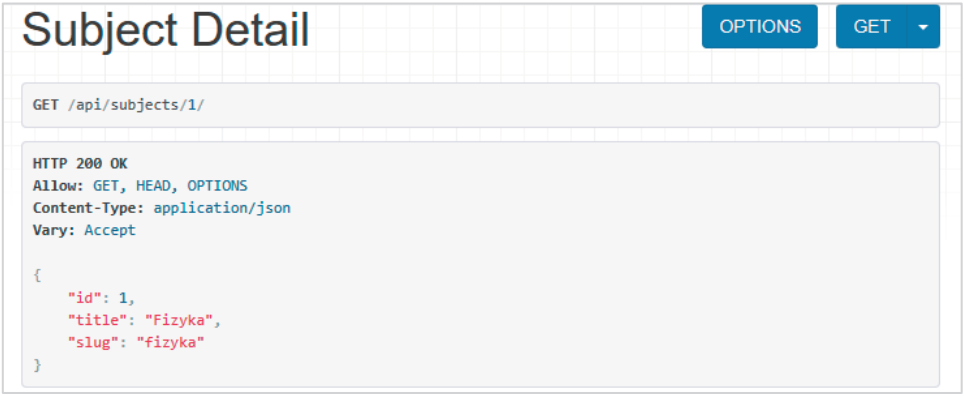
W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/api/subjects/`. Powinieneś zobaczyć łatwe do przeglądania API wygenerowane za pomocą frameworka DRF (rysunek 15.2).



Rysunek 15.2. Strona listy tematów uzyskana za pomocą API frameworka REST

Interfejs HTML został dostarczony przez generator `BrowsableAPIRenderer`. Wyświetla on nagłówki wyniku i treść oraz pozwala na wykonywanie żądań. Ponadto można uzyskać dostęp do widoku szczegółowego API dla obiektu `Subject` przez umieszczenie w adresie URL jego identyfikatora.

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/api/subjects/1/`. Powinieneś zobaczyć pojedynczy obiekt `Subject` wygenerowany w formacie JSON (rysunek 15.3).



Rysunek 15.3. Strona szczegółów tematu uzyskana za pomocą API frameworka REST

To jest odpowiedź dla widoku `SubjectDetailView`. Zobaczmy, jak można wzbogacić treść zwracaną dla każdego tematu. W następnym podrozdziale zajmiemy się rozszerzeniem serializatorów o dodatkowe pola i metody.

Rozszerzanie serializatorów

Nauczyłeś się, jak serializować obiekty modelu. Często jednak powstaje potrzeba wzbogacenia odpowiedzi o dodatkowe istotne dane lub pola obliczeniowe. Przyjrzyjmy się kilku możliwościom rozszerzania serializatorów.

Dodawanie pól do serializatorów

Przystępujemy do zmodyfikowania widoków tematów w taki sposób, aby dla każdego z nich uwzględnić liczbę dostępnych kursów. W celu dodania dla każdego przedmiotu liczby powiązanych kursów skorzystamy z funkcji agregacji frameworka Django.

W pliku *api/views.py* aplikacji `courses` umieść poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
from django.db.models import Count
# ...

class SubjectListView(generics.ListAPIView):
    queryset = Subject.objects.annotate(total_courses=Count('courses'))
    serializer_class = SubjectSerializer

class SubjectDetailView(generics.RetrieveAPIView):
    queryset = Subject.objects.annotate(total_courses=Count('courses'))
    serializer_class = SubjectSerializer
```

Teraz dla widoków `SubjectListView` i `SubjectDetailView` wykorzystujemy kolekcję `QuerySet`, która do wyświetlania powiązanej liczby kursów używa funkcji agregacji `Count`.

W pliku *api/serializers.py* aplikacji `courses` umieść poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
from rest_framework import serializers
from courses.models import Subject

class SubjectSerializer(serializers.ModelSerializer):
    total_courses = serializers.IntegerField()

    class Meta:
        model = Subject
        fields = ['id', 'title', 'slug', 'total_courses']
```

Do klasy `SubjectSerializer` dodałeś pole `total_courses`. Jest to pole `IntegerField`, używane do reprezentowania liczb całkowitych. To pole automatycznie pobierze swoją wartość z atrybutu `total_courses` serializowanego obiektu. Dzięki wykorzystaniu funkcji `annotate()` dodałeś do wynikowych obiektów `QuerySet` atrybut `total_courses`.

Otwórz w przeglądarce stronę `http://127.0.0.1:8000/api/subjects/1/`. Zserializowany obiekt JSON zawiera teraz atrybut `total_courses`, co pokazałem na rysunku 15.4.

```
HTTP 200 OK
Allow: GET, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

{
    "id": 1,
    "title": "Fizyka",
    "slug": "fizyka",
    "total_courses": 0
}
```

Rysunek 15.4. Strona szczegółów tematu włącznie z atrybutem total_courses

Do widoków listy tematów i szczegółowego pomyślnie dodałeś atrybut `total_courses`. Przyjrzyjmy się teraz, w jaki sposób dodawać atrybuty za pomocą niestandardowych metod serializatorów.

Implementacja w serializatorach pól obliczanych za pomocą metod

Framework DRF udostępnia klasę `SerializerMethodField`, pozwalającą na implementowanie pól tylko do odczytu, które obliczają swoją wartość przez wywołanie metody klasy serializatora. Może to być szczególnie przydatne, gdy do zserializowanego obiektu chcesz dołączyć niestandardowo sformatowane dane lub wykonać złożone obliczenia na liczbach niebędących częścią egzemplarzy modelu.

W tym punkcie stworzymy metodę, która dla danego tematu serializuje trzy najpopularniejsze kursy. Kursy uszeregujemy według liczby zapisanych na nie uczestników. Przeprowadź edycję pliku *api/serializers.py* aplikacji `courses` i umieść w nim poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
from django.db.models import Count
from rest_framework import serializers
from courses.models import Subject


class SubjectSerializer(serializers.ModelSerializer):
    total_courses = serializers.IntegerField()
    popular_courses = serializers.SerializerMethodField()

    def get_popular_courses(self, obj):
        courses = obj.courses.annotate(
            total_students=Count('students')
        ).order_by('total_students')[:3]
        return [
            f'{c.title} ({c.total_students})' for c in courses
        ]

    class Meta:
        model = Subject
        fields = [
            'id',
            'title',
            'slug',
            'total_courses',
            'popular_courses'
        ]
```

W nowym kodzie dodaliśmy do klasy `SubjectSerializer` metodę serializatora `popular_courses`. Pole oblicza swoją wartość za pomocą metody `get_popular_courses()`. Nazwę metody serializatora do wywołania można podać za pomocą argumentu `method_field` w klasie `SerializerMethodField`. Jeśli nie podano tego argumentu, domyślnie metoda przyjmuje nazwę `get_<nazwa_pola>`.

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/api/subjects/3/`. Zserializowany obiekt JSON zawiera teraz atrybut `popular_courses`, co pokazałem na rysunku 15.5.

```
HTTP 200 OK
Allow: GET, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

{
    "id": 3,
    "title": "Informatyka",
    "slug": "informatyka",
    "total_courses": 2,
    "popular_courses": [
        "Python dla początkujących (0)",
        "Kurs Django (3)"
    ]
}
```

Rysunek 15.5. Strona szczegółów tematu włącznie z atrybutem popular_courses

Pomyślnie zaimplementowałeś obliczane za pomocą metody pole `SerializerMethodField`. Zauważ, że teraz dla każdego wyniku zwróconego przez `SubjectListView` jest generowane dodatkowe zapytanie SQL. W następnym punkcie pokażę, jak można kontrolować liczbę zwracanych wyników przez dodanie do widoku `SubjectListView` mechanizmu stronicowania.

Dodawanie stronicowania do widoków

Framework DRF zawiera wbudowany mechanizm stronicowania pozwalający na zarządzanie liczbą obiektów przesyłanych w odpowiedziach API. Po jakimś czasie liczba tematów i kursów w witrynie może osiągnąć znaczne rozmiary. Stronicowanie może być szczególnie przydatne do poprawy wydajności i komfortu użytkownika podczas pracy z dużymi zbiorami danych.

Zaktualizujmy widok `SubjectListView`, aby zawierał mechanizm stronicowania. Zaczniemy od zdefiniowania klasy implementującej stronicowanie.

Utwórz w katalogu *courses/api* nowy plik i nadaj mu nazwę *pagination.py*. Następnie umieść w nim poniższy fragment kodu.

```
from rest_framework.pagination import PageNumberPagination


class StandardPagination(PageNumberPagination):
    page_size = 10
    page_size_query_param = 'page_size'
    max_page_size = 50
```

Powyższa klasa dziedziczy po klasie `PageNumberPagination`. Klasa ta zapewnia obsługę stronicowania opartą na numerach stron. Ustawiamy atrybuty:

- `page_size`. Określa domyślny rozmiar strony (liczbę elementów zwracanych na stronie), gdy w żądaniu nie podano rozmiaru strony.
- `page_size_query_params`. Definiuje nazwę parametru zapytania, który ma być używany dla rozmiaru strony.
- `max_page_size`. Wskazuje maksymalny dozwolony rozmiar żądanej strony.

Przeprowadź edycję pliku *api/views.py* aplikacji `courses` i umieść w nim poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
from django.db.models import Count
from rest_framework import generics
from courses.models import Subject
from courses.api.pagination import StandardPagination
from courses.api.serializers import SubjectSerializer


class SubjectListView(generics.ListAPIView):
    queryset = Subject.objects.annotate(total_courses=Count('courses'))
    serializer_class = SubjectSerializer
    pagination_class = StandardPagination
# ...
```

Możesz teraz stronicować obiekty zwracane przez widok `SubjectListView`. W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/api/subjects/`. Jak można zauważyć, ze względu na stronicowanie struktura JSON zwracana przez widok jest teraz inna. Zobaczysz taką strukturę:

```
{
    "count": 4,
    "next": null,
    "previous": null,
    "results": [
        {
            "id": 1,
            "title": "Matematyka",
             ...
        },         ...
    ]
}
```

Zwracany tekst w formacie JSON zawiera teraz elementy:

- `count`. Całkowita liczba wyników.
- `next`. Adres URL do pobrania następnej strony. Wartość `null`, gdy nie ma kolejnej strony.
- `previous`. Adres URL do pobrania poprzedniej strony. Wartość `null`, gdy nie ma poprzedniej strony.
- `results`. Lista zawierająca zserializowane obiekty zwrócone na tej stronie.

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/api/subjects/?page_size=2&page=1`. Spowoduje to stronicowanie wyników po dwa elementy na stronę i pobranie pierwszej strony wyników, jak pokazałem na rysunku 15.6.

Zaimplementowaliśmy stronicowanie oparte na numerze strony, ale framework DRF zapewnia również klasę do implementacji stronicowania na podstawie liczby wyników, przesunięcia i kursorów. Więcej informacji na temat stronicowania można znaleźć na stronie *https://www.django-rest-framework.org/api-guide/pagination/*.

Stworzyłeś punkty końcowe API dla widoków do prezentowania tematów. W następnym podrozdziale dodamy do interfejsu API punkty końcowe do prezentowania kursów.

Tworzenie serializatora kursów

Przechodzimy teraz do przygotowania serializatora dla modelu `Course`. Przeprowadź edycję pliku *api/serializers.py* aplikacji `courses` i umieść w nim poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
# …
from courses.models import Course, Subject
class CourseSerializer(serializers.ModelSerializer):
    class Meta:
        model = Course
        fields = [
            'id',
            'subject',
            'title',
            'slug',
            'overview',
            'created',
            'owner',
            'modules'
        ]
```

```
GET /api/subjects/?page_size=2&page=1
```

```
HTTP 200 OK
Allow: GET, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

{
    "count": 4,
    "next": "http://127.0.0.1:8000/api/subjects/?page=2&page_size=2",
    "previous": null,
    "results": [
        {
            "id": 1,
            "title": "Fizyka",
            "slug": "fizyka",
            "total_courses": 1,
            "popular_courses": [
                "Dynamika i prawa Newtona (0)"
            ]
        },
        {
            "id": 3,
            "title": "Informatyka",
            "slug": "informatyka",
            "total_courses": 2,
            "popular_courses": [
                "Python dla początkujących (0)",
                "Kurs Django (3)"
            ]
        }
    ]
}
```

Rysunek 15.6. Pierwsza strona wyników dla stronicowania listy tematów, gdy rozmiar strony wynosi 2

Zobaczmy, w jaki sposób odbywa się serializacja obiektu `Course`. Przejdź do powłoki i wydaj poniższe polecenie.

```
python manage.py shell
```

Uruchom następujący fragment kodu.

```
>>> from rest_framework.renderers import JSONRenderer
>>> from courses.models import Course
>>> from courses.api.serializers import CourseSerializer
>>> course = Course.objects.latest('id')
>>> serializer = CourseSerializer(course)
>>> JSONRenderer().render(serializer.data)
```

Otrzymasz obiekt JSON wraz z kolumnami wymienionymi w `CourseSerializer`. Możesz zobaczyć, że powiązane obiekty menedżera `modules` są serializowane jako lista kluczy podstawowych. Oto przykład:

```
"modules": [6, 7, 9, 10]
```

Są to identyfikatory powiązanych obiektów `Module`. W następnym punkcie zapoznasz się z różnymi sposobami serializacji powiązanych obiektów.

Serializacja relacji

Framework DRF zawiera różne typy powiązanych pól do reprezentowania relacji modelu. Można je zastosować do relacji `ForeignKey`, `ManyToManyField` i `OneToOneField`, a także do generycznych relacji modelu.

W celu zmiany sposobu serializacji powiązanych obiektów `Module` zamierzamy wykorzystać pole `StringRelatedField`. Klasa `StringRelatedField` reprezentuje powiązany obiekt za pomocą metody `__str__()`.

Przeprowadź edycję pliku *api/serializers.py* aplikacji `courses` i umieść w nim poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
# ...

class CourseSerializer(serializers.ModelSerializer):
    modules = serializers.StringRelatedField(many=True, read_only=True)

    class Meta:
        # ...
```

W nowym kodzie zdefiniowałeś pole `modules`, które zapewnia serializację powiązanych obiektów `Module`. Ustawiliśmy `many=True`, wskazując tym samym na serializację wielu obiektów. Parametr `read_only` określa, że kolumna jest tylko do odczytu i nie powinna być uwzględniana w jakichkolwiek danych wejściowych podczas tworzenia lub uaktualniania obiektów.

Przejdź do powłoki i ponownie utwórz egzemplarz `CourseSerializer`. Za pomocą obiektu klasy `JSONRenderer` wyrenderuj atrybut `data` serializatora. Tym razem wymienione moduły są serializowane za pomocą metody `__str__()` w następujący sposób.

```
"modules": ["0. Instalacja Django", "1. Konfiguracja Django"]
```

Zwróć uwagę, że framework DRF nie optymalizuje kolekcji `QuerySet`. Podczas serializacji listy kursów dla każdego wyniku kursu zostanie wygenerowane zapytanie SQL w celu pobrania powiązanych obiektów `Module`. By zmniejszyć liczbę dodatkowych zapytań SQL, można skorzystać w zapytaniu `QuerySet` z funkcji `prefetch_related()`, na przykład `Course.objects. prefetch_related('modules')`. Omówię to w dalszej części tego rozdziału, w podrozdziale „Utworzenie kolekcji ViewSet i routerów”.

Więcej informacji na temat relacji serializatorów można znaleźć na stronie *https://www.django-rest-framework.org/api-guide/serializers/.*

W kolejnym punkcie pokażę, jak zdefiniować serializację powiązanych obiektów za pomocą zagnieżdżonego serializatora.

Opracowanie zagnieżdżonych serializatorów

Aby dołączyć więcej informacji o poszczególnych modułach, trzeba serializować obiekty `Module` i je zagnieżdżać. Zmodyfikuj wcześniejszy kod w pliku *api/serializers.py* aplikacji `courses`, aby przedstawiał się tak:

```
from django.db.models import Count
from rest_framework import serializers
from courses.models import Course, Module, Subject


class ModuleSerializer(serializers.ModelSerializer):
    class Meta:
        model = Module
        fields = ['order', 'title', 'description']

class CourseSerializer(serializers.ModelSerializer):
    modules = ModuleSerializer(many=True, read_only=True)

    class Meta:
        # …
```

W nowym kodzie zdefiniowałeś serializator `ModuleSerializer`, by zapewnić serializację modelu `Module`. Następnie, aby zagnieździć serializator `ModuleSerializer`, dodajemy do serializatora `CourseSerializer` atrybut `modules`. Zachowujemy ustawienie `many=True`, aby wskazać, że serializujemy wiele obiektów, i `read_only=True`, aby zachować tryb tylko do odczytu tego pliku.

Przejdź do powłoki i ponownie utwórz egzemplarz serializatora `CourseSerializer`. Za pomocą obiektu klasy `JSONRenderer` wyrenderuj atrybut `data` serializatora. Tym razem wskazane moduły będą serializowane wraz z zagnieżdżonym serializatorem `Module` ↪`Serializer`, tak jak przedstawiłem poniżej.

```
"modules": [
    {
        "order": 0,
        "title": "Ogólne omówienie frameworka",
        "description": "Krótkie omówienie frameworka sieciowego."
    },
    {
        "order": 1,
        "title": "Konfiguracja Django",
        "description": "Proces instalacji Django."
    },
    ...
]
```

Utworzenie kolekcji ViewSet i routerów

Klasa `ViewSet` pozwala na zdefiniowanie interakcji API i umożliwia frameworkowi DRF dynamiczne tworzenie adresów URL za pomocą obiektu `Router`.

Przy użyciu kolekcji `ViewSet` można uniknąć powtarzania tej samej logiki w wielu widokach.

Kolekcja `ViewSet` zawiera metody dla tych typowych operacji:

- tworzenie: metoda `create()`,
- pobieranie: metody `list()` i `retrieve()`,
- uaktualnianie: metody update() i `partial_update()`,
- usuwanie: metoda `destroy()`.

Przystępujemy do utworzenia kolekcji `ViewSet` dla modelu `Course`. Przeprowadź edycję pliku *api/views.py* aplikacji `courses` i umieść w nim poniższy fragment kodu wyróżniony pogrubioną czcionką.

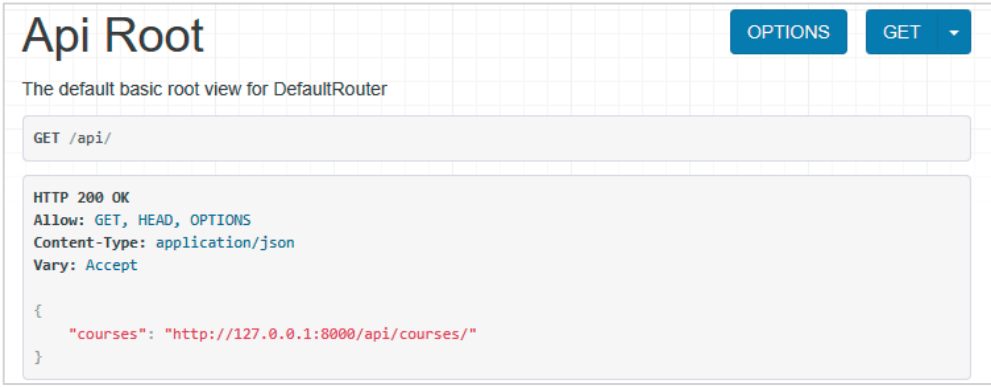
```
from django.db.models import Count
from rest_framework import generics
from rest_framework import viewsets
from courses.api.pagination import StandardPagination
from courses.api.serializers import CourseSerializer, SubjectSerializer
from courses.models import Course, Subject


class CourseViewSet(viewsets.ReadOnlyModelViewSet):
    queryset = Course.objects.prefetch_related('modules')
    serializer_class = CourseSerializer
    pagination_class = StandardPagination
```

Nowa klasa `CourseViewSet` dziedziczy po klasie `ReadOnlyModelViewSet`, udostępniającej operacje tylko do odczytu `list()` i `retrieve()` — zarówno w przypadku wyświetlania listy obiektów, jak i pobierania pojedynczego obiektu. Do pobierania obiektów określiliśmy bazowy obiekt `QuerySet`. Do wydajnego pobierania powiązanych obiektów `Module` użyliśmy wywołania `prefetch_related('modules')`. Pozwoli to uniknąć dodatkowych zapytań SQL podczas serializacji zagnieżdżonych modułów dla każdego kursu. W tej klasie zdefiniowaliśmy również klasę serializatora i klasę obsługi stronicowania, które mają być wykorzystywane dla tego obiektu `ViewSet`. Przeprowadź edycję pliku *api/urls.py* i utwórz router dla obiektu `ViewSet`, tak jak przedstawiłem poniżej.

```
from django.urls import include, path
from rest_framework import routers
from . import views


app_name = 'courses'


router = routers.DefaultRouter()
router.register('courses', views.CourseViewSet)

urlpatterns = [
    # …
    path('', include(router.urls)),
]
```

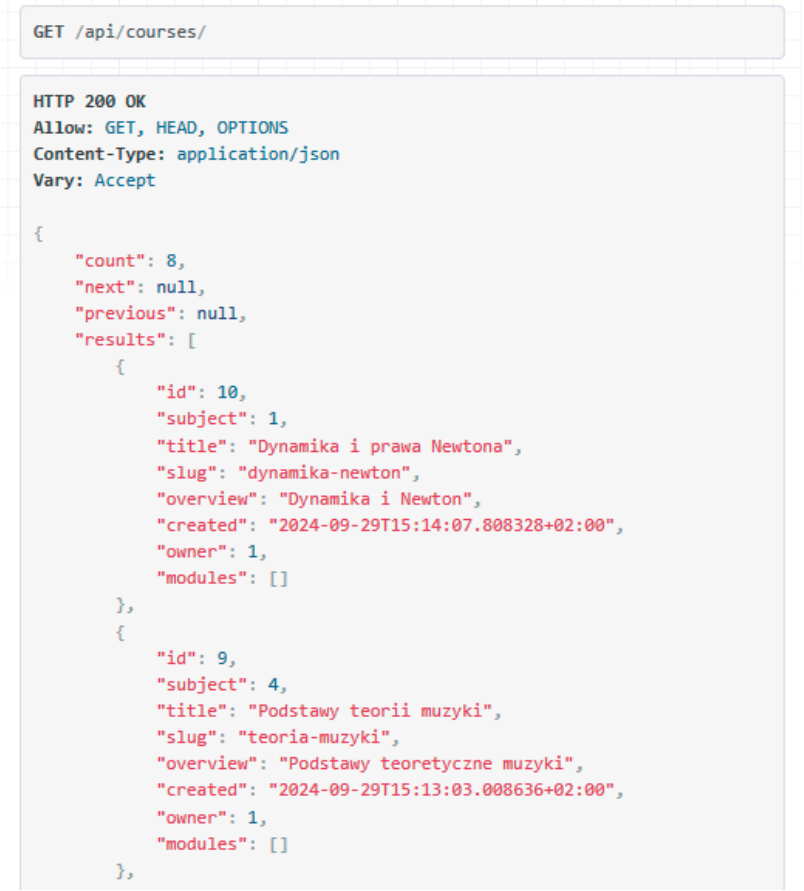
Utworzyłeś obiekt `DefaultRouter` i zarejestrowałeś obiekt `CourseViewSet` z prefiksem `courses`. Ten router staje się odpowiedzialny za automatyczne generowanie adresów URL dla kolekcji `ViewSet`.

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/api/`. Powinieneś zobaczyć, że router wyświetla listę wszystkich obiektów `ViewSet` w bazowym adresie URL, co pokazałem na rysunku 15.7.



Rysunek 15.7. Główna strona API frameworka REST

W celu pobrania listy kursów przejdź pod adres `http://127.0.0.1:8000/api/courses/`. Pobraną listę kursów pokazałem na rysunku 15.8.



Rysunek 15.8. Strona z listą kursów interfejsu API REST

Przekształćmy widoki `SubjectListView` i `SubjectDetailView` na jeden obiekt `ViewSet`. W pliku *api/views.py* usuń lub oznacz komentarzem klasy `SubjectListView` i `SubjectDetailView`. Następnie dodaj poniższe wiersze zaznaczone pogrubioną czcionką:

```
# ...
class SubjectViewSet(viewsets.ReadOnlyModelViewSet):
    queryset = Subject.objects.annotate(total_courses=Count('courses'))
    serializer_class = SubjectSerializer
    pagination_class = StandardPagination
```

W pliku *api/urls.py* oznacz komentarzem poniższe wzorce adresów URL, ponieważ nie będą już potrzebne:

```
# path(
#    subjects/',
#    views.SubjectListView.as_view(),
#    name=subject_list'
# ),
# path(
#    subjects/<pk>/ ',
#     views.SubjectDetailView.as_view(),
#     name='subject_detail'
#),
```

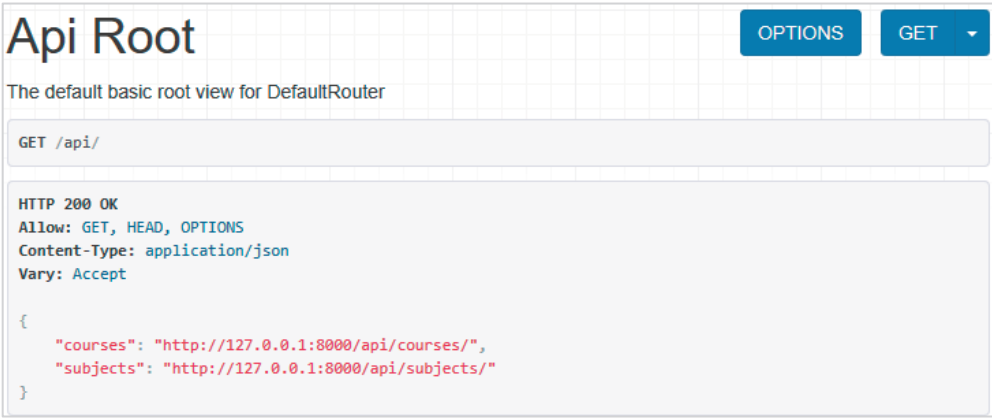
Następnie dodaj poniższe wiersze zaznaczone pogrubioną czcionką:

```
from django.urls import include, path
from rest_framework import routers
from . import views

app_name = 'courses'

router = routers.DefaultRouter()
router.register('courses', views.CourseViewSet)
router.register('subjects', views.SubjectViewSet)

urlpatterns = [
    path('', include(router.urls)),
]
```

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/api`. Zobaczysz, że router zawiera teraz adresy URL zarówno dla obiektów `ViewSet` `"courses"`, jak i `"subjects"`, co pokazałem na rysunku 15.9.

Więcej informacji na temat obiektów `ViewSet` możesz znaleźć na stronie *https://www.django-rest-framework.org/api-guide/viewsets/*. Natomiast więcej informacji o routerach znajdziesz na stronie *http://www.django-rest-framework.org/api-guide/routers/*.

Generyczne widoki interfejsu API i obiekty `ViewSet` są bardzo przydatne do tworzenia interfejsów API REST na podstawie modeli i serializatorów. Czasami jednak może być konieczne zaimplementowanie własnych widoków z niestandardową logiką. Przyjrzyjmy się, w jaki sposób można tworzyć niestandardowe widoki API.



Rysunek 15.9. Główna strona API frameworka REST

Tworzenie własnych widoków API

Framework DRF udostępnia klasę `APIView`, która buduje funkcjonalność API na podstawie klasy `View` frameworka Django. Różnica między klasami `APIView` i `View` polega na tym, że pierwsza z wymienionych korzysta z obiektów `Request` i `Response` frameworka DRF oraz zapewnia obsługę wyjątków `APIException`, zwracających właściwe odpowiedzi HTTP. Ponadto ma wbudowany system uwierzytelniania i autoryzacji pozwalający na zarządzanie dostępem do widoków.

Przystępujemy teraz do utworzenia widoku umożliwiającego użytkownikowi zapisanie się na kursy. Przeprowadź edycję pliku *api/views.py* aplikacji `courses` i umieść w nim poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
from django.db.models import Count
from django.shortcuts import get_object_or_404
from rest_framework import generics
from rest_framework import viewsets
from rest_framework.response import Response
from rest_framework.views import APIView
from courses.api.pagination import StandardPagination
from courses.api.serializers import CourseSerializer, SubjectSerializer
from courses.models import Course, Subject

# ...

class CourseEnrollView(APIView):
    def post(self, request, pk, format=None):
        course = get_object_or_404(Course, pk=pk)
        course.students.add(request.user)
        return Response({'enrolled': True})
```

Widok `CourseEnrollView` obsługuje zapisanie się użytkownika na kursy. Oto w jaki sposób działa powyższy kod:

1. Utworzenie niestandardowego widoku będącego podklasą klasy `APIView`.
2. Zdefiniowanie metody `post()` dla operacji `POST`. W przypadku tego widoku nie będzie dozwolona żadna inna metoda HTTP.
3. Oczekiwane jest użycie parametru `pk` adresu URL zawierającego identyfikator kursu. Pobieramy kurs na podstawie wartości wymienionego parametru, a jeśli nie będzie podany, zgłaszamy wyjątek `404`.
4. Dodanie bieżącego użytkownika do relacji `students` typu „wiele do wielu" obiektu `Course` i zwrócenie odpowiedzi wskazującej na sukces.

Przeprowadź edycję pliku *api/urls.py* i dodaj poniższy wzorzec adresów URL dla widoku `CourseEnrollView`.

```
path(
    'courses/<pk>/enroll/',
    views.CourseEnrollView.as_view(),
    name='course_enroll'
),
```

Teoretycznie możemy teraz wykonywać żądania `POST` w celu zapisania bieżącego użytkownika na kurs. Jednak konieczne jest zapewnienie możliwości identyfikacji użytkownika i uniemożliwienie nieuwierzytelnionym użytkownikom uzyskania dostępu do przygotowanego widoku. Zobaczmy więc, jak w API działa system uwierzytelniania i uprawnień.

Obsługa uwierzytelniania

Framework DRF oferuje klasy uwierzytelniania pozwalające na identyfikację użytkownika wykonującego żądanie. Jeżeli uwierzytelnianie kończy się powodzeniem, framework umieszcza w `request.user` obiekt `User`, reprezentujący uwierzytelnionego użytkownika. W przypadku braku uwierzytelnionego użytkownika następuje umieszczenie egzemplarza Django o nazwie `AnonymousUser`.

Framework DRF zapewnia obsługę wymienionych poniżej mechanizmów uwierzytelniania.

- `BasicAuthentication`. Uwierzytelnianie podstawowe HTTP. Nazwa użytkownika i hasło są wysyłane przez klienta w nagłówku HTTP `Authorization` zakodowanym za pomocą `Base64`. Więcej informacji o tym uwierzytelnianiu znajdziesz na stronie *https://en.wikipedia.org/wiki/Basic_access_authentication*.
- `TokenAuthentication`. Uwierzytelnianie na podstawie tokenu. Model `Token` jest używany do przechowywania tokenów użytkownika. Użytkownik umieszcza token w nagłówku HTTP `Authorization` i tym sposobem przeprowadza uwierzytelnienie.
- `SessionAuthentication`. Wykorzystanie do obsługi uwierzytelniania backendu sesji frameworka Django. Ten backend przydaje się do przeprowadzania uwierzytelnionych żądań AJAX do API z poziomu frontendu witryny internetowej.
- `RemoteUserAuthentication`. Umożliwia delegowanie uwierzytelniania do serwera WWW, który ustawia zmienną środowiskową `REMOTE_USER`.

Istnieje możliwość przygotowania własnego mechanizmu uwierzytelniania. W tym celu można utworzyć podklasę klasy `BaseAuthentication` dostarczanej przez framework DRF oraz nadpisać metodę `authenticate()`.

Implementacja podstawowego uwierzytelniania

Uwierzytelnianie można zdefiniować dla poszczególnych widoków lub globalnie za pomocą opcji `DEFAULT_AUTHENTICATION_CLASSES`.

> **Uwaga**
>
> Uwierzytelnianie jedynie identyfikuje użytkownika wykonującego żądanie. Nie zezwala na uzyskanie dostępu do widoków lub nie zabrania tego. Jeżeli chcesz ograniczyć dostęp do widoków, musisz wykorzystać system uprawnień.

Wszelkie informacje dotyczące uwierzytelniania znajdziesz na stronie *https://www.django-rest-framework.org/api-guide/authentication/*.

Do widoku dodamy teraz uwierzytelnianie `BasicAuthentication`. Przeprowadź edycję pliku *api/views.py* aplikacji `courses` i umieść atrybut `authentication_classes` w widoku `Course` ↪`EnrollView`, tak jak pokazałem poniżej.

```
# ...
from rest_framework.authentication import BasicAuthentication


class CourseEnrollView(APIView):
    authentication_classes = [BasicAuthentication]
    # ...
```

Użytkownik będzie identyfikowany za pomocą danych uwierzytelniających podanych w nagłówku `Authorization` żądania HTTP.

Określenie uprawnień do widoków

Framework DRF zawiera system uprawnień, za pomocą którego można ograniczyć dostęp do widoków. Oto niektóre z wybranych uprawnień obsługiwanych przez framework DRF:

- `AllowAny`. Niczym nieograniczony dostęp niezależnie od tego, czy użytkownik został uwierzytelniony.
- `IsAuthenticated`. Dostęp jedynie dla uwierzytelnionych użytkowników.
- `IsAuthenticatedOrReadOnly`. Pełny dostęp dla uwierzytelnionych użytkowników. Z kolei użytkownicy anonimowi mogą jedynie wykonywać żądania odczytu danych, które wykorzystują metody `GET`, `HEAD` lub `OPTIONS`.
- `DjangoModelPermissions`. Uprawienia powiązane z `django.contrib.auth`. Widok ten wymaga atrybutu `queryset`. Jedynie uwierzytelniony użytkownik wraz z przydzielonymi uprawnieniami do danego modelu otrzymuje to uprawnienie.

- `DjangoObjectPermissions`. Uprawnienia Django do poszczególnych obiektów.

Jeżeli użytkownikowi zostanie odmówiony dostęp, najczęściej otrzymuje odpowiedź wraz z jednym z poniższych kodów błędu HTTP.

- `HTTP 401`. Brak autoryzacji.
- `HTTP 403`. Brak uprawnień.

Więcej informacji na temat uprawnień znajdziesz na stronie *https://www.django-rest-framework.org/api-guide/permissions/*.

Przeprowadź edycję pliku *api/views.py* aplikacji `courses` i umieść atrybut `permission_classes` w widoku `CourseEnrollView`, co pokazałem poniżej.

```
# …
from rest_framework.authentication import BasicAuthentication
from rest_framework.permissions import IsAuthenticated

class CourseEnrollView(APIView):
    authentication_classes = [BasicAuthentication]
    permission_classes = [IsAuthenticated]
    # …
```

Zdefiniowaliśmy uprawnienie `IsAuthenticated`. W ten sposób anonimowy użytkownik nie będzie mógł uzyskać dostępu do tego widoku. Teraz możesz wykonywać żądania `POST` do nowej metody API.

Upewnij się, że serwer programistyczny jest uruchomiony. Przejdź do powłoki, a następnie wydaj poniższe polecenie.

```
curl -i -X POST http://127.0.0.1:8000/api/courses/1/enroll/
```

Powinieneś otrzymać taką odpowiedź:

```
HTTP/1.1 401 Unauthorized
…
{"detail": "Nie podano danych uwierzytelniających."
```

Zgodnie z oczekiwaniami otrzymałeś kod HTTP `401`, ponieważ nie jesteś uwierzytelnionym użytkownikiem. Wykorzystamy teraz uwierzytelnianie podstawowe wraz z jednym z użytkowników. Wydaj poniższe polecenie. Zastąp *użytkownik*:*hasło* danymi uwierzytelniającymi istniejącego użytkownika.

```
curl -i -X POST -u użytkownik:hasło http://127.0.0.1:8000/api/courses/1/enroll/
```

Powinieneś otrzymać taką odpowiedź:

```
HTTP/1.1 200 OK
…
{"enrolled": true}
```

W witrynie administracyjnej Django możesz teraz potwierdzić, że wskazany użytkownik został zapisany na kurs.

Dołączanie dodatkowych operacji do kolekcji ViewSet

Istnieje możliwość dołączania dodatkowych operacji do kolekcji `ViewSet`. Zmienimy wcześniejszy widok `CourseEnrollView` na niestandardową operację kolekcji `ViewSet`. Przeprowadź edycję pliku *api/views.py* i zmodyfikuj klasę `CourseViewSet` w następujący sposób.

```
# ...
from rest_framework.decorators import action

class CourseViewSet(viewsets.ReadOnlyModelViewSet):
    queryset = Course.objects.prefetch_related('modules')
    serializer_class = CourseSerializer

    @action(
        detail=True,
        methods=['post'],
        authentication_classes=[BasicAuthentication],
        permission_classes=[IsAuthenticated]
    )
    def enroll(self, request, *args, **kwargs):
        course = self.get_object()
        course.students.add(request.user)
        return Response({'enrolled': True})
```

Dodaliśmy własną metodę `enroll()` reprezentującą dodatkowe operacje dla danej kolekcji `ViewSet`. Oto w jaki sposób działa powyższy kod:

1. Używamy oferowanego przez framework dekoratora `action` z parametrem `detail=True` w celu określenia, że operacja będzie wykonywana na pojedynczym obiekcie.
2. Dekorator pozwala na dodanie własnych atrybutów dla operacji. Określamy więc, że jedynie metoda `POST` jest dozwolona dla danego widoku oraz definiujemy odpowiednie klasy uwierzytelniania i uprawnień.
3. Za pomocą wywołania `self.get_object()` pobieramy obiekt `Course`.
4. Dodajemy bieżącego użytkownika do relacji `students` typu „wiele do wielu” i zwracamy własną odpowiedź wskazującą na sukces.

W pliku *api/urls.py* usuń lub oznacz komentarzem poniższy wzorzec adresu URL, ponieważ nie będzie już potrzebny.

```
path(
    'courses/<pk>/enroll/',
    views.CourseEnrollView.as_view(),
    name='course_enroll'
),
```

Następnie przeprowadź edycję pliku *api/views.py* i usuń klasę `CourseEnrollView`.

Adres URL pozwalający uczestnikowi zapisać się na kurs będzie teraz generowany automatycznie przez router. Adres ten pozostaje taki sam, ponieważ jest tworzony dynamicznie za pomocą zdefiniowanej wcześniej operacji o nazwie `enroll`.

Po zarejestrowaniu się na kurs uczniowie muszą uzyskać dostęp do treści kursu. W kolejnym podrozdziale pokażę, jak zapewnić dostęp do kursu tylko zarejestrowanym uczestnikom.

Tworzenie niestandardowych uprawnień

Chcemy, aby uczestnicy mieli dostęp do treści jedynie tych kursów, na które się zapisali. Tylko uczestnik zapisany na dany kurs powinien mieć dostęp do jego zawartości. W tym zakresie najlepszym rozwiązaniem będzie przygotowanie własnej klasy uprawnień. Framework DRF oferuje klasę `BasePermission`, pozwalającą na definiowanie wymienionych poniżej metod.

- `has_permission()`. Sprawdzenie uprawnień na poziomie widoku.
- `has_object_permission()`. Sprawdzenie uprawnień na poziomie egzemplarza.

Wymienione powyżej metody powinny zwracać `True`, aby został przydzielony dostęp lub `False` w przeciwnym razie.

W katalogu *courses/api* utwórz nowy plik o nazwie *permissions.py*. Następnie w pliku umieść poniższy fragment kodu.

```
from rest_framework.permissions import BasePermission


class IsEnrolled(BasePermission):
    def has_object_permission(self, request, view, obj):
        return obj.students.filter(id=request.user.id).exists()
```

Tworzymy podklasę klasy `BasePermission` i nadpisujemy jej metodę `has_object_permission()`. Sprawdzamy, czy użytkownik wykonujący żądanie znajduje się w relacji `students` obiektu `Course`. Następnie wykorzystujemy uprawnienie `IsEnrolled`.

Serializacja treści kursu

Konieczne jest przeprowadzenie serializacji treści kursu. Model `Content` zawiera generyczny klucz obcy pozwalający na powiązanie obiektów pochodzących z różnych modeli treści. W poprzednim rozdziale do wszystkich modeli treści dodaliśmy również metodę `render()`. Teraz tę metodę możemy wykorzystać w celu dostarczenia do API wyrenderowanej treści.

Przeprowadź edycję pliku *api/serializers.py* aplikacji `courses` i umieść w nim poniższy fragment kodu.

```
from courses.models import Content, Course, Module, Subject


class ItemRelatedField(serializers.RelatedField):
    def to_representation(self, value):
        return value.render()

class ContentSerializer(serializers.ModelSerializer):
```

```
    item = ItemRelatedField(read_only=True)
    class Meta:
        model = Content
        fields = ['order', 'item']
```

W tym kodzie definiujemy własną kolumnę za pomocą podklasy serializatora `Related ↪Field` dostarczanego przez framework REST i nadpisujemy metodę `to_representation()`. Zdefiniowaliśmy również serializator `ContentSerializer` dla modelu `Content` i użyliśmy własnej kolumny dla ogólnego klucza zewnętrznego `item`.

Dla modelu `Module` potrzebny jest alternatywny serializator, który uwzględni jego treść, a także rozszerzony serializator modelu `Course`. Przeprowadź edycję pliku *api/serializers.py* i umieść w nim poniższy fragment kodu.

```
class ModuleWithContentsSerializer(serializers.ModelSerializer):
    contents = ContentSerializer(many=True)
    class Meta:
        model = Module
        fields = ['order', 'title', 'description', 'contents']

class CourseWithContentsSerializer(serializers.ModelSerializer):
    modules = ModuleWithContentsSerializer(many=True)
    class Meta:
        model = Course
        fields = [
            'id',
            'subject',
            'title',
            'slug',
            'overview',
            'created',
            'owner',
            'modules'
        ]
```

Przystępujemy do utworzenia widoku odzwierciedlającego zachowanie operacji `retrieve()`, ale uwzględniającego treść kursu. Przeprowadź edycję pliku *api/views.py* i dodaj poniższą metodę do klasy `CourseViewSet`.

```
# …

from courses.api.permissions import IsEnrolled
from courses.api.serializers import CourseWithContentsSerializer

class CourseViewSet(viewsets.ReadOnlyModelViewSet):
    # …
    @action(
        detail=True,
        methods=['get'],
        serializer_class=CourseWithContentsSerializer,
        authentication_classes=[BasicAuthentication],
        permission_classes=[IsAuthenticated, IsEnrolled]
    )
    def contents(self, request, *args, **kwargs):
        return self.retrieve(request, *args, **kwargs)
```

Oto omówienie nowej metody:

1. Użyliśmy dekoratora `action` z parametrem `detail=True` w celu wskazania, że ta operacja jest przeprowadzana na pojedynczym obiekcie.
2. Określiliśmy, że dla tej operacji jest dozwolona jedynie metoda `GET`.
3. Użyliśmy nowej klasy serializatora o nazwie `CourseWithContentsSerializer`, która uwzględnia wyrenderowaną treść kursu.
4. Użyliśmy uprawnień zarówno standardowego `IsAuthenticated`, jak i własnego `IsEnrolled`. W ten sposób mamy pewność, że jedynie użytkownik zapisany na dany kurs będzie mógł uzyskać dostęp do jego treści.
5. Istniejącą operację `retrieve()` wykorzystujemy do zwrócenia obiektu kursu.

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/api/courses/1/contents/`. Jeżeli podałeś odpowiednie dane uwierzytelniające, uzyskasz dostęp do wskazanego widoku i zobaczysz, że każdy moduł tego kursu ma wygenerowaną dla niego treść HTML podobną do pokazanej poniżej.

```
{
    "order": 0,
    "title": "Wprowadzenie do Django",
    "description": "Krótkie wprowadzenie do frameworka webowego Django.",
    "contents": [
        {
            "order": 0,
            "item": "<p>Poznaj Django. Django jest wysokopoziomowym frameworkiem
            webowym w Pythonie
            ...</p>"
        },
        {
            "order": 1,
            "item": "\n<iframe width=\"480\" height=\"360\"
            src=\"http://www.youtube.com/embed/bgV39DlmZ2U?
            wmode=opaque\"
            frameborder=\"0\" allowfullscreen></iframe>\n"
        }
    ]
}
```

W ten sposób opracowałeś proste API pozwalające innym usługom na programowe uzyskanie dostępu do aplikacji `courses`. Framework DRF umożliwia zarządzanie tworzeniem i edycją obiektów za pomocą klasy `ModelViewSet`. Przedstawiłem tutaj najważniejsze aspekty frameworka REST w Django, a więcej informacji na jego temat znajdziesz w obszernej dokumentacji dostępnej w witrynie *https://www.django-rest-framework.org/*.

Wykorzystanie API RESTful

Po zaimplementowaniu interfejsu API można z niego skorzystać w sposób programowy z innych aplikacji. Możesz komunikować się z API z poziomu JavaScript w warstwie frontend Twojej aplikacji w sposób podobny do funkcjonalności, którą zbudowaliśmy w rozdziale 6., „Udostępnianie treści w witrynie internetowej”. Możesz również korzystać z API aplikacji budowanych w Pythonie lub dowolnych innych językach programowania.

W tym rozdziale utworzymy prostą aplikację w Pythonie, która używa API RESTful w celu pobrania wszystkich dostępnych kursów. Następnie zapiszemy się na jeden z nich. Dowiesz się, w jaki sposób przeprowadzać uwierzytelnianie z wykorzystaniem API za pomocą podstawowego uwierzytelniania HTTP oraz jak wykonywać żądania `GET` i `POST`.

Aby skorzystać z API, użyjemy biblioteki Pythona `Requests`. Z biblioteki `Requests` korzystaliśmy w rozdziale 6., „Udostępnianie treści w witrynie internetowej", w celu pobierania zdjęć z podanego adresu URL. Biblioteka ta dodaje warstwę abstrakcji nad operacjami związanymi z żądaniami HTTP i zapewnia bardzo prosty interfejs do korzystania z usług HTTP. Dokumentację biblioteki `Requests` można znaleźć pod adresem *https://requests.readthedocs.io/en/master/*.

Otwórz powłokę i zainstaluj bibliotekę `Requests` za pomocą polecenia

```
python -m pip install requests==2.31.0
```

Utwórz nowy katalog obok katalogu projektu `educa` i nadaj mu nazwę *api_examples*. Wewnątrz katalogu *api_examples* utwórz nowy plik i nadaj mu nazwę *enroll_all.py*. Struktura plików powinna teraz wyglądać tak:

```
api_examples/
    enroll_all.py
educa/
    ...
```

Przeprowadź edycję pliku *enroll_all.py* i dodaj do niego kod:

```
import requests

base_url = 'http://127.0.0.1:8000/api/'
url = f'{base_url}courses/'
available_courses = []
while url is not None:
    print(f'Ładowanie kursów z {url}')
    r = requests.get(url)
    response = r.json()
    url = response['next']
    courses = response['results']
    available_courses += [course['title'] for course in courses]
print(f'Dostępne kursy: {", ".join(available_courses)}')
```

W powyższym kodzie wykonujemy następujące działania.

1. Importujemy bibliotekę `requests` i definiujemy bazowy adres URL interfejsu API oraz adres URL punktu końcowego listy kursów.
2. Definiujemy pustą listę `available_courses`.
3. Do stronicowania wszystkich stron wyników używamy instrukcji `while`.
4. Za pomocą metody `requests.get()` pobieramy dane z API, wysyłając żądanie `GET` pod adres URL `http://127.0.0.1:8000/api/courses/`. Ten punkt końcowy API jest dostępny publicznie, a więc nie wymaga żadnego uwierzytelniania.
5. Za pomocą metody `json()` obiektu `response` dekodujemy dane w formacie JSON zwracane przez interfejs API.

6. W zmiennej `url` zapisujemy atrybut `next`, co pozwala na pobranie w instrukcji `while` następnej strony wyników.
7. Dodajemy atrybut `title` każdego kursu do listy `available_courses`.
8. Gdy zmienna `url` ma wartość `None`, następuje przejście do ostatniej strony wyników i nie są pobierane żadne dodatkowe strony.
9. Wyświetlamy listę dostępnych kursów.

Uruchom serwer programistyczny z katalogu projektu `educa` za pomocą polecenia:

```
python manage.py runserver
```

W innym egzemplarzu powłoki uruchom z katalogu *api_examples/* polecenie:

```
python enroll_all.py
```

Dane wyjściowe będą zawierać listę wszystkich nazw kursów, tak jak pokazałem poniżej.

```
Dostępne kursy: Wprowadzenie do Django, Python dla początkujących, Podstawy algebry
```

To pierwsze zautomatyzowane wywołanie do Twojego API.

W pliku *enroll_all.py* dodaj następujące wiersze zaznaczone pogrubioną czcionką:

```
import requests

username = ''
password = ''

base_url = 'http://127.0.0.1:8000/api/'
url = f'{base_url}courses/'
available_courses = []

while url is not None:
    print(f'Ładowanie kursów z {url}')
    r = requests.get(url)
    response = r.json()
    url = response['next']
    courses = response['results']
    available_courses += [course['title'] for course in courses]
print(f'Dostępne kursy: {", ".join(available_courses)}')

for course in courses:
    course_id = course['id']
    course_title = course['title']
    r = requests.post(
        f'{base_url}courses/{course_id}/enroll/',
        auth=(username, password)
    )
    if r.status_code == 200:
        # Żądanie zakończone sukcesem
        print(f'Pomyślnie zapisałeś się na kurs {course_title}')
```

Zastąp wartości zmiennych `username` i `password` poświadczeniami istniejącego użytkownika lub załaduj wartości ze zmiennych środowiskowych. Do załadowania poświadczeń

ze zmiennych środowiskowych możesz użyć modułu `python-decouple`, podobnie jak w punkcie „Korzystanie ze zmiennych środowiskowych" w rozdziale 2., „Usprawnianie bloga i dodanie funkcjonalności społecznościowych".

W nowej wersji kodu wykonujemy następujące działania.

1. Definiujemy nazwę użytkownika i hasło uczestnika, który ma się zapisać na kursy.
2. Iterujemy po liście dostępnych kursów pobranych za pomocą interfejsu API.
3. Zapisujemy identyfikator kursu w zmiennej `course_id` oraz atrybut `title` w zmiennej `course_title`.
4. Za pomocą wywołania `requests.post()` wysyłamy dla każdego kursu żądanie `POST` pod adres URL `http://127.0.0.1:8000/api/[id]/enroll/`. Ten adres URL odpowiada widokowi API `CourseEnrollView`, który pozwala użytkownikowi zapisać się na kurs. Budujemy adres URL dla każdego kursu za pomocą zmiennej `course_id`. Widok `CourseEnrollView` wymaga uwierzytelniania. Wykorzystuje uprawnienie `IsAuthenticated` i klasę uwierzytelniania `BasicAuthentication`. Biblioteka `Requests` zawiera wbudowaną obsługę prostego uwierzytelniania HTTP. W celu przeprowadzenia uwierzytelniania użytkownika za pomocą podstawowego uwierzytelniania HTTP wykorzystujemy parametr `auth` po to, by przekazać krotkę z nazwą użytkownika i hasłem.
5. Jeśli kod odpowiedzi to `200 OK`, wyświetlamy komunikat informujący o tym, że użytkownik został pomyślnie zapisany na kurs.

Biblioteka `Requests` pozwala na wykorzystywanie innych rodzajów uwierzytelniania. Więcej informacji na temat uwierzytelniania z wykorzystaniem biblioteki `Requests` można znaleźć pod adresem *https://requests.readthedocs.io/en/master/user/authentication/*.

Z poziomu katalogu *api_examples/* uruchom polecenie:

```
python enroll_all.py
```

Zobaczysz takie dane wyjściowe.

```
Dostępne kursy: Wprowadzenie do Django, Python dla początkujących, Podstawy algebry
Pomyślnie zapisałeś się na kurs Wprowadzenie do Django
Pomyślnie zapisałeś się na kurs Python dla początkujących
Pomyślnie zapisałeś się na kurs Podstawy algebry
```

Doskonale! Korzystając z API, pomyślnie zapisałeś użytkownika na wszystkie dostępne kursy. Dla każdego kursu dostępnego na platformie zobaczysz komunikat zaczynający się od słów: „Pomyślnie zapisałeś się na kurs". Jak mogłeś się przekonać, korzystanie z API z poziomu dowolnej aplikacji jest bardzo łatwe.

Podsumowanie

W tym rozdziale dowiedziałeś się, jak użyć frameworka DRF do zbudowania w swoim projekcie API typu RESTful. Utworzyłeś serializatory i widoki dla modeli oraz zbudowałeś własne widoki API. Dodałeś również do Twojego API mechanizm uwierzytelniania

i ograniczyłeś dostęp do widoków API z wykorzystaniem uprawnień. Następnie dowiedziałeś się, jak tworzyć niestandardowe uprawnienia i zaimplementowałeś zestawy widoków i routery. Na koniec skorzystałeś z biblioteki `Requests`, aby wykorzystać API z zewnętrznego skryptu Pythona.

W następnym rozdziale pokażę, jak utworzyć serwer czatu za pomocą frameworka Django Channels. Zaimplementujesz komunikację asynchroniczną z wykorzystaniem mechanizmu WebSocket i skorzystasz z bazy danych Redis do utworzenia warstwy kanałów.

Dodatkowe zasoby

Oto lista zasobów zawierających dodatkowe informacje związane z tematami omawianymi w tym rozdziale:

- Kod źródłowy przykładów zamieszczonych w tym rozdziale — *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter15*.
- Witryna internetowa frameworka DRF — *https://www.django-rest-framework.org/*.
- Ustawienia frameworka REST — *https://www.django-rest-framework.org/api-guide/settings/*.
- Serializatory frameworka DRF — *https://www.django-rest-framework.org/api-guide/serializers/*.
- Klasy renderujące frameworka DRF — *https://www.django-rest-framework.org/api-guide/renderers/*.
- Parsery frameworka DRF — *https://www.django-rest-framework.org/api-guide/parsers/*.
- Generyczne domieszki i widoki frameworka DRF — *https://www.django-rest-framework.org/api-guide/generic-views/*.
- Strona pobierania narzędzia `curl` — *https://curl.se/download.html*.
- Platforma API Postman — *https://www.getpostman.com/*.
- Stronicowanie we frameworku DRF — *https://www.django-rest-framework.org/api-guide/pagination/*.
- Relacje serializatorów frameworka DRF — *https://www.django-rest-framework.org/api-guide/relations/*.
- Podstawowe uwierzytelnianie HTTP — *https://en.wikipedia.org/wiki/Basic_access_authentication*.
- Uwierzytelnianie we frameworku DRF — *https://www.django-rest-framework.org/api-guide/authentication/*.
- Uprawnienia we frameworku DRF — *https://www.django-rest-framework.org/api-guide/permissions/*.
- Kolekcje `ViewSet` frameworka DRF — *https://www.django-rest-framework.org/api-guide/viewsets/*.
- Routery frameworka DRF — *https://www.django-rest-framework.org/api-guide/routers/*.

- Dokumentacja biblioteki Pythona `Requests` — *https://requests.readthedocs.io/en/master/*.
- Uwierzytelnianie za pomocą biblioteki `Requests` — *https://requests.readthedocs.io/en/master/user/authentication/*.

Budowanie serwera czatu

Rozdział 16

W poprzednim rozdziale utworzyłeś dla swojego projektu API RESTful, które zapewnia programowalny interfejs dla Twojej aplikacji.

W tym rozdziale zbudujesz za pomocą frameworka Django Channels serwer czatu dla uczestników kursów. Uczestnicy kursów będą mogli uzyskać dostęp do różnych pokojów czatu dla każdego kursu, do którego są zapisani. Dzięki zapoznaniu się z technikami programowania asynchronicznego z wykorzystaniem Django Channels nauczysz się, jak tworzyć aplikacje działające w czasie rzeczywistym. Dzięki serwowaniu projektu Django za pośrednictwem interfejsu **ASGI** (ang. *Asynchronous Server Gateway Interface*) i implementacji komunikacji asynchronicznej zwiększysz responsywność i skalowalność swojego serwera. Wiadomości czatu będą utrwalane w bazie danych, co pozwoli na utworzenie kompleksowej historii czatu oraz zwiększy wygodę użytkowania i funkcjonalność aplikacji czatu.

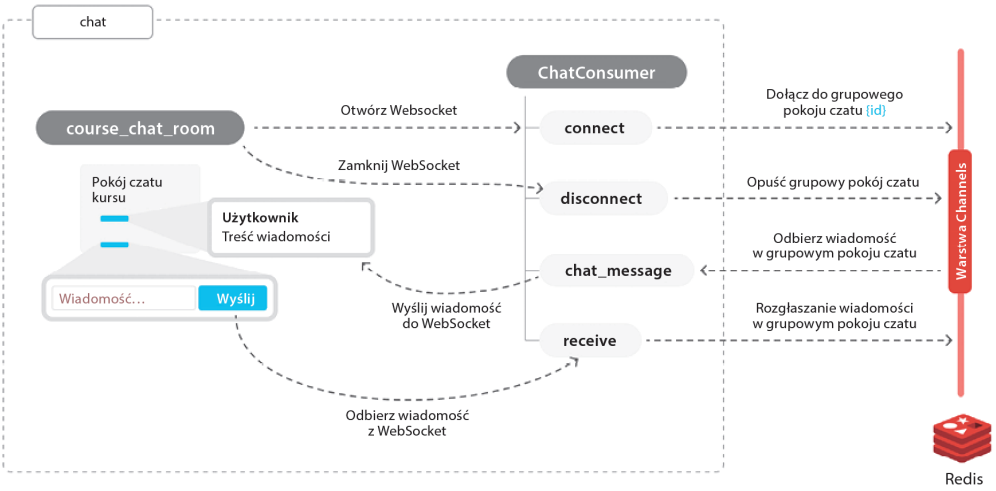
W tym rozdziale skoncentruję się na następujących zagadnieniach:

- Dodanie frameworka `Channels` do projektu
- Zbudowanie konsumenta `WebSocket` i odpowiedniego routingu
- Implementacja klienta `WebSocket`
- Uaktywnienie warstwy kanału z wykorzystaniem bazy danych Redis
- Utworzenie w pełni asynchronicznego klienta
- Utrwalanie wiadomości czatu w bazie danych

Przegląd funkcjonalności

Reprezentację widoków, szablonów i funkcjonalności, które zbudujemy w tym rozdziale, przedstawiłem na rysunku 16.1.

W tym rozdziale zaimplementujesz w aplikacji `chat` widok `course_chat_room`. Ten widok będzie służył jako szablon do wyświetlania pokoju czatu dla wybranego kursu. Gdy użytkownik dołączy do pokoju czatu, zostaną wyświetlone najnowsze wiadomości, jakie się na nim pojawiły. Skorzystamy z JavaScript w celu nawiązania połączenia `WebSocket` w przeglądarce, a następnie utworzymy konsumenta gniazda WebSocket `ChatConsumer`, który będzie obsługiwał połączenia `WebSocket` i wymieniał wiadomości. Do zaimplementowania warstwy kanału, która umożliwia nadawanie wiadomości do wszystkich użytkowników w pokoju czatu, wykorzystamy bazę danych Redis.



Rysunek 16.1. Schemat funkcjonalności zbudowanych w rozdziale 16.

Kod źródłowy przykładów z tego rozdziału można znaleźć pod adresem *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter16*.

Wszystkie pakiety Pythona użyte w tym rozdziale są zawarte w pliku *requirements.txt* dołączonym do plików z kodem źródłowym przykładów z tego rozdziału. Aby zainstalować każdy pakiet Pythona, możesz postępować zgodnie z instrukcjami zamieszczonymi w poniższych podrozdziałach. Możesz także zainstalować wszystkie wymagania jednocześnie za pomocą polecenia `pip install -r requirements.txt`.

Utworzenie aplikacji czatu

Zamierzamy zaimplementować serwer czatu dający uczestnikom kursów dostęp do pokoju czatu dla każdego kursu. Uczestnicy kursu będą mogli wejść do pokoju czatu kursu i w czasie rzeczywistym wymieniać wiadomości. Do zbudowania tej funkcjonalności wykorzystamy framework `Channels`. Jest to aplikacja będąca rozszerzeniem frameworka Django, służąca do obsługi protokołów wymagających długotrwałych połączeń, takich jak WebSocket, chatboty lub MQTT (prosta warstwa transportu wiadomości typu publikuj-subskrybuj używana w projektach **IoT**, czyli w internecie rzeczy).

Za pomocą frameworka `Channels` można łatwo zaimplementować w projekcie funkcjonalności działające w czasie rzeczywistym lub asynchroniczne, które mogą uzupełnić standardowe synchroniczne widoki HTTP. Zaczniemy od dodania do projektu nowej aplikacji. Nowa aplikacja będzie zawierać logikę serwera czatu.

Dokumentację frameworka Django Channels można znaleźć na stronie *https://channels.readthedocs.io/*.

Przystępujemy do implementacji serwera czatu. Aby utworzyć strukturę plików nowej aplikacji, uruchom z poziomu katalogu projektu `educa` polecenie:

```
django-admin startapp chat
```

Przeprowadź edycję pliku *settings.py* projektu `educa` i uaktywnij aplikację czatu w Twoim projekcie, czyli zmodyfikuj ustawienie `INSTALLED_APPS` w następujący sposób.

```
INSTALLED_APPS = [
    # ...
    'chat.apps.ChatConfig',
]
```

Właśnie uaktywniłeś w projekcie nową aplikację czatu. W następnym punkcie utworzysz widok dla pokojów czatu kursów.

Implementacja widoku pokoju czatu

Uczestnicy kursów będą mieli dostęp do oddzielnego pokoju czatu dla każdego kursu. Należy utworzyć widok pozwalający uczestnikom kursów na dołączenie do pokoju czatu dla wybranego kursu. Dostęp do pokoju czatu danego kursu będą mogli uzyskać wyłącznie uczestnicy tego kursu.

W pliku *views.py* nowej aplikacji `chat` dodaj kod:

```
from django.contrib.auth.decorators import login_required
from django.http import HttpResponseForbidden
from django.shortcuts import render
from courses.models import Course


@login_required
def course_chat_room(request, course_id):
    try:
        # Pobranie kursu o identyfikatorze zgodnym z identyfikatorem kursu, do którego
        # dołączył bieżący użytkownik.
        course = request.user.courses_joined.get(id=course_id)
    except Course.DoesNotExist:
        # Użytkownik nie jest uczestnikiem tego kursu lub kurs nie istnieje.
        return HttpResponseForbidden()
    return render(request, 'chat/room.html', {'course': course})
```

To jest widok `course_chat_room`. W tym widoku skorzystaliśmy z dekoratora `@login_required`, aby uniemożliwić dostęp do widoku użytkownikom, którzy nie zostali uwierzytelnieni. Oto działanie tego widoku:

1. Widok otrzymuje obowiązkowy parametr `course_id`, który służy do pobierania kursu o podanym identyfikatorze `id`.
2. Dostęp do kursów, na które użytkownik jest zapisany, uzyskujemy za pośrednictwem relacji `courses_joined`. Z tego podzbioru kursów pobieramy kurs o podanym identyfikatorze. Jeśli kurs o podanym identyfikatorze nie istnieje lub użytkownik nie jest zapisany na ten kurs, zwracamy odpowiedź `HttpResponseForbidden`, która oznacza odpowiedź HTTP ze statusem `403`.
3. Jeśli kurs o podanym identyfikatorze istnieje, a użytkownik jest zapisany na ten kurs, renderujemy szablon *chat/room.html*, przekazując do kontekstu szablonu obiekt `course`.

Dla tego widoku trzeba dodać wzorzec URL. Utwórz nowy plik w katalogu aplikacji `chat` i nadaj mu nazwę *urls.py*. Wprowadź do niego kod:

```
from django.urls import path
from . import views


app_name = 'chat'

urlpatterns = [
    path(
        'room/<int:course_id>/',
        views.course_chat_room,
        name='course_chat_room'),
]
```

To początkowa zawartość pliku wzorców URL dla aplikacji czatu. Zdefiniowaliśmy wzorzec URL `course_chat_room` włącznie z parametrem `course_id` poprzedzonym przedrostkiem `int`, ponieważ w tym miejscu oczekujemy wyłącznie liczby całkowitej.

Uwzględnij nowe wzorce URL aplikacji czatu w głównym pliku wzorców URL projektu. Przeprowadź edycję głównego pliku *urls.py* projektu `educa` i dodaj do niego wiersz:

```
urlpatterns = [
    # …
    path('chat/', include('chat.urls', namespace='chat')),
]
```

Wzorce adresów URL aplikacji czatu są dodawane do projektu w ramach ścieżki *chat/*.

Należy teraz utworzyć szablon dla widoku `course_chat_room`. Ten szablon będzie zawierać obszar do wizualizacji komunikatów wymienianych na czacie oraz pole wprowadzania tekstu z przyciskiem *Prześlij* służące do wysyłania na czacie wiadomości tekstowych.

W katalogu aplikacji `chat` utwórz następującą strukturę plików.

```
templates/
    chat/
        room.html
```

Przeprowadź edycję szablonu *chat/room.html* i dodaj do niego kod:

```
{% extends "base.html" %}

{% block title %}Pokój czatu dla "{{ course.title }}"{% endblock %}

{% block content %}
  <div id="chat">
  </div>
  <div id="chat-input">
    <input id="chat-message-input" type="text">
    <input id="chat-message-submit" type="submit" value="Prześlij">
  </div>
{% endblock %}

{% block include_js %}
```

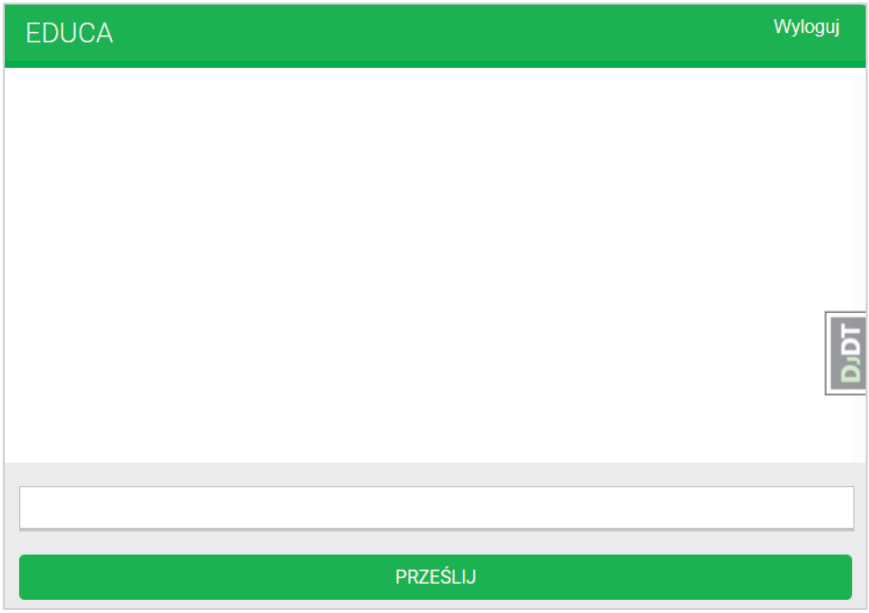
```
{% endblock %}

{% block domready %}
{% endblock %}
```

To jest szablon pokoju czatu dla kursu. W tym szablonie wykonaliśmy następujące działania:

1. Rozszerzyliśmy szablon *base.html* projektu i wypełniliśmy jego blok `content`.
2. Zdefiniowaliśmy element HTML `<div>` z identyfikatorem czatu, który wykorzystamy do wyświetlania wiadomości czatu wysyłanych przez użytkownika oraz innych uczestników kursu.
3. Zdefiniowaliśmy także drugi element `<div>`, z polem wprowadzania tekstu i przyciskiem *Prześlij*, pozwalającym użytkownikowi na przesyłanie wiadomości.
4. W szablonie umieściliśmy bloki `include_js` i `domready` zdefiniowane przez szablon *base.html*, który zaimplementujemy później z wykorzystaniem języka JavaScript w celu nawiązania połączenia z gniazdem WebSocket i wysyłania bądź odbierania wiadomości.

Uruchom serwer programistyczny i otwórz w przeglądarce adres URL `http://127.0.0.1:8000/chat/room/1/`, zastępując `1` identyfikatorem istniejącego kursu z bazy danych.

Spróbuj uzyskać dostęp do pokoju czatu, kiedy zalogujesz się jako użytkownik, który jest zapisany na kurs. Powinieneś zobaczyć ekran pokazany na rysunku 16.2.



Rysunek 16.2. Strona pokoju czatu kursu

To jest ekran pokoju czatu kursu, który uczestnicy będą wykorzystywać do dyskutowania na tematy dotyczące kursu.

Utworzyłeś bazowy widok dla pokoju czatu kursu. Teraz trzeba obsłużyć wiadomości przesyłane pomiędzy uczestnikami kursu. W następnym podrozdziale omawiam obsługę asynchronicznego trybu wykorzystania frameworka `Channels` do komunikacji w czasie rzeczywistym.

Komunikacja w czasie rzeczywistym w Django za pomocą frameworka Channels

Zamierzamy zbudować serwer czatu dający uczestnikom kursów dostęp do pokoju czatu dla każdego kursu. Uczestnicy kursu będą mogli uzyskać dostęp do pokoju czatu kursu i wymieniać pomiędzy sobą wiadomości. Implementacja tej funkcjonalności wymaga komunikacji pomiędzy serwerem a klientem w czasie rzeczywistym.

Do tego celu nie sprawdza się standardowy model HTTP żądanie-odpowiedź, ponieważ chcemy, aby w przeglądarce powiadomienia wyświetlały się bezpośrednio po pojawieniu się nowych wiadomości. Istnieje kilka sposobów, na jakie można zaimplementować tę funkcjonalność za pomocą odpytywania AJAX lub długotrwałego odpytywania w połączeniu z przechowywaniem wiadomości w bazie danych lub magazynie Redis. Jednak nie ma skutecznego sposobu implementacji serwera czatu za pomocą standardowej, synchronicznej aplikacji webowej.

Potrzebna jest komunikacja asynchroniczna, która umożliwia interakcje w czasie rzeczywistym. Serwer powinien mieć możliwość przesyłania aktualizacji do klienta, gdy tylko pojawią się nowe wiadomości, bez konieczności wysyłania przez klienta okresowego żądania aktualizacji. Komunikacja asynchroniczna ma również inne zalety, takie jak mniejsze opóźnienia, lepsza wydajność pod obciążeniem i lepszy ogólny komfort pracy użytkownika. Serwer czatu zbudujemy z wykorzystaniem komunikacji asynchronicznej za pośrednictwem ASGI.

Aplikacje asynchroniczne z wykorzystaniem ASGI

Django jest zazwyczaj wdrażany z wykorzystaniem serwera **WSGI** (ang. *Web Server Gateway Interface*), który jest standardowym interfejsem dla aplikacji Pythona wykorzystywanym do obsługi żądań HTTP. Jednak do pracy z aplikacjami asynchronicznymi trzeba użyć innego interfejsu — o nazwie ASGI — który pozwala obsługiwać również żądania WebSocket. ASGI to nowy standard Pythona dla asynchronicznych serwerów WWW i aplikacji. Dzięki wykorzystaniu ASGI umożliwimy frameworkowi Django obsługę każdej wiadomości niezależnie i w czasie rzeczywistym. W ten sposób uzyskamy możliwość płynnego i dynamicznego czatu pomiędzy uczestnikami kursów.

Wprowadzenie do tematyki interfejsu ASGI można znaleźć pod adresem *https://asgi.readthedocs.io/en/latest/introduction.html.*

Django zawiera obsługę asynchronicznych aplikacji Pythona działających z wykorzystaniem ASGI. Pisanie widoków asynchronicznych jest obsługiwane od wydania Django 3.1, a w wersji Django 4.1 wprowadzono asynchroniczne procedury obsługi dla widoków opartych na klasach. W Django 5.0 dodano w asynchronicznych widokach obsługę zdarzeń rozłączenia przed wygenerowaniem odpowiedzi. Dodano także funkcje asynchroniczne

do frameworka uwierzytelniania, zapewniono obsługę asynchronicznego wysyłania sygnałów i dodano obsługę trybu asynchronicznego dla wielu wbudowanych dekoratorów.

Framework `Channels` opiera się na natywnej obsłudze ASGI dostępnej w Django, a oprócz niej dostarcza dodatkowe funkcjonalności obsługi protokołów wymagających długotrwałych połączeń, takich jak WebSocket, protokoły IoT i protokoły czatu.

WebSocket zapewnia komunikację w trybie pełnego dupleksu poprzez ustanowienie trwałego, otwartego i dwukierunkowego połączenia **TCP** (ang. *Transmission Control Protocol*) pomiędzy serwerami a klientami. Zamiast wysyłać do serwera żądania HTTP, nawiązujemy z nim połączenie. Po otwarciu kanału można wymieniać wiadomości w obu kierunkach bez konieczności nawiązywania za każdym razem nowego połączenia. Do zaimplementowania serwera czatu w tym projekcie użyjemy połączenia WebSocket.

Więcej o WebSocket można przeczytać na stronie *https://en.wikipedia.org/wiki/WebSocket*.

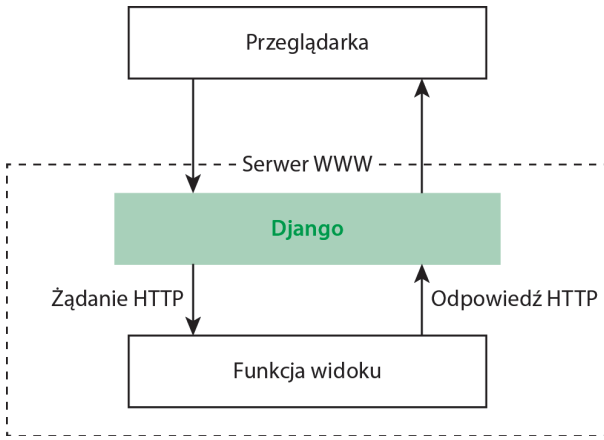
Więcej informacji na temat wdrażania Django z ASGI można znaleźć na stronie *https://docs.djangoproject.com/en/5.0/howto/deployment/asgi/*.

Więcej informacji o obsłudze we frameworku Django asynchronicznych widoków można znaleźć na stronie *https://docs.djangoproject.com/en/5.0/topics/async/*, a obsługę w Django asynchronicznych widoków opartych na klasach opisano na stronie *https://docs.djangoproject.com/en/5.0/topics/class-based-views/#async-classbased-views*.

W następnym punkcie opowiem o modyfikowaniu cyklu żądanie-odpowiedź w Django za pomocą frameworka `Channels`.

Cykl żądanie-odpowiedź z wykorzystaniem frameworka Channels

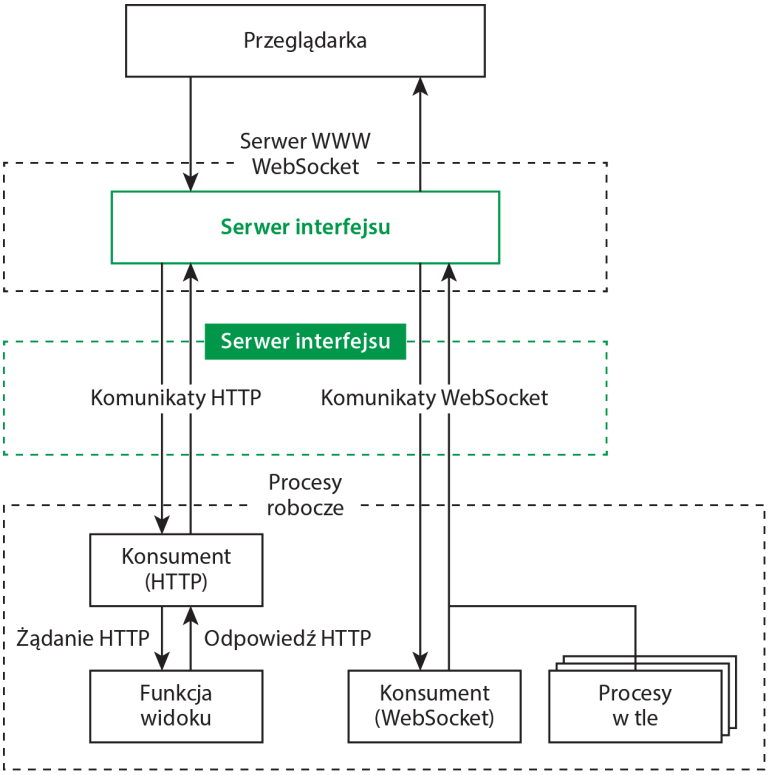
Bardzo ważne jest zrozumienie różnic w cyklu pomiędzy żądaniem standardowym a żądaniem wykonywanym z wykorzystaniem implementacji opartej na frameworku `Channels`. Schemat cyklu żądania w synchronicznej konfiguracji Django pokazałem na rysunku 16.3.



Rysunek 16.3. Cykl żądanie-odpowiedź frameworka Django

Gdy przeglądarka prześle żądanie HTTP do serwera WWW, Django obsługuje żądanie i przekazuje do odpowiedniego widoku obiekt `HttpRequest`. Widok przetwarza żądanie i zwraca obiekt `HttpResponse`, który jest wysyłany z powrotem do przeglądarki w odpowiedzi HTTP. Nie istnieje żaden mechanizm, który utrzymywałby otwarte połączenie lub wysyłał dane do przeglądarki bez powiązanego z nim żądania HTTP.

Cykl żądania w projekcie Django z wykorzystaniem frameworka `Channels` i mechanizmu WebSocket pokazałem na rysunku 16.4.



Rysunek 16.4. Cykl żądanie-odpowiedź w Django z wykorzystaniem frameworka Channels

Framework `Channels` zastępuje cykl Django żądanie-odpowiedź komunikatami przesyłanymi w różnych kanałach. Żądania HTTP są nadal kierowane do funkcji widoku za pomocą Django, ale przechodzą przez kanały. Pozwala to również na obsługę komunikacji WebSocket, w której występują producenci i konsumenci wymieniający komunikaty przez warstwę kanału. Framework `Channels` zachowuje synchroniczną architekturę Django. To pozwala wybrać pomiędzy pisaniem kodu synchronicznego a asynchronicznego bądź połączenia kodu synchronicznego z asynchronicznym.

Istniejące widoki synchroniczne będą współistnieć z funkcjonalnością WebSocket, którą zaimplementujemy za pomocą Daphne. Dzięki temu będziemy obsługiwać zarówno żądania HTTP, jak i WebSocket.

W następnym podrozdziale pokażę, jak zainstalować aplikację `Channels` i dodać ją do projektu.

Instalacja frameworków Channels i Daphne

Przystępujemy do dodania do projektu frameworka `Channels` oraz konfiguracji wymaganego routingu ASGI w celu zarządzania żądaniami HTTP.

Aby zainstalować framework `Channels` w swoim środowisku wirtualnym, użyj polecenia:

```
python -m pip install -U channels[daphne]==4.1.0
```

Spowoduje to zainstalowanie frameworka `Channels` wraz z serwerem aplikacji ASGI Daphne. Serwer ASGI jest niezbędny do obsługi żądań asynchronicznych. Wybrałem Daphne ze względu na jego prostotę i kompatybilność, wynikającą z tego, że jest dostarczany w pakiecie z frameworkiem `Channels`.

W pliku *settings.py* projektu dodaj `'daphne'` na liście `INSTALLED_APPS`, tak jak pokazałem poniżej.

```
INSTALLED_APPS = [
    'daphne',
    # …
]
```

Po dodaniu `daphne` do ustawienia `INSTALLED_APPS` to Daphne przejmuje kontrolę nad poleceniem `runserver` — zastępując standardowy serwer programistyczny Django. Pozwoli to na obsługę asynchronicznych żądań podczas programowania aplikacji. Oprócz obsługi routingu URL do widoków Django dla synchronicznych żądań serwer programistyczny Daphne zarządza również trasami dla konsumentów WebSocket. Więcej informacji o Daphne można znaleźć na stronie *https://github.com/django/daphne*.

Framework `Channels` wymaga zdefiniowania w projekcie jednej, głównej aplikacji obsługi kanałów, która będzie uruchamiana dla wszystkich żądań. Tę aplikację możesz zdefiniować, dodając do projektu ustawienie `ASGI_APPLICATION`. Jest to ustawienie podobne do opcji `ROOT_URLCONF` wskazującej na bazowe wzorce URL Twojego projektu. Aplikację obsługi kanałów możesz umieścić w dowolnym miejscu projektu, ale zaleca się, aby umieścić ją w pliku *routing.py* na poziomie projektu. Główną konfigurację routingu możesz dodać bezpośrednio do pliku *asgi.py*, w którym zostanie zdefiniowana aplikacja ASGI.

Przeprowadź edycję pliku *asgi.py* w katalogu projektu `educa` i dodaj następujący kod wyróżniony pogrubioną czcionką.

```
import os
from channels.routing import ProtocolTypeRouter
from django.core.asgi import get_asgi_application

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'educa.settings')

django_asgi_app = get_asgi_application()
application = ProtocolTypeRouter({
    'http': django_asgi_app,
})
```

W powyższym kodzie zdefiniowaliśmy główną aplikację ASGI, która będzie uruchamiana w przypadku serwowania Twojej aplikacji Django za pomocą ASGI. W roli głównego punktu wejścia systemu routingu postawiliśmy klasę `ProtocolTypeRouter` należącą do frameworka

`Channels`. Klasa `ProtocolTypeRouter` pobiera słownik, który mapuje typy komunikacji, takie jak `http` lub `websocket` na aplikacje ASGI. Egzemplarz tej klasy tworzymy z domyślą aplikacją dla protokołu HTTP. Później dodamy protokół dla protokołu WebSocket.

Przeprowadź edycję pliku *settings.py* projektu i dodaj w nim poniższy wiersz kodu.

```
ASGI_APPLICATION = 'educa.asgi.application'
```

Do zlokalizowania głównej konfiguracji routingu kanały wykorzystują ustawienie `ASGI_APPLICATION`.

Uruchom serwer programistyczny za pomocą polecenia:

```
python manage.py runserver
```

Zobaczysz wynik podobny do tego:

```
Watching for file changes with StatReloader
Performing system checks...
System check identified no issues (0 silenced).
April 14, 2024 - 08:02:57
Django version 5.0.4, using settings 'educa.settings'
Starting ASGI/Daphne version 4.1.0 development server at http://127.0.0.1:8000/
Quit the server with CONTROL-C.
```

Upewnij się, że dane wyjściowe zawierają wiersz `Starting ASGI/Daphne version 4.1.0 development server`. Ten wiersz potwierdza, że zamiast standardowego serwera programistycznego Django używasz serwera programistycznego Daphne, który jest zdolny do zarządzania żądaniami zarówno synchronicznymi, jak i asynchronicznymi. Żądania HTTP nadal zachowują się tak jak wcześniej, ale są kierowane przez framework `Channels`.

Po zainstalowaniu frameworka `Channels` w Twoim projekcie możesz zbudować serwer czatu dla aplikacji `courses`. Aby zaimplementować serwer czatu w swoim projekcie, wykonaj następujące czynności.

1. **Skonfiguruj konsumenta**. Jest to fragment kodu służący do obsługi komunikacji WebSocket w sposób bardzo podobny do tradycyjnych widoków HTTP. Zamierzamy napisać konsumenta odpowiedzialnego za czytanie komunikatów z kanału komunikacyjnego i zapisywanie ich do niego.
2. **Skonfiguruj routing**. Framework `Channels` oferuje klasy obsługi routingu pozwalające na łączenie konsumentów i tworzenie z nich stosu. Zamierzamy skonfigurować routing URL dla konsumenta czatu.
3. **Zaimplementuj klienta WebSocket**. Gdy uczestnik kursu zażąda dostępu do pokoju czatu, nawiążemy połączenie do gniazda WebSocket z przeglądarki i za pomocą JavaScript będziemy wysyłać wiadomości lub je odbierać.
4. **Włącz warstwę kanału komunikacyjnego**. Warstwa kanału komunikacyjnego pozwala komunikować się pomiędzy różnymi egzemplarzami aplikacji. Stanowią one przydatną część rozproszonej, działającej w czasie rzeczywistym aplikacji. Warstwę kanału komunikacyjnego skonfigurujemy z wykorzystaniem bazy danych Redis.

Zacznijmy od napisania własnego konsumenta do obsługi połączeń z WebSocket, odbierania i wysyłania wiadomości oraz rozłączania.

Pisanie konsumenta

Konsumenty w aplikacjach asynchronicznych są odpowiednikami widoków Django. Jak wspominałem wcześniej, konsumenty obsługują połączenia WebSocket w sposób bardzo podobny do tego, w jaki tradycyjne widoki obsługują żądania HTTP. Konsumenty są aplikacjami ASGI zdolnymi do obsługi wiadomości, powiadomień i innych informacji. W przeciwieństwie do widoków frameworka Django, konsumenty są budowane do obsługi długotrwałej komunikacji. Adresy URL są mapowane na konsumenty za pośrednictwem klas routingu, które pozwalają łączyć konsumenty i tworzyć ich stosy.

Spróbujmy zaimplementować prostego konsumenta, który jest zdolny do przyjmowania połączeń WebSocket i wysyła echo każdej otrzymanej wiadomości. Ta początkowa funkcjonalność pozwoli uczestnikowi kursu na wysyłanie wiadomości do konsumentów i odbieranie wysyłanych przez nich wiadomości.

Utwórz nowy plik wewnątrz katalogu aplikacji `chat` i nadaj mu nazwę *consumers.py*. Wprowadź do niego kod:

```
import json
from channels.generic.websocket import WebsocketConsumer


class ChatConsumer(WebsocketConsumer):
    def connect(self):
        # Akceptacja połączenia.
        self.accept()

    def disconnect(self, close_code):
        pass

    # Odbiór komunikatu z WebSocket.
    def receive(self, text_data):
        text_data_json = json.loads(text_data)
        message = text_data_json['message']
        # Wysłanie komunikatu do WebSocket.
        self.send(text_data=json.dumps({'message': message}))
```

To jest klasa `ChatConsumer`. Klasa ta dziedziczy po klasie `WebsocketConsumer` frameworka `Channels` i zawiera implementację podstawowego konsumenta `WebSocket`. W tym konsumencie implementujemy następujące metody.

- `connnect()`. Wywoływana po otrzymaniu nowego połączenia. Każde połączenie akceptujemy za pomocą wywołania `self.accept()`. Możemy również odrzucić połączenie za pomocą wywołania `self.close()`.
- `disconnect()`. Wywoływana, gdy następuje zamknięcie gniazda. Używamy w niej instrukcji `pass`, ponieważ nie trzeba implementować żadnego działania, kiedy klient zamyka połączenie.
- `receive()`. Wywoływana każdorazowo przy odbieraniu danych w WebSocket. Oczekujemy, że będziemy odbierać tekst jako `text_data` (mogą to być również dane binarne odbierane jako `binary_data`). Odbierane dane tekstowe są w formacie JSON. Z tego powodu korzystamy z metody `json.loads()` w celu

załadowania odebranych danych JSON do słownika Pythona. Sięgamy do klucza `message`, którego oczekujemy w odebranej strukturze JSON. Aby wykonać echo komunikatu, odsyłamy wiadomość do gniazda WebSocket za pomocą metody `self.send()`, przekształcając ją ponownie na format JSON za pomocą metody `json.dumps()`.

Wstępna wersja konsumenta `ChatConsumer` akceptuje dowolne połączenie WebSocket i wykonuje echo każdego odebranego komunikatu do klienta WebSocket. Zwróć uwagę, że konsument jeszcze nie rozgłasza wiadomości do innych klientów. Tę funkcjonalność zbudujemy później, kiedy zaimplementujemy warstwę kanału komunikacyjnego.

Najpierw ujawnijmy naszego konsumenta przez dodanie go do adresów URL projektu.

Routing

Należy zdefiniować adres URL w celu kierowania połączeń do zaimplementowanego konsumenta `ChatConsumer`. Framework `Channels` oferuje klasy obsługi routingu pozwalające na łączenie konsumentów i tworzenie z nich stosu w celu rozsyłania komunikatów na podstawie połączenia. Można je interpretować jako system routingu URL frameworka Django dla aplikacji asynchronicznych.

Utwórz nowy plik wewnątrz katalogu aplikacji czatu i nadaj mu nazwę *routing.py*. Wprowadź do niego kod:

```
from django.urls import re_path
from . import consumers


websocket_urlpatterns = [
    re_path(
        r'ws/chat/room/(?P<course_id>\d+)/$',
        consumers.ChatConsumer.as_asgi()
    ),
]
```

W tym kodzie mapujemy wzorzec URL za pomocą klasy `ChatConsumer` zdefiniowanej w pliku *chat/consumers.py*. Poza tym jest kilka szczegółów, które warto przeanalizować:

- Do zdefiniowania ścieżki z wykorzystaniem wyrażeń regularnych użyliśmy metody Django `re_path` zamiast metody `path()`. W przypadku gdy wewnętrzne routery są opakowane przez dodatkową warstwę middleware, routing URL frameworka `Channels` może nie działać poprawnie z trasami tworzonymi za pomocą metody `path()`, więc to podejście pomaga uniknąć wszelkich problemów.
- Adres URL zawiera parametr w postaci liczby całkowitej o nazwie `course_id`. Parametr ten będzie dostępny w zakresie konsumenta i pozwoli zidentyfikować pokój czatu kursu, z którym użytkownik się łączy.
- Wywołujemy metodę `as_asgi()` klasy konsumenta, aby uzyskać aplikację ASGI, która utworzy egzemplarz konsumenta dla każdego połączenia użytkownika. To zachowanie jest podobne do wykorzystania metody `as_view()` frameworka Django dla widoków opartych na klasach.

Uwaga

Dobrą praktyką jest poprzedzanie adresów URL WebSocket prefiksem `/ws/`, aby odróżnić je od adresów URL wykorzystywanych do standardowych, synchronicznych żądań HTTP. Stosowanie tej praktyki upraszcza również konfigurację produkcyjną, gdy serwer HTTP przekierowuje żądania na podstawie ścieżki.

Przeprowadź edycję globalnego pliku *asgi.py* znajdującego się obok pliku *settings.py* tak, aby miał następującą zawartość.

```
import os
from channels.auth import AuthMiddlewareStack
from channels.routing import ProtocolTypeRouter, URLRouter
from django.core.asgi import get_asgi_application

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'educa.settings')

django_asgi_app = get_asgi_application()

from chat.routing import websocket_urlpatterns

application = ProtocolTypeRouter({
    'http': django_asgi_app,
    'websocket': AuthMiddlewareStack(
        URLRouter(websocket_urlpatterns)
    ),
})
```

W tym kodzie dodajemy takie elementy:

- Nową trasę dla protokołu `websocket`. Skorzystaliśmy z klasy `URLRouter` w celu zmapowania połączeń `websocket` na wzorce URL zdefiniowane na liście `websocket_urlpatterns` modułu `chat.routing`.
- `AuthMiddlewareStack` jako funkcję opakowującą dla routera URL. Klasa `AuthMiddlewareStack` dostarczana przez framework `Channels` obsługuje standardowe uwierzytelnianie Django, kiedy szczegółowe dane użytkownika są przechowywane w sesji. Później do identyfikacji użytkownika wysyłającego wiadomość będziemy korzystać z egzemplarza użytkownika w zakresie konsumenta.

Zauważ, że import `websocket_urlpatterns` znajduje się poniżej wywołania funkcji `get_asgi_application()`. Jest to konieczne, by rejestr aplikacji wypełniał się przed zaimportowaniem kodu, który może importować modele ORM.

Po utworzeniu działającego konsumenta WebSocket, który jest dostępny za pośrednictwem adresu URL, możemy zaimplementować klienta WebSocket.

Implementacja klienta WebSocket

Dotychczas utworzyliśmy widok `course_chat_room` i odpowiadający mu szablon, które służą uczestnikom kursu do uzyskania dostępu do pokoju czatu kursu. Zaimplementowaliśmy konsumenta WebSocket dla serwera czatu i powiązaliśmy go za pomocą routingu URL.

Teraz zbudujemy klienta WebSocket umożliwiającego nawiązywanie połączeń z gniazdem WebSocket w szablonie pokoju czatu kursu i wysyłanie oraz odbieranie wiadomości.

Aby otworzyć i utrzymać połączenie w przeglądarce, zaimplementujesz klienta WebSocket za pomocą JavaScript. Za pomocą JavaScript będziesz również wchodzić w interakcje z **obiektowym modelem dokumentu** (ang. *Document Object Model* — DOM).

Wykonasz zadania związane z klientem WebSocket.

1. Po załadowaniu strony otworzysz połączenie WebSocket z serwerem.
2. Po otrzymaniu danych przez WebSocket będziesz dodawać wiadomości do kontenera HTML.
3. Aby wysyłać wiadomości przez WebSocket, gdy użytkownik kliknie przycisk *WYŚLIJ* lub naciśnie klawisz *Enter*, podłączysz do przycisku *Wyślij* obiekt nasłuchujący.

Zacznijmy od otwarcia połączenia WebSocket.

W szablonie *chat/room.html* aplikacji `chat` zmodyfikuj blok `domready` w taki sposób:

```
{% block include_js %}
  {{ course.id|json_script:"course-id" }}
{% endblock %}

{% block domready %}
  const courseId = JSON.parse(
    document.getElementById('course-id').textContent
  );
  const url = 'ws://' + window.location.host +
              '/ws/chat/room/' + courseId + '/';
  const chatSocket = new WebSocket(url);
{% endblock %}
```

W bloku `include_js` użyliśmy filtra szablonu `json_script`, aby bezpiecznie skorzystać z wartości `course.id` z JavaScript. Filtr szablonu `json_script` dostarczony przez Django generuje obiekt Pythona jako JSON opakowany w znacznik `<script>`, dzięki czemu można z niego bezpiecznie korzystać w kodzie JavaScript. Kod `{{ course. id|json_script: ↪"course-id" }}` jest renderowany jako `<script id="course-id" type="application/json" ↪>6</script>`. Ta wartość jest następnie pobierana w bloku `domready` poprzez parsowanie zawartości elementu z `id="course-id"` za pomocą metody `JSON.parse()`. Jest to bezpieczny sposób używania obiektów Pythona w kodzie JavaScript.

Uwaga

Filtr szablonu `json_script` bezpiecznie koduje obiekty Pythona jako JSON i bezpiecznie osadza je w znaczniku HTML `<script>`, co dzięki wyeliminowaniu potencjalnie szkodliwych znaków zabezpiecza przed atakami typu *cross-site scripting* (**XSS**).

Więcej informacji na temat filtra szablonów `json_script` znajdziesz na stronie *https://docs.djangoproject.com/en/5.0/ref/templates/builtins/#json-script*.

W bloku `domready` zdefiniowaliśmy adres URL z protokołem WebSocket, który ma postać *ws://* (lub *wss://* dla połączeń bezpiecznych WebSocket, analogicznie do *https://*). Adresy URL budujemy z wykorzystaniem bieżącej lokalizacji w przeglądarce, którą odczytujemy za pomocą wywołania `window.location.host`. Pozostała część adresu URL jest zbudowana z wykorzystaniem ścieżki wzorca URL pokoju czatu zdefiniowanej w pliku *routing.py* aplikacji czatu.

Zapisujemy cały adres URL zamiast jego budowania na podstawie nazwy, ponieważ framework `Channels` nie pozwala na odwracanie adresów URL. Używamy identyfikatora bieżącego kursu w celu wygenerowania adresu URL bieżącego kursu, a następnie zapisujemy adres URL w nowej zmiennej o nazwie `url`.

Następnie otwieramy połączenie WebSocket do zapisanego adresu URL za pomocą wywołania `new WebSocket(url)`. Do nowej zmiennej `chatSocket` przypisujemy egzemplarz utworzonego obiektu klienta WebSocket.

Utworzyłeś konsumenta WebSocket, dołączyłeś dla niego routing i zaimplementowałeś prostego klienta WebSocket. Teraz wypróbujemy wstępną wersję czatu.

Uruchom serwer programistyczny za pomocą polecenia:

```
python manage.py runserver
```

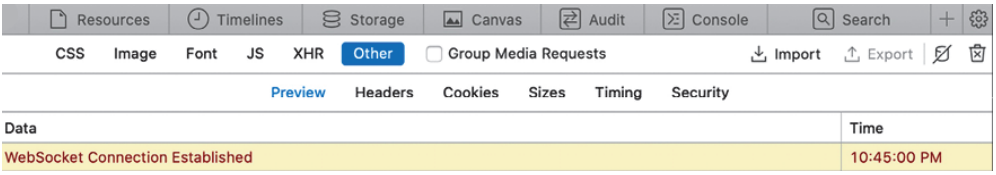
Otwórz w przeglądarce adres URL `http://127.0.0.1:8000/chat/room/1/`, zastępując `1` identyfikatorem istniejącego kursu z bazy danych. Przyjrzyj się danym wyjściowym w konsoli. Oprócz żądań HTTP `GET` strony i jej plików statycznych, powinieneś zobaczyć dwa wiersze `WebSocket HANDSHAKING` i `WebSocket CONNECT`, tak jak pokazałem poniżej:

```
HTTP GET /chat/room/1/ 200 [0.02, 127.0.0.1:57141]
WebSocket HANDSHAKING /ws/chat/room/1/ [127.0.0.1:57144]
WebSocket CONNECT /ws/chat/room/1/ [127.0.0.1:57144]
```

Serwer Daphne nasłuchuje przychodzących połączeń do gniazda z wykorzystaniem standardowego gniazda TCP. Proces uzgadniania jest pomostem z HTTP do WebSocket. W procesie uzgadniania ustalane są szczegóły połączenia, a każda ze stron może zamknąć połączenie przed zakończeniem negocjacji. Pamiętajmy, że aby zaakceptować dowolne połączenie w metodzie `connect()` klasy `ChatConsumer` zaimplementowanej w pliku *consumers.py* aplikacji czatu, używamy wywołania `self.accept()`. Ponieważ połączenie zostało zaakceptowane, widzimy w konsoli komunikat `WebSocket CONNECT`.

Jeśli do śledzenia połączeń sieciowych korzystasz z narzędzi programistycznych przeglądarki, możesz również za ich pomocą uzyskać informacje na temat ustanowionego połączenia WebSocket.

Powinno to wyglądać tak, jak pokazałem na rysunku 16.5.



Rysunek 16.5. Za pomocą narzędzi programistycznych przeglądarki można zobaczyć, że zostało nawiązane połączenie WebSocket

Teraz, gdy jesteśmy w stanie nawiązać połączenie z gniazdem WebSocket, możemy wykonywać działania za jego pośrednictwem. Zaimplementujemy metody obsługi popularnych zdarzeń, takich jak odbieranie komunikatów i zamykanie połączenia. Przeprowadź edycję szablonu *chat/room.html* aplikacji czatu i zmodyfikuj blok `domready` w taki sposób:

```
{% block domready %}
  const courseId = JSON.parse(
    document.getElementById('course-id').textContent
  );
  const url = 'ws://' + window.location.host +
              '/ws/chat/room/' + courseId + '/';
  const chatSocket = new WebSocket(url);

  chatSocket.onmessage = function(event) {
    const data = JSON.parse(event.data);
    const chat = document.getElementById('chat');

    chat.innerHTML += '<div class="message">' +
                      data.message + '</div>';
    chat.scrollTop = chat.scrollHeight;
  };

  chatSocket.onclose = function(event) {
    console.error('Chat socket closed unexpectedly');
  };
{% endblock %}
```

W powyższym kodzie zdefiniowaliśmy następujące zdarzenia dla klienta WebSocket.

- `onmessage`. Wyzwalane, gdy przez gniazdo WebSocket zostaną odebrane dane. Parsujemy wiadomość, której oczekiwanym formatem jest JSON i odczytujemy atrybut `message`. Następnie do elementu HTML `<div>` z identyfikatorem czatu dodajemy nowy element z odebraną wiadomością. Spowoduje to dodanie nowych wiadomości do logu czatu, a jednocześnie pozwoli zachować wszystkie poprzednie wiadomości dodane do logu. Aby upewnić się, że nowa wiadomość będzie widoczna, przewijamy element `<div>` logu czatu do dołu. Aby to uzyskać, przewijamy zawartość do łącznej wysokości logu czatu dostępnej do przewijania. Wartość tę można uzyskać za pomocą atrybutu `scrollHeight`.
- `onclose`. Wyzwalane przy zamykaniu połączenia z gniazdem WebSocket. Nie oczekujemy zamknięcia połączenia, dlatego gdy coś takiego się zdarzy, piszemy na konsoli komunikat „Nastąpiło nieoczekiwane zamknięcie gniazda czatu”.

W ten sposób zaimplementowaliśmy operację wyświetlania komunikatu w odpowiedzi na odebranie wiadomości. Trzeba jeszcze zaimplementować funkcjonalność wysyłania komunikatów do gniazda.

W szablonie *chat/room.html* aplikacji `chat` wprowadź u dołu bloku `domready` taki kod JavaScript:

```
const input = document.getElementById('chat-message-input');
const submitButton = document.getElementById('chat-message-submit');

submitButton.addEventListener('click', function(event) {
  const message = input.value;   if(message) {
```

```
        // Wysłanie wiadomości w formacie JSON.
        chatSocket.send(JSON.stringify({'message': message}));
        // Wyczyszczenie pola tekstowego.
        input.value = '';
        input.focus();
    }
});
```

W powyższym kodzie zdefiniowaliśmy funkcję nasłuchu dla zdarzenia kliknięcia przycisku *Prześlij*, który wybieramy za pomocą identyfikatora `chat-message-submit`. Po kliknięciu przycisku wykonujemy następujące działania.

1. Odczytujemy wiadomość wprowadzoną przez użytkownika na podstawie wartości elementu wprowadzania tekstu z identyfikatorem `chat-message-input`.
2. Za pomocą instrukcji `if(message)` sprawdzamy, czy wiadomość ma jakąś treść.
3. Jeśli użytkownik wprowadził wiadomość, formujemy treść w formacie JSON, np. `{'message': 'ciąg wprowadzony przez użytkownika'}`, za pomocą wywołania `JSON.stringify()`.
4. Wysyłamy zawartość w formacie JSON przez gniazdo WebSocket za pomocą wywołania metody `send()` klienta `chatSocket`.
5. Czyścimy zawartość pola tekstowego, ustawiając jego wartość na pusty ciąg znaków za pomocą instrukcji `input.value = ''`.
6. Przy użyciu instrukcji `input.focus()` zwracamy fokus do pola tekstowego. Dzięki temu użytkownik może natychmiast przystąpić do pisania nowej wiadomości.

Użytkownik może teraz wysyłać wiadomości poprzez wprowadzenie ich do pola tekstowego i kliknięcie przycisku *Prześlij*.

Aby poprawić komfort użytkownika, przekazujemy fokus do pola tekstowego natychmiast po załadowaniu strony, zatem użytkownik może bezpośrednio wpisywać do niego tekst. Przechwytujemy także klawisze wciskane na klawiaturze w celu identyfikacji klawisza *Enter*, a po kliknięciu przycisku *Prześlij* wyzwalamy zdarzenie `click`. Aby wysłać wiadomość, użytkownik może kliknąć przycisk albo nacisnąć klawisz *Enter*.

Przeprowadź edycję szablonu *chat/room.html* aplikacji `chat` i wprowadź u dołu bloku `domready` kod JavaScript:

```
input.addEventListener('keypress', function(event) {
    if (event.key === 'Enter') {
      // jeśli trzeba, anuluj domyślną operację
      event.preventDefault();
      // wyzwól zdarzenie kliknięcia przycisku
      submitButton.click();
    }
  });

  input.focus();
```

W tym kodzie zdefiniowaliśmy również funkcję dla zdarzenia naciśnięcia klawisza elementu `input`. Za każdym razem, gdy użytkownik naciśnie dowolny klawisz, wykonaj następujące czynności.

1. Sprawdź, czy naciśnięty klawisz to *Enter*.
2. Jeżeli naciśnięto klawisz *Enter*:
   A. Za pomocą wywołania `event.preventDefault()` blokujemy domyślne zachowanie tego klawisza.
   B. Jeśli użytkownik wciśnie klawisz *Enter*, wyzwalamy zdarzenie `click` przycisku *Prześlij*, aby wysłać wiadomość do gniazda WebSocket.

Na zewnątrz procedury obsługi zdarzeń, w głównym kodzie JavaScript bloku `domready`, przekazujemy za pomocą wywołania `input.focus()` fokus do elementu `input`. W ten sposób po załadowaniu DOM fokus zostanie ustawiony na elemencie wprowadzania tekstu, dzięki czemu użytkownik będzie mógł wpisać wiadomość.

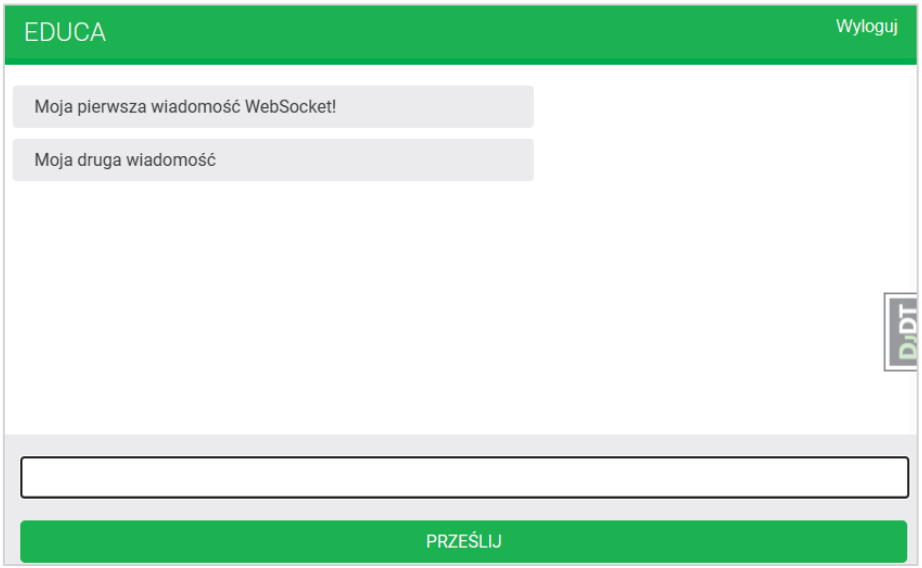
Kompletny blok `domready` szablonu *chat/room.html* powinien teraz wyglądać tak:

```
{% block domready %}
  const courseId = JSON.parse(
    document.getElementById('course-id').textContent
  );
  const url = 'ws://' + window.location.host +
              '/ws/chat/room/' + courseId + '/';
  const chatSocket = new WebSocket(url);

  chatSocket.onmessage = function(event) {
    const data = JSON.parse(event.data);
    const chat = document.getElementById('chat');

    chat.innerHTML += '<div class="message">' +
                      data.message + '</div>';
    chat.scrollTop = chat.scrollHeight;
  };

  chatSocket.onclose = function(event) {
    console.error('Nastąpiło nieoczekiwane zamknięcie gniazda czatu');
  };

  const input = document.getElementById('chat-message-input');
  const submitButton = document.getElementById('chat-message-submit');

  submitButton.addEventListener('click', function(event) {
    const message = input.value;
    if(message) {
      // Wysłanie wiadomości w formacie JSON.
      chatSocket.send(JSON.stringify({'message': message}));
      // Wyczyszczenie pola tekstowego.
     input.value = '';
      input.focus();
    }
  });

  input.addEventListener('keypress', function(event) {
    if (event.key === 'Enter') {
```

```
            // jeśli trzeba, anuluj domyślną operację
            event.preventDefault();
            // wyzwól zdarzenie kliknięcia przycisku
            submitButton.click();
        }
    });

    input.focus();
{% endblock %}
```

Otwórz w przeglądarce adres URL `http://127.0.0.1:8000/chat/room/1/`, zastępując `1` identyfikatorem istniejącego kursu z bazy danych. Zaloguj się jako użytkownik zapisany na kurs, wpisz tekst w polu tekstowym, a następnie kliknij przycisk *Prześlij* albo wciśnij klawisz *Enter*.

Zauważysz, że Twoja wiadomość wyświetli się w logu czatu (rysunek 16.6).



Rysunek 16.6. Strona pokoju czatu z wiadomościami przesłanymi przez gniazdo WebSocket

Doskonale! Przez gniazdo WebSocket została wysłana wiadomość, którą odebrał konsument `ChatConsumer` i odesłał ją z powrotem przez gniazdo WebSocket. Klient `chatSocket` otrzymał zdarzenie `message`. W odpowiedzi została wyzwolona funkcja `onmessage`, która spowodowała dodanie wiadomości do logu czatu.

Zaimplementowałeś funkcjonalność z konsumentem WebSocket oraz klientem WebSocket w celu nawiązania komunikacji klient-serwer i wysyłania oraz odbierania zdarzeń. Serwer czatu nie może jednak rozgłaszać wiadomości do innych klientów. Jeśli otworzysz drugą zakładkę przeglądarki i wpiszesz wiadomość, wiadomość ta nie pojawi się w pierwszej zakładce. W celu zbudowania komunikacji pomiędzy konsumentami należy utworzyć warstwę kanału komunikacyjnego.

Warstwa kanału komunikacyjnego

Warstwa kanału komunikacyjnego pozwala komunikować się pomiędzy różnymi egzemplarzami aplikacji. Warstwa kanału komunikacyjnego to mechanizm transportowy, który umożliwia wielu egzemplarzom aplikacji wzajemne komunikowanie się ze sobą oraz z innymi częściami Django.

W serwerze czatu planujemy utworzyć wiele egzemplarzy konsumenta `ChatConsumer` dla tego samego pokoju czatu kursu. Każdy uczestnik kursu dołączający do pokoju czatu utworzy w swojej przeglądarce klienta WebSocket, który otworzy połączenie z egzemplarzem konsumenta WebSocket. Potrzebujemy wspólnej warstwy kanału komunikacyjnego w celu dystrybucji wiadomości pomiędzy konsumentami.

Kanały komunikacyjne i grupy

Warstwy kanałów komunikacyjnych zapewniają dwie abstrakcje do zarządzania komunikacją; są to kanały i grupy.

- **Kanał**. Można go porównać do skrzynki odbiorczej, do której można przesyłać wiadomości, lub do kolejki zadań. Każdy kanał ma nazwę. Wiadomości są wysyłane do kanału przez każdego, kto zna nazwę kanału, a następnie przekazywane konsumentom, którzy nasłuchują na tym kanale.
- **Grupa**. Wiele kanałów może tworzyć grupę. Każda grupa ma nazwę. Każdy, kto zna nazwę grupy, może dodać do niej kanał lub go z niej usunąć. Posługując się nazwą grupy, można także wysłać wiadomość do wszystkich kanałów, które do niej należą.

Grupy kanałów wykorzystamy do zaimplementowania serwera czatu. Kiedy utworzymy grupy kanału dla pokoju rozmów każdego kursu, egzemplarze klasy `ChatConsumer` będą mogły komunikować się ze sobą.

Dodajmy do budowanego projektu warstwę kanału.

Konfiguracja warstwy kanału komunikacyjnego z wykorzystaniem Redis

Redis jest opcją zalecaną dla warstwy kanału komunikacyjnego, chociaż framework `Channels` zapewnia obsługę dla innych typów warstw. Redis spełnia funkcję magazynu komunikacyjnego dla warstwy kanału. Pamiętaj, że z bazy danych Redis korzystałeś już w rozdziale 7., „Śledzenie działań użytkownika”, rozdziale 10., „Rozbudowa sklepu internetowego”, i w rozdziale 14., „Renderowanie i buforowanie treści”.

Jeśli dotąd nie zainstalowałeś magazynu Redis, instrukcje jego instalacji znajdziesz w rozdziale 7., „Śledzenie działań użytkownika”.

Aby wykorzystać Redis w roli kanału komunikacyjnego, trzeba zainstalować pakiet `channels-redis`. Aby zainstalować pakiet `channel-redis` w swoim środowisku wirtualnym, użyj polecenia:

```
python -m pip install channels-redis==4.2.0
```

Zmodyfikuj plik *settings.py* w projekcie `educa` poprzez dodanie do niego poniższego fragmentu kodu.

```
CHANNEL_LAYERS = {
    'default': {
        'BACKEND': 'channels_redis.core.RedisChannelLayer',
        'CONFIG': {
            'hosts': [('127.0.0.1', 6379)],
        },
    },
}
```

Ustawienie `CHANNEL_LAYERS` określa konfigurację warstw kanałów komunikacyjnych dostępnych dla projektu. Zdefiniowaliśmy domyślną warstwę kanału, używając backendu `RedisChannelLayer` dostarczonego przez pakiet `channels-redis` i określiliśmy host `127.0.0.1` i port `6379`, na którym działa Redis.

Wypróbujmy warstwę kanału komunikacyjnego. Zainicjuj kontener Docker bazy danych Redis za pomocą polecenia:

```
docker run -it --rm --name redis -p 6379:6379 redis:7.2.4
```

Aby uruchomić polecenie w tle (w trybie odłączonym), możesz użyć opcji `-d`.

Otwórz powłokę Django za pomocą polecenia:

```
python manage.py shell
```

Aby sprawdzić, czy warstwa kanału może komunikować się z Redis, wpisz następujący kod, który wysyła wiadomość do testowego kanału o nazwie `test_channel` oraz go z niego odbiera.

```
>>> import channels.layers
>>> from asgiref.sync import async_to_sync
>>> channel_layer = channels.layers.get_channel_layer()
>>> async_to_sync(channel_layer.send)('test_channel', {'message': 'witaj'})
>>> async_to_sync(channel_layer.receive)('test_channel')
```

Powinieneś otrzymać taki wynik:

```
{"message": 'witaj'}
```

W powyższym kodzie wysłałeś wiadomość do kanału testowego przez warstwę kanału, a następnie odebrałeś ją z warstwy kanału. Warstwa kanału komunikacyjnego pomyślnie komunikuje się z systemem Redis.

Następnie dodamy do projektu warstwę kanału.

Aktualizacja konsumenta w celu rozgłaszania wiadomości

W celu skorzystania z warstwy kanału komunikacyjnego przeprowadzimy edycję konsumenta `ChatConsumer`. Użyjemy grupy kanału dla każdego pokoju rozmów kursu. Z tego powodu wykorzystamy identyfikator kursu w celu zbudowania nazwy grupy. Egzemplarze konsumenta `ChatConsumer` będą znały nazwę grupy i będą mogły się komunikować ze sobą.

W pliku *consumers.py* aplikacji `chat` zaimportuj funkcję `async_to_sync()` i zmodyfikuj metodę `connect()` klasy `ChatConsumer` w taki sposób:

```
import json
from asgiref.sync import async_to_sync
from channels.generic.websocket import WebsocketConsumer

class ChatConsumer(WebsocketConsumer):
    def connect(self):
        self.id = self.scope['url_route']['kwargs']['course_id']
        self.room_group_name = f'chat_{self.id}'
        # Dołączenie do grupy pokoju.
        async_to_sync(self.channel_layer.group_add)(
            self.room_group_name, self.channel_name
        )
        # Akceptacja połączenia.
        self.accept()
    # ...
```

W tym kodzie zaimportowaliśmy funkcję pomocniczą `async_to_sync()` w celu opakowania wywołań asynchronicznych metod warstwy kanału komunikacyjnego. `ChatConsumer` to synchroniczny konsument `WebsocketConsumer`, który wymaga wywoływania asynchronicznych metod warstwy kanału.

W nowej metodzie `connect()` wykonujemy następujące zadania.

1. Pobieramy identyfikator kursu z zakresu, aby poznać kurs, z którym jest powiązany pokój rozmów. Aby odczytać parametr `course_id` z adresu URL, używamy wywołania `self.scope['url_route'] ['kwargs ']['course_id']`. Z każdym konsumentem jest powiązany zakres zawierający informacje dotyczące jego połączenia, argumentów przekazanych w adresie URL oraz uwierzytelnionego użytkownika.
2. Nazwę grupy budujemy z wykorzystaniem identyfikatora kursu, z którym jest powiązana grupa. Zapamiętaj, że grupa kanału istnieje dla każdego pokoju rozmów kursu. Nazwę grupy zapisujemy w atrybucie `room_group_name` konsumenta.
3. Dołączamy do grupy poprzez dodanie do niej bieżącego kanału. Nazwę kanału uzyskujemy z atrybutu `channel_name` konsumenta. Aby dodać kanał do grupy, korzystamy z metody `group_add` warstwy kanału komunikacyjnego. W celu skorzystania z asynchronicznej metody kanału komunikacyjnego używamy wrappera `async_to_sync()`.
4. W celu zaakceptowania połączenia WebSocket korzystamy z metody `self.accept()`.

Gdy konsument `ChatConsumer` otrzyma nowe połączenie WebSocket, dodaje kanał do grupy związanej z kursem w swoim zakresie. Konsument może teraz odbierać wszystkie wiadomości wysyłane do grupy.

W tym samym pliku *consumers.py* zmodyfikuj metodę `disconnect()` klasy `ChatConsumer` w ten sposób:

```
class ChatConsumer(WebsocketConsumer):
    # …
    def disconnect(self, close_code):
        # Opuść grupę pokoju rozmów.
        async_to_sync(self.channel_layer.group_discard)(
            self.room_group_name, self.channel_name
        )
    # …
```

Kiedy połączenie zostanie zamknięte, wywołujemy metodę `group_discard()` warstwy kanału komunikacyjnego, aby opuścić grupę. W celu skorzystania z asynchronicznej metody kanału komunikacyjnego używamy wrappera `async_to_sync()`.

W tym samym pliku *consumers.py* zmodyfikuj metodę `receive()` klasy `ChatConsumer` w ten sposób:

```
class ChatConsumer(WebsocketConsumer):
    # …
    # Odbiór komunikatu z WebSocket.
    def receive(self, text_data):
        text_data_json = json.loads(text_data)
        message = text_data_json['message']
        # Wyślij komunikat do grupy pokoju.
        async_to_sync(self.channel_layer.group_send)(
            self.room_group_name,
            {
                'type': 'chat_message',
                'message': message,
            }
        )
```

Po odebraniu wiadomości z połączenia WebSocket zamiast wysyłania jej do powiązanego z połączeniem kanału teraz wysyłamy wiadomość do grupy. W tym celu wywołujemy metodę `group_send()` warstwy kanału komunikacyjnego. W celu skorzystania z asynchronicznej metody kanału komunikacyjnego używamy wrappera `async_to_sync()`. W zdarzeniu `sent` wysyłanym do grupy przekazujemy następujące informacje.

- `type`. Typ zdarzenia. Jest to specjalny klucz odpowiadający nazwie metody, która powinna być wywoływana przez konsumentów odbierających zdarzenie. Metodę tę można zaimplementować w konsumencie o takiej samej nazwie, co typ wiadomości. Dzięki temu metoda ta będzie wywołana każdorazowo po odebraniu wiadomości tego konkretnego typu.
- `message`. Właściwa wiadomość, którą wysyłamy.

W tym samym pliku *consumers.py* dodaj w klasie `ChatConsumer` nową metodę `chat_message()` tak, jak pokazałem poniżej.

```
class ChatConsumer(WebsocketConsumer):
    # …
    # Odebranie wiadomości z grupy pokoju rozmów.
    def chat_message(self, event):
        # Wysłanie komunikatu do WebSocket.
        self.send(text_data=json.dumps(event))
```

Nadaliśmy tej metodzie nazwę `chat_message()`, co odpowiada kluczowi `type` wysłanemu do grupy kanału po odebraniu wiadomości z gniazda WebSocket. Gdy do grupy zostanie wysłana wiadomość typu `chat_message`, otrzymają ją wszyscy konsumenci, którzy utworzyli subskrypcję do grupy i w odpowiedzi uruchomią metodę `chat_message()`. W metodzie `chat_message()` wysyłamy wiadomość odebraną z gniazda WebSocket.

Kompletny plik *consumers.py* ma następującą zawartość.

```
import json
from asgiref.sync import async_to_sync
from channels.generic.websocket import WebsocketConsumer


class ChatConsumer(WebsocketConsumer):
    def connect(self):
        self.id = self.scope['url_route']['kwargs']['course_id']
        self.room_group_name = f'chat_{self.id}'
        # Dołączenie do grupy pokoju.
        async_to_sync(self.channel_layer.group_add)(
            self.room_group_name, self.channel_name
        )
        # Akceptacja połączenia.
        self.accept()

    def disconnect(self, close_code):
        # Opuść grupę pokoju rozmów.
        async_to_sync(self.channel_layer.group_discard)(
            self.room_group_name, self.channel_name
        )

    # Odbiór komunikatu z WebSocket.
    def receive(self, text_data):
        text_data_json = json.loads(text_data)
        message = text_data_json['message']
        # Wyślij komunikat do grupy pokoju.
        async_to_sync(self.channel_layer.group_send)(
            self.room_group_name,
            {
                'type': 'chat_message',
                'message': message,
            }
        )

    # Odebranie wiadomości z grupy pokoju rozmów.
    def chat_message(self, event):
        # Wysłanie komunikatu do WebSocket.
        self.send(text_data=json.dumps(event))
```

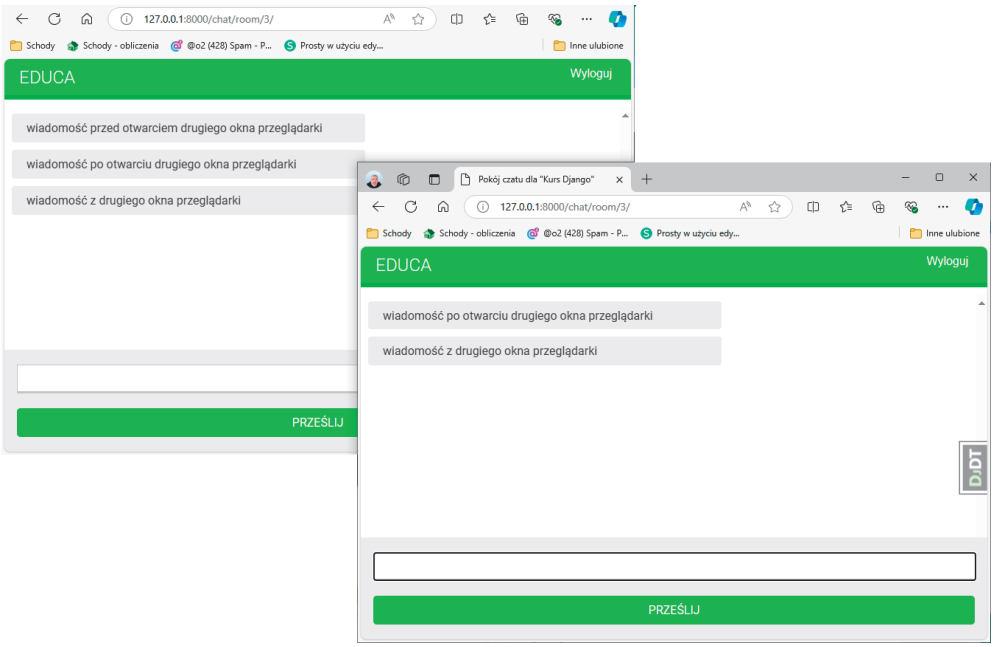
Właśnie zaimplementowałeś warstwę kanału w klasie `ChatConsumer`, która pozwala konsumentom na rozgłaszanie wiadomości i komunikowanie się ze sobą.

Uruchom serwer programistyczny za pomocą polecenia:

```
python manage.py runserver
```

Otwórz w przeglądarce adres URL `http://127.0.0.1:8000/chat/room/1/` zastępując `1` identyfikatorem istniejącego kursu z bazy danych. Napisz wiadomość i ją wyślij. Następnie otwórz drugie okno przeglądarki i spróbuj sięgnąć do tego samego adresu URL. Wyślij wiadomość z każdego okna przeglądarki.

Wynik pokazałem na rysunku 16.7.



Rysunek 16.7. Strona pokoju rozmów z wiadomościami wysłanymi z różnych okien przeglądarki

Widać, że pierwsza wiadomość jest wyświetlana tylko w pierwszym oknie przeglądarki. Gdy otworzysz drugie okno przeglądarki, wiadomości wysłane w dowolnym oknie przeglądarki zostaną wyświetlone w obu oknach. Gdy otworzysz nowe okno przeglądarki i sięgniesz do adresu URL pokoju rozmów, zostanie nawiązane nowe połączenie WebSocket pomiędzy klientem JavaScript w przeglądarce a konsumentem WebSocket na serwerze. Każdy kanał zostanie dodany do grupy powiązanej z identyfikatorem kursu przekazanym w adresie URL do konsumenta. Wiadomości są wysyłane do grupy i odbierane przez wszystkich konsumentów.

W kolejnym kroku zamierzamy wzbogacić wiadomości o dodatkowy kontekst.

Dodawanie kontekstu do wiadomości

Teraz, gdy można wymieniać wiadomości pomiędzy wszystkimi użytkownikami w pokoju rozmów, warto dodać funkcjonalność pozwalającą zobaczyć, kto wysłał określone wiadomości oraz kiedy zostały wysłane. Spróbujmy dodać kontekst do wiadomości.

W pliku *consumers.py* aplikacji `chat` wprowadź następujące zmiany.

```
import json
from asgiref.sync import async_to_sync
from channels.generic.websocket import WebsocketConsumer
from django.utils import timezone


class ChatConsumer(WebsocketConsumer):
    def connect(self):
        self.user = self.scope['user']
        self.id = self.scope['url_route']['kwargs']['course_id']
        self.room_group_name = f'chat_{self.id}'
        # Dołączenie do grupy pokoju.
        async_to_sync(self.channel_layer.group_add)(
            self.room_group_name, self.channel_name
        )
        # Akceptacja połączenia.
        self.accept()

    def disconnect(self, close_code):
        # Opuść grupę pokoju rozmów.
        async_to_sync(self.channel_layer.group_discard)(
            self.room_group_name, self.channel_name
        )

    # Odbiór komunikatu z WebSocket.
    def receive(self, text_data):
        text_data_json = json.loads(text_data)
        message = text_data_json['message']
        now = timezone.now()
        # Wyślij komunikat do grupy pokoju.
        async_to_sync(self.channel_layer.group_send)(
            self.room_group_name,
            {
                'type': 'chat_message',
                'message': message,
                'user': self.user.username,
                'datetime': now.isoformat(),
            }
        )

    # Odebranie wiadomości z grupy pokoju rozmów.
    def chat_message(self, event):
        # Wysłanie komunikatu do WebSocket.
        self.send(text_data=json.dumps(event))
```

Zaimportowałeś moduł `timezone` z frameworka Django. W metodzie `connect()` konsumenta pobrałeś aktualnego użytkownika z zakresu za pomocą wywołania `self.scope['user']` i zapisałeś go w nowym atrybucie `user` konsumenta.

Gdy konsument odbierze wiadomość przez gniazdo WebSocket, pobiera aktualny czas za pomocą wywołania `timezone.now()` i przekazuje w zdarzeniu wysyłanym do grupy kanału bieżącego użytkownika wartość `datetime` w formacie ISO 8601.

Przeprowadź edycję szablonu *chat/room.html* aplikacji czatu i dodaj do bloku `include_js` następujący wiersz oznaczony pogrubieniem.

```
{% block include_js %}
  {{ course.id|json_script:"course-id" }}
  {{ request.user.username|json_script:"request-user" }}
{% endblock %}
```

Korzystając z szablonu `json_script`, możesz bezpiecznie wyświetlić nazwę użytkownika żądającego, aby użyć jej w kodzie JavaScript.

W bloku `domready` szablonu *chat/room.html* dodaj poniższe wiersze wyróżnione pogrubioną czcionką:

```
{% block domready %}
  const courseId = JSON.parse(
    document.getElementById('course-id').textContent
  );
  const requestUser = JSON.parse(
    document.getElementById('request-user').textContent
  );
  # ...

{% endblock %}
```

W nowym kodzie bezpiecznie sparsowaliśmy dane elementu z identyfikatorem `request-user` i zapisaliśmy je w stałej `requestUser`.

Następnie w bloku `domready` znajdź wiersze:

```
const data = JSON.parse(event.data);
const chat = document.getElementById('chat');

chat.innerHTML += '<div class="message">' +
                  data.message + '</div>';
chat.scrollTop = chat.scrollHeight;
```

Zastąp te wiersze poniższym fragmentem kodu.

```
const data = JSON.parse(event.data);
const chat = document.getElementById('chat');

const dateOptions = {hour: 'numeric', minute: 'numeric', hour12: true};
const datetime = new Date(data.datetime).toLocaleString('en', dateOptions);
const isMe = data.user === requestUser;
const source = isMe ? 'me' : 'other';
const name = isMe ? 'Me' : data.user;

chat.innerHTML += '<div class="message ' + source + '">' +
                  '<strong>' + name + '</strong> ' +
                  '<span class="date">' + datetime + '</span><br>' +
                  data.message + '</div>';
chat.scrollTop = chat.scrollHeight;
```

W tym kodzie zaimplementowaliśmy następujące zmiany.

1. Wykonaliśmy konwersję wartości `datetime` odebranej w wiadomości na obiekt JavaScript `Date` i sformatowaliśmy ją odpowiednio do bieżących ustawień międzynarodowych.
2. Porównaliśmy nazwę użytkownika otrzymaną w wiadomości z dwiema różnymi stałymi, które pomagają zidentyfikować użytkownika.
3. Stała `source` pobiera wartość `me`, jeśli użytkownik wysyłający wiadomość to bieżący użytkownik, lub `other` w przeciwnym przypadku.
4. Jeżeli użytkownik wysyłający wiadomość jest bieżącym użytkownikiem, stała `name` przyjmuje wartość `Me`, w przeciwnym razie przyjmuje nazwę użytkownika wysyłającego wiadomość. Wykorzystamy ją w celu wyświetlenia nazwy użytkownika wysyłającego wiadomość.
5. Następnie skorzystaliśmy z wartości `source` jako klasy głównego elementu `<div>` w celu odróżnienia wiadomości wysłanych przez bieżącego użytkownika od wiadomości wysłanych przez innych. Na podstawie atrybutu `class` zastosowaliśmy różne style CSS. Te style CSS są zadeklarowane w statycznym pliku *css/base.css*.
6. Nazwę użytkownika i wartość `datetime` wykorzystujemy w wiadomości dołączanej do logu czatu.

Otwórz w przeglądarce adres URL `http://127.0.0.1:8000/chat/room/1/` zastępując `1` identyfikatorem istniejącego kursu z bazy danych. Zaloguj się jako użytkownik zapisany na kurs, napisz wiadomość i ją wyślij.

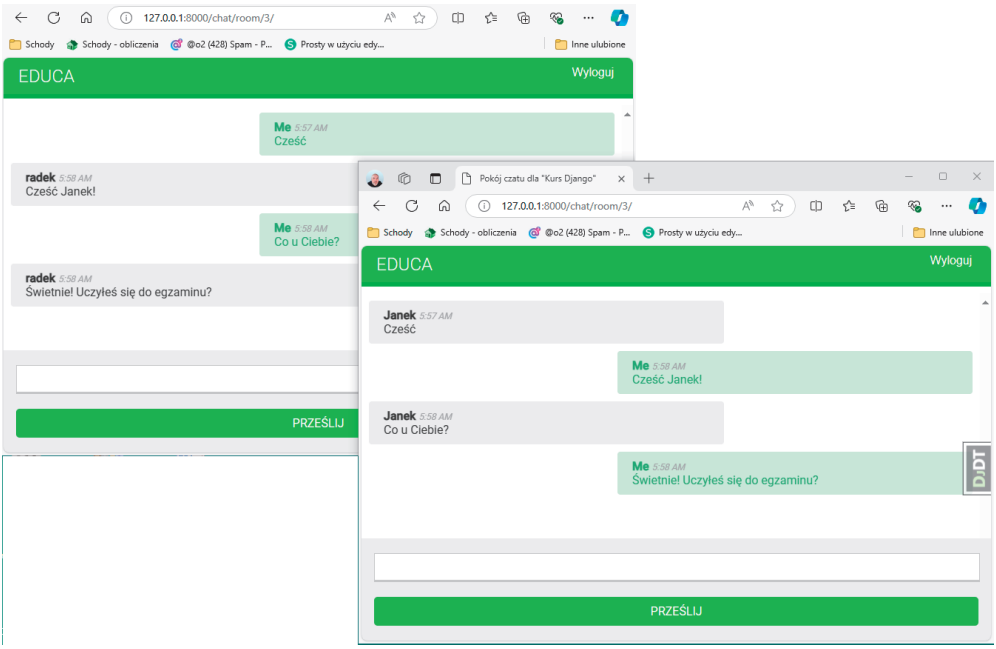
Następnie, aby uniemożliwić korzystanie z tej samej sesji, otwórz drugie okno przeglądarki w trybie incognito. Zaloguj się jako inny użytkownik zapisany na ten sam kurs i wyślij wiadomość.

Będziesz mógł wymieniać wiadomości z wykorzystaniem dwóch różnych użytkowników, obserwować użytkownika i czas. Wiadomości są wyświetlane z wyraźnym rozróżnieniem tych, które wysłał bieżący użytkownik, od tych, które wysłali inni. Przykładową konwersację pomiędzy dwoma użytkownikami pokazałem na rysunku 16.8.

Doskonale! Korzystając z frameworka `Channels` zbudowałeś działającą aplikację do czatowania w czasie rzeczywistym. Z kolejnego podrozdziału dowiesz się, jak usprawnić konsumenta czatu, tak by stał się w pełni asynchroniczny.

Modyfikacja konsumenta w celu uzyskania pełnej asynchroniczności

Zaimplementowana klasa `ChatConsumer` dziedziczy po klasie bazowej `WebsocketConsumer`, która jest synchroniczna. Synchroniczne konsumenty działają w taki sposób, że każde żądanie musi być przetwarzane po kolei, jedno po drugim. Synchroniczne konsumenty pozwalają na wygodny dostęp do modeli Django i wywoływanie standardowych, synchronicznych funkcji wejścia-wyjścia. Konsumenty asynchroniczne działają jednak wydajniej ze względu na ich zdolność do wykonywania operacji bez blokowania, tzn. przechodzenia do nowego zadania bez oczekiwania na zakończenie zadania poprzedniego. Nie



Rysunek 16.8. Strona pokoju rozmów z wiadomościami wysłanymi przez dwóch różnych użytkowników

wymagają dodatkowych wątków podczas obsługi żądań, co skraca czas oczekiwania i zwiększa możliwość skalowania w celu jednoczesnego obsłużenia większej liczby użytkowników i żądań.

Dzięki temu, że używamy asynchronicznych funkcji warstwy kanału komunikacyjnego, możemy łatwo przekształcić klasę `ChatConsumer` w taki sposób, by była asynchroniczna.

Przeprowadź edycję pliku *consumers.py* aplikacji `chat` i zaimplementuj następujące zmiany.

```
import json
from channels.generic.websocket import AsyncWebsocketConsumer
from django.utils import timezone

class ChatConsumer(AsyncWebsocketConsumer):
    async def connect(self):
        self.user = self.scope['user']
        self.id = self.scope['url_route']['kwargs']['course_id']
        self.room_group_name = 'chat_%s' % self.id
        # Dołączenie do grupy pokoju.
        await self.channel_layer.group_add(
            self.room_group_name, self.channel_name
        )
        # Akceptacja połączenia.
        await self.accept()

    async def disconnect(self, close_code):
        # leave room group
        await self.channel_layer.group_discard(
```

```
            self.room_group_name, self.channel_name
        )

    # Odbiór wiadomości z WebSocket.
    async def receive(self, text_data):
        text_data_json = json.loads(text_data)
        message = text_data_json['message']
        now = timezone.now()
        # Wysłanie wiadomości do grupy pokoju rozmów.
        await self.channel_layer.group_send(
            self.room_group_name,
            {
                'type': 'chat_message',
                'message': message,
                'user': self.user.username,
                'datetime': now.isoformat(),
            }
        )
    # Odebranie wiadomości z grupy pokoju rozmów.
    async def chat_message(self, event):
        # send message to WebSocket
        await self.send(text_data=json.dumps(event))
```

Zaimplementowałeś następujące zmiany.

1. Konsument `ChatConsumer` dziedziczy teraz po klasie `AsyncWebsocketConsumer` co pozwala na implementację wywołań asynchronicznych.
2. Zmodyfikowałeś definicję wszystkich metod z `def` na `async def`.
3. W celu wywoływania asynchronicznych funkcji, które wykonują operacje wejścia-wyjścia, wykorzystałeś słowo kluczowe `await`.
4. Do wywoływania metod warstwy kanału komunikacyjnego nie używasz już pomocniczej funkcji `async_to_sync()`.

Ponownie otwórz adres URL `http://127.0.0.1:8000/chat/room/1/` w dwóch różnych oknach przeglądarki i sprawdź, czy serwer czatu nadal działa. Serwer czatu jest teraz w pełni asynchroniczny!

W następnym podrozdziale zaimplementujesz historię czatu dzięki zapisywaniu wiadomości w bazie danych.

Utrwalanie wiadomości w bazie danych

Przystępujemy do rozbudowy aplikacji `chat` przez dodanie mechanizmu utrwalania wiadomości. Opracujemy funkcjonalność do przechowywania wiadomości w bazie danych, co pozwoli na zaprezentowanie historii czatu użytkownikom dołączającym do pokoju rozmów. Ta funkcjonalność jest niezbędna w aplikacjach działających w czasie rzeczywistym, kiedy konieczne jest wyświetlanie zarówno bieżących, jak i wcześniej wygenerowanych danych. Weźmy na przykład aplikację do obrotu akcjami: użytkownicy po zalogowaniu powinni zobaczyć nie tylko bieżące wartości akcji, ale także wartości historyczne od momentu otwarcia giełdy.

Aby zaimplementować funkcjonalność historii czatu, wykonamy takie czynności:

1. Stworzymy model Django do przechowywania wiadomości czatu i dodamy go do strony administracyjnej.
2. Zmodyfikujemy konsumenta WebSocket tak, aby utrwalał wiadomości.
3. Będziemy pobierać historię czatu w celu wyświetlania najnowszych wiadomości po dołączeniu użytkownika do pokoju rozmów.

Pracę rozpoczynamy od opracowania modelu.

Utworzenie modelu dla wiadomości czatu

W pliku *models.py* aplikacji `chat` umieść poniższe wiersze kodu wyróżnione pogrubioną czcionką.

```
from django.conf import settings
from django.db import models


class Message(models.Model):
    user = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.PROTECT,
        related_name='chat_messages'
    )
    course = models.ForeignKey(
        'courses.Course',
        on_delete=models.PROTECT,
        related_name='chat_messages'
    )
    content = models.TextField()
    sent_on = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f'{self.user} on {self.course} at {self.sent_on}'
```

Jest to model danych służący do utrwalania wiadomości czatu. Przyjrzyjmy się polom modelu `Message`.

- `user`. Obiekt `User` reprezentujący użytkownika, który napisał wiadomość. Jest to pole klucza obcego (`ForeignKey`), ponieważ definiuje relację typu „wiele do jednego”: użytkownik może wysłać wiele wiadomości, ale każda wiadomość jest przesłana tylko przez jednego użytkownika. Dzięki użyciu dla parametru `on_delete` słowa kluczowego `PROTECT` obiekt `User` nie może zostać usunięty, jeśli istnieją powiązane z nim wiadomości.
- `course`. Relacja z obiektem `Course`. Każda wiadomość należy do pokoju czatu danego kursu. Dzięki użyciu dla parametru `on_delete` słowa kluczowego `PROTECT` obiekt `Course` nie może zostać usunięty, jeśli istnieją powiązane z nim wiadomości.
- `content`. Pole `TextField` do przechowywania treści wiadomości.
- `sent_on`. Pole `DateTimeField` do przechowywania daty i godziny zapisania obiektu wiadomości po raz pierwszy.

Aby wygenerować migracje bazy danych dla aplikacji `chat`, wykonaj w wierszu powłoki polecenie:

```
python manage.py makemigrations chat
```

Powinieneś otrzymać taki wynik:

```
Migrations for 'chat':
    chat/migrations/0001_initial.py
        - Create model Message
```

Zastosuj utworzoną migrację do bazy danych za pomocą polecenia:

```
python manage.py migrate
```

Wyświetli się wynik, który kończy się wierszem:

```
Applying chat.0001_initial… OK
```

Teraz baza danych jest zsynchronizowana z nowym modelem. Dodajmy model `Message` do witryny administracyjnej.

Dodanie modelu Message do witryny administracyjnej

W pliku *admin.py* aplikacji `chat` zarejestruj model `Message` w witrynie administracyjnej, co pokazałem poniżej. Nowy kod został wyróżniony pogrubioną czcionką.

```
from django.contrib import admin
from chat.models import Message

@admin.register(Message)
class MessageAdmin(admin.ModelAdmin):
    list_display = ['sent_on', 'user', 'course', 'content']
    list_filter = ['sent_on', 'course']
    search_fields = ['content']
    raw_id_fields = ['user', 'course']
```

Uruchom serwer programistyczny i przejdź w przeglądarce internetowej pod adres `http://127.0.0.1:8000/admin/`. Na stronie administracyjnej powinieneś zobaczyć blok *CHAT* i sekcję *Messages* (rysunek 16.9).

Rysunek 16.9. Aplikacja chat i sekcja Messages na stronie administracyjnej

W kolejnym punkcie pokażę, jak zapisywać wiadomości przesłane przez użytkowników do bazy danych.

Zapisywanie wiadomości w bazie danych

Zmodyfikujemy konsumenta `WebSocket` w taki sposób, aby wszystkie wiadomości otrzymane za pośrednictwem `WebSocket` były utrwalane w bazie danych.

W pliku *consumers.py* aplikacji `chat` umieść poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
import json
from channels.generic.websocket import AsyncWebsocketConsumer
from django.utils import timezone
from chat.models import Message

class ChatConsumer(AsyncWebsocketConsumer):
    # ...
    async def persist_message(self, message):
        # Wysłanie wiadomości do gniazda WebSocket.
        await Message.objects.acreate(
            user=self.user, course_id=self.id, content=message
        )

    # Odbiór wiadomości z WebSocket.
    async def receive(self, text_data):
        text_data_json = json.loads(text_data)
        message = text_data_json['message']
        now = timezone.now()
        # Wysłanie wiadomości do grupy pokoju rozmów.
        await self.channel_layer.group_send(
            self.room_group_name,
            {
                'type': 'chat_message',
                'message': message,
                'user': self.user.username,
                'datetime': now.isoformat(),
            },
        )
        # Utrwalenie wiadomości.
        await self.persist_message(message)
    # ...
```

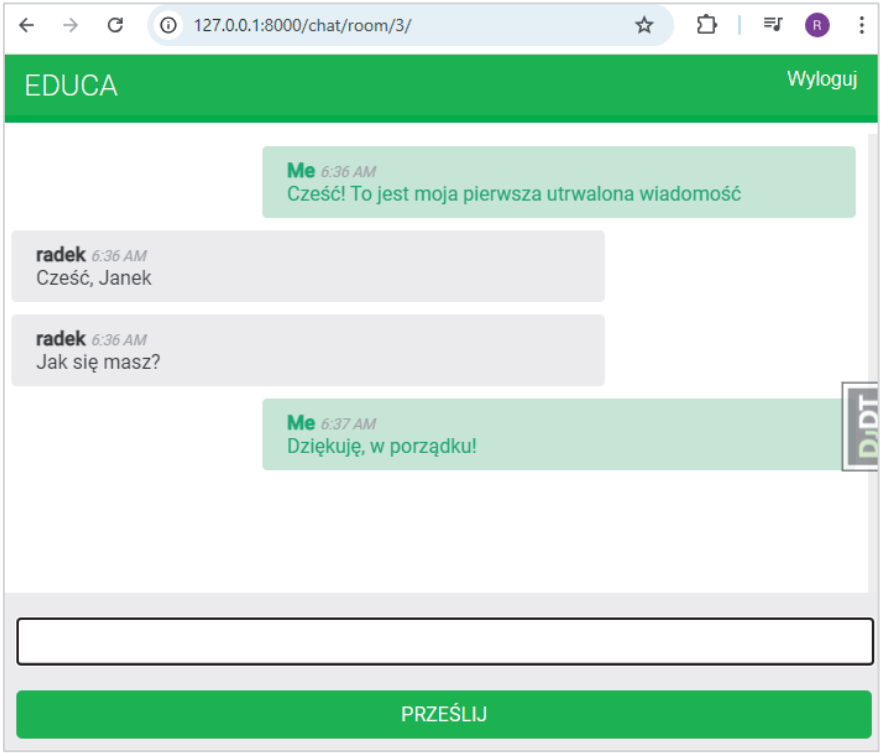
W powyższym kodzie dodajemy do klasy `ChatConsumer` asynchroniczną metodę `persist_message()`. Ta metoda przyjmuje parametr `message` i tworzy w bazie danych obiekt `Message` z podaną wiadomością, powiązanym uwierzytelnionym użytkownikiem i identyfikatorem obiektu `Course`, do którego należy pokój rozmów grupowych. Ponieważ obiekt `ChatConsumer` jest w pełni asynchroniczny, używamy metody `acreate()` obiektu `QuerySet`, będącej asynchroniczną wersją metody `create()`. Więcej informacji o sposobie pisania asynchronicznych zapytań za pomocą Django ORM można przeczytać na stronie *https://docs.djangoproject.com/en/5.0/topics/db/queries/#asynchronousqueries*.

Metodę `persist_message()` wywołujemy asynchronicznie w metodzie `receive()`, wykonywanej po odebraniu wiadomości przez konsumenta za pośrednictwem WebSocket.

Uruchom serwer programistyczny i otwórz w przeglądarce adres URL `http://127.0.0.1:8000/chat/room/1/`, zastępując `1` identyfikatorem istniejącego kursu z bazy danych. Zaloguj się jako użytkownik zapisany na kurs, napisz wiadomość i ją wyślij.

Następnie, aby uniemożliwić korzystanie z tej samej sesji, otwórz drugie okno przeglądarki w trybie incognito. Zaloguj się jako inny użytkownik, również zapisany na ten sam kurs, i także wyślij kilka wiadomości.

Przykład wiadomości wysłanych przez dwóch różnych użytkowników jest pokazany na rysunku 16.10.

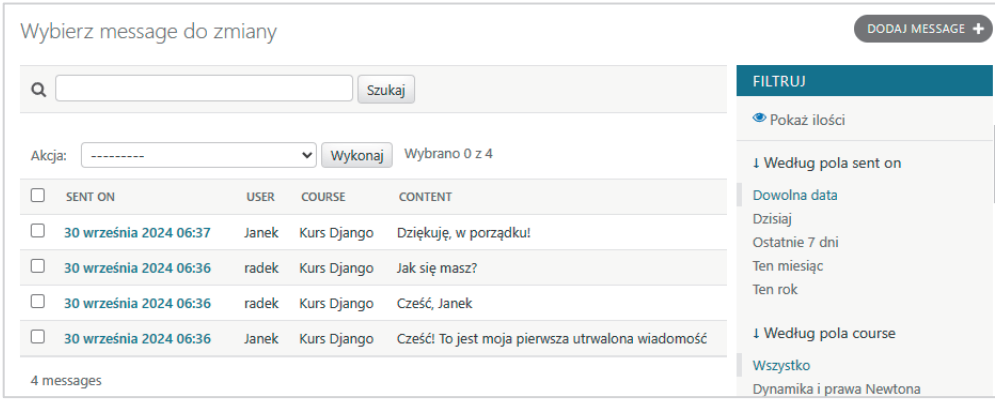


Rysunek 16.10. Przykład czatu z wiadomościami wysłanymi przez dwóch różnych użytkowników

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/admin/chat/message/`. Na stronie administracyjnej powinny wyświetlić się wysłane wiadomości, jak pokazałem na rysunku 16.11.

Wszystkie wiadomości są teraz utrwalane w bazie danych.

Należy pamiętać, że wiadomości mogą zawierać złośliwy kod, na przykład fragmenty JavaScript. W szablonie nie oznaczamy wiadomości jako bezpiecznych, co zapewnia początkową warstwę ochrony przed złośliwą zawartością. Aby jednak poprawić bezpieczeństwo, warto rozważyć wprowadzenie mechanizmu unieszkodliwiania wiadomości przed zapisaniem ich w bazie danych. Niezawodną opcją unieszkodliwiania zawartości



Rysunek 16.11. Widok wiadomości przechowywanych w bazie danych w witrynie administracyjnej

jest pakiet `nh3`. Więcej informacji na temat pakietu `nh3` można znaleźć na stronie *https://docs.djangoproject.com/en/5.0/ref/contrib/postgres/search.* Dodatkowo jest dostępny pakiet `django-nh3`, który zapewnia integrację z frameworkiem Django oraz niestandardowe pola modelu `nh3` i pola formularzy. Więcej informacji na jego temat można znaleźć na stronie *https://github.com/marksweb/django-nh3*.

Teraz gdy pełna historia czatów jest przechowywana w bazie danych, pokażę, jak zaprezentować użytkownikom najnowsze wiadomości w historii czatów po dołączeniu do pokoju rozmów.

Wyświetlanie historii czatu

Po dołączeniu użytkownika do pokoju czatu kursu wyświetlimy pięć ostatnich wiadomości z historii czatu. Zapewni to użytkownikom natychmiastowy kontekst dla trwających rozmów.

W pliku *views.py* aplikacji `chat` umieść w widoku `course_chat_room` poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
@login_required
def course_chat_room(request, course_id):
    try:
        # Pobranie kursu o identyfikatorze zgodnym z identyfikatorem kursu, do którego dołączył
        # bieżący użytkownik.
        course = request.user.courses_joined.get(id=course_id)
    except Course.DoesNotExist:
        # Użytkownik nie jest uczestnikiem tego kursu lub kurs nie istnieje.
        return HttpResponseForbidden()
    # retrieve chat history
    latest_messages = course.chat_messages.select_related(
        'user'
    ).order_by('-id')[:5]
    latest_messages = reversed(latest_messages)
    return render(
```

```
            request,
            'chat/room.html',
            {'course': course, 'latest_messages': latest_messages}
        )
```

Pobieramy wiadomości czatu związane z kursem i używamy funkcji `select_related()`, aby w tym samym zapytaniu pobrać powiązanego użytkownika. Dzięki temu nie będzie trzeba generować dodatkowych zapytań SQL, aby uzyskać dostęp do nazwy użytkownika w celu wyświetlenia jej obok każdej wiadomości. Mechanizm ORM frameworka Django nie obsługuje ujemnego indeksowania, więc pobieramy pierwszych pięć wiadomości w odwrotnej kolejności chronologicznej i używamy funkcji `reversed()`, by zmienić ich kolejność na chronologiczną.

Teraz dodamy historię czatu do szablonu pokoju rozmów. W szablonie *chat/room.html* dodaj następujące wiersze wyróżnione pogrubioną czcionką.

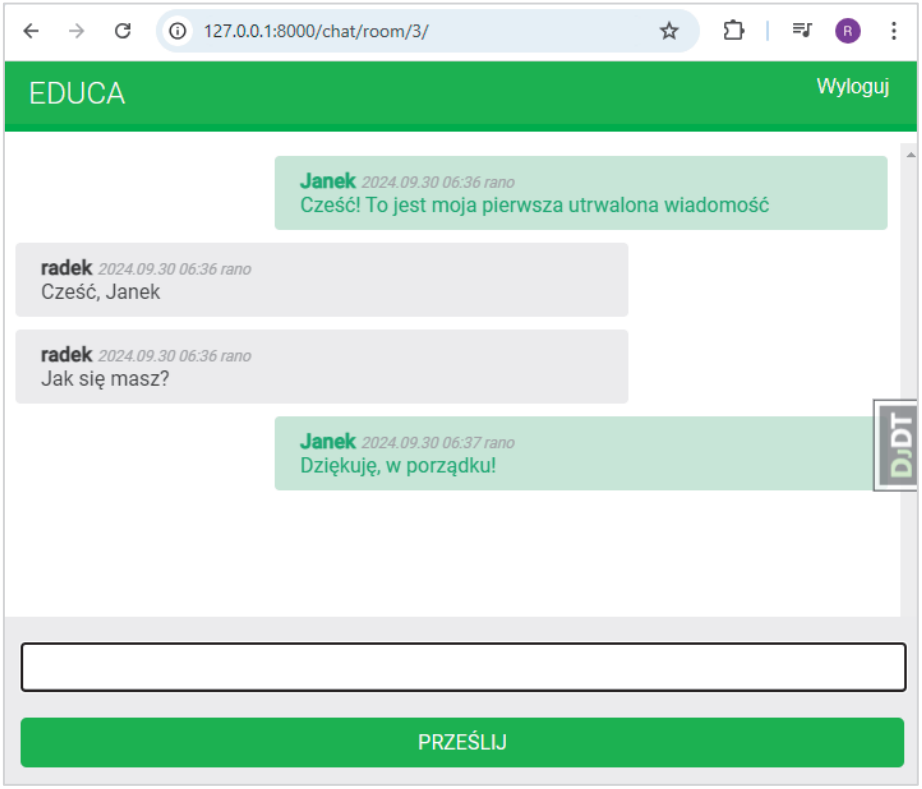
```
# …
{% block content %}
  <div id="chat">
    {% for message in latest_messages %}
      <div class="message {% if message.user == request.user %}me{% else %} other
      ↪{% endif %}">
        <strong>{{ message.user.username }}</strong>
        <span class="date">
          {{ message.sent_on|date:"Y.m.d H:i A" }}
        </span>
        <br>
        {{ message.content }}
      </div>
    {% endfor %}
  </div>
  <div id="chat-input">
    <input id="chat-message-input" type="text">
    <input id="chat-message-submit" type="submit" value="Prześlij">
  </div>
{% endblock %}
# …
```

Otwórz w przeglądarce adres URL `http://127.0.0.1:8000/chat/room/1/`, zastąpiwszy `1` identyfikatorem istniejącego kursu z bazy danych. Powinieneś teraz zobaczyć najnowsze wiadomości, tak jak pokazałem na rysunku 16.12.

Po dołączeniu do pokoju rozmów użytkownicy mogą teraz zobaczyć najnowsze wiadomości w historii czatu. W kolejnym podrozdziale dodamy link do menu umożliwiający wejście do pokoju czatu kursu.

Integracja aplikacji czatu z istniejącymi widokami

Serwer czatu jest teraz w pełni zaimplementowany, a uczestnicy kursu mogą się ze sobą komunikować. Dodajmy link dla uczestników kursów pozwalający im dołączyć do pokoju rozmów.



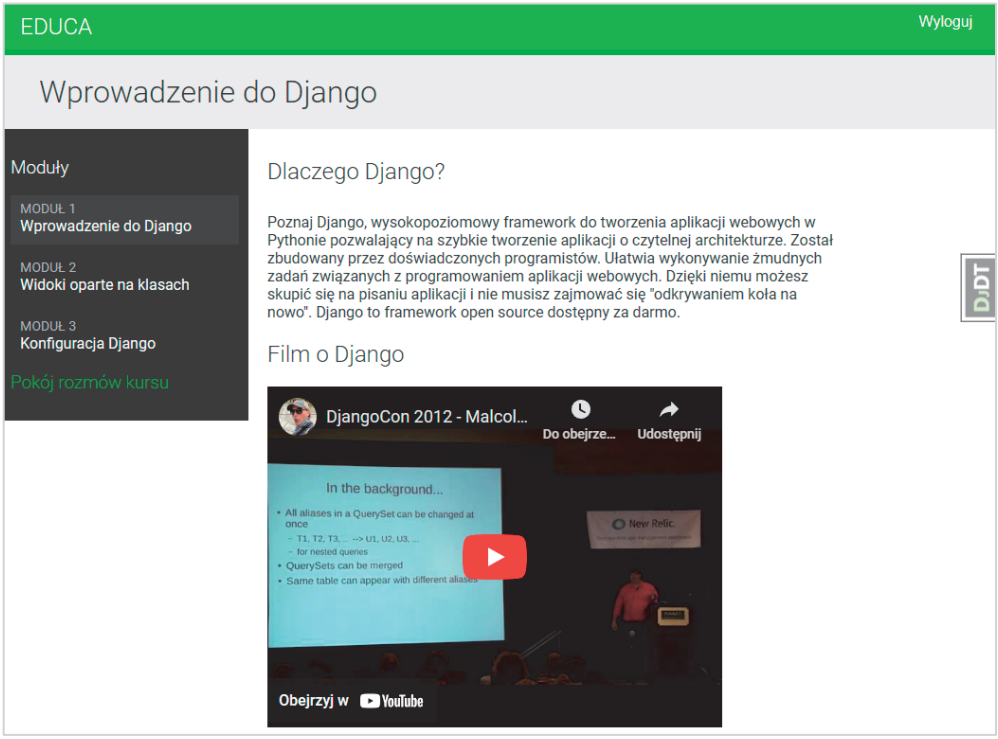
Rysunek 16.12. Pokój czatu, w którym po dołączeniu wyświetlają się najnowsze wiadomości

Przeprowadź edycję szablonu *students/course/detail.html* aplikacji `educa` i dodaj kod elementu HTML `<h3>` u dołu elementu `<div class="contents">`.

```
<div class="contents">
  ...
  <h3>
    <a href="{% url "chat:course_chat_room" object.id %}">
      Pokój rozmów kursu
    </a>
  </h3>
</div>
```

Otwórz przeglądarkę i w celu obejrzenia zawartości kursu wejdź na stronę dowolnego kursu, na który użytkownik jest zapisany. W ramce bocznej pojawi się teraz link *Pokój rozmów kursu* prowadzący do widoku pokoju rozmów. Kliknięcie tego linka prowadzi do strony pokoju rozmów (rysunek 16.13).

Gratulacje! Pomyślnie zbudowałeś swoją pierwszą aplikację asynchroniczną przy użyciu frameworka `Channels` Django.



Rysunek 16.13. Strona szczegółów kursu zawierająca link do pokoju rozmów

Podsumowanie

W tym rozdziale dowiedziałeś się, jak utworzyć serwer czatu z wykorzystaniem frameworka `Channels`. Zaimplementowaliśmy konsumenta i klienta `WebSocket`. Umożliwiliśmy także komunikację pomiędzy konsumentami za pomocą warstwy kanału komunikacyjnego z wykorzystaniem bazy danych Redis oraz zmodyfikowaliśmy konsumenta w taki sposób, aby był w pełni asynchroniczny. Ponadto zaimplementowaliśmy funkcjonalność utrwalania wiadomości czatu, co pozwoliło na poprawę wygody użytkowania czatu oraz zachowanie historii czatu użytkowników. Umiejętności zdobyte w tym rozdziale pomogą Ci implementować asynchroniczne funkcjonalności w aplikacjach czasu rzeczywistego.

W następnym rozdziale nauczysz się, w jaki sposób za pomocą systemu Docker Compose opracować środowisko produkcyjne dla projektu Django z wykorzystaniem NGINX, uWSGI i Daphne. Dowiesz się również, jak wdrożyć niestandardowe oprogramowanie middleware do przetwarzania żądań i odpowiedzi w całej aplikacji, a także jak zaprogramować niestandardowe polecenia zarządzające umożliwiające automatyzację zadań i wykonywanie ich za pomocą wiersza poleceń.

Dodatkowe zasoby

Oto lista zasobów zawierających dodatkowe informacje związane z tematami omawianymi w tym rozdziale:

- Kod źródłowy przykładów zamieszczonych w tym rozdziale — *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter16*.
- Wprowadzenie do ASGI — *https://asgi.readthedocs.io/en/latest/introduction.html*.
- Obsługa widoków asynchronicznych w Django — *https://docs.djangoproject.com/en/5.0/topics/async/.*
- Obsługa widoków asynchronicznych opartych na klasach w Django — *https://docs.djangoproject.com/en/5.0/topics/class-based-views/#async-class-based-views*.
- Serwer ASGI Daphne — *https://github.com/django/daphne*.
- Dokumentacja frameworka Django Channels — *https://channels.readthedocs.io/*.
- Wdrażanie Django z ASGI — *https://docs.djangoproject.com/en/5.0/howto/deployment/#asgi/*.
- Wprowadzenie do WebSocket — *https://en.wikipedia.org/wiki/WebSocket*.
- Sposób użycia filtra szablonu `json_script` — *https://docs.djangoproject.com/en/5.0/ref/templates/builtins/#json-script*.
- Asynchroniczne zapytania Django ORM — *https://docs.djangoproject.com/en/5.0/topics/db/queries/#asynchronous-queries*.
- Dokumentacja pakietu `nh3` — *https://nh3.readthedocs.io/en/latest/*.
- Projekt `django-nh3` — *https://github.com/marksweb/django-nh3*.

Rozdział

17 Wdrożenie

W poprzednim rozdziale zbudowałeś za pomocą frameworka Django Channels serwer czatu dla uczestników kursów. Teraz, gdy utworzyłeś w pełni funkcjonalną platformę e-learningową, trzeba skonfigurować środowisko produkcyjne na serwerze online, tak byś mógł uzyskać do niej dostęp przez internet. Dotychczas do uruchamiania swojej witryny korzystałeś z środowiska programistycznego, wykorzystując serwer programistyczny Django. W tym rozdziale dowiesz się, jak skonfigurować środowisko produkcyjne zdolne do serwowania Twojego projektu Django w bezpieczny i wydajny sposób.

Oto zagadnienia, na których skoncentruję się w tym rozdziale:

- Konfigurowanie ustawień Django dla wielu środowisk
- Korzystanie z systemu Docker Compose do uruchamiania wielu usług
- Konfigurowanie serwera WWW z wykorzystaniem uWSGI i Django
- Obsługa PostgreSQL i Redis za pomocą Docker Compose
- Korzystanie z frameworka Django kontroli systemu
- Obsługa NGINX z wykorzystaniem Dockera
- Udostępnianie statycznych zasobów za pomocą NGINX
- Zabezpieczanie połączeń z wykorzystaniem **TLS** (ang. *Transport Layer Security*) lub **SSL** (ang. *Secure Sockets Layer*)
- Wykorzystanie serwera **ASGI** (ang. *Asynchronous Server Gateway Interface*) Daphne do obsługi Django Channels
- Utworzenie własnej warstwy middleware dla frameworka Django
- Implementacja własnych poleceń zarządzających frameworka Django

W poprzednich rozdziałach diagramy na początku rozdziału reprezentowały widoki, szablony i kompleksowe funkcjonalności. W tym rozdziale skupimy się jednak na konfiguracji środowiska produkcyjnego. Konkretne diagramy ilustrujące konfigurację środowiska zostały zamieszczone w treści rozdziału.

Kod źródłowy przykładów z tego rozdziału można znaleźć pod adresem *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter17*.

Wszystkie pakiety Pythona użyte w tym rozdziale są zawarte w pliku *requirements.txt* dołączonym do plików z kodem źródłowym przykładów z tego rozdziału. Aby zainstalować każdy pakiet Pythona, możesz postępować zgodnie z instrukcjami zamieszczonymi w poniższych podrozdziałach. Możesz także zainstalować wszystkie wymagania jednocześnie za pomocą polecenia `pip install -r requirements.txt`.

Tworzenie środowiska produkcyjnego

Nadeszła pora na wdrożenie projektu Django w środowisku produkcyjnym. Zaczniemy od skonfigurowania ustawień Django dla wielu środowisk, a następnie skonfigurujemy środowisko produkcyjne.

Zarządzanie ustawieniami dla wielu środowisk

W rzeczywistych projektach będziesz musiał uwzględniać istnienie wielu środowisk. Zwykle należy zapewnić obsługę co najmniej jednego środowiska lokalnego do programowania oraz środowiska produkcyjnego do serwowania aplikacji. Mogą istnieć również inne środowiska, na przykład środowisko testowe lub przejściowe (ang. *staging environment*).

Część ustawień projektu pozostaje taka sama we wszystkich środowiskach, natomiast inne zależą całkowicie od danego środowiska. Zwykle korzysta się z pliku bazowego, w którym są zdefiniowane wspólne ustawienia, oraz pliku ustawień dla poszczególnych środowisk, który nadpisuje wszelkie niezbędne ustawienia i definiuje dodatkowe.

W tym rozdziale opiszę środowiska:

- `local`. Lokalne środowisko do uruchamiania projektu na komputerze.
- `prod`. Środowisko do wdrażania projektu na serwerze produkcyjnym.

W katalogu projektu `educa`, obok pliku *settings.py* utwórz podkatalog o nazwie *settings*. Nazwę istniejącego w projekcie pliku *settings.py* zmień na *base.py* i przenieść go do nowego katalogu *settings/*.

Następnie w katalogu *settings* utwórz kilka dodatkowych plików, aby struktura katalogu przedstawiała się tak:

```
settings/
    __init__.py
    base.py
    local.py
    prod.py
```

Utworzone pliki mają następującą zawartość.

- *base.py*. Podstawowy plik ustawień zawierający domyślne i najczęściej stosowane ustawienia (wcześniej *settings.py*).
- *local.py*. Własne ustawienia dla środowiska lokalnego.
- *prod.py*. Własne ustawienia dla środowiska produkcyjnego.

Ze względu na przeniesienie plików ustawień do katalogu o jeden poziom niżej, trzeba zaktualizować ustawienie `BASE_DIR` w pliku *settings/base.py*, aby wskazywało główny katalog projektu.

Gdy obsługujesz wiele środowisk, utwórz plik z ustawieniami podstawowymi oraz oddzielne pliki dla każdego z używanych środowisk. Pliki z ustawieniami dla poszczególnych środowisk powinny dziedziczyć ustawienia podstawowe i nadpisywać ustawienia charakterystyczne dla danego środowiska.

Przeprowadź edycję pliku *settings/base.py* i poniższy wiersz:

```
BASE_DIR = Path(__file__).resolve().parent.parent
```

zastąp następującym:

```
BASE_DIR = Path(__file__).resolve().parent.parent.parent
```

Wskazujemy jeden katalog wyżej poprzez dodanie do ścieżki `BASE_DIR` fragmentu *.parent*. Teraz skonfigurujemy ustawienia dla środowiska lokalnego.

Ustawienia środowiska lokalnego

Zamiast używać domyślnej konfiguracji dla ustawień `DEBUG` i `DATABASES`, zdefiniujemy je jawnie dla każdego środowiska. Te ustawienia będą zależne od środowiska. Przeprowadź edycję pliku *settings/local.py* i umieść w nim poniższy fragment kodu.

```
from .base import *

DEBUG = True

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

Jest to plik ustawień dla środowiska lokalnego. Na początku importujemy wszystkie ustawienia określone w pliku *base.py*, a następnie definiujemy ustawienia `DEBUG` i `DATA ↪BASES` charakterystyczne dla tego środowiska. Ustawienia `DEBUG` i `DATABASES` pozostają takie same, jak w przypadku środowiska programistycznego. Teraz można usunąć z pliku *base.py* ustawienia `DEBUG` i `DATABASES`.

Polecenia zarządzania Django nie wykryją automatycznie pliku ustawień, ponieważ plik ustawień projektu nie jest domyślnym plikiem *settings.py*. Gdy uruchamiasz polecenia zarządzania, powinieneś wskazać używany moduł ustawień. W tym celu należy dodać opcję `--settings` w następujący sposób:

```
python manage.py runserver --settings=educa.settings.local
```

Przejdziemy teraz do walidacji projektu i konfiguracji środowiska lokalnego.

Uruchamianie środowiska lokalnego

Uruchom środowisko lokalne z wykorzystaniem nowej struktury ustawień. Upewnij się, że Redis działa lub uruchom w powłoce kontener Dockera za pomocą polecenia:

```
docker run -it --rm --name redis -p 6379:6379 redis:7.2.4
```

W innej powłoce uruchom z katalogu projektu następujące polecenie zarządzania:

```
python manage.py runserver --settings=educa.settings.local
```

Otwórz w przeglądarce stronę `http://127.0.0.1:8000/` i sprawdź, czy strona poprawnie się ładuje. Teraz serwujesz swoją witrynę przy użyciu ustawień dla środowiska lokalnego.

Jeśli nie chcesz przekazywać opcji `--settings` za każdym razem, gdy uruchamiasz polecenie zarządzania, możesz zdefiniować zmienną środowiskową `DJANGO_SETTINGS_MODULE`. Django użyje jej do zidentyfikowania modułu ustawień, który ma być wykorzystany. Jeśli używasz systemu Linux lub macOS, możesz zdefiniować zmienną środowiskową poprzez uruchomienie w powłoce polecenia:

```
export DJANGO_SETTINGS_MODULE=educa.settings.local
```

Jeśli używasz systemu Windows, możesz uruchomić w powłoce polecenie:

```
set DJANGO_SETTINGS_MODULE=educa.settings.local
```

Każde polecenie zarządzania, które wykonasz później, użyje ustawień zdefiniowanych w zmiennej środowiskowej `DJANGO_SETTINGS_ MODULE`.

Zatrzymaj z poziomu powłoki serwer programistyczny Django. W tym celu naciśnij klawisze *Ctrl+C*. Kontener Docker bazy danych Redis również można zatrzymać za pomocą klawiszy *Ctrl+C*.

Lokalne środowisko działa poprawnie. Przygotujmy teraz ustawienia dla środowiska produkcyjnego.

Ustawienia środowiska produkcyjnego

Zacznijmy od określenia początkowych ustawień dla środowiska produkcyjnego. Przeprowadź edycję pliku *educa/settings/prod.py* i umieść w nim poniższy fragment kodu.

```
from .base import *

DEBUG = False

ADMINS = [
    ('Antonio M', 'email@mydomain.com'),
]

ALLOWED_HOSTS = ['*']

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

To są ustawienia dla środowiska produkcyjnego.

- `DEBUG`. Przypisanie wartości `False` opcji `DEBUG` jest obowiązkowe dla każdego środowiska produkcyjnego. Jeżeli o tym zapomnisz, ryzykujesz wyświetlenie pełnych informacji o stosie wywołań, a także poufnych danych konfiguracyjnych, które nie powinny być udostępniane publicznie.
- `ADMINS`. Gdy opcja `DEBUG` ma przypisaną wartość `False`, a widok zgłosi wyjątek, wówczas wszystkie informacje zostaną za pomocą wiadomości e-mail przekazane do osób wymienionych w opcji `ADMINS`. Upewnij się, że zastąpiłeś przykładową krotkę imienia i nazwiska oraz adresu e-mail odpowiednimi danymi.

- `ALLOWED_HOSTS`. Ze względów bezpieczeństwa Django zezwoli na serwowanie projektu tylko z hostów wymienionych na tej liście. W tym przykładzie, dzięki wykorzystaniu symbolu gwiazdki (*), zezwoliliśmy na to dowolnym hostom. Zbiór hostów, które będą mogły być wykorzystane do serwowania projektu, zostanie ograniczony później.
- `DATABASES`. Zachowujemy domyślne ustawienia bazy danych wskazujące na bazę danych SQLite środowiska lokalnego. Produkcyjną bazę danych skonfigurujesz później.

Plik ustawień dla środowiska produkcyjnego uzupełnimy w następnych punktach tego rozdziału.

W ten sposób z powodzeniem przygotowałeś ustawienia dla wielu środowisk. W następnym podrozdziale zbudujemy kompletne środowisko produkcyjne, poprzez skonfigurowanie różnych usług za pomocą Dockera.

Korzystanie z systemu Docker Compose

Z Dockera korzystałeś już w rozdziale 3., „Rozbudowa aplikacji bloga". Używałeś go również w całej tej książce do uruchamiania kontenerów dla różnych usług, takich jak PostgreSQL, Redis i RabbitMQ.

Każdy kontener Docker łączy kod źródłowy aplikacji z bibliotekami systemu operacyjnego oraz zależnościami wymaganymi do uruchomienia aplikacji. Zastosowanie kontenerów aplikacji pozwala poprawić jej przenośność. W środowisku produkcyjnym do zbudowania i uruchomienia różnych kontenerów Docker skorzystamy z systemu Docker Compose.

Docker Compose to narzędzie do definiowania i uruchamiania aplikacji wielokontenerowych. Aby zdefiniować różne usługi, możesz utworzyć plik konfiguracyjny, a następnie użyć jednego polecenia, aby na podstawie tej konfiguracji uruchomić wszystkie usługi. Informacje o systemie Docker Compose można znaleźć na stronie *https://docs.docker.com/compose/*.

W środowisku produkcyjnym utworzymy aplikację rozproszoną, która będzie działać w wielu kontenerach Docker. W każdym kontenerze Docker będzie uruchomiona inna usługa. Najpierw zdefiniujemy następujące trzy usługi, a w następnych wprowadzimy dodatkowe.

- **Usługa WWW.** Serwer WWW do serwowania projektu Django.
- **Usługa bazy danych**. Usługa bazy danych do uruchomienia PostgreSQL.
- **Usługa pamięci podręcznej**. Usługa do uruchomienia bazy danych Redis.

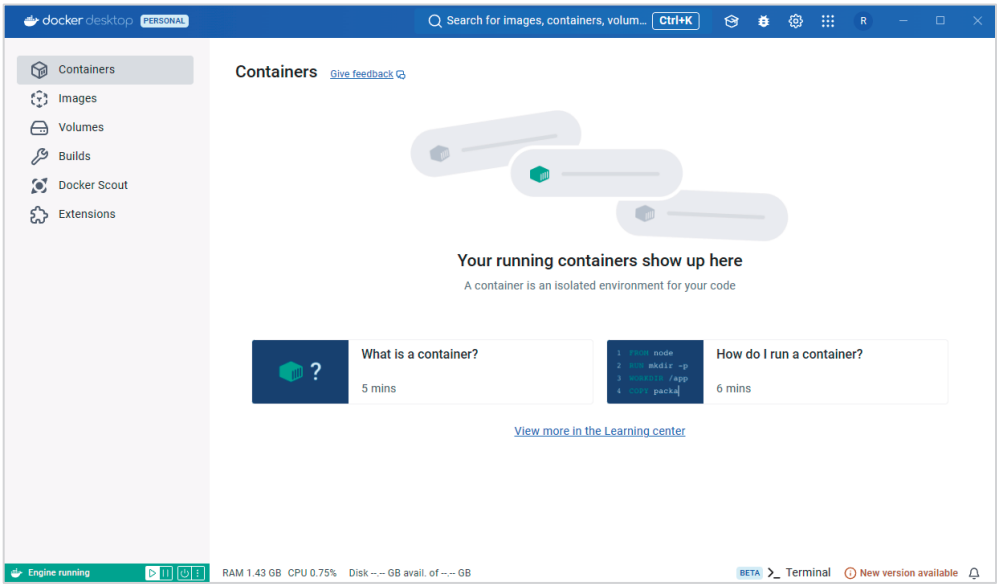
Zacznijmy od zainstalowania systemu Docker Compose.

Instalacja Docker Compose za pośrednictwem Docker Desktop

System Docker Compose możesz uruchomić w systemie macOS, 64-bitowym systemie Linux, a także w systemie Windows. Najszybszym sposobem instalacji systemu Docker Compose jest zainstalowanie środowiska Docker Desktop. Instalacja obejmuje system Docker Engine, interfejs wiersza poleceń oraz wtyczkę Docker Compose.

Zainstaluj Docker Desktop zgodnie z instrukcjami na stronie *https://docs.docker.com/compose/install/compose-desktop/*.

Otwórz aplikację Docker Desktop i kliknij link *Containers*. Powinieneś zobaczyć stronę podobną do pokazanej na rysunku 17.1.



Rysunek 17.1. Interfejs Docker Desktop

Po zainstalowaniu systemu Docker Compose, należy stworzyć obraz Docker dla Twojego projektu Django.

Tworzenie pliku Dockerfile

Aby uruchomić projekt Django, należy utworzyć obraz Docker. *Dockerfile* to plik tekstowy, który zawiera polecenia Dockera do utworzenia obrazu Docker. Przygotujemy plik *Dockerfile* z poleceniami potrzebnymi do zbudowania obrazu Docker dla projektu Django.

Obok katalogu projektu `educa` utwórz nowy plik i nadaj mu nazwę *Dockerfile*. W nowym pliku umieść poniższy fragment kodu.

```
# Pobranie oficjalnego bazowego obrazu Docker Pythona
FROM python:3.12.3

# Ustawienie zmiennych środowiskowych
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

# Ustawienie katalogu roboczego
WORKDIR /code

# Instalacja zależności
RUN pip install --upgrade pip
COPY requirements.txt .
RUN pip install -r requirements.txt

# Skopiowanie projektu Django
COPY . .
```

W przedstawionym kodzie wykonujemy następujące zadania.

1. Używamy macierzystego obrazu Docker Pythona wersji w 3.12.3. Oficjalny obraz Docker Pythona można znaleźć pod adresem *https://hub.docker.com/_/python*.
2. Ustawiamy zmienne środowiskowe:
   A. `PYTHONDONTWRITEBYTECODE`. Blokuje zapisywanie przez Pythona plików *.pyc*.
   B. `PYTHONUNBUFFERED`. Włącza bezpośrednie wysyłanie strumieni `stdout` i `stderr` do terminala, bez ich uprzedniego buforowania.
3. Do zdefiniowania katalogu roboczego obrazu, wykorzystujemy polecenie `WORKDIR`.
4. Aktualizujemy pakiet `pip` obrazu.
5. Do katalogu roboczego (.) macierzystego obrazu Pythona kopiujemy plik *requirements.txt*.
6. Za pomocą menedżera `pip` instalujemy w obrazie pakiety języka Python wymienione w pliku *requirements.txt*.
7. Kopiujemy kod źródłowy projektu Django z katalogu lokalnego do katalogu roboczego (.) obrazu.

Za pomocą tego pliku *Dockerfile* zdefiniowałeś sposób utworzenia obrazu Docker do serwowania projektu Django. Dokumentację pliku *Dockerfile* można znaleźć pod adresem *https://docs.docker.com/reference/dockerfile/*.

Dodanie wymagań Pythona

Plik *requirements.txt* wymieniony w pliku *Dockerfile*, który utworzyłeś przed chwilą zawiera listę wszystkich niezbędnych pakietów języka Python, które powinny być zainstalowane dla projektu.

Obok katalogu projektu `educa` utwórz nowy plik i nadaj mu nazwę *requirements.txt*. Być może już wcześniej utworzyłeś ten plik i skopiowałeś zawartość pliku *requirements.txt*

z pliku *https://github.com/PacktPublishing/Django-5-by-example/blob/main/Chapter17/requirements.txt*. Jeśli jeszcze tego nie zrobiłeś, dodaj do nowo utworzonego pliku *requirements.txt* następujące wiersze:

```
asgiref==3.8.1
Django~=5.0.4
Pillow==10.3.0
sqlparse==0.5.0
django-braces==1.15.0
django-embed-video==1.4.9
pymemcache==4.0.0
django-debug-toolbar==4.3.0
redis==5.0.4
django-redisboard==8.4.0
djangorestframework==3.15.1
requests==2.31.0
channels[daphne]==4.1.0
channels-redis==4.2.0
psycopg==3.1.18
uwsgi==2.0.25.1
python-decouple==3.8
```

Oprócz pakietów Pythona, które zainstalowałeś w poprzednich rozdziałach, plik *requirements.txt* zawiera pakiety:

- `psycopg`. Adapter PostgreSQL. W środowisku produkcyjnym będziemy używać bazy danych PostgreSQL.
- `uwsgi`. Serwer WWW WSGI. Ten serwer WWW skonfigurujemy później w celu serwowania Django w środowisku produkcyjnym.
- `python-decouple`. Pakiet ułatwiający ładowanie zmiennych środowiskowych.

Zacznijmy od skonfigurowania aplikacji Docker w systemie Docker Compose. Stworzymy plik Docker Compose wraz z definicją serwera WWW, bazy danych i usług Redis.

Tworzenie pliku Docker Compose

Do zdefiniowania usług, które będą działać w różnych kontenerach Docker, użyjemy pliku Docker Compose. Konfiguracja Docker Compose jest zapisana w pliku tekstowym w formacie YAML zawierającym definicje usług, sieci i woluminów danych dla aplikacji Docker. YAML to czytelny dla człowieka język serializacji danych. Przykład pliku YAML możesz znaleźć na stronie *https://yaml.org/*.

Obok katalogu projektu `educa` utwórz nowy plik i nadaj mu nazwę *docker-compose.yml*. Następnie w pliku umieść poniższy fragment kodu.

```
services:

  web:
    build: .
    command: python /code/educa/manage.py runserver 0.0.0.0:8000
    restart: always
    volumes:
      - .:/code
```

```
    ports:
      - "8000:8000"
    environment:
      - DJANGO_SETTINGS_MODULE=educa.settings.prod
```

W tym pliku definiujemy usługę sieciową (`web`). Oto sekcje służące do definicji tej usługi:

- `build`. Określa wymagania dotyczące budowania obrazu kontenera usługi. Może to być pojedynczy ciąg definiujący ścieżkę kontekstu lub szczegółowa definicja kompilacji. Należy podać względną ścieżkę z pojedynczą kropką (`.`), aby wskazywała ten sam katalog, w którym znajduje się plik *Compose*. W tej lokalizacji system Docker Compose będzie szukać pliku *Dockerfile*. Więcej informacji na temat sekcji `build` znajdziesz na stronie *https://docs.docker.com/compose/compose-file/build/*.
- `command`. Nadpisuje domyślne polecenie kontenera. Serwer programistyczny Django uruchamiamy za pomocą polecenia `python manage.py runserver`. Projekt jest serwowany na hoście `0.0.0.0`, który jest domyślnym adresem IP platformy Docker, na porcie `8000`.
- `restart`. Definiuje reguły restartowania dla kontenera. Ustawienie `always` powoduje, że kontener jest uruchamiany ponownie zawsze wtedy, kiedy się zatrzyma. Jest to przydatne w środowisku produkcyjnym, kiedy chcemy zminimalizować przestoje. Więcej informacji na temat reguł restartowania można znaleźć pod adresem *https://docs.docker.com/config/containers/start-containers-automatically/*.
- `volumes`. Dane w kontenerach Docker nie są trwałe. Każdy kontener Docker ma wirtualny system plików, który jest wypełniany plikami obrazu i który jest niszczony po zatrzymaniu kontenera. Woluminy są preferowaną metodą utrwalania danych generowanych i wykorzystywanych przez kontenery platformy Docker. W tej sekcji montujemy lokalny katalog `.` do katalogu */code* obrazu. Więcej informacji o woluminach Docker można znaleźć na stronie *https://docs.docker.com/storage/volumes/*.
- `ports`. Wskazuje porty kontenera. Port hosta `8000` jest mapowany na port kontenera `8000`, na którym działa serwer programistyczny Django.
- `environment`. Definiuje zmienne środowiskowe. Ustawiliśmy zmienną środowiskową `DJANGO_SETTINGS_MODULE` w celu wskazania produkcyjnego pliku ustawień Django *educa.settings.prod*.

Zauważ, że w definicji pliku Docker Compose, do serwowania aplikacji używamy serwera programistycznego Django. Serwer programistyczny Django nie nadaje się do wykorzystania w produkcji, więc później zastąpisz go pythonowym serwerem WWW WSGI.

Informacje o specyfikacji Docker Compose znajdziesz na stronie *https://docs.docker.com/compose/compose-file/*.

Przy założeniu, że Twój katalog główny nosi nazwę *Rozdział17*, struktura plików powinna w tym momencie wyglądać tak:

```
Rozdział17/
    Dockerfile
```

```
    docker-compose.yml
    educa/
        manage.py
        ...
    requirements.txt
```

Otwórz terminal w katalogu, w którym znajduje się plik *docker-compose.yml* i uruchom polecenie:

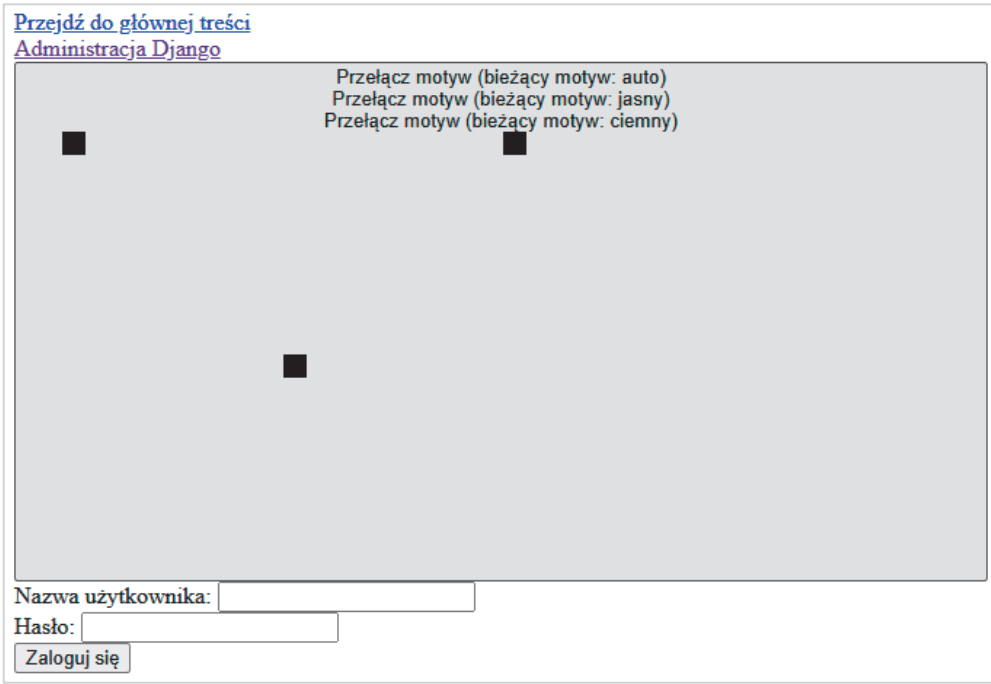
```
docker compose up
```

Spowoduje to uruchomienie aplikacji Docker zdefiniowanej w pliku Docker Compose. Powinieneś otrzymać wynik zawierający m.in. następujące wiersze.

```
web-1  | Performing system checks...
web-1  | rozdzia17-web-1  | System check identified no issues (0 silenced).
web-1  | March 10, 2024 - 12:03:28
web-1  | Django version 5.0.4, using settings 'educa.settings.prod'
web-1  | Starting ASGI/Daphne version 4.1.0 development server at
http://0.0.0.0:8000/
web-1  | Quit the server with CONTROL-C.
```

Kontener Docker dla Twojego projektu Django jest uruchomiony!

W przeglądarce internetowej przejdź pod adres `http://127.0.0.1:8000/admin/`. Powinieneś zobaczyć formularz logowania do strony administracyjnej Django (rysunek 17.2).

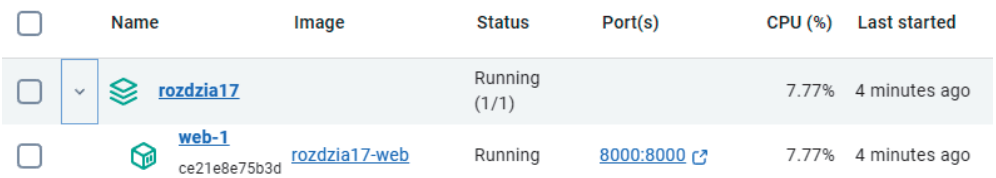


Rysunek 17.2. Ekran logowania do strony administracyjnej Django

Style CSS nie zostały załadowane. Używasz ustawienia `DEBUG=False`, więc wzorce adresów URL do serwowania statycznych plików w głównym pliku *urls.py* projektu nie zostały uwzględnione. Pamiętaj, że serwer programistyczny Django nie nadaje się do serwowania plików statycznych. W celu serwowania plików statycznych skonfigurujesz serwer w dalszej części tego rozdziału.

Jeśli spróbujesz uzyskać dostęp do dowolnego innego adresu URL swojej witryny, możesz otrzymać błąd HTTP 500, ponieważ nie skonfigurowałeś jeszcze bazy danych dla środowiska produkcyjnego.

Spójrz na aplikację Docker Desktop. Wyświetlają się w niej następujące kontenery (rysunek 17.3):



Rysunek 17.3. Aplikacja rozdzia17 i kontener web-1 w aplikacji Docker Desktop

Aplikacja Docker *rozdzia17* jest uruchomiona i zawiera pojedynczy kontener o nazwie `web-1`, który działa w porcie 8000. Nazwa aplikacji Docker jest generowana dynamicznie na podstawie nazwy katalogu, w którym znajduje się plik Docker Compose, w tym przypadku jest to nazwa *rozdzia17*.

W obszarze *Images* zobaczysz obraz utworzony dla usługi `web`, jak pokazałem na rysunku 17.4.

Rysunek 17.4. Aplikacja rozdzia17 i kontener web-1 w aplikacji Docker Desktop

Obraz *rozdzia17-web* został zbudowany z wykorzystaniem zdefiniowanego wcześniej pliku *Dockerfile* i jest używany przez kontener `web-1`.

W kolejnym punkcie dodamy do aplikacji Docker usługi PostgreSQL i Redis.

Konfigurowanie usługi PostgreSQL

W tej książce korzystaliśmy głównie z bazy danych SQLite. Wprawdzie jest prosta i łatwa w konfiguracji, ale w środowisku produkcyjnym będzie potrzebna baza danych oferująca znacznie większe możliwości, np. PostgreSQL, MySQL lub Oracle. Do zainstalowania PostgreSQL skorzystałeś z Dockera (w rozdziale 3., „Rozbudowa aplikacji bloga"). Informacje dotyczące oficjalnego obrazu Docker bazy danych PostgreSQL można znaleźć pod adresem *https://hub.docker.com/_/postgres*.

Przeprowadź edycję pliku *docker-compose.yml* i dodaj do niego następujące wiersze wyróżnione pogrubioną czcionką.

```
services:

  db:
    image: postgres:16.2
    restart: always
    volumes:
      - ./data/db:/var/lib/postgresql/data
    environment:
      - POSTGRES_DB=postgres
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=postgres

  web:
    build: .
    command: python /code/educa/manage.py runserver 0.0.0.0:8000
    restart: always
    volumes:
      - .:/code
    ports:
      - "8000:8000"
    environment:
      - DJANGO_SETTINGS_MODULE=educa.settings.prod
      - POSTGRES_DB=postgres
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=postgres
    depends_on:
      - db
```

Dzięki tym zmianom zdefiniowałeś usługę o nazwie `db`. Jej konfiguracja składa się z następujących podsekcji:

- `image`. Usługa korzysta z podstawowego obrazu Docker `postgres`.
- `restart`. Reguły restartowania zostały ustawione na `always`.
- `volumes`. Aby utrwalić bazę danych, montujemy katalog *./data/db* do katalogu obrazu */var/lib/postgresql/data*. Dzięki temu po zatrzymaniu aplikacji Docker dane przechowywane w bazie danych będą zachowane. Spowoduje to utworzenie lokalnej ścieżki *data/db/*.
- `environment`. Użyliśmy zmiennych `POSTGRES_DB` (nazwa bazy danych), `POSTGRES_USER` i `POSTGRES_ PASSWORD` wraz z ich wartościami domyślnymi.

Definicja usługi `web` zawiera teraz zmienne środowiskowe do konfiguracji PostgreSQL w Django. Zależność usługi stworzyliśmy z wykorzystaniem ustawienia `depends_on`, tak aby usługa `web` była uruchamiana po usłudze `db`. To da nam gwarancję kolejności inicjalizacji kontenera, ale nie zagwarantuje kompletnego zainicjowania PostgreSQL przed uruchomieniem serwera WWW Django. Aby rozwiązać ten problem, trzeba użyć skryptu, który będzie czekał na dostępność hosta bazy danych i jego portu TCP. Do zarządzania inicjalizacją kontenera Docker zaleca używanie narzędzia `wait-for-it`.

Pobierz skrypt bash *wait-for-it.sh* ze strony *https://github.com/vishnubob/wait-for-it/blob/master/wait-for-it.sh* i zapisz plik obok pliku *docker-compose.yml*. Następnie w pliku *dockercompose.yml* zmodyfikuj definicję usługi `web` w pokazany niżej sposób. Nowy kod został wyróżniony pogrubioną czcionką.

```
web:
  build: .
  command: ["./wait-for-it.sh", "db:5432", "--",
            "python", "/code/educa/manage.py", "runserver",
            "0.0.0.0:8000"]
  restart: always
  volumes:
      - .:/code
    environment:
      - DJANGO_SETTINGS_MODULE=educa.settings.prod
      - POSTGRES_DB=postgres
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=postgres
    depends_on:
      - db
```

W tej definicji usługi używasz skryptu bash `wait-for-it.sh`, którego zadaniem jest czekanie, przed uruchomieniem serwera programistycznego Django, na gotowość hosta `db` do akceptowania połączeń na porcie `5432` — domyślnym porcie dla PostgreSQL. Więcej informacji o kolejności uruchamiania usług w Docker Compose możesz przeczytać na stronie *https://docs.docker.com/compose/startup-order/*.

Przystępujemy do modyfikacji ustawień Django. Przeprowadź edycję pliku *educa/settings/prod.py* projektu i umieść w nim poniższy fragment kodu wyróżniony pogrubioną czcionką.

```
from decouple import config
from .base import *

DEBUG = False

ADMINS = [
    ('Antonio M', 'email@mydomain.com'),
]

ALLOWED_HOSTS = ['*']

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': config('POSTGRES_DB'),
        'USER': config('POSTGRES_USER'),
        'PASSWORD': config('POSTGRES_PASSWORD'),
        'HOST': 'db',
        'PORT': 5432,
    }
}
```

W pliku ustawień środowiska produkcyjnego używamy opcji:

- `ENGINE`. Używamy backendu bazy danych Django dla PostgreSQL.
- `NAME`, `USER` i `PASSWORD`. Do pobrania zmiennych środowiskowych `POSTGRES_DB` (nazwa bazy danych), `POSTGRES_USER` i `POSTGRES_PASSWORD` skorzystaliśmy z metody `os.environ.get()`. Te zmienne środowiskowe zostały ustawione w pliku Docker Compose.

- `HOST`. Użyliśmy ustawienia `db`, która jest nazwą hosta kontenera dla usługi bazy danych zdefiniowanej w pliku Docker Compose. Domyślną nazwą hosta kontenera jest identyfikator kontenera w systemie Docker. Dlatego użyliśmy nazwy hosta `db`.
- `PORT`. Użyliśmy wartości `5432`, która jest domyślnym portem dla PostgreSQL.

Zatrzymaj aplikację Docker poprzez naciśnięcie w powłoce klawiszy *Ctrl + C* lub za pomocą przycisku *stop* w aplikacji Docker Desktop. Następnie ponownie uruchom system Docker Compose za pomocą polecenia.

```
docker compose up
```

Pierwsze uruchomienie po dodaniu usługi `db` do pliku Docker Compose potrwa dłużej, ponieważ PostgreSQL musi zainicjować bazę danych. Dane wyjściowe będą zawierały m.in. następujące dwa wiersze:

```
db-1   | ... database system is ready to accept connections
...
web-1  | Starting ASGI/Daphne version 4.1.0 development server at
http://0.0.0.0:8000/
```

Zarówno baza danych PostgreSQL, jak i aplikacja Django są gotowe. Produkcyjna baza danych jest pusta, dlatego należy zastosować migracje bazy danych.

Stosowanie migracji bazy danych i tworzenie superużytkownika

Otwórz terminal w katalogu, w którym znajduje się plik *docker-compose.yml* i uruchom następujące polecenie:

```
docker compose exec web python /code/educa/manage.py migrate
```

Polecenie `docker compose exec` umożliwia uruchamianie poleceń w kontenerze. To polecenie służy do wykonywania polecenia zarządzania migracją w kontenerze `web` Dockera.

Na końcu utwórz superużytkownika za pomocą polecenia:

```
docker compose exec web python /code/educa/manage.py createsuperuser
```

Migracje do bazy danych zostały zastosowane i utworzyłeś superużytkownika. Korzystając z poświadczeń superużytkownika możesz teraz uzyskać dostęp do strony `http://local`
`↪host:8000/admin/`. Style CSS nadal się nie załadują, ponieważ jeszcze nie skonfigurowałeś serwowania statycznych plików.

Zdefiniowałeś usługi do obsługi Django i PostgreSQL za pomocą Docker Compose. W kolejnym punkcie dodasz usługę do obsługi w środowisku produkcyjnym bazy danych Redis.

Konfigurowanie usługi Redis

Przystępujemy do dodania do pliku Docker Compose usługi Redis. W tym celu skorzystamy z oficjalnego obrazu Docker bazy danych Redis. Informacje dotyczące oficjalnego obrazu Docker bazy danych Redis można znaleźć na stronie *https://hub.docker.com/_/redis*.

Przeprowadź edycję pliku *docker-compose.yml* i dodaj do niego następujące wiersze wyróżnione pogrubioną czcionką.

```
services:

  db:
    # …

  cache:
    image: redis:7.2.4
    restart: always
    volumes:
      - ./data/cache:/data

  web:
    # ...
    depends_on:
      - db
      - cache
```

W powyższym kodzie zdefiniowaliśmy usługę `cache` zawierającą podsekcje:

- `image`. Usługa korzysta z podstawowego obrazu Docker bazy danych Redis.
- `restart`. Reguły restartowania zostały ustawione na `always`.
- `volumes`. W katalogu obrazu */data* montujemy katalog *./data/cache*, w którym zostaną utrwalone wszelkie zapisy bazu danych Redis. Spowoduje to utworzenie lokalnej ścieżki *data/cache/*.

W definicji usługi `web` dodajemy usługę `cache` jako zależność, dzięki czemu usługa web będzie uruchomiona po usłudze `cache`. Serwer Redis inicjuje się szybko, dlatego w tym przypadku nie trzeba używać narzędzia `wait-for-it`.

Przeprowadź edycję pliku *educa/settings/prod.py* i umieść w nim poniższy fragment kodu.

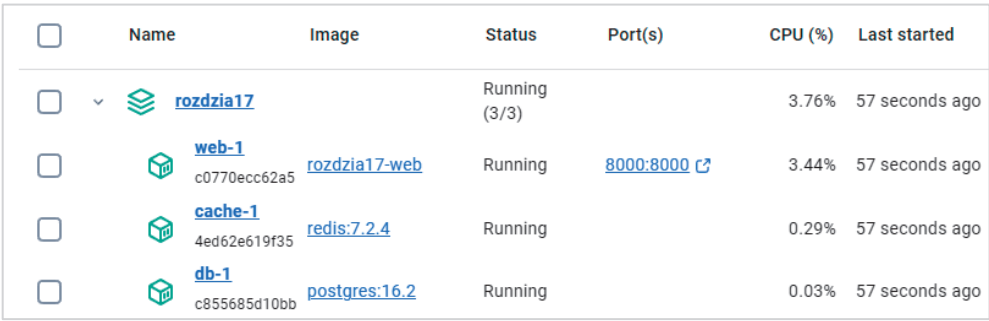
```
REDIS_URL = 'redis://cache:6379'
CACHES['default']['LOCATION'] = REDIS_URL
CHANNEL_LAYERS['default']['CONFIG']['hosts'] = [REDIS_URL]
```

W tych ustawieniach używamy nazwy hosta `cache`, która jest automatycznie generowana przez Docker Compose przy użyciu wykorzystywanych przez Redis nazwy usługi `cache` i portu `6379`. Modyfikujemy ustawienie `CACHE` frameworka Django oraz ustawienie `CHANNEL_LAYERS` używane przez framework `Channels` tak, by korzystały z produkcyjnego adresu URL bazy danych Redis.

Zatrzymaj aplikację Docker poprzez naciśnięcie w powłoce klawiszy *Ctrl + C* lub za pomocą przycisku *stop* w aplikacji Docker Desktop. Następnie ponownie uruchom system Docker Compose za pomocą polecenia.

```
docker compose up
cache-1   | ... Ready to accept connections tcp
```

Otwórz aplikację Docker Desktop. Powinieneś teraz zobaczyć aplikację Docker `rozdzia17`, która uruchamia kontener dla każdej usługi zdefiniowanej w pliku Docker Compose: `db`, `cache` i `web` (rysunek 17.5).



Rysunek 17.5. Aplikacja rozdzia17 z kontenerami db-1, web-1 i cache-1 w aplikacji Docker Desktop

Nadal serwujesz aplikację Django za pomocą serwera programistycznego, który nie nadaje się do użytku w środowisku produkcyjnym. Zastąpimy go pythonowym serwerem WWW WSGI.

Serwowanie Django za pomocą WSGI i NGINX

Podstawową platformą wdrożeniową Django jest **WSGI**. Ten skrót oznacza *Web Server Gateway Interface*, czyli technologię będącą standardem w zakresie udostępniania aplikacji Pythona w internecie.

Kiedy generujesz nowy projekt za pomocą polecenia `startproject`, Django tworzy w katalogu projektu plik o nazwie *wsgi.py*. Plik ten zawiera polecenie uruchamiające aplikację WSGI, które jest punktem wyjścia do budowanej aplikacji.

Interfejs WSGI jest używany do uruchomienia tworzonego projektu zarówno w serwerze programistycznym, jak i aplikacji wdrożonej w wybranym środowisku produkcyjnym. Więcej informacji na temat WSGI znajdziesz na stronie *https://wsgi.readthedocs.io/en/latest/*.

W następnych punktach skorzystamy z **uWSGI** — serwera WWW typu open source, który implementuje specyfikację WSGI.

Korzystanie z uWSGI

W książce wykorzystywaliśmy serwer programistyczny Django do uruchamiania projektów w środowisku lokalnym. Serwer programistyczny nie nadaje się jednak do wykorzystania produkcyjnego, a wdrożenie aplikacji w środowisku produkcyjnym wymaga standardowego serwera WWW.

Serwer uWSGI to niezwykle szybki serwer aplikacji Pythona. Komunikacja z aplikacją Pythona odbywa się według specyfikacji WSGI. Serwer uWSGI przekształca żądania sieciowe na format możliwy do przetworzenia przez Django.

Skonfigurujmy uWSGI do obsługi projektu Django. Wcześniej w pliku *requirements.txt* projektu dodałeś ustawienie `uwsgi==2.0.20`, więc serwer uWSGI jest już zainstalowany w obrazie Docker usługi `web`.

W pliku *docker-compose.yml* zmodyfikuj definicję usługi `web` w pokazany niżej sposób. Nowy kod został wyróżniony pogrubioną czcionką.

```
web:
    build: .
    command: ["./wait-for-it.sh", "db:5432", "--",
              "uwsgi", "--ini", "/code/config/uwsgi/uwsgi.ini"]
    restart: always
    volumes:
      - .:/code
    environment:
      - DJANGO_SETTINGS_MODULE=educa.settings.prod
      - POSTGRES_DB=postgres
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=postgres
    depends_on:
      - db
      - cache
```

Pamiętaj, aby usunąć sekcję `ports`. uWSGI będzie dostępny za pomocą gniazda, więc nie trzeba udostępniać portu w kontenerze.

Nowe polecenie uruchamia uWSGI i przekazuje do niego plik konfiguracyjny */code/config/uwsgi/uwsgi.ini*. Stwórzmy plik konfiguracyjny dla uWSGI.

Konfiguracja uWSGI

Serwer uWSGI pozwala na zdefiniowanie własnej konfiguracji w pliku *.ini*. Utwórz plik *config/uwsgi/uwsgi.ini* obok pliku *docker-compose.yml*. Przy założeniu, że Twój katalog główny nosi nazwę *Rozdział17*, struktura plików powinna w tym momencie wyglądać następująco:

```
Rozdział17/
    config/
        uwsgi/
            uwsgi.ini
    Dockerfile
    docker-compose.yml
    educa/
        manage.py
        ...
    requirements.txt
```

Przeprowadź edycję pliku *config/uwsgi/uwsgi.ini* i umieść w nim poniższy fragment kodu.

```
[uwsgi]
socket=/code/educa/uwsgi_app.sock
chdir = /code/educa/
module=educa.wsgi:application
master=true
chmod-socket=666
uid=www-data
gid=www-data
vacuum=true
```

W pliku *uwsgi.ini* zdefiniowaliśmy zmienne:

- `socket`. Gniazdo UNIX/TCP przeznaczone do powiązania serwera.
- `chdir`. Ścieżka dostępu do katalogu projektu, aby serwer uWSGI mógł do niego przejść przed załadowaniem aplikacji Pythona.
- `module`. Moduł WSGI do wykorzystania. W tym miejscu wskazujemy aplikację wywoływalną zdefiniowaną w module `wsgi` projektu.
- `master`. Włączenie procesu głównego.
- `chmod-socket`. Uprawnienia do plików, które będą zastosowane do pliku gniazda. W tym przypadku korzystamy z ustawienia `666`, dzięki czemu NGINX może zarówno czytać dane z gniazda, jak i je do niego zapisywać.
- `uid`. Identyfikator użytkownika procesu po jego uruchomieniu.
- `gid`. Identyfikator grupy procesu po jego uruchomieniu.
- `vacuum`. Ustawienie `true` jest instrukcją dla uWSGI wyczyszczenia wszystkich utworzonych plików tymczasowych lub gniazd UNIX.

Opcja `socket` jest przeznaczona do komunikacji z routerami podmiotów zewnętrznych, takimi jak NGINX. Serwer uWSGI uruchomimy za pomocą gniazda. Skonfigurujemy NGINX jako serwer WWW i poprzez gniazdo będziemy prowadzić komunikację z uWSGI.

Listę wszystkich dostępnych opcji uWSGI znajdziesz na stronie *https://uwsgi-docs.readthedocs.io/en/latest/Options.html*.

W tym momencie nie można uzyskać dostępu do egzemplarza uWSGI za pomocą przeglądarki internetowej, ponieważ serwer został uruchomiony z użyciem gniazda. Aby dokończyć przygotowanie środowiska, użyjemy NGINX przed uWSGI. Dzięki temu będziemy zarządzać żądaniami HTTP i przekazywać żądania aplikacji do uWSGI przez gniazdo. W następnym punkcie dokończmy przygotowanie środowiska produkcyjnego.

Korzystanie z NGINX

Kiedy udostępniasz witrynę internetową, oferujesz nie tylko treść generowaną dynamicznie, ale musisz również zapewnić obsługę plików statycznych, takich jak arkusze stylów CSS, skrypty JavaScript i obrazy. Chociaż istnieje możliwość serwowania plików statycznych za pomocą uWSGI, to taka konfiguracja dodaje niepotrzebny narzut do żądań HTTP. Z tego powodu zalecanym podejściem do udostępniania plików statycznych jest skonfigurowanie takiego serwera WWW jak NGINX.

NGINX to serwer WWW charakteryzujący się doskonałą obsługą współbieżności, dobrą wydajnością i niskim zapotrzebowaniem na pamięć operacyjną. NGINX działa w charakterze odwrotnego proxy, otrzymuje żądania HTTP i kieruje je do różnych warstw backend.

Ogólnie rzecz biorąc, serwera WWW, takiego jak NGINX, używasz w celu szybkiego i wydajnego udostępniania plików statycznych, a żądania treści generowanej dynamicznie są przekazywane do procesów roboczych uWSGI. Podczas korzystania z NGINX można również stosować reguły i wykorzystać oferowane przez ten serwer możliwości w zakresie odwrotnego proxy.

Usługę NGINX dodamy do pliku Docker Compose z wykorzystaniem oficjalnego obrazu Docker NGINX. Informacje dotyczące oficjalnego obrazu Docker serwera NGINX można znaleźć na stronie *https://hub.docker.com/_/nginx*.

Przeprowadź edycję pliku *docker-compose.yml* i dodaj do niego następujące wiersze wyróżnione pogrubioną czcionką.

```
services:
  db:
    # …
  cache:
    # …
  web:
    # …

  nginx:
    image: nginx:1.25.5
    restart: always
    volumes:
      - ./config/nginx:/etc/nginx/templates
      - .:/code
    ports:
      - "80:80"
```

Dodałeś definicję usługi `nginx` z podsekcjami:

- `image`. Usługa korzysta z podstawowego obrazu Docker serwera `nginx`.
- `restart`. Reguły restartowania zostały ustawione na `always`.
- `volumes`. Montujemy wolumin *./config/nginx* w katalogu */etc/nginx/templates* obrazu Docker. W tym miejscu NGINX będzie szukać domyślnego szablonu konfiguracji. Montujemy także lokalny katalog *.* do katalogu */code* obrazu, tak aby NGINX miał dostęp do statycznych plików.
- `ports`. Udostępniamy port 80, który jest mapowany na port kontenera 80. Jest to domyślny port dla protokołu HTTP.

Przystępujemy do skonfigurowania serwera WWW NGINX.

Konfiguracja NGINX

W katalogu *config/* utwórz następującą ścieżkę do pliku wyróżnioną pogrubioną czcionką:

```
config/
    uwsgi/
```

```
      uwsgi.ini
    nginx/
        default.conf.template
```

Przeprowadź edycję pliku *nginx/default.conf.template* i dodaj do niego kod:

```
# Komponent upstream dla uWSGI
upstream uwsgi_app {
    server unix:/code/educa/uwsgi_app.sock;
}

server {
    listen       80;
    server_name  www.educaproject.com educaproject.com;
    error_log    stderr warn;
    access_log   /dev/stdout main;

    location / {
        include      /etc/nginx/uwsgi_params;
        uwsgi_pass   uwsgi_app;
    }
}
```

Jest to konfiguracja podstawowa dla NGINX. W tej konfiguracji zdefiniowaliśmy nowy strumień `upstream` o nazwie `uwsgi_app` wskazujący na gniazdo utworzone przez uWSGI. Skorzystaliśmy przy tym z bloku `server` wraz z następującą konfiguracją.

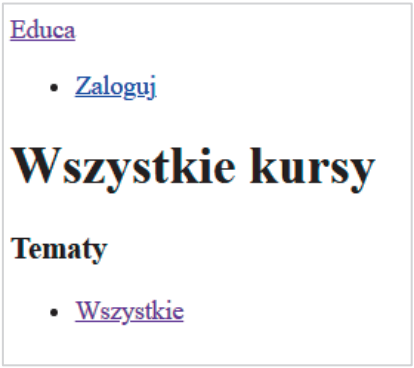
- Nakazujemy serwerowi NGINX nasłuchiwanie na porcie 80.
- Jako nazwę serwera podajemy *www.educaproject.com* i *educaproject.com*. NGINX będzie obsługiwać żądania przychodzące dla obu wymienionych domen.
- Aby pobierać logi błędów zapisane w standardowym strumieniu błędów, używamy `stderr` wewnątrz dyrektywy `error_log`. Drugi parametr określa poziom logowania. Aby odbierać ostrzeżenia i ważniejsze błędy, używamy opcji `warn`.
- W opcji `access_log` za pomocą */dev/stdout* wskazujemy standardowe wyjście.
- Wskazujemy, że każde żądanie znajdujące się w ścieżce dostępu / powinno być przekierowane do gniazda `uwsgi_app` (uWSGI).
- Dołączamy domyślne parametry konfiguracyjne uWSGI związane z NGINX. Znajdują się one w pliku */etc/nginx/uwsgi_params*.

NGINX jest teraz skonfigurowany. Dokumentację serwera NGINX znajdziesz na stronie *https://nginx.org/en/docs/*.

Zatrzymaj aplikację Docker poprzez naciśnięcie w powłoce klawiszy *Ctrl+C* lub za pomocą przycisku *stop* w aplikacji Docker Desktop. Następnie ponownie uruchom system Docker Compose za pomocą polecenia.

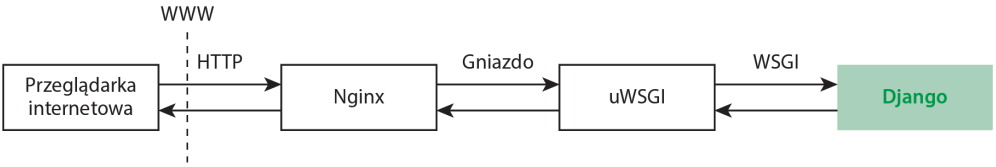
```
docker compose up
```

W przeglądarce internetowej przejdź pod adres `http://localhost/`. Nie ma potrzeby dodawania portu do adresu URL, ponieważ uzyskujesz dostęp do hosta przez standardowy port HTTP 80. Powinieneś zobaczyć stronę z listą kursów bez stylów CSS, jak na rysunku 17.6.



Rysunek 17.6. Strona listy kursów serwowana za pośrednictwem NGINX i uWSGI

Schemat żądanie-odpowiedź przygotowywanego środowiska produkcyjnego pokazałem na rysunku 17.7.



Rysunek 17.7. Cykl żądanie-odpowiedź w środowisku produkcyjnym

Kiedy klient w postaci przeglądarki internetowej wykonuje żądanie HTTP, zachodzą następujące zdarzenia.

1. NGINX otrzymuje żądanie HTTP.
2. NGINX poprzez gniazdo deleguje to żądanie do uWSGI.
3. Serwer uWSGI przekazuje żądanie do Django w celu jego przetworzenia.
4. Django zwraca odpowiedź HTTP, która jest przekazywana z powrotem do serwera NGINX, który z kolei przekazuje ją dalej do klienta, czyli przeglądarki internetowej.

Jeśli sprawdzisz aplikację Docker Desktop, powinieneś zobaczyć, że teraz działają 4 kontenery:

- usługa `db` z uruchomionym PostgreSQL,
- usługa pamięci podręcznej (`cache`) z systemem Redis,
- usługa serwera WWW (`web`) z systemem uWSGI + Django,
- usługa `nginx` z uruchomionym systemem NGINX.

Kontynuujmy konfigurację środowiska produkcyjnego. Zamiast uzyskiwania dostępu do projektu za pomocą nazwy *localhost*, skonfigurujemy projekt w taki sposób, aby używał nazwy hosta *educaproject.com*.

Korzystanie z nazwy hosta

W budowanej witrynie będziemy używać nazwy hosta *educaproject.com*. Ponieważ używamy przykładowej nazwy domeny, konieczne jest przekierowanie jej do komputera lokalnego.

W systemach Linux lub OS X rozwiązaniem jest edycja pliku */etc/hosts* i dodanie do niego wiersza.

```
127.0.0.1 educaproject.com www.educaproject.com
```

Jeśli używasz systemu Windows, w pliku *C:\Windows\System32\drivers\etc\hosts* dodaj taki sam wiersz.

W ten sposób żądania do hostów *educaproject.com* i *www.educaproject.com* będą przekierowywane do serwera lokalnego. W serwerze produkcyjnym nie trzeba tego robić, ponieważ nazwa hosta serwera jest już podana w konfiguracji DNS domen.

W przeglądarce internetowej przejdź pod adres *http://educaproject.com/*. Powinieneś zobaczyć przygotowaną witrynę internetową, choć nadal bez wczytanych zasobów statycznych. Środowisko produkcyjne projektu jest już niemal gotowe.

Teraz możemy wprowadzić ograniczenia dla hostów, które będą obsługiwać Twój projekt Django. Zmodyfikuj opcje pliku ustawień produkcyjnych *educa/settings/prod.py* projektu i zmień ustawienie `ALLOWED_HOSTS` w taki sposób.

```
ALLOWED_HOSTS = ['educaproject.com', 'www.educaproject.com']
```

Django będzie teraz obsługiwał Twoją aplikację tylko wtedy, gdy będzie działał na jednym z tych hostów. Więcej informacji o ustawieniu `ALLOWED_HOSTS` możesz znaleźć pod adresem *https://docs.djangoproject.com/en/5.0/ref/settings/#allowed-hosts.*

Środowisko produkcyjne projektu jest już prawie gotowe. W następnej kolejności skonfigurujemy NGINX do obsługi plików statycznych.

Udostępnianie zasobów statycznych i multimedialnych

uWSGI pozwala na bezproblemowe serwowanie plików statycznych, ale nie jest tak szybki i wydajny jak NGINX. W celu zapewnienia najlepszej wydajności zasoby statyczne będą udostępniane przez NGINX. Skonfigurujemy NGINX do serwowania zarówno statycznych plików Twojej aplikacji (arkuszy stylów CSS, plików JavaScript i obrazów), jak i plików multimedialnych wgrywanych przez instruktorów jako treść kursu.

Przeprowadź edycję pliku *settings/base.py* i umieść w nim poniższy fragment kodu bezpośrednio poniżej ustawienia `STATIC_URL`.

```
STATIC_ROOT = BASE_DIR / 'static'
```

To jest katalog główny dla wszystkich plików statycznych projektu. Następnie spróbujemy zebrać pliki statyczne z różnych aplikacji Django do wspólnego katalogu.

Zbieranie plików statycznych

Każda aplikacja w Twoim projekcie Django może zawierać pliki statyczne w katalogu *static/*. Django zapewnia polecenie pozwalające na zebranie plików statycznych ze wszystkich aplikacji do jednej lokalizacji. To upraszcza konfigurację niezbędną do serwowania plików statycznych w środowisku produkcyjnym. Polecenie `collectstatic` kopiuje wszystkie pliki statyczne z aplikacji i umieszcza je w katalogu wskazanym przez zmienną `STATIC_ROOT`.

Zatrzymaj aplikację Docker poprzez naciśnięcie w powłoce klawiszy *Ctrl + C* lub za pomocą przycisku *stop* w aplikacji Docker Desktop. Następnie ponownie uruchom system Docker Compose za pomocą polecenia.

```
docker compose up
```

Otwórz nowy terminal w katalogu, w którym znajduje się plik *docker-compose.yml* i uruchom polecenie:

```
docker compose exec web python /code/educa/manage.py collectstatic
```

Alternatywną opcją jest uruchomienie w powłoce, z katalogu *educa/project*, polecenia:

```
python manage.py collectstatic --settings=educa.settings.local
```

Uruchomienie dowolnego z tych poleceń przyniesie ten sam efekt, ponieważ podstawowy katalog lokalny jest zamontowany w obrazie Docker. Django zapyta, czy chcesz zastąpić istniejące pliki w katalogu głównym. Wpisz **yes** i naciśnij *Enter*. Zobaczysz wynik podobny do następującego.

```
171 static files copied to '/code/educa/static'.
```

Pliki z katalogów *static/* wszystkich aplikacji wymienionych w ustawieniu `INSTALLED_APPS` zostały skopiowane do globalnego katalogu projektu */educa/static/*.

Serwowanie statycznych plików za pomocą NGINX

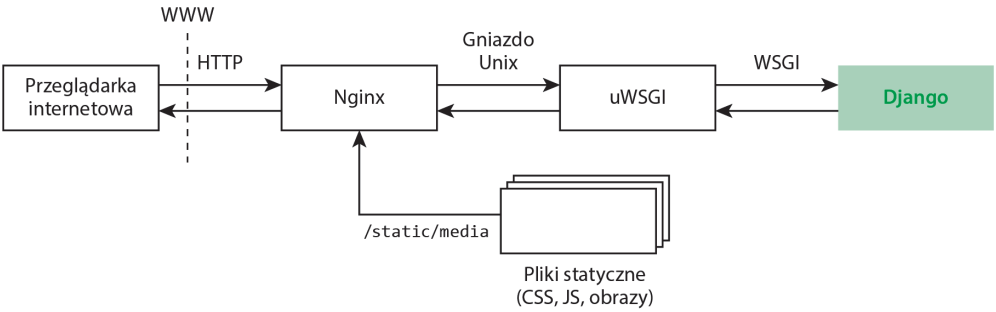
Przeprowadź edycję pliku *config/nginx/default.conf.template* i dodaj do bloku `server` następujące wiersze wyróżnione pogrubioną czcionką.

```
server {
    # …
    location / {
        include      /etc/nginx/uwsgi_params;
        uwsgi_pass   uwsgi_app;
    }

    location /static/ {
        alias /code/educa/static/;
    }

    location /media/ {
        alias /code/educa/media/;
    }
}
```

Wymienione dyrektywy nakazują serwerowi NGINX bezpośrednie udostępnianie zasobów statycznych znajdujących się w katalogach */static* i */media*. Oto opis tych ścieżek:

- */static/*. Ta ścieżka jest zgodna z ustawioną w opcji `STATIC_URL`. Ścieżka docelowa odpowiada wartości ustawienia `STATIC_ROOT`. Używamy jej do serwowania statycznych plików aplikacji z katalogu zamontowanego do obrazu Docker serwera NGINX.
- */media/*. Ta ścieżka jest zgodna z ustawioną w opcji `MEDIA_URL`, a jej ścieżka docelowa odpowiada wartości ustawienia `MEDIA_ROOT`. Używamy jej do obsługi plików multimedialnych przesłanych do treści kursu z katalogu zamontowanego do obrazu Docker NGINX.

Schemat środowiska produkcyjnego pokazałem na rysunku 17.8.



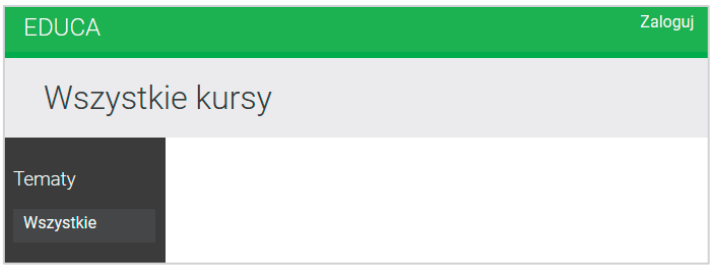
Rysunek 17.8. Cykl żądanie-odpowiedź środowiska produkcyjnego włącznie z plikami statycznymi

Pliki zlokalizowane w katalogach */static/* i */media/* są teraz serwowane przez NGINX bezpośrednio zamiast przekazywania ich do uWSGI. Żądania do dowolnych innych ścieżek nadal są przekazywane przez NGINX do uWSGI przez gniazdo UNIX.

Zatrzymaj aplikację Docker poprzez naciśnięcie w powłoce klawiszy *Ctrl+C* lub za pomocą przycisku *stop* w aplikacji Docker Desktop. Następnie ponownie uruchom system Docker Compose za pomocą polecenia.

```
docker compose up
```

W przeglądarce internetowej przejdź pod adres *http://educaproject.com/*. Powinieneś zobaczyć ekran podobny do pokazanego na rysunku 17.9.



Rysunek 17.9. Strona listy kursów serwowana za pośrednictwem NGINX i uWSGI

Teraz zasoby statyczne, takie jak arkusze stylów CSS i obrazy, ładują się prawidłowo. Żądania HTTP plików statycznych są teraz serwowane bezpośrednio przez NGINX zamiast przekazywania do uWSGI.

Z powodzeniem skonfigurowałeś NGINX do udostępniania plików statycznych. Następnie przygotujesz swój projekt Django do wdrożenia w środowisku produkcyjnym i serwowania jego zawartości za pomocą protokołu HTTPS.

Zabezpieczanie witryny za pomocą protokołu SSL/TLS

Protokół **TLS** (ang. *Transport Layer Security*) jest standardem dla serwowania stron internetowych za pośrednictwem bezpiecznego połączenia. Poprzednikiem TLS jest SSL (ang. *Secure Sockets Layer*). Chociaż protokół SSL jest obecnie uważany za przestarzały, w wielu bibliotekach i dokumentacji online można znaleźć odwołania zarówno do TLS, jak i SSL. Gorąco zachęcam do stosowania HTTPS w udostępnianych witrynach internetowych.

W tym podrozdziale sprawdzimy budowany projekt Django pod kątem wszelkich błędów i zweryfikujemy go pod kątem wdrożenia produkcyjnego. Przygotujemy go również do serwowania przez HTTPS. Następnie, aby udostępniać witrynę internetową w bezpieczny sposób, skonfigurujemy w NGINX certyfikat SSL/TLS.

Sprawdzenie gotowości projektu do wdrożenia do produkcji

Framework Django dostarcza mechanizmu weryfikacji, za pomocą którego można w dowolnym momencie sprawdzić stan projektu. Framework kontrolny (`check`) sprawdza aplikacje zainstalowane w Twoim projekcie Django i wykrywa typowe problemy. Sprawdzenia są uruchamiane niejawnie po uruchomieniu poleceń zarządzających, takich jak `runserver` i `migrate`. Można jednak zainicjować je jawnie za pomocą polecenia zarządzającego `check`.

Więcej informacji na temat frameworka `check` w Django można znaleźć na stronie *https://docs.djangoproject.com/en/5.0/topics/checks/*.

Przekonajmy się, czy framework `check` nie znajduje w Twoim projekcie żadnych problemów. Aby sprawdzić projekt, przejdź do powłoki z poziomu katalogu projektu `educa` i wydaj poniższe polecenie.

```
python manage.py check --settings=educa.settings.prod
```

Powinieneś zobaczyć wynik podobny do następującego.

```
System check identified no issues (0 silenced).
```

Framework kontroli systemu nie znalazł żadnych problemów. Jeśli użyjesz opcji `--deploy`, framework kontroli systemu przeprowadzi dodatkowe testy istotne z punktu widzenia wdrożenia do produkcji.

Z katalogu projektu `educa` uruchom polecenie.

```
python manage.py check --deploy --settings=educa.settings.prod
```

Powinieneś zobaczyć wynik podobny do następującego.

```
System check identified some issues:
WARNINGS:
(security.W004) You have not set a value for the SECURE_HSTS_SECONDS setting.
...
(security.W008) Your SECURE_SSL_REDIRECT setting is not set to True...
(security.W009) Your SECRET_KEY has less than 50 characters, less than 5 unique
characters, or it's prefixed with 'django-insecure-'...
(security.W012) SESSION_COOKIE_SECURE is not set to True. ...
(security.W016) You have 'django.middleware.csrf.CsrfViewMiddleware' in your
MIDDLEWARE, but you have not set CSRF_COOKIE_SECURE ...
System check identified 5 issues (0 silenced).
```

Framework `check` zidentyfikował pięć problemów (0 błędów, 5 ostrzeżeń). Wszystkie ostrzeżenia dotyczą ustawień związanych z zabezpieczeniami.

Zajmijmy się problemem `security.W009`. Przeprowadź edycję pliku *educa/settings/base.py* i zmodyfikuj ustawienie `SECRET_KEY`. Usuń przedrostek `django-insecure-` i wprowadź dodatkowe losowe znaki, aby wygenerować ciąg o długości co najmniej 50 znaków.

Ponownie uruchom polecenie `check`, aby się przekonać, czy problem `security.W009` nie jest już zgłaszany. Pozostałe ostrzeżenia dotyczą konfiguracji SSL/TLS. Zajmiemy się nimi w następnej kolejności.

Konfiguracja projektu do obsługi SSL/TLS

Django zawiera pewne ustawienia charakterystyczne dla SSL/TLS. Zmodyfikujemy ustawienia produkcyjne w taki sposób, aby serwować witrynę za pośrednictwem protokołu HTTPS.

Przeprowadź edycję pliku *educa/settings/prod.py* i umieść w nim poniższe ustawienie.

```
# Zabezpieczenia
CSRF_COOKIE_SECURE = True
SESSION_COOKIE_SECURE = True
SECURE_SSL_REDIRECT = True
```

Oto omówienie użytych opcji:

- `CSRF_COOKIE_SECURE`. Wykorzystanie bezpiecznego pliku cookie do ochrony przed fałszowaniem żądań między witrynami (**CSRF**). W przypadku ustawienia `True` przeglądarki będą przesyłać pliki cookie tylko przez protokół HTTPS.
- `SESSION_COOKIE_SECURE`. Użyj bezpiecznego pliku cookie sesji. W przypadku ustawienia `True` przeglądarki będą przesyłać pliki cookie tylko przez protokół HTTPS.
- `SECURE_SSL_REDIRECT`. Określa, czy żądanie HTTP ma zostać przekierowane do HTTPS.

Teraz Django będzie przekierowywać żądania HTTP do HTTPS, a pliki cookie sesji i CSRF będą wysyłane tylko przez HTTPS.

Z głównego katalogu Twojego projektu uruchom polecenie:

```
python manage.py check --deploy --settings=educa.settings.prod
```

Pozostało tylko jedno ostrzeżenie — `security.W004`:

```
(security.W004) You have not set a value for the SECURE_HSTS_SECONDS setting.
...
```

To ostrzeżenie jest związane z regułą **HSTS** (ang. *HTTP Strict Transport Security*). Reguły HSTS uniemożliwiają użytkownikom omijanie ostrzeżeń i nawiązywanie połączeń z witrynami, których certyfikaty utraciły ważność, są podpisane podpisem własnym lub z innego powodu mają nieważny certyfikat SSL. W następnym punkcie zamierzamy użyć dla naszej witryny certyfikatu z podpisem własnym, dlatego zignorujemy to ostrzeżenie.

Gdy jesteś właścicielem domeny, możesz ubiegać się o certyfikat SSL/TLS wydany przez zaufany urząd certyfikacji. Dzięki temu przeglądarki internetowe mogą zweryfikować tożsamość witryny. W takim przypadku możesz wprowadzić w ustawieniu `SECURE_HSTS_SECONDS` wartość większą od 0, która jest wartością domyślną. Więcej informacji o regułach HSTS możesz znaleźć na stronie *https://docs.djangoproject.com/en/5.0/ref/middleware/#http-strict-transport-security.*

Pomyślnie rozwiązałeś pozostałe problemy zgłaszane przez framework `check`. Więcej informacji o liście kontrolnej wdrażania aplikacji Django możesz znaleźć na stronie *https://docs.djangoproject.com/en/5.0/howto/deployment/checklist/*.

Utworzenie certyfikatu SSL/TLS

W katalogu projektu `educa` utwórz nowy katalog o nazwie `ssl`. Następnie wygeneruj certyfikat SSL/TLS za pomocą poniższego polecenia wydanego w powłoce.

```
openssl req -x509 -newkey rsa:2048 -sha256 -days 3650 -nodes \
  -keyout ssl/educa.key -out ssl/educa.crt \
  -subj '/CN=*.educaproject.com' \
  -addext 'subjectAltName=DNS:*.educaproject.com'
```

W ten sposób generujemy klucz prywatny i 2048-bitowy certyfikat SSL/TLS ważny przez rok. Ten certyfikat został wydany dla hosta o nazwie **.educaproject.com*. To jest certyfikat wieloznaczny. Dzięki wykorzystaniu w nazwie domeny symbolu wieloznacznego * certyfikat może być używany dla dowolnej subdomeny *educaproject.com*, na przykład *www.educaproject.com* lub *django.educaproject.com*. Po wygenerowaniu certyfikatu w katalogu *educa/ssl/* będą znajdować się dwa pliki: *educa.key* (klucz prywatny) oraz *educa.crt* (certyfikat).

Aby skorzystać z opcji `-addext`, będziesz potrzebować co najmniej OpenSSL 1.1.1 lub Libre SSL 3.1.0. Lokalizację OpenSSL na Twoim komputerze możesz sprawdzić za pomocą polecenia `which openssl`, a wersję możesz sprawdzić za pomocą polecenia `openssl version`.

Możesz również użyć certyfikatu SSL/TLS dołączonego do materiałów z kodem źródłowym do tego rozdziału. Certyfikat jest dostępny pod adresem *https://github.com/Packt Publishing/Django-5-by-example/blob/main/Chapter17/educa/ssl/*. Pamiętaj, że powinieneś wygenerować klucz prywatny i nie powinieneś używać tego certyfikatu w środowisku produkcyjnym.

Konfiguracja serwera NGINX do wykorzystania SSL/TLS

Przeprowadź edycję pliku *docker-compose.yml* i dodaj do niego następujące wiersze wyróżnione pogrubioną czcionką.

```
services:
  # …

  nginx:
    #…
    ports:
      - "80:80"
      - "443:443"
```

Host kontenera NGINX będzie dostępny przez port 80 (HTTP) i port 443 (HTTPS). Port hosta `443` jest zmapowany na port kontenera `443`.

Przeprowadź edycję pliku *config/nginx/default.conf.template* projektu `educa` i zmodyfikuj dyrektywę `server`, umieszczając w niej wskazane dyrektywy dotyczące SSL/TLS.

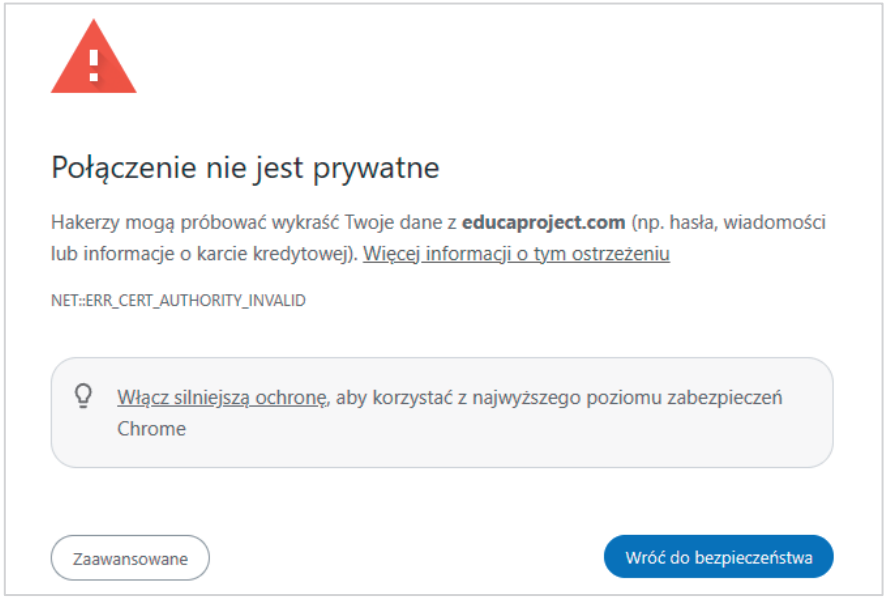
```
server {
   listen              80;
   listen              443 ssl;
   ssl_certificate     /code/educa/ssl/educa.crt;
   ssl_certificate_key /code/educa/ssl/educa.key;
   server_name         www.educaproject.com educaproject.com;
   # …
}
```

Dzięki temu ustawieniu NGINX nasłuchuje teraz zarówno ruchu HTTP w porcie 80, jak i ruchu HTTPS w porcie 443. Ścieżka dostępu do certyfikatu SSL/TLS została wskazana za pomocą zmiennej `ssl_certificate`, natomiast klucz certyfikatu — z wykorzystaniem zmiennej `ssl_certificate_key`.

Zatrzymaj aplikację Docker poprzez naciśnięcie w powłoce klawiszy *Ctrl+C* lub za pomocą przycisku *stop* w aplikacji Docker Desktop. Następnie ponownie uruchom system Docker Compose za pomocą polecenia.

```
docker compose up
```

W przeglądarce internetowej przejdź pod adres *https://educaproject.com/*. Powinieneś zobaczyć komunikat ostrzeżenia, podobny do pokazanego na rysunku 17.10.

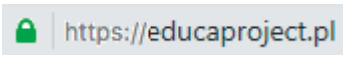


Rysunek 17.10. Ostrzeżenie o nieprawidłowym certyfikacie

Dokładna treść i postać tego komunikatu zależą od używanej przeglądarki internetowej. Komunikat ostrzega o użyciu niezaufanego certyfikatu, ponieważ przeglądarka internetowa nie jest w stanie zweryfikować tożsamości witryny internetowej. To z kolei wynika z wykorzystania certyfikatu z podpisem własnym zamiast uzyskania go z jednego z zaufanych urzędów certyfikacji. Gdy jesteś właścicielem domeny, możesz ubiegać się o certyfikat SSL/TLS z zaufanego urzędu certyfikacji, aby przeglądarki internetowe mogły zweryfikować tożsamość witryny. Jeżeli chcesz uzyskać zaufany certyfikat dla rzeczywistej domeny internetowej, zainteresuj się projektem Let's Encrypt utworzonym przez fundację Linux Foundation. Jest to urząd certyfikacji typu *non profit*, który ma na celu ułatwienie bezpłatnego uzyskiwania i odnawiania zaufanych certyfikatów SSL/TLS. Więcej informacji na ten temat można znaleźć pod adresem *https://letsencrypt.org*.

Kliknij link lub przycisk udostępniający dodatkowe informacje i zdecyduj o wizycie w witrynie, ignorując ostrzeżenia. Przeglądarka może poprosić o dodanie wyjątku dla tego certyfikatu lub sprawdzić, czy mu ufasz. W przeglądarce Chrome może nie pojawić się opcja przejścia do niezaufanej witryny. W takim przypadku wpisz **`thisisunsafe`** bezpośrednio w oknie przeglądarki Chrome wyświetlającym stronę z ostrzeżeniem, po czym wciśnij *Enter*. Chrome załaduje wtedy tę stronę. Należy pamiętać, aby robić to tylko w przypadku certyfikatów z podpisem własnym. Nie ufaj żadnym nieznanym certyfikatom ani nie omijaj mechanizmów weryfikacji certyfikatów SSL/TSL dla innych domen.

Gdy sięgniesz do strony zabezpieczonej certyfikatem SSL/TLS, zobaczysz, że przeglądarka obok adresu URL wyświetla ikonę kłódki, co pokazałem na rysunku 17.11.



Rysunek 17.11. Pasek adresu w przeglądarce z ikoną kłódki oznaczającą bezpieczne połączenie

Inne przeglądarki mogą wyświetlać ostrzeżenie informujące, że certyfikat nie jest zaufany, jak na rysunku 17.12.

Rysunek 17.12. Pasek adresu przeglądarki wraz z komunikatem ostrzegawczym

Twoja przeglądarka mogła oznaczyć certyfikat jako niebezpieczny, ale używasz go tylko do celów testowych. Teraz serwujemy naszą witrynę w bezpieczny sposób, za pośrednictwem protokołu HTTPS.

Przekierowywanie ruchu HTTP do HTTPS

Django przekierowuje żądania HTTP do HTTPS przy użyciu ustawienia `SECURE_SSL_REDIRECT`. Każde żądanie korzystające z prefiksu *http://* jest przekierowywane do tego samego adresu URL z prefiksem *https://*. Można to jednak obsłużyć wydajniej za pomocą NGINX.

Przeprowadź edycję pliku *config/nginx/default.conf.template* i dodaj następujące wiersze wyróżnione pogrubioną czcionką.

```
# Komponent upstream dla uWSGI
upstream uwsgi_app {
    server unix:/code/educa/uwsgi_app.sock;
}

server {
    listen      80;
    server_name www.educaproject.com educaproject.com;
    return 301 https://$host$request_uri;
}

server {
    listen               443 ssl;
    ssl_certificate      /code/educa/ssl/educa.crt;
    ssl_certificate_key  /code/educa/ssl/educa.key;
    server_name   www.educaproject.com educaproject.com;
    # ...
}
```

W tym kodzie usunęliśmy dyrektywę `listen 80` z oryginalnego bloku `server`, dzięki czemu platforma jest dostępna tylko przez SSL/TLS (port `443`). Nad pierwotnym blokiem `server` zdefiniowaliśmy dodatkowy blok `server`, który nasłuchuje wyłącznie na porcie 80 i przekierowuje wszystkie żądania HTTP do HTTPS. Aby to osiągnąć, należy zwrócić kod odpowiedzi HTTP `301` (trwałe przekierowanie), który powoduje przekierowanie żądanego adresu URL do wersji *https://* z wykorzystaniem zmiennych `$host` oraz `$request_uri`.

Otwórz terminal w katalogu nadrzędnym, w którym znajduje się plik *docker-compose.yml* i uruchom następujące polecenie NGINX.

```
docker compose exec nginx nginx -s reload
```

Spowoduje to uruchomienie w kontenerze `nginx` polecenia `nginx -s reload`. Cały ruch HTTP zostanie teraz, za pomocą serwera NGINX, przekierowany na HTTPS.

Środowisko aplikacji jest teraz zabezpieczone za pomocą TLS/SSL. Aby dokończyć konfigurowanie środowiska produkcyjnego, musimy już tylko zintegrować serwer Daphne w celu obsługi żądań asynchronicznych i uruchomić w produkcji pokoje rozmów.

Konfiguracja serwera Daphne do wykorzystania z frameworkiem Django Channels

W rozdziale 16., „Budowanie serwera czatu", użyliśmy frameworka Django Channels do zbudowania serwera czatu z wykorzystaniem WebSocket. W celu serwowania asynchronicznych żądań zastąpiliśmy standardowe polecenie `runserver` frameworka Django wywołaniem serwera Daphne. W tym podrozdziale dodamy Daphne do środowiska produkcyjnego.

Aby uruchomić serwer WWW Daphne, utworzymy w pliku Docker Compose nową usługę.

W pliku *docker-compose.yml* dodaj następujące wiersze wewnątrz bloku *services*.

```
daphne:
    build: .
    working_dir: /code/educa/
    command: ["../wait-for-it.sh", "db:5432", "--",
              "daphne", "-b", "0.0.0.0", "-p", "9001",
              "educa.asgi:application"]
    restart: always
    volumes:
      -.:/code
   environment:
      - DJANGO_SETTINGS_MODULE=educa.settings.prod
      - POSTGRES_DB=postgres
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=postgres
    depends_on:
      - db
      - cache
```

Definicja usługi `daphne` jest bardzo podobna do konfiguracji usługi `web`. Obraz usługi `daphne` również jest tworzony przy użyciu pliku *Dockerfile* utworzonego wcześniej dla usługi `web`. Oto najważniejsze różnice:

- Ustawienie `working_dir` zmienia katalog roboczy obrazu na */code/educa/*.
- Ustawienie `command` uruchamia aplikację `educa.asgi:application` zdefiniowaną w pliku *educa/asgi.py* z Daphne przy użyciu hosta `0.0.0.0` i portu 9001. Wykorzystuje również skrypt bash *wait-for-it*, aby przed zainicjowaniem serwera WWW czekać na gotowość bazy danych PostgreSQL.

Ponieważ wykorzystujemy Django w produkcji, to podczas odbierania żądań HTTP Django sprawdza ustawienie `ALLOWED_HOSTS`. Te same mechanizmy walidacji zaimplementujemy dla połączeń WebSocket.

Przeprowadź edycję pliku *educa/asgi.py* w Twoim projekcie i dodaj następujący kod wyróżniony pogrubioną czcionką.

```
import os
from django.core.asgi import get_asgi_application
from channels.routing import ProtocolTypeRouter, URLRouter
from channels.security.websocket import AllowedHostsOriginValidator
from channels.auth import AuthMiddlewareStack

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'educa.settings')

django_asgi_app = get_asgi_application()

from chat.routing import websocket_urlpatterns

application = ProtocolTypeRouter({
    'http': django_asgi_app,
    'websocket': AllowedHostsOriginValidator(
        AuthMiddlewareStack(
            URLRouter(websocket_urlpatterns)
        )
    ),
})
```

Konfiguracja frameworka `Channels` jest teraz przygotowana do produkcji.

Wykorzystanie bezpiecznych połączeń dla gniazd WebSocket

Wcześniej skonfigurowaliśmy NGINX do wykorzystania bezpiecznych połączeń za pośrednictwem SSL/TLS. Teraz należy zmodyfikować połączenia `ws` (WebSocket) w taki sposób, by korzystały z protokołu `wss` (ang. *WebSocket Secure* — dosłownie: bezpieczne WebSocket) analogicznie do tego, jak połączenia HTTP są teraz serwowane przez HTTPS.

Przeprowadź edycję szablonu *chat/room.html* aplikacji `chat` i odszukaj w bloku `domready` wiersz:

```
const url = 'ws://' + window.location.host +
```

Zastąp gotym:

```
const url = 'wss://' + window.location.host +
```

Dzięki wykorzystaniu prefiksu *wss://* zamiast *ws://*, jawnie wskazujemy na połączenie z bezpiecznym gniazdem WebSocket.

Uwzględnienie Daphne w konfiguracji NGINX

W konfiguracji produkcyjnej uruchomimy Daphne w gnieździe UNIX, a w wyższej warstwie skorzystamy z serwera NGINX. NGINX na podstawie ustalonej ścieżki przekaże żądania do Daphne. Daphne zostanie udostępniony serwerowi NGINX za pośrednictwem interfejsu gniazda UNIX, analogicznie do konfiguracji uWSGI.

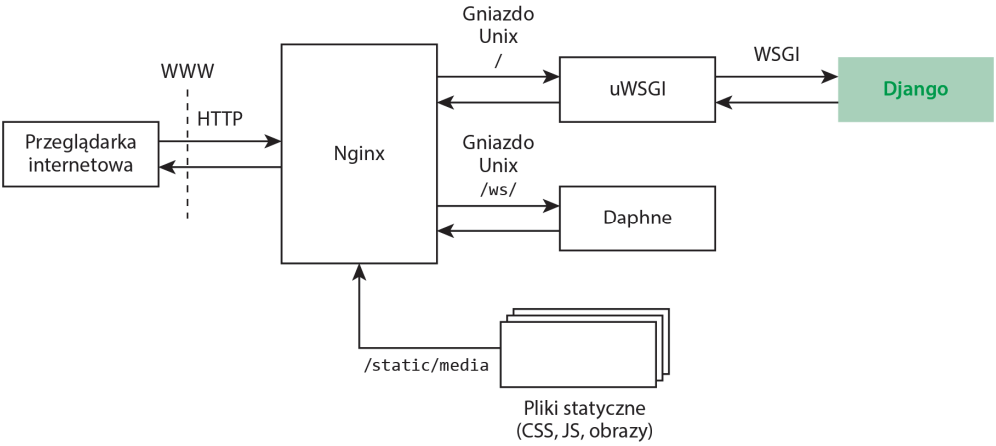
Przeprowadź edycję pliku *config/nginx/default.conf.template* i zmodyfikuj go w następujący sposób.

```
# Komponent upstream dla uWSGI
upstream uwsgi_app {
    server unix:/code/educa/uwsgi_app.sock;
}

# komponent upstream dla Daphne
upstream daphne {
    server daphne:9001;
}

server {
    listen       80;
    server_name www.educaproject.com educaproject.com;
    return 301 https://$host$request_uri;
}

server {
    listen               443 ssl;
    ssl_certificate      /code/educa/ssl/educa.crt;
    ssl_certificate_key  /code/educa/ssl/educa.key;
    server_name  www.educaproject.com educaproject.com;
    error_log    stderr warn;
    access_log   /dev/stdout main;

    location / {
        include      /etc/nginx/uwsgi_params;
        uwsgi_pass   uwsgi_app;
    }

    location /ws/ {
        proxy_pass           http://daphne;
        proxy_http_version   1.1;
        proxy_set_header     Upgrade $http_upgrade;
        proxy_set_header     Connection "upgrade";
        proxy_redirect       off;
    }

    location /static/ {
        alias /code/educa/static/;     }

    location /media/ {
        alias /code/educa/media/;
    }
}
```

W tej konfiguracji zdefiniowaliśmy nowy strumień `upstream` o nazwie *daphne*, wskazujący na host *daphne* i port 9001. W bloku `server`, w celu przekazywania żądań do Daphne, skonfigurowaliśmy lokalizację */ws/*. Aby przekazywać żądania do Daphne, skorzystaliśmy z dyrektywy `proxy_pass` oraz uwzględniliśmy kilka dodatkowych dyrektyw proxy.

Dzięki tej konfiguracji NGINX przekaże każde żądanie URL, które rozpoczyna się od prefiksu */ws/* do Daphne, a pozostałe do uWSGI. Wyjątkiem są pliki zapisane w katalogach */static/* lub */media/*, które będą serwowane bezpośrednio przez NGINX.

Środowisko produkcyjne z uwzględnieniem Daphne pokazałem na rysunku 17.13.



Rysunek 17.13. Cykl żądanie-odpowiedź w środowisku produkcyjnym z wykorzystaniem Daphne

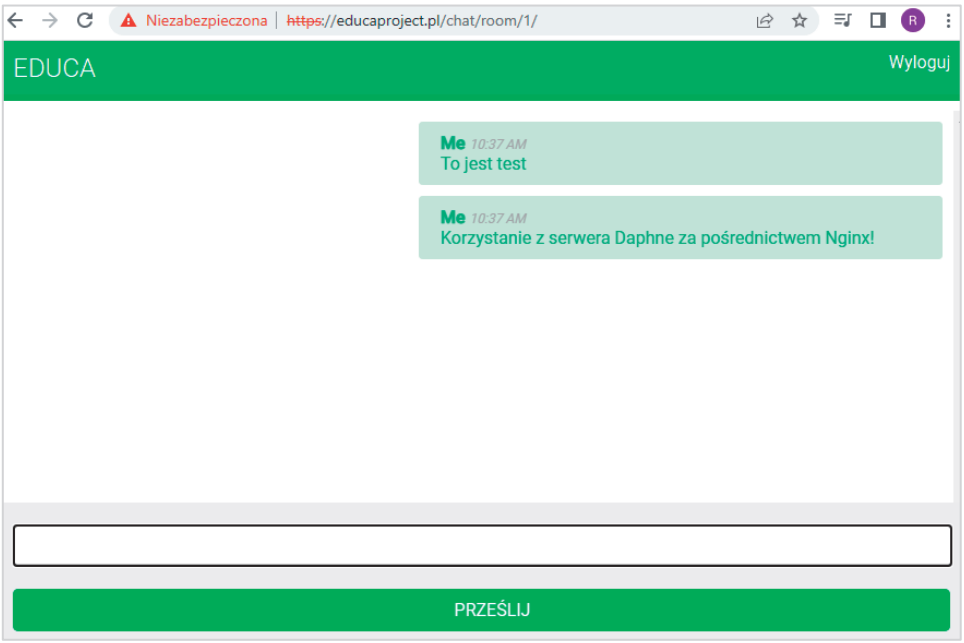
NGINX działa przed uWSGI, a Daphne spełnia rolę odwróconego serwera proxy. Serwer NGINX przyjmuje żądania z internetu i przekazuje je do serwera aplikacji (uWSGI lub Daphne) na podstawie prefiksu ścieżki. Poza tym NGINX obsługuje również pliki statyczne i przekierowuje niezabezpieczone żądania do bezpiecznych lokalizacji. Taka konfiguracja pozwala na ograniczenie czasu przestojów, zużycie mniejszej ilości zasobów serwerowych oraz zapewnia większą wydajność i bezpieczeństwo.

Zatrzymaj aplikację Docker poprzez naciśnięcie w powłoce klawiszy *Ctrl+C* lub za pomocą przycisku *stop* w aplikacji Docker Desktop. Następnie ponownie uruchom system Docker Compose za pomocą polecenia.

```
docker compose up
```

Zaloguj się jako użytkownik *instruktor* i skorzystaj z przeglądarki, aby utworzyć przykładowy kurs. Następnie zaloguj się jako użytkownik, który jest zapisany na kurs, i otwórz w przeglądarce stronę *https://educaproject.com/chat/room/1/*. Powinieneś być w stanie wysyłać i odbierać wiadomości, tak jak pokazałem w poniższym przykładzie (rysunek 17.14).

Daphne działa poprawnie, a NGINX przesyła do niego żądania WebSocket. Wszystkie połączenia są zabezpieczone za pomocą SSL/TLS.



Rysunek 17.14. Wiadomości w pokoju rozmów kursu serwowane za pomocą NGINX i Daphne

Gratulacje! Właśnie zbudowałeś własny, gotowy do produkcji stos z wykorzystaniem NGINX, uWSGI i Daphne. Za pomocą ustawień konfiguracyjnych NGINX, uWSGI i Daphne możesz wprowadzić dodatkowe optymalizacje w celu zwiększenia wydajności i poprawy bezpieczeństwa. Jednak ta konfiguracja produkcyjna doskonale nadaje się na początek.

W celu zdefiniowania i uruchamiania usług w wielu kontenerach, skorzystaliśmy z systemu Docker Compose. Pamiętaj, że możesz używać systemu Docker Compose zarówno w lokalnych środowiskach programistycznych, jak i w środowiskach produkcyjnych. Więcej informacji na temat korzystania z Docker Compose w środowisku produkcyjnym można znaleźć pod adresem *https://docs.docker.com/compose/production/*.

W przypadku bardziej zaawansowanych środowisk produkcyjnych konieczna będzie dynamiczna dystrybucja kontenerów na wiele maszyn. W tym celu zamiast Docker Compose będziesz potrzebować orkiestratora, takiego jak Docker Swarm lub Kubernetes. Więcej informacji na temat trybu Docker Swarm można znaleźć pod adresem *https://docs.docker.com/engine/swarm/*, a informacje o Kubernetes są dostępne na stronie *https://kubernetes.io/docs/home/*.

Zwróć uwagę, że zarządzanie systemami i infrastrukturą chmury wymaga specjalistycznej wiedzy w zakresie konfiguracji, optymalizacji i bezpieczeństwa. Aby zapewnić bezpieczne i wydajne środowisko produkcyjne, warto rozważyć zatrudnienie eksperta ds. systemów i DevOps lub poszerzenie własnej wiedzy w tych dziedzinach.

Gdy środowisko przetwarzające żądania HTTP w wydajny sposób jest gotowe, nadszedł czas, aby zapoznać się dokładniej z oprogramowaniem middleware do przetwarzania w aplikacji żądań i odpowiedzi.

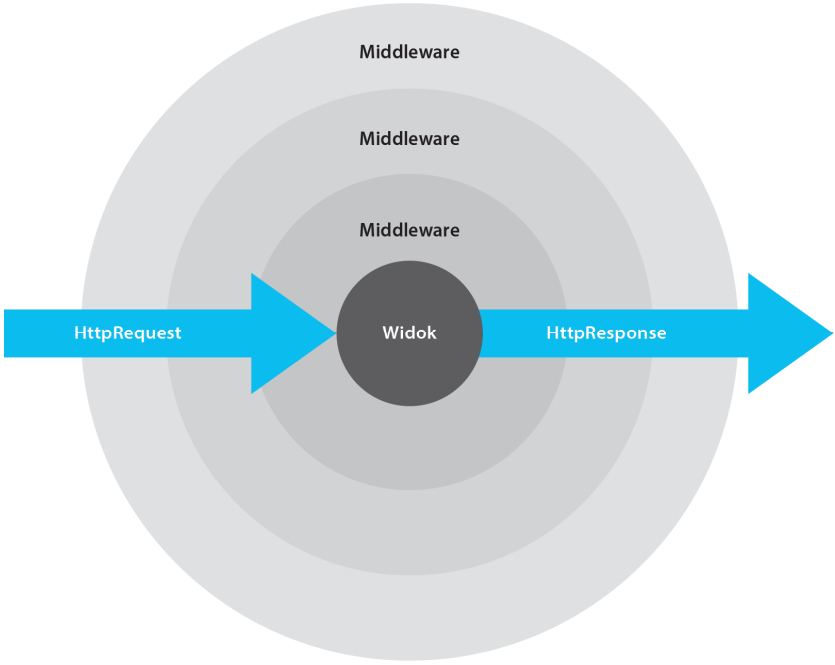
Utworzenie własnej warstwy middleware

Wiesz już o istnieniu opcji `MIDDLEWARE` zawierającej listę oprogramowania middleware, które zostało dodane do projektu. Tego rodzaju oprogramowanie możesz potraktować jako niskopoziomowy system wtyczek pozwalający na implementację haków uruchamianych podczas przetwarzania żądań i odpowiedzi. Każde oprogramowanie middleware jest odpowiedzialne za konkretną operację, która będzie wykonana dla wszystkich żądań lub odpowiedzi HTTP.

Uwaga

Staraj się unikać wykonywania kosztownych operacji przez oprogramowanie middleware, ponieważ operacje te będą wykonywane w trakcie każdego żądania.

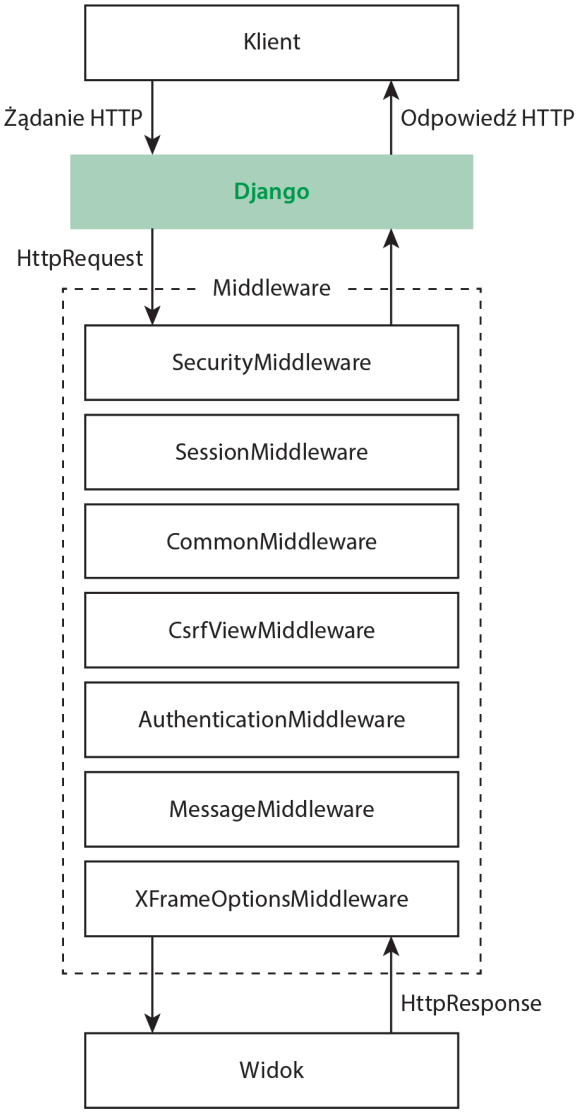
Sposób uruchamiania oprogramowania middleware w Django pokazałem na rysunku 17.15.



Rysunek 17.15. Uruchamianie oprogramowania middleware w Django

Po otrzymaniu żądania HTTP oprogramowanie middleware jest wywoływane w kolejności pojawiania się na liście `MIDDLEWARE`. Natomiast w trakcie generowania odpowiedzi HTTP przez Django oprogramowanie middleware jest wykonywane w kolejności odwrotnej do jego umieszczenia na wspomnianej liście.

Kolejność uruchamiania komponentów middleware zawartych w ustawieniu `MIDDLEWARE` podczas tworzenia projektu za pomocą polecenia zarządzania `startproject` przedstawiłem na rysunku 17.16.



Rysunek 17.16. Kolejność uruchamiania domyślnych komponentów oprogramowania middleware

Oprogramowanie middleware można napisać w postaci na przykład następującej funkcji.

```
def my_middleware(get_response):
    def middleware(request):
        # Kod wykonywany dla każdego żądania przed wywołaniem widoku
        # (i innego oprogramowania middleware).
        response = get_response(request)
        # Kod wykonywany dla każdego żądania (odpowiedzi) po wywołaniu widoku.
        return response
    return middleware
```

Fabryka oprogramowania middleware to obiekt wywoływalny, który pobiera wywołanie `get_response` i zwraca oprogramowanie middleware. Oprogramowanie `middleware` to obiekt wywoływalny, który — podobnie jak widok — pobiera żądanie i zwraca odpowiedź. Wywołanie `get_response` może być kolejnym oprogramowaniem middleware w łańcuchu lub rzeczywistym widokiem w przypadku ostatniego oprogramowania middleware na liście.

Jeśli dowolne oprogramowanie middleware zwróci odpowiedź bez wywoływania odpowiedzi `get_response`, spowoduje to tzw. „zwarcie procesu". Dalsze warstwy middleware (w tym widok) nie będą uruchamiane, a odpowiedź będzie przekazana przez te same warstwy, przez które przekazywane jest żądanie.

Kolejność programów w ustawieniu `MIDDLEWARE` jest bardzo ważna, ponieważ warstwy middleware mogą zależeć od zestawu danych w żądaniu obsłużonym wcześniej przez inną warstwę middleware. Podczas dodawania nowego oprogramowania do ustawienia `MIDDLEWARE`, pamiętaj, aby umieścić je we właściwej kolejności.

Więcej informacji na temat oprogramowania middleware można znaleźć pod adresem *https://docs.djangoproject.com/en/5.0/topics/http/middleware/*.

Utworzenie oprogramowania middleware do obsługi subdomeny

Zamierzamy utworzyć własną warstwę middleware, aby umożliwić dostęp do kursów za pośrednictwem subdomeny. Każdy adres URL szczegółów kursu w postaci *https://educaproject.com/course/django/*, *educaproject.com/course/django/* będzie również dostępny za pośrednictwem subdomeny używającej slugu kursu, np. *https://django/educaproject.com/*. Aby uzyskać dostęp do szczegółów kursu, użytkownicy będą mogli korzystać z subdomeny, tak jak ze skrótu. Wszelkie żądania dotyczące subdomen zostaną przekierowane do właściwych adresów URL kursu.

Oprogramowanie middleware może zostać umieszczone w dowolnym miejscu projektu. Jednak zaleca się utworzenie dla niego pliku o nazwie *middleware.py* w katalogu aplikacji.

W katalogu aplikacji `courses` utwórz nowy plik *middleware.py*, a następnie umieść w nim poniższy fragment kodu.

```
from django.urls import reverse
from django.shortcuts import get_object_or_404, redirect
from .models import Course

def subdomain_course_middleware(get_response):
    """
    Subdomeny dla kursów
    """
    def middleware(request):
        host_parts = request.get_host().split('.')
        if len(host_parts) > 2 and host_parts[0] != 'www':
            # Pobranie kursu dla subdomeny.
            course = get_object_or_404(Course, slug=host_parts[0])
            course_url = reverse('course_detail', args=[course.slug])
```

```
                # Przekierowanie bieżącego żądania do widoku course_detail.
                url = '{}://{}{}'.format(
                    request.scheme, '.'.join(host_parts[1:]), course_url
                )
                return redirect(url)
            response = get_response(request)
            return response
        return middleware
```

Po otrzymaniu żądania HTTP wykonywane są następujące zadania.

Pobranie nazwy hosta użytej w żądaniu oraz podział jej na poszczególne człony. Jeśli np. użytkownik uzyskuje dostęp do *mycourse.educaproject.com*, generowana jest lista `['mycourse', 'educaproject', 'com']`.

Sprawdzenie, czy nazwa hosta zawiera szukaną subdomenę, o ile operacja podziału na człony wygenerowała więcej niż dwa elementy. Jeżeli nazwa hosta zawiera subdomenę i nie jest to wartość `'www'`, próbujemy pobrać kurs za pomocą wartości `slug` podanej w subdomenie.

Jeżeli kurs nie zostanie znaleziony, następuje zgłoszenie wyjątku `Http404`. W przeciwnym razie przeglądarka internetowa zostanie przekierowana na adres URL prowadzący do widoku zawierającego szczegóły kursu.

Przeprowadź edycję pliku *settings/base.py* w projekcie `educa` i umieść `'courses.middleware.subdomain_course_ middleware'` na końcu listy `MIDDLEWARE`, tak jak pokazałem poniżej.

```
MIDDLEWARE = [
    # ...
    'courses.middleware.subdomain_course_middleware',
]
```

Tak przygotowana warstwa middleware będzie wykonywana w trakcie każdego żądania.

Pamiętaj, że nazwy hostów, które mogą obsługiwać budowany projekt Django, są określone w ustawieniu `ALLOWED_HOSTS`. Zmienimy to ustawienie, aby naszą aplikację mogła obsługiwać każda możliwa subdomena *educaproject.com*.

Zmodyfikuj ustawienie `ALLOWED_HOSTS` w pliku *educa/settings/prod.py* w taki sposób:

```
ALLOWED_HOSTS = ['.educaproject.com']
```

Wartość rozpoczynająca się kropką jest używana jako symbol wieloznaczny subdomeny; `'.educaproject.com'` będzie pasować do *educaproject.com* i dowolnej subdomeny, np. *course.django.educaproject.com*.

Obsługa wielu subdomen za pomocą NGINX

Chcemy, aby serwer NGINX mógł obsługiwać witrynę internetową wraz z wszelkimi używanymi subdomenami. Przeprowadź edycję pliku *config/nginx/default.conf.template* i zastąp w nim te oto dwa wystąpienia poniższego wiersza:

```
server_name www.educaproject.com educaproject.com;
```

następującym:

```
server_name *.educaproject.com educaproject.com;
```

Dzięki użyciu gwiazdki reguła ta będzie miała zastosowanie we wszystkich subdomenach *educaproject.com*. W celu lokalnego przetestowania przygotowanej warstwy middleware należy do pliku */etc/hosts* dodać wszystkie subdomeny, które mają być sprawdzone. Jeżeli chcesz przetestować oprogramowanie middleware wraz z obiektem `Course` i slugiem `django`, do wymienionego pliku dodaj wiersz:

```
127.0.0.1 django.educaproject.com
```

Zatrzymaj aplikację Docker poprzez naciśnięcie w powłoce klawiszy *Ctrl + C* lub za pomocą przycisku *stop* w aplikacji Docker Desktop. Następnie ponownie uruchom system Docker Compose za pomocą polecenia.

```
docker compose up
```

Teraz w przeglądarce internetowej przejdź pod adres *https://django.educaproject.com/*. Oprogramowanie middleware znajdzie kurs na podstawie podanej subdomeny, a następnie przeprowadzi przekierowanie na adres *https://educaproject.com/course/django/*.

Niestandardowa warstwa middleware obsługi subdomeny działa poprawnie!

W kolejnym podrozdziale zaprezentuję ostatni temat, który jest niezwykle przydatny w projektach: automatyzację zadań i udostępnianie ich jako poleceń.

Implementacja własnych poleceń administracyjnych

Framework Django pozwala aplikacjom na rejestrowanie własnych poleceń przeznaczonych dla narzędzia *manage.py*. Przykładowo w rozdziale 11., „Rozbudowa sklepu internetowego", użyliśmy poleceń administracyjnych `makemessages` i `compilemessages` w celu utworzenia i kompilacji plików zawierających tłumaczenie witryny internetowej na inny język.

Polecenie zarządzające składa się z modułu Pythona zawierającego klasę `Command` dziedziczącej po `django.core.management.base.BaseCommand` lub jednej z jej podklas. Masz możliwość tworzenia prostych poleceń, które jako dane wejściowe mogą pobierać argumenty pozycyjne i opcjonalne.

Django szuka poleceń administracyjnych w katalogu *management/commands* każdej aktywnej aplikacji wymienionej na liście `INSTALLED_APPS`. Każdy znaleziony moduł zostanie zarejestrowany jako polecenie administracyjne o nazwie takiej samej jak danego modułu.

Więcej informacji na temat własnych poleceń administracyjnych znajdziesz na stronie *https://docs.djangoproject.com/en/5.0/howto/custom-management-commands/*.

Tutaj utworzymy własne polecenie administracyjne, które będzie przypominało użytkownikom o konieczności zapisania się na co najmniej jeden kurs. Polecenie będzie wysyłać przypomnienie e-mail do użytkowników, którzy są zarejestrowani dłużej niż podana ilość czasu, a mimo to nie zapisali się jeszcze na żaden kurs.

Utwórz poniższą strukturę plików w aplikacji `students`.

```
management/
    __init__.py
    commands/
        __init__.py
        enroll_reminder.py
```

Przeprowadź edycję pliku *enroll_reminder.py* i umieść w nim poniższy fragment kodu.

```
import datetime
from django.conf import settings
from django.contrib.auth.models import User
from django.core.mail import send_mass_mail
from django.core.management.base import BaseCommand
from django.db.models import Count
from django.utils import timezone


class Command(BaseCommand):
    help = 'Wysyła wiadomość e-mail do użytkowników zarejestrowanych co najmniej od' \
           'N dni z przypomnieniem o konieczności zapisania się na dowolny kurs.'

    def add_arguments(self, parser):
        parser.add_argument('--days', dest='days', type=int)

    def handle(self, *args, **options):
        emails = []
        subject = 'Zapisz się na kurs'
        date_joined = timezone.now().today() - datetime.timedelta(
            days=options['days'] or 0
        )
        users = User.objects.annotate(
            course_count=Count('courses_joined')
        ).filter(course_count=0, date_joined__date__lte=date_joined)
        for user in users:
            message = f"""Witaj {user.first_name},
            Zauważyliśmy, że jeszcze nie zapisałeś się na kurs.
            Na co jeszcze czekasz?"""
            emails.append(
                (
                    subject,
                    message,
                    settings.DEFAULT_FROM_EMAIL,
                    [user.email]
                )
            )
        send_mass_mail(emails)
        self.stdout.write(f'Wysłano {len(emails)} przypomnień')
```

To jest kod polecenia `enroll_reminder`. Oto w jaki sposób działa powyższy kod:

- Klasa `Command` dziedziczy po klasie `BaseCommand`.
- Dołączyliśmy atrybut `help`. Zadaniem tego atrybutu jest wyświetlenie krótkiego opisu działania polecenia po jego wydaniu w postaci `python manage.py help ↪enroll_reminder`.

- Metody `add_arguments()` użyliśmy w celu dodania argumentu o nazwie `--days`. Za pomocą tego argumentu można określić minimalną liczbę dni, jaką użytkownik ma na zapisanie się na dowolny kurs, zanim otrzyma to przypomnienie.
- Metoda `handle()` zawiera treść rzeczywistego polecenia. Sparsowany atrybut `days` pobieramy z wiersza polecenia. Jeśli ta wartość nie jest ustawiona, użyjemy wartości 0, aby przypomnienie zostało wysłane do wszystkich użytkowników, którzy nie zapisali się na kurs, niezależnie od tego, kiedy się zarejestrowali. Do pobrania bieżącej daty z uwzględnieniem strefy czasowej używamy wywołania `timezone.now().data()` dostępnego w dostarczonym przez Django module `timezone` (strefę czasową dla swojego projektu możesz ustawić za pomocą opcji `TIME_ZONE`). Pobieramy również nazwy użytkowników zarejestrowanych dłużej niż podana liczba dni i nadal niezapisanych na żaden kurs. Otrzymujemy te dane przez oznaczenie kolekcji `QuerySet` całkowitą liczbą kursów, na które zapisał się dany użytkownik. Przypomnienie generujemy dla każdego pobranego użytkownika i dodajemy go do listy `emails`. Na końcu wysyłamy wiadomości e-mail za pomocą funkcji `send_mass_mail()` zoptymalizowanej do otworzenia pojedynczego połączenia SMTP w celu jednoczesnego wysłania wszystkich wiadomości, zamiast otwierania po jednym połączeniu dla każdej wysyłanej wiadomości e-mail.

W ten sposób utworzyłeś pierwsze polecenie administracyjne. Możesz teraz przejść do powłoki i wydać je.

```
docker compose exec web python /code/educa/manage.py \
  enroll_reminder --days=20 --settings=educa.settings.prod
```

Jeżeli nie masz działającego lokalnie serwera SMTP, w rozdziale 2., „Usprawnianie bloga i dodanie funkcjonalności społecznościowych", znajdziesz informacje, jak skonfigurować serwer SMTP dla projektu Django. Ewentualnie możesz dodać poniższy wiersz do pliku *settings/base.py*, aby podczas prac nad projektem framework Django przekazywał wiadomości e-mail do standardowego wyjścia.

```
EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'
```

Django zawiera również narzędzie przeznaczone do wywoływania poleceń administracyjnych za pomocą Pythona. Tego rodzaju polecenia możesz wywoływać z poziomu kodu w następujący sposób.

```
from django.core import management
management.call_command('enroll_reminder', days=20)
```

Gratulacje! W taki sposób utworzyłeś własne polecenie administracyjne dla aplikacji.

Polecenia zarządzające frameworka Django można zaplanować do automatycznego uruchamiania za pomocą takich narzędzi jak `cron` lub Celery Beat. `cron` to narzędzie obsługi harmonogramu zadań w uniksowych systemach operacyjnych, umożliwiające użytkownikom planowanie uruchamiania skryptów lub poleceń w określonych godzinach i odstępach czasu. Więcej na temat narzędzia `cron` możesz przeczytać na stronie *https://en.wikipedia.org/wiki/Cron*. A Celery Beat to system obsługi harmonogramu współpracujący z Celery, który pozwala na uruchamianie funkcji w wyznaczonych odstępach czasu. Więcej o Celery Beat można dowiedzieć się na stronie *https://docs.celeryq.dev/en/stable/userguide/periodic-tasks.html*. Dzięki wykorzystaniu mechanizmów `cron` lub Celery Beat możesz zapewnić regularne wykonywanie zadań bez konieczności ręcznej interwencji.

Podsumowanie

W tym rozdziale stworzyłeś środowisko produkcyjne przy użyciu Docker Compose. Skonfigurowałeś NGINX, uWSGI i Daphne do obsługi aplikacji w środowisku produkcyjnym. Zabezpieczyłeś swoje środowisko za pomocą SSL/TLS. Ponadto zaimplementowałeś własną warstwę middleware oraz dowiedziałeś się, jak można samodzielnie utworzyć polecenie administracyjne.

Dotarłeś do końca tej książki. Gratulacje! Zdobyłeś umiejętności pozwalające na tworzenie za pomocą frameworka Django aplikacji sieciowych, które mogą odnieść sukces. W książce przeprowadziłem Cię przez procesy tworzenia rzeczywistych projektów oraz integracji Django z innymi technologiami. Teraz jesteś gotów do rozpoczęcia pracy nad własnymi projektami Django zarówno w postaci prostego prototypu, jak i ogromnej aplikacji sieciowej.

Życzę Ci powodzenia podczas kolejnej przygody z Django!

Rozszerzanie projektu przy użyciu sztucznej inteligencji

W tym podrozdziale zaprezentuję zadanie rozszerzenia projektu, do którego wykorzystam przykładowe zapytanie do ChatGPT. ChatGPT jest dostępny pod adresem *https://chat.openai.com/*. Jeśli to jest Twoja pierwsza styczność z ChatGPT, zapoznaj się z podstawami korzystania z tego narzędzia w podrozdziale „Rozszerzanie projektu z wykorzystaniem sztucznej inteligencji" w rozdziale 3., „Rozbudowa aplikacji bloga".

Opracowaliśmy kompleksową platformę e-learningową. Jednakże gdy uczestnicy są zapisani na wiele kursów, z których każdy zawiera kilka modułów, mogą mieć trudności z zapamiętaniem, gdzie ostatnio skończyli. Aby temu zaradzić, użyjemy ChatGPT w połączeniu z Redis do przechowywania i pobierania postępów każdego ucznia w ramach kursu. Wskazówki można znaleźć w podpowiedzi dostępnej pod adresem *https://github.com/PacktPublishing/Django-5-by-example/blob/main/Chapter17/prompts/task.md*.

Wskazówka

Podczas refaktoryzacji kodu Pythona ChatGPT ułatwia zbadanie różnych strategii. Opisz swoje obecne podejście, a ChatGPT udzieli Ci porad, dzięki którym Twój kod stanie się bardziej pythoniczny dzięki wykorzystaniu takich zasad jak **DRY** (ang. *do not repeat yourself* — dosłownie: nie powtarzaj się) i stosowaniu zasad projektowania modułowego. W ten sposób Twój kod stanie się czystszy, bardziej estetyczny i łatwiejszy w utrzymaniu.

Dodatkowe zasoby

Oto lista zasobów zawierających dodatkowe informacje związane z tematami omawianymi w tym rozdziale:

- Kod źródłowy przykładów zamieszczonych w tym rozdziale — *https://github.com/PacktPublishing/Django-5-by-example/tree/main/Chapter17*.
- Przegląd informacji na temat Docker Compose — *https://docs.docker.com/compose/*.
- Instalowanie systemu Docker Desktop — *https://docs.docker.com/compose/install/compose-desktop/*.
- Oficjalny obraz Docker Pythona — *https://hub.docker.com/_/python*.
- Dokumentacja Dockerfile — *https://docs.docker.com/reference/dockerfile/*.
- Plik *requirements.txt* do tego rozdziału — *https://github.com/PacktPublishing/Django-5-by-example/blob/main/Chapter17/requirements.txt*.
- Przykład pliku YAML — *https://yaml.org/*.
- Sekcja `build` pliku *Dockerfile* — *https://docs.docker.com/compose/compose-file/build/*.
- Reguły restartowania w systemie Docker — *https://docs.docker.com/config/containers/start-containers-automatically/*.
- Woluminy Dockera — *https://docs.docker.com/storage/volumes/*.
- Specyfikacja Docker Compose — *https://docs.docker.com/compose/compose-file/*.
- Oficjalny obraz Docker PostgreSQL — *https://hub.docker.com/_/postgres*.
- Skrypt bash *wait-for-it.sh* dla Dockera — *https://github.com/vishnubob/wait-for-it/blob/master/wait-for-it.sh*.
- Kolejność uruchamiania usług w Compose — *https://docs.docker.com/compose/startup-order/*.
- Oficjalny obraz Docker bazy danych Redis — *https://hub.docker.com/_/redis*.
- Dokumentacja WSGI — *https://wsgi.readthedocs.io/en/latest/*.
- Lista opcji uWSGI — *https://uwsgi-docs.readthedocs.io/en/latest/Options.html*.
- Oficjalny obraz Docker systemu NGINX — *https://hub.docker.com/_/nginx*.
- Dokumentacja NGINX — *https://nginx.org/en/docs/*.
- Ustawienie `ALLOWED_HOSTS` — *https://docs.djangoproject.com/en/5.0/ref/settings/#allowedhosts*.
- Framework `check` Django — *https://docs.djangoproject.com/en/5.0/topics/checks/*.
- Reguły HTTP Strict Transport Security w Django — *https://docs.djangoproject.com/en/5.0/ref/middleware/#http-strict-transport-security*.
- Lista kontrolna wdrażania aplikacji Django — *https://docs.djangoproject.com/en/5.0/howto/deployment/checklist/*.
- Katalog certyfikatów SSL/TLS z podpisem własnym — *https://github.com/PacktPublishing/Django-5-by-example/blob/main/Chapter17/educa/ssl/*.

- Urząd certyfikacji Let's Encrypt — *https://letsencrypt.org/*.
- Korzystanie z Docker Compose w produkcji — *https://docs.docker.com/compose/production/*.
- Tryb Docker Swarm — *https://docs.docker.com/engine/swarm/*.
- Kubernetes — *https://kubernetes.io/docs/home/*.
- Warstwy middleware w Django — *https://docs.djangoproject.com/en/5.0/topics/http/middleware/*.
- Tworzenie własnych poleceń administracyjnych — *https://docs.djangoproject.com/en/5.0/howto/custom-management-commands/*.
- System `cron` — *https://en.wikipedia.org/wiki/Cron*.
- `celery beat` — *https://docs.celeryq.dev/en/stable/userguide/periodic-tasks.html*.

PROGRAM
PARTNERSKI
GRUPY HELION
1. ZAREJESTRUJ SIĘ
2. PREZENTUJ KSIĄŻKI
3. ZBIERAJ PROWIZJĘ
Zmień swoją stronę WWW w działający bankomat!
Dowiedz się więcej i dołącz już dzisiaj!
http://program-partnerski.helion.pl
GRUPA
Helion